

Numerical Partial Differential Equations

James Adler
Tufts University
jadler.math.tufts.edu

Hans De Sterck
University of Waterloo
uwaterloo.ca/scholar/hdesterc

Scott MacLachlan
Memorial University of Newfoundland
www.math.mun.ca/~smaclachlan

Luke Olson
University of Illinois Urbana-Champaign
lukeo.cs.illinois.edu

May 28, 2025

Contents

Preface	xi
I Introduction and background	1
1 About partial differential equations	3
Overview	3
1.1 Introduction	4
1.1.1 Motivation	4
1.1.2 Partial differential equations	6
1.2 First-order PDEs	7
1.2.1 Scalar first-order conservation laws	7
1.2.2 The linear advection equation in 1D	8
1.2.3 Conservation with flow	9
1.3 Second-order linear PDEs	11
1.3.1 Classification	12
1.4 Some further thoughts	16
2 Background material in numerical methods	19
Overview	19
2.1 Polynomial approximation and interpolation	20
2.1.1 Polynomial approximation	20
2.1.2 Polynomial interpolation	22
2.2 Taylor series	23
2.3 Computational grids	28
2.3.1 Quasi-uniform grids	31
2.4 Approximating derivatives	32
2.5 Approximating integrals: Quadrature	37
2.5.1 Integrating interpolating polynomials	38
2.5.2 Gaussian quadrature	39
2.6 Integration properties	42
2.6.1 Integration by parts in 1D, 2D, and 3D	42
2.6.2 Further integration-by-parts formulas	44
2.7 Some further thoughts	45
II Basic topics	47
3 Finite-difference methods for elliptic partial differential equations	49
Overview	49

3.1	Finite-difference methods for elliptic PDEs on uniform meshes	50
3.1.1	1D model problem	51
3.1.2	2D model problem	53
3.2	Finite-difference methods on nonuniform meshes	56
3.3	Convergence theory for finite-difference methods	59
3.3.1	Consistency	62
3.3.2	Stability	63
3.4	Treatment of Neumann and periodic boundary conditions	70
3.5	Structure-preserving finite-difference methods	75
3.5.1	Finite-volume methods for elliptic PDEs in 1D	77
3.5.2	Finite-volume methods for elliptic PDEs in 2D	83
3.6	Variants of finite-difference methods	86
3.6.1	Finite differences in curvilinear coordinates	87
3.6.2	Meshless finite-difference methods	95
3.7	Some further thoughts	99
3.8	Exercises	100
4	Finite-element methods for elliptic differential equations in 1D	103
	Overview	103
4.1	Weak forms	104
4.2	Ritz–Galerkin approximation	107
4.3	Piecewise polynomial finite elements in 1D	112
4.4	Matrix assembly for linear finite elements in 1D	120
4.5	Some further thoughts	123
4.6	Exercises	126
5	Finite-difference methods for time-dependent partial differential equations	129
	Overview	129
5.1	Finite-difference methods for model equations	130
5.1.1	Finite-difference stencils	131
5.1.2	Method of lines	135
5.1.3	Two-level schemes and matrix notation	136
5.2	Convergence theory	139
5.2.1	Consistency, convergence, and stability	142
5.2.2	Lax convergence and equivalence theorems	145
5.3	Stability of two-level schemes	148
5.3.1	Stability analysis by directly bounding matrix norms	148
5.3.2	Discrete-time Fourier transforms	150
5.3.3	Von Neumann stability of two-level schemes	153
5.3.4	Examples of von Neumann stability analysis	156
5.3.5	Black–Scholes equation for option pricing	162
5.4	Numerical dissipation and dispersion	164
5.4.1	Dissipation and dispersion in linear PDEs	165
5.4.2	Numerical dissipation and dispersion	167
5.4.3	Implications for approximating hyperbolic and parabolic PDEs	171
5.5	Some further thoughts	172
5.6	Exercises	174
6	Finite-volume methods for scalar conservation laws in 1D	177
	Overview	177

6.1	Some properties of scalar hyperbolic conservation laws	178
6.1.1	Nonlinear scalar conservation laws in 1D	178
6.1.2	Weak solutions	181
6.1.3	Shock speed: The Rankine–Hugoniot relation	183
6.1.4	Entropy weak solutions	184
6.2	Fundamentals of finite-volume methods	186
6.2.1	Difficulties with finite-difference methods applied to conservation laws	187
6.2.2	Finite-volume formulation and numerical flux functions	189
6.2.3	Numerical conservation and the Lax–Wendroff theorem	193
6.3	Some finite-volume methods	194
6.3.1	Numerical flux functions for linear advection	194
6.3.2	Numerical flux functions for nonlinear conservation laws	195
6.3.3	Finite-volume methods and the Riemann problem	196
6.3.4	Numerical examples	200
6.4	Total variation and finite-volume methods	202
6.4.1	Total-variation properties of scalar conservation law solutions	202
6.4.2	TVD schemes for conservation laws	205
6.5	Second-order accurate methods	208
6.5.1	Second-order time integration	208
6.5.2	Linear reconstruction and slope-limiting methods	208
6.5.3	Slope-limited second-order TVD schemes	211
6.5.4	Application of a slope-limited second-order TVD scheme	216
6.6	Some further thoughts	217
6.7	Exercises	220
III	Intermediate topics	223
7	Linear systems of equations	225
	Overview	225
7.1	Gaussian elimination	227
7.1.1	Taking advantage of sparsity	230
7.2	Reordering algorithms for Gaussian elimination	233
7.2.1	The reverse Cuthill–McKee algorithm	236
7.2.2	Elimination graphs and the minimum-degree algorithm	238
7.2.3	Nested dissection	241
7.3	Iterative methods	243
7.3.1	Polynomial iterative methods and preconditioning	244
7.3.2	Jacobi, Gauss–Seidel, and successive overrelaxation	246
7.4	Convergence theory for stationary iterative methods	250
7.5	Optimizing convergence of SOR	255
7.6	Incomplete factorizations	261
7.7	Polynomial methods and the Chebyshev iteration	264
7.7.1	Symmetric and positive-definite systems	266
7.7.2	Nonsymmetric systems	270
7.8	Krylov methods: The Arnoldi algorithm and GMRES	273
7.8.1	The Arnoldi algorithm	274
7.8.2	GMRES	275
7.9	Krylov methods: The Lanczos algorithm, MINRES, and conjugate gradient	281

7.9.1	MINRES	283
7.9.2	Conjugate gradient	284
7.10	Preconditioning	287
7.11	Generalized methods for nonsymmetric matrices	290
7.12	Some further thoughts	295
7.13	Exercises	297
8	Finite-element methods for elliptic partial differential equations	301
	Overview	301
8.1	Review of basic tools from functional analysis	302
8.1.1	All about function spaces	302
8.1.2	Function spaces C^0 , C^1 , and C^k	304
8.1.3	Normed vector spaces and Banach spaces	305
8.1.4	Inner product spaces and Hilbert spaces	307
8.1.5	Weak derivatives	309
8.1.6	Sobolev spaces	312
8.2	Existence and uniqueness	314
8.2.1	Riesz representation theorem	315
8.2.2	The Lax–Milgram theorem	317
8.3	Types of elements and approximation properties	322
8.3.1	Triangular and tetrahedral elements, P_k	323
8.3.2	Quadrilateral and hexahedral elements, Q_k	329
8.4	More general problems	332
8.4.1	Nonsymmetric bilinear forms	332
8.4.2	Nonhomogeneous Dirichlet boundary conditions	335
8.4.3	Non-Dirichlet boundary conditions	336
8.5	Finite-element assembly	337
8.5.1	Weak-form assembly	339
8.5.2	Boundary conditions	345
8.5.3	Example	347
8.6	A posteriori error estimates	347
8.6.1	Residual-based error estimates	349
8.6.2	Other types of error estimates	351
8.7	Finite-element methods for time-dependent PDEs	353
8.8	Some further thoughts	355
8.9	Exercises	356
9	Finite-volume methods for systems of conservation laws	361
	Overview	361
9.1	Systems of hyperbolic conservation laws	362
9.1.1	Linear hyperbolic systems in 1D	363
9.1.2	Systems of nonlinear hyperbolic conservation laws in 1D	368
9.2	Finite-volume methods for systems of equations	372
9.2.1	The Lax–Friedrichs numerical flux function for systems	373
9.2.2	The Roe numerical flux function	373
9.2.3	Numerical example	377
9.3	Multidimensional problems	378
9.3.1	Finite-volume methods on Cartesian grids	378
9.3.2	Boundary conditions and ghost cells	379
9.3.3	Finite-volume methods on unstructured grids	383

9.4	Essentially nonoscillatory finite-volume methods	384
9.4.1	Stencils for high-order reconstruction	387
9.4.2	ENO reconstruction	389
9.4.3	WENO reconstruction	392
9.4.4	TVD Runge–Kutta methods for high-order time integration	395
9.4.5	Example: WENO for Burgers’ equation in 1D	397
9.4.6	WENO for systems and in multiple spatial dimensions	399
9.5	Discontinuous Galerkin schemes for hyperbolic conservation laws	400
9.5.1	The discontinuous Galerkin method	400
9.5.2	Implementation	404
9.5.3	Nonlinear equations	408
9.6	Implicit time-integration for nonlinear PDEs	411
9.6.1	Backward differentiation methods	413
9.6.2	Implicit Runge–Kutta methods	414
9.6.3	Implicit-explicit Runge–Kutta methods	416
9.7	Some further thoughts	418
9.8	Exercises	420
IV	Advanced topics	425
10	Advanced discretizations	427
	Overview	427
10.1	Staggered and mimetic finite-difference discretizations	428
10.1.1	Marker-and-cell discretizations	428
10.1.2	The Yee scheme and mimetic finite-volume discretizations	433
10.2	Finite-element discretizations in spaces other than H^1	434
10.2.1	Raviart–Thomas and Nédélec finite elements	435
10.2.2	H^2 -conforming discretizations	445
10.3	Mixed finite-element methods	450
10.3.1	Continuous inf-sup conditions	452
10.3.2	Discrete inf-sup conditions and mixed Poisson	455
10.3.3	Weak formulation of the Stokes equations	460
10.3.4	Mixed finite-element discretization of the Stokes equations	463
10.4	Least-squares finite-element methods	469
10.4.1	FOSLS discretization	471
10.4.2	A posteriori error estimates	475
10.4.3	Stokes example	477
10.5	Discontinuous Galerkin finite-element methods	480
10.5.1	Example	486
10.6	Spectral methods	487
10.6.1	Spectral discretization	488
10.6.2	High-order spectral differencing	490
10.6.3	The fast Fourier transform	492
10.6.4	Fast Poisson solvers	494
10.7	Some further thoughts	495
11	Advanced linear solvers	497
	Overview	497
11.1	Domain decomposition methods	498

11.1.1	Basic Schwarz methods	499
11.1.2	Advanced Schwarz methods	505
11.1.3	Two-level methods	510
11.2	Multigrid methods	515
11.2.1	The smoothing effect	516
11.2.2	Coarse-grid correction	519
11.2.3	The multigrid μ -cycle	524
11.2.4	Analysis of multigrid methods	530
11.3	Algebraic multigrid	533
11.3.1	Basics of algebraic multigrid	536
11.3.2	Smoothed aggregation	539
11.3.3	Algebraic multigrid: C/F splittings	547
11.4	Block-structured systems of equations	553
11.4.1	Block-factorization preconditioners	554
11.4.2	Monolithic multigrid methods	556
11.4.3	Time-dependent problems	558
11.5	Some further thoughts	559
Bibliography		563
Index		585

Preface

The study of partial differential equations (PDEs) plays a central role in applied mathematics, as PDEs can be used to model many of the physical systems in the world around us. Since the invention of digital computers, the *numerical solution* of differential equations has been essential for a vast array of scientific and engineering applications, both in conveniences of modern life, such as accurate short- and medium-term weather forecasting, and in major scientific research efforts or industrial projects. The latter includes the design of nuclear fusion reactors; climate modeling; the design of advanced aircraft, spacecraft, ships, and cars; modeling in biology and finance, and increasingly in medical applications, such as understanding muscle response and biomechanical abilities. With so many potential avenues for the use of numerical algorithms in this area, it is no surprise that courses in numerical methods for differential equations are widespread in undergraduate and graduate programs in mathematics, computer science, physics, engineering, and numerous inter- and multidisciplinary programs of study that span these and other areas.

This book arose out of several such courses taught by the authors across their universities over a period of many years. Each of these courses had its own emphasis, driven by our individual interests, but there were many common threads. In writing this book, we have aimed to compile a broad selection of topics at the level of final-year undergraduate or first-year graduate students who have a sufficient background in applied mathematics. A basic understanding of the underlying differential equations, along with a first course in numerical methods, covering polynomial interpolation and approximations of derivatives and integrals, is also assumed. In places where more advanced mathematics is needed, such as in the analysis of finite-element methods or the development of shock waves in hyperbolic equations, we have included brief summaries of the material with references to more in-depth treatments.

Mission statement

The main goal of this book is to prepare students for research or professional work in the field of numerical methods for differential equations, whether that is in the development and analysis of new discretizations and/or solution algorithms or in applying numerical methods to PDEs in science or engineering. We emphasize that this book is intended as a broad survey of methods, aimed to introduce the reader to the central concepts for various families of discretizations and solution algorithms, and not as a research monograph highlighting the latest advances in any one area. Our goal, rather, is to lay the foundation necessary to read such research monographs and/or journal papers that assume sufficient background in the general techniques to explain their advances.

Road map

The book contains four parts, featuring background, introductory, intermediate, and advanced materials. Part I contains some basic background on PDEs and numerical methods. Part II introduces the three main classes of numerical methods for PDEs that this book focuses on: finite-difference, finite-element, and finite-volume methods. Part III discusses linear solvers and extends the material on finite-element and finite-volume methods at a more advanced level. Part IV, finally, presents further higher-level topics on discretizations and solvers.

Just as there are many reasons to study numerical methods for PDEs, there are many possible paths through this book. Some courses will take a broad survey approach, covering the main topics in Part II, introducing finite-difference and finite-element methods for elliptic differential equations and finite-difference and finite-volume methods for time-dependent differential equations. Other courses may focus on just the elliptic case or just the time-dependent case, covering selected chapters from Parts II, III, and IV. Among ourselves, we have taught courses using several variants.

Even for a broad survey course, there are several possible paths through Part II, depending on instructor preference and what, if anything, the instructor wishes to include from later chapters. Figure P.1 shows three possible configurations:

- one that omits the finite-volume method for hyperbolic equations in favor of including some numerical linear algebra, leading to a deeper traversal of the text while leaving several large topics to later semesters;
- one that surveys methods for time-dependent PDEs before elliptic PDEs, offering a path through the text with wide, but not exhaustive, coverage; and
- one that proceeds by method, first covering finite-difference methods, then finite-volume methods, then finite-element methods, yielding a breadth of coverage in methods, yet omitting some extended topics in each case.

All of these are structured as one-semester courses necessitating some hard choices in what material to include and exclude from later sections in each chapter and/or later chapters in the book. An alternative approach is to consider a two-semester sequence with topics grouped by the type of equations being considered, as shown in Figure P.2, where one semester focuses on elliptic PDEs, including either some linear algebra or details of more advanced discretizations, and the other semester focuses on time-dependent PDEs.

Other paths are, of course, possible, as is supplementing the material presented here with more advanced material from research monographs in a particular topic area. We note that there are few (if any) dependencies between chapters within a part of the book, and only the obvious dependencies between chapters in the basic, intermediate, and advanced parts of the book. While many excellent books present numerical linear algebra at the same level as this book, we include a survey on both iterative methods for linear systems and advanced topics on domain decomposition and multigrid methods, as these techniques are often tightly entwined with the discretization techniques that we teach in our numerical PDEs courses.

Other books on numerical PDEs

There are many excellent books in the field of numerical solutions of differential equations. Our motivation in writing *this* book was to collate material appropriate for teaching broad survey courses at the level of final-year undergraduate or beginning graduate students. Having taught such courses over many years, we found ourselves reaching for a wide variety of books needed to cover the material presented here. These books are outstanding resources, and we want to

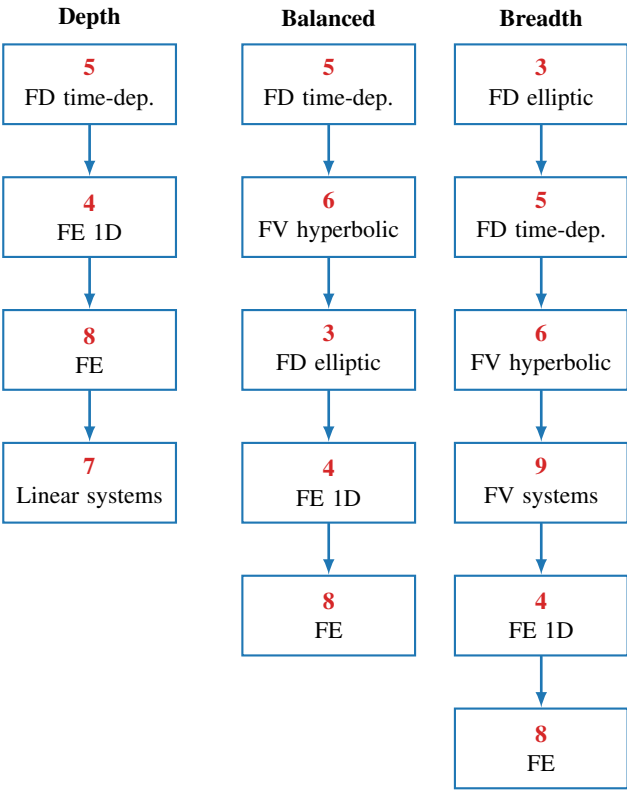


Figure P.1. Examples of one-semester paths through the chapters.

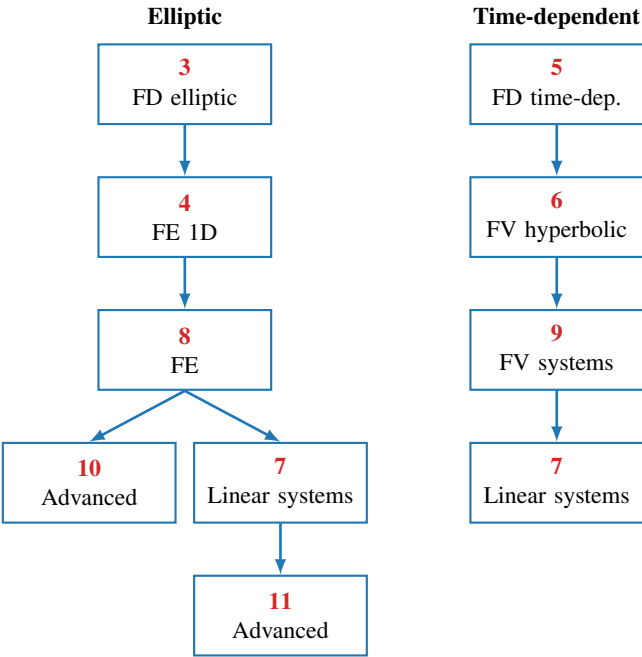


Figure P.2. Example of a two-semester sequence.

acknowledge the inspiration and impact that these books have had, both on our teaching and on our writing here. In the text we have included over one hundred well-established definitions, theorems, corollaries, and lemmas as part of the presentation. Some of these include classical results such as Taylor's theorem, for which we do not provide references. Others are drawn from more recent literature, and for these we cite either original works or standard texts for additional information.

Much of Part II focuses on the finite-difference methodology (Chapters 3 and 5), which is covered in detail in the books by LeVeque [152] and by Strikwerda [208]. Our presentation of finite-element methods, in Chapters 4 and 8, was heavily influenced by the classic text of Brenner and Scott [52]. A major part of Part III is Chapter 7, focusing on algorithms for the solution of large linear systems. The material on iterative methods was informed by the well-known book of Saad [194], supplemented by the monograph of Varga [226], while the introductory material on direct methods was gathered from several sources [105, 90, 109]. Chapters 6 and 9, on finite-volume methods, was informed by LeVeque [151] and the more recent book by Hesthaven [131]. Part IV contains material drawn from a much wider range of sources, including both journals and research monographs.

Acknowledgments

Books take time to write, of course. They also take time to research, to review, and to polish. For this, we are indebted to our colleagues and friends who have helped both to shape the initial vision for the book and to refine the text by reading and by using early drafts in their classrooms.

First and foremost, we thank Tom Manteuffel for pushing us along this path. All of us first learned much of what we know about this subject from Tom and his introductory graduate course on numerical methods for PDEs at CU-Boulder. We are grateful for his deep insights, his guidance, and his suggestion that started us on this project.

We also give a special thanks to an incomplete list of amazing colleagues and students who have helped to stress-test the content and find errors in the book. Thanks to Professors Andreas Klöckner and Paul Fischer at the University of Illinois Urbana-Champaign, Ludmil Zikatanov at The Pennsylvania State University, Ron Haynes at Memorial University of Newfoundland, and Xiaozhe Hu at Tufts University. In addition, we thank James Jackaman, Oliver Krzysik, Amin Rafiei, and Tareq Uz Zaman for finding numerous errors and weak points (including several quite hard-to-find mistakes!) in the text.

Part I

Introduction and background

Chapter 1

About partial differential equations

Overview

Partial differential equations (PDEs) play an important role in modeling physical phenomena in nearly every field of the physical and engineering sciences and in areas such as finance and the biomedical sciences. Given the vast range of applications, PDEs take on a variety of forms, and this requires careful attention when developing numerical methods that are suitable for these different types of PDE problems. Even so, a basic set of prototypical PDEs can often be used to guide numerical development and exploit features that may be common to different problems.

The PDEs discussed in this book can be broadly categorized as one of three types: *elliptic*, *parabolic*, or *hyperbolic*. For example, a potential problem of the form

$$-u_{xx} - u_{yy} = f(x, y) \quad (1.1)$$

is classified as elliptic, while a wave problem of the form

$$u_{tt} = c^2 u_{xx} \quad (1.2)$$

is classified as hyperbolic.

Objectives

In this chapter, we will introduce salient features of some first-order and second-order PDEs. We will uncover key properties of prototypical PDEs of different types. From this introductory overview, you will be able to identify linear and nonlinear PDEs and define key properties of PDEs. You will also recognize the origins of some important first- and second-order PDEs that commonly appear in mathematical models.

Chapter Outline

Section 1.1 Introduction motivates the use of PDEs and simple prototypes for applications in science and engineering and introduces PDEs and some of their basic properties.

Section 1.2 First-order PDEs focuses on first-order PDEs in the context of conservation laws for physical systems. The *linear advection equation* is introduced as a model problem that will be used throughout the book.

Section 1.3 Second-order linear PDEs considers second-order PDEs and provides an overview of their classification. The *heat equation* and *Poisson's equation* will be introduced as model problems that are considered later in the book. Some key properties, such as characteristic curves and maximum principles, will be highlighted.

1.1 ■ Introduction

Objectives

This section highlights the important role PDEs play in modeling applications in the natural sciences and beyond. You will learn that the development of numerical methods focuses on prototype examples of PDEs, but the resulting techniques can be used to solve challenging modeling problems in the real world. PDEs are introduced, and some of their basic properties, such as order and linearity, are discussed.

1.1.1 ■ Motivation

PDEs play a central role in mathematical models of the physical world, as they naturally arise when expressing conservation laws and equilibrium conditions. As such, there is a long history of interest in their solution, dating back to the 18th century and work of d'Alembert, Euler, Lagrange, Laplace, and others. In the 18th and 19th centuries, analytical techniques for expressing solutions, in terms of Fourier series, power series, or Green's functions, were developed. These methods now form the core of many introductory courses and textbooks on PDEs.

While such methods are important, both from a theoretical perspective and in many practical settings, they are not universally applicable, and, even when they do yield expressions for the solutions, recovering accurate estimates of pointwise or averaged values may be difficult. Motivated by these essential complications, two fields of study have emerged since the mid-20th century. *Computational science and engineering (CSE)* (also referred to as *scientific computing*) focuses on the use of numerical algorithms to simulate real-world scientific and engineering phenomena, while *numerical analysis* studies the accuracy and efficiency of these algorithmic approaches. Numerical methods for PDEs play an important role in both of these fields, as many of the physical processes in CSE applications are modeled by PDEs. Thus, their simulation is essential. Additionally, an analytical understanding of the simulation tools helps refine and improve the models.

The overall process of CSE can be viewed as a series of steps that begins with a question from the underlying scientific or engineering application. To answer that question, a mathematical model is developed that must include sufficient physical detail to be reliable, but not so much detail that the simulation is limited by resolving physical effects that have only a small impact on the question at hand. From this mathematical model (which, in the context of this book, we consider to be a governing PDE or system of PDEs with appropriate initial and boundary-value data), a discrete (computational) model is derived and numerical results are computed. These results are, typically, postprocessed to a quantitative answer to the posed question. In practice, this process is iterative (see Figure 1.1.1), with refinements to the mathematical and computational models naturally incorporated to improve physical fidelity and numerical accuracy once initial results are obtained.

While identifying the correct equations to be solved is a fundamental step in CSE, it is also very application specific, depending on the physical process to be simulated and the relevant values of physical parameters within the model. Our approach to numerical methods for PDEs is to develop the methods and analysis for prototypical equations from several important classes, starting from an assumption of familiarity with these basic equations and their properties (reviewed in this chapter and as needed), focusing on understanding both the “discretization” process and the efficiency with which the resulting discrete systems of equations can be solved.

As a motivating example, consider modeling the flow of a single, homogeneous fluid in a given spatial domain. This problem arises in many scientific and engineering applications, such as in meteorology and climate modeling (considering the atmospheric flow around Earth); in design of cars, ships, airplanes, and spacecraft (considering the flow of air or water around

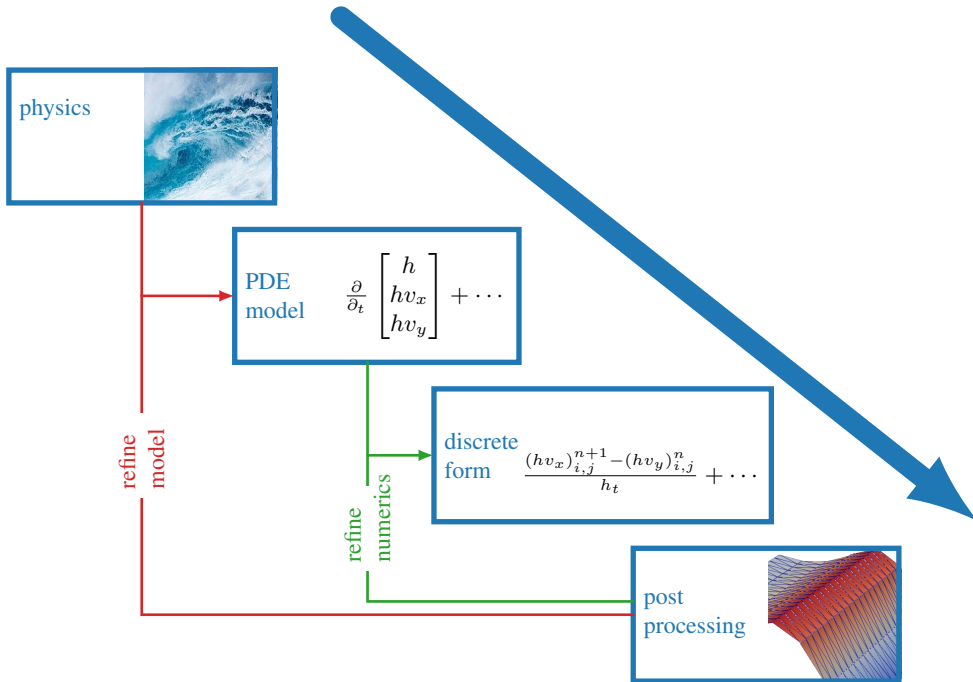


Figure 1.1.1. CSE simulation process. Ocean wave image credit: CC0 public domain.

the structure of a vehicle); and in many other industrial processes where fluid flow plays a key role. Many common fluids are *Newtonian* and *incompressible* and, as a result, their flow can be modeled by the incompressible Navier–Stokes equations. Considering a three-dimensional spatial domain, the Navier–Stokes equations are a coupled nonlinear PDE system with four scalar equations,

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) - \nabla \cdot (\mu (\nabla \mathbf{u} + \nabla \mathbf{u}^T)) + \nabla p = \rho \mathbf{f}, \quad (1.3a)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (1.3b)$$

in four unknown scalar functions: the fluid velocity $\mathbf{u}(x, y, z, t)$ (a vector with three components) and the pressure $p(x, y, z, t)$. Here, ρ and μ are the density and viscosity of the fluid, respectively, which can be assumed to be known constants, and $\mathbf{f}(x, y, z)$ represents known external (body) forces acting on the fluid, such as gravity.

Analytical solutions of the incompressible Navier–Stokes equations are available only in a few cases, typically considering idealized geometry and/or boundary conditions. Indeed, it remains an open question as to whether or not suitably smooth and bounded solutions to the equations exist and are unique in three space dimensions. Given this, it is no surprise that a subfield of CSE, computational fluid dynamics or CFD, has been a major driver of research into numerical methods for PDEs since the 1960s. Finding numerical approximations to the solution of these equations has many challenges, as they are a coupled system of nonlinear, time-dependent equations. Adding additional physical effects, such as a Coriolis force (for atmospheric flow), compressibility of the fluid, or non-Newtonian behavior, as seen in many common fluids, such as latex paint and ketchup, further complicates the simulation. In order to develop the building blocks for such a simulation, we first study the much simpler cases of scalar first- and second-order PDEs. We develop the basic methodology for each of these pieces and others, which can be combined into complex simulations, such as for Navier–Stokes flow.

1.1.2 ■ Partial differential equations

We now introduce *PDEs*, providing some definitions and notations that we use throughout the book. To begin we consider an open, connected domain $\Omega \in \mathbb{R}^d$ and an unknown, real-valued function $u(\mathbf{x})$ of $\mathbf{x} = (x_1, x_2, \dots, x_d) \in \Omega$. Then, the partial derivatives of $u(\mathbf{x})$,

$$u_{x_i} = \frac{\partial u}{\partial x_i} = \partial_{x_i} u, \quad u_{x_i x_j} = \frac{\partial^2 u}{\partial x_j \partial x_i} = \partial_{x_i x_j} u, \quad \dots, \quad (1.4)$$

are also real-valued functions on Ω . Throughout this book, we may refer to one-, two-, and three-dimensional spaces as 1D, 2D, and 3D, respectively. This leads to a formal definition of a PDE given in Definition 1.1 and the definition of the order of a PDE in Definition 1.2.

Definition 1.1: A partial differential equation

A scalar partial differential equation is a scalar equation involving $\mathbf{x}$, u , and the partial derivatives of u .

We also consider systems of PDEs where some of the unknown functions may be vector valued—e.g., $\mathbf{u} = (u, v, w)$ in equation (1.3).

Definition 1.2: Order of a PDE

The order of a PDE is the order of the highest-order partial derivative present in the PDE.

For example, the PDE

$$u_{xx} + uu_{yy} + u_x = f(x, y), \quad (1.5)$$

is second order, while

$$u_t + u_x = f(x, t) \quad (1.6)$$

is first order. The Navier–Stokes equations are an example of a *second-order* system of PDEs.

In addition, the PDE may be linear or nonlinear in u and the partial derivatives of u . It is also useful to define the following subsets of types of nonlinear PDEs in Definitions 1.3 and 1.4.

Definition 1.3: A quasi-linear PDE

A nonlinear PDE in u in which the highest-order partial derivatives appear linearly with coefficients depending only on the lower-order derivatives of u and the independent variables, $\mathbf{x}$, is called a quasi-linear PDE.

For example, the PDE

$$xu^2 u_{xx} + (u_x + y) u_{yy} + y u_y^3 = f(x, y) \quad (1.7)$$

is quasi-linear since u_{xx} and u_{yy} both appear linearly. Clearly, the PDE is nonlinear because of the u^2 factor and the u_x and u_y^3 terms.

Definition 1.4: A semilinear PDE

A nonlinear PDE is called semilinear if it is quasi-linear and if the highest-order terms have coefficients that depend only on the independent variables, $\mathbf{x}$.

For example, the following PDE is semilinear:

$$(x + y + z) u_x + z^2 u_y + \sin(x) u_z + u^2 = f(x, y, z). \quad (1.8)$$

With these basic definitions in hand, we next discuss a few important properties of some prototype examples of PDEs. Understanding these properties makes the development of numerical

methods to solve these equations more robust, as we aim to capture similar properties at the discrete level. We focus on first-order and second-order PDEs, as they are the most ubiquitous in applications.

1.2 ■ First-order PDEs

Objectives

In this section, you will learn about some first-order PDEs that are related to the concept of *conservation* in a physical system. You will be able to associate the first-order linear advection problem with characteristic curves in the space-time domain and derive some classical PDEs from fluid mechanics.

The first major focus of this book is on time-dependent *first-order* PDEs that describe fundamental conservation principles. These may be used to model a variety of applications in science and engineering, for example, traffic flow, the propagation of waves in shallow water, and the dynamics of compressible fluids, including the flow of air around airplanes [151]. We consider scalar PDEs and PDE systems, linear and nonlinear, and in one or multiple spatial dimensions. The second major focus of this book is on second-order PDEs, which are introduced in the next section.

1.2.1 ■ Scalar first-order conservation laws

In three spatial dimensions, the general form of a conservation law for the scalar conserved quantity $u(x, y, z, t)$ is given by

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{f}(u) = 0, \quad (1.9)$$

where $\mathbf{f}(u) = (f(u), g(u), h(u))$ is called the *flux vector*. In this section, we assume that $\mathbf{f}(u)$ depends on u but not on derivatives of u . Then, equation (1.9) is a first-order PDE. We also assume that the flux function $\mathbf{f}(u)$ is a differentiable function of u , noting that the flux function is nonlinear in many applications. The scalar first-order conservation law (1.9) is called *hyperbolic* because there are characteristic curves $(x(t), y(t), z(t))$ along which the PDE reduces to an ordinary differential equation (ODE).

Integrating equation (1.9) over any spatial region $\Omega \subset \mathbb{R}^3$ leads to an explanation for why equation (1.9) is called a *conservation law*. Indeed, using the divergence theorem (see equation (2.90)), we obtain the *integral form* of the conservation law,

$$\frac{d\bar{u}(t)}{dt} + \int_{\partial\Omega} \mathbf{f}(u(\mathbf{x}, t)) \cdot \mathbf{n} \, ds = 0 \quad \text{for any } \Omega \subset \mathbb{R}^3, \quad (1.10)$$

where $\mathbf{n}$ is the outward-pointing unit vector normal to the boundary surface $\partial\Omega$ of domain Ω and

$$\bar{u}(t) = \int_{\Omega} u(\mathbf{x}, t) \, d\mathbf{x}. \quad (1.11)$$

This expresses conservation: the integrated quantity $\bar{u}(t)$ remains constant in time unless there is a net flux through the boundary $\partial\Omega$. For continuously differentiable functions $u \in C^1$, equations (1.9) and (1.10) are equivalent. If a solution, u , of conservation law (1.9) is in C^1 , then u is called a *classical solution*.

In addition, integrating equation (1.10) over time interval $[t_1, t_2]$ gives another *integral form* of the conservation law,

$$\bar{u}(t_2) - \bar{u}(t_1) + \int_{t_1}^{t_2} \int_{\partial\Omega} \mathbf{f}(u(\mathbf{x}, t)) \cdot \mathbf{n} \, ds \, dt = 0, \quad (1.12)$$

which shows that the net change in $\bar{u}$ is balanced by the temporal integral of the net flux through the boundary. Similar expressions are obtained for conservation laws in 1D and 2D, and any other dimension, in fact. In this form, we easily identify u as the *density* of a conserved quantity in Ω , with $\bar{u}$ denoting the total amount of the conserved quantity in Ω .

Finally, it is important to note that in some physical models a source term may be added to first-order conservation law (1.9):

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{f}(u) = q. \quad (1.13)$$

Here, the source term q may be of the form $q(u(\mathbf{x}, t), \mathbf{x}, t)$ or may depend on second derivatives of u in dissipative systems. In this sense, conservation law (1.9) is often called an *ideal* conservation law because there are no dissipative effects.

1.2.2 ■ The linear advection equation in 1D

One of the simplest scalar conservation laws is the 1D *linear advection equation*,

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0, \quad (1.14)$$

where a is a given constant representing the advection speed. This equation is obtained by considering equation (1.9) in 1D and assuming a linear flux: $f(u) = a u$. The 1D linear advection equation is also called the *one-way wave equation*, since the second-order wave equation can be factored as

$$\frac{\partial^2 u}{\partial t^2} - a^2 \frac{\partial^2 u}{\partial x^2} = \left(\frac{\partial}{\partial t} + a \frac{\partial}{\partial x} \right) \left(\frac{\partial}{\partial t} - a \frac{\partial}{\partial x} \right) u = 0, \quad (1.15)$$

representing a combination of right-traveling and left-traveling waves.

Given initial condition $u(x, 0) = g(x)$ at $t = 0$, the exact solution of the 1D linear advection equation is given by

$$u(x, t) = g(x - a t). \quad (1.16)$$

Indeed, defining $s = x - a t$, we obtain

$$\frac{\partial u}{\partial t} = \frac{dg}{ds} \frac{\partial s}{\partial t} = -a \frac{dg}{ds} \quad \text{and} \quad \frac{\partial u}{\partial x} = \frac{dg}{ds} \frac{\partial s}{\partial x} = \frac{dg}{ds}, \quad (1.17)$$

such that equation (1.14) is satisfied.

The concept of characteristic curves is fundamental for conservation laws. Consider how a solution $u(x, t)$ to the linear advection equation varies along a curve $x(t)$ —i.e., consider $u(x(t), t)$. By the chain rule, we have

$$\frac{du(x(t), t)}{dt} = \frac{\partial u}{\partial x} \frac{dx}{dt} + \frac{\partial u}{\partial t}, \quad (1.18)$$

which implies that

$$\frac{du(x(t), t)}{dt} = 0 \quad \text{if} \quad \frac{dx}{dt} = a. \quad (1.19)$$

Hence, the linear advection solution $u(x, t)$ is constant along curves $x(t) = x_0 + a t$ with slope a . Along these *characteristic curves*, the linear advection equation, equation (1.14), reduces to the ODE

$$\frac{dw(t)}{dt} = 0, \quad (1.20)$$

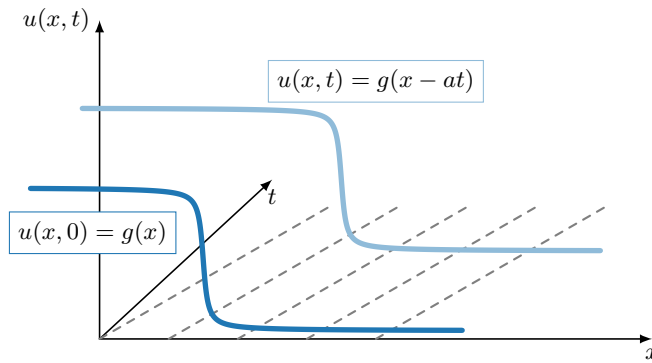


Figure 1.2.1. Linear advection in 1D. The dotted lines are characteristic curves in the x - t plane, with slope a . The initial profile $g(x)$ is advected with speed a . The solution $u(x, t)$ is constant along the characteristics $x(t) = x_0 + at$.

for function $w(t) = u(x(t), t)$, so the solution $u(x, t)$ is constant along the characteristic curves (see Figure 1.2.1). As a consequence, initial and boundary data cannot be specified along characteristic curves (or they could violate the properties observed above), and the PDE is classified as hyperbolic.

The 1D linear advection equation naturally extends to higher spatial dimensions. For example, the linear advection equation in 3D is given by

$$\frac{\partial u}{\partial t} + (a, b, c) \cdot \nabla u = 0, \quad (1.21)$$

which corresponds to flux function $\mathbf{f}(u) = (au, bu, cu)$ in equation (1.9). Here, we see that a , b , and c are the advection speeds in the x , y , and z directions, respectively.

1.2.3 ■ Conservation with flow

Conservation laws (1.13) with source term q take on a specific form when we consider conserved quantities that are being “carried” by a flow. If the conserved quantity $u(\mathbf{x}, t)$ is being “pushed” through a control volume $\Omega \subset \mathbb{R}^d$ with velocity $\mathbf{v}$ (see Figure 1.2.2 for a 2D example), we find a specific structure in the flux function $\mathbf{f}(u)$.

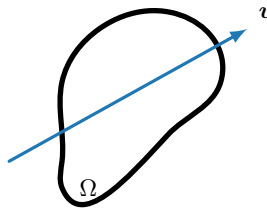


Figure 1.2.2. Fluid moving through a control volume $\Omega \subset \mathbb{R}^2$ with velocity $\mathbf{v}$.

In order to derive conservation laws in this setting, we need to consider how the total amount of the conserved quantity changes with time. That is, we seek to find an expression for

$$\frac{d}{dt} \int_{\Omega} u(\mathbf{x}, t) d\mathbf{x}. \quad (1.22)$$

Here, the fact that u is “flowing” over the boundary of Ω is important, since this results in a net flux of u in Ω that depends on the value of uv on $\partial\Omega$. The amount of u that flows out of the domain at time t is given by $\int_{\partial\Omega} u(\mathbf{x}, t) \mathbf{v} \cdot \mathbf{n} d\mathbf{s}$, representing the total normal flux of u through the boundary $\partial\Omega$. This gives us a general form of the conservation law as

$$\frac{d}{dt} \int_{\Omega} u(\mathbf{x}, t) d\mathbf{x} = - \underbrace{\int_{\partial\Omega} u(\mathbf{x}, t) \mathbf{v} \cdot \mathbf{n} d\mathbf{s}}_{\text{net flux}} + \int_{\Omega} \underbrace{q}_{\text{source/sink}} d\mathbf{x}. \quad (1.23)$$

As before, using the divergence theorem gives

$$\int_{\Omega} \frac{\partial u}{\partial t} d\mathbf{x} = - \int_{\Omega} \nabla \cdot (u\mathbf{v}) d\mathbf{x} + \int_{\Omega} q d\mathbf{x} \quad (1.24a)$$

$$\Rightarrow \int_{\Omega} \left(\frac{\partial u}{\partial t} + \nabla \cdot (u\mathbf{v}) - q \right) d\mathbf{x} = 0. \quad (1.24b)$$

Since this is true for any arbitrary volume Ω , we obtain a PDE in the form of conservation law (1.13),

$$\frac{\partial u}{\partial t} + \nabla \cdot (u\mathbf{v}) = q, \quad (1.25)$$

where the flux vector is now given by $\mathbf{f}(u) = u\mathbf{v}$. Next we highlight some examples of different conserved quantities, which ultimately lead to some classical PDEs in fluid mechanics, such as the Navier–Stokes and Euler equations. In some cases, the sources and sinks, q , can be represented as part of the flux term. For example, in 3D, if $q = \frac{\partial p}{\partial x}$ for some function p , then we can write $q = \nabla \cdot (p, 0, 0)$ and p can be added to the flux function in the x -direction.

Consider $u(\mathbf{x}, t) = \rho(\mathbf{x}, t)$ to be the mass density of a fluid moving with velocity $\mathbf{v}$, and assume that there are no sources or sinks for mass, so $q = 0$ in this context. Then, conservation of mass becomes

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho\mathbf{v}) = 0. \quad (1.26)$$

In the case of an *incompressible* fluid (i.e., $\rho = \text{constant}$), this simplifies to $\nabla \cdot \mathbf{v} = 0$.

Next, consider $\mathbf{u}(\mathbf{x}, t)$ to represent the momentum of a fluid moving with velocity $\mathbf{v}$, where the momentum equals mass density times velocity, i.e., $\mathbf{u} = \rho\mathbf{v}$. In this case, the momentum $\mathbf{u}$ is a *vector* quantity, and we can derive conservation laws for each component of $\mathbf{u}$, writing in 3D say, $u_i = \rho v_i$ for $i = 1, 2, 3$, to get the PDE system in conservation form:

$$\frac{\partial(\rho v_i)}{\partial t} + \nabla \cdot (\rho v_i \mathbf{v}) = q_i. \quad (1.27)$$

Rewriting this system in nonconservative form gives

$$\frac{\partial \rho}{\partial t} v_i + \rho \frac{\partial v_i}{\partial t} + v_i \nabla \cdot (\rho\mathbf{v}) + \rho \mathbf{v} \cdot \nabla v_i = q_i. \quad (1.28)$$

If we rearrange the equation slightly, we notice that the equation for conservation of mass appears within this expression and can be eliminated:

$$\underbrace{v_i \left(\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho\mathbf{v}) \right)}_{\text{conservation of mass}} + \rho \left(\frac{\partial v_i}{\partial t} + \mathbf{v} \cdot \nabla v_i \right) = q_i. \quad (1.29)$$

This simplification can often be applied in fluid dynamics equations (for example, in the Navier–Stokes equations). Rewriting this in vector notation, we obtain an evolution equation for the velocity:

$$\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} = \frac{1}{\rho} \mathbf{q}. \quad (1.30)$$

Here, the sinks and sources of momentum are force densities acting on the fluid, according to Newton’s second law,

$$\rho \frac{d\mathbf{v}}{dt} = \mathbf{q}, \quad (1.31)$$

where

$$\frac{d\mathbf{v}(\mathbf{x}(t), t)}{dt} = \frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} \quad (1.32)$$

is the total derivative of the velocity of a fluid element moving with velocity $\mathbf{v}$, with

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}(\mathbf{x}, t). \quad (1.33)$$

Possible forces include fluid pressure ($\mathbf{q}_p = -\nabla p$), fluid viscosity ($\mathbf{q}_\mu = \mu \nabla^2 \mathbf{v}$, which makes the system second order), gravity, and the Coriolis force.

1.3 ■ Second-order linear PDEs

Objectives

In this section, you will derive diffusion models from conservation laws to obtain some second-order PDEs. You will then learn how to characterize general second-order PDEs as elliptic, hyperbolic, or parabolic. In addition, you will see how to associate each of these types of equations with key properties of a physical system.

In Section 1.2, we considered examples of conservation laws that led to first-order PDEs, namely,

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{f}(u) = 0, \quad (1.34)$$

where the flux of the conserved quantity, $\mathbf{f}(u)$, depends on u but not on derivatives of u . For a material that interacts with the surrounding medium as a fluid, $\mathbf{f}$ was related to the velocity of that fluid. If we now assume that the quantity does not interact with the surrounding medium, then there are more options for the form of the flux. Consider conservation of mass once again in this setting:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{f}(\rho) = 0. \quad (1.35)$$

To “close” the system, we need to specify how $\mathbf{f}$ depends on ρ and possibly derivatives of ρ . Below are some examples where $\mathbf{f}$ depends on the gradient of ρ , leading to standard diffusion-type second-order PDEs.

Remark 1.5: Diffusion equations

In 1811, Joseph Fourier experimented with heat conduction and observed that heat flows from hotter to colder places with a flux that is proportional to the temperature difference. This led to Fourier’s law of heat conduction:

$$\mathbf{f}(\rho) = -k \nabla \rho. \quad (1.36)$$

In 1855, Adolf Fick observed the same relation for diffusion of chemicals:

$$\mathbf{f}(\rho) = -D \nabla \rho. \quad (1.37)$$

Here, D and k are positive constants determined by the medium being modeled. In more refined models, the coefficients may depend on x or ρ . In either case, the resulting conservation law is called a *diffusion equation*:

$$\frac{\partial \rho}{\partial t} - \nabla \cdot (k \nabla \rho) = q \quad (\text{heat equation}), \quad (1.38a)$$

$$\frac{\partial \rho}{\partial t} - \nabla \cdot (D \nabla \rho) = q \quad (\text{Fick's second law of diffusion}). \quad (1.38b)$$

Note that while the above are derived using macroscopic models for the flux, they can also be used to describe microscopic models, as Einstein did in 1905 to describe Brownian motion.

A special case of diffusion is the steady-state case, where $\frac{\partial \rho}{\partial t} = 0$ and $k(x)$ is independent of time:

$$-\nabla \cdot (k \nabla \rho) = q. \quad (1.39)$$

For a homogeneous material, k is a constant, and we obtain Poisson's equation:

$$-\nabla \cdot (\nabla \rho) = -\nabla^2 \rho = -\Delta \rho = \frac{q}{k}. \quad (1.40)$$

If $q = 0$, we get Laplace's equation:

$$-\nabla^2 \rho = -\Delta \rho = 0. \quad (1.41)$$

1.3.1 ■ Classification

In the examples of first-order and second-order linear PDEs that we have considered so far in this chapter, we have obtained a few different types of PDEs: wave like, diffusion based, and time independent. All were derived from conservation laws, but depending on the physics being modeled, the properties of the PDE and its solutions may be very different. To explore this further, we consider second-order linear PDEs in two spatial dimensions and then classify them. The benefit of this classification is that it highlights certain features of each PDE and its solutions, which ultimately helps to guide the development of numerical methods for their approximation. To this end, consider the linear second-order PDE in two independent variables,

$$a(x, y) u_{xx} + 2b(x, y) u_{xy} + c(x, y) u_{yy} + d(x, y) u_x + e(x, y) u_y + f(x, y) u = g(x, y), \quad (1.42)$$

where we note that $d(x, y) u_x + e(x, y) u_y + f(x, y) u$ are the low-order terms.

Table 1.3.1 classifies the PDE at a given point $(x, y) \in \Omega$ as elliptic, parabolic or hyperbolic, depending on the coefficients of the second-order terms. In this section, we just posit this classification and interpret it using example solutions of prototype PDEs of the three classes. We will later derive and interpret this classification based on the existence of characteristic curves; see Section 9.1.1.

Prototypical examples of the classification are as follows. If $b(x, y) = 0$ and $a(x, y) = c(x, y) = -1$ (and the low-order coefficients are zero), then equation (1.42) leads to the familiar Laplace operator, $-u_{xx} - u_{yy}$, which according to the classification is *elliptic*. Similarly, Table 1.3.2 also gives the two prototypical examples of the *parabolic* and *hyperbolic* cases, where we use t instead of y as the second independent variable, as is typical for these problems.

Table 1.3.1. *Classification of linear second-order PDEs, equation (1.42).*

	Criterion	Classification
(i)	$b^2 - ac < 0$	elliptic
(ii)	$b^2 - ac = 0$	parabolic
(iii)	$b^2 - ac > 0$	hyperbolic

Table 1.3.2. *Prototypical examples of linear second-order PDE operators of the three different types: elliptic, parabolic, hyperbolic.*

Operator	Common name	Classification
$-u_{xx} - u_{yy}$	2D Laplace operator	elliptic
$u_t - u_{xx}$	1D heat (or diffusion) operator	parabolic
$u_{tt} - u_{xx}$	1D wave operator	hyperbolic

These simple examples give only a basic view of second-order PDEs. An equation may change type at different points in the domain Ω . Consider the case of

$$u_{xx} + yu_{yy} = 0. \quad (1.43)$$

This PDE exhibits elliptic, parabolic, and hyperbolic properties, each in different parts of the domain. This is depicted in Figure 1.3.1, which indicates how the behavior changes as y varies.

With more than two independent variables, we have a more general classification scheme. Consider the general form of a second-order, d -dimensional, semilinear PDE operator,

$$Lu := \sum_{i=1}^d \sum_{j=1}^d a_{i,j}(\mathbf{x}) \frac{\partial^2 u}{\partial x_i \partial x_j} + \text{low-order terms}, \quad (1.44)$$

where $A = (a_{i,j}(\mathbf{x}))$ is a $d \times d$ matrix of functions that depend only on the independent variables, $\mathbf{x}$. We can usually assume that A is symmetric and, therefore, has real eigenvalues. Looking at

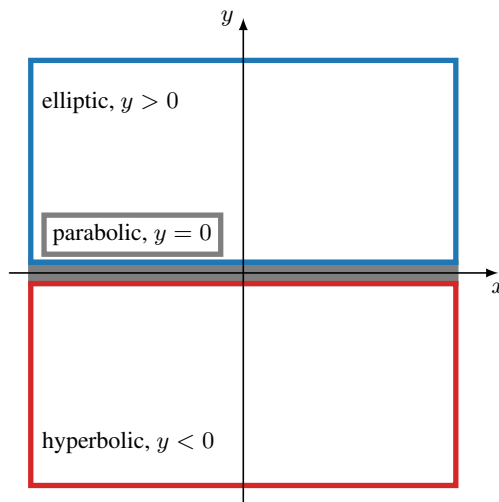
**Figure 1.3.1.** *Classification of equation (1.43).*

Table 1.3.3. *Higher-dimensional classification.*

	Criterion	Classification
(i)	$\lambda_j(\mathbf{x}) = 0$ for some j	parabolic
(ii)	$\lambda_j(\mathbf{x})$ all have the same sign	elliptic
(iii)	All but one $\lambda_j(\mathbf{x})$ have the same sign	hyperbolic
(iv)	A has more than one eigenvalue of each sign	ultrahyperbolic

the eigenvalue spectrum of A at point $\mathbf{x}$, denote the j th eigenvalue by $\lambda_j(\mathbf{x})$ for $j = 1, \dots, d$, ordered such that $\lambda_1 \leq \dots \leq \lambda_d$. For a point $\mathbf{x} \in \Omega$, we use the classification scheme in Table 1.3.3. We note that in 3D only (i)–(iii) apply, since (iv) is possible only in dimensions $d > 3$.

Elliptic

Unlike hyperbolic and parabolic PDEs, the behavior of an elliptic PDE does not expose a characteristic direction along which information propagates. Indeed, as we see in the following model problem, the character of the solution is *global*. This will impact the numerical methods in Chapters 3 and 4 that we develop for these problems and the sparse linear solvers in Chapter 7 used in the solution process.

The prototypical example of an elliptic problem is the 2D Poisson problem. Given an open domain $\Omega \subset \mathbb{R}^2$, consider the elliptic *boundary-value problem* (BVP)

$$\text{(BVP)} \quad \begin{cases} -\nabla \cdot (\nabla u) = f(\mathbf{x}) & \text{in } \Omega, \\ u = g(\mathbf{x}) & \text{on } \partial\Omega. \end{cases} \quad (1.45)$$

We consider two specific examples of this BVP that highlight the elliptic nature of the PDE. First, let Ω be the unit circle, $f(\mathbf{x}) = 0$, and $g(\mathbf{x}) = \cos(2\theta)$ (where $\theta = \arctan(y/x)$ is the angle in polar coordinates, with $r^2 = x^2 + y^2$), so that the unique solution of the BVP is given by $u(x, y) = r^2 \cos(2\theta)$. This solution is shown in the left panel of Figure 1.3.2. Here we see that the maximum (and minimum) of the solution are attained on the boundary of the domain, which is a general property of this type of elliptic BVP when $f(\mathbf{x}) = 0$.

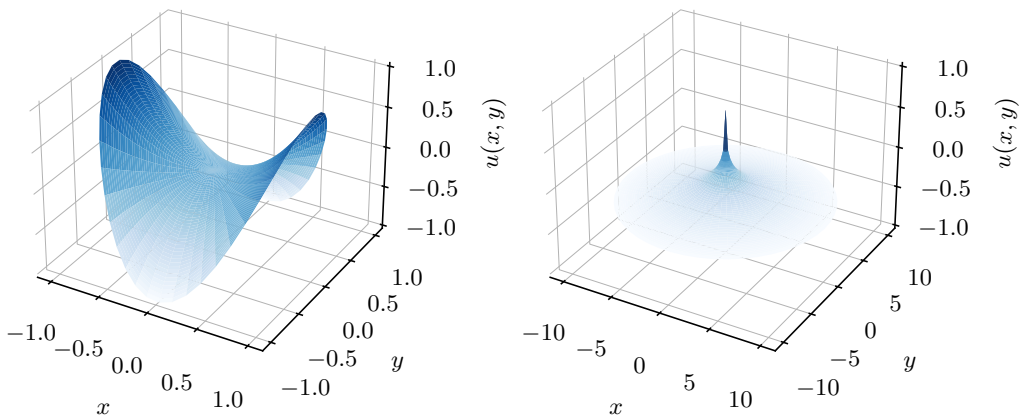


Figure 1.3.2. Two example solutions to elliptic BVP (1.45). (left) A solution that illustrates the maximum principle, with maxima/minima found on the boundary. (right) A point source has global impact on a solution.

Alternatively, consider $f(\mathbf{x}) = \delta(\mathbf{x} - \mathbf{0})$, the Dirac delta function, on an infinite domain $\Omega = \mathbb{R}^2$ and with free-space boundary conditions $u \rightarrow 0$ as $r \rightarrow \infty$. The problem describes a unit charge at the origin, and the unique solution of the BVP is the so-called fundamental solution $u(x, y) = -\frac{1}{2\pi} \ln r$, which describes the potential that is due to the unit charge. The solution is shown in the right panel of Figure 1.3.2, highlighting the global solution behavior. That is, the value and location of the point charge has a global impact on the solution, a trait that is not present in other types of PDEs.

Parabolic

The 1D heat or diffusion equation is an example of a parabolic PDE. A typical example of a parabolic *initial boundary-value problem* (IBVP) in a 1D spatial domain is given by

$$(IBVP) \quad \begin{cases} \frac{\partial u}{\partial t} - \frac{\partial^2 u}{\partial x^2} = 0 & \text{in } (0, 1) \times (0, T], \\ u(x, 0) = \sin(\pi x) & \text{for } x \in (0, 1), \\ u(0, t) = u(1, t) = 0 & \text{for } t \in (0, T]. \end{cases} \quad (1.46)$$

Here, $u(x, 0)$ may represent an initial concentration of a chemical substance, for example. This problem has a unique solution which is given in closed form by

$$u(x, t) = e^{-\pi^2 t} \sin(\pi x). \quad (1.47)$$

The initial concentration profile diffuses out of the domain over time, as shown in Figure 1.3.3. From this, we see that the parabolic problem models diffusion forward in time.

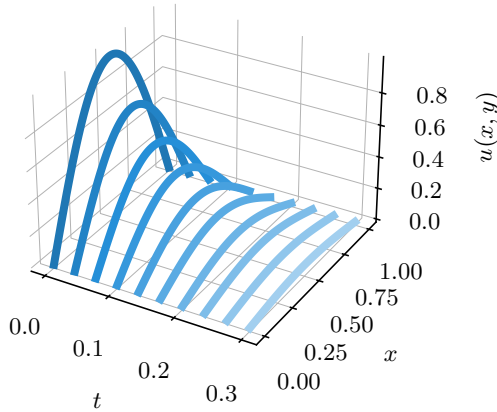


Figure 1.3.3. Solution profiles of the diffusion problem (1.46) over time.

Hyperbolic

The prototypical example of a hyperbolic second-order PDE is the 1D wave equation, given by

$$u_{tt} = c^2 u_{xx}, \quad (1.48)$$

where c is the wave speed. We define the problem on an infinite spatial domain and with an initial condition given by $u(x, 0) = g(x)$. In the particular case of $g(x) = \sin(x)$, we can consider the following two specific solutions:

$$u(x, t) = \sin(x \pm ct), \quad (1.49)$$

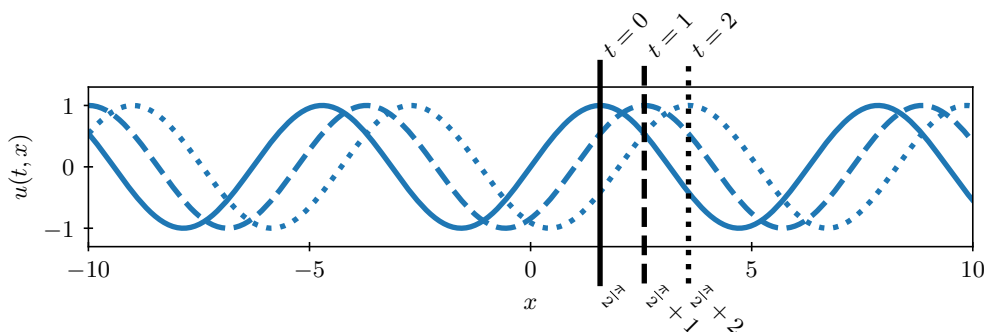


Figure 1.3.4. Example solution $u(x, t) = \sin(x - ct)$ of equation (1.48) with $c = 1$.

which are sine waves traveling at speeds $\pm c$ (see Figure 1.3.4). This indicates that the hyperbolic PDE in equation (1.48) exhibits *two characteristic* directions with slopes c and $-c$; prescribing a second initial condition on $\frac{\partial u}{\partial t}$ at $t = 0$ would fully specify the problem, resulting in a unique solution.

This can be understood better, for example, by relating the second-order wave equation to a system of first-order wave equations as follows. If we denote $v_1 = u_t$ and $v_2 = u_x$, then equation (1.48) can be written as

$$\begin{bmatrix} v_1 \\ v_2 \end{bmatrix}_t + \begin{bmatrix} 0 & -c^2 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}_x = 0, \quad (1.50)$$

or $\mathbf{v}_t + A\mathbf{v}_x = 0$. Decomposing A into $A = R\Lambda R^{-1}$, where the columns of R contain the right eigenvectors of A and Λ is a diagonal matrix with the eigenvalues c and $-c$ of A on the diagonal, we can decouple equation system (1.50). Indeed, setting $\mathbf{w} = R^{-1}\mathbf{v}$, we obtain

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix}_t + \begin{bmatrix} c & 0 \\ 0 & -c \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}_x = 0. \quad (1.51)$$

This represents a system of two first-order wave equations, i.e., the previously discussed scalar linear advection equation: the wave associated with w_1 moves to the right with speed c , and the wave associated with w_2 moves to the left with speed $-c$. The two associated families of characteristic curves have slopes c and $-c$. In Chapters 5, 6, and 9, we consider numerical methods for first-order hyperbolic PDEs.

1.4 ■ Some further thoughts

Partial differential equations are well studied in mathematics, with the fundamental knowledge on *hyperbolic*, *elliptic*, and *parabolic* types in place by the early 1800s. A fascinating review of the history of PDEs can be found in *Partial Differential Equations in the 20th Century* by Brezis and Browder [54], where the second-order hyperbolic wave equation of Section 1.3 is attributed to d'Alembert (1747), Euler (1758), and Bernoulli (1762). Likewise, the elliptic second-order PDE, the Laplace equation, is due to Laplace (1782), while the parabolic heat equation is due to Fourier (1822). These PDEs form the foundation of the model problems we introduced in this chapter, and PDEs arose quickly in a range of applications. We summarize many of the contributions in Figure 1.4.1, following the historical outline in Brezis and Browder [54], although more detailed information is available from many sources, such as [33].

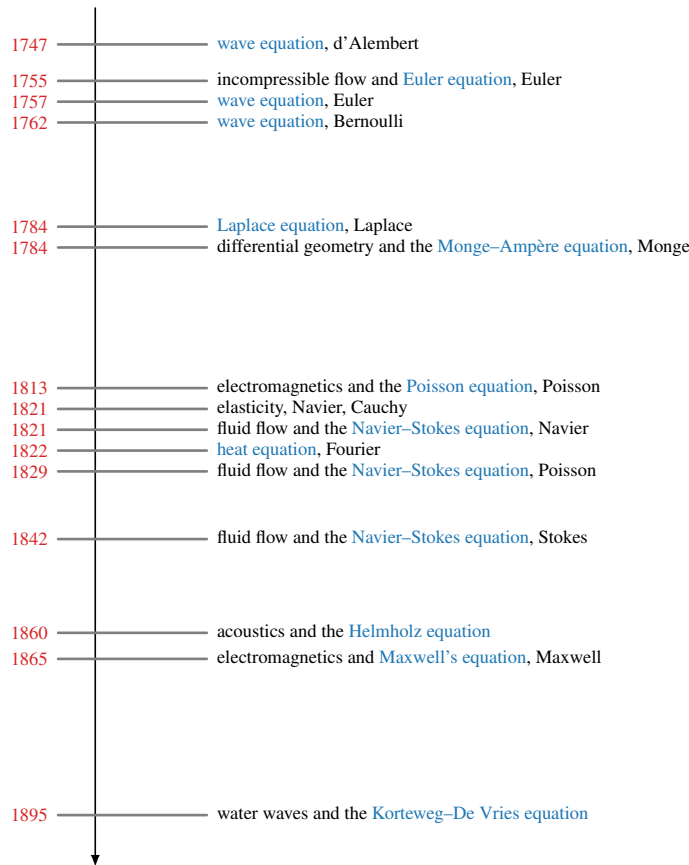


Figure 1.4.1. *Timeline of fundamental work in PDEs.*

Extensive theoretical properties of PDEs were established over the last two centuries, and we can only provide a glimpse of this rich landscape here. There are several in-depth, yet focused, texts on PDEs that can accompany this text. *Partial Differential Equations* by Evans [94] has become a standard tome on the subject, aimed at graduate-level students in mathematics. There are also numerous textbooks on the development of Fourier series–type solutions and other analytical approaches to expressing solutions for PDEs, such as [157, 119, 180]. We also refer the reader to texts more focused on specific numerical methods, such as *Finite Volume Methods for Hyperbolic Problems* by LeVeque [151], *Finite Elements: Theory, Fast Solvers, and Applications in Solid Mechanics* by Braess [48], and *High-Order Methods for Incompressible Fluid Flow* by Deville, Fischer, and Mund [86], all of which provide excellent insight into the underlying PDEs while still focusing on the numerical methods to approximate their solution.

Chapter 2

Background material in numerical methods

Overview

Interpolation, integration, and differentiation of functions lie at the heart of many numerical methods for approximating solutions to partial differential equations. In this chapter, we provide a brief overview of these basic operations to highlight key properties and to introduce notation that will be used throughout the text.

Given a 1D function, $f(x)$, we can learn quite a bit about our ability to approximate it as a solution to a PDE by considering polynomial interpolants of the function and by considering its Taylor expansion. If the function—i.e., the solution to the PDE—can be approximated easily by a low-order polynomial interpolation, then the job of constructing associated numerical methods becomes much easier. Similarly, the Taylor-series expansion of a function can be used to quantify the accuracy of a numerical approximation. Moreover, for many of the numerical methods we consider, we will need to approximate integrals or derivatives. These types of approximations are well understood, and we outline key properties for them in this chapter.

We do not provide every detail and derivation of the results presented in this chapter, as one would for a course on numerical analysis. However, we do state the main results and motivations, so that when the concepts are used in the framework of a numerical method for PDEs, they can be easily referenced. At the end of this chapter, we provide references to texts that do provide a more complete and rigorous study of such ideas.

Objectives

In this chapter, we will discuss basic procedures and key properties of interpolation, integration, and differentiation, from both calculus and numerical analysis. In later chapters of the text, we use many of these results and, therefore, key notation will be introduced here. Specifically, in this chapter, you will review ideas about approximating a function's values through either polynomial interpolation or Taylor approximations. You will see how we classify computational grids, one of the key ingredients in our methods for approximating solutions to PDEs. Then, you will see how we derive numerical differentiation and integration formulae.

Chapter Outline

Section 2.1 Polynomial approximation and interpolation poses the question of how we can approximate a function's values over an interval based on its values at a few nodes. We recall the ideas of polynomial interpolation and how the error in such an approximation depends on the choice of nodes for the approximation and properties of the function.

Section 2.2 Taylor series reviews the many forms of Taylor's theorem, with an emphasis on the remainder terms. We also extend the standard results in one dimension to functions in $\mathbb{R}^d$ as needed for approximating solutions to PDEs in multiple dimensions.

Section 2.3 Computational grids discusses the different types of computational meshes or grids that will be used throughout the book to formulate numerical methods for PDEs and represent numerical solutions. This section considers both structured and unstructured grids.

Section 2.4 Approximating derivatives looks at how we approximate derivatives of functions using Taylor series. Here, we derive several of the basic approximations that commonly arise in finite-difference approximations, as are used in Chapters 3 and 5.

Section 2.5 Approximating integrals: Quadrature derives common numerical quadrature schemes that are used to approximate integrals of functions. We review 1D quadrature routines based both on interpolating polynomials and on orthogonal polynomials, as well as extensions to two-dimensional domains.

Section 2.6 Integration properties discusses a variety of integration-by-parts formulae in 1D and in multiple spatial dimensions. These formulae will be useful in the finite-element methods presented in Chapters 4, 8, and 10.

2.1 ■ Polynomial approximation and interpolation

Objectives

In this section, you will see how the *best* polynomial approximation to a function can be found. The polynomial interpolant will be constructed, and you will see how accurately this approximates a function.

The ultimate goal of all of the numerical schemes that we will discuss is to approximate solutions to PDEs. Our approaches will select such approximations from finite-dimensional function spaces, so it is natural to first think about how we would approximate a known function, $f(x)$, in such a space before we consider the case where we want to approximate the unknown solution to a PDE.

2.1.1 ■ Polynomial approximation

We first consider approximating a known function $f(x)$ by a polynomial. For example, consider the function $f(x)$ in Figure 2.1.1. We could ask for the best approximation to this function in the finite-dimensional space of polynomials of degree no more than k , which we can write as

$$P_k = \{p(x) \mid p(x) = a_0 + a_1x + \cdots + a_kx^k, a_j \in \mathbb{R}\}. \quad (2.1)$$

Note that P_k is a subspace of the infinite-dimensional vector space of all polynomials, since the linear combination $\alpha p(x) + \beta q(x)$ for real values α, β is always in P_k if $p(x)$ and $q(x)$ are, and the zero polynomial is also in P_k . It is finite-dimensional because any $p(x) \in P_k$ can be written as a linear combination of the $k+1$ basis elements $\{x^i\}_{i=0}^k$, with $p(x) = \sum_{i=0}^k a_i x^i$.

In order to find the *best* polynomial approximation to the function $f(x)$, we seek a polynomial in P_k that minimizes the difference between the function and that polynomial in some norm. For a function $f(x)$ defined on $\Omega = [0, 1]$, for example, we seek $p_k^*(x) \in P_k$ such that

$$p_k^*(x) = \operatorname{argmin}_{p_k \in P_k} \|f(x) - p_k(x)\|_{L^2(\Omega)}^2, \quad (2.2)$$

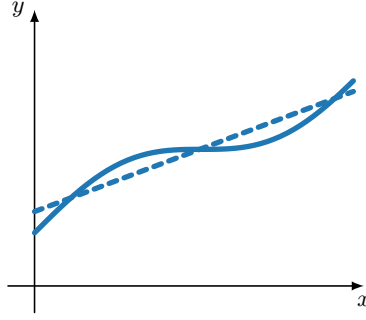


Figure 2.1.1. Polynomial approximation (dashed) of a function $f(x)$ (solid).

where $L^2(\Omega)$ is the vector space of functions that are square integrable over domain Ω , with inner product $\langle g(x), h(x) \rangle = \int_{\Omega} g(x)h(x) dx$ and norm $\|g(x)\|_{L^2(\Omega)} = (\int_{\Omega} g(x)^2 dx)^{1/2}$. We note that different norms could be used in equation (2.2), leading to different optimal polynomials, but we do not consider this here. Given a basis, $\{\phi_i(x)\}_{i=0}^k$, for P_k , we can describe the optimal $p_k^*(x)$ in equation (2.2) by a vector $\mathbf{a} \in \mathbb{R}^{k+1}$ such that

$$p_k^*(x) = \sum_{i=0}^k a_i \phi_i(x). \quad (2.3)$$

To find $p_k^*(x)$ in equation (2.2), we then minimize

$$F(\mathbf{a}) = \|f(x) - p_k(x)\|_{L^2(\Omega)}^2 \quad (2.4a)$$

$$= \int_0^1 \left(f(x) - \sum_{i=0}^k a_i \phi_i(x) \right)^2 dx \quad (2.4b)$$

$$= \int_0^1 \left(f(x)^2 - 2 \sum_i f(x) a_i \phi_i(x) + \sum_i \sum_j a_i a_j \phi_i(x) \phi_j(x) \right) dx \quad (2.4c)$$

over the coefficients $\mathbf{a} \in \mathbb{R}^{k+1}$. To find a minimum of $F(\mathbf{a})$, we set

$$\frac{\partial F}{\partial a_i} = -2 \int_0^1 f(x) \phi_i(x) dx + \sum_j \int_0^1 2a_j \phi_j(x) \phi_i(x) dx \equiv 0 \quad (2.5)$$

or, in matrix form,

$$M\mathbf{a} = \begin{bmatrix} \langle \phi_0, \phi_0 \rangle & \langle \phi_0, \phi_1 \rangle & \langle \phi_0, \phi_k \rangle \\ \langle \phi_1, \phi_0 \rangle & \ddots & \\ \vdots & & \\ \langle \phi_k, \phi_0 \rangle & \cdots & \langle \phi_k, \phi_k \rangle \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_k \end{bmatrix} = \begin{bmatrix} \langle f, \phi_0 \rangle \\ \langle f, \phi_1 \rangle \\ \vdots \\ \langle f, \phi_k \rangle \end{bmatrix} = \mathbf{f}. \quad (2.6)$$

Here, M is the so-called *mass matrix*, the condition number of which depends on the choice of basis functions, ϕ_i , but can be large for “easy” choices of basis functions (such as the monomials) and for large values of k . Achieving the best approximation is greatly simplified by using an

orthogonal polynomial basis so that M is a diagonal matrix. For the $L^2(\Omega)$ norm on an interval, Ω , the Legendre polynomials are an excellent choice of basis to achieve this.

2.1.2 ■ Polynomial interpolation

A related, but not equivalent, problem is that of polynomial interpolation of a known function $f(x)$. Given a set of $k + 1$ distinct points $x_0, \dots, x_k$ in a closed interval Ω , let $p_k(x)$ be the unique polynomial of degree at most k that interpolates the function $f(x)$ at the $k + 1$ points, i.e., so that $p_k(x_i) = f(x_i)$ for $0 \leq i \leq k$. When $f(x) \in C^{k+1}(\Omega)$, the vector space of functions with $k + 1$ continuous derivatives in Ω , basic interpolation theory tells us that the approximation error of the interpolating polynomial, $e_k(x) = p_k(x) - f(x)$, at any $x \in \Omega$, is given by

$$e_k(x) = p_k(x) - f(x) = \frac{f^{(k+1)}(\xi(x))}{(k+1)!} (x - x_0)(x - x_1) \cdots (x - x_k) \quad (2.7)$$

for some point $\xi \in \Omega$ that depends on the point x at which we evaluate the interpolating polynomial.

There are two observations that follow from equation (2.7). First, the error in an interpolating polynomial is controlled by the placement of the interpolation points, $\{x_0, \dots, x_k\}$, and the size of the $(k + 1)$ th derivative of $f(x)$ at $\xi(x)$, $f^{(k+1)}(\xi(x))$. Second, if $|f^{(k+1)}(\xi(x))|$ does not vary too much over Ω , then we can approximately minimize the maximum interpolation error over Ω by choosing interpolation points that minimize the factor $\prod_{i=0}^k (x - x_i)$. It can be shown that the so-called *Chebyshev nodes*,

$$x_i = \cos\left(\frac{2i + 1}{2k + 2}\pi\right), \quad i = 0, \dots, k, \quad (2.8)$$

minimize the maximum of $\prod_{i=0}^k (x - x_i)$ over $\Omega = [-1, 1]$. The interpolating polynomial that uses the Chebyshev nodes as interpolation points is often a good approximation for the optimal polynomial $p_k^*(x)$ that minimizes $\max_{x \in \Omega} |f(x) - p_k(x)|$ on $\Omega = [-1, 1]$, noting that this is a different norm than that which leads to equation (2.6). In particular, the function

$$G(\mathbf{a}) = \max_{x \in \Omega} |f(x) - p_k(x)| \quad (2.9)$$

is not differentiable, so we cannot minimize $G(\mathbf{a})$ in the same way that we minimize $F(\mathbf{a})$ in equation (2.4) above. However, since $F(\mathbf{a}) \leq |\Omega|G^2(\mathbf{a})$, approximately minimizing $G(\mathbf{a})$ still gives us useful information about the polynomial approximation problem considered above.

More generally, to derive an upper bound on the interpolation error $e_k(x)$ in equation (2.7), assume $x_0 < x_1 < \dots < x_k$ and let $\Omega = [x_0, x_k]$. Let h be the maximum mesh spacing of the interpolation points: $h = \max_i |x_{i+1} - x_i|$. Then, for any $x \in \Omega$ we have

$$|e_k(x)| = |f(x) - p_k(x)| \leq \max_{\xi \in \Omega} |f^{(k+1)}(\xi)| \frac{h^{k+1}}{4k + 4}. \quad (2.10)$$

This can be seen as follows. Without loss of generality, assume that $\prod_{i=0}^k (x - x_i)$ attains its maximum for some $x \in [x_0, x_1]$. Note that the absolute value of the factor $(x - x_0)(x - x_1)$ in equation (2.7) attains its maximum at $x = (x_0 + x_1)/2$. From this, it follows that $|(x - x_0)(x - x_1)| \leq h^2/4$. Then,

$$\frac{|\prod_{i=0}^k (x - x_i)|}{(k+1)!} \leq \frac{h^2}{4} \frac{2h \cdot 3h \cdots kh}{(k+1)!} = \frac{h^{k+1}}{4(k+1)}. \quad (2.11)$$

Note that if $\Pi_{i=0}^k (x - x_i)$ attains its maximum for a point, x , in some other interval, the bounds on $|x - x_i|$ for x_i away from that interval are smaller than those above, so this is the worst-possible case, giving a bound that holds no matter where the maximum is attained. The bound on the interpolation error from equation (2.10) is illustrated in the following example.

Example 2.1: Interpolation error bounds

Consider the function $f(x) = \sin(x) + x + 2$, as shown in Figure 2.1.2a. We consider interpolating polynomials of order k from 1 to 10 with $k + 1$ equispaced interpolation points in the interval $\Omega = [0, 6]$. Figure 2.1.2b compares the maximum interpolation error over interval $[0, 6]$ with the bound of equation (2.10). Note that $|f^{(k+1)}| \leq 1$ for $k \geq 1$. In this case, the bound of equation (2.10) is close to the maximum error.

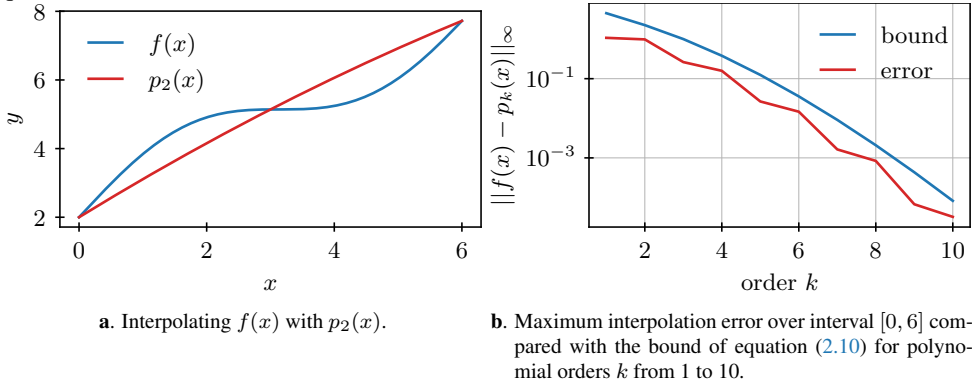


Figure 2.1.2. Polynomial interpolation example for $f(x) = \sin(x) + x + 2$.

2.2 ■ Taylor series

Objectives

In this section, you will recall the ideas behind Taylor series and derive several different expressions for the remainder term when we consider a Taylor polynomial of finite degree. To derive similar tools in multiple dimensions, we will use the results from the one-dimensional case to establish similar remainder terms for functions in higher-dimensional spaces.

The Taylor series and Taylor polynomials of a function give approximations of the function via a truncated power series expansion. The coefficients of the series depend on the derivatives of the function and, thus, are useful in later sections where we compute approximations of derivatives of functions and seek to understand the errors in those approximations.

First, recall Taylor's theorem in 1D, with an integral expression for the remainder.

Theorem 2.2: Taylor's theorem in 1D with remainder

Let $n \in \mathbb{Z}, n \geq 0$. If $f : [a, b] \rightarrow \mathbb{R}$ is $n + 1$ times continuously differentiable on $[a, b]$ and $x_0 \in [a, b]$, then

$$f(x) = f(x_0) + \frac{f'(x_0)}{1!}(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n + \int_{x_0}^x \frac{f^{(n+1)}(t)}{n!}(x - t)^n dt. \quad (2.12)$$

Proof. By the fundamental theorem of calculus,

$$\int_{x_0}^x f'(t) dt = f(x) - f(x_0) \Rightarrow f(x) = f(x_0) + \int_{x_0}^x f'(t) dt, \quad (2.13)$$

giving the stated result for $n = 0$. For the general case, we proceed by induction, assuming that, for $1 \leq k \leq n$,

$$\begin{aligned} f(x) = f(x_0) &+ \frac{f'(x_0)}{1!}(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots \\ &+ \frac{f^{(k-1)}(x_0)}{(k-1)!}(x - x_0)^{k-1} + \int_{x_0}^x \frac{f^{(k)}(t)}{(k-1)!}(x - t)^{k-1} dt. \end{aligned} \quad (2.14)$$

Noting that $f^{(k)}(t)$ is continuously differentiable for $k \leq n$ by the original assumption on f , we use integration by parts to get

$$\int_{x_0}^x \frac{f^{(k)}(t)}{(k-1)!}(x - t)^{k-1} dt = -\frac{f^{(k)}(t)}{k!}(x - t)^k \Big|_{x_0}^x + \int_{x_0}^x \frac{f^{(k+1)}(t)}{k!}(x - t)^k dt \quad (2.15a)$$

$$= \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k + \int_{x_0}^x \frac{f^{(k+1)}(t)}{k!}(x - t)^k dt. \quad (2.15b)$$

Thus, by induction,

$$\begin{aligned} f(x) = f(x_0) &+ \frac{f'(x_0)}{1!}(x - x_0) + \cdots \\ &+ \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k + \int_{x_0}^x \frac{f^{(k+1)}(t)}{k!}(x - t)^k dt \end{aligned} \quad (2.16)$$

for $1 \leq k \leq n$. Taking $k = n$ gives the stated result. $\square$

We generally call

$$\begin{aligned} p_n(x) = f(x_0) &+ \frac{f'(x_0)}{1!}(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots \\ &+ \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n \end{aligned} \quad (2.17)$$

the degree- n Taylor polynomial approximating $f(x)$ about the point x_0 . The last term in equation (2.12) is referred to as the *remainder term* and allows us to compute the error in using the Taylor polynomial $p_n(x)$ as an approximation for $f(x)$ (or its derivatives). However, the integral term can be a bit cumbersome. Therefore, we use the following corollaries to write or bound the remainder in a more practical form.

Corollary 2.3: Lagrange form of remainder

Under the same assumptions as in Theorem 2.2, there is a point ξ between x and x_0 such that

$$f(x) = f(x_0) + \frac{f'(x_0)}{1!}(x - x_0) + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n + \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)^{n+1}. \quad (2.18)$$

Proof. By the integral mean value theorem, there is a point ξ between x and x_0 such that

$$\int_{x_0}^x \frac{f^{(n+1)}(t)}{n!} (x-t)^n dt = f^{(n+1)}(\xi) \int_{x_0}^x \frac{(x-t)^n}{n!} dt = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-x_0)^{n+1}. \quad (2.19)$$

□

Using this, we can now bound the error between a function and its Taylor polynomial of degree n .

Corollary 2.4: Bound on remainder

Under the same assumptions as in Theorem 2.2, if $\exists M_n$ such that $|f^{(n+1)}(t)| \leq M_n$ for all t between x and x_0 , then

$$|f(x) - p_n(x)| \leq M_n \left| \frac{(x-x_0)^{n+1}}{(n+1)!} \right| \quad (2.20)$$

for $p_n(x)$ defined as in equation (2.17).

Note that we assume t and ξ are “between x and x_0 ” in the above results to allow both of the cases where $x < x_0$ and $x > x_0$.

Example 2.5: Taylor approximation of $\sin(x)$

As an example, consider the function

$$f(x) = \sin(x) \quad (2.21)$$

and the expansion point $x_0 = \frac{\pi}{4}$. Figure 2.2.1 highlights the increase in accuracy as more terms are added to the Taylor expansion. In addition, while the error is large over much of the interval shown, the rightmost figure shows the high accuracy near x_0 .

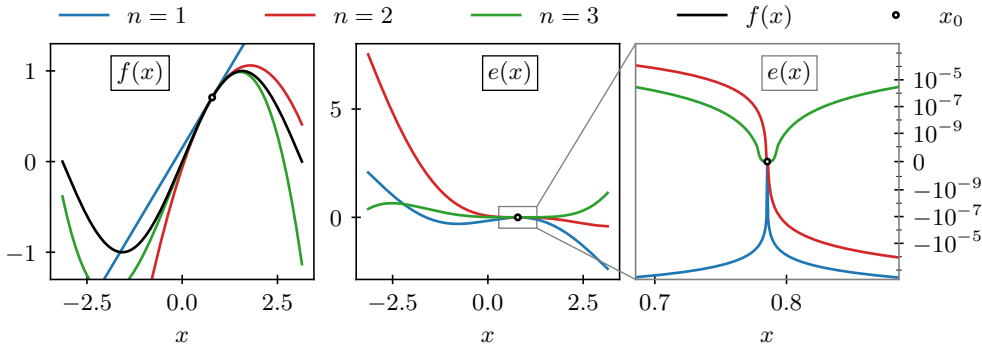


Figure 2.2.1. (left) Taylor polynomials for $f(x) = \sin(x)$ about $x_0 = \frac{\pi}{4}$ with $n = 1$, $n = 2$, and $n = 3$. (center) Error in each polynomial. (right) Zoomed-in view of the error.

While Theorem 2.2 is already useful by itself, we now give the multivariable version of Taylor’s theorem, the proof of which uses the one-dimensional version. To do this, we need concise notation for higher-order derivatives, for which we will use multi-index notation.

Definition 2.6: Multi-index notation for high-order derivatives

Let $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_d)$ be a sequence of indices such that $\alpha_j \in \mathbb{Z}$ and $\alpha_j \geq 0 \forall j$, and let $|\alpha| = \alpha_1 + \dots + \alpha_d$. Define

$$D^\alpha f = \frac{\partial^\alpha f}{\partial x^\alpha} = \frac{\partial^{|\alpha|} f}{\partial x_1^{\alpha_1} \partial x_2^{\alpha_2} \dots \partial x_d^{\alpha_d}}. \quad (2.22)$$

Additionally, note that $\sum_{|\alpha|=k}(\dots)$ means to sum over all multi-indices such that $|\alpha| = k$.

The idea behind proving the multivariable version of Taylor's theorem is to apply Theorem 2.2 to the single-variable function $g(t) = f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)}))$. However, it is cumbersome to directly compute $g^{(k)}(t) = \left(\frac{d}{dt}\right)^k f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)}))$. The multinomial theorem, Theorem 2.7, gives a useful tool for this.

Theorem 2.7: Multinomial theorem

$$\left(\sum_{i=1}^d y_i\right)^k = k! \sum_{|\alpha|=k} \prod_{j=1}^d \frac{(y_j)^{\alpha_j}}{\alpha_j!}. \quad (2.23)$$

Proof. This result is proven by induction, using the binomial theorem for the inductive step. $\square$

With this result, we can find a useful form for expressing $g^{(k)}(t)$ for $g(t) = f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)}))$.

Lemma 2.8: Multivariable derivatives

$$\left(\frac{d}{dt}\right)^k f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) = ((\mathbf{x} - \mathbf{x}^{(0)}) \cdot \nabla)^k f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) \quad (2.24a)$$

$$= k! \sum_{|\alpha|=k} \frac{\partial^\alpha f}{\partial x^\alpha}(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) \left(\prod_{j=1}^d \frac{(x_j - x_j^{(0)})^{\alpha_j}}{\alpha_j!} \right). \quad (2.24b)$$

Proof. By the chain rule,

$$\frac{d}{dt} f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) = \nabla f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) \cdot (\mathbf{x} - \mathbf{x}^{(0)}). \quad (2.25)$$

Now, let $\mathbf{y} = \mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})$ and $h(\mathbf{y}) = (\mathbf{x} - \mathbf{x}^{(0)}) \cdot \nabla f(\mathbf{y})$ so that

$$\frac{d}{dt} f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) = h(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})). \quad (2.26)$$

Using the previous expression, now for dh/dt , gives

$$\left(\frac{d}{dt}\right)^2 f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) = (\mathbf{x} - \mathbf{x}^{(0)}) \cdot \nabla h(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) \quad (2.27a)$$

$$= [(\mathbf{x} - \mathbf{x}^{(0)}) \cdot \nabla]^2 f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})). \quad (2.27b)$$

Inductively applying this step gives the first expression. From here, the second expression follows using the multinomial theorem, writing

$$(\mathbf{x} - \mathbf{x}^{(0)}) \cdot \nabla = (x_1 - x_1^{(0)})\partial_{x_1} + \cdots + (x_d - x_d^{(0)})\partial_{x_d}. \quad (2.28)$$

□

We can now state and easily prove the multidimensional extension of Theorem 2.2.

Theorem 2.9: Multivariable Taylor's theorem

Let $\mathbf{x}^{(0)} \in \mathbb{R}^d$, $\delta \in \mathbb{R}$, $\delta > 0$, $n \in \mathbb{Z}$, $n \geq 0$, and let $f : cl(D(\mathbf{x}^{(0)}, \delta)) \rightarrow \mathbb{R}$ be a function that is $(n+1)$ times continuously differentiable on $cl(D(\mathbf{x}^{(0)}, \delta))$, where $cl(D(\mathbf{x}^{(0)}, \delta))$ is the closure of the disk in $\mathbb{R}^d$ centered at $\mathbf{x}^{(0)}$ with radius δ . Then, for any $\mathbf{x} \in D(\mathbf{x}^{(0)}, \delta)$,

$$\begin{aligned} f(\mathbf{x}) &= \sum_{|\alpha|=0}^n \frac{\partial^\alpha f(\mathbf{x}^{(0)})}{\partial \mathbf{x}^\alpha} \left(\prod_{j=1}^d \frac{(x_j - x_j^{(0)})^{\alpha_j}}{\alpha_j!} \right) \\ &\quad + (n+1) \sum_{|\alpha|=n+1} \left[\int_0^1 (1-t)^n \frac{\partial^\alpha f}{\partial \mathbf{x}^\alpha}(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) dt \right] \left(\prod_{j=1}^d \frac{(x_j - x_j^{(0)})^{\alpha_j}}{\alpha_j!} \right). \end{aligned} \quad (2.29)$$

Proof. Defining $g(t) = f(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)}))$, we see that $g(t)$ satisfies the assumptions of Theorem 2.2 on $[0, 1]$. Thus,

$$f(\mathbf{x}) = g(1) = \sum_{k=0}^n \frac{g^{(k)}(0)}{k!} (1-0)^k + \int_0^1 \frac{g^{(n+1)}(t)}{n!} (1-t)^n dt \quad (2.30a)$$

$$= \sum_{k=0}^n \sum_{|\alpha|=k} \frac{\partial^\alpha f}{\partial \mathbf{x}^\alpha}(\mathbf{x}^{(0)}) \left(\prod_{j=1}^d \frac{(x_j - x_j^{(0)})^{\alpha_j}}{\alpha_j!} \right) \quad (2.30b)$$

$$+ (n+1) \sum_{|\alpha|=n+1} \int_0^1 (1-t)^n \frac{\partial^\alpha f}{\partial \mathbf{x}^\alpha}(\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) dt \left(\prod_{j=1}^d \frac{(x_j - x_j^{(0)})^{\alpha_j}}{\alpha_j!} \right). \quad (2.30c)$$

□

The above expressions are, of course, not very intuitive when first encountered. Consider, for example, the case where $d = 2$, and let

$$I_{\alpha_1, \alpha_2} = \int_0^1 (1-t)^n \frac{\partial^{n+1} f}{\partial x^{\alpha_1} \partial y^{\alpha_2}} (\mathbf{x}^{(0)} + t(\mathbf{x} - \mathbf{x}^{(0)})) dt. \quad (2.31)$$

Then the expression from Theorem 2.9 becomes

$$f(x, y) = f(x_0, y_0) + \frac{\partial f}{\partial x}(x_0, y_0)(x - x_0) + \frac{\partial f}{\partial y}(x_0, y_0)(y - y_0) \quad (2.32a)$$

$$+ \frac{1}{2!} \frac{\partial^2 f}{\partial x^2} (x - x_0)^2 + \frac{1}{2!} \frac{\partial^2 f}{\partial y^2} (y - y_0)^2 + \frac{\partial^2 f}{\partial x \partial y} (x - x_0)(y - y_0) \quad (2.32b)$$

$$+ \cdots + (n+1) \sum_{\alpha_1 + \alpha_2 = n+1} I_{\alpha_1, \alpha_2} \frac{(x - x_0)^{\alpha_1}}{\alpha_1!} \frac{(y - y_0)^{\alpha_2}}{\alpha_2!}. \quad (2.32c)$$

Thus, in two dimensions, the first-order Taylor polynomial is linear in both x and y , while the second-order Taylor polynomial is quadratic, with one mixed-derivative term. Note also the unequal weighting among the second derivatives ∂x^2 , ∂y^2 and the mixed term $\partial x \partial y$. The remainder term is more complicated in this case but can be bounded similarly to Corollary 2.4.

With the above theorems, we can now proceed to approximate functions and derivatives of functions at given points using a Taylor polynomial. The bounds on the remainder term, then, automatically give us a bound on the error in the approximation.

2.3 • Computational grids

Objectives

In this section, you will learn about the various types of computational grids or meshes that are used to formulate numerical methods for PDEs and represent their approximate solutions.

The numerical methods for PDEs discussed in this book are typically formulated on a computational grid or mesh that *discretizes* the domain where the PDE is posed by dividing it into a finite number of cells. For 1D intervals, we have a simple grid structure, with grid points or nodes separating the grid cells. On 2D domains, we often represent a grid in terms of the grid points, edges that connect two points, and cells or elements that are bounded by edges. On 3D domains, sets of edges define faces, and sets of faces define the cells or elements. This grid structure is used in formulating key components of the numerical methods. For example, finite-element methods may use piecewise polynomial basis functions defined on collections of grid elements, while finite-difference methods may use approximations of derivatives defined at grid points. The grid is then also used to represent the numerical approximation of the PDE solution.

It is useful to distinguish between two main classes of computational grids: *structured grids*, which feature a regular, logically rectangular neighbor structure of grid cells (or elements) and/or grid points (or nodes), and *unstructured grids*, which do not have such a regular structure. Within these two classes, we can distinguish further subtypes. Figure 2.3.1 gives some examples of 2D grid subtypes within the classes of structured and unstructured grids that we now further define and explain.

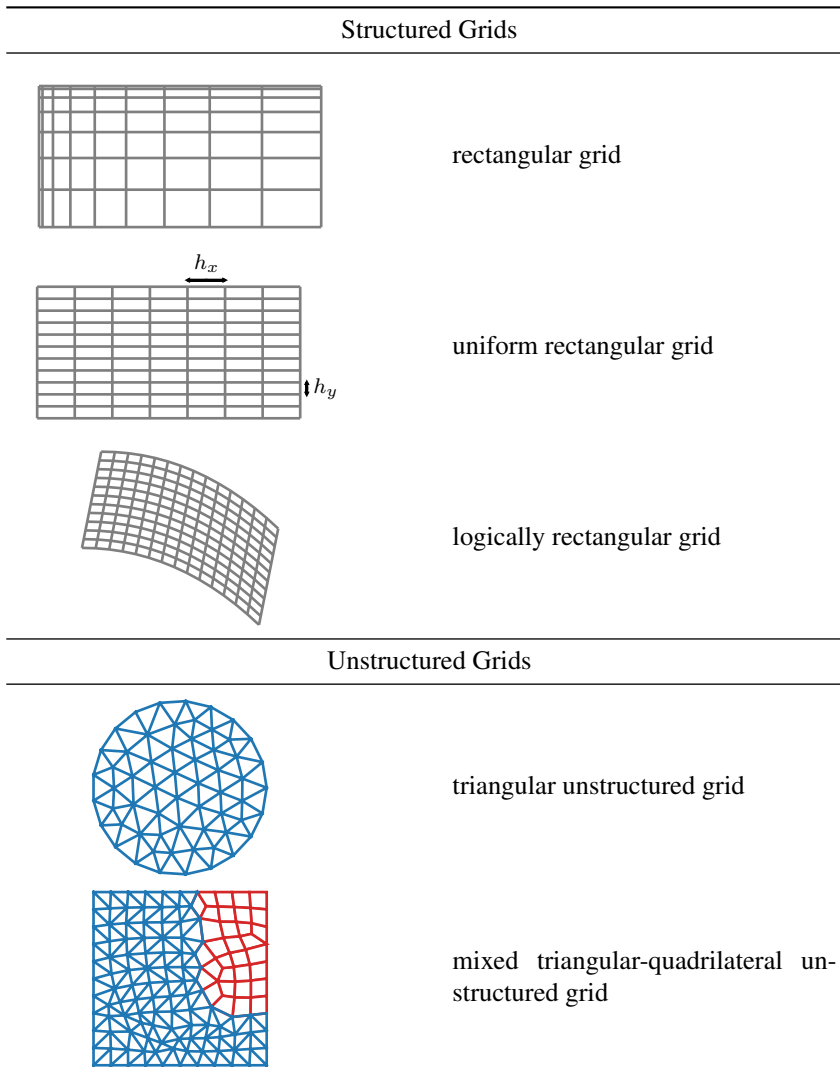


Figure 2.3.1. Example 2D grid types within the two main classes of structured and unstructured grids.

We begin with defining a 1D computational grid on an interval of the real line.

Definition 2.10: 1D grid

- A 1D grid on interval $[a, b]$ is a set of points $\Omega^h = \{x_i\}_{i=0}^{n_x}$ with $x_0 = a < x_1 < \dots < x_{n_x} = b$.
- A 1D grid $\Omega^h = \{x_i\}_{i=0}^{n_x}$ on $[a, b]$ is called uniform if the points x_i are equally spaced, i.e., $x_i = a + i h$ ($i = 0, \dots, n_x$), with the grid spacing h given by $h = (b - a)/n_x$.

Rectangular grids in two or more dimensions can be defined using tensor products of 1D grids.

Definition 2.11: Rectangular grid

- A rectangular grid on a 2D domain $[a, b] \times [c, d]$ is a set of points Ω^h that is obtained by the tensor product of two 1D grids, $\Omega^{h_x} = \{x_i\}_{i=0}^{n_x}$ on $[a, b]$ and $\Omega^{h_y} = \{y_j\}_{j=0}^{n_y}$ on $[c, d]$:

$$\Omega^h = \Omega^{h_x} \otimes \Omega^{h_y} = \{(x_i, y_j)\}_{i=0, j=0}^{i=n_x, j=n_y}. \quad (2.33)$$

This can be extended to three or more dimensions in a similar fashion.

- We call a rectangular grid uniform if it is a tensor product of uniform 1D grids. For example, for a 2D uniform rectangular grid on $[a, b] \times [c, d]$, there are grid spacings h_x and h_y such that $(x_i, y_j) = (a + i h_x, c + j h_y)$ ($i = 0, \dots, n_x; j = 0, \dots, n_y$), with $h_x = (b - a)/n_x$ and $h_y = (d - c)/n_y$.
- A logically rectangular grid in two or more dimensions has the same neighbor connectivity as a rectangular grid, but the coordinates of its grid points are not fixed by a tensor-product structure. Similarly, a 1D grid can also be defined on a curved line segment.

The grids defined in Definitions 2.10 and 2.11 all belong to the class of structured grids due to their regular neighbor connectivity. In contrast, *unstructured grids* (see Definition 2.12) have irregular connectivity of neighboring grid points or nodes and grid cells or elements. For example, in 2D, we can have an unstructured grid of triangles, where the number of triangles that a node of the grid belongs to varies between nodes in the grid (not just at the domain boundary, but also in the interior). We can also have unstructured 2D grids of quadrilateral cells, or with a mixture of quadrilaterals and triangles; see Figure 2.3.1 for some examples. In 3D, we can similarly have unstructured grids of tetrahedra or other element shapes, such as prisms, pyramids, or hexahedra. For high-order accurate methods, it may also be useful to consider grid cells or elements with curved edges and/or surfaces.

Definition 2.12: Meshes

Given an open and connected domain Ω in $\mathbb{R}^d$ with $d = 1, 2$, or 3 , we denote a mesh of Ω as a set $\mathcal{T}^h$ composed of n_{elem} mesh elements τ^k ,

$$\mathcal{T}^h = \{\tau^k\}_{k=0}^{n_{\text{elem}}-1}, \quad (2.34)$$

where $\mathcal{T}^h$ is a tessellation of an open computational domain Ω^h that approximates Ω , satisfying the following properties:

- Each τ^k is a closed d -dimensional element, defined by vertices, along with edges (in 2D) and faces (in 3D). Examples of τ^k include triangles and quadrilaterals in 2D and tetrahedra and hexagons in 3D.
- For each $\tau \in \mathcal{T}^h$, the vertex coordinates, $\mathbf{v}$, of τ satisfy $\mathbf{v} \in \overline{\Omega}$.
- The elements form a partition or tessellation of the computational domain Ω^h :
 - the elements are disjoint: $\tau_o^i \cap \tau_o^j = \emptyset$ for any $\tau^i, \tau^j \in \mathcal{T}^h$, where $\tau_o = \tau \setminus \partial\tau$ for any $\tau \in \mathcal{T}^h$,
 - the elements cover $\overline{\Omega}^h$: $\overline{\Omega}^h = \bigcup_{\tau \in \mathcal{T}^h} \tau$.

The mesh is called unstructured if it has irregular neighbor connectivity. The superscript h represents the mesh size; one example of this is the maximal diameter of an element; see equation (8.158). In later chapters, we will simply say that $\mathcal{T}^h$ is a tessellation of Ω and refer to Ω^h as the corresponding computational domain.

2.3.1 ■ Quasi-uniform grids

We now discuss the concept of a quasi-uniform sequence of meshes. This concept is useful in the context of repeated refinements of a mesh and when analyzing convergence of numerical methods for PDEs. A quasi-uniform family of structured grids is defined as follows.

Definition 2.13: Quasi-uniform grid family in 1D

Let $\Omega = [a, b]$ be a fixed interval, and, for an infinite sequence of h values with $0 < h < b - a$ and limit 0, let $\{\Omega^h\}$ be a family of grids such that, for nodes $\{x_j^h\}_{j=0}^{n_x}$,

$$\max_{1 \leq j \leq n_x} |x_j^h - x_{j-1}^h| = h. \quad (2.35)$$

The family of grids is called *quasi-uniform* if $\exists \rho \in (0, 1]$ such that

$$\min_{1 \leq j \leq n_x} |x_j^h - x_{j-1}^h| \geq \rho h \quad (2.36)$$

for all h .

Note that the condition $h < b - a$ in the definition excludes grids with $n_x = 1$ and is a technical restriction that will be needed in some proofs.

Quasi uniformity is most useful for higher-dimensional problems but easily understood for one-dimensional meshes, where the definition simply states that the widths of the largest and smallest mesh intervals must, asymptotically, be reduced at the same rate. This is convenient (but not always necessary) for convergence analysis of some numerical methods, since some quantities will naturally be bounded by the largest mesh width and some by the smallest; a quasi-uniformity assumption relates these two quantities. While it is easy to construct families of grids with largest mesh interval going to zero that are not quasi-uniform, many standard constructions, for example, in the context of repeated grid refinement, naturally yield the quasi-uniform property, particularly in one dimension.

Example 2.14: Some examples of quasi-uniform grid families

First, it is easy to see that any sequence of uniform grids on $[a, b]$ with largest mesh interval going to zero satisfies Definition 2.13 for a quasi-uniform grid family: for each grid in the sequence with n_x grid intervals, we set $h = (b - a)/n_x$, and then the definition holds for any $\rho \in (0, 1]$.

Figure 2.3.2 gives two further examples of quasi-uniform grid families. The left example illustrates a family of uniform grids that is obtained by starting from a uniform coarsest grid that is repeatedly refined by dividing each mesh interval into two equal parts. Since each grid is uniform, this family is quasi-uniform. (Note, also, that this is just a specific example of a sequence of uniform grids, as described more generally above.)

The right example illustrates a family of nonuniform grids that is quasi-uniform. The family is obtained by starting from a nonuniform coarsest grid of Chebyshev nodes (as in equation (2.8)) that is repeatedly refined by dividing each mesh interval into two equal parts.

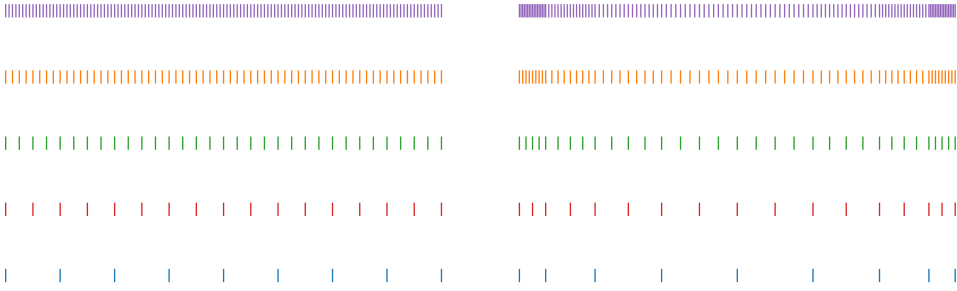


Figure 2.3.2. (left) Quasi-uniform grid family consisting of repeatedly refined uniform grids. (right) Quasi-uniform grid family consisting of repeatedly refined nonuniform grids.

2.4 - Approximating derivatives

Objectives

The main goal of this section is to use Taylor series to approximate derivatives. You will derive several 1D approximation formulas for derivatives, quantify their accuracy, and extend these approaches to functions of two variables.

We start with the simple task of using Taylor's theorem to approximate the derivative of a function $f(x)$ at a point x_0 , $f'(x_0)$ (see Figure 2.4.1).

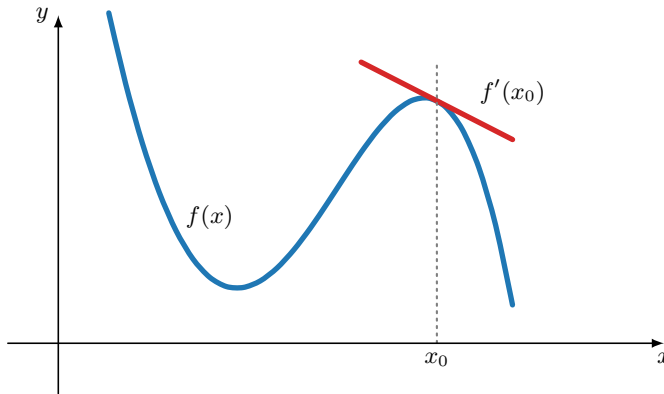


Figure 2.4.1. Differentiable function $f(x)$. The red line has slope equal to $f'(x_0)$.

For the moment, consider the one-dimensional case with domain $\Omega = \mathbb{R}$ and with an infinite regular grid with grid spacing $h \in \mathbb{R}$ defined as

$$\Omega^h = \{x_j \mid x_j = jh, j \in \mathbb{Z}\}. \quad (2.37)$$

How do we find an approximation to $f'(x)$ at some point $x_j \in \Omega^h$ using the values of $f(x)$ at points $x_i \in \Omega^h$? To start, we return to Taylor's theorem as introduced in the previous section.

We expand about the point x_j to arrive at

$$f(x) = f(x_j) + f'(x_j)(x - x_j) + \frac{f''(\xi)}{2}(x - x_j)^2, \quad (2.38)$$

so that

$$f(x_j + h) = f(x_j) + f'(x_j)h + \frac{f''(\xi)}{2}h^2, \quad (2.39)$$

which gives an approximation to the derivative that is first-order accurate as $h \rightarrow 0$:

$$f'(x_j) = \frac{f(x_j + h) - f(x_j)}{h} + \mathcal{O}(h). \quad (2.40)$$

Using $x_j - h$ instead of $x_j + h$, or a combination of the two, we arrive at a standard collection of first- and second-order finite-difference formulas for the first derivative:

Forward difference:

$$f'(x_j) = \frac{f(x_{j+1}) - f(x_j)}{h} - \frac{f''(\xi_+)}{2}h, \quad (2.41)$$

Backward difference:

$$f'(x_j) = \frac{f(x_j) - f(x_{j-1})}{h} + \frac{f''(\xi_-)}{2}h, \quad (2.42)$$

Centered difference:

$$f'(x_j) = \frac{f(x_{j+1}) - f(x_{j-1}))}{2h} - \frac{h^2}{12}(f'''(\xi_+) + f'''(\xi_-)). \quad (2.43)$$

Here, we use the notation $\xi_{\pm}$ to denote points in the intervals to the left and right of x_j that are determined by the Lagrange form of the remainder term in Taylor's theorem. For the centered difference formula, it is easy to verify that the second-order error terms for $f(x_j + h)$ and $f(x_j - h)$ cancel, resulting in an approximation that is second-order accurate and depends on the third derivative of f at points $\xi_{\pm}$. These finite-difference formulas can be used to approximate derivatives of a function given its values on a mesh. As an example, approximations to the derivative of $f(x) = \sin(\pi x)$ at $x_j = 0.4$ are shown in Figure 2.4.2.

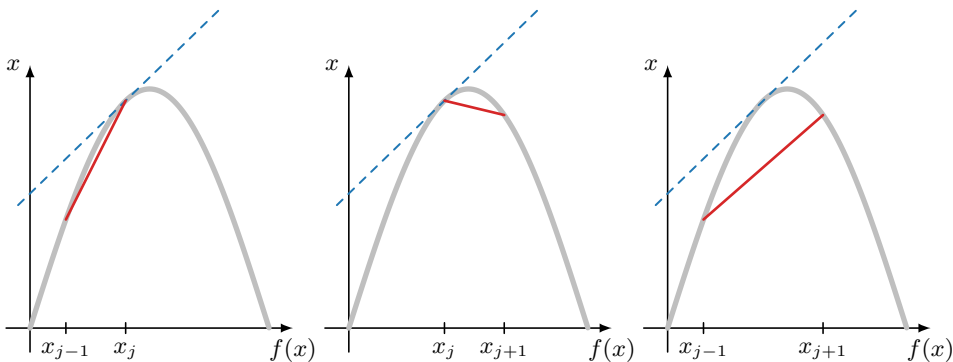


Figure 2.4.2. (left) Backward, (center) forward, and (right) centered finite-difference approximations of $f'(x)$ for $f(x) = \sin(\pi x)$, shown in red, compared with the exact derivative at x_j , shown in blue.

Higher accuracy requires more information in the approximation formula. Each of the computations above uses only the immediate neighbor points of x_j , namely x_{j-1} and x_{j+1} , in order to approximate $f'(x)$ at x_j . We can *expand* the pattern to two neighboring grid points on each side, aiming to increase the order of accuracy by using more terms in the Taylor series,

$$\begin{aligned} f(x_j + h) = f(x_j) + f'(x_j)h + \frac{f''(x_j)}{2}h^2 + \frac{f^{(3)}(x_j)}{3!}h^3 \\ + \frac{f^{(4)}(x_j)}{4!}h^4 + \frac{f^{(5)}(\xi_+)}{5!}h^5, \end{aligned} \quad (2.44a)$$

$$\begin{aligned} f(x_j - h) = f(x_j) - f'(x_j)h + \frac{f''(x_j)}{2}h^2 - \frac{f^{(3)}(x_j)}{3!}h^3 \\ + \frac{f^{(4)}(x_j)}{4!}h^4 - \frac{f^{(5)}(\xi_-)}{5!}h^5, \end{aligned} \quad (2.44b)$$

and similarly for $2h$ spacing,

$$\begin{aligned} f(x_j + 2h) = f(x_j) + 2f'(x_j)h + 4\frac{f''(x_j)}{2}h^2 + 8\frac{f^{(3)}(x_j)}{3!}h^3 \\ + 16\frac{f^{(4)}(x_j)}{4!}h^4 + 32\frac{f^{(5)}(\eta_+)}{5!}h^5, \end{aligned} \quad (2.45a)$$

$$\begin{aligned} f(x_j - 2h) = f(x_j) - 2f'(x_j)h + 4\frac{f''(x_j)}{2}h^2 - 8\frac{f^{(3)}(x_j)}{3!}h^3 \\ + 16\frac{f^{(4)}(x_j)}{4!}h^4 - 32\frac{f^{(5)}(\eta_-)}{5!}h^5. \end{aligned} \quad (2.45b)$$

Combining the four terms $f(x_j \pm h)$ and $f(x_j \pm 2h)$ in a way that annihilates the most terms in the right-hand side (other than $f'(x_j)$) leads to the following fourth-order accurate difference formula:

$$f'(x_j) = \frac{f(x - 2h) - 8f(x - h) + 8f(x + h) - f(x + 2h)}{12h} + \mathcal{O}(h^4). \quad (2.46)$$

Further increasing the order of accuracy requires more terms in the computation. Alternative constructions exist, however. One common approach is to first interpolate the function $f(x)$ using a degree- q polynomial, followed by taking the derivative of the interpolant. This way, the schemes above are easily recovered, but variants exist that use different collocation points or extrapolation techniques.

Example 2.15: Approximations to the derivative of $\sin(3\pi x)$

As a numerical test, we approximate the derivative of $f(x) = \sin(3\pi x)$ in the interval $[0, 1]$. We select $n_x = 8$ equidistant grid points, so $h = 1/(n_x - 1)$, and successively double the number of points, observing the maximum pointwise error in the interval, denoted $\|e(x)\|_\infty$. Figure 2.4.3 shows the expected results. For example, the second-order formula above yields a reduction by 4 in the error as the mesh spacing h is halved.

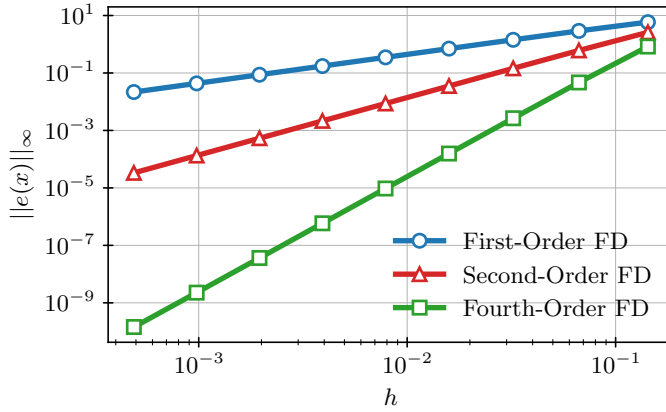


Figure 2.4.3. Finite difference (FD) approximation to the derivative of $f(x) = \sin(3\pi x)$ in $\Omega = [0, 1]$.

So far, we have made an implicit assumption when discussing the error in terms of mesh spacing h : we have to assume that the highest derivative in the Taylor expansion exists. Also, for example, if the third derivative $f^{(3)}(\xi)$ in the second-order schemes above is large, then h is required to be small in order to yield an accurate approximation. For higher-order schemes, existence of higher derivatives needs to be assumed. Thus, one needs to consider the smoothness of the problem data when choosing an appropriate scheme.

The same techniques can be used to generate approximations for higher-order derivatives, such as approximations to $f''(x_j)$. Here, we consider two Taylor expansions like the ones given in equations (2.44a) and (2.44b); however, we now add these expressions instead of subtracting them in order to preserve the terms including $f'''(x_j)$. This gives

$$f(x_j + h) + f(x_j - h) = 2f(x_j) + f''(x_j)h^2 + \frac{h^4}{24}(f^{(4)}(\xi_+) + f^{(4)}(\xi_-)). \quad (2.47)$$

Rearranging, we get the second-order approximation of the second derivative as

$$f''(x_j) = \frac{1}{h^2}(f(x_{j+1}) - 2f(x_j) + f(x_{j-1})) - \frac{h^2}{24}(f^{(4)}(\xi_+) + f^{(4)}(\xi_-)). \quad (2.48)$$

Similar approximations can be made for higher-order derivatives of $f(x)$, or for higher-order approximations of the second derivative. For example, we could similarly add equations (2.45a) and (2.45b) to get a second $\mathcal{O}(h^2)$ -accurate approximation to $f''(x_j)$ and use a weighted difference of this and the expression above to get a fourth-order approximation. As a general rule of thumb, at least $k + 1$ points must be used to approximate the k th derivative of f , but higher-order approximations are possible if more points are used, or if symmetries in the mesh spacing and differencing weights can be exploited.

Finally, the finite-difference schemes above also extend to higher dimensions. For example, for $f(x, y)$ in 2D with h_x and h_y mesh spacing, we can obtain approximations to the partial derivatives in their respective coordinates. For example, centered-difference formulas for the first partial derivatives along the x and y coordinates are given by

$$\frac{\partial f}{\partial x}(x_i, y_j) = \frac{f(x_{i+1}, y_j) - f(x_{i-1}, y_j)}{2h_x} - \frac{h_x^2}{12} \left(\frac{\partial^3 f}{\partial x^3}(\xi_+, y_j) + \frac{\partial^3 f}{\partial x^3}(\xi_-, y_j) \right), \quad (2.49a)$$

$$\frac{\partial f}{\partial y}(x_i, y_j) = \frac{f(x_i, y_{j+1}) - f(x_i, y_{j-1})}{2h_y} - \frac{h_y^2}{12} \left(\frac{\partial^3 f}{\partial y^3}(x_i, \eta_+) + \frac{\partial^3 f}{\partial y^3}(x_i, \eta_-) \right), \quad (2.49b)$$

and centered-difference formulas for the second partial derivatives with respect to x and y are given by

$$\begin{aligned} \frac{\partial^2 f}{\partial x^2}(x_i, y_j) &= \frac{f(x_{i+1}, y_j) - 2f(x_i, y_j) + f(x_{i-1}, y_j)}{h_x^2} \\ &\quad - \frac{h_x^2}{24} \left(\frac{\partial^4 f}{\partial x^4}(\xi_+, y_j) + \frac{\partial^4 f}{\partial x^4}(\xi_-, y_j) \right), \end{aligned} \quad (2.50a)$$

$$\begin{aligned} \frac{\partial^2 f}{\partial y^2}(x_i, y_j) &= \frac{f(x_i, y_{j+1}) - 2f(x_i, y_j) + f(x_i, y_{j-1}))}{h_y^2} \\ &\quad - \frac{h_y^2}{24} \left(\frac{\partial^4 f}{\partial y^4}(x_i, \eta_+) + \frac{\partial^4 f}{\partial y^4}(x_i, \eta_-) \right). \end{aligned} \quad (2.50b)$$

We now also have to consider the mixed partial derivatives that may, for example, arise in 2D elliptic PDEs. These approximations involve points that have different coordinates in both the x - and y -directions. Considering points (x_{i+1}, y_{j+1}) and (x_{i-1}, y_{j-1}) , the two-dimensional version of Taylor's theorem provides the following expansions, assuming $f(x, y)$ is at least five times continuously differentiable:

$$\begin{aligned} f(x_{i+1}, y_{j+1}) &= f(x_i, y_j) + h_x \frac{\partial f(x_i, y_j)}{\partial x} + h_y \frac{\partial f(x_i, y_j)}{\partial y} \\ &\quad + \frac{h_x^2}{2} \frac{\partial^2 f(x_i, y_j)}{\partial x^2} + h_x h_y \frac{\partial^2 f(x_i, y_j)}{\partial x \partial y} + \frac{h_y^2}{2} \frac{\partial^2 f(x_i, y_j)}{\partial y^2} \\ &\quad + \frac{h_x^3}{6} \frac{\partial^3 f(x_i, y_j)}{\partial x^3} + \frac{h_x^2 h_y}{2} \frac{\partial^3 f(x_i, y_j)}{\partial x^2 \partial y} \\ &\quad + \frac{h_x h_y^2}{2} \frac{\partial^3 f(x_i, y_j)}{\partial x \partial y^2} + \frac{h_y^3}{6} \frac{\partial^3 f(x_i, y_j)}{\partial y^3} \\ &\quad + \frac{h_x^4}{24} \frac{\partial^4 f(x_i, y_j)}{\partial x^4} + \frac{h_x^3 h_y}{6} \frac{\partial^4 f(x_i, y_j)}{\partial x^3 \partial y} + \frac{h_x^2 h_y^2}{4} \frac{\partial^4 f(x_i, y_j)}{\partial x^2 \partial y^2} \\ &\quad + \frac{h_x h_y^3}{6} \frac{\partial^4 f(x_i, y_j)}{\partial x \partial y^3} + \frac{h_y^4}{24} \frac{\partial^4 f(x_i, y_j)}{\partial y^4} + \dots, \end{aligned} \quad (2.51)$$

with a similar expression for $f(x_{i-1}, y_{j-1})$, with opposites signs on all odd-order derivatives. Adding these two together then gives immediate cancellation, with

$$\begin{aligned} f(x_{i+1}, y_{j+1}) + f(x_{i-1}, y_{j-1}) &= 2f(x_i, y_j) \\ &\quad + h_x^2 \frac{\partial^2 f(x_i, y_j)}{\partial x^2} + 2h_x h_y \frac{\partial^2 f(x_i, y_j)}{\partial x \partial y} + h_y^2 \frac{\partial^2 f(x_i, y_j)}{\partial y^2} \\ &\quad + \frac{h_x^4}{12} \frac{\partial^4 f(x_i, y_j)}{\partial x^4} + \frac{h_x^3 h_y}{3} \frac{\partial^4 f(x_i, y_j)}{\partial x^3 \partial y} + \frac{h_x^2 h_y^2}{2} \frac{\partial^4 f(x_i, y_j)}{\partial x^2 \partial y^2} \\ &\quad + \frac{h_x h_y^3}{3} \frac{\partial^4 f(x_i, y_j)}{\partial x \partial y^3} + \frac{h_y^4}{12} \frac{\partial^4 f(x_i, y_j)}{\partial y^4} + \dots \end{aligned} \quad (2.52)$$

Then, substituting equations (2.50a) and (2.50b) into the above and rearranging yields

$$\begin{aligned} \frac{\partial^2 f(x_i, y_j)}{\partial x \partial y} = & \frac{1}{2h_x h_y} \left(f(x_{i+1}, y_{j+1}) + f(x_{i-1}, y_{j-1}) + 2f(x_i, y_j) \right. \\ & \left. - f(x_{i+1}, y_j) - f(x_{i-1}, y_j) - f(x_i, y_{j+1}) - f(x_i, y_{j-1}) \right) \\ & - \frac{h_x^2}{6} \frac{\partial^4 f(x_i, y_j)}{\partial x^3 \partial y} - \frac{h_x h_y}{4} \frac{\partial^4 f(x_i, y_j)}{\partial x^2 \partial y^2} - \frac{h_y^2}{6} \frac{\partial^4 f(x_i, y_j)}{\partial x \partial y^3} + \dots \end{aligned} \quad (2.53)$$

This shows that the approximation given by

$$\begin{aligned} \frac{\partial^2 f(x_i, y_j)}{\partial x \partial y} \approx & \frac{1}{2h_x h_y} \left(f(x_{i+1}, y_{j+1}) + f(x_{i-1}, y_{j-1}) + 2f(x_i, y_j) \right. \\ & \left. - f(x_{i+1}, y_j) - f(x_{i-1}, y_j) - f(x_i, y_{j+1}) - f(x_i, y_{j-1}) \right) \end{aligned} \quad (2.54)$$

is second-order accurate when h_x and h_y are reduced at the same rate. It is important to note that equation (2.54) is not the only second-order approximation of the mixed partial derivative; we could instead derive a similar form using the points (x_{i+1}, y_{j-1}) and (x_{i-1}, y_{j+1}) .

An alternative way to evaluate the accuracy of a finite-difference formula like equation (2.54) is to realize that an approximation of a derivative operator should correctly approximate the derivatives of low-order polynomials. Thus, we can consider $f(x, y) = x^p y^q$ for small values of p and q in equation (2.54). A straightforward calculation verifies that if $p = 0$ or $q = 0$, the formula correctly gives $\frac{\partial^2 f}{\partial x \partial y} \equiv 0$, while for $p = q = 1$, the formula also correctly gives $\frac{\partial^2 f}{\partial x \partial y} \equiv 1$. We revisit this idea in Section 3.6.2 to construct finite-difference approximations in the more general context of so-called meshless methods that use arbitrary clouds of points in any number of dimensions. As we will see later on, the finite-difference method relies on these approximations of the derivatives to represent the PDE and obtain an approximate solution.

2.5 ■ Approximating integrals: Quadrature

Objectives

In this section you will develop formulae to approximate the integral of a function, which are also referred to as quadrature rules. Using orthogonal polynomials, you will assess the accuracy of such schemes in 1D. You will see how we can extend these schemes to higher-dimensional integrals.

We first consider the task of approximating the integral of a function of one variable, $f(x)$, over a finite interval $a \leq x \leq b$, known as (numerical) quadrature. That is, we aim to approximate

$$I = \int_a^b f(x) dx. \quad (2.55)$$

There are two common families of techniques for doing this that are used in various settings. The first family is based on considering (piecewise) polynomial interpolants of $f(x)$, which are then integrated exactly. The second family, known as Gaussian integration methods, is based on approximating the integral by a linear combination of function values at locations and with weights that are chosen to integrate polynomials up to a certain degree exactly.

2.5.1 ■ Integrating interpolating polynomials

Approximating $f(x)$ directly by a polynomial leads to a well-known set of approximations of I . For a piecewise constant interpolant that interpolates $f(x)$ at the midpoint of the interval, we get

$$\int_a^b f(x) dx \approx \int_a^b f\left(\frac{a+b}{2}\right) dx = (b-a)f\left(\frac{a+b}{2}\right), \quad (2.56)$$

which is known as the midpoint rule. Similarly, for piecewise linear interpolation at the interval boundaries, we get

$$\int_a^b f(x) dx \approx \int_a^b \frac{b-x}{b-a} f(a) + \frac{x-a}{b-a} f(b) dx = \frac{b-a}{2} (f(a) + f(b)), \quad (2.57)$$

which is known as the trapezoidal rule, since it approximates the area under the curve of $f(x)$ by a trapezoid determined by the points $(a, f(a))$ and $(b, f(b))$. As a final example of methods in this class, we approximate $f(x)$ by a quadratic interpolating polynomial. Taking $c = (a+b)/2$, we set

$$\int_a^b f(x) dx \approx \int_a^b \frac{(b-x)(c-x)}{(b-a)(c-a)} f(a) + \frac{(x-a)(b-x)}{(c-a)(b-c)} f(c) + \frac{(x-a)(c-x)}{(b-a)(c-b)} f(b) dx \quad (2.58a)$$

$$= \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right), \quad (2.58b)$$

which is known as Simpson's rule or Simpson's 1/3 rule, since $\frac{b-a}{6} = \frac{c-a}{3} = \frac{b-c}{3}$. Of course, one can continue to derive methods by going to higher- and higher-order polynomial interpolants (using a cubic interpolant, for example, yields what is known as Simpson's 3/8 rule), but it is more common to gain accuracy in these approximations using the composite approaches below.

The accuracy in the above schemes is estimated by using the Lagrange form of the remainder from Taylor's theorem, as given in Corollary 2.3. For the midpoint rule, we show that

$$\left| I - (b-a)f\left(\frac{a+b}{2}\right) \right| \leq \frac{(b-a)^3}{24} \max_{a \leq \xi \leq b} |f''(\xi)|. \quad (2.59)$$

Similarly, the trapezoidal rule error is bounded as

$$\left| I - \frac{b-a}{2} (f(a) + f(b)) \right| \leq \frac{(b-a)^3}{12} \max_{a \leq \xi \leq b} |f''(\xi)|, \quad (2.60)$$

and the error for Simpson's rule is bounded as

$$\left| I - \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right) \right| \leq \frac{1}{90} \left(\frac{b-a}{2} \right)^5 \max_{a \leq \xi \leq b} |f'''(\xi)|. \quad (2.61)$$

While these bounds show improvement with increasing order of polynomial approximation, they also make it clear that the approximation is going to be fundamentally limited by the length of the interval, $b-a$.

To overcome this limitation, we use *composite* integration rules that arise from piecewise polynomial interpolation over a mesh of the interval from a to b . For example, dividing the interval $[a, b]$ into n_x evenly spaced subintervals, $[x_i, x_{i+1}]$ for $0 \leq i \leq n_x - 1$, with $h = \frac{b-a}{n_x}$

and $x_i = a + ih$, we apply the midpoint rule to approximate the integral on each subinterval, writing the composite midpoint rule as

$$\int_a^b f(x) dx = \sum_{i=0}^{n_x-1} \int_{x_i}^{x_{i+1}} f(x) dx \approx \sum_{i=0}^{n_x-1} f\left(\frac{x_i + x_{i+1}}{2}\right) h. \quad (2.62)$$

We directly sum the error terms associated with the approximation of each term $\int_{x_i}^{x_{i+1}} f(x) dx$ to show that

$$\left| I - \sum_{i=0}^{n_x-1} f\left(\frac{x_i + x_{i+1}}{2}\right) h \right| \leq \frac{(b-a)^3}{24n_x^2} \max_{a \leq \xi \leq b} |f''(\xi)|. \quad (2.63)$$

Note that the denominator scales like n_x^2 and not n_x^3 , since we arrive at the expression by summing n_x terms that each contain n_x^3 in the denominator. Similarly, we derive the composite trapezoidal rule as

$$\int_a^b f(x) dx \approx \sum_{i=0}^{n_x-1} \frac{h}{2} (f(x_i) + f(x_{i+1})), \quad (2.64)$$

with error bound

$$\left| I - \sum_{i=0}^{n_x-1} \frac{h}{2} (f(x_i) + f(x_{i+1})) \right| \leq \frac{(b-a)^3}{12n_x^2} \max_{a \leq \xi \leq b} |f''(\xi)|. \quad (2.65)$$

2.5.2 ■ Gaussian quadrature

Our second family of methods for approximating integrals is known as the Gaussian quadrature rules. We first consider the interval $[-1, 1]$ and will later use a change of variables to map these rules to general intervals. Gaussian quadrature is based on determining optimal *quadrature points*, x_i , and *quadrature weights*, w_i , in the approximation

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^n w_i f(x_i), \quad (2.66)$$

where we define *optimal* to mean that we maximize the precision of the quadrature rule, defined as the largest degree of polynomial for which the quadrature rule is exact. To determine the optimal quadrature points and weights, it is useful to consider a basis of orthogonal polynomials for the vector space of polynomials of degree up to k . Specifically, for $k \geq 0$, we define the Legendre polynomial, $p_k(x)$, as the polynomial of degree k such that $p_k(1) = 1$ and $p_k(x)$ satisfies the orthogonality relation

$$\int_{-1}^1 p_k(x) p_\ell(x) dx = 0 \text{ for all } \ell < k. \quad (2.67)$$

This gives the degree-zero polynomial $p_0(x) = 1$ and the degree-one polynomial $p_1(x) = x$. Higher-order polynomials are shown to satisfy the recurrence relation

$$(k+1)p_{k+1}(x) = (2k+1)xp_k(x) - kp_{k-1}(x). \quad (2.68)$$

With this polynomial basis defined, it can be shown that any piecewise continuous function $f(x)$ defined on $[-1, 1]$ (with finitely many points of discontinuity) can be written as

$$f(x) = \sum_{\ell=0}^{\infty} a_\ell p_\ell(x), \text{ where } a_\ell = \frac{2\ell+1}{2} \int_{-1}^1 f(x) p_\ell(x) dx. \quad (2.69)$$

We now make use of Legendre polynomials to investigate how the quadrature points and weights in the Gaussian integration rule (2.66) are determined optimally to maximize the degree of polynomials that are integrated exactly. A first question that arises is, if $f(x)$ is a polynomial of degree at most k , how many pointwise values are needed in equation (2.66) to guarantee that its integral is evaluated exactly? The somewhat surprising answer to this question is that, when the points are optimally sampled, we can exactly integrate all polynomials up to degree $2n - 1$ with n samples. That is, there exist n quadrature weights, w_i , and points x_i , such that

$$\int_{-1}^1 f(x) dx = \sum_{i=1}^n w_i f(x_i) \quad (2.70)$$

for any function $f(x)$ that is a polynomial of degree at most $2n - 1$. We find these points by making use of the Legendre polynomials above, and by noticing that if $f(x)$ is a polynomial of degree at most $2n - 1$, then it can be written as

$$f(x) = q_{n-1}(x)p_n(x) + r_{n-1}(x), \quad (2.71)$$

where $p_n(x)$ is the Legendre polynomial of degree n and $q_{n-1}(x)$ and $r_{n-1}(x)$ are both polynomials of degree at most $n - 1$. Noticing that $q_{n-1}(x)$ can be written as a linear combination of $p_0(x)$ through $p_{n-1}(x)$, it must be the case that $\int_{-1}^1 q_{n-1}(x)p_n(x) dx = 0$, due to the orthogonality of the Legendre polynomials. Thus, we look to choose $\{w_i\}$ and $\{x_i\}$ such that

$$\int_{-1}^1 f(x) dx = \int_{-1}^1 r_{n-1}(x) dx = \sum_{i=1}^n w_i f(x_i). \quad (2.72)$$

This is possible if we select $\{x_i\}$ to be the n roots of $p_n(x) = 0$, giving $f(x_i) = r_{n-1}(x_i)$ for $1 \leq i \leq n$. The n weights, w_i , are then found by requiring

$$\int_{-1}^1 r_{n-1}(x) dx = \sum_{i=1}^n w_i r_{n-1}(x_i) \quad (2.73)$$

for all polynomials $r_{n-1}(x)$ of degree at most $n - 1$. With this, we solve for the weights, for example, by simply noting that $\int_{-1}^1 x^k dx = \frac{1 - (-1)^{k+1}}{k+1}$, so that the weights are uniquely determined by the set of equations

$$2 = \sum_{i=1}^n w_i, \quad (2.74a)$$

$$0 = \sum_{i=1}^n w_i x_i, \quad (2.74b)$$

$$\frac{2}{3} = \sum_{i=1}^n w_i x_i^2, \quad (2.74c)$$

$\vdots$

$$\frac{1 - (-1)^{n-1}}{n} = \sum_{i=1}^n w_i x_i^{n-1}. \quad (2.74d)$$

While direct solution of this Vandermonde system is often numerically unstable (particularly for large n), there are alternative forms that express the quadrature weights directly in terms of the Legendre polynomials.

For $n = 1$, this recovers the midpoint rule,

$$\int_{-1}^1 f(x) dx \approx 2f(0), \quad (2.75)$$

which we know to be exact for all linear functions (degree-one polynomials). For $n = 2$, we find a new two-point quadrature rule,

$$\int_{-1}^1 f(x) dx \approx f\left(\frac{-1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right), \quad (2.76)$$

which is exact for all polynomials of degree three or less. Similarly, for $n = 3$, we get a three-point quadrature rule,

$$\int_{-1}^1 f(x) dx \approx \frac{5}{9}f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9}f(0) + \frac{5}{9}f\left(\sqrt{\frac{3}{5}}\right), \quad (2.77)$$

which is exact for all polynomials of degree five or less.

Next, we extend the approach to general intervals $[a, b]$. For this, we make use of a standard change of variables to map the integral of $f(x)$ over the interval $[a, b]$ to an equivalent integral over $[-1, 1]$,

$$\int_a^b f(x) dx = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{a+b}{2} + \frac{b-a}{2}y\right) dy, \quad (2.78)$$

where $x = \frac{a+b}{2} + \frac{b-a}{2}y$. We then directly use the quadrature rules on $[-1, 1]$, obtaining the approximations

$$\int_a^b f(x) dx \approx (b-a)f\left(\frac{a+b}{2}\right), \quad (2.79a)$$

$$\int_a^b f(x) dx \approx \frac{b-a}{2} \left(f\left(\frac{a+b}{2} - \frac{b-a}{2\sqrt{3}}\right) + f\left(\frac{a+b}{2} + \frac{b-a}{2\sqrt{3}}\right) \right), \quad (2.79b)$$

and so on. The standard error estimate for Gaussian quadrature with nodes x_i and weights w_i mapped to the interval $[a, b]$ is

$$\left| \int_a^b f(x) dx - \sum_{i=1}^n w_i f(x_i) \right| \leq \left(\int_a^b p_n(x)^2 dx \right) \max_{a \leq \xi \leq b} \frac{|f^{(2n)}(\xi)|}{(2n)!}, \quad (2.80)$$

where $p_n(x)$ is the n th Legendre polynomial mapped to the interval $[a, b]$ (cf. [206, Thm. 3.6.24]). Thus, while composite versions of Gaussian quadrature can be constructed as above, it is often more attractive to increase n to achieve a better error bound, particularly when the integrand is known to be polynomial (so that there exists an n for which the Gaussian quadrature becomes exact) or is well approximated by a polynomial of moderate degree.

For higher-dimensional integrals, we often use iterated integrals to apply these quadrature rules. For example, for any quadrilateral region in 2D, we write the affine mapping from the square $[-1, 1]^2$ to the quadrilateral Q with vertices (x_j, y_j) , $1 \leq j \leq 4$, as

$$\begin{aligned} \begin{bmatrix} u \\ v \end{bmatrix} &= \frac{(1-x)(1-y)}{4} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + \frac{(1+x)(1-y)}{4} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} \\ &+ \frac{(1+x)(1+y)}{4} \begin{bmatrix} x_3 \\ y_3 \end{bmatrix} + \frac{(1-x)(1+y)}{4} \begin{bmatrix} x_4 \\ y_4 \end{bmatrix}, \end{aligned} \quad (2.81)$$

mapping every point $(x, y) \in [-1, 1]^2$ into Q . Then, we use a standard change of variables to map an integral over Q into one over $[-1, 1]^2$ and approximate the integral over $[-1, 1]^2$ by applying one of the quadrature rules above to the iterated form of that area integral. The same approach can be used over hexahedra in three dimensions. For more complicated domains in two dimensions, one approach is to first subdivide the domain into quadrilaterals (possibly introducing an approximation of the domain if it features curved edges) and then map each quadrilateral onto $[-1, 1]^2$ as above.

In two dimensions, it is also common to directly define quadrature rules over triangles, which are useful when considering a triangulation of a complicated domain (such as we use in Chapter 8). These quadrature rules can be directly defined on triangles or can be based on a singular transformation from a triangular region to a square. While there is no systematic construction of these rules as in the quadrilateral case, many low-order rules are known and in common use; see [207, Table 4.1] for a list of rules on triangles and [144] for rules on tetrahedra. For example, the equivalent of the midpoint rule for the triangle T with vertices at $(0, 0)$, $(1, 0)$, and $(0, 1)$ is

$$\int_T f(x, y) d\mathbf{x} \approx \frac{1}{2} f\left(\frac{1}{3}, \frac{1}{3}\right), \quad (2.82)$$

which is exact for all bivariate polynomials that are linear in both x and y (but not bilinear polynomials, which include terms like xy). A three-point scheme is given by

$$\int_T f(x, y) d\mathbf{x} \approx \frac{1}{6} \left(f\left(\frac{1}{6}, \frac{1}{6}\right) + f\left(\frac{1}{6}, \frac{2}{3}\right) + f\left(\frac{2}{3}, \frac{1}{6}\right) \right), \quad (2.83)$$

which is exact for all bivariate polynomials of the form $c_{0,0} + c_{1,0}x + c_{0,1}y + c_{1,1}xy + c_{2,0}x^2 + c_{0,2}y^2$. Higher-precision rules are also known.

2.6 ■ Integration properties

Objectives

Integration plays an important role in the derivation of many numerical methods for PDEs. In this section, you will review several identities that are frequently used in this context.

When deriving numerical methods for PDEs, it is often useful to transition from the usual “strong” form of a differential equation to its corresponding “weak” form, which typically requires some use of integration by parts to weaken the assumed smoothness of a solution. An important aspect of this transition is to properly keep track of the boundary terms produced. In this section, we review integration-by-parts theorems, both in 1D and the two- and three-dimensional analogues that arise from applying Green’s and the divergence theorems. As we will see, these theorems can, in general, be thought of as providing a similar set of tools in various settings, although they produce subtly different boundary integrals in their different forms.

2.6.1 ■ Integration by parts in 1D, 2D, and 3D

The classical formulation for integration by parts in one dimension is given in Theorem 2.16.

Theorem 2.16: Integration by parts in 1D

Let $[a, b] \subset \mathbb{R}$, and let $f : [a, b] \rightarrow \mathbb{R}$ and $g : [a, b] \rightarrow \mathbb{R}$ be continuously differentiable. Then,

$$\int_a^b f(x)g'(x)dx = f(x)g(x)\Big|_a^b - \int_a^b f'(x)g(x)dx. \quad (2.84)$$

Proof. By the product rule,

$$(f(x)g(x))' = f'(x)g(x) + f(x)g'(x) \quad (2.85a)$$

$$\Rightarrow \int_a^b (f(x)g(x))' dx = \int_a^b (f'(x)g(x) + f(x)g'(x)) dx. \quad (2.85b)$$

However, by the fundamental theorem of calculus,

$$\int_a^b (f(x)g(x))' dx = f(x)g(x) \Big|_a^b \quad (2.86a)$$

$$\Rightarrow \int_a^b f(x)g'(x) dx = f(x)g(x) \Big|_a^b - \int_a^b f'(x)g(x) dx. \quad (2.86b)$$

□

We note some properties of this one-dimensional form. First, the “goal” of integration by parts is to move the derivative on $g(x)$ in the left-hand integral onto $f(x)$ in the right-hand integral. The integration-by-parts formula tells us that two changes are then necessary to accommodate this. First, the sign on the integral changes. In addition, a “boundary term” appears, evaluating $f(x)g(x)$ at the two endpoints of the interval $[a, b]$. We will see similar changes arise in two and three dimensions.

In two dimensions, the fundamental theorem of calculus is replaced by the second vector form (flux form) of Green’s theorem,

$$\int_{\Omega} \nabla \cdot \mathbf{f}(x, y) d\mathbf{x} = \int_{\partial\Omega} \mathbf{f} \cdot \mathbf{n} ds, \quad (2.87)$$

where $\partial\Omega$ is the boundary of the region $\Omega \subset \mathbb{R}^2$ with positive (i.e., counterclockwise) orientation and $\mathbf{n}$ is the outward unit normal. We note that we will use $d\mathbf{x}$ as the differential for integrals over domains in more than one dimension and ds for the differential over a boundary line or surface. While it is sometimes customary to denote integrals in multiple dimensions by double or triple integration symbols, we avoid this notation here.

In the context of finite-element methods for second-order elliptic PDEs, an important case is to take $\mathbf{f} = u \nabla v$, which gives

$$\int_{\Omega} \nabla \cdot (u \nabla v) d\mathbf{x} = \int_{\partial\Omega} u (\nabla v \cdot \mathbf{n}) ds. \quad (2.88)$$

Now, using standard vector calculus identities, we have $\nabla \cdot (u \nabla v) = u(\nabla \cdot \nabla v) + (\nabla u) \cdot (\nabla v)$, giving

$$\int_{\Omega} \nabla \cdot (u \nabla v) d\mathbf{x} = \int_{\Omega} u(\nabla \cdot \nabla v) + (\nabla u) \cdot (\nabla v) d\mathbf{x} \quad (2.89a)$$

$$\Rightarrow \int_{\Omega} u(\nabla \cdot \nabla v) d\mathbf{x} = \int_{\partial\Omega} u(\nabla v \cdot \mathbf{n}) ds - \int_{\Omega} (\nabla u) \cdot (\nabla v) d\mathbf{x}. \quad (2.89b)$$

Note, again, the change in sign between the integrals on the left- and right-hand sides, as well as the integral over $\partial\Omega$ that appears.

In 3D, an analogous result is obtained using the divergence theorem,

$$\int_{\Omega} \nabla \cdot \mathbf{f} d\mathbf{x} = \int_{\partial\Omega} \mathbf{f} \cdot \mathbf{n} ds, \quad (2.90)$$

where $\partial\Omega$ is the closed surface that encloses the volume Ω , with outward positive orientation. Again taking $\mathbf{f} = u(\nabla v)$ gives

$$\int_{\Omega} u(\nabla \cdot \nabla v) d\mathbf{x} = \int_{\partial\Omega} (u \nabla v \cdot \mathbf{n}) ds - \int_{\Omega} (\nabla u) \cdot (\nabla v) d\mathbf{x}. \quad (2.91)$$

2.6.2 ■ Further integration-by-parts formulas

Two further situations that may arise in the context of finite-element methods involve integrands of the form $\int_{\Omega} \mathbf{u} \cdot (\nabla \nabla \cdot \mathbf{v}) d\mathbf{x}$ and $\int_{\Omega} \mathbf{u} \cdot (\nabla \times \nabla \times \mathbf{v}) d\mathbf{x}$ and their two-dimensional analogues. In these cases, similar results hold. For the former, we again use the flux form of Green's theorem in 2D, equation (2.87), or the divergence theorem in 3D, equation (2.90), now applied to the vector field $\mathbf{f} = \mathbf{u} \nabla \cdot \mathbf{v}$, giving (in either case)

$$\int_{\Omega} \mathbf{u} \cdot (\nabla \nabla \cdot \mathbf{v}) d\mathbf{x} = \int_{\partial\Omega} (\nabla \cdot \mathbf{v}) \mathbf{u} \cdot \mathbf{n} ds - \int_{\Omega} (\nabla \cdot \mathbf{u}) (\nabla \cdot \mathbf{v}) d\mathbf{x}. \quad (2.92)$$

For the latter, in 3D, we do the same with $\mathbf{f} = \mathbf{u} \times (\nabla \times \mathbf{v})$, using the identity that $\nabla \cdot (\mathbf{p} \times \mathbf{q}) = (\nabla \times \mathbf{p}) \cdot \mathbf{q} - \mathbf{p} \cdot (\nabla \times \mathbf{q})$. This gives

$$\int_{\Omega} \mathbf{u} \cdot (\nabla \times \nabla \times \mathbf{v}) d\mathbf{x} = \int_{\Omega} (\nabla \times \mathbf{u}) \cdot (\nabla \times \mathbf{v}) d\mathbf{x} - \int_{\partial\Omega} (\mathbf{u} \times (\nabla \times \mathbf{v})) \cdot \mathbf{n} ds. \quad (2.93)$$

Note that in this case there is no change of sign between the volume integrals on the left- and right-hand sides. Two further manipulations of the boundary term come from standard vector identities, giving

$$\int_{\Omega} \mathbf{u} \cdot (\nabla \times \nabla \times \mathbf{v}) d\mathbf{x} = \int_{\Omega} (\nabla \times \mathbf{u}) \cdot (\nabla \times \mathbf{v}) d\mathbf{x} + \int_{\partial\Omega} (\nabla \times \mathbf{v}) \cdot (\mathbf{u} \times \mathbf{n}) ds, \quad (2.94a)$$

$$\int_{\Omega} \mathbf{u} \cdot (\nabla \times \nabla \times \mathbf{v}) d\mathbf{x} = \int_{\Omega} (\nabla \times \mathbf{u}) \cdot (\nabla \times \mathbf{v}) d\mathbf{x} - \int_{\partial\Omega} \mathbf{u} \cdot ((\nabla \times \mathbf{v}) \times \mathbf{n}) ds. \quad (2.94b)$$

Two-dimensional analogues of these also hold, but we first need to differentiate between the scalar and vector versions of the curl operator. To do so, we consider vector fields

$$\mathbf{u} = \begin{bmatrix} u^{(1)}(x, y) \\ u^{(2)}(x, y) \\ 0 \end{bmatrix} \text{ and } \mathbf{v} = \begin{bmatrix} v^{(1)}(x, y) \\ v^{(2)}(x, y) \\ 0 \end{bmatrix} \quad (2.95)$$

as the “embeddings” of two-dimensional vectors in 3D. With this, we note that

$$\nabla \times \mathbf{v} = \begin{bmatrix} 0 \\ 0 \\ v_x^{(2)}(x, y) - v_y^{(1)}(x, y) \end{bmatrix}. \quad (2.96)$$

Thus, we can think of the “curl” of a two-dimensional vector field as a scalar function, defining $\text{rot } \mathbf{v} = v_x^{(2)}(x, y) - v_y^{(1)}(x, y)$. To make sense of $\nabla \times \nabla \times \mathbf{v}$ in 2D, we again use the embedding in 3D to find

$$\nabla \times \nabla \times \mathbf{v} = \begin{bmatrix} v_{xy}^{(2)}(x, y) - v_{yy}^{(1)}(x, y) \\ v_{xy}^{(1)}(x, y) - v_{xx}^{(2)}(x, y) \\ 0 \end{bmatrix}, \quad (2.97)$$

showing that we can interpret $\mathbf{u} \cdot (\nabla \times \nabla \times \mathbf{v})$ as the usual dot product between two 2D vectors. Similarly, we can define the 2D vector field $\mathbf{f} = \mathbf{u} \times (\nabla \times \mathbf{v})$ by its embedding in 3D and apply the analogous identity as in the 3D case. This yields

$$\int_{\Omega} \mathbf{u} \cdot (\nabla \times \nabla \times \mathbf{v}) \, d\mathbf{x} = \int_{\Omega} (\text{rot } \mathbf{u}) (\text{rot } \mathbf{v}) \, d\mathbf{x} + \int_{\partial\Omega} (\text{rot } \mathbf{v}) \left(u^{(1)} n^{(2)} - u^{(2)} n^{(1)} \right) ds, \quad (2.98)$$

where the outward normal vector to $\Omega \in \mathbb{R}^2$ is $\mathbf{n} = \begin{bmatrix} n^{(1)} \\ n^{(2)} \end{bmatrix}$. Note that we can write the unit tangential vector to $\partial\Omega$ as $\mathbf{t} = \begin{bmatrix} n^{(2)} \\ -n^{(1)} \end{bmatrix}$, which allows us to write the above identity as

$$\int_{\Omega} \mathbf{u} \cdot (\nabla \times \nabla \times \mathbf{v}) \, d\mathbf{x} = \int_{\Omega} (\text{rot } \mathbf{u}) (\text{rot } \mathbf{v}) \, d\mathbf{x} + \int_{\partial\Omega} (\text{rot } \mathbf{v}) (\mathbf{u} \cdot \mathbf{t}) \, ds. \quad (2.99)$$

2.7 ■ Some further thoughts

Function interpolation, numerical integration, and numerical differentiation represent a major portion of any course on numerical methods. It is impossible to complete the study of finite-element methods, for example, without an in-depth understanding of both quadrature schemes and various polynomial representations. Likewise, there are extensive classes of numerical methods that use differencing schemes, for example, finite-difference timestepping methods and spectral methods. Here, we have given a brief overview of a few of the tools that we will use throughout the text, but it is important to point out that we have only given a brief taste of the vast field of numerical analysis, as well as reviewing key tools from calculus. We could embark on a more complete overview, which could start, for example, with the method of exhaustion from Eudoxus and others for approximating the area of a circular disk, which originated around 350 BCE (see Figure 2.7.1), but instead we point to additional resources.

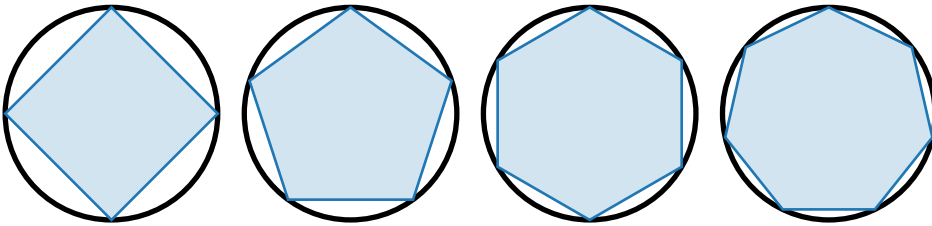


Figure 2.7.1. Method of exhaustion in approximating the area of a circular disk.

Numerical analysis texts abound, but a standout for its readability and accessibility is *A First Course in Numerical Methods* by Ascher and Greif [20]. A classic and comprehensive treatment of numerical analysis, covering both the core topics of this chapter in detail as well as more advanced topics in numerical analysis, is *An Introduction to Numerical Analysis* by Atkinson [22]. *Scientific Computing: An Introductory Survey* by Heath [127] takes a different approach, by offering a survey of methods and focusing on the motivation. *Numerical Mathematics* by Quarteroni, Sacco, and Saleri [181] is another example of a comprehensive and modern treatment of scientific computing. Their sharp and concise coverage of the key components of numerical analysis would be a natural reference to consider.

Part II

Basic topics

Chapter 3

Finite-difference methods for elliptic partial differential equations

Overview

Finite-difference methods are some of the most approachable techniques for the discretization of differential equations and can be used in a wide variety of applications. In this chapter, we present the basic finite-difference methodology for elliptic PDEs, with a particular focus on boundary-value problems governed by the Poisson equation on simple domains in one and two dimensions:

$$-u'' = f \quad \text{and} \quad -\Delta u = f. \quad (3.1)$$

Finite-difference methods have a great advantage in their simplicity for one-dimensional problems or structured domains in two or more dimensions. Later in the chapter, we explore how these ideas can be extended to more general settings.

The topics in this chapter lay the groundwork for several later sections of the text, including Chapter 5, where you will see the same tools applied to time-dependent differential equations, and Chapter 7, where you will explore techniques for solving the linear systems of equations that arise from finite-difference (and other) discretizations.

Objectives

In this chapter, we will introduce the main ideas of finite-difference discretizations and their analysis. In particular, you will see how to use pointwise values of a function to *approximate* its derivatives, and how to use this to approximate solutions to PDEs. In this setting, you will understand the connection between PDE discretizations and linear systems, which we explore many times in this book. Using the techniques from this chapter, you will be able to

- express the finite-difference discretization of a differential equation as a linear system of equations for approximations to the values of the function at the points of the mesh;
- understand and analyze both the stability and accuracy of such an approximation and how boundary conditions impact these; and
- investigate variants such as structure-preserving methods and finite-difference approaches in curvilinear coordinates or when no mesh exists.

Chapter Outline

Section 3.1 Finite-difference methods for elliptic PDEs on uniform meshes presents the fundamental ideas of finite-difference discretizations for elliptic PDEs in the context of elliptic boundary-value problems with Dirichlet boundary conditions on uniform grids. The finite-difference discretizations are obtained by approximating derivatives of the unknown function using Taylor's theorem, resulting in pointwise approximations to the solution of the PDE.

Section 3.2 Finite-difference methods on nonuniform meshes extends the ideas from Section 3.1 to *nonuniform* meshes in one and two dimensions. An important takeaway from this section is that the truncation error is larger on nonuniform meshes than on uniform meshes, which will impact the later convergence analysis.

Section 3.3 Convergence theory for finite-difference methods addresses the important question of what it means for the approximations generated by solving these problems to be “good.” This section introduces the key ideas of truncation error and stability, along with the formal tools needed to use these to prove convergence of a finite-difference approximation to the function that solves the PDE problem. While there are large classes of problems for which the “easy” tools work, we also discuss what might be done in cases where they cannot be applied.

Section 3.4 Treatment of Neumann and periodic boundary conditions develops techniques for extending the “basic” finite-difference methodology from Section 3.1 to handle some other boundary conditions that we commonly encounter. Here, we see that careful treatment of the boundary conditions is needed, both in practice and in the analysis of the schemes.

Section 3.5 Structure-preserving finite-difference methods discusses how the finite-difference methodology can be extended to preserve certain types of structure to the differential equation whose solution we are approximating. This is closely connected to the finite-volume methodology (which we revisit for hyperbolic equations in Chapter 6), and we show how to extend the finite-difference methodology to also approximate *fluxes*, which play a key role in the finite-volume setting.

Section 3.6 Variants of finite-difference methods presents two variants on the finite-difference methodology that serve as a starting point for some modern research in this field. First, we discuss how to apply the finite-difference methodology when our sense of a grid is better represented in curvilinear coordinates than Cartesian coordinates. Second, we discuss the so-called “meshless” finite-difference methodology that can be applied when there is even less structure in the points at which we seek an approximation.

3.1 ■ Finite-difference methods for elliptic PDEs on uniform meshes

Objectives

In this section, you will see how the finite-difference formulas to approximate derivatives that were derived from Taylor's theorem in Section 2.4 are used to transform elliptic PDEs into an algebraic system of equations, in a process known as the finite-difference discretization of the differential equation. The solution of this system defines a pointwise approximation of the solution of the differential equation that will be considered throughout this chapter.

The basic premise of a finite-difference discretization is the use of Taylor's theorem to approximate the derivatives of the unknown function in the differential equation at points on a mesh. Specifically, for elliptic PDEs like the 1D and 2D model problems in equation (3.1), we use the following centered finite-difference approximations from Section 2.4 to approximate the second derivatives on uniform meshes in 1D and 2D. From (2.48), we have

$$u''(x_j) = \frac{1}{h^2} (u(x_{j+1}) - 2u(x_j) + u(x_{j-1})) - \frac{h^2}{24} (u''''(\xi_+) + u''''(\xi_-)) \quad (3.2)$$

and from (2.50), we have

$$\begin{aligned} \frac{\partial^2 u}{\partial x^2}(x_i, y_j) &= \frac{u(x_{i+1}, y_j) - 2u(x_i, y_j) + u(x_{i-1}, y_j))}{h_x^2} \\ &\quad - \frac{h_x^2}{24} \left(\frac{\partial^4 u}{\partial x^4}(\xi_+, y_j) + \frac{\partial^4 u}{\partial x^4}(\xi_-, y_j) \right), \end{aligned} \quad (3.3a)$$

$$\begin{aligned} \frac{\partial^2 u}{\partial y^2}(x_i, y_j) &= \frac{u(x_i, y_{j+1}) - 2u(x_i, y_j) + u(x_i, y_{j-1}))}{h_y^2} \\ &\quad - \frac{h_y^2}{24} \left(\frac{\partial^4 u}{\partial y^4}(x_i, \eta_+) + \frac{\partial^4 u}{\partial y^4}(x_i, \eta_-) \right). \end{aligned} \quad (3.3b)$$

Section 2.4 viewed these formulas in the sense of the *forward problem* of approximating second derivatives of a known function $u(x)$ or $u(x, y)$ given values $u(x_i)$ or $u(x_i, y_j)$ sampled in grid points. In this section, we take the opposite viewpoint, where these formulas and relevant boundary conditions are used to derive a system of linear equations,

$$A \mathbf{u} = \mathbf{f}, \quad (3.4)$$

which allows us to solve the *inverse problem* of determining approximations for the values of the unknown function $u(x)$ or $u(x, y)$ at the grid points, given a PDE and boundary conditions. The unknown approximations are then stored in a vector, $\mathbf{u}$.

3.1.1 ■ 1D model problem

Our first example derives a finite-difference discretization of the 1D boundary-value problem $-u''(x) + u(x) = f(x)$ on domain $0 < x < 1$ with Dirichlet boundary conditions on a uniform mesh, as defined in Definition 2.10.

Example 3.1: 1D boundary-value problem

Consider approximating the solution to $-u''(x) + u(x) = \sin(3\pi x)$ for $0 < x < 1$ and $u(0) = u(1) = 0$, with a uniform mesh $\{x_i\}_{i=0}^{n_x}$ and mesh spacing $h = 1/n_x$.

Let $u_i \approx u(x_i)$ be the finite-difference approximation to the solution at mesh point x_i and let $f_i = \sin(3\pi x_i)$ be the value of the right-hand side evaluated at the mesh point. At each interior mesh point, the differential equation requires that $u(x)$ satisfies

$$-u''(x_i) + u(x_i) = \sin(3\pi x_i). \quad (3.5)$$

We now use the second-order accurate finite-difference approximation of the second derivative, as given in equation (3.2), to generate the equations of the finite-difference approximation: for

each grid point x_i , we replace $u''(x_i)$ by its finite-difference approximation and the exact values $u(x_i)$ by the approximations u_i , giving

$$\frac{1}{h^2} (-u_{i+1} + 2u_i - u_{i-1}) + u_i = f_i \text{ for } 1 \leq i \leq n_x - 1. \quad (3.6)$$

At the boundary points, we use the Dirichlet conditions to set $u_0 = u_{n_x} = 0$.

Together, this is a linear system of $n_x + 1$ equations in $n_x + 1$ unknowns, which we write in matrix-vector form as

$$\begin{bmatrix} 1 & 0 & \cdots & 0 & \cdots \\ -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} & 0 & \cdots \\ 0 & -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} & \cdots \\ & & \ddots & \ddots & \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} \\ & & & & 1 \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{n_x-1} \\ u_{n_x} \end{bmatrix} = \begin{bmatrix} 0 \\ f_1 \\ f_2 \\ \vdots \\ f_{n_x-1} \\ 0 \end{bmatrix}. \quad (3.7)$$

Noting that the equations for u_0 and u_{n_x} are trivial, it is common to rewrite this as a system of $n_x - 1$ equations in the $n_x - 1$ interior unknowns as

$$\begin{bmatrix} \frac{2}{h^2} + 1 & -\frac{1}{h^2} & 0 & \cdots & \cdots \\ -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} & 0 & \cdots \\ & & \ddots & \ddots & \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} \\ & & & & \frac{2}{h^2} + 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n_x-2} \\ u_{n_x-1} \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_{n_x-2} \\ f_{n_x-1} \end{bmatrix}. \quad (3.8)$$

In this case, the matrix is symmetric and Toeplitz (having a constant value on each diagonal of the matrix) and is also diagonally dominant—these properties will prove useful in later analysis. In the case of nonhomogeneous Dirichlet boundary conditions, with prescribed nonzero values of u_0 and u_{n_x} , the right-hand side would be slightly modified, with terms u_0/h^2 and u_{n_x}/h^2 added to the first and last entries, respectively, to account for the terms in the first and last columns of the original linear system.

For any given forcing function $f(x)$ and boundary conditions on $u(x)$, we write the final linear system as either $A^h \mathbf{u}^h = \mathbf{f}^h$, or as $A\mathbf{u} = \mathbf{f}$ when the superscript is not needed for clarity.

Note that the tridiagonal matrix

$$A^h = \frac{1}{h^2} \begin{bmatrix} 2 & -1 & 0 & \cdots \\ -1 & 2 & -1 & \cdots \\ & & \ddots & \ddots \\ & & & -1 & 2 & -1 \\ & & & 0 & -1 & 2 \end{bmatrix}, \quad (3.9)$$

which is the finite-difference discretization of the differential operator $-\frac{d^2}{dx^2}$ on a uniform grid with grid spacing h , is called the 1D Laplacian matrix. We will often encounter tridiagonal Toeplitz matrices, such as this A^h , so we now introduce notation to express these.

Definition 3.2: Tridiagonal Toeplitz matrix

An $n \times n$ matrix, A , is Toeplitz if it has entries that are constant along its diagonals, i.e., $a_{i,j} = a_{i-j}$ for $1 \leq i, j \leq n$:

$$A = \begin{bmatrix} a_0 & a_{-1} & a_{-2} & a_{-3} & \cdots & a_{1-n} \\ a_1 & a_0 & a_{-1} & a_{-2} & \cdots & a_{2-n} \\ a_2 & a_1 & a_0 & a_{-1} & \cdots & a_{3-n} \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ a_{n-1} & \cdots & a_3 & a_2 & a_1 & a_0 \end{bmatrix}. \quad (3.10)$$

When a Toeplitz matrix, A , is also tridiagonal, we write $A = \text{tridiag}(a_1, a_0, a_{-1})$, implying $a_k = 0$ for $k \neq -1, 0, 1$.

With this, we can write the 1D Laplacian matrix as $A^h = \text{tridiag}(\frac{-1}{h^2}, \frac{2}{h^2}, \frac{-1}{h^2})$.

The previous 1D example discussed the case of Dirichlet boundary conditions of the form $u(a) = u_a$ and $u(b) = u_b$ for known values u_a, u_b . Section 3.4 discusses details that arise with Neumann or periodic boundary conditions. In all these cases, the finite-difference approach results in $n_x + 1$ equations in $n_x + 1$ unknowns. The finite-difference approximation to the solution of the differential equation is, simply, the discrete set of approximate function values $\{u_i\}_{i=0}^{n_x}$ that satisfy this set of equations. The well-posedness of these linear systems is discussed in detail in Section 3.3.

3.1.2 ■ 2D model problem

We next consider the finite-difference discretization of elliptic PDEs in 2D, such as the 2D model problem in equation (3.1), on 2D uniform rectangular meshes as defined in Definition 2.11. The following example derives the standard second-order accurate finite-difference discretization of the elliptic PDE boundary-value problem $-\Delta u(x, y) + u(x, y) = f(x, y)$ on domain $[0, 1] \times [0, 1]$ with homogeneous Dirichlet boundary conditions on a uniform rectangular mesh.

Example 3.3: 2D boundary-value problem

Consider approximating the solution to $-\Delta u(x, y) + u(x, y) = \sin(3\pi x) \sin(5\pi y)$ for $[0, 1] \times [0, 1]$ with homogeneous Dirichlet boundary conditions on the four edges of the unit square:

$$u(x, 0) = u(x, 1) = 0 \quad \text{for } 0 < x < 1, \quad (3.11a)$$

$$u(0, y) = u(1, y) = 0 \quad \text{for } 0 < y < 1. \quad (3.11b)$$

We use a uniform tensor-product mesh, with $h_x = 1/n_x$ and $h_y = 1/n_y$, to approximate the solution. The discrete representation of the boundary condition is given by

$$u_{i,0} = u_{i,n_y} = 0 \quad \text{for } 0 \leq i \leq n_x, \quad (3.12a)$$

$$u_{0,j} = u_{n_x,j} = 0 \quad \text{for } 0 \leq j \leq n_y, \quad (3.12b)$$

where the boundary conditions may naturally include the four corners of the unit square.

At interior points on the mesh, we use the centered finite-difference approximations for the second partial derivatives of $u(x, y)$ in the x and y directions from equation (3.3a) and equation (3.3b) to approximate $-\Delta u$ at a mesh point $(x_i, y_j) = (i/n_x, j/n_y)$ as

$$-\Delta u(x_i, y_j) \approx -\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} - \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2}, \quad (3.13)$$

yielding the finite-difference equation at that mesh point as

$$-\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} - \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} + u_{i,j} = \sin\left(\frac{3\pi i}{n_x}\right) \sin\left(\frac{5\pi j}{n_y}\right). \quad (3.14)$$

To write these equations in matrix form, we have to define an ordering when placing the unknowns at the grid points in a vector of unknowns $\mathbf{u}$. To do this, we adopt the convention of a column-based *lexicographical* ordering, where nodes are labeled in order of their indices as follows: using a block-vector notation, we set

$$\mathbf{u} = \begin{bmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \\ \vdots \\ \mathbf{u}_{n_x} \end{bmatrix} \quad \text{with } \mathbf{u}_i = \begin{bmatrix} u_{i,0} \\ u_{i,1} \\ \vdots \\ u_{i,n_y} \end{bmatrix}. \quad (3.15)$$

This means that the nodes are ordered by column (i.e., constant x_i) in the grid, from left to right, and in order of increasing y_j within each column.

Using lexicographic ordering, we can assemble equation (3.14) into a linear system in block form. We write $A\mathbf{u} = \mathbf{f}$ as

$$\begin{bmatrix} B & -\alpha I & & & \\ -\alpha I & B & -\alpha I & & \\ & -\alpha I & B & -\alpha I & \\ & & \ddots & \ddots & \ddots \\ & & & -\alpha I & B & -\alpha I \\ & & & & -\alpha I & B \end{bmatrix} \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \mathbf{u}_3 \\ \vdots \\ \mathbf{u}_{n_x-2} \\ \mathbf{u}_{n_x-1} \end{bmatrix} = \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \\ \vdots \\ \mathbf{f}_{n_x-2} \\ \mathbf{f}_{n_x-1} \end{bmatrix}, \quad (3.16)$$

where $\alpha = 1/h_x^2$,

$$B = \text{tridiag}\left(\frac{-1}{h_y^2}, \frac{2}{h_x^2} + \frac{2}{h_y^2} + 1, \frac{-1}{h_y^2}\right), \quad (3.17)$$

and we have truncated the degrees of freedom associated with homogeneous Dirichlet boundary conditions from the system. We see again that we can approximate the solution of the PDE boundary-value problem by solving a linear system.

The finite-difference discretization of the 2D differential operator $-\Delta$ on a uniform grid with constant grid spacing $h = h_x = h_y$ leads to the block tridiagonal matrix

$$A^h = \frac{1}{h^2} \begin{bmatrix} T & -I & & & \\ -I & T & -I & & \\ & -I & T & -I & \\ & & \ddots & \ddots & \ddots \\ & & & -I & T & -I \\ & & & & -I & T \end{bmatrix}, \quad (3.18)$$

3.2 ■ Finite-difference methods on nonuniform meshes

Objectives

While the finite-difference method is often considered only on uniform meshes, as in the previous section, there is no inherent need for a rectangular mesh to be uniform. In this section, you will see the detailed calculations of how we form first- and second-order accurate approximations of first and second derivatives on nonuniform meshes in 1D and 2D, and how those approximations can be used in the finite-difference methodology.

We first consider derivatives of functions on nonuniform 1D grids, as defined in Definition 2.10, by extending the results of finite-difference formulas on uniform grids that were given in Section 2.4. Consider a mesh on $[a, b]$, with $a = x_0 < x_1 < \dots < x_{n_x} = b$, and function $u(x)$. If $u(x)$ is sufficiently differentiable, we can use Taylor's theorem to write

$$u(x_{j+1}) = u(x_j) + (x_{j+1} - x_j)u'(x_j) + (x_{j+1} - x_j)^2 \frac{u''(\xi_+)}{2}, \quad (3.24a)$$

$$u(x_{j-1}) = u(x_j) + (x_{j-1} - x_j)u'(x_j) + (x_{j-1} - x_j)^2 \frac{u''(\xi_-)}{2}, \quad (3.24b)$$

where we use the notation from Corollary 2.3 that ξ is an unknown point between the two points used in Taylor's theorem, and we use $-$ and $+$ subscripts to differentiate between the points in the intervals (x_{j-1}, x_j) and (x_j, x_{j+1}) , respectively.

Defining $h_j = x_j - x_{j-1}$, we rewrite the above equations to obtain expressions for $u'(x_j)$ in terms of $u(x_j)$ and either $u(x_{j-1})$ or $u(x_{j+1})$ as follows:

$$\text{Forward difference: } u'(x_j) = \frac{u(x_{j+1}) - u(x_j)}{h_{j+1}} - h_{j+1} \frac{u''(\xi_+)}{2}, \quad (3.25a)$$

$$\text{Backward difference: } u'(x_j) = \frac{u(x_j) - u(x_{j-1})}{h_j} + h_j \frac{u''(\xi_-)}{2}. \quad (3.25b)$$

Assuming $u(x)$ is twice continuously differentiable in the interval $[a, b]$, we know that there exists an $M > 0$ such that $|u''(\xi)| < M$ for all $\xi \in [a, b]$. With this, we can bound the truncation error terms above by $M/2$ times either the local step size, h_j or h_{j+1} , or the maximum mesh size, $h = \max_{1 \leq j \leq n_x} h_j$. While this establishes that both approximations improve as h gets smaller (typically meaning that the number of points in the interval gets larger), it also establishes that both give only an $\mathcal{O}(h)$ approximation to $u'(x_j)$, unless $u(x)$ is a linear function and, thus, $u''(x) = 0$.

To get a second-order approximation for the first derivative, we can take a weighted average of two derivative approximations with the weight chosen to cancel some of the error. Assuming that $u(x)$ is three times continuously differentiable on the interval $[a, b]$, we have

$$u(x_{j+1}) = u(x_j) + h_{j+1}u'(x_j) + h_{j+1}^2 \frac{u''(x_j)}{2} + h_{j+1}^3 \frac{u'''(\xi_+)}{6}, \quad (3.26a)$$

$$u(x_{j-1}) = u(x_j) - h_j u'(x_j) + h_j^2 \frac{u''(x_j)}{2} - h_j^3 \frac{u'''(\xi_-)}{6}, \quad (3.26b)$$

noting that ξ_+ and ξ_- are not the same points as before. Now, the forward and backward difference formulas become the following:

$$\text{Forward difference: } u'(x_j) = \frac{u(x_{j+1}) - u(x_j)}{h_{j+1}} - h_{j+1} \frac{u''(x_j)}{2} - h_{j+1}^2 \frac{u'''(\xi_+)}{6}, \quad (3.27a)$$

$$\text{Backward difference: } u'(x_j) = \frac{u(x_j) - u(x_{j-1})}{h_j} + h_j \frac{u''(x_j)}{2} - h_j^2 \frac{u'''(\xi_-)}{6}. \quad (3.27b)$$

Since $u'(x_j) = \frac{h_j}{h_j+h_{j+1}}u'(x_j) + \frac{h_{j+1}}{h_j+h_{j+1}}u'(x_j)$, we can cancel out the first-order truncation error terms with this average, yielding the centered-difference formula

$$u'(x_j) = \frac{h_j}{h_j+h_{j+1}} \left(\frac{u(x_{j+1}) - u(x_j)}{h_{j+1}} - h_{j+1} \frac{u''(x_j)}{2} - h_{j+1}^2 \frac{u'''(\xi_+)}{6} \right) \quad (3.28a)$$

$$+ \frac{h_{j+1}}{h_j+h_{j+1}} \left(\frac{u(x_j) - u(x_{j-1}))}{h_j} + h_j \frac{u''(x_j)}{2} - h_j^2 \frac{u'''(\xi_-)}{6} \right) \quad (3.28b)$$

$$= \frac{1}{h_j+h_{j+1}} \left(\frac{h_j}{h_{j+1}} u(x_{j+1}) + \left(\frac{h_{j+1}}{h_j} - \frac{h_j}{h_{j+1}} \right) u(x_j) - \frac{h_{j+1}}{h_j} u(x_{j-1}) \right) \quad (3.28c)$$

$$- \frac{h_j h_{j+1}}{6(h_j+h_{j+1})} (h_{j+1} u'''(\xi_+) + h_j u'''(\xi_-)). \quad (3.28d)$$

Since $\frac{h_j h_{j+1}}{h_j+h_{j+1}} < h$, we conclude that this scheme is second-order accurate, again assuming that $u(x)$ is three times continuously differentiable on the interval.

Similar techniques can be applied to approximate higher derivatives in one dimension. Writing out even more terms in the Taylor expansions for $u(x_{j\pm 1})$ (assuming $u(x)$ is smooth enough for this to be valid), we have

$$u(x_{j+1}) = u(x_j) + h_{j+1} u'(x_j) + h_{j+1}^2 \frac{u''(x_j)}{2} + h_{j+1}^3 \frac{u'''(x_j)}{6} + h_{j+1}^4 \frac{u''''(\xi_+)}{24}, \quad (3.29a)$$

$$u(x_{j-1}) = u(x_j) - h_j u'(x_j) + h_j^2 \frac{u''(x_j)}{2} - h_j^3 \frac{u'''(x_j)}{6} + h_j^4 \frac{u''''(\xi_-)}{24}. \quad (3.29b)$$

Multiplying the first equation by $\frac{h_j}{h_j+h_{j+1}}$ and the second by $\frac{h_{j+1}}{h_j+h_{j+1}}$ and adding gives

$$\frac{h_j}{h_j+h_{j+1}} (u(x_{j+1}) - u(x_j)) + \frac{h_{j+1}}{h_j+h_{j+1}} (u(x_{j-1}) - u(x_j)) \quad (3.30a)$$

$$= \left(\frac{h_j h_{j+1}^2}{2(h_j+h_{j+1})} + \frac{h_j^2 h_{j+1}}{2(h_j+h_{j+1})} \right) u''(x_j) \quad (3.30b)$$

$$+ \left(\frac{h_j h_{j+1}^3}{6(h_j+h_{j+1})} - \frac{h_j^3 h_{j+1}}{6(h_j+h_{j+1})} \right) u'''(x_j) \quad (3.30c)$$

$$+ \left(\frac{h_j h_{j+1}^4}{24(h_j+h_{j+1})} u''''(\xi_+) + \frac{h_j^4 h_{j+1}}{24(h_j+h_{j+1})} u''''(\xi_-) \right). \quad (3.30d)$$

Note that we can simplify the coefficient on $u''(x_j)$ to $h_j h_{j+1}/2$, allowing us to rearrange the equation as

$$u''(x_j) = \frac{2}{h_{j+1}(h_j+h_{j+1})} (u(x_{j+1}) - u(x_j)) + \frac{2}{h_j(h_j+h_{j+1})} (u(x_{j-1}) - u(x_j)) \quad (3.31)$$

$$- \frac{h_{j+1} - h_j}{3} u'''(x_j) - \frac{1}{12(h_j+h_{j+1})} (h_{j+1}^3 u''''(\xi_+) + h_j^3 u''''(\xi_-)).$$

If we make no further assumptions on the mesh, it is clear that $|h_{j+1} - h_j| < h$, so this provides a first-order accurate approximation of $u''(x_j)$ using the function values at points x_{j-1} , x_j , and x_{j+1} , assuming f is four times continuously differentiable so that the original Taylor expansions are valid. In fact, the same conclusion holds if f is only three times continuously differentiable,

with a slightly different expression for the final truncation error term. This result is also valid on a uniform mesh, where $h_{j+1} - h_j = 0$, yielding the second-order accurate approximation discussed in Section 2.4.

To use this to solve for a finite-difference approximation on a nonuniform mesh, we often introduce the quantity $\bar{h}_j = (h_j + h_{j+1})/2$ as the average of the mesh widths around point x_j . Then,

$$u''(x_j) \approx \frac{1}{h_{j+1}\bar{h}_j} (u(x_{j+1}) - u(x_j)) + \frac{1}{h_j\bar{h}_j} (u(x_{j-1}) - u(x_j)) \quad (3.32a)$$

$$= \frac{1}{\bar{h}_j} \left(\frac{u(x_{j+1})}{h_j} - \left(\frac{1}{h_j} + \frac{1}{h_{j+1}} \right) u(x_j) + \frac{u(x_{j+1})}{h_{j+1}} \right). \quad (3.32b)$$

This form of the approximation is often most convenient to deal with, particularly since we can multiply through by a factor of $\bar{h}_j$ to get a modified, but symmetric, linear system to solve when we use this in a finite-difference approximation of the solution to a PDE.

Example 3.4: 1D boundary-value problem on nonuniform mesh

Consider approximating the solution to $-u''(x) + u(x) = \sin(3\pi x)$ for $0 < x < 1$ with $u(0) = u(1) = 0$ on a nonuniform mesh $\{x_i\}_{i=0}^{n_x}$. Let $u_i \approx u(x_i)$ be the finite-difference approximation to the solution and let $f_i = \sin(3\pi x_i)$ be the values of the right-hand side evaluated at the mesh points.

At each interior mesh point, the differential equation requires

$$-u''(x_i) + u(x_i) = \sin(3\pi x_i), \quad (3.33)$$

so we generate the equations of the finite-difference approximation, giving

$$\frac{1}{\bar{h}_i} \left(\frac{-u_{i-1}}{h_i} + \left(\frac{1}{h_i} + \frac{1}{h_{i+1}} \right) u_i - \frac{u_{i+1}}{h_{i+1}} \right) + u_i = f_i \text{ for } 1 \leq i \leq n_x - 1. \quad (3.34)$$

While this is perfectly valid, as a practical point, we often multiply through by $\bar{h}_i$ so that the resulting linear system is symmetric, giving a modified form of the equation as

$$\frac{-u_{i-1}}{h_i} + \left(\frac{1}{h_i} + \frac{1}{h_{i+1}} \right) u_i - \frac{u_{i+1}}{h_{i+1}} + \bar{h}_i u_i = \bar{h}_i f_i. \quad (3.35)$$

Note that, while this is often a more convenient form to work with at the discrete level, the convergence and stability analysis in subsequent sections must be performed on the original (unscaled) linear system. At the boundary points, we use the Dirichlet conditions to conclude that $u_0 = u_{n_x} = 0$.

With rescaling and homogeneous Dirichlet boundary conditions, the resulting matrix and linear system are

$$A = \begin{bmatrix} \frac{1}{h_1} + \frac{1}{h_2} + \bar{h}_1 & -\frac{1}{h_2} & & & \\ -\frac{1}{h_2} & \frac{1}{h_2} + \frac{1}{h_3} + \bar{h}_2 & & & \\ & \ddots & \ddots & \ddots & \\ & -\frac{1}{h_{n_x-2}} & \frac{1}{h_{n_x-2}} + \frac{1}{h_{n_x-1}} + \bar{h}_{n_x-2} & -\frac{1}{h_{n_x-1}} & \\ & & -\frac{1}{h_{n_x-1}} & \frac{1}{h_{n_x-1}} + \frac{1}{h_{n_x}} + \bar{h}_{n_x-1} & \end{bmatrix} \quad (3.36)$$

and

$$A \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n_x-2} \\ u_{n_x-1} \end{bmatrix} = \begin{bmatrix} \bar{h}_1 f_1 \\ \bar{h}_2 f_2 \\ \vdots \\ \bar{h}_{n_x-2} f_{n_x-2} \\ \bar{h}_{n_x-1} f_{n_x-1} \end{bmatrix}. \quad (3.37)$$

An example solution is given in Figure 3.2.1 for the case of $n_x = 25$ randomly selected mesh points.

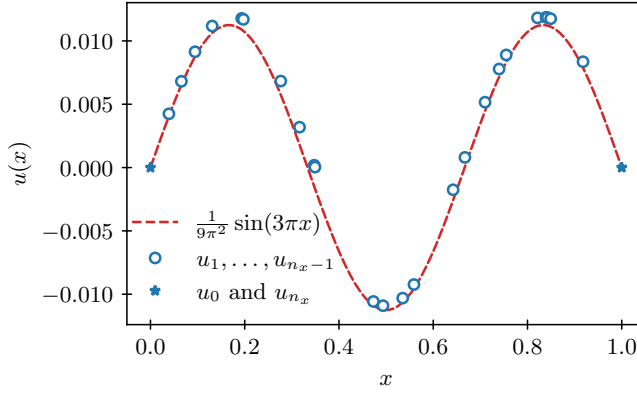


Figure 3.2.1. Example with $n_x = 25$ randomly selected mesh points for the case of $f(x) = \sin(3\pi x)$.

Importantly, the matrix is now symmetric and diagonally dominant, but no longer Toeplitz. In the case of nonhomogeneous Dirichlet boundary conditions, with prescribed values of u_0 and u_{n_x} , the right-hand side would be slightly modified to account for the terms in the first and last columns of the original linear system.

On nonuniform rectangular grids in 2D, we again make use of the Kronecker product construction, as detailed in Section 3.1. The 1D operators, $A_{1d}^{h_x}$ and $A_{1d}^{h_y}$, are constructed for the nonuniform discretization on meshes Ω^{h_x} and Ω^{h_y} , which are not the uniform-grid operators described in the previous section. Then, the Kronecker products are used to compute the discretization matrix $A = A_{1d}^{h_x} \otimes I^{n_y} + I^{n_x} \otimes A_{1d}^{h_y}$.

3.3 - Convergence theory for finite-difference methods

Objectives

The main goals of this section are to establish a theoretical framework for when the finite-difference method provides a good approximate solution of the PDE, quantifying what it really means to be a “good approximation.” You will develop concepts such as consistency and stability of a discretization and see how these relate to convergence of the approximation to the solution of the PDE.

Having established how to compute the finite-difference approximation to the solution of a differential equation, we now ask the obvious question: how does that approximation relate to the true solution of the differential equation? Answering this question requires developing some mathematical language to talk about finite-difference approximations in a rigorous way.

We first ask a simpler question: how do we measure the difference between the finite-difference approximation and the true solution of a differential equation? There are several ways to

consider this, but we adopt a *collocation* viewpoint; that is, we seek to control the error in the approximation at the mesh points. To do this, let $\{x_j\}_{j=0}^{n_x}$ be a one-dimensional mesh for the interval $[a, b]$ with $h := \frac{b-a}{n_x}$, $u(x)$ be the solution to a given differential equation posed on the interval $[a, b]$, and u_j be a finite-difference approximation to $u(x_j)$ for $0 \leq j \leq n_x$. We define the pointwise error in the finite-difference approximation to be $e_j = u(x_j) - u_j$ for $0 \leq j \leq n_x$. We consider three vectors of length $n_x + 1$, namely, the (pointwise) exact solution, the numerical approximation, and the pointwise error:

$$\mathbf{U} = \begin{bmatrix} u(x_0) \\ u(x_1) \\ \vdots \\ u(x_{n_x}) \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{n_x} \end{bmatrix}, \quad \mathbf{e} = \begin{bmatrix} e_0 \\ e_1 \\ \vdots \\ e_{n_x} \end{bmatrix} = \mathbf{U} - \mathbf{u}. \quad (3.38)$$

A natural way to measure the size of the vector of pointwise errors, $\mathbf{e}$, is through a vector norm, such as the scaled Euclidean (ℓ_2) norm,

$$\|\mathbf{e}\|_2 = \left(h \sum_{j=0}^{n_x} e_j^2 \right)^{1/2}, \quad (3.39)$$

or the maximum (ℓ_∞) norm,

$$\|\mathbf{e}\|_\infty = \max_{0 \leq j \leq n_x} |e_j|. \quad (3.40)$$

The scaling on the Euclidean norm is in place to make it more natural to compare norms of vectors of different lengths (corresponding to finite-difference approximations on meshes with different numbers of points). To facilitate this, the Euclidean vector norm is scaled such that it scales in the same way as a finite-sum approximation of a continuous L^2 norm of a function on the computational domain. For example, for the exact solution $u(x)$ on domain $\Omega = [a, b]$, we have

$$\|u(x)\|_{L^2(\Omega)} = \sqrt{\int_a^b u(x)^2 dx} \approx \left(\frac{b-a}{n_x} \sum_{j=0}^{n_x-1} u(x_j)^2 \right)^{1/2}. \quad (3.41)$$

This scaling will be important for error norms when considering the accuracy of the finite-difference approximation and the order of convergence of the finite-difference method as the grid is refined.

As always, vector norms induce norms on matrices (the natural operators on vectors) through the relation that

$$\|A\| = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|}, \quad (3.42)$$

where A is any matrix, and the supremum is taken over all nonzero vectors of appropriate dimension. For the two norms above, standard calculations simplify the general form of the matrix norm to the specific forms

$$\|A\|_2 = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_2}{\|\mathbf{x}\|_2} = (\lambda_{\max}(A^T A))^{1/2}, \quad (3.43a)$$

$$\|A\|_\infty = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \max_{0 \leq i \leq n_x} \sum_{j=0}^{n_x} |a_{i,j}|, \quad (3.43b)$$

where $\lambda_{\max}(A^T A)$ is the largest (in magnitude) eigenvalue of $A^T A$. Note that when A is symmetric, then the largest singular value of the matrix equals the absolute value of the eigenvalue

of largest size:

$$\sigma_{\max}(A) = (\lambda_{\max}(A^T A))^{1/2} = |\lambda|_{\max}(A). \quad (3.44)$$

A simple consequence of the definition of the matrix norm is the useful inequality that $\|Av\| \leq \|A\|\|v\|$ for any vector v .

With this machinery in place, we now consider the convergence theory of finite-difference methods for elliptic PDEs written in matrix form. As we saw in Section 3.1, there is a matrix, A , representing the finite-difference discretization scheme on a grid, and the finite-difference approximation, u , is the solution to the linear system $Au = f$, where the vector f encodes the right-hand side of the differential equation as well as any necessary information from the boundary conditions. Writing $u = A^{-1}f$ (assuming, for now, that A is invertible), we can derive the relation

$$e = U - u = U - A^{-1}f = A^{-1}(AU - f). \quad (3.45)$$

This then gives the bound

$$\|e\| = \|A^{-1}(AU - f)\| \leq \|A^{-1}\| \|AU - f\|. \quad (3.46)$$

Here, we have assumed that we are using the same norm (e.g., the ℓ_2 norm) on both sides of the inequality, although this approach can be generalized to allow different norms. The quantity $AU - f$ in equation (3.46) plays a central role in the convergence analysis of finite-difference methods and is known as the *truncation error* of the finite-difference scheme.

Definition 3.5: Truncation error (cf. [152, Sec. 2.5])

Given a finite-difference scheme described as $Au = f$, the truncation error of the scheme is the vector $\tau = AU - f$, where U is the vector of true solution values at the mesh points.

By its definition, the truncation error is the difference between the finite-difference scheme applied to the true solution, AU , and the finite-difference right-hand side. A useful consequence of this is the relation that

$$Ae = A(U - u) = AU - f = \tau, \quad (3.47)$$

or $e = A^{-1}\tau$. The main idea of our approach to convergence theory is to reconsider equation (3.46), written as $\|e\| \leq \|A^{-1}\| \|\tau\|$, and to bound $\|e\|$ by separately bounding $\|A^{-1}\|$ and $\|\tau\|$ as the mesh is refined.

To define convergence of a finite-difference scheme, consider a family of approximations that are generated by a family of quasi-uniform meshes (see Definition 2.13 in Section 2.3) defined on an interval $[a, b]$. Given a mesh Ω^h in this family, let A^h be the finite-difference discretization matrix for a given differential equation discretized on Ω^h , and let U^h , u^h , e^h , and τ^h be the associated vectors of the true solution, the finite-difference approximation, the error, and the truncation error at the nodes of Ω^h . Our goal is to measure the dependence of $\|e^h\|$, $\|(A^h)^{-1}\|$, and $\|\tau^h\|$ on h .

Definition 3.6: Consistency, stability, and convergence [152, Ch. 2]

Given a quasi-uniform family of meshes, $\{\Omega^h\}$, as defined in Definition 2.13, we say that a discretization scheme is

- *consistent with order μ if $\|\tau^h\| = \mathcal{O}(h^\mu)$ as $h \rightarrow 0$,*
- *stable if $\exists C_{stab} > 0$ such that $\|(A^h)^{-1}\| < C_{stab}$ for all $0 < h \leq 1$, and*
- *convergent with order μ if $\|e^h\| = \mathcal{O}(h^\mu)$ as $h \rightarrow 0$.*

Each definition depends on the vector norm in which we choose to measure convergence. To account for this, when stating one of the above properties of the scheme, it is necessary to specify for which norm the property is established and to establish the properties independently for each norm of interest.

Theorem 3.7: Lax convergence theorem (cf. [152, Ch. 2])

Given a quasi-uniform family of meshes, $\{\Omega^h\}$, if a discretization scheme is consistent with order μ and stable, then it is convergent with order μ .

Proof. Consider a discretization scheme that is consistent and stable for the family $\{\Omega^h\}$. Stability implies that

$$\|e^h\| \leq \left\| (A^h)^{-1} \right\| \|\tau^h\| \leq C_{\text{stab}} \|\tau^h\| \text{ for all } 0 < h \leq 1. \quad (3.48)$$

With this, consistency implies that $\|e^h\| = \mathcal{O}(h^\mu)$ as $h \rightarrow 0$, since $C_{\text{stab}} \|\tau^h\| = \mathcal{O}(h^\mu)$ as $h \rightarrow 0$. $\square$

3.3.1 ■ Consistency

Consistency is generally straightforward to check and follows, in many cases, directly from the Taylor-series derivation of the scheme itself. In Section 3.1, we used formulas that express the derivative of a smooth function in terms of its finite-difference approximation plus an error term. Here, we reverse that and write the finite-difference approximation of a smooth function as the difference between the true derivative (used to eliminate the right-hand side of the differential equation) and that same error term. Thus, if we have error terms bounded in terms of h for all derivatives appearing in the scheme, we automatically satisfy the property of consistency. This is shown in the following example.

Example 3.8: Consistency for a 1D boundary-value problem

Consider $\Omega = [0, 1]$, with a family of uniform meshes, $\{\Omega^h\}$, and $h = 1/n_x$. Approximate the solution of $-u''(x) + u(x) = f(x)$ for $0 < x < 1$ with $u(0) = u(1) = 0$ by the finite-difference scheme of Example 3.1:

$$\frac{1}{h^2} (-u_{j+1} + 2u_j - u_{j-1}) + u_j = f_j \text{ for } 1 \leq j \leq n_x - 1. \quad (3.49)$$

In the notation of this section, we write the matrix from Example 3.1 as

$$A^h = \text{tridiag} \left(-\frac{1}{h^2}, \frac{2}{h^2} + 1, -\frac{1}{h^2} \right). \quad (3.50)$$

From the analysis of Section 3.1, we know that

$$\frac{1}{h^2} (-u(x_{j+1}) + 2u(x_j) - u(x_{j-1})) = -u''(x_j) - \frac{h^2}{24} (u''''(\xi_+) + u''''(\xi_-)) \quad (3.51)$$

so long as $u(x)$ is four times continuously differentiable. From this, we have that

$$\tau_j = f(x_j) - \frac{1}{h^2} (-u(x_{j+1}) + 2u(x_j) - u(x_{j-1})) - u(x_j) \quad (3.52a)$$

$$= f(x_j) + u''(x_j) + \frac{h^2}{24} (u''''(\xi_+) + u''''(\xi_-)) - u(x_j) \quad (3.52b)$$

$$= \frac{h^2}{24} (u''''(\xi_+) + u''''(\xi_-)), \quad (3.52c)$$

since $u(x)$ is the solution of the differential equation $-u''(x) + u(x) = f(x)$. Assuming u is four times continuously differentiable on the closed interval, $[0, 1]$, we have that there exists a constant, M , such that $|u''''(x)| \leq M$ for all $0 \leq x \leq 1$, giving the bound that

$$|\tau_j| \leq \frac{M}{12} h^2. \quad (3.53)$$

From here, we conclude that both $\|\tau^h\|_\infty \leq Mh^2/12$ and $\|\tau^h\|_2 \leq Mh^2/12$, where it is important to note again that we have measured $\|\cdot\|_2$ in the *scaled* ℓ_2 norm. This gives consistency with order $\mu = 2$ in both the maximum and Euclidean norms.

Aside from the details of the definition of a family of quasi-uniform meshes as in Definition 2.13, none of the development above is specific to one-dimensional differential equations. One complication in going to multidimensional problems is in the lack of a natural ordering of points on a tensor-product mesh, which we address by consistently adopting the convention of a *lexicographic* ordering, as described in Section 3.1. Once this order is established, we can again interpret a finite-difference approximation of a differential operator on the tensor-product mesh as a matrix, A , acting on the approximation u and, consequently, analyze convergence of the finite-difference approximation using the tools of stability and consistency in the same manner as was described here in 1D. However, as we will see next for stability, we may in each case need to consider the specific details of the matrices, A , that go with such approximations.

3.3.2 ■ Stability

In contrast to consistency, establishing stability requires additional effort beyond the derivation of the finite-difference scheme using Taylor's theorem. We consider two common techniques that are used for proving stability, as well as possible approaches for when these techniques fail. Stability is most easily established by referring to properties of the discretization matrix; thus, we begin with a discussion of matrix properties.

As stated in Definition 3.6, stability of a discretization scheme on a quasi-uniform family of meshes is established by proving a mesh-independent bound on the so-called *stability constant*, C_{stab} , for which $\|(A^h)^{-1}\| < C_{\text{stab}}$ for all meshes Ω^h . Naturally, the specific techniques for doing this depend strongly on the vector norm used to induce the matrix norm in this inequality. The most common norms to consider are the Euclidean (ℓ_2) and maximum (ℓ_∞) norms, which yield the matrix norms given in equations (3.43a) and (3.43b).

When considering the ℓ_2 norm of a symmetric matrix, A , we have the added relations that $\|A\|_2 = |\lambda|_{\max}(A)$ and $\|A^{-1}\|_2 = (|\lambda|_{\min}(A))^{-1}$, where $|\lambda|_{\min}(A)$ and $|\lambda|_{\max}(A)$ are the minimum and maximum of the absolute values of the eigenvalues of A . When the discretization leads to symmetric discretization matrices, A^h , we can often establish stability using a classic theorem from linear algebra.

Theorem 3.9: Geršgorin's theorem [226, Thm. 1.11]

Let A be an $n \times n$ matrix. For $1 \leq i \leq n$, define

$$R_i = \sum_{j \neq i} |a_{i,j}| \quad (3.54)$$

and $D_i = \{z \in \mathbb{C} \mid |a_{i,i} - z| \leq R_i\}$. Then, every eigenvalue, λ , of A satisfies

$$\lambda \in \bigcup_{i=1}^n D_i. \quad (3.55)$$

Proof. Let λ be an eigenvalue of A and $\mathbf{x} \neq \mathbf{0}$ satisfy $A\mathbf{x} = \lambda\mathbf{x}$. Choose i so that $|x_i| = \max_j |x_j| > 0$. Since $A\mathbf{x} = \lambda\mathbf{x}$, we have that

$$\sum_{j=1}^n a_{i,j} x_j = \lambda x_i, \quad (3.56)$$

or

$$(\lambda - a_{i,i})x_i = \sum_{j \neq i} a_{i,j} x_j, \quad (3.57)$$

which implies that

$$|\lambda - a_{i,i}| \leq \sum_{j \neq i} |a_{i,j}| \left| \frac{x_j}{x_i} \right| \leq \sum_{j \neq i} |a_{i,j}|. \quad (3.58)$$

Thus $\lambda \in D_i$, so it is in the union of all such disks. $\square$

Example 3.10: Stability for a 1D boundary-value problem

Picking up from Examples 3.1 and 3.8 with a family of uniform meshes on $\Omega = [0, 1]$ and the discretization matrix

$$A^h = \text{tridiag} \left(-\frac{1}{h^2}, \frac{2}{h^2} + 1, -\frac{1}{h^2} \right), \quad (3.59)$$

we now look to apply Geršgorin's theorem to bound the minimal eigenvalue of A^h .

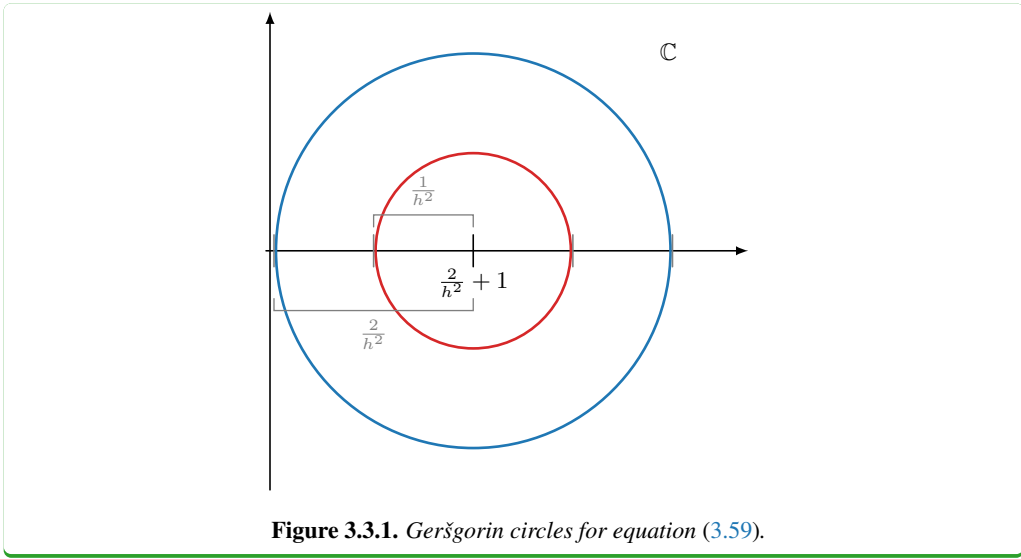
By direct computation, we have that

$$D_1 = D_{n_x} = \left\{ z \in \mathbb{C} \mid \left| z - \left(\frac{2}{h^2} + 1 \right) \right| \leq \frac{1}{h^2} \right\}, \quad (3.60a)$$

$$D_i = \left\{ z \in \mathbb{C} \mid \left| z - \left(\frac{2}{h^2} + 1 \right) \right| \leq \frac{2}{h^2} \right\} \text{ for } 2 \leq i \leq n_x - 2, \quad (3.60b)$$

as shown in Figure 3.3.1. Since A^h is symmetric, we also know that all of its eigenvalues are real valued, leading to the conclusion that every eigenvalue, λ , of A^h , satisfies $1 \leq \lambda \leq \frac{4}{h^2} + 1$.

From here, we conclude that $\lambda_{\min}(A^h) \geq 1$ for all h and, consequently, $\|(A^h)^{-1}\|_2 \leq 1$ for all meshes in the family Ω^h . Thus, the discretization is stable in the ℓ_2 norm. This, coupled with the conclusion from Example 3.8 that the discretization is consistent in this norm with order $\mu = 2$, implies (via Theorem 3.9) that the discretization is convergent in this norm with order $\mu = 2$.



Clearly, this technique is not limited to the matrices A^h from this example. In general, establishing stability in the ℓ_2 norm is straightforward using Geršgorin's theorem whenever the discretization matrices are symmetric and *diagonally dominant*.

Definition 3.11: Diagonal dominance (see also [194, Def. 4.5])

The $n \times n$ matrix A is (row) diagonally dominant if

$$|a_{i,i}| \geq \sum_{j \neq i} |a_{i,j}| \quad \text{for } 1 \leq i \leq n. \quad (3.61)$$

The $n \times n$ matrix A is strictly (row) diagonally dominant if $\exists \alpha > 0$ such that

$$|a_{i,i}| - \sum_{j \neq i} |a_{i,j}| \geq \alpha \quad \text{for } 1 \leq i \leq n. \quad (3.62)$$

The family of matrices, $\{A^h\}$, is uniformly strictly diagonally dominant if each matrix A^h is strictly diagonally dominant with a constant α that is independent of h .

Theorem 3.12: Stability in the ℓ_2 norm

If the discretization matrices, A^h , corresponding to a quasi-uniform family of meshes, $\{\Omega^h\}$, are all symmetric and uniformly strictly diagonally dominant, then the discretization is stable in the ℓ_2 norm.

Proof. Let α be a constant such that every matrix A^h is strictly diagonally dominant with respect to α . Since A^h is symmetric, it has real eigenvalues and, applying Geršgorin's theorem, strict diagonal dominance of A^h implies that $|\lambda|_{\min}(A^h) \geq \alpha$ because each Geršgorin circle is at least a distance α away from the origin of the complex plane. Thus, $\|(A^h)^{-1}\|_2 \leq \frac{1}{\alpha}$ for any h , establishing stability in the ℓ_2 norm. $\square$

When A^h is not symmetric, we cannot directly apply Geršgorin's theorem to A^h to prove stability but must, instead, apply it to $(A^h)^T A^h$ (or $A^h (A^h)^T$) to bound the minimum singular value of A^h (which equals $\|A^h\|_2$; see equation (3.43a)) from below. In this case, if $(A^h)^T A^h$ is uniformly strictly diagonally dominant, then the discretization is seen to be stable by the same argument as above.

One consequence of having matrix A^h be strictly diagonally dominant is that Geršgorin's theorem guarantees that all of its eigenvalues are strictly in the right half of the complex plane. This is equivalent to a useful property that we now define.

Definition 3.13: Positive-definite and negative-definite matrices [194, Sec. 1.11]

Matrix A is said to be *positive definite* if $\mathbf{u}^T A \mathbf{u} > 0$ for all $\mathbf{u} \neq \mathbf{0}$. If $\mathbf{u}^T A \mathbf{u} < 0$ for all $\mathbf{u} \neq \mathbf{0}$, we say that A is *negative definite*. If either of these hold without strict inequality, we say that A is (positive or negative) *semidefinite*.

Geršgorin's theorem is a natural tool for verifying positive (or negative) definiteness for symmetric matrices, for which positive definiteness is equivalent to having all positive eigenvalues.

A similar theorem can be applied to establish stability in the ℓ_∞ norm, without restrictions on the symmetry of A^h .

Theorem 3.14: Varah's theorem [225, Thm. 1]

Let A be an $n \times n$ matrix that is strictly diagonally dominant with respect to constant $\alpha > 0$. Then, $\|A^{-1}\|_\infty \leq \frac{1}{\alpha}$.

Proof. Since A is strictly diagonally dominant, by Geršgorin's theorem it is invertible. Thus, we can write

$$\|A^{-1}\|_\infty = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|A^{-1} \mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \sup_{\mathbf{y} \neq \mathbf{0}} \frac{\|\mathbf{y}\|_\infty}{\|A \mathbf{y}\|_\infty}. \quad (3.63)$$

Then, taking reciprocals of both sides, we have that $\frac{1}{\|A^{-1}\|_\infty} = \inf_{\mathbf{y} \neq \mathbf{0}} \frac{\|A \mathbf{y}\|_\infty}{\|\mathbf{y}\|_\infty}$. We can bound this using a similar argument as in the proof of Geršgorin's theorem, giving that

$$\inf_{\mathbf{y} \neq \mathbf{0}} \frac{\|A \mathbf{y}\|_\infty}{\|\mathbf{y}\|_\infty} \geq \alpha. \quad (3.64)$$

To show that $\inf_{\mathbf{y} \neq \mathbf{0}} \frac{\|A \mathbf{y}\|_\infty}{\|\mathbf{y}\|_\infty} \geq \alpha$, it is sufficient to show that $\|A \mathbf{y}\|_\infty \geq \alpha \|\mathbf{y}\|_\infty$ for all $\mathbf{y}$. Let $|y_i| = \|\mathbf{y}\|_\infty$. Then, $0 < \alpha \leq |a_{i,i}| - \sum_{j \neq i} |a_{i,j}|$ implies that

$$0 < \alpha |y_i| \leq |a_{i,i}| |y_i| - \sum_{j \neq i} |a_{i,j}| |y_i| \quad (3.65a)$$

$$\leq |a_{i,i}| |y_i| - \sum_{j \neq i} |a_{i,j}| |y_j| \quad (3.65b)$$

$$\leq |a_{i,i} y_i| - \left| \sum_{j \neq i} a_{i,j} y_j \right| \quad (3.65c)$$

$$\leq \left| \sum_j a_{i,j} y_j \right| \leq \max_i \left| \sum_j a_{i,j} y_j \right|, \quad (3.65d)$$

where the penultimate inequality follows from the reverse triangle inequality, $||a| - |b|| \leq |a + b|$. Thus, $\alpha \|y\|_\infty \leq \|Ay\|_\infty$ and $\|A^{-1}\|_\infty \leq \frac{1}{\alpha}$. $\square$

Corollary 3.15: Stability in the ℓ_∞ norm

If the discretization matrices, A^h , corresponding to a quasi-uniform family of meshes, Ω^h , are uniformly strictly diagonally dominant, then the discretization is stable in the ℓ_∞ norm.

When the discretization matrices are not uniformly strictly diagonally dominant, establishing stability can be more difficult. Here, we turn to directly establishing stability by finding explicit forms for $(A^h)^{-1}$ or its eigenvalues. These must, necessarily, be adapted for each problem considered, although there are a few general classes of problems for which this is possible. While the broad theory of which matrix algebras allow us to get the explicit forms that we need is beyond the scope of this book, the next example demonstrates the general trend: if we can find a suitable ansatz for the form of the eigenvectors, we can easily compute the eigenvalues and, consequently, examine the stability bound.

Stability for a nonstrictly diagonally dominant matrix (ℓ_2 norm)

Consider $\Omega = [0, 1]$ and a family of uniform meshes, $\{\Omega^h\}$, with $h = 1/n_x$, and approximate the solution of $-u''(x) = f(x)$ for $0 < x < 1$ with $u(0) = u(1) = 0$ by the finite-difference scheme

$$\frac{1}{h^2} (-u_{j+1} + 2u_j - u_{j-1}) = f_j \text{ for } 1 \leq j \leq n_x - 1. \quad (3.66)$$

Here, we arrive at the discretization matrices

$$A^h = \text{tridiag} \left(-\frac{1}{h^2}, \frac{2}{h^2}, -\frac{1}{h^2} \right). \quad (3.67)$$

Note that A^h is diagonally dominant, but not strictly so: for all but the first and last rows, the sum of the absolute values of the off-diagonals matches the diagonal entry. Thus, the arguments centered on Geršgorin's theorem fail to establish stability.

For a symmetric, tridiagonal Toeplitz (i.e., diagonal-constant) matrix, such as A^h , we can make use of the ansatz that the $n_x - 1$ eigenvectors of A^h have the form $v_j^{(k)} = \sin\left(\frac{jk\pi}{n_x}\right)$ for $1 \leq k \leq n_x - 1$. From this, we compute the eigenvalues, λ_k , directly from the expression $A^h v^{(k)} = \lambda_k v^{(k)}$. First, we have

$$\left(A^h v^{(k)} \right)_j = \frac{1}{h^2} \left(-\sin\left(\frac{(j-1)k\pi}{n_x}\right) + 2\sin\left(\frac{jk\pi}{n_x}\right) - \sin\left(\frac{(j+1)k\pi}{n_x}\right) \right). \quad (3.68)$$

Applying an angle sum identity yields

$$\sin\left(\frac{(j-1)k\pi}{n_x}\right) + \sin\left(\frac{(j+1)k\pi}{n_x}\right) = 2\sin\left(\frac{jk\pi}{n_x}\right) \cos\left(\frac{k\pi}{n_x}\right). \quad (3.69)$$

This allows us to directly calculate that

$$\left(A^h v^{(k)} \right)_j = \frac{1}{h^2} \left(2\sin\left(\frac{jk\pi}{n_x}\right) - 2\sin\left(\frac{jk\pi}{n_x}\right) \cos\left(\frac{k\pi}{n_x}\right) \right) \quad (3.70a)$$

$$= \frac{2}{h^2} \left(1 - \cos\left(\frac{k\pi}{n_x}\right) \right) \sin\left(\frac{jk\pi}{n_x}\right) \quad (3.70b)$$

$$= \frac{4}{h^2} \sin^2\left(\frac{k\pi}{2n_x}\right) v_j^{(k)}. \quad (3.70c)$$

Note that this calculation works for any j : it follows directly if $2 \leq j \leq n_x - 2$, while, for $j = 1$ or $j = n_x - 1$, it is validated because the definition of $\mathbf{v}^{(k)}$ is trivially extended to include $v_0^{(k)} = v_{n_x}^{(k)} = 0$.

Thus, $\mathbf{v}^{(k)}$ given above is an eigenvector of A^h , with eigenvalue $\lambda_k = \frac{4}{h^2} \sin^2 \left(\frac{k\pi}{2n_x} \right)$. Since $\sin^2(\theta)$ is an increasing function of θ for $0 \leq \theta \leq \frac{\pi}{2}$, we have that $\lambda_{\min}(A^h) = \frac{4}{h^2} \sin^2 \left(\frac{\pi}{2n_x} \right)$. We now consider what happens to this eigenvalue as $n_x \rightarrow \infty$ and $h \rightarrow 0$, making use of the relation that $n_x = \frac{1}{h}$.

Writing $\lambda_{\min}(A^h) = \frac{4}{h^2} \sin^2 \left(\frac{\pi h}{2} \right)$ and applying L'Hôpital's rule to the fraction $\frac{\sin(\pi h/2)}{h}$, we find that $\lim_{h \rightarrow 0} \lambda_{\min}(A^h) = \pi^2$. This is interesting by itself, since it shows that the smallest eigenvalue of our discretization matrix converges to that of the differential eigenproblem $-u''(x) = \lambda u(x)$ with homogeneous Dirichlet boundary conditions on the interval $\Omega = [0, 1]$. For our purposes, though, it is primarily useful in that it allows us to bound $\lambda_{\min}(A^h)$ independently of h .

An extended exercise in differential calculus can be done to show that $\frac{d}{dh} (\lambda_{\min}(A^h)) < 0$ for $0 < h < 1$. This shows that $\lambda_{\min}(A^h)$ is maximized at $h = 0$ and minimized at $h = 1/2$ (the largest value of h that allows for a nontrivial discretization matrix). For $h = 1/2$, $\lambda_{\min}(A^h) = 8$, giving the bounds that

$$8 \leq \lambda_{\min}(A^h) \leq \pi^2 \text{ or} \quad (3.71a)$$

$$\frac{1}{\pi^2} \leq (\lambda_{\min}(A^h))^{-1} \leq \frac{1}{8}. \quad (3.71b)$$

Finally, we conclude that $\|(A^h)^{-1}\|_2 \leq \frac{1}{8}$ for all uniform meshes, Ω^h , and the discretization is stable in the ℓ_2 norm.

Similarly, establishing stability in the maximum norm is greatly complicated when the discretization matrices are not uniformly strictly diagonally dominant. While, in theory, we could use the explicit form of the eigenvalues and eigenvectors (as computed above) to express $(A^h)^{-1}$ in closed form, this is generally not very enlightening, since it involves complicated expressions for the individual entries in $(A^h)^{-1}$. Next, we continue the example and derive an explicit expression for the entries of $(A^h)^{-1}$ and, consequently, the stability bound in the maximum norm.

Stability for a nonstrictly diagonally dominant matrix (max norm)

Consider A^h defined as in equation (3.67) for a discretization of $-u''(x) = f(x)$ on a uniform mesh, Ω^h , of $\Omega = [0, 1]$. Since A^h is symmetric, we calculate $\|(A^h)^{-1}\|_\infty$ directly from its definition in equation (3.43b), or by finding the maximum column sum instead of the maximum row sum. Thus, we look to express $(A^h)^{-1}$ columnwise, finding that the j th column of $(A^h)^{-1}$, which we denote by $\mathbf{w}^{(j)}$, must satisfy $A^h \mathbf{w}^{(j)} = \mathbf{e}^{(j)}$, where $\mathbf{e}^{(j)}$ is the j th canonical unit vector, with

$$e_i^{(j)} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{otherwise.} \end{cases} \quad (3.72)$$

Thus, writing $A^h \mathbf{w}^{(j)} = \mathbf{e}^{(j)}$ componentwise, we have

$$\frac{1}{h^2} \left(-w_{j-1}^{(j)} + 2w_j^{(j)} - w_{j+1}^{(j)} \right) = 1, \quad (3.73a)$$

$$\frac{1}{h^2} \left(-w_{i-1}^{(j)} + 2w_i^{(j)} - w_{i+1}^{(j)} \right) = 0 \text{ for } i \neq j, \quad (3.73b)$$

where we either truncate these expressions at the two boundaries or extend the definition of $\mathbf{w}^{(j)}$ by $w_0^{(j)} = w_{n_x}^{(j)} = 0$.

Away from the endpoints, we note that $-w_{i-1}^{(j)} + 2w_i^{(j)} - w_{i+1}^{(j)} = 0$ for any linear function in mesh index, $w_i^{(j)} = c_1 + c_2 i$. This leads to the “inspired guess” that we take $w^{(j)}$ to be a linear function to both the right and left of node j , with the value at node j scaled so that the first equation holds. That is, we write

$$w_i^{(j)} = \begin{cases} c \frac{i}{j} & \text{if } i \leq j, \\ c \frac{1-i/n_x}{1-j/n_x} & \text{if } i \geq j \end{cases} \quad (3.74)$$

and choose the constant, c , so that $-w_{j-1}^{(j)} + 2w_j^{(j)} - w_{j+1}^{(j)} = h^2$. Direct calculation then gives $c = \frac{(n_x - j)j}{n_x^3}$ and

$$w_i^{(j)} = \begin{cases} \frac{1}{n_x} \left(1 - \frac{j}{n_x}\right) \left(\frac{i}{n_x}\right) & \text{if } i \leq j, \\ \frac{1}{n_x} \left(1 - \frac{i}{n_x}\right) \left(\frac{j}{n_x}\right) & \text{if } i \geq j. \end{cases} \quad (3.75)$$

Both the ansatz for $w_i^{(j)}$ and its final form satisfy $w_0^{(j)} = w_{n_x}^{(j)} = 0$, so no further adaptation is needed to satisfy the boundary conditions, and each column of $(A^h)^{-1}$ is written as in equation (3.75).

Finally, we turn to bounding $\max_{1 \leq j \leq n_x-1} \sum_{i=1}^{n_x-1} |w_i^{(j)}|$, for which we notice that $0 \leq w_i^{(j)} \leq \frac{1}{n_x}$ for all i, j . Thus, $\sum_{i=1}^{n_x-1} |w_i^{(j)}| \leq 1$ for all j , and $\|(A^h)^{-1}\|_\infty \leq 1$, independent of h , and the discretization is stable in the maximum norm.

The inescapable conclusion from the examples in this section is that establishing stability when A^h is not uniformly strictly diagonally dominant is significantly more difficult than when it is. Indeed, it is also clear that the techniques above are difficult to generalize. For certain classes of matrix algebras (e.g., Toeplitz matrices and their subset of circulant matrices), we can adopt fixed forms for the eigenvectors and, consequently, compute eigenvalues of A^h . This is also applicable to discretizations in two or more dimensions, where the discretization matrices can be written (as discussed above) as sums of Kronecker products of one-dimensional discretization matrices and standard linear-algebra results on eigenvalues and eigenvectors of tensor products can sometimes be used to bound the eigenvalues of A^h . For matrices outside of these classes, however, it is quite difficult to find closed-form expressions for eigenvalues or eigenvectors and, consequently, stability analysis is quite difficult. Similarly, the technique used above for establishing stability in the maximum norm is difficult to extend even to simple finite-difference operators in two dimensions.

Interestingly, it is often easier to establish lack of stability of a finite-difference discretization scheme than to prove stability. For the ℓ_2 norm, we note that if A^h is symmetric, then

$$\lambda_{\min}(A^h) \leq \frac{\mathbf{x}^T A^h \mathbf{x}}{\mathbf{x}^T \mathbf{x}} \text{ for any } \mathbf{x} \neq \mathbf{0}. \quad (3.76)$$

So, if A^h is symmetric, and we can find a vector, $\mathbf{x}$, such that $\mathbf{x}^T A^h \mathbf{x} = f(h) \mathbf{x}^T \mathbf{x}$, then an immediate consequence is that $\|(A^h)^{-1}\|_2 \geq 1/f(h)$. As a result, if $f(h) \rightarrow 0$ as $h \rightarrow 0$, then the discretization cannot possibly be stable, since no upper bound, $\|(A^h)^{-1}\|_2 < C_{\text{stab}}$, can possibly exist. The same is true for maximum norm stability of a symmetric family of discretization matrices, since

$$\|(A^h)^{-1}\|_\infty^{-1} = \inf_{\mathbf{y} \neq \mathbf{0}} \frac{\|A^h \mathbf{y}\|_\infty}{\|\mathbf{y}\|_\infty} \leq \lambda_{\min}(A^h). \quad (3.77)$$

Thus, if we can show that $\lambda_{\min}(A^h) \rightarrow 0$ as $h \rightarrow 0$, the discretization cannot possibly be stable in the maximum norm or the ℓ_2 norm.

3.4 ■ Treatment of Neumann and periodic boundary conditions

Objectives

In this section, you will see how to extend the finite-difference technique to deal with more complicated sets of boundary conditions. Two issues arise, one regarding the accuracy of the discretization and the other the solvability of the resulting linear systems. You will learn standard techniques to address both of these, along with general principles that can be applied in many situations.

While Dirichlet boundary conditions are easiest to handle from the perspective of deriving a finite-difference discretization, it is important to recognize that we rarely get to choose the boundary conditions to apply in a real-world setting. One common alternative boundary condition is the Neumann boundary condition, prescribing $u'(a)$ at endpoint a in one dimension and $\frac{\partial u}{\partial \mathbf{n}} = \nabla u \cdot \mathbf{n}$ along a (smooth) segment of the boundary of the domain with outward normal vector $\mathbf{n}$ in more dimensions. Such boundary conditions naturally model flux in a physical system, with homogeneous Neumann boundary conditions used to model impermeable boundaries and nonhomogeneous conditions used to model sources or sinks of material along the boundary. A natural first approach to discretizing such a condition is to use the standard differencing schemes of Section 2.4.

For example, considering a second-order equation posed on the interval $[0, 1]$, with mesh $\{x_j\}_{j=0}^{n_x}$, the boundary condition $u'(0) = \alpha$ can be directly approximated as

$$\frac{1}{x_1 - x_0} (u_1 - u_0) = \alpha, \quad (3.78)$$

where u_0 and u_1 are the approximations to $u(x_0) = u(0)$ and $u(x_1)$, respectively. Coupling this equation with n_x equations associated with the nodes x_1 through x_{n_x} (coming from discretization of the differential operator at the interior nodes and a corresponding discretization of the boundary condition at $x_{n_x} = 1$) leads to a system of $n_x + 1$ equations in $n_x + 1$ unknowns. The one-sided approximation to $u'(x_0)$ above, however, is inherently a first-order approximation. This may be fine if the discretization at the interior nodes is also only first-order accurate, but it can lead to degradation of accuracy at all mesh points (often called “pollution” of the numerical solution) if the scheme is otherwise second order. Consider, for example, the case of a uniform mesh discretization of $-u''(x) + u(x) = f(x)$ using the standard second-order approximation of the second derivative on a uniform mesh. Here, even though we consider a first-order discretization at only one or two boundary points, numerical experiments show that the overall solution is only first-order accurate.

To attain second order, we can look for a second-order approximation to the first derivative at the endpoint. On a uniform mesh of $[0, 1]$, this is not difficult to do, as Taylor’s theorem gives us

$$u(0) = u(0), \quad (3.79a)$$

$$u(h) = u(0) + u'(0)h + u''(0)\frac{h^2}{2} + u'''(0)\frac{h^3}{6} + \cdots, \quad (3.79b)$$

$$u(2h) = u(0) + u'(0)2h + u''(0)\frac{4h^2}{2} + u'''(0)\frac{8h^3}{6} + \cdots. \quad (3.79c)$$

To cancel out the second-derivative terms, we simply need to consider

$$4u(h) - u(2h) = 3u(0) + u'(0)2h - u'''(0)\frac{4h^3}{6} + \cdots. \quad (3.80)$$

Rearranging gives us

$$u'(0) = \frac{1}{2h} (-3u(0) + 4u(h) - u(2h)) + u'''(0)\frac{2h^2}{6} + \dots \quad (3.81)$$

Thus, a second-order approximation to the boundary condition $u'(0) = \alpha$ on the uniform mesh with mesh spacing h is given by

$$\frac{-3u_0 + 4u_1 - u_2}{2h} = \alpha. \quad (3.82)$$

Coupling this with the second-order discretization of $-u''(x) + u(x) = f(x)$ on this mesh and the eliminated Dirichlet boundary condition $u(1) = \beta$ yields the linear system

$$\begin{bmatrix} -\frac{3}{2h} & \frac{2}{h} & -\frac{1}{2h} & 0 & \dots & \dots \\ -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} & 0 & \dots & \dots \\ & -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} & \dots & \dots \\ & & \ddots & \ddots & \ddots & \ddots \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} \\ & & & & -\frac{1}{h^2} & \frac{2}{h^2} + 1 \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{n_x-2} \\ u_{n_x-1} \end{bmatrix} = \begin{bmatrix} \alpha \\ f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_{n_x-2} \\ f_{n_x-1} + \frac{\beta}{h^2} \end{bmatrix}. \quad (3.83)$$

Solution of this linear system now *does* lead to a second-order approximation to the solution of the differential equation; however, the matrix is clearly no longer symmetric or strictly diagonally dominant. Thus, proving that the system is stable and second-order accurate is difficult but can be avoided by using a better technique.

To gain second-order accuracy and keep symmetry, we introduce another technique known as using “ghost points.” For this approach, we suppose that the solution, $u(x)$, extends smoothly outside of the domain, so that we can make the usual centered approximation to the first derivative:

$$u'(0) \approx \frac{u(h) - u(-h)}{2h} \approx \frac{u_1 - u_{-1}}{2h}. \quad (3.84)$$

This introduces the fictitious point, x_{-1} , and approximation, $u_{-1} \approx u(x_{-1})$, into the finite-difference scheme, resulting in $n_x + 2$ degrees of freedom with only $n_x + 1$ equations. To overcome this mismatch, we introduce an additional equation by enforcing the finite-difference approximation to the differential equation at the boundary point. For the example above, this would yield the additional equation that

$$\frac{-u_{-1} + 2u_0 - u_1}{h^2} + u_0 = f_0. \quad (3.85)$$

In general, the differential equation is not expected to hold at the boundary; however, this can be interpreted as an approximation at an interior point that is infinitesimally close to the boundary. Simply adding this equation again yields a well-posed system whose solution is a second-order approximation to the solution of the differential equation but still does not restore symmetry (or strict diagonal dominance) in the resulting matrix.

The second step in using a ghost-point approximation is to eliminate the extra variable, u_{-1} , using the boundary condition. Here, for the boundary condition $u'(0) = \alpha$, we can solve the centered-difference approximation to this to give $u_{-1} = u_1 - 2h\alpha$ and substitute this into the

discretization of the differential equation at the boundary point, yielding

$$\frac{-(u_1 - 2h\alpha) + 2u_0 - u_1}{h^2} + u_0 = f_0, \quad (3.86a)$$

$$\frac{2u_0 - 2u_1}{h^2} + u_0 = f_0 - \frac{2\alpha}{h}, \quad (3.86b)$$

$$\text{or, equivalently, } \frac{u_0 - u_1}{h^2} + \frac{1}{2}u_0 = \frac{1}{2}f_0 - \frac{\alpha}{h}. \quad (3.86c)$$

Incorporating this into the linear system of equations results in

$$\begin{bmatrix} \frac{1}{h^2} + \frac{1}{2} & -\frac{1}{h^2} & & & & \\ -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} & 0 & \cdots & \\ & -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} & 0 & \cdots \\ & & \ddots & \ddots & \ddots & \\ & & -\frac{1}{h^2} & \frac{2}{h^2} + 1 & -\frac{1}{h^2} & \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} + 1 & \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{n_x-2} \\ u_{n_x-1} \end{bmatrix} = \begin{bmatrix} \frac{1}{2}f_0 - \frac{\alpha}{h} \\ f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_{n_x-2} \\ f_{n_x-1} + \frac{\beta}{h^2} \end{bmatrix}, \quad (3.87)$$

where, as in equation (3.83), we incorporate the eliminated Dirichlet boundary condition $u(1) = \beta$ into the linear system. This is seen to be symmetric and strictly diagonally dominant, establishing the desired stability results in both the Euclidean and maximum norm. The same ghost-point approach can easily be extended to the second example considered earlier, of $-u''(x) = f(x)$, but now requires modification of the direct approaches to proving stability demonstrated in Section 3.3.2.

More difficulty arises in the case where the equation is $-u''(x) = f(x)$ but both boundary conditions are of Neumann type: $u'(0) = \alpha$ and $u'(1) = \beta$. Here, we follow the same prescription as above, now at both endpoints, writing

$$\frac{u_1 - u_{-1}}{2h} = \alpha, \quad \frac{u_{n_x+1} - u_{n_x-1}}{2h} = \beta, \quad (3.88)$$

solving these for the ghost-point values of u_{-1} and u_{n_x+1} , which are then substituted into the discretized differential equation at the endpoints to yield

$$\frac{-2u_1 + 2u_0}{h^2} = f_0 - \frac{2\alpha}{h}, \quad \frac{2u_{n_x} - 2u_{n_x-1}}{h^2} = f_{n_x} + \frac{2\beta}{h}. \quad (3.89)$$

Dividing by two in both terms gives the symmetric system

$$\begin{bmatrix} \frac{1}{h^2} & -\frac{1}{h^2} & & & & \\ -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} & 0 & \cdots & \\ & -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} & 0 & \cdots \\ & & \ddots & \ddots & \ddots & \\ & & -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} & \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} & \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{n_x-1} \\ u_{n_x} \end{bmatrix} = \begin{bmatrix} \frac{1}{2}f_0 - \frac{\alpha}{h} \\ f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_{n_x-1} \\ \frac{1}{2}f_{n_x} + \frac{\beta}{h} \end{bmatrix}. \quad (3.90)$$

While it is straightforward to arrive at this point, we now must confront the fact that the discretization matrix above is singular, seeing that

$$\begin{bmatrix} \frac{1}{h^2} & -\frac{1}{h^2} & & & & \\ -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} & 0 & \cdots & \\ & -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} & 0 & \cdots \\ & & & \ddots & \ddots & \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} \\ & & & & -\frac{1}{h^2} & \frac{1}{h^2} \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ \vdots \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}. \quad (3.91)$$

While this appears quite problematic at first, it is, in fact, a reflection of a problem with the continuum differential equation. From $-u''(x) = f(x)$, we can integrate both sides to arrive at

$$-\int_0^1 f(x) dx = \int_0^1 u''(x) dx = u'(1) - u'(0) = \beta - \alpha. \quad (3.92)$$

Thus, unless $-\int_0^1 f(x) dx = \beta - \alpha$, the continuum differential equation is ill posed. Furthermore, if this condition is satisfied, then the problem has a nonunique solution, since if $u(x)$ satisfies the differential equation and boundary conditions, then so does $u(x) + c$ for any constant c . This is, in fact, a reflection of the well-known Fredholm alternative from the theory of differential equations: since the homogeneous system $-u''(x) = 0$ with boundary conditions $u'(0) = u'(1) = 0$ has a nontrivial solution (in the form of any constant function), the nonhomogeneous equation that we are considering cannot have a unique solution.

The singularity of the discretization matrix reflects this at the discrete level. We can easily show that the nullspace of this matrix is the span of the constant vector, which says that the discrete problem $A^h \mathbf{u}^h = \mathbf{f}^h$ is solvable if and only if $\mathbf{f}^h \cdot \mathbf{1} = 0$, where $\mathbf{1}$ is the vector whose entries are all unity. Computing this, we see that

$$\mathbf{f}^h \cdot \mathbf{1} = \frac{1}{2}f_0 - \frac{\alpha}{h} + f_1 + f_2 + \cdots + f_{n_x-1} + f_{n_x} + \frac{\beta}{h} \quad (3.93a)$$

$$= \frac{1}{h}(\beta - \alpha) + \sum_{j=0}^{n_x-1} \frac{1}{2}(f_j + f_{j+1}). \quad (3.93b)$$

Thus, $\mathbf{f}^h \cdot \mathbf{1} = 0$ when $\beta - \alpha = -\frac{h}{2} \sum_{j=0}^{n_x-1} (f_j + f_{j+1})$. We recognize the right-hand side of this expression as the trapezoidal rule approximation of $-\int_0^1 f(x) dx$, so this is simply the discrete analogue of the continuum condition. When this condition is satisfied, then the discretization has a solution that is unique up to a constant shift, again just as in the continuum.

While this understanding is helpful, it leads us to a practical problem. Supposing the discrete compatibility condition is satisfied, how do we find a solution of the resulting linear system? First, we need to address the nonuniqueness problem: if $\mathbf{u}^h$ satisfies $A^h \mathbf{u}^h = \mathbf{f}^h$, then $A^h (\mathbf{u}^h + c\mathbf{1}) = \mathbf{f}^h$ for any constant c . Which of these solutions should we choose? Two common approaches arise:

1. Often, we choose to impose a compatibility condition on the continuum solution, that $\int_0^1 u(x) dx = 0$ (or another fixed constant). In this case, we can find any solution $\mathbf{v}^h$ to the discretized system, $A^h \mathbf{v}^h = \mathbf{f}^h$, then take $\mathbf{u}^h = \mathbf{v}^h - \frac{\mathbf{1}^T \mathbf{v}^h}{\mathbf{1}^T \mathbf{1}} \mathbf{1}$, so that $\mathbf{u}^h \cdot \mathbf{1} = 0$. If another normalization for $\mathbf{u}^h$ is desired, this can be shifted by any constant multiple of the vector $\mathbf{1}$. Alternatively, a vector that takes value one at all interior points and values of

$1/2$ at the two endpoints can be used, analogously to the trapezoidal rule approximation of the integral discussed above. For any of these, we can solve the singular linear system for any solution, $\mathbf{v}^h$, then postprocess to find the desired solution, $\mathbf{u}^h$. Iterative methods have been used successfully here. We could also directly impose that $\mathbf{1}^T \mathbf{u}^h = 0$ by adding a Lagrange multiplier to enforce the constraint, which leads to a well-posed (but no longer positive-definite) linear system.

2. A second approach is to consider fixing the solution at one node on the mesh, for example, to add the constraint that $u_0 = 0$. With this, we can either modify the linear system to remove u_0 from the calculation (leading to a well-posed linear system for the remaining n_x values of $\mathbf{u}^h$) or again find any solution $\mathbf{v}^h$ and postprocess to get $\mathbf{u}^h = \mathbf{v}^h - v_0 \mathbf{1}$.

The same problems arise with periodic boundary conditions, such as for $-u''(x) = f(x)$ for $0 < x < 1$, with $u(0) = u(1)$ and $u'(0) = u'(1)$. Here, standard discretization plus the correspondence that $u_0 = u_{n_x}$ leads to the linear system

$$\begin{bmatrix} \frac{2}{h^2} & -\frac{1}{h^2} & & & & -\frac{1}{h^2} \\ -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} & 0 & \cdots & \\ & -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} & 0 & \cdots \\ & & \ddots & \ddots & \ddots & \\ & & & -\frac{1}{h^2} & \frac{2}{h^2} & -\frac{1}{h^2} \\ -\frac{1}{h^2} & & & -\frac{1}{h^2} & \frac{2}{h^2} & \frac{2}{h^2} \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{n_x-2} \\ u_{n_x-1} \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_{n_x-2} \\ f_{n_x-1} \end{bmatrix}. \quad (3.94)$$

Again, the constant vector is in the nullspace of A^h , and requiring that the right-hand side be orthogonal to the nullspace leads to the condition that $\sum_{j=0}^{n_x-1} f_j = 0$. This, in fact, is the same trapezoidal rule integration of $f(x)$ as above, just adjusted to the periodicity of $f(x)$ in order to match the required properties of $u(x)$. Thus, the same approaches as outlined above can be used here.

In two (or more) dimensions, we can generalize this approach to Neumann boundary conditions. Considering, for example, the Neumann problem posed on the unit square, with $-\Delta u = f(x, y)$ for $x, y \in (0, 1) \times (0, 1)$, with $u_y(x, 0) = u_y(x, 1) = 0$ for $0 < x < 1$ and $u_x(0, y) = u_x(1, y) = 0$ for $0 < y < 1$, discretized on a uniform tensor-product mesh with mesh widths $h_x = 1/n_x$ and $h_y = 1/n_y$, we easily arrive at the usual stencil for the interior mesh points, with $1 \leq i \leq n_x - 1$ and $1 \leq j \leq n_y - 1$, as

$$\frac{1}{h_x^2} (-u_{i+1,j} + 2u_{i,j} - u_{i-1,j}) + \frac{1}{h_y^2} (-u_{i,j+1} + 2u_{i,j} - u_{i,j-1}) = f_{i,j}. \quad (3.95)$$

Now, we adjust just one term at the boundaries. For example, at $i = 0$, $1 \leq j \leq n_y - 1$, we have

$$\frac{1}{h_x^2} (-u_{1,j} + 2u_{0,j} - u_{-1,j}) + \frac{1}{h_y^2} (-u_{0,j+1} + 2u_{0,j} - u_{0,j-1}) = f_{0,j}. \quad (3.96)$$

Using the boundary condition and the ghost point, we write

$$\frac{u_{1,j} - u_{-1,j}}{2h_x} = 0, \quad (3.97)$$

which simplifies (in the case of homogeneous Neumann conditions) to $u_{-1,j} = u_{1,j}$, giving

$$\frac{1}{h_x^2} (-2u_{1,j} + 2u_{0,j}) + \frac{1}{h_y^2} (-u_{0,j+1} + 2u_{0,j} - u_{0,j-1}) = f_{0,j}. \quad (3.98)$$

The same approach can be used along all four edges of the domain, and two ghost points are needed at each corner. The resulting discrete linear system can be rescaled, if needed, to regain symmetry by noting that the same scaling needs to be applied to all rows corresponding to degrees of freedom along the boundary.

3.5 ■ Structure-preserving finite-difference methods

Objectives

In this section, we explore variants of the standard finite-difference technique that are suitable for more complex problems—e.g., applications where continuous structure in the differential equation should be preserved at the discrete level. First, you will look at how to preserve symmetry for variable-coefficient diffusion equations in a standard finite-difference setting. Then, you will see the closely related finite-volume method in the setting of elliptic PDEs.

As a motivating example, we consider a variant of the one-dimensional diffusion equation, where the diffusion coefficient varies in space,

$$-(K(x)u'(x))' = f(x) \text{ for } 0 < x < 1, \quad (3.99)$$

supplemented with Dirichlet boundary conditions at $x = 0$ and $x = 1$. A natural temptation is to expand the derivative term using the product rule, writing

$$-K(x)u''(x) - K'(x)u'(x) = f(x) \text{ for } 0 < x < 1. \quad (3.100)$$

On a uniform mesh with spacing $h = 1/n_x$, using centered differences to approximate both first- and second-derivative terms, this leads to the discretization at mesh point x_j as

$$\begin{aligned} -K(x_j)u''(x_j) - K'(x_j)u'(x_j) &\approx K(x_j) \frac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} \\ &\quad + \frac{K(x_{j+1}) - K(x_{j-1}))}{2h} \left(\frac{u_{j+1} - u_{j-1}}{2h} \right) = f_j. \end{aligned} \quad (3.101)$$

Rewriting, we have the discrete equations as

$$\begin{aligned} \frac{1}{h^2} \left(- \left(K(x_j) + \frac{K(x_{j+1}) - K(x_{j-1}))}{4} \right) u_{j-1} \right. \\ \left. + 2K(x_j)u_j - \left(K(x_j) + \frac{K(x_{j-1}) - K(x_{j+1}))}{4} \right) u_{j+1} \right) = f_j \end{aligned} \quad (3.102)$$

for $1 \leq j \leq n_x - 1$. Under the reasonable assumption that nodal values of $K(x)$ are available (and, possibly, modified if nodal values of $K'(x)$ are available), this gives a second-order truncation error in the approximation of the original differential equation. However, it must be noted that the resulting approximation yields a nonsymmetric matrix, even though the original differential equation is self-adjoint.

An alternative approach is to view the second-order differential equation as pair of first-order equations, separated as

$$q(x) = -K(x)u'(x), \quad (3.103a)$$

$$q'(x) = f(x) \quad (3.103b)$$

for $0 < x < 1$. If $f(x)$ is known at the nodes, a natural centered-difference approximation to the second equation is, simply,

$$\frac{q_{j+\frac{1}{2}} - q_{j-\frac{1}{2}}}{h} = f_j \quad (3.104)$$

for $1 \leq j \leq n_x - 1$, where $q_{j\pm\frac{1}{2}}$ represents the values of $q(x)$ at the midpoints of the mesh, $x_{j\pm\frac{1}{2}} = (x_j + x_{j\pm 1})/2$. Using centered differences again on the first equation, we can write the flux terms as

$$q(x_{j+\frac{1}{2}}) \approx -K(x_{j+\frac{1}{2}}) \frac{u_{j+1} - u_j}{h} \text{ and } q(x_{j-\frac{1}{2}}) \approx -K(x_{j-\frac{1}{2}}) \frac{u_j - u_{j-1}}{h}. \quad (3.105)$$

Combining equations (3.104) and (3.105) yields

$$\frac{1}{h^2} \left(-K(x_{j-\frac{1}{2}})u_{j-1} + \left(K(x_{j-\frac{1}{2}}) + K(x_{j+\frac{1}{2}}) \right) u_j - K(x_{j+\frac{1}{2}})u_{j+1} \right) = f_j. \quad (3.106)$$

Though this approach now requires midpoint values of $K(x)$, the resulting discretization naturally preserves the symmetry present in the continuum differential equation.

While this seems like a simple trick applicable in only this setting, it is the starting point for a wide class of methods known as “structure-preserving” or “mimetic” finite-difference methods, where we seek discretizations that preserve standard identities from calculus at the discrete level. The main idea is that differences of nodal values are naturally located at the midpoints of the mesh, giving two discrete derivative operators, one mapping from nodes to midpoints (used to define $q_{j\pm\frac{1}{2}}$ above) and another from midpoints back to nodes. On tensor-product meshes in two dimensions, this leads to the *staggered* location of the two components of the gradient, with partial derivatives with respect to x naturally located at the midpoints of edges between two adjacent nodes with the same y -coordinate, and partial derivatives with respect to y naturally located at the midpoints of edges between two adjacent nodes with the same x -coordinate; see Figure 3.5.1. At first, this may seem inconvenient, but this structure is key to achieving stable discretizations of many coupled systems of differential equations, as it leads to natural definitions of the divergence operator at the nodes and the curl operator at the cell centers.

Generalizing the above approach to the two-dimensional analogue of the equation,

$$-\nabla \cdot (K(x, y) \nabla u(x, y)) = f(x, y), \quad (3.107)$$

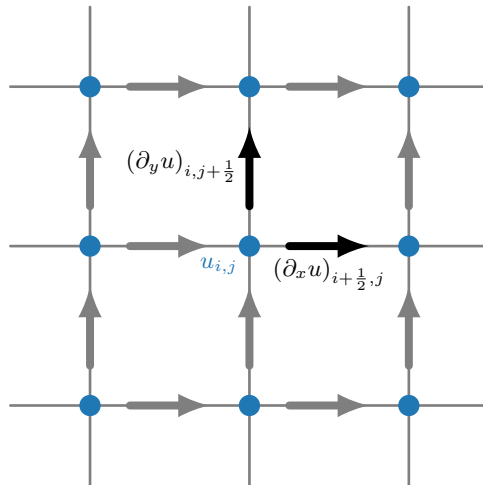


Figure 3.5.1. Mimetic location of partial derivatives (indicated by arrows) for a uniform mesh in 2D.

on a tensor product of uniform meshes yields the discretization

$$\begin{aligned} & \frac{1}{h_x^2} \left(-K_{i-\frac{1}{2},j} u_{i-1,j} + \left(K_{i-\frac{1}{2},j} + K_{i+\frac{1}{2},j} \right) u_{i,j} - K_{i+\frac{1}{2},j} u_{i+1,j} \right) \\ & + \frac{1}{h_y^2} \left(-K_{i,j-\frac{1}{2}} u_{i,j-1} + \left(K_{i,j-\frac{1}{2}} + K_{i,j+\frac{1}{2}} \right) u_{i,j} - K_{i,j+\frac{1}{2}} u_{i,j+1} \right) = f_{i,j}, \end{aligned} \quad (3.108)$$

where we have used $K_{\alpha,\beta} = K(x_\alpha, y_\beta)$ as shorthand. Again, this discretization is well defined so long as the values of $K(x, y)$ are available at the midpoints of the edges, $(x_{i\pm\frac{1}{2}}, y_j)$ and $(x_i, y_{j\pm\frac{1}{2}})$. When $K(x, y)$ is continuous at these points, this is naturally satisfied; however, many practical applications deal with discontinuous coefficients, and many times such discontinuities arise along the edges of the mesh. In such cases, a natural choice is to approximate the coefficient value using the average of the values within the mesh cells on either side of the edge; however, there is little connection between this intuition and the analysis of standard finite-difference schemes. In particular, discontinuities in the coefficient values generally lead to solutions that do not possess the smoothness assumed of standard finite-difference schemes. In order to properly understand both approximation and convergence in this setting, we turn to the finite-volume technique.

3.5.1 ■ Finite-volume methods for elliptic PDEs in 1D

The finite-volume methodology is a well-developed family of approaches that has been applied to many problems in fluid dynamics, solid mechanics, electromagnetism, and more. While the philosophy behind these approaches is fundamentally different from that of finite differences, for simple problems in simple settings, they produce equivalent schemes. Here, and in Section 3.5.2, we develop the start of the finite-volume methodology, aiming at basic elliptic second-order time-steady differential equations, following the discussion in [95]. In Chapter 6, the methodology is developed in detail for the case of hyperbolic equations, where it is most commonly used.

One of the main ideas behind the finite-volume methodology is, as above, to switch from considering the equation in second-order form and, instead, view the equation in *conservative* form, writing $-u''(x) = f(x)$ for $0 < x < 1$ with $u(0) = u(1) = 0$ as

$$q(x) = -u'(x), \quad (3.109a)$$

$$q'(x) = f(x). \quad (3.109b)$$

In general, we view this as two equations, one prescribing the *flux*, $q(x)$, in terms of $u(x)$, and the second enforcing that the divergence of this flux is given by $f(x)$. In one dimension, the divergence operator coincides with the usual derivative operator, but the use of this conservative/divergence form is key to the generalizations in two and three dimensions. In one dimension, the fundamental theorem of calculus will be used to express the integral of the derivative of $q(x)$ over an interval in terms of the values of $u(x)$ at the endpoints; in two or three dimensions, this will be replaced by the divergence theorem, interchanging a volume integral of the divergence of $q(x)$ with a surface integral of its outward normal component.

Within the finite-volume methodology, it is convenient to slightly generalize our mesh notation from above, introducing two “types” of points, with integer indices to represent our approximation points, x_j , for $0 < j < n_x$, and half-integer indices to represent *cell boundaries*, so that $x_{j-1/2} < x_j < x_{j+1/2}$. This also introduces new notation for mesh spacing, which is summarized in Definition 3.16. Our finite-volume approximation consists of a function value, u_j , that represents $u(x)$ over the cell $x_{j-1/2} < x < x_{j+1/2}$. There are several ways to view u_j , with the most common being either as a pointwise approximation of $u(x_j)$ or as an approximation to the mean of $u(x)$ over the cell, $\frac{1}{h_j} \int_{x_{j-1/2}}^{x_{j+1/2}} u(x) dx$.

Definition 3.16: Finite-volume mesh in 1D [95, Def. 5.1]

A mesh of the interval $[a, b]$ is given by partitioning it into $n_x - 1$ nonoverlapping cells. We denote by x_j for $j = 1, \dots, n_x - 1$, a point in the interior of each cell. The boundaries are then defined as $x_{j \pm 1/2}$. For convenience, we choose $x_0 = x_{1/2}$ and $x_{n_x} = x_{n_x - 1/2}$ such that the partitioning is

$$a = x_0 = x_{1/2} < x_1 < x_{3/2} < \dots < x_{n_x - 3/2} < x_{n_x - 1} < x_{n_x - 1/2} = x_{n_x} = b, \quad (3.110)$$

with

$$h_j = x_{j+1/2} - x_{j-1/2} \quad \text{for } 1 \leq j \leq n_x - 1, \quad (3.111a)$$

$$h_{j+1/2} = x_{j+1} - x_j \quad \text{for } 0 \leq j \leq n_x - 1, \quad (3.111b)$$

$$h_j^- = x_j - x_{j-1/2} \quad \text{for } 1 \leq j \leq n_x - 1, \quad (3.111c)$$

$$h_j^+ = x_{j+1/2} - x_j \quad \text{for } 1 \leq j \leq n_x - 1, \quad (3.111d)$$

as in Figure 3.5.2. For such a mesh, we take $h = \max_{1 \leq j \leq n_x - 1} h_j$.

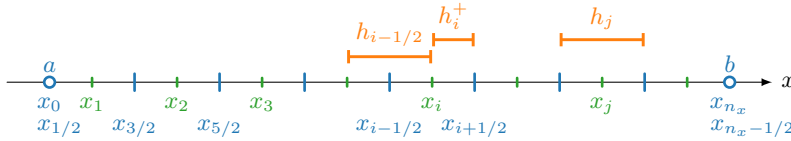


Figure 3.5.2. A mesh of the interval $[a, b]$.

To generate the discrete equations that determine u_j , we start by integrating $q'(x) = f(x)$ over the cell $x_{j-1/2} < x < x_{j+1/2}$:

$$\int_{x_{j-1/2}}^{x_{j+1/2}} q'(x) dx = \int_{x_{j-1/2}}^{x_{j+1/2}} f(x) dx, \quad (3.112a)$$

$$q(x_{j+1/2}) - q(x_{j-1/2}) = \int_{x_{j-1/2}}^{x_{j+1/2}} f(x) dx, \quad (3.112b)$$

or

$$-u'(x_{j+1/2}) + u'(x_{j-1/2}) = \int_{x_{j-1/2}}^{x_{j+1/2}} f(x) dx. \quad (3.112c)$$

The *cell-centered* finite-volume scheme then emerges by introducing the approximation $q(x_{j+1/2}) \approx -\frac{1}{h_{j+1/2}}(u_{j+1} - u_j)$ and writing $f_j = \frac{1}{h_j} \int_{x_{j-1/2}}^{x_{j+1/2}} f(x) dx$, giving

$$q_{j+1/2} - q_{j-1/2} = h_j f_j \quad \text{for } 1 \leq j \leq n_x - 1, \quad (3.113a)$$

$$q_{j+1/2} = -\frac{1}{h_{j+1/2}}(u_{j+1} - u_j) \quad \text{for } 1 \leq j \leq n_x - 2, \quad (3.113b)$$

$$q_{1/2} = -\frac{u_1}{h_{1/2}}, \quad (3.113c)$$

$$q_{n_x-1/2} = \frac{u_{n_x-1}}{h_{n_x-1/2}}, \quad (3.113d)$$

where the equations for $q_{1/2}$ and $q_{n_x-1/2}$ take into account both the homogeneous boundary conditions $u_0 = u_{n_x} = 0$ and any changes in mesh spacing near the boundaries. Furthermore,

we can combine the definitions of the fluxes, $q_{j+1/2}$, to construct a linear system just for the values, u_j , as

$$-\frac{u_{j+1} - u_j}{h_{j+1/2}} + \frac{u_j - u_{j-1}}{h_{j-1/2}} = h_j f_j \text{ for } 1 \leq j \leq n_x - 1, \quad (3.114a)$$

$$u_0 = u_{n_x} = 0. \quad (3.114b)$$

In the case of a uniform mesh, where $h_j = h_{j+1/2}$ for all j , this is equivalent to a scaled version of the standard finite-difference discretization.

Considering the consistency of the scheme, we notice that the equation

$$-u'(x_{j+1/2}) + u'(x_{j-1/2}) = \int_{x_{j-1/2}}^{x_{j+1/2}} f(x) dx \quad (3.115)$$

is *exact*: it holds for the true solution of the differential equation. Thus, for consistency, we only need to check that $-\frac{1}{h_{j+1/2}}(u_{j+1} - u_j)$ is a consistent approximation of $-u'(x_{j+1/2})$. However, this depends on how we interpret the approximation u_j . If it is taken as the approximation to the pointwise value $u(x_j)$, then this is a consistent approximation if $u \in C^2([a, b])$, since

$$u(x_{j+1}) = u(x_{j+1/2}) + h_{j+1}^- u'(x_{j+1/2}) + \frac{(h_{j+1}^-)^2}{2} u''(x_{j+1/2}) + \dots, \quad (3.116a)$$

$$u(x_j) = u(x_{j+1/2}) - h_j^+ u'(x_{j+1/2}) + \frac{(h_j^+)^2}{2} u''(x_{j+1/2}) + \dots \quad (3.116b)$$

leads to

$$u(x_{j+1}) - u(x_j) = h_{j+1/2} u'(x_{j+1/2}) + \frac{1}{2} ((h_{j+1}^-)^2 - (h_j^+)^2) u''(x_{j+1/2}) + \dots \quad (3.117)$$

A similar argument shows that this is also a consistent discretization if we interpret u_j as an approximation to the average value of $u(x)$ on the cell $x_{j-1/2} < x < x_{j+1/2}$.

Rather than proving convergence based on a combination of the above consistency plus a stability condition, we will directly prove convergence of the resulting finite-volume approximation, as the proof technique used is more suitable for generalization to the case of the variable-coefficient equations discussed above, including the case when the diffusion coefficient is discontinuous and the solution of the differential equation is not expected to be twice continuously differentiable or smoother on $[a, b]$.

Theorem 3.17: Convergence of the finite-volume method in 1D [95, Thm. 6.1]

Let $f \in C^0([0, 1])$ and $u \in C^2([0, 1])$ be the solution of $-u''(x) = f(x)$ for $0 < x < 1$ with $u(0) = u(1) = 0$. Let the mesh, $\{x_j\}_{j=0}^{n_x}$, satisfy the conditions in Definition 3.16. Then, there is a unique solution, $\mathbf{u} \in \mathbb{R}^{n_x-1}$, to the linear system in equation (3.114a). Furthermore, defining $e_j = u(x_j) - u_j$ for $1 \leq j \leq n_x - 1$, with $e_0 = e_{n_x} = 0$, there exists $C \geq 0$ such that

$$\sum_{j=0}^{n_x-1} \frac{(e_{j+1} - e_j)^2}{h_{j+1/2}} \leq C^2 h^2 \text{ and } |e_j| \leq Ch \text{ for } 1 \leq j \leq n_x - 1. \quad (3.118)$$

Proof. First, multiplying equation (3.114a) by u_j and summing over j yields

$$-\sum_{j=1}^{n_x-1} \frac{u_j(u_{j+1} - u_j)}{h_{j+1/2}} + \sum_{j=1}^{n_x-1} \frac{u_j(u_j - u_{j-1})}{h_{j-1/2}} \quad (3.119a)$$

$$= \frac{u_{n_x-1}^2}{h_{n_x-1/2}} - \sum_{j=1}^{n_x-2} \frac{u_j(u_{j+1} - u_j)}{h_{j+1/2}} + \sum_{j=2}^{n_x-1} \frac{u_j(u_j - u_{j-1})}{h_{j-1/2}} + \frac{u_1^2}{h_{1/2}} \quad (3.119b)$$

$$= \frac{u_{n_x-1}^2}{h_{n_x-1/2}} + \sum_{j=1}^{n_x-2} \frac{(u_{j+1} - u_j)^2}{h_{j+1/2}} + \frac{u_1^2}{h_{1/2}} \quad (3.119c)$$

$$= \sum_{j=1}^{n_x-1} h_j u_j f_j. \quad (3.119d)$$

Thus, the only solution when $f_j = 0$ for $1 \leq j \leq n_x - 1$ is $u_j = 0$ for $1 \leq j \leq n_x - 1$, which implies that the linear system in equation (3.114a) has a unique solution for any f_j , $1 \leq j \leq n_x - 1$.

Now, let $\bar{q}_{j+1/2} = -u'(x_{j+1/2})$ for $0 \leq j \leq n_x - 1$. Integrating $q'(x) = f(x)$ from $x_{j-1/2}$ to $x_{j+1/2}$ gives

$$\bar{q}_{j+1/2} - \bar{q}_{j-1/2} = h_j f_j \quad \text{for } 1 \leq j \leq n_x - 1, \quad (3.120a)$$

and, since

$$q_{j+1/2} - q_{j-1/2} = h_j f_j \quad \text{for } 1 \leq j \leq n_x - 1, \quad (3.120b)$$

we have

$$(\bar{q}_{j+1/2} - q_{j+1/2}) - (\bar{q}_{j-1/2} - q_{j-1/2}) = 0 \quad \text{for } 1 \leq j \leq n_x - 1. \quad (3.120c)$$

Next, write

$$d_{j+1/2} = -\frac{1}{h_{j+1/2}}(u(x_{j+1}) - u(x_j)) + u'(x_{j+1/2}) \quad \text{for } 0 \leq j \leq n_x \quad (3.121)$$

and note that

$$\bar{q}_{j+1/2} - q_{j+1/2} = -\frac{u(x_{j+1}) - u(x_j)}{h_{j+1/2}} + \frac{u_{j+1} - u_j}{h_{j+1/2}} - d_{j+1/2} \quad (3.122a)$$

$$= -\frac{e_{j+1} - e_j}{h_{j+1/2}} - d_{j+1/2}. \quad (3.122b)$$

Substituting this into equation (3.120c) gives

$$-\frac{e_{j+1} - e_j}{h_{j+1/2}} + \frac{e_j - e_{j-1}}{h_{j-1/2}} = d_{j+1/2} - d_{j-1/2} \quad \text{for } 1 \leq j \leq n_x - 1. \quad (3.123)$$

Now, multiplying by e_j and summing, as with u_j in the first step of the proof, gives

$$\sum_{j=0}^{n_x-1} \frac{(e_{j+1} - e_j)^2}{h_{j+1/2}} = \sum_{j=0}^{n_x-1} d_{j+1/2} e_j - \sum_{j=1}^{n_x} d_{j-1/2} e_j = -\sum_{j=0}^{n_x-1} d_{j+1/2} (e_{j+1} - e_j). \quad (3.124)$$

Finally, following the consistency calculation above, there is a $C > 0$ such that

$$|d_{j+1/2}| = \left| -\frac{1}{h_{j+1/2}}(u(x_{j+1}) - u(x_j)) + u'(x_{j+1/2}) \right| \leq Ch \quad (3.125)$$

for $0 \leq j \leq n_x - 1$. From this, we get the bound that

$$\sum_{j=0}^{n_x-1} \frac{(e_{j+1} - e_j)^2}{h_{j+1/2}} \leq Ch \sum_{j=0}^{n_x-1} |e_{j+1} - e_j|. \quad (3.126)$$

Considering the sum on the right-hand side, we use the Cauchy–Schwarz inequality to show that

$$\begin{aligned} \sum_{j=0}^{n_x-1} |e_{j+1} - e_j| &\leq \left(\sum_{j=0}^{n_x-1} \frac{(e_{j+1} - e_j)^2}{h_{j+1/2}} \right)^{1/2} \left(\sum_{j=0}^{n_x-1} h_{j+1/2} \right)^{1/2} \\ &= \left(\sum_{j=0}^{n_x-1} \frac{(e_{j+1} - e_j)^2}{h_{j+1/2}} \right)^{1/2}, \end{aligned} \quad (3.127)$$

as $\sum_{j=0}^{n_x-1} h_{j+1/2} = 1$. This, at last, allows us to derive first that

$$\sum_{j=0}^{n_x-1} \frac{(e_{j+1} - e_j)^2}{h_{j+1/2}} \leq C^2 h^2 \quad (3.128)$$

and, consequently, that

$$\sum_{j=0}^{n_x-1} |e_{j+1} - e_j| \leq Ch. \quad (3.129)$$

Since $e_j = \sum_{i=0}^{j-1} (e_{i+1} - e_i)$, we also have that

$$|e_j| \leq \sum_{i=0}^{j-1} |e_{i+1} - e_i| \leq Ch \text{ for } 1 \leq j \leq n_x - 1. \quad (3.130)$$

□

The same discretization and proof technique can be applied more generally. Again considering

$$-(K(x)u'(x))' = f(x) \text{ for } 0 < x < 1, \quad (3.131)$$

the natural definition of the flux is $q(x) = -K(x)u'(x)$, and integrating both sides of the equation from $x_{j-1/2}$ to $x_{j+1/2}$ leads to the relation that

$$-K(x_{j+1/2})u'(x_{j+1/2}) + K(x_{j-1/2})u'(x_{j-1/2}) = \int_{x_{j-1/2}}^{x_{j+1/2}} f(x) dx \text{ for } 1 \leq j \leq n_x - 1. \quad (3.132)$$

As above, we consider specifically the case where $K(x)$ is discontinuous, so that if $f \in L^2([0, 1])$, then the solution of this integrated form satisfies $u, u' \in L^2([0, 1])$, but we do not require that $u \in C^2([0, 1])$. Since the strong form of the equation requires $K(x)u'(x)$ to be continuous

(so that it can be differentiated), this integration step is always valid. However, discontinuities in $K(x)$ lead to difficulties in translating the integrated form of the differential equation into a discretization, since $K(x_{j\pm 1/2})$ may not be well defined. To account for this, we suppose that we know the value $x_{j+1/2}$ and wish to define a value of $K(x_{j+1/2})$ so that our discrete approximation also has a continuous flux approximation at $x_{j+1/2}$. Suppose, additionally, that $K(x)$ is constant on the two intervals adjacent to this point, with

$$K(x) = \begin{cases} K_j & \text{for } x_{j-1/2} < x < x_{j+1/2}, \\ K_{j+1} & \text{for } x_{j+1/2} < x < x_{j+3/2}. \end{cases} \quad (3.133)$$

Continuity in the flux approximation at $x_{j+1/2}$ can be expressed as requiring that

$$-K_j \frac{u_{j+1/2} - u_j}{h_j^+} = -K_{j+1} \frac{u_{j+1} - u_{j+1/2}}{h_{j+1}^-}, \quad (3.134)$$

where the left- and right-hand sides of this expression come from simple finite-difference approximations of the flux at $x_{j+1/2}$ from the left and right sides of the point, respectively. Solving this equation for the midpoint value gives

$$u_{j+1/2} = \frac{\frac{K_j}{h_j^+} u_j + \frac{K_{j+1}}{h_{j+1}^-} u_{j+1}}{\frac{K_j}{h_j^+} + \frac{K_{j+1}}{h_{j+1}^-}}. \quad (3.135)$$

Using this expression in either side of the flux approximation leads to

$$q_{j+1/2} = - \left(\frac{h_j^+}{K_j} + \frac{h_{j+1}^-}{K_{j+1}} \right)^{-1} (u_{j+1} - u_j), \quad (3.136)$$

and this expression can be used for $0 \leq j \leq n_x - 1$, coupled with boundary conditions that $u_0 = u_{n_x} = 0$.

With this, the applicability of the finite-volume scheme can be further expanded to advection-diffusion-reaction equations such as

$$- (K(x)u'(x))' + bu'(x) + cu(x) = f(x) \text{ for } 0 < x < 1 \quad (3.137)$$

for constant coefficients $b, c \in \mathbb{R}$ with $c \geq 0$. Again, integrating from $x_{j-1/2}$ to $x_{j+1/2}$ yields

$$\begin{aligned} -K(x_{j+1/2})u'(x_{j+1/2}) + K(x_{j-1/2})u'(x_{j-1/2}) + b(u(x_{j+1/2}) - u(x_{j-1/2})) \\ + c \int_{x_{j-1/2}}^{x_{j+1/2}} u(x) dx = \int_{x_{j-1/2}}^{x_{j+1/2}} f(x) dx \text{ for } 1 \leq j \leq n_x - 1. \end{aligned} \quad (3.138)$$

Whether we view the discrete value u_j as a pointwise approximation to $u(x_j)$ or as an approximation to its average value on the interval from $x_{j-1/2}$ to $x_{j+1/2}$, a reasonable approximation of the reaction term is

$$c \int_{x_{j-1/2}}^{x_{j+1/2}} u(x) dx \approx ch_j u_j, \quad (3.139)$$

noting that this is exact for the latter interpretation. For the advection terms, two approximations can be used. Commonly, we consider the first-order “upwind” approximation, where we take

$$bu(x_{j+1/2}) \approx \begin{cases} bu_j & \text{if } b \geq 0, \\ bu_{j+1} & \text{if } b \leq 0. \end{cases} \quad (3.140)$$

This can be shown to guarantee stability in this case and will be discussed more in Chapter 6. We could also use a centered averaging,

$$bu(x_{j+1/2}) \approx b \left(\frac{h_{j+1}^- u_j + h_j^+ u_{j+1}}{h_{j+1/2}} \right), \quad (3.141)$$

but this can be unstable if b is large compared to the mesh spacing and $K(x)$.

Pulling these together, we write a discretization of equation (3.138) with $b \geq 0$ as

$$q_{j+1/2} - q_{j-1/2} + b(u_j - u_{j-1}) + ch_j u_j = h_j f_j \quad (3.142)$$

for $1 \leq j \leq n_x - 1$ and

$$q_{j+1/2} = - \left(\frac{h_j^+}{K_j} + \frac{h_{j+1}^-}{K_{j+1}} \right)^{-1} (u_{j+1} - u_j) \quad (3.143)$$

for $1 \leq j \leq n_x - 2$, with suitable flux definitions for $q_{1/2}$ and $q_{n_x-1/2}$ that account for the boundary conditions, $u_0 = u_{n_x} = 0$. Theorem 3.17 can be extended to show that this scheme still gives a unique solution with $\mathcal{O}(h)$ approximation error; see [95, Thm. 7.1].

3.5.2 ■ Finite-volume methods for elliptic PDEs in 2D

The 1D ideas are naturally extended to polygonal domains in two spatial dimensions, polyhedral domains in three dimensions, and more general domains (by approximation of the domains). Here, we focus on the case of polygonal domains in two dimensions. As above, we consider a general advection-diffusion-reaction equation, but we write it in *divergence form* as

$$-\nabla \cdot \nabla u(x, y) + \nabla \cdot (\mathbf{b}(x, y)u(x, y)) + c(x, y)u(x, y) = f(x, y) \quad \text{for } (x, y) \in \Omega, \quad (3.144a)$$

$$u(x, y) = 0 \quad \text{for } (x, y) \in \partial\Omega. \quad (3.144b)$$

We consider the case of a constant diffusion coefficient (taken to be one, without loss of generality) for simplicity, although this could be adapted in the same manner as above in the general case. While we write the advection term in divergence form, we note that

$$\nabla \cdot (\mathbf{b}(x, y)u(x, y)) = \mathbf{b}(x, y) \cdot \nabla u(x, y) + (\nabla \cdot \mathbf{b}(x, y))u(x, y), \quad (3.145)$$

so that any advection-diffusion-reaction equation can be written in this divergence form, possibly by redefining the reaction coefficient. For regularity, we assume that the right-hand side $f(x, y) \in L^2(\Omega)$ and that the forcing velocity $\mathbf{b}(x, y) \in C^1(\Omega)$. For existence and uniqueness of the solution (in a weak sense to be defined), we further assume that both $c(x, y) \geq 0$ and $\nabla \cdot \mathbf{b}(x, y) \geq 0$ pointwise (which is stricter than necessary for the continuum equation (see Section 8.4) but matches conditions discussed below on the discretization). Under additional smoothness assumptions, further complications can be introduced, such as nonzero Dirichlet boundary conditions.

The basic idea of the finite-volume methodology is now to divide Ω into polygonal volumes and to integrate the divergence form of the equation using the divergence theorem. For any region $T \subset \Omega$, integrating equation (3.144a) over T and using the fact that $\int_T \nabla \cdot \mathbf{F}(x, y) dx = \int_{\partial T} \mathbf{F}(x, y) \cdot \mathbf{n} ds$, where $\mathbf{n}$ is the outward unit normal direction on ∂T , we have

$$- \int_{\partial T} (\nabla u) \cdot \mathbf{n} ds + \int_{\partial T} (\mathbf{b} \cdot \mathbf{n}) u ds + \int_T cu dx = \int_T f dx. \quad (3.146)$$

Let u_T be some approximation to the value of $u(x, y)$ on T , either pointwise (e.g., at the centroid of the polygon) or in an average sense, with $u_T \approx \frac{1}{A_T} \int_T u(x, y) d\mathbf{x}$, where A_T is the area of T . Then, natural approximations can be used for the area integrals, with

$$f_T = \frac{1}{A_T} \int_T f(x, y) d\mathbf{x}, \quad (3.147a)$$

$$c_T = \frac{1}{A_T} \int_T c(x, y) d\mathbf{x}, \quad (3.147b)$$

$$\int_T c(x, y) u(x, y) d\mathbf{x} \approx u_T \int_T c(x, y) d\mathbf{x} = A_T c_T u_T. \quad (3.147c)$$

For the line integrals, we now use the fact that T is polygonal to write these as sums over the faces (edges in 2D) of T , and using $\partial T = \cup_{i=1}^{n_f^T} f_i^T$, where we assume T has n_f^T faces, f_i^T . Then,

$$-\int_{\partial T} (\nabla u) \cdot \mathbf{n} ds + \int_{\partial T} (\mathbf{b} \cdot \mathbf{n}) u ds = -\sum_{i=1}^{n_f^T} \int_{f_i^T} (\nabla u) \cdot \mathbf{n} ds + \sum_{i=1}^{n_f^T} \int_{f_i^T} (\mathbf{b} \cdot \mathbf{n}) u ds. \quad (3.148)$$

For regular meshes, approximating the line integrals across faces is straightforward. Consider the example shown in Figure 3.5.3, where we consider the line integral along the face $f_{1,2}$ between the two cells, T_1 and T_2 , with the outward normal vector, $\mathbf{n}_{1,2}$, pointing from T_1 into T_2 . Given approximate values u_1 on T_1 and u_2 on T_2 , a natural approximation for $\int_{f_{1,2}} \nabla u \cdot \mathbf{n}_{1,2} ds$ is to approximate the normal derivative of u across this face by a simple finite difference to give a value at the midpoint,

$$\int_{f_{1,2}} \nabla u \cdot \mathbf{n}_{1,2} ds \approx |f_{1,2}| \frac{u_2 - u_1}{x_2 - x_1}, \quad (3.149)$$

where $|f_{1,2}|$ is simply the area (length in 2D) of face $f_{1,2}$. For the convection term, a similar (weighted) averaging of u_1 and u_2 is possible, although an upwind differencing is more common for stability, approximating

$$\int_{f_{1,2}} \mathbf{b} \cdot \mathbf{n}_{1,2} u ds \approx \begin{cases} |f_{1,2}| \mathbf{b}(\mathbf{x}_{1,2}) \cdot \mathbf{n}_{1,2} u_1 & \text{if } \mathbf{b}(\mathbf{x}_{1,2}) \cdot \mathbf{n}_{1,2} \geq 0, \\ |f_{1,2}| \mathbf{b}(\mathbf{x}_{1,2}) \cdot \mathbf{n}_{1,2} u_2 & \text{if } \mathbf{b}(\mathbf{x}_{1,2}) \cdot \mathbf{n}_{1,2} \leq 0, \end{cases} \quad (3.150)$$

where $\mathbf{x}_{1,2}$ is the midpoint of face $f_{1,2}$ (see Figure 3.5.3).

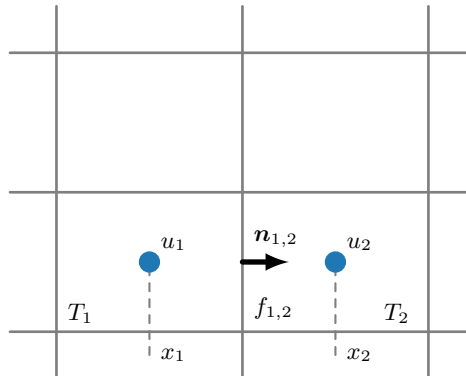


Figure 3.5.3. Finite-volume definition of face-based fluxes on a regular mesh in two dimensions.

While more complex, the approach above extends to meshes with less structure. Indeed, the basic approach remains the same: find approximate values for the integrands at the midpoint of the faces and sum these contributions weighted by face area (length in 2D). Two common settings are worth special attention, however. The first is the case of triangular meshes. In order for simple approximations to be effective, assumptions need to be made on the *regularity* of the meshes. One commonly used sufficient condition is that no triangle in the mesh have a vertex with an internal angle equal to or greater than $\pi/2$. This, however, is sometimes seen as being too restrictive, and a more general condition is the *strict Delaunay condition*, requiring that no triangle in the mesh be completely contained within the circumscribed circle of any other triangle in the mesh. This, of course, allows some triangles in the mesh to have angles greater than $\pi/2$, but only if the neighboring triangles have compatible shape. An example of this is shown in Figure 3.5.4, where the blue triangle has an angle greater than $\pi/2$, with neighboring elements in red that violate (left) and satisfy (right) the strict Delaunay condition.

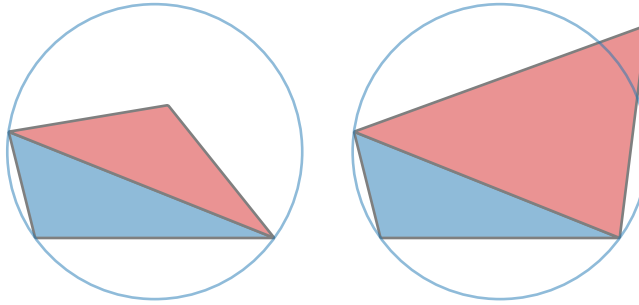


Figure 3.5.4. Example of the strict Delaunay condition on mesh elements: (left) violates the condition with the red triangle; (right) satisfies the condition.

The second common setting is that of a *Voronoi* mesh, where, given a set of “seed points,” $\{\mathbf{x}_i\}_{i=1}^{n_p} \subset \Omega$, we define

$$T_i = \{\mathbf{y} \in \Omega \mid |\mathbf{x}_i - \mathbf{y}| < |\mathbf{x}_j - \mathbf{y}| \forall j \neq i\} \text{ for } 1 \leq i \leq n_p. \quad (3.151)$$

The edges of a Voronoi mesh are, then, the lines equidistant from two seed points, $\mathbf{x}_i$ and $\mathbf{x}_j$, and the “nodes” of the mesh (which do not normally enter into the finite-volume scheme) are those points equidistant from more than two seed points—see Figure 3.5.5. Voronoi meshes have some inherent shape regularity due to their construction, although they may require some adaptation near the boundary of Ω .

In both cases, the key step in developing a finite-volume approach is the approximation of the face integrals in equation (3.148). Given two adjacent cells, T_1 and T_2 , either in a triangulation satisfying the strict Delaunay condition or in a Voronoi mesh, if $f_{1,2}$ is the face between them, then the same approximations of the line integrals over $f_{1,2}$ used in the uniform-grid case above can be used here, only replacing the one-dimensional distance $x_2 - x_1$ with the Euclidean distance between points $\mathbf{x}_1 \in T_1$ and $\mathbf{x}_2 \in T_2$. The choice of these points is, of course, fundamental to the accuracy of the scheme, and natural choices are the orthocenter for the triangular case (the point where the lines through each vertex and perpendicular to the opposite edge intersect) or the generating points for the Voronoi case. As above, these contributions are summed over all edges around a volume to determine the discrete equations.

Depending on assumptions about the mesh, the PDE, and its solution, different conclusions can be reached about the discrete solution. If the meshes are *admissible* (as they are in triangulations that satisfy the strict Delaunay condition or for Voronoi meshes), then the linear system has a unique solution under our original assumptions that $f \in L^2(\Omega)$, $\mathbf{b} \in C^1(\Omega)$,

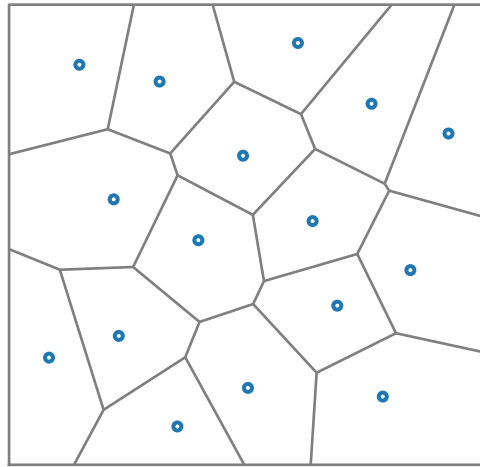


Figure 3.5.5. Voronoi mesh with 16 quasi-random seed points.

and both $c(x, y) \geq 0$ and $\nabla \cdot \mathbf{b}(x, y) \geq 0$ pointwise, with Dirichlet boundary conditions. If we consider a sequence of admissible meshes, $\{\Omega^h\}$, such that the element diameters approach zero and consider the function u^h defined to be the piecewise constant interpolant of the discrete solution value on each volume, then it can be shown that u^h converges to the *weak solution* of the PDE as the maximum element diameter approaches zero. If, furthermore, the PDE in the strong form has a solution, $u \in C^2(\Omega)$, then there exists a constant, $C > 0$, such that $\|u - u^h\|_{L^2(\Omega)} \leq C \max_{T \in \Omega^h} \text{diam}(T)$, where the norm is the continuum $L^2(\Omega)$ norm, $\|u - u^h\|_{L^2(\Omega)}^2 = \int_{\Omega} (u - u^h)^2 dx$; see [95, Thm. 9.3]. This is a first-order bound that suffers in comparison to some other discretizations that offer a bound on the error proportional to the square of $\max_{T \in \Omega^h} \text{diam}(T)$. However, this is not unexpected given the piecewise constant interpolant u^h . Indeed, under further assumptions on the mesh and the PDE (for example, that $\mathbf{b}(x, y) = \mathbf{0}$ and $c(x, y) = 0$), the error bound on the finite-volume solution can also be improved to second order.

Finally, it must be noted that the finite-volume methodology is much more relevant to the time-dependent case explored later, in Chapters 6 and 9, due to its natural *conservation* properties. Thus, while we present it here as an extension of the finite-difference methodology in the elliptic case (which is useful when considering more general meshes), this is only a “first taste” of the approach.

3.6 ■ Variants of finite-difference methods

Objectives

The appeal of finite-difference schemes is both in the relative ease of their construction and in the high efficiency that can be achieved due to the structure assumed in the grid. Unfortunately, for even slightly more complicated settings, finite-difference methods can require significant modifications. In this section, you will consider two popular variants that can extend the applicability of finite-difference methods to more general settings. You will be introduced to differencing in more general (curvilinear) coordinate systems; be able to associate this differencing with more familiar cases, such as polar coordinates; and learn to use Taylor’s theorem to derive approximations in *neighborhoods* for use in more general domains, leading to so-called *meshless* methods.

3.6.1 ■ Finite differences in curvilinear coordinates

Discretizations on structured grids are often highly computationally efficient, as we can take advantage of blocked-memory layout and straightforward finite-difference schemes. In this section, we take a look at generalizing this to *logically* structured grids and the specific case where a separate coordinate system can define the space.

Consider Figure 3.6.1, where we have a structured rectangular grid on the left, defined on the *reference* domain, $\hat{\Omega}$, and a logically structured grid (or curvilinear grid) on the right, defined on the *physical* domain, Ω . We first assume that there is a known coordinate transformation from the coordinates $\mathbf{r} = (r, s)$ in the reference domain to the coordinates $\mathbf{x} = (x, y)$ in the physical domain, given by

$$\mathbf{x}(\mathbf{r}) : \hat{\Omega} \rightarrow \Omega. \quad (3.152)$$

This transformation (and its inverse) provide a mapping between the reference grid, which we denote by $\hat{\Omega}^h$, and the physical grid, denoted by Ω^h .

Remark 3.18: General dimensions

In this section, we derive the transformed problem and its resulting discretization only in 2D, noting that generalizations are easily described using Einstein notation. We give an example at the end of the section.

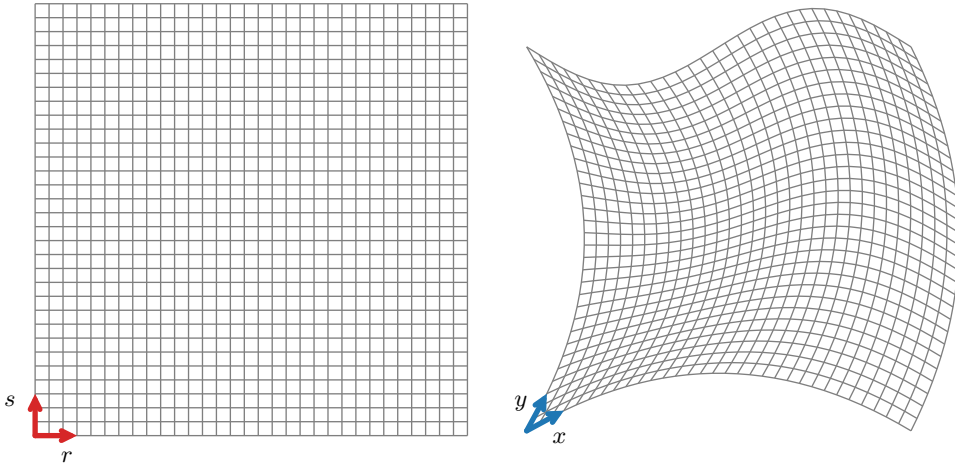


Figure 3.6.1. (left) Reference mesh $\hat{\Omega}$ and (right) curvilinear mesh in the physical domain Ω .

If we consider the Poisson problem on the physical domain,

$$-\Delta u = f(\mathbf{x}) \quad \text{in } \Omega, \quad (3.153)$$

the goal is to represent this problem on the regular structure of the reference domain defined by $\hat{\Omega}$, where we can easily construct finite-difference schemes.

We begin with the mapping $\mathbf{x}(\mathbf{r})$ and define the Jacobian matrix of the transformation as

$$\mathcal{J} = \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial s} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial s} \end{bmatrix} \quad (3.154)$$

and its (Jacobian) determinant as

$$J = |\mathcal{J}| = \frac{\partial x}{\partial r} \frac{\partial y}{\partial s} - \frac{\partial y}{\partial r} \frac{\partial x}{\partial s}. \quad (3.155)$$

Likewise, the *inverse* of the Jacobian matrix is

$$\mathcal{J}^{-1} = \frac{1}{J} \begin{bmatrix} \frac{\partial y}{\partial s} & -\frac{\partial x}{\partial s} \\ -\frac{\partial y}{\partial r} & \frac{\partial x}{\partial r} \end{bmatrix} \quad (3.156a)$$

$$= \begin{bmatrix} \frac{\partial r}{\partial x} & \frac{\partial r}{\partial y} \\ \frac{\partial s}{\partial x} & \frac{\partial s}{\partial y} \end{bmatrix}, \quad (3.156b)$$

where the latter step associates the inverse of the Jacobian matrix with the Jacobian matrix of the inverse transformation $\mathbf{r}(\mathbf{x})$ (from the inverse function theorem). Moreover, the standard identity gives $|\mathcal{J}^{-1}| = 1/J$ (where we implicitly assume that $\mathcal{J}$ is not singular, as is expected to be the case). From the identity in equation (3.156b), we have several handy identities that we use next:

$$\frac{\partial r}{\partial x} = \frac{1}{J} \frac{\partial y}{\partial s}, \quad \frac{\partial s}{\partial x} = -\frac{1}{J} \frac{\partial y}{\partial r}, \quad \frac{\partial r}{\partial y} = -\frac{1}{J} \frac{\partial x}{\partial s}, \quad \frac{\partial s}{\partial y} = \frac{1}{J} \frac{\partial x}{\partial r}. \quad (3.157)$$

These identities lead to equalities that help with the change of variables from Ω to $\hat{\Omega}$, specifically,

$$\frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial x} \right) + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial x} \right) = \frac{\partial}{\partial r} \frac{\partial y}{\partial s} - \frac{\partial}{\partial s} \frac{\partial y}{\partial r} = 0 \quad (3.158a)$$

and

$$\frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial y} \right) + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial y} \right) = -\frac{\partial}{\partial r} \frac{\partial x}{\partial s} + \frac{\partial}{\partial s} \frac{\partial x}{\partial r} = 0. \quad (3.158b)$$

With these identities in place, we next turn to computing a change of variables, allowing us to express the *gradient* of a function, say $u(\mathbf{x})$, in terms of the function $\hat{u}(\mathbf{r})$, with the relation that $u(\mathbf{x}) = \hat{u}(\mathbf{r}(\mathbf{x}))$. After we compute this, we continue the calculation to relate the Laplacian of u to the Laplacian of $\hat{u}$. Considering the first component of the gradient of $u(\mathbf{x})$, the derivative with respect to x , we have

$$\frac{\partial u}{\partial x} = \frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial x} + \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial x} \quad \text{Chain rule} \quad (3.159a)$$

$$= \frac{1}{J} \left(J \frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial x} + J \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial x} \right) \quad \text{Extracting } J \quad (3.159b)$$

$$= \frac{1}{J} \left(J \frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial x} + \frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial x} \right) \hat{u} + J \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial x} + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial x} \right) \hat{u} \right) \quad \text{From equation (3.158a)} \quad (3.159c)$$

$$= \frac{1}{J} \left(\frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial x} \hat{u} \right) + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial x} \hat{u} \right) \right). \quad \text{Product rule} \quad (3.159d)$$

Similarly, for the y component, we have

$$\frac{\partial u}{\partial y} = \frac{1}{J} \left(\frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial y} \hat{u} \right) + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial y} \hat{u} \right) \right). \quad (3.159e)$$

If we now consider having data for $\hat{u}(r, s)$ on a mesh in (r, s) space, we see how we can compute approximate values of ∇u on the corresponding (mapped) mesh in (x, y) space. Knowing $\mathbf{r}(\mathbf{x})$ analytically, as well as its associated Jacobian matrix and determinant, we evaluate these at any point in (x, y) space, in particular, at any point mapped forward from the mesh in (r, s) space. Thus, we evaluate (for example) $J \frac{\partial r}{\partial x} \hat{u}$ at any point in (r, s) space. This allows us to apply standard finite-difference schemes in (r, s) space to approximate either $\frac{\partial u}{\partial x}$ or $\frac{\partial u}{\partial y}$ at the mapped grid points in (x, y) space.

We would like to use these formulas recursively to represent the second-derivative operators, ∂_{xx} and ∂_{yy} in equation (3.153), but that requires writing $\frac{\partial u}{\partial x}(x, y)$ and $\frac{\partial u}{\partial y}(x, y)$ as mapped functions from (r, s) space (so that the formulas in equations (3.159d) and (3.159e) are valid). To do this, we first use the chain rule on $u(\mathbf{x}) = \hat{u}(\mathbf{r}(\mathbf{x}))$, writing

$$\frac{\partial u}{\partial x} = \left(\frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial x} + \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial x} \right) \text{ and } \frac{\partial u}{\partial y} = \left(\frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial y} + \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial y} \right). \quad (3.160)$$

We recognize that each of these formulas gives a value of the partial derivative of u at a point, (x, y) , in terms of another function that we can evaluate at the mapped point, $\mathbf{r}(\mathbf{x})$, since we naturally evaluate $\frac{\partial \hat{u}}{\partial r}$ at the mapped point and we trivially evaluate the coordinate derivatives using the identity mapping $\mathbf{x} = \mathbf{x}(\mathbf{r}(\mathbf{x}))$. Thus, at a point (x, y) , with $(r, s) = \mathbf{r} = \mathbf{r}(\mathbf{x})$, we have

$$\frac{\partial u}{\partial x}(x, y) = \left(\frac{\partial \hat{u}}{\partial r}(r, s) \frac{\partial r}{\partial x}(\mathbf{x}(\mathbf{r})) + \frac{\partial \hat{u}}{\partial s}(r, s) \frac{\partial s}{\partial x}(\mathbf{x}(\mathbf{r})) \right), \quad (3.161a)$$

$$\frac{\partial u}{\partial y}(x, y) = \left(\frac{\partial \hat{u}}{\partial r}(r, s) \frac{\partial r}{\partial y}(\mathbf{x}(\mathbf{r})) + \frac{\partial \hat{u}}{\partial s}(r, s) \frac{\partial s}{\partial y}(\mathbf{x}(\mathbf{r})) \right). \quad (3.161b)$$

Turning to the Laplace operator, $-\partial_{xx} - \partial_{yy}$, we now substitute these expressions for $\frac{\partial u}{\partial x}$ and $\frac{\partial u}{\partial y}$ into equations (3.159d) and (3.159e), which results in

$$-u_{xx} - u_{yy} = -\frac{1}{J} \left(\frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial x} \frac{\partial u}{\partial x} \right) + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial x} \frac{\partial u}{\partial x} \right) \right) \quad (3.162a)$$

$$- \frac{1}{J} \left(\frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial y} \frac{\partial u}{\partial y} \right) + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial y} \frac{\partial u}{\partial y} \right) \right) \quad (3.162b)$$

$$= -\frac{1}{J} \left(\frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial x} \left(\frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial x} + \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial x} \right) \right) + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial x} \left(\frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial x} + \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial x} \right) \right) \right) \quad (3.162c)$$

$$- \frac{1}{J} \left(\frac{\partial}{\partial r} \left(J \frac{\partial r}{\partial y} \left(\frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial y} + \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial y} \right) \right) + \frac{\partial}{\partial s} \left(J \frac{\partial s}{\partial y} \left(\frac{\partial \hat{u}}{\partial r} \frac{\partial r}{\partial y} + \frac{\partial \hat{u}}{\partial s} \frac{\partial s}{\partial y} \right) \right) \right). \quad (3.162d)$$

We then group the terms based on which second derivatives are being applied to $\hat{u}$, giving the

more compact form

$$-u_{xx} - u_{yy} = -\frac{1}{J} \left(\frac{\partial}{\partial r} \left(J \left(\frac{\partial r}{\partial x} \frac{\partial r}{\partial x} + \frac{\partial r}{\partial y} \frac{\partial r}{\partial y} \right) \frac{\partial \hat{u}}{\partial r} \right) \right) \quad (3.163a)$$

$$-\frac{1}{J} \left(\frac{\partial}{\partial r} \left(J \left(\frac{\partial r}{\partial x} \frac{\partial s}{\partial x} + \frac{\partial r}{\partial y} \frac{\partial s}{\partial y} \right) \frac{\partial \hat{u}}{\partial s} \right) \right) \quad (3.163b)$$

$$-\frac{1}{J} \left(\frac{\partial}{\partial s} \left(J \left(\frac{\partial s}{\partial x} \frac{\partial r}{\partial x} + \frac{\partial s}{\partial y} \frac{\partial r}{\partial y} \right) \frac{\partial \hat{u}}{\partial r} \right) \right) \quad (3.163c)$$

$$-\frac{1}{J} \left(\frac{\partial}{\partial s} \left(J \left(\frac{\partial s}{\partial x} \frac{\partial s}{\partial x} + \frac{\partial s}{\partial y} \frac{\partial s}{\partial y} \right) \frac{\partial \hat{u}}{\partial s} \right) \right). \quad (3.163d)$$

The terms in red in equation (3.163) have a special geometric meaning (discussed in Remark 3.19); for now, we label them as

$$g^{rr} = \frac{\partial r}{\partial x} \frac{\partial r}{\partial x} + \frac{\partial r}{\partial y} \frac{\partial r}{\partial y}, \quad (3.164a)$$

$$g^{rs} = \frac{\partial r}{\partial x} \frac{\partial s}{\partial x} + \frac{\partial r}{\partial y} \frac{\partial s}{\partial y}, \quad (3.164b)$$

$$g^{sr} = \frac{\partial s}{\partial x} \frac{\partial r}{\partial x} + \frac{\partial s}{\partial y} \frac{\partial r}{\partial y}, \quad (3.164c)$$

$$g^{ss} = \frac{\partial s}{\partial x} \frac{\partial s}{\partial x} + \frac{\partial s}{\partial y} \frac{\partial s}{\partial y}, \quad (3.164d)$$

noting that $g^{rs} = g^{sr}$. This leads to the following (general) form for writing the Laplacian, to which we apply a differencing scheme, as in the earlier sections:

$$\begin{aligned} -\nabla_{(x,y)} \cdot \nabla_{(x,y)} u &= -\frac{1}{J} \partial_r (J g^{rr} \partial_r \hat{u}) - \frac{1}{J} \partial_r (J g^{rs} \partial_s \hat{u}) \\ &\quad - \frac{1}{J} \partial_s (J g^{sr} \partial_r \hat{u}) - \frac{1}{J} \partial_s (J g^{ss} \partial_s \hat{u}), \end{aligned} \quad (3.165a)$$

$$= -\frac{1}{J} \nabla_{(r,s)} \cdot J \begin{bmatrix} g^{rr} & g^{rs} \\ g^{sr} & g^{ss} \end{bmatrix} \nabla_{(r,s)} \hat{u}. \quad (3.165b)$$

We emphasize that in the tensor form in equation (3.165b), the inner diffusion tensor,

$$\begin{bmatrix} G^{rr} & G^{rs} \\ G^{sr} & G^{ss} \end{bmatrix} := J \begin{bmatrix} g^{rr} & g^{rs} \\ g^{sr} & g^{ss} \end{bmatrix}, \quad (3.166)$$

is evaluated on the (r, s) mesh, so that we can use finite differences on the (r, s) mesh to approximate the Laplacian at points in the (x, y) mesh.

There are many options to construct a finite-difference scheme for equation (3.165a). We could expand the derivatives and then use straightforward centered differencing, but this can easily result in nonsymmetric systems of equations to be solved, as discussed in Section 3.5. To avoid this, we employ mimetic-type finite differencing, as we did there. Figure 3.6.2 outlines the general strategy for this type of scheme. For example, the left figure shows the approximation to $\partial_r \hat{u}$ on a regular grid, where we assume that we know values of $\hat{u}$ at the cell centers and want to approximate $\partial_r \hat{u}$ at the node at index (i, j) . To do this, we use centered differencing to approximate the derivative at two points, $(i, j \pm \frac{1}{2})$, followed by averaging of the two to get

$$(\partial_r \hat{u})_{i,j} = \frac{\hat{u}_{i+\frac{1}{2},j+\frac{1}{2}} - \hat{u}_{i-\frac{1}{2},j+\frac{1}{2}} + \hat{u}_{i+\frac{1}{2},j-\frac{1}{2}} - \hat{u}_{i-\frac{1}{2},j-\frac{1}{2}}}{2h}. \quad (3.167)$$

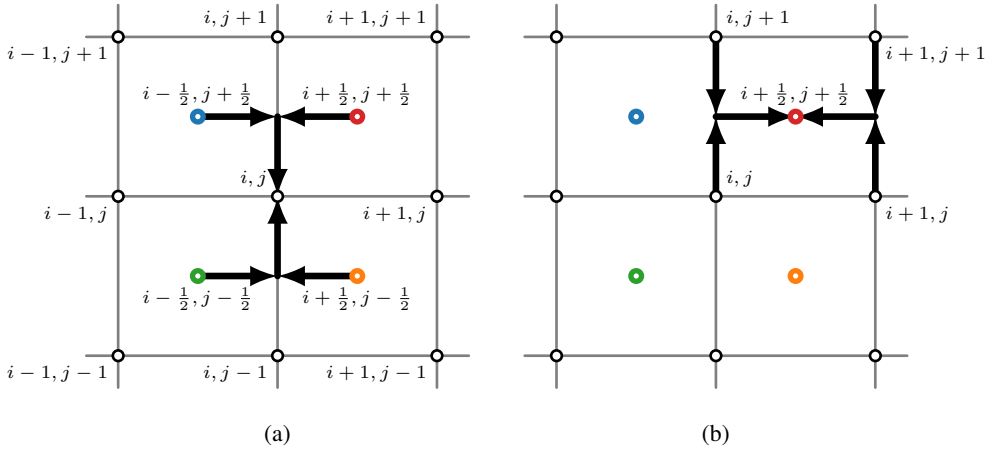


Figure 3.6.2. Mimetic finite differencing in the (left) r -direction evaluated at node (i, j) using cell center values, and in the (right) s -direction evaluated at cell centers using node values.

In a similar fashion, an approximation to $\partial_s \hat{u}$, but now evaluated at the cell center $(i + \frac{1}{2}, j + \frac{1}{2})$, can be constructed with centered differencing of nodal values to approximate the derivative at the two points $(i \pm \frac{1}{2}, j)$, followed by averaging (as in the right schematic in Figure 3.6.2):

$$(\partial_s \hat{u})_{i+\frac{1}{2}, j+\frac{1}{2}} = \frac{\hat{u}_{i+1, j+1} - \hat{u}_{i+1, j} + \hat{u}_{i, j+1} - \hat{u}_{i, j}}{2h}. \quad (3.168)$$

In both cases above, we assume a uniform grid spacing, with $h = h_x = h_y$, although generalizations easily follow.

In the following, we use combinations of differencing between nodal and cell-centered values, as in equations (3.167) and (3.168), on each of the terms $\partial_r (G^{rr} \partial_r \hat{u})$, $\partial_r (G^{rs} \partial_s \hat{u})$, $\partial_s (G^{sr} \partial_r \hat{u})$, and $\partial_s (G^{ss} \partial_s \hat{u})$ in equation (3.165b). In all cases, we approximate the “inner” derivative by differencing nodal values to get cell-centered approximations of $\partial_r u$ and $\partial_s u$ at locations $(i \pm \frac{1}{2}, j \pm \frac{1}{2})$, followed by differencing these cell-centered values to get nodal approximations of the second derivatives. For clarity, we mark the terms evaluated at cell centers with colors: $(i + \frac{1}{2}, j + \frac{1}{2})$, $(i - \frac{1}{2}, j + \frac{1}{2})$, $(i + \frac{1}{2}, j - \frac{1}{2})$, and $(i - \frac{1}{2}, j - \frac{1}{2})$.

First consider the mixed derivative $\partial_r (G^{rs} \partial_s)$ term. We approximate this as

$$(\partial_r (G^{rs} \partial_s \hat{u}))_{i, j} = \frac{1}{2h} \left(\overbrace{G_{i+\frac{1}{2}, j+\frac{1}{2}}^{rs} (\partial_s \hat{u})_{i+\frac{1}{2}, j+\frac{1}{2}}}^{\text{see Figure 3.6.2(a)}} - G_{i-\frac{1}{2}, j+\frac{1}{2}}^{rs} (\partial_s \hat{u})_{i-\frac{1}{2}, j+\frac{1}{2}} \right. \\ \left. + G_{i+\frac{1}{2}, j-\frac{1}{2}}^{rs} (\partial_s \hat{u})_{i+\frac{1}{2}, j-\frac{1}{2}} - G_{i-\frac{1}{2}, j-\frac{1}{2}}^{rs} (\partial_s \hat{u})_{i-\frac{1}{2}, j-\frac{1}{2}} \right) \quad (3.169a)$$

$$= \frac{1}{4h^2} \left(\overbrace{G_{i+\frac{1}{2}, j+\frac{1}{2}}^{rs} (\hat{u}_{i+1, j+1} - \hat{u}_{i+1, j} + \hat{u}_{i, j+1} - \hat{u}_{i, j})}^{\text{see Figure 3.6.2(b)}} \right. \\ - G_{i-\frac{1}{2}, j+\frac{1}{2}}^{rs} (\hat{u}_{i, j+1} - \hat{u}_{i, j} + \hat{u}_{i-1, j+1} - \hat{u}_{i-1, j}) \\ + G_{i+\frac{1}{2}, j-\frac{1}{2}}^{rs} (\hat{u}_{i+1, j} - \hat{u}_{i+1, j-1} + \hat{u}_{i, j} - \hat{u}_{i, j-1}) \\ \left. - G_{i-\frac{1}{2}, j-\frac{1}{2}}^{rs} (\hat{u}_{i, j} - \hat{u}_{i, j-1} + \hat{u}_{i-1, j} - \hat{u}_{i-1, j-1}) \right). \quad (3.169b)$$

As noted above, $G^{rs} = G^{sr}$, so that $(\partial_s (G^{sr} \partial_r \hat{u}))_{i,j} = (\partial_s (G^{rs} \partial_r \hat{u}))_{i,j}$. This gives

$$(\partial_s (G^{rs} \partial_r \hat{u}))_{i,j} = \frac{1}{2h} \left(G_{i+\frac{1}{2},j+\frac{1}{2}}^{rs} (\partial_r \hat{u})_{i+\frac{1}{2},j+\frac{1}{2}} - G_{i+\frac{1}{2},j-\frac{1}{2}}^{rs} (\partial_r \hat{u})_{i+\frac{1}{2},j-\frac{1}{2}} \right. \\ \left. + G_{i-\frac{1}{2},j+\frac{1}{2}}^{rs} (\partial_r \hat{u})_{i-\frac{1}{2},j+\frac{1}{2}} - G_{i-\frac{1}{2},j-\frac{1}{2}}^{rs} (\partial_r \hat{u})_{i-\frac{1}{2},j-\frac{1}{2}} \right) \quad (3.170a)$$

$$= \frac{1}{4h^2} \left(G_{i+\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i+1,j+1} - \hat{u}_{i,j+1} + \hat{u}_{i+1,j} - \hat{u}_{i,j}) \right. \\ - G_{i+\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i+1,j} - \hat{u}_{i,j} + \hat{u}_{i+1,j-1} - \hat{u}_{i,j-1}) \\ + G_{i-\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i,j+1} - \hat{u}_{i-1,j+1} + \hat{u}_{i,j} - \hat{u}_{i-1,j}) \\ \left. - G_{i-\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i,j} - \hat{u}_{i-1,j} + \hat{u}_{i,j-1} - \hat{u}_{i-1,j-1}) \right). \quad (3.170b)$$

While summing these two expressions is a bit of a daunting task, most terms cancel and we are left with an expression that resembles “differencing along the diagonals”:

$$4h^2 (\partial_r (G^{rs} \partial_s \hat{u}))_{i,j} + (\partial_s (G^{sr} \partial_r \hat{u}))_{i,j} \\ = G_{i+\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i+1,j+1} - \hat{u}_{i+1,j} + \hat{u}_{i,j+1} - \hat{u}_{i,j}) \\ - G_{i-\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i,j+1} - \hat{u}_{i,j} + \hat{u}_{i-1,j+1} - \hat{u}_{i-1,j}) \\ + G_{i+\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i+1,j} - \hat{u}_{i+1,j-1} + \hat{u}_{i,j} - \hat{u}_{i,j-1}) \\ - G_{i-\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i,j} - \hat{u}_{i,j-1} + \hat{u}_{i-1,j} - \hat{u}_{i-1,j-1}) \\ + G_{i+\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i+1,j+1} - \hat{u}_{i,j+1} + \hat{u}_{i+1,j} - \hat{u}_{i,j}) \\ - G_{i+\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i+1,j} - \hat{u}_{i,j} + \hat{u}_{i+1,j-1} - \hat{u}_{i,j-1}) \\ + G_{i-\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i,j+1} - \hat{u}_{i-1,j+1} + \hat{u}_{i,j} - \hat{u}_{i-1,j}) \\ - G_{i-\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i,j} - \hat{u}_{i-1,j} + \hat{u}_{i,j-1} - \hat{u}_{i-1,j-1}) \quad (3.171a)$$

$$= G_{i+\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i+1,j+1} - \cancel{\hat{u}_{i+1,j}} + \cancel{\hat{u}_{i,j+1}} - \hat{u}_{i,j} + \hat{u}_{i+1,j+1} - \cancel{\hat{u}_{i,j+1}} + \cancel{\hat{u}_{i+1,j}} - \hat{u}_{i,j}) \\ + G_{i-\frac{1}{2},j+\frac{1}{2}}^{rs} (-\cancel{\hat{u}_{i,j+1}} + \hat{u}_{i,j} - \hat{u}_{i-1,j+1} + \cancel{\hat{u}_{i-1,j+1}} + \cancel{\hat{u}_{i,j+1}} - \hat{u}_{i-1,j+1} + \hat{u}_{i,j} - \cancel{\hat{u}_{i-1,j}}) \\ + G_{i+\frac{1}{2},j-\frac{1}{2}}^{rs} (\cancel{\hat{u}_{i+1,j}} - \hat{u}_{i+1,j-1} + \hat{u}_{i,j} - \cancel{\hat{u}_{i,j-1}} - \cancel{\hat{u}_{i+1,j-1}} + \hat{u}_{i,j} - \hat{u}_{i+1,j-1} + \cancel{\hat{u}_{i,j-1}}) \\ - G_{i-\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i,j} - \cancel{\hat{u}_{i,j-1}} + \cancel{\hat{u}_{i-1,j-1}} - \hat{u}_{i-1,j-1} + \hat{u}_{i,j} - \cancel{\hat{u}_{i-1,j}} + \cancel{\hat{u}_{i,j-1}} - \hat{u}_{i-1,j-1}) \quad (3.171b)$$

$$= 2 G_{i+\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i+1,j+1} - \hat{u}_{i,j}) - 2 G_{i-\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i,j} - \hat{u}_{i-1,j-1}) \\ + 2 G_{i-\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i,j} - \hat{u}_{i-1,j+1}) - 2 G_{i+\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i+1,j-1} - \hat{u}_{i,j}). \quad (3.171c)$$

For the nonmixed terms, $\partial_r (G^{rr} \partial_r \hat{u})$ and $\partial_s (G^{ss} \partial_s \hat{u})$, we use the standard centered-differencing approach and arrive at

$$\partial_r (G^{rr} \partial_r \hat{u}) = \frac{1}{h^2} \left(G_{i+\frac{1}{2},j}^{rr} (\hat{u}_{i+1,j} - \hat{u}_{i,j}) - G_{i-\frac{1}{2},j}^{rr} (\hat{u}_{i,j} - \hat{u}_{i-1,j}) \right) \quad (3.172a)$$

and

$$\partial_s (G^{ss} \partial_s \hat{u}) = \frac{1}{h^2} \left(G_{i,j+\frac{1}{2}}^{ss} (\hat{u}_{i,j+1} - \hat{u}_{i,j}) - G_{i,j-\frac{1}{2}}^{ss} (\hat{u}_{i,j} - \hat{u}_{i,j-1}) \right). \quad (3.172b)$$

Combining equations (3.171c), (3.172a), and (3.172b) completes the discretization. At grid point (i, j) , we have the approximation

$$\begin{aligned}
 & -\partial_r (G^{rr} \partial_r \hat{u}) - \partial_r (G^{rs} \partial_s \hat{u}) - \partial_s (G^{sr} \partial_r \hat{u}) - \partial_s (G^{ss} \partial_s \hat{u}) \\
 & \approx -\frac{1}{h^2} \left(G_{i+\frac{1}{2},j}^{rr} (\hat{u}_{i+1,j} - \hat{u}_{i,j}) - G_{i-\frac{1}{2},j}^{rr} (\hat{u}_{i,j} - \hat{u}_{i-1,j}) \right) \\
 & \quad -\frac{1}{2h^2} \left(G_{i+\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i+1,j+1} - \hat{u}_{i,j}) - G_{i-\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i,j} - \hat{u}_{i-1,j-1}) \right. \\
 & \quad \left. + G_{i-\frac{1}{2},j+\frac{1}{2}}^{rs} (\hat{u}_{i,j} - \hat{u}_{i-1,j+1}) - G_{i+\frac{1}{2},j-\frac{1}{2}}^{rs} (\hat{u}_{i+1,j-1} - \hat{u}_{i,j}) \right) \\
 & \quad -\frac{1}{h^2} \left(G_{i,j+\frac{1}{2}}^{ss} (\hat{u}_{i,j+1} - \hat{u}_{i,j}) - G_{i,j-\frac{1}{2}}^{ss} (\hat{u}_{i,j} - \hat{u}_{i,j-1}) \right). \tag{3.173}
 \end{aligned}$$

Since we have used second-order centered differencing in each case, the expected accuracy remains second order. In addition, the structure of the diffusion tensor guarantees that the resulting system of algebraic equations is symmetric and positive definite (so long as $\mathcal{J}$ is not singular, as we have assumed). There are two final details that we need to clean up to complete the scheme.

First, the scheme is specified in terms of the values of the function $\hat{u}(\mathbf{r})$ on a uniform mesh of $\hat{\Omega}$, with the property that $u(\mathbf{x}) = \hat{u}(\mathbf{r}(\mathbf{x}))$. This implicitly defines a mesh of Ω by mapping the nodes (and, if needed, edges) of the mesh of $\hat{\Omega}$ forward to Ω by the mapping $\mathbf{x}(\mathbf{r})$. This allows us to remove the dependence on $\hat{u}(\mathbf{r})$ from our scheme by using the same indexing of points in Ω , defining $\mathbf{x}_{i,j} = \mathbf{x}(\mathbf{r}_{i,j})$ and then equating the approximations $u_{i,j} = \hat{u}_{i,j}$, based on the continuum relationship that $u(\mathbf{x}_{i,j}) = \hat{u}(\mathbf{r}_{i,j})$, since $\mathbf{r}(\mathbf{x}(\mathbf{r}_{i,j})) = \mathbf{r}_{i,j}$. Finally, recalling equation (3.165b), we rewrite our approximation directly in terms of $u(\mathbf{x})$ as

$$\begin{aligned}
 -\nabla_{(x,y)} \cdot \nabla_{(x,y)} u & \approx -\frac{1}{J_{i,j} h^2} \left(G_{i+\frac{1}{2},j}^{rr} (u_{i+1,j} - u_{i,j}) - G_{i-\frac{1}{2},j}^{rr} (u_{i,j} - u_{i-1,j}) \right) \\
 & \quad -\frac{1}{2J_{i,j} h^2} \left(G_{i+\frac{1}{2},j+\frac{1}{2}}^{rs} (u_{i+1,j+1} - u_{i,j}) - G_{i-\frac{1}{2},j-\frac{1}{2}}^{rs} (u_{i,j} - u_{i-1,j-1}) \right. \\
 & \quad \left. + G_{i-\frac{1}{2},j+\frac{1}{2}}^{rs} (u_{i,j} - u_{i-1,j+1}) - G_{i+\frac{1}{2},j-\frac{1}{2}}^{rs} (u_{i+1,j-1} - u_{i,j}) \right) \\
 & \quad -\frac{1}{J_{i,j} h^2} \left(G_{i,j+\frac{1}{2}}^{ss} (u_{i,j+1} - u_{i,j}) - G_{i,j-\frac{1}{2}}^{ss} (u_{i,j} - u_{i,j-1}) \right). \tag{3.174}
 \end{aligned}$$

If one or both of the maps $\mathbf{x}(\mathbf{r})$ or $\mathbf{r}(\mathbf{x})$ is known exactly, then we can evaluate the inner diffusion tensor in equation (3.166) analytically by sampling the coefficient values at the edge midpoints or cell centers given in equation (3.174). In practice, however, we derive adequately accurate approximations of these values based solely on the coordinates of the curvilinear mesh points, $\{\mathbf{x}_{i,j}\}$. Consider, for example, the G^{rr} coefficient defined in equation (3.166), which must be sampled at the edge midpoints:

$$G_{i+\frac{1}{2},j}^{rr} = J_{i+\frac{1}{2},j} \left(\frac{\partial r}{\partial x} \frac{\partial r}{\partial x} + \frac{\partial r}{\partial y} \frac{\partial r}{\partial y} \right)_{i+\frac{1}{2},j}. \tag{3.175}$$

Assuming a uniform mesh in (r, s) space, we infer a local mapping $\mathbf{r}(\mathbf{x})$ by requiring that it interpolate the mesh-point locations $\{\mathbf{x}_{k,\ell}\}$ for points (k, ℓ) in some neighborhood of (i, j) (for example, for points where $i-1 \leq k \leq i+2$ and $j-1 \leq \ell \leq j+1$, to evenly sample around the edge midpoint $(i+\frac{1}{2}, j)$). From this interpolatory mapping, we then infer the derivatives needed in equation (3.175), including those in the values in the Jacobian determinant. We omit specific details, as the computational aspects are highly dependent on the problem.

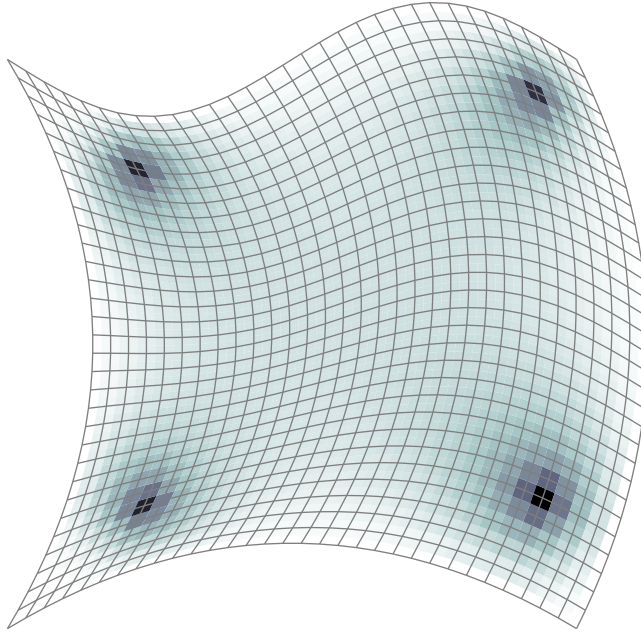


Figure 3.6.3. Example approximation on a curvilinear mesh with four point sources.

To complete this section, we return to the mesh in Figure 3.6.1, where the transformation on $\mathbf{r}$ is given by

$$x = r(1 - s) + rs + 0.6(1 - r)s(1 - s) + 0.5rs(1 - s), \quad (3.176a)$$

$$y = (1 - r)s + rs + 0.6(1 - s)r(1 - r) - 0.1s \sin(2\pi r). \quad (3.176b)$$

Letting the right-hand side, f , be a sum of unit point sources, each located near one of the four corners of the domain, we see that the finite-difference approximation recovers the expected diffusion in the problem—see Figure 3.6.3.

Remark 3.19: Curvilinear coordinates in 3D and Einstein notation

Equation (3.165b) gives the two-dimensional expression; however, by adopting Einstein notation, the form is straightforward in 3D as well. To do this, consider variables on the physical space given by x^1 , x^2 , and x^3 (rather than x , y , and z) and the variables in the reference space to be given by r^1 , r^2 , and r^3 (rather than r , s , and t). With Einstein notation (where repeated indices represent a summation of the respective index), equation (3.165b) becomes

$$-\Delta_{(x^1, x^2, x^3)} u = -\frac{1}{J} \frac{\partial}{\partial r^\alpha} \left(J g^{\alpha\beta} \frac{\partial u}{\partial r^\beta} \right), \quad (3.177)$$

where $g^{\alpha\beta}$ is defined similarly to equation (3.164). Note that defining the Jacobian matrix of the mapping, $\mathcal{J}$, and extending the two-dimensional case from equation (3.154), the values $g^{\alpha\beta}$ are the entries in the matrix $\mathcal{J}^{-1} \mathcal{J}^{-T}$. The tensor $g^{\alpha\beta}$ is commonly known as the *contravariant metric tensor*.

It is also worth noting that if the tensor $g^{\alpha\beta}$ is diagonal, then the curvilinear grids have an orthogonal basis and the expressions for the Laplacian and other operators are greatly simplified.

Polar coordinates are one example, leading to the common expression for the Laplacian,

$$-\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial u}{\partial r} \right) - \frac{1}{r^2} \frac{\partial^2 u}{\partial \theta^2} = -\frac{\partial^2 u}{\partial r^2} - \frac{1}{r} \frac{\partial u}{\partial r} - \frac{1}{r^2} \frac{\partial^2 u}{\partial \theta^2}, \quad (3.178)$$

and specialized numerical schemes for approximating values of the Laplacian on grids in (r, θ) space. A full treatment of curvilinear coordinates and grids can be found in [155].

3.6.2 ■ Meshless finite-difference methods

One fundamental problem in applying the finite-difference methodology on irregular domains is the difficulty in deriving consistent and accurate schemes. When the domain can be mapped to a rectangle (or rectangular prism), generalizations of the curvilinear coordinates used above can be employed, but when the domain and/or approximation points truly lack any regular structure, there is little to take advantage of in the use of Taylor's theorem to derive difference approximations. One approach to overcome this difficulty is to give up altogether on Taylor's theorem and employ what is known as a “meshless” methodology that assumes no structure whatsoever at the discretization points.

The obvious question, then, is how do we approximate a PDE on an arbitrary set of points? A first step is to identify a “neighborhood” around any given point, $\mathbf{x}_0$, and use all other points in that neighborhood to define the approximation. In the continuum domain, Ω , this can often be taken simply as the intersection between the point set under consideration and a ball in Ω :

$$B(\mathbf{x}_0, r) = \{\mathbf{x} \in \Omega \mid \|\mathbf{x} - \mathbf{x}_0\| < r\}. \quad (3.179)$$

See Figure 3.6.4 for an example.

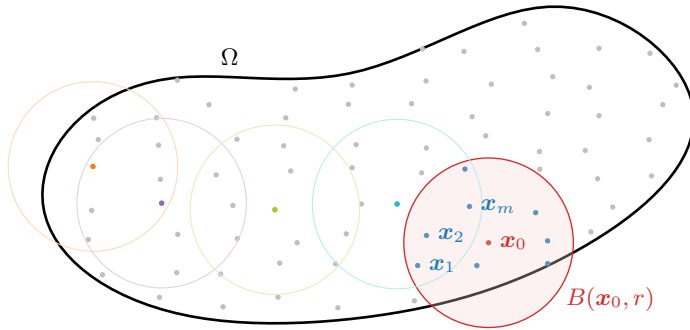


Figure 3.6.4. Example domain with neighborhood $B(\mathbf{x}_0, r)$ at $\mathbf{x}_0$, along with neighborhoods at four selected points.

For sufficiently small r , we expect that standard approximations using Taylor's theorem centered at $\mathbf{x}_0$ should be accurate enough to be reasonable; however, we must also ensure that there are enough discretization points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m$ in $B(\mathbf{x}_0, r)$ in order to be able to create a good discretization. We take this as a given in order to apply the meshless finite-difference methodology. In other words, r is chosen so that we have m points:

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m \in B(\mathbf{x}_0, r). \quad (3.180)$$

Then, we use the function values $u(\mathbf{x}_0), u(\mathbf{x}_1), \dots, u(\mathbf{x}_m)$ to approximate derivatives of $u(\mathbf{x})$ at $\mathbf{x}_0$. Note that we could do this directly from Taylor's theorem, expanding the function value

at each point in terms of a Taylor series at $\mathbf{x}_0$ and solving an algebraic equation to combine these expansions and derive a derivative approximation. This, however, turns out to be overly complicated. An alternative idea is to realize that, if the method is to be consistent, then the truncation error in any approximation must be proportional to higher-order partial derivatives of $u(\mathbf{x})$. This suggests that we can derive the weights in a finite-difference-style approximation by requiring the approximation to hold for low-order polynomials. This idea was already introduced in Section 2.4 as a way to check that a scheme on a regular mesh is consistent; now, we will use it to *derive* a consistent scheme.

As an example, consider a two-dimensional domain, Ω , on which we want to approximate the Poisson equation, $-u_{xx} - u_{yy} = f(x, y)$. At any node $\mathbf{x}_0$, we intend to write the finite-difference approximation using points $\mathbf{x}_0, \dots, \mathbf{x}_m$ as $\sum_{i=0}^m s_i u_i = f(\mathbf{x}_0)$, where $\{s_i\}$ are the weights in the finite-difference scheme, and $\{u_i\}$ are the approximate values of $u(\mathbf{x})$ at the nodes $\mathbf{x}_i = (x_i, y_i)$. If this discretization is to be consistent, then the following conditions must hold:

- Let $u(x, y) = 1$; then $-u_{xx} - u_{yy} = 0 \Rightarrow \sum_{i=0}^m s_i = 0$.
- Let $u(x, y) = x - x_0$; then $-u_{xx} - u_{yy} = 0 \Rightarrow \sum_{i=0}^m s_i (x_i - x_0) = 0$.
- Let $u(x, y) = y - y_0$; then $-u_{xx} - u_{yy} = 0 \Rightarrow \sum_{i=0}^m s_i (y_i - y_0) = 0$.
- Let $u(x, y) = (x - x_0)^2$; then $-u_{xx} - u_{yy} = -2 \Rightarrow \sum_{i=0}^m s_i (x_i - x_0)^2 = -2$.
- Let $u(x, y) = (x - x_0)(y - y_0)$; then $-u_{xx} - u_{yy} = 0 \Rightarrow \sum_{i=0}^m s_i (x_i - x_0)(y_i - y_0) = 0$.
- Let $u(x, y) = (y - y_0)^2$; then $-u_{xx} - u_{yy} = -2 \Rightarrow \sum_{i=0}^m s_i (y_i - y_0)^2 = -2$.

This gives six equations that must be satisfied for the scheme to be consistent, in $m+1$ unknowns. Extending the technique to three dimensions leads to 10 equations, still in $m+1$ unknowns, as there are three first partial derivative terms and six second partial derivative terms.

If we want a scheme with better accuracy, we can simply require that the scheme match the continuum derivatives of more terms, leading to a higher-order consistency. It is natural to add such terms in groups, corresponding to all third-order terms, then all fourth-order terms, and so forth. For example, requiring consistency for all third-order polynomials in two dimensions requires the following conditions:

- Let $u(x, y) = (x - x_0)^3$; then $-u_{xx} - u_{yy} = -6(x - x_0) \Rightarrow \sum_{i=0}^m s_i (x_i - x_0)^3 = 0$.
- Let $u(x, y) = (x - x_0)^2(y - y_0)$; then $-u_{xx} - u_{yy} = -2(y - y_0) \Rightarrow \sum_{i=0}^m s_i (x_i - x_0)^2(y_i - y_0) = 0$.
- Let $u(x, y) = (x - x_0)(y - y_0)^2$; then $-u_{xx} - u_{yy} = -2(x - x_0) \Rightarrow \sum_{i=0}^m s_i (x_i - x_0)(y_i - y_0)^2 = 0$.
- Let $u(x, y) = (y - y_0)^3$; then $-u_{xx} - u_{yy} = -6(y - y_0) \Rightarrow \sum_{i=0}^m s_i (y_i - y_0)^3 = 0$.

Here, we have 10 equations that must be satisfied, but still in $m+1$ unknowns. Considering all terms up to and including third-order polynomials in three dimensions leads to 20 equations that must be satisfied, in $m+1$ unknowns.

In principle, we could continue in this way, to get progressively better order in our schemes by driving the truncation error into higher and higher derivatives, but we are limited by two practical factors. First, as we have seen, the number of terms needed grows with each order, which would

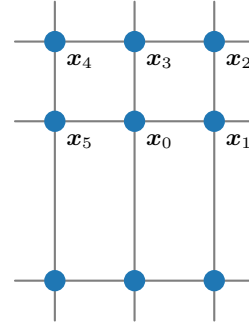
require increasing the number of points in the stencil to achieve higher-order schemes. Having more points in the stencil amounts to a higher cost to use the scheme, and it is important to balance the cost of the scheme with the practical accuracy that results. Second, requiring higher-order consistency can also lead to issues with the stability of the resulting schemes.

At this point, it is important to stress that we can only say that the above are *necessary* conditions for stability of the resulting scheme, but it is not apparent whether they are also *sufficient* conditions. In general, this is true, but only under additional assumptions on the quality of the set of points used in the approximation (extending the idea of a quasi-uniform mesh to the meshless setting). Additionally, some structure of the points on the boundary of the domain is needed in order to be able to accurately impose boundary conditions in this setting.

Accepting that these schemes are consistent, and that the resulting discretizations can be stable, a pragmatic question to ask is which points we should choose to use in the approximation at x_0 . From our original setting, one idea is to choose the radius, r , such that only m points other than x_0 are in $B(x_0, r)$, and $m + 1$ matches the number of constraints to be imposed. As the following example shows, this strategy may not be successful because the $m + 1$ equations may not have a unique solution.

Example 3.20: Meshless approximation without geometric spread

Consider the simple case of the tensor-product mesh shown at right, where the mesh spacing is uniformly h in the x -direction, but $2h$ below the line on which x_0 lies and h above it. Requiring the minimum consistency conditions suggests that we should take $\sqrt{2}h < r < 2h$, so that $m + 1 = 6$ points (including x_0) lie in $B(x_0, r)$.



Writing out the system of equations gives

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & h & h & 0 & -h & -h \\ 0 & 0 & h & h & h & 0 \\ 0 & h^2 & h^2 & 0 & h^2 & h^2 \\ 0 & 0 & h^2 & 0 & -h^2 & 0 \\ 0 & 0 & h^2 & h^2 & h^2 & 0 \end{bmatrix} \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -2 \\ 0 \\ -2 \end{bmatrix}. \quad (3.181)$$

Considering the third and sixth equations, we have

$$hs_2 + hs_3 + hs_4 = 0, \quad (3.182a)$$

$$h^2s_2 + h^2s_3 + h^2s_4 = -2, \quad (3.182b)$$

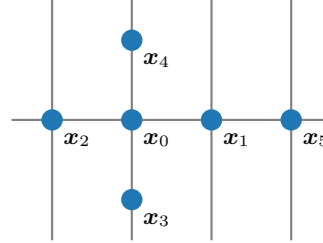
which cannot hold simultaneously. This shows us that even if we have enough points to match the number of equations, we are not guaranteed to have a solution to the resulting system of equations that defines the weights.

What fails in the previous example is that all of the points chosen lie on one side (in the positive y -direction) of the point under consideration. Thus, there is no way for the terms to cancel properly to correctly differentiate both $y - y_0$ and $(y - y_0)^2$. In general, for the meshless

finite-difference method to be successful, we must ensure that we have some geometric “spread” of the points used. This is generally needed to ensure the existence of the solution but also is quite helpful from the perspective of stability, as the following example shows.

Example 3.21: Meshless approximation with geometric spread

Taking points as pictured at right, we again have $m + 1 = 6$, and the resulting system of equations becomes



$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & h & -h & 0 & 0 & 2h \\ 0 & 0 & 0 & -h & h & 0 \\ 0 & h^2 & h^2 & 0 & 0 & 4h^2 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & h^2 & h^2 & 0 \end{bmatrix} \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -2 \\ 0 \\ -2 \end{bmatrix}. \quad (3.183)$$

From the above system, we extract a 2×2 subsystem,

$$\begin{bmatrix} -h & h \\ h^2 & h^2 \end{bmatrix} \begin{bmatrix} s_3 \\ s_4 \end{bmatrix} = \begin{bmatrix} 0 \\ -2 \end{bmatrix}, \quad (3.184)$$

which leads to $s_3 = s_4 = -1/h^2$. Similarly, we extract a 2×3 subsystem,

$$\begin{bmatrix} -h & h & 2h \\ h^2 & h^2 & 4h^2 \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ s_5 \end{bmatrix} = \begin{bmatrix} 0 \\ -2 \end{bmatrix}, \quad (3.185)$$

which is underdetermined, with solution $s_1 = -1/h^2 - 3s_5$ and $s_2 = -1/h^2 - s_5$, with s_5 as a free parameter. With these, we solve for

$$s_0 = -s_1 - s_2 - s_3 - s_4 - s_5 = 4/h^2 + 3s_5. \quad (3.186)$$

Now, some choices of the free parameter, s_5 , will cause problems with stability, even though the scheme is consistent for any choice. For example, taking $s_5 = -1/h^2$ gives the discretization stencil

$$\frac{1}{h^2} (u_0 + 2u_1 - u_3 - u_4 - u_5) = f_0, \quad (3.187)$$

which no longer corresponds to a diagonally dominant matrix. In contrast, any choice of $-\frac{1}{3h^2} < s_5 < 0$ leads to a scheme where all of the off-diagonal coefficients are negative, and the diagonal coefficient of u_0 is positive.

The last example illustrates a general principle that is often used to guide the construction of meshless finite-difference schemes. We aim to keep “enough” points to ensure the existence of solutions, but also to keep them geometrically balanced so that stability is less problematic. Thinking back to our arguments about stability, we recognize that having all negative off-diagonal coefficients is often a desirable matrix property, one that we seek to enforce in the

construction of the scheme, since we always have $s_0 = -\sum_{i=1}^m s_i$. Thus, if the off-diagonal coefficients are all negative, then s_0 is necessarily positive, and this often leads to stability. Algorithms exist to ensure that this can be done, based on including enough points in the stencil to satisfy certain geometric conditions.

3.7 ■ Some further thoughts

While the main focus above has been on deriving the lowest-order consistent finite-difference schemes in various settings, the techniques proposed generalize directly to higher-order methods. A standard approach is to simply use wider stencils, generating a fourth-order accurate approximation of $-u''$ on a uniform mesh, for example, as

$$\frac{u_{j+2} - 16u_{j+1} + 30u_j - 16u_{j-1} + u_{j-2}}{30h^2}. \quad (3.188)$$

While such schemes are useful, there are complications in applying them close to boundaries, where the wide stencils would naturally reach outside the discretized domain. One alternative is to use so-called compact stencils (or “mehrstellenverfahren” [76]) to derive higher-order discretizations [201]. Here, we note that on uniform meshes, we have

$$-u''(x_j) = \frac{1}{h^2} (-u(x_{j+1}) + 2u(x_j) - u(x_{j-1})) + \frac{h^2}{12} u''''(x_j) + \mathcal{O}(h^4), \quad (3.189a)$$

$$u''''(x_j) = \frac{1}{h^2} (u''(x_{j+1}) - 2u''(x_j) + u''(x_{j-1})) + \mathcal{O}(h^2). \quad (3.189b)$$

If our PDE is of the form $-u''(x) = f(x)$, we can replace the second of these by

$$u''''(x_j) = \frac{1}{h^2} (-f(x_{j+1}) + 2f(x_j) - f(x_{j-1})) + \mathcal{O}(h^2), \quad (3.190)$$

leading to the fourth-order accurate scheme given by

$$\frac{1}{h^2} (-u_{j+1} + 2u_j - u_{j-1}) + \frac{1}{12} (-f(x_{j+1}) + 2f(x_j) - f(x_{j-1})) = f(x_j) \quad (3.191)$$

or

$$\frac{1}{h^2} (-u_{j+1} + 2u_j - u_{j-1}) = f(x_j) - \frac{1}{12} (-f(x_{j+1}) + 2f(x_j) - f(x_{j-1})). \quad (3.192)$$

Here, we achieve higher-order accuracy not by taking wider stencils in approximating $-u''(x_j)$ but, instead, by combining values of the forcing function, $f(x)$, evaluated both at x_j and at nearby points. Similar schemes are in common use for two- and three-dimensional problems, due to their easy construction and relatively low cost. In addition, modifications are available to account for nonsmooth boundaries of domains and similar complications that often impact finite-difference convergence [238].

Another approach to improving accuracy of finite-difference methods is to design them to satisfy further continuum properties, such as those approaches discussed in Section 3.5. One common family of approaches are those based on a *summation-by-parts* methodology. For first-order derivatives, we say that an approximation of $u'(x)$ on mesh Ω^h satisfies a summation-by-parts property if we can write the matrix, D , representing the approximation as $D = H^{-1}Q$ for symmetric and positive-definite matrix, H , and Q such that $Q + Q^T$ is a diagonal matrix with entries

$$(Q + Q^T)_{i,j} = \begin{cases} -1 & \text{if } i = j = 0, \\ 1 & \text{if } i = j = n_x, \\ 0 & \text{otherwise.} \end{cases} \quad (3.193)$$

This formula is the discrete analogue to the standard integration-by-parts formula, which we can write as

$$\int_a^b u(x)v'(x)dx + \int_a^b v(x)u'(x) = u(b)v(b) - u(a)v(a). \quad (3.194)$$

Here, we consider vectors $\mathbf{u}$ and $\mathbf{v}$ that correspond to nodal values approximating $u(x)$ and $v(x)$, respectively, and the integrals are approximated by inner products in the norm induced by H , giving the discrete analogue of equation (3.194) as

$$\mathbf{u}^T H D \mathbf{v} + \mathbf{v}^T H D \mathbf{u} = u_{n_x} v_{n_x} - u_0 v_0. \quad (3.195)$$

Rewriting the left-hand side of this using $HD = Q$ leads to the condition in equation (3.193). On a uniform grid, this can be satisfied, for example, by taking H to be a diagonal matrix with

$$H_{i,i} = \begin{cases} h/2 & i = 0 \text{ or } i = n_x, \\ h & \text{otherwise} \end{cases} \quad (3.196)$$

and using the centered-difference approximation to define D at all interior mesh points along with the first-order approximations $(u_1 - u_0)/h$ for $u'(a)$ and $(u_{n_x} - u_{n_x-1})/h$ for $u'(b)$. These ideas can be further developed and extended to lead to a summation-by-parts methodology for second derivatives and schemes that give both second-order and higher-order accuracy [168].

3.8 ■ Exercises

1. Consider a uniform grid, $\{x_j\}_{j=0}^{n_x}$, for the interval $[0, 1]$, with grid spacing $h = 1/n_x$. Rederive the $\mathcal{O}(h^2)$ approximation to the second derivative of $u(x)$ at a grid point x_j for $1 \leq j \leq n_x - 1$, including the terms for $u^{(4)}(x_j)$, $u^{(5)}(x_j)$, $u^{(6)}(x_j)$, and $u^{(7)}(x_j)$ in the analysis. To do this, keep all terms up to the eighth derivative of u in your Taylor expansions for $u(x_{j\pm 1})$. Which terms cancel and which do not? Do the same for the centered finite-difference approximation for $u'(x)$, and again notice which terms cancel and which do not.
2. Consider a uniform grid, $\{x_j\}_{j=0}^{n_x}$, for the interval $[0, 1]$, with grid spacing $h = 1/n_x$. Derive an $\mathcal{O}(h^2)$ approximation to the second derivative of $u(x)$ at a grid point x_j for $2 \leq j \leq n_x - 2$ using the points $x_{j\pm 2}$ and x_j , but not the points $x_{j\pm 1}$. Be sure to include enough terms in your derivation to guarantee that the approximation is second order. Consider the function $u(x) = \cos(n_x \pi x)$ on this grid. What are the values of $u(x_j)$ and $u''(x_j)$ evaluated at grid point j ? Is the approximation to $u''(x_j)$ generated by this scheme accurate? Justify your answer using the remainder term(s) from your derivation.
3. Consider a uniform grid, $\{x_j\}_{j=0}^{n_x}$, for the interval $[0, 1]$, with grid spacing $h = 1/n_x$. Derive an $\mathcal{O}(h^2)$ approximation to the fourth derivative of $u(x)$ at a grid point x_j for $2 \leq j \leq n_x - 2$, and include enough terms in the remainder to establish second-order accuracy. How many derivatives of $u(x)$ must be continuous in order for your approximation to be valid? Why can you not use this approximation for $j = 1$ or $j = n_x - 1$?
4. Consider a *nonuniform* grid $\{x_j\}_{j=0}^{n_x}$ for the interval $[0, 1]$, with grid spacings $h_j = x_j - x_{j-1}$. Use the centered finite-difference approximation to approximate $u'(x_{j\pm 1/2})$, and then a centered finite-difference approximation of these two values to approximate $u''(x_j)$. Be sure to include the error expression in terms of higher derivatives of u evaluated at x_j and/or $x_{j\pm 1/2}$.

5. Consider a rectangular tensor-product grid of uniform grids, $\Omega^{h_x} = \{x_i\}_{i=0}^{n_x}$, $\Omega^{h_y} = \{y_j\}_{j=0}^{n_y}$, and $\Omega^{h_z} = \{z_k\}_{k=0}^{n_z}$, in three dimensions. Derive an approximation to the Laplacian of $u(x, y, z)$,

$$-u_{xx}(x, y, z) - u_{yy}(x, y, z) - u_{zz}(x, y, z),$$

at point (x_i, y_j, z_k) using values at that point as well as at $(x_{i\pm 1}, y_j, z_k)$, $(x_i, y_{j\pm 1}, z_k)$, and $(x_i, y_j, z_{k\pm 1})$, assuming that all partial derivatives of $u(x, y, z)$ up to fourth order are continuous. What is the leading-order remainder term for your approximation?

6. Consider the boundary-value problem

$$\begin{aligned} -u''(x) + u(x) &= x \quad \text{for } 0 < x < 1, \\ u(0) &= 0, \\ u(1) &= 1. \end{aligned}$$

First, find the analytical solution to the boundary-value problem. Next, consider a nonuniform grid $\{x_j\}_{j=0}^{n_x}$ and state the centered finite-difference scheme for the problem with this specific right-hand side function, $f(x) = x$. Verify that $u_j = u(x_j)$ (i.e., the analytical solution evaluated at the mesh points) is a solution of the finite-difference scheme.

7. Consider the boundary-value problem

$$\begin{aligned} -u''(x) + u(x) &= x^3 \quad \text{for } 0 < x < 1, \\ u(0) &= 0, \\ u(1) &= 7. \end{aligned}$$

First, find the analytical solution to the boundary-value problem. Next, consider a nonuniform grid $\{x_j\}_{j=0}^{n_x}$ and state the centered finite-difference scheme for the problem with this specific right-hand side function, $f(x) = x^3$. Is $u_j = u(x_j)$ (i.e., the analytical solution evaluated at the mesh points) a solution of the finite-difference scheme? What about on a uniform mesh? Explain your answers by relating this to the truncation error terms derived in this chapter for uniform and nonuniform meshes.

8. Consider a uniform grid $\{x_j\}_{j=0}^{n_x}$ for the interval $[0, 1]$, with grid spacing $h = 1/n_x$, and the problem

$$\begin{aligned} -u''(x) + au'(x) + u(x) &= f(x) \quad \text{for } 0 < x < 1, \\ u(0) &= u(1) = 0. \end{aligned}$$

For the centered finite-difference discretization of the equation

$$\frac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} + a \left(\frac{u_{j+1} - u_{j-1}}{2h} \right) + u_j = f_j, \quad 1 \leq j \leq n_x - 1,$$

state conditions on the coefficient a that guarantee stability in the maximum norm for a given h .

9. Consider the centered finite-difference discretization of

$$-u''(x) = f(x) \quad \text{for } 0 < x < 1,$$

with one Neumann boundary condition and one Dirichlet boundary condition,

$$u'(0) = \alpha,$$

$$u(1) = \beta,$$

on a uniform grid, $\{x_j\}_{j=0}^{n_x}$, with $x_j = jh$ for $h = 1/n_x$. Using the ghost-point approach to discretize the Neumann boundary condition, show that this discretization is stable in the maximum norm. *Hint:* Consider “guessing” that the j th column of the inverse of the discretization matrix takes a constant value at points x_i with $i \leq j$ and is linear over points x_i with $i \geq j$.

10. Consider a *cell-centered discretization* with n_x points on the interval $[0, 1]$, where the approximate solution is calculated at points $x_j = (j - \frac{1}{2})h$ for $h = 1/n_x$ and $1 \leq j \leq n_x$, and not at the nodes of the uniform grid.
- (a) Write down the $\mathcal{O}(h^2)$ discretization of $-u''(x) + u(x) = f(x)$ at an interior point, x_j , for $2 \leq j \leq n_x - 1$.
 - (b) Use a finite-difference scheme on a nonuniform grid to derive the discretization of $-u''(x) + u(x) = f(x)$ at point $x_1 = h/2$, subject to the boundary condition $u(0) = 0$. What is the leading-order term of the truncation error associated with this point?
 - (c) Use a ghost-point finite-difference scheme to derive the discretization of $-u''(x) + u(x) = f(x)$ at point $x_1 = h/2$, subject to the boundary condition $u'(0) = 0$. What is the leading-order term of the truncation error associated with this point?

Chapter 4

Finite-element methods for elliptic differential equations in 1D

Overview

We continue with elliptic PDEs and study the finite-element method, which is a powerful tool for constructing numerical solutions to differential equations. It is commonly used in many application areas, including fluid flow, solid mechanics, and electromagnetics. In this chapter, we present a first introduction to the finite-element method, developing key concepts in the simplified setting of solving the 1D boundary-value problem

$$-u''(x) = f(x) \quad \text{for } x \in (0, 1), \quad (4.1a)$$

$$u(0) = 0, \quad (4.1b)$$

$$u'(1) = 0. \quad (4.1c)$$

The advantage of the 1D setting considered here is that we can analyze many of the properties of the discretization using simple tools from calculus. The topics in this chapter will be revisited in Chapter 8, where you will see the same approach applied in much greater generality, beyond the one-dimensional context of this chapter, but using more powerful tools from functional analysis. There, the focus is on formulating more general results based on the development here—nonetheless, the same key steps in the process will be followed.

Objectives

In this chapter, we introduce the main concepts of the finite-element method in a setting where advanced tools from functional analysis, such as Hilbert spaces, can be avoided. Using the tools developed in this chapter, you will be able to transition from the *functional* description of a finite-element discretization to the *matrix* description, needed for practical solution of finite-element discretizations. In particular, you will be able to

- express a problem like equation (4.1) in the so-called *weak* form of the problem, thereby taking advantage of a view of the problem that loosens requirements on the differentiability of the solution;
- construct a framework for approximating the solution to equation (4.1) using the weak form, and understand basic properties of that approximation; and
- understand how to transition from the weak form to a linear-algebra problem whose solution gives our approximation.

Chapter Outline

Section 4.1 Weak forms presents the strong and weak forms of the differential equation, highlighting when a solution of one is a solution of the other and providing a few basic facts about function spaces.

Section 4.2 Ritz–Galerkin approximation defines the Ritz–Galerkin approximation setting and details the existence and uniqueness of the discrete solution. Several key properties of the Ritz–Galerkin approximation are proven, including an abstract framework for an error bound, telling us about the quality of the approximation to the solution.

Section 4.3 Piecewise polynomial finite elements in 1D defines the concept of a piecewise polynomial approximation space associated with a mesh. Proofs of the approximation properties for piecewise linear and piecewise quadratic finite-element spaces are given.

Section 4.4 Matrix assembly for linear finite elements in 1D discusses how to transition from the functional description of the finite-element discretization to one in terms of matrices and vectors. While there are several feasible approaches to this in one dimension, we emphasize an elementwise construction that will be generalized in Section 8.5.

4.1 ■ Weak forms

Objectives

In this section, you will identify the *strong* and *weak* forms of a differential equation, apply the theorems of Section 2.6 to move between the two forms, and compare the solutions associated with each. A valuable tool in the discussion of the weak form is a Hilbert space (and vector spaces in general). To this end, we also cover some basic definitions and properties of these spaces that will be used throughout the text.

To start, we consider the boundary-value problem

$$-u''(x) = f(x) \quad \text{for } x \in (0, 1), \quad (4.2a)$$

$$u(0) = 0, \quad (4.2b)$$

$$u'(1) = 0. \quad (4.2c)$$

Notice that this problem is defined with a Dirichlet boundary condition at $x = 0$ and a Neumann boundary condition at $x = 1$. This is because these two types of boundary conditions are treated very differently within the finite-element method, and we will keep track of this as we develop the components of the method. The problem defined in equation (4.2) is called “the differential equation” or specifically the “the strong form of the differential equation.” To introduce a *weak* form of the problem, we consider a smooth function $v(x)$ with $v(0) = 0$ and an integral form of equation (4.2),

$$\int_0^1 f(x)v(x) dx = - \int_0^1 u''(x)v(x) dx \quad (4.3a)$$

$$= -u'(x)v(x) \Big|_{x=0}^{x=1} + \int_0^1 u'(x)v'(x) dx \quad (4.3b)$$

$$= \int_0^1 u'(x)v'(x) dx, \quad (4.3c)$$

where the boundary term at $x = 1$ vanishes because $u'(1) = 0$, while at $x = 0$ the boundary term vanishes because $v(0) = 0$.

For notation, we again define the space of square-integrable functions on $[0, 1]$ as

$$L^2([0, 1]) = \left\{ f : [0, 1] \rightarrow \mathbb{R} \left| \int_0^1 f^2(x) dx < \infty \right. \right\}. \quad (4.4)$$

It is important to note that $L^2([0, 1])$ is, in fact, a Hilbert space, with an inner product defined by $\langle u, v \rangle = \int_0^1 u(x)v(x) dx$ and a norm defined by $\|u\|^2 = \langle u, u \rangle$. In addition, we define the bilinear form

$$a(u, v) = \int_0^1 u'(x)v'(x) dx = \langle u', v' \rangle, \quad (4.5)$$

which simplifies the presentation of equation (4.3c). With this, we see that if $u(x)$ satisfies equation (4.2a) along with the boundary conditions in equations (4.2b) and (4.2c), then $a(u, v) = \langle f, v \rangle$ for any function $v(x)$ that satisfies $v(0) = 0$ and for which the integrals are well defined such that the integration-by-parts steps in equation (4.3) are valid. This motivates the space of functions

$$\mathcal{V} = \{ v \in L^2([0, 1]) \mid a(v, v) < \infty, v(0) = 0 \}. \quad (4.6)$$

With this space, we see that if $v \in \mathcal{V}$, then v is necessarily continuous, since $a(v, v)$ cannot be finite if v is discontinuous (see Remark 4.1). In the end, the *weak form of the differential equation* is, then, to find $u \in \mathcal{V}$ such that

$$a(u, v) = \langle f, v \rangle \quad \forall v \in \mathcal{V}. \quad (4.7)$$

Remark 4.1: Equivalence classes of functions

We consider functions in the space $L^2([0, 1])$ in equation (4.6); however, we actually have something more: *equivalence classes* of functions. That is, by $u \in L^2([0, 1])$, we mean that u is an equivalence class of functions or a group of functions that differ in value on at most a set of points of measure zero. When we discuss a function, u , in a space such as $L^2([0, 1])$ or $\mathcal{V}$, we *implicitly assume* that we are considering the simplest function in the equivalence class associated with u . Thus, when we say that $u \in \mathcal{V}$ is continuous, what we really mean is that we are considering the continuous representation of every equivalence class in $\mathcal{V}$. Since functions in $\mathcal{V}$ can only differ on small sets of points (specifically, those of measure zero), this is to be considered as a small concession. As a concrete example, the function u in Figure 4.1.1a represents a continuous function, which is in the same equivalence class as the function in Figure 4.1.1b, which has a removable discontinuity.

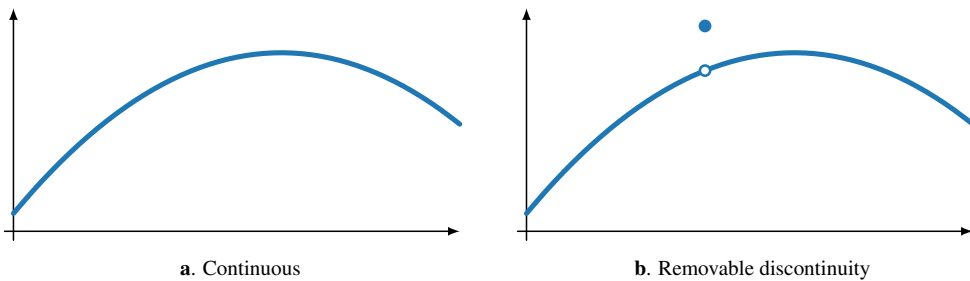


Figure 4.1.1. Example of functions in the same equivalence class.

Since a solution, $u(x)$, of the strong form of the differential equation is guaranteed to be a solution of the weak form, it is natural to ask if the converse holds as well. We note, however, that the definition of $\mathcal{V}$ makes no requirements on the second derivative of $u \in \mathcal{V}$. Consequently,

there is no guarantee that the solution of the weak form is sufficiently smooth to be a solution of the strong form. If the solution to the weak form is sufficiently smooth, then it is also a solution of the strong form. To prove this, we introduce the sets

$$C^k([0, 1]) = \{v \mid v(x) \text{ is } k \text{ times continuously differentiable on } [0, 1]\} \quad (4.8)$$

for all nonnegative integers k .

Theorem 4.2: Weak solution implies strong solution (see also [52, Thm. 0.1.4])

Suppose $f \in C^0([0, 1])$ and $u \in C^2([0, 1]) \cap \mathcal{V}$ satisfy $a(u, v) = \langle f, v \rangle \forall v \in \mathcal{V}$. Then, $u(x)$ satisfies equation (4.2).

Proof. By assumption,

$$\langle f, v \rangle = a(u, v) = \int_0^1 u'(x)v'(x) dx = u'(1)v(1) - \int_0^1 u''(x)v(x) dx \quad (4.9)$$

for any $v \in \mathcal{V} \cap C^1([0, 1])$. This gives

$$\int_0^1 (u'' + f)v dx = \langle u'' + f, v \rangle = 0 \quad (4.10)$$

for all $v \in \mathcal{V} \cap C^1([0, 1])$ that also satisfy $v(1) = 0$. From here, we wish to show that $u'' + f$ necessarily equals zero. To do this, suppose that $w(x) = u''(x) + f(x) \neq 0$ for some $x \in (0, 1)$. We next show that this results in a contradiction.

Take $\hat{x} \in (0, 1)$ to be any point such that $w(\hat{x}) \neq 0$. Then, since w is continuous, we can find a $\delta > 0$ such that $(\hat{x} - \delta, \hat{x} + \delta) \subset (0, 1)$, with $w(x) \neq 0$ and $w(x)$ having the same sign as $w(\hat{x})$ for all x with $|x - \hat{x}| < \delta$. We now define the function $v \in \mathcal{V} \cap C^1([0, 1])$ by

$$v(x) = \begin{cases} (x - (\hat{x} + \delta))^2(x - (\hat{x} - \delta))^2 w(\hat{x}) & \text{if } |x - \hat{x}| < \delta, \\ 0 & \text{otherwise.} \end{cases} \quad (4.11)$$

We note that the construction of $v(x)$ ensures several important properties, including that $v(x) \in C^1([0, 1])$, since both $v(x)$ and $v'(x)$ are defined and equal to zero at $x = \hat{x} \pm \delta$, and that $v(x)$ has the same sign as $w(x)$ over the interval $(\hat{x} - \delta, \hat{x} + \delta)$. In addition, we have that $v(1) = 0$, so $v \in \mathcal{V}$. Thus, we have constructed a function $v \in \mathcal{V} \cap C^1([0, 1])$ such that $\int_0^1 (u'' + f)v dx > 0$.

This construction of $v(x)$ leads to a contradiction with equation (4.10) and the hypothesis that $w(x) = u''(x) + f(x) \neq 0$ for all $x \in (0, 1)$. Consequently, it must be the case that $-u''(x) = f(x)$ for all $x \in (0, 1)$.

To check that u satisfies the boundary conditions, since $u \in \mathcal{V}$ we have $u(0) = 0$. To see that $u'(1) = 0$, consider $v(x) = x$ and observe from equation (4.9) that $\langle f, x \rangle = \int_0^1 (-u'')x dx + u'(1)$. However, since $-u'' = f$ in $(0, 1)$, we have that $\langle f, x \rangle = \int_0^1 (-u'')x dx$. This implies that $u'(1) = 0$. $\square$

Two important observations follow from this result. First, the boundary conditions $u(0) = 0$ and $u'(1) = 0$ are treated differently. The Dirichlet condition, $u(0) = 0$, is called an *essential* boundary condition and must be explicitly included in the definition of $\mathcal{V}$ for the initial setup of the proof. In contrast, the homogeneous Neumann condition, $u'(1) = 0$, is called a *natural*

boundary condition since it is satisfied automatically by the solution of the weak problem, without needing to impose it in the space $\mathcal{V}$. We address this concept in more detail in Chapter 8. Second, the theorem does not offer insight into the existence (or uniqueness) of a solution to the weak form. We introduce general theory to answer this question later on.

While the inner product on $L^2([0, 1])$ arises in many contexts, a second inner product arises in the weak formulation. Note that $\langle u, v \rangle_e = a(u, v)$ defines an inner product on $\mathcal{V}$, which we call the a -inner product or the *energy* inner product. To see that $\langle \cdot, \cdot \rangle_e$ forms an inner product, we verify the standard four conditions for u, v , and $w \in \mathcal{V}$, and for $\alpha, \beta \in \mathbb{R}$:

- $a(u, v) = a(v, u)$,
- $a(\alpha u + \beta w, v) = \alpha a(u, v) + \beta a(w, v)$,
- $a(u, u) \geq 0$, and
- $a(u, u) = 0$ if and only if $u(x) = 0 \forall x$.

The first three conditions hold by standard integration properties, while the last holds because $u \in \mathcal{V}$ must be continuous, since $a(u, u)$ is finite. Since $a(u, v)$ defines an inner product, it also defines a norm on $\mathcal{V}$, defined as $\|v\|_e = \sqrt{a(v, v)}$, which we call the energy norm on $\mathcal{V}$. We note that both inner products satisfy an important inequality, known as the *Cauchy–Schwarz inequality*, that

$$|\langle u, v \rangle| \leq \|u\| \|v\| \text{ and } |\langle u, v \rangle_e| \leq \|u\|_e \|v\|_e. \quad (4.12)$$

For more details, see Section 8.1.4.

4.2 ■ Ritz–Galerkin approximation

Objectives

The goal of this section is to transition from the infinite-dimensional setting of the weak form of a differential equation to the *finite*-dimensional setting; this introduces what is known as a Ritz–Galerkin approximation. Thus, first, you will establish the existence and uniqueness of the Ritz–Galerkin approximation in Theorem 4.5. Then, you will identify two important properties of Ritz–Galerkin approximations that form the basis of the error analysis used in the finite-element method. Furthermore, you will see how Theorem 4.9 allows us to characterize the quality of a finite-element approximation in terms of a tractable property of the underlying finite-element space.

The development of the weak form of a PDE is often independent of considerations for the numerical approximation of the solution and has played a central role in the analysis of PDEs in many settings. In turn, the finite-element method originated from approximate solutions to the weak form by restricting the infinite-dimensional setting of the weak form to that of a finite-dimensional problem. We start by defining the key piece in Definition 4.3, which identifies the finite-dimensional *weak* problem.

Definition 4.3: Ritz–Galerkin approximation

Let $a : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{R}$ be a bilinear form, and let $\mathcal{V}^h$ be a finite-dimensional subspace of $\mathcal{V}$. Consider the weak form restricted to $\mathcal{V}^h$: find $u^h \in \mathcal{V}^h$ such that

$$a(u^h, v) = \langle f, v \rangle \quad \forall v \in \mathcal{V}^h. \quad (4.13)$$

Here, u^h is called the Ritz–Galerkin approximation of the weak solution $u \in \mathcal{V}$.

Example 4.4: Polynomial subspace for the model problem

As an example, consider constructing an approximation of the form

$$\hat{u}(x) = a_0 + a_1x + a_2x^2 + a_3x^3 \quad (4.14)$$

to the 1D model problem in equation (4.2) with solution $u(x) = \sin(\frac{\pi x}{2})$ and $f = \frac{\pi^2}{4} \sin(\frac{\pi x}{2})$, using the weak form in equation (4.7). To begin, observe that imposing boundary condition $\hat{u}(0) = 0$ on $\mathcal{V}$ forces the first coefficient $a_0 = 0$. The monomials x , x^2 , and x^3 span a subspace $\hat{\mathcal{V}}$ of $\mathcal{V}$ and can be used as a basis for constructing $\hat{u}$, although we discuss more suitable bases later in this chapter.

Thus, we are left with three coefficients, determined by three constraints from the weak form in equation (4.7):

$$\left\langle \frac{\partial \hat{u}(x)}{\partial x}, \frac{\partial}{\partial x} x \right\rangle = \langle f, x \rangle, \quad (4.15a)$$

$$\left\langle \frac{\partial \hat{u}(x)}{\partial x}, \frac{\partial}{\partial x} x^2 \right\rangle = \langle f, x^2 \rangle, \quad (4.15b)$$

$$\left\langle \frac{\partial \hat{u}(x)}{\partial x}, \frac{\partial}{\partial x} x^3 \right\rangle = \langle f, x^3 \rangle. \quad (4.15c)$$

Using the form of $\hat{u}(x)$ and f above yields the linear system

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & \frac{4}{3} & \frac{3}{2} \\ 1 & \frac{3}{2} & \frac{9}{5} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{4}{\pi} + 2 \\ -\frac{24}{\pi^2} + 3 \end{bmatrix}. \quad (4.16)$$

Solving for the coefficients in $\hat{u}(x)$ results in

$$\hat{u}(x) = 1.60x - 0.16x^2 - 0.44x^3, \quad (4.17)$$

where the coefficients have been rounded. The error is displayed in Figure 4.2.1.

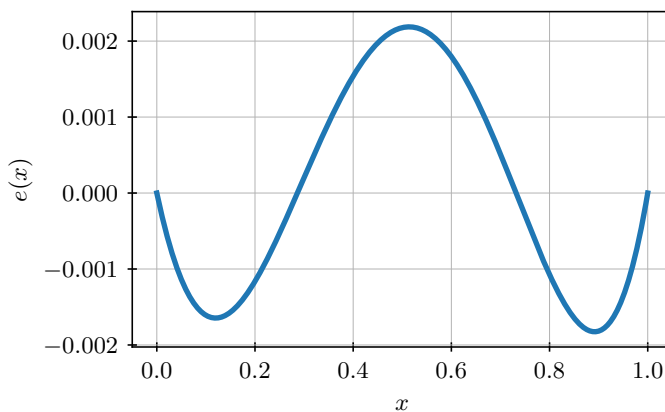


Figure 4.2.1. Approximation error, $e(x) = \sin(\frac{\pi x}{2}) - \hat{u}(x)$, for Example 4.4.

Several natural questions arise when considering the weak form:

- Does such a u^h always exist?
- Is u^h unique?
- How does u^h relate to the exact solution of the differential equation?

To answer these questions, we develop several important theoretical tools, motivated by this 1D setting. Later on, we will see that these tools are directly applicable to more complex problems.

We first consider the important question of existence and uniqueness of the solution to the Ritz–Galerkin approximation in Definition 4.3. To prove this, we make use of the finite-dimensional nature of $\mathcal{V}^h$ to transform the problem into a question of linear algebra. Later on, we will see that the assumption $f \in L^2([0, 1])$ in Theorem 4.5 is somewhat stricter than necessary.

Theorem 4.5: Existence and uniqueness [52, Thm. 0.2.2]

If $f \in L^2([0, 1])$, then there exists a unique solution, u^h , to the Ritz–Galerkin approximation in Definition 4.3 when $\mathcal{V}^h \subset \mathcal{V}$ is finite-dimensional.

Proof. Since $\mathcal{V}^h$ is finite-dimensional, it has a finite-dimensional basis, so we can write $\mathcal{V}^h = \text{span}\{\phi_1, \dots, \phi_n\}$. From this, for any $u^h \in \mathcal{V}^h$ we have $u^h = \sum_{j=1}^n \alpha_j \phi_j$ for coefficients $\alpha = \{\alpha_j\}_{j=1}^n$. The Ritz–Galerkin approximation seeks α such that

$$a\left(\sum_{j=1}^n \alpha_j \phi_j, v\right) = \langle f, v \rangle \quad \forall v \in \mathcal{V}^h. \quad (4.18)$$

Since any $v \in \mathcal{V}^h$ is also a linear combination of the basis functions, this is equivalent to finding α such that

$$a\left(\sum_{j=1}^n \alpha_j \phi_j, \phi_i\right) = \langle f, \phi_i \rangle \quad \text{for } 1 \leq i \leq n, \quad (4.19)$$

where the right-hand side is finite since $f \in L^2([0, 1])$. Since $a(\cdot, \cdot)$ is bilinear, we rewrite this as a system of linear equations for α :

$$\sum_{j=1}^n \alpha_j a(\phi_j, \phi_i) = \langle f, \phi_i \rangle \quad \text{for } 1 \leq i \leq n. \quad (4.20)$$

Defining the $n \times n$ matrix A by $a_{i,j} = a(\phi_j, \phi_i)$ and the n -dimensional vector $\mathbf{f}$ by $f_i = \langle f, \phi_i \rangle$, the Ritz–Galerkin approximation is equivalent to solving the linear system $A\alpha = \mathbf{f}$. The question of existence and uniqueness of u^h is equivalent to asking if this linear system has a unique solution for all $\mathbf{f}$. We show this by contradiction. If $A\alpha = \mathbf{f}$ does not have a unique solution, then A must be singular, meaning that there is a vector $\beta \neq 0$ such that $A\beta = 0$. This requires that $\beta^T A\beta = 0$.

Defining $\hat{v}(x) = \sum_{j=1}^n \beta_j \phi_j(x)$ results in

$$\beta^T A \beta = \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j a(\phi_j, \phi_i) = a \left(\sum_{j=1}^n \beta_j \phi_j, \sum_{i=1}^n \beta_i \phi_i \right) \quad (4.21a)$$

$$= a(\hat{v}, \hat{v}) = \int_0^1 ((\hat{v})')^2 dx. \quad (4.21b)$$

If $\beta^T A \beta = 0$, then we must have that $\int_0^1 ((\hat{v})')^2 dx = 0$, which implies that $\hat{v}'(x) = 0$ for all x . In turn, this means that $\hat{v}(x) = c$ for some constant c . However, since $\hat{v} \in \mathcal{V}^h \subset \mathcal{V}$, it must satisfy $\hat{v}(0) = 0$, which implies that $c = 0$ and, thus, that $\beta = \mathbf{0}$. This is a contradiction, so A must be nonsingular, which implies that $A\alpha = \mathbf{f}$ has a unique solution, establishing the theorem. $\square$

To understand the relationship between the solution of the weak form in equation (4.7) and that of the Ritz–Galerkin approximation, we develop three central results. The first establishes an orthogonality property in the energy inner product.

Lemma 4.6: Orthogonality relation (cf. [52, Eq. (0.3.1)])

If $u \in \mathcal{V}$ is the solution of the weak form in equation (4.7) and $u^h \in \mathcal{V}^h$ is the solution of the Ritz–Galerkin approximation from Definition 4.3, then $a(u - u^h, v) = 0 \forall v \in \mathcal{V}^h$.

Proof. Since u^h comes from the Ritz–Galerkin approximation, it must satisfy $a(u^h, v) = \langle f, v \rangle \forall v \in \mathcal{V}^h$. Similarly, since u is the solution of the weak form, it must satisfy $a(u, v) = \langle f, v \rangle \forall v \in \mathcal{V}$. Noting that $\mathcal{V}^h \subset \mathcal{V}$, this gives $a(u, v) = \langle f, v \rangle \forall v \in \mathcal{V}^h$. Thus,

$$a(u - u^h, v) = a(u, v) - a(u^h, v) = \langle f, v \rangle - \langle f, v \rangle = 0 \quad \forall v \in \mathcal{V}^h. \quad (4.22)$$

$\square$

Using this, we arrive at a second observation, which shows that the Ritz–Galerkin approximation generates the “best approximation” in the subspace $\mathcal{V}^h$ when measured in the energy norm.

Lemma 4.7: Céa’s lemma [52, Thm. 0.3.3]

If $u \in \mathcal{V}$ is the solution of the weak form and $u^h \in \mathcal{V}^h$ is the solution of the Ritz–Galerkin approximation, then $\|u - u^h\|_e = \min_{v \in \mathcal{V}^h} \|u - v\|_e$.

Proof. By the definition of the energy norm, we have

$$\|u - u^h\|_e^2 = a(u - u^h, u - u^h). \quad (4.23)$$

For any $v \in \mathcal{V}^h$, we can rewrite this as

$$a(u - u^h, u - u^h) = a(u - u^h, u - v) + a(u - u^h, v - u^h) \quad (4.24)$$

using the fact that the weak form is bilinear. Note that the second term, though, has $v - u^h \in \mathcal{V}^h$

as its second argument. Thus, by the orthogonality relationship of Lemma 4.6, we have

$$a(u - u^h, u - u^h) = a(u - u^h, u - v) \quad \forall v \in \mathcal{V}^h. \quad (4.25)$$

Now, since $a(\cdot, \cdot)$ is an inner product, we can apply the Cauchy–Schwarz inequality to the right-hand side, arriving at

$$a(u - u^h, u - u^h) = \|u - u^h\|_e^2 \leq \|u - u^h\|_e \|u - v\|_e \quad \forall v \in \mathcal{V}^h. \quad (4.26)$$

Dividing through both sides by $\|u - u^h\|_e$ then gives $\|u - u^h\|_e \leq \|u - v\|_e \quad \forall v \in \mathcal{V}^h$, which is equivalent to the stated result. $\square$

Our final task is to quantify the accuracy of the “best” approximation. In general, this depends on the choice of the finite-dimensional subspace $\mathcal{V}^h$. To this end, we state what is known as an *approximation assumption* and show how this is closely related to how well u^h approximates the solution of the weak form.

Definition 4.8: Approximation assumption (see [52, Eq. (0.3.4)])

Given $\mathcal{V}^h \subset \mathcal{V}$, the approximation assumption for $\mathcal{V}^h$ is that $\exists \epsilon > 0$ such that $\forall w \in C^2([0, 1]) \cap \mathcal{V}$, $\min_{v \in \mathcal{V}^h} \|w - v\|_e \leq \epsilon \|w''\|$.

While quantifying ϵ for a given $\mathcal{V}^h$ remains an important question, the following results show how we can use ϵ to bound the difference between the solution of the weak form, u , and that of the Ritz–Galerkin approximation, u^h . We first establish a relationship between $u - u^h$ in the L^2 and energy norms and then consider a bound in the case that $u \in C^2([0, 1])$.

Theorem 4.9: Aubin–Nitsche duality [52, Thm. 0.3.5]

Let $f \in L^2([0, 1])$, $u \in \mathcal{V}$ be the solution of the weak form and $u^h \in \mathcal{V}^h$ be the solution of the Ritz–Galerkin approximation. If the approximation assumption for $\mathcal{V}^h$ in Definition 4.8 holds, then

$$\|u - u^h\| \leq \epsilon \|u - u^h\|_e. \quad (4.27)$$

Proof. Since $u - u^h \in \mathcal{V} \subset C^0([0, 1])$, let $w \in C^2([0, 1]) \cap \mathcal{V}$ be the solution of the dual equation for the error, where $-w'' = u - u^h$ with boundary conditions $w(0) = 0$, $w'(1) = 0$. Then, w is also the solution of the weak form

$$a(w, v) = \langle u - u^h, v \rangle \quad \forall v \in \mathcal{V}. \quad (4.28)$$

Since $u - u^h \in \mathcal{V}$, we have that

$$\|u - u^h\|^2 = \langle u - u^h, u - u^h \rangle \quad (4.29a)$$

$$= a(w, u - u^h) \quad (4.29b)$$

$$= a(u - u^h, w) - a(u - u^h, v) \quad \forall v \in \mathcal{V}^h, \quad (4.29c)$$

where the last equality follows from the orthogonality relationship of Lemma 4.6. By the Cauchy–Schwarz inequality, this gives

$$\|u - u^h\|^2 \leq \|u - u^h\|_e \|w - v\|_e \quad \forall v \in \mathcal{V}^h. \quad (4.30)$$

Now, taking $v \in \mathcal{V}^h$ to be the minimizer of $\|w - v\|_e$ in Definition 4.8, we can use the approximation assumption to see that

$$\|u - u^h\|^2 \leq \epsilon \|u - u^h\|_e \|w''\|. \quad (4.31)$$

However, we chose w to be the solution of $-w'' = u - u^h$, giving

$$\|u - u^h\|^2 \leq \epsilon \|u - u^h\|_e \|u - u^h\|. \quad (4.32)$$

This simplifies to $\|u - u^h\| \leq \epsilon \|u - u^h\|_e$. $\square$

In turn, the theorem of Aubin–Nitsche leads to an error bound on the finite-dimensional approximation, as shown next.

Corollary 4.10: Error bound (see [52, Thm. 0.3.5])

If, in addition to the assumptions of Theorem 4.9, $f \in C^0([0, 1])$ and $u \in C^2([0, 1])$, then

$$\|u - u^h\| \leq \epsilon \|u - u^h\|_e \leq \epsilon^2 \|f\|. \quad (4.33)$$

Proof. Since $\|u - u^h\|_e = \min_{v \in \mathcal{V}^h} \|u - v\|_e$ by Céa's lemma, the approximation assumption on $\mathcal{V}^h$ for $u \in C^2([0, 1])$ gives $\|u - u^h\|_e \leq \epsilon \|u''\|$. From Theorem 4.9,

$$\|u - u^h\| \leq \epsilon \|u - u^h\|_e \leq \epsilon^2 \|u''\|. \quad (4.34)$$

Since the weak solution $u \in C^2([0, 1])$, it is also the strong solution of the differential equation $-u'' = f$. Thus, $\|u''\| = \|f\|$. $\square$

With these results, we see that the quality of the approximation of u^h to u depends only on the approximation assumption, which is a property of the subspace $\mathcal{V}^h$. In the next section, we give examples of bounds on ϵ for piecewise linear and piecewise quadratic approximations.

4.3 ■ Piecewise polynomial finite elements in 1D

Objectives

In this section, you will derive approximation assumptions for specific finite-dimensional spaces. In particular, for a fixed mesh in 1D, you will identify how well continuous piecewise linear functions can approximate a given solution and how the approximation property improves with piecewise quadratic functions.

For a given finite-dimensional space, Theorem 4.9 bounds the error of an approximation from that space to the weak solution $u \in \mathcal{V}$ using ϵ . To give specific bounds on ϵ , we fix the finite-dimensional space $\mathcal{V}^h \subset \mathcal{V}$. In the finite-element method, we make two (independent) choices that determine $\mathcal{V}^h$:

1. identifying *elements* or subdivisions of the domain $\Omega = [0, 1]$ and
2. choosing a piecewise polynomial space to represent u^h on each element.

For this, we recall the definition of a mesh from Section 2.3.

Definition 4.11: 1D mesh

A mesh on $[0, 1]$ is determined by a set of nodes (points) satisfying $0 = x_0 < x_1 < x_2 < \dots < x_{n_x} = 1$ that divides $[0, 1]$ into n_x elements, $[x_{i-1}, x_i]$ for $1 \leq i \leq n_x$. The mesh spacing is defined by $h_i = x_i - x_{i-1}$ for $1 \leq i \leq n_x$.

With this mesh, we define piecewise polynomial spaces as follows.

Definition 4.12: Piecewise polynomials

For integer $k \geq 1$, define

$$\mathcal{V}_k^h = \left\{ u \in C^0(\Omega) \mid \begin{array}{l} u(x) \text{ is a polynomial of degree at most } k \\ \text{on each interval } [x_{i-1}, x_i] \text{ and } u(0) = 0 \end{array} \right\}. \quad (4.35)$$

As a direct consequence of this definition, we have $\mathcal{V}_k^h \subset \mathcal{V}$. A natural question to ask is whether the definition of $\mathcal{V}_k^h$ extends to $k = 0$, giving piecewise constant functions that are discontinuous at the nodes of the mesh. While this space of piecewise-constant functions is well defined, such functions are not in $\mathcal{V}$, since their first derivatives are not square integrable. Thus, $a(v, v)$ is not finite for a discontinuous piecewise constant function v . However, we will consider these functions in other contexts, for example, in Chapter 10.

The space defined by $k = 1$, $\mathcal{V}_1^h$, is the space of continuous piecewise linear functions. In contrast to piecewise constants, we can determine a function $u \in \mathcal{V}_1^h$ by its values at the nodes, $\{x_i\}_{i=1}^{n_x}$ (recalling that $u(0) = 0$ for any function in $\mathcal{V}$). Thus, we define a basis for $\mathcal{V}_1^h$ using basis functions $\phi_i(x)$ that satisfy the property

$$\phi_i(x_j) = \delta_{i,j} \quad (4.36)$$

for all grid points x_j . These are called *nodal basis functions*. For $1 \leq i < n_x$, we define them as

$$\phi_i(x) = \begin{cases} \frac{x-x_{i-1}}{x_i-x_{i-1}} & \text{for } x \in [x_{i-1}, x_i], \\ \frac{x_{i+1}-x}{x_{i+1}-x_i} & \text{for } x \in [x_i, x_{i+1}], \\ 0 & \text{otherwise.} \end{cases} \quad (4.37)$$

The last basis function is given by

$$\phi_{n_x}(x) = \begin{cases} \frac{x-x_{n_x-1}}{x_{n_x}-x_{n_x-1}} & \text{for } x \in [x_{n_x-1}, x_{n_x}], \\ 0 & \text{otherwise.} \end{cases} \quad (4.38)$$

Note that we do not need to define a nodal basis function, ϕ_0 , with $\phi_0(x_0) = 1$, due to the boundary condition $u(0) = 0$ that is strongly enforced on $\mathcal{V}$. The basis functions ϕ_i , $i = 1, \dots, n_x$, are called *compactly supported* because they are nonzero only in a compact (i.e., bounded) interval centered on x_i . Due to their shape, they are sometimes called *hat functions* or *tent functions* (see Figure 4.3.1 below).

From here, we check that $a(\phi_i, \phi_i) < \infty$ for $1 \leq i \leq n_x$ and again see that $\mathcal{V}_1^h = \text{span}\{\phi_1, \dots, \phi_{n_x}\}$. In other words, these functions define a basis for $\mathcal{V}_1^h$, and $\mathcal{V}_1^h$ is a subset of $\mathcal{V}$. With this basis, for any function $u \in \mathcal{V}_1^h$, we write

$$u(x) = \sum_{i=1}^{n_x} u(x_i) \phi_i(x). \quad (4.39)$$

As a result, each $u \in \mathcal{V}_1^h$ can be thought of as being defined by its values at the n_x nodes $\{x_i\}_{i=1}^{n_x}$.

To show that an approximation property as in Definition 4.8 holds for $\mathcal{V}_1^h$, we must show that for any $w \in C^2([0, 1])$, $\exists w^h \in \mathcal{V}_1^h$ such that $\|w - w^h\|_e \leq \epsilon \|w''\|$. In most settings in finite-element theory, we prove such a relation for the *interpolant* of w .

Definition 4.13: Interpolant [52, Def. 0.4.3]

Let $w \in \mathcal{V}$. Given a mesh on $[0, 1]$, the (piecewise linear) interpolant of w is the function $w_I^h(x) \in \mathcal{V}_1^h$ given by $w_I^h(x) = \sum_{i=1}^{n_x} w(x_i) \phi_i(x)$, where x_i are the nodes of the mesh.

From this definition, we see that if $w \in \mathcal{V}_1^h$, then $w = w_I^h$ —i.e., any function in $\mathcal{V}_1^h$ is its own interpolant. In contrast, the interpolant of any function in $\mathcal{V}$ is simply determined by its values at the nodes.

We now bound $\|w - w_I^h\|_e$, proving the needed approximation property in terms of the maximum value of h_i for $1 \leq i \leq n_x$.

Theorem 4.14: Approximation property for $\mathcal{V}_1^h$ (cf. [52, Thm. 0.4.5])

Given a mesh on $[0, 1]$, let $h = \max_{1 \leq i \leq n_x} h_i$. Given $w \in C^2([0, 1]) \cap \mathcal{V}$, let w_I^h be the piecewise linear interpolant of w given by Definition 4.13. Then,

$$\|w - w_I^h\|_e \leq \frac{h}{\sqrt{2}} \|w''\|. \quad (4.40)$$

Proof. We prove this result by first “localizing” the bound to a single element, $[x_{i-1}, x_i]$, noting that

$$\|w - w_I^h\|_e^2 = \sum_{i=1}^{n_x} \int_{x_{i-1}}^{x_i} \left([w(x) - w_I^h(x)]' \right)^2 dx \quad (4.41)$$

and

$$\|w''\|^2 = \sum_{i=1}^{n_x} \int_{x_{i-1}}^{x_i} (w''(x))^2 dx = \sum_{i=1}^{n_x} \int_{x_{i-1}}^{x_i} \left([w(x) - w_I^h(x)]'' \right)^2 dx, \quad (4.42)$$

since $(w_I^h)'' = 0$ on (x_{i-1}, x_i) . From this, the stated result follows if we show that

$$\int_{x_{i-1}}^{x_i} \left([w(x) - w_I^h(x)]' \right)^2 dx \leq \frac{(x_i - x_{i-1})^2}{2} \int_{x_{i-1}}^{x_i} \left([w(x) - w_I^h(x)]'' \right)^2 dx. \quad (4.43)$$

To simplify notation, we write $e(x) = w(x) - w_I^h(x)$. Then, equation (4.43) becomes

$$\int_{x_{i-1}}^{x_i} (e'(x))^2 dx \leq \frac{(x_i - x_{i-1})^2}{2} \int_{x_{i-1}}^{x_i} (e''(x))^2 dx. \quad (4.44)$$

We prove this inequality by changing variables, mapping the interval $[x_{i-1}, x_i]$ onto $[0, 1]$. Let $x = x_{i-1} + s(x_i - x_{i-1})$ for $s \in [0, 1]$, and take $\hat{e}(s) = e(x_{i-1} + s(x_i - x_{i-1}))$. Using the chain rule, we then arrive at

$$\frac{d\hat{e}}{ds} = (x_i - x_{i-1}) e'(x_{i-1} + s(x_i - x_{i-1})), \quad (4.45a)$$

$$\frac{d^2\hat{e}}{ds^2} = (x_i - x_{i-1})^2 e''(x_{i-1} + s(x_i - x_{i-1})). \quad (4.45b)$$

Changing variables from x to s in the integrals in equation (4.43) then shows that equation (4.43) holds if

$$\int_0^1 \left[\frac{d\hat{e}}{ds} \right]^2 ds \leq \frac{1}{2} \int_0^1 \left[\frac{d^2\hat{e}}{ds^2} \right]^2 ds. \quad (4.46)$$

Proving this bound has the advantage that it no longer depends on h_i . Also, since the interpolant matches w at the nodes, $e(x_{i-1}) = e(x_i) = 0$, so $\hat{e}(0) = \hat{e}(1) = 0$.

Since w and its interpolant are both continuous and differentiable, as is the change of variables formula, $\hat{e}$ is a continuous function on $[0, 1]$ that is differentiable on $(0, 1)$. By Rolle's theorem, $\exists \xi \in (0, 1)$ such that $\hat{e}'(\xi) = 0$. From this, $\hat{e}'(s) = \int_\xi^s \hat{e}''(t) dt$ for any $s \in [0, 1]$. This allows us to use the Cauchy–Schwarz inequality to bound $|\hat{e}'(s)|$ as

$$|\hat{e}'(s)| = \left| \int_\xi^s \hat{e}''(t) dt \right| \quad (4.47a)$$

$$\leq \left(\left| \int_\xi^s 1^2 dt \right| \right)^{\frac{1}{2}} \left(\int_\xi^s [\hat{e}''(t)]^2 dt \right)^{\frac{1}{2}} \leq |s - \xi|^{\frac{1}{2}} \left(\int_0^1 [\hat{e}''(t)]^2 dt \right)^{\frac{1}{2}}. \quad (4.47b)$$

Squaring both sides and integrating with respect to s gives

$$\int_0^1 [\hat{e}'(s)]^2 ds \leq \left(\int_0^1 |s - \xi| ds \right) \left(\int_0^1 [\hat{e}''(s)]^2 ds \right), \quad (4.48)$$

where we have changed the variable of integration in the last integral for consistency. Noting that

$$\int_0^1 |s - \xi| ds = \xi^2 - \xi + \frac{1}{2} \leq \frac{1}{2} \quad (4.49)$$

for all $\xi \in (0, 1)$ then gives

$$\int_0^1 [\hat{e}'(s)]^2 ds \leq \frac{1}{2} \int_0^1 [\hat{e}''(s)]^2 ds. \quad (4.50)$$

This implies equation (4.43), from which the stated result follows. $\square$

With this result and Corollary 4.10, we summarize the approximation property of $\mathcal{V}_1^h$ in the following.

Corollary 4.15: Error bound for $\mathcal{V}_1^h$

Let $f \in C^0([0, 1])$ and $u \in C^2([0, 1]) \cap \mathcal{V}$. Let u^h be the Ritz–Galerkin approximation of $u \in \mathcal{V}$ over $\mathcal{V}_1^h$, defined by $a(u^h, v) = \langle f, v \rangle \forall v \in \mathcal{V}_1^h$. Then,

$$\|u - u^h\| \leq \frac{h}{\sqrt{2}} \|u - u^h\|_e \leq \frac{h^2}{2} \|f\|. \quad (4.51)$$

By noting that $\mathcal{V}_1^h \subset \mathcal{V}_k^h$ for all $k > 1$, we see that the piecewise linear interpolant is an element of each subspace $\mathcal{V}_k^h$, $k \geq 1$. This leads to two immediate consequences of Theorem 4.14. The first establishes an approximation property for the subspace $\mathcal{V}_k^h$.

Corollary 4.16: Approximation property for $\mathcal{V}_k^h$

Let $h = \max_{1 \leq i \leq n_x} h_i$, $w \in C^2([0, 1]) \cap \mathcal{V}$. Then,

$$\min_{v \in \mathcal{V}_k^h} \|w - v\|_e \leq \frac{h}{\sqrt{2}} \|w''\|. \quad (4.52)$$

Second, we can now derive error bounds for the general case of approximation spaces $\mathcal{V}_k^h$, $k \geq 1$.

Corollary 4.17: Error bound for $\mathcal{V}_k^h$

Let $f \in L^2([0, 1])$, $u \in \mathcal{V}$ be the solution of the weak form and $u^h \in \mathcal{V}_k^h$ be the Ritz–Galerkin approximation of u . Then,

$$\|u - u^h\| \leq \frac{h}{\sqrt{2}} \|u - u^h\|_e. \quad (4.53)$$

If, additionally, $f \in C^0([0, 1])$ and $u \in C^2([0, 1])$, then

$$\|u - u^h\| \leq \frac{h}{\sqrt{2}} \|u - u^h\|_e \leq \frac{h^2}{2} \|f\|. \quad (4.54)$$

The advantage of the spaces of “higher-order” finite elements, $\mathcal{V}_k^h$ for $k > 1$, comes from the fact that we can prove a stronger bound than that in Theorem 4.14 by using the interpolant in the higher-order space, with a higher order of convergence. To illustrate, we prove the analogous result to Theorem 4.14 for piecewise quadratic polynomials ($k = 2$) on the same mesh—similar results hold for polynomial approximation spaces with $k > 2$.

While the basis we used for $\mathcal{V}_1^h$ is naturally defined by the mesh using nodal basis functions, we cannot directly use this approach for $\mathcal{V}_2^h$, since the function values at the mesh points only provide two conditions of the three needed to specify a quadratic polynomial on each element. While a specific choice of basis is not needed for the following result, it is helpful to think about defining a nodal basis for $\mathcal{V}_2^h$ by using the midpoint of each element, $x_{i-\frac{1}{2}} = (x_{i-1} + x_i)/2$ for $1 \leq i \leq n_x$, to specify the third function value. This gives basis functions

$$\phi_i(x) = \begin{cases} \frac{(x-x_{i-1})(x-x_{i-\frac{1}{2}})}{(x_i-x_{i-1})(x_i-x_{i-\frac{1}{2}})} & \text{for } x \in [x_{i-1}, x_i], \\ \frac{(x_{i+1}-x)(x_{i+\frac{1}{2}}-x)}{(x_{i+1}-x_i)(x_{i+\frac{1}{2}}-x_i)} & \text{for } x \in [x_i, x_{i+1}], \\ 0 & \text{otherwise} \end{cases} \quad (4.55)$$

associated with each node $1 \leq i < n_x$ of the mesh, as well as

$$\phi_{n_x}(x) = \begin{cases} \frac{(x-x_{n_x-1})(x-x_{n_x-\frac{1}{2}})}{(x_{n_x}-x_{n_x-1})(x_{n_x}-x_{n_x-\frac{1}{2}})} & \text{for } x \in [x_{n_x-1}, x_{n_x}], \\ 0 & \text{otherwise} \end{cases} \quad (4.56)$$

associated with the node of the mesh at $x_{n_x} = 1$. Furthermore, associated with the midpoint of each element $1 \leq i \leq n_x$, we have the basis functions

$$\phi_{i-\frac{1}{2}}(x) = \begin{cases} \frac{(x-x_{i-1})(x_i-x)}{(x_{i-\frac{1}{2}}-x_{i-1})(x_{i-\frac{1}{2}}-x_{i-\frac{1}{2}})} & \text{for } x \in [x_{i-1}, x_i], \\ 0 & \text{otherwise.} \end{cases} \quad (4.57)$$

Note that this is a nodal basis satisfying $\phi_i(x_j) = \delta_{i,j}$ if one includes the midpoints as nodes and considers half-integer indices. This is shown in Figure 4.3.1.

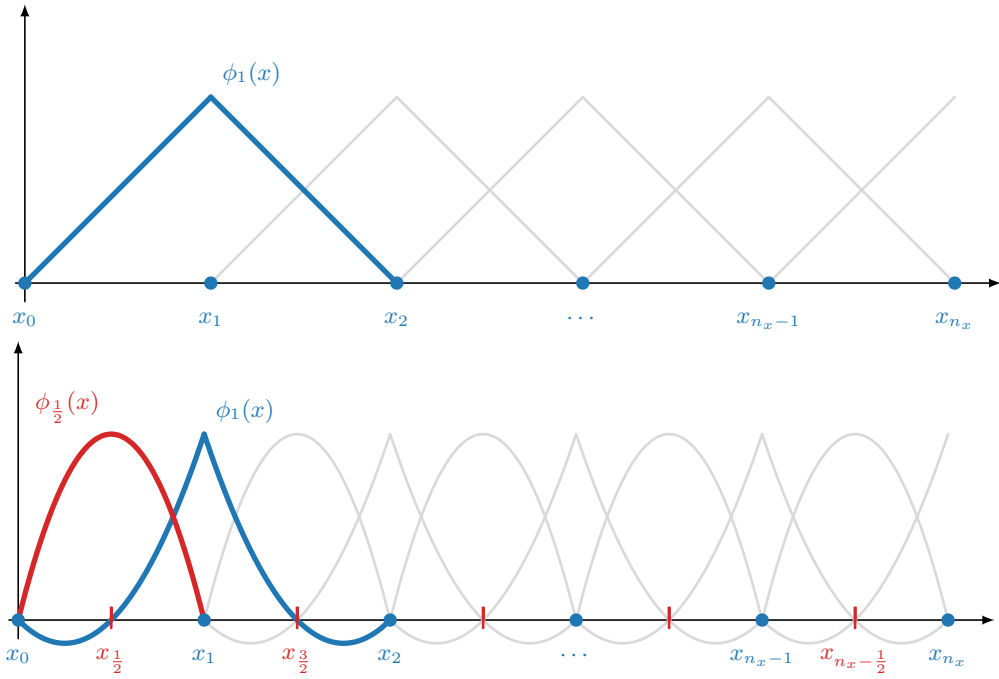


Figure 4.3.1. (top) Linear and (bottom) quadratic basis functions.

With this basis, we define the (piecewise quadratic) interpolant in $\mathcal{V}_2^h$ of a function $w \in \mathcal{V}$ as

$$w_I^h = \sum_{i=1}^{n_x} w(x_i) \phi_i(x) + \sum_{i=1}^{n_x} w(x_{i-\frac{1}{2}}) \phi_{i-\frac{1}{2}}(x). \quad (4.58)$$

It is this higher-order interpolant that provides a better approximation than the piecewise linear approximation considered in Theorem 4.14, under the assumption that w is sufficiently smooth. While the following proof repeats many of the same arguments seen there, we present it in its entirety for sake of clarity.

Theorem 4.18: Approximation property for $\mathcal{V}_2^h$

Let $h = \max_{1 \leq i \leq n_x} h_i$, $w \in C^3([0, 1]) \cap \mathcal{V}$. Then,

$$\|w - w_I^h\|_e \leq \frac{h^2}{2} \|w'''\|. \quad (4.59)$$

Proof. We prove this result by first “localizing” the bound to a single element, $[x_{i-1}, x_i]$, noting that

$$\|w - w_I^h\|_e^2 = \sum_{i=1}^{n_x} \int_{x_{i-1}}^{x_i} \left([w(x) - w_I^h(x)]' \right)^2 dx \quad (4.60)$$

and

$$\|w'''\|^2 = \sum_{i=1}^{n_x} \int_{x_{i-1}}^{x_i} (w'''(x))^2 dx = \sum_{i=1}^{n_x} \int_{x_{i-1}}^{x_i} ([w(x) - w_I^h(x)]''')^2 dx, \quad (4.61)$$

since $(w_I^h)''' = 0$ on (x_{i-1}, x_i) . From this, the stated result follows if we can show that

$$\int_{x_{i-1}}^{x_i} ([w(x) - w_I^h(x)]')^2 dx \leq \frac{(x_i - x_{i-1})^4}{4} \int_{x_{i-1}}^{x_i} ([w(x) - w_I^h(x)]''')^2 dx. \quad (4.62)$$

To simplify notation, we write $e(x) = w(x) - w_I^h(x)$. Then, rewriting equation (4.62) results in

$$\int_{x_{i-1}}^{x_i} (e'(x))^2 dx \leq \frac{(x_i - x_{i-1})^4}{4} \int_{x_{i-1}}^{x_i} (e'''(x))^2 dx. \quad (4.63)$$

We prove this inequality by changing variables, mapping the interval $[x_{i-1}, x_i]$ onto $[0, 1]$. Let $x = x_{i-1} + s(x_i - x_{i-1})$ for $s \in [0, 1]$, and take $\hat{e}(s) = e(x_{i-1} + s(x_i - x_{i-1}))$. Using the chain rule, we then get

$$\frac{d\hat{e}}{ds} = (x_i - x_{i-1})e'(x_{i-1} + s(x_i - x_{i-1})), \quad (4.64a)$$

$$\frac{d^2\hat{e}}{ds^2} = (x_i - x_{i-1})^2 e''(x_{i-1} + s(x_i - x_{i-1})), \quad (4.64b)$$

$$\frac{d^3\hat{e}}{ds^3} = (x_i - x_{i-1})^3 e'''(x_{i-1} + s(x_i - x_{i-1})). \quad (4.64c)$$

Changing variables from x to s in the integrals in equation (4.62) then shows that equation (4.62) holds if

$$\int_0^1 \left[\frac{d\hat{e}}{ds} \right]^2 ds \leq \frac{1}{4} \int_0^1 \left[\frac{d^3\hat{e}}{ds^3} \right]^2 ds. \quad (4.65)$$

Proving this bound has the advantage that it no longer depends on h_i . In addition, since the interpolant matches w at the nodes, $e(x_{i-1}) = e(x_{i-\frac{1}{2}}) = e(x_i) = 0$, we have $\hat{e}(0) = \hat{e}(\frac{1}{2}) = \hat{e}(1) = 0$.

Since w and its interpolant are both continuous and twice differentiable, as is the change of variables formula, $\hat{e}$ is a continuous function on $[0, 1]$ with continuous first and second derivatives on $(0, 1)$. We apply Rolle's theorem twice, first to see that $\exists \xi_1 \in (0, \frac{1}{2})$ and $\xi_2 \in (\frac{1}{2}, 1)$ such that $\hat{e}'(\xi_1) = \hat{e}'(\xi_2) = 0$, and then to see that $\exists \xi_3 \in (\xi_1, \xi_2)$ such that $\hat{e}''(\xi_3) = 0$. Then, $\hat{e}''(s) = \int_{\xi_3}^s \hat{e}'''(t) dt$ for any $s \in [0, 1]$. This allows us to use the Cauchy–Schwarz inequality to bound $|\hat{e}''(s)|$ as

$$|\hat{e}''(s)| = \left| \int_{\xi_3}^s \hat{e}'''(t) dt \right| \quad (4.66a)$$

$$\leq \left(\int_{\xi_3}^s 1^2 dt \right)^{\frac{1}{2}} \left(\int_{\xi_3}^s [\hat{e}'''(t)]^2 dt \right)^{\frac{1}{2}} \leq |s - \xi_3|^{\frac{1}{2}} \left(\int_0^1 [\hat{e}'''(t)]^2 dt \right)^{\frac{1}{2}}. \quad (4.66b)$$

Squaring both sides and integrating with respect to s gives

$$\int_0^1 [\hat{e}''(s)]^2 ds \leq \left(\int_0^1 |s - \xi_3| ds \right) \left(\int_0^1 [\hat{e}'''(s)]^2 ds \right), \quad (4.67)$$

where we have changed the variable of integration in the last integral for consistency. Noting that

$$\int_0^1 |s - \xi_3| ds = \xi_3^2 - \xi_3 + \frac{1}{2} \leq \frac{1}{2} \quad (4.68)$$

for all $\xi_3 \in (0, 1)$ then gives

$$\int_0^1 [\hat{e}''(s)]^2 ds \leq \frac{1}{2} \int_0^1 [\hat{e}'''(s)]^2 ds. \quad (4.69)$$

Repeating this using ξ_1 (or ξ_2) yields a similar bound that

$$\int_0^1 [\hat{e}'(s)]^2 ds \leq \frac{1}{2} \int_0^1 [\hat{e}''(s)]^2 ds, \quad (4.70)$$

giving

$$\int_0^1 [\hat{e}'(s)]^2 ds \leq \frac{1}{4} \int_0^1 [\hat{e}'''(s)]^2 ds. \quad (4.71)$$

This implies equation (4.62), from which the stated result follows. $\square$

Note that this bound does not change the relationship between the L^2 and energy norms of the error $u - u^h$, since that is determined by the duality argument in Theorem 4.9. In that proof, we apply the approximation assumption to the function w that solves $-w'' = u - u^h$. Since we can only guarantee that $u^h \in C^0([0, 1])$, this only allows us to consider $w \in C^2([0, 1])$, and the requirement that $w \in C^3([0, 1])$ in Theorem 4.18 precludes improving this bound. Theorem 4.18 does, however, provide a better bound on the energy norm of the error, which improves L^2 convergence from second to third order. In addition, the bound provides a direct improvement for $\mathcal{V}_k^h$ for $k > 2$, analogous to Corollary 4.17, when $u \in C^3([0, 1])$.

Corollary 4.19: Error bound for $\mathcal{V}_2^h$

Let $f \in L^2([0, 1])$, $u \in \mathcal{V}$ be the solution of the weak form and $u^h \in \mathcal{V}_2^h$ be the Ritz–Galerkin approximation of u . Then,

$$\|u - u^h\| \leq \frac{h}{\sqrt{2}} \|u - u^h\|_e. \quad (4.72)$$

If, additionally, $f \in C^1([0, 1])$ and $u \in C^3([0, 1])$, then

$$\|u - u^h\| \leq \frac{h}{\sqrt{2}} \|u - u^h\|_e \leq \frac{h^3}{2\sqrt{2}} \|u'''\|. \quad (4.73)$$

The proofs of Theorems 4.14 and 4.18 generalize easily to higher-order approximations (larger k) if additional smoothness is assumed on f and u . This is an attractive property of higher-order finite-element methods: when the weak solution, u , is sufficiently smooth, an improved approximation is possible using higher-order elements. As will be discussed later on, however, there are many other important factors to be balanced in practice, comparing the accuracy (determined by the order of the approximation) with the cost of storage for the resulting matrices and of solving the resulting linear systems.

4.4 ■ Matrix assembly for linear finite elements in 1D

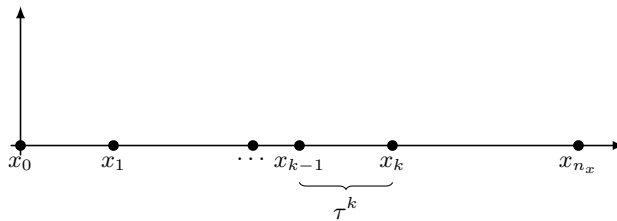
Objectives

The goal of this section is to outline steps needed to construct a 1D finite-element solution. The approach we take is an *element-by-element* construction of the linear system, and a similar approach will be used in higher dimensions, for example, in Chapter 8. Here, you will visualize the basis functions in a 1D element and learn how to form an elementwise view of the weak form. Then, you will understand how to associate basis functions with degrees of freedom and specific entries in a matrix form of the problem, and you will finally construct a three-element example using this process.

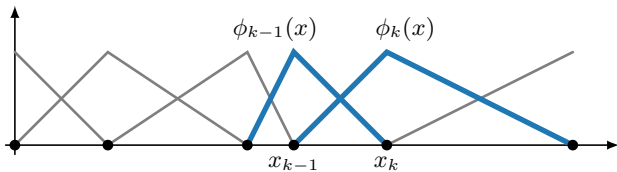
To begin, we consider the model Poisson problem of equation (4.2) with weak form as in equation (4.7): find $u \in \mathcal{V}$ such that

$$\langle u', v' \rangle = \langle f, v \rangle \quad \forall v \in \mathcal{V}, \quad (4.74)$$

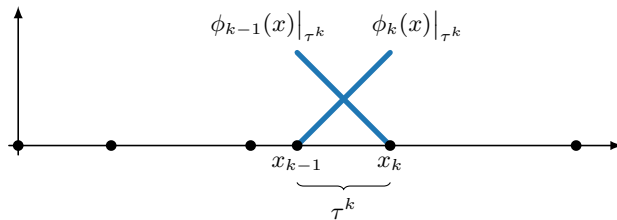
with $\mathcal{V}$ as defined in equation (4.6).



a. Finite-element mesh.



b. Basis functions.



c. Basis functions in element τ^k .

Figure 4.4.1. Finite-element mesh and basis functions.

Consider a 1D grid of points in $\Omega = [0, 1]$ as in Figure 4.4.1a, with node set $\{x_0, \dots, x_{n_x}\}$ and element set $\mathcal{T}^h = \{\tau^1, \dots, \tau^{n_x}\}$, where element $\tau^k = [x_{k-1}, x_k]$.

As a first pass at a discretization of equation (4.74), we follow Section 4.3 and consider the space of continuous, piecewise linear functions on the grid above. That is,

$$\mathcal{V}_1^h = \{u(x) \in C^0([0, 1]) \mid u(x) \text{ is linear for } x \in \tau^k \forall k\}. \quad (4.75)$$

There are many options for a *global* basis for $\mathcal{V}_1^h$, the most common being defined in equations (4.37) and (4.38), as illustrated in Figure 4.4.1b. Here we seek the Ritz–Galerkin approximation $u^h \in \mathcal{V}_1^h$ of the form

$$u^h(x) = \sum_{j=0}^{n_x} u_j \phi_j(x), \quad (4.76)$$

where $u_0 = 0$ since $u(0) = 0$. Then, the Ritz–Galerkin approximation u^h is obtained from

$$\langle (u^h)', \phi_i' \rangle = \langle f, \phi_i \rangle \quad \text{for } i = 1, \dots, n_x. \quad (4.77)$$

To form an element-by-element view of the weak form, we consider the basis functions $\phi_{k-1}(x)$ and ϕ_k restricted to element τ^k (see Figure 4.4.1c), which define the *local* basis functions

$$\lambda_{k-1}(x) = \phi_{k-1}(x)|_{\tau^k} = \frac{x_k - x}{x_k - x_{k-1}} \quad \text{for } x \in \tau^k, \quad (4.78a)$$

$$\lambda_k(x) = \phi_k(x)|_{\tau^k} = \frac{x - x_{k-1}}{x_k - x_{k-1}} \quad \text{for } x \in \tau^k. \quad (4.78b)$$

These local basis functions, $\lambda_{k-1}(x)$ and $\lambda_k(x)$, have several properties that are helpful in constructing the matrix form of the discrete weak problem. Each $\lambda_k(x)$ is linear, but also $\lambda_{k-1}(x) + \lambda_k(x) = 1$, the unit constant function over τ^k . In addition, if we compute the gradient of each basis function, we have

$$\frac{d\lambda_{k-1}}{dx} = -\frac{1}{x_k - x_{k-1}}, \quad (4.79a)$$

$$\frac{d\lambda_k}{dx} = \frac{1}{x_k - x_{k-1}}. \quad (4.79b)$$

Another key observation is that the basis is *local*, meaning that $u^h(x)|_{\tau^k} = \sum_{j=0}^{n_x} u_j \phi_j(x)|_{\tau^k} = u_{k-1} \lambda_{k-1}(x) + u_k \lambda_k(x)$ for $x \in \tau^k$.

We now revisit the 1D weak form of the model problem given by equation (4.74). We rewrite the finite-dimensional subspace $\mathcal{V}_1^h \subset \mathcal{V}$ in slightly more typical notation as

$$\mathcal{V}_1^h = \{\phi(x) \mid \phi(x) \in C^0([0, 1]), \phi(x)|_{\tau} \in P_1(\tau) \text{ for all } \tau \in \mathcal{T}^h, \text{ and } \phi(0) = 0\}, \quad (4.80)$$

where $P_1(\Omega)$ is the set of all polynomials of degree one or less on Ω . This space satisfies the left, or *essential*, boundary condition ($u(0) = 0$), while the weak form naturally includes the constraint imposed by the right boundary condition ($u'(1) = 0$) or *natural* boundary condition. The weak form of the problem is then stated as follows: find $u^h(x) \in \mathcal{V}_1^h$ such that

$$\int_0^1 (u^h)' \phi_i' dx = \int_0^1 f \phi_i dx \quad \text{for } i = 1, \dots, n_x. \quad (4.81)$$

Since $u^h(x) = \sum_{j=1}^{n_x} u_j \phi_j(x)$, the weak form in equation (4.81) leads to

$$\sum_{j=1}^{n_x} u_j \int_0^1 \frac{d\phi_j}{dx} \frac{d\phi_i}{dx} dx = \int_0^1 f \phi_i dx \quad \text{for } i = 1, \dots, n_x. \quad (4.82)$$

The *element-by-element* view of this expression is then

$$\sum_{j=1}^{n_x} u_j \underbrace{\sum_{\tau^k \in \mathcal{T}^h} \int_{\tau^k} \frac{d\phi_j}{dx} \frac{d\phi_i}{dx} dx}_{a_{i,j}^k} = \underbrace{\sum_{\tau^k \in \mathcal{T}^h} \int_{\tau^k} f \phi_i dx}_{f_i^k} \quad \text{for } i = 1, \dots, n_x, \quad (4.83)$$

where $a_{i,j}$ and f_i denote the matrix and vector entries in the resulting linear system,

$$A\mathbf{u} = \mathbf{f}. \quad (4.84)$$

The expression in equation (4.83) also introduces the elementwise contributions to the matrix and vector, denoted $a_{i,j}^k$ and f_i^k . These are also called the local or elementwise stiffness matrix and right-hand side, respectively.

The full matrix can be assembled by inserting the elementwise contributions into the global matrix. For this, it is helpful to write out the explicit forms of A^k and $\mathbf{f}^k$ in element τ^k :

$$A^k = \begin{bmatrix} \int_{\tau^k} \frac{d\phi_{k-1}}{dx} \frac{d\phi_{k-1}}{dx} dx & \int_{\tau^k} \frac{d\phi_{k-1}}{dx} \frac{d\phi_k}{dx} dx \\ \int_{\tau^k} \frac{d\phi_k}{dx} \frac{d\phi_{k-1}}{dx} dx & \int_{\tau^k} \frac{d\phi_k}{dx} \frac{d\phi_k}{dx} dx \end{bmatrix}, \quad (4.85a)$$

$$\mathbf{f}^k = \begin{bmatrix} \int_{\tau^k} f \phi_{k-1} dx \\ \int_{\tau^k} f \phi_k dx \end{bmatrix}. \quad (4.85b)$$

How these values get accumulated into the global matrix is straightforward in 1D, but it is worth discussing to set the stage for the more complicated case in Section 8.5. The key additional ingredient is known as the local-to-global map. For each element τ^k , we also define a matrix P^k , with entries given by selecting the columns of the global identity matrix that correspond to the degrees of freedom on element τ^k , with order chosen so that the j th column of P^k is the column of the global identity that corresponds to the j th basis function on τ^k . In this setting, these are just $n_x \times 2$ matrices, and the first column of P^k is column $k-1$ of the $n_x \times n_x$ identity matrix, while the second column of P^k is column k of the $n_x \times n_x$ identity matrix. With this, we have the simple relation that $A = \sum_{\tau^k \in \mathcal{T}^h} P^k A^k (P^k)^T$, while $\mathbf{f} = \sum_{\tau^k \in \mathcal{T}^h} P^k \mathbf{f}^k$. This corresponds to the summations on the left- and right-hand sides of equation (4.83), with $a_{i,j} = \sum_{\tau^k \in \mathcal{T}^h} a_{i,j}^k$, where $a_{i,j}^k$ is the entry in position (i,j) of matrix $P^k A^k (P^k)^T$, and $f_i = \sum_{\tau^k \in \mathcal{T}^h} f_i^k$, where f_i^k is the i th entry in $P^k \mathbf{f}^k$. To keep the notation general, we write A as an $n \times n$ matrix with $n = n_x$ in this example.

Example 4.20: Example with three elements

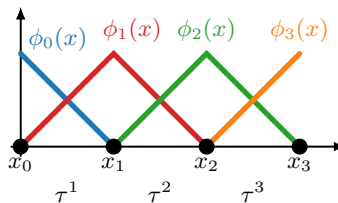


Figure 4.4.2. Basis functions on three elements.

Consider the 1D uniform mesh on $[0, 1]$ with $h = 1/3$ (see Figure 4.4.2), and let $f(x) = 1$.

From above, we have

$$\int_{\tau^k} \phi'_{k-1}(x) \phi'_{k-1}(x) dx = \frac{1}{h}, \quad (4.86a)$$

$$\int_{\tau^k} \phi'_{k-1}(x) \phi'_k(x) dx = -\frac{1}{h}, \text{ and} \quad (4.86b)$$

$$\int_{\tau^k} 1 \phi_{k-1}(x) dx = \frac{h}{2}, \quad (4.86c)$$

which yield identical element stiffness matrices and element right-hand sides of

$$A^k = \frac{1}{h} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad \text{and} \quad f^k = \frac{h}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{ for all } k. \quad (4.87)$$

To assemble the resulting matrix, the element contributions are of the form $P^k A^k (P^k)^T$, which just “shifts” the positioning of the 2×2 element matrix down the diagonal:

$$A = \frac{1}{h} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \frac{1}{h} \begin{bmatrix} & 1 & -1 \\ & -1 & 1 \end{bmatrix} + \frac{1}{h} \begin{bmatrix} & & 1 & -1 \\ & & -1 & 1 \end{bmatrix} \quad (4.88a)$$

$$= \frac{1}{h} \begin{bmatrix} 1 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ & & -1 & 1 \end{bmatrix}. \quad (4.88b)$$

Likewise, the right-hand side becomes

$$f = \frac{h}{2} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} + \frac{h}{2} \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} + \frac{h}{2} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad (4.89a)$$

$$= \frac{h}{2} \begin{bmatrix} 1 \\ 2 \\ 2 \\ 1 \end{bmatrix}. \quad (4.89b)$$

Finally, setting $u_0 = 0$ reduces the system to

$$\frac{1}{h} \begin{bmatrix} 2 & -1 & \\ -1 & 2 & -1 \\ & -1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \frac{h}{2} \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}. \quad (4.90)$$

4.5 ■ Some further thoughts

In this chapter, we have taken advantage of a simple problem to present the theory and construction of finite-element approximations in 1D, leaving many of the more technical details for later chapters. It is worth noting, however, that the *secret sauce* of finite elements is largely in the selection of the *basis* for the function space, to which we have given a bit more attention in the

final sections of this chapter. Here, we have considered continuous piecewise linear or piecewise quadratic basis functions (see Figure 4.3.1, for example). On a single element, this will be a poor approximation if the solution varies a lot, so we rely on having a mesh made up of small intervals of size h in order to refine the approximation. This is called the h version of the finite-element method, symbolizing that we improve our resolution by making h smaller.

We can, however, generalize this notion, using the same process as Ritz–Galerkin. First, consider a homogeneous Dirichlet problem (although this readily extends to other settings, including the mixed boundary conditions considered earlier):

$$-u''(x) = f(x) \quad \text{for } x \in (0, 1), \quad (4.91a)$$

$$u(0) = u(1) = 0. \quad (4.91b)$$

Here, we consider only a single element, fixing $h = 1.0$. Similar to the construction with hat functions in earlier sections, we seek an approximation $u^p(x)$ to $u(x)$,

$$u^p(x) = \sum_{i=1}^p \alpha_i \phi_i(x), \quad (4.92)$$

where α_i are the coefficients of the expansion of $u^p(x)$ in terms of the basis set $\{\phi_i(x)\}_{i=1}^p$. Notably, we have no reference to h , leaving the index over i to range over a p -dimensional space.

One option for the set of $\phi_i(x)$ that satisfies the homogeneous boundary conditions is to take

$$\phi_i(x) = x^i(1-x) = x^i - x^{i+1}, \quad (4.93)$$

resulting in a basis that spans the space of at most $(p+1)$ -degree polynomials on the unit interval. Recalling equation (4.7), we have

$$a(\phi_j, \phi_i) = \int_0^1 \phi_i' \phi_j' dx \quad (4.94a)$$

$$= \frac{ij}{i+j-1} - \frac{2ij+i+j}{i+j} + \frac{ij+i+j+1}{i+j+1}, \quad i = 1 \dots, p, \quad (4.94b)$$

resulting in a matrix problem of $A\alpha = f$ where $a_{i,j} = a(\phi_j, \phi_i)$, since we have only one element. This is called the p version of the finite-element method. In practice, the two versions are often combined, resulting in an hp version, where both polynomial order and element size are varied over the mesh. Despite the improvement in accuracy that we would expect with higher-order polynomials, from the construction above, we see an immediate issue with the basis: the conditioning of the matrix A . In this case, the conditioning quickly grows:

$p+1$	$\text{cond}(A)$
2	1.0
3	1.1×10^1
4	1.8×10^2
5	3.4×10^3
6	7.3×10^4
7	1.7×10^6
8	4.1×10^7
9	1.0×10^9
10	2.7×10^{10}

There are several options to consider. First, we could consider a different function space entirely, say with

$$\phi_i(x) = \sin(i\pi x), \quad i = 1, \dots, p. \quad (4.95)$$

Here, the conditioning has a closed form, $\text{cond}(A) = p^2$, and it is clear from viewing the basis functions that they are not close to being linearly dependent as in the polynomial case above (cf. Figure 4.5.1).

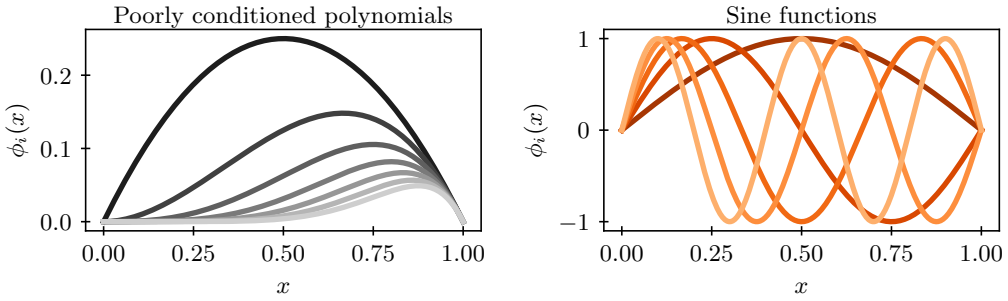


Figure 4.5.1. Polynomial and sine basis functions.

At the same time, extending this basis to elements in higher dimensions will present a challenge, and well-behaved polynomial bases do exist. Legendre polynomials, for example, are L^2 orthogonal on $[-1, 1]$ and can be expressed through the recurrence relation

$$\phi_0(x) = 1, \quad (4.96a)$$

$$\phi_1(x) = x, \quad (4.96b)$$

$$\phi_{p+1}(x) = \frac{1}{p+1} ((2p+1)x\phi_p(x) - p\phi_{p-1}(x)). \quad (4.96c)$$

Similarly, (first kind) Chebyshev polynomials provide a weighted form of L^2 orthogonality and are defined by the recurrence

$$\phi_0(x) = 1, \quad (4.97a)$$

$$\phi_1(x) = x, \quad (4.97b)$$

$$\phi_{p+1}(x) = 2x\phi_p(x) - \phi_{p-1}(x) \quad (4.97c)$$

on $[-1, 1]$. Examples are shown in Figure 4.5.2.

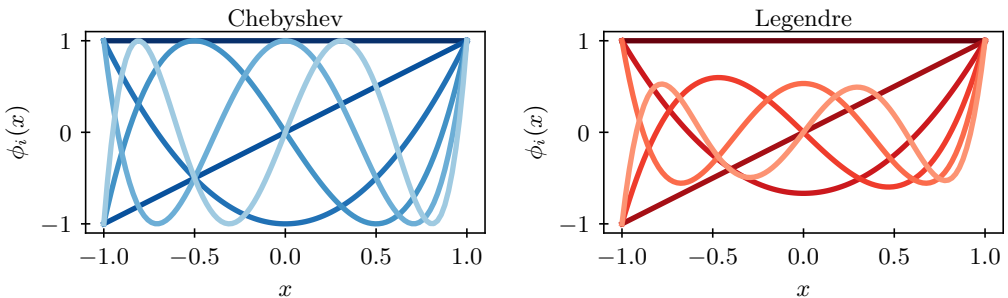


Figure 4.5.2. Chebyshev and Legendre polynomial basis functions.

For more on basis functions and spectral methods in general, we point the reader to the fantastic and complete work on the subject, *Chebyshev and Fourier Spectral Methods* by John P. Boyd [47].

While the form of the basis functions plays a major role in the accuracy and computational cost of the finite-element method, generalizing the approach taken in this chapter will follow similar steps:

1. Convert the PDE from strong form to weak form.
2. Approximate the weak form using a Ritz–Galerkin approximation over a finite-dimensional subspace.
3. Analyze the existence and uniqueness of the discrete approximation, as well as its accuracy, using Céa’s lemma and an approximation assumption.
4. Prove the accuracy of the approximation by establishing an approximation property.

4.6 ■ Exercises

1. Consider the function $u_k(x) = \sin\left((k + \frac{1}{2})\pi x\right)$ for any integer k . Verify that $u_k(x)$ is a solution of the PDE

$$\begin{aligned} -u''(x) &= f(x) \quad \text{for } 0 < x < 1, \\ u(0) &= 0, \\ u'(1) &= 0, \end{aligned}$$

and state the form of the right-hand side, $f(x)$, for this to hold.

2. Find the weak form for the problem

$$\begin{aligned} -u''(x) &= f(x) \quad \text{for } 0 < x < 1, \\ u(0) &= 0, \\ u'(1) &= \beta. \end{aligned}$$

Provide your answer in the form of finding $u \in \mathcal{V}$ such that $a(u, v) = L(v)$ for all $v \in \mathcal{V}$, where $a(u, v)$ is a bilinear form and $L(v)$ is a linear operator on $v \in \mathcal{V}$.

3. Given a nonuniform grid, $\{x_j\}_{j=0}^{n_x}$, on the unit interval $[0, 1]$ and the nodal basis for the space of piecewise linear approximations that satisfy $u(0) = 0$, $\mathcal{V}_1^h = \text{span}\{\phi_j(x)\}_{j=1}^{n_x}$, compute the entries of the *finite-element stiffness matrix*, $a_{i,j} = a(\phi_j, \phi_i)$, for the bilinear form $a(u, v) = \int_0^1 u'(x)v'(x) dx$.
4. Given a nonuniform grid, $\{x_j\}_{j=0}^{n_x}$, on the unit interval $[0, 1]$, consider the spaces $\mathcal{V}$ and $\mathcal{V}_1^h$, both constrained to take value zero at $x = 0$. Define the bilinear form $a(u, v) = \int_0^1 u'(x)v'(x) dx$, and consider the weak solution, u , of $a(u, v) = \int_0^1 f(x)v(x) dx$ for all $v \in \mathcal{V}$; the corresponding Ritz–Galerkin solution, $u^h \in \mathcal{V}_1^h$; and the interpolant of u , u_I^h , in $\mathcal{V}_1^h$.

- (a) Show that $a(u - u_I^h, v) = 0$ for all $v \in \mathcal{V}_1^h$ by showing that

$$\int_{x_{i-1}}^{x_i} (u(x) - u_I^h(x))' v'(x) dx = 0$$

for any $v \in \mathcal{V}_1^h$. Be sure to explain why only this is needed to reach the stated conclusion.

- (b) Explain why this forces $u^h = u_I^h$ in this case, which we call *superconvergence*. Could we draw the same conclusion if the bilinear form were

$$a(u, v) = \int_0^1 u(x)v(x) + u'(x)v'(x) dx?$$

5. Consider the boundary-value problem,

$$\begin{aligned} -u''(x) + 2u'(x) + u(x) &= f(x) \quad \text{for } 0 < x < 1, \\ u(0) &= u(1) = 0. \end{aligned}$$

- (a) Show that any $u \in C^2([0, 1])$ that is a solution of this boundary-value problem is also a solution of

$$a(u, v) = \langle f, v \rangle \text{ for all } v \in \mathcal{V},$$

with

$$\begin{aligned} a(u, v) &= \int_0^1 u'(x)v'(x) + 2u'(x)v(x) + u(x)v(x) dx \\ \text{and } \mathcal{V} &= \left\{ v \in L^2([0, 1]) \mid v(0) = v(1) = 0 \text{ and } \int_0^1 (v'(x))^2 dx < \infty \right\}. \end{aligned}$$

- (b) Let $\{x_j\}_{j=0}^{n_x}$ be a uniform grid on $[0, 1]$, with $h = 1/n_x$ and $x_j = jh$ for $0 \leq j \leq n_x$. Express the Ritz–Galerkin approximation of finding $u^h \in \mathcal{V}_1^h \subset \mathcal{V}$ (the space of piecewise linear functions that satisfy both boundary conditions) such that $a(u^h, v) = \langle f, v \rangle$ for all $v \in \mathcal{V}_1^h$ as a linear system, $A\mathbf{u} = \mathbf{f}$, giving the entries in both matrix A and vectors $\mathbf{u}$ and $\mathbf{f}$. You should provide values for the entries in A that depend only on h (i.e., you should evaluate all integrals explicitly).

- (c) Define

$$\langle u, v \rangle_E = \int_0^1 u'(x)v'(x) + u(x)v(x) dx, \text{ with } \|u\|_E^2 = \langle u, u \rangle_E.$$

Show that $a(u, u) = \|u\|_E^2$ for all $u \in \mathcal{V}$.

- (d) Show that $a(u, v) \leq C\|u\|_E\|v\|_E$ for some constant C .

- (e) Use parts (c) and (d) to prove a modified version of Céa's lemma:

If $u \in \mathcal{V}$ is the solution of the weak form and $u^h \in \mathcal{V}^h$ is the solution of the Ritz–Galerkin approximation, then $\|u - u^h\|_E \leq C(\min_{v \in \mathcal{V}^h} \|u - v\|_E)$. Here, you can assume that both the solution of the weak form and the Ritz–Galerkin approximation exist and are unique, but you should justify all other steps in your proof.

Chapter 5

Finite-difference methods for time-dependent partial differential equations

Overview

We now return to the finite-difference method, as discussed in Chapter 3, but applied to time-dependent PDEs. We start with prototypical *hyperbolic* and *parabolic* PDEs and develop the methodology for these equations. For the hyperbolic case, we consider the one-way wave equation, also called the linear advection equation,

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0, \quad (5.1)$$

where a is a given constant representing the speed of advection. Given a smooth initial condition, $u(x, 0) = g(x)$, we know the exact solution, $u(x, t) = g(x - at)$, making this a great problem with which to investigate the notions of numerical stability, consistency, and convergence. For the parabolic case, we consider the heat equation,

$$\frac{\partial u}{\partial t} - D \frac{\partial^2 u}{\partial x^2} = 0, \quad (5.2)$$

where $D > 0$ is a given constant representing how quickly heat diffuses in the system.

In this chapter, we consider *linear* hyperbolic and parabolic PDEs. For nonlinear hyperbolic PDEs, more specialized techniques are required, which are developed in Chapters 6 and 9 on finite-volume methods. Parabolic PDEs are often also discretized using finite-element methods in space, extending the approach from Chapter 4, as we discuss in Section 8.7.

Objectives

In this chapter, we present the main concepts of the finite-difference method applied to two simple time-dependent test problems of hyperbolic and parabolic type. Using the tools developed in this chapter, you will be able to create numerical schemes that are consistent with the particular properties of the PDE, and will learn how to properly simulate both wave-like and diffusive phenomena with finite differences. In particular, you will be able to

- approximate solutions to problems like those in equations (5.1) and (5.2) discretely with a variety of finite-difference schemes on uniform rectangular meshes;
- analyze the consistency, stability, and convergence of each of these schemes, using tools such as operator norms, Fourier and von Neumann analysis, and Toeplitz matrices; and
- verify and validate how physical properties of the original PDE are preserved by each scheme.

Chapter Outline

Section 5.1 Finite-difference methods for model equations presents two-level finite-difference schemes for discretizing the one-way wave equation and the heat equation. Basic notation is introduced, and the numerical schemes are written in matrix form to help with later analysis. We introduce the *method of lines* for discretization of time-dependent PDEs, which is a common viewpoint for these schemes.

Section 5.2 Convergence theory defines the notions of truncation error, consistency, convergence, and stability similarly to what is discussed in Chapter 3 for elliptic PDEs. The Lax equivalence and convergence theorems are presented and proven, emphasizing the notion that it takes consistency and stability to obtain convergence.

Section 5.3 Stability of two-level schemes analyzes the stability of two-level finite-difference schemes. Using the Lax convergence theorems requires bounding operator norms to prove stability, which may be cumbersome in practice. Thus, von Neumann analysis is presented for determining numerical stability using Fourier analysis of Toeplitz matrices. It is shown that implicit schemes are often beneficial for parabolic PDEs, as illustrated by the example of the Black–Scholes equation from computational finance.

Section 5.4 Numerical dissipation and dispersion discusses the concepts of dissipation and dispersion for PDEs and their numerical solution. The PDE is analyzed in order to determine what type of physical properties one should expect. The extent to which dissipation and dispersion properties are preserved by the numerical schemes is investigated, and some broader conclusions are drawn about approximating solutions to hyperbolic and parabolic PDEs.

5.1 ■ Finite-difference methods for model equations

Objectives

In this section, several finite-difference schemes are introduced for the one-way wave equation and the heat equation in 1D, which are prototypical hyperbolic and parabolic PDEs. You will see how to rewrite the numerical schemes for these equations in matrix form, which will set the notation that will be used throughout this chapter. You will also learn about the relationship between these schemes and traditional numerical methods for ODEs via a discretization concept called the method of lines.

A prototype equation for hyperbolic PDEs is the one-way wave equation in 1D (also called the linear advection equation), which was described in Section 1.2:

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0, \quad (5.3a)$$

$$u(x, 0) = g(x), \quad (5.3b)$$

where a is a given constant representing the speed of advection and $g(x)$ is the initial condition defined on $\mathbb{R}$. Using the method of characteristics, we know that the exact solution is $u(x, t) = g(x - at)$.

A prototype for parabolic PDEs is the 1D heat equation,

$$\frac{\partial u}{\partial t} - D \frac{\partial^2 u}{\partial x^2} = 0, \quad (5.4a)$$

$$u(x, 0) = g(x), \quad (5.4b)$$

where $D > 0$ is a given diffusion constant and $g(x)$ is, again, an initial condition. The method of characteristics cannot be used to find the exact solution of the heat equation; however, several other solution techniques are known (including the use of Fourier series and transforms [157, 119, 180], commonly discussed in first courses on PDEs). If we choose $g(x) = \delta(x)$, where $\delta(x)$ is the Dirac delta function, then the solution for $t > 0$ on an unbounded spatial domain is known to be

$$u(x, t) = \frac{1}{\sqrt{4\pi t}} e^{-\frac{x^2}{4t}}. \quad (5.5)$$

Thus, while the initial data is nonzero at only one point, $x = 0$, $u(x, t) > 0$ for all x if $t > 0$. This example illustrates that the domain of dependence for parabolic PDEs is infinite, in contrast to the finite domains of dependence of hyperbolic PDEs.

5.1.1 ■ Finite-difference stencils

As described in Chapter 3, the main idea of the finite-difference method is to represent $u(x, t)$ by its values at points on a grid and then use interpolation to get values at points not on the grid. Consider the regular space-time grid

$$\{(x_k, t_\ell) \mid x_k = kh_x, t_\ell = \ell h_t, \text{ with } k, \ell \in \mathbb{Z} \text{ and } \ell \geq 0\} \quad \text{on } \mathbb{R} \times [0, \infty). \quad (5.6)$$

Remark 5.1: Finite versus infinite grids

Note that the grid considered in equation (5.6) is an *infinite* grid. For the one-way wave equation, we are often less concerned with boundary conditions and assume the wave propagates continuously. Equivalently, we could consider a periodic mesh such that x_k is defined on a finite interval, $[a, b]$, but with $u(a, t_\ell) = u(b, t_\ell)$ for all $t_\ell \in [0, \infty)$. However, for many practical applications, boundary conditions are important (such as inflow or outflow conditions) and, then, a nonperiodic finite grid needs to be considered. The same is true for the heat equation, where nonperiodic finite grids are most common, combined with Dirichlet or Neumann boundary conditions, as discussed in Chapter 3. In what follows, the analysis can be somewhat different if we are assuming a finite grid versus an infinite one. In general, we will not assume either type unless otherwise stated.

As discussed in Section 2.4, there are many ways to use Taylor's theorem to generate difference approximations for the spatial and temporal derivatives in our time-dependent PDEs. We identify the exact solution of the PDE at grid point (x_k, t_ℓ) as $u(x_k, t_\ell)$ and denote the numerical approximation of $u(x_k, t_\ell)$ at the grid point by

$$u_{k,\ell} \approx u(x_k, t_\ell). \quad (5.7)$$

Then, using these finite-difference approximations of derivatives of $u(x, t)$, we can rewrite the one-way wave equation or the heat equation as a finite-difference scheme whose solution yields an approximation to the solution of the PDE.

Terminology for space-time finite-difference stencils

Terminology for space-time finite-difference stencils varies across the literature. In this chapter, we adopt the use of *explicit* versus *implicit* in time, and the use of *centered*, *backward*, or *forward* in space. To explain this, consider the stencil diagram in Figure 5.1.1 for the one-way wave equation. Here, *forward* spatial differencing is to the right of x_k , meaning that we use $(u_{k+1,\ell} - u_{k,\ell})/h_x$ as a first-order accurate approximation of $\partial u / \partial x$ at x_k . Similarly, backward spatial

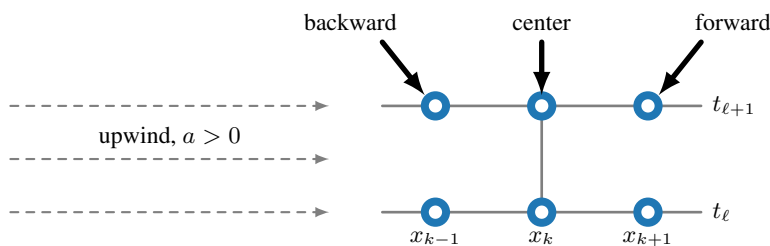


Figure 5.1.1. Stencil notation for space-time finite differences.

differencing is to the left, meaning that we use $(u_{k,\ell} - u_{k-1,\ell})/h_x$ as the first-order accurate approximation of $\partial u/\partial x$ at x_k .

In the time direction, we call a finite-difference method *explicit* if the difference formula allows us to directly compute a value at the next timestep—e.g., $u_{k,\ell+1}$ —using only values at the previous timestep, i.e., $u_{k-1,\ell}$, $u_{k,\ell}$, or $u_{k+1,\ell}$. In the same manner, a finite-difference method is called *implicit* if the difference formula for $u_{k,\ell+1}$ also involves other unknown values at timestep $t_{\ell+1}$, i.e., $u_{k-1,\ell+1}$ or $u_{k+1,\ell+1}$. In this case, a linear system needs to be solved at every timestep to advance the solution. This naming convention leads, for example, to the *explicit-in-time, backward-in-space (ETBS)* scheme of Figure 5.1.2a and the *explicit-in-time, forward-in-space (ETFS)* scheme of Figure 5.1.2b.

Alternative terminology for first-order approximations of the spatial derivative distinguishes between *upwind* and *downwind* discretizations. This terminology is commonly used in the context of hyperbolic conservation laws and finite-volume methods. When the wave speed $a > 0$ (i.e., the wind blows from the left), the difference approximation of Figure 5.1.2a that uses the value $u_{k-1,\ell}$ situated in the upwind direction from x_k is called the upwind discretization. Similarly, the difference approximation of Figure 5.1.2b uses the value $u_{k+1,\ell}$ situated downwind from x_k and is called the downwind discretization. The upwind scheme always uses the approximation in the upwind direction, so when $a < 0$ the scheme from Figure 5.1.2b is called upwind. Similarly, what we call explicit in time in this chapter is often also called forward in time, and implicit in time is often called backward in time. For example, in this alternative terminology, when $a > 0$, the ETBS scheme from Figure 5.1.2a can also be called forward in time, upwind in space. In this chapter, we use only the terminology from Figure 5.1.2.

The one-way wave equation

Several examples of finite-difference schemes and their stencils for the one-way wave equation are shown in Figure 5.1.2. To demonstrate how a numerical solution of such schemes can differ from the analytical one, consider the one-way wave equation with $a = 1$ on the periodic interval $-1 \leq x < 1$, with initial condition

$$u(x, 0) = \begin{cases} 100 e^{1/(x^2 - 0.25)} & \text{if } -0.5 \leq x \leq 0.5, \\ 0 & \text{otherwise,} \end{cases} \quad (5.8)$$

which is a so-called *bump function* and is infinitely differentiable (see Example 8.5). We use 100 points to discretize in space and take timesteps of size $h_t = 1/300$ for the interval $0 \leq t \leq 1$. Figure 5.1.3 shows the analytical solution and the numerical solution at times $t = 0.2j$ for $0 \leq j \leq 5$, using the ETBS and Crank–Nicolson schemes. We notice that the quality of approximation is slightly different for the two numerical schemes. The ETBS scheme results in a decline in amplitude as t gets larger, while the second-order Crank–Nicolson scheme significantly

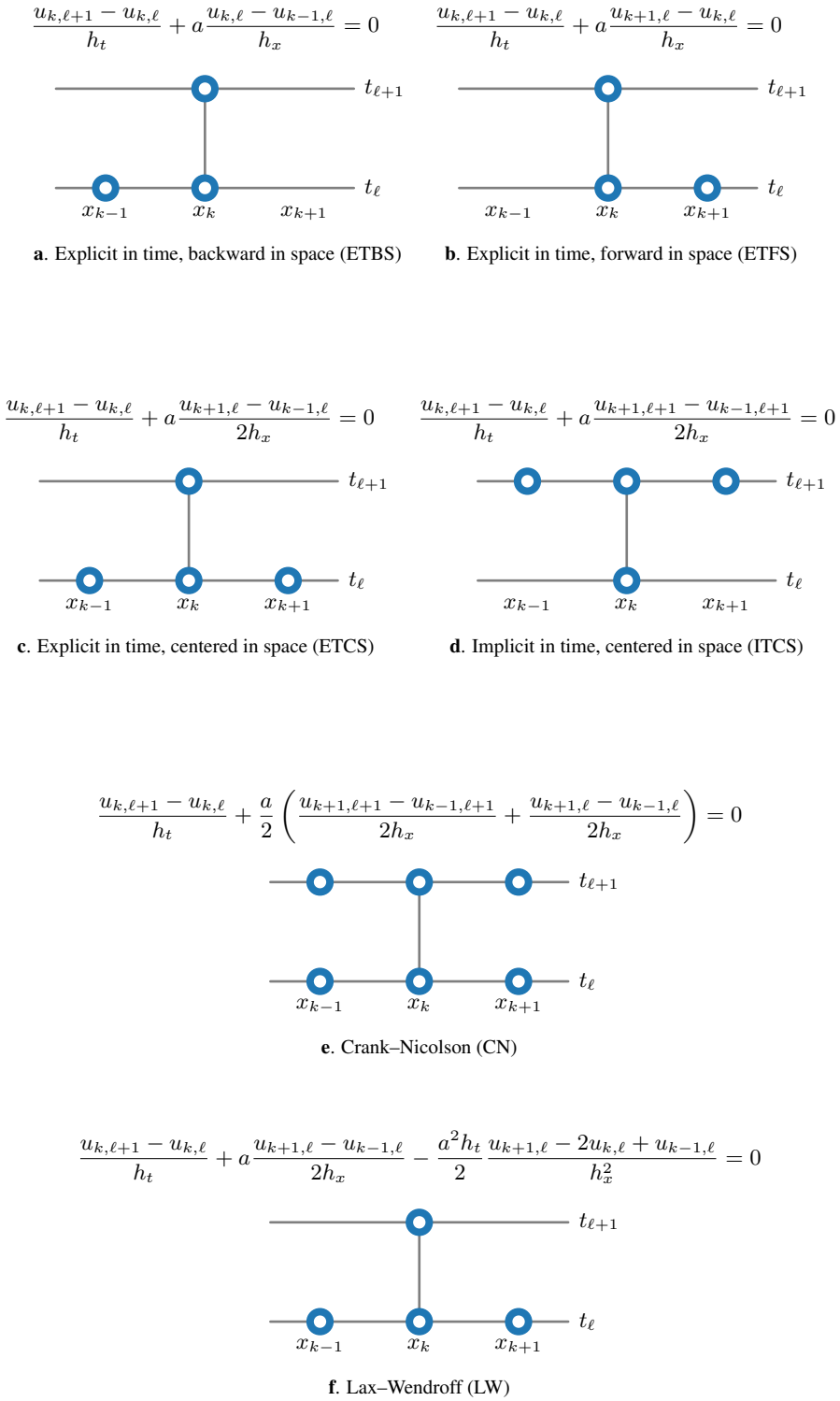


Figure 5.1.2. Examples of finite-difference stencils for the one-way wave equation.

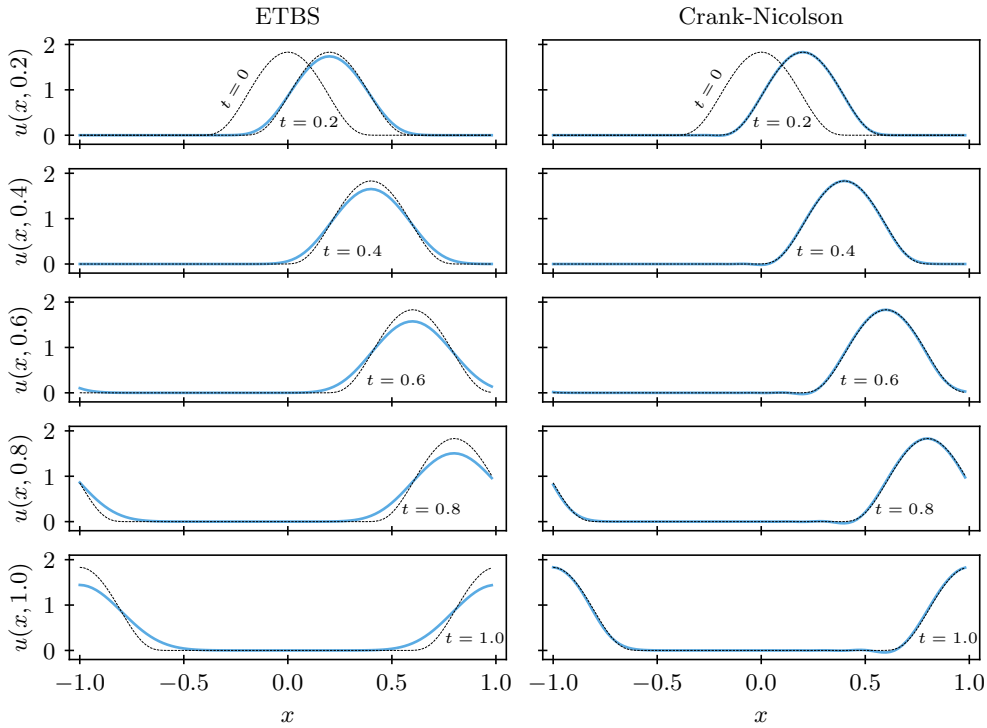


Figure 5.1.3. Exact solution (black line) and numerical solutions (blue lines) of the one-way wave equation using ETBS (left) and Crank–Nicolson (right) schemes. Solution times range from $t = 0.2$ (top) to $t = 1$ (bottom).

improves the results (with similar results achieved by Lax–Wendroff). We investigate why this happens later in this chapter.

The heat equation

To illustrate finite-difference schemes for the heat equation, (5.4), we discretize the second-order spatial derivative by the second-order centered differencing scheme derived in Section 3.1:

$$u_{xx}(x_k, t_\ell) \approx \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2}. \quad (5.9)$$

Then, we can write three finite-difference schemes as follows:

$$\textbf{Explicit Euler:} \quad \frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} - D \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2} = 0, \quad (5.10a)$$

$$\textbf{Implicit Euler:} \quad \frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} - D \frac{u_{k+1,\ell+1} - 2u_{k,\ell+1} + u_{k-1,\ell+1}}{h_x^2} = 0, \quad (5.10b)$$

$$\begin{aligned} \textbf{Crank–Nicolson:} \quad & \frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} - \frac{D}{2} \left(\frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2} \right. \\ & \left. + \frac{u_{k+1,\ell+1} - 2u_{k,\ell+1} + u_{k-1,\ell+1}}{h_x^2} \right) = 0. \end{aligned} \quad (5.10c)$$

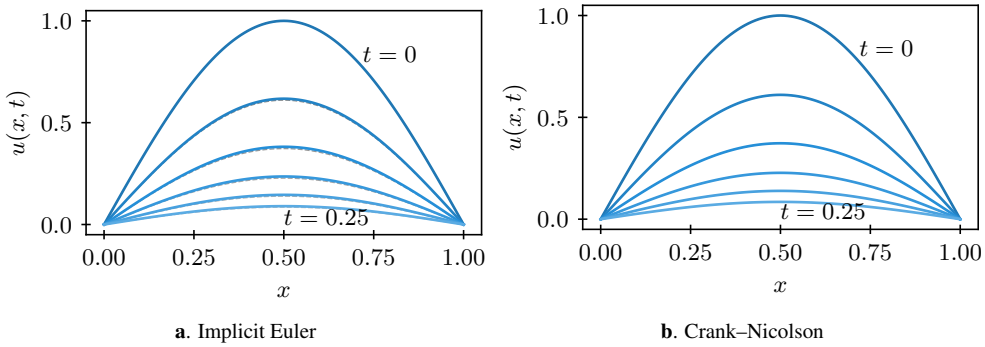


Figure 5.1.4. Exact solution (dotted black line) and numerical solutions (blue lines) of the heat equation using the **a.** implicit Euler and **b.** Crank–Nicolson finite-difference schemes. Dark blue curves indicate $t = 0$ and become lighter as t increases to $t = 0.25$.

As an example, consider the interval $0 \leq x \leq 1$, with homogeneous Dirichlet boundary conditions, and the initial condition $u(x, 0) = \sin(\pi x)$, so that the analytical solution is given by $u(x, t) = \sin(\pi x)e^{-\pi^2 t}$. We discretize using $h_x = 0.01$ (corresponding to a uniform mesh with 101 points, including the two endpoints) and $h_t = 0.05$. Figure 5.1.4 shows the analytical and numerical solutions at times $t = jh_t$ for $0 \leq j \leq 5$ using the implicit Euler and Crank–Nicolson schemes. It is harder to see the differences in this case, although we note that the analytical solution is slightly smaller than its numerical approximation, particularly for the implicit Euler case.

5.1.2 ■ Method of lines

In Section 5.1.1, we considered directly discretizing time-dependent PDEs using finite-difference techniques. An alternative approach is to use the *method of lines*, where we first discretize the problem in space (we often say that we *semidiscretize* the problem) and then apply a timestepping technique to the spatially discretized problem. This lets us leverage the vast body of work on numerical methods for systems of ODEs, which focus on accurate and efficient approximation of solutions to problems like

$$\frac{d\mathbf{u}}{dt} = \mathbf{F}(t, \mathbf{u}(t)). \quad (5.11)$$

Considering again the one-way wave equation, equation (5.1), we can first discretize the spatial term, $\frac{\partial u}{\partial x}$, using a standard finite-difference approach on a spatial mesh. This gives, for example,

$$\frac{\partial u}{\partial x}(x_k, t) \approx \frac{u(x_k, t) - u(x_{k-1}, t)}{h_x} \quad (5.12)$$

if we use the backward-difference approximation on a spatial mesh with a uniform mesh width, $h_x = x_k - x_{k-1}$. Alternatively, we could approximate

$$\frac{\partial u}{\partial x}(x_k, t) \approx \frac{u(x_{k+1}, t) - u(x_{k-1}, t)}{2h_x} \quad (5.13)$$

if we use the centered-difference approximation on the same mesh.

Defining $u_k(t)$ as our approximation of $u(x_k, t)$, these lead to systems of ODEs of the form

$$\frac{du_k}{dt} = -\frac{a}{h_x} (u_k(t) - u_{k-1}(t)) \quad \text{or} \quad (5.14a)$$

$$\frac{du_k}{dt} = -\frac{a}{2h_x} (u_{k+1}(t) - u_{k-1}(t)) \quad (5.14b)$$

for the two spatial discretization schemes. Similarly, we can use the centered finite-difference approximation of $\frac{\partial^2 u}{\partial x^2}$ to write a system of ODEs for the semidiscretized heat equation as

$$\frac{du_k}{dt} = \frac{D}{h_x^2} (u_{k+1}(t) - 2u_k(t) + u_{k-1}(t)). \quad (5.15)$$

Any of these can be written in the vector form of equation (5.11), where $\mathbf{u}(t)$ has components $u_k(t)$ for a suitable range of k (depending on problem domain and mesh size). This is a coupled system of ODEs, which we can now integrate using any ODE-based time-integration method.

In this chapter, we mostly focus on implicit and explicit Euler discretizations of these equations, where we write $\mathbf{u}_\ell \approx \mathbf{u}(t_\ell)$, giving

$$\textbf{Explicit Euler: } \frac{\mathbf{u}_{\ell+1} - \mathbf{u}_\ell}{h_t} = \mathbf{F}(t_\ell, \mathbf{u}_\ell), \quad (5.16a)$$

$$\textbf{Implicit Euler: } \frac{\mathbf{u}_{\ell+1} - \mathbf{u}_\ell}{h_t} = \mathbf{F}(t_{\ell+1}, \mathbf{u}_{\ell+1}). \quad (5.16b)$$

From Figure 5.1.2c, the ETCS scheme can thus also be derived using the centered-difference approximation in space and the explicit Euler discretization in time. As another example, the Crank–Nicolson method can be derived as a method-of-lines discretization using the centered-difference discretization in space and the implicit trapezoid method in time (a two-stage Runge–Kutta integrator). Other combinations also arise, as will schemes that do not come (directly) from using the method of lines.

It is easy to recognize the simple schemes using implicit and explicit Euler as coming either from the method of lines or from direct *space-time* discretization. We emphasize that both ways of deriving these schemes are equally valid, and that the two perspectives each offer their own advantages. In many ways, using direct space-time discretizations is a much more powerful technique, so we use it when it provides us with some benefit. However, some powerful timestepping techniques, such as Runge–Kutta integrators, are most naturally utilized from the method-of-lines perspective.

5.1.3 ■ Two-level schemes and matrix notation

In order to generalize the analysis for the convergence theory presented later, we now rewrite the schemes in matrix form. For simplicity, we consider only *two-level schemes*. These are schemes that only require the solution at two timesteps (typically the current time and the time one grid point back: $t_{\ell+1}$ and t_ℓ , respectively). Thus, we reuse some notation from Chapter 3. Consider $\mathbf{u}$ as a stacked vector of values of the approximation at each grid point in space and time,

$$\mathbf{u} = \begin{bmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \\ \vdots \end{bmatrix}, \quad \text{where} \quad \mathbf{u}_\ell = \begin{bmatrix} \vdots \\ u_{-1,\ell} \\ u_{0,\ell} \\ u_{1,\ell} \\ \vdots \end{bmatrix}, \quad (5.17)$$

as well as the exact solution evaluated at the grid points in vector form,

$$U = \begin{bmatrix} U_0 \\ U_1 \\ \vdots \end{bmatrix}, \quad \text{where} \quad U_\ell = \begin{bmatrix} \vdots \\ u(x_{-1}, t_\ell) \\ u(x_0, t_\ell) \\ u(x_1, t_\ell) \\ \vdots \end{bmatrix}. \quad (5.18)$$

Then, define the error at a grid point as $e_{k,\ell} = u(x_k, t_\ell) - u_{k,\ell}$ to obtain

$$e = U - u \quad \text{and} \quad e_\ell = U_\ell - u_\ell. \quad (5.19)$$

Next, each of the finite-difference schemes presented in Figure 5.1.2 and equation (5.10) can be rewritten as a matrix equation,

$$P_h u_{\ell+1} = Q_h u_\ell + h_t b_\ell, \quad (5.20)$$

where b_ℓ may arise from a source term in the PDE. In all cases, these schemes involve matrices P_h and Q_h that are tridiagonal Toeplitz matrices (see Definition 3.2), including where one of them is the identity matrix.

Definition 5.2: Two-level linear finite-difference scheme [188]

A finite-difference scheme that can be written as

$$P_h u_{\ell+1} = Q_h u_\ell + h_t b_\ell \quad (5.21)$$

is called a two-level linear finite-difference scheme. Each iteration depends on only two instances of time.

Consider the following four finite-difference schemes for $u_t + au_x = 0$:

1. ETCS: $\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k-1,\ell}}{2h_x} = 0,$
2. ETFS: $\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k,\ell}}{h_x} = 0,$
3. ETBS: $\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k,\ell} - u_{k-1,\ell}}{h_x} = 0,$
4. Crank–Nicolson: $\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + \frac{a}{2} \left(\frac{u_{k+1,\ell+1} - u_{k-1,\ell+1}}{2h_x} + \frac{u_{k+1,\ell} - u_{k-1,\ell}}{2h_x} \right) = 0.$

Each of the above schemes is rewritten so that it is in the form of equation (5.21), providing the matrices for each:

1. ETCS:

$$u_{k,\ell+1} = \frac{ah_t}{2h_x} u_{k-1,\ell} + u_{k,\ell} - \frac{ah_t}{2h_x} u_{k+1,\ell}, \quad (5.22a)$$

$$P_h = I \quad Q_h = \text{tridiag} \left(\frac{ah_t}{2h_x}, 1, -\frac{ah_t}{2h_x} \right). \quad (5.22b)$$

2. ETFS:

$$u_{k,\ell+1} = \left(1 + \frac{ah_t}{h_x}\right) u_{k,\ell} - \frac{ah_t}{h_x} u_{k+1,\ell}, \quad (5.23a)$$

$$P_h = I \quad Q_h = \text{tridiag}\left(0, 1 + \frac{ah_t}{h_x}, -\frac{ah_t}{h_x}\right). \quad (5.23b)$$

3. ETBS:

$$u_{k,\ell+1} = \frac{ah_t}{h_x} u_{k-1,\ell} + \left(1 - \frac{ah_t}{h_x}\right) u_{k,\ell}, \quad (5.24a)$$

$$P_h = I \quad Q_h = \text{tridiag}\left(\frac{ah_t}{h_x}, 1 - \frac{ah_t}{h_x}, 0\right). \quad (5.24b)$$

4. Crank–Nicolson:

$$-\frac{ah_t}{4h_x} u_{k-1,\ell+1} + u_{k,\ell+1} + \frac{ah_t}{4h_x} u_{k+1,\ell+1} = \frac{ah_t}{4h_x} u_{k-1,\ell} + u_{k,\ell} - \frac{ah_t}{4h_x} u_{k+1,\ell}, \quad (5.25a)$$

$$P_h = \text{tridiag}\left(-\frac{ah_t}{4h_x}, 1, \frac{ah_t}{4h_x}\right) \quad Q_h = \text{tridiag}\left(\frac{ah_t}{4h_x}, 1, -\frac{ah_t}{4h_x}\right). \quad (5.25b)$$

Similarly, we can rewrite our schemes for the heat equation in this form:

1. Explicit Euler:

$$u_{k,\ell+1} = u_{k,\ell} + \frac{Dh_t}{h_x^2} (u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}), \quad (5.26a)$$

$$P_h = I, \quad Q_h = \text{tridiag}\left(\frac{Dh_t}{h_x^2}, 1 - 2\frac{Dh_t}{h_x^2}, \frac{Dh_t}{h_x^2}\right). \quad (5.26b)$$

2. Implicit Euler:

$$u_{k,\ell+1} - \frac{Dh_t}{h_x^2} (u_{k+1,\ell+1} - 2u_{k,\ell+1} + u_{k-1,\ell+1}) = u_{k,\ell}, \quad (5.27a)$$

$$P_h = \text{tridiag}\left(-\frac{Dh_t}{h_x^2}, 1 + 2\frac{Dh_t}{h_x^2}, -\frac{Dh_t}{h_x^2}\right), \quad Q_h = I. \quad (5.27b)$$

3. Crank–Nicolson:

$$\begin{aligned} u_{k,\ell+1} - \frac{Dh_t}{2h_x^2} (u_{k+1,\ell+1} - 2u_{k,\ell+1} + u_{k-1,\ell+1}) \\ = u_{k,\ell} + \frac{Dh_t}{2h_x^2} (u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}), \end{aligned} \quad (5.28a)$$

$$P_h = \text{tridiag}\left(-\frac{Dh_t}{2h_x^2}, 1 + \frac{Dh_t}{h_x^2}, -\frac{Dh_t}{2h_x^2}\right), \quad Q_h = \text{tridiag}\left(\frac{Dh_t}{2h_x^2}, 1 - \frac{Dh_t}{h_x^2}, \frac{Dh_t}{2h_x^2}\right). \quad (5.28b)$$

Remark 5.3: Forcing term and boundary conditions

In all of the examples above, $\mathbf{b}_\ell = 0$ in equation (5.21) because we are considering the homogeneous PDEs $u_t + au_x = 0$ and $u_t - Du_{xx} = 0$. Also, at the moment, we are ignoring any boundary conditions. Thus, P_h and Q_h may be viewed as “infinite-grid” matrices that act on infinite vectors $\mathbf{u}_{\ell+1}$ and $\mathbf{u}_\ell$ or, with periodic boundary conditions, as equivalent to enforcing periodic initial conditions. This viewpoint will help with the analysis later. However, we could consider finite matrices with appropriate boundary conditions and will do so in later sections.

A natural question to ask is how to choose between the schemes. In the next section, we answer that by first investigating the errors introduced by the differencing scheme to approximate the derivative. We then discuss the properties of consistency, stability, and convergence.

5.2 ■ Convergence theory

Objectives

In this section, you will reconsider the definitions of truncation error, consistency, stability, and convergence from Chapter 3 and apply them to several of the schemes presented in the previous section. You will learn about the Lax equivalence and Lax convergence theorems and get a formal understanding of the notions of consistency, stability, and convergence using operator norms.

For the elliptic case, Definition 3.5 defines the truncation error of a scheme. For the time-dependent case, we have to be a bit more careful since our standard way of writing a two-level finite-difference scheme as $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + h_t \mathbf{b}_\ell$ comes from rescaling our approximation of the PDE by a factor of h_t . To define the truncation error for these schemes, we “undo” this scaling (see Definition 5.4). As in Remark 5.3, we again note that we are primarily interested in the case where $\mathbf{b}_\ell = 0$, since we consider homogeneous (unforced) PDEs, but we keep $\mathbf{b}_\ell$ in the calculations here for completeness.

Definition 5.4: Truncation error [188, Eq. (3.6b)]

The local truncation error, $\tau_{k,\ell}$, is the error that remains when a finite-difference formula is applied to the exact solution, $u(x_k, t_\ell)$. For a two-level finite-difference scheme with $\mathbf{U}_\ell$ defined as in equation (5.18), we define

$$\tau_\ell = \frac{1}{h_t} (P_h \mathbf{U}_{\ell+1} - Q_h \mathbf{U}_\ell) - \mathbf{b}_\ell, \quad \text{where} \quad \tau_\ell = \begin{bmatrix} \vdots \\ \tau_{-1,\ell} \\ \tau_{0,\ell} \\ \tau_{1,\ell} \\ \vdots \end{bmatrix}. \quad (5.29)$$

The truncation error relates to the best-case scenario where the solution of the PDE is approximated by sampling the exact solution at the grid points. Thus, to find the truncation error, we substitute the sampled exact solution into our schemes in place of the numerical approximation

and use Taylor-series expansion of the exact solution around the point (x_k, t_ℓ) . This is illustrated in Examples 5.5 and 5.6 for the ETCS and ETFS schemes applied to the one-way wave equation and in Example 5.7 for the heat equation.

Example 5.5: ETCS truncation error for the one-way wave equation

We begin by rewriting the scheme in equation (5.22) in the form of equation (5.29). This gives us

$$\tau_{k,\ell} = \frac{u(x_k, t_{\ell+1}) - u(x_k, t_\ell)}{h_t} + a \frac{u(x_{k+1}, t_\ell) - u(x_{k-1}, t_\ell)}{2h_x} \quad (5.30a)$$

$$\begin{aligned} &= \frac{1}{h_t} \left(u(x_k, t_\ell) + u_t(x_k, t_\ell)h_t + u_{tt}(x_k, \varsigma) \frac{h_t^2}{2} - u(x_k, t_\ell) \right) \\ &\quad + \frac{a}{2h_x} \left(u(x_k, t_\ell) + u_x(x_k, t_\ell)h_x + u_{xx}(x_k, t_\ell) \frac{h_x^2}{2} + u_{xxx}(\xi^+, t_\ell) \frac{h_x^3}{6} \right. \\ &\quad \left. - u(x_k, t_\ell) + u_x(x_k, t_\ell)h_x - u_{xx}(x_k, t_\ell) \frac{h_x^2}{2} + u_{xxx}(\xi^-, t_\ell) \frac{h_x^3}{6} \right) \end{aligned} \quad (5.30b)$$

$$= u_t(x_k, t_\ell) + u_{tt}(x_k, \varsigma) \frac{h_t}{2} + au_x(x_k, t_\ell) + \frac{a}{12} (u_{xxx}(\xi^+, t_\ell) + u_{xxx}(\xi^-, t_\ell)) h_x^2 \quad (5.30c)$$

$$= u_{tt}(x_k, \varsigma) \frac{h_t}{2} + \frac{a}{12} (u_{xxx}(\xi^+, t_\ell) + u_{xxx}(\xi^-, t_\ell)) h_x^2 \quad (5.30d)$$

$$= \mathcal{O}(h_t, h_x^2). \quad (5.30e)$$

Example 5.6: ETFS truncation error for the one-way wave equation

As in Example 5.5, we rewrite the scheme from equation (5.23) in the form of equation (5.29) to get

$$\tau_{k,\ell} = \frac{u(x_k, t_{\ell+1}) - u(x_k, t_\ell)}{h_t} + a \frac{u(x_{k+1}, t_\ell) - u(x_k, t_\ell)}{h_x} \quad (5.31a)$$

$$\begin{aligned} &= \frac{1}{h_t} \left(u(x_k, t_\ell) + u_t(x_k, t_\ell)h_t + u_{tt}(x_k, \varsigma) \frac{h_t^2}{2} - u(x_k, t_\ell) \right) \\ &\quad + \frac{a}{h_x} \left(u(x_k, t_\ell) + u_x(x_k, t_\ell)h_x + u_{xx}(\xi^+, t_\ell) \frac{h_x^2}{2} - u(x_k, t_\ell) \right) \end{aligned} \quad (5.31b)$$

$$= u_{tt}(x_k, \varsigma) \frac{h_t}{2} + au_{xx}(\xi^+, t_\ell) \frac{h_x}{2} \quad (5.31c)$$

$$= \mathcal{O}(h_t, h_x). \quad (5.31d)$$

Example 5.7: Implicit Euler truncation error for the heat equation

For implicit schemes, such as the implicit Euler discretization of the heat equation in equation (5.27), it is often easier to use the Taylor expansion around point $(x_k, t_{\ell+1})$ in

simplifying equation (5.29), giving

$$\tau_{k,\ell} = \frac{u(x_k, t_{\ell+1}) - u(x_k, t_\ell)}{h_t} - D \frac{u(x_{k+1}, t_{\ell+1}) - 2u(x_k, t_{\ell+1}) + u(x_{k-1}, t_{\ell+1}))}{h_x^2} \quad (5.32a)$$

$$= \frac{1}{h_t} \left(u(x_k, t_{\ell+1}) - \left(u(x_k, t_{\ell+1}) - u_t(x_k, t_{\ell+1})h_t + u_{tt}(x_k, \varsigma) \frac{h_t^2}{2} \right) \right) \\ - D \left(u_{xx}(x_k, t_{\ell+1}) + \frac{h_x^2}{24} (u_{xxxx}(\xi_+, t_{\ell+1}) + u_{xxxx}(\xi_-, t_{\ell+1})) \right) \quad (5.32b)$$

$$= -u_{tt}(x_k, \varsigma) \frac{h_t}{2} - \frac{Dh_x^2}{24} (u_{xxxx}(\xi_+, t_{\ell+1}) + u_{xxxx}(\xi_-, t_{\ell+1})) \quad (5.32c)$$

$$= \mathcal{O}(h_t, h_x^2). \quad (5.32d)$$

Notably, we would have reached the same conclusion about the dependence of the truncation error on h_t and h_x had we used the Taylor expansion around (x_k, t_ℓ) , but that would require more work to simplify the results.

Similar procedures produce the following truncation errors for the other schemes mentioned earlier, which we summarize below:

Scheme	PDE	Order
ETFS	$u_t + au_x = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t, h_x)$
ETBS	$u_t + au_x = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t, h_x)$
ETCS	$u_t + au_x = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t, h_x^2)$
ITCS	$u_t + au_x = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t, h_x^2)$
Crank–Nicolson	$u_t + au_x = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t^2, h_x^2)$
Lax–Wendroff	$u_t + au_x = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t^2, h_x^2)$
Explicit Euler	$u_t - Du_{xx} = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t, h_x^2)$
Implicit Euler	$u_t - Du_{xx} = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t, h_x^2)$
Crank–Nicolson	$u_t - Du_{xx} = 0$	$\tau_{k,\ell} = \mathcal{O}(h_t^2, h_x^2)$

It is important to note that all of these examples rely on writing the finite-difference scheme in its “time-derivative” form given in equation (5.29), as is done for the stencils in Figure 5.1.2. In contrast, while the “update” forms used in the two-level matrix notation of equation (5.21) would achieve similar, but not identical, results, we derive the truncation error in the “time-derivative” form of the difference formula because it leads to the practically relevant conclusion in the error analysis that follows in Section 5.2.2. Additionally, in this form, we consider the truncation error in approximating both the temporal and spatial derivatives in the PDE, while the “update” form emphasizes the spatial derivatives but not the temporal ones (because it has been scaled by a factor of h_t).

To study the error more rigorously, we return to the matrix notation and consider the truncation error vector, τ_ℓ , as in equation (5.29), which we can rewrite as

$$P_h U_{\ell+1} = Q_h U_\ell + \mathbf{b}_\ell h_t + \tau_\ell h_t. \quad (5.33)$$

The factor of h_t multiplying τ_ℓ in this equation arises naturally from the definition in equation (5.29) and is illustrated in Example 5.8 for the ETCS scheme applied to the one-way wave equation, consistent with Example 5.5.

Example 5.8: ETCS truncation error for the one-way wave equation (matrix form)

For ETCS, $P_h = I$ and

$$(P_h \mathbf{U}_{\ell+1})_k = u(x_k, t_{\ell+1}) = u(kh_x, (\ell+1)h_t), \quad (5.34a)$$

$$(Q_h \mathbf{U}_\ell)_k = u(kh_x, \ell h_t) - \frac{ah_t}{2h_x} \left(u((k+1)h_x, \ell h_t) - u((k-1)h_x, \ell h_t) \right). \quad (5.34b)$$

Thus,

$$\begin{aligned} (P_h \mathbf{U}_{\ell+1} - Q_h \mathbf{U}_\ell)_k &= u_t(kh_x, \ell h_t)h_t + u_{tt}(kh_x, \varsigma) \frac{h_t^2}{2} + ah_t u_x(kh_x, \ell h_t) \\ &\quad + \frac{ah_t}{12} \left(u_{xxx}(\xi^+, \ell h_t) + u_{xxx}(\xi^-, \ell h_t) \right) h_x^2 \end{aligned} \quad (5.35a)$$

$$= \left(u_{tt}(kh_x, \varsigma) \frac{h_t}{2} + \frac{a}{12} (u_{xxx}(\xi^+, \ell h_t) + u_{xxx}(\xi^-, \ell h_t)) h_x^2 \right) h_t \quad (5.35b)$$

$$= \tau_{k,\ell} h_t. \quad (5.35c)$$

As for the elliptic case, we would like the truncation error of our finite-difference schemes to go to zero as the mesh and temporal spacing goes to zero. This notion was defined in Chapter 3 as *consistency* of the finite-difference scheme. However, as we recall from Chapter 3, consistency alone is not enough to guarantee convergence.

5.2.1 Consistency, convergence, and stability

We now combine the two-level finite-difference update formula (5.21) with expression (5.33) for the truncation error to obtain

$$P_h(\mathbf{U}_{\ell+1} - \mathbf{u}_{\ell+1}) = Q_h(\mathbf{U}_\ell - \mathbf{u}_\ell) + \boldsymbol{\tau}_\ell h_t \quad (5.36a)$$

$$\Rightarrow P_h \mathbf{e}_{\ell+1} = Q_h \mathbf{e}_\ell + \boldsymbol{\tau}_\ell h_t \quad (5.36b)$$

$$\Rightarrow \mathbf{e}_{\ell+1} = P_h^{-1} Q_h \mathbf{e}_\ell + P_h^{-1} \boldsymbol{\tau}_\ell h_t, \quad (5.36c)$$

noting that $\mathbf{e}_0 = 0$ since $\mathbf{u}_0 = \mathbf{U}_0$ can be assumed exact for the initial condition. This allows us to then restate questions about consistency, stability, and convergence in terms of the vectors $\boldsymbol{\tau}_\ell$ and $\mathbf{e}_\ell$ and the operators P_h and Q_h . To obtain precise conclusions, we have to select a specific metric or norm to use when measuring convergence. Possibilities include

- the max-norm (ℓ_∞), $\|\mathbf{e}\|_\infty = \max_{k,\ell} |e_{k,\ell}|$;
- the scaled Euclidean norm (ℓ_2), $\|\mathbf{e}\|_2 = \left(\sum_k \sum_\ell h_x h_t e_{k,\ell}^2 \right)^{\frac{1}{2}}$; and
- the scaled Euclidean norm for fixed t_ℓ , $\|\mathbf{e}_\ell\|_2 = \left(\sum_k h_x e_{k,\ell}^2 \right)^{\frac{1}{2}}$.

As in Section 3.3, we scale the Euclidean norms so that we can more naturally compare errors measured on different meshes. In the rest of this section, we use the vector norm $\|\cdot\|$, with no

subscript, to represent the scaled Euclidean norm at a fixed time t_ℓ and state results in terms of this norm, but we note that convergence analysis can also be considered in other norms.

We again define the matrix norm induced by a vector norm as

$$\|T\| = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|T\mathbf{x}\|}{\|\mathbf{x}\|}. \quad (5.37)$$

An immediate consequence of this definition is that $\|T\mathbf{x}\| \leq \|T\| \|\mathbf{x}\|$. For the (scaled or unscaled) Euclidean norm, this can be evaluated as

$$\|T\| = \max_i |\sigma_i(T)|, \quad (5.38)$$

where the values $\sigma_i(T)$ are the singular values of T , defined as the square roots of the eigenvalues of $T^T T$ (see Section 3.3). This reduces to the absolute values of the eigenvalues of T when T is symmetric. For finite matrices, we often use the tools of Section 3.3.2 to bound these matrix norms.

When investigating convergence in a setting where we formally allow for infinite spatial grids, we have to put some conditions on the PDE solution $u(x, t)$ and its partial derivatives so that Euclidean norms at any fixed time t_ℓ are finite. Thinking about either functions with “compact support”—i.e., functions defined on all of $\mathbb{R}$ but which take a zero value outside some closed and bounded set—or functions that decay “sufficiently fast,” we consider spatial functions in

$$L^2(\mathbb{R}) = \left\{ f : \mathbb{R} \rightarrow \mathbb{R} \mid \int_{-\infty}^{\infty} |f(x)|^2 dx < \infty \right\}. \quad (5.39)$$

Using the scaled Euclidean norm at a fixed time t_ℓ , we now redefine, in Definition 5.9, the notions of consistency, stability, and convergence from Section 3.3.

Definition 5.9: Consistency, stability, and convergence [188, Ch. 3]

Let $\frac{\partial^\mu u}{\partial x^\mu}$ denote the μ th partial derivative in x and $\frac{\partial^\nu u}{\partial t^\nu}$ denote the ν th partial derivative in t . Given a fixed time, t^* , assume that $\frac{\partial^\mu u(x, t)}{\partial x^\mu}, \frac{\partial^\nu u(x, t)}{\partial t^\nu} \in L^2(\mathbb{R})$ for all fixed $t < t^*$ for some (known) values of μ and ν .

A two-level linear finite-difference scheme, $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + \mathbf{b}_\ell h_t$, is

- consistent in the (scaled) ℓ_2 norm with order ν in time and μ in space if

$$\max_{\ell, \ell h_t \leq t^*} \|\tau_\ell\| = \mathcal{O}(h_x^\mu, h_t^\nu); \quad (5.40)$$

- stable in the (scaled) ℓ_2 norm if $\exists c > 0$, independent of h_t and h_x , such that

$$\|(P_h^{-1} Q_h)^\ell P_h^{-1}\| \leq c \quad \forall \ell \text{ and } h_t \text{ such that } \ell h_t \leq t^*; \text{ and} \quad (5.41)$$

- convergent in the (scaled) ℓ_2 norm with order ν in time and μ in space if

$$\max_{\ell, \ell h_t \leq t^*} \|\mathbf{e}_\ell\| = \mathcal{O}(h_x^\mu, h_t^\nu). \quad (5.42)$$

Remark 5.10: Differences from the elliptic case

The definitions in Definition 5.9 for consistency and convergence are very similar to the definitions in Definition 3.6, but the notion of stability is now quite different. Here, it depends on how the approximation to the solution behaves in time.

Consistency tells us whether or not our finite-difference scheme is “right” for our PDE. We get consistent schemes when we use finite-difference formulas that properly approximate the partial derivatives as the grid is refined. For example, consider approximating the second partial derivative u_{xx} in the 1D heat equation, equation (5.2), by the centered finite-difference formula in equation (5.9) with implicit Euler timestepping, but replacing the coefficient 2 in equation (5.9) by $2 + \epsilon$ with, e.g., $\epsilon = 0.001$. Then the error caused by ϵ may be small on a given fixed grid, but the resulting finite-difference formula is inconsistent with the PDE and the resulting numerical approximations cannot converge to the exact solution as the grid is refined. All of the schemes outlined above are consistent. However, it is again important to reiterate that consistency alone is not enough to guarantee convergence. The following example demonstrates this nicely.

Example 5.11: Consistent but unstable scheme for the one-way wave equation

Consider $u_t + au_x = 0$, $a > 0$, with the ETFS discretization:

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k,\ell}}{h_x} = 0. \quad (5.43)$$

We rewrite this as

$$u_{k,\ell+1} = u_{k,\ell} - \frac{ah_t}{h_x}(u_{k+1,\ell} - u_{k,\ell}) = (1 + \gamma)u_{k,\ell} - \gamma u_{k+1,\ell}, \quad (5.44)$$

with $\gamma = \frac{ah_t}{h_x}$. Consider a solution to $u_t + au_x = 0$, $a > 0$, with

$$u(x, 0) = g(x) = \begin{cases} 1 & \text{if } -1 \leq x \leq 0, \\ 0 & \text{otherwise.} \end{cases} \quad (5.45)$$

The known solution is $u(x, t) = g(x - at)$ and, for example, $u(1, \frac{3}{2a}) = 1$. However, consider formula (5.44) with initial condition

$$u_{k,0} = \begin{cases} 1 & \text{if } -1 \leq kh_x \leq 0, \\ 0 & \text{otherwise.} \end{cases} \quad (5.46)$$

Propagating this forward with the discretization scheme gives

$$u_{0,0} = 1, \quad u_{1,0} = 0, \quad u_{2,0} = 0, \quad \dots, \quad (5.47a)$$

$$u_{0,1} = 1 + \gamma, \quad u_{1,1} = 0, \quad u_{2,1} = 0, \quad \dots, \quad (5.47b)$$

$$u_{0,2} = (1 + \gamma)^2, \quad u_{1,2} = 0, \quad u_{2,2} = 0, \quad \dots \quad (5.47c)$$

Thus, for all $k > 0$ and $\ell \geq 0$, $u_{k,\ell} = 0$. Therefore, $u_{k,\ell} = 0$ for $(kh_x, \ell h_t) \rightarrow (1, \frac{3}{2a})$ as $h_x, h_t \rightarrow 0$, which contradicts $u(1, \frac{3}{2a}) = 1$. While ETFS is a consistent scheme, it does not converge to the correct solution in this case.

Thus, there must be more to the story than just consistency. In particular, we observe that $u_{0,\ell} = (1 + \gamma)^\ell \forall \ell \geq 0$. Then, to investigate this further, consider the approximation at $x = 0$ for some fixed time, say $t = 1$:

$$u(0, 1) \approx u_{0, \frac{1}{h_t}} = (1 + \gamma)^{\frac{1}{h_t}}. \quad (5.48)$$

Assuming that $\gamma > 0$ is kept constant as h_x and h_t approach zero, this shows that the approximation of $u(0, 1)$ blows up as the grid is refined. It is easy to verify numerically that $u_{-1,\ell}$, $u_{-2,\ell}$, etc., also blow up as the grid is refined. Thus, as $h_t, h_x \rightarrow 0$, we observe that the numerical approximation converges to an incorrect solution in some points of the domain, and in some points the solution blows up. This is a result of the scheme being unstable.

As Example 5.11 indicates, stability is a crucial property for convergence of a finite-difference scheme. While the definitions for consistency and convergence in Definition 5.9 are natural, the definition of stability is justified by the notion of stability that we need in the convergence proofs of the next subsection.

5.2.2 ■ Lax convergence and equivalence theorems

The following two theorems by Lax establish general links between consistency, stability, and convergence.

Theorem 5.12: Lax convergence theorem [188, Sec. 3.5]

If a two-level linear finite-difference scheme $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + \mathbf{b}_\ell h_t$ is consistent in the ℓ_2 norm with order ν in time and μ in space and stable in the ℓ_2 norm, then it is convergent in the ℓ_2 norm with order ν in time and μ in space.

Proof. Applying the definitions above as in (5.36c), we start from

$$P_h \mathbf{e}_{\ell+1} = Q_h \mathbf{e}_\ell + \boldsymbol{\tau}_\ell h_t \quad (5.49)$$

or

$$\mathbf{e}_{\ell+1} = (P_h^{-1} Q_h) \mathbf{e}_\ell + h_t P_h^{-1} \boldsymbol{\tau}_\ell. \quad (5.50)$$

Since we know that $\mathbf{e}_0 = 0$, we have

$$\mathbf{e}_1 = h_t P_h^{-1} \boldsymbol{\tau}_0, \quad (5.51a)$$

$$\mathbf{e}_2 = h_t (P_h^{-1} Q_h) P_h^{-1} \boldsymbol{\tau}_0 + h_t P_h^{-1} \boldsymbol{\tau}_1. \quad (5.51b)$$

By induction, it follows that $\mathbf{e}_\ell = h_t \sum_{m=1}^{\ell} (P_h^{-1} Q_h)^{\ell-m} P_h^{-1} \boldsymbol{\tau}_{m-1}$. Taking the norm of both sides and using the triangle inequality gives

$$\|\mathbf{e}_\ell\| \leq h_t \sum_{m=1}^{\ell} \|(P_h^{-1} Q_h)^{\ell-m} P_h^{-1} \boldsymbol{\tau}_{m-1}\|. \quad (5.52)$$

Using a property of operator norms,

$$\|\mathbf{e}_\ell\| \leq h_t \sum_{m=1}^{\ell} \|(P_h^{-1} Q_h)^{\ell-m} P_h^{-1}\| \|\boldsymbol{\tau}_{m-1}\|. \quad (5.53)$$

Now, assuming that ℓ and h_t are such that $\ell h_t \leq t^*$, we have

$$\|(P_h^{-1} Q_h)^{\ell-m} P_h^{-1}\| \leq c \text{ for } m = 1, 2, \dots, \ell, \quad (5.54)$$

since the finite-difference scheme is stable. This implies that

$$\|\mathbf{e}_\ell\| \leq h_t \sum_{m=1}^{\ell} c \|\boldsymbol{\tau}_{m-1}\| \leq c h_t \cdot \ell \max_{n, n h_t \leq t^*} \|\boldsymbol{\tau}_n\| \leq c t^* \max_{n, n h_t \leq t^*} \|\boldsymbol{\tau}_n\| \quad (5.55)$$

for all ℓ and h_t such that $\ell h_t \leq t^*$. Here, $c t^*$ is a constant.

Since the finite-difference scheme is also consistent, we have

$$\max_{n, n h_t \leq t^*} \|\boldsymbol{\tau}_n\| = \mathcal{O}(h_x^\mu, h_t^\nu) \text{ when } \frac{\partial^\mu u}{\partial x^\mu}, \frac{\partial^\nu u}{\partial t^\nu} \in L^2(\mathbb{R}), \quad (5.56)$$

and it then follows that

$$\max_{\ell, \ell h_t \leq t^*} \|e_\ell\| = \mathcal{O}(h_x^\mu, h_t^\nu) \text{ when } \frac{\partial^\mu u}{\partial x^\mu}, \frac{\partial^\nu u}{\partial t^\nu} \in L^2(\mathbb{R}). \quad (5.57)$$

Thus, the finite-difference scheme is convergent. $\square$

It is important to recognize that the proof applies to any two-level linear finite-difference method of the form (5.21) for any time-dependent PDE. It thus applies to both the one-way wave and heat equations, as well as other PDEs. Additionally, the proof does not depend on the norm. Finally, it turns out that a stronger result holds.

Theorem 5.13: Lax equivalence theorem [188, Sec. 3.5]

Let the two-level linear finite-difference scheme, $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + h_t \mathbf{b}_\ell$, be consistent in the ℓ_2 norm with order ν in time and μ in space. Then it is stable in the ℓ_2 norm if and only if it is convergent in the ℓ_2 norm with order ν in time and μ in space.

The proof of the Lax equivalence theorem is similar to the proof of the Lax convergence theorem. The equivalence result is important because it shows that stability is required for convergence of the linear scheme.

Lax–Richtmyer stability

The Lax equivalence theorem is also referred to as the Lax–Richtmyer theorem in certain contexts and is often considered as the *fundamental theorem of numerical analysis* (along with Dahlquist’s similar equivalence theorem for ODEs). In general, proving stability can be somewhat cumbersome. The Lax convergence theorem requires a bound on the stability of the two-level linear finite-difference scheme such that

$$\|(P_h^{-1} Q_h)^\ell P_h^{-1}\| \leq c \quad \forall \ell, \ell h_t \text{ with } \ell h_t \leq t^*. \quad (5.58)$$

Considering a consequence of the definition of operator norms, we have

$$\|(P_h^{-1} Q_h)^\ell P_h^{-1}\| \leq \|P_h^{-1} Q_h\|^\ell \|P_h^{-1}\|. \quad (5.59)$$

Thus, a sufficient set of conditions for stability that is often easier to verify is given by

$$\|P_h^{-1} Q_h\| \leq 1 \quad \text{and} \quad \|P_h^{-1}\| \leq c. \quad (5.60)$$

We refer to these conditions as *Lax–Richtmyer stability*.

One alternative definition for truncation error (used in [188]) is to define it to be $P_h^{-1} \tau_\ell$ (i.e., multiplying both sides of equation (5.29) by P_h^{-1}). This allows a somewhat simpler grouping of terms in the proof of Theorem 5.12, requiring only $\|P_h^{-1} Q_h\| \leq 1$ for stability. We choose not to adopt this approach here, since verifying a bound on the truncation error in this approach is complicated in the implicit case (when $P_h \neq I$), and the easiest way to do so is by showing that $\|P_h^{-1}\| \leq c$ for some constant c .

In the next section, we consider the Lax–Richtmyer conditions for stability from equation (5.60) and discuss their practical use. While Lax–Richtmyer stability is easy to show directly in some instances using matrix norm inequalities, it can be complicated in many cases.

Therefore, we introduce Fourier analysis techniques to come up with the notion of *von Neumann stability*, which is much easier to apply in practice as an indirect way to demonstrate Lax–Richtmyer stability.

Example 5.14: Convergence order for the one-way wave equation

As a final illustration, we show how the order of convergence can be measured numerically for some two-level finite-difference schemes of the form (5.21). We return to the example of the one-way wave equation and investigate how the error varies at the final time $t = 1$ for the ETBS and CN schemes. On the left of Figure 5.2.1, we consider a nondifferentiable initial condition given by

$$u(x, 0) = \begin{cases} (-100(x - 0.4)(x - 0.6))^p & \text{if } 0.4 \leq x \leq 0.6, \\ 0 & \text{otherwise,} \end{cases} \quad (5.61)$$

with $p = 1$ (to start), as shown in the left panels of Figure 5.2.1. To measure convergence of the ETBS and Crank–Nicolson schemes, we use a periodic spatial domain with $-1 \leq x < 1$ and a temporal domain $0 \leq t \leq 1$. The coarsest grid has $h_x = 1/64$ and $h_t = 1/128$, and we decrease the grid spacings by a factor of two over several levels of refinement. While the analysis above says that the truncation error for ETBS should be $\mathcal{O}(h_t, h_x)$ and for Crank–Nicolson should be $\mathcal{O}(h_t^2, h_x^2)$, the convergence we observe appears linear in both cases. This is because the solution that we are considering is continuous, but not continuously differentiable, and the truncation error analysis assumes that $u(x, t)$ is sufficiently smooth for the Taylor expansions to be valid.

To see the effect of increased smoothness, the right panel of Figure 5.2.1 considers the same experiment, but with an initial condition given by taking the third power ($p = 3$) of the initial condition from equation (5.61). Taking this higher power has the effect of giving the solution added smoothness, with continuous first and second partial derivatives in both x and t . With sufficient smoothness in the solution, we observe the convergence rates predicted by the theory.

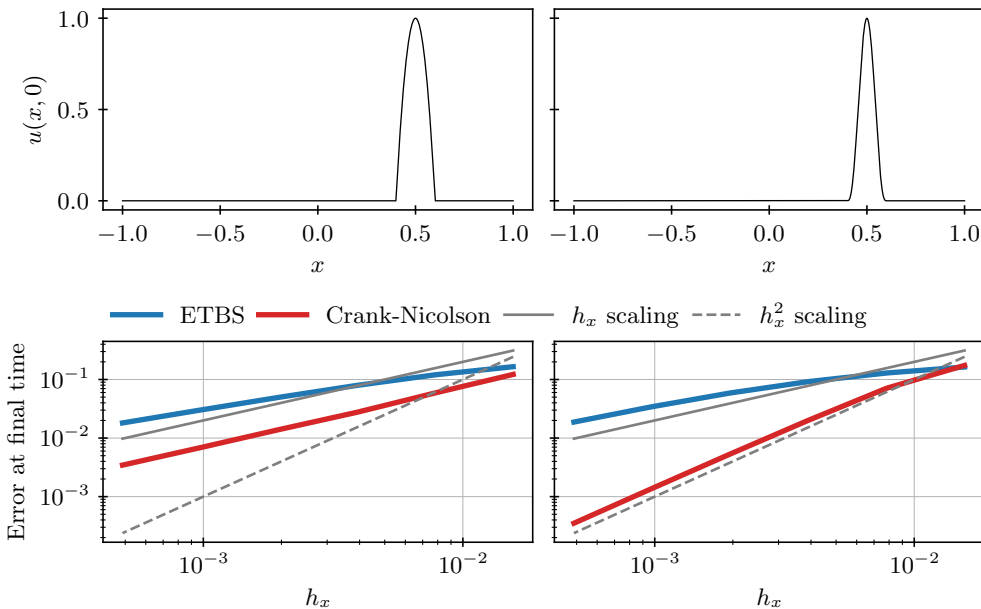


Figure 5.2.1. Convergence of two schemes for the one-way wave equation, with a continuous but not continuously differentiable solution (left) and a smoother solution (right).

5.3 ■ Stability of two-level schemes

Objectives

In this section, you will consider the notion of stability more rigorously in terms of matrix norms of the operators in a finite-difference scheme. You will see that while the analysis can sometimes be straightforward, it can become quite complex even for basic problems. Thus, the notion of von Neumann analysis is introduced, which is easier to use in practice.

5.3.1 ■ Stability analysis by directly bounding matrix norms

In the previous section, we introduced the notion of Lax–Richtmyer stability. Namely, a two-level linear finite-difference scheme, $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + \mathbf{b}_\ell h_t$, is stable in the ℓ_2 norm if there exists a constant $c > 0$ such that $\|P_h^{-1} Q_h\| \leq 1$ and $\|P_h^{-1}\| \leq c$. We also know from the Lax convergence theorem that if a finite-difference scheme is both stable and consistent, it is convergent. We begin by considering an example of a consistent scheme and attempting to show stability by bounding matrix norms. Recall the ETBS scheme:

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k,\ell} - u_{k-1,\ell}}{h_x} = 0. \quad (5.62)$$

Let $\gamma = \frac{ah_t}{h_x}$, so we can rewrite the scheme as

$$u_{k,\ell+1} = \gamma u_{k-1,\ell} + (1 - \gamma) u_{k,\ell}. \quad (5.63)$$

As before, we define $P_h = I$ and $Q_h = \text{tridiag}(\gamma, 1 - \gamma, 0)$. We do not yet make any assumptions about the sign of a . To make this example more concrete, we can consider a finite spatial domain with regular grid spacing, $\{x_k\}_{k=0}^{n_x}$, and a Dirichlet inflow boundary condition on either the left or the right side of the domain, depending on the sign of a . At this boundary point, x_b , we impose that $u(x_b, t) = f(t)$ for some known function, $f(t)$, for all $t > 0$. This leads to P_h and Q_h being matrices of size $n_x \times n_x$.

Since P_h is the identity matrix, the norm of its inverse is one, and the condition $\|P_h^{-1}\| \leq c$ is easily satisfied independent of h_x and h_t no matter what induced matrix norm we consider. On the other hand, $\|P_h^{-1} Q_h\| = \|Q_h\|$, but the mechanics of bounding this norm can be quite different depending on the choice of norm. If, for example, we consider either the ℓ_1 or the ℓ_∞ norm, standard results in linear algebra easily give us what we need. The matrix norm induced by the ℓ_1 vector norm is the maximum value of the sum of the absolute values of the matrix entries in any column, and the matrix norm induced by the ℓ_∞ vector norm is the maximum value of the sum of the absolute values of the matrix entries in any row:

$$\|Q_h\|_1 = \max_{1 \leq j \leq n_x} \sum_{k=1}^{n_x} |(Q_h)_{k,j}| \quad (\text{maximum absolute column sum}), \quad (5.64a)$$

$$\|Q_h\|_\infty = \max_{1 \leq k \leq n_x} \sum_{j=1}^{n_x} |(Q_h)_{k,j}| \quad (\text{maximum absolute row sum}). \quad (5.64b)$$

For ETBS, these yield $\|Q_h\|_1 = \|Q_h\|_\infty = |1 - \gamma| + |\gamma|$. If $0 \leq \gamma \leq 1$, then both terms are nonnegative, and we get that $\|Q_h\|_1 = \|Q_h\|_\infty = 1$; otherwise, one of the terms is strictly greater than one, so we have $\|Q_h\|_1 = \|Q_h\|_\infty > 1$.

When considering the ℓ_2 norm, we have

$$\|P_h^{-1} Q_h\|_2 = \|Q_h\|_2 = \max \text{ singular value of } Q_h \quad (5.65a)$$

$$= \sqrt{\max \text{ eigenvalue of } Q_h^T Q_h}, \quad (5.65b)$$

where $Q_h^T Q_h$ is a tridiagonal matrix with sub- and superdiagonal entries equal to $\gamma(1 - \gamma)$ and diagonal entries of $(1 - \gamma)^2 + \gamma^2$ in each row except the last, where the diagonal entry is $(1 - \gamma)^2$. Since $Q_h^T Q_h$ is symmetric positive semidefinite, its eigenvalues are real and nonnegative. Furthermore, we can bound the eigenvalues using Geršgorin's theorem (see Theorem 3.9), which says that every eigenvalue, λ , of $A = Q_h^T Q_h$ satisfies $|\lambda - a_{k,k}| \leq \sum_{j \neq k} |a_{k,j}|$ for some k . Considering these sets, we see that for $0 \leq \gamma \leq 1$, a general row for $1 < k < n_x$ gives the inclusion

$$(1 - 2\gamma)^2 = (1 - \gamma)^2 + \gamma^2 - 2\gamma(1 - \gamma) \leq \lambda \leq (1 - \gamma)^2 + \gamma^2 + 2\gamma(1 - \gamma) = 1. \quad (5.66)$$

For $k = 1$, we have

$$(1 - 2\gamma)^2 \leq (1 - \gamma)^2 + \gamma^2 - \gamma(1 - \gamma) \leq \lambda \leq (1 - \gamma)^2 + \gamma^2 + \gamma(1 - \gamma) \leq 1, \quad (5.67)$$

while, for $k = n_x$, we have

$$-1 \leq (1 - \gamma)(1 - 2\gamma) = (1 - \gamma)^2 - \gamma(1 - \gamma) \leq \lambda \leq (1 - \gamma)^2 + \gamma(1 - \gamma) = 1 - \gamma \leq 1. \quad (5.68)$$

Thus, $0 \leq \lambda \leq 1$ for all eigenvalues, λ , of $Q_h^T Q_h$ when $0 \leq \gamma \leq 1$, so $\|Q_h\| \leq 1$. For $\gamma < 0$, the off-diagonal entries become negative, while the diagonal entries are all greater than one. In this case, many of the inequalities above “flip,” and we can directly conclude that $\|Q_h\| \geq 1$. Using a strengthened form of Geršgorin's theorem, we can, in fact, conclude that $\|Q_h\| > 1$ in this case. For $\gamma > 1$, Geršgorin's theorem fails to provide a conclusive answer, since the interval associated with the last row of $Q_h^T Q_h$ always intersects the interval $[-1, 1]$ as well as the intervals associated with the other rows of the matrix. Here, more detailed analysis is needed to show that $\|Q_h\| > 1$ as well.

From this, we conclude that ETBS is stable in the ℓ_1 , ℓ_2 , and ℓ_∞ norms if $0 \leq \gamma \leq 1$. This requires that $a \geq 0$ and

$$\frac{ah_t}{h_x} \leq 1 \iff h_t \leq \frac{h_x}{a}. \quad (5.69)$$

This type of stability is called *conditional stability* (and we say that the scheme is conditionally stable), since it depends on a relationship between the grid spacing h_x , the temporal spacing h_t , and the wave velocity a . Such a condition is known as a Courant–Friedrichs–Lewy (CFL) condition and determines, for instance, how small h_t needs to be relative to h_x in order to guarantee stability and, hence, convergence.

In this case, we find that the backward spatial derivative leads to a stable scheme only when $a \geq 0$ (the spatial derivative uses information from the upwind direction, i.e., the direction from which “the wind blows” and from which information propagates along characteristic curves) and only when the timestep, h_t , is small enough. By symmetry, we can conclude that ETFS is stable only when $a \leq 0$ and $h_t \leq \frac{h_x}{-a}$.

Example 5.15: Stability of implicit Euler for the heat equation

Recall the implicit Euler discretization of the heat equation:

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} - D \frac{u_{k+1,\ell+1} - 2u_{k,\ell+1} + u_{k-1,\ell+1}}{h_x^2} = 0. \quad (5.70)$$

Let $\gamma = \frac{Dh_t}{h_x^2}$ so that we can rewrite the scheme as

$$-\gamma u_{k+1,\ell+1} + (1 + 2\gamma)u_{k,\ell+1} - \gamma u_{k-1,\ell+1} = u_{k,\ell}. \quad (5.71)$$

Here, we consider homogeneous Dirichlet boundary conditions at $k = 0$ and $k = n_x$, enforced as $u_{0,\ell+1} = u_{n_x,\ell+1} = 0$ for all $\ell \geq 0$, making P_h and Q_h matrices of dimension $n_x - 1$.

As in Section 5.1.3, we recognize that P_h is a symmetric and tridiagonal matrix, with diagonal entries of $1 + 2\gamma$ and off-diagonal entries of $-\gamma$, while Q_h is the identity matrix.

When considering stability in the ℓ_2 norm, we use the fact that P_h is strictly diagonally dominant to apply Geršgorin's theorem (Theorem 3.9) and show that $\|P_h^{-1}\|_2 = \|P_h^{-1}Q_h\|_2 \leq 1$. Similarly, we use Varah's theorem (Theorem 3.14) to show that $\|P_h^{-1}\|_\infty = \|P_h^{-1}Q_h\|_\infty \leq 1$, establishing Lax–Richtmyer stability in the ℓ_∞ norm.

In this setting, Neumann boundary conditions are more complicated to implement and analyze than in the elliptic setting of Section 3.4. Here, the rescaling used to restore symmetry when implementing Neumann boundary conditions using the ghost-point approach scales both P_h and Q_h , and, consequently, care must be taken in order to be able to apply Geršgorin's or Varah's theorem to establish stability.

As above, even for explicit finite-difference schemes, where P_h is the identity, bounding the norms of the operators can be somewhat tedious or require technical analysis. Thus, proving stability (and, as a result, convergence) can be quite complicated. Moreover, implicit schemes, such as Crank–Nicolson, where both P_h and Q_h are tridiagonal matrices, can make the analysis even harder, particularly in the nonsymmetric case. In both examples above, our work could also have been simplified by assuming periodic boundary conditions, leading to circulant matrices, where the eigenvalue analysis gets somewhat easier to handle. We now introduce another general way of proving stability that uses Fourier analysis and Toeplitz matrices, which includes the periodic case. This process is referred to as *von Neumann stability analysis*. In order to use this analysis, though, we must consider problems on infinite grids or periodic domains. This restricts the analysis in the sense that it does not guarantee stability for all types of boundary conditions but generally turns out to be a sufficient condition for stability in practice.

5.3.2 ■ Discrete-time Fourier transforms

We first consider von Neumann analysis for problems on infinite grids. In this setting, the main tool that we use to determine bounds on the norms of finite-difference operators is the Fourier transform. Given an infinite-grid vector, $\mathbf{x}$, only on the spatial domain, we define $\hat{\mathbf{x}}(\theta) = \sum_k x_k e^{-i\theta k}$ to be the Fourier transform of $\mathbf{x}$. To recover $\mathbf{x}$ from $\hat{\mathbf{x}}(\theta)$, we use the definition of the inverse Fourier transform,

$$x_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} \hat{\mathbf{x}}(\theta) e^{i\theta k} d\theta = \frac{1}{2\pi} \int_{-\pi}^{\pi} \left(\sum_j x_j e^{-i\theta j} \right) e^{i\theta k} d\theta \quad (5.72a)$$

$$= \sum_j x_j \left(\frac{1}{2\pi} \int_{-\pi}^{\pi} e^{i\theta(k-j)} d\theta \right) = x_k, \quad (5.72b)$$

noting that

$$\int_{-\pi}^{\pi} e^{i\theta(k-j)} d\theta = \begin{cases} 2\pi & \text{if } k = j, \\ 0 & \text{otherwise.} \end{cases} \quad (5.73)$$

Further, since

$$|\hat{\mathbf{x}}(\theta)| = \left| \sum_k x_k e^{-i\theta k} \right| \leq \sum_k |x_k e^{-i\theta k}| = \sum_k |x_k|, \quad (5.74)$$

$\widehat{\mathbf{x}}(\theta)$ is well defined when $\sum_k |x_k| < \infty$ —i.e., its existence is determined by the absolute convergence of the series.

In general, since we care about functions in L^2 (see equation (5.39)), we consider

$$\widehat{\mathbf{x}}(\theta) \in L^2([-\pi, \pi]) = \left\{ f : [-\pi, \pi] \rightarrow \mathbb{C} \left| \int_{-\pi}^{\pi} |f(\theta)|^2 d\theta < \infty \right. \right\}. \quad (5.75)$$

For $f \in L^2([-\pi, \pi])$, we define $\|f\|^2 = \int_{-\pi}^{\pi} |f(\theta)|^2 d\theta$. Parseval's theorem, Theorem 5.16, establishes a relation between the ℓ_2 norms of $\mathbf{x}$ and $\widehat{\mathbf{x}}(\theta)$.

Theorem 5.16: Parseval's theorem

$$\text{If } \|\mathbf{x}\| < \infty, \text{ then } \sum_k |x_k|^2 = \frac{1}{2\pi} \int_{-\pi}^{\pi} |\widehat{\mathbf{x}}(\theta)|^2 d\theta.$$

Using Parseval's theorem plus the weighted Euclidean norm, $\|\mathbf{x}\|^2 = h_x \sum_k |x_k|^2$, we get that

$$\|P_h^{-1}\|^2 = \sup_{\mathbf{x} \neq 0} \frac{\|P_h^{-1}\mathbf{x}\|^2}{\|\mathbf{x}\|^2} = \sup_{\mathbf{x} \neq 0} \frac{\|P_h^{-1}\mathbf{x}\|^2}{\frac{h_x}{2\pi} \int_{-\pi}^{\pi} |\widehat{\mathbf{x}}(\theta)|^2 d\theta}, \quad (5.76a)$$

$$\|P_h^{-1}Q_h\|^2 = \sup_{\mathbf{x} \neq 0} \frac{\|P_h^{-1}Q_h\mathbf{x}\|^2}{\|\mathbf{x}\|^2} = \sup_{\mathbf{x} \neq 0} \frac{\|P_h^{-1}Q_h\mathbf{x}\|^2}{\frac{h_x}{2\pi} \int_{-\pi}^{\pi} |\widehat{\mathbf{x}}(\theta)|^2 d\theta}. \quad (5.76b)$$

Thus, if we can express $\|P_h^{-1}Q_h\mathbf{x}\|^2$ and $\|P_h^{-1}\mathbf{x}\|^2$ in terms of $\int_{-\pi}^{\pi} |\widehat{\mathbf{x}}(\theta)|^2 d\theta$, we can compute (or bound) the norms as needed to see if a scheme is stable. As it turns out, for our finite-difference schemes on uniform meshes, P_h and Q_h are both always operators of Toeplitz form.

Definition 5.17: Toeplitz operators

The operator T is a Toeplitz operator if $(T\mathbf{x})_j = \sum_k x_k t_{j-k}$. We call $\mathbf{t}$ its Toeplitz vector.

For a finite matrix, Toeplitz form means that the matrix elements on each diagonal are constant (see Definition 3.2). For finite-difference operators, Toeplitz form means that coefficients of the stencil between two grid points depend only on their relative locations and are independent of where in the grid you look. Example 5.18 shows the Toeplitz form for the ETCS scheme of the one-way wave equation.

Example 5.18: Toeplitz operators for the one-way wave equation

Let $\gamma = \frac{ah_t}{2h_x}$. For the ETCS scheme,

$$u_{k,\ell+1} = \gamma u_{k-1,\ell} + u_{k,\ell} - \gamma u_{k+1,\ell}, \quad (5.77)$$

we have

$$(P_h \mathbf{u}_{\ell+1})_j = u_{j,\ell+1} = \sum_k u_{k,\ell+1} p_{j-k} \quad (5.78a)$$

$$\text{for } p_{j-k} = \begin{cases} 1 & \text{if } k = j, \\ 0 & \text{otherwise,} \end{cases} \quad \text{i.e.,} \quad p_m = \begin{cases} 1 & \text{if } m = 0, \\ 0 & \text{otherwise.} \end{cases} \quad (5.78b)$$

This corresponds to P_h being the identity operator since the method is explicit. Similarly,

$$(Q_h \mathbf{u}_\ell)_j = \gamma u_{j-1,\ell} + u_{j,\ell} - \gamma u_{j+1,\ell} = \sum_k u_{k,\ell} q_{j-k} \quad (5.79a)$$

$$\text{for } q_{j-k} = \begin{cases} \gamma & \text{if } k = j-1, \\ 1 & \text{if } k = j, \\ -\gamma & \text{if } k = j+1, \\ 0 & \text{otherwise,} \end{cases} \quad \text{i.e.,} \quad q_m = \begin{cases} \gamma & \text{if } m = 1, \\ 1 & \text{if } m = 0, \\ -\gamma & \text{if } m = -1, \\ 0 & \text{otherwise.} \end{cases} \quad (5.79b)$$

Both P_h and Q_h are Toeplitz operators: $P_h = I$ and $Q_h = \text{tridiag}(\gamma, 1, -\gamma)$.

In general, the operators we consider in all of the two-level finite-difference schemes we have discussed are Toeplitz operators. Moreover, the Fourier transform of $y_j = \sum_k x_k p_{j-k}$ for a given Toeplitz vector $\mathbf{p}$ takes on a convenient form:

$$\hat{\mathbf{y}}(\theta) = \sum_j \left(\sum_k x_k p_{j-k} \right) e^{-i\theta j} \quad (5.80a)$$

$$= \sum_j \left(\sum_k \left(\frac{1}{2\pi} \int_{-\pi}^{\pi} \hat{\mathbf{x}}(\varphi) e^{i\varphi k} d\varphi \right) p_{j-k} \right) e^{-i\theta j} \quad (5.80b)$$

$$= \frac{1}{2\pi} \int_{-\pi}^{\pi} \hat{\mathbf{x}}(\varphi) \sum_j \sum_k p_{j-k} e^{i\varphi k} e^{-i\theta j} d\varphi \quad (5.80c)$$

$$= \frac{1}{2\pi} \int_{-\pi}^{\pi} \hat{\mathbf{x}}(\varphi) \sum_j \left(\sum_k p_{j-k} e^{i\varphi(k-j)} \right) e^{i(\varphi-\theta)j} d\varphi. \quad (5.80d)$$

Here, for any j , we rename the index on the innermost summation to get

$$\sum_{k=-\infty}^{\infty} p_{j-k} e^{i\varphi(k-j)} = \sum_{m=-\infty}^{\infty} p_m e^{-i\varphi m} = \hat{\mathbf{p}}(\varphi), \quad (5.81)$$

which is the Fourier transform of the Toeplitz vector $\mathbf{p}$. Thus,

$$\hat{\mathbf{y}}(\theta) = \int_{-\pi}^{\pi} \hat{\mathbf{x}}(\varphi) \hat{\mathbf{p}}(\varphi) \left(\frac{1}{2\pi} \sum_j e^{i(\varphi-\theta)j} \right) d\varphi. \quad (5.82)$$

Note the following properties:

- the function $\hat{\mathbf{w}}(\theta) = \frac{1}{2\pi} \sum_j e^{i(\varphi-\theta)j}$ in equation (5.82) is the Fourier transform of the vector $\mathbf{w}$, where $w_j = \frac{1}{2\pi} e^{i\varphi j}$;
- by taking the inverse Fourier transform, $w_j = \frac{1}{2\pi} \int_{-\pi}^{\pi} \hat{\mathbf{w}}(\theta) e^{i\theta j} d\theta$;
- also $w_j = \frac{1}{2\pi} e^{i\varphi j} = \frac{1}{2\pi} \int_{-\pi}^{\pi} \delta(\varphi - \theta) e^{i\theta j} d\theta$, where $\delta(\varphi)$ is the Dirac delta function.

Therefore,

$$\hat{\mathbf{w}}(\theta) = \delta(\varphi - \theta) \quad (5.83)$$

and

$$\hat{\mathbf{y}}(\theta) = \int_{-\pi}^{\pi} \hat{\mathbf{x}}(\varphi) \hat{\mathbf{p}}(\varphi) \delta(\varphi - \theta) d\varphi = \hat{\mathbf{x}}(\theta) \hat{\mathbf{p}}(\theta), \quad (5.84)$$

so we finally obtain that

$$\hat{\mathbf{y}}(\theta) = \hat{\mathbf{x}}(\theta)\hat{\mathbf{p}}(\theta) \text{ when } y_j = \sum_k x_k p_{j-k}. \quad (5.85)$$

This is the well-known result that convolution in the spatial domain corresponds to multiplication in the Fourier domain, so we see that applying a Toeplitz operator to an infinite-grid vector is equivalent to multiplication of the Fourier transform of the vector by the Fourier transform of the Toeplitz vector. Hence, $\hat{\mathbf{p}}(\theta)\hat{\mathbf{x}}(\theta)$ is the Fourier transform of $P_h \mathbf{x}$.

Also note that, since $\hat{\mathbf{x}}(\theta) = \frac{1}{\hat{\mathbf{p}}(\theta)}\hat{\mathbf{p}}(\theta)\hat{\mathbf{x}}(\theta)$ and $\hat{\mathbf{x}}(\theta)$ is the Fourier transform of $\mathbf{x} = P_h^{-1}P_h \mathbf{x}$, the Fourier transform of the Toeplitz vector for P_h^{-1} is $\frac{1}{\hat{\mathbf{p}}(\theta)}$. Thus, in general, multiplication by the inverse of a Toeplitz operator leads to division by the Fourier transform of the Toeplitz vector. This then tells us that the Fourier transform of the Toeplitz vector for $P_h^{-1}Q_h$ is $\frac{\hat{\mathbf{q}}(\theta)}{\hat{\mathbf{p}}(\theta)}$.

5.3.3 ■ Von Neumann stability of two-level schemes

We now return to trying to bound $\|P_h^{-1}Q_h\|$ and $\|P_h^{-1}\|$. From the proof of Lax's convergence theorem (see Theorem 5.12 and equation (5.53) in particular), we see how bounding these operator norms bounds the error at each timestep and, hence, with consistency, implies convergence. Using Parseval's theorem and the properties of Toeplitz operators, we arrive at the following:

$$\|P_h^{-1}Q_h\|^2 = \sup_{\mathbf{x} \neq 0} \frac{\|P_h^{-1}Q_h \mathbf{x}\|^2}{\|\mathbf{x}\|^2} = \sup_{\mathbf{x} \neq 0} \frac{\frac{h_x}{2\pi} \int_{-\pi}^{\pi} |\frac{\hat{\mathbf{q}}(\theta)}{\hat{\mathbf{p}}(\theta)} \hat{\mathbf{x}}(\theta)|^2 d\theta}{\frac{h_x}{2\pi} \int_{-\pi}^{\pi} |\hat{\mathbf{x}}(\theta)|^2 d\theta} \quad (5.86a)$$

$$\leq \sup_{\mathbf{x} \neq 0} \frac{\max_{\varphi} |\frac{\hat{\mathbf{q}}(\varphi)}{\hat{\mathbf{p}}(\varphi)}|^2 \int_{-\pi}^{\pi} |\hat{\mathbf{x}}(\theta)|^2 d\theta}{\int_{-\pi}^{\pi} |\hat{\mathbf{x}}(\theta)|^2 d\theta} = \max_{\varphi} \left| \frac{\hat{\mathbf{q}}(\varphi)}{\hat{\mathbf{p}}(\varphi)} \right|^2. \quad (5.86b)$$

Similarly,

$$\|P_h^{-1}\|^2 \leq \max_{\varphi} \left| \frac{1}{\hat{\mathbf{p}}(\varphi)} \right|^2. \quad (5.87)$$

Moreover, it is easy to see that the inequalities in equations (5.86b) and (5.87) are actually equalities. By choosing $\mathbf{x}$ as a specific Fourier mode, $x_k = \frac{1}{2\pi} e^{i\varphi^* k}$, where φ^* is where the maximum is attained, we get

$$\|P_h^{-1}Q_h\| = \max_{\varphi} \left| \frac{\hat{\mathbf{q}}(\varphi)}{\hat{\mathbf{p}}(\varphi)} \right|, \quad \|P_h^{-1}\| = \max_{\varphi} \left| \frac{1}{\hat{\mathbf{p}}(\varphi)} \right|. \quad (5.88)$$

To see this, note that the Fourier mode $\mathbf{x}$ with $x_k = \frac{1}{2\pi} e^{i\varphi k}$ has Fourier transform $\hat{\mathbf{x}}(\theta) = \delta(\varphi - \theta)$ (see equation (5.83)), so the Fourier transform of $\mathbf{y} = P_h^{-1}Q_h \mathbf{x}$ is given by $\frac{\hat{\mathbf{q}}(\theta)}{\hat{\mathbf{p}}(\theta)} \delta(\varphi - \theta)$. By using the inverse Fourier transform, we find that

$$y_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} \frac{\hat{\mathbf{q}}(\theta)}{\hat{\mathbf{p}}(\theta)} \delta(\varphi - \theta) e^{i\theta k} d\theta \quad (5.89a)$$

$$= \frac{1}{2\pi} \frac{\hat{\mathbf{q}}(\varphi)}{\hat{\mathbf{p}}(\varphi)} e^{i\varphi k} \quad (5.89b)$$

$$= \frac{\hat{\mathbf{q}}(\varphi)}{\hat{\mathbf{p}}(\varphi)} x_k. \quad (5.89c)$$

This shows that any Fourier mode with $x_k = \frac{1}{2\pi} e^{i\varphi k}$ is an eigenvector of $P_h^{-1}Q_h$ with eigenvalue $\frac{\widehat{q}(\varphi)}{\widehat{p}(\varphi)}$: when $\mathbf{x}$ is a Fourier mode with $x_k = \frac{1}{2\pi} e^{i\varphi k}$, we have that

$$P_h^{-1}Q_h \mathbf{x} = \frac{\widehat{q}(\varphi)}{\widehat{p}(\varphi)} \mathbf{x}, \quad (5.90)$$

and the equivalent result holds for $P_h^{-1}\mathbf{x}$ (and, in fact, any Toeplitz operator). This leads directly to equation (5.88) by taking into account the bounds (5.86b) and (5.87).

So, to compute the norms of the operators $P_h^{-1}Q_h$ and P_h^{-1} , we simply compute the Fourier transforms of the Toeplitz vectors corresponding to the finite-difference scheme operators and maximize the appropriate quotient involving $\widehat{q}(\varphi)$ and $\widehat{p}(\varphi)$. For further use, we introduce the following definition of the amplification factor in Definition 5.19, sometimes referred to as the symbol of the two-level finite-difference scheme.

Definition 5.19: Amplification factor of a two-level finite-difference scheme

Consider a two-level linear finite-difference scheme, $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + \mathbf{b}_\ell h_t$, where P_h and Q_h are Toeplitz operators with Toeplitz vectors p_k and q_k , and let

$$\widehat{p}(\varphi) = \sum_k p_k e^{-i\varphi k} \text{ and } \widehat{q}(\varphi) = \sum_k q_k e^{-i\varphi k} \quad (5.91)$$

be the Fourier transforms of the Toeplitz vectors. Then, the amplification factor (or symbol) of the two-level finite-difference method is given by

$$s(\varphi) = \frac{\widehat{q}(\varphi)}{\widehat{p}(\varphi)}. \quad (5.92)$$

With this notation, we summarize the above discussion in the following definition of von Neumann stability, Definition 5.20.

Definition 5.20: Von Neumann stability

Consider a two-level linear finite-difference scheme, $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + \mathbf{b}_\ell h_t$, with P_h and Q_h Toeplitz operators and amplification factor $s(\varphi)$. If

$$\max_\varphi |s(\varphi)| \leq 1 \quad \text{and} \quad \max_\varphi \left| \frac{1}{\widehat{p}(\varphi)} \right| \leq c \quad (5.93)$$

for some constant $c > 0$, then we say the scheme is von Neumann stable.

It is clear from equation (5.88) that von Neumann stability is equivalent to Lax–Richtmyer stability in the case of infinite grids and hence implies stability as defined in Definition 5.9 and as used as a condition for convergence in the Lax convergence theorem, Theorem 5.12. It can be shown (see more on this below) that von Neumann stability also implies Lax–Richtmyer stability in the case of finite domains with periodic boundary conditions.

Remark 5.21: Bounding only the amplification factor

Often, the process of bounding the amplification factor alone in equation (5.93) is referred to as von Neumann stability analysis. However, we actually need $\|(P_h^{-1}Q_h)^\ell P_h^{-1}\| \leq c$ for stability, and bounding the amplification factor only checks for $\|P_h^{-1}Q_h\| \leq 1$. Therefore, the condition $\|P_h^{-1}\| \leq c$ is also required to prove stability in the ℓ_2 norm. For explicit schemes, $P_h = I$, so we always have that $\|P_h^{-1}\| \leq 1$. For implicit schemes, the condition $\|P_h^{-1}\| \leq c$ also almost

always holds, since if there is no constant c such that $\|P_h^{-1}\| \leq c \forall h_x, h_t$, then P_h becomes increasingly singular as the grid is refined, and problems also occur with $\|P_h^{-1}Q_h\|$. This also relates to our discussion in Section 5.2.2 of the definition of truncation error and Lax–Richtmyer stability. So, most of the time, showing consistency and a bound on the amplification factor is good enough to expect convergence of the scheme.

Finally, we note that von Neumann stability analysis is usually also a good indicator for stability in the ℓ_2 norm on finite grids with boundary conditions that are different from periodic. However, if a scheme is not von Neumann stable, then it is usually also not Lax–Richtmyer stable when the grid is finite with boundary conditions different from periodic. Thus, we often do not worry about differences with the case where von Neumann analysis can be strictly applied and use it directly as an indicator of stability (or lack thereof) of a scheme.

Von Neumann stability on finite periodic domains

While our derivation of von Neumann stability analysis focuses on the infinite-grid case, it also provides a sufficient condition for Lax–Richtmyer stability in the case of finite uniform grids with periodic boundary conditions. This follows from the fact that the discretization matrices associated with finite periodic grids are circulant matrices (which are a subset of Toeplitz matrices), with eigenvalues given by *sampling* the amplification factor at a finite set of equidistant points in the interval from zero to 2π . Thus, knowing that a scheme is von Neumann stable (i.e., the bounds in Definition 5.20 hold for all ϕ) clearly implies that the same bounds hold for any finite subset of ϕ values on the same interval, and this ensures Lax–Richtmyer stability for the finite circulant matrices P_h and Q_h on the periodic grid.

Specifically, consider a periodic domain, $\Omega = [0, L]$, of length L , and an equispaced grid with $n_x + 1$ grid points given by

$$0 = x_0 < x_1 < \cdots < x_{n_x} = L \quad (5.94)$$

and grid spacing

$$h_x = \frac{L}{n_x}. \quad (5.95)$$

Consider a two-level finite-difference method $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + \mathbf{b}_\ell h_t$ for unknowns $\mathbf{u}_\ell = (u_{0,\ell}, u_{1,\ell}, \dots, u_{n_x-1,\ell})^T \in \mathbb{R}^{n_x}$. By periodicity,

$$u_{0,\ell} = u_{n_x,\ell} \quad \forall \ell, \quad (5.96)$$

so we do not include $u_{n_x,\ell}$ in $\mathbf{u}_\ell$. As usual, we assume that $P_h, Q_h \in \mathbb{R}^{n_x \times n_x}$ are Toeplitz matrices, but, in addition, we assume that they are circulant matrices by periodicity.

Next, consider the error-propagation formula

$$\mathbf{e}_{\ell+1} = (P_h^{-1}Q_h)\mathbf{e}_\ell + h_t P_h^{-1} \boldsymbol{\tau}_\ell \quad (5.97)$$

from equation (5.50) in the proof of the Lax convergence theorem. The *discrete Fourier transform* $\hat{\mathbf{e}}_\ell \in \mathbb{C}^{n_x}$ of error $\mathbf{e}_\ell \in \mathbb{R}^{n_x}$ is defined by

$$\mathbf{e}_\ell = \sum_{m=0}^{n_x-1} \hat{\mathbf{e}}_{m,\ell} \mathbf{f}_{\theta_m}, \quad (5.98)$$

where the n_x vectors $\mathbf{f}_{\theta_m} \in \mathbb{C}^{n_x}$ are the Fourier basis vectors given by

$$f_{k,\theta_m} = e^{ik\theta_m}, \quad (5.99)$$

with the *phase angle* $\theta_m \in [0, 2\pi)$ given by

$$\theta_m = 2\pi \frac{m}{n_x}, \quad m = 0, \dots, n_x - 1. \quad (5.100)$$

Equation (5.98) shows that the discrete Fourier transform decomposes a spatial vector $e_\ell \in \mathbb{R}^{n_x}$ as a linear combination of the n_x Fourier basis vectors f_{θ_m} , and the discrete Fourier transform vector $\widehat{e}_\ell$ contains the coefficients of the linear combination.

To provide some more insight, we rewrite expression (5.99) for the components of the Fourier basis vectors in terms of a wavelength as follows:

$$f_{k, \theta_m} = e^{ik\theta_m}, \quad (5.101a)$$

$$= e^{i2\pi x_k / \lambda_m}. \quad (5.101b)$$

Here, $x_k = k h_x$ is the k th grid point, and the m th *wavelength*, λ_m , is given by

$$\lambda_m = \frac{L}{m}. \quad (5.102)$$

The wavelength is the length of one full sinusoidal oscillation. For example, the Fourier mode with $m = 1$ describes oscillations with wavelength $\lambda_1 = L$ (one oscillation over the length of the domain), and the wavelength of the $m = 2$ mode is given by $\lambda_2 = L/2$, so there are $m = 2$ oscillations in the domain. This corresponds to phase angles $\theta_1 = 2\pi \frac{1}{n_x}$ and $\theta_2 = 2\pi \frac{2}{n_x}$, respectively. The $m = 0$ mode corresponds to the constant mode, with $\theta_0 = 0$ (one can think of this as a wave with infinite wavelength).

The crucial point is now that the n_x Fourier basis vectors, f_{θ_m} , form a basis of eigenvectors for any $n_x \times n_x$ circulant matrix. In particular, the f_{θ_m} are eigenvectors of $P_h^{-1}Q_h$ with eigenvalues $s(\theta_m)$, where $s(\varphi) = \frac{\widehat{q}(\varphi)}{\widehat{p}(\varphi)}$ is the symbol of $P_h^{-1}Q_h$. Therefore, the von Neumann stability condition $\max_\varphi |s(\varphi)| \leq 1$ implies that $\max_{\theta_m} |s(\theta)| \leq 1$. It can be shown that the ℓ_2 norm of a circulant matrix is the size of its largest eigenvalue, so $\|P_h^{-1}Q_h\|_2 = \max_{\theta_m} |s(\theta)| \leq 1$, and we conclude that von Neumann stability implies Lax–Richtmyer stability on periodic domains.

Finally, looking back at error-propagation equation (5.97), we now understand that the condition $\|P_h^{-1}Q_h\|_2 = \max_{\theta_m} |s(\theta)| \leq 1$ means that error vectors composed of any single Fourier mode on the periodic grid are damped in time. Considering equation (5.98), this implies by linearity that any error mode e_ℓ is damped in time, which is exactly what Lax–Richtmyer stability provides.

5.3.4 ■ Examples of von Neumann stability analysis

The following examples show how von Neumann analysis is performed in practice and what conclusions it provides.

We first consider the ETBS method for the one-way wave equation. Writing $\gamma = \frac{a h_t}{h_x}$, this scheme is given by

$$u_{k, \ell+1} = \gamma u_{k-1, \ell} + (1 - \gamma) u_{k, \ell}, \quad (5.103a)$$

$$P_h = I, \quad Q_h = \text{tridiag}(\gamma, 1 - \gamma, 0), \quad (5.103b)$$

$$p_k = \begin{cases} 1 & \text{if } k = 0, \\ 0 & \text{otherwise,} \end{cases} \quad q_k = \begin{cases} \gamma & \text{if } k = 1, \\ 1 - \gamma & \text{if } k = 0, \\ 0 & \text{otherwise.} \end{cases} \quad (5.103c)$$

Thus, recalling that the Fourier transform of the Toeplitz vector is given by $\widehat{\mathbf{p}}(\varphi) = \sum_k p_k e^{-i\varphi k}$, we obtain

$$\widehat{\mathbf{p}}(\varphi) = 1, \quad (5.104a)$$

$$\widehat{\mathbf{q}}(\varphi) = \gamma e^{-i\varphi} + (1 - \gamma) = 1 - \gamma(1 - e^{-i\varphi}) \quad (5.104b)$$

$$\Rightarrow s(\varphi) = 1 - \gamma(1 - e^{-i\varphi}). \quad (5.104c)$$

Checking $|s(\varphi)|^2$, we have

$$|s(\varphi)|^2 = (1 - \gamma(1 - e^{-i\varphi}))(1 - \gamma(1 - e^{i\varphi})) \quad (5.105a)$$

$$= 1 - 2\gamma + 2\gamma^2 + 2(\gamma - \gamma^2) \cos(\varphi), \quad (5.105b)$$

where we have used the following standard formulas:

$$e^{i\varphi} = \cos \varphi + i \sin \varphi, \quad \cos \varphi = \frac{e^{i\varphi} + e^{-i\varphi}}{2}, \quad \sin \varphi = \frac{e^{i\varphi} - e^{-i\varphi}}{2i}. \quad (5.106)$$

To find the maximum of the modulus with respect to the choice of φ , we differentiate:

$$\frac{d}{d\varphi}(1 - 2\gamma + 2\gamma^2 + 2(\gamma - \gamma^2) \cos \varphi) = -2(\gamma - \gamma^2) \sin \varphi. \quad (5.107)$$

This is equal to zero if and only if $\varphi = m\pi$, with $m \in \mathbb{Z}$. When $\varphi = m\pi$,

$$|s(\varphi)|^2 = 1 - 2\gamma + 2\gamma^2 + 2(\gamma - \gamma^2)(\pm 1). \quad (5.108)$$

If m is even, $|s(\varphi)|^2 = 1$, and if m is odd, $|s(\varphi)|^2 = 1 - 4\gamma + 4\gamma^2 = (1 - 2\gamma)^2$. Thus, $|s(\varphi)|^2 \leq 1$ if and only if

$$|1 - 2\gamma| \leq 1 \Leftrightarrow 0 \leq \gamma \leq 1 \Leftrightarrow 0 \leq h_t \leq \frac{h_x}{a}. \quad (5.109)$$

In other words, the ETBS scheme is von Neumann stable if and only if $h_t \leq \frac{h_x}{a}$, which requires $a \geq 0$. This is the same CFL condition we found in equation (5.69) using eigenvalue analysis for the Dirichlet case, but it is easier to compute in the current setting. As in that case, we note that ETBS is conditionally stable, meaning that it only converges when $a \geq 0$ and, given h_x , that h_t is chosen such that it satisfies the CFL condition. Again, our interpretation is that the backward spatial derivative leads to a stable scheme only when $a \geq 0$ and only when the timestep, h_t , is small enough. As before, this means that ETFS is stable only when $a \leq 0$ and $h_t \leq \frac{h_x}{-a}$.

We next revisit this calculation from a slightly different viewpoint, illustrating a simplified approach to von Neumann stability analysis that applies in some common cases and may provide some additional insight.

Again, consider the ETBS method for the one-way wave equation,

$$u_{k,\ell+1} = \gamma u_{k-1,\ell} + (1 - \gamma) u_{k,\ell}, \quad (5.110)$$

where $\gamma = \frac{ah_t}{h_x}$. With initial condition $u(x, 0) = f(x)$ on an infinite grid, the exact solution of the PDE is given by $u(x, t) = f(x - at)$. If $f(x)$ is bounded in $\mathbb{R}$, it is clear that $u(x, t)$ is bounded for all time $t \geq 0$, and it is natural to expect that a stable numerical method will produce numerical approximations that are also bounded for all t , assuming that the initial condition is bounded.

Consider an infinite equispaced grid with grid points

$$x_k = k h_x, \quad k \in \mathbb{Z}. \quad (5.111)$$

Assume that the numerical approximation at time t_ℓ is a Fourier mode with wavelength $\lambda \in \mathbb{R}$ and amplitude $\tilde{v}_\ell$, given by

$$u_{k,\ell} = \tilde{v}_\ell e^{i2\pi x_k/\lambda}, \quad (5.112a)$$

$$= \tilde{v}_\ell e^{i2\pi h_x k/\lambda}. \quad (5.112b)$$

The wavelength λ represents the length of one oscillation in the Fourier mode. Defining the *phase angle* φ as

$$\varphi = \frac{2\pi h_x}{\lambda}, \quad (5.113)$$

we write the Fourier mode with amplitude $\tilde{v}_\ell$ as

$$u_{k,\ell} = \tilde{v}_\ell e^{i\varphi k}, \quad (5.114)$$

where we can restrict φ to $[0, 2\pi]$ due to the periodicity of the complex exponential, noting that k is an integer.

We now plug $u_{k,\ell} = \tilde{v}_\ell e^{i\varphi k}$ into the update equation (5.110) to find the numerical approximation at time level $\ell + 1$,

$$u_{k,\ell+1} = \gamma \tilde{v}_\ell e^{i\varphi(k-1)} + (1 - \gamma) \tilde{v}_\ell e^{i\varphi k}, \quad (5.115a)$$

$$= \tilde{v}_\ell (\gamma e^{-i\varphi} + (1 - \gamma)) e^{i\varphi k}, \quad (5.115b)$$

$$= \tilde{v}_{\ell+1} e^{i\varphi k}, \quad (5.115c)$$

where

$$\tilde{v}_{\ell+1} = \tilde{v}_\ell (\gamma e^{-i\varphi} + (1 - \gamma)). \quad (5.116)$$

We find that, if $\mathbf{u}_\ell$ is a Fourier mode as in equation (5.114), then $\mathbf{u}_{\ell+1}$ is also a Fourier mode, with amplitude $\tilde{v}_{\ell+1}$ given in equation (5.116). This, in fact, occurs for all two-level finite-difference methods $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + \mathbf{b}_\ell h_t$ with $\mathbf{b}_\ell = \mathbf{0}$ and P_h and Q_h being Toeplitz operators, since then all Fourier modes are eigenvectors of P_h and Q_h . We can then define the *symbol* of the finite-difference method as the ratio of the Fourier amplitudes $\tilde{v}_{\ell+1}$ and $\tilde{v}_\ell$:

$$s(\varphi) = \frac{\tilde{v}_{\ell+1}}{\tilde{v}_\ell} = \gamma e^{-i\varphi} + (1 - \gamma). \quad (5.117)$$

The symbol now has a simple interpretation: $|s(\varphi)|$ is the growth (or amplification) factor of the amplitude of a Fourier mode with phase angle φ for one iteration of the finite-difference method. Clearly, for the method to be stable, we require

$$\max_{\varphi} |s(\varphi)| \leq 1, \quad (5.118)$$

because otherwise there would be Fourier initial conditions for which the amplitude would grow without bound as ℓ increases, and there could not be convergence to the exact solution.

This simplified analysis, which applies when $\mathbf{b}_\ell = \mathbf{0}$ in equation (5.21), is, of course, closely related to the more formal presentation of von Neumann analysis that we gave earlier in this section. For example, the symbol we obtain in equation (5.117) by taking the ratio of the Fourier amplitudes $\tilde{v}_{\ell+1}$ and $\tilde{v}_\ell$ is, of course, equal to the ratio of the Fourier transforms of the Toeplitz vectors of the Q_h and P_h operators, as given in equation (5.92), and we still have that $\|P_h^{-1} Q_h\|_2 = \max_{\varphi} |s(\varphi)| \leq 1$, which implies Lax–Richtmyer stability and, hence, convergence.

However, we have learned in this example that the symbol can also be computed simply by plugging the Fourier mode of equation (5.114) into the propagation equation (with $\mathbf{b}_\ell = \mathbf{0}$) and

computing the ratio of the amplitudes at time levels $\ell + 1$ and ℓ , and we have illustrated a simple interpretation of the size of the symbol as the growth (or amplification) factor of the amplitude of a Fourier mode over one timestep, similar to what we learned in Section 5.3.3 for the periodic case.

Letting $\gamma = \frac{ah_t}{2h_x}$, the ETCS scheme is given by

$$u_{k,\ell+1} = \gamma u_{k-1,\ell} + u_{k,\ell} - \gamma u_{k+1,\ell}, \quad (5.119a)$$

$$P_h = I, \quad Q_h = \text{tridiag}(\gamma, 1, -\gamma), \quad (5.119b)$$

$$p_k = \begin{cases} 1 & \text{if } k = 0, \\ 0 & \text{otherwise,} \end{cases} \quad q_k = \begin{cases} \gamma & \text{if } k = 1, \\ 1 & \text{if } k = 0, \\ -\gamma & \text{if } k = -1, \\ 0 & \text{otherwise.} \end{cases} \quad (5.119c)$$

Thus,

$$\hat{p}(\varphi) = e^{-i\varphi(0)} = 1, \quad (5.120a)$$

$$\hat{q}(\varphi) = \gamma e^{-i\varphi(1)} + e^{-i\varphi(0)} - \gamma e^{-i\varphi(-1)} \quad (5.120b)$$

$$= \gamma e^{-i\varphi} + 1 - \gamma e^{i\varphi} \quad (5.120c)$$

$$= 1 - \frac{ah_t}{h_x} \sin(\varphi)i \quad (5.120d)$$

$$\Rightarrow s(\varphi) = \frac{\hat{q}(\varphi)}{\hat{p}(\varphi)} = \frac{1 - \frac{ah_t}{h_x} \sin(\varphi)i}{1}. \quad (5.120e)$$

To investigate under which conditions von Neumann stability may hold, we consider $|s(\varphi)|^2$:

$$|s(\varphi)|^2 = \left(1 + \frac{a^2 h_t^2}{h_x^2} \sin^2(\varphi) \right) \geq 1 \quad (5.121a)$$

$$\Rightarrow \max_{\varphi} |s(\varphi)| > 1 \text{ since } \frac{|a|h_t}{h_x} > 0. \quad (5.121b)$$

Thus, ETCS is never von Neumann stable for any choice of the timestep h_t . Since it is not stable, it is not convergent.

With $\gamma = \frac{ah_t}{4h_x}$, the Crank–Nicolson scheme is given by

$$-\gamma u_{k-1,\ell+1} + u_{k,\ell+1} + \gamma u_{k+1,\ell+1} = \gamma u_{k-1,\ell} + u_{k,\ell} - \gamma u_{k+1,\ell}, \quad (5.122a)$$

$$P_h = \text{tridiag}(-\gamma, 1, \gamma), \quad Q_h = \text{tridiag}(\gamma, 1, -\gamma), \quad (5.122b)$$

$$p_k = \begin{cases} -\gamma & \text{if } k = 1, \\ 1 & \text{if } k = 0, \\ \gamma & \text{if } k = -1, \\ 0 & \text{otherwise,} \end{cases} \quad q_k = \begin{cases} \gamma & \text{if } k = 1, \\ 1 & \text{if } k = 0, \\ -\gamma & \text{if } k = -1, \\ 0 & \text{otherwise.} \end{cases} \quad (5.122c)$$

Thus,

$$\hat{p}(\varphi) = -\gamma e^{-i\varphi} + 1 + \gamma e^{i\varphi} = 1 + 2\gamma i \sin(\varphi), \quad (5.123a)$$

$$\hat{q}(\varphi) = \gamma e^{-i\varphi} + 1 - \gamma e^{i\varphi} = 1 - 2\gamma i \sin(\varphi) \quad (5.123b)$$

$$\Rightarrow s(\varphi) = \frac{1 - 2\gamma i \sin(\varphi)}{1 + 2\gamma i \sin(\varphi)}. \quad (5.123c)$$

Checking $|s(\varphi)|^2$,

$$|s(\varphi)|^2 = \frac{|1 - 2\gamma i \sin(\varphi)|^2}{|1 + 2\gamma i \sin(\varphi)|^2} \quad (5.124a)$$

$$= \frac{1 + 4\gamma^2 \sin^2(\varphi)}{1 + 4\gamma^2 \sin^2(\varphi)} = 1. \quad (5.124b)$$

Thus, $\max_{\varphi} |s(\varphi)| = 1$, and the Crank–Nicolson scheme is unconditionally von Neumann stable and converges unconditionally, i.e., for any choice of the timestep h_t . Again, this result could be found using the eigenvalue analysis for Lax–Richtmyer stability, but the calculations are much easier with the approach taken here.

We next consider both explicit and implicit schemes for the heat equation (see equation (5.10)). Taking $\gamma = \frac{Dh_t}{h_x^2}$, we write the explicit Euler scheme for the heat equation as

$$u_{k,\ell+1} = u_{k,\ell} + \gamma(u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}), \quad (5.125a)$$

$$P_h = I, \quad Q_h = \text{tridiag}(\gamma, 1 - 2\gamma, \gamma), \quad (5.125b)$$

$$p_k = \begin{cases} 1 & \text{if } k = 0, \\ 0 & \text{otherwise,} \end{cases} \quad q_k = \begin{cases} \gamma & \text{if } k = 1, \\ 1 - 2\gamma & \text{if } k = 0, \\ \gamma & \text{if } k = -1, \\ 0 & \text{otherwise.} \end{cases} \quad (5.125c)$$

Thus,

$$\widehat{p}(\varphi) = 1, \quad (5.126a)$$

$$\widehat{q}(\varphi) = \gamma e^{-i\varphi} + (1 - 2\gamma) + \gamma e^{i\varphi} = 1 - 2\gamma + 2\gamma \cos(\varphi) \quad (5.126b)$$

$$\Rightarrow s(\varphi) = 1 + 2\gamma(\cos(\varphi) - 1). \quad (5.126c)$$

Since we want $|s(\varphi)| \leq 1$, we need

$$-2 \leq 2\gamma(\cos(\varphi) - 1) \leq 0. \quad (5.127)$$

Since $-1 \leq \cos(\varphi) \leq 1$, we have $-2 \leq \cos(\varphi) - 1 \leq 0$, and the upper bound is always satisfied. Then, to satisfy the lower bound, we need

$$-2 \leq -4\gamma, \quad (5.128a)$$

$$\frac{1}{2} \geq \frac{Dh_t}{h_x^2} \quad (5.128b)$$

$$\Rightarrow h_t \leq \frac{h_x^2}{2D}. \quad (5.128c)$$

Notice that we need $h_t = O(h_x^2)$, which is much smaller than the typical CFL requirement for hyperbolic PDEs. Nevertheless, explicit Euler is conditionally stable, and thus convergent, for the 1D heat equation.

For the implicit Euler scheme, we again take $\gamma = \frac{Dh_t}{h_x^2}$ and write the scheme as

$$u_{k,\ell+1} - \gamma(u_{k+1,\ell+1} - 2u_{k,\ell+1} + u_{k-1,\ell+1}) = u_{k,\ell}, \quad (5.129a)$$

$$P_h = \text{tridiag}(-\gamma, 1 + 2\gamma, -\gamma), \quad Q_h = I, \quad (5.129b)$$

$$p_k = \begin{cases} -\gamma & \text{if } k = 1, \\ 1 + 2\gamma & \text{if } k = 0, \\ -\gamma & \text{if } k = -1, \\ 0 & \text{otherwise,} \end{cases} \quad q_k = \begin{cases} 1 & \text{if } k = 0, \\ 0 & \text{otherwise.} \end{cases} \quad (5.129c)$$

Thus,

$$\hat{p}(\varphi) = -\gamma e^{-i\varphi} + (1 + 2\gamma) - \gamma e^{i\varphi} = 1 + 2\gamma - 2\gamma \cos(\varphi), \quad (5.130a)$$

$$\hat{q}(\varphi) = 1 \quad (5.130b)$$

$$\Rightarrow s(\varphi) = \frac{1}{1 - 2\gamma(\cos(\varphi) - 1)}. \quad (5.130c)$$

Again, we want $|s(\varphi)| \leq 1$, implying that

$$1 \leq |1 - 2\gamma(\cos(\varphi) - 1)| = |1 + 2\gamma(1 - \cos(\varphi))|, \quad (5.131)$$

which is always true since $1 - \cos(\varphi) \geq 0$ and $D \geq 0$. Thus, implicit Euler is von Neumann stable for all values of γ , i.e., for all timesteps h_t . We call this scheme unconditionally stable.

A natural question to ask at this point is what the consequences are for violating stability. Theorem 5.13 (the Lax equivalence theorem) tells us that a consistent scheme is stable *if and only if* it is convergent, meaning that we must see some form of divergence when we apply a scheme outside of its stability region. To explore this, we take a closer look at the explicit Euler scheme for the heat equation, which we know has a CFL requirement that $h_t \leq \frac{h_x^2}{2D}$. We pose the problem on the interval $0 < x < 1$ with homogeneous Dirichlet boundary conditions and $D = 1$, discretized on a uniform grid with $h_x = \frac{1}{16}$. To explore the effects of violating the CFL condition, we choose two values for h_t , one “good” value $h_t = 0.48 h_x^2$ that obeys the CFL condition, and a “bad” value $h_t = 0.528 h_x^2$ that violates the CFL condition. With a ratio of 1.1 between these two timestep sizes, if we integrate 11 steps with the smaller value, it obtains the same final time value as integrating 10 steps with the larger value.

In the left panel of Figure 5.3.1, we consider the initial condition $u(x, 0) = \sin(\pi x)$, which we integrate forward in time to $t = 5.28 h_x^2$ using either 11 steps with the smaller value of h_t or 10 steps with the larger. At the scale of this plot, both solutions look equally good and are visually indistinguishable from the exact solution. This might lead us to question the necessity of satisfying the CFL condition, if not for the counterpoint in the right panel of Figure 5.3.1. There, we start with a much more oscillatory initial condition, $u(x, 0) = \sin(15\pi x)$, and integrate for the same length of time with the same two choices of h_t . Here, the exact solution,

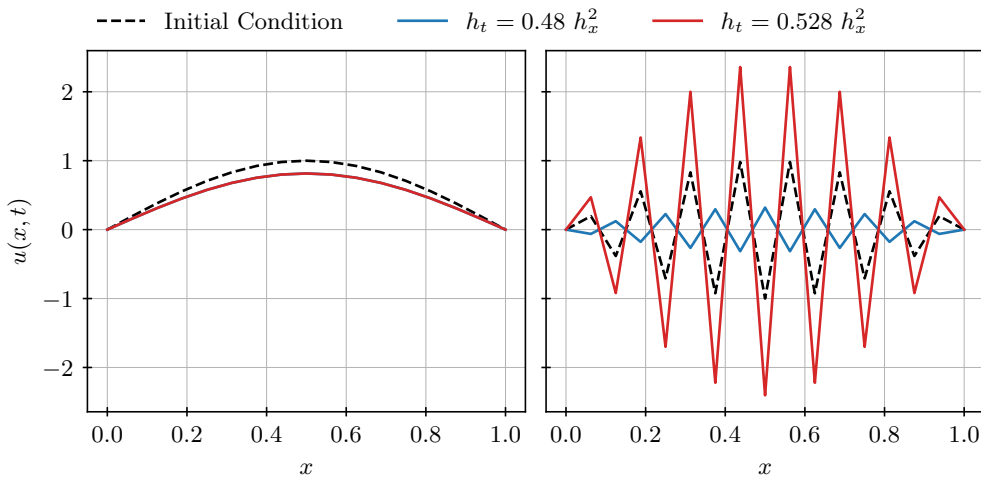


Figure 5.3.1. Convergence of explicit Euler for the heat equation, with (left) smooth initial data and (right) oscillatory initial data and h_t chosen just below and just above the CFL limit.

$u(x, t) = e^{-15^2 \pi^2 t} \sin(15\pi x)$ (not shown on the plot), quickly decays to zero. The numerical approximation with $h_t = 0.48h_x^2$ is decaying, but much more slowly than the analytical solution, with its maximum value at time $t = 5.28h_x^2$ being about 0.32, compared with the analytical solution value of about 10^{-20} . This accuracy can be expected, given the relatively large values of h_x and h_t and the truncation-error analysis of the scheme. However, the numerical approximation with $h_t = 0.528h_x^2$ is *growing* in time, with a maximum value of about 2.36 after 10 timesteps. If we integrate out to later times, this mode continues to grow.

Both of these examples are somewhat contrived—a typical initial condition for the heat equation would be a function whose Fourier-series representation contains contributions from many terms. When we discretize the heat equation, the numerical approximation can only represent a finite number of these modes, with the discrete analogues of the continuous Fourier modes being the eigenvectors of the spatial discretization matrix, which become the eigenvectors of the two-level finite-difference scheme. When we violate the CFL condition, the two-level finite-difference scheme has at least one eigenvalue that is larger than one in magnitude, which corresponds to a mode like that shown in the right panel of Figure 5.3.1 and which grows with each timestep. Thus, while the numerical approximation of the smooth modes in the solution to the heat equation is reasonable, numerical errors in the oscillatory modes are quickly amplified and dominate the numerical error even after only a few timesteps. Indeed, even for solutions that involve only smooth Fourier modes, numerical round-off errors can introduce small amounts of high-frequency noise in an approximation that quickly grow and dominate the errors. This growth in high-frequency modes is why it is critical to obey the CFL limits.

In general, the Lax convergence or equivalence theorem tells us almost everything we need to know about the convergence of a finite-difference scheme. The main strategy is to show that the scheme is stable (via von Neumann analysis) and compute orders of consistency in time and space. Then, the scheme is convergent with the same orders. If a solution is sought such that the error $\max_\ell \|e_\ell\| \leq \alpha$ (as in the proof of the Lax convergence theorem, Theorem 5.12), then choosing h_x and h_t such that $h_x^\mu < \mathcal{O}(\alpha)$ and $h_t^\nu < \mathcal{O}(\alpha)$, and ensuring that any CFL conditions are satisfied, guarantees the required accuracy.

5.3.5 ■ Black–Scholes equation for option pricing

To illustrate the benefits of implicit time integration for parabolic PDEs, we now consider the example of the Crank–Nicolson finite-difference method for the Black–Scholes PDE, which is a well-known, *backward*, parabolic PDE that models the price of financial options. Consider the boundary-value problem

$$u_t + \frac{1}{2}\sigma^2 x^2 u_{xx} + r x u_x - r u = 0 \quad \text{on } \Omega = (0, x_{\max}) \times (0, T), \quad (5.132a)$$

$$u(x, T) = \max(x - K, 0), \quad (5.132b)$$

$$u(0, t) = 0, \quad (5.132c)$$

$$u(x_{\max}, t) = x_{\max} - K e^{-r(T-t)}, \quad (5.132d)$$

which models the price, $u(x, t)$, of a *European call option*, where x is the price of stock S that is *underlying* the option and t is time. In a European call option, at time $t = 0$, the holder acquires the right, but not the obligation, to buy stock S for the *strike price* K at the *expiry time* T . Here, r is the risk-free interest rate (e.g., 5% per year) and σ is the volatility of the stock (e.g., 30% per year). The European call option has value for the holder because the holder will be able to make a profit $x - K$ if the price x of stock S at time T is greater than the strike price K ; the holder would then exercise the option to buy the stock at price K and sell at price $x > K$. If $x < K$ at $t = T$, the option is not exercised and has zero value. Hence, at the expiry time T , the value of

the European call option is given by the *pay-off function*,

$$g(x) = \max(x - K, 0), \quad (5.133)$$

where x is the value at time T of stock S . Other types of options, e.g., the European put option, can be modeled by considering different pay-off functions.

The value of the option varies over time and, at any given time $t \in [0, T]$, depends on the price x of stock S at time t . The Black–Scholes PDE in equation (5.132a) provides a model for computing the value, $u(x, t)$, of the option for any price x of stock S at any time $t \in [0, T]$ and can be derived by considering a random price, $X(t)$, for stock S that follows a geometric Brownian motion process, subject to volatility σ . The PDE then models a hedging strategy where a portfolio of stock S can be made risk free by adding options (assuming that all risk-free portfolios earn the risk-free rate of return, r , as is the case in a so-called *arbitrage-free* market). An important characteristic of the Black–Scholes equation is that it is solved *backward* in time. That is, we know the value of the stock option at time $T > 0$, and we wish to compute the value of the option at time $t = 0$ in order to assess whether the option is worth its cost of purchase at the present time.

Clearly, at time $t = T$ the value of the option is given by the pay-off function $g(x)$, and this gives the *end condition* for the backward parabolic equation in equation (5.132b). To derive *boundary conditions* for the PDE, we consider two scenarios. First, if the price, x , of stock S is zero at any time t , then the value of the option is $u(0, t) = 0$, since the potential for profit is greater if we buy the stock at zero price rather than an option. This provides the first boundary condition in equation (5.132c). On the other hand, for a large stock price, x , a reasonable approximation of the option price at time t is given by $u(x, t) \approx x - Ke^{-r(T-t)}$ because, for large x , it becomes increasingly likely that the holder will be able to exercise the option for a profit, paying K to get x (or likely more), and the cost K that will be needed to exercise the option at time T can be discounted using the risk-free rate r (because the holder can put $Ke^{-r(T-t)}$ in the bank at time t to obtain K at the expiry time T). This provides the second boundary condition in equation (5.132d). The following example explains how the Crank–Nicolson finite-difference method can be used to compute option prices modeled by the Black–Scholes equation.

Considering the Black–Scholes boundary-value problem in equations (5.132a)–(5.132d), we approximate the option price on a Cartesian grid in $\Omega = (0, x_{\max}) \times (0, T)$ by

$$u_{k,l} \approx u(x_k, t_l). \quad (5.134)$$

Since explicit finite-difference methods generally lead to stringent timestep limitations for the parabolic Black–Scholes PDE, implicit methods are often preferred. The Crank–Nicolson method is a popular method for the Black–Scholes PDE because there is no timestep limitation for stability and the approximation is second-order accurate. The Crank–Nicolson discretization for the Black–Scholes PDE is given by

$$\frac{u_{k,l+1} - u_{k,l}}{h_t} + \frac{1}{2}q_{k,l} + \frac{1}{2}q_{k,l+1} = 0, \quad (5.135)$$

where

$$q_{k,l} = \frac{1}{2}\sigma^2 x_k^2 \frac{u_{k+1,l} - 2u_{k,l} + u_{k-1,l}}{h_x^2} + r x_k \frac{u_{k+1,l} - u_{k-1,l}}{2h_x} - r u_{k,l}. \quad (5.136)$$

Figure 5.3.2 shows numerical results for Crank–Nicolson applied to the European call option problem in equations (5.132a)–(5.132d) for strike price $K = 100$ and expiry time $T = 4$ years. As desired, the option price $u(x, t)$ equals the pay-off function at time $t = T$. If the initial stock

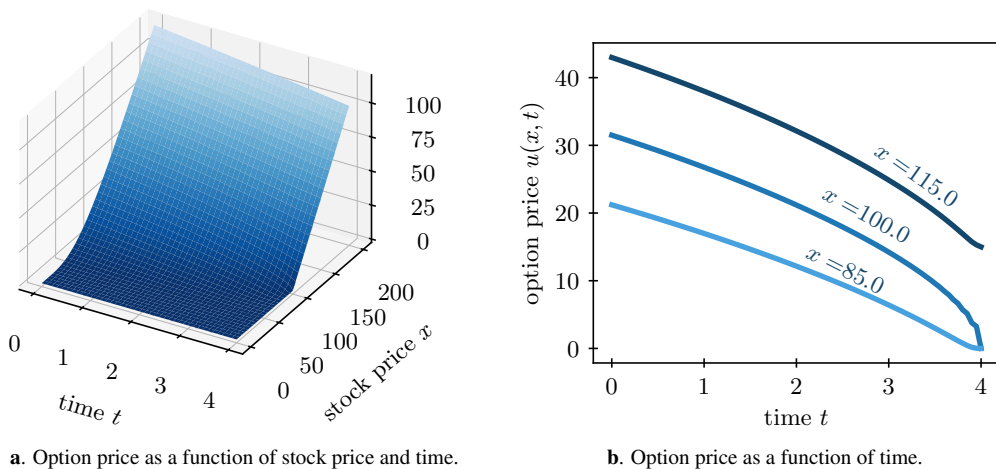


Figure 5.3.2. Option price $u(x, t)$ for a European call option with strike price $K = 100$ and expiry time $T = 4$ years. The risk-free cash rate is $r = 5\%$ /year and the volatility is $\sigma = 30\%$ /year. The option price is obtained by solving the Black–Scholes equation backward in time using the Crank–Nicolson method on a grid with 80 time intervals and 80 intervals of the stock price x over a range $[0, 200]$.

price is $x = 85$ at time $t = 0$, the price of the option at $t = 0$ is approximately 20. This option price reflects the risk-free value of the option to the holder and is influenced by the risk-free rate r (a larger rate corresponds to a larger expected increase over time in the value of the risk-free portfolio) and by the volatility σ of the underlying stock S (a larger volatility increases the option value). If the stock price $x = 85$ were to remain constant over time, the value of the option would steadily decrease to zero over time, because it would become increasingly likely that the holder would not be able to exercise the option for a profit. Considering now, for example, the value of the option at $t = 2$, we see that the option would increase in value if the stock price x were to rise to, say, $x = 100$, reflecting the higher probability for a larger payout at $t = T$. If the stock price rises to $x = 115$, the option price increases further, and the option value at $t = T$ indeed equals the profit $115 - K = 15$. In the Black–Scholes PDE, the constant r reflects the expected increase in value of the portfolio, and the constant σ reflects a backward diffusion process resulting from the volatility in the price of the underlying stock S .

Finally, we observe that the Crank–Nicolson approximation shows some small spurious oscillations near the expiry time $T = 4$, especially for stock price x close to the strike price $K = 100$. While these oscillations are small and reduce in amplitude as the grid is refined, they are undesirable. In this problem, they occur due to the nonsmooth pay-off function, and they can be reduced by replacing the Crank–Nicolson time integration by implicit Euler, which, however, reduces the method to first-order accuracy. Note that explicit methods would require very small timesteps for stability, which renders them impractical for this parabolic equation.

5.4 ■ Numerical dissipation and dispersion

Objectives

Continuous PDEs can model many physical properties, but numerical schemes might differ in the degree to which they respect them. Thus, in this section, you will learn how numerical schemes may introduce numerical dissipation or dispersion of waves in the solution.

While the Lax convergence theorem tells us about the convergence of finite-difference schemes, we would still like to understand more about the qualitative nature of the solutions we would expect from each scheme. To study this, we first analyze how different types of PDEs behave qualitatively in the continuum. For the remainder of this section, we consider the following three linear differential operators, whose fundamental solutions have very different properties:

$$L_1 u = u_t + a u_x, \quad (5.137a)$$

$$L_2 u = u_t + a u_x - D u_{xx}, \quad D > 0, \quad (5.137b)$$

$$L_3 u = u_t + a u_x - \mu u_{xxx}. \quad (5.137c)$$

5.4.1 ■ Dissipation and dispersion in linear PDEs

We consider wave-like functions of the form

$$z(x, t) = z_0 e^{i(kx - \omega t)}, \quad (5.138)$$

where $z_0 \in \mathbb{C}$, $k \in \mathbb{R}$, and $\omega \in \mathbb{C}$. This function is periodic in space, since k is real valued. Here, k denotes the wave number, and the wavelength of the wave, i.e., the length of one full sinusoidal oscillation, is given by $\frac{2\pi}{k}$. The initial amplitude of the wave is $|z_0|$. Since $\omega = \alpha + i\beta$ is allowed to be complex, the amplitude of the wave, $|z_0 e^{\beta t}|$, may grow or shrink over time.

Our first observation is that wave-like solutions of type (5.138) are always eigenfunctions of linear PDE operators (see Theorem 5.22).

Theorem 5.22: Wave-like eigenfunctions of linear PDEs

Let L be any linear partial differential operator in two variables, x and t . Define $z(x, t) = z_0 e^{i(kx - \omega t)}$. Then,

$$Lz = \lambda(\omega, k)z \quad (5.139)$$

for some polynomial function $\lambda(\omega, k)$.

Proof. Note that $z_t = -i\omega z_0 e^{i(kx - \omega t)} = -i\omega z$ and $z_x = ikz_0 e^{i(kx - \omega t)} = ikz$. Since any linear differential operator L is a polynomial in ∂_t and ∂_x without constant term, we have that $Lz = \lambda(\omega, k)z$ for some polynomial function $\lambda(\omega, k)$. $\square$

A major consequence of this theorem, Corollary 5.23, is that wave-like functions of type (5.138) are solutions of the unforced linear PDE $Lu = 0$ whenever ω and k satisfy $\lambda(\omega, k) = 0$. This condition is called the *dispersion relation*, illustrated in Definition 5.24.

Corollary 5.23: Wave-like solutions

The wave-like function $z(x, t)$ in (5.138) is a solution to $Lz = 0$ if and only if $\lambda(\omega, k) = 0$ in equation (5.139).

Definition 5.24: Dispersion relation [214, Sec. 7.2]

The equation $\lambda(\omega, k) = 0$ is called the *dispersion relation* for the PDE $Lz = 0$.

For the three linear operators defined in equations (5.137a)–(5.137c), we arrive at the following dispersion relations:

For $L_1 u = u_t + au_x$, we observe a linear relation:

$$L_1 z = -i\omega z(x, t) + iakz(x, t) = i(ak - \omega)z(x, t) \quad (5.140a)$$

$$\Rightarrow \lambda(\omega, k) = i(ak - \omega), \text{ or} \quad (5.140b)$$

$$\lambda(\omega, k) = 0 \iff \omega = ak. \quad (5.140c)$$

For $L_2 u = u_t + au_x - Du_{xx}$, the relation is quadratic:

$$L_2 z = -i\omega z(x, t) + iakz(x, t) - D(ik)^2 z(x, t) \quad (5.141a)$$

$$\Rightarrow \lambda(\omega, k) = -i\omega + iak + Dk^2, \text{ or} \quad (5.141b)$$

$$\lambda(\omega, k) = 0 \iff \omega = ak - iDk^2. \quad (5.141c)$$

Finally, for $L_3 u = u_t + au_x - \mu u_{xxx}$, the relation is cubic:

$$L_3 z = -i\omega z(x, t) + iakz(x, t) - \mu(ik)^3 z(x, t) \quad (5.142a)$$

$$\Rightarrow \lambda(\omega, k) = -i\omega + iak + i\mu k^3, \text{ or} \quad (5.142b)$$

$$\lambda(\omega, k) = 0 \iff \omega = ak + \mu k^3. \quad (5.142c)$$

For any of these cases, we write $\omega(k) = \alpha(k) + i\beta(k)$, so that

$$z(x, t) = z_0 e^{i(kx - \omega t)} \quad (5.143a)$$

$$= z_0 e^{i(kx - \alpha(k)t - i\beta(k)t)} \quad (5.143b)$$

$$= z_0 e^{\beta(k)t} e^{i(kx - \alpha(k)t)}. \quad (5.143c)$$

From this, we observe that if $\beta(k) \neq 0$, then the amplitude of the wave changes in time. In many applications we have that $\beta(k) \leq 0$, so that solutions do not blow up in time. On the other hand, if $\alpha(k) = ak$, then $z(x, t) = z_0 e^{\beta(k)t} e^{i k(x - at)}$. The term $e^{i k(x - at)}$ describes a wave with fixed wavelength, moving with fixed velocity. This leads to the concepts of *dissipation* (see Definition 5.25) and *dispersion* (see Definition 5.26).

Definition 5.25: Dissipation [214, Sec. 7.2]

Let $z(x, t) = z_0 e^{i(kx - \omega t)}$ be a wave-like solution of $Lu = 0$ with $\omega(k) = \alpha(k) + i\beta(k)$. In the case where $\beta(k) < 0$, the wave-like solution decays in time. We call this type of behavior *dissipation*.

Definition 5.26: Phase velocity and dispersion [214, Sec. 7.2]

Let $z(x, t) = z_0 e^{i(kx - \omega t)}$ be a wave-like solution of $Lu = 0$ with $\omega(k) = \alpha(k) + i\beta(k)$ and $\omega(k)$ satisfying $\lambda(\omega(k), k) = 0$. The *phase velocity* of $z(x, t)$ is given by

$$v_{ph} = \frac{Re(\omega(k))}{k} = \frac{\alpha(k)}{k}. \quad (5.144)$$

If v_{ph} is a nonconstant function of k , then waves with different wavelengths move at different velocities, and we call this *dispersion*. Conversely, if v_{ph} is a constant (i.e., $\alpha(k)$ is linear in k), then all waves move with the same velocity, and there is no dispersion.

In the case of $L_1 u = u_t + au_x$, we have $\omega(k) = ak$, resulting in $v_{ph} = a$ and $\beta(k) = 0$. Thus, L_1 is neither dissipative nor dispersive, and solutions to $L_1 z = 0$ are given by $z(x, t) = z_0 e^{ik(x-at)}$. Conversely, for $L_2 u = u_t + au_x - Du_{xx}$, we have $\omega(k) = ak - iDk^2$, so that $v_{ph} = a$ and $\beta(k) < 0$. Here, L_2 is not dispersive, but it is dissipative, with solutions to $L_2 z = 0$ given by $z(x, t) = z_0 e^{-Dk^2 t} e^{ik(x-at)}$. Furthermore, for $L_3 u = u_t + au_x - \mu u_{xxx}$, we have $\omega(k) = ak + \mu k^3$, so that $v_{ph} = a + \mu k^2$ and $\beta(k) = 0$. In this case, L_3 is not dissipative, but it is dispersive, with solutions to $L_3 z = 0$ given by $z(x, t) = z_0 e^{ik(x-(a+\mu k^2)t)}$.

So, why do we care about this? Understanding the qualitative behavior of solutions to the PDE in the continuum is important because we would like our finite-difference schemes to produce similar behavior. In the next subsection, we consider numerical schemes for the solution of $L_1 u = u_t + au_x = 0$, where we now know that L_1 is neither dissipative nor dispersive. However, finite-difference discretizations may (to varying extents) introduce *numerical* dissipation or dispersion that we should be aware of when choosing a scheme.

5.4.2 ■ Numerical dissipation and dispersion

To quantify numerical dissipation and dispersion, we introduce the discrete wave-like function

$$z_{j,\ell} = z_0 e^{i(kjh_x - \omega\ell h_t)}, \quad (5.145)$$

which we consider as a solution to the finite-difference matrix equation $P_h z_{\ell+1} = Q_h z_\ell$ approximating the one-way wave equation $L_1 u = 0$. The following theorem provides a useful result for our discrete wave function.

Theorem 5.27: Wave-like eigenfunctions of Toeplitz operators

Let T be a Toeplitz operator and z_ℓ be a discrete wave-like function as defined in equation (5.145). Then $Tz_\ell = \lambda(k)z_\ell$ for some function $\lambda(k)$.

Proof. By definition,

$$(Tz_\ell)_j = \sum_m t_{j-m} z_{m,\ell} \quad (5.146a)$$

$$= \sum_m t_{j-m} z_0 e^{i(kmh_x - \omega\ell h_t)} \quad (5.146b)$$

$$= \sum_m t_{j-m} z_0 e^{i(k(m-j)h_x)} e^{i(kjh_x - \omega\ell h_t)} \quad (5.146c)$$

$$= \left(\sum_{\hat{m}} t_{\hat{m}} e^{-i\hat{m}kh_x} \right) z_0 e^{i(kjh_x - \omega\ell h_t)}. \quad (5.146d)$$

Thus, we have $Tz_\ell = \lambda(k)z_\ell$ for $\lambda(k) = \sum_{\hat{m}} t_{\hat{m}} e^{-i\hat{m}kh_x}$. $\square$

We now show that the discrete wave-like function z_ℓ from equation (5.145) is a solution of the finite-difference matrix equation $P_h z_{\ell+1} = Q_h z_\ell$ if ω and k satisfy a discrete dispersion relation. This discrete dispersion relation gives us insight into the numerical dissipation and dispersion in numerical schemes for the one-way wave equation, $L_1 u = 0$. We know that both P_h and Q_h are Toeplitz. As a result, there are functions $\lambda_1(k)$ and $\lambda_2(k)$ such that $P_h z_{\ell+1} = \lambda_1(k)z_{\ell+1}$ and

$Q_h z_\ell = \lambda_2(k) z_\ell$. So if z_ℓ is a solution to $P_h z_{\ell+1} = Q_h z_\ell$, then we have

$$\lambda_1(k) z_{\ell+1} = \lambda_2(k) z_\ell, \quad (5.147a)$$

$$\lambda_1(k) z_0 e^{\imath(kjh_x - \omega(\ell+1)h_t)} = \lambda_2(k) z_0 e^{\imath(kjh_x - \omega\ell h_t)} \quad (5.147b)$$

$$\Rightarrow e^{-\imath\omega h_t} = \frac{\lambda_2(k)}{\lambda_1(k)}. \quad (5.147c)$$

Moreover, $\lambda_1(k) = \sum_{\widehat{m}} p_{\widehat{m}} e^{-\imath k \widehat{m} h_x} = \widehat{p}(kh_x)$ and $\lambda_2(k) = \sum_{\widehat{m}} q_{\widehat{m}} e^{-\imath k \widehat{m} h_x} = \widehat{q}(kh_x)$, which leads to

$$\frac{\lambda_2(k)}{\lambda_1(k)} = \frac{\widehat{q}(kh_x)}{\widehat{p}(kh_x)} = s(kh_x), \quad (5.148)$$

which is the *symbol* of the finite-difference method used in the von Neumann analysis—see Definition 5.19. Therefore, $z_{j,\ell} = z_0 e^{\imath(kjh_x - \omega\ell h_t)}$ is a solution of the finite-difference scheme applied to $L_1 u = 0$ if $\omega = \omega(k)$ satisfies the *discrete dispersion relation*

$$e^{-\imath\omega(k)h_t} = s(kh_x). \quad (5.149)$$

To investigate this further, we first rewrite $s(kh_x)$ in polar form as

$$s(kh_x) = |s(kh_x)| e^{\imath\phi(kh_x)} = e^{\ln |s(kh_x)| + \imath\phi(kh_x)}. \quad (5.150)$$

Then, since $e^{-\imath\omega(k)h_t} = s(kh_x)$, we obtain

$$\omega(k) = \frac{-\phi(kh_x) + \imath \ln |s(kh_x)|}{h_t}. \quad (5.151)$$

As in the continuum, we now look at the growth in the discrete wave amplitude

$$z_{j,\ell} = z_0 \exp(\imath(kjh_x - \omega\ell h_t)) \quad (5.152a)$$

$$= z_0 \exp\left(\imath\left(kjh_x - \left(\frac{-\phi(kh_x) + \imath \ln |s(kh_x)|}{h_t}\right)\ell h_t\right)\right) \quad (5.152b)$$

$$= z_0 |s(kh_x)|^\ell \exp\left(\imath k\left(jh_x - \left(\frac{-\phi(kh_x)}{kh_t}\right)\ell h_t\right)\right). \quad (5.152c)$$

For a stable scheme, we expect $|s(kh_x)| \leq 1 \forall k$, which implies that

$$|s(kh_x)|^\ell \leq 1 \quad \forall \ell, \quad (5.153)$$

and the amplitude does not grow without bound over time.

We say that a two-level finite-difference scheme for $L_1 u = 0$, $P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell$, with symbol $s(kh_x)$, has numerical dissipation when $|s(kh_x)| < 1$. The size of $s(kh_x)$ measures the amount of numerical dissipation of the scheme as a function of k and h_x . Similarly, a scheme with symbol $s(kh_x) = |s(kh_x)| e^{\imath\phi(kh_x)}$ has phase velocity $v_{ph} = \frac{-\phi(kh_x)}{kh_t}$. If v_{ph} is not constant in k , then there is numerical dispersion caused by the finite-difference scheme. Conversely, if v_{ph} is independent of k , then all waves move with the same velocity, and there is no numerical dispersion. The difference between v_{ph} and a measures the amount of numerical dispersion as a function of k and h_x . The following examples show how these concepts can help one understand the dominant error effects of numerical methods for the one-way wave equation.

Example 5.28: ETBS for the one-way wave equation: numerical dissipation and dispersion

For ETBS, $\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k,\ell} - u_{k-1,\ell}}{h_x} = 0$, we have

$$s(kh_x) = 1 - \gamma(1 - e^{-ikh_x}) = (1 - \gamma) + \gamma e^{-ikh_x}, \quad (5.154)$$

with $\gamma = \frac{ah_t}{h_x}$. In this case, $s(kh_x)$ describes points on a circle in the complex plane, centered at $(1 - \gamma)$, with radius γ , as sketched in Figure 5.4.1. For von Neumann stability, we require that $|s(kh_x)| \leq 1$, which requires in turn that $0 \leq \gamma \leq 1$, i.e., $a \geq 0$ and $h_t \leq \frac{h_x}{a}$. If γ is close to zero (i.e., $ah_t \ll h_x$), then $s(k) \approx 1$ for all k . Thus, we would expect low numerical dissipation when γ is small. For larger values of γ , we expect more numerical dissipation, since the center of the circle in the complex plane moves toward the origin and there are some modes, k , for which the associated value of $|s(k)|$ can get quite small. This is observed in the example of Figure 5.1.3, where we applied this scheme to the one-way wave equation with $\gamma = \frac{1/300}{2/100} = \frac{1}{6}$ and observed notable dissipation in the numerical solution after 300 timesteps at $t = 1$.

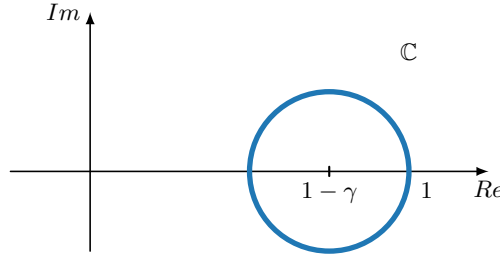


Figure 5.4.1. Symbol of ETBS for the one-way wave equation.

Next, we consider the dispersive properties of this scheme. If kh_x is small, then $e^{-ikh_x} \approx 1 - ikh_x$. So we can approximate $s(kh_x)$ as

$$s(kh_x) \approx (1 - \gamma) + \gamma(1 - ikh_x) = 1 - i\gamma kh_x. \quad (5.155)$$

Using the discrete dispersion relation, equation (5.149), we write

$$e^{-i\omega(k)h_t} = \cos(\omega(k)h_t) - i \sin(\omega(k)h_t) \approx 1 - i\gamma kh_x, \quad (5.156)$$

and we see that if $\omega(k)h_t$ is also small, we obtain $\omega(k)h_t \approx \gamma kh_x$, or $\omega(k) \approx \gamma k \frac{h_x}{h_t} = ak$, since $\gamma = \frac{ah_t}{h_x}$. This then gives $v_{ph} \approx \frac{ak}{k} = a$. Thus, we get no or little dispersion when both kh_x and $\omega(k)h_t$ are small. For fixed values of h_x and h_t , these are expected to be small for some set of suitably small k . From the opposite perspective, for any fixed k , we can find bounds on h_x and h_t such that these approximations are reasonable, and there is little dispersion, as seen in Figure 5.1.3. This gives the general idea that modes that are “smooth” relative to the grid suffer little from dispersion, while those that are oscillatory relative to the grid may suffer from more dispersion.

Another viewpoint on numerical dissipation and dispersion comes from *modified equation analysis*. To perform this analysis, we go back to the computation of the truncation error for the scheme and consider

$$\frac{1}{h_t} (P_h \mathbf{u}_{\ell+1} - Q_h \mathbf{u}_\ell). \quad (5.157)$$

Assuming no forcing terms, this defines the truncation error of the scheme; in modified equation analysis, however, we ask the question of what PDE this corresponds to if we include the dominant terms of the truncation error. For the ETBS scheme above, we have that

$$\frac{1}{h_t} (P_h \mathbf{u}_{\ell+1} - Q_h \mathbf{u}_\ell) = \frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k,\ell} - u_{k-1,\ell}}{h_x} \quad (5.158a)$$

$$= u_t(x_k, t_\ell) + \frac{h_t}{2} u_{tt}(x_k, t_\ell) + a \left(u_x(x_k, t_\ell) - \frac{h_x}{2} u_{xx}(x_k, t_\ell) \right) \quad (5.158b)$$

$$+ \mathcal{O}(h_t^2, h_x^2). \quad (5.158c)$$

Discarding the second-order terms, we rewrite this as

$$u_t + au_x = \frac{1}{2} (ah_x u_{xx} - h_t u_{tt}). \quad (5.159)$$

Differentiating this expression with respect to both x and t gives

$$u_{tt} + au_{xt} = \frac{1}{2} (ah_x u_{xxt} - h_t u_{ttt}), \quad (5.160a)$$

$$u_{tx} + au_{xx} = \frac{1}{2} (ah_x u_{xxx} - h_t u_{ttx}), \quad (5.160b)$$

which we combine to get

$$u_{tt} = a^2 u_{xx} + \mathcal{O}(h_t, h_x). \quad (5.161)$$

Now discarding the $\mathcal{O}(h_t, h_x)$ terms, we use this to rewrite equation (5.159) to have only one time-derivative term, yielding the modified PDE

$$u_t + au_x = \frac{1}{2} (ah_x - a^2 h_t) u_{xx}. \quad (5.162)$$

We recognize this as an advection-diffusion equation of the form in equation (5.137b) when $ah_t < h_x$ (which is the CFL condition for the scheme). This tells us that solutions of the ETBS scheme are expected to be dissipative, since the dominant terms in the truncation error are second-order spatial derivatives.

Similar analyses can be carried out for other schemes. In fact, some of the more elaborate schemes, such as Crank–Nicolson and Lax–Wendroff, were designed specifically to address numerical dispersion and/or dissipation, as shown in Example 5.29.

Example 5.29: Lax–Wendroff for the one-way wave equation: Numerical dissipation and dispersion

For Lax–Wendroff on the one-way wave equation,

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k-1,\ell}}{2h_x} - \frac{a^2 h_t}{2} \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2} = 0, \quad (5.163)$$

similar modified equation analysis shows that the resulting modified PDE is

$$u_t + au_x + \frac{1}{6} (ah_x^2 - a^3 h_t^2) u_{xxx} = 0. \quad (5.164)$$

As for the operator from equation (5.137c) above, we know that adding μu_{xxx} to the PDE has a dispersive effect. Thus, we expect dispersion to be the dominant error effect in the Lax–Wendroff scheme. More generally, for first-order accurate discretizations of the one-way wave

equation, the leading term in the truncation error is proportional to u_{xx} , and, hence, dissipation is the dominant error effect. For second-order accurate methods, such as Lax–Wendroff, the leading term in the truncation error is proportional to u_{xxx} , and dispersion is the main error.

To verify this, specifically for the Lax–Wendroff scheme, we consider a similar setup as the examples with smooth initial conditions in Section 5.1.1, with the same values of $h_x = 2/100$ and $h_t = 1/300$, but with the initial condition

$$u(x, 0) = \begin{cases} -100(x + 0.6)(x + 0.4) & \text{if } -0.6 \leq x \leq -0.4, \\ 0 & \text{otherwise,} \end{cases} \quad (5.165)$$

which achieves a maximum value of one at $x = -0.5$ and is continuous at the two transition points at $x = -0.6$ and $x = -0.4$ but not continuously differentiable there (see also equation (5.61)). In Figure 5.4.2, we see that, at the final time, the magnitude of the solution has only decayed a very small amount, agreeing with the expectation that the Lax–Wendroff scheme should have small numerical dissipation effects. However, in contrast to the smooth solutions in Figure 5.1.3, the numerical solution at $t = 1$ using the Lax–Wendroff scheme for this case shows that significant oscillations have been introduced into the approximation. These oscillations come from numerical dispersion: the initial solution is composed of a superposition of Fourier modes, but not all of these modes are advected at the same speed by the numerical scheme, losing the coherent form of the initial condition. Dispersion errors are characterized by nonphysical oscillations, as we observe in this case.

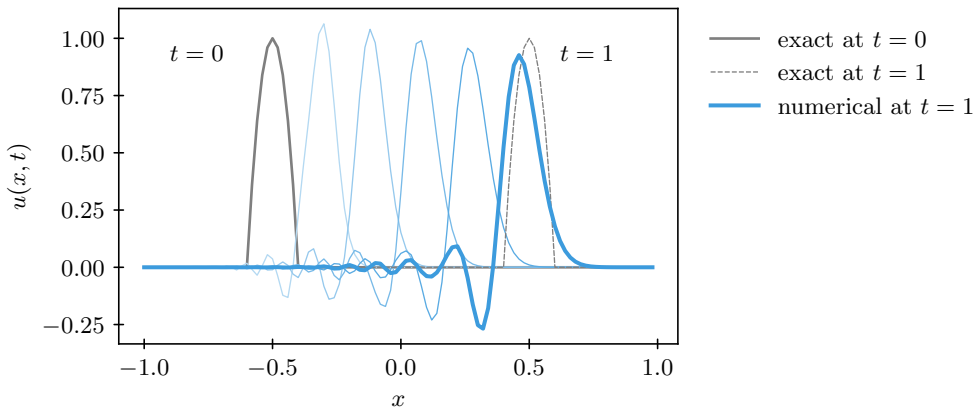


Figure 5.4.2. Numerical solution of the one-way wave equation using the Lax–Wendroff scheme. Numerical solutions from $t > 0$ (lighter blue shade) to $t = 1$ (darker blue shade) are shown, along with the exact solutions at $t = 0$ and $t = 1$.

5.4.3 ■ Implications for approximating hyperbolic and parabolic PDEs

It is interesting to reflect for a moment on the implications of what we have learned about dissipation and dispersion, and about numerical stability, for the numerical approximation of hyperbolic and parabolic PDEs. In Section 5.1 on finite-difference discretization stencils, there was little difference in the treatment of our prototypical hyperbolic (one-way wave equation) and parabolic (heat equation) PDEs. Both can be discretized to yield a variety of two-level finite-difference schemes, and the Lax convergence theorem (Theorem 5.12) gives the same convergence conditions for both types of problems. In both cases, we can use standard Taylor-series arguments to establish consistency, and either Lax–Richtmyer or von Neumann stability analysis to establish

stability. However, despite these similarities, there are two more full chapters of this book devoted to further treatment of the discretization of hyperbolic PDEs (Chapters 6 and 9), and only one short section (Section 8.7) on the parabolic case. The main motivation for this stems from what we have learned in the current section on dissipation and dispersion. Fundamentally, continuum solutions to parabolic PDEs are dissipative in nature—even if the initial data for a problem is discontinuous, high-frequency Fourier modes are damped faster than low-frequency modes, and the solution both becomes smooth and decays rapidly (unless forcing terms are present). This is in contrast to purely hyperbolic PDEs, where continuum solutions are typically not dissipative and may not be dispersive. In this case, discontinuities in the initial data can propagate through space and time. Moreover, as discussed in Chapter 6 for nonlinear hyperbolic PDEs, even smooth initial data can lead to discontinuous solutions at later times.

In Section 5.4.2, the focus was on examples of numerical dissipation and dispersion for the hyperbolic case. The symbols for the implicit and explicit Euler discretizations of the heat equation derived in Section 5.3.4, however, are real valued, which corresponds to the case of no numerical dispersion, but possible numerical dissipation. In the parabolic setting, however, dissipation is the *expected* behavior of both the continuum and numerical solutions. This makes developing effective numerical schemes for the parabolic case much easier than in the hyperbolic case, where special treatment is needed to match the expected physical behavior of propagating sharp changes in the solution.

In the earlier sections of this chapter, we established that both the implicit and explicit Euler schemes for the heat equation are consistent and von Neumann stable (possibly conditionally so), and, as a result, they are convergent with order $\mathcal{O}(h_t, h_x^2)$. We could easily expand this set of results and show that the Crank–Nicolson method achieves $\mathcal{O}(h_t^2, h_x^2)$ truncation error and is unconditionally stable. Of these schemes, explicit Euler is the least expensive per timestep but suffers due to the CFL condition that may require many more timesteps to reach the final time required in a stable fashion. There are a few key takeaways:

- Explicit methods for the heat equation require $h_t = \mathcal{O}(h_x^2)$ for stability.
- Implicit methods often have less stringent stability restrictions; for example, Crank–Nicolson is stable for any h_t , and accuracy considerations only require $h_t = \mathcal{O}(h_x)$, which asymptotically is much more forgiving than the explicit stability bound.
- Implicit methods may, thus, require many fewer timesteps to reach the same accuracy; however, the benefit of using implicit methods also depends on the cost of solving the implicit linear system (the cost per timestep may be substantially larger for implicit methods).
- The above is typically true for all parabolic equations and depends little on the specific discretization chosen for the elliptic part of the PDE, as long as it leads to a scheme that is consistent (and stable in the elliptic sense).

The last statement implies that we can focus on the elliptic part separately and know that everything we do there can be related back to parabolic equations. Thus, the development of numerical methods for parabolic PDEs mainly focuses on the analysis of the elliptic equations (e.g., as studied in Chapters 3, 4, and 8) and then applying implicit or explicit discretizations in time for the time-dependent part using the method-of-lines approach.

5.5 ■ Some further thoughts

One emphasis in this chapter has been on discretizations that can be derived using the method-of-lines approach, with standard spatial finite-difference discretizations of u_x or u_{xx} coupled with time integration using the implicit or explicit Euler methods or Crank–Nicolson. An exception

is Lax–Wendroff for the one-way wave equation, which can be derived from simultaneously requiring second-order accuracy in space and time using Taylor series (see Section 6.3.2 for the derivation in the nonlinear case). A natural next step in the development of the method-of-lines approach is the use of higher-order discretizations in both space and time. Higher-order spatial discretizations can be derived using Taylor-series methods, as in Chapter 3, but more work is needed to move to higher order in time. We discuss this in a few spots later in this book, notably Sections 6.5, 9.4.4, and 9.6, noting that higher-order methods like backward-differentiation formulas (which generalize the Euler methods) and Runge–Kutta methods (which generalize Crank–Nicolson) are well known and can be used in the same way as we use their simplest forms here. More thorough treatments of these time-integration schemes are found in many standard textbooks on numerical methods, including [127, 22, 20].

A common classification of these time-integration schemes are as multistep versus multi-stage methods. Multistage approaches can be analyzed in the framework of the two-level finite-difference methods considered here, with the intermediate approximations in such schemes (the so-called stage values) depending only on $\mathbf{u}_\ell$ to determine $\mathbf{u}_{\ell+1}$. Multistep schemes, in contrast, make use of $\mathbf{u}_{\ell-1}$ and possibly more previous timesteps. To analyze these schemes, we would need to generalize the two-level analysis to include more levels. For instance, we could discretize $u_t + au_x = 0$ using an explicit second-order backward-differentiation formula, BDF2, in time and a centered scheme in space, yielding

$$\frac{3u_{k,\ell+1} - 4u_{k,\ell} + u_{k,\ell-1}}{2h_t} + a \frac{u_{k+1,\ell} - u_{k-1,\ell}}{2h_x} = 0. \quad (5.166)$$

In matrix notation, we would obtain a scheme of the form

$$R_h \mathbf{u}_{\ell+1} + P_h \mathbf{u}_\ell + Q_h \mathbf{u}_{\ell-1} = 0. \quad (5.167)$$

Then, the notion of stability requires bounding the operator norms of the three matrices appearing here. The proof of the Lax convergence theorem is similar but requires a three-term recursion to rewrite the error propagation in terms of the truncation error.

While the concepts of consistency and stability and the use of the Lax convergence theorem to establish convergence are quite standard, Section 5.4 establishes that these are not the only concerns in understanding the convergence and solution quality of a finite-difference scheme. *Modified equation analysis* [231], as discussed in Section 5.4.2, derives a PDE for which our numerical scheme gives an accurate approximation using Taylor’s theorem to first derive a higher-order PDE consistent with the given discrete scheme. Then, temporal derivatives are exchanged for spatial ones (as is done for the Lax–Wendroff discretization in Section 6.3) to derive a PDE with the same first-order time derivative but additional spatial terms. These extra terms vanish as the grid is refined but offer us insight into the dominant error terms for the discretization. With this analysis, we can understand not only the consistency error in a scheme but also the dissipation and dispersion properties associated with the method.

Finally, we note that while much of the remainder of this book is devoted to alternative schemes that outperform the finite-difference methods described in this chapter for wide classes of problems, this is not to say that these initial schemes are not widely used. As we discuss in Chapters 6 and 9, there are very good reasons why finite-volume schemes are generally preferred for hyperbolic equations. For parabolic equations, however, it is quite common to consider either the generalizations of the finite-difference methods proposed here or approaches based on using finite-element methods in space coupled with the method of lines for temporal discretization. One family of equations where finite-difference methods remain quite commonly used consists of the nonlinear Schrödinger equations such as

$$i \frac{\partial u}{\partial t} - \Delta u + \lambda |u|^{p-1} u = 0, \quad (5.168)$$

where $i^2 = -1$ and $p > 1$. Depending on the values of p and λ and the domain on which the problem is posed, the solutions to this equation may or may not exist and may or may not have certain regularity properties or associated conservation laws. Effective finite-difference discretizations of these equations have been widely studied (cf. [83]) and are a common tool, particularly when they are able to preserve discrete analogues of the associated conservation laws of the equation.

5.6 ■ Exercises

1. Let $a > 0$, and consider the one-way wave equation $u_t + au_x = 0$. Find the truncation error for the Lax–Wendroff scheme

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k-1,\ell}}{2h_x} - \frac{a^2 h_t}{2} \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2} = 0.$$

In particular, show that the scheme satisfies $\tau_{k,\ell} = \mathcal{O}(h_t^2, h_x^2)$. *Hint:* You will need to manipulate the PDE in order to cancel out the $\mathcal{O}(h_t)$ term in the truncation error.

2. Let $D > 0$, and consider the heat equation $u_t - Du_{xx} = 0$. Find the truncation error for the Crank–Nicolson scheme

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} - \frac{D}{2} \left(\frac{u_{k+1,\ell+1} - 2u_{k,\ell+1} + u_{k-1,\ell+1}}{h_x^2} + \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2} \right) = 0.$$

In particular, show that the scheme satisfies $\tau_{k,\ell} = \mathcal{O}(h_t^2, h_x^2)$.

3. Let $a > 0$, and consider the one-way wave equation $u_t + au_x = 0$. Perform a von Neumann analysis for both the ETFS scheme,

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k,\ell}}{h_x} = 0,$$

and the Lax–Wendroff scheme,

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k-1,\ell}}{2h_x} - \frac{a^2 h_t}{2} \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2} = 0.$$

4. Consider the following finite-difference method for the one-way wave equation $u_t + au_x = 0$:

$$\frac{u_{k,\ell+1} - \frac{u_{k+1,\ell} + u_{k-1,\ell}}{2}}{h_t} + a \frac{u_{k+1,\ell} - u_{k-1,\ell}}{2h_x} = 0.$$

- (a) Find the order of the truncation error $\tau_{k,\ell}$ as a function of h_x and/or h_t .
- (b) Find the symbol $s(\varphi)$ of the method, and determine the von Neumann stability bound on h_t .

5. Let $D > 0$, and consider the heat equation $u_t - Du_{xx} = 0$. Consider the Crank–Nicolson scheme

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} - \frac{D}{2} \left(\frac{u_{k+1,\ell+1} - 2u_{k,\ell+1} + u_{k-1,\ell+1}}{h_x^2} + \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2} \right) = 0.$$

Perform a Lax–Richtmyer stability analysis for the scheme with Dirichlet boundary conditions in the Euclidean norm, assuming that $u_{0,\ell} = u_{0,\ell+1} = u_{n_x,\ell} = u_{n_x,\ell+1} = 0$. Also perform a von Neumann stability analysis for the scheme. Do the two approaches agree or not?

6. Consider the wave equation $u_{tt} - a^2 u_{xx} = 0$ and the finite-difference scheme

$$\frac{u_{k,\ell+1} - 2u_{k,\ell} + u_{k,\ell-1}}{h_t^2} - a^2 \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2} = 0.$$

Verify that the truncation error is second order in both h_t and h_x . Under what conditions on h_t and h_x is the scheme stable? *Hint:* Consider the spatial modes $e^{i\varphi k}$, but also include their amplification in time, writing $u_{k,\ell} = c_\ell^{(\varphi)} e^{i\varphi k}$. Find and solve a recurrence relation for $c_\ell^{(\varphi)}$, and give conditions so that $|c_{\ell+1}^{(\varphi)}| \leq |c_\ell^{(\varphi)}|$ for all k and ℓ .

7. Let $\alpha, \beta \in \mathbb{R}$ be known constants. Show that the bounds

$$\|P_h^{-1}Q_h\| \leq 1 + \alpha h_t \text{ and } \|P_h^{-1}\| \leq \beta$$

are sufficient conditions to prove stability of the finite-difference method

$$P_h \mathbf{u}_{\ell+1} = Q_h \mathbf{u}_\ell + h_t \mathbf{b}_\ell$$

on a domain with $\ell h_t \leq t^*$. *Hint:* Show that these conditions are sufficient conditions for satisfying the definition of a stable scheme in Definition 5.9.

8. Let $a > 0$, and consider the one-way wave equation $u_t + au_x = 0$. Compute the dispersion relations for the ITCS, Crank–Nicholson, and Lax–Wendroff schemes. Fix $a = 0.9$ and $h_t/h_x = 1$ and plot the phase velocity and amplitude of the symbol for the three schemes as a function of kh_x , and compare these for the three schemes. For which scheme is the phase velocity closest to that of the continuum equation? Which scheme is least dissipative? Which scheme is most dissipative?
9. Derive the dispersion relation of the PDE

$$u_t + a u_x = \delta u_{xxxx} + \epsilon u_{xxxxx}.$$

Give an expression for the standard wave-like solution that satisfies this PDE. Is there dissipation? Is there dispersion? What is the phase velocity of the wave?

Chapter 6

Finite-volume methods for scalar conservation laws in 1D

Overview

Finite-volume methods are widely used to approximate solutions of nonlinear hyperbolic conservation laws of type

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0, \quad (6.1)$$

where the nonlinearity of the *flux function*, $f(u)$, may lead to the formation of discontinuities in the solution, $u(x, t)$. Finite-volume methods are close cousins of the finite-difference methods that are the topic of previous chapters. However, they are derived from integral forms of the PDE in order to maintain conservation properties that are crucial for obtaining the correct propagation speed of discontinuities, while also controlling spurious oscillations that can easily arise at discontinuities.

The topics in this chapter introduce basic building blocks for finite-volume methods in the simplified setting of scalar and one-dimensional conservation laws. In Chapter 9, these building blocks will be extended to coupled systems of conservation laws in multiple spatial dimensions, which have broad applications in areas such as fluid dynamics.

Objectives

In this chapter, we delve into the main principles and ideas behind finite-volume methods for nonlinear hyperbolic conservation laws. Grounded in an understanding of the theoretical properties of smooth and discontinuous solutions to these PDEs, you will derive a general form for a finite-volume method from an integral form of the PDE. This leads directly to first-order accurate finite-volume methods that are closely related to finite-difference methods, but with a crucial added numerical conservation property. A more sophisticated approach is needed to attain second-order accuracy, with the concept of total variation aiding in controlling spurious oscillations. Using the techniques from this chapter, you will be able to

- reconstruct how information propagates and discontinuities form in solutions of hyperbolic conservation laws;
- derive families of efficient and accurate finite-volume discretizations for hyperbolic conservation laws with general flux functions; and
- compute numerical approximations of smooth and discontinuous conservation law solutions with first- and second-order accuracy.

Chapter Outline

Section 6.1 Some properties of scalar hyperbolic conservation laws introduces properties of solutions to conservation laws that are important for deriving numerical methods that properly handle discontinuities. We discuss the concepts of characteristic curves, integral forms, weak solutions, and shock speed, which figure prominently in the derivation of finite-volume methods in the following sections.

Section 6.2 Fundamentals of finite-volume methods presents a general framework for finite-volume methods derived from an integral form of the PDE. The conservation form of finite-volume methods directly leads to a numerical conservation property that is crucial for shock propagation with correct speeds.

Section 6.3 Some finite-volume methods completes the abstract framework introduced in Section 6.2 by prescribing several numerical flux functions that can be used. The first-order upwind (FOU) flux function connects these new schemes with the finite-difference case, while the Lax–Friedrichs approach works well for the nonlinear hyperbolic conservation laws that motivate finite-volume schemes. Finally, finite-volume methods are interpreted in the physically intuitive context of Riemann problems, leading to Godunov’s method.

Section 6.4 Total variation and finite-volume methods covers the concept of total variation, both at the continuum level and within a finite-volume discretization. We see that the solutions of hyperbolic conservation laws naturally satisfy a *total-variation diminishing* (TVD) property and examine how we might extend that to the presented numerical methods.

Section 6.5 Second-order accurate methods extends the finite-volume approach to second-order accuracy in space and time. Linear polynomial reconstruction is employed to increase accuracy, and slope limiters are derived within a total-variation framework to control spurious oscillations.

6.1 ■ Some properties of scalar hyperbolic conservation laws

Objectives

The goal of this section is to outline several key properties of solutions to scalar nonlinear conservation laws. These properties are useful for formulating numerical methods in the subsequent sections of this chapter. In particular, you will use the prototype model of Burgers’ equation to help establish and illustrate these properties. As such, you will learn how to calculate and illustrate solutions to Burgers’ equation and characterize solutions as rarefactions or steepening. Then, you will be able to construct solutions with shocks and compute shock speeds. Finally, you will see how to define a Riemann problem and derive its solution.

6.1.1 ■ Nonlinear scalar conservation laws in 1D

In Section 1.2, we introduced scalar conservation laws in three spatial dimensions and established the concept of characteristic curves for the linear advection equation in 1D. We now take a closer look at *nonlinear* conservation laws in 1D and their characteristic curves. Consider quantity

$u(x, t)$ and the 1D scalar conservation law

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0. \quad (6.2)$$

If $f(u)$ is differentiable, the *quasi-linear* form of the conservation law is given by

$$\frac{\partial u}{\partial t} + \frac{df}{du} \frac{\partial u}{\partial x} = 0. \quad (6.3)$$

Then, similar to the linear case, considering

$$\frac{du(x(t), t)}{dt} = \frac{\partial u}{\partial x} \frac{dx}{dt} + \frac{\partial u}{\partial t} \quad (6.4)$$

for the change in $u(x, t)$ along a curve $x(t)$, we define the characteristic curves for equation (6.2) as follows.

Definition 6.1: Characteristic curve

A curve $x(t)$ satisfying

$$\frac{dx(t)}{dt} = \frac{df(u)}{du} \quad (6.5)$$

is a characteristic curve of scalar conservation law (6.2).

Since

$$\frac{du(x(t), t)}{dt} = 0 \quad (6.6)$$

on characteristic curves, $u(x, t)$ is constant on the curves, and characteristic curves are straight lines.

Inviscid Burgers' equation

As a specific example and to highlight some key properties, we consider the *inviscid form of Burgers' equation*, given by

$$\frac{\partial u}{\partial t} + \frac{\partial(u^2/2)}{\partial x} = 0, \quad (6.7)$$

with flux function

$$f(u) = \frac{u^2}{2}. \quad (6.8)$$

In quasi-linear form, the equation is

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0, \quad (6.9)$$

and the characteristic curves satisfy

$$\frac{dx(t)}{dt} = u. \quad (6.10)$$

Notice that the slopes of the characteristic curves depend on the values of the solution, u . This plays an important role in the formation of shocks and other aspects of the flow solution, as we see in the following examples.

Inviscid Burgers' equation—rarefaction wave

Consider Burgers' equation (6.7) on domain $(x, t) \in \mathbb{R} \times \mathbb{R}^+$ with initial condition

$$u(x, 0) = \begin{cases} 0 & \text{if } x \leq 0, \\ x & \text{if } 0 \leq x \leq 1, \\ 1 & \text{if } x \geq 1. \end{cases} \quad (6.11)$$

Characteristics in the x - t plane are shown in Figure 6.1.1, and solution profiles from $t = 0$ to $t = 1$ are shown in Figure 6.1.2. In this case, the slope of the initial solution profile becomes less steep as time progresses. In fluid mechanics (where Burgers' equation comes from), this is called a rarefaction wave, since it describes a wave in which gas becomes less dense (or rarefies).

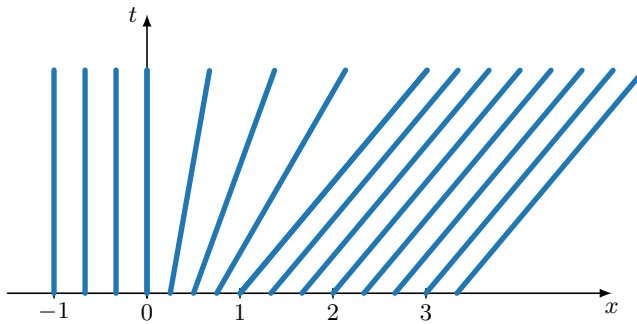


Figure 6.1.1. Characteristics for Burgers' equation—rarefaction wave (initial condition in equation (6.11)).

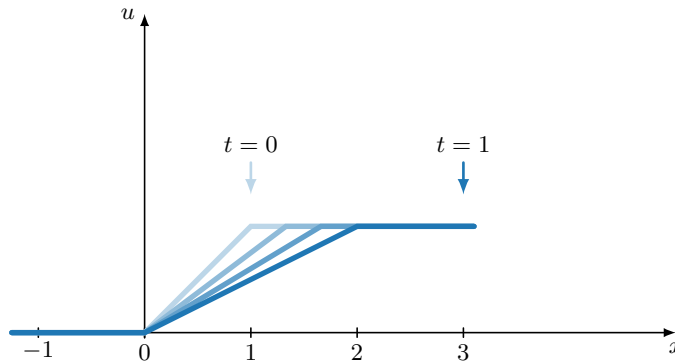


Figure 6.1.2. Solution profiles for Burgers' equation from $t = 0$ (lighter shade) to $t = 1$ (darker shade)—rarefaction wave (initial condition in equation (6.11)).

Inviscid Burgers' equation—steepening wave

We now consider Burgers' equation (6.7) on domain $(x, t) \in \mathbb{R} \times \mathbb{R}^+$ with initial condition

$$u(x, 0) = \begin{cases} 1 & \text{if } x \leq 0, \\ 1 - x & \text{if } 0 \leq x \leq 1, \\ 0 & \text{if } x \geq 1. \end{cases} \quad (6.12)$$

Characteristics in the x - t plane are shown in Figure 6.1.3, and solution profiles from $t = 0$ to $t = 1$ are shown in Figure 6.1.4. In this case, the slope of the initial solution profile steepens as time progresses, and the profile becomes discontinuous at $t = 1$.

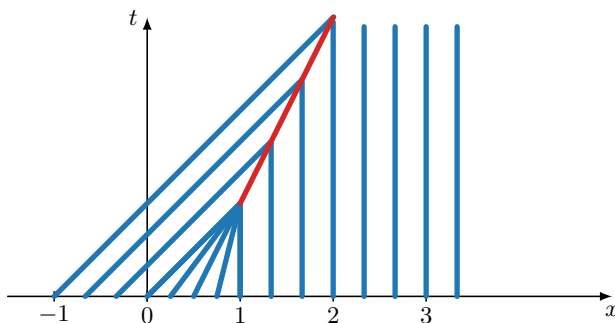


Figure 6.1.3. Characteristics for Burgers' equation—steepening wave (initial condition in equation (6.12)).

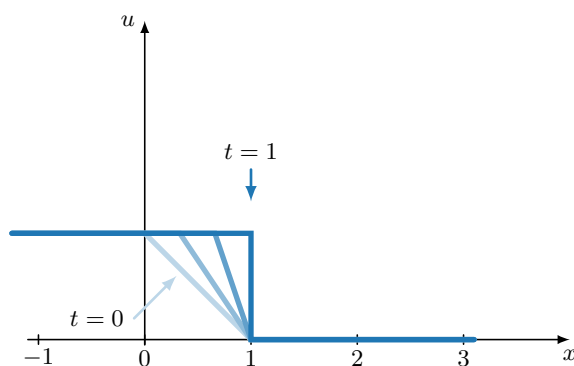


Figure 6.1.4. Solution profiles for Burgers' equation from $t = 0$ (lighter shade) to $t = 1$ (darker shade)—steepening wave (initial condition in equation (6.12)).

For Burgers' equation, discontinuities may form even when the initial condition is infinitely differentiable—i.e., in $C^\infty(\mathbb{R})$ —for example, for $u(x, 0) = \sin(x)$. The formation of discontinuities from smooth initial data in finite time is an important property of nonlinear conservation laws. It requires introducing the concept of weak solutions (see Section 6.1.2) and the development of numerical methods that can properly handle these discontinuities.

6.1.2 ■ Weak solutions

Since discontinuities may form in solutions of first-order conservation laws, we require a robust mathematical framework to describe in which sense solutions with discontinuities may satisfy the PDE conservation law (6.2); clearly, derivatives of the solution do not exist at the discontinuities in the classical sense. As such, we consider the concept of weak solutions of conservation law (6.2)—i.e., solutions that are not in C^1 , the set of continuously differentiable functions. A first definition is given in Definition 6.2.

Definition 6.2: Weak solution of conservation law, version I [233, Eq. (2.37)]

Function $u(x, t)$ is called a weak solution of the 1D conservation law of equation (6.2) if $u(x, t)$ satisfies the integral form of the conservation law,

$$\frac{d}{dt} \left(\int_a^b u(x, t) dx \right) + f(u(b, t)) - f(u(a, t)) = 0, \quad (6.13)$$

for almost all intervals (a, b) and times t .

This definition extends to functions that may not be in C^1 , and it is intuitive because the integral form of the conservation law expresses the basic conservation principle that should remain valid for solutions not in C^1 . A more formal definition is motivated as follows. As in Section 4.1, we consider a compactly supported smooth test function $\phi(x, t) \in C_0^1(\mathbb{R} \times \mathbb{R}^+)$ —that is, a function in $C^1(\mathbb{R} \times \mathbb{R}^+)$ that is zero outside of some bounded set in $\mathbb{R} \times \mathbb{R}^+$. We multiply conservation law (6.2) by such a $\phi(x, t)$ and integrate over domain $(x, t) \in \mathbb{R} \times \mathbb{R}^+$ to get

$$\int_0^\infty \int_{-\infty}^\infty \left(\frac{\partial u}{\partial t} \phi(x, t) + \frac{\partial f(u)}{\partial x} \phi(x, t) \right) dx dt = 0. \quad (6.14)$$

We now split the integral into its two natural terms, integrating by parts on both of these, but with respect to time for the first term and space for the second term. This gives

$$\begin{aligned} - \int_0^\infty \int_{-\infty}^\infty (u \phi_t + f(u) \phi_x) dx dt + \int_{-\infty}^\infty u(x, t) \phi(x, t) dx \Big|_{t=0}^{t=\infty} \\ + \int_0^\infty f(u(x, t)) \phi(x, t) dt \Big|_{x=-\infty}^{x=\infty} = 0. \end{aligned} \quad (6.15)$$

Since $\phi(x, t)$ vanishes for $x \rightarrow \pm\infty$ and $t \rightarrow \infty$, this leads to Definition 6.3, which is equivalent to Definition 6.2.

Definition 6.3: Weak solution of conservation law, version II [233, Eq. (2.33)]

Function $u(x, t)$ is called a weak solution of the 1D conservation law in equation (6.2) if, for any $\phi(x, t) \in C_0^1(\mathbb{R} \times \mathbb{R}^+)$,

$$\int_0^\infty \int_{-\infty}^\infty (u \phi_t + f(u) \phi_x) dx dt + \int_{-\infty}^\infty u(x, 0) \phi(x, 0) dx = 0. \quad (6.16)$$

Solutions with jump discontinuities—i.e., discontinuities where the one-sided limits exist and are finite but not equal—are important examples of weak solutions of conservation laws. For instance, we observed in the problem of Figure 6.1.4 that jump discontinuities may form in solutions of Burgers' equation. These jump discontinuities are also called shocks, because they are related to shock waves such as the sonic boom in gas dynamics. As is discussed in Section 6.1.3, a specific formula can be derived for the propagation speed of jump discontinuities in weak solutions of conservation laws, based on the integral form. Furthermore, the rarefaction wave in Figure 6.1.1 also describes a weak solution of Burgers' equation: in this case, $u(x, t)$ does not have a jump discontinuity, but its partial derivatives have jump discontinuities, so it is not in C^1 .

6.1.3 ■ Shock speed: The Rankine–Hugoniot relation

We now derive an expression for the propagation speed of jump discontinuities in weak solutions of conservation laws. This expression is called the Rankine–Hugoniot relation for shock speed, as described in Theorem 6.4.

Theorem 6.4: Rankine–Hugoniot relation [233, Eq. (2.18)]

Let $\hat{x}(t)$ be a curve describing a jump discontinuity in a weak solution of the 1D conservation law in equation (6.2). Then,

$$\hat{x}'(t) = \frac{f(u^+) - f(u^-)}{u^+ - u^-}, \quad (6.17)$$

where u^- and u^+ are the values of $u(x, t)$ to the left and to the right of the jump discontinuity.

Proof. Consider a and b to bound the location of the jump discontinuity at time t . Then, the weak solution satisfies the integral form in equation (6.13), and, hence,

$$\frac{d}{dt} \left(\int_a^{\hat{x}(t)} u(x, t) dx + \int_{\hat{x}(t)}^b u(x, t) dx \right) + f(u(b, t)) - f(u(a, t)) = 0. \quad (6.18)$$

Defining $G_a(\hat{x}(t), t) = \int_a^{\hat{x}(t)} u(x, t) dx$ and $G_b(\hat{x}(t), t) = \int_{\hat{x}(t)}^b u(x, t) dx$, we have

$$\frac{dG_a(\hat{x}(t), t)}{dt} = \frac{\partial G_a}{\partial \hat{x}} \frac{d\hat{x}}{dt} + \frac{\partial G_a}{\partial t} \quad (6.19a)$$

$$= u(\hat{x}(t), t) \hat{x}'(t) + \int_a^{\hat{x}(t)} u_t(x, t) dx \quad (6.19b)$$

$$= u(\hat{x}(t), t) \hat{x}'(t) - \int_a^{\hat{x}(t)} f(u)_x dx \quad (6.19c)$$

$$= u(\hat{x}(t), t) \hat{x}'(t) - (f(u(\hat{x}(t), t)) - f(u(a, t))), \quad (6.19d)$$

and the expression for the derivative of $G_b(\hat{x}(t), t)$ is similar. Since $u(x, t)$ is discontinuous at $\hat{x}(t)$, we need to consider the derivative of $G_a(\hat{x}(t), t)$ from the left and the derivative of $G_b(\hat{x}(t), t)$ from the right:

$$\left[\frac{dG_a(\hat{x}(t), t)}{dt} \right]^- = u^- \hat{x}'(t) - (f(u^-) - f(u(a, t))), \quad (6.20a)$$

$$\left[\frac{dG_b(\hat{x}(t), t)}{dt} \right]^+ = -u^+ \hat{x}'(t) - (f(u(b, t)) - f(u^+)). \quad (6.20b)$$

Substituting these into equation (6.18), we obtain

$$u^- \hat{x}'(t) - f(u^-) - u^+ \hat{x}'(t) + f(u^+) = 0, \quad (6.21)$$

which gives the desired result. $\square$

For Burgers' equation with initial conditions as in equation (6.12), a jump discontinuity forms at $t = 1$. Then, the Rankine–Hugoniot relation establishes a speed of propagation of

$$\hat{x}'(t) = \frac{f(u^+) - f(u^-)}{u^+ - u^-} = \frac{0^2/2 - 1^2/2}{0 - 1} = \frac{1}{2}, \quad (6.22)$$

as shown in Figure 6.1.5.

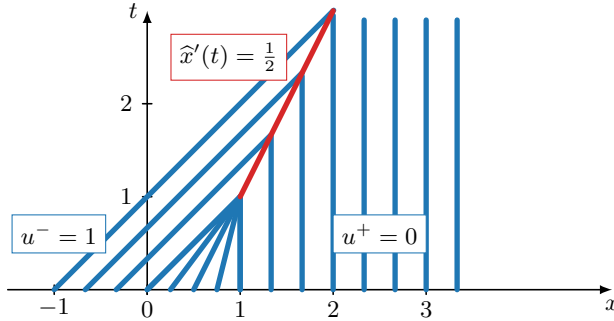


Figure 6.1.5. Characteristics for Burgers' equation—steepening wave and shock propagation (initial condition in equation (6.12)).

As we find in upcoming sections, the propagation of discontinuous solutions is fundamentally important to the development of effective numerical methods for hyperbolic conservation laws. As such, it is useful to set up a model problem that exhibits this behavior. To this end, Definition 6.5 introduces the so-called *Riemann problem*.

Definition 6.5: Riemann problem

Consider scalar conservation law (6.2) with initial condition

$$u(x, 0) = \begin{cases} u^- & \text{if } x \leq 0, \\ u^+ & \text{if } x > 0. \end{cases} \quad (6.23)$$

This problem, consisting of constant left and right states separated by a jump discontinuity, is called a *Riemann problem*.

As a specific example of a Riemann problem, consider Burgers' equation (6.7) for $(x, t) \in \mathbb{R} \times \mathbb{R}^+$ with initial condition

$$u(x, 0) = \begin{cases} 1 & \text{if } x \leq 0, \\ 0 & \text{if } x > 0. \end{cases} \quad (6.24)$$

With the help of Theorem 6.4, we see that the shock propagating with speed $\hat{x}'(t) = 1/2$, connecting constant regions in the x - t plane with values $u^- = 1$ and $u^+ = 0$, is a weak solution, and it can be shown that it is unique. Characteristics in the x - t plane are shown in Figure 6.1.6.

6.1.4 ■ Entropy weak solutions

Riemann problems (see Definition 6.5) for scalar conservation laws may also result in a rarefaction solution. Consider Burgers' equation (6.7) for $(x, t) \in \mathbb{R} \times \mathbb{R}^+$ with initial condition

$$u(x, 0) = \begin{cases} 0 & \text{if } x \leq 0, \\ 1 & \text{if } x > 0. \end{cases} \quad (6.25)$$

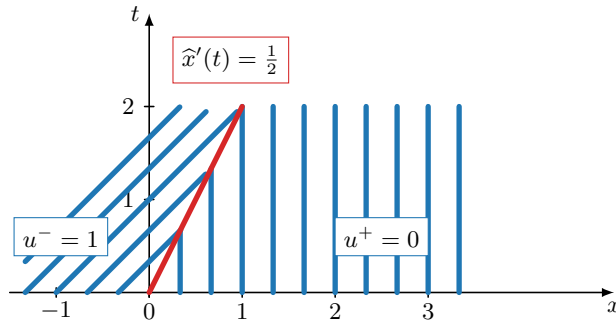


Figure 6.1.6. Characteristics for Burgers' equation with initial condition equation (6.24)—a Riemann problem with a shock solution.

Again, with the help of Theorem 6.4, we see that this Riemann problem has a weak solution with a shock that propagates with speed $\hat{x}'(t) = 1/2$, connecting constant regions with value $u^- = 0$ and $u^+ = 1$. Characteristics in the x - t plane are shown in Figure 6.1.7a. However, a second weak solution to this problem is given by a rarefaction wave with linear variation between u^- and u^+ (cf. equation (6.11) and Figure 6.1.7b). Therefore, we conclude that there is no unique weak solution to this problem. In fact, there are infinitely many weak solutions to this problem consisting of combinations of shocks and rarefactions.

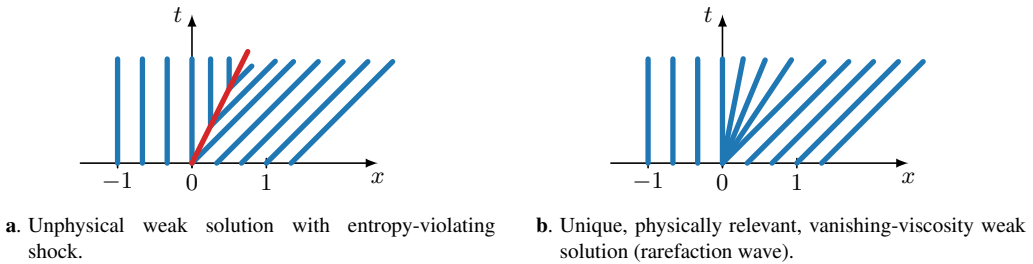


Figure 6.1.7. Characteristics for Burgers' equation—Riemann problem with (right) rarefaction solution and (left) unphysical shock solution.

So, how do we select a single weak solution? Luckily, the only physically valid solution in this case is the (continuous) weak solution with the single rarefaction wave, as depicted in Figure 6.1.7b. This solution is called the *vanishing-viscosity* weak solution of the problem. To see the connection to viscous solutions, consider the *viscous* form of Burgers' equation:

$$\frac{\partial u}{\partial t} + \frac{\partial u^2/2}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}. \quad (6.26)$$

In the limit of vanishing viscosity ($\nu \rightarrow 0$), the single rarefaction wave is the unique solution to this problem.

The vanishing-viscosity solution is also called the *entropy-satisfying* weak solution, because it is the unique weak solution that satisfies a so-called *entropy condition*, reflecting the fact that, in fluid mechanics, entropy across a shock cannot decrease. For Burgers' equation, entropy-satisfying shocks have characteristics that converge into the shock on both sides (as in Figure 6.1.5), whereas entropy-violating shocks have characteristics that diverge away from the

shock on both sides (as in Figure 6.1.7a). Intuitively, information travels forward in time along characteristics: information should always propagate from the initial and boundary data. In the case of the entropy-violating shock of Figure 6.1.7a, characteristics emanate from the shock, which is unphysical, since it would amount to new information being created in the shock. For the physically admissible, entropy-satisfying shock of Figure 6.1.5, information is carried into the shock from the initial or boundary conditions. It will be important to design numerical methods that converge to the physically relevant vanishing-viscosity solution among the possible weak solutions.

To formalize the notion of an entropy-satisfying weak solution, we consider the specific cases of convex or concave flux functions—see Definition 6.6. In this situation, there is a straightforward criterion to select admissible shocks.

Definition 6.6: Convex and concave flux functions

A flux function $f(u)$ is called convex if

$$f''(u) > 0 \quad \text{for all } u \quad (6.27)$$

and concave if

$$f''(u) < 0 \quad \text{for all } u. \quad (6.28)$$

For convex or concave flux functions, the Lax entropy condition in Definition 6.7 gives the condition under which a shock is entropy satisfying.

Definition 6.7: Lax entropy condition [131, Eq. (2.10)]

Let $f(u)$ be a convex or concave flux function. Then a shock curve $\hat{x}(t)$ is said to satisfy the Lax entropy condition if

$$f'(u^-(t)) > \hat{x}'(t) > f'(u^+(t)) \quad \forall t. \quad (6.29)$$

This condition for entropy-satisfying shocks can be directly interpreted in terms of characteristics converging into the shock. The shock in Figure 6.1.7a is not entropy satisfying, according to the Lax entropy condition—the characteristics emanate from the shock. Instead, the rarefaction solution in Figure 6.1.7b is the unique entropy-satisfying weak solution of the Riemann problem. In contrast, the shock in Figure 6.1.6 is entropy satisfying according to the Lax entropy condition—the characteristics enter the shock.

6.2 ■ Fundamentals of finite-volume methods

Objectives

The goal of this section is to introduce the fundamental ideas behind classical finite-volume discretizations for scalar nonlinear hyperbolic conservation laws in one spatial dimension. In this section, you will recognize that standard finite-difference methods (see Chapter 5) may produce oscillations and incorrect shock speeds when applied to nonlinear conservation laws. Thus, you will learn how to construct finite-volume methods based on numerical flux functions and associate the concept of discrete conservation with that of shocks moving with the correct speed.

6.2.1 ■ Difficulties with finite-difference methods applied to conservation laws

In this section, we consider numerical methods for scalar nonlinear conservation law (6.2). It is tempting to simply consider nonlinear generalizations of the finite-difference methods for the linear advection equation as discussed in Section 5.1 and apply them to the quasi-linear form of equation (6.2):

$$\frac{\partial u}{\partial t} + f'(u) \frac{\partial u}{\partial x} = 0. \quad (6.30)$$

However, in this section, we give two examples that highlight the limitations of directly using finite-difference techniques to solve nonlinear hyperbolic PDEs with shocks. In the first example, we see that the finite-difference discretization may converge to a solution with the incorrect shock speed, while, in the second example, we observe that finite-difference methods with order of accuracy greater than one may easily produce strong numerical oscillations at discontinuities that are not reduced in amplitude as the grid is refined; in fact, this may lead to unphysical values for some solution variables.

These examples motivate methods of *finite-volume* type. Finite-volume methods for hyperbolic conservation laws are closely related to finite-difference methods, but they differ in two crucial aspects. First, they enforce conservation in a discrete way, thus ensuring that shocks propagate at correct speeds. Second, the higher-order versions of finite-volume methods are carefully designed to avoid or limit unphysical oscillations at shocks.

A finite-difference method with incorrect shock speed

Consider a Riemann problem (Definition 6.5) with a shock for the quasi-linear form of Burgers' equation (6.9) with initial condition

$$u(x, 0) = \begin{cases} 1 & \text{if } x \leq 0, \\ 0 & \text{if } x > 0. \end{cases} \quad (6.31)$$

The solution of this problem is a shock with propagation speed $\frac{1}{2}$, as determined from the Rankine–Hugoniot relation, equation (6.17). Applying a first-order discretization that is explicit in time and backward in space (ETBS) to the quasi-linear form in equation (6.9) on a uniform mesh leads to the nonlinear finite-difference method

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + u_{k,\ell} \frac{u_{k,\ell} - u_{k-1,\ell}}{h_x} = 0, \quad (6.32)$$

where we have assumed that the wave speed, $f'(u) = u$, evaluated over the entire numerical solution, is never negative. Consequently, we choose the stable direction of the spatial finite-difference approximation as a backward difference.

For this example, at $t = 0$, we have $u_{k,0} = 1$ for $k \leq 0$ and $u_{k,0} = 0$ for $k > 0$. From equation (6.32), we see that at any time $\ell > 0$ the initial condition is retained— $u_{k,\ell} = 1$ for $k \leq 0$ and $u_{k,\ell} = 0$ for $k > 0$ —and this remains true as the grid is refined. As a result, the numerical solution converges to a discontinuous solution with an incorrect shock speed equal to zero, yielding a solution that is not a weak solution of Burgers' equation. While the example may appear specific to the zero initial condition on the right, the same issue also occurs numerically when the right side of the Riemann problem is modified to a value different from zero—e.g., $\frac{1}{3}$.

The origin of this deficiency can be traced to the fact that the finite-difference method in equation (6.32) does not properly adhere to the conservation principle that is expressed in the integral form of the conservation law, which led to the Rankine–Hugoniot relation for shock speed.

Specifically, the numerical scheme does not conserve “mass” in this case: the mass that is continually added or removed in the Riemann problem at a constant rate leads to an incorrect shock speed. As we see in the next section, finite-volume methods are constructed to satisfy a discrete conservation principle that precludes convergence to solutions with incorrect shock speeds.

Unphysical oscillations at discontinuities with a second-order finite-difference method

Even in the linear case of the one-way wave equation—i.e., conservation law (6.2) with linear flux function $f(u) = a u$ —finite-difference methods with order higher than one may exhibit strong oscillations at discontinuities. Figure 6.2.1 highlights an example of oscillations in the case of linear advection with a discontinuous profile using the Lax–Wendroff scheme, introduced in Section 5.1. The figure shows the numerical solution at $t = 0.3$ for initial condition

$$u(x, 0) = \begin{cases} 0.1 & \text{if } x \leq 0.5, \\ 1.0 & \text{if } x > 0.5 \end{cases} \quad (6.33)$$

and with periodic boundary conditions. Results for two different grid resolutions of $h_x = 0.02$ (corresponding to 50 intervals on the domain $[0, 1]$) and $h_x = 0.01$ (or 100 intervals) are shown. Spurious numerical oscillations are observed that do not decrease in amplitude as the grid is refined, leading to negative solution values, which may be unphysical in some applications. These oscillations are also related to the well-known Gibbs phenomenon that arises when approximating discontinuous functions using a Fourier series, and they are predicted by the dispersion analysis of finite-difference methods for linear advection in Section 5.4. That is, for second-order accurate methods, such as Lax–Wendroff, the dominant numerical error is dispersive. The Fourier transform of a discontinuous function contains a broad continuum of Fourier frequencies, which perfectly interfere to form the discontinuity at the initial time but get out of phase at later times due to the dispersion, resulting in strong oscillations. In some applications, the function u

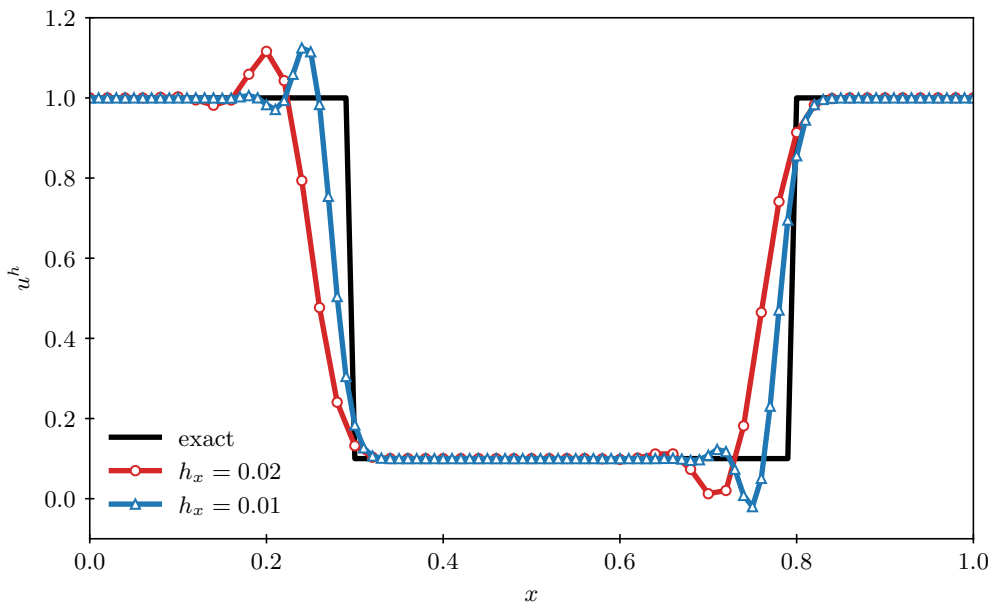


Figure 6.2.1. Lax–Wendroff approximation for linear advection of a discontinuous profile with wave speed $a = 1$.

may be physically restricted to nonnegative values, for example, if u represents a fluid density or pressure. In such applications, negative solution values near discontinuities due to unphysical oscillations may have to be avoided at all cost, since they may preclude simulations from completing in a meaningful way.

In Section 6.5, we look more closely at a class of finite-volume methods of order two (away from shocks) that are designed to carefully avoid these oscillations. Section 9.4 describes a different class of finite-volume methods with orders of accuracy that may exceed two.

6.2.2 ■ Finite-volume formulation and numerical flux functions

Finite-volume methods divide the spatial domain into n_{cells} intervals, $\Omega_k = [x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}]$, as shown in Figure 6.2.2. This is related to the 1D mesh used in the finite-difference method with $n_x + 1$ grid points, such that $n_{\text{cells}} = n_x$. In each cell, Ω_k , a discrete variable $u_k(t)$ approximates the spatial average of $u(x, t)$ over the cell; thus, we depict $u_k(t)$ as constant over the cell in the figure. A finite-volume method updates the approximate cell average, $u_k(t)$ in cell Ω_k , by using so-called *numerical fluxes* $\hat{f}_{k+\frac{1}{2}}(t)$ and $\hat{f}_{k-\frac{1}{2}}(t)$ that are defined at the right and left interfaces of the cell. The numerical flux function $\hat{f}_{k+\frac{1}{2}}(t)$, for example, is computed based on cell averages to the left and to the right of the cell interface at $x_{k+\frac{1}{2}}$.

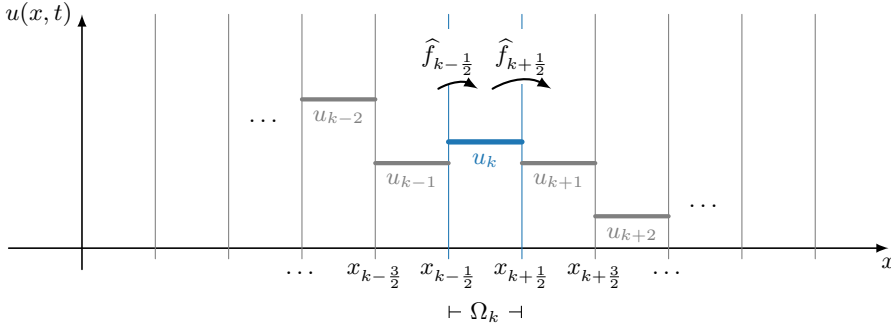


Figure 6.2.2. Finite-volume cells $\Omega_k = [x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}]$, with finite-volume approximations u_k .

To arrive at a definition of a finite-volume method, we consider the integral form of the 1D scalar conservation law,

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0, \quad (6.34)$$

on a uniform mesh, with $h_x = x_{k+\frac{1}{2}} - x_{k-\frac{1}{2}}$. We assume a uniform mesh in this chapter since it simplifies the presentation; however, it is not essential in what follows. By spatial integration over finite-volume cell $\Omega_k = [x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}]$, we obtain

$$h_x \frac{d\bar{u}_k(t)}{dt} + f(u(x_{k+\frac{1}{2}}, t)) - f(u(x_{k-\frac{1}{2}}, t)) = 0, \quad (6.35)$$

where

$$\bar{u}_k(t) = \frac{1}{h_x} \int_{\Omega_k} u(x, t) dx \quad (6.36)$$

is the cell average of $u(x, t)$ at time t in cell Ω_k . We then rewrite equation (6.35) as

$$\frac{d\bar{u}_k(t)}{dt} + \frac{f(u(x_{k+\frac{1}{2}}, t)) - f(u(x_{k-\frac{1}{2}}, t))}{h_x} = 0. \quad (6.37)$$

Notably, the conservation law solution $u(x, t)$ *exactly* satisfies equation (6.37)—no approximations have been introduced in deriving equation (6.37) from conservation law (6.34). We can turn equation (6.37) into a numerical formula of *finite-volume* type by considering discrete variables $u_k(t)$ that approximate cell averages of $u(x, t)$, that is,

$$u_k(t) \approx \bar{u}_k(t), \quad (6.38)$$

and by defining a *numerical flux function*,

$$\hat{f}_{k+\frac{1}{2}}(t) \approx f(u(x_{k+\frac{1}{2}}, t)), \quad (6.39)$$

that approximates the flux at cell interface $k + \frac{1}{2}$. In general, the numerical flux function $\hat{f}_{k+\frac{1}{2}}$ can depend on any number of cell averages to the left and to the right of cell interface $k + \frac{1}{2}$. This leads to the following general definition of a finite-volume method in semidiscrete form:

$$\frac{du_k(t)}{dt} + \frac{\hat{f}_{k+\frac{1}{2}}(t) - \hat{f}_{k-\frac{1}{2}}(t)}{h_x} = 0. \quad (6.40)$$

We call the form in equation (6.40) a *semidiscretized* form because the discretization is only in space and not yet in time. In what follows, we explain how the numerical flux functions $\hat{f}_{k+\frac{1}{2}}(t)$ can be chosen. The semidiscrete finite-volume formula in equation (6.40) is an ODE system that can be turned into a fully discretized finite-volume method by considering explicit or implicit time discretization methods of various order, as in Chapter 5.

For the specific finite-volume methods discussed in this book, we employ numerical flux functions $\hat{f}_{k+\frac{1}{2}}(t)$ that are defined as functions $f^*(\cdot, \cdot)$ of discrete solution approximations to the immediate left and right of the cell interface at $x_{k+\frac{1}{2}}$, which are denoted by $u_{k+\frac{1}{2}}^-(t)$ and $u_{k+\frac{1}{2}}^+(t)$. So we write

$$\hat{f}_{k+\frac{1}{2}}(t) = f^*(u_{k+\frac{1}{2}}^-(t), u_{k+\frac{1}{2}}^+(t)), \quad (6.41)$$

where the left and right states $u_{k+\frac{1}{2}}^-(t)$ and $u_{k+\frac{1}{2}}^+(t)$ at interface $k + \frac{1}{2}$ are computed using approximate cell averages close to interface $k + \frac{1}{2}$, for example, by polynomial interpolation. The form of the numerical flux function in equation (6.41) can be motivated by the notion of solving a Riemann problem at cell interface $k + \frac{1}{2}$ as in the well-known *Godunov method*; we explore this in detail in Section 6.3.3.

A finite-volume method of low spatial order is obtained from equation (6.40) by simply choosing

$$u_{k+\frac{1}{2}}^-(t) = u_k(t), \quad (6.42a)$$

$$u_{k+\frac{1}{2}}^+(t) = u_{k+1}(t) \quad (6.42b)$$

in the numerical flux function $f^*(\cdot, \cdot)$ in equation (6.41). Similarly, spatial accuracy of higher order can be obtained by *reconstructing* $u_{k+\frac{1}{2}}^-(t)$ and $u_{k+\frac{1}{2}}^+(t)$ using polynomial approximation on wider stencils of approximate cell averages that go beyond the cell averages immediately to the left and right of the cell interface. Specifically, we write

$$u_{k+\frac{1}{2}}^-(t) = u_{k+\frac{1}{2}}^-(u_{k-p}(t), \dots, u_k(t), \dots, u_{k+q}(t)), \quad (6.43a)$$

$$u_{k+\frac{1}{2}}^+(t) = u_{k+\frac{1}{2}}^+(u_{k+1-p}(t), \dots, u_{k+1}(t), \dots, u_{k+1+q}(t)) \quad (6.43b)$$

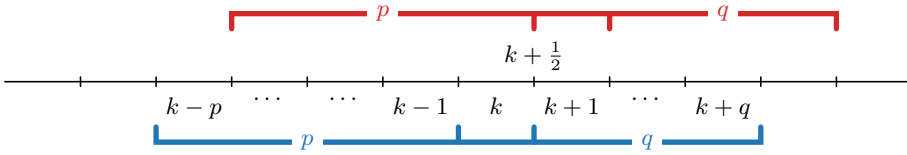


Figure 6.2.3. Collections of cell averages of width $p + q + 1$.

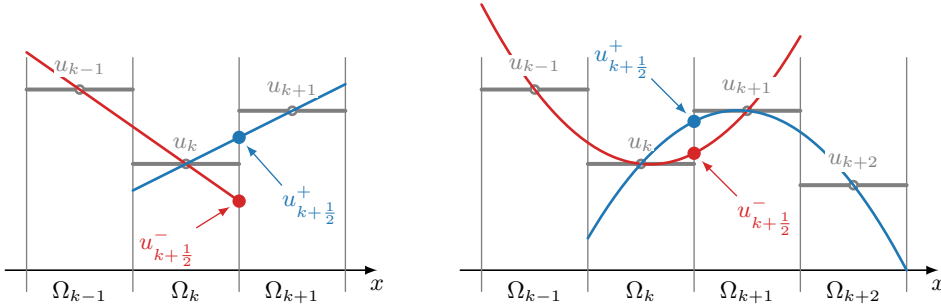


Figure 6.2.4. (left) Left-biased linear reconstruction and (right) central quadratic reconstruction.

for spatial *reconstruction stencils* of width $p + q + 1$ —see Figure 6.2.3. We can use polynomial interpolation of cell averages on these stencils to determine more accurate values for $u_{k+\frac{1}{2}}^-(t)$ and $u_{k+\frac{1}{2}}^+(t)$.

The left panel of Figure 6.2.4 shows an example of a reconstruction stencil with $p = 1$ and $q = 0$ using linear reconstruction. When combined with the Lax–Friedrichs numerical flux function that is introduced in Section 6.3.2, this gives a left-biased spatial discretization that achieves second-order accuracy. For linear advection with $a > 0$, this can be combined with a two-stage explicit Runge–Kutta method (see Section 6.5.1) to obtain a second-order accurate upwind-biased method that is numerically stable when the timestep satisfies a CFL condition with CFL constant 0.5. However, this type of discretization suffers from spurious oscillations at discontinuities similar to the Lax–Wendroff method in Figure 6.2.1 and is not suitable for nonlinear hyperbolic conservation laws. Therefore, Section 6.5 will present suitable second-order accurate finite-volume discretizations for problems that may exhibit shocks, considering a reconstruction stencil with $p = 1$ and $q = 1$ that uses linear reconstructions on pairs of adjacent cells of the stencil, combined with a nonlinear limiting mechanism to select the linear reconstruction slope in each cell that suppresses oscillations at discontinuities.

The right panel of Figure 6.2.4 shows an example of parabolic reconstruction on a reconstruction stencil with $p = 1$ and $q = 1$. This results in a central discretization that achieves third-order spatial accuracy, but again this discretization suffers from spurious oscillations at discontinuities and is not suitable for nonlinear hyperbolic conservation laws. In the rest of this chapter, we consider first-order accurate methods using reconstruction stencils with $p = 0$ and $q = 0$ (i.e., piecewise constant reconstruction as in equation (6.42)), except for Section 6.5, which discusses second-order methods using slope-limited linear reconstruction on reconstruction stencils with $p = 1$ and $q = 1$.

If we combine the finite-volume method in equation (6.40) and the reconstructions in equation (6.42) for the numerical flux function $f^*(\cdot, \cdot)$ in equation (6.41) with a simple explicit (or forward) Euler discretization in time, using the notation $u_{k,\ell}$ for a variable approximating the cell average of $u(x, t)$ in cell Ω_k at time t_ℓ ,

$$u_{k,\ell} \approx \bar{u}_k(t_\ell), \quad (6.44)$$

we obtain the following low-order explicit finite-volume method:

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + \frac{f^*(u_{k,\ell}, u_{k+1,\ell}) - f^*(u_{k-1,\ell}, u_{k,\ell})}{h_x} = 0. \quad (6.45)$$

With the finite-volume method defined as in equation (6.45), we revisit the idea of *consistency* introduced in Section 5.2. For finite-volume schemes, the consistency property places a requirement on the numerical flux function $f^*(\cdot, \cdot)$.

Definition 6.8: Consistent numerical flux function [131, Sec. 4.1]

A numerical flux function $f^*(u^-, u^+)$ for conservation law (6.34) is called *consistent* if

- $f^*(u, u) = f(u) \quad \forall u$,
- $f^*(u^-, u^+)$ is Lipschitz continuous in each argument.

It can be shown that, when using a consistent numerical flux, equation (6.40) is consistent with conservation law (6.34) in the sense that the spatial discretization error converges to zero as the grid is refined, as we have seen previously in the case of finite-difference methods. The first consistency requirement is also reasonable in light of equation (6.41)—for example, for a constant solution $u(x, t) = \hat{u}$, consistency implies that the numerical flux equals the actual flux, $f^*(\hat{u}, \hat{u}) = f(\hat{u})$.

We first focus on low-order explicit finite-volume methods as defined in equation (6.45), where the numerical flux at an interface depends only on the adjacent cell averages. In later sections, we consider finite-volume methods with higher orders of accuracy and implicit time integration.

Exact discrete conservation

We say that the semidiscrete finite-volume method, (6.40), is in *conservation form*, mimicking the conservation property of the PDE at a discrete level. Indeed, if we sum the equation over adjacent cells from cell index k_a to k_b , we obtain a telescoping sum in which numerical fluxes at intermediate cell interfaces cancel (see Figure 6.2.5):

$$\frac{d}{dt} \left(\sum_{k=k_a}^{k_b} u_k(t) \right) + \frac{\hat{f}_{k_b+\frac{1}{2}}(t) - \hat{f}_{k_a-\frac{1}{2}}(t)}{h_x} = 0. \quad (6.46)$$

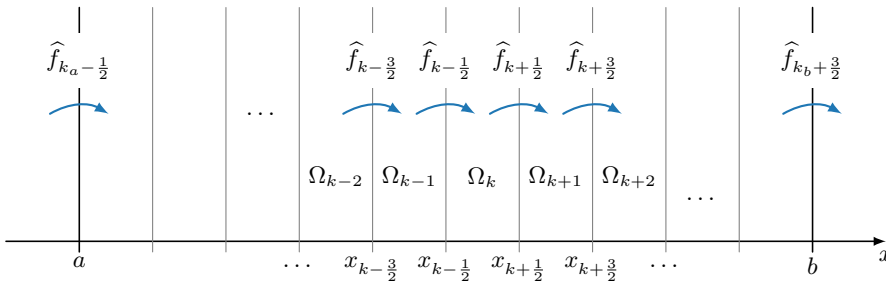


Figure 6.2.5. Finite-volume methods divide the interval $[a, b]$ into finite-volume cells $\Omega_k = [x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}]$.

This directly reflects the conservation property at the PDE level, which says

$$\frac{d}{dt} \left(\int_a^b u(x, t) dx \right) + f(u(b, t)) - f(u(a, t)) = 0. \quad (6.47)$$

We now have the notion of *exact discrete* conservation: updates in the total mass between a and b are only determined by the difference in the numerical fluxes at $x = a$ and $x = b$, due to the telescopic cancellation of the interior fluxes in the updates of equation (6.40). This exact discrete conservation property is central to preventing the issue of incorrect shock speeds that we observed in the example of Section 6.2.1 for a finite-difference method that was not in conservation form. According to Definition 6.2, weak solutions satisfy the integral form in equation (6.47), and equation (6.46) gives us a discrete analogue of this for finite-volume methods.

6.2.3 ■ Numerical conservation and the Lax–Wendroff theorem

We next introduce the Lax–Wendroff theorem (see Theorem 6.9), which asserts that finite-volume methods in conservative form (6.45) cannot converge to a solution that is not a weak solution, precluding incorrect shock speeds as in Section 6.2.1. Thus, if we construct convergent and conservative finite-volume schemes, we can guarantee that they will converge to a weak solution of the conservation law.

Intuitively, one can see that this guarantee arises due to the exact discrete conservation property that was discussed at the end of Section 6.2.2. More formally, we consider the finite-volume method

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + \frac{f_{k+\frac{1}{2},\ell}^* - f_{k-\frac{1}{2},\ell}^*}{h_x} = 0 \quad (6.48)$$

for scalar conservation law (6.34) on the domain $(x, t) \in (\mathbb{R}, \mathbb{R}^+)$ with initial condition $u(x, 0) = u^0(x)$, writing $f_{k+\frac{1}{2},\ell}^* = f^*(u_{k,\ell}, u_{k+1,\ell})$ as shorthand notation. We multiply equation (6.48) by a test function $\phi(x, t) \in C_0^1(\mathbb{R}^2)$ evaluated at x_k and t_ℓ and define $\phi_{k,\ell} = \phi(x_k, t_\ell)$. Then,

$$0 = \sum_{\ell=1}^{\infty} \sum_{k=-\infty}^{\infty} \left(\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + \frac{f_{k+\frac{1}{2},\ell}^* - f_{k-\frac{1}{2},\ell}^*}{h_x} \right) \phi_{k,\ell} h_x h_t, \quad (6.49a)$$

$$\begin{aligned} 0 &= - \sum_{\ell=1}^{\infty} \sum_{k=-\infty}^{\infty} \left(\frac{\phi_{k,\ell} - \phi_{k,\ell-1}}{h_t} u_{k,\ell} + \frac{\phi_{k,\ell} - \phi_{k-1,\ell}}{h_x} f_{k+\frac{1}{2},\ell}^* \right) h_x h_t \\ &\quad - \sum_{k=-\infty}^{\infty} \phi_{k,0} u_{k,1} h_x, \end{aligned} \quad (6.49b)$$

where we have used the summation-by-parts formula

$$\sum_{k=1}^N a_k (\phi_k - \phi_{k-1}) + \sum_{k=1}^N \phi_k (a_{k+1} - a_k) = -a_1 \phi_0 + \phi_N a_{N+1} \quad (6.50)$$

and the fact that $\phi_{k,\ell}$ vanishes for $k \rightarrow \pm\infty$ and $\ell \rightarrow \infty$. Using some further technical details (see [150], for example), it can then be shown that, if the values $u_{k,\ell}$ converge to a function $u(x, t)$ in the limit of $h_t \rightarrow 0$ and $h_x \rightarrow 0$, equation (6.49b) implies

$$- \int_0^\infty \int_{-\infty}^\infty (u \phi_t + f(u) \phi_x) dx dt - \int_{-\infty}^\infty u(x, 0) \phi(x, 0) dx = 0. \quad (6.51)$$

This implies that $u(x, t)$ is a weak solution of the conservation law according to Definition 6.3, as expressed in the following classical theorem.

Theorem 6.9: Lax–Wendroff theorem [131, Thm. 4.3]

If the numerical approximation $\{u_{k,\ell}\}$ obtained by finite-volume method (6.48) for conservation law (6.34) converges to a function $u(x, t)$ as $h_t \rightarrow 0$ and $h_x \rightarrow 0$, then $u(x, t)$ is a weak solution of the conservation law.

As discussed below in Section 6.3.2, Theorem 6.9 does not guarantee convergence to an entropy-satisfying weak solution, but only to a weak solution. We still need to take care in our construction of the numerical flux function to ensure that we get convergence to weak solutions that satisfy the entropy condition discussed in Section 6.1.4.

6.3 ■ Some finite-volume methods

Objectives

The goal of this section is to present some specific finite-volume schemes (through the definition of numerical flux functions) for scalar hyperbolic conservation laws in one spatial dimension. In this section, you will construct numerical flux functions that lead to first-order accurate discretizations and understand Godunov’s method as a finite-volume method that solves Riemann problems at cell interfaces.

6.3.1 ■ Numerical flux functions for linear advection

We have already considered a number of schemes in Section 5.1 for the linear advection equation

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0 \quad (6.52)$$

that can be written in the conservative form of equation (6.45): these are finite-difference methods that can be interpreted as finite-volume schemes. A couple of examples follow.

First-order upwind flux function

When $a > 0$ in equation (6.52), we can use the ETBS method from Section 5.1:

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k,\ell} - u_{k-1,\ell}}{h_x} = 0. \quad (6.53)$$

In this case, the numerical flux at interface $k + \frac{1}{2}$ in equation (6.45) is given by $f_{k+\frac{1}{2},\ell}^* = au_{k,\ell}$. This is called an “upwind” flux, since it is determined by the solution value to the left of the interface—i.e., in the direction the wind blows from, which is called the upwind direction. Similarly, if $a < 0$, we can use the explicit-in-time, forward-in-space (ETFS) method,

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k,\ell}}{h_x} = 0, \quad (6.54)$$

where the numerical flux at interface $k + \frac{1}{2}$ is now given by $f_{k+\frac{1}{2},\ell}^* = au_{k+1,\ell}$, determined by the solution value to the right of the interface (which is now the upwind direction).

These two cases are combined in the so-called (explicit-in-time) first-order upwind (FOU) method, with numerical flux $f_{k+\frac{1}{2},\ell}^* = f^*(u_{k,\ell}, u_{k+1,\ell})$ given by

$$f^*(u_{k,\ell}, u_{k+1,\ell}) = \frac{au_{k,\ell} + au_{k+1,\ell}}{2} - \frac{|a|}{2}(u_{k+1,\ell} - u_{k,\ell}). \quad (6.55)$$

It should be noted that the FOU method resulting from numerical flux function (6.55) can also be written as

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k+1,\ell} - u_{k-1,\ell}}{2h_x} = \frac{|a|h_x}{2} \frac{u_{k+1,\ell} - 2u_{k,\ell} + u_{k-1,\ell}}{h_x^2}. \quad (6.56)$$

This shows that the first term in numerical flux function (6.55) corresponds to a second-order accurate centered discretization of the spatial derivative $\partial u / \partial x$, which we know from Section 5.3.4 is by itself numerically unstable. The right-hand side of equation (6.56) can be viewed as a diffusive term proportional to a second-order accurate centered discretization of $\partial^2 u / \partial x^2$. This shows that the second term in numerical flux function (6.55) can be interpreted as providing numerical stability proportional to the *numerical diffusion coefficient*, $|a|h_x/2$, which approaches zero as the spatial grid is refined.

As discussed in Section 5.3, the associated finite-difference schemes (ETBS and ETFS, depending on the sign of a) are numerically stable under the CFL condition $h_t \leq h_x/|a|$, and we now see that this method is also a finite-volume method with (upwind) numerical flux (6.55) that is stable under this CFL condition. We take the upwind flux function in equation (6.55) for linear advection as a starting point and extend it in the next section to an upwind flux for nonlinear scalar conservation law (6.34).

Lax–Wendroff flux function

Similarly to the first-order schemes, the Lax–Wendroff finite-difference method from Section 5.1 can be written in the conservative form of equation (6.45) by using the numerical flux function

$$f^*(u_{k,\ell}, u_{k+1,\ell}) = \frac{au_{k,\ell} + au_{k+1,\ell}}{2} - \frac{a^2}{2} \frac{h_t}{h_x} (u_{k+1,\ell} - u_{k,\ell}). \quad (6.57)$$

This implies that the Lax–Wendroff scheme is also a finite-volume method, and we extend it to nonlinear scalar conservation law (6.34) in the next section.

6.3.2 ■ Numerical flux functions for nonlinear conservation laws

Lax–Friedrichs flux function

The factor $|a|$ in the FOU flux function given in equation (6.55) for linear advection represents the magnitude of the wave speed—i.e., the slope of the characteristic curve. Rewriting nonlinear conservation law (6.34) in quasi-linear form (see equation (6.30)) results in

$$\frac{\partial u}{\partial t} + f'(u) \frac{\partial u}{\partial x} = 0, \quad (6.58)$$

where $f'(u)$ is the slope of the characteristic curve. This provides inspiration to generalize the FOU flux function in equation (6.55) to the nonlinear case by considering

$$f^*(u_{k,\ell}, u_{k+1,\ell}) = \frac{f(u_{k,\ell}) + f(u_{k+1,\ell})}{2} - \frac{\alpha_{k+\frac{1}{2}}}{2} (u_{k+1,\ell} - u_{k,\ell}), \quad (6.59)$$

where we have replaced the linear fluxes au by nonlinear fluxes $f(u)$. Here, $\alpha_{k+\frac{1}{2}}$ is chosen as a suitable approximation to $|f'(u)|$ at $x_{k+\frac{1}{2}}$ and t_ℓ . As a start, we may be tempted to use

$$\alpha_{k+\frac{1}{2}} = \left| f' \left(\frac{u_{k,\ell} + u_{k+1,\ell}}{2} \right) \right|, \quad (6.60)$$

but it turns out that the resulting scheme may converge to an entropy-violating weak solution. A more suitable and commonly used choice for $\alpha_{k+\frac{1}{2}}$ is

$$\alpha_{k+\frac{1}{2}} = \max(|f'(u_{k,\ell})|, |f'(u_{k+1,\ell})|), \quad (6.61)$$

which can be shown to lead to a scheme that converges to the entropy solution (i.e., the vanishing-viscosity solution) if the flux function, $f(u)$, is convex ($f''(u) > 0$). This is at least plausible, since equation (6.61) provides more numerical diffusion than equation (6.60) when $f'(u)$ is monotone, simply because the maximum value of $f'(u)$ over the interval between $u_{k,\ell}$ and $u_{k+1,\ell}$ is achieved at one of its endpoints in this case.

Numerical flux function (6.59) with $\alpha_{k+\frac{1}{2}}$ chosen as in equation (6.61) is called the local Lax–Friedrichs flux function. In the global variant, $\alpha_{k+\frac{1}{2}}$ is taken to be a constant (independent of k), for example, as the maximum of $|f'(u_{k,\ell})|$ over the entire spatial domain. The Lax–Friedrichs method is first-order accurate and conservative, and, as we show in a later section, it provides enough numerical diffusion to handle shocks without numerical oscillations.

Lax–Wendroff flux function for nonlinear conservation laws

Here, we derive a nonlinear version of the Lax–Wendroff finite-difference/finite-volume scheme for conservation law (6.34). At point (x_k, t_ℓ) , we consider a Taylor series in time:

$$u(x_k, t_\ell + h_t) = u(x_k, t_\ell) + u_t(x_k, t_\ell)h_t + u_{tt}(x_k, t_\ell)\frac{h_t^2}{2} + \mathcal{O}(h_t^3). \quad (6.62)$$

Next, we replace temporal derivatives by spatial derivatives using the PDE

$$u_t = -f(u)_x \quad u_{tt} = -f(u)_{xt} = -(f(u)_t)_x = -(f'(u)u_t)_x = (f'(u)f(u)_x)_x. \quad (6.63)$$

Using centered differencing for the spatial derivatives and retaining terms up to second order, this results in

$$\begin{aligned} & \frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + \frac{f(u_{k+1,\ell}) - f(u_{k-1,\ell})}{2h_x} \\ &= \frac{h_t}{2h_x} \left(f'(u_{k+\frac{1}{2},\ell}) \frac{f(u_{k+1,\ell}) - f(u_{k,\ell})}{h_x} - f'(u_{k-\frac{1}{2},\ell}) \frac{f(u_{k,\ell}) - f(u_{k-1,\ell})}{h_x} \right), \end{aligned} \quad (6.64)$$

where we write

$$u_{k+\frac{1}{2},\ell} = \frac{u_{k,\ell} + u_{k+1,\ell}}{2}. \quad (6.65)$$

This is a finite-volume method with numerical flux function

$$f^*(u_{k,\ell}, u_{k+1,\ell}) = \frac{f(u_{k,\ell}) + f(u_{k+1,\ell})}{2} - \frac{h_t}{2h_x} f'(u_{k+\frac{1}{2},\ell})(f(u_{k+1,\ell}) - f(u_{k,\ell})). \quad (6.66)$$

This method is conservative and second-order accurate in space and time, yet it is not useful for problems with shocks, because it does not have enough numerical diffusion to avoid oscillations. We consider second-order accurate methods that can handle shocks in later sections.

6.3.3 ■ Finite-volume methods and the Riemann problem

Historically, *Godunov's method* was one of the first numerical methods for nonlinear hyperbolic conservation laws that took on the finite-volume form of equation (6.40). In its original, first-order accurate form, Godunov's method is a specific example of the low-order accurate explicit

finite-volume method of equation (6.45), with a specific choice of flux function $f^*(u_{k,\ell}, u_{k+1,\ell})$ at cell interface $k + \frac{1}{2}$. This so-called Godunov flux uses an exact solution of the Riemann problem at the cell interface $k + \frac{1}{2}$ with the cell averages $u_{k,\ell}$ and $u_{k+1,\ell}$ as the left and right constant states. To illustrate the approach in a concrete situation, we first consider the general solution of the Riemann problem for Burgers' equation, followed by an explanation of the physical reasoning behind and the practical implementation of Godunov's method. We conclude with the more general case of the Riemann problem solution and Godunov flux for a general scalar conservation law.

Exact solution of the Riemann problem for Burgers' equation

Consider the Riemann problem of Figure 6.3.1 for scalar conservation law (6.2), with initial condition $u(x, 0) = u^-$ for $x < 0$ and $u(x, 0) = u^+$ for $x \geq 0$. For Burgers' equation, (6.7), the flux is $f(u) = u^2/2$, which is a convex flux since $f''(u) = 1 > 0$. For convex flux functions, the unique entropy-satisfying solution to any Riemann problem is either a single rarefaction, as in the example of Figure 6.1.1, or a single shock propagating with the Rankine–Hugoniot shock speed $s = (f(u^+) - f(u^-))/(u^+ - u^-)$ of equation (6.17), as in the example of Figure 6.1.3. According to the Lax entropy condition (see Definition 6.7), the solution to the Riemann problem for Burgers' equation contains a shock when $u^- > u^+$, and a rarefaction with linear variation between u^- and u^+ results otherwise. Therefore, the exact solution $u(x, t)$ of the Riemann problem of Figure 6.3.1 for Burgers' equation is given by

$$\text{if } u^- > u^+ : u(x, t) = \begin{cases} u^- & \text{if } x < st, \\ u^+ & \text{if } x > st, \end{cases} \quad (6.67a)$$

$$\text{if } u^- \leq u^+ : u(x, t) = \begin{cases} u^- & \text{if } x < u^-t, \\ x/t & \text{if } u^-t \leq x \leq u^+t, \\ u^+ & \text{if } u^+t < x. \end{cases} \quad (6.67b)$$

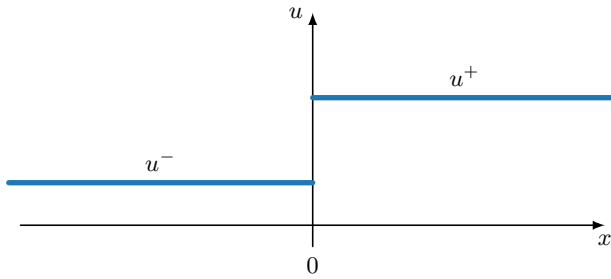


Figure 6.3.1. Riemann problem for scalar conservation law (6.2) with initial condition at $t = 0$ given by $u(x, 0) = u^-$ for $x < 0$ and $u(x, 0) = u^+$ for $x \geq 0$.

The Godunov numerical flux function will evaluate the flux $f(u)$ at the exact Riemann problem solution $u(x, t)$ in the point $x = 0$ —i.e., at the state $u^* = u(0, t)$, which is constant in time due to the self-similar nature of the Riemann problem solution. In the very special case of a shock solution with speed $s = 0$, $u(0, t)$ and u^* are not defined, but it is natural in this case to take the flux at $x = 0$ to be $f^*(u^-, u^+) = f(u^-) = f(u^+)$.

Godunov's method

Godunov's method for scalar conservation law (6.2) starts by considering a partition of the computational domain into finite-volume cells Ω_k and averages the initial condition $u(x, t_0)$ over

each Ω_k to define $u_{k,0}$. In general, we represent averages at each time t_ℓ as a grid function:

$$\mathbf{u}_\ell = (u_{1,\ell}, \dots, u_{k,\ell}, \dots, u_{n_{\text{cells}},\ell})^T \quad \text{for volumes } \Omega_k, k = 1, \dots, n_{\text{cells}}. \quad (6.68)$$

At each cell interface, $k + \frac{1}{2}$, we consider the *Riemann problem* with constant left and right states, $u_{k,0}$ and $u_{k+1,0}$, and evolve the solution by solving the Riemann problem *exactly* up to time $t_1 = t_0 + h_t$. The exact evolution is ensured by making the timestep small enough—i.e., $h_t \leq h_x / (2 \max_j |f'(u_{j,0})|)$ —so that the waves emanating from each interface will not interact with those of Riemann problems at an adjacent interface. To arrive at an approximation at timestep t_1 , the averages $\mathbf{u}_1$ are computed over each cell. This procedure is applied repeatedly in a sequence of operations that average, evolve, and average again at each timestep, where the only approximation occurs at the averaging steps. In practice, though, these averages are computed in a straightforward manner by evaluating equation (6.45) with the use of flux functions $f^*(u_{k,\ell}, u_{k+1,\ell})$ at interface $k + \frac{1}{2}$ that compute the exact flux of the Riemann problem solution at $x_{k+\frac{1}{2}}$ at time $t_{\ell+1}$. If we use such an *exact Riemann solver* to compute $f^*(u_{k,\ell}, u_{k+1,\ell})$ as the exact flux $f^*_{k+\frac{1}{2},\ell}$, then the fluxes between t_ℓ and $t_{\ell+1}$ are computed exactly. Thus, due to the conservation law form of the PDE, the resulting update in average values from $\mathbf{u}_\ell$ to $\mathbf{u}_{\ell+1}$ is computed exactly—i.e., the averaging of the exactly evolved Riemann solutions is exact, and the only approximation in the scheme is the repeated averaging.

Figure 6.3.2 sketches the Godunov procedure for an example where the averaged initial condition at time t_0 is given by

$$u(x, t_0) = \begin{cases} 1.0 & \text{if } x \in \Omega_0, \\ -0.2 & \text{if } x \in \Omega_1, \\ 0.2 & \text{if } x \in \Omega_2, \\ 0.6 & \text{if } x \in \Omega_3, \\ 0.2 & \text{if } x \in \Omega_4. \end{cases} \quad (6.69)$$

Here, we consider cells of unit length (i.e., $h_x = 1$), which leads to a bound of $h_t = 0.5$ in the CFL condition. The figure shows the averaging, evolving, averaging step from t_0 to t_1 .

The update formula for Godunov's method can be viewed as having the conservative form

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + \frac{f^*_{k+\frac{1}{2},\ell} - f^*_{k-\frac{1}{2},\ell}}{h_x} = 0, \quad (6.70)$$

where $f^*_{k+\frac{1}{2},\ell}$ is the exact Riemann flux for the Riemann problem with initial data (at time t_ℓ) of $u_{k,\ell}$ and $u_{k+1,\ell}$, evaluated at interface $x_{k+\frac{1}{2}}$ and time $t_{\ell+1}$. The Riemann fluxes, in fact, remain exact up to $h_t < h_x / \max_j |f'(u_{j,\ell})|$ (twice the timestep limitation mentioned above that precludes waves interacting), since waves emanating from interface $k + \frac{1}{2}$ cannot reach either of its two adjacent interfaces and will not influence the state or the flux there before that amount of time has passed. While interactions between waves from adjacent interfaces can occur before this time, these interactions cannot yet affect the fluxes at either interface.

The flux $f^*_{k+\frac{1}{2},\ell}$ obtained by the exact Riemann solver is called the Godunov flux. The exact Riemann flux computations imply consistency with the PDE, numerical stability follows from the timestep constraint that precludes errors arising from waves reaching adjacent interfaces before averaging, and correct shock speeds follow from the conservative discretized form. It is remarkable that the physically motivated principles behind Godunov's method automatically lead to these desirable numerical properties; for this reason, Godunov's method is considered a milestone in the derivation of numerical methods for nonlinear hyperbolic conservation laws.

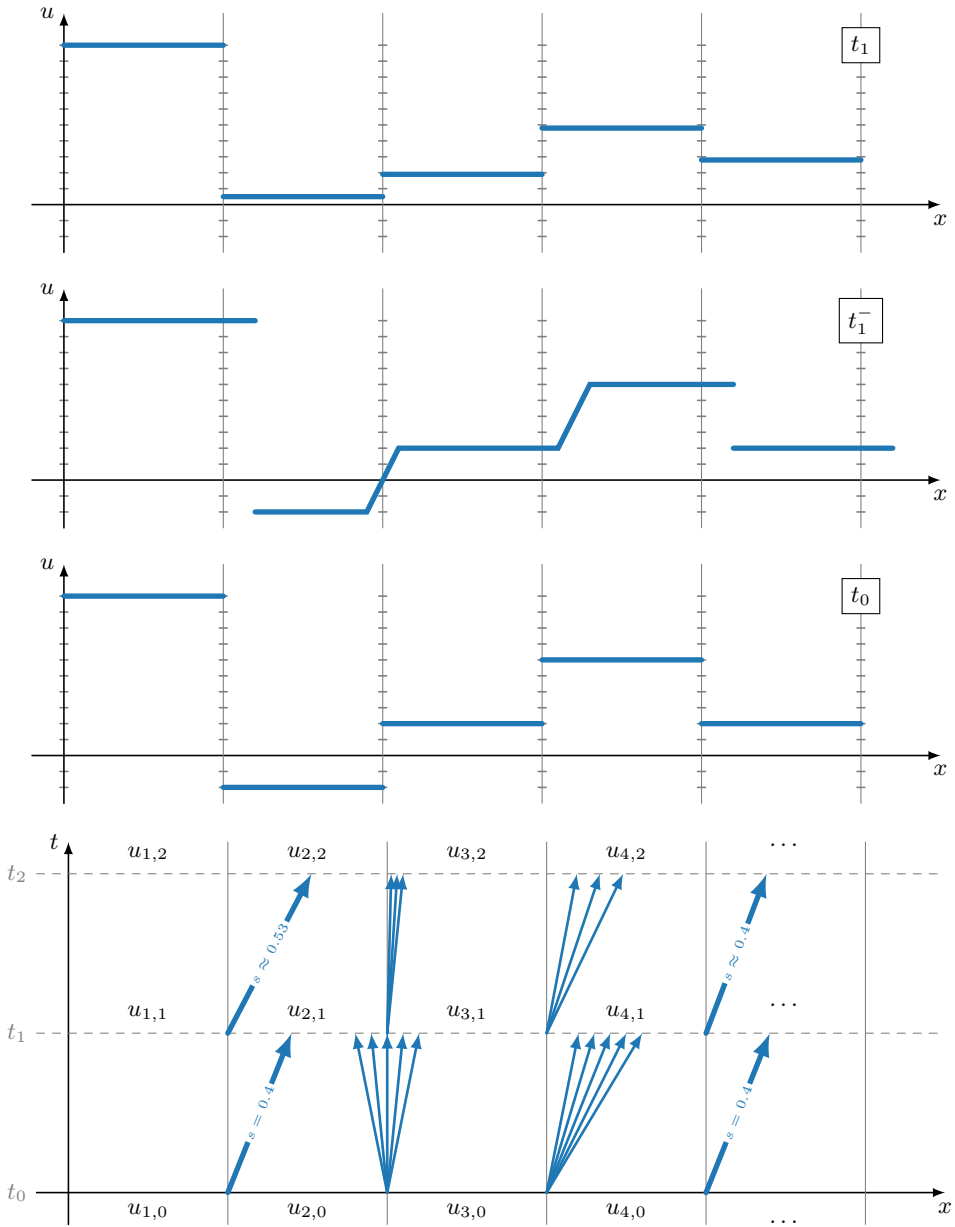


Figure 6.3.2. Godunov's method: (top three figures) the initial cell averages at time t_0 that form Riemann problems at cell interfaces, the evolved state at t_1^- (before averaging) obtained by exactly solving the Riemann problems, and the averaged values representing the approximation at time t_1 ; (bottom figure) characteristics for the Riemann problems at each time level and at each interface. The timestep is $h_t = 0.5$.

From this viewpoint, the original Godunov method is a specific example of a low-order finite-volume method (6.45) that uses an exact Riemann solver as numerical flux function. Higher-order extensions of Godunov's method can be considered by applying the exact Riemann solver (or an approximate Riemann solver) to compute the numerical flux $f^*(u_{k+\frac{1}{2}}^-, u_{k+\frac{1}{2}}^+)$ of equation (6.41) based on left and right states in the Riemann problem that are obtained from multiple

cell averages adjacent to the cell interface, using, for example, polynomial interpolation approaches. In this context, Godunov's approach is often referred to as the *REA algorithm*:

1. **Reconstruct** the solution as a piecewise polynomial on each cell.
2. **Evolve** the Riemann interface problems by a distance of h_t .
3. **Average** the evolved quantities.

The ideas behind Godunov's method apply to both scalar equations and systems. For example, exact Riemann solvers can be implemented for hyperbolic systems from compressible fluid dynamics. Next, we describe an exact Riemann solver for the case of general flux functions in scalar conservation law (6.2), generalizing the exact Riemann flux for Burgers' equation that follows from equations (6.67a) and (6.67b).

Exact Riemann solver for general scalar conservation law

Consider the Riemann problem from Figure 6.3.1 for general scalar conservation law (6.2), and assume, as a first case, that the flux function is convex: $f''(u) > 0$. Let u_s be the unique state where $f(u)$ attains its minimum, i.e., $f'(u_s) = 0$. Here, the subscript "s" denotes "stagnation," because $f'(u)$ is the speed of the wave and is zero at this state. Then, the flux f^* for the Riemann problem solution at $x = 0$ is given by

$$f^* = \begin{cases} f(u^-), & \text{shock with } s > 0, \\ f(u^+), & \text{shock with } s \leq 0, \\ f(u^-), & \text{rarefaction with } f'(u^-) \geq 0, \\ f(u^+), & \text{rarefaction with } f'(u^+) \leq 0, \\ f(u_s), & \text{rarefaction with } f'(u^-) \leq 0 \leq f'(u^+). \end{cases} \quad (6.71)$$

By carefully considering all cases, it can be shown that the expressions for the Riemann solution from equation (6.71) are equivalent to

$$f^* = \begin{cases} \max_{u^+ \leq u \leq u^-} f(u) & \text{if } u^- > u^+, \\ \min_{u^- \leq u \leq u^+} f(u) & \text{if } u^- \leq u^+, \end{cases} \quad (6.72)$$

where the first case corresponds to a shock and the second to a rarefaction if $f(u)$ is convex. Equation (6.72) may not be practical for computing and is mostly of theoretical relevance. It can be shown that the same expression is valid for concave flux functions and for more general flux functions that are neither convex nor concave [151].

6.3.4 ■ Numerical examples

We illustrate both the local Lax–Friedrichs and Godunov methods by considering two initial conditions for Burgers' equation on the domain $-1 \leq x \leq 6$. In the first, we take

$$u(x, 0) = \begin{cases} 1 & \text{if } x < 0, \\ 1 + x & \text{if } 0 \leq x \leq 1, \\ 2 & \text{if } 1 < x, \end{cases} \quad (6.73)$$

leading to a rarefaction wave solution. We use "ghost cells" (see Section 9.3) to impose boundary conditions so that the left-hand and right-hand boundaries of the interval keep fixed values for all time. Figure 6.3.3 shows the numerical solutions computed for this initial condition at time

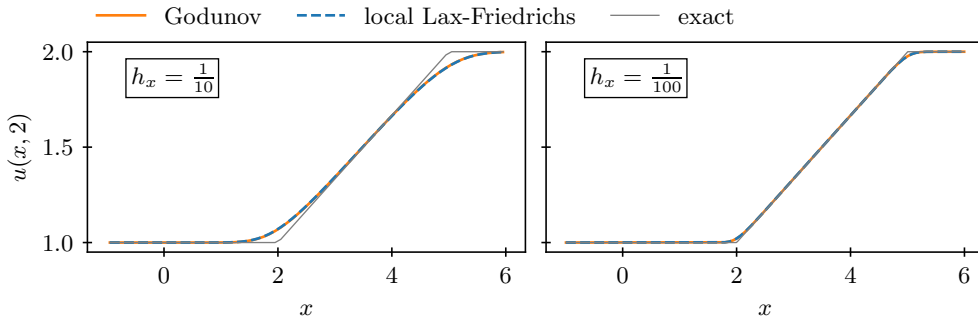


Figure 6.3.3. Numerical solutions to Burgers' equation with initial condition as given in equation (6.73). Note that the solution using the local Lax–Friedrichs flux function is “on top of” that using the Godunov flux.

$t = 2$, with timestep fixed as $h_t = h_x / (2 \max_j |f'(u_{j,\ell})|)$, which is stable for both schemes. We note that there is very little difference between the solutions computed using the two methods.

Our second example considers the same domain and boundary conditions, but with the initial condition

$$u(x, 0) = \begin{cases} 2 & \text{if } x < 0, \\ 2 - x & \text{if } 0 \leq x \leq 1, \\ 1 & \text{if } 1 < x, \end{cases} \quad (6.74)$$

leading to a shock that develops at time $t = 1$. Figure 6.3.4 shows the numerical solutions computed for this initial condition at times $t = 1/2$ (before the shock forms) and $t = 2$ (after the

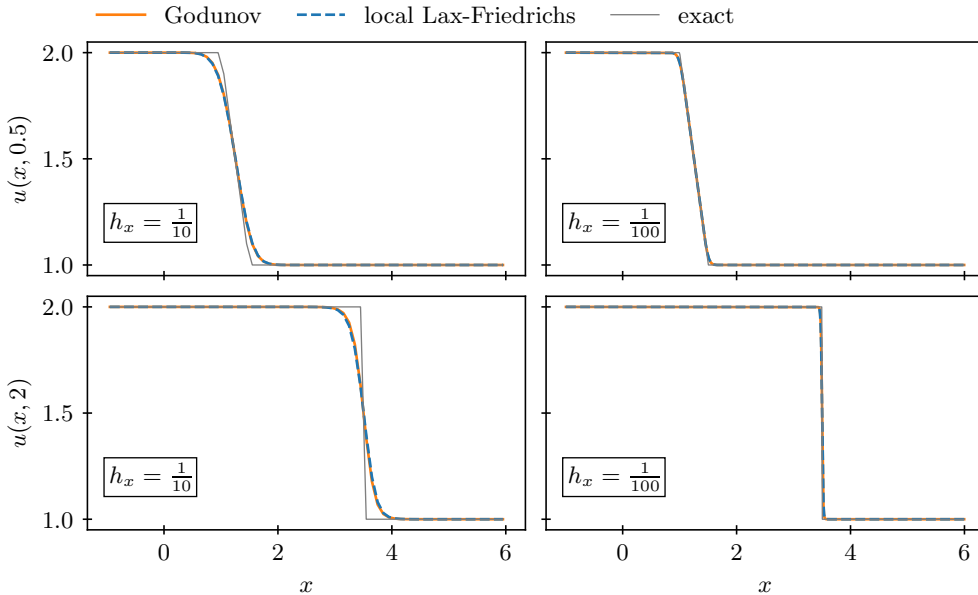


Figure 6.3.4. Numerical solutions to Burgers' equation with initial condition as given in equation (6.74). Note that the solution using the local Lax–Friedrichs flux function is “on top of” that using the Godunov flux.

shock forms), using the same timestep as above. We note that there is, again, very little difference between the solutions computed using the two methods.

6.4 ■ Total variation and finite-volume methods

Objectives

A key property that we aim for in finite-volume methods is that they do not introduce spurious oscillations into the numerical solutions. In the finite-volume framework, this is often considered in the context of *total-variation diminishing* (TVD) methods. In this section, you will be introduced to the concepts of total-variation and TVD schemes and examine some of their basic properties. Specifically, you will develop insight into the concept of total variation to describe local extrema in solutions to conservation laws. You will also identify what it means for a scheme to be TVD and outline some basic conditions and properties of TVD methods.

6.4.1 ■ Total-variation properties of scalar conservation law solutions

Recall, from Section 6.1.1, that the characteristic curves of scalar nonlinear conservation law (6.2) are straight lines on which the solution $u(x, t)$ is constant. This immediately implies that any local minima or maxima in the solution $u(x, t_1)$ at time $t_1 > t_0$ are present in the initial condition $u(x, t_0)$ —in other words, no new local minima or maxima can appear over time for solutions of the conservation law. This property is captured by the concept of the *total variation* of a function $u(x, t)$, which we define in Definition 6.10. We see that, for any entropy-satisfying solution $u(x, t)$ of conservation law (6.2) with a convex flux function, the total variation decreases or stays constant over time. In the next section, we design schemes that satisfy a discrete version of this TVD property, thus provably avoiding the occurrence of spurious numerical oscillations at shocks. The following definition of total variation is valid when $u(x)$ is differentiable.

Definition 6.10: Total variation [151, Sec. 6.7]

The total variation of differentiable function $u(x)$ over interval Ω is defined by

$$TV(u(x), \Omega) = \int_{\Omega} \left| \frac{du(x)}{dx} \right| dx. \quad (6.75)$$

If $u(x)$ contains jump discontinuities, then a more general definition is required, as in the following:

$$TV(u(x), \Omega) = \limsup_{\epsilon \rightarrow 0} \frac{1}{\epsilon} \int_{\Omega} |u(x) - u(x - \epsilon)| dx. \quad (6.76)$$

Alternatively, equation (6.75) can be interpreted in the distributional sense for functions with jump discontinuities, and we use Definition 6.10 for the general case (including for functions with discontinuities) in what follows.

Figure 6.4.1 shows an example of how, for a smooth function $u(x)$ on the interval $[x_1, x_2]$, the total variation can be computed as the sum of the absolute values of the differences in function values between successive local minima and maxima:

$$TV(u(x), [x_1, x_2]) = |u_b - u_a| + |u_c - u_b| + |u_d - u_c| + |u_e - u_d|. \quad (6.77)$$

From this example, it is easy to see that the total variation would increase if an additional local minimum or maximum were to be introduced into the function.

If we consider the total variation over time for a function $u(x, t)$ that is a solution of the conservation law in equation (6.2) with convex flux function, then it can be shown that the total

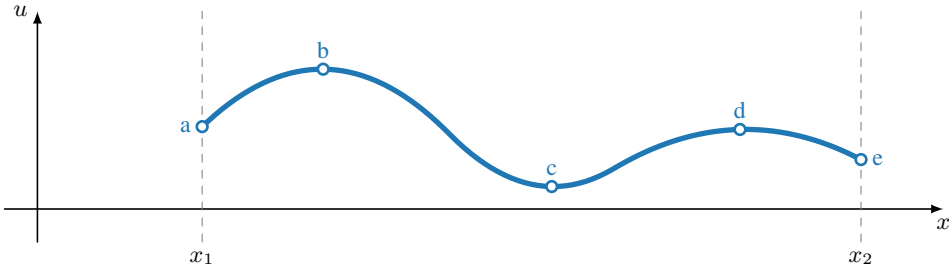


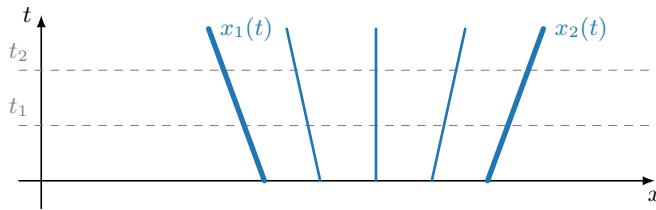
Figure 6.4.1. Smooth function $u(x)$ with total variation $TV(u(x), [x_1, x_2]) = |u_b - u_a| + |u_c - u_b| + |u_d - u_c| + |u_e - u_d|$.

variation of $u(x, t)$ is nonincreasing. We say that these solutions $u(x, t)$ satisfy the TVD property. That is, for any two characteristic curves $x_1(t)$ and $x_2(t)$ of a solution $u(x, t)$ of the conservation law (6.2), we have

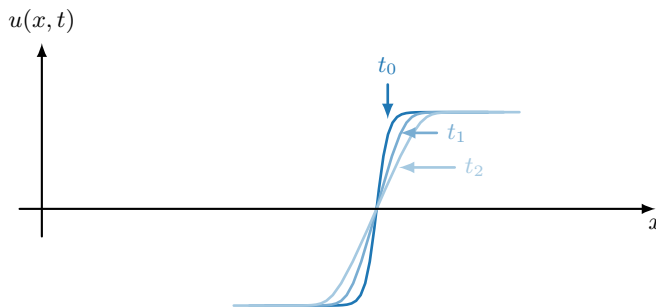
$$\frac{d}{dt} \left(\int_{x_1(t)}^{x_2(t)} \left| \frac{\partial u(x, t)}{\partial x} \right| dx \right) \leq 0. \quad (6.78)$$

This property can be illustrated by some examples. Figure 6.4.2 illustrates the case of a smooth solution $u(x, t)$ of the conservation law in equation (6.2), where the total variation between two characteristics, $x_1(t)$ and $x_2(t)$, does not change in time, because the solution is constant on characteristics and no new local extrema can arise between the characteristics. With $\Omega(t) = [x_1(t), x_2(t)]$, the time-dependent total variation becomes

$$TV(u(x, t), \Omega(t)) = \int_{x_1(t)}^{x_2(t)} \left| \frac{\partial u(x, t)}{\partial x} \right| dx, \quad (6.79)$$



a. Characteristics.



b. Solutions at time t_0 , t_1 , and t_2 .

Figure 6.4.2. Characteristics and solutions for smooth solution $u(x, t)$ of the conservation law in equation (6.2), where the total variation between two characteristics $x_1(t)$ and $x_2(t)$ does not change in time.

giving

$$TV(u(x, t_1), \Omega(t_1)) = TV(u(x, t_2), \Omega(t_2)) \quad (6.80)$$

and

$$\frac{dTV(u(x, t), \Omega(t))}{dt} = 0. \quad (6.81)$$

Figure 6.4.3 illustrates the case of a solution, $u(x, t)$, of conservation law (6.2) for which a shock develops, and the total variation between the two characteristics, $x_1(t)$ and $x_2(t)$, decreases when characteristics $y_1(t)$ and $y_2(t)$ merge to form a shock along curve $y(t)$. In the schematic, there is a local extremum between $y_1(t)$ and $y_2(t)$ that disappears when the shock is formed. In this case, we have

$$TV(u(x, t_2), \Omega(t_2)) < TV(u(x, t_1), \Omega(t_1)), \quad (6.82)$$

with $\Omega(t) = [x_1(t), x_2(t)]$.

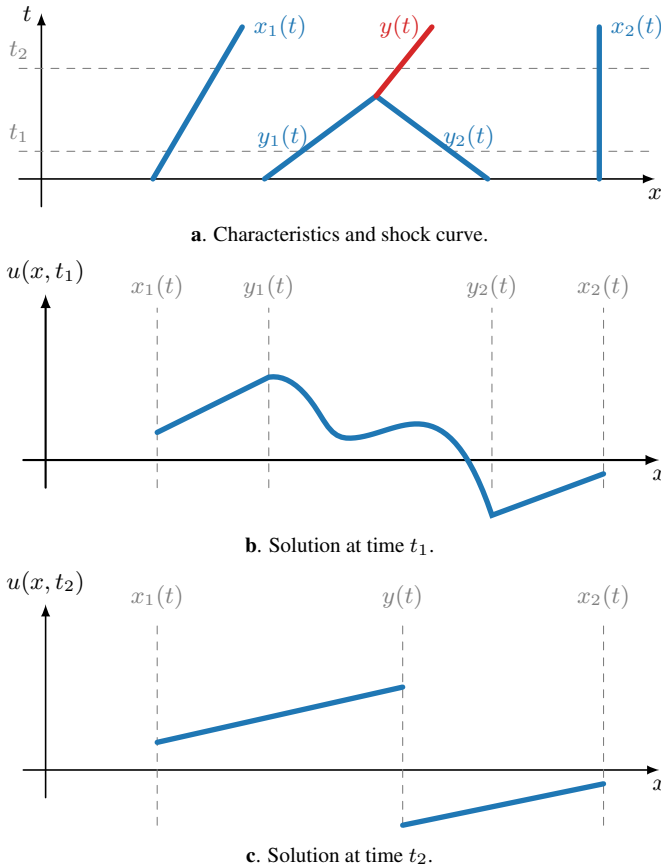


Figure 6.4.3. Characteristics and solutions for solution $u(x, t)$ of the conservation law in equation (6.2) for which a shock develops and the total variation between the two characteristics $x_1(t)$ and $x_2(t)$ decreases when characteristics $y_1(t)$ and $y_2(t)$ merge to form a shock along curve $y(t)$.

6.4.2 ■ TVD schemes for conservation laws

Our overall goal is to design numerical methods that avoid spurious oscillations. One approach is to formulate numerical schemes that satisfy a discrete version of the TVD property in equation (6.78). To do this, we extend the concept of total variation to the discrete case as in Definition 6.11.

Definition 6.11: Total variation of a grid function [151, Sec. 6.7]

Consider grid function $\mathbf{u}_\ell = (u_{1,\ell}, \dots, u_{k,\ell}, \dots, u_{n_{\text{cells}},\ell})^T$ at time t_ℓ and volumes Ω_k , with $k = 1, \dots, n_{\text{cells}}$. In addition, assume periodic boundary conditions in space. Then, the total variation of $\mathbf{u}_\ell$ is defined by

$$TV(\mathbf{u}_\ell) = \sum_{k=1}^{n_{\text{cells}}} |u_{k,\ell} - u_{k-1,\ell}|, \quad (6.83)$$

where $u_{0,\ell} = u_{n_{\text{cells}},\ell}$ by periodicity.

Then, we define a discrete version of the TVD property in Definition 6.12.

Definition 6.12: Discrete TVD property [151, Sec. 6.7]

$$TV(\mathbf{u}_{\ell+1}) \leq TV(\mathbf{u}_\ell). \quad (6.84)$$

As in the continuous case, this property ensures that no new local extrema are introduced into the numerical solution as time evolves. The assumption on periodicity is both convenient and a common use case. The definition above and the theory below can be extended to other cases, but the definition of $TV(\mathbf{u}_\ell)$ must be adapted to the specific type of boundary condition.

TVD conditions

We now consider explicit one-step update methods for the conservation law in equation (6.2) of the form

$$u_{k,\ell+1} = u_{k,\ell} - \beta_{k-\frac{1}{2}}(u_{k,\ell} - u_{k-1,\ell}) + \eta_{k+\frac{1}{2}}(u_{k+1,\ell} - u_{k,\ell}), \quad (6.85)$$

where the coefficients $\beta_{k-\frac{1}{2}}$ and $\eta_{k+\frac{1}{2}}$ may depend on the approximation $\mathbf{u}_\ell$ at time t_ℓ . For example, the FOU method for advection in equation (6.56) is in this form, as we explore below. For ease of notation, we define the forward and backward differences

$$\Delta^+ u_{k,\ell} = u_{k+1,\ell} - u_{k,\ell}, \quad (6.86a)$$

$$\Delta^- u_{k,\ell} = u_{k,\ell} - u_{k-1,\ell}. \quad (6.86b)$$

Theorem 6.13 provides explicit conditions under which the numerical method defined in equation (6.85) satisfies the TVD property of Definition 6.12.

Theorem 6.13: TVD conditions [125, Lem. 2.2]

Consider the numerical method

$$u_{k,\ell+1} = u_{k,\ell} - \beta_{k-\frac{1}{2}} \Delta^- u_{k,\ell} + \eta_{k+\frac{1}{2}} \Delta^+ u_{k,\ell} \quad (6.87)$$

with periodic boundary conditions. If, for all k ,

$$\beta_{k+\frac{1}{2}} \geq 0, \quad (6.88a)$$

$$\eta_{k+\frac{1}{2}} \geq 0, \quad (6.88b)$$

$$\beta_{k+\frac{1}{2}} + \eta_{k+\frac{1}{2}} \leq 1, \quad (6.88c)$$

then the numerical method is TVD.

Proof. We have

$$TV(\mathbf{u}_{\ell+1}) = \sum_k |u_{k+1,\ell+1} - u_{k,\ell+1}| \quad (6.89a)$$

$$= \sum_k |\Delta^+ u_{k,\ell} - \beta_{k+\frac{1}{2}} \Delta^- u_{k+1,\ell} + \eta_{k+\frac{3}{2}} \Delta^+ u_{k+1,\ell} + \beta_{k-\frac{1}{2}} \Delta^- u_{k,\ell} - \eta_{k+\frac{1}{2}} \Delta^+ u_{k,\ell}|. \quad (6.89b)$$

Using $\Delta^- u_{k+1,\ell} = \Delta^+ u_{k,\ell}$, $\Delta^- u_{k,\ell} = \Delta^+ u_{k-1,\ell}$, and the triangle inequality, we get

$$TV(\mathbf{u}_{\ell+1}) \leq \sum_k \left(|(1 - \beta_{k+\frac{1}{2}} - \eta_{k+\frac{1}{2}}) \Delta^+ u_{k,\ell}| + |\eta_{k+\frac{3}{2}} \Delta^+ u_{k+1,\ell}| + |\beta_{k-\frac{1}{2}} \Delta^+ u_{k-1,\ell}| \right) \quad (6.90a)$$

$$= \sum_k \left(|(1 - \beta_{k+\frac{1}{2}} - \eta_{k+\frac{1}{2}}) \Delta^+ u_{k,\ell}| + |\eta_{k+\frac{1}{2}} \Delta^+ u_{k,\ell}| + |\beta_{k+\frac{1}{2}} \Delta^+ u_{k,\ell}| \right), \quad (6.90b)$$

where the last equality is obtained by shifting the indices in the sum and using periodicity. Using the assumptions on the coefficients $\beta_{k+\frac{1}{2}}$ and $\eta_{k+\frac{1}{2}}$, we obtain the desired result:

$$TV(\mathbf{u}_{\ell+1}) \leq \sum_k (1 - \beta_{k+\frac{1}{2}} - \eta_{k+\frac{1}{2}} + \beta_{k+\frac{1}{2}} + \eta_{k+\frac{1}{2}}) |\Delta^+ u_{k,\ell}| \quad (6.91a)$$

$$= \sum_k |u_{k+1,\ell} - u_{k,\ell}| \quad (6.91b)$$

$$= TV(\mathbf{u}_\ell). \quad (6.91c)$$

□

FOU for linear advection is TVD

Consider the FOU finite-volume method

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + a \frac{u_{k,\ell} - u_{k-1,\ell}}{h_x} = 0 \quad (6.92)$$

for linear advection equation

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0, \quad (6.93)$$

with $a > 0$. Rewriting the method as

$$u_{k,\ell+1} = u_{k,\ell} - a \frac{h_t}{h_x} (u_{k,\ell} - u_{k-1,\ell}), \quad (6.94)$$

we identify the coefficients in equation (6.85) as

$$\beta_{k-\frac{1}{2}} = a \frac{h_t}{h_x}, \quad (6.95a)$$

$$\eta_{k+\frac{1}{2}} = 0. \quad (6.95b)$$

Thus, for all k , we have that

$$\beta_{k+\frac{1}{2}} \geq 0, \quad (6.96a)$$

$$\eta_{k+\frac{1}{2}} \geq 0, \quad (6.96b)$$

$$\beta_{k+\frac{1}{2}} + \eta_{k+\frac{1}{2}} \leq 1, \quad (6.96c)$$

provided that the CFL stability condition ($h_t \leq h_x/a$) is satisfied. This confirms that the FOU method for linear advection is TVD when the CFL condition is satisfied. Recalling that FOU is the first-order finite-volume method in equation (6.45) with a Lax–Friedrichs flux function (see equation (6.59)) applied to linear advection, this confirms our earlier observations that the first-order method provides sufficient numerical diffusion to avoid spurious oscillations at discontinuities, unlike the second-order Lax–Wendroff method used in Figure 6.2.1, which is not TVD.

Linear schemes and Godunov's theorem

The example above shows that the FOU scheme for linear advection is TVD. However, it is only first-order accurate. Our next question is whether we can obtain second-order accurate schemes for linear advection that are TVD. If we consider so-called linear schemes (see Definition 6.14), then we find that they cannot be both second-order accurate and TVD. This leads us to consider nonlinear schemes in the next section.

Definition 6.14: Linear schemes [131, Eq. (4.6)]

A numerical scheme

$$u_{k,\ell+1} = \sum_j c_j u_{k-j,\ell} \quad (6.97)$$

applied to the conservation law in equation (6.2) with linear flux function $f(u) = au$ is called a linear scheme if all coefficients c_j are constant—i.e., they do not depend on the approximation u_ℓ at time t_ℓ . Otherwise, the scheme is called nonlinear.

Godunov's theorem (which we state without proof) establishes that linear schemes of order higher than one cannot be TVD.

Theorem 6.15: Godunov's theorem [131, Thm. 4.11]

Linear TVD schemes are at most first-order accurate.

Godunov's theorem indicates that, when pursuing TVD methods with order of accuracy higher than one, one needs to consider nonlinear schemes, with coefficients that depend on the approximation. The slope-limiting schemes presented below in Section 6.5.2, for example, are nonlinear schemes, because the reconstruction formulas in equations (6.107a) and (6.107b) introduce coefficients into the finite-volume method that depend on the approximation $\mathbf{u}_\ell$ at time t_ℓ . As a consequence, the slope-limiting methods, which aim at improving accuracy beyond first order, are not subject to the accuracy restrictions of Godunov's theorem. As we see in Section 6.5.3, conditions can be found under which they are TVD.

6.5 ■ Second-order accurate methods

Objectives

The goal of this section is to extend the first-order accurate finite-volume methods introduced in the previous section to second-order accuracy in space and time. Here, you will recall second-order integration in time using a Runge–Kutta method for ODE systems and apply linear polynomial reconstruction to increase spatial accuracy. Finally, you will use TVD principles to construct second-order accurate finite-volume methods that control oscillations using slope limiters.

6.5.1 ■ Second-order time integration

To begin, we extend our method of time integration from first order to second order. While we limit ourselves to second order for simplicity, many of the concepts extend to higher orders, as discussed in Section 9.4.4. We consider the finite-volume method in semidiscretized form from equation (6.40) for nonlinear scalar conservation law (6.2). Furthermore, we write this as a *system* of ODEs in vector form,

$$\frac{d\mathbf{u}(t)}{dt} + \mathbf{z}(\mathbf{u}(t)) = \mathbf{0}, \quad (6.98)$$

where the components of $\mathbf{u}(t)$ are the cell average approximations, $u_k(t)$, of $u(t)$ in the finite-volume cells, Ω_k , and the components of the spatial residual vector $\mathbf{z}(\mathbf{u}(t))$ are given by the relative differences of the numerical fluxes at the interfaces of the cells Ω_k :

$$z_k(\mathbf{u}(t)) = \frac{\hat{f}_{k+\frac{1}{2}}(\mathbf{u}(t)) - \hat{f}_{k-\frac{1}{2}}(\mathbf{u}(t))}{h_x}. \quad (6.99)$$

We consider discretizations of the spatial derivatives that are second-order accurate and match this with second-order time integration. Using a method-of-lines approach, any second-order ODE integrator can be employed to integrate the ODE system in equation (6.98) in time. Here, we use the following two-stage explicit Runge–Kutta method of (global) order two, known as the explicit midpoint method:

$$\mathbf{u}_{\ell+\frac{1}{2}} = \mathbf{u}_\ell - \frac{h_t}{2} \mathbf{z}(\mathbf{u}_\ell), \quad (6.100a)$$

$$\mathbf{u}_{\ell+1} = \mathbf{u}_\ell - h_t \mathbf{z}(\mathbf{u}_{\ell+\frac{1}{2}}). \quad (6.100b)$$

6.5.2 ■ Linear reconstruction and slope-limiting methods

Consider the semidiscrete finite-volume method from equation (6.40) for nonlinear scalar conservation law (6.2), with a numerical flux function written in terms of the left and right states at

cell interfaces, as in equation (6.41):

$$\frac{du_k(t)}{dt} + \frac{f^*(u_{k+\frac{1}{2}}^-(t), u_{k+\frac{1}{2}}^+(t)) - f^*(u_{k-\frac{1}{2}}^-(t), u_{k-\frac{1}{2}}^+(t))}{h_x} = 0. \quad (6.101)$$

If we take the left and right states at the cell interfaces equal to the cell averages, as in equation (6.42), we obtain a spatial finite-volume discretization that is generally first-order accurate. In this section, we seek to extend the accuracy to second order by using the cell averages to derive a piecewise linear reconstruction in each finite-volume cell, rather than a piecewise constant reconstruction as we used previously—see Figure 6.5.1. In what follows, we generally suppress the dependence on time, t_ℓ , since we are focused on evaluating $z(\mathbf{u})$, where $\mathbf{u}$ is given explicitly by the Runge–Kutta scheme in equation (6.100). Thus, below when we use a single subscript for $\mathbf{u}$, we are indicating the (spatial) finite-volume cell, where u_k is the piecewise constant approximation to $u(x, t)$ in cell Ω_k .

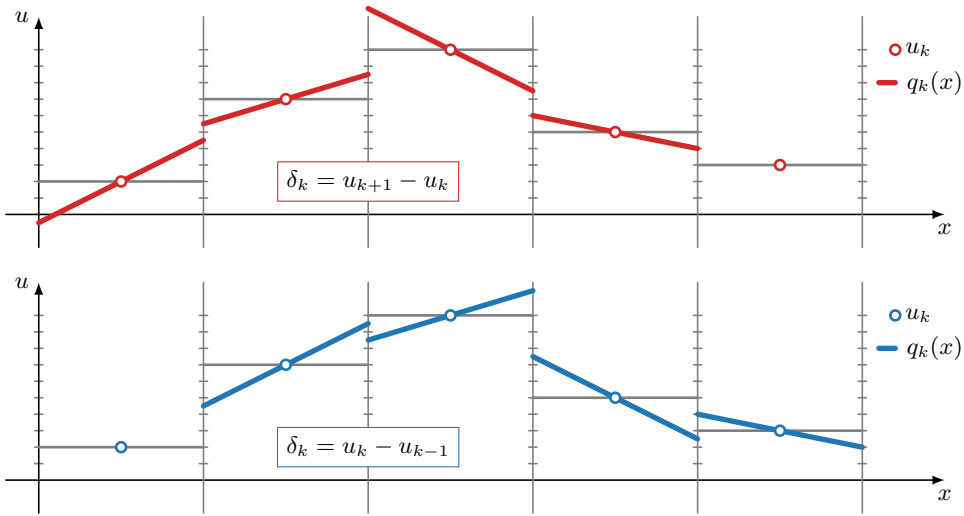


Figure 6.5.1. Linear reconstruction of the cell average. The average value is marked with circles, while the linear reconstruction from equation (6.102) is labeled with $q_k(x)$, using (bottom) $\delta_k = u_k - u_{k-1}$ or (top) $\delta_k = u_{k+1} - u_k$.

In each cell Ω_k , we consider the piecewise linear function

$$q_k(x) = u_k + \frac{\delta_k}{h_x}(x - x_k), \quad (6.102)$$

where x_k is the midpoint of cell Ω_k , and the slope of the linear approximation in cell Ω_k , δ_k/h_x , is determined by the choice of the quantity δ_k . For example, we could choose

$$\delta_k = u_k - u_{k-1} \quad (6.103a)$$

or

$$\delta_k = u_{k+1} - u_k \quad (6.103b)$$

or a convex combination of these two forms. Using such a reconstruction as $q_k(x)$, we choose

the left and right states at cell interface $x_{k+\frac{1}{2}}$ in $f^*(u_{k+\frac{1}{2}}^-, u_{k+\frac{1}{2}}^+)$ as follows:

$$u_{k+\frac{1}{2}}^- = q_k(x_{k+\frac{1}{2}}) = u_k + \frac{1}{2}\delta_k, \quad (6.104a)$$

$$u_{k+\frac{1}{2}}^+ = q_{k+1}(x_{k+\frac{1}{2}}) = u_{k+1} - \frac{1}{2}\delta_{k+1}. \quad (6.104b)$$

Numerical simulations show that fixing a choice of δ_k as in either equation (6.103a) or equation (6.103b) and using the Lax–Friedrichs flux function of equation (6.59) results in unphysical numerical oscillations at shocks, similar to the behavior observed in Figure 6.2.1. This is because the slopes in these equations lead to updates that do not satisfy TVD conditions similar to those in Theorem 6.13.

The main approach that we explore for obtaining a second-order accurate method without unphysical oscillations is the technique of *slope limiting*. We know that using the cell averages, u_k , in the Lax–Friedrichs flux function of equation (6.59) leads to a solution at shocks without spurious oscillations. Using the cell averages corresponds to choosing the reconstruction slope, δ_k/h_k , in equation (6.102) equal to zero. With a slope-limiting approach, we design an automatic procedure to select a second-order accurate slope based on equation (6.103) away from discontinuities and then *limit* the slope to a smaller change when close to a discontinuity. Considering cell Ω_k , we define the ratio, r_k , of the slopes determined by the cell average in cell Ω_k and the cell average of its left and right neighbor cells as

$$r_k = \frac{u_k - u_{k-1}}{u_{k+1} - u_k}. \quad (6.105)$$

A common approach used in practice, to avoid division by zero, is to add a small constant, ϵ , of appropriate sign to the denominator.

One well-known *slope limiter* is the so-called minmod slope limiter, defined in Definition 6.16.

Definition 6.16: Minmod slope limiter

The *minmod* slope limiter $\phi(r)$ is defined as

$$\phi(r) = \max(0, \min(r, 1)). \quad (6.106)$$

The slope limiter is used to compute slope-limited reconstruction values at interface $k + \frac{1}{2}$ by modifying equation (6.104) as follows:

$$u_{k+\frac{1}{2}}^- = u_k + \frac{1}{2}\phi(r_k)(u_{k+1} - u_k), \quad (6.107a)$$

$$u_{k+\frac{1}{2}}^+ = u_{k+1} - \frac{1}{2}\phi(r_{k+1})(u_{k+2} - u_{k+1}). \quad (6.107b)$$

If the differences, $u_k - u_{k-1}$ and $u_{k+1} - u_k$, have the same sign, then r_k in equation (6.105) is positive, and the minmod slope limiter combined with equation (6.107a) selects the smallest of the two slopes in the linear reconstruction of $u_{k+\frac{1}{2}}^-$. For smooth solutions, we expect second-order accuracy for this linear reconstruction. In the other case, where the differences, $u_k - u_{k-1}$ and $u_{k+1} - u_k$, have opposite signs, the minmod limiter selects a zero reconstruction slope and the method locally reverts to first-order accuracy. As a result, we expect this to help avoid the formation of spurious oscillations, where the solution slopes may change sign from one cell to the next. When determining the cell interface values in the numerical flux formula (6.41) as in equation (6.107), we effectively use reconstruction stencils with $p = 1$ and $q = 1$ in

equation (6.43). On these stencils, we use linear reconstructions on adjacent pairs of cells of the stencil, combined with a nonlinear slope-limiting mechanism, e.g., using equation (6.106), which selects the linear (or constant) reconstruction slope in each cell that suppresses oscillations at discontinuities.

Next, we investigate theoretical conditions on slope limiters that guarantee oscillation-free solutions at shocks and second-order accuracy away from shocks (when combined with second-order accurate time integration, as above). These conditions are used to derive and justify slope-limiter formulas like the minmod formula in equation (6.106).

6.5.3 ■ Slope-limited second-order TVD schemes

Our goal now is to construct second-order accurate approximations by employing linear reconstruction within the cells. We derive conditions on the slope-limiter functions $\phi(r)$ that yield approximations that are TVD. Contrary to the notation used in Section 6.5.2, we now concretely consider the approximations at time t_ℓ and $t_{\ell+1}$. In this section, we largely follow the original development of such schemes, due to Sweby [211], as is also presented in other places [150, 215].

Throughout the section, we consider the linear advection equation

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0 \quad (6.108)$$

with $a > 0$. The resulting methods, however, can be applied to nonlinear conservation law (6.2) by using the appropriate nonlinear numerical fluxes—e.g., the Lax–Friedrichs flux in equation (6.59)—instead of the linear upwind fluxes that we consider below.

Recalling linear reconstruction

$$q_{k,\ell}(x) = u_{k,\ell} + \frac{\delta_{k,\ell}}{h_x}(x - x_k) \quad (6.109)$$

in cell Ω_k at time t_ℓ , we consider the following general form for the slope, $\delta_{k,\ell}$, in cell Ω_k :

$$\delta_{k,\ell} = \frac{1}{2}(1 + \omega)\Delta^- u_{k,\ell} + \frac{1}{2}(1 - \omega)\Delta^+ u_{k,\ell}, \quad (6.110)$$

where ω is a constant chosen in $[-1, 1]$. This is a convex combination of the forward and backward differences

$$\Delta^+ u_{k,\ell} = u_{k+1,\ell} - u_{k,\ell}, \quad (6.111a)$$

$$\Delta^- u_{k,\ell} = u_{k,\ell} - u_{k-1,\ell}. \quad (6.111b)$$

In this case, any choice of $\omega \in [-1, 1]$ leads to second-order accuracy in the nonlimited scheme.

We fix ω and perform the steps given in Algorithm 6.1 to obtain an initial nonlimited method that can be shown to be second-order accurate in space and time. The reconstructed variables at the cell interfaces are computed in line 4 (assuming that the flux at the leftmost boundary of the domain is provided). We rewrite these values as

$$u_{k+\frac{1}{2}}^- = u_{k,\ell} - \frac{h_t}{2} \frac{a}{h_x} \delta_{k,\ell} + \frac{\delta_{k,\ell}}{2} \quad (6.112a)$$

$$= u_{k,\ell} + \frac{1 - \frac{ah_t}{h_x}}{2} \delta_{k,\ell} \quad (6.112b)$$

$$= u_{k,\ell} + \gamma \delta_{k,\ell}, \quad (6.112c)$$

where

$$\gamma = \frac{1 - \lambda}{2} \quad (6.113)$$

and λ is defined as the CFL number,

$$\lambda = \frac{ah_t}{h_x}. \quad (6.114)$$

For stability, selecting $\lambda \in [0, 1]$ results in $\gamma \in [0, 1/2]$. Then, the upwind linear flux at cell interface $x_{k+\frac{1}{2}}$ using the linear reconstruction is given by

$$f_{k+\frac{1}{2},\ell}^* = a u_{k,\ell} + a \gamma \delta_{k,\ell}. \quad (6.115)$$

Algorithm 6.1: Nonlimited linear reconstruction scheme for linear advection

Input : u_ℓ , grid function at time t_ℓ

$u_{\frac{1}{2}}$, flux value at the boundary

ω , weight parameter in $[-1, 1]$

a , advection speed

h_t, h_x , grid spacing

$\{\Omega_k\}_{k=1}^{n_{\text{cells}}}$, list of finite-volume cells

Output : $u_{\ell+1}$, grid function at time $t_{\ell+1}$

```

1 for each  $k = 1, \dots, n_{\text{cells}}$ 
2    $\delta_{k,\ell} = \frac{1}{2}(1 + \omega)\Delta^- u_{k,\ell} + \frac{1}{2}(1 - \omega)\Delta^+ u_{k,\ell}$  {compute slope}
3    $u_{k,\ell+\frac{1}{2}} = u_{k,\ell} - \frac{h_t}{2}a \frac{\delta_{k,\ell}}{h_x}$  {compute cell value at intermediate time  $t_{\ell+\frac{1}{2}}$ }
4    $u_{k+\frac{1}{2}}^- = u_{k,\ell+\frac{1}{2}} + \frac{\delta_{k,\ell}}{2}$  {compute linear reconstruction at right cell interface of  $\Omega_k$ }
5    $u_{k,\ell+1} = u_{k,\ell} - \frac{h_t}{h_x} \left( a u_{k+\frac{1}{2}}^- - a u_{k-\frac{1}{2}}^- \right)$  {use upwind flux  $f^* = au$ }

```

It is easy to verify that, with $\omega = -1$ in equation (6.110), we recover the Lax–Wendroff flux (see equation (6.66)) in equation (6.115) and the Lax–Wendroff method from Algorithm 6.1, which we know from Figure 6.2.1 can result in spurious oscillations. Other choices of ω lead to other well-known second-order accurate methods for linear advection. For example, $\omega = 0$ leads to Fromm’s scheme, and $\omega = 1$ leads to the Beam–Warming scheme:

Lax–Wendroff: $\delta_{k,\ell} = u_{k+1,\ell} - u_{k,\ell},$

Fromm: $\delta_{k,\ell} = \frac{1}{2}(u_{k+1,\ell} - u_{k-1,\ell}),$

Beam–Warming: $\delta_{k,\ell} = u_{k,\ell} - u_{k-1,\ell}.$

All of these methods are linear and second-order accurate, and, by Godunov’s theorem, Theorem 6.15, they cannot be TVD.

Instead, consider the case of $\omega = 1$ and the Lax–Wendroff flux expressed as in equation (6.115) after linear reconstruction. This gives

$$f_{k+\frac{1}{2},\ell}^* = a u_{k,\ell} + a \gamma \Delta^+ u_{k,\ell}, \quad (6.116)$$

since $\delta_{k,\ell}$ equals the forward difference $\Delta^+ u_{k,\ell}$ when $\omega = -1$. The reconstruction slope is then limited in this flux by multiplication with a limiting factor $\phi_{k+\frac{1}{2},\ell}$:

$$f_{k+\frac{1}{2},\ell}^* = a u_{k,\ell} + a \gamma \phi_{k+\frac{1}{2},\ell} \Delta^+ u_{k,\ell}. \quad (6.117)$$

Next, we establish conditions on the limiting factors, $\phi_{k+\frac{1}{2},\ell}$, that result in TVD schemes.

Continuing, and still assuming that $a > 0$, we use the numerical flux in equation (6.117) to write the update formula in line 5 of Algorithm 6.1 in the form of the update formula in equation (6.87) of Theorem 6.13. This results in

$$u_{k,\ell+1} = u_{k,\ell} - \frac{h_t}{h_x} (f_{k+\frac{1}{2},\ell}^* - f_{k-\frac{1}{2},\ell}^*) \quad (6.118a)$$

$$= u_{k,\ell} - \frac{h_t}{h_x} \left(a u_{k,\ell} + a \gamma \phi_{k+\frac{1}{2},\ell} \Delta^+ u_{k,\ell} - a u_{k-1,\ell} - a \gamma \phi_{k-\frac{1}{2},\ell} \Delta^+ u_{k-1,\ell} \right) \quad (6.118b)$$

$$= u_{k,\ell} - \lambda (1 - \gamma \phi_{k-\frac{1}{2},\ell}) \Delta^- u_{k,\ell} + (-\lambda) \gamma \phi_{k+\frac{1}{2},\ell} \Delta^+ u_{k,\ell}. \quad (6.118c)$$

However, the last expression does not directly identify useful TVD conditions, because $\eta_{k+\frac{1}{2}} = (-\lambda) \gamma \phi_{k+\frac{1}{2},\ell}$ is negative for $\phi_{k+\frac{1}{2},\ell}$ close to one, the case of no limiting. Instead, we rewrite equation (6.118c) in the following form to arrive at TVD conditions that can be evaluated more readily using Theorem 6.13:

$$u_{k,\ell+1} = u_{k,\ell} - \lambda \left((1 - \gamma \phi_{k-\frac{1}{2},\ell}) + \gamma \frac{\phi_{k+\frac{1}{2},\ell}}{r_{k,\ell}} \right) \Delta^- u_{k,\ell}, \quad (6.119)$$

where

$$r_{k,\ell} = \frac{\Delta^- u_{k,\ell}}{\Delta^+ u_{k,\ell}}. \quad (6.120)$$

Comparing with equation (6.87) in Theorem 6.13, we have

$$\beta_{k+\frac{1}{2}} = \lambda \left(1 + \frac{1-\lambda}{2} \left(\frac{\phi_{k+\frac{1}{2},\ell}}{r_{k,\ell}} - \phi_{k-\frac{1}{2},\ell} \right) \right), \quad (6.121a)$$

$$\eta_{k+\frac{1}{2}} = 0. \quad (6.121b)$$

Thus, Theorem 6.13 requires

$$0 \leq \beta_{k+\frac{1}{2}} \leq 1 \quad \forall k. \quad (6.122)$$

We know from Godunov's theorem, Theorem 6.15, that the $\phi_{k+\frac{1}{2},\ell}$ need to depend on the approximation u_ℓ at time t_ℓ if we want to obtain a TVD scheme. To this end, we consider the slope limiters $\phi_{k+\frac{1}{2},\ell}$ to be a function of $r_{k,\ell}$, the ratio of the left and right slopes as defined in equation (6.120), leading to

$$\phi_{k+\frac{1}{2},\ell} = \phi(r_{k,\ell}), \quad (6.123a)$$

$$\phi_{k-\frac{1}{2},\ell} = \phi(r_{k-1,\ell}). \quad (6.123b)$$

Our goal is, then, to find conditions on the function $\phi(r)$ so that equation (6.122) is satisfied, yielding a scheme that is TVD.

We first consider the case where $r \leq 0$ in $\phi(r)$. For cell Ω_k at time t_ℓ , this corresponds to when the two adjacent slopes defined by $\Delta^- u_{k,\ell}$ and $\Delta^+ u_{k,\ell}$ have opposite signs. Thus, we select $\phi(r) = 0$. As we have observed, this selects a zero reconstruction slope and the

method locally reverts to first-order accuracy, which we can intuitively see will help to avoid the formation of spurious oscillations.

The choice of $\phi(r) = 0$ for $r \leq 0$ can also be motivated by our discussion of the reconstruct-evolve-average approach in Godunov's first-order method in Section 6.3.3. We can interpret the linear reconstruction scheme as a reconstruct-evolve-average (REA) approach. If the evolve step evolves the reconstructed solution exactly, then it cannot increase total variation. In addition, it can be shown that averaging also cannot increase total variation. When $r \leq 0$, reconstruction—e.g., with slopes defined by $\Delta^- u_{k,\ell}$ or $\Delta^+ u_{k,\ell}$ —increases total variation, leading to an REA procedure that is total-variation increasing. For this reason, we choose $\phi(r) = 0$ when $r \leq 0$.

Second, for $r \geq 0$, it is natural to choose $\phi(r) \geq 0$, because the limiter should retain the sign of the slope when both adjacent differences, $\Delta^- u_{k,\ell}$ and $\Delta^+ u_{k,\ell}$, have the same sign. Defining

$$\psi_{k,\ell} := \frac{\phi(r_{k,\ell})}{r_{k,\ell}} - \phi(r_{k-1,\ell}), \quad (6.124)$$

the condition on $\beta_{k+\frac{1}{2}}$ defined in equation (6.121a) requires that

$$0 \leq \beta_{k+\frac{1}{2}} \leq 1, \quad (6.125)$$

with

$$\beta_{k+\frac{1}{2}} = \lambda \left(1 + \frac{1-\lambda}{2} \psi_{k,\ell} \right). \quad (6.126)$$

It is not difficult to see that the condition expressed in equation (6.125) requires that $\psi_{k,\ell} \in [-2, 2]$, because $\beta_{k+\frac{1}{2}}$ is a parabolic function in λ , and requiring $\psi_{k,\ell} \in [-2, 2]$ guarantees that the parabola's extremum achieved at $\lambda_{ext} = 1/\psi_{k,\ell} + 1/2$ occurs outside $\lambda \in (0, 1)$. As a result, we require that

$$-2 \leq \frac{\phi(r_1)}{r_1} - \phi(r_2) \leq 2 \quad (6.127)$$

for any $r_1, r_2 \geq 0$, which can be met by requiring $\phi(r)$ to satisfy

$$0 \leq \phi(r) \leq 2, \quad (6.128a)$$

$$0 \leq \frac{\phi(r)}{r} \leq 2. \quad (6.128b)$$

This defines the *TVD region* for limiter functions $\phi(r)$, as shown in Figure 6.5.2. Note that the TVD condition in equation (6.122) is also satisfied when one or both of $r_{k,\ell}$ and $r_{k-1,\ell}$ are negative, relying on conditions (6.128) when $r \geq 0$ and on $\phi(r) = 0$ when $r \leq 0$.

Next, it can be shown that second-order accuracy of the reconstruction-based scheme for smooth flow requires that $\phi(1) = 1$. Intuitively, this reflects the requirement that, when two adjacent slopes are equal, slope should be used in the reconstruction to obtain second-order accuracy. Indeed, Figure 6.5.3 illustrates that each of the second-order linear schemes of Lax and Wendroff ($\phi(r) = 1$), Beam and Warming ($\phi(r) = r$), and Fromm ($\phi(r) = (r+1)/2$) satisfy $\phi(1) = 1$; however, they are not TVD because their implied limiter functions do not lie within the TVD region for $r \geq 0$.

Furthermore, it has been observed that the best results are obtained when reconstructed schemes use limiter functions that lie between the Lax–Wendroff and Beam–Warming curves in the diagram of Figure 6.5.3. This condition, superimposed on the TVD condition, determines *Sweby's second-order TVD region*, as indicated in Figure 6.5.3. Three well-known slope limiters

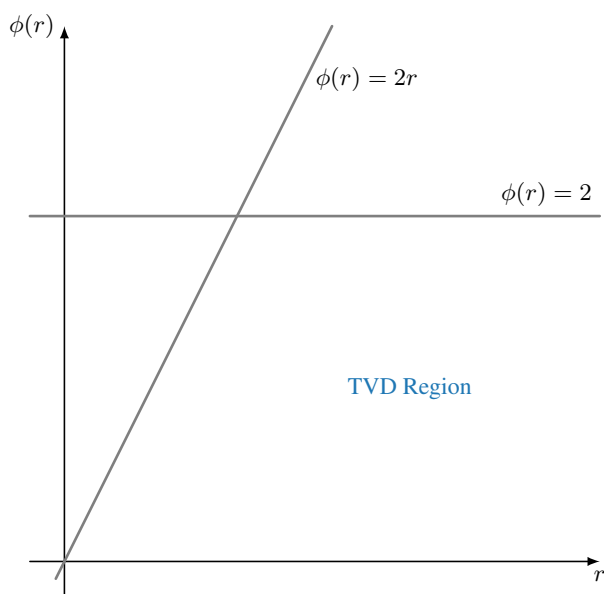


Figure 6.5.2. TVD region for limiter function $\phi(r)$.

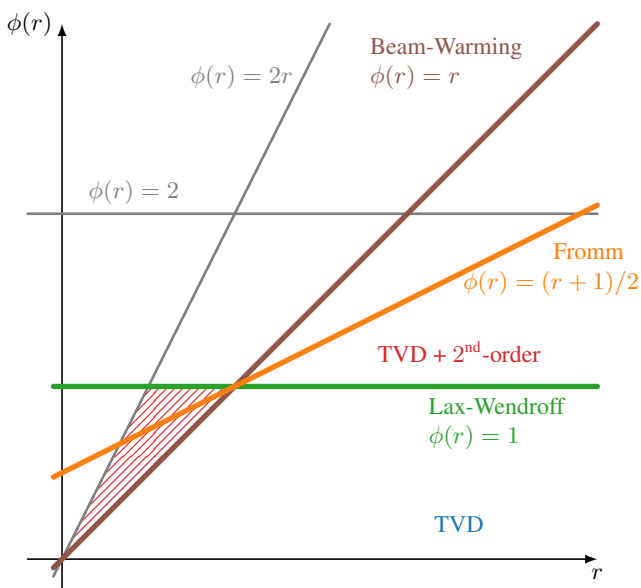


Figure 6.5.3. TVD second-order region.

that lie in this second-order TVD region are illustrated in Figure 6.5.4:

Minmod: $\phi(r) = \max(0, \min(r, 1)),$ (6.129a)

Superbee: $\phi(r) = \max(0, \min(2r, 1), \min(r, 2)),$ (6.129b)

Van Leer: $\phi(r) = \frac{r + |r|}{1 + |r|}.$ (6.129c)

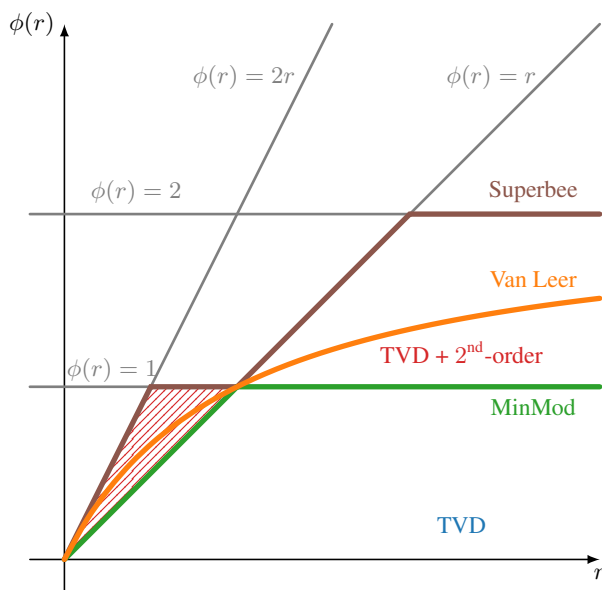


Figure 6.5.4. Examples of TVD limiters.

6.5.4 ■ Application of a slope-limited second-order TVD scheme

The TVD slope limiters in the previous section were derived and justified for the linear advection equation but can also be applied successfully to nonlinear conservation laws. Consider Burgers' equation (6.7) with initial data as used in Section 6.3.4, with initial conditions in equation (6.73) leading to a rarefaction wave and those in equation (6.74) leading to a shock. Figure 6.5.5 compares the second-order method derived in this section (using the two-stage explicit Runge–Kutta method, minmod limiter, and local Lax–Friedrichs flux function) and the first-order local Lax–Friedrichs method discussed earlier in Section 6.3.2 for these two initial conditions. We note that, as expected, no spurious oscillations arise in either case, particularly near the shock. We also note that the second-order solution is clearly closer to the exact solution than the first-order solution. The first-order result shows more diffusion at the shock, but the difference in

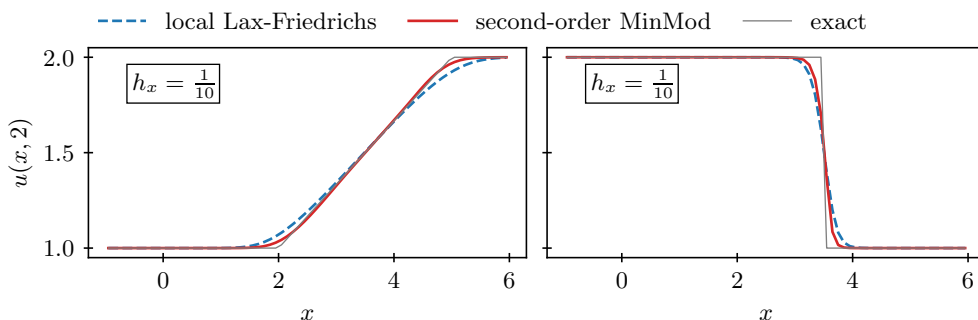


Figure 6.5.5. Comparison of second-order TVD scheme with minmod limiter compared to first-order result, both using the Lax–Friedrichs numerical flux function for initial conditions in (left) equation (6.73) and (right) equation (6.74). Grid spacing $h_x = 1/10$ is used, and the timestep is chosen as $0.5 h_x / \max_j |f'(u_j)|$.

accuracy there is not large, consistent with the fact that the limited scheme drops to first-order accuracy near the shock. More substantial differences in accuracy between the two schemes would occur for problems in which the solution contains high-frequency content away from the shock, where the limited method is second-order accurate, but this difference is not revealed in this simple problem.

6.6 ■ Some further thoughts

Nonlinear hyperbolic conservation laws have a rich mathematical structure, even in the simple scalar form in one spatial dimension given by

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0. \quad (6.130)$$

Standard textbooks on numerical methods for hyperbolic conservation laws such as [151] and [131] discuss a broad set of topics and applications for equation (6.130) that go substantially beyond what we have covered in this chapter.

Traffic flow

Much of our understanding of hyperbolic conservation laws arose historically from studying problems in compressible fluid dynamics, but there are other instructive areas of application, for example, the modeling of traffic flow. Let $\rho(x, t)$ denote the number of cars per car length at location x and time t along a one-lane highway (such that $\rho \in [0, 1]$), and assume that, given the car density $\rho(x, t)$, the velocity of a car can be modeled as a function, $u(\rho)$, that only depends on ρ . Then, the following conservation law provides a simple model for the evolution of the car density, $\rho(x, t)$, as a function of space and time [151]:

$$\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x}(u(\rho) \rho) = 0. \quad (6.131)$$

Here, the flux of cars per unit time at any position x and time t is given by $f(\rho) = u(\rho) \rho$.

A simple model for the velocity function $u(\rho)$ is given by

$$u(\rho) = (1 - \rho) u_{\max}, \quad (6.132)$$

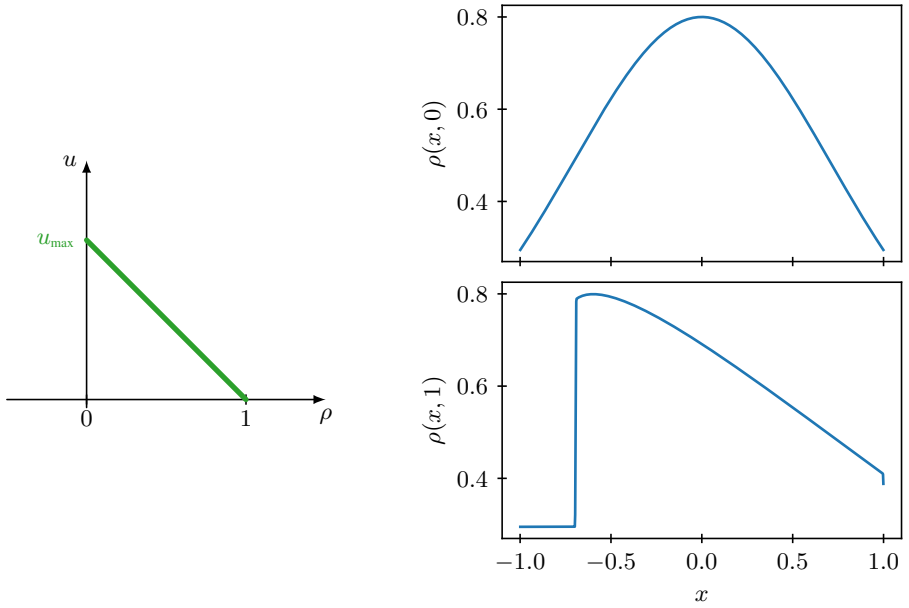
where $u = u_{\max}$ when there is no traffic and $u = 0$ when traffic is bumper-to-bumper; see Figure 6.6.1a. The flux of cars is then given by

$$f(\rho) = \rho(1 - \rho) u_{\max}. \quad (6.133)$$

The flux $f(\rho)$ is maximal when $\rho = 1/2$. As ρ increases, the flux decreases because the velocity decreases, and as ρ decreases, the flux decreases because there are fewer cars. The slope of the characteristic curves is given by

$$f'(\rho) = (1 - 2\rho) u_{\max}. \quad (6.134)$$

Consider an initial car density at $t = 0$ as sketched in Figure 6.6.1b. Where $\rho'(x) < 0$, cars move into lighter traffic and speed up. Since ρ decreases, the slope $f'(\rho)$ in (6.134) of the characteristic curves increases as a function of x , and the car density rarefies. Conversely, when $\rho'(x) > 0$, cars move into denser traffic and slow down. The slope of the characteristic curves

a. Traffic velocity u as a function of car density ρ .b. Traffic density, ρ , at $t = 0$ and $t = 1$ with $u_{\max} = 1$.**Figure 6.6.1.** Traffic flow model.

steepens until a shock occurs. In this simple model, a shock happens when cars suddenly reach the head of a traffic jam. After the shock, the traffic jam slowly resolves into a rarefaction wave.

Entropy functions

Another topic we have not covered is a more formal mathematical way to determine unique entropy weak solutions by using the theoretical construct of convex entropy functions. Given a conservation law with flux function, $f(u)$, we can choose a convex entropy function, $\eta(u)$, i.e., a function $\eta(u)$ with $\eta''(u) > 0$ for all u . We first seek an entropy flux function, $\psi(u)$, that is compatible with the entropy function, $\eta(u)$, in the sense that any smooth solution of conservation law (6.130) also satisfies the PDE

$$\frac{\partial \eta(u)}{\partial t} + \frac{\partial \psi(u)}{\partial x} = 0. \quad (6.135)$$

By considering the quasi-linear forms of equation (6.130) and equation (6.135), it can be seen that $\psi(u)$ must satisfy the compatibility condition $\psi' = \eta' f'$. For example, for Burgers' equation with $f(u) = u^2/2$, we choose $\eta(u) = u^2$ (satisfying $\eta'' = 1 > 0$), and then the compatibility condition gives $\psi(u) = 2u^3/3$.

We know that weak solutions of the conservation law satisfy equation (6.130) in the integral sense. However, weak solutions do not satisfy equation (6.135) in the integral sense, but it can be shown, using the fact that $\eta''(u) > 0$, that any vanishing-viscosity weak solution satisfies the following inequality:

$$\frac{\partial \eta(u)}{\partial t} + \frac{\partial \psi(u)}{\partial x} \leq 0. \quad (6.136)$$

For a given problem with a convex flux function, all weak solutions satisfy conservation law (6.130) (in the integral sense), but only the unique physical weak solution also satisfies the entropy inequality in equation (6.136) (again in the integral sense). Along a characteristic curve,

since $\psi' = \eta' f'$, we can also write

$$\frac{d}{dt}\eta(x(t), t) = \frac{\partial\eta(u)}{\partial t} + f'(u)\frac{\partial\eta(u)}{\partial x} \leq 0, \quad (6.137)$$

so the entropy $\eta(u)$ cannot increase over time. This is the second law of thermodynamics, except that the sign of the mathematical entropy is, by convention, chosen opposite to the sign of entropy in physics. The physical entropy can be thought of as corresponding to the amount of disorder in a system, and this can only go up over time.

For the case of shocks, from equation (6.136) one can derive the shock inequality

$$s (\eta(u^+) - \eta(u^-)) \geq \psi(u^+) - \psi(u^-), \quad (6.138)$$

essentially paralleling the proof of Theorem 6.4. For example, for Burgers' equation we have $s = (u^- + u^+)/2$, and using the entropy function and flux given above, equation (6.138) reduces to

$$\frac{1}{6} (u^- - u^+)^3 \geq 0. \quad (6.139)$$

That is, $u^- \geq u^+$, which is the Lax entropy condition from Definition 6.7.

In a similar manner, the concepts of entropy function and entropy flux can be applied in the discrete setting to show that certain finite-volume methods converge to the unique entropy weak solution [151, 131]. These concepts also extend to nonlinear conservation law systems. For example, for the Euler equations of compressible gas dynamics that will be introduced in Chapter 9, the physical entropy of the gas can be chosen as the entropy function in the mathematical formulation of an entropy inequality as in equation (6.136).

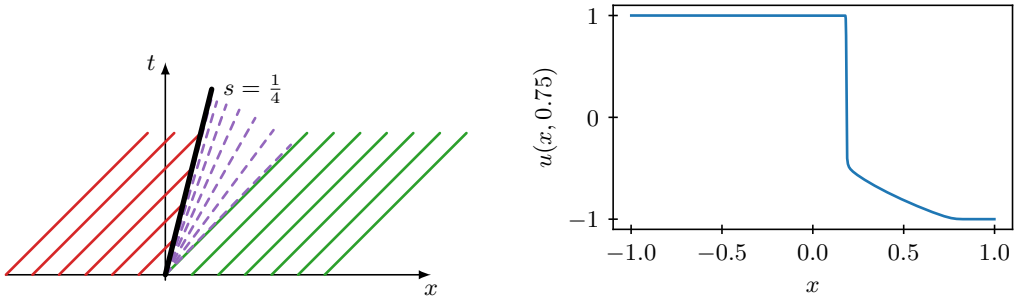
Nonconvex flux functions

Finally, most of our discussion in this chapter has focused on the case where the flux function, $f(u)$, in conservation law (6.130) is convex or concave (see Definition 6.6). However, some physical problems are modeled by nonconvex flux functions. Consider, for example, a conservation law with nonconvex flux function

$$f(u) = u^3/3. \quad (6.140)$$

In this case, it is no longer true that the solution of a Riemann problem is either a shock or a rarefaction. Consider a Riemann problem with $u^- = 1$ and $u^+ = -1$. The Rankine–Hugoniot relation gives shock speed, $s = 1/3$, but this shock is not admissible under the Lax entropy condition, with $f'(u^-) = 1$ but also $f'(u^+) = 1$, so the characteristics on the right would leave the shock. Therefore, the solution to this Riemann problem cannot be simply a shock or a rarefaction. Instead, the unique vanishing-viscosity solution to this problem is a so-called *compound shock*: a shock followed by an attached rarefaction, with the characteristic where the rarefaction is attached to the shock being parallel to the shock (see Figure 6.6.2). The intermediate state where the rarefaction is attached to the shock is determined by the conditions that the shock needs to have to satisfy the Rankine–Hugoniot relations and that the Lax condition is satisfied at this state in a borderline manner: the characteristic is parallel to the shock.

Physical models where nonconvex flux functions and compound shocks arise include, for example, the Buckley–Leverett equation for modeling two-phase flows in porous media with applications to oil recovery [151], compound shocks in sedimentation processes [35], and the equations of magnetohydrodynamics (MHD) for compressible plasmas [79]. The MHD equations are a hyperbolic system with eight equations in eight unknowns, and are discussed briefly at the end of Chapter 9, which extends the methods presented in this chapter to *systems* of hyperbolic conservation laws and to multiple spatial dimensions.



a. Characteristic curves for Riemann problem with $u^- = 1$ and $u^+ = -1$.

b. Compound shock at $t = 0.75$.

Figure 6.6.2. Example of a compound shock for a conservation law with $f(u) = u^3/3$.

6.7 ■ Exercises

1. Consider the nonlinear hyperbolic conservation law

$$\frac{\partial u}{\partial t} + \frac{\partial (u^4)}{\partial x} = 0.$$

- (a) Let curve $x(t)$ be a characteristic curve of the PDE. Find an expression for $dx(t)/dt$ as a function of u for this characteristic curve. Are the characteristic curves straight lines? Explain why. What is the slope of the characteristic curve that passes through a point in the x - t plane where $u = 3$?
- (b) Consider a discontinuity with left and right states $u^- = 3$ and $u^+ = 1$. What is the shock speed? Verify that this shock satisfies the Lax entropy condition.

2. Consider Burgers' equation,

$$\frac{\partial u}{\partial t} + \frac{\partial (u^2/2)}{\partial x} = 0,$$

with initial conditions

$$u(x, 0) = \begin{cases} 0 & \text{if } x \leq 0, \\ x & \text{if } 0 \leq x \leq 1, \\ 0 & \text{if } x > 1. \end{cases}$$

- (a) Draw the characteristic curves in the x - t plane, and explain why these initial conditions might result in a solution with a shock that has a curved shock front. *Hint:* The left and right states, u^- and u^+ , may depend on the time t .
- (b) What are the shock speed and position at $t = 0$?
- (c) Use the Rankine–Hugoniot relation to solve for $\hat{x}(t)$, the location of the shock front as a function of time.

3. Consider the following two PDEs on the domain $x \in \mathbb{R}, t > 0$:

$$(1) \quad u_t + \left(\frac{u^2}{2} \right)_x = 0,$$

$$(2) \quad \left(\frac{u^2}{2} \right)_t + \left(\frac{u^3}{3} \right)_x = 0.$$

- (a) Show that $u(x, t) = x/(t+a)$ is a solution of both (1) and (2) for any constant $a > 0$.
- (b) Consider a Riemann problem with initial conditions $u^- = 2$ and $u^+ = 1$ at $t = 0$. With these initial conditions, what is the solution for $t \geq 0$ for equation (1)? What is the solution for equation (2)?
- (c) It appears that (1) and (2) have the same smooth solutions but different discontinuous solutions. Resolve this apparent contradiction by explaining why the smooth solutions are the same, but the discontinuous solutions are not the same. *Hint:* How does equation (1) relate to equation (2)?
4. Consider a numerical conservation law on a periodic domain given by

$$u_t + (f(u))_x = 0.$$

Suppose a numerical flux is given by $f^*(u, v)$ and is differentiable in both u and v . In addition, assume that f^* is consistent: $f^*(u, u) = f(u)$. Show that the finite-volume method

$$\frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + \frac{f^*(u_{k,\ell}, u_{k+1,\ell}) - f^*(u_{k-1,\ell}, u_{k,\ell})}{h_x} = 0$$

is first-order accurate (consistent) in x and t , assuming a solution $u(x, t)$ with sufficient smoothness.

5. Consider the (global) Lax–Friedrichs numerical flux given by

$$f^*(u_{k,\ell}, u_{k+1,\ell}) = \frac{f(u_{k,\ell}) + f(u_{k+1,\ell})}{2} - \frac{h_t}{2h_x}(u_{k+1,\ell} - u_{k,\ell})$$

for the conservation law $u_t + (f(u))_x = 0$. Show that using the Lax–Friedrichs flux in a low-order explicit finite-volume scheme can be viewed as a finite-difference discretization of the conservation law with diffusion:

$$u_t + (f(u))_x = cu_{xx}.$$

Why is the scheme consistent with the conservation law? *Hint:* Keep h_t/h_x fixed as $h_t \rightarrow 0$ and $h_x \rightarrow 0$.

6. Suppose we have a spatial finite-volume discretization scheme for the linear advection equation $u_t + au_x = 0$ with $a > 0$ and periodic boundary conditions, written as $\mathbf{u}_t + \mathbf{z}(\mathbf{u}) = 0$ for $\mathbf{u}$ describing the (piecewise constant) value of the finite-volume approximation on each of n_{cells} intervals. Suppose also that the explicit Euler time discretization of this scheme is TVD (that is, $TV(\mathbf{u} - h_t \mathbf{z}(\mathbf{u})) \leq TV(\mathbf{u})$ for any $\mathbf{u}$) for all $h_t \leq h_t^*$ for some value h_t^* .

- (a) With $TV(\mathbf{u}) = \sum_{k=1}^{n_{\text{cells}}} |u_k - u_{k-1}|$ (periodically extending $u_0 = u_{n_{\text{cells}}}$), show that, for any $\mathbf{u}$ and $\mathbf{v}$, $TV(\mathbf{u} + \mathbf{v}) \leq TV(\mathbf{u}) + TV(\mathbf{v})$.
- (b) Show that the explicit trapezoid rule, which computes $\mathbf{u}_{\ell+1}$ as

$$\begin{aligned} \mathbf{U} &= \mathbf{u}_\ell - h_t \mathbf{z}(\mathbf{u}_\ell), \\ \mathbf{u}_{\ell+1} &= \mathbf{u}_\ell - \frac{h_t}{2} (\mathbf{z}(\mathbf{u}_\ell) + \mathbf{z}(\mathbf{U})), \end{aligned}$$

applied to the same spatial semidiscretization, is also TVD if $h_t \leq h_t^*$.

(c) Show that the two-stage TVD Runge–Kutta scheme, which computes $\mathbf{u}_{\ell+1}$ as

$$\begin{aligned}\mathbf{U} &= \mathbf{u}_\ell - h_t \mathbf{z}(\mathbf{u}_\ell), \\ \mathbf{u}_{\ell+1} &= \frac{1}{2}(\mathbf{u}_\ell + \mathbf{U}) - \frac{h_t}{2} \mathbf{z}(\mathbf{U}),\end{aligned}$$

applied to the same spatial semidiscretization, is also TVD if $h_t \leq h_t^*$.

7. Consider the linear advection equation

$$u_t + au_x = 0,$$

with $a > 0$. Using the REA algorithm (see Algorithm 6.1), consider the Fromm slope

$$\delta_{k,\ell} = \frac{1}{2}(u_{k+1,\ell} - u_{k-1,\ell}).$$

(a) Derive the *difference* form of the resulting second-order scheme, and write it as the upwind scheme plus some extra terms:

$$u_{k,\ell+1} = u_{k,\ell} - \frac{ah_t}{h_x}(u_{k,\ell} - u_{k-1,\ell}) + \text{extra terms}.$$

The method is a four-point stencil, in contrast to the two-point stencil for the upwind method.

(b) Derive the *flux* form of the scheme, written in conservation form as

$$u_{k,\ell+1} = u_{k,\ell} - \frac{h_t}{h_x} (f^*(u_{k,\ell}, u_{k+1,\ell}) - f^*(u_{k-1,\ell}, u_{k,\ell})).$$

To do so, derive the flux $f^*(u_{k,\ell}, u_{k+1,\ell})$. Notably, the Fromm flux is an average of the Lax–Wendroff and Beam–Warming fluxes.

Part III

Intermediate topics

Chapter 7

Linear systems of equations

Overview

As seen in the previous chapters, linear (and nonlinear) systems of equations play an important role in the numerical solution of PDEs. In many problems, the discretization process yields a large set of discrete equations to be solved, resulting in sometimes globally coupled but sparse linear systems—e.g., in the case of elliptic PDEs. In this chapter, we explore a range of numerical techniques for their solution.

As an example, consider a finite-difference approximation of a two-dimensional Poisson problem, $-\nabla \cdot \nabla u = f$ on $[0, 1]^2$. Using an $n_x \times n_y$ regular grid with $n_x = n_y$ and mesh size $h = \frac{1}{n_x}$ results in a set of discrete equations of the form

$$\frac{1}{h^2} (4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}) = f_{i,j} \quad (7.1)$$

for $0 < i, j < n_x$. For example, with $n_x = 4$ and eliminated Dirichlet boundary conditions, the system for this set of equations is

$$\frac{1}{h^2} \begin{bmatrix} 4 & -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 4 & 0 & 0 & -1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 4 & -1 & 0 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 4 & -1 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 & -1 & 4 & 0 & 0 & -1 \\ 0 & 0 & 0 & -1 & 0 & 0 & 4 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} u_{1,1} \\ u_{2,1} \\ u_{3,1} \\ u_{1,2} \\ u_{2,2} \\ u_{3,2} \\ u_{1,3} \\ u_{2,3} \\ u_{3,3} \end{bmatrix} = \begin{bmatrix} f_{1,1} \\ f_{2,1} \\ f_{3,1} \\ f_{1,2} \\ f_{2,2} \\ f_{3,2} \\ f_{1,3} \\ f_{2,3} \\ f_{3,3} \end{bmatrix}, \quad (7.2)$$

where the degrees of freedom are ordered first in the x -direction. In this chapter, we consider systems more generally of the form

$$A\mathbf{u} = \mathbf{f}, \quad (7.3)$$

where A and $\mathbf{f}$ are formed through the discretization, as in the example above, and where $\mathbf{u}$ represents the discrete solution. In the example above, the entries in $\mathbf{u}$ are the values $\{u_{i,j}\}$ for the interior grid points, $1 \leq i, j \leq n_x - 1$. The matrix A is $n \times n$ for $n = (n_x - 1)^2$.

Objectives

In this chapter, we illustrate the main concepts of direct and iterative methods for the solution of linear systems of equations. You will analyze a breadth of solution methods, distinguishing their individual advantages and disadvantages. While there are many specialized techniques used to achieve the highest efficiency possible for the solution of the linear systems that arise from specific discretization approaches, this chapter focuses on broader classes of methods from which these techniques are derived. In particular, you will be able to

- compare the differences in the direct solution of a linear system with dense and sparse coefficient matrices, as well as with methods based on iterative schemes;
- predict the resulting convergence characteristics of various solution methods based on properties of the linear system;
- analyze polynomial and Krylov methods leading to common approaches such as the Chebyshev iteration, generalized minimum residual (GMRES), minimum residual (MINRES), and conjugate gradient (CG); and
- develop preconditioners for Krylov methods and investigate their convergence properties.

Chapter Outline

Section 7.1 Gaussian elimination presents a review of the standard methodology for directly solving a linear system of equations. We investigate the associated costs for both dense and sparse systems, summarizing the fundamental limitations of sparse direct methods.

Section 7.2 Reordering algorithms for Gaussian elimination modifies the standard elimination approach to take advantage of sparseness in the linear system. Using principles of graph theory, we show how reordering the linear system can reduce the cost of factorization. Standard examples of reordering methods, such as the Cuthill–McKee algorithm, the basic minimum-degree algorithm, and nested dissection, are presented.

Section 7.3 Iterative methods presents the classical theory of matrix splittings, which leads to the well-known Jacobi, Gauss–Seidel, and successive overrelaxation (SOR) iterative methods. We consider sparse linear systems with n degrees of freedom resulting from discretizations of PDEs and ask whether it is possible to solve for the n unknowns in $\mathcal{O}(n)$ cost.

Section 7.4 Convergence theory for stationary iterative methods analyzes and presents convergence theory for standard stationary iterative methods such as Jacobi, Gauss–Seidel, and SOR. We investigate various properties of matrices, including the notion of an M-matrix, and see how they affect the convergence behavior of such linear systems.

Section 7.5 Optimizing convergence of SOR presents more details on the convergence theory for the SOR method, focusing on a case where a sharp convergence bound can be given.

Section 7.6 Incomplete factorizations introduces a splitting method which aims to balance cost and accuracy to find an iteration that converges quickly. To do so, we consider sparse approximations of the LU factorization of a linear system, as well as threshold-based incomplete factorization techniques.

Section 7.7 Polynomial methods and the Chebyshev iteration presents the broad class of polynomial iterative methods, which includes many standard techniques for the solution

of linear systems. We focus on writing the approach as a “minimax” problem and introduce the idea of the Chebyshev iteration, analyzing convergence properties for both symmetric and nonsymmetric linear systems.

Section 7.8 Krylov methods: The Arnoldi algorithm and GMRES introduces the idea of a *Krylov space*. We derive the Arnoldi algorithm to find an orthogonal basis for the Krylov space, leading to the most general Krylov method, GMRES. We also discuss the method’s efficient implementation.

Section 7.9 Krylov methods: The Lanczos algorithm, MINRES, and CG explores what can happen when we make additional assumptions on the linear system to which we apply an iterative method, such as symmetry of the linear system. Specifically, we discuss how the Arnoldi algorithm is modified to the Lanczos algorithm when the system is symmetric, leading to the MINRES method. We also derive and analyze the CG method by choosing iterates whose residuals are orthogonal to the Krylov space.

Section 7.10 Preconditioning outlines the most common framework for accelerating convergence of a Krylov method by exchanging the linear system to be solved for one that has the same solution but offers faster convergence. Here, we present the notion of both “left” and “right” preconditioning and discuss adaptations such as the flexible GMRES (FGMRES) algorithm.

Section 7.11 Generalized methods for nonsymmetric matrices discusses extensions of Krylov methods designed for symmetric matrices, such as CG and MINRES, to nonsymmetric linear systems. We introduce the Faber–Manteuffel theorem and develop the biconjugate gradient (BiCG) algorithm and BiCGSTAB to address stability.

7.1 ■ Gaussian elimination

Objectives

Central to the approximation of many PDEs is a linear system of the form $Au = f$. A natural first approach to solving such systems is by way of Gaussian elimination (or row reduction). Gaussian elimination is certainly a viable approach for small systems; however, the algorithm is computationally expensive (and inefficient) compared to the best algorithms available for many of the matrices that arise in the discretization of PDEs. Indeed, Gaussian elimination is a valuable general-purpose algorithm that can be used to solve $Au = f$ for any nonsingular matrix. However, in this section, you will explore specialized algorithms that are more efficient than naïve Gaussian elimination when A has added structure. Specifically, you will consider the cost of using a standard Gaussian elimination process on a dense matrix and compare that with specialized versions of Gaussian elimination that take into account the banded structure of a matrix and use sparsity to reduce the overall cost.

There are many variants of Gaussian elimination that have slightly different properties. Here, we consider an “in-place” factorization, where the entries in an $n \times n$ matrix A are overwritten by those of a lower-triangular matrix, L , and an upper-triangular matrix, U , such that $A = LU$. Algorithm 7.1 is often referred to as the *ikj* version of the factorization, due to the ordering of the loops within the algorithm.

The algorithm loops first over the rows (i) of matrix A . For each row, each column (k) before the diagonal is updated to store the multiplier used to eliminate that entry ($a_{i,k}$) in the row-reduction process; then, all subsequent columns (j) are updated based on that multiplier and the entries ($a_{k,j}$) in the corresponding row of A . In the in-place form, the entries in the strict

Algorithm 7.1: *ikj* in-place LU factorization**Input :** A , matrix to factorize (in-place)

```

1 for  $i = 2$  to  $n$ 
2   for  $k = 1$  to  $i - 1$ 
3      $a_{i,k} = a_{i,k}/a_{k,k}$ 
4     for  $j = k + 1$  to  $n$ 
5        $a_{i,j} = a_{i,j} - a_{i,k}a_{k,j}$ 

```

lower triangle of A are overwritten by those of L , which has unit entries on its diagonal and a zero upper-triangular part, and those in the upper triangle of A (including its diagonal) are overwritten by those of U . Mathematically, this is written as

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix} \rightarrow \begin{bmatrix} u_{1,1} & u_{1,2} & \cdots & u_{1,n} \\ \ell_{2,1} & u_{2,2} & \cdots & u_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{n,1} & \ell_{n,2} & \cdots & u_{n,n} \end{bmatrix}, \quad (7.4a)$$

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ \ell_{2,1} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{n,1} & \ell_{n,2} & \cdots & 1 \end{bmatrix}, \quad U = \begin{bmatrix} u_{1,1} & u_{1,2} & \cdots & u_{1,n} \\ 0 & u_{2,2} & \cdots & u_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{n,n} \end{bmatrix}. \quad (7.4b)$$

Remark 7.1: Pivoting

In practice, a pivoting strategy is often employed with Gaussian elimination in order to maintain numerical stability (see, for example, [20, Sec. 5.3]). This avoids dividing by very small numbers that might appear on the main diagonal during the row-reduction process. We omit the details here but assume that pivoting is performed when needed in the analysis below.

An important consideration for us is computational cost (i.e., *how long does the algorithm take?*). A count of the floating-point operations (flops), including both additions and multiplications, shows two flops per step in the innermost loop (over j) and one additional flop in the middle loop (over k). Thus, the total cost in flops is

$$\sum_{i=2}^n \sum_{k=1}^{i-1} \left(1 + \sum_{j=k+1}^n 2 \right) = \sum_{i=2}^n \sum_{k=1}^{i-1} (1 + 2(n-k)) \quad (7.5a)$$

$$= \sum_{i=2}^n \sum_{k=1}^{i-1} (1 + 2n) - 2k = \sum_{i=1}^{n-1} \sum_{k=1}^i (1 + 2n) - 2k \quad (7.5b)$$

$$= \sum_{i=1}^{n-1} (1 + 2n)i - i(i+1) = \sum_{i=1}^{n-1} 2ni - i^2 \quad (7.5c)$$

$$= \frac{2n(n-1)n}{2} - \frac{(n-1)n(2n-1)}{6} \quad (7.5d)$$

$$= \frac{2}{3}n^3 - \frac{1}{2}n^2 - \frac{1}{6}n. \quad (7.5e)$$

If A is symmetric and positive definite, the Cholesky factorization can be used in place of this but only saves a factor of two in the total cost.

This approach assumes that A is a *dense* matrix, meaning that most of its entries are nonzero. However, in considering the example in equation (7.2), we see that linear systems can be (and often are) very *sparse*, with only a fixed number of nonzero entries in each row or column of the matrix. Consequently, an efficient algorithm should take advantage of this structure in the factorization algorithm. A first approach takes a small step by assuming that the matrix is *banded*.

Definition 7.2: Banded sparse matrix [90, Sec. 5.3.3]

A banded matrix, A , is a matrix whose nonzero entries are confined to be in a band around the main diagonal. Thus, there exists a $k < n$ such that $a_{i,j} = 0$ if $|i - j| > k$. The smallest such k is called the bandwidth of A .

For tridiagonal matrices, which might result from discretization of a 1D elliptic PDE, the resulting bandwidth is one. In contrast, for two-dimensional problems on regular $n_x \times n_y$ meshes using lexicographic ordering with $n_x = n_y$, the discretization matrices for second-order differential operators typically have a bandwidth of about n_x , with the exact value depending on the discretization (e.g., finite difference vs. finite element) and boundary conditions (eliminated Dirichlet vs. noneliminated). For example, in equation (7.2), we immediately observe the banded structure, with a bandwidth of $n_x - 1 = 3$.

To see how this helps control costs, consider the LU factorization of a banded matrix with bandwidth $B > 0$, without pivoting. From Algorithm 7.1, we see that if entry $a_{i,k} = 0$ when the algorithm reaches row i , then the inner loops over k and j can be skipped as no productive computation is done. For a matrix position outside the bandwidth, it must always be the case that $a_{i,k} = 0$ when the algorithm reaches it, because the only updates done so far to row i are those for columns 1 through $k-1$, which are also all outside of the bandwidth and, consequently, have zero entries. Similarly, the final loop over j can be truncated when $a_{k,j}$ falls outside the bandwidth, since such an entry remains zero even after earlier updates. Thus, we only need to compute the LU factors within the band of the matrix, and we know that $\ell_{i,k} = 0$ if $i - k > B$ and $u_{i,j} = 0$ if $j - i > B$. With this, we can modify the factorization in Algorithm 7.1 to Algorithm 7.2, where the limits on the loops distinguish the two.

Counting flops, as above, we now have the total cost of the banded factorization algorithm as

$$\sum_{i=2}^n \sum_{k=\max(1, i-B)}^{i-1} \left(1 + \sum_{j=k+1}^{\min(k+B, n)} 2 \right) \leq \sum_{i=2}^n \sum_{k=\max(1, i-B)}^{i-1} (1 + 2(k + B - k)) \quad (7.6a)$$

$$\leq \sum_{i=2}^n \sum_{k=i-B}^{i-1} (1 + 2B) \quad (7.6b)$$

$$= \sum_{i=2}^n (B(1 + 2B)) \leq (2B^2 + B)n. \quad (7.6c)$$

Thus, banded factorization on an $n \times n$ matrix with bandwidth $B < n$ has a cost of $\mathcal{O}(B^2 n)$ compared to a dense factorization cost of $\mathcal{O}(n^3)$. This clearly offers a significant advantage when $B \ll n$. Most notably, when $B = \mathcal{O}(1)$, banded factorization is an $\mathcal{O}(n)$ algorithm, making it *asymptotically optimal* since at least n operations are required to compute a vector of length n , and banded factorization (plus banded triangular solves) calculates the solution to $A\mathbf{u} = \mathbf{f}$ in $\mathcal{O}(n)$ operations.

Algorithm 7.2: *ikj* banded in-place LU factorization**Input :** A , matrix to factorize (in place)

```

1 for  $i = 2$  to  $n$ 
2   for  $k = \max(1, i - B)$  to  $i - 1$ 
3      $a_{i,k} = a_{i,k} / a_{k,k}$ 
4     for  $j = k + 1$  to  $\min(k + B, n)$ 
5        $a_{i,j} = a_{i,j} - a_{i,k} a_{k,j}$ 

```

Example 7.3: Banded factorization for discretizations of second-order PDEs

The discretization matrices corresponding to a one-dimensional second-order differential equation have bandwidth $B = 1$, giving a cost bounded by $3n_x$ operations to compute the LU factors on a grid with n_x intervals. While this is optimal, things get worse in higher dimensions. For those corresponding to a two-dimensional problem on an $n_x \times n_y$ mesh with $n_x = n_y$, $B \approx n_x$ and $n \approx n_x^2$ is the size of the matrix (again, depending on the details of the discretization and boundary conditions), giving a cost of approximately $2n_x^4$. This is, of course, a substantial improvement on the cost of $\frac{2}{3}n_x^6$ achieved by the dense factorization algorithm, but it is far from optimal, as there are only $\mathcal{O}(n_x^2) \sim \mathcal{O}(n)$ nonzero entries in the matrix A and in the solution vector for $Au = f$.

While we focus primarily on the cost of factorization, it is important to also consider the cost of the forward-backward substitution phase of solving a linear system once its LU factorization has been computed. While the solution can be done in place (similarly to the factorization itself), it is convenient to think of the solution of $LUu = f$ as first solving for the vector v such that $Lv = f$ and then solving $Uu = v$. The first of these solves has a cost of $n(n - 1)$ flops, since L has unit diagonal, so $v_i = f_i - \sum_{j=1}^{i-1} \ell_{i,j} v_j$, costing $2(i - 1)$ flops to compute v_i . The second of these solves requires n additional flops to divide by the nonunit diagonal entries of U after the off-diagonal terms are accumulated. Thus, the total cost of solving $LUu = f$ after the factorization has been computed is an additional $2n^2 - n$ flops. In the banded case, the cost of row i in the solve of $Lv = f$ is now bounded by $2B$ (independent of i), so the cost of solving $LUu = f$ is bounded by $(4B + 1)n$, a substantial savings on the $\mathcal{O}(n^2)$ cost in the dense case when n is large.

7.1.1 ■ Taking advantage of sparsity

While the banded factorization algorithm offers some significant improvement in cost compared to the fully dense algorithm, it is still a “dense” algorithm, making no use of zeros that appear within the bandwidth. While this is reasonable in some settings, it ignores the possibility that the bandwidth is large because of only a few rows or columns in the matrix. For example, equation (7.2) shows a bandwidth of three, but many zero entries within the outermost band of nonzeros. To account for this, we can refine the ideas that led us to the banded algorithm at a finer level of detail. In what follows, we assume that the matrix, A , has a symmetric nonzero pattern, meaning that $a_{i,j} \neq 0$ if and only if $a_{j,i} \neq 0$. This is not strictly necessary, but it greatly simplifies the calculations. Moreover, this assumption is generally true (or nearly so) for many of the discretization matrices that we have considered so far. Thus, we would like to see how taking this structure into account affects the cost of the factorization algorithm. Before beginning, we need some definitions to simplify the notation.

Definition 7.4: Sparse matrix properties [105, Ch. 4]

Given an $n \times n$ matrix A , we define

- $f_i(A) = \min\{j \mid a_{i,j} \neq 0\}$, giving the index of the first column with a nonzero entry in row i ;
- $\beta_i(A) = i - f_i(A)$, the i th bandwidth of A , implying that $B = \max_i\{\beta_i(A)\}$;
- $\text{Env}(A) = \{\{i, j\} \mid f_i(A) \leq j < i\} = \{\{i, j\} \mid 0 < i - j \leq \beta_i(A)\}$, the envelope of A ;
- $\omega_i(A) = \{k \mid k > i \text{ and } \exists \ell \leq i \text{ such that } a_{k,\ell} \neq 0\}$, the i th front of A , with front width $|\omega_i(A)|$; and
- $|\text{Env}(A)| = \sum_{i=1}^n \beta_i(A) = \sum_{i=1}^n |\omega_i(A)|$, the envelope size of A .

Note that we use the absolute value notation on sets, with $|S|$ denoting the cardinality of the set S . An important feature of this definition is that the envelope of A includes all of the nonzero entries in A , but also (potentially) some zero entries that appear between the first column in each row with a nonzero entry and the main diagonal. The i th front of A contains all rows of A that are modified in the i th step of elimination; these rows are identified as the rows for which $\text{Env}(A)$ intersects column i below row i .

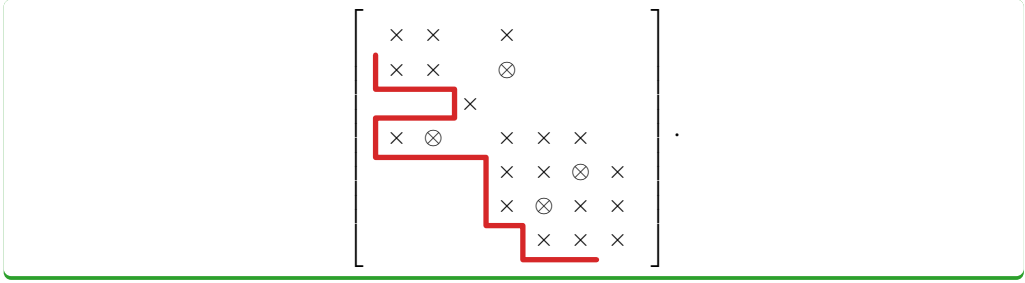
While the entries of A within its envelope can be either zero or nonzero, we generally expect the entries of L and U within the envelope of A to be nonzero, due to *fill-in* during the row-reduction process, where we overwrite entries based on the computed multipliers in L . There are two ways in which entries with value zero can occur within the envelope of a matrix through this process. First, as shown in Example 7.5, there can be an element (i, j) within the envelope where fill-in occurs in row i but in columns before j , but no fill-in can possibly occur in column j . Such entries are rare in the factorization process, so we do not make an effort to track them. The second possibility is that the fill-in introduced in position (i, j) at one step is canceled out by a calculation at a later step. This is usually difficult to predict in practice (and does not occur in Example 7.5). Consequently, when analyzing the cost of a factorization, we ignore the possibility of such coincidental cancellation.

Example 7.5: Fill-in during factorization

Consider the factorization of a matrix, A , whose sparsity pattern is given as

$$\begin{bmatrix} \times & \times & & \times & & & \\ \times & \times & & & & & \\ & & \times & & & & \\ \times & & & \times & \times & \times & \\ & & & \times & \times & & \times \\ & & & & \times & \times & \times \\ & & & & & \times & \times & \times \end{bmatrix}.$$

By drawing $\text{Env}(A)$, we see that $\omega_i(A)$ is the set of rows where $\text{Env}(A)$ intersects column i below row i in this setting. Importantly, this is where the zero entries in A may “fill in” with nonzero values during the factorization process. For the matrix above, the lines below show $\text{Env}(A)$, and the $\otimes$ symbols show fill-in that occurs during the factorization:



A central question to answer is how the costs of the factorization and solve stages depend on $\text{Env}(A)$ (or other properties) when we take advantage of sparsity within the envelope. This is accomplished by modifying the ordering of the unbanded algorithm, Algorithm 7.1, into what is known as the *kij* form (or outer product form) of the factorization. This is shown in Algorithm 7.3.

Algorithm 7.3: *kij* in-place LU factorization

Input : A , matrix to factorize (in place)

```

1 for  $k = 1$  to  $n - 1$ 
2   for  $i = k + 1$  to  $n$ 
3      $a_{i,k} = a_{i,k} / a_{k,k}$ 
4     for  $j = k + 1$  to  $n$ 
5        $a_{i,j} = a_{i,j} - a_{i,k} a_{k,j}$ 

```

In the sparse version of this algorithm, we recognize that the loops over i and j can be seen as

1. a vector scaling of entries below the diagonal in column k ;
2. an outer product of the scaled vector with the row vector of entries in row k after the diagonal; and
3. an update to the corresponding entries in A .

Thus, in the *sparse* case, we only need to compute the nonzero entries in each of these, or $|\omega_k(A)|$ in total, assuming a nonzero sparsity structure. The cost of each outer loop (over k) is then $|\omega_k(A)| + 2|\omega_k(A)|^2$. Summing these gives the total cost as

$$\sum_{k=1}^{n-1} |\omega_k(A)| (1 + 2|\omega_k(A)|). \quad (7.7)$$

In the case of a banded matrix, $|\omega_k(A)| \approx B$ for all k and, so, we recover the same banded algorithm cost of $\mathcal{O}(B^2n)$. While this accounting is a direct result of the *sparse kij* algorithm, the same cost estimates hold for all orderings of the loops in the LU factorization, but obtaining the bound is more technical. However, the main result remains: the cost of sparse Gaussian elimination depends strongly on the sizes of $\omega_k(A)$. If all of these are small, then the overall cost is small. In the next section, we discuss how to take advantage of this idea by *reordering* the linear system and representing the matrices as graphs. We then give some examples of such sparse-direct algorithms.

7.2 ■ Reordering algorithms for Gaussian elimination

Objectives

For a given linear system $A\mathbf{u} = \mathbf{f}$, one problem when using Gaussian elimination is that we generally have no control over the sizes of the fronts of A , $\omega_k(A)$. We can, however, *reorder* the linear system and solve an equivalent system in its place. In this section, you will learn some strategies for reordering that allow for a reduced cost of factorization. Specifically, you will consider examples showing how different reorderings can help or not help for a particular matrix structure. Then, using concepts from graph theory, you will analyze the reordering algorithms and see three examples that reduce the overall cost of Gaussian elimination.

Let Π be a permutation of $\{1, 2, \dots, n\}$, and define the permutation matrix, P , such that

$$p_{i,j} = \begin{cases} 1 & \text{if } j = \Pi(i), \\ 0 & \text{otherwise.} \end{cases} \quad (7.8)$$

Since P is an orthogonal matrix, with $P^{-1} = P^T$, the solution to $A\mathbf{u} = \mathbf{f}$ is also the solution to $(P^T A P)P^T \mathbf{u} = P^T \mathbf{f}$. The reordered matrix, $P^T A P$, is a similarity transformation of the original system (so it is positive definite when A is positive definite) and also preserves the assumed symmetry of the nonzero pattern. Ideally, we would then like to answer the question of what is the best possible choice of permutation matrix, P . That is, of all possible permutations matrices, P , which should we choose to minimize either $|\text{Env}(P^T A P)| = \sum_{i=1}^n |\omega_i(P^T A P)|$ or the total cost of factorization, $\sum_{k=1}^{n-1} |\omega_k(P^T A P)| (1 + 2|\omega_k(P^T A P)|)$. This is, unfortunately, a very difficult question to answer, turning out to be an NP complete problem.

Instead, we turn to heuristics. To get a sense of how much reordering might help, we consider a (somewhat contrived) example (Example 7.6) of an $n \times n$ matrix with all zero off-diagonal entries except for a single dense row/column pair. We then show two different “reorderings,” only one of which leads to an asymptotic improvement in the cost of factorization.

Example 7.6: Reordering example

Consider the case of an $n \times n$ matrix with sparsity pattern for $n = 5$ given by

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & & & \\ \times & & \times & & \\ \times & & & \times & \\ \times & & & & \times \end{bmatrix}.$$

For the general case of a matrix, A , with nonzero entries $a_{1,i}$, $a_{i,1}$, and $a_{i,i}$ for $1 \leq i \leq n$, we observe that $|\omega_k(A)| = n - k$ for all k , giving a cost of factorization in this ordering of

$$\sum_{k=1}^{n-1} (n - k) (2(n - k) + 1) = \mathcal{O}(n^3). \quad (7.9)$$

Intuitively, this makes sense, since elimination of the entries in the first column using multiples of the first row introduces nonzero entries in all rows after the first, leading to a cost equal to that of dense LU. If, instead, we choose a permutation Π to minimize the bandwidth of the

matrix, we have

$$\begin{bmatrix} \times & & \times & & \\ & \times & \times & & \\ \times & \times & \times & \times & \times \\ & & \times & \times & \\ & & \times & & \times \end{bmatrix},$$

with

$$\Pi(i) = \begin{cases} \lceil \frac{n}{2} \rceil & \text{if } i = 1, \\ 1 & \text{if } i = \lceil \frac{n}{2} \rceil, \\ i & \text{otherwise.} \end{cases} \quad (7.10)$$

Taking P to be the associated permutation matrix, we now have $|\omega_k(P^T AP)| = 1$ for $1 \leq k < \lceil \frac{n}{2} \rceil$, while $|\omega_k(P^T AP)| = n - k$ for $\lceil \frac{n}{2} \rceil \leq k \leq n$. Unfortunately, by direct computation we see that

$$\sum_{k=1}^{n-1} |\omega_k(P^T AP)| (2 |\omega_k(P^T AP)| + 1) = \mathcal{O}(n^3), \quad (7.11)$$

with a slight improvement in the constant in comparison to no permutation.

Another option is to choose a reverse permutation, with $\Pi(i) = n - i + 1$, and the resulting nonzero structure becomes amenable to efficient elimination:

$$\begin{bmatrix} \times & & & \times \\ & \times & & \times \\ & & \times & \times \\ & & & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix}.$$

Now, again calling the associated permutation matrix P , we have $|\omega_k(P^T AP)| = 1$ for all k , giving an $\mathcal{O}(n)$ factorization cost. This example highlights the importance of selecting the “right” permutation for a given problem, which can have a notable impact on the factorization and overall solve cost.

To approach a more systematic way of defining a permutation matrix, P , for a given matrix, A , many algorithms make use of a graph representation (see Definition 7.7) of the nonzero pattern of A .

Definition 7.7: Graphs

A *graph*, $G = (V, E)$, consists of a set of vertices (or nodes), V , and a set of edges, E , which are two-element subsets of V .

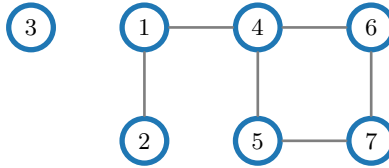
For an $n \times n$ matrix, A , we consider $V = \{1, \dots, n\}$ as the set of nodes, with each element of V associated with the corresponding row/column of A , and require that $\{i, j\} \in E$ if and only if $i \neq j$ and $a_{i,j} \neq 0$. In this case, we still assume a symmetric nonzero pattern in the matrix, so that this “undirected” graph definition (which makes no distinction between an edge “from” i “to” j and one “from” j “to” i) is sufficient. In addition, we assume that pivoting is unnecessary and that we can ignore the influence of the diagonal entries (in the graph).

Example 7.8: Graph of the arrow matrix

First, we consider the graph associated with the “arrow” matrix from Example 7.6. Here, there are five nodes in V and edges to each node (except for itself) from node 1:

**Example 7.9: Graph of a matrix**

As a second example, we consider the graph associated with the matrix considered in Example 7.5. Here, we see that node 3 has no off-diagonal entries in the associated row and column of A , so it appears as an *isolated* node, with no incident edges:



The key reason why we introduce the graph formality is to relate structure in the graph to $\text{Env}(A)$ and $\omega_i(A)$. In particular, we use the adjacency set, which is the set of all nodes that have an incident edge on another node in the set. From Definition 7.10, we then get two results (Theorem 7.11 and Corollary 7.12) that allow us to describe some specific algorithms.

Definition 7.10: Adjacency set

Given a graph, $G = (V, E)$, and a subset of vertices $S \subset V$, the *adjacency set* of S is defined as $\text{Adj}_G(S) = \{v \in V \setminus S \mid \{v, s\} \in E \text{ for some } s \in S\}$. For vertex v , the *adjacency set*, $\text{Adj}_G(v)$, is the set of all vertices incident to v .

Theorem 7.11: Envelopes and adjacency sets [105, Thm. 4.3.5]

For $i < j$, $\{j, i\} \in \text{Env}(A)$ if and only if $j \in \text{Adj}_G(\{1, 2, \dots, i\})$.

Proof. If $j \in \text{Adj}_G(\{1, 2, \dots, i\})$, then $a_{j,k} \neq 0$ for some $k \leq i$, so that $\{j, k\} \in E$. This implies that $f_j(A) \leq i$ (see Definition 7.4) and, consequently, $\{j, i\} \in \text{Env}(A)$.

Conversely, if $f_j(A) \leq i < j$, then $\{j, f_j(A)\} \in E$, since $a_{j, f_j(A)} \neq 0$. This implies that $j \in \text{Adj}_G(\{1, 2, \dots, i\})$. $\square$

Corollary 7.12: Fronts and adjacency sets [105, Cor. 4.3.6]

For $i = 1, \dots, n$, $\omega_i(A) = \text{Adj}_G(\{1, 2, \dots, i\})$.

Proof. By its definition, $\omega_i(A) = \{j > i \mid \{j, i\} \in \text{Env}(A)\} = \text{Adj}_G(\{1, 2, \dots, i\})$. $\square$

From the above results, we now view the question of making both $|\omega_i(A)|$ and $\sum |\omega_i(A)|(2|\omega_i(A)| + 1)$ small in terms of $|\text{Adj}_G(\{1, 2, \dots, i\})|$.

7.2.1 ■ The reverse Cuthill–McKee algorithm

The first algorithm we consider is the *Cuthill–McKee* algorithm, which aims to minimize the bandwidth of the permuted matrix, $P^T AP$. While this does not directly relate to $|\omega_i(A)|$, it is used in subsequent algorithms. To define the algorithm, we introduce some more concepts from graph theory.

Definition 7.13: Graph distance [105, Sec. 4.4.2]

Given a graph, $G = (V, E)$, we have the following definitions:

- The *distance*, $d_G(u, v)$, between two vertices, $u, v \in V$, is the length of the shortest path between them using edges in E . If no path exists, $d_G(u, v) = \infty$. If $u = v$, then $d_G(u, v) = 0$.

- The *eccentricity*, $\varepsilon_G(v)$, of a vertex, $v \in V$, is the maximum distance from v to any other vertex:

$$\varepsilon_G(v) = \max_{w \in V} d_G(v, w). \quad (7.12)$$

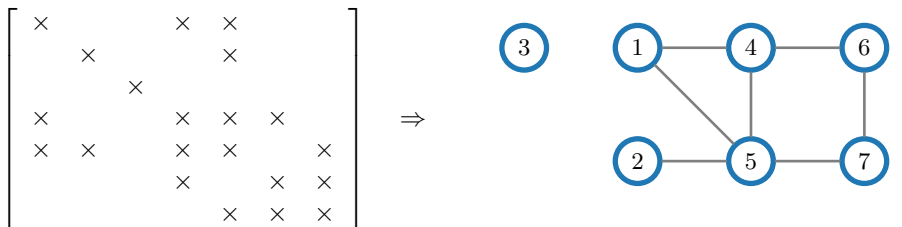
- The *diameter* of G is $\text{diam}(G) = \max_{v \in V} \varepsilon_G(v)$.
- A *peripheral* vertex of G is a vertex, v , where $\varepsilon_G(v) = \text{diam}(G)$.
- The *degree* of vertex v in the (undirected) graph G is the number of incident edges, denoted by $\text{degree}_G(v) = |\text{Adj}_G(v)|$.

The key ideas behind the Cuthill–McKee algorithm can be summarized by three heuristic principles related to the sequential construction of the permutation, Π , by enumerating the vertices in the graph of A :

1. Attempt to minimize bandwidth by ordering adjacent nodes sequentially.
2. When enumerating neighbors of a node, do so in order from highest degree to lowest. The high-degree neighbors are likely to have direct or indirect connections to other neighbors, so numbering these together can improve ordering later on in the process.
3. Start the numbering from a peripheral vertex. We aim to sweep through the graph in some sort of structured manner and so it is disadvantageous to start “in the middle.”

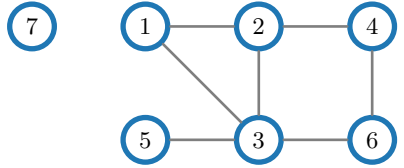
An important fourth principle, discovered by accident by Alan George (in his PhD thesis [107]), is to take the ordering generated by the above algorithm and reverse it, leading to the *reverse Cuthill–McKee* algorithm. This approach generally improves the reduction in envelope size and, as such, is useful from the perspective of reducing the cost of factorization, although it does not change the bandwidth.

To illustrate the Cuthill–McKee algorithm, we consider a simple model problem whose matrix is given as a modification of Examples 7.5 and 7.9:



In this ordering, the matrix has bandwidth four, and four zero entries are contained within $\text{Env}(A)$.

The first principle of the Cuthill–McKee algorithm is to order adjacent nodes sequentially, which is clearly not the case here, since node 1 is connected to nodes 4 and 5, but not to node 2 or 3. Thus, an “improved” numbering may be given by the graph

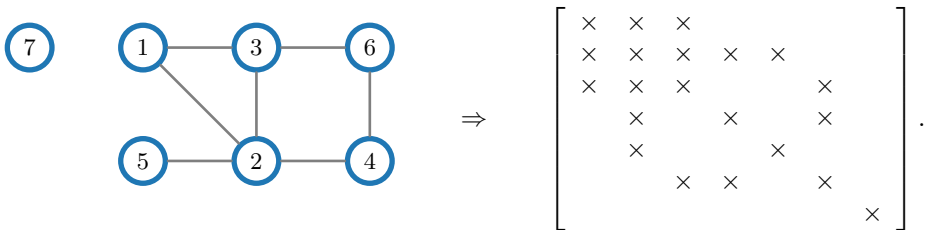


which leads to the nonzero pattern matrix

$$\begin{bmatrix} \times & \times & \times & & & & \\ \times & \times & \times & \times & & & \\ \times & \times & \times & & \times & \times & \\ & \times & & \times & \times & & \\ & & \times & \times & & \times & \\ & & \times & \times & & \times & \\ & & & & & & \times \end{bmatrix}.$$

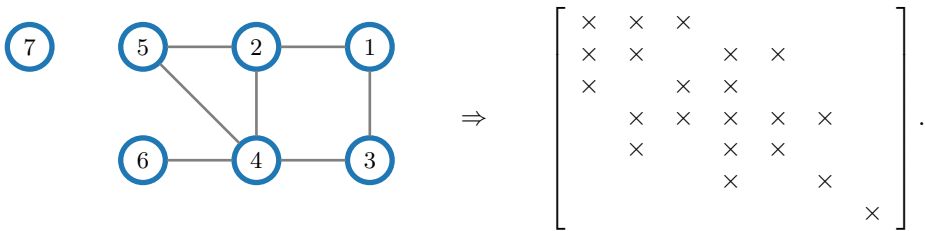
As a result, in this ordering there are three zero entries within the envelope of the matrix, and the bandwidth has also been reduced to three.

The next refinement to this ordering is, then, to add vertices in order from the highest-degree neighbor to the lowest-degree neighbor, leading to the following graph and corresponding nonzero pattern matrix:



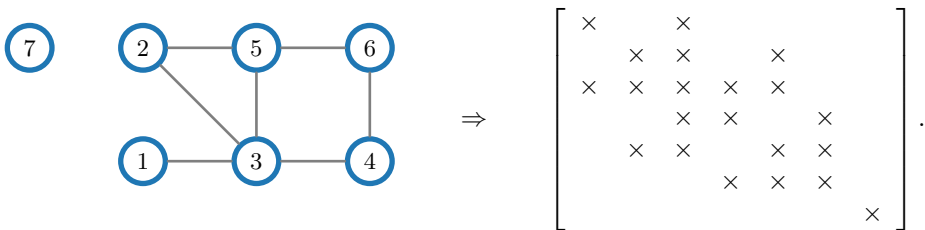
Consequently, this ordering has increased the number of zeros within the envelope back to four, while maintaining a bandwidth of three.

Then, observe that the process of identifying the peripheral vertices is complicated by the presence of an isolated vertex. However, the reorderings above show that as long as this vertex is numbered last or first, it does not contribute any zero entries within the band of the matrix, or to the bandwidth. Thus, we can safely order this vertex first (or last) and consider only the ordering of the *connected component* of the graph. This is true in general: when the graph of A is disconnected, we determine a reordering of A by ordering each connected component of the graph in sequence. For the connected component of the graph in this example, node 1 in the above numbering is not a peripheral vertex, since its eccentricity is only two, while nodes 5 and 6 in the last ordering (nodes 2 and 6 in the original ordering) are peripheral vertices, with eccentricity three. Following the heuristic to add higher-degree vertices first, we now renumber vertex 6 as vertex 1 to produce the following graph and nonzero pattern matrix:



Here, we see a reduction back to three zero entries within the envelope of the reordered matrix, and the bandwidth remains three.

Finally, we reverse the above ordering to obtain the *reverse Cuthill–McKee* ordering. As above, we do this only over the connected component of the graph (although it makes no difference to the envelope or bandwidth if the disconnected vertex is also included in the reverse ordering). This gives the following graph and nonzero pattern matrix:



In this ordering, only one zero entry appears within the envelope, although the bandwidth remains three.

7.2.2 ■ Elimination graphs and the minimum-degree algorithm

By returning to the connection between the envelope of a matrix and the adjacency sets in the associated graph, more sophisticated reordering schemes arise. The key results in Theorem 7.11 and Corollary 7.12 lead to a basic “greedy” strategy for choosing the permutation that, of all as yet unnumbered nodes, we should choose a node of minimum degree to number next. This agrees with the intuition given by Corollary 7.12 that we want $\text{Adj}_G(\{1, \dots, i+1\})$ to be as small as possible in order to minimize $\omega_{i+1}(A)$. However, this relies on choosing a node with a small number of connections relative to the fill-in already introduced. Thus, to optimize the selection, we need to track how the connectivity of the graph changes as the elimination process proceeds. To accomplish this, we use the idea of an *elimination graph* (defined in Definition 7.14).

Definition 7.14: Elimination graphs [105, Sec. 5.2.1]

Given a graph, $G = (V, E)$, the elimination graph of G when eliminating vertex $v \in V$ is the graph

$$G' = (V', E'), \quad (7.13)$$

where $V' = V \setminus \{v\}$ and where $\{u, w\} \in E'$ for $u, w \in V'$ if either $\{u, w\} \in E$ or both $\{u, v\} \in E$ and $\{v, w\} \in E$.

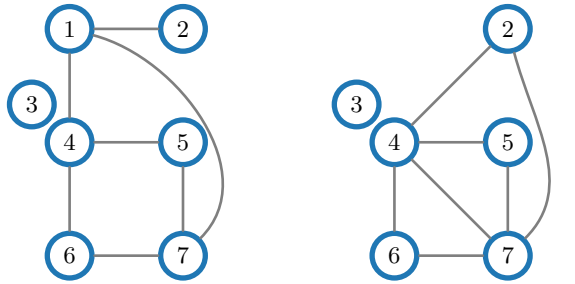
An important feature of elimination graphs is that if $\{u, w\} \in E'$ but $\{u, w\} \notin E$, then the elimination of v has added a connection between u and w , corresponding to a new nonzero entry introduced in the corresponding LU factors. See Example 7.15.

Example 7.15: Elimination graph example

Consider the matrix with nonzero pattern

$$\begin{bmatrix} \times & \times & & \times & & \times \\ \times & \times & & & & \\ & & \times & & & \\ \times & & & \times & \times & \times \\ & & & \times & \times & \times \\ & & & \times & & \times \\ \times & & & & \times & \times \end{bmatrix},$$

and suppose the first row has already been used to eliminate the nonzero entries in the first column. This elimination introduces three more nonzero entries in each of the lower- and upper-triangular portions of the matrix, corresponding to three “new” edges in the graph associated with the matrix after the first row has been processed. The nonzero fill-in entries in positions $(4, 2)$, $(7, 2)$, and $(7, 4)$ clearly change the degrees of nodes 4 and 7, which affects the best decisions to be made for further elimination. From the graph perspective, we are eliminating node 1 (from the graph at left below), and we obtain the connections in the elimination graph (shown at right below):



We track the fill-in introduced in the elimination process by constructing a sequence of elimination graphs. Given $G_0 = (V_0, E_0)$ corresponding to the original $n \times n$ matrix, A , we construct a series of graphs, $G_0, G_1, G_2, \dots, G_{n-1}$, where each elimination graph $G_i = (V_i, E_i)$ has $|V_i| = n - i$. The elimination graph G_i tells us about the connections between vertices that have yet to be eliminated. We then use this to choose a node of minimum degree over E_i to be eliminated to form G_{i+1} . In this way, we directly minimize $|\omega_{i+1}(A)|$ over all possible choices for the node to be eliminated at step i . This leads to the basic minimum-degree algorithm, given next as Algorithm 7.4. We then apply this to the graph from Example 7.15 in Example 7.16.

Algorithm 7.4: Basic minimum-degree algorithm

Input : $G_0 = (V_0, E_0)$, initial graph of vertices and edges

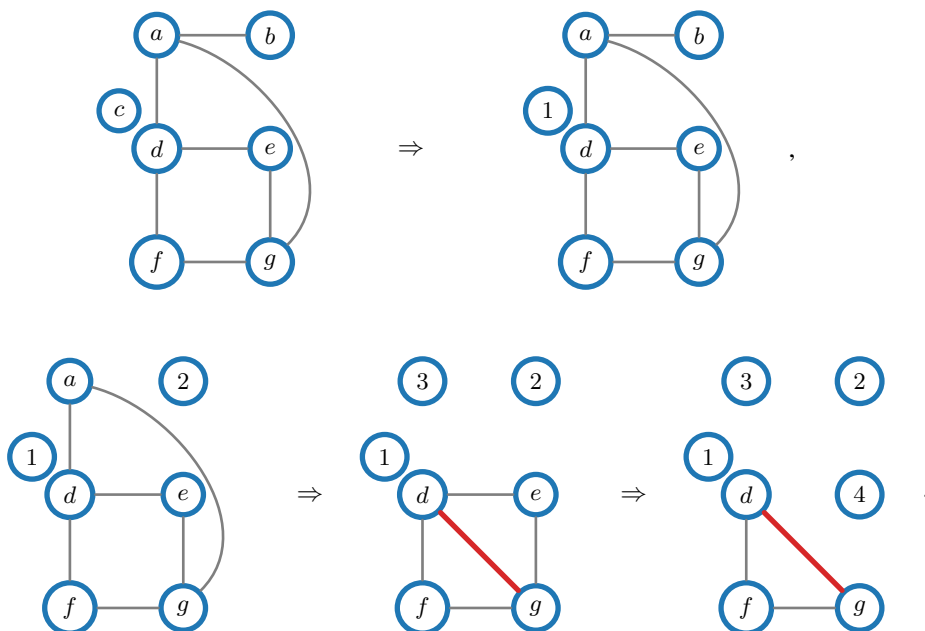
Output : $\{z_i\}_{i=1}^n$, minimum-degree ordering

- 1 $n := |V_0|$
- 2 **for** $i = 1$ **to** $n - 1$
- 3 $z_i \leftarrow \operatorname{argmin} \{\operatorname{degree}_{G_i}(v) \text{ for } v \in V_i\}$ {choose the node with minimum degree}
- 4 $G_{i+1} = (V_i \setminus \{z_i\}, E_{i+1})$ {form the elimination graph from equation (7.13)}

Example 7.16: Basic minimum-degree algorithm

We apply Algorithm 7.4 directly to the matrix/graph shown in Example 7.15. To simplify things, we replace the initial node numbers by letters and assign numbers to the nodes in the order that they are eliminated.

Of course, with degree zero, node c is the first to be eliminated. After this, node b is the lowest-degree node, with degree one. Eliminating both of these nodes introduces no fill in the algorithm. At this point, we have three nodes of degree two, a , e , and f . Eliminating any of these will introduce some fill-in, but only a single edge between nodes d and g . We break the tie arbitrarily and eliminate a next. Now, nodes e and f both have minimum degree of two, and eliminating e next gives a complete graph on nodes d , f , and g , which we take in order (any order will work):



The resulting reordered matrix has the following form, with only one zero entry in its envelope in the minimum-degree ordering, corresponding to the single edge that was introduced in the elimination process:

$$\begin{bmatrix} \times & & & & & \\ & \times & \times & & & \\ & \times & \times & & \times & \times \\ & & & \times & \times & \times \\ & & \times & \times & \times & \times \\ & & & & \times & \times & \times \\ & \times & \times & & \times & \times \end{bmatrix}.$$

There are several natural improvements to the basic minimum-degree algorithm that improve the efficiency of the approach. The first is known as “mass elimination” and is based on the observation that after elimination of a node, y , some of its neighbors can be eliminated directly, without further deliberation, as they are the natural next choices for elimination. If we define G_y to be the elimination graph of G after eliminating vertex y , then the result in Theorem 7.17 holds, where $\text{degree}_G(z)$ denotes the degree of node z (the number of edges incident on the node) in the graph, G .

Theorem 7.17: Mass elimination [106, Thm. 3.1]

If y is selected as the minimum degree in G , then

$$\{z \in \text{Adj}_G(y) \mid \text{degree}_{G_y}(z) = \text{degree}_G(y) - 1\} \quad (7.14)$$

can be eliminated after y in any order.

This is seen directly by noting that if z is a node in this set, then its degree must have been the same as the node of minimum degree, y , before y was eliminated, and that no new edges incident on these nodes were created in the elimination graph G_y . This is particularly useful in the final stages of elimination, where the graph G is (close to) a complete graph on the remaining vertices. A second improvement is possible by noting that some nodes can be treated together in elimination. We say that nodes u and v are *indistinguishable* if $\text{Adj}_G(u) \cup \{u\} = \text{Adj}_G(v) \cup \{v\}$. This terminology is inspired by the result in Theorem 7.18.

Theorem 7.18: Indistinguishability [106, Thm. 3.3]

If u and v are indistinguishable in G , they are also indistinguishable in G_y (if $y \neq u, v$).

Finally, we note that we can generalize this observation to cases where the adjacency sets are fully nested. We say that node v is *outmatched* by u if $\text{Adj}_G(u) \cup \{u\} \subset \text{Adj}_G(v) \cup \{v\}$.

Theorem 7.19: Outmatched nodes [156]

If v is outmatched by u in G , then it is also outmatched by u in G_y (if $y \neq u, v$).

An important consequence of this is that the degree of u must be less than that of v , which means that if v is outmatched by u at any stage of the elimination process, we can ignore v as a candidate to be eliminated until after u has been eliminated.

7.2.3 ■ Nested dissection

While minimum degree and similar algorithms offer a robust approach for general sparse matrices, many such matrices do have a structured nonzero pattern, coming from discretizations on structured meshes. Thus, we now consider a method developed for this situation. Here, we focus on the standard 2D Poisson model problem with a five-point finite-difference discretization on a tensor-product mesh (see for example equation (7.2)), where each row of A corresponds to a point in the finite-difference mesh with index (i, j) , and (aside from those corresponding to nodes near the boundary of the mesh) A has off-diagonal nonzero entries corresponding to columns associated with the finite-difference mesh points $(i \pm 1, j)$ and $(i, j \pm 1)$. The graph of such a matrix is a lattice graph that matches the finite-difference mesh. Since such graphs arise regularly, it is worthwhile to have specialized methods that take advantage of structure. In addition, the possibility exists for much deeper theoretical understanding because of the regular structure.

In 1973, two major results were published, strongly characterizing the best possible reorderings for graphs in this family. First, Hoffman, Martin, and Rose [137] proved that the cost of elimination of $P^T A P$ must be at least $\mathcal{O}(n_x^3)$ when A comes from discretization on an $n_x \times n_y$ mesh with $n_x = n_y$, and where P is any permutation matrix. The proof of this result depends on properties of elimination graphs for this problem, showing that a clique (i.e., a complete subgraph) of a certain size must arise, resulting in a lower bound on the complexity. Second, George proposed the “nested dissection” algorithm [104] and proved that it yields a permutation, P , such that the cost of factoring $P^T A P$ is $\mathcal{O}(n_x^3)$, again when A comes from discretization on an

$n_x \times n_y$ mesh with $n_x = n_y$. Since this matches (asymptotically) the best possible complexity, we say that this is an optimal algorithm.

The idea behind the nested dissection algorithm is to define a *separator* in a graph as a set of nodes that, when removed, splits the graph into two or more disconnected pieces. The ordering of nodes in the permutation is then determined based on choosing a hierarchy of separators on the regular graph. With a target of $\mathcal{O}(n_x^3)$ cost on an $n_x \times n_y$ mesh with $n_x = n_y$, we use $\mathcal{O}(n_x)$ separator nodes so that dense Gaussian elimination on a matrix over this set has an $\mathcal{O}(n_x^3)$ cost for factorization. Consider a 7×7 mesh, as pictured in Figure 7.2.1. We use the “central cross” in this graph as a separator (the nodes adjacent to the red edges in the figure) and order these nodes last (and lexicographically) in the permutation. Then, we recurse on the four smaller 3×3 meshes, again defining a central cross for each (those nodes adjacent to the green edges) and four subproblems (now on 1×1 meshes, or single nodes). Here, we order each 3×3 separator immediately after the nodes in its four subproblems.

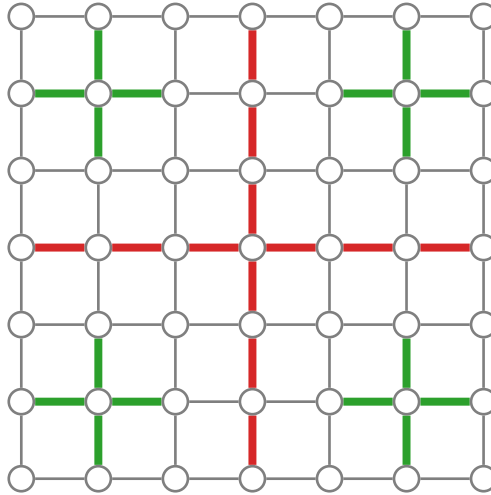


Figure 7.2.1. Graph of a 7×7 regular grid, with nested dissection separators highlighted.

To prove the complexity of factorization based on this reordering, we consider the partial problem after the first reordering. Assuming n_x is odd, then, as in the example above, there are $2n_x - 1$ nodes in the central cross and four subproblems, each of size $\left(\frac{n_x-1}{2}\right) \times \left(\frac{n_x-1}{2}\right)$. These subproblems are completely disjoint, so we can write the reordered system matrix as

$$P^T A P = \begin{bmatrix} A_{1,1} & 0 & 0 & 0 & A_{1,s} \\ 0 & A_{2,2} & 0 & 0 & A_{2,s} \\ 0 & 0 & A_{3,3} & 0 & A_{3,s} \\ 0 & 0 & 0 & A_{4,4} & A_{4,s} \\ A_{s,1} & A_{s,2} & A_{s,3} & A_{s,4} & A_{s,s} \end{bmatrix}, \quad (7.15)$$

where $A_{i,i}$, $1 \leq i \leq 4$, correspond to the matrix restricted to each of the disjoint subdomains, while $A_{i,s}$ and $A_{s,i}$ for $1 \leq i \leq 4$ contain the connections from each subdomain to/from the separator (the central cross). Similarly, $A_{s,s}$ contains the connections in the original matrix between nodes in the separator. The cost of factoring the matrix in this reordering is easily accounted for by summing the costs of factoring each of the $A_{i,i}$ plus those associated with the separator. As above, we assume that the cost of accounting for the separator is $\mathcal{O}(n_x^3)$, equal to that of dense Gaussian elimination on $A_{s,s}$.

While a careful count of operations is possible, the general form of the proof considers the case when $n_x = 2^k - 1$ and all recursively defined subproblems also have sizes that are one less than a power of two. Here, we let $\theta(n_x)$ denote the cost of factoring the reordered matrix $P^T A P$ when A corresponds to an operator on an $n_x \times n_y$ mesh with $n_x = n_y$. By the above argument, we have the basic recursion that

$$\theta(n_x) = 4\theta\left(\frac{n_x - 1}{2}\right) + \mathcal{O}(n_x^3). \quad (7.16)$$

Writing $\theta_k = \theta(2^k - 1)$, this gives the recurrence relation that $\theta_k = 4\theta_{k-1} + c8^k$ for some constant c associated with the bound on the $\mathcal{O}(n_x^3)$ term. Considering this expression for θ_{k+1} yields the two-term homogeneous recurrence relation

$$\theta_{k+1} - 12\theta_k + 32\theta_{k-1} = 0. \quad (7.17)$$

Following the theory of recurrence relations, we use the *ansatz* that $\theta_k = s^k$, which transforms this into the quadratic form

$$s^2 - 12s + 32 = 0. \quad (7.18)$$

This has two solutions, $s = 4$ and $s = 8$, giving the general solution of the recurrence relation as

$$\theta_k = c_1 4^k + c_2 8^k = \mathcal{O}((2^k - 1)^3). \quad (7.19)$$

Following the more careful count arrives at a similar conclusion, that $\theta(n_x) = \mathcal{O}(n_x^3)$, with a constant that is about 10. As remarked above, this matches the asymptotic lower bound of Hoffman, Martin, and Rose [137], proving that nested dissection is asymptotically optimal.

Unfortunately, the above example does not readily extend to the general case. However, some insights hold for certain classes of graphs. For example, with planar graphs with n nodes, the *planar separator theorem* [154] establishes the existence of a separator of size $\mathcal{O}(\sqrt{n})$ that separates the graph into two disconnected pieces, of which the larger can have no more than twice the number of nodes of the smaller. Furthermore, such separators can be found in linear time. From this, we arrive at a similar bound on the cost of factorization for certain classes of discretizations of two-dimensional problems on both regular and irregular grids. On regular $n_x \times n_y \times n_z$ grids in three dimensions with $n_x = n_y = n_z$, using geometric separators akin to the “central crosses” used in two dimensions gives an $\mathcal{O}(n_x^5)$ factorization cost. This improves (dramatically) on the $\mathcal{O}(n_x^7)$ cost of factorization for the same systems in lexicographical order by banded Gaussian elimination but remains impractical in terms of cost, even for modest values of n_x . This result is also much harder to generalize to unstructured meshes of three-dimensional geometries, motivating the study of alternative approaches. In particular, we now turn to iterative methods in order to move toward solvers that have optimal cost.

7.3 ■ Iterative methods

Objectives

Consider again a sparse linear system, $Au = f$, resulting from a finite-difference (or other) discretization on an $n_x \times n_y$ mesh in two dimensions with $n_x = n_y$, as in equation (7.2). In Section 7.2.3, we observed that the best complexity for solving such a system with sparse Gaussian elimination comes when using the nested dissection ordering, leading to $\mathcal{O}(n_x^3)$ cost. Yet, we use this algorithm to compute only $\mathcal{O}(n_x^2)$ values (the values of the solution at the n_x^2 mesh points), and there are only $\mathcal{O}(n_x^2)$ nonzero entries in the matrix, A , and right-hand side f . An important question is then whether we can use $\mathcal{O}(n_x^2)$ data to compute $\mathcal{O}(n_x^2)$

values in only $\mathcal{O}(n_x^2)$ cost. In this section, you will explore the basics of some simple *iterative* methods in order to achieve this goal, building on the ideas of *matrix splitting*. Specifically, you will consider the concept of polynomial iterative methods and the notion of preconditioning in order to accelerate convergence. Then, you will analyze methods based on matrix splitting, such as the Jacobi, Gauss–Seidel, and successive overrelaxation (SOR) schemes.

To start, we ask the following question:

- Can we construct an $\mathcal{O}(n_x^2)$ algorithm to solve $A\mathbf{u} = \mathbf{f}$ when the problem corresponds to a discretized PDE on an $n_x \times n_y$ mesh with $n_x = n_y$?

The short answer to this question is “yes,” with some caveats. Primarily, we remove the goal of *exactly* solving the linear system. This is reasonable, in general, due to the well-known results about solving linear systems in finite-precision arithmetic and the expected loss of accuracy proportional to the condition number of A . Furthermore, if we identify the vector $\mathbf{u}$ with an approximation to the solution of a discretized PDE, then it further motivates solving the linear system with no higher accuracy than the inherent discretization error. As a result, we recast the question:

- Can we find an approximation of the solution of $A\mathbf{u} = \mathbf{f}$ that is accurate to within discretization error with complexity of $\mathcal{O}(\text{nnz}(A))$, where $\text{nnz}(A)$ is the number of nonzero entries in the matrix A ?

This viewpoint enables the development of *iterative methods*, which are approaches for solving a linear system, $A\mathbf{u} = \mathbf{f}$, that take an approximation, $\mathbf{u}_k$, to $\mathbf{u}$ and form another approximation, $\mathbf{u}_{k+1}$. The goal is to develop a scheme where $\mathbf{u}_{k+1}$ improves upon $\mathbf{u}_k$ —e.g., $\|\mathbf{u}_{k+1} - \mathbf{u}\| \leq c\|\mathbf{u}_k - \mathbf{u}\|$ for a constant, $c < 1$, and in some norm $\|\cdot\|$. It is important to note that this is not a requirement to be an iterative method, and some iterative methods will, indeed, return worse approximations for some problems. Understanding when a particular method is expected (or guaranteed) to perform well, and when it might fail, is an important goal for this chapter.

7.3.1 ■ Polynomial iterative methods and preconditioning

In the following, we define $\mathbf{e}_k = \mathbf{u} - \mathbf{u}_k$ as the error in the approximation, $\mathbf{u}_k$, which is generally unknown and not computable, since it requires knowledge of the exact solution, $\mathbf{u}$. We focus on iterative methods of the general form

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \omega_k \delta \mathbf{u}_k \quad (7.20)$$

for scalar ω_k and vector $\delta \mathbf{u}_k$. In this case, the *ideal* choice for the update would satisfy $\omega_k \delta \mathbf{u}_k = \mathbf{e}_k$. Therefore, we typically look to approximate this ideal relation.

One common approach is to use the residual, $\mathbf{r}_k = \mathbf{f} - A\mathbf{u}_k$, as an approximation for $\mathbf{e}_k$. This is effective if A is close to the identity matrix in some sense, since

$$\mathbf{r}_k = \mathbf{f} - A\mathbf{u}_k = A\mathbf{u} - A\mathbf{u}_k = A\mathbf{e}_k, \quad (7.21)$$

so if $A \approx I$, then $\mathbf{r}_k \approx \mathbf{e}_k$. Considering an initial approximation or *guess*, $\mathbf{u}_0$, we iterate to arrive at

$$\mathbf{u}_1 = \mathbf{u}_0 + \omega_1(\mathbf{f} - A\mathbf{u}_0), \quad (7.22a)$$

$$\mathbf{u}_2 = \mathbf{u}_1 + \omega_2(\mathbf{f} - A\mathbf{u}_1), \quad (7.22b)$$

$$\mathbf{u}_3 = \mathbf{u}_2 + \omega_3(\mathbf{f} - A\mathbf{u}_2), \quad (7.22c)$$

$$\vdots$$

$$\mathbf{u}_k = \mathbf{u}_{k-1} + \omega_k(\mathbf{f} - A\mathbf{u}_{k-1}). \quad (7.22d)$$

To analyze the convergence behavior of this iteration, we look at the evolution of the *error*. Subtracting both sides of the above expression defining $\mathbf{u}_k$ from the true solution, $\mathbf{u}$, we have

$$\mathbf{u} - \mathbf{u}_k = \mathbf{u} - \mathbf{u}_{k-1} - \omega_k(\mathbf{f} - A\mathbf{u}_{k-1}) \quad (7.23a)$$

$$\Rightarrow \mathbf{e}_k = \mathbf{e}_{k-1} - \omega_k A \mathbf{e}_{k-1} = (I - \omega_k A) \mathbf{e}_{k-1} \quad (7.23b)$$

$$\Rightarrow \mathbf{e}_k = \left[\prod_{i=1}^k (I - \omega_i A) \right] \mathbf{e}_0. \quad (7.23c)$$

We often write the matrix in the last expression as $p_k(A) = \prod_{i=1}^k (I - \omega_i A)$, recognizing this as a degree- k polynomial in matrix A with the added property that $p_k(0) = I$, where we denote the all-zero matrix by 0. From this property, we identify approaches based on this methodology as *polynomial methods*. Several different families of polynomial methods exist, primarily differing in their choice of the coefficients, $\omega_1, \dots, \omega_k$, and the assumptions that they make on the matrix A .

As we will discover, by a judicious choice of the coefficients, we can design a polynomial method that matches the $\mathcal{O}(n_x^3)$ complexity of factorization using nested dissection ordering for the Poisson equation discretized on an $n_x \times n_y$ mesh with $n_x = n_y$. However, we cannot generally achieve $\mathcal{O}(n_x^2)$ complexity without additional ingredients. To achieve this, we need to combine polynomial methods with the concept of *preconditioning*. Consider

$$\underbrace{MA\mathbf{u} = M\mathbf{f}}_{\text{left preconditioning}} \text{ or } \underbrace{AM(M^{-1}\mathbf{u}) = \mathbf{f}}_{\text{right preconditioning}}, \quad (7.24)$$

where the *preconditioning matrix* (or *preconditioner*), M , is an invertible matrix chosen to accelerate convergence. If we (left) precondition the system and iterate as above, then the iteration becomes

$$\mathbf{u}_k = \mathbf{u}_{k-1} + \omega_k M(\mathbf{f} - A\mathbf{u}_{k-1}) \quad (7.25a)$$

$$\Rightarrow \mathbf{e}_k = (I - \omega_k MA) \mathbf{e}_{k-1} \quad (7.25b)$$

$$= p_k(MA) \mathbf{e}_0. \quad (7.25c)$$

We will revisit other preconditioning strategies in later sections. In particular, Chapter 11 is focused on the construction of two families of preconditioners that lead to excellent performance for matrices, A , that come from the discretization of elliptic PDEs.

For now, our focus is on choosing M with two properties. First, we want $M(\mathbf{f} - A\mathbf{u}_{k-1})$ to be relatively inexpensive to compute, at least compared to the (required) computation of the regular residual, $\mathbf{f} - A\mathbf{u}_{k-1}$. Second, we seek M such that the coefficients of the polynomial yield a small $\mathbf{e}_k$ for any $\mathbf{e}_0$:

$$\mathbf{e}_k = p_k(MA) \mathbf{e}_0 = \prod_{i=1}^k (I - \omega_i MA) \mathbf{e}_0. \quad (7.26)$$

This last point is noteworthy: if we are willing to accept assumptions on $\mathbf{e}_0$, then it can be trivial to select a polynomial that yields a small error. For example, if $\mathbf{e}_0$ is an eigenvector of A or MA , then setting ω_1 to be the inverse of the corresponding eigenvalue will result in zero error after only one step. Instead, our goal is to do this without assuming any *a priori* knowledge of the initial error.

At this point, we have two open questions that are necessarily intertwined: *how do we choose M* and *how do we choose $\omega_1, \omega_2, \dots$* ? We cannot completely separate these questions, as some “good choices” for M will rule out “good choices” of the polynomial coefficients and vice versa. Going forward, we need to be conscious of making trade-offs between these two questions. Consequently, we will iterate between these questions for the remainder of the chapter.

7.3.2 ■ Jacobi, Gauss–Seidel, and successive overrelaxation

One approach to developing iterative methods is through a “matrix splitting,” writing $A = M^{-1} - N$. Then, $A\mathbf{u} = \mathbf{f}$ is equivalent to $M^{-1}\mathbf{u} = N\mathbf{u} + \mathbf{f}$, which extends to an iteration of the form

$$M^{-1}\mathbf{u}_k = N\mathbf{u}_{k-1} + \mathbf{f} \quad (7.27a)$$

$$\Rightarrow \mathbf{u}_k = MN\mathbf{u}_{k-1} + M\mathbf{f} \quad (7.27b)$$

$$= \mathbf{u}_{k-1} + M(\mathbf{f} - A\mathbf{u}_{k-1}). \quad (7.27c)$$

We equate this iteration with a preconditioned polynomial method (cf. equation (7.25b)) with all weights equal, $\omega_i = 1$, and with preconditioner M —this is termed a *stationary* iterative method.

Remark 7.20: Preconditioner notation

There is notational inconsistency in the preconditioning literature between the choice of the preconditioner as M (as we have done in this book) and as M^{-1} (as it typically appears in the case of the literature on matrix splittings). Unfortunately, there are good arguments for both notations and both are used widely in the literature. Here, we generally adopt the former notation, since it allows the possibility that A and M are singular—for example, in the case of Neumann or periodic boundary conditions for the Poisson equation.

Matrix splittings allow us to refine the general questions posed above and to ask how to choose the matrix splitting so that M^{-1} is easy to invert (or so that it is easy to solve for $\mathbf{u}_k$ as defined above) and so that the stationary iteration thus defined is quick to converge. Many classical methods are based on addressing the invertibility but do not necessarily focus on convergence. Here, we follow that approach and consider classical iterations for which it is computationally efficient to compute $\mathbf{u}_k$ from $\mathbf{u}_{k-1}$ and then ask what conditions must be imposed on A so that we can guarantee fast convergence of the iteration.

We adopt the notation $A = D - L - U$, where D is a diagonal matrix and L and U are strictly lower- and upper-triangular matrices, respectively. With this, four standard choices for matrix splittings are as follows:

Jacobi: $M^{-1} = D$,

Weighted Jacobi: $M^{-1} = \frac{1}{\omega}D$,

Gauss–Seidel: $M^{-1} = D - L$, and

Successive Overrelaxation (SOR): $M^{-1} = \frac{1}{\omega}D - L$.

In iteration forms, these yield

$$\text{Jacobi: } \mathbf{u}_k = \mathbf{u}_{k-1} + D^{-1}(\mathbf{f} - A\mathbf{u}_{k-1}) \quad (7.28a)$$

$$= D^{-1}((L + U)\mathbf{u}_{k-1} + \mathbf{f}), \quad (7.28b)$$

$$\text{Weighted Jacobi: } \mathbf{u}_k = \mathbf{u}_{k-1} + \omega D^{-1}(\mathbf{f} - A\mathbf{u}_{k-1}) \quad (7.28c)$$

$$= (1 - \omega)\mathbf{u}_{k-1} + \omega D^{-1}((L + U)\mathbf{u}_{k-1} + \mathbf{f}), \quad (7.28d)$$

$$\text{Gauss-Seidel: } \mathbf{u}_k = \mathbf{u}_{k-1} + (D - L)^{-1}(\mathbf{f} - A\mathbf{u}_{k-1}) \quad (7.28e)$$

$$= (D - L)^{-1}(U\mathbf{u}_{k-1} + \mathbf{f}), \quad (7.28f)$$

$$\text{SOR: } \mathbf{u}_k = \mathbf{u}_{k-1} + \left(\frac{1}{\omega}D - L\right)^{-1}(\mathbf{f} - A\mathbf{u}_{k-1}) \quad (7.28g)$$

$$= \left(\frac{1}{\omega}D - L\right)^{-1} \left(\left(\left(\frac{1}{\omega} - 1 \right) D + U \right) \mathbf{u}_{k-1} + \mathbf{f} \right). \quad (7.28h)$$

The idea behind all of these methods is that systems governed by diagonal and lower-triangular matrices are easy to solve, so they naturally control the cost per iteration (relative to the cost of forming residuals with A). In fact, using the matrix-splitting form of $\mathbf{u}_k = M(\mathbf{f} + N\mathbf{u}_{k-1})$ allows iterations for Jacobi and Gauss-Seidel to be done at the same cost as a residual evaluation, while there is a slightly higher cost for SOR, since D appears in both M^{-1} and N .

To analyze convergence of these methods, we consider them in error-propagation form, writing

$$\mathbf{u}_k = \mathbf{u}_{k-1} + M(\mathbf{f} - A\mathbf{u}_{k-1}) \quad (7.29a)$$

$$\Rightarrow \mathbf{e}_k = (I - MA)\mathbf{e}_{k-1} \quad (7.29b)$$

$$= (I - MA)^2\mathbf{e}_{k-2} \quad (7.29c)$$

$$\vdots$$

$$= (I - MA)^k\mathbf{e}_0. \quad (7.29d)$$

Thus, the error evolves via a power iteration with the matrix $I - MA$. Suppose that this is diagonalizable, so that we find a full set of eigenvectors and eigenvalues:

$$(I - MA)\mathbf{v}_i = \lambda_i\mathbf{v}_i \text{ for } 1 \leq i \leq n. \quad (7.30)$$

This allows us to expand $\mathbf{e}_0$ in terms of the eigenvector basis, writing $\mathbf{e}_0 = \sum_{i=1}^n c_i\mathbf{v}_i$ for some coefficients, $\{c_i\}$, which then gives

$$\mathbf{e}_k = (I - MA)^k\mathbf{e}_0 = \sum_{i=1}^n c_i(I - MA)^k\mathbf{v}_i = \sum_{i=1}^n c_i\lambda_i^k\mathbf{v}_i. \quad (7.31)$$

From here, we observe that the rate of convergence is governed by the largest eigenvalue (in absolute value or modulus) of $I - MA$. One quantity that is used to assess convergence is the *asymptotic convergence factor*, which is defined as

$$\bar{\rho} := \lim_{k \rightarrow \infty} \left(\frac{\|\mathbf{e}_k\|}{\|\mathbf{e}_0\|} \right)^{\frac{1}{k}}. \quad (7.32)$$

In contrast, the *convergence rate* is defined as $\mu = -\ln \bar{\rho}$. Note that the asymptotic convergence factor tells us the relative factor by which we expect $\|\mathbf{e}_k\|$ to be decreased with each iteration, while the convergence rate tells us about the number of iterations needed to reduce the error by a fixed factor. Setting $\bar{\rho}^k = e^{-\tau}$ and taking logarithms of both sides, we see that it is expected to take $k = \tau/\mu$ iterations of the iterative method to achieve a reduction in the error norm by $e^{-\tau}$; similarly, it is expected to take $k = \tau \ln 10/\mu$ iterations to achieve a reduction in the error norm by $10^{-\tau}$. For the iterative scheme $\mathbf{u}_k = \mathbf{u}_{k-1} + M(\mathbf{f} - A\mathbf{u}_{k-1})$, with error propagation as $\mathbf{e}_k = (I - MA)^k\mathbf{e}_0$, the asymptotic convergence factor equals the spectral radius of $I - MA$:

$$\bar{\rho} = \rho(I - MA). \quad (7.33)$$

Example 7.21: Nondiagonalizable propagation matrix

The spectral radius $\rho(I - MA)$ is well defined when $I - MA$ is not diagonalizable; however, the convergence behavior of the resulting iteration may not be monotone. As an example, consider the linear system

$$\begin{bmatrix} 1 & \alpha \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \end{bmatrix} = \begin{bmatrix} 2 + \alpha \\ 1 \end{bmatrix} \quad (7.34)$$

for a large value of α and where $\mathbf{u} = \begin{bmatrix} u_0 \\ u_1 \end{bmatrix}$. Using the Jacobi iteration, we have that D is the 2×2 identity matrix, so

$$I - D^{-1}A = \begin{bmatrix} 0 & -\alpha \\ 0 & 0 \end{bmatrix}. \quad (7.35)$$

This is clearly not diagonalizable, having only one eigenvalue, of zero, with algebraic multiplicity of two, but geometric multiplicity of one. Knowing that the solution is $u_0 = 2$, $u_1 = 1$, we take an initial guess of the zero vector and find that

$$\mathbf{u}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow \mathbf{e}_0 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad (7.36a)$$

$$\mathbf{u}_1 = \begin{bmatrix} 2 + \alpha \\ 1 \end{bmatrix} \Rightarrow \mathbf{e}_1 = (I - D^{-1}A)\mathbf{e}_0 = \begin{bmatrix} -\alpha \\ 0 \end{bmatrix}, \quad (7.36b)$$

$$\mathbf{u}_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \Rightarrow \mathbf{e}_2 = (I - D^{-1}A)\mathbf{e}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \quad (7.36c)$$

Here, since $\rho(I - D^{-1}A) = 0$, we expect the asymptotic convergence factor to be zero, meaning that the exact solution is achieved in a finite number of steps (as it is). However, computing

$$\|\mathbf{e}_1\|/\|\mathbf{e}_0\| = |\alpha|/\sqrt{5} \quad (7.37)$$

reveals that the error after the first iteration is proportional to α , showing that the error can grow substantially (if α is large) before the asymptotic convergence factor becomes dominant.

Substantial theory has been developed to cover the convergence of these matrix splittings, primarily focused on bounding $\rho(I - MA)$ for certain classes of matrices. For instance, a general result about the spectrum of a matrix, known as Geršgorin's theorem, can be used. We note that this result has already appeared as Theorem 3.9 in Section 3.3.2, but we restate it here with a slight change in notation to match what is used in this chapter. We include the proof for convenience.

Theorem 7.22: Geršgorin's theorem [226, Thm. 1.11]

Let A be an $n \times n$ matrix and $\Lambda_i = \sum_{j=1, j \neq i}^n |a_{i,j}|$ for $1 \leq i \leq n$. If λ is an eigenvalue of A , then $\exists r$, $1 \leq r \leq n$, such that

$$|\lambda - a_{r,r}| \leq \Lambda_r. \quad (7.38)$$

Proof. Let λ be an eigenvalue of A , and let $\mathbf{u} \neq \mathbf{0}$ be the eigenvector associated with λ , so $A\mathbf{u} = \lambda\mathbf{u}$, normalized so that $\max_{1 \leq i \leq n} |u_i| = 1$. Then, since $A\mathbf{u} = \lambda\mathbf{u}$, we have

$$(\lambda - a_{i,i})u_i = \sum_{\substack{j=1 \\ j \neq i}}^n a_{i,j}u_j \text{ for } 1 \leq i \leq n. \quad (7.39)$$

Choose r such that $|u_r| = 1$. Then,

$$|\lambda - a_{r,r}| \leq \sum_{\substack{j=1 \\ j \neq r}}^n |a_{r,j}| |u_j| \leq \sum_{\substack{j=1 \\ j \neq r}}^n |a_{r,j}| = \Lambda_r. \quad (7.40)$$

□

Two results immediately follow from Theorem 7.22 that can be useful in analyzing the spectral properties of a problem. First, Corollary 7.23 identifies sets (known as Geršgorin circles) or unions of sets where eigenvalues must lie. With this, Corollary 7.24 establishes a bound on the spectral radius—a useful tool that is used to gather a rough estimate based on a quick inspection of the matrix entries.

Corollary 7.23: Geršgorin circles [226, Thm. 1.11]

Under the assumptions and definitions of Theorem 7.22, all eigenvalues, λ , of A lie in the set

$$\bigcup_{i=1}^n \{z \mid |z - a_{i,i}| \leq \Lambda_i\}. \quad (7.41)$$

Corollary 7.24: Geršgorin bound on $\rho(A)$ [226, Cor. 1.12]

Let A be an $n \times n$ matrix. Then,

$$\rho(A) \leq \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{i,j}|. \quad (7.42)$$

Geršgorin's theorem has several important consequences, one of which is that it can be used to prove convergence of the Jacobi iteration for a class of matrices that arise in some PDE discretizations. To this end, we introduce the concept of diagonal dominance in Definition 7.25. Convergence of the Jacobi iteration for diagonally dominant matrices is then detailed in Theorem 7.26.

Definition 7.25: Diagonally dominant matrices (see also [194, Def. 4.5])

The $n \times n$ matrix, A , is strictly diagonally dominant if

$$|a_{i,i}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{i,j}| \text{ for } 1 \leq i \leq n. \quad (7.43)$$

Theorem 7.26: Jacobi convergence [194, Thm. 4.9]

If the $n \times n$ matrix, A , is strictly diagonally dominant, then $\rho(I - D^{-1}A) < 1$.

Proof. First, we write the entries of $D^{-1}A$ in terms of A : $(D^{-1}A)_{i,j} = a_{i,i}^{-1}a_{i,j}$. This gives that

$$(I - D^{-1}A)_{i,j} = \begin{cases} 0 & \text{if } i = j, \\ -a_{i,i}^{-1}a_{i,j} & \text{if } i \neq j. \end{cases} \quad (7.44)$$

Then, by the corollary to Geršgorin's theorem, Corollary 7.24,

$$\rho(I - D^{-1}A) \leq \max_{1 \leq i \leq n} \sum_{\substack{j=1 \\ j \neq i}}^n |a_{i,i}^{-1} a_{i,j}| = \max_{1 \leq i \leq n} \frac{1}{|a_{i,i}|} \sum_{\substack{j=1 \\ j \neq i}}^n |a_{i,j}| < 1. \quad (7.45)$$

□

While Theorem 7.26 is useful in some cases, it is rather limited. For example, it applies only to the Jacobi iteration and only to strictly diagonally dominant matrices, which excludes its use in important cases such as the discretization of $-u'' = f$ by finite-difference or finite-element techniques. To handle these, in the next section, we introduce some more general classes of matrices and present several key theorems in this direction, giving convergence proofs and bounds on convergence rates.

7.4 ■ Convergence theory for stationary iterative methods

Objectives

In Theorem 7.26, we prove convergence of the Jacobi method for diagonally dominant matrices. In this section, you will dive a bit further into how well the Jacobi, weighted Jacobi, Gauss–Seidel, and SOR methods introduced in Section 7.3 perform for a wider class of problems. Specifically, you will consider various types of matrices that may result from standard discretizations of PDEs, including the notion of an M-matrix. Then, you will analyze the convergence properties of Jacobi, Gauss–Seidel, and SOR on such matrices.

In order to analyze the convergence properties of the splitting methods introduced in Section 7.3, we first introduce some important classes of matrices. As in Section 3.3, we note the connection between matrix A being strictly diagonally dominant and positive definiteness, repeating Definition 3.13 in Definition 7.27. Thus, Geršgorin's theorem is a common tool for verifying positive (or negative) definiteness, particularly for symmetric matrices, where positive definiteness is equivalent to having all positive eigenvalues. Additionally, we consider nonnegative and irreducible matrices in Definitions 7.28 and 7.29.

Definition 7.27: Positive-definite and negative-definite matrices [194, Sec. 1.11]

Matrix A is said to be positive definite if $\mathbf{u}^T A \mathbf{u} > 0$ for all $\mathbf{u} \neq \mathbf{0}$. If $\mathbf{u}^T A \mathbf{u} < 0$ for all $\mathbf{u} \neq \mathbf{0}$, we say that A is negative definite. If either of these holds without strict inequality, we say that A is (positive or negative) semidefinite.

Definition 7.28: Nonnegative matrices [226, Def. 2.1]

An $n \times n$ matrix, A , is nonnegative if $a_{i,j} \geq 0$ for $1 \leq i, j \leq n$.

Definition 7.29: Reducible matrices [226, Def. 1.15]

For $n \geq 2$, the $n \times n$ matrix, A , is reducible if there is a permutation matrix, P , such that $P^T A P$ is block upper triangular. That is,

$$P^T A P = \begin{bmatrix} A_{1,1} & A_{1,2} \\ 0 & A_{2,2} \end{bmatrix}, \quad (7.46)$$

where $A_{1,1}$ is an $r \times r$ matrix and $A_{2,2}$ is an $(n-r) \times (n-r)$ matrix. If no such permutation exists, then A is irreducible.

Assuming that a matrix is irreducible is important because it rules out many special cases, allowing us to prove stronger results. An equivalent definition of irreducibility comes from considering a directed graph representation of the nonzero structure in A , where edge $(i, j) \in E$ if and only if $a_{i,j} \neq 0$. Then, A is irreducible if and only if there is a directed path from any node in the graph of the matrix to any other node in the graph. Assuming irreducibility gives us many more tools to use in proving convergence results, including Theorem 7.30.

Theorem 7.30: Perron–Frobenius [226, Thm. 2.7]

Let B be an irreducible, nonnegative $n \times n$ matrix. Then,

1. B has a positive real eigenvalue equal to $\rho(B)$;
2. $\rho(B)$ is associated with an eigenvector, $\mathbf{u}$, such that $u_i > 0$ for $1 \leq i \leq n$;
3. $\rho(B)$ increases when any entry in B increases; and
4. $\rho(B)$ is a simple eigenvalue of B .

We omit the proof, since it does not give direct insight into the performance of the iterative methods that we are interested in.

Clearly, this theorem is not applicable directly to our discretization matrices, since they are not expected to be nonnegative. However, a common property of many of our discretization matrices leads to a nonnegative iteration matrix for Jacobi. These are called M-matrices (see Definition 7.31).

Definition 7.31: M-matrices [41, Ch. 6]

The $n \times n$ matrix, A , is an M-matrix if $a_{i,j} \leq 0$ for all $i \neq j$ and A is positive definite.

One consequence of A being positive definite is that its diagonal entries must be positive. As a result, if A is an irreducible M-matrix, then $B = I - D^{-1}A$ is a nonnegative irreducible matrix, with the additional property that $b_{i,i} = 0$ for all i . Thus, we can write $B = L + U$ for strictly lower- and upper-triangular matrices, L and U , respectively. (Note that these are not the same as the matrices L and U from the matrix splitting $A = D - L - U$ —see below.) This leads to some useful properties.

Definition 7.32: Jacobi convergence functions [226, Def. 3.6]

Let L and U be strictly lower- and upper-triangular matrices, respectively, with $B = L + U$. For any $\sigma \geq 0$, define

$$m(\sigma) = \rho(\sigma L + U) \text{ and } n(\sigma) = \rho(L + \sigma U). \quad (7.47)$$

The functions $m(\sigma)$ and $n(\sigma)$ prove useful in analyzing convergence of the Jacobi and Gauss–Seidel iterations. Some notable properties of these functions that are easy to check are that

- $m(0) = n(0) = 0$,
- $m(1) = n(1) = \rho(L + U) = \rho(B)$, and
- $n(\sigma) = \sigma m(1/\sigma)$ for $\sigma > 0$.

A consequence of Definition 7.32 is Lemma 7.33.

Lemma 7.33: Jacobi convergence [226, Lem. 3.7]

Let $B = L + U$ be nonnegative and irreducible with $b_{i,i} = 0$ for all i , and let L and U be strictly lower- and upper-triangular matrices, respectively. Then, $m(\sigma)$ and $n(\sigma)$ are strictly increasing for $\sigma \geq 0$.

Proof. Since B is an irreducible matrix, neither L nor U can be the zero matrix, implying that $\rho(B) > 0$. In addition, the matrix $M(\sigma) = \sigma L + U$ is also irreducible for $\sigma > 0$ and is nonnegative. Thus, by the Perron–Frobenius theorem, $m(\sigma) = \rho(M(\sigma))$ is increasing for $\sigma \geq 0$. Similarly, we can define $N(\sigma) = L + \sigma U$ and conclude that $n(\sigma) = \rho(N(\sigma))$ is increasing for $\sigma \geq 0$. $\square$

This result also holds in the case that B is not irreducible and $\rho(B) > 0$. If $\rho(B) = 0$, however, then $m(\sigma) = n(\sigma) = 0$ for all $\sigma \geq 0$. An important question now is how to relate $B = I - D^{-1}A$ to the Gauss–Seidel iteration matrix. This is easily done but, unfortunately, introduces a bit of conflicting notation. If $B = I - D^{-1}A = L + U$, then we can write $A = D(I - L - U) = D(I - L) - DU$, where L and U are not the same matrices as those defined in the matrix splitting of A . We define the Gauss–Seidel iteration matrix to be $\mathcal{L}_1 = (D(I - L))^{-1}DU = (I - L)^{-1}U$. With this, we can state Theorem 7.34.

Theorem 7.34: Stein–Rosenberg [226, Thm. 3.8]

Let $B = L + U$ be nonnegative, with $b_{i,i} = 0$ for all i , and L and U be strictly lower- and upper-triangular matrices, respectively. Let $\mathcal{L}_1 = (I - L)^{-1}U$. Then one, and only one, of the following holds:

1. $\rho(B) = \rho(\mathcal{L}_1) = 0$;
2. $0 < \rho(\mathcal{L}_1) < \rho(B) < 1$;
3. $\rho(B) = \rho(\mathcal{L}_1) = 1$; and
4. $1 < \rho(B) < \rho(\mathcal{L}_1)$.

Proof. We prove only the second statement, and only in the case when B is irreducible (since we use the lemma above). Proof of the fourth statement is similar, and the other two follow as special cases (e.g., when A is singular with nonzero diagonal entries, the third case follows automatically).

First, note that

$$\mathcal{L}_1 = (I - L)^{-1}U = (I + L + L^2 + \cdots + L^{n-1})U \quad (7.48)$$

is nonnegative since L and U are. Here, we have used the truncated Neumann series expansion for $(I - L)^{-1}$, since we must have $L^n = 0$ for any strictly lower-triangular matrix L .

Considering the second case, assume that $0 < \rho(B) < 1$. From the Neumann series expansion above, we conclude that $\rho(\mathcal{L}_1) > 0$ as well. Now, let $\mathbf{u}$ be an eigenvector of $\mathcal{L}_1$ so that

$$(I - L)^{-1}U\mathbf{u} = \lambda\mathbf{u} \quad (7.49a)$$

$$\Rightarrow U\mathbf{u} = \lambda(I - L)\mathbf{u} \quad (7.49b)$$

$$\Rightarrow (\lambda L + U)\mathbf{u} = \lambda\mathbf{u} \text{ and } (L + (1/\lambda)U)\mathbf{u} = \mathbf{u}. \quad (7.49c)$$

Thus, we conclude that $m(\lambda) \geq \lambda$ and that $n(1/\lambda) \geq 1$ for any eigenvalue of $\mathcal{L}_1$. Since $\rho(B) = n(1) < 1$, we obtain that $1/\lambda > 1$, or $\lambda < 1$. However, that means that $m(\lambda) < m(1) = \rho(B)$ for all eigenvalues, λ , of $\mathcal{L}_1$. Thus, $\rho(\mathcal{L}_1) < \rho(B)$. $\square$

A major consequence of Theorem 7.34, in the case of M-matrices, is that either both Jacobi and Gauss–Seidel are convergent (with Gauss–Seidel converging faster than Jacobi, unless their spectral radii are both zero) or they both are not (and, in the case of divergence, Gauss–Seidel diverges faster than Jacobi). Similar relations can be proven for SOR, where we make use of a similar transformation to write the SOR iteration matrix in terms of the splitting $B = L + U$ of the Jacobi iteration matrix. As above, recovering that $A = D - DL - DU$ (where D is the diagonal of A), we can write the SOR iteration matrix as

$$\mathcal{L}_\omega := \left(\frac{1}{\omega} D - DL \right)^{-1} \left(\left(\frac{1}{\omega} - 1 \right) D + DU \right) \quad (7.50a)$$

$$= \left(\frac{1}{\omega} I - L \right)^{-1} \left(\left(\frac{1}{\omega} - 1 \right) I + U \right) \quad (7.50b)$$

$$= (I - \omega L)^{-1} ((1 - \omega) I + \omega U). \quad (7.50c)$$

This leads to Theorem 7.35.

Theorem 7.35: Kahan's theorem [226, Thm. 3.11]

Let $B = L + U$ be an $n \times n$ matrix with $b_{i,i} = 0$ for all i . Then, for any $\omega \in \mathbb{C}$, $\rho(\mathcal{L}_\omega) \geq |\omega - 1|$, with equality only if $|\lambda| = |\omega - 1|$ for all eigenvalues, λ , of $\mathcal{L}_\omega$.

Proof. Consider the characteristic polynomial of $\mathcal{L}_\omega$, $\phi(\lambda) = \det(\lambda I - \mathcal{L}_\omega)$. Since L is strictly lower triangular, $\det(I - \omega L) = 1$. Thus,

$$\phi(\lambda) = \det(I - \omega L) \det(\lambda I - \mathcal{L}_\omega) \quad (7.51a)$$

$$= \det(\lambda(I - \omega L) - ((1 - \omega)I + \omega U)) \quad (7.51b)$$

$$= \det((\lambda + \omega - 1)I - \omega \lambda L - \omega U). \quad (7.51c)$$

In addition,

$$\phi(0) = \det(-\mathcal{L}_\omega) = (-1)^n \prod_{i=1}^n \lambda_i(\mathcal{L}_\omega) \quad (7.52a)$$

$$= \det((\omega - 1)I - \omega U) = (\omega - 1)^n. \quad (7.52b)$$

Thus, $\rho(\mathcal{L}_\omega) = \max_{1 \leq i \leq n} |\lambda_i(\mathcal{L}_\omega)| \geq |\omega - 1|$, and equality holds only if $|\lambda_i(\mathcal{L}_\omega)| = |\omega - 1|$ for all i . $\square$

Theorem 7.35 establishes an important fact: when using a real weight, ω , SOR is only convergent when $0 < \omega < 2$. However, the theorem does not provide any more insight into when SOR is actually convergent, since it only provides a lower bound on the eigenvalues of the iteration matrix. Under the additional assumption that A is Hermitian, a better result can be proven. Here, we make a slight generalization to the splitting of Hermitian matrix $A = D - L - L^H$ for diagonal D and strictly lower-triangular L (where superscript H denotes the Hermitian (conjugate)

transpose). This does not complicate the proof but provides a result that is naturally applicable to block-structured SOR-type relaxation. We emphasize that C below in Theorem 7.36 need not be a diagonal matrix, and that E need not be strictly lower triangular, although the theorem also holds in that case.

Theorem 7.36: Ostrowski's theorem [226, Thm. 3.12]

Let $A = C - E - E^H$ be Hermitian, where C is Hermitian and positive definite and $C - \omega E$ is nonsingular for $0 < \omega < 2$. Define $\mathcal{L}_\omega := (C - \omega E)^{-1}((1 - \omega)C + \omega E^H)$. Then, $\rho(\mathcal{L}_\omega) < 1$ if and only if A is positive definite and $0 < \omega < 2$.

Proof. Let e_0 be an arbitrary nonzero vector, and define $e_{m+1} = \mathcal{L}_\omega e_m$ for $m \geq 0$. Given the definition of $\mathcal{L}_\omega$, this is equivalent to

$$(C - \omega E)e_{m+1} = ((1 - \omega)C + \omega E^H)e_m. \quad (7.53)$$

Writing $y_m = e_m - e_{m+1}$ for $m \geq 0$, we deduce the following relations. First, since $(C - \omega E)(e_m - y_m) = ((1 - \omega)C + \omega E^H)e_m$, we have

$$(C - \omega E)y_m = (C - \omega E)e_m - (1 - \omega)Ce_m - \omega E^H e_m = \omega A e_m. \quad (7.54)$$

Similarly, since $(C - \omega E)(e_{m+1}) = ((1 - \omega)C + \omega E^H)(e_{m+1} - y_m)$, we have

$$\omega A e_{m+1} = ((1 - \omega)C + \omega E^H)y_m. \quad (7.55)$$

Taking inner products with these relations leads to

$$y_m^H (C - \omega E)y_m = \omega y_m^H A e_m, \quad (7.56a)$$

$$e_m^H (C - \omega E)y_m = \omega e_m^H A e_m, \quad (7.56b)$$

$$\omega e_{m+1}^H A e_{m+1} = e_{m+1}^H ((1 - \omega)C + \omega E^H)y_m. \quad (7.56c)$$

Subtracting equation (7.56c) from equation (7.56b) gives

$$\omega (e_m^H A e_m - e_{m+1}^H A e_{m+1}) = e_m^H (C - \omega E)y_m - e_{m+1}^H ((1 - \omega)C + \omega E^H)y_m \quad (7.57a)$$

$$= e_m^H (C - \omega E)y_m - e_m^H ((1 - \omega)C + \omega E^H)y_m + y_m^H ((1 - \omega)C + \omega E^H)y_m \quad (7.57b)$$

$$= \omega e_m^H A y_m + y_m^H ((1 - \omega)C + \omega E^H)y_m. \quad (7.57c)$$

Now, using equation (7.56a) on the right-hand side, we get

$$\omega (e_m^H A e_m - e_{m+1}^H A e_{m+1}) = (2 - \omega)y_m^H C y_m. \quad (7.58)$$

Assume that A is positive definite and $0 < \omega < 2$. Take e_0 to be an eigenvector of $\mathcal{L}_\omega$ with eigenvalue λ , so that $e_1 = \lambda e_0$ and $y_0 = (1 - \lambda)e_0$. Substituting this in the above yields

$$(1 - |\lambda|^2) e_0^H A e_0 = \frac{2 - \omega}{\omega} |1 - \lambda| 2 e_0^H C e_0. \quad (7.59)$$

Note first that we cannot have $\lambda = 1$, since this would give y_0 as the all-zero vector and, from equation (7.56b), $e_0^H A e_0 = 0$ for a nonzero vector e_0 , contradicting the positive

definiteness of A . Thus, the right-hand side of equation (7.59) must be strictly positive, since $0 < \omega < 2$. This implies that the left-hand side of equation (7.59) is also strictly positive, so $1 - |\lambda|^2 > 0$, implying that $|\lambda| < 1$ and, consequently, $\rho(\mathcal{L}_\omega) < 1$.

Conversely, assume that $\rho(\mathcal{L}_\omega) < 1$ for $0 < \omega < 2$. Then, for any e_0 , $\lim_{m \rightarrow \infty} e_m = 0$ and $\lim_{m \rightarrow \infty} e_m^H A e_m = 0$. Furthermore, since C is positive definite,

$$e_m^H A e_m = e_{m+1}^H A e_{m+1} + \frac{2-\omega}{\omega} \mathbf{y}_m^H C \mathbf{y}_m \geq e_{m+1}^H A e_{m+1}. \quad (7.60)$$

Suppose that A is not positive definite. Then, there exists a nonzero e_0 such that $e_0^H A e_0 = 0$, but since $\rho(\mathcal{L}_\omega) < 1$, we must have $\mathbf{y}_0 = e_0 - \mathcal{L}_\omega e_0 \neq 0$. This implies that $\mathbf{y}_0^H C \mathbf{y}_0 > 0$. From the above,

$$e_m^H A e_m \leq e_1^H A e_1 < e_0^H A e_0 \quad (7.61)$$

for all $m \geq 1$, contradicting the limit on $e_m^H A e_m$. Consequently, A must be positive definite. $\square$

In the notation of the theorem, taking C to be the diagonal of A and E to be the strictly lower-triangular part results in the standard SOR algorithm. A notable result, originally due to Reich [186], is given in Corollary 7.37 below. In the standard case of a Hermitian matrix with positive diagonal entries, this implies that Gauss–Seidel is convergent if and only if the matrix is also positive definite.

Corollary 7.37: Reich’s theorem [226, Cor. 3.13]

Let $A = C - E - E^H$ be Hermitian, with C Hermitian and positive definite and $C - E$ nonsingular. Define $\mathcal{L}_1 := (C - E)^{-1} E^H$. Then, $\rho(\mathcal{L}_1) < 1$ if and only if A is positive definite.

In the following section, we present some more results on the convergence of Gauss–Seidel, Jacobi, and SOR, but they are rather technical and require more significant assumptions than those presented herein. For this reason, we defer these results to an “optional” section that can easily be skipped without impacting later sections in the book.

7.5 - Optimizing convergence of SOR

Objectives

In the previous sections, we have looked at a few cases where we can make concrete statements about convergence of the Jacobi, Gauss–Seidel, and SOR iterations. Before the development of the multigrid and domain decomposition iterations discussed in Chapter 11, a major research question was how quickly SOR could be made to converge and, in particular, if it could lead to an iteration that could solve typical discretizations of elliptic PDEs in $\mathcal{O}(n)$ time (for a problem with n degrees of freedom). In this section, you will focus on answering this question, which requires more quantitative theory than was developed previously.

To further understand the convergence of SOR, it is best to put more restrictions on the class of matrices that we consider. There are many ways of specifying properties of the discretization matrix, A , that achieve this goal. While these are of great historical interest, we omit them here in the interest of space. At their root, many of these approaches aim to find properties of A that

give some freedom in the subsequent convergence analysis, to help in finding the optimal value of ω . This work can be traced back to that of Young [237] and was considered state of the art for many years, but it has since been surpassed by performance of Krylov methods and modern techniques, such as multigrid and domain decomposition (see Chapter 11). We begin by defining one class of matrices for which we can say more about convergence than in the previous sections.

Definition 7.38: Property A^π [14]

A matrix, A , has property A^π if there is a permutation matrix, P , such that $P^T A P$ is block tridiagonal,

$$P^T A P = \begin{bmatrix} D_1 & U_1 & 0 & \cdots & \cdots \\ L_1 & D_2 & U_2 & 0 & \cdots \\ 0 & L_2 & D_3 & U_3 & \\ & & \ddots & \ddots & \ddots \\ & & & L_{r-2} & D_{r-1} & U_{r-1} \\ & & & & L_{r-1} & D_r \end{bmatrix}, \quad (7.62)$$

where each D_i , $1 \leq i \leq r$, is nonsingular.

Standard discretizations of our model Poisson problem have this property already, with $P = I$, when ordered lexicographically; however, we can also order the two-dimensional Poisson problem “by diagonal” to achieve a reordering with further properties. To define the “by-diagonal” permutation for a problem discretized on an $n_x \times n_y$ tensor-product mesh in 2D, we define block i of the permutation such that all grid points (k, ℓ) for $1 \leq k \leq n_x$ and $1 \leq \ell \leq n_y$ satisfy $k + \ell = i + 1$. As is easily seen for a five-point finite-difference discretization on this mesh, this permutation leads to matrix $P^T A P$ that has diagonal matrices D_i . This greatly simplifies the analysis that follows, which we can see by considering the Jacobi and SOR iterations on matrix $P^T A P$.

Going back to the notation of the matrix splitting $A = D - L - U$, where, as usual, D is diagonal and L and U are strictly lower and upper triangular, respectively, we write

$$I - D^{-1}A = P(I - P^T D^{-1} P P^T A P) P^T = P(I - (P^T D P)^{-1} (P^T A P)) P^T \quad (7.63)$$

using the property of a permutation matrix that $P P^T = P^T P = I$. Thus, if the diagonal of $P^T A P$ is $P^T D P$, then the spectral radius of the Jacobi iteration matrix is the same for both A and $P^T A P$. This is always the case, however, since permutations map diagonal entries of A to diagonal entries of $P^T A P$. For SOR, in contrast, more is required. Now, we write

$$I - \left(\frac{1}{\omega} D - L \right)^{-1} A = P \left(I - \left(\frac{1}{\omega} P^T D P - P^T L P \right)^{-1} P^T A P \right) P^T \quad (7.64)$$

and see that the spectral radius of the SOR iteration matrix is independent of P only when the lower-triangular part of $P^T A P$ is $P^T L P$. This, unfortunately, is not always the case; however, we can order the blocks within the “by-diagonal” permutation above so that it does hold for our five-point finite-difference model problem. In general, we call permutations such that the lower-triangular part of $P^T A P$ is $P^T L P$ “nonmigratory.”

In the remainder of this section, we assume that our matrices have property A^π with the additional properties that each D_i is diagonal and that the permutation is nonmigratory. With this, we can work directly with the analysis of SOR convergence for $P^T A P = D - L - U$,

where

$$D = \begin{bmatrix} D_1 & 0 & & & \\ 0 & D_2 & 0 & & \\ & 0 & D_3 & 0 & \\ & & \ddots & \ddots & \ddots \\ & & & D_r \end{bmatrix}, \quad (7.65a)$$

$$L = \begin{bmatrix} 0 & & & & \\ L_1 & 0 & & & \\ 0 & L_2 & 0 & & \\ & \ddots & \ddots & & \\ & & & L_{r-1} & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & U_1 & & & \\ & 0 & U_2 & & \\ & & \ddots & \ddots & \\ & & & U_{r-1} & \\ & & & 0 & \end{bmatrix}. \quad (7.65b)$$

Below, when we assume that A has property A^π , it should be interpreted to mean that all of these properties hold and that A has already been permuted into $P^T A P$ by such a permutation.

Lemma 7.39: SOR scaling lemma [237, Thm. 2.1]

If $A = D - L - U$ has property A^π , then

$$\det \left(\alpha L + \frac{1}{\alpha} U - kD \right) \quad (7.66)$$

is independent of α for all $\alpha \neq 0$ and any k .

Proof. Define I_i to be the identity matrix with the same size as D_i for $1 \leq i \leq r$, and let

$$Q_\alpha = \begin{bmatrix} I_1 & 0 & & & \\ 0 & \alpha I_2 & 0 & & \\ & 0 & \alpha^2 I_3 & 0 & \\ & & \ddots & \ddots & \ddots \\ & & & \alpha^{r-1} I_r \end{bmatrix}. \quad (7.67)$$

Then, $Q_\alpha^{-1} (\alpha L + \frac{1}{\alpha} U - kD) Q_\alpha = L + U - kD$ because of the block-tridiagonal structure of A . Thus,

$$\det \left(\alpha L + \frac{1}{\alpha} U - kD \right) = \det (L + U - kD) \quad (7.68)$$

and is independent of α . $\square$

Theorem 7.40: Young's theorem [237, Thm. 2.2]

Let $A = D - L - U$ have property A^π , and define $B = D^{-1}(L + U)$ and $\mathcal{L}_\omega = (D - \omega L)^{-1}((1 - \omega)D + \omega U)$. Then,

1. if μ is any eigenvalue of B of multiplicity p , then $-\mu$ is also an eigenvalue of B of multiplicity p ;
2. $\lambda \in \mathbb{C}$ satisfies

$$(\lambda + \omega - 1)^2 = \omega^2 \mu^2 \lambda, \quad (7.69)$$

for some eigenvalue, μ , of B if and only if λ satisfies

$$\lambda + \omega - 1 = \omega\mu\lambda^{1/2} \quad (7.70)$$

for some eigenvalue, μ , of B ;

3. if λ satisfies either equation (7.69) or equation (7.70), then it is an eigenvalue of $\mathcal{L}_\omega$; and
4. if λ is an eigenvalue of $\mathcal{L}_\omega$, then there exists an eigenvalue, μ , of B such that equations (7.69) and (7.70) hold.

Proof. We prove each result below.

1. From Lemma 7.39,

$$\det(L + U + \mu D) = \det(-L - U + \mu D) = (-1)^n \det(L + U - \mu D). \quad (7.71)$$

Therefore, $\det(B - \mu I)$ must take the form of a power of μ times a polynomial in μ^2 .

2. If μ is an eigenvalue of B and λ satisfies equation (7.70), then it clearly satisfies equation (7.69). If, on the other hand, λ satisfies equation (7.69), then $\lambda + \omega - 1 = \pm\omega\mu\lambda^{1/2}$. Since both $\pm\mu$ are eigenvalues of B , then one of them yields equation (7.70).
3. λ is an eigenvalue of $\mathcal{L}_\omega$ if and only if $\det(\lambda I - \mathcal{L}_\omega) = 0$. By properties of determinants,

$$\det(\lambda I - \mathcal{L}_\omega) = \det(I - \omega D^{-1}L)^{-1} \det(\lambda(I - \omega D^{-1}L) - ((1 - \omega)I + \omega D^{-1}U)) \quad (7.72a)$$

$$= \det(-\omega D^{-1}U - \lambda\omega D^{-1}L + (\lambda + \omega - 1)I). \quad (7.72b)$$

From this, note that $\lambda = 0$ can only be an eigenvalue of $\mathcal{L}_\omega$ if $\omega = 1$, since

$$\det(-\mathcal{L}_\omega) = \det(-\omega D^{-1}U + (\omega - 1)I) = (\omega - 1)^n. \quad (7.73)$$

Thus, $\lambda = 0$ is an eigenvalue of $\mathcal{L}_\omega$ if and only if it satisfies both equation (7.69) and equation (7.70).

Furthermore, if $\lambda \neq 0$, then

$$\det(\lambda I - \mathcal{L}_\omega) = (-\omega)^n \lambda^{n/2} \det\left(\lambda^{1/2} D^{-1}L + \lambda^{-1/2} D^{-1}U - \frac{\lambda + \omega - 1}{\omega\lambda^{1/2}} I\right) \quad (7.74a)$$

$$= (-\omega)^n \lambda^{n/2} \det\left(D^{-1}L + D^{-1}U - \frac{\lambda + \omega - 1}{\omega\lambda^{1/2}} I\right). \quad (7.74b)$$

Thus, if λ satisfies equation (7.70) for some eigenvalue, μ , of $B = D^{-1}(L + U)$, then $\det(\lambda I - \mathcal{L}_\omega) = 0$ and λ is an eigenvalue of $\mathcal{L}_\omega$.

4. Conversely, if $\lambda \neq 0$ is an eigenvalue of $\mathcal{L}_\omega$, then

$$\det\left(D^{-1}(L + U) - \frac{\lambda + \omega - 1}{\omega\lambda^{1/2}} I\right) = 0, \quad (7.75)$$

and equation (7.70) holds.

□

Corollary 7.41: Jacobi and Gauss–Seidel convergence [226, Cor. 4.6]

Under the assumptions of Theorem 7.40, when $\omega = 1$, the eigenvalues of the Gauss–Seidel iteration matrix, $\mathcal{L}_1$, satisfy either $\lambda = 0$ or $\lambda = \mu^2$ when μ is an eigenvalue of B . Consequently, $\rho(\mathcal{L}_1) = (\rho(B))^2$.

Theorem 7.40 and its corollary provide key insight into the convergence rates of SOR and Gauss–Seidel, in relation to that of Jacobi. In particular, from Corollary 7.41, we see that (asymptotically) we expect Gauss–Seidel to converge for a given problem in half as many iterations as Jacobi. This is significant, since it can be implemented with the same cost per iteration as Jacobi. Theorem 7.42 tells us how much better using SOR can be than this.

Theorem 7.42: Optimal SOR convergence [226, Sec. 4.3]

Assume that $A = D - L - U$ has property A^π and that $B = D^{-1}(L + U)$ has only real eigenvalues. Then, $\rho(\mathcal{L}_\omega) < 1$ if and only if $\rho(B) < 1$ and $0 < \omega < 2$. Furthermore, $\rho(\mathcal{L}_\omega)$ is minimized for

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - (\rho(B))^2}}, \quad (7.76)$$

with $\rho(\mathcal{L}_{\omega_{\text{opt}}}) = \omega_{\text{opt}} - 1$.

Proof. From Theorem 7.40, we know that λ is an eigenvalue of $\mathcal{L}_\omega$ if $\lambda - \omega\mu\lambda^{1/2} + (\omega - 1) = 0$, where μ is an eigenvalue of B . Treating this as a quadratic equation for $\lambda^{1/2}$, we conclude that $|\lambda| < 1$ if and only if $|\omega - 1| < 1$ and $|\omega\mu| < \omega$. These conditions hold if and only if $0 < \omega < 2$ and $|\mu| < 1$. Thus, for given values of ω and μ ,

$$\lambda(\mu, \omega) = \left(\frac{1}{2}\mu\omega \pm \left(1 - \omega + \left(\frac{1}{2}\omega\mu \right)^2 \right)^{1/2} \right)^2 \quad (7.77)$$

are both eigenvalues of $\mathcal{L}_\omega$. Maximizing $|\lambda(\mu, \omega)|$ over $0 < \mu \leq \rho(B)$ gives a maximum value at $\mu = \rho(B)$. Minimizing $|\lambda(\rho(B), \omega)|$ over $0 < \omega < 2$ gives the expression

$$1 - \omega_{\text{opt}} + \left(\frac{1}{2}\omega_{\text{opt}}\rho(B) \right)^2 = 0. \quad (7.78)$$

Solving this gives the expression for ω_{opt} in the statement of the theorem. This then yields

$$\rho(\mathcal{L}_{\omega_{\text{opt}}}) = \lambda(\rho(B), \omega_{\text{opt}}) = \left(\frac{1}{2}\omega_{\text{opt}}\rho(B) \right)^2 = \omega_{\text{opt}} - 1, \quad (7.79)$$

noting that $\omega_{\text{opt}} > 1$ by its definition. $\square$

Recall from Theorem 7.35 that $\rho(\mathcal{L}_\omega) = |\omega - 1|$ if and only if all eigenvalues of $\mathcal{L}_\omega$ have $|\lambda| = |\omega - 1|$. This says that when we use the optimal overrelaxation factor for SOR, the eigenvalues of $\mathcal{L}_{\omega_{\text{opt}}}$ must lie on a circle in the complex plane. In contrast, those of $\mathcal{L}_1$ are real valued and are either zero or equal to the squares of the eigenvalues of $B = D^{-1}(L + U)$. As we increase the relaxation parameter from one to ω_{opt} , the zero eigenvalues of $\mathcal{L}_1$ move along

a circle in the complex plane of radius $\omega - 1$, while the real (nonzero) eigenvalues of $\mathcal{L}_1$ move along the real axis and into the complex plane so that, at ω_{opt} , all eigenvalues are on this circle.

Revisiting the one-dimensional Poisson problem, we consider $-u''(x) = g(x)$ for $0 < x < 1$, with $u(0) = u(1) = 0$ on a uniform mesh, with $h = 1/n_x$. The eigenvalues of the discretization matrix A^h in equation (3.67) are $\lambda_k = \frac{4}{h^2} \sin^2\left(\frac{k\pi}{2n_x}\right)$ for $0 < k < n_x$. Since $\sin^2(\theta)$ is an increasing function of θ for $0 \leq \theta \leq \frac{\pi}{2}$, we see that the largest eigenvalue of A^h is $\lambda_{n_x-1} = \frac{4}{h^2} \sin^2\left(\frac{\pi}{2} - \frac{\pi}{2n_x}\right) \approx \frac{4}{h^2}$. Since the diagonal of A^h is constant, with each entry equal to $\frac{2}{h^2}$, we then have that the eigenvalues of the Jacobi iteration satisfy

$$1 - 2 \sin^2\left(\frac{\pi}{2} - \frac{\pi h}{2}\right) \leq \lambda(I - D^{-1}A^h) \leq 1 - 2 \sin^2\left(\frac{\pi h}{2}\right). \quad (7.80)$$

A bit of calculus should convince us that both of these values are within $\mathcal{O}(h^2)$ of ± 1 , and that $\rho(I - D^{-1}A^h) \approx 1 - \frac{h^2}{2}$.

From here, we ask the question of how many iterations of Jacobi are needed to achieve an error reduction of $\mathcal{O}(h^2)$, consistent with an $\mathcal{O}(1)$ initial error (e.g., by approximating the solution to $A^h \mathbf{u}^h = \mathbf{f}^h$ by the zero vector) that is reduced to the level of discretization error by the Jacobi iteration. This requires at least k iterations, where

$$\left(1 - \frac{h^2}{2}\right)^k \leq h^2. \quad (7.81)$$

Taking logarithms of both sides and using the fact that $\ln(1+z) \approx z$ when z is small gives

$$k \approx Ch^{-2} \ln(1/h), \quad (7.82)$$

meaning that $n_x^2 \ln(n_x)$ iterations of Jacobi are needed to reduce the residual norm below discretization error for the Poisson problem in 1D with n_x degrees of freedom. A similar analysis for the two-dimensional Poisson problem on an $n_x \times n_y$ mesh with $n_x = n_y$ shows that $n_x^2 \ln(n_x)$ iterations are needed to achieve discretization error in this case.

Using Corollary 7.41, we have that the asymptotic convergence factor for Gauss–Seidel satisfies $\rho(\mathcal{L}_1) = (\rho(B))^2 \approx 1 - h^2$, and we can repeat the above calculation with the same conclusion, that $k \approx n_x^2 \ln(n_x)$ iterations are required to solve the 1D Poisson problem to discretization error using Gauss–Seidel, and that $k \approx n_x^2 \ln(n_x)$ iterations are needed for the two-dimensional Poisson problem on an $n_x \times n_y$ mesh with $n_x = n_y$ using Gauss–Seidel. In two dimensions, in particular, each iteration of Jacobi or Gauss–Seidel costs $\mathcal{O}(n_x^2)$ arithmetic operations, so the total cost of solving a problem to within machine precision using *either* Jacobi or Gauss–Seidel is roughly quadratic in the number of degrees of freedom. In one dimension, it is even worse: approximately cubic in the number of degrees of freedom. Note that these estimates for Jacobi and Gauss–Seidel convergence neglect an important constant. As shown in Corollary 7.41, Gauss–Seidel takes half as many iterations as Jacobi for all of these problems.

Now, let's consider Theorem 7.42, which gives

$$\rho(\mathcal{L}_{\omega_{\text{opt}}}) = \omega_{\text{opt}} - 1 = \frac{2}{1 + \sqrt{1 - (\rho(B))^2}} - 1 = \frac{1 - \sqrt{1 - (\rho(B))^2}}{1 + \sqrt{1 - (\rho(B))^2}}. \quad (7.83)$$

Taking $\rho(B) \approx 1 - \frac{h^2}{2}$, we have $1 - (\rho(B))^2 \approx h^2$, which gives $\rho(\mathcal{L}_{\omega_{\text{opt}}}) \approx 1 - h$. By a similar calculation to that above, this says that we need $k \approx Ch^{-1} \ln(1/h)$ iterations of SOR with the optimal weight to reduce the error to discretization error from an initial $\mathcal{O}(1)$ error. For the one-dimensional Poisson problem, this reduces the cost of solution from roughly cubic in

n_x to quadratic. For the two-dimensional Poisson problem on an $n_x \times n_y$ mesh with $n_x = n_y$, this reduces the needed number of iterations from $\mathcal{O}(n_x^2 \ln(n_x))$ to $\mathcal{O}(n_x \ln(n_x))$, resulting in a total cost that is proportional to $n_x^3 \ln(n_x)$. This is a notable improvement on cost from Jacobi or Gauss–Seidel, but it is still not *optimal*, which would be a cost of $\mathcal{O}(n_x^2)$ or $\mathcal{O}(n_x^2 \ln(n_x))$. To further improve this cost requires the use of improved preconditioners, such as those discussed in Chapter 11.

7.6 ■ Incomplete factorizations

Objectives

The fundamental philosophy behind the approaches discussed in the previous sections is to choose the matrix splitting, $A = M^{-1} - N$, so that $M\mathbf{r}_{k-1}$ is inexpensive to compute. By choosing either a diagonal or a triangular matrix for M^{-1} , the cost of computing this matrix-vector product is bounded by the cost of forming the residual, and the weighted Jacobi and Gauss–Seidel algorithms can even be implemented to have total cost equal to that of a residual calculation. In this section, we change perspective and seek splittings so that N is guaranteed to be “small” in some sense, resulting in an iteration that should be quick to converge. From this perspective, the splitting $A = A - 0$ is ideal, giving convergence in one iteration when we compute $A^{-1}\mathbf{r}_{k-1}$ using, for example, an LU factorization of A . Thus, in this section you will make use of sparse approximations to A^{-1} or its LU factorization form to derive and understand matrix splittings known as incomplete factorization or approximate inverse methods. Next, you will study the convergence properties of incomplete factorization methods with regular splittings and investigate alternative approaches such as threshold-based incomplete factorization and other modified techniques.

If we consider again Algorithm 7.1 and its sparse counterparts from Section 7.2, we observe that the cost of factorization is strongly dependent on the number of new nonzero entries introduced within the innermost loop—those entries where $a_{i,j} = 0$ prior to factorization, but for which there is a k with both $a_{i,k} \neq 0$ and $a_{k,j} \neq 0$. *Incomplete* factorization algorithms aim to control this cost by sacrificing exactness and discarding some or all of this fill-in. This naturally leads to factorizations that can be expressed as $A = LU - R$, where R represents the “residual” of the factorization, including the terms that were discarded to control costs.

A common family of approaches to incomplete factorization are based on considering a fixed *zero-pattern set*,

$$\mathcal{P} \subset \{(i, j) \mid i \neq j, 1 \leq i, j \leq n\}, \quad (7.84)$$

and computing entries within the LU factors only outside of this set, as shown in Algorithm 7.5. As before, once the in-place factorization has been done, the nonzero entries are “unpacked” into the matrices L and U . Then, we simply define $R = A - LU$ using L and U .

A natural question is whether the resulting iteration,

$$LU\mathbf{u}_k = R\mathbf{u}_{k-1} + \mathbf{f}, \quad (7.85)$$

converges to the solution of $A\mathbf{u} = \mathbf{f}$ for some initial guess, $\mathbf{u}_0$. This is known to be true when A is an M-matrix, based on the theory outlined below. This theory, developed in more detail in [226], has applicability beyond that of incomplete factorizations, extending regular splittings (and weak regular splittings). To start, we define the notion of a *regular splitting*.

Algorithm 7.5: *ikj* in-place incomplete LU factorization based on zero-pattern set

Input : A , matrix to factorize (in place)
 $\mathcal{P}$, zero-pattern set

```

1 for  $i = 2$  to  $n$ 
2   for  $k = 1$  to  $i - 1$ 
3     if  $(i, k) \notin \mathcal{P}$ 
4        $a_{i,k} = a_{i,k} / a_{k,k}$ 
5     for  $j = k + 1$  to  $n$ 
6       if  $(i, j) \notin \mathcal{P}$ 
7          $a_{i,j} = a_{i,j} - a_{i,k} a_{k,j}$ 

```

Definition 7.43: Regular splitting [226, Def. 3.28]

Let A , M , and N be real-valued matrices. Then $A = M^{-1} - N$ is a regular splitting of A if M^{-1} is nonsingular with $m_{i,j} \geq 0$ and $n_{i,j} \geq 0$ for all (i, j) .

Then, Theorem 7.44 shows a relationship with the inverse (being positive) and a bounded iteration matrix for the splitting.

Theorem 7.44: Regular splitting convergence [226, Thm. 3.29]

Let $A = M^{-1} - N$ be a regular splitting of A . Then, A is nonsingular with $(A^{-1})_{i,j} \geq 0$ for all (i, j) if and only if $\rho(MN) < 1$.

In the case of an M-matrix, these properties are satisfied (see Theorem 7.45 and Corollary 7.46), showing that any matrix splitting of an M-matrix, A , that is also a regular splitting leads to a convergent iteration. In addition, the inequality in the final corollary provides some intuition: the convergence of the iteration improves as $\rho(A^{-1}N)$ gets small.

Theorem 7.45: Inverse positivity for M-matrices [41, Ch. 6]

Let A be an M-matrix. Then, A is nonsingular with $(A^{-1})_{i,j} \geq 0$ for all (i, j) .

Corollary 7.46: Regular splitting convergence (M-matrix) [226, Thm. 3.29]

If A is an M-matrix and $A = M^{-1} - N$ is a regular splitting of A , then

$$\rho(MN) = \frac{\rho(A^{-1}N)}{1 + \rho(A^{-1}N)} < 1. \quad (7.86)$$

An interesting aspect in our setting is that no conditions are needed on the zero-pattern set, $\mathcal{P}$, in order for Algorithm 7.5 to produce a regular splitting [194].

Theorem 7.47: Regular splitting ILU [194, Thm. 10.2]

Let A be an M-matrix and $\mathcal{P}$ be a zero-pattern set. Then, the incomplete LU factorization based on $\mathcal{P}$ does not break down and the resulting splitting, $A = LU - R$, is a regular splitting of A .

While these results are useful, they offer little guidance on the choice of $\mathcal{P}$. A common approach is to base the choice of $\mathcal{P}$ on the structure of the nonzero entries in A , defined as the

set

$$NZ(A) = \{(i, j) \mid a_{i,j} \neq 0\}. \quad (7.87)$$

Then, we select $\mathcal{P}$ based on $NZ(A)$. For example, the choice of

$$\mathcal{P} = \{(i, j) \mid i \neq j, 1 \leq i, j \leq n\} \setminus NZ(A) \quad (7.88)$$

restricts the nonzero entries in L and U to appear on the diagonals or in off-diagonal positions where A has nonzero entries. This choice leads to the algorithm known as $ILU(0)$, which gives controlled costs per iteration but does not always lead to a quickly converging iteration. A common approach to improve performance by expanding the zero-pattern set leads to the $ILU(k)$ algorithms, which choose

$$\mathcal{P} = \{(i, j) \mid i \neq j, 1 \leq i, j \leq n\} \setminus NZ(A^{k+1}). \quad (7.89)$$

An alternative set of approaches abandons the use of fixed zero-pattern sets in order to allow fill-in to occur in places where it seems significant. Algorithm 7.6 gives a general sketch of such approaches, known as incomplete LU with thresholding or ILUT algorithms.

Algorithm 7.6: *ikj* in-place incomplete LU factorization with thresholding (ILUT)

Input : A , matrix to factorize (in place)

```

1 for  $i = 2$  to  $n$ 
2    $w = a_{i,*}$  {  $w$  is the  $i$ th row of  $A$  in its entirety }
3   for  $k = 1$  to  $i - 1$  when  $w_k \neq 0$ 
4      $w_k = w_k / a_{k,k}$ 
5     scalar_drop( $w_k$ ) { Apply dropping rule to  $w_k$  }
6     if  $w_k$  not dropped
7       for  $j = k + 1$  to  $n$ 
8          $w_j = w_j - w_k u_{k,j}$ 
9   vector_drop( $w$ ) { Apply dropping rule to  $w$  }
10  for  $j = 1$  to  $i - 1$ 
11     $\ell_{i,j} = w_j$ 
12  for  $j = i$  to  $n$ 
13     $u_{i,j} = w_j$ 

```

There are two “dropping” calls in the algorithm: one for a single entry and one on an entire row. In both cases, some entries may be dropped from the resulting factorization if they are deemed “small enough.” Many different dropping rules are possible for the two points in the algorithm where they must be applied. One common approach (proposed in [193]) is to base the dropping on two parameters: a global dropping tolerance, τ , and a “level-of-fill” limit, p . For each row i of A , we denote the i th row by $w = a_{i,*}$, and a relative dropping tolerance, $\tau_i = \tau \|w\|$, is computed. With this, the first dropping rule can be implemented as dropping w_k in the inner loop when $|w_k| < \tau_i$. After the inner loop, when candidates for the i th rows of L and U have been computed, entries in w are dropped in two stages. First, any resulting entry where $|w_j| < \tau_i$ is dropped due to its size. Second, at most p entries in each of the lower- and upper-triangular parts are kept, keeping only the largest entries in each. Thus, p provides a strict upper bound on the memory needed and cost per iteration of the algorithm by enforcing the number of nonzero entries in the $n \times n$ matrices L and U to be bounded above by pn . However, it must

be noted that controlling the sparsity of the factorization by p alone is typically ineffective, and much better performance (relative to cost) results from adjusting τ to limit fill entrywise before dropping based on p .

Many other variants of incomplete factorizations exist and are well known in the literature. For example, *modified* ILU approaches, based on the idea of modifying the nonunit diagonal entries in U to improve stability and convergence of the iteration, are often popular, particularly for M-matrices. Under certain conditions, they can be proven to offer significant improvements on ILU with the same nonzero patterns. Even with such modifications, however, the resulting iterations still do not achieve optimal complexity. Approximate inverse preconditioners, which aim to define the splitting $A = M^{-1} - N$ by choosing M^{-1} so that $\|I - AM\|$ is minimized (for example, in the Frobenius norm) over all matrices M^{-1} , with a certain sparsity pattern for M , are also common but have the same limitations [115]. Thus, we now turn to some nonstationary iterative methods that can improve upon the performance.

7.7 ■ Polynomial methods and the Chebyshev iteration

Objectives

While the matrix-splitting ideas considered in the past sections are appealing due to their straightforward construction, they fail to generate optimal solution algorithms, even for simple model problems. While more finely tuned matrix splittings (such as symmetric SOR) are possible, research in these directions has yet to produce iterations that are competitive with those discussed in the remainder of this chapter and in Chapter 11. Thus, in this section, you will revisit the basic structure of polynomial iterative methods introduced in Section 7.3. Specifically, you will learn how to rewrite the polynomial iteration process as a “minimax” problem and be introduced to the idea of a Chebyshev iteration. Then, you will show that for symmetric and positive-definite linear systems, the convergence of the Chebyshev iteration depends on the spectrum of the matrix. Finally, you will understand how the results can be extended to nonsymmetric and non-Hermitian matrices.

Consider again the general polynomial method for solving the $n \times n$ linear system $Au = f$ given an initial guess, u_0 , described in Section 7.3.1:

$$u_1 = u_0 + \omega_1(f - Au_0), \quad (7.90a)$$

$$u_2 = u_1 + \omega_2(f - Au_1), \quad (7.90b)$$

$$u_3 = u_2 + \omega_3(f - Au_2), \quad (7.90c)$$

$$\vdots$$

$$u_k = u_{k-1} + \omega_k(f - Au_{k-1}). \quad (7.90d)$$

For notational simplicity, we have opted for the unpreconditioned form, although the calculations below can be extended to the preconditioned system, $MAu = Mf$.

In this section, we consider the question of how to choose the coefficients ω_j . As above, we look at this primarily in the error-propagation form, writing

$$e_k = \left[\prod_{j=1}^k (I - \omega_j A) \right] e_0 = p_k(A) e_0 \quad (7.91)$$

for $p_k(A) = \prod_{j=1}^k (I - \omega_j A)$. A natural goal is to select the coefficients of $p_k(A)$ in order to make e_k as small as possible. Making the additional assumption that A is diagonalizable, with

eigenvectors that satisfy $Av_i = \lambda_i v_i$ for $1 \leq i \leq n$, we expand $e_0 = \sum_{i=1}^n c_i v_i$, giving

$$e_k = \left[\prod_{j=1}^k (I - \omega_j A) \right] \left[\sum_{i=1}^n c_i v_i \right] \quad (7.92a)$$

$$= \sum_{i=1}^n c_i \left(\prod_{j=1}^k (I - \omega_j A) v_i \right) \quad (7.92b)$$

$$= \sum_{i=1}^n c_i \left(\prod_{j=1}^k (1 - \omega_j \lambda_i) v_i \right) = \sum_{i=1}^n c_i p_k(\lambda_i) v_i, \quad (7.92c)$$

where we write $p_k(z) = \prod_{j=1}^k (1 - \omega_j z)$ for $z \in \mathbb{C}$.

Note that the optimal choice for the coefficients in $p_k(A)$ strongly depends on e_0 and, in particular, its expansion into the eigenvectors of A . If, for example, e_0 is an eigenvector of A with eigenvalue λ , a natural choice is to take $\omega_1 = 1/\lambda$, which gives e_1 as the zero vector. Similarly, if e_0 is a linear combination of $k < n$ eigenvectors, then by a judicious choice of $\omega_1, \dots, \omega_k$ as the inverses of the k associated eigenvalues, we can ensure that e_k is the zero vector, giving the exact solution in k steps. Sections 7.8 and 7.9 discuss polynomial methods that use an optimization perspective that enables such choices. Here, we want to choose $p_k(A)$ to guarantee that e_k is small without using any information about e_0 to choose the coefficients.

To guarantee that $\|e_k\|$ is small, we want to make $p_k(\lambda_i)$ small for all eigenvalues λ_i . This gives

$$\|e_k\| \leq \sum_{i=1}^n |p_k(\lambda_i)| |c_i| \|v_i\| \leq \left(\max_{1 \leq i \leq n} |p_k(\lambda_i)| \right) \sum_{i=1}^n |c_i| \|v_i\|. \quad (7.93)$$

Recognizing that we cannot make the sum on the right small without imposing assumptions on e_0 , we instead ask to minimize $\max_{1 \leq i \leq n} |p_k(\lambda_i)|$. In the case where A is symmetric and positive definite, so that all of the eigenvalues are real valued and positive, this leads to a natural connection to Chebyshev polynomials, whose *minimax* properties are well known (see, for example, [20, 22] for more details).

Chebyshev polynomials are defined by the recurrence relation that, given $T_0(x) = 1$ and $T_1(x) = x$,

$$T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x) \text{ for } -1 \leq x \leq 1. \quad (7.94)$$

Among their many useful properties is the identity that $T_k(x) = \cos(k \arccos(x))$ for $-1 \leq x \leq 1$. Additionally, the recurrence relation above is easily extended for values of x along the entire real line, or even for complex values, leading to the similar identity that $T_k(z) = \cosh(k \operatorname{arccosh}(z))$ outside of the real interval $[-1, 1]$. A key property of Chebyshev polynomials is given in Theorem 7.48.

Theorem 7.48: Chebyshev minimax property [20, Sec. 12.4]

Given $k > 0$, define

$$\tau_k = \inf_{\deg(Q) \leq k-1} \left[\max_{-1 \leq x \leq 1} |x^k + Q(x)| \right] \quad (7.95)$$

for real-valued polynomials, $Q(x)$, of degree no more than $k-1$. Then, τ_k is uniquely attained when $x^k + Q(x) = 2^{1-k} T_k(x)$ and $\tau_k = 2^{1-k}$.

7.7.1 ■ Symmetric and positive-definite systems

Eigenvalues of symmetric matrices are always real, and those of positive-definite matrices must lie in the right half of the complex plane. When A is symmetric and positive definite, this allows us to approximate the minimum of $\max_{1 \leq i \leq n} |p_k(\lambda_i)|$ by taking $p_k(z)$ to be a scaled and transformed Chebyshev polynomial. Suppose we know that $0 < \alpha \leq \lambda_i \leq \beta$ for $1 \leq i \leq n$. Then, taking $\theta = (\alpha + \beta)/2$ and $\delta = (\beta - \alpha)/2$, we consider

$$p_k(z) = T_k \left(\frac{\theta - z}{\delta} \right) / T_k \left(\frac{\theta}{\delta} \right). \quad (7.96)$$

Note that $p_k(0) = 1$, as required by the factored form of $p_k(z)$. In fact, $p_k(z)$ is the minimax polynomial over the interval $\alpha \leq z \leq \beta$ subject to the normalization that $p_k(0) = 1$. Furthermore, we have $\max_{\alpha \leq z \leq \beta} |p_k(z)| = 1/T_k(\theta/\delta)$, which gets smaller as β/α approaches one, again by the properties of the Chebyshev polynomials. Note that this only *approximates* the minimum, since it minimizes $|p_k(z)|$ over the interval from α to β , rather than minimizing the discrete set of values $\{|p_k(\lambda_i)|\}$.

Example 7.49: Chebyshev polynomial result

As an example of the polynomial defined in equation (7.96), we consider two cases. First, consider matrix A_1 with eigenvalues

$$\Lambda(A_1) = 1.2, 1.32, 1.44, 1.56, 1.68, 1.8. \quad (7.97)$$

Figure 7.7.1a displays the polynomials, $p_k(z)$, for $k = 1, 2, 3$. Since the distribution of eigenvalues in this example is relatively compact, even low-order polynomials remain small over the entire interval containing the spectrum, $[1.2, 1.8]$. In fact, in this example, $p_3(z)$ already achieves a maximum of 0.002 over the interval.

Alternatively, consider matrix A_2 with eigenvalues

$$\Lambda(A_2) = 0.2, 0.52, 0.84, 1.16, 1.48, 1.8, \quad (7.98)$$

which are more widely distributed in the interval $[0, 2]$ than the eigenvalues of A_1 . In contrast to A_1 , the polynomials are relatively large over the subinterval containing the spectrum, $[0.2, 1.8]$, as shown in Figure 7.7.1b. Since $p_k(z)$ is monic (meaning $p_k(0) = 1$), reducing the size of $|p_k(z)|$ over $\Lambda(A_2)$ is inherently difficult since the minimum eigenvalue is close to zero.

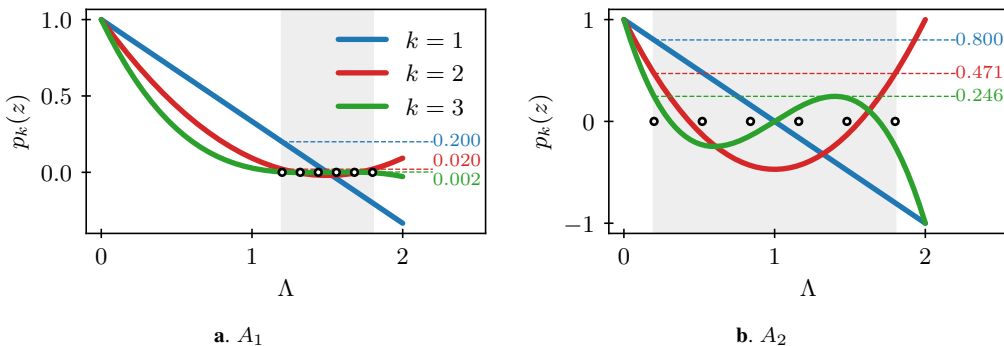


Figure 7.7.1. Examples of polynomial $p_k(z)$ in equation (7.96) for $k = 1, 2, 3$. The region of eigenvalues (circled) is highlighted in gray and the maximal value of $|p_k(x)|$ over this interval is noted on the right.

From this perspective, choosing the weights, ω_j , so that $p_k(z)$ is the scaled k th Chebyshev polynomial is promising. Unfortunately, this would require fixing the total number of steps in the iterative process beforehand, since the weights that correspond to the k th scaled Chebyshev polynomial, $p_k(z)$, are completely different than those of $p_{k+1}(z)$. Instead, we look to modify the polynomial iteration to define an iteration that satisfies $\mathbf{e}_k = p_k(z)\mathbf{e}_0$ but no longer takes the form $\mathbf{u}_k = \mathbf{u}_{k-1} + \omega_k \mathbf{r}_k$. The “trick” that makes this work is to derive a recurrence relation for $p_k(z)$ that depends on the bounds on the spectrum, α and β , and use this to get recurrences that define $\mathbf{u}_k$ and $\mathbf{r}_k = \mathbf{f} - A\mathbf{u}_k$. As above, we do not work directly with α and β but, instead, phrase the algorithm in terms of $\theta = (\alpha + \beta)/2$ and $\delta = (\beta - \alpha)/2$. For simplicity of notation, we write $\sigma_k = T_k(\theta/\delta)$ and note that σ_k must satisfy the original Chebyshev recurrence relation, with $\sigma_k = 2(\theta/\delta)\sigma_{k-1} - \sigma_{k-2}$, for $\sigma_0 = 1$ and $\sigma_1 = \theta/\delta$.

With that notation, we write

$$p_k(z) = (1/\sigma_k)T_k\left(\frac{\theta - z}{\delta}\right) \quad (7.99a)$$

$$= (1/\sigma_k) \left(2 \left(\frac{\theta - z}{\delta} \right) T_{k-1} \left(\frac{\theta - z}{\delta} \right) - T_{k-2} \left(\frac{\theta - z}{\delta} \right) \right) \quad (7.99b)$$

$$= \frac{\sigma_{k-1}}{\sigma_k} \left(2 \left(\frac{\theta - z}{\delta} \right) p_{k-1}(z) - \frac{\sigma_{k-2}}{\sigma_{k-1}} p_{k-2}(z) \right), \quad (7.99c)$$

with initial relations that $p_0(z) = 1$ and $p_1(z) = 1 - z/\theta$. Noting the two ratios of successive σ 's, we find it is easier to work with

$$\rho_{k-1} = \frac{\sigma_{k-1}}{\sigma_k} = \frac{\sigma_{k-1}}{2\sigma_1\sigma_{k-1} - \sigma_{k-2}} = \frac{1}{2\sigma_1 - \rho_{k-2}}, \quad (7.100)$$

allowing us to write the recurrence as

$$p_k(z) = \rho_{k-1} \left(2 \frac{\theta - z}{\delta} p_{k-1}(z) - \rho_{k-2} p_{k-2}(z) \right). \quad (7.101)$$

With this, we turn back to the basic polynomial iteration, writing $\mathbf{e}_k = p_k(A)\mathbf{e}_0$ for all k , which implies that we can write $\mathbf{r}_k = p_k(A)\mathbf{r}_0$ and $\mathbf{r}_{k-1} = p_{k-1}(A)\mathbf{r}_0$. This gives

$$\mathbf{r}_k - \mathbf{r}_{k-1} = (p_k(A) - p_{k-1}(A))\mathbf{r}_0 \quad (7.102a)$$

$$= \left(\rho_{k-1} \left(2 \frac{\theta - A}{\delta} p_{k-1}(A) - \rho_{k-2} p_{k-2}(A) \right) - \frac{\rho_{k-1}}{\rho_{k-1}} p_{k-1}(A) \right) \mathbf{r}_0 \quad (7.102b)$$

$$= \rho_{k-1} \left(\left(2 \left(\sigma_1 I - \frac{1}{\delta} A \right) p_{k-1}(A) - \rho_{k-2} p_{k-2}(A) \right) - (2\sigma_1 - \rho_{k-2}) p_{k-1}(A) \right) \mathbf{r}_0 \quad (7.102c)$$

$$= \rho_{k-1} \left(-\frac{2}{\delta} A p_{k-1}(A) \mathbf{r}_0 + \rho_{k-2} (p_{k-1}(A) - p_{k-2}(A)) \mathbf{r}_0 \right) \quad (7.102d)$$

$$= \rho_{k-1} \left(-\frac{2}{\delta} A \mathbf{r}_{k-1} + \rho_{k-2} (\mathbf{r}_{k-1} - \mathbf{r}_{k-2}) \right). \quad (7.102e)$$

Rewriting the last relationship, we have

$$\mathbf{r}_k = \mathbf{r}_{k-1} + \rho_{k-1} \left(\rho_{k-2} (\mathbf{r}_{k-1} - \mathbf{r}_{k-2}) - \frac{2}{\delta} A \mathbf{r}_{k-1} \right). \quad (7.103)$$

Noting that $\mathbf{r}_k = \mathbf{f} - A\mathbf{u}_k$, with similar relationships for $\mathbf{r}_{k-1}$ and $\mathbf{r}_{k-2}$, allows us to rewrite this for updates to $\mathbf{u}_k$ as

$$\mathbf{u}_k = \mathbf{u}_{k-1} + \rho_{k-1} \left(\rho_{k-2}(\mathbf{u}_{k-1} - \mathbf{u}_{k-2}) + \frac{2}{\delta} \mathbf{r}_{k-1} \right). \quad (7.104)$$

These recurrences are now *three-term recurrences* and provide the values of $\mathbf{u}_k$ and $\mathbf{r}_k$ in terms of $\mathbf{u}_{k-1}$, $\mathbf{r}_{k-1}$, $\mathbf{u}_{k-2}$, and $\mathbf{r}_{k-2}$. A typical implementation of these methods is to define the update vector,

$$\mathbf{d}_k = \rho_{k-1} \left(\rho_{k-2}(\mathbf{u}_{k-1} - \mathbf{u}_{k-2}) + \frac{2}{\delta} \mathbf{r}_{k-1} \right), \quad (7.105)$$

and then to update $\mathbf{u}_k = \mathbf{u}_{k-1} + \mathbf{d}_k$ and $\mathbf{r}_k = \mathbf{r}_{k-1} - A\mathbf{d}_k$. With this, the method requires more storage than the two-term recurrence, $\mathbf{u}_k = \mathbf{u}_{k-1} + \omega_k \mathbf{r}_{k-1}$, as we must store five vectors ($\mathbf{u}_{k-1}$, $\mathbf{u}_{k-2}$, $\mathbf{r}_{k-1}$, $\mathbf{d}_k$, and $A\mathbf{d}_k$) in order to compute both $\mathbf{u}_k$ and $\mathbf{r}_k$ but still only require a single matrix-vector product per iteration. We reduce this storage cost by one vector by noting that $\mathbf{d}_{k-1} = \mathbf{u}_{k-1} - \mathbf{u}_{k-2}$, so we need not store $\mathbf{u}_{k-2}$ in order to compute the next iteration. Thus, while the cost of a Chebyshev iteration is slightly higher than that of a two-term recurrence, the cost is likely to pay off when β/α is far from one. The iteration is summarized in Algorithm 7.7.

Algorithm 7.7: Chebyshev iteration to solve $A\mathbf{u} = \mathbf{f}$

Input : A , linear system matrix
 $\mathbf{f}$, linear system right-hand side
 $\mathbf{u}_0$, initial guess
 α, β , lower and upper bounds on the spectrum of A
Output : $\mathbf{u}_k$, k th iterate and approximation to $\mathbf{u}$

```

1  $\theta = (\alpha + \beta)/2, \delta = (\beta - \alpha)/2, \sigma_1 = \theta/\delta, \rho_0 = 1/\sigma_1$ 
2  $\mathbf{r}_0 = \mathbf{f} - A\mathbf{u}_0$ 
3  $\mathbf{u}_1 = \mathbf{u}_0 + (1/\theta)\mathbf{r}_0$ 
4  $\mathbf{r}_1 = \mathbf{r}_0 - (1/\theta)A\mathbf{r}_0$ 
5  $\mathbf{d}_1 = (1/\theta)\mathbf{r}_0$ 
6 for  $k = 2, 3, \dots$ 
7    $\rho_{k-1} = 1/(2\sigma_1 - \rho_{k-2})$ 
8    $\mathbf{d}_k = \rho_{k-1}(\rho_{k-2}\mathbf{d}_{k-1} + \frac{2}{\delta}\mathbf{r}_{k-1})$ 
9    $\mathbf{u}_k = \mathbf{u}_{k-1} + \mathbf{d}_k$ 
10   $\mathbf{r}_k = \mathbf{r}_{k-1} - A\mathbf{d}_k$ 
```

Since we derived the algorithm under the assumption that A is symmetric and positive definite, we can measure the convergence of the iteration in the norm induced by A , defined as $\|\mathbf{e}\|_A^2 = \mathbf{e}^T A \mathbf{e}$. This turns out to be a natural measure of performance of iterative methods for symmetric and positive-definite matrices, since it is easier to analyze than the Euclidean norm of the error, but it is stronger than analyzing the Euclidean norm of the residual. An important property of A that also holds under the symmetric and positive-definite assumption is that A is diagonalizable by a unitary matrix, meaning that there exist n vectors, $\mathbf{v}_i$, such that $A\mathbf{v}_i = \lambda_i \mathbf{v}_i$ for $1 \leq i \leq n$ and such that $\mathbf{v}_i^T \mathbf{v}_j = \delta_{i,j}$ for $1 \leq i, j \leq n$, where $\delta_{i,j}$ is the Kronecker delta function, which takes value one when $i = j$ and zero otherwise. (See [109, Thm. 8.1.1] for details.) A consequence of this is that any vector in $\mathbb{R}^n$ can be written as a linear combination of the eigenvectors $\{\mathbf{v}_i\}_{i=1}^n$. We use these results in the following theorem.

Theorem 7.50: Chebyshev convergence [24, Sec. 5.3]

Let A be symmetric and positive definite, and let $\alpha \leq \lambda \leq \beta$ for any eigenvalue, λ , of A . As above, let $\theta = (\alpha + \beta)/2$ and $\delta = (\beta - \alpha)/2$. Let initial guess, $\mathbf{u}_0$, to the solution, $\mathbf{u}$, of $A\mathbf{u} = \mathbf{f}$ be given, and let $\mathbf{u}_k$ be the k th iterate generated by Algorithm 7.7. Then,

$$\|\mathbf{u} - \mathbf{u}_k\|_A \leq \frac{1}{|T_k(\theta/\delta)|} \|\mathbf{u} - \mathbf{u}_0\|_A. \quad (7.106)$$

Proof. Since A is diagonalizable, we write $\mathbf{e}_0 = \mathbf{u} - \mathbf{u}_0 = \sum_{i=1}^n c_i \mathbf{v}_i$ in terms of the eigenvectors of A . Then, as above, $\mathbf{e}_k = p_k(A)\mathbf{e}_0 = \sum_{i=1}^n c_i p_k(\lambda_i) \mathbf{v}_i$. Thus,

$$\|\mathbf{e}_k\|_A^2 = \sum_{i=1}^n c_i^2 \lambda_i p_k^2(\lambda_i) \leq \left(\max_{1 \leq i \leq n} p_k^2(\lambda_i) \right) \sum_{i=1}^n c_i^2 \lambda_i \quad (7.107a)$$

$$= \left(\max_{1 \leq i \leq n} p_k^2(\lambda_i) \right) \|\mathbf{e}_0\|_A^2, \quad (7.107b)$$

where we have used the orthonormality of the eigenvectors of A in both the first and last equalities.

Since $|\theta - \lambda_i| \leq \delta$ for all i , then $|T_k((\theta - \lambda_i)/\delta)| \leq 1$ by the properties of the Chebyshev polynomials. Thus,

$$|p_k(\lambda_i)| = |T_k((\theta - \lambda_i)/\delta)| / |T_k(\theta/\delta)| \leq 1 / |T_k(\theta/\delta)|, \quad (7.108)$$

proving the stated inequality. $\square$

Corollary 7.51: Chebyshev convergence bound [24, Sec. 5.3]

Under the assumptions of Theorem 7.50,

$$\|\mathbf{u} - \mathbf{u}_k\|_A \leq 2 \left(\frac{\sqrt{\beta/\alpha} - 1}{\sqrt{\beta/\alpha} + 1} \right)^k \|\mathbf{u} - \mathbf{u}_0\|_A. \quad (7.109)$$

Proof. Note that $\theta/\delta = 1 + 2\alpha/(\beta - \alpha)$. Thus, $T_k(\theta/\delta) = T_k(1 + 2\alpha/(\beta - \alpha))$. From the recurrence relation above, for real $x > 1$,

$$T_k(x) = \frac{1}{2} \left(\left(x + \sqrt{x^2 - 1} \right)^{-k} + \left(x + \sqrt{x^2 - 1} \right)^k \right) \geq \frac{1}{2} \left(x + \sqrt{x^2 - 1} \right)^k. \quad (7.110)$$

Writing $\eta = \frac{\alpha}{\beta - \alpha}$, we obtain

$$T_k(\theta/\delta) = T_k(1 + 2\eta) \geq \frac{1}{2} \left(1 + 2\eta + \sqrt{(1 + 2\eta)^2 - 1} \right)^k \quad (7.111a)$$

$$= \frac{1}{2} \left(1 + 2\eta + 2\sqrt{\eta(\eta + 1)} \right)^k = \frac{1}{2} \left(\left(\sqrt{\eta} + \sqrt{\eta + 1} \right)^2 \right)^k. \quad (7.111b)$$

Now, substituting back in the value of η , we further manipulate this as

$$T_k(\theta/\delta) \geq \frac{1}{2} \left(\left(\sqrt{\frac{\alpha}{\beta - \alpha}} + \sqrt{\frac{\beta}{\beta - \alpha}} \right)^2 \right)^k \quad (7.112a)$$

$$= \frac{1}{2} \left(\frac{(\sqrt{\alpha} + \sqrt{\beta})^2}{\beta - \alpha} \right)^k = \frac{1}{2} \left(\frac{\sqrt{\alpha} + \sqrt{\beta}}{\sqrt{\beta} - \sqrt{\alpha}} \right)^k = \frac{1}{2} \left(\frac{\sqrt{\beta/\alpha} + 1}{\sqrt{\beta/\alpha} - 1} \right)^k. \quad (7.112b)$$

Thus,

$$\frac{1}{|T_k(\theta/\delta)|} \leq 2 \left(\frac{\sqrt{\beta/\alpha} - 1}{\sqrt{\beta/\alpha} + 1} \right)^k. \quad (7.113)$$

□

A key consequence of this result is that the performance of the Chebyshev iteration depends on the bounds available for the spectrum of A . The best-possible bounds are the extremal eigenvalues, with $\alpha = \lambda_{\min}(A)$ and $\beta = \lambda_{\max}(A)$, giving $\beta/\alpha = \text{cond}(A)$. If only poorer bounds are available, then the performance of the Chebyshev iteration suffers accordingly. This is a key disadvantage to the iteration in comparison to the CG algorithm, which will be introduced in Section 7.9, although there are still some settings where the Chebyshev iteration is preferred.

Example 7.52: One-dimensional Poisson problem

Again considering the finite-difference discretization of a one-dimensional Poisson problem (see equation (3.67)), we know that the smallest eigenvalue of A^h is bounded below by eight, and the largest eigenvalue is bounded above by $4/h^2$. Taking $h = 1/n_x$ and using these to define $\alpha = 8$ and $\beta = 4n_x^2$, Corollary 7.51 gives us the bound that

$$\|\mathbf{u} - \mathbf{u}_k\|_A \leq 2 \left(\frac{\sqrt{n_x^2/2} - 1}{\sqrt{n_x^2/2} + 1} \right)^k \|\mathbf{u} - \mathbf{u}_0\|_A. \quad (7.114)$$

We can rewrite the term inside the brackets as $1 - \frac{2}{\sqrt{n_x^2/2} + 1}$, which allows us to follow the same argument that leads to equation (7.82) and estimate that we need $k \approx n_x \ln(n_x)$ iterations of the Chebyshev algorithm to achieve discretization-level error in the iterative solution of the 1D Poisson problem. A similar analysis is possible in 2D for the Poisson problem on an $n_x \times n_y$ mesh with $n_x = n_y$, requiring $k \approx n_x \ln(n_x)$ iterations to achieve discretization error accuracy.

The asymptotic bound here allows some flexibility in the estimation of α and β , giving the same asymptotic performance so long as α is $\mathcal{O}(1)$ and β is $\mathcal{O}(h^{-2})$. If either of these estimates is replaced by one with a worse dependence on h —e.g., estimating $\alpha = h$, which still gives an interval containing the spectrum, just not one that is a tight bound—then the performance of the resulting algorithm will suffer.

7.7.2 ■ Nonsymmetric systems

The Chebyshev iteration is naturally extended to nonsymmetric or non-Hermitian matrices, as the recurrence relation for the Chebyshev polynomials stays the same for complex arguments.

What changes, however, is that the recurrence relation now leads to the definition $T_k(z) = \cosh(k \operatorname{arccosh}(z))$ for $z \in \mathbb{C}$. We now consider diagonalizable matrices $A = V\Lambda V^{-1}$, with diagonal matrix Λ whose entries are the eigenvalues of A . We further assume that all of the eigenvalues of A lie in an ellipse in the complex plane that does not include the origin.

To fix notation, we define $E(\theta, \delta, \gamma)$ to be the ellipse in the complex plane with center at $\theta \in \mathbb{C}$, focal distance δ , and semimajor axis γ . Both δ and γ are naturally complex in this description, but γ/δ is real valued. Two examples of ellipses are shown in Figure 7.7.2. The shifted and scaled Chebyshev polynomials associated with the minimax property on $E(\theta, \delta, \gamma)$ remain

$$p_k(z) = T_k\left(\frac{\theta - z}{\delta}\right) / T_k\left(\frac{\theta}{\delta}\right); \quad (7.115)$$

however, the relevant bound is shown to be

$$\max_{z \in E(\theta, \delta, \gamma)} |p_k(z)| = T_k(\gamma/\delta) / |T_k(\theta/\delta)|. \quad (7.116)$$

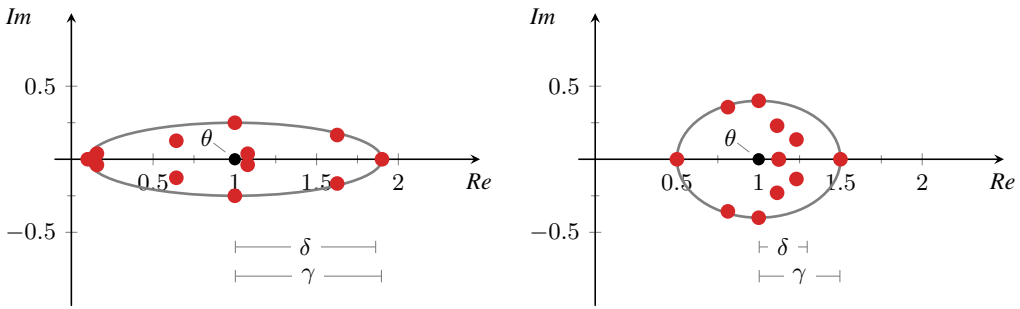


Figure 7.7.2. Two ellipses defined in terms of their center (θ), focal distance (δ), and semimajor axis (γ). An example of 12 eigenvalues is shown in red.

The real case considered above corresponds to a degenerate ellipse, where $\gamma = \delta$, and the numerator in the above is equal to one. Since the shifted and scaled Chebyshev polynomials remain the same, the Chebyshev iteration is still determined by θ and δ (which would now be direct inputs in place of α and β). As a result, the only changes needed in the algorithm are to include complex parameters, σ_1 and ρ_k , as well as complex-valued vectors. Now, however, we measure convergence in terms of the Euclidean norm of the residual, since the matrix A no longer defines a norm. A result is given in Theorem 7.53.

Theorem 7.53: Nonsymmetric Chebyshev convergence [163]

Assume that $A = V\Lambda V^{-1}$ is diagonalizable, with eigenvector matrix V and diagonal matrix of eigenvalues Λ . Assume that θ , δ , and γ are given such that each eigenvalue, λ_i , for $1 \leq i \leq n$, satisfies $\lambda_i \in E(\theta, \delta, \gamma)$. Then,

$$\|\mathbf{r}_k\| \leq \operatorname{cond}(V) \frac{T_k(\gamma/\delta)}{|T_k(\theta/\delta)|} \|\mathbf{r}_0\|. \quad (7.117)$$

Proof. From the recursion relations, we have that

$$\mathbf{r}_k = p_k(A)\mathbf{r}_0 = Vp_k(\Lambda)V^{-1}\mathbf{r}_0, \quad (7.118)$$

since any power of A satisfies $A^p = V\lambda^p V^{-1}$. Thus,

$$\|\mathbf{r}_k\| \leq \|V\| \|p_k(\Lambda)\| \|V^{-1}\| \|\mathbf{r}_0\|. \quad (7.119)$$

Noting that

$$\|p_k(\Lambda)\| = \max_{1 \leq i \leq n} |p_k(\lambda_i)| \leq \max_{z \in E(\theta, \delta, \gamma)} |p_k(z)| \quad (7.120)$$

completes the proof with the inequality stated above. $\square$

It is important to note that, for a nonnormal matrix, A , $\text{cond}(V)$ can be quite large and, consequently, this bound is potentially much worse than in the symmetric case, even before considering extensions of the bounds on Chebyshev polynomials to the complex case. In the next sections, we consider methods that use these basic ideas, but without needing to know the bounds on the spectrum a priori.

Example 7.54: Nonsymmetric matrices

As an example of a matrix where the condition number of V is potentially large, we consider

$$A = \begin{bmatrix} 2 & 1 \\ \epsilon^2 & 2 \end{bmatrix} \quad (7.121)$$

for $\epsilon > 0$. If we were to consider the limiting case of $\epsilon = 0$, then A would take the form of a Jordan block and no longer be diagonalizable. By direct calculation, we find the eigenvalues and eigenvectors of A so that $A = V\Lambda V^{-1}$ for

$$\Lambda = \begin{bmatrix} 2 + \epsilon & 0 \\ 0 & 2 - \epsilon \end{bmatrix} \text{ and } V = \begin{bmatrix} 1 & 1 \\ \epsilon & -\epsilon \end{bmatrix}. \quad (7.122)$$

The Euclidean norm condition number of V is given by its singular values, which are the square roots of the eigenvalues of

$$VV^T = \begin{bmatrix} 1 & 1 \\ \epsilon & -\epsilon \end{bmatrix} \begin{bmatrix} 1 & \epsilon \\ 1 & -\epsilon \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2\epsilon^2 \end{bmatrix}, \quad (7.123)$$

giving $\text{cond}(V) = \frac{1}{\epsilon}$.

While this introduces an arbitrarily large factor into the bound in Theorem 7.53, it must be noted that the remaining terms counteract this somewhat, since the eigenvalues of A in this case are quite close together. For a concrete example, consider solving $A\mathbf{u} = \mathbf{f}$ for $\mathbf{f} = \begin{bmatrix} 3 \\ 2 + \epsilon^2 \end{bmatrix}$, so that the exact solution is $\mathbf{u} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Taking $\theta = 2$ and $\gamma = \delta = \epsilon$ (so that eigenvalues $2 \pm \epsilon$ lie in the ellipse), after one step of the Chebyshev iteration with $\mathbf{u}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, we have

$$\mathbf{u}_1 = \mathbf{u}_0 + \frac{1}{\theta} \mathbf{r}_0 = \begin{bmatrix} \frac{3}{2} \\ 1 + \frac{\epsilon^2}{2} \end{bmatrix}, \text{ with } \mathbf{r}_1 = \begin{bmatrix} -1 - \frac{\epsilon^2}{2} \\ -\frac{3\epsilon^2}{2} \end{bmatrix}. \quad (7.124)$$

While $\|\mathbf{r}_1\|$ is not as large as the value of $\text{cond}(V)$ might suggest, it is still $\mathcal{O}(1)$, as is the error in $\mathbf{u}_1$. Contrast this with the symmetric case, taking

$$A = \begin{bmatrix} 2 & \epsilon \\ \epsilon & 2 \end{bmatrix}, \text{ with } \mathbf{f} = \begin{bmatrix} 2 + \epsilon \\ 2 + \epsilon \end{bmatrix}, \quad (7.125)$$

with the same (zero) initial guess and exact solution, $\mathbf{u}$. The symmetric matrix here has the same eigenvalues as our initial nonsymmetric matrix, and we have the same initial error, but we have a different initial residual. In this case, we get

$$\mathbf{u}_1 = \mathbf{u}_0 + \frac{1}{\theta} \mathbf{r}_0 = \begin{bmatrix} 1 + \frac{\epsilon}{2} \\ 1 + \frac{\epsilon}{2} \end{bmatrix}, \text{ with } \mathbf{r}_1 = \begin{bmatrix} -\epsilon - \frac{\epsilon^2}{2} \\ -\epsilon - \frac{\epsilon^2}{2} \end{bmatrix}. \quad (7.126)$$

Clearly, both the error in $\mathbf{u}_1$ and the residual, $\mathbf{r}_1$, are much smaller in the symmetric case than in the nonsymmetric case, even though the eigenvalues are the same. In fact, the *only* difference between these two cases is that the eigenvalue matrix, V , is unitary (with unit condition number) in the symmetric case, but not in the nonsymmetric case.

7.8 ■ Krylov methods: The Arnoldi algorithm and GMRES

Objectives

The Chebyshev iteration in the previous section introduces a fundamental new idea, that we can construct a polynomial method with more general updates than $\mathbf{u}_k = \mathbf{u}_{k-1} + \omega_k(\mathbf{f} - A\mathbf{u}_{k-1})$, although we may need to do more work per iteration to do so. The next major step forward in the development of iterative methods comes from asking what are the *best* polynomial methods that we can construct. To gain insight into this question, we first consider the space of possible iterates in a polynomial method. In this section, you will be introduced to a large class of iterative polynomial methods known as Krylov methods, where the space of possible iterates is the Krylov subspace. Specifically, you will be able to define a Krylov space and discuss the difficulties in finding good approximations within such a space. You will derive the Arnoldi algorithm that finds an *orthogonal* basis for a Krylov space and, then, construct and analyze the generalized minimum residual, or GMRES, method, which adds additional computational improvements to finding the best solution in the Krylov space.

For a classical polynomial method,

$$\mathbf{u}_1 = \mathbf{u}_0 + \omega_1 \mathbf{r}_0, \quad (7.127a)$$

$$\begin{aligned} \mathbf{u}_2 &= \mathbf{u}_1 + \omega_2(\mathbf{f} - A\mathbf{u}_1) \\ &= \mathbf{u}_0 + \omega_1 \mathbf{r}_0 + \omega_2(\mathbf{f} - A(\mathbf{u}_0 + \omega_1 \mathbf{r}_0)) \\ &= \mathbf{u}_0 + (\omega_1 + \omega_2) \mathbf{r}_0 - \omega_1 \omega_2 A \mathbf{r}_0, \end{aligned} \quad (7.127b)$$

$$\begin{aligned} \mathbf{u}_3 &= \mathbf{u}_2 + \omega_3(\mathbf{f} - A\mathbf{u}_2) \\ &= \mathbf{u}_0 + (\omega_1 + \omega_2 + \omega_3) \mathbf{r}_0 - (\omega_1 \omega_2 + \omega_1 \omega_3 + \omega_2 \omega_3) A \mathbf{r}_0 + \omega_1 \omega_2 \omega_3 A^2 \mathbf{r}_0. \end{aligned} \quad (7.127c)$$

By induction, we can show the more general result for this iteration,

$$\mathbf{u}_{k-1} = \mathbf{u}_0 + c_0 \mathbf{r}_0 + c_1 A \mathbf{r}_0 + \cdots + c_{k-2} A^{k-2} \mathbf{r}_0 \quad (7.128a)$$

$$\Rightarrow \mathbf{u}_k = \mathbf{u}_0 + c'_0 \mathbf{r}_0 + c'_1 A \mathbf{r}_0 + \cdots + c'_{k-2} A^{k-2} \mathbf{r}_0 + c'_{k-1} A^{k-1} \mathbf{r}_0, \quad (7.128b)$$

where coefficients $c'_0, \dots, c'_{k-1}$ are determined by $c_0, \dots, c_{k-2}$ and the choice of ω_k . The same is true of the Chebyshev iteration by a similar inductive argument. These expressions lead to a useful characterization of the space of iterates that are reached by a polynomial method by noting that

$$\mathbf{u}_k - \mathbf{u}_0 \in \text{span}\{\mathbf{r}_0, A \mathbf{r}_0, \dots, A^{k-1} \mathbf{r}_0\}. \quad (7.129)$$

This important space is defined in Definition 7.55.

Definition 7.55: Krylov subspace [194, Eq. (6.2)]

Given a matrix, A , and a vector, $\mathbf{r}_0$, the Krylov subspace of dimension k is

$$\mathcal{K}_k(A, \mathbf{r}_0) = \text{span}\{\mathbf{r}_0, A\mathbf{r}_0, \dots, A^{k-1}\mathbf{r}_0\}. \quad (7.130)$$

The above argument shows that a classical one-step polynomial method generates approximations, $\mathbf{u}_k$, that satisfy

$$\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0). \quad (7.131)$$

A much larger class of iterative methods, known as Krylov methods, also choose approximations that satisfy this relation. Unlike the one-step and Chebyshev iterations, Krylov methods add the requirement that $\mathbf{u}_k$ be chosen to satisfy some (quasi-)optimality property. Different types of optimality lead to the different methods in the Krylov family of algorithms.

7.8.1 ■ The Arnoldi algorithm

While not the first historically, a natural algorithm to consider first is the generalized minimum residual, or GMRES, algorithm, introduced by Saad and Schultz in 1986 [195]. Within GMRES, we choose $\mathbf{u}_k$ to minimize $\|\mathbf{f} - A\mathbf{u}_k\|$ (i.e., minimize the norm of the residual) over all possible vectors, $\mathbf{u}_k$, that satisfy $\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)$, with $\mathbf{r}_0 = \mathbf{f} - A\mathbf{u}_0$. What sets GMRES apart from other methods (including the earlier minimum residual, or MINRES, algorithm that leads to the slightly nonintuitive name of GMRES) is that it makes no further assumptions on A , other than A being nonsingular (and even this can be avoided [59]).

A brute-force approach to achieving the minimization within GMRES can be found by introducing matrices, P_k , whose columns are the natural bases for the Krylov spaces based on A and $\mathbf{r}_0$:

$$P_k = [\mathbf{r}_0, A\mathbf{r}_0, \dots, A^{k-1}\mathbf{r}_0]. \quad (7.132)$$

Then, any vector $\mathbf{u}_k$ that satisfies $\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)$ can be written in the form

$$\mathbf{u}_k = \mathbf{u}_0 + P_k \mathbf{y}_k \text{ for } \mathbf{y}_k \in \mathbb{R}^k \text{ (or } \mathbb{C}^k). \quad (7.133)$$

Next, we solve the resulting least-squares problem for $\mathbf{y}_k$:

$$\min_{\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)} \|\mathbf{f} - A\mathbf{u}_k\| = \min_{\mathbf{y}_k} \|\mathbf{f} - A\mathbf{u}_0 - AP_k \mathbf{y}_k\| = \min_{\mathbf{y}_k} \|\mathbf{r}_0 - AP_k \mathbf{y}_k\|. \quad (7.134)$$

The solution to this naturally satisfies the associated normal equations, giving

$$P_k^T A^T AP_k \mathbf{y}_k = P_k^T A^T \mathbf{r}_0. \quad (7.135)$$

There are two computational issues with this approach. First, the cost of updating $P_k^T A^T AP_k$ (and, presumably, its factorization if we are to solve the least-squares system using a direct method) is potentially high. Secondly, when A is ill conditioned, the conditioning of $A^T A$ is even worse, so it is difficult to accurately solve such systems. The standard “fix” for the conditioning problem in a least-squares setting is to use a QR factorization of the matrix in order to avoid forming the normal equations. We could do that here as well; however, there is some benefit to being more clever in the algorithmic design.

Thus, instead of computing and updating a QR factorization of AP_k , we directly compute orthogonal bases for $\mathcal{K}_k(A, \mathbf{r}_0)$ and use this to modify the least-squares problem. One approach is to take $\mathbf{r}_0, A\mathbf{r}_0, \dots, A^{k-1}\mathbf{r}_0$ and then use the Gram–Schmidt (or QR factorization) algorithm to orthogonalize this basis for the Krylov space. Instead, we use a modified form that computes an orthonormal basis for the same subspace in a more stable manner. The standard algorithm for

Algorithm 7.8: Arnoldi algorithm to compute orthogonal basis of $\mathcal{K}_{k+1}(A, \mathbf{q}_1)$ **Input :** A , matrix $\mathbf{q}_1$, initial vector with $\|\mathbf{q}_1\| = 1$ **Output :** $\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_{k+1}$, orthogonal basis

```

1 for  $j = 1, 2, \dots, k$ 
2    $\tilde{\mathbf{q}}_{j+1} = A\mathbf{q}_j$ 
3   for  $i = 1, \dots, j$ 
4      $h_{i,j} = \mathbf{q}_i^T \tilde{\mathbf{q}}_{j+1}$ 
5    $\tilde{\mathbf{q}}_{j+1} = \tilde{\mathbf{q}}_{j+1} - \sum_{i=1}^j h_{i,j} \mathbf{q}_i$ 
6    $h_{j+1,j} = \|\tilde{\mathbf{q}}_{j+1}\|$ 
7    $\mathbf{q}_{j+1} = \frac{1}{h_{j+1,j}} \tilde{\mathbf{q}}_{j+1}$ 

```

this is known as the Arnoldi algorithm (based on the modified Gram–Schmidt process), given in Algorithm 7.8.

Several properties of this algorithm are worth noting. First, when $h_{j+1,j} = 0$, the algorithm fails or *breaks down*, since $A\mathbf{q}_j - \sum_{i=1}^j h_{i,j} \mathbf{q}_i = \mathbf{0}$, which says that $A\mathbf{q}_j \in \text{span}\{\mathbf{q}_1, \dots, \mathbf{q}_j\}$. In this setting, there is no “new” direction to add to the space already generated, and $\mathcal{K}_k(A, \mathbf{q}_1) = \mathcal{K}_j(A, \mathbf{q}_1)$ for all $k \geq j$. Such a condition occurs, for example, when $\mathbf{q}_1$ is a linear combination of only a few eigenvectors of A . In general, though, $A\mathbf{q}_j = h_{j+1,j} \mathbf{q}_{j+1} + \sum_{i=1}^j h_{i,j} \mathbf{q}_i$. From this, we see that $\mathbf{q}_2 = -\frac{h_{1,1}}{h_{2,1}} \mathbf{q}_1 + \frac{1}{h_{2,1}} A\mathbf{q}_1$, and we use this fact to prove (by induction) that

$$\text{span}\{\mathbf{q}_1, A\mathbf{q}_1, \dots, A^{k-1} \mathbf{q}_1\} = \text{span}\{\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_k\} \quad (7.136)$$

for any $k \geq 1$.

7.8.2 ■ GMRES

To simplify notation, we assume A is an $n \times n$ matrix and introduce the $n \times k$ matrix Q_k , whose columns are given by $\mathbf{q}_1, \dots, \mathbf{q}_k$; the $n \times (k+1)$ matrix Q_{k+1} , whose columns are given by $\mathbf{q}_1, \dots, \mathbf{q}_{k+1}$; and the $(k+1) \times k$ matrix, $\bar{H}_k$, with entries

$$(\bar{H}_k)_{i,j} = \begin{cases} h_{i,j} & \text{if } 1 \leq j \leq k, 1 \leq i \leq \min(j+1, k+1), \\ 0 & \text{otherwise.} \end{cases} \quad (7.137)$$

With this, the set of relations that $A\mathbf{q}_j = \sum_{i=1}^{j+1} h_{i,j} \mathbf{q}_i$ for $1 \leq j \leq k$ can be rewritten simply as

$$AQ_k = Q_{k+1} \bar{H}_k. \quad (7.138)$$

We also note that the matrices Q_k and Q_{k+1} have orthogonal columns. GMRES uses the basis for $\mathcal{K}_k(A, \mathbf{r}_0)$ given by Q_k , generated by the Arnoldi algorithm started from $\mathbf{q}_1 = \frac{1}{\|\mathbf{r}_0\|} \mathbf{r}_0$, rather than the natural basis considered above. Writing $\mathbf{u}_k = \mathbf{u}_0 + Q_k \mathbf{y}_k$, we then have

$$\min_{\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)} \|\mathbf{f} - A\mathbf{u}_k\| = \min_{\mathbf{y}_k} \|\mathbf{r}_0 - AQ_k \mathbf{y}_k\| \quad (7.139a)$$

$$= \min_{\mathbf{y}_k} \|\mathbf{r}_0 - Q_{k+1} \bar{H}_k \mathbf{y}_k\| \quad (7.139b)$$

$$= \min_{\mathbf{y}_k} \|Q_{k+1}^T \mathbf{r}_0 - \bar{H}_k \mathbf{y}_k\|, \quad (7.139c)$$

where we have used the fact that $\min \|x\| = \min \|Q^T x\|$ for any orthogonal matrix Q . Since the first column of Q_{k+1} is $\frac{1}{\|r_0\|} r_0$ and Q_{k+1} has orthogonal columns, then

$$Q_{k+1}^T r_0 = \beta e_{k+1}^{(1)}, \quad (7.140)$$

where $\beta = \|r_0\|$, and $e_{k+1}^{(1)}$ is the first canonical unit vector of length $k+1$ whose first entry is one and has all other entries equal to zero. With this,

$$\min_{u_k - u_0 \in \mathcal{K}_k(A, r_0)} \|f - Au_k\| = \min_{y_k} \|\beta e_{k+1}^{(1)} - \bar{H}_k y_k\|. \quad (7.141)$$

At this stage, we now consider using a QR factorization to solve the least-squares problem, but for the $(k+1) \times k$ matrix, $\bar{H}_k$, rather than the $n \times n$ matrix, A . We write $\bar{H}_k = U_k R_k$, where U_k is a $(k+1) \times (k+1)$ orthogonal matrix and R_k is a $(k+1) \times k$ upper-triangular matrix. Note that we depart from a common convention here and that u_k is *not* the k th column of U_k —i.e., there is no direct relation between the (small) square matrix, U_k , and the n vector u_k that is our best approximation to the solution. Then,

$$\min_{u_k - u_0 \in \mathcal{K}_k(A, r_0)} \|f - Au_k\| = \min_{y_k} \|\beta e_{k+1}^{(1)} - \bar{H}_k y_k\| = \min_{y_k} \|\beta U_k^T e_{k+1}^{(1)} - R_k y_k\|. \quad (7.142)$$

This exposes some further structure to the problem, since $\beta U_k^T e_{k+1}^{(1)}$ is a vector of length $k+1$, but all entries in the last row of R_k are zero, $(R_k)_{k+1,j} = 0$ for $1 \leq j \leq k$. This means that for any choice of y_k , $R_k y_k$ is a vector of length $k+1$ whose last entry is zero. This yields a natural strategy for the minimization, since we have k degrees of freedom in y_k and can, at best, match k values in the vector $\beta U_k^T e_{k+1}^{(1)}$. Thus, from the perspective of the least-squares problem, the best possible strategy is to pick y_k so that $R_k y_k$ matches the first k entries of $\beta U_k^T e_{k+1}^{(1)}$. With this choice, the norm of the residual in that solution is easily expressed as $|\gamma_{k+1}|$, where we use γ_{k+1} to denote the $(k+1)$ th entry of the vector $\beta U_k^T e_{k+1}^{(1)}$.

An important consequence of this choice is that we no longer need to directly compute y_k in order to know the value of

$$\min_{u_k - u_0 \in \mathcal{K}_k(A, r_0)} \|f - Au_k\| = \min_{y_k} \|\beta U_k^T e_{k+1}^{(1)} - R_k y_k\|. \quad (7.143)$$

This leads to the GMRES algorithm, which, at each iteration,

- uses the Arnoldi form of the modified Gram–Schmidt algorithm to generate Q_{k+1} and $\bar{H}_k$ from Q_k and $\bar{H}_{k-1}$;
- updates the QR factorization of $\bar{H}_{k-1}$ to get the QR factorization of $\bar{H}_k = U_k R_k$; and
- updates $\beta U_k^T e_{k+1}^{(1)}$ from $\beta U_{k-1}^T e_k^{(1)}$, isolating γ_{k+1} .

Only after iterating until $|\gamma_{k+1}|$ is small enough to satisfy a stopping criterion do we solve for y_k from a $k \times k$ triangular system (at cost $\mathcal{O}(k^2)$) and then compute $u_k = u_0 + Q_k y_k$ (at cost $\mathcal{O}(nk)$). Each iteration of the Arnoldi algorithm requires one matrix-vector product to generate Aq_j in addition to j dot products and vector updates to orthogonalize this against the j vectors in the basis for the Krylov space, resulting in a total cost of $\mathcal{O}(nj)$ for the j th step and $\mathcal{O}(nk^2)$ in total for k steps.

For a complete picture of both the algorithm and its cost, we need to consider the specifics of how we update the QR factorization of $\bar{H}_{k-1}$ at each step and, subsequently, compute $\beta U_k^T e_{k+1}^{(1)}$. Here, we economize over the standard cost of QR factorization of a $(k+1) \times k$ matrix, $\mathcal{O}(k^3)$,

by taking advantage of the nearly upper-triangular structure of $\bar{H}_k$. Consider, first, the case of directly computing the QR factorization of a small system, $\bar{H}_4$, using Givens rotations. The original system, generated by four steps of the Arnoldi algorithm, is

$$\bar{H}_4 = \begin{bmatrix} h_{1,1} & h_{1,2} & h_{1,3} & h_{1,4} \\ h_{2,1} & h_{2,2} & h_{2,3} & h_{2,4} \\ 0 & h_{3,2} & h_{3,3} & h_{3,4} \\ 0 & 0 & h_{4,3} & h_{4,4} \\ 0 & 0 & 0 & h_{5,4} \end{bmatrix}. \quad (7.144)$$

To transform $\bar{H}_4$ into upper-triangular form, we multiply on the left by a series of Givens rotations computed to zero out the subdiagonal entries. Due to the nearly upper-triangular structure, only one Givens rotation per column of $\bar{H}_4$ is needed. The product of these Givens rotations (in order) becomes the matrix U_4^T . To zero out $h_{2,1}$, we compute $s_1 = h_{2,1}/\sqrt{h_{1,1}^2 + h_{2,1}^2}$ and $c_1 = h_{1,1}/\sqrt{h_{1,1}^2 + h_{2,1}^2}$ and define

$$\Omega_1^{(4)} = \begin{bmatrix} c_1 & s_1 & 0 & 0 & 0 \\ -s_1 & c_1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (7.145)$$

This gives

$$\Omega_1^{(4)} \bar{H}_4 = \begin{bmatrix} h_{1,1}^{(1)} & h_{1,2}^{(1)} & h_{1,3}^{(1)} & h_{1,4}^{(1)} \\ 0 & h_{2,2}^{(1)} & h_{2,3}^{(1)} & h_{2,4}^{(1)} \\ 0 & h_{3,2} & h_{3,3} & h_{3,4} \\ 0 & 0 & h_{4,3} & h_{4,4} \\ 0 & 0 & 0 & h_{5,4} \end{bmatrix}. \quad (7.146)$$

Next, we define $s_2 = h_{3,2}/\sqrt{(h_{2,2}^{(1)})^2 + h_{3,2}^2}$ and $c_2 = h_{2,2}^{(1)}/\sqrt{(h_{2,2}^{(1)})^2 + h_{3,2}^2}$ and multiply this matrix by

$$\Omega_2^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & c_2 & s_2 & 0 & 0 \\ 0 & -s_2 & c_2 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.147)$$

in order to zero out $h_{3,2}$. Continuing in this way, we define $\Omega_3^{(4)}$ and $\Omega_4^{(4)}$ so that

$$\Omega_4^{(4)} \Omega_3^{(4)} \Omega_2^{(4)} \Omega_1^{(4)} \bar{H}_4 = R_4, \quad (7.148)$$

where R_4 is an upper-triangular 5×4 matrix.

An important observation from this process is that, aside from adjusting for dimensions, the QR factorization of $\bar{H}_k$ differs from that of $\bar{H}_{k-1}$ by only a single Givens rotation, to eliminate $h_{k+1,k}$ once the Givens rotations from the QR factorization of $\bar{H}_{k-1}$ have been applied to the last column of $\bar{H}_k$. Thus, if we have the factorization of

$$\Omega_{k-1}^{(k-1)} \Omega_{k-2}^{(k-1)} \cdots \Omega_1^{(k-1)} \bar{H}_{k-1} = R_{k-1}, \quad (7.149)$$

we can compute that of

$$\Omega_k^{(k)} \Omega_{k-1}^{(k)} \cdots \Omega_1^{(k)} \bar{H}_k = R_k \quad (7.150)$$

by simply “augmenting” $\Omega_j^{(k-1)}$ by the $(k+1)$ th row and column of the identity matrix to construct $\Omega_j^{(k)}$ for $1 \leq j \leq k-1$, applying these transformations to the k th column of $\bar{H}_k$ to generate the first $k-1$ entries of the k th column of R_k , then computing the new Givens rotation, $\Omega_k^{(k)}$. The cost of this is $\mathcal{O}(k)$ to apply the $k-1$ known Givens rotations to the last column of $\bar{H}_k$ (once that column is computed) and $\mathcal{O}(1)$ to compute and apply $\Omega_k^{(k)}$. Similarly, since we have already computed

$$\beta U_{k-1}^T \mathbf{e}_k^{(1)} = \beta \Omega_{k-1}^{(k-1)} \Omega_{k-2}^{(k-1)} \cdots \Omega_1^{(k-1)} \mathbf{e}_k^{(1)}, \quad (7.151)$$

we need only extend this by zero and apply $\Omega_k^{(k)}$ to arrive at

$$\beta U_k^T \mathbf{e}_{k+1}^{(1)} = \beta \Omega_k^{(k)} \Omega_{k-1}^{(k)} \cdots \Omega_1^{(k)} \mathbf{e}_{k+1}^{(1)}, \quad (7.152)$$

at $\mathcal{O}(1)$ cost.

Adding these costs, we see that the total cost of k steps of GMRES is expressed as

$$\left(\sum_{j=1}^k c_1 n j + c_2 j + c_3 \right) + c_4 k^2 + c_5 n k, \quad (7.153)$$

where the constants above are those associated with the Arnoldi orthogonalization step (c_1), updating the QR factorization (c_2), and updating $\beta U_k^T \mathbf{e}_{k+1}^{(1)}$ (c_3) at each step of the iteration, along with solving for $\mathbf{y}_k$ (c_4) and $\mathbf{u}_k$ (c_5) once convergence has been reached after k steps. The dominant cost here, after summation, is $\mathcal{O}(k^2 n)$ from the orthogonalization step. While this is expensive, it is also critical to the numerical stability of the algorithm and, as such, is unavoidable.

At a high level, the GMRES algorithm is given in Algorithm 7.9. We do not know how many steps are required in advance and, generally, want to iterate until a stopping criterion is satisfied, requiring $|\gamma_{j+1}|$ to be below a given (absolute or relative) tolerance. Since the most efficient implementations of any algorithm are those with fixed memory footprints (so that memory can be allocated once at the start of the algorithm and not reallocated if initial arrays prove too small), having an “open-ended” loop should be avoided. Most implementations of GMRES address this issue by requiring a parameter, $k_{\max}$, that controls the maximum number of iterations in the Arnoldi loop. Of course, this introduces a complication, in that the maximum number of iterations may be reached without a suitable reduction in the residual. To overcome this, the standard approach is to use *restarted* GMRES, where, if $k_{\max}$ iterations are reached without convergence, the solution $\mathbf{u}_{k_{\max}}$ is constructed following the algorithm, and then a second iteration of GMRES is performed using this vector in place of $\mathbf{u}_0$. This “outer” iteration continues until either a maximum number of outer iterations is reached or the convergence tolerance is satisfied. We note that restarted GMRES is a necessary evil in order to maintain a fixed memory allocation for the algorithm but that the act of restarting throws out all of the evolving Krylov space and starts again from scratch. Thus, it is common to have convergence difficulties if the iteration restarts too soon, before a good approximation is obtained. In practice, it is wise to allow $k_{\max}$ to be as large as feasible and to rely on building up a quality Krylov space that can achieve the convergence bounds discussed below. If restarting is necessary, it is better to try and “save” some information from the current Krylov space to restart without a complete loss of the work that has been done to develop that space (see, e.g., [171]).

Now that we have the full GMRES algorithm, it is important to ask how well it works. A natural bound (without restarting) comes from the optimality property that, after k steps, we minimize $\|\mathbf{r}_k\|$ over all approximate solutions where $\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)$. Thus, the bound on $\|\mathbf{r}_k\|$ must be at least as good as that of the Chebyshev iterates from Theorem 7.53, as stated in Theorem 7.56.

Algorithm 7.9: Restarted GMRES algorithm

Input : A , linear system matrix
 $\mathbf{f}$, linear system right-hand side
 $\mathbf{u}_0$, initial guess
 $k_{\max}$, maximum subspace size and restart parameter
 tol , stopping tolerance
Output : $\mathbf{u}_k$, approximation to $\mathbf{u}$

```

1 while Not converged
2    $\mathbf{r}_0 = \mathbf{f} - A\mathbf{u}_0$ ,  $\beta = \|\mathbf{r}_0\|$ ,  $\mathbf{q}_1 = \mathbf{r}_0/\beta$ 
3   Initialize matrices  $Q = [\mathbf{q}_1]$  and  $\bar{H}$ 
4   for  $j = 1, 2, \dots, k_{\max}$ 
5      $\tilde{\mathbf{q}}_{j+1} = A\mathbf{q}_j$ 
6     for  $i = 1, \dots, j$ 
7        $h_{i,j} = \mathbf{q}_i^T \tilde{\mathbf{q}}_{j+1}$ 
8        $\tilde{\mathbf{q}}_{j+1} = \tilde{\mathbf{q}}_{j+1} - h_{i,j}\mathbf{q}_i$ 
9      $h_{j+1,j} = \|\tilde{\mathbf{q}}_{j+1}\|$ 
10     $\mathbf{q}_{j+1} = \frac{1}{h_{j+1,j}} \tilde{\mathbf{q}}_{j+1}$ 
11     $Q = [Q, \mathbf{q}_{j+1}]$ ,  $(\bar{H})_{i,j} = h_{i,j}$  for  $1 \leq i \leq j+1$ 
12    Compute Givens rotation to update  $\bar{H} = UR$ 
13    Update  $\beta U^T \mathbf{e}^{(1)}$ , with last entry  $\gamma_{j+1}$ 
14    if  $|\gamma_{j+1}| < tol$  or  $j = k_{\max}$ 
15       $k = j$ ,  $U_k = U$ ,  $R_k = R$ ,  $Q_k = Q$ 
16      Break
17   $\mathbf{y}_k = \operatorname{argmin}_{\mathbf{y}} \|\beta U_k^T \mathbf{e}_{k+1}^{(1)} - R_k \mathbf{y}\|$ 
18   $\mathbf{u}_k = \mathbf{u}_0 + Q_k \mathbf{y}_k$ 
19  if Converged
20    Break
21  else
22     $\mathbf{u}_0 = \mathbf{u}_k$ 

```

Theorem 7.56: GMRES convergence [194, Prop. 6.32]

Assume that $A = V\Lambda V^{-1}$ is diagonalizable, with eigenvector matrix V and diagonal matrix of eigenvalues Λ , with entries λ_i for $1 \leq i \leq n$. Let $p_k(z)$ be any polynomial of degree at most k with the property that $p_k(0) = 1$. Further, let θ , δ , and γ be any constants such that each eigenvalue λ_i for $1 \leq i \leq n$ satisfies $\lambda_i \in E(\theta, \delta, \gamma)$. Then, letting $\mathbf{r}_k$ be the residual for the k th iterate using the GMRES algorithm, both of the following bounds hold:

$$\|\mathbf{r}_k\| \leq \operatorname{cond}(V) \max_{1 \leq i \leq n} |p_k(\lambda_i)| \|\mathbf{r}_0\|, \quad (7.154a)$$

$$\|\mathbf{r}_k\| \leq \operatorname{cond}(V) \frac{T_k(\gamma/\delta)}{|T_k(\theta/\delta)|} \|\mathbf{r}_0\|. \quad (7.154b)$$

Proof. By construction, for the GMRES iterate $\mathbf{u}_k$, we have

$$\|\mathbf{r}_k\| = \min_{\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)} \|\mathbf{f} - A\mathbf{u}_k\|. \quad (7.155)$$

As in Theorem 7.53,

$$\mathbf{r}_k = p_k(A)\mathbf{r}_0 = Vp_k(\Lambda)V^{-1}\mathbf{r}_0 \quad (7.156)$$

for *some* polynomial, $p_k(A)$, which is chosen to minimize $\|\mathbf{r}_k\|$ over all possible vectors in the Krylov space, corresponding to all possible polynomials of degree at most k with constraint that $p_k(0) = 1$. The bound in the theorem follows from bounding $\|\mathbf{r}_k\|$ termwise in the above.

Since the Chebyshev iterate, $\mathbf{u}_k$, satisfies the condition that $\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)$, the GMRES residual satisfies the bound from Theorem 7.53 for any θ , δ , and γ that satisfy the assumptions of that theorem. $\square$

The second bound in Theorem 7.56 is slightly stronger than Theorem 7.53, since it holds for any θ , δ , and γ such that $\lambda_i \in E(\theta, \delta, \gamma)$ for $1 \leq i \leq n$, while the Chebyshev bound is dependent on knowing these values *a priori*. This is an important improvement of GMRES (and Krylov methods in general): we no longer need to have detailed knowledge of the spectrum of A in order to get the best-possible performance from the method. The first bound also allows us to achieve better bounds on convergence in cases where the spectrum is “clustered” around multiple points in the complex plane, rather than contained in a single ellipse.

For example, consider the case when we know constants θ , δ , and γ such that $\lambda_i \in E(\theta, \delta, \gamma)$ for $1 \leq i \leq n-1$, but λ_n lies far outside this ellipse. A good bound on the error from the Chebyshev iteration requires adjusting the bounds on the ellipse in order to include λ_n within $E(\theta, \delta, \gamma)$; this can easily have a negative impact on the convergence properties of the method. However, for GMRES, we make a specific choice of $\mathbf{u}_k$ to construct an upper bound on $\|\mathbf{r}_k\|$, choosing $\mathbf{u}_k - \mathbf{u}_0 = p_k(A)\mathbf{r}_0$ for $p_k(A) = p_{k-1}(A) \cdot \left(I - \frac{1}{\lambda_n}A\right)$, where $p_{k-1}(A)$ is the Chebyshev polynomial of degree $k-1$ acting on A . This gives the bound

$$\|\mathbf{r}_k\| \leq \text{cond}(V) \frac{T_{k-1}(\gamma/\delta)}{|T_{k-1}(\theta/\delta)|} \|\mathbf{r}_0\| \quad (7.157)$$

for the original parameters, since the chosen approximation has no error in the direction associated with λ_n . This is sketched for the case of a real-valued spectrum in Figure 7.8.1,

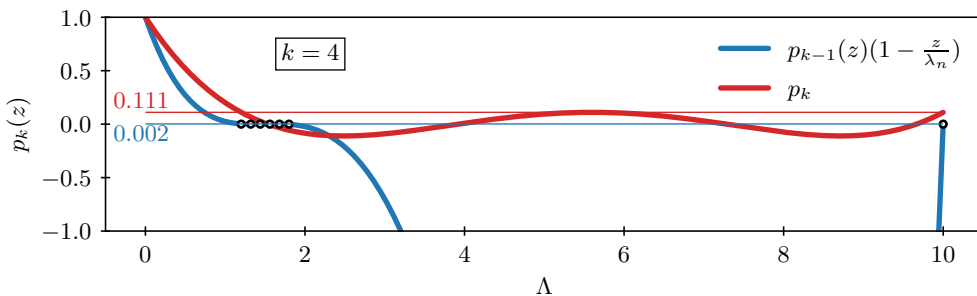


Figure 7.8.1. Comparison of GMRES convergence bounds given in Theorem 7.56 for $k = 4$, with the red line indicating the second bound, constructed by using a Chebyshev polynomial over the smallest interval that contains all of the eigenvalues, and the blue line constructed by using the first bound with polynomial chosen as the product of the third-degree Chebyshev polynomial over the cluster of eigenvalues at left and the linear function that is zero at λ_n at right.

where the first bound with an appropriately chosen polynomial gives a convergence bound of $\|r_4\| \leq 0.002\|r_0\|$, while the second bound, using the Chebyshev polynomial over the interval containing the entire spectrum, gives a convergence bound of $\|r_4\| \leq 0.111\|r_0\|$. Similar extensions are possible, for example, in the case where there are two ellipses that contain all of the spectrum but are far apart in the complex plane.

In the next section, we consider what happens when the matrix is symmetric and when it is symmetric and positive definite. Here, the GMRES algorithm simplifies significantly, leading to other classical Krylov methods: MINRES and CG.

7.9 ■ Krylov methods: The Lanczos algorithm, MINRES, and conjugate gradient

Objectives

In the previous section, we considered the GMRES algorithm for solving the system $Au = f$, which makes no assumptions on the properties of the matrix A . Many of the linear systems that we encounter in practice, however, have significant structure or specific algebraic properties. Thus, it is natural to ask what happens to the problem of approximating solutions to $Au = f$ by a Krylov method when A has additional properties. Therefore, in this section, you will understand how the Arnoldi algorithm is modified to the Lanczos algorithm when A is symmetric and discuss the derivation of the MINRES algorithm, departing from GMRES to take advantage of symmetry. Next, you will consider a new approach by forming iterates whose residuals are orthogonal to the Krylov space, resulting in the CG algorithm when A is symmetric and positive definite.

We start with the case where A is symmetric, $A = A^T$. While we consider only the case of real-valued matrices here, the same argument extends to the complex-valued case when A is Hermitian. If we revisit the Arnoldi algorithm (see Algorithm 7.8) and look more closely at the relationship $AQ_k = Q_{k+1}\bar{H}_k$, we conclude that $Q_k^T A Q_k = Q_k^T Q_{k+1} \bar{H}_k$. Since $Q_k^T Q_{k+1}$ is a $k \times (k+1)$ matrix and Q_{k+1} is formed by appending a column to Q_k that is orthogonal to all of its columns, we obtain

$$Q_k^T Q_{k+1} = [I \quad \mathbf{0}], \quad (7.158)$$

where $\mathbf{0}$ is the zero vector of length k . We then define

$$(H_k)_{i,j} = \begin{cases} h_{i,j} & \text{if } 1 \leq j \leq k, 1 \leq i \leq \min(j+1, k), \\ 0 & \text{otherwise,} \end{cases} \quad (7.159)$$

so that $Q_k^T A Q_k = Q_k^T Q_{k+1} \bar{H}_k = H_k$.

Since A is symmetric, so is $Q_k^T A Q_k$. This implies that H_k must also be symmetric. However, H_k is an *upper Hessenberg* matrix, with entries appearing only in positions (i, j) where $1 \leq i \leq j+1$. In order for such a matrix to be symmetric, it must be tridiagonal, simply by a matching argument:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}^T = \begin{bmatrix} \times & \times & & & \\ \times & \times & \times & & \\ \times & \times & \times & \times & \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix}. \quad (7.160)$$

Algorithm 7.10: Lanczos algorithm to compute orthogonal basis of $\mathcal{K}_{k+1}(A, \mathbf{q}_1)$ when A is symmetric

Input : A , matrix

$\mathbf{q}_1$, initial vector with $\|\mathbf{q}_1\| = 1$

Output : $\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_{k+1}$, orthogonal basis

```

1  $\beta_1 = 0, \mathbf{q}_0 = \mathbf{0}$ 
2 for  $j = 1, 2, \dots, k$ 
3    $\tilde{\mathbf{q}}_{j+1} = A\mathbf{q}_j - \beta_j\mathbf{q}_{j-1}$ 
4    $\alpha_j = \mathbf{q}_j^T \tilde{\mathbf{q}}_{j+1}$ 
5    $\tilde{\mathbf{q}}_{j+1} = \tilde{\mathbf{q}}_{j+1} - \alpha_j\mathbf{q}_j$ 
6    $\beta_{j+1} = \|\tilde{\mathbf{q}}_{j+1}\|$ 
7    $\mathbf{q}_{j+1} = \frac{1}{\beta_j}\tilde{\mathbf{q}}_{j+1}$ 

```

This implies that $h_{i,j} = 0$ if $|i - j| > 1$, meaning that all entries not in the tridiagonal band must be zero. This, in turn, guarantees that $\mathbf{q}_i^T \tilde{\mathbf{q}}_{j+1} = 0$ whenever $j > i + 1$, allowing for a major simplification in the Arnoldi algorithm when A is symmetric, leading to the Lanczos algorithm, given in Algorithm 7.10.

Comparing Algorithm 7.10 with Algorithm 7.8, we see that the loop over i is eliminated. This comes about due to the symmetric tridiagonal nature of H_k , which requires that

$$h_{i,j} = 0 \text{ for } 1 \leq i \leq j - 2, \quad (7.161a)$$

$$\beta_j := h_{j-1,j} = h_{j,j-1}, \quad (7.161b)$$

$$\alpha_j := h_{j,j} = \mathbf{q}_j^T \tilde{\mathbf{q}}_{j+1}, \quad (7.161c)$$

$$\text{and } \beta_{j+1} := h_{j+1,j} = \|\tilde{\mathbf{q}}_{j+1}\|. \quad (7.161d)$$

From this, we see that

$$Q_k^T A Q_k = \begin{bmatrix} \alpha_1 & \beta_2 & & & \\ \beta_2 & \alpha_2 & \beta_3 & & \\ & \beta_3 & \ddots & \ddots & \\ & & \ddots & \ddots & \beta_k \\ & & & \beta_k & \alpha_k \end{bmatrix} =: T_k. \quad (7.162)$$

Similarly, we define $\bar{T}_k$ in place of $\bar{H}_k$, writing

$$\bar{T}_k := \begin{bmatrix} \alpha_1 & \beta_2 & & & \\ \beta_2 & \alpha_2 & \beta_3 & & \\ & \beta_3 & \ddots & \ddots & \\ & & \ddots & \ddots & \beta_k \\ & & & \beta_k & \alpha_k \\ & & & & \beta_{k+1} \end{bmatrix}, \quad (7.163)$$

with $AQ_k = Q_{k+1}^T \bar{T}_k$.

7.9.1 ■ MINRES

Now, just as in GMRES, we take an approximation $\mathbf{u}_k$ with $\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)$ and write it as $\mathbf{u}_k = \mathbf{u}_0 + Q_k \mathbf{y}_k$. Then, minimizing the norm of the residual gives us

$$\min_{\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)} \|\mathbf{f} - A\mathbf{u}_k\| = \min_{\mathbf{y}_k} \|\mathbf{r}_0 - AQ_k \mathbf{y}_k\| \quad (7.164a)$$

$$= \min_{\mathbf{y}_k} \|\mathbf{r}_0 - Q_{k+1} \bar{T}_k \mathbf{y}_k\| = \min_{\mathbf{y}_k} \|Q_{k+1}^T \mathbf{r}_0 - \bar{T}_k \mathbf{y}_k\|. \quad (7.164b)$$

Writing $\beta_1 = \|\mathbf{r}_0\|$ results in

$$\min_{\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)} \|\mathbf{f} - A\mathbf{u}_k\| = \min_{\mathbf{y}_k} \|\beta_1 \mathbf{e}_{k+1}^{(1)} - \bar{T}_k \mathbf{y}_k\|, \quad (7.165)$$

where we can solve this system by computing the QR factorization of $\bar{T}_k$, as we did with GMRES. Here, we keep the advantage of GMRES that $\bar{T}_k = U_k R_k$ can be easily computed as an update to $\bar{T}_{k-1} = U_{k-1} R_{k-1}$ by “augmenting” the dimension of U_{k-1} and applying one more Givens rotation, giving

$$U_k^T = \Omega_k^{(k)} \Omega_{k-1}^{(k)} \cdots \Omega_1^{(k)}. \quad (7.166)$$

However, since $\bar{T}_k$ is tridiagonal, R_k is upper triangular but with only three nonzeros per row, further reducing the work needed in this case.

One final step to improve efficiency comes from taking advantage of the structure of the QR factorization itself. Instead of taking the full QR factorization, where U_k is a $(k+1) \times (k+1)$ matrix and R_k is a $(k+1) \times k$ upper-triangular matrix, we can make use of the “thin” QR, where we compute only the first k columns of U_k . This is feasible since the $(k+1)$ th row of R_k will be all zeros. As with GMRES, we again note that this introduces a departure from a common notation: the columns of U_k are *not* denoted by $\mathbf{u}_k$, the solution vector.

Using thin QR gives $\bar{T}_k = \bar{U}_k \bar{R}_k$, but with $\bar{U}_k$ now being a $(k+1) \times k$ matrix and R_k being a $k \times k$ matrix. Writing $\mathbf{y}_k = \bar{R}_k^{-1} \bar{U}_k^T (\beta_1 \mathbf{e}_{k+1}^{(1)})$ then gives

$$\mathbf{u}_k = \mathbf{u}_0 + Q_k \mathbf{y}_k = \mathbf{u}_0 + Q_k \bar{R}_k^{-1} \bar{U}_k^T (\beta_1 \mathbf{e}_{k+1}^{(1)}). \quad (7.167)$$

Instead of evolving Q_k and $\bar{R}_k$ explicitly (as in GMRES), we now write $W_k = Q_k \bar{R}_k^{-1}$ and determine the columns of W_k iteratively as the solutions of $W_k \bar{R}_k = Q_k$. The structure of $\bar{R}_k$ guarantees that the last column of W_k depends only on the last column of Q_k (the vector $\mathbf{q}_k$) and the prior two columns of W_k . This allows us to combine

$$\mathbf{u}_{k-1} = \mathbf{u}_0 + W_{k-1} U_{k-1}^T (\beta_1 \mathbf{e}_k^{(1)}) \quad \text{and} \quad \mathbf{u}_k = \mathbf{u}_0 + W_k U_k^T (\beta_1 \mathbf{e}_{k+1}^{(1)}) \quad (7.168)$$

to write $\mathbf{u}_k = \mathbf{u}_{k-1} + \xi_k \mathbf{w}_k$ for a value, ξ_k , that is computed from the Givens rotation that updates the QR factorization of $\bar{T}_{k-1}$ into that of $\bar{T}_k$.

The resulting algorithm is known as MINRES and is presented in Algorithm 7.11. The algorithm becomes notationally complex because much of the explicit computation from GMRES has been replaced by implicit computation; however, it is also much more efficient because we take advantage of the full structure of the symmetric system. Algorithm 7.11 can be naturally broken down into three parts. The first stage is exactly the k th step of the Lanczos algorithm, slightly compressed from Algorithm 7.10. The second, computing several scalar values, comes from the update to the QR factorization by computing a new Givens rotation. The final stage computes the new vector $\mathbf{w}_k$ as described above and then updates $\mathbf{u}_k$ and the associated residual norm.

Algorithm 7.11: MINRES algorithm

Input : A , linear system matrix
 $\mathbf{f}$, linear system right-hand side
 $\mathbf{u}_0$, initial guess

Output : $\mathbf{u}_k$, k th iterate and approximation to $\mathbf{u}$

```

1  $\mathbf{q}_1 = \mathbf{f} - A\mathbf{u}_0$ ,  $\beta_1 = \|\mathbf{q}_1\|$ 
2 Set  $\mathbf{q}_0 = \mathbf{0}$ ,  $\mathbf{w}_{-1} = \mathbf{w}_0 = \mathbf{0}$ 
3 Set  $\gamma_0 = \gamma_1 = 1$ ,  $\sigma_0 = \sigma_1 = 0$ , and  $\eta = \beta_1$ 
4 for  $k = 1, 2, \dots$  {Lanczos step}
5    $\mathbf{q}_k = \frac{1}{\beta_k} \mathbf{q}_k$ 
6    $\alpha_k = \mathbf{q}_k^T A \mathbf{q}_k$ 
7    $\mathbf{q}_{k+1} = A \mathbf{q}_k - \alpha_k \mathbf{q}_k - \beta_k \mathbf{q}_{k-1}$ 
8    $\beta_{k+1} = \|\mathbf{q}_{k+1}\|$ 

9    $\delta = \gamma_k \alpha_k - \gamma_{k-1} \sigma_k \beta_k$  {Givens rotations and QR}
10   $\rho_1 = \sqrt{\delta^2 + \beta_{k+1}^2}$ ,  $\rho_2 = \sigma_k \alpha_k + \gamma_{k-1} \gamma_k \beta_k$ ,  $\rho_3 = \sigma_{k-1} \beta_k$ 
11   $\gamma_{k+1} = \delta / \rho_1$ ,  $\sigma_{k+1} = \beta_{k+1} / \rho_1$ 

12   $\mathbf{w}_k = (\mathbf{q}_k - \rho_2 \mathbf{w}_{k-1} - \rho_3 \mathbf{w}_{k-2}) / \rho_1$  {update  $\mathbf{w}_k$ }
13   $\mathbf{u}_k = \mathbf{u}_{k-1} + \eta \gamma_{k+1} \mathbf{w}_k$  {new approximation}
14   $\|\mathbf{r}_k\| = |\sigma_{k+1}| \|\mathbf{r}_{k-1}\|$  {update residual norm}
15   $\eta = -\sigma_{k+1} \eta$ 

```

7.9.2 • Conjugate gradient

In general, the MINRES algorithm arises from using the Lanczos relation, that is, $AQ_k = Q_{k+1}\bar{T}_k$, and then minimizing the residual over the Krylov space, $\mathbf{u}_k = \mathbf{u}_0 + Q_k \mathbf{y}_k$, giving

$$\min_{\mathbf{u}_k} \|\mathbf{f} - A\mathbf{u}_k\| = \min_{\mathbf{y}_k} \|\mathbf{r}_0 - AQ_k \mathbf{y}_k\| = \min_{\mathbf{y}_k} \|Q_{k+1}^T \mathbf{r}_0 - \bar{T}_k \mathbf{y}_k\|. \quad (7.169)$$

Another approach is to require that the residual, $\mathbf{r}_k = \mathbf{f} - A\mathbf{u}_k$, be orthogonal (conjugate) to all vectors in the Krylov space $\mathcal{K}_k(A, \mathbf{r}_0) = \text{span}(Q_k)$. This gives

$$Q_k^T (\mathbf{f} - A\mathbf{u}_k) = Q_k^T (\mathbf{r}_0 - AQ_k \mathbf{y}_k) = 0 \quad (7.170a)$$

$$\Rightarrow Q_k^T A Q_k \mathbf{y}_k = T_k \mathbf{y}_k = Q_k^T \mathbf{r}_0 = \beta_1 \mathbf{e}_k^{(1)}, \quad (7.170b)$$

where T_k is defined by the Lanczos iteration as above in equation (7.162).

We now consider this idea, but also the case when A is both symmetric and positive definite. Then, T_k will retain both properties, and we can compute its Cholesky factorization as

$$T_k = L_k U_k = \begin{bmatrix} 1 & 0 & & & \\ \lambda_2 & 1 & & & \\ & \ddots & \ddots & \ddots & \\ & & \lambda_{k-1} & 1 & 0 \\ & & & \lambda_k & 1 \end{bmatrix} \begin{bmatrix} \eta_1 & \beta_2 & & & \\ 0 & \eta_2 & \beta_3 & & \\ & \ddots & \ddots & \ddots & \\ & & 0 & \eta_{k-1} & \beta_k \\ & & & 0 & \eta_k \end{bmatrix}, \quad (7.171)$$

with the relations that $\eta_1 = \alpha_1$, $\lambda_k = \beta_k / \eta_{k-1}$, and $\eta_k = \alpha_k - \lambda_k \beta_k$. Here, again, we continue to have matrix, U_k (now from an LU factorization, not a QR factorization), whose columns are only indirectly related to the solution vector, $\mathbf{u}_k$. Writing $\mathbf{u}_k = \mathbf{u}_0 + Q_k \mathbf{y}_k$ with $\mathbf{y}_k = T_k^{-1} (\beta_1 \mathbf{e}_k^{(1)})$, we then have

$$\mathbf{u}_k = \mathbf{u}_0 + Q_k U_k^{-1} L_k^{-1} (\beta_1 \mathbf{e}_k^{(1)}). \quad (7.172)$$

To simplify the calculation, we split the inverses of the two factors into two pieces, writing $P_k = Q_k U_k^{-1}$ and $\mathbf{z}_k = L_k^{-1} (\beta_1 \mathbf{e}_k^{(1)})$. For the first, note that this implies that $P_k U_k = Q_k$ or, componentwise, $\eta_j \mathbf{p}_j + \beta_j \mathbf{p}_{j-1} = \mathbf{q}_j$ for $1 < j \leq k$, where $\mathbf{p}_j$ and $\mathbf{q}_j$ are the j th columns of P_k and Q_k , respectively. This gives us a natural iteration to compute the columns of P_k as $\mathbf{p}_j = \frac{1}{\eta_j} (\mathbf{q}_j - \beta_j \mathbf{p}_{j-1})$, where $\mathbf{p}_0 = \mathbf{r}_0$ arises as a special case. Similarly, we derive an iteration for the entries in $\mathbf{z}_k$ by noting that we have, for $1 < j \leq k$, the relationship $L_j \mathbf{z}_j = \beta_1 \mathbf{e}_j^{(1)}$, so we can iteratively define the last entry of $\mathbf{z}_j$ as $\zeta_j = -\lambda_j \zeta_{j-1}$, where ζ_{j-1} is the last entry in $\mathbf{z}_{j-1}$. This gives

$$\mathbf{z}_j = L_j^{-1} (\beta_1 \mathbf{e}_j^{(1)}) = \begin{bmatrix} \mathbf{z}_{j-1} \\ \zeta_j \end{bmatrix} \text{ for } 1 < j \leq k. \quad (7.173)$$

These two together allow us to iteratively compute the approximations for $1 \leq j \leq k$ by writing

$$\mathbf{u}_j = \mathbf{u}_0 + P_j \mathbf{z}_j = \mathbf{u}_0 + P_{j-1} \mathbf{z}_{j-1} + \zeta_j \mathbf{p}_j = \mathbf{u}_{j-1} + \zeta_j \mathbf{p}_j \quad (7.174)$$

and using the iteration above to compute $\mathbf{p}_j$ at each step.

While the above iteration could be used to generate approximate solutions to $A\mathbf{u} = \mathbf{f}$, it turns out to be mildly unstable when implemented in this way. Instead, we make use of two important properties of the matrices introduced above. First, consider

$$P_k^T A P_k = U_k^{-T} Q_k^T A Q_k U_k^{-1} = U_k^{-T} T_k U_k = U_k^{-T} L_k, \quad (7.175)$$

since $T_k = L_k U_k$. Note, however, that $U_k^{-T} L_k$ must be a lower-triangular matrix, but that $P_k^T A P_k$ must be symmetric. This implies that $P_k^T A P_k$ must be diagonal and, so, its columns must be orthogonal in the inner product induced by A , $\mathbf{p}_i^T A \mathbf{p}_j = 0$ when $i \neq j$. We call $\{\mathbf{p}_j\}$ an A -conjugate set of vectors because of this property. Second, we notice that the sequence of residuals has an important additional property,

$$\mathbf{f} - A\mathbf{u}_k = \mathbf{r}_0 - A Q_k \mathbf{y}_k = \mathbf{r}_0 - Q_{k+1} \bar{T}_k \mathbf{y}_k \quad (7.176a)$$

$$= \mathbf{r}_0 - Q_k T_k \mathbf{y}_k - \beta_{k+1} \mathbf{q}_{k+1} = -\beta_{k+1} \mathbf{q}_{k+1}, \quad (7.176b)$$

since $T_k \mathbf{y}_k = Q_k^T \mathbf{r}_0 = \beta_1 \mathbf{e}_k^{(1)}$. Thus, not only is the residual after k steps orthogonal to the previous Krylov space, but it lies precisely in the direction of the next vector in the Lanczos basis, $\mathbf{q}_{k+1}$.

We use these properties to recast the Lanczos algorithm without explicitly doing the Lanczos process. Since we know that $\mathbf{p}_k = \frac{1}{\eta_k} (\mathbf{q}_k - \beta_k \mathbf{p}_{k-1})$, we can compute a *scaled* version of $\{\mathbf{p}_j\}$ by using the following facts:

1. $\mathbf{q}_k$ points in the same direction as $\mathbf{r}_{k-1}$.
2. $\mathbf{p}_k$ is a linear combination of $\mathbf{q}_k$ and $\mathbf{p}_{k-1}$.
3. $\mathbf{p}_i^T A \mathbf{p}_j = 0$ for all $i \neq j$.

To do this, we first increment the indices for $\{\mathbf{p}_j\}$ and redefine β_k for later convenience, writing

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k, \quad (7.177)$$

and then we use $\mathbf{p}_k^T A \mathbf{p}_{k+1} = 0$ to find that $\beta_k = -\mathbf{p}_k^T A \mathbf{r}_{k+1} / \mathbf{p}_k^T A \mathbf{p}_k$. Next, knowing that the increments in the solution and residual take the forms

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \alpha_k \mathbf{p}_k, \quad (7.178a)$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k \quad (7.178b)$$

for some constant α_k , we use the orthogonality of the residuals, $\mathbf{r}_k^T \mathbf{r}_{k+1} = 0$, to solve for $\alpha_k = \mathbf{r}_k^T \mathbf{r}_{k+1} / \mathbf{r}_k^T A \mathbf{p}_k$. One final optimization to save work comes from noticing that in the expressions for α_k and β_k above, we can use the relations that

$$\mathbf{r}_k^T A \mathbf{p}_k = (\mathbf{p}_k - \beta_{k-1} \mathbf{p}_{k-1})^T, \quad A \mathbf{p}_k = \mathbf{p}_k^T A \mathbf{p}_k, \quad (7.179)$$

so $\alpha_k = \mathbf{r}_k^T \mathbf{r}_{k+1} / \mathbf{p}_k^T A \mathbf{p}_k$. Then, writing $A \mathbf{p}_k = -\frac{1}{\alpha_k}(\mathbf{r}_{k+1} - \mathbf{r}_k)$ yields

$$\beta_k = -\frac{\mathbf{p}_k^T A \mathbf{r}_{k+1}}{\mathbf{p}_k^T A \mathbf{p}_k} = \frac{1}{\alpha_k} \frac{(\mathbf{r}_{k+1} - \mathbf{r}_k)^T \mathbf{r}_{k+1}}{\mathbf{p}_k^T A \mathbf{p}_k} = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}. \quad (7.180)$$

This gives the usual form of the CG algorithm, as given in Algorithm 7.12.

Algorithm 7.12: CG algorithm

Input : A , linear system matrix
 $\mathbf{f}$, linear system right-hand side
 $\mathbf{u}_0$, initial guess

Output : $\mathbf{u}_{k+1}$, $(k+1)$ th iterate and approximation to $\mathbf{u}$

```

1  $\mathbf{r}_0 = \mathbf{f} - A\mathbf{u}_0$ ,  $\mathbf{p}_0 = \mathbf{r}_0$ 
2 for  $k = 0, 1, 2, \dots$ 
3    $\alpha_k = \mathbf{r}_k^T \mathbf{r}_k / \mathbf{p}_k^T A \mathbf{p}_k$ 
4    $\mathbf{u}_{k+1} = \mathbf{u}_k + \alpha_k \mathbf{p}_k$ 
5    $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k$ 
6    $\beta_k = \mathbf{r}_{k+1}^T \mathbf{r}_{k+1} / \mathbf{r}_k^T \mathbf{r}_k$ 
7    $\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$ 

```

Finally, we assess the computational cost. If we store the inner products $\mathbf{r}_k^T \mathbf{r}_k$, then this requires only one matrix-vector product, two dot products, and three vector updates per iteration and requires storage of only four vectors, since $\mathbf{u}_k$ and $\mathbf{r}_k$ can be overwritten immediately when their new values are computed, as can $\mathbf{p}_k$ and $A \mathbf{p}_k$. As in the case of GMRES, a convergence bound is immediate, since the iterates generated must be at least as good as the iterates from the Chebyshev iteration (see Corollary 7.51). This is summarized in Theorem 7.57, though we omit the proof.

Theorem 7.57: CG convergence [194, Thm. 6.29]

Let A be symmetric and positive definite, with $\text{cond}(A) = \lambda_{\max}(A) / \lambda_{\min}(A)$, and let $\mathbf{u}_k$ be the k th iterate of the CG algorithm. Then, for any right-hand side, $\mathbf{f}$, and initial guess $\mathbf{u}_0$, we have

$$\|\mathbf{u} - \mathbf{u}_k\|_A \leq 2 \left(\frac{\sqrt{\text{cond}(A)} - 1}{\sqrt{\text{cond}(A)} + 1} \right)^k \|\mathbf{u} - \mathbf{u}_0\|_A. \quad (7.181)$$

As for GMRES, the values of the extremal eigenvalues of A need not be known in order to arrive at a bound that depends only on these. Furthermore, just like the Chebyshev iteration, small numbers of isolated eigenvalues also do not have a large effect on the convergence of CG, since the bound above can be similarly modified to account for them.

7.10 ■ Preconditioning

Objectives

Having explored a broad (but not exhaustive) family of polynomial iterations, we now turn back to the question of choosing a preconditioner for the system, with the aim of solving an equivalent linear system to $Au = f$ but for which we get improved convergence behavior from the above Krylov methods. Therefore, in this section you will be introduced to the ideas of “left” and “right” preconditioning. You will analyze how right-preconditioned GMRES can be adapted for different preconditioners at each iteration, leading to the flexible GMRES algorithm, and, in the symmetric case, derive the preconditioned MINRES and CG algorithms.

Preconditioning is most straightforward for a general method like GMRES, where we simply replace $Au = f$ with an equivalent system:

$$MAu = Mf \text{ or,} \quad (7.182a)$$

$$AMz = f, \text{ with } u = Mz. \quad (7.182b)$$

When A is full rank, we typically assume that M is also full rank, leading to a unique solution for u . When A is singular, it is natural to make *compatible* assumptions on the nullspace of M , so that the set of solutions is unchanged by the preconditioning.

We distinguish between the two types of preconditioning introduced above as “left” and “right” preconditioning. For left-preconditioned GMRES, there are essentially no changes to Algorithm 7.9 introduced in Section 7.8. We now define the initial residual as the *preconditioned* residual, $r_0 = M(f - Au_0)$, and use $\tilde{q}_{j+1} = MAq_j$ in the Arnoldi step. Otherwise, the steps of the algorithm stay the same, and it is equivalent to applying GMRES directly to the system $MAu = Mf$. For right-preconditioned GMRES, similarly small changes are needed. Now, given an initial guess, u_0 , we begin with the true initial residual, $r_0 = f - Au_0$, but replace the Arnoldi step with $\tilde{q}_{j+1} = AMq_j$. At solution, $Q_k y_k$ is now an approximation to z and, so, we write the GMRES approximation as $u_k = u_0 + MQ_k y_k$. Again, this is equivalent to applying GMRES to the linear system $AMz = f$, with initial guess z_0 that satisfies $u_0 = Mz_0$. However, the vector z_0 is not needed, since neither the original residual nor the generated approximation to u depends on it.

These two approaches generate the Krylov spaces $\mathcal{K}_k(MA, M(f - Au_0))$ for left preconditioning and $\mathcal{K}_k(AM, f - Au_0)$ for right preconditioning. If MA and/or AM have much better spectral properties than A (in the sense of the GMRES convergence bounds in Theorem 7.56), then convergence can be greatly improved. For the case that M is invertible, $M^{-1}(MA)M = AM$, and, so, MA and AM are similar matrices, with equal eigenvalues but different eigenvector matrices. Thus, they may lead to different convergence bounds. An inherent danger with left preconditioning, however, is that it changes the minimization problem for GMRES from $\min_{u_k} \|f - Au_k\|$ to $\min_{u_k} \|Mf - MAu_k\|$. If M is very poorly conditioned (as might be expected for a good preconditioner of an ill-conditioned matrix, A), this can greatly change the approximation generated. Fortunately, this does not happen for right preconditioning, since we still solve the original minimization problem, now posed as $\min_{z_k} \|f - AMz_k\|$. Thus, so long as we can accurately compute the right-preconditioned solution update, $MQ_k y_k$,

we recover a solution that minimizes the residual of the original problem over the modified Krylov space.

An additional advantage of right preconditioning is demonstrated via the more general *flexible* GMRES (FGMRES) algorithm (outlined in Algorithm 7.13), where the preconditioner is allowed to change from one iteration to the next. For simple problems, this does not necessarily seem to be a desirable property; however, for many difficult systems of PDEs, it is natural to want to embed one Krylov solver within another, and FGMRES allows this to be done. In this approach, we replace the initial step of each Arnoldi iteration, $\tilde{\mathbf{q}}_{j+1} = A\mathbf{M}\mathbf{q}_j$ for right-preconditioned GMRES, with $\tilde{\mathbf{q}}_{j+1} = A\mathbf{M}_j\mathbf{q}_j$, where the preconditioner now depends on j . In addition to storing Q_k , we now store a matrix Z_k , whose columns are given by $\mathbf{M}_j\mathbf{q}_j$, and reconstruct the approximation solution as $\mathbf{u}_k = \mathbf{u}_0 + Z_k\mathbf{y}_k$. In this setting, the Arnoldi relation becomes $AZ_k = Q_{k+1}\bar{H}_k$, so there are no changes in the resulting minimization algorithm. When $\mathbf{M}_j = \mathbf{M}$ for all j , it is equivalent to right-preconditioned GMRES. When $\mathbf{M}_j = \mathbf{I}$ for

Algorithm 7.13: FGMRES algorithm

Input : A , linear system matrix
 $\mathbf{M}_j$, preconditioning matrix defined at iteration j
 $\mathbf{f}$, linear system right-hand side
 $\mathbf{u}_0$, initial guess
 $k_{\max}$, maximum subspace size and restart parameter
 tol , stopping tolerance

Output : $\mathbf{u}_k$, approximation to $\mathbf{u}$

```

1 while Not converged
2    $\mathbf{r}_0 = \mathbf{f} - A\mathbf{u}_0$ ,  $\beta = \|\mathbf{r}_0\|$ ,  $\mathbf{q}_1 = \mathbf{r}_0/\beta$ 
3   Initialize matrices  $Q = [\mathbf{q}_1]$ ,  $Z = []$ , and  $\bar{H}$ 
4   for  $j = 1, 2, \dots, k_{\max}$ 
5      $\mathbf{z}_j = \mathbf{M}_j\mathbf{q}_j$ 
6      $\tilde{\mathbf{q}}_{j+1} = A\mathbf{z}_j$ 
7     for  $i = 1, \dots, j$ 
8        $h_{i,j} = \mathbf{q}_i^T \tilde{\mathbf{q}}_{j+1}$ 
9        $\tilde{\mathbf{q}}_{j+1} = \tilde{\mathbf{q}}_{j+1} - h_{i,j}\mathbf{q}_i$ 
10     $h_{j+1,j} = \|\tilde{\mathbf{q}}_{j+1}\|$ 
11     $\mathbf{q}_{j+1} = \tilde{\mathbf{q}}_{j+1}/h_{j+1,j}$ 
12     $Q = [Q, \mathbf{q}_{j+1}]$ ,  $Z = [Z, \mathbf{z}_j]$ ,  $(\bar{H})_{i,j} = h_{i,j}$  for  $1 \leq i \leq j+1$ 
13    Compute Givens rotation to update  $\bar{H} = UR$ 
14    Update  $\beta U^T \mathbf{e}^{(1)}$ , with last entry  $\gamma_{j+1}$ 
15    if  $|\gamma_{j+1}| < tol$  or  $j = k_{\max}$ 
16       $k = j$ ,  $U_k = U$ ,  $R_k = R$ ,  $Z_k = Z$ 
17      Break
18   $\mathbf{y}_k = \operatorname{argmin}_{\mathbf{y}} \|\beta U_k^T \mathbf{e}_{k+1}^{(1)} - R_k \mathbf{y}\|$ 
19   $\mathbf{u}_k = \mathbf{u}_0 + Z_k \mathbf{y}_k$ 
20  if Converged
21    Break
22  else
23     $\mathbf{u}_0 = \mathbf{u}_k$ 

```

all j , it is equivalent to unpreconditioned GMRES. One important point here is that FGMRES has an additional memory cost over classical right-preconditioned GMRES due to the storage of both $\mathbf{z}_j$ and $\mathbf{q}_j$; however, this cost also has a benefit, as it requires one less application of the preconditioner, since we can directly compute $Z_k \mathbf{y}_k$ at the final step, instead of $M Q_k \mathbf{y}_k$ in classical right-preconditioned GMRES. In many applications, this is a favorable trade-off, giving improved run-time (when matrix-vector multiplication with M is expensive) when sufficient storage is available for the “extra” vectors in Z_k .

For MINRES and CG, we need to be more careful in the choice of a preconditioner, since MA (or AM) is generally not symmetric, even if A and M are both symmetric. In these cases, we typically require M to be both symmetric and positive definite, so that we can write $M = LL^T$. This decomposition might denote a Cholesky decomposition of M or another matrix square root, $L = M^{1/2}$. While this is generally expensive, it is only formally required and not needed in practical implementations. With this, $L^T AL = L^{-1}(LL^T A)L$ is symmetric and similar to MA . If A is also positive definite, then so is $L^T AL$. From here, similar derivations yield both the preconditioned MINRES and preconditioned CG algorithms; we consider only the latter case, assuming that A is both symmetric and positive definite. Directly applying CG to $L^T AL \mathbf{y} = L^T \mathbf{f}$ yields Algorithm 7.14. If $\hat{\mathbf{y}}_{k+1}$ is the converged solution, then $\mathbf{u}_{k+1} = L \hat{\mathbf{y}}_{k+1}$.

Algorithm 7.14: CG applied to $L^T AL \mathbf{y} = L^T \mathbf{f}$

Input : A , linear system matrix
 L , factorized preconditioner with $M = LL^T$
 $\mathbf{f}$, linear system right-hand side
 $\mathbf{u}_0$, initial guess
Output : $\hat{\mathbf{y}}_{k+1}$, $(k+1)$ th iterate with $\mathbf{u} \approx L \hat{\mathbf{y}}_{k+1}$

```

1  $\hat{\mathbf{r}}_0 = L^T(\mathbf{f} - A\mathbf{u}_0)$ ,  $\hat{\mathbf{p}}_0 = \hat{\mathbf{r}}_0$ ,  $\hat{\mathbf{y}}_0 = L^{-1}\mathbf{u}_0$ 
2 for  $k = 1, 2, \dots$ 
3    $\alpha_k = \hat{\mathbf{r}}_k^T \hat{\mathbf{r}}_k / \hat{\mathbf{p}}_k^T L^T AL \hat{\mathbf{p}}_k$ 
4    $\hat{\mathbf{y}}_{k+1} = \hat{\mathbf{y}}_k + \alpha_k \hat{\mathbf{p}}_k$ 
5    $\hat{\mathbf{r}}_{k+1} = \hat{\mathbf{r}}_k - \alpha_k L^T AL \hat{\mathbf{p}}_k$ 
6    $\beta_k = \hat{\mathbf{r}}_{k+1}^T \hat{\mathbf{r}}_{k+1} / \hat{\mathbf{r}}_k^T \hat{\mathbf{r}}_k$ 
7    $\hat{\mathbf{p}}_{k+1} = \hat{\mathbf{r}}_{k+1} + \beta_k L^T AL \hat{\mathbf{p}}_k$ 

```

In Algorithm 7.15, we directly update $\mathbf{u}_{k+1}$ in place of $\hat{\mathbf{y}}_{k+1}$, since

$$\hat{\mathbf{y}}_{k+1} = \hat{\mathbf{y}}_k + \alpha_k \hat{\mathbf{p}}_k \quad \Rightarrow \quad \mathbf{u}_{k+1} = \mathbf{u}_k + \alpha_k L \hat{\mathbf{p}}_k. \quad (7.183)$$

Likewise, $\hat{\mathbf{r}}_{k+1} = L^T(\mathbf{f} - A\mathbf{u}_{k+1}) = L^T \mathbf{r}_{k+1}$, so

$$\hat{\mathbf{r}}_{k+1} = \hat{\mathbf{r}}_k - \alpha_k L^T AL \hat{\mathbf{p}}_k, \quad (7.184a)$$

which gives

$$L^T \mathbf{r}_{k+1} = L^T \mathbf{r}_k - \alpha_k L^T AL \hat{\mathbf{p}}_k \quad (7.184b)$$

or

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k AL \hat{\mathbf{p}}_k. \quad (7.184c)$$

If we then take $\mathbf{p}_{k+1} = L \hat{\mathbf{p}}_{k+1}$, the above relations simplify to match those within the unpreconditioned CG algorithm, giving $\mathbf{u}_{k+1} = \mathbf{u}_k + \alpha_k \mathbf{p}_k$ and $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k$, while the update

for $\mathbf{p}_{k+1}$ becomes

$$\mathbf{p}_{k+1} = L\hat{\mathbf{p}}_{k+1} = L\hat{\mathbf{r}}_{k+1} + \beta_k L\hat{\mathbf{p}}_k = LL^T \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k. \quad (7.185)$$

Defining $\mathbf{z}_k = M\mathbf{r}_k$ gives $\mathbf{p}_{k+1} = \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k$. Similar transformations are possible for the constants in the CG algorithm, with

$$\alpha_k = \hat{\mathbf{r}}_k^T \hat{\mathbf{r}}_k / \hat{\mathbf{p}}_k^T L^T A L \hat{\mathbf{p}}_k = \mathbf{r}_k^T L L^T \mathbf{r}_k / \mathbf{p}_k^T A \mathbf{p}_k = \mathbf{r}_k^T \mathbf{z}_k / \mathbf{p}_k^T A \mathbf{p}_k, \quad (7.186a)$$

$$\beta_k = \hat{\mathbf{r}}_{k+1}^T \hat{\mathbf{r}}_{k+1} / \hat{\mathbf{r}}_k^T \hat{\mathbf{r}}_k = \mathbf{r}_{k+1}^T L L^T \mathbf{r}_{k+1} / \mathbf{r}_k^T L L^T \mathbf{r}_k = \mathbf{r}_{k+1}^T \mathbf{z}_{k+1} / \mathbf{r}_k^T \mathbf{z}_k. \quad (7.186b)$$

These give us an iteration that can be written directly in terms of A and M , with no need for the factorization $M = LL^T$. The final preconditioned CG algorithm is presented in Algorithm 7.15.

Algorithm 7.15: Preconditioned CG algorithm

Input : A , linear system matrix

M , preconditioner matrix

$\mathbf{f}$, linear system right-hand side

$\mathbf{u}_0$, initial guess

Output : $\mathbf{u}_{k+1}$, $(k+1)$ th iterate and approximation to $\mathbf{u}$

1 $\mathbf{r}_0 = \mathbf{f} - A\mathbf{u}_0$, $\mathbf{z}_0 = M\mathbf{r}_0$, $\mathbf{p}_0 = \mathbf{z}_0$

2 **for** $k = 1, 2, \dots$

3 $\alpha_k = \mathbf{r}_k^T \mathbf{z}_k / \mathbf{p}_k^T A \mathbf{p}_k$

4 $\mathbf{u}_{k+1} = \mathbf{u}_k + \alpha_k \mathbf{p}_k$

5 $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k$

6 $\mathbf{z}_{k+1} = M\mathbf{r}_{k+1}$

7 $\beta_k = \mathbf{r}_{k+1}^T \mathbf{z}_{k+1} / \mathbf{r}_k^T \mathbf{z}_k$

8 $\mathbf{p}_{k+1} = \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k$

7.11 ■ Generalized methods for nonsymmetric matrices

Objectives

It is difficult to ignore the substantial gap between the cost of optimal Krylov methods in the symmetric case and the general (nonsymmetric) case. When A is symmetric, CG and MINRES both give optimal cost per iteration of one matrix-vector product and a handful of vector operations, while GMRES has a growing cost per iteration, with the k th step requiring one matrix-vector product and $\mathcal{O}(k)$ vector operations. Thus, in this final section of the chapter, you will explore the natural question of whether or not we can close this gap; the short answer is that we cannot. Specifically, in this section, you will see the Faber–Manteuffel theorem that says that there are no optimal-cost Krylov methods for general matrices, only for specific classes. Then, you will learn how to generate *biorthogonal* bases via the bi-Lanczos algorithm, in order to develop the biconjugate gradient (BiCG) algorithm for nonsymmetric systems. Finally, to address the stability issues that arise, you will derive the BiCGSTAB algorithm.

To understand the gap between the costs of CG and MINRES versus GMRES, we consider the essence of a Krylov method, where we write $\mathbf{u}_k = \mathbf{u}_0 + P_k \mathbf{y}_k$ such that $\text{span}(P_k) = \mathcal{K}_k(A, \mathbf{r}_0)$. For stability, we have required that P_k be an orthogonal matrix in some inner product,

meaning that there is some symmetric and positive-definite matrix, B , such that

$$\mathbf{p}_i^T B \mathbf{p}_j = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{otherwise.} \end{cases} \quad (7.187)$$

For a given matrix, A , the relationship between A and the inner product induced by B leads to the definition of B -normal(t) matrices.

Definition 7.58: B -normal matrices [112, Sec. 6.1]

The matrix A is B -normal(t) for $t \in \mathbb{N}$ if and only if there exists a polynomial $p_t(z)$ of degree t such that

$$(B^{1/2} A B^{-1/2})^T = p_t(B^{1/2} A B^{-1/2}). \quad (7.188)$$

The intuition here is that this extends the usual definition of a normal matrix (where $AA^T = A^T A$) to other inner products. For general B , being B -normal(1) means that A is self-adjoint in the B inner product. If B is the identity operator, then any symmetric matrix, A , satisfies $A^T = A$, so A is B -normal(1). The Faber–Manteuffel theorem, Theorem 7.59, uses this idea to classify when an s -term CG-like method is possible, meaning a Krylov method where the next column of P_k can be determined using at most $s - 1$ previous columns.

Theorem 7.59: Faber–Manteuffel [96]

An s -term CG method exists for matrix A if and only if either

1. there is a monic polynomial of degree at most s , $p(z)$, such that $p(A) = 0$ or
2. A is B -normal($s - 2$) for some symmetric and positive-definite matrix B .

The first case implies that A has at most s distinct eigenvalues, independent of the size of A , which is generally not true for “interesting” matrices, such as those we normally consider. Considering the second case, a subsequent result shows that if A is B -normal(t) for $t > 1$, then there is a polynomial of degree at most t^2 , $p_{t^2}(z)$, such that $p_{t^2}(A) = 0$. Again, this is generally not true for interesting matrices. Another result shows that if A is B -normal(1), then $A = \alpha I$ for constant α , A is self-adjoint in the B inner product, $A = B^{-1} A^T B$, or $B^{1/2} A B^{-1/2} = e^{\theta} (\frac{r}{2} I + F)$ for real values θ and r and antisymmetric matrix $F = -F^T$. While this offers slightly more generality than the requirements for MINRES or CG, the general conclusion is that we cannot generalize CG to treat general matrices while maintaining both an optimal approximation and a short-term recurrence to define the algorithm. GMRES arises from choosing to keep an optimal approximation and sacrificing short-term recurrences. Another class of algorithms, which we discuss next, arises from maintaining short-term recurrence relations but losing control over optimality in the approximation.

The first of these methods arises by constructing two Krylov spaces, one based on A and a second based on A^T , and then requiring that the residual of the k th approximation be orthogonal to all vectors in the second Krylov space. To achieve this, we use the Lanczos biorthogonalization (or bi-Lanczos) algorithm to generate these bases, as given in Algorithm 7.16. The properties of the bases produced by this algorithm are summarized in Theorem 7.60.

Theorem 7.60: Bi-Lanczos bases [194, Prop. 7.1]

Let $V_k = [\mathbf{v}_1, \dots, \mathbf{v}_k]$ and $W_k = [\mathbf{w}_1, \dots, \mathbf{w}_k]$, where the vectors defining the columns of these matrices are given by Algorithm 7.16. If no breakdown occurs before step k , then the following

equations hold:

$$W_k^T V_k = I, \quad (7.189a)$$

$$\mathcal{K}_k(A, \mathbf{v}_1) = \text{span}(V_k), \quad \mathcal{K}_k(A^T, \mathbf{w}_1) = \text{span}(W_k), \quad (7.189b)$$

$$W_k^T A V_k = T_k = \begin{bmatrix} \alpha_1 & \beta_2 & & \\ \delta_2 & \alpha_2 & \beta_3 & \\ & \ddots & \ddots & \ddots \\ & & \delta_k & \alpha_k \end{bmatrix}, \quad (7.189c)$$

$$A V_k = V_k T_k + \delta_{k+1} \mathbf{v}_{k+1} \left(\mathbf{e}_k^{(1)} \right)^T, \quad (7.189d)$$

$$A^T W_k = W_k T_k^T + \beta_{k+1} \mathbf{w}_{k+1} \left(\mathbf{e}_k^{(1)} \right)^T. \quad (7.189e)$$

Algorithm 7.16: Lanczos biorthogonalization algorithm to compute biorthogonal bases of $\mathcal{K}_k(A, \mathbf{v}_1)$ and $\mathcal{K}_k(A^T, \mathbf{w}_1)$

Input : A , linear system matrix

$\mathbf{v}_1, \mathbf{w}_1$, initial vectors with $\mathbf{w}_1^T \mathbf{v}_1 = 1$

Output : $\left. \begin{array}{l} V_k = [\mathbf{v}_1, \dots, \mathbf{v}_k] \\ W_k = [\mathbf{w}_1, \dots, \mathbf{w}_k] \end{array} \right\}$ biorthogonal bases

```

1  $\mathbf{v}_0 = \mathbf{w}_0 = \mathbf{0}, \beta_1 = \delta_1 = 0$ 
2 for  $j = 1, 2, \dots, k-1$ 
3    $\alpha_j = \mathbf{w}_j^T A \mathbf{v}_j$ 
4    $\hat{\mathbf{v}}_{j+1} = A \mathbf{v}_j - \alpha_j \mathbf{v}_j - \beta_j \mathbf{v}_{j-1}$ 
5    $\hat{\mathbf{w}}_{j+1} = A^T \mathbf{w}_j - \alpha_j \mathbf{w}_j - \delta_j \mathbf{w}_{j-1}$ 
6   Choose  $\delta_{j+1}, \beta_{j+1}$  so that  $\delta_{j+1} \beta_{j+1} = \hat{\mathbf{w}}_{j+1}^T \hat{\mathbf{v}}_{j+1}$ 
7    $\mathbf{v}_{j+1} = \frac{1}{\delta_{j+1}} \hat{\mathbf{v}}_{j+1}$ 
8    $\mathbf{w}_{j+1} = \frac{1}{\beta_{j+1}} \hat{\mathbf{w}}_{j+1}$ 

```

It is important to note that the set of vectors produced by Algorithm 7.16 are now biorthogonal, but that V_k and W_k are not orthogonal matrices. Nonetheless, we can define a CG-like method by taking $\mathbf{u}_k = \mathbf{u}_0 + V_k \mathbf{y}_k$ for some $\mathbf{y}_k$ and requiring that $W_k^T \mathbf{r}_k = \mathbf{0}$. This gives

$$W_k^T (\mathbf{r}_0 - A V_k \mathbf{y}_k) = \mathbf{0} \text{ or} \quad (7.190a)$$

$$T_k \mathbf{y}_k = W_k^T A V_k \mathbf{y}_k = W_k^T \mathbf{r}_0. \quad (7.190b)$$

If we further take $\mathbf{v}_1 = \frac{1}{\|\mathbf{r}_0\|} \mathbf{r}_0$, then we can define $\beta = \mathbf{w}_1^T \mathbf{r}_0$ and write this as $T_k \mathbf{y}_k = \beta \mathbf{e}_k^{(1)}$, similarly to before. This then gives the k th approximation as

$$\mathbf{u}_k = \mathbf{u}_0 + V_k T_k^{-1} \left(\beta \mathbf{e}_k^{(1)} \right). \quad (7.191)$$

From here, we follow the same derivation as the CG algorithm to get the biconjugate gradient, or BiCG, algorithm, given in Algorithm 7.17. As for CG, we can prove by induction that $\hat{\mathbf{r}}_i^T \mathbf{r}_j = 0$ and $\hat{\mathbf{p}}_i^T A \mathbf{p}_j = 0$ for $i \neq j$.

Algorithm 7.17: BiCG algorithm

Input : A , linear system matrix
 f , linear system right-hand side
 u_0 , initial guess

Output : u_{k+1} , approximation to u

```

1  $r_0 = b - Au_0$ 
2 Choose  $\hat{r}_0$  such that  $\hat{r}_0^T r_0 \neq 0$ 
3  $p_0 = r_0, \hat{p}_0 = \hat{r}_0$ 
4 for  $k = 1, 2, \dots$ 
5    $\alpha_k = \hat{r}_k^T r_k / \hat{p}_k^T A p_k$ 
6    $u_{k+1} = u_k + \alpha_k p_k$ 
7    $r_{k+1} = r_k - \alpha_k A p_k$ 
8    $\hat{r}_{k+1} = \hat{r}_k - \alpha_k A^T \hat{p}_k$ 
9    $\beta_k = \hat{r}_{k+1}^T r_{k+1} / \hat{r}_k^T r_k$ 
10   $p_{k+1} = r_{k+1} + \beta_k p_k$ 
11   $\hat{p}_{k+1} = \hat{r}_{k+1} + \beta_k \hat{p}_k$ 

```

The BiCG algorithm now requires two matrix-vector products per iteration, one with A and one with A^T . Implicitly, the algorithm is solving the linear system $A^T \hat{u} = \hat{b}$ at the same time as we solve $Au = b$ for a vector $\hat{b}$ determined by the choice of $\hat{r}_0$. However, since we do not care about the solution to that system, we do not bother computing the approximations to $\hat{u}$. The important difference, though, is that while CG gives an orthogonal projection onto the Krylov space, BiCG gives an oblique projection. The resulting residual is still orthogonal to *some* space, but we get no optimality because the approximation comes from this oblique projection. Thus, we cannot effectively control the error. Furthermore, this algorithm is unstable, since it implicitly relies on the LU factorization of T_k without pivoting, which is not guaranteed to be stable unless A is symmetric and positive definite.

Aside from its lack of stability, the main disadvantage of BiCG is that it requires matrix-vector products with both A and A^T . In practice, this doubles the storage required for the algorithm, since most sparse matrix data structures do not directly support multiplication by the transpose of a matrix. The conjugate gradient squared (CGS) algorithm fixes this issue by examining and rewriting the recurrences in BiCG to avoid the use of A^T . In particular, at step k of BiCG, we have the two residual updates written as

$$r_{k+1} = r_k - \alpha_k A p_k, \quad \hat{r}_{k+1} = \hat{r}_k - \alpha_k A^T \hat{p}_k \quad (7.192a)$$

and two direction updates written as

$$p_{k+1} = r_{k+1} + \beta_k p_k, \quad \hat{p}_{k+1} = \hat{r}_{k+1} + \beta_k \hat{p}_k. \quad (7.192b)$$

Since $p_0 = r_0$ and $\hat{p}_0 = \hat{r}_0$, we rewrite these as

$$r_{k+1} = \phi_{k+1}(A) r_0, \quad \hat{r}_{k+1} = \phi_{k+1}(A^T) \hat{r}_0, \quad (7.193a)$$

$$p_{k+1} = \pi_{k+1}(A) r_0, \quad \hat{p}_{k+1} = \pi_{k+1}(A^T) \hat{r}_0 \quad (7.193b)$$

for polynomials $\phi_{k+1}(z)$ and $\pi_{k+1}(z)$. Since we naturally have $(\phi_{k+1}(A^T))^T = \phi_{k+1}(A)$ and

$$(\pi_{k+1}(A^T))^T = \pi_{k+1}(A),$$

$$\begin{aligned} \alpha_k &= \hat{\mathbf{r}}_k^T \mathbf{r}_k / \hat{\mathbf{p}}_k^T A \mathbf{p}_k = (\phi_k(A^T) \hat{\mathbf{r}}_0)^T (\phi_k(A) \mathbf{r}_0) / (\pi_k(A^T) \hat{\mathbf{r}}_0)^T A (\pi_k(A) \mathbf{r}_0) \\ &= \hat{\mathbf{r}}_0^T \phi_k^2(A) \mathbf{r}_0 / \hat{\mathbf{r}}_0^T A \pi_k^2(A) \mathbf{r}_0 \text{ and} \end{aligned} \quad (7.194a)$$

$$\beta_k = \hat{\mathbf{r}}_{k+1}^T \mathbf{r}_{k+1} / \hat{\mathbf{r}}_k^T \mathbf{r}_k = \hat{\mathbf{r}}_0^T \phi_{k+1}^2(A) \mathbf{r}_0 / \hat{\mathbf{r}}_0^T \phi_k^2(A) \mathbf{r}_0. \quad (7.194b)$$

These relations lead us to try to directly derive recurrences for $\phi_k^2(A) \mathbf{r}_0$ and $\pi_k^2(A) \mathbf{r}_0$ from those for $\phi_k(A) \mathbf{r}_0$ and $\pi_k(A) \mathbf{r}_0$. Writing the polynomial recurrences as

$$\phi_{k+1}(z) = \phi_k(z) - \alpha_k z \pi_k(z) \text{ and} \quad (7.195a)$$

$$\pi_{k+1}(z) = \phi_{k+1}(z) + \beta_k \pi_k(z), \quad (7.195b)$$

we directly write

$$\phi_{k+1}^2(z) = \phi_k^2(z) - 2\alpha_k z \phi_k(z) \pi_k(z) + \alpha_k^2 z^2 \pi_k^2(z) \text{ and} \quad (7.196a)$$

$$\pi_{k+1}^2(z) = \phi_{k+1}^2(z) + 2\beta_k \phi_{k+1}(z) \pi_k(z) + \beta_k^2 \pi_k^2(z). \quad (7.196b)$$

Of the two cross-terms that appear in the above, we use the recurrence for $\pi_k(z)$ to write

$$\phi_k(z) \pi_k(z) = \phi_k^2(z) + \beta_{k-1} \phi_k(z) \pi_{k-1}(z). \quad (7.197)$$

Then, for the “offset-indexed” cross-terms, we use both initial recurrences to derive

$$\begin{aligned} \phi_k(z) \pi_{k-1}(z) &= \phi_{k-1}(z) \pi_{k-1}(z) - \alpha_{k-1} z \pi_{k-1}^2 \\ &= \phi_{k-1}^2(z) + \beta_{k-2} \phi_{k-1}(z) \pi_{k-2}(z) - \alpha_{k-1} z \pi_{k-1}^2. \end{aligned} \quad (7.198)$$

This leads to a natural trick, where we evolve three vectors instead of two, writing $\mathbf{r}_k = \phi_k^2(A) \mathbf{r}_0$, $\mathbf{p}_k = \pi_k^2(A) \mathbf{r}_0$, and $\mathbf{q}_k = \phi_{k+1}(A) \pi_k(A) \mathbf{r}_0$ (noting that $\mathbf{r}_k$ is now different from its use in BiCG). Given the recurrence relations above, it is convenient to also track a fourth vector:

$$\mathbf{s}_k = \mathbf{r}_k + \beta_{k-1} \mathbf{q}_{k-1} = \phi_k(A) (\phi_k(A) + \beta_{k-1} \pi_{k-1}(A)) \mathbf{r}_0. \quad (7.199)$$

This is the final step needed to derive the CGS algorithm, given as Algorithm 7.18.

CGS eliminates the need for direct computations with A^T but, unfortunately, does not solve the instability problem inherent in BiCG. To overcome this, BiCGSTAB breaks the biorthogonality in BiCG and introduces a new way to evolve the shadow residual, $\hat{\mathbf{r}}_k$, and search direction, $\hat{\mathbf{p}}_k$, in order to improve local stability by solving a minimization problem. Fundamentally, we replace the recurrences for $\mathbf{r}_k$ and $\mathbf{p}_k$ by writing $\mathbf{r}_k = \psi_k(A) \phi_k(A) \mathbf{r}_0$ and $\mathbf{p}_k = \psi_k(A) \pi_k(A) \mathbf{r}_0$ for a new polynomial $\psi_k(A)$ chosen to ensure stability. To make the computations with $\psi_k(A)$ tractable, we choose it to satisfy the recurrence relation $\psi_{k+1}(z) = (1 - \omega_k z) \psi_k(z)$, leaving just a single parameter, ω_k , to be chosen at each iteration. With this, the recurrences for the residual and search direction become

$$\mathbf{r}_{k+1} = (I - \omega_k A) (\mathbf{r}_k - \alpha_k A \mathbf{p}_k) \text{ and} \quad (7.200a)$$

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k (I - \omega_k A) \mathbf{p}_k. \quad (7.200b)$$

Following from BiCG and CGS, we take $\alpha_k = \hat{\mathbf{r}}_0^T \mathbf{r}_k / \hat{\mathbf{r}}_0^T A \mathbf{p}_k$, but we update β_k to be

$$\beta_k = (\alpha_k / \omega_k) (\hat{\mathbf{r}}_0^T \mathbf{r}_{k+1} / \hat{\mathbf{r}}_0^T \mathbf{r}_k), \quad (7.201)$$

noting that this coincides with BiCG and CGS when $\omega_k = \alpha_k$. Instead of making that choice, however, we split the residual update into two steps, writing $\mathbf{s}_k = \mathbf{r}_k - \alpha_k A \mathbf{p}_k$ and,

Algorithm 7.18: CGS algorithm

Input : A , linear system matrix
 $\mathbf{f}$, linear system right-hand side
 $\mathbf{u}_0$, initial guess

Output : $\mathbf{u}_{k+1}$, approximation to $\mathbf{u}$

```

1  $\mathbf{r}_0 = \mathbf{b} - A\mathbf{u}_0$ 
2 Choose  $\hat{\mathbf{r}}_0$  such that  $\hat{\mathbf{r}}_0^T \mathbf{r}_0 \neq 0$ 
3  $\mathbf{p}_0 = \mathbf{s}_0 = \mathbf{r}_0$ 
4 for  $k = 1, 2, \dots$ 
5    $\alpha_k = \hat{\mathbf{r}}_0^T \mathbf{r}_k / \hat{\mathbf{r}}_0^T A\mathbf{p}_k$ 
6    $\mathbf{q}_k = \mathbf{s}_k - \alpha_k A\mathbf{p}_k$ 
7    $\mathbf{u}_{k+1} = \mathbf{u}_k + \alpha_k (\mathbf{s}_k + \mathbf{q}_k)$ 
8    $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A(\mathbf{s}_k + \mathbf{q}_k)$ 
9    $\beta_k = \hat{\mathbf{r}}_0^T \mathbf{r}_{k+1} / \hat{\mathbf{r}}_0^T \mathbf{r}_k$ 
10   $\mathbf{s}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{q}_k$ 
11   $\mathbf{p}_{k+1} = \mathbf{s}_{k+1} + \beta_k (\mathbf{q}_k + \beta_k \mathbf{p}_k)$ 

```

subsequently, $\mathbf{r}_{k+1} = \mathbf{s}_k - \omega_k A\mathbf{s}_k$. We now choose ω_k to minimize $\|\mathbf{r}_{k+1}\|^2$, giving $\omega_k = \mathbf{s}_k^T A\mathbf{s}_k / (A\mathbf{s}_k)^T (A\mathbf{s}_k)$. To get the final form of the BiCGSTAB algorithm, given in Algorithm 7.19, we note that the accumulated residual update of

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A\mathbf{p}_k - \omega_k A\mathbf{s}_k \quad (7.202)$$

implies that

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \alpha_k \mathbf{p}_k + \omega_k \mathbf{s}_k. \quad (7.203)$$

While $\hat{\mathbf{r}}_0$ appears as a parameter in the algorithm, most implementations of BiCGSTAB take $\hat{\mathbf{r}}_0 = \mathbf{r}_0$ for simplicity, since it will always satisfy the required relationship (so long as $\mathbf{r}_0 \neq \mathbf{0}$). Many implementations also note that when $\mathbf{s}_k$ satisfies the desired residual convergence condition (usually that $\|\mathbf{s}_k\|$ is sufficiently small), then the algorithm can terminate before computing ω_k , etc., and return $\mathbf{u}_{k+1/2}$ as the approximate solution. In this sense, it is good to recall that BiCGSTAB requires two matrix-vector products per iteration, and that one is performed to generate $\mathbf{u}_{k+1/2}$ and $\mathbf{s}_k$, while the second is used to generate $\mathbf{u}_{k+1}$ and $\mathbf{r}_{k+1}$. When comparing performance between BiCGSTAB and GMRES, it is important to account for this cost. Finally, the minimization that leads to the choice of ω_k can be generalized to take larger steps and solve a larger minimization problem at each iteration, leading to the BiCGSTAB(ℓ) algorithms (see [202]).

7.12 ■ Some further thoughts

One indicator of just how important the numerical solution of PDEs has been to the history of computing is the extensive history of algorithms for the numerical solution of linear systems. While there is always interest in improving the accuracy of the discretizations, the utility of a good discretization scheme is questionable if it cannot be solved in a practical amount of time. This is emphasized in Figure 7.12.1, where we compare the methods developed in this chapter and their *estimated* compute times for solving $A\mathbf{u} = \mathbf{f}$ for a 3D finite-difference Poisson equation discretized on a uniform $n_x \times n_x \times n_x$ mesh, yielding an $n \times n$ matrix with

Algorithm 7.19: BiCGSTAB algorithm

Input : A , linear system matrix
 $\mathbf{f}$, linear system right-hand side
 $\mathbf{u}_0$, initial guess

Output : $\mathbf{u}_{k+1}$, approximation to $\mathbf{u}$

```

1  $\mathbf{r}_0 = \mathbf{b} - A\mathbf{u}_0$ 
2 Choose  $\hat{\mathbf{r}}_0$  such that  $\hat{\mathbf{r}}_0^T \mathbf{r}_0 \neq 0$ 
3  $\mathbf{p}_0 = \mathbf{r}_0$ 
4 for  $k = 1, 2, \dots$ 
5    $\alpha_k = \hat{\mathbf{r}}_0^T \mathbf{r}_k / \hat{\mathbf{r}}_0^T A\mathbf{p}_k$ 
6    $\mathbf{u}_{k+1/2} = \mathbf{u}_k + \alpha_k \mathbf{p}_k$ 
7    $\mathbf{s}_k = \mathbf{r}_k - \alpha_k A\mathbf{p}_k$ 
8    $\omega_k = \mathbf{s}_k^T A\mathbf{s}_k / (A\mathbf{s}_k)^T (A\mathbf{s}_k)$ 
9    $\mathbf{u}_{k+1} = \mathbf{u}_{k+1/2} + \omega_k \mathbf{s}_k$ 
10   $\mathbf{r}_{k+1} = \mathbf{s}_k - \omega_k A\mathbf{s}_k$ 
11   $\beta_k = (\alpha_k / \omega_k) (\hat{\mathbf{r}}_0^T \mathbf{r}_{k+1} / \hat{\mathbf{r}}_0^T \mathbf{r}_k)$ 
12   $\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k (\mathbf{p}_k - \omega_k A\mathbf{p}_k)$ 

```

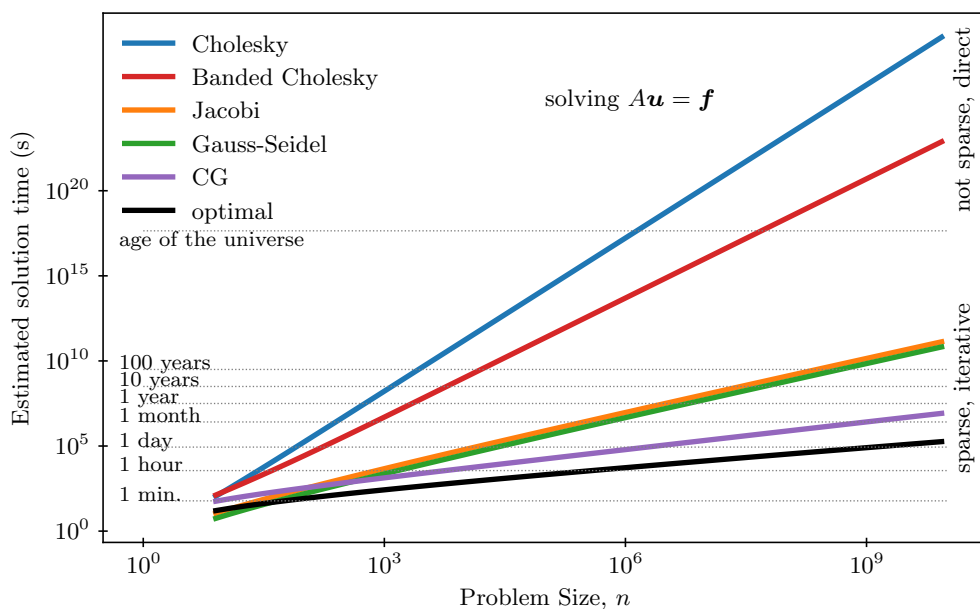


Figure 7.12.1. Estimated solution time for solving linear system resulting from a finite-difference discretization of the 3D Poisson equation using a uniform $n_x \times n_x \times n_x$ mesh using different solvers. Here, the matrix or problem size is $n = n_x^3$.

$n = n_x^3$ degrees of freedom. It is particularly interesting to note that dense direct methods become infeasible for even modest problem sizes, around that of a $100 \times 100 \times 100$ mesh ($n = 10^6$), and even taking advantage of banded factorizations yields limited benefits. In contrast, the scaling offered by basic iterations, such as Jacobi or Gauss–Seidel, remains quite reasonable but still requires compute times of months or years to handle these problems. In contrast, CG is much better, with a large but not impractical estimated CPU time of a day for the $100 \times 100 \times 100$ mesh. As we will see in Chapter 11, there are still better methods available that make these problems feasible (particularly in parallel computing environments) at even larger scales. An important aspect to emphasize here is that waiting for faster computing hardware is not a viable solution to this challenge. Indeed, it has been argued that computational science has advanced equally due to advances in computer hardware (that largely follow Moore’s law) and advances in these core algorithms over the past decades.

We emphasize that the discussion in this chapter is a brief summary of a wide range of material that is discussed in more detail in other books. Dense and sparse Gaussian elimination are the focus of [109, 105, 90], and the theory for matrix splittings and classical iterative methods is found in [226, 237]. The theory and development of Krylov methods is given in [194, 220]. Other great resources for an in-depth treatment of some of the material in this chapter include [218, 84, 34].

7.13 ■ Exercises

1. Write out the “classical” in-place LU factorization algorithm in kij ordering that corresponds to the standard version of Gaussian elimination from introductory linear algebra. Show that this is equivalent to the ikj ordering.
2. Assume n is odd. Let the $n \times n$ matrix, A , have nonzero entries along only its main diagonal and first row/column: $a_{i,i} \neq 0$, $a_{i,1} \neq 0$, and $a_{1,j} \neq 0$.
 - (a) Compute $|\text{Env}(A)|$ and the total cost of factorization (including all details).
 - (b) Show that, for any permutation matrix, P , $\text{Env}(P^T A P)$ depends only on how the permutation changes the first row/column of A .
 - (c) Find a permutation, P , that minimizes the bandwidth of $P^T A P$, and compute $|\text{Env}(P^T A P)|$ and the total cost of factorization in this ordering (including all details). How much smaller is the constant than in the original ordering?
 - (d) Consider the permutation, P , that reorders A to have nonzeros only along its main diagonal and last row/column, and compute $|\text{Env}(P^T A P)|$ and the total cost of factorization in this ordering (including all details).
3. Let A be the $n \times n$ tridiagonal matrix with entries $a_{i,i} = 2 + \alpha$ for $\alpha \geq 0$ and $a_{i,i \pm 1} = -1$. Assume n is even.
 - (a) Show that the vectors $\mathbf{v}^{(k)}$ for $1 \leq k \leq n$ defined by $v_i^{(k)} = \sin\left(\frac{ik\pi}{n+1}\right)$ are eigenvectors of both the system matrix, A , and the Jacobi iteration matrix. What are the eigenvalues of these matrices? What is the spectral radius of the Jacobi iteration matrix as a function of n ?
 - (b) To find the eigenvalues of SOR (and Gauss–Seidel), use the matrix splitting $A = D - L - U$, and consider the *generalized eigenvalues*, γ_k , for which there is a nonzero vector, $\mathbf{v}^{(k)}$, such that $A\mathbf{v}^{(k)} = \gamma_k\left(\frac{1}{\omega}D - L\right)\mathbf{v}^{(k)}$. *Hint:* Use the ansatz that $v_i^{(k)} = \sigma_k^i \sin(ik\pi/(n+1))$ when $\gamma_k \neq 1$. First, derive an expression for σ_k , and then derive one for the eigenvalues of SOR, $\lambda_k = 1 - \gamma_k$.

- (c) For the case of Gauss–Seidel, find the explicit form of $I - (D - L)^{-1}A = (D - L)^{-1}U$ and compute $\text{tr}((D - L)^{-1}U)$; use this form to argue that each nonzero eigenvalue from above must have algebraic multiplicity one and, consequently, the zero eigenvalue must have algebraic multiplicity $n/2$. What is the geometric multiplicity of the zero eigenvalue?
- (d) What is the optimal SOR relaxation parameter for this system? What is the spectral radius of the SOR iteration matrix with this parameter choice?
- (e) Consider these in the case of $\alpha = 0$. Explain why your results above give the spectral radii for the Jacobi, Gauss–Seidel, and SOR iteration matrices for the linear system obtained from discretizing the one-dimensional Poisson problem with finite differences.
4. (a) Let $\rho = \rho(I - M^{-1}A)$ for fixed M . Using the estimate that $\|e_\ell\| \approx \rho^\ell \|e_0\|$ for large ℓ , estimate the number of steps, ℓ , needed to ensure that the relative error in x_ℓ , $\|e_\ell\|/\|e_0\|$, is less than a fixed tolerance, ϵ .
- (b) Use your results from (a) and the previous problem to estimate the number of steps of Jacobi, Gauss–Seidel, and SOR (with optimal choice of ω) applied to that problem with $\alpha = 0$ that are needed to reach a fixed tolerance, ϵ , as a function of N . It may be helpful to recall that when z is small, $\log(1 + z) \approx z$. How much “faster” is the Gauss–Seidel iteration than the Jacobi iteration? Using “big-O” notation, explain the advantage offered by SOR with optimal ω .
5. Let $A = D - L - U$ be the splitting of $n \times n$ matrix A into its diagonal and strictly lower- and upper-triangular parts. Assume that A is nonsingular and that $a_{i,i} \neq 0$ for $1 \leq i \leq n$. The symmetric Gauss–Seidel method for solving $Au = f$ with initial guess u_0 is given by

$$\begin{aligned}(D - L)u_{k+1/2} &= f + Uu_k, \\ (D - U)u_{k+1} &= f + Lu_{k+1/2}.\end{aligned}$$

- (a) Show that if $u_k = u$, then $u_{k+1} = u$.
- (b) Show that if A is either a lower- or an upper-triangular matrix, then $u_1 = u$.
6. The Kaczmarz iteration for $Au = f$ is defined to be the Gauss–Seidel iteration applied to $AA^T w = f$, with iterates related by $u_k = A^T w_k$.
- (a) Show that for any given u_0 , we can compute u_1 (and, more generally, u_k) without explicitly computing w_0 .
- (b) Give the componentwise form of the iteration to compute $(u_k)_j$ corresponding to the update to the j th component of w_k .
- (c) Explain why the Kaczmarz iteration is guaranteed to converge for any nonsingular matrix A .
7. For $\varepsilon > 0$, consider the matrix

$$A = \begin{bmatrix} 4 & -1 & -2 \\ -1 & 2 & 0 \\ -2 & 0 & 2 + 2\varepsilon \end{bmatrix}.$$

- (a) Compute the ILU(0) factorization of A .
 - (b) Write down the error-propagation matrix for the ILU(0) iteration matrix for A .
 - (c) Estimate the asymptotic convergence factor of this iteration based on Geršgorin's theorem. What happens as $\varepsilon \rightarrow 0$?
8. Let A be the system matrix for the one-dimensional discretization of the Poisson problem with finite differences on a uniform grid of the unit interval. Using the extreme eigenvalues of A to determine the parameters for the Chebyshev iteration, give a bound on the error after k iterations of the Chebyshev method, relative to the initial error. Estimate how many iterations are needed to reach a fixed tolerance, ϵ , as a function of n_x . Discuss how this bound compares to those for Jacobi, Gauss–Seidel, and SOR.
 9. Prove (by induction) that the Chebyshev iteration with initial guess $\mathbf{u}_0$ produces iterates that satisfy $\mathbf{u}_k - \mathbf{u}_0 \in \mathcal{K}_k(A, \mathbf{r}_0)$, the Krylov space of dimension k with initial vector $\mathbf{r}_0 = \mathbf{f} - A\mathbf{u}_0$.
 10. Let A be the $n \times n$ matrix with entries $a_{i+1,i} = 1$ for $i = 1, \dots, n-1$, $a_{1,n} = 1$, and all other entries zero. Let $\mathbf{f}$ have entries $f_1 = 1$ and $f_i = 0$ for $i = 2, \dots, n$, and let $\mathbf{u}_0$ be the zero vector.
 - (a) Prove that GMRES applied to the system $A\mathbf{u} = \mathbf{f}$ with initial guess $\mathbf{u}_0$ takes n steps to find the true solution, and that $\|\mathbf{f} - A\mathbf{u}_k\| = 1$ for $1 \leq k \leq n-1$.
 - (b) Find the eigenvalues of A , and explain why the GMRES convergence bound does not apply to this example.
 11. Let A be a block matrix with $k \times k$ blocks, each of size $n \times n$, of the form

$$A = \begin{bmatrix} I & A_1 & & & & \\ & I & A_2 & & & \\ & & I & \ddots & & \\ & & & \ddots & A_{k-2} & \\ & & & & I & A_{k-1} \\ & & & & & I \end{bmatrix}.$$

- (a) Show that $(I - A)^k = 0$.
 - (b) Show that GMRES applied to the system $A\mathbf{u} = \mathbf{f}$ converges in at most k iterations, independent of n , for any $\mathbf{f}$.
12. Let $A = I + \alpha B$, where B is a real-valued $n \times n$ matrix with the property that $B^T = -B$. We call B a *skew-symmetric matrix*.
 - (a) Show that $\mathbf{x}^T A \mathbf{x} = \mathbf{x}^T \mathbf{x}$ for any vector, $\mathbf{x}$, for any value of α .
 - (b) Show that the upper Hessenberg matrix, $H_k = Q_K^T A Q_k$, generated by the Arnoldi algorithm applied to A is tridiagonal.

13. Let $\epsilon > 0$ and define

$$A = \begin{bmatrix} 1 + \epsilon & -1 & 1 \\ -1 & 1 + \epsilon & -1 \\ 1 & -1 & 0 \end{bmatrix}.$$

(a) Show that

$$\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

is an eigenvector of A with eigenvalue ϵ .

(b) Show that A has two eigenvectors of the form

$$\begin{bmatrix} 1 \\ -1 \\ z \end{bmatrix},$$

and find the associated eigenvalues as functions of ϵ . Show that in the limit as $\epsilon \rightarrow 0$, these eigenvalues approach $1 \pm \sqrt{3}$.

(c) Let

$$B = \begin{bmatrix} \epsilon & 0 & 1 \\ 0 & \epsilon & -1 \\ 1 & -1 & 0 \end{bmatrix}.$$

What are the eigenvalues of $B^{-1}A$?

Chapter 8

Finite-element methods for elliptic partial differential equations

Overview

In Chapter 4, we took a first look at the finite-element method (FEM) applied to a one-dimensional elliptic PDE. Finite elements are used in many applications, including fluid flow, solid mechanics, and electromagnetics, and this chapter focuses on generalizing of the ideas of Chapter 4. We focus here on second-order elliptic equations, such as

$$-\nabla \cdot K(\mathbf{x})\nabla u + \mathbf{b}(\mathbf{x}) \cdot \nabla u + c(\mathbf{x})u = f(\mathbf{x}) \text{ for all } \mathbf{x} \in \Omega, \quad (8.1)$$

with suitable boundary conditions on domain Ω , for various choices of the coefficients of the PDE, $K(\mathbf{x})$, $\mathbf{b}(\mathbf{x})$, and $c(\mathbf{x})$. The ideas in this chapter are further extended in Chapter 10, where we consider systems of PDEs.

Objectives

In this chapter, you will explore the finite-element methodology in a more general setting. To do this requires first reviewing some tools from functional analysis. Later sections will focus on important issues with boundary conditions, implementation details, and error estimates. Specifically, you will

- develop the theoretical tools needed to understand weak forms of more general problems and the existence and uniqueness of their solutions;
- explore the connections between the continuum PDEs and weak forms and their discrete counterparts, including the accuracy of the approximate solutions and important details of how we assemble the linear systems corresponding to the discretizations;
- apply these approaches to general second-order elliptic and parabolic PDEs, with general coefficients and boundary conditions; and
- understand the interplay between error estimates and adaptive mesh refinement techniques that are a natural complement to the finite-element methodology.

Chapter Outline

Section 8.1 Review of basic tools from functional analysis reviews tools from functional analysis, including the weak derivative and properties of vector, Banach, Hilbert, and Sobolev spaces. This machinery will be used to formalize the analysis of the weak form of the problem in later sections.

Section 8.2 Existence and uniqueness introduces concepts such as coercivity, continuity, and ellipticity in order to establish existence and uniqueness for weak forms of PDEs. Key theorems such as the Riesz representation theorem and the Lax–Milgram theorem are presented.

Section 8.3 Types of elements and approximation properties reintroduces Céa’s lemma in the context of Hilbert spaces and discusses several examples of finite elements and their approximation properties. Both triangular and quadrilateral meshes are considered (and their 3D counterparts).

Section 8.4 More general problems generalizes the well-posedness of model problems for the case of a nonsymmetric bilinear form. The section also looks at more general boundary conditions, such as nonhomogeneous Dirichlet, Neumann, and Robin conditions.

Section 8.5 Finite-element assembly describes the methodology for implementing the finite-element method on a given two-dimensional test problem using triangular elements. We present algorithms for constructing the linear systems, including treatment of the boundary conditions.

Section 8.6 A posteriori error estimates presents the notion and examples of a posteriori error estimates and how they can be used in the context of adaptive mesh refinement.

Section 8.7 Finite-element methods for time-dependent PDEs extends the discussion on finite-element methods to time-dependent PDEs, using the example of a parabolic PDE.

8.1 ■ Review of basic tools from functional analysis

Objectives

The focus of this section is on tools from functional analysis with application to finite-element theory and design. Thus, you will be introduced to the mathematical framework necessary for developing and discussing finite-element methods in two and higher dimensions. In particular, *weak* solutions play a fundamental role in the construction of numerical approximations. In this section, you will identify relevant function spaces for different settings and identify key properties and use cases of these function spaces. You will also look at the function spaces associated with the idea of weak derivatives, gaining insight through examples.

8.1.1 ■ All about function spaces

The underlying mechanics of the finite-element method rely on function spaces: selecting the *best* function from a space under certain conditions. Consequently, we need to put proper care into our choice of function spaces. In particular, we need our spaces to be *complete*, so that we do not get into situations where a sequence of approximations does not converge to something in the space.

As an example, consider the space of piecewise polynomial functions that are in $C^1([-1, 1])$; that is, they are continuous functions with continuous derivatives at all points in the open interval

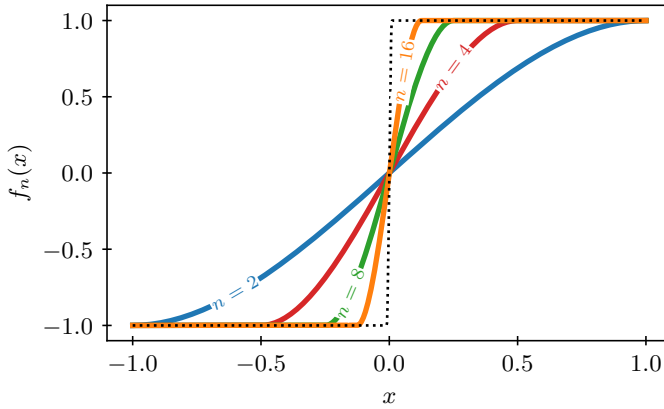


Figure 8.1.1. Sequence of piecewise polynomial C^1 functions.

$(-1, 1)$, but they can be piecewise defined with point discontinuities in their higher derivatives at a finite number of points in the interval. This space is incomplete: we can construct a sequence of functions $\{f_n(x)\}$ in this space, illustrated in Figure 8.1.1, that converges to a step function which is not in $C^1([-1, 1])$. Here,

$$f_n(x) = \begin{cases} -1 & \text{if } x \leq -\frac{1}{n}, \\ \frac{3n}{2}x - \frac{n^3}{2}x^3 & \text{if } -\frac{1}{n} < x < \frac{1}{n}, \\ 1 & \text{if } x \geq \frac{1}{n}, \end{cases} \quad (8.2)$$

and $f_n(x) \in C^1([-1, 1])$ for all n , since $\lim_{x \rightarrow 1/n^-} f_n(x) = 1$, $\lim_{x \rightarrow -1/n^+} f_n(x) = -1$, and $\lim_{x \rightarrow 1/n^-} f'_n(x) = \lim_{x \rightarrow -1/n^+} f'_n(x) = 0$. Since much of the finite-element theory relies on various smoothness criteria, it is important to have spaces that are complete under these properties.

In the following sections, we outline several commonly used function spaces with their inner products, norms, and weak derivatives. In each case, an example is provided to illustrate properties of the space. To begin, we define vector and function spaces.

Definition 8.1: Vector space

A vector space $\mathcal{V}$ over field $\mathbb{F}$ is a set that is closed under vector addition and scalar multiplication: for any $u, v \in \mathcal{V}$, $u + v \in \mathcal{V}$, and for any $\alpha \in \mathbb{F}$ and $v \in \mathcal{V}$, $\alpha v \in \mathcal{V}$, where the addition and multiplication operations satisfy the standard axioms (associativity and commutativity of vector addition, existence of addition and multiplication identities, existence of an additive inverse, and associativity and distributivity of scalar multiplication; see [23]).

Example 8.2: Vector space

Let $\mathcal{V} = \mathbb{R}^3$ and $\mathbb{F} = \mathbb{R}$. Then, the following hold for any $u, v \in \mathcal{V}$ and $\alpha \in \mathbb{F}$:

- $v + u = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} + \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \begin{bmatrix} v_1 + u_1 \\ v_2 + u_2 \\ v_3 + u_3 \end{bmatrix} \in \mathcal{V}$, and
- $\alpha v = \alpha \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} \alpha v_1 \\ \alpha v_2 \\ \alpha v_3 \end{bmatrix} \in \mathcal{V}$.

In the special case where $\mathcal{V}$ is a set of functions, we call the vector space a *function space*.

Example 8.3: Function space

Let $\Omega \subset \mathbb{R}^n$ be open, let $\mathbb{F} = \mathbb{R}$, and let $\mathcal{V} = C^0(\Omega) = \{f \mid f \text{ is continuous on } \Omega\}$. Then, for any $f, g \in \mathcal{V}$ and $\alpha \in \mathbb{F}$ we have that $f + g \in \mathcal{V}$ and $\alpha f \in \mathcal{V}$, where the function $f + g$ is defined by $(f + g)(x) = f(x) + g(x)$ and the function αf is defined by $(\alpha f)(x) = \alpha f(x)$.

8.1.2 ■ Function spaces C^0 , C^1 , and C^k

In Example 8.3, we consider $C^0(\Omega)$, the space of all continuous functions on the open domain $\Omega \subset \mathbb{R}^n$. This space and similar ones, such as $C^k(\Omega)$, $C^\infty(\Omega)$, and $L^2(\Omega)$, were introduced earlier in this book. However, we reintroduce them here in order to dive into their properties a bit more and to see how they are used for finite-element methods. We start with a general definition of differentiability classes of functions.

Definition 8.4: Vector space of functions with k continuous derivatives

The function space $C^k(\Omega)$ is defined as

$$C^k(\Omega) = \{f \in C^0(\Omega) \mid D^\alpha f \in C^0(\Omega) \text{ for all } \alpha \text{ such that } |\alpha| \leq k\}, \quad (8.3)$$

where $D^\alpha f = \frac{\partial^{|\alpha|} f}{\partial x_1^{\alpha_1} \partial x_2^{\alpha_2} \dots \partial x_n^{\alpha_n}}$ and $|\alpha| = \alpha_1 + \alpha_2 + \dots + \alpha_n$.

We generally assume that a domain $\Omega \subset \mathbb{R}^n$ is open, but spaces of functions with k continuous derivatives can also be defined over $\overline{\Omega}$, the closure of Ω . In addition to $C^k(\Omega)$, we define two additional commonly used spaces:

- the space of infinite regularity, $C^\infty(\Omega)$,

$$C^\infty(\Omega) = \{f \in C^0(\Omega) \mid D^\alpha f \in C^0(\Omega) \text{ for all } \alpha\}, \text{ and} \quad (8.4)$$

- the space of continuous functions with compact support,

$$C_0^k(\Omega) = \{f \in C^k(\Omega) \mid f \text{ has compact support}\}, \quad (8.5)$$

where the support of a function $f(x)$ is defined as the closure of the set of points where $f(x) \neq 0$, and $f(x)$ is said to have compact support when the support of $f(x)$ is a compact set in $\mathbb{R}^n$, i.e., a closed and bounded set.

Example 8.5: Functions in C_0^k

As an example, consider the case where $\Omega = \mathbb{R}$. It is easy to construct functions in $C_0^0(\mathbb{R})$, the set of continuous functions with compact support. A nice example is

$$f(x) = \begin{cases} 4(x - x^2) & \text{if } 0 < x < 1, \\ 0 & \text{otherwise,} \end{cases} \quad (8.6)$$

noting that $x - x^2$ takes value zero at $x = 0$ and $x = 1$, while the scaling by a factor of four gives $f(x)$ a maximum value of one at $x = 1/2$. This function is in $C_0^0(\mathbb{R})$, but not $C_0^1(\mathbb{R})$, because the derivative of $x - x^2$ is not zero at $x = 0$ or $x = 1$.

To construct functions in $C_0^k(\mathbb{R})$ for $0 < k < \infty$, we can simply take powers of $f(x)$. For example, $f^2(x) = 16(x - x^2)^2$ has derivative $2f(x)f'(x)$ and, consequently, has a zero derivative for $x = 0$ and $x = 1$. Thus, $f^2(x)$ is a prototypical function in $C_0^1(\mathbb{R})$. We used a similarly

constructed function in the proof of Theorem 4.2, in equation (4.11). Similar calculations show that $f^3(x)$ is in $C_0^2(\mathbb{R})$, and so on.

This example does not directly extend to $C_0^\infty(\mathbb{R})$, since the pointwise values of $f^{k+1}(x)$ tend to zero for all $x \neq 1/2$ as $k \rightarrow \infty$ (confirming that the pointwise limit of continuous functions need not be continuous). Constructing a function in $C^\infty(\mathbb{R})$ that has compact support is a bit trickier, but one example is the so-called bump function

$$f(x) = \begin{cases} e^{\frac{1}{x^2-1}} & \text{if } |x| < 1, \\ 0 & \text{if } |x| \geq 1, \end{cases} \quad (8.7)$$

which has infinitely many continuous derivatives at $x = \pm 1$, since the decay of the exponential as $|x| \rightarrow 1^-$ outweighs the polynomial growth of the rational function that multiplies the exponential term in the derivatives of $f(x)$. Thus, the function and its derivative are continuous with compact support (in $[-1, 1]$). As a result, $f \in C_0^\infty(\mathbb{R})$.

8.1.3 ■ Normed vector spaces and Banach spaces

Vector spaces defined as in Definition 8.1 can be equipped with a norm. A *norm* on vector space $\mathcal{V}$ is defined as a function,

$$\|\cdot\| : \mathcal{V} \rightarrow \mathbb{R}, \quad (8.8)$$

such that the following hold for all elements $f, g \in \mathcal{V}$ and for every scalar $\alpha \in \mathbb{R}$:

1. $\|f\| \geq 0$, and $\|f\| = 0$ if and only if $f = 0$;
2. $\|\alpha f\| = |\alpha| \|f\|$; and
3. $\|f + g\| \leq \|f\| + \|g\|$.

If a norm exists on a function space, the pairing $(\mathcal{V}, \|\cdot\|)$ is termed a *normed* vector space. Given a normed vector space $(\mathcal{V}, \|\cdot\|)$, or simply $\mathcal{V}$, and a sequence of functions, namely, $\{f_i\}$ where $f_i \in \mathcal{V}$, the sequence is said to converge to a limit $f \in \mathcal{V}$ if $\lim_{i \rightarrow \infty} \|f_i - f\| = 0$.

The concept of a *Cauchy sequence* is central in formulating completeness properties of function spaces. The sequence $\{f_i\}$ is called a Cauchy sequence if, for any $\epsilon > 0$, there exists $N > 0$ such that

$$\|f_i - f_j\| < \epsilon \quad \text{for all } i, j > N. \quad (8.9)$$

This allows us to formally define a complete normed vector space as a space where each Cauchy sequence converges to a limit in the space. This is called a Banach space and is formally defined in Definition 8.6.

Definition 8.6: Banach space

A complete normed vector space $(\mathcal{V}, \|\cdot\|)$, or Banach space, is a normed vector space for which every Cauchy sequence, $\{f_i\}$, in $\mathcal{V}$ has a limit $f \in \mathcal{V}$.

Example 8.7: $C^0(\Omega)$ versus $C^0(\overline{\Omega})$

If we consider $C^0(\Omega)$, where Ω is an open, bounded set, then the space cannot be a normed vector space under the sup-norm because functions in $C^0(\Omega)$ can be unbounded on the boundary of Ω . The function $f(x) = \frac{1}{x}$, for example, is unbounded on $(0, 1)$, so the sup-norm $\|f\|_\infty = \sup_{x \in \overline{\Omega}} |f(x)|$ does not define a norm on $C^0((0, 1))$.

However, if we define C^0 over the closure of Ω , $\bar{\Omega}$, then we can show completeness under the sup-norm—i.e., $(C^0(\bar{\Omega}), \|\cdot\|_\infty)$ is a Banach space. Let $\mathcal{V} = C^0(\bar{\Omega})$; then, since each $f \in \mathcal{V}$ is continuous on a closed and bounded domain, there exists an $M \in \mathbb{R}$ such that $\|f\|_\infty \leq M < \infty$. As a result, $(C^0(\bar{\Omega}), \|\cdot\|_\infty)$ is a normed vector space (since $\|\cdot\|_\infty$ satisfies the definition of a norm).

To prove completeness, consider a Cauchy sequence of functions, $\{f_i\}$, in $C^0(\bar{\Omega})$. For each fixed $x \in \bar{\Omega}$, the sequence $\{f_i(x)\}$ is a Cauchy sequence in $\mathbb{R}$ and, so, $\lim_{i \rightarrow \infty} f_i(x)$ exists and is finite. Define $f(x) = \lim_{i \rightarrow \infty} f_i(x)$ for each $x \in \bar{\Omega}$. Then, we need to show that $f(x)$ is continuous and that $\lim_{i \rightarrow \infty} \|f - f_i\|_\infty = 0$, so that $f(x)$ is the limit of the Cauchy sequence in the space.

To see that $f(x)$ is continuous, let $\epsilon > 0$ be given. Since $\{f_i\}$ is Cauchy with respect to the sup-norm, there exists an N such that $|f_n(x) - f_m(x)| < \epsilon$ for all $n, m > N$ and all $x \in \bar{\Omega}$. Taking the limit of this as $m \rightarrow \infty$ gives $|f_n(x) - f(x)| \leq \epsilon$ for all $n > N$ and all $x \in \bar{\Omega}$. Thus, $\{f_i\}$ converges *uniformly* to f , which implies that $f(x)$ is continuous on $\bar{\Omega}$ since the f_i are continuous. This also directly shows that $\lim_{i \rightarrow \infty} \|f - f_i\|_\infty = 0$. Consequently, $C^0(\bar{\Omega})$ is a Banach space with norm $\|\cdot\|_\infty$.

As illustrated throughout this book, we are often concerned with functions with “compact support,” or those that decay “sufficiently fast.” Thus, we consider another example of a class of Banach spaces, outlined in Definition 8.8.

Definition 8.8: L^p spaces

L^p spaces are defined as follows for $1 \leq p < \infty$:

$$L^p(\Omega) = \{u \mid u \text{ is real and measurable, } \int_{\Omega} |u|^p dx < \infty\}. \quad (8.10)$$

The following L^p norm defines a norm on this space:

$$\|u\|_p = \left(\int_{\Omega} |u|^p dx \right)^{1/p}. \quad (8.11)$$

In the case of $p = \infty$, we define the space as

$$L^\infty(\Omega) = \{u \mid |u(x)| < \infty \text{ almost everywhere}\}, \quad (8.12)$$

with norm

$$\|u\|_\infty = \inf\{C \mid |u(x)| \leq C \text{ almost everywhere}\}. \quad (8.13)$$

This is a good place to remind the reader of the discussion in Remark 4.1 of Chapter 4: elements of the L^p spaces defined in Definition 8.8 can be viewed as *equivalence classes* of functions that differ in value on at most a set of points of measure zero. This explains why we define the sup-norm of functions $u(x)$ in L^p spaces as in equation (8.13), whereas the sup-norm for the $C^0(\bar{\Omega})$ space of continuous functions $f(x)$ in Example 8.7 can be defined in a simpler pointwise manner as $\|f\|_\infty = \sup_{x \in \bar{\Omega}} |f(x)|$. Likewise, the issue of integrability in Definition 8.8 (and elsewhere in the text) is closely tied to the concept of *measurability*, namely, the assumption of a measurable function. While this plays an important role in a rigorous study of functional analysis, we sidestep the definition here and point to a discussion in [23, Sec. 1.2.3].

L^p spaces are Banach spaces with several useful properties, two of which are:

Hölder's inequality: For $1 \leq p, q \leq \infty$ with $\frac{1}{p} + \frac{1}{q} = 1$, with $u \in L^p(\Omega)$ and $v \in L^q(\Omega)$,

$$\|uv\|_1 \leq \|u\|_p \|v\|_q. \quad (8.14)$$

Minkowski's inequality: For $1 \leq p \leq \infty$, $u, v \in L^p(\Omega)$,

$$\|u + v\|_p \leq \|u\|_p + \|v\|_p. \quad (8.15)$$

8.1.4 ■ Inner product spaces and Hilbert spaces

While a complete normed vector space is a useful tool, many of the spaces encountered in the study of the numerical approximation of solutions to PDEs are based on an inner product $\langle \cdot, \cdot \rangle : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{R}$ for some vector space $\mathcal{V}$. For any f, g , and $h \in \mathcal{V}$ and scalar $\alpha \in \mathbb{R}$, the inner product satisfies

1. $\langle f, f \rangle \geq 0$ and $\langle f, f \rangle = 0$ if and only if $f = 0$,
2. $\langle f, g \rangle = \langle g, f \rangle$, and
3. $\langle \alpha f + g, h \rangle = \alpha \langle f, h \rangle + \langle g, h \rangle$.

The inner product induces a norm, $\|f\| = \sqrt{\langle f, f \rangle}$, so that every inner product space is also a normed vector space. If the space is complete under this induced norm, then it is a Banach space, but a complete inner product space is more specifically called a *Hilbert space*.

Definition 8.9: Hilbert space

A Hilbert space is an inner product space that is complete with respect to the norm induced by the inner product.

Example 8.10: $C^k(\overline{\Omega})$ spaces and the Hilbert space $L^2(\Omega)$

The $C^k(\overline{\Omega})$ spaces of continuous functions do not directly define Hilbert spaces. The natural norms on these spaces, such as $\|\cdot\|_\infty$, do not correspond to inner products, and for standard norms induced by inner products, such as the L^2 norm defined in Definition 8.8, the vector space is not complete. For instance, the sequence of $C^1([-1, 1])$ functions in equation (8.2) converges to a discontinuous function in the L^2 norm (i.e., one that is not in $C^1([-1, 1])$).

Hilbert spaces can, however, be defined by *completing* these spaces, considering the sets of functions reached as limits of those in $C^k(\overline{\Omega})$ under suitable norms (with associated inner products). For example, it can be shown that the completion of $C_0^\infty(\Omega)$ under $\|\cdot\|_2$ is the Hilbert space $L^2(\Omega) = \{u \mid \|u\|_2 < \infty\}$, which has inner product

$$\langle u, v \rangle = \int_{\Omega} uv \, d\mathbf{x}. \quad (8.16)$$

A useful tool when considering Hilbert spaces is the Hilbert projection theorem (Theorem 8.12). The theorem establishes an orthogonal decomposition when subspace $\mathcal{M} \subset \mathcal{V}$ is a closed subset of Hilbert space $\mathcal{V}$ —see Figure 8.1.2. The orthogonal complement of the subspace, $\mathcal{M}^\perp$, is defined in Definition 8.11. When subspace $\mathcal{M}$ is a closed set, it is called a closed subspace. The Hilbert projection theorem will be a useful tool in establishing upcoming results, such as the Riesz representation theorem, Theorem 8.22.

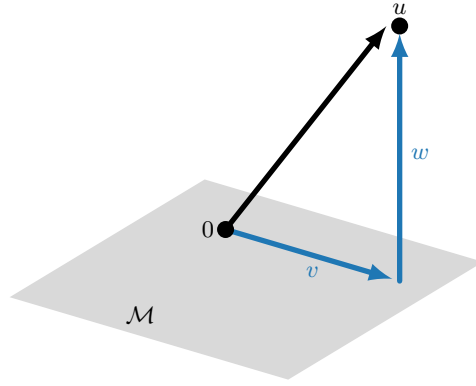


Figure 8.1.2. Projection of u onto closed subspace $\mathcal{M} \subset \mathcal{V}$.

Definition 8.11: Orthogonal complement

Let $\mathcal{M}$ be a closed subspace of Hilbert space $\mathcal{V}$ with inner product $\langle \cdot, \cdot \rangle_{\mathcal{V}}$. Then,

$$\mathcal{M}^{\perp} = \{w \in \mathcal{V} \mid \langle v, w \rangle_{\mathcal{V}} = 0 \ \forall v \in \mathcal{M}\}. \quad (8.17)$$

Theorem 8.12: Hilbert projection theorem (see, e.g., [101, Thm. 6.2.2])

Let $\mathcal{M}$ be a closed subspace of Hilbert space $\mathcal{V}$ with inner product $\langle \cdot, \cdot \rangle_{\mathcal{V}}$ and (induced) norm $\|\cdot\|_{\mathcal{V}}$. For any $u \in \mathcal{V}$ there exists a unique $v \in \mathcal{M}$ and a unique $w \in \mathcal{M}^{\perp}$ such that $u = v + w$, i.e., $\mathcal{V} = \mathcal{M} \oplus \mathcal{M}^{\perp}$.

Proof. Given u , v_1 , and $v_2 \in \mathcal{V}$, the following identity is used:

$$\|u - v_1\|_{\mathcal{V}}^2 + \|u - v_2\|_{\mathcal{V}}^2 = \frac{1}{2}\|v_1 - v_2\|_{\mathcal{V}}^2 + 2\left\|u - \frac{v_1 + v_2}{2}\right\|_{\mathcal{V}}^2. \quad (8.18)$$

First, let $\delta^2 = \inf_{\hat{v} \in \mathcal{M}} \|u - \hat{v}\|_{\mathcal{V}}^2$. We establish existence of a unique minimizer $v \in \mathcal{M}$.

Existence of $v \in \mathcal{M}$. Since $\mathcal{M}$ is closed (by assumption), we can consider a sequence $\{v_n\}$ of functions in $\mathcal{M}$ such that $\|u - v_n\|_{\mathcal{V}} \leq \delta^2 + \frac{1}{n}$ for each n . We show that $\{v_n\}$ is Cauchy (thus showing that $v_n \rightarrow v \in \mathcal{M}$, since $\mathcal{M}$ is complete because a closed subset of a complete space is automatically complete). From the triangle inequality and the identity in equation (8.18), we have

$$\|v_n - v_m\|_{\mathcal{V}}^2 \leq \|u - v_n\|_{\mathcal{V}}^2 + \|u - v_m\|_{\mathcal{V}}^2 \leq 2\delta^2 + \frac{1}{n} + \frac{1}{m} \quad (8.19a)$$

$$\Rightarrow \frac{1}{2}\|v_n - v_m\|_{\mathcal{V}}^2 + 2\left\|u - \frac{v_n + v_m}{2}\right\|_{\mathcal{V}}^2 \leq 2\delta^2 + \frac{1}{n} + \frac{1}{m}. \quad (8.19b)$$

Since $\frac{v_n + v_m}{2} \in \mathcal{M}$, $\delta^2 \leq \|u - \frac{v_n + v_m}{2}\|_{\mathcal{V}}^2$ and

$$\frac{1}{2}\|v_n - v_m\|_{\mathcal{V}}^2 + 2\delta^2 \leq 2\delta^2 + \frac{1}{n} + \frac{1}{m}, \quad (8.20)$$

showing that $\{v_n\}$ is Cauchy: for any ϵ , $\|v_n - v_m\|_{\mathcal{V}}^2 \leq \sqrt{2(\frac{1}{n} + \frac{1}{m})} \leq \epsilon$ if $n \geq 4/\epsilon^2$ and $m \geq 4/\epsilon^2$. Thus, the limit of this sequence, v , is in the space $\mathcal{M}$.

Uniqueness of $v \in \mathcal{M}$. Let v_1 and v_2 both satisfy $\delta^2 = \|u - v_1\|_{\mathcal{V}}^2 = \|u - v_2\|_{\mathcal{V}}^2$. Then, from equation (8.18) and the definition of δ ,

$$\|v_1 - v_2\|_{\mathcal{V}}^2 = 2\|u - v_1\|_{\mathcal{V}}^2 + 2\|u - v_2\|_{\mathcal{V}}^2 - 4 \left\| u - \frac{v_1 + v_2}{2} \right\|_{\mathcal{V}}^2 \leq 2\delta^2 + 2\delta^2 - 4\delta^2 = 0. \quad (8.21)$$

So, we must have $v_1 = v_2$.

Orthogonality: $w = u - v \in \mathcal{M}^\perp$. Let $w = u - v$, and suppose that $\langle w, \tilde{v} \rangle_{\mathcal{V}} = \alpha > 0$ for some $\tilde{v} \in \mathcal{M}$ with $\|\tilde{v}\| = 1$. Then, consider $v_1 = v + \alpha\tilde{v}$, for which we have

$$\|u - (v + \alpha\tilde{v})\|_{\mathcal{V}}^2 = \|w\|_{\mathcal{V}}^2 - 2\alpha \langle w, \tilde{v} \rangle_{\mathcal{V}} + \alpha^2 \|\tilde{v}\|_{\mathcal{V}}^2 \quad (8.22a)$$

$$= \|w\|_{\mathcal{V}}^2 - \alpha^2 \quad (8.22b)$$

$$< \|w\|_{\mathcal{V}}^2, \quad (8.22c)$$

which is a contradiction, since v minimizes $\|u - \hat{v}\|_{\mathcal{V}}$ over $\hat{v} \in \mathcal{M}$. Therefore, $\langle w, \tilde{v} \rangle_{\mathcal{V}} = 0$ for all $\tilde{v} \in \mathcal{M}$. $\square$

One of the advantages of working in an inner-product space is that we always have a norm induced by the inner product, as $\|f\|_{\mathcal{V}} = \sqrt{\langle f, f \rangle_{\mathcal{V}}}$. Such norms satisfy an important inequality, the *Cauchy–Schwarz inequality*, that we now prove in Theorem 8.13.

Theorem 8.13: Cauchy–Schwarz inequality

Let $\mathcal{V}$ be an inner-product space with inner product $\langle \cdot, \cdot \rangle_{\mathcal{V}}$ and induced norm $\|\cdot\|_{\mathcal{V}}$. Then, for any $u, v \in \mathcal{V}$,

$$|\langle u, v \rangle_{\mathcal{V}}| \leq \|u\|_{\mathcal{V}} \|v\|_{\mathcal{V}}. \quad (8.23)$$

Proof. Consider the function $p(t) = \langle tu + v, tu + v \rangle_{\mathcal{V}}$, noting that $p(t) \geq 0$ for all t . By direct computation, we have

$$p(t) = \langle tu, tu \rangle_{\mathcal{V}} + \langle tu, v \rangle_{\mathcal{V}} + \langle v, tu \rangle_{\mathcal{V}} + \langle v, v \rangle_{\mathcal{V}} = \|u\|_{\mathcal{V}}^2 t^2 + 2\langle u, v \rangle_{\mathcal{V}} t + \|v\|_{\mathcal{V}}^2. \quad (8.24)$$

Now, either $\|u\|_{\mathcal{V}} = 0$ (and the inequality is trivially satisfied, since $u = 0$) or $p(t)$ is a quadratic equation in t , which has at most one root, since $p(t) \geq 0$.

Assuming that $\|u\|_{\mathcal{V}} \neq 0$, $p(t)$ will have at most one real root if and only if

$$(2\langle u, v \rangle_{\mathcal{V}})^2 - 4\|u\|_{\mathcal{V}}^2 \|v\|_{\mathcal{V}}^2 \leq 0. \quad (8.25)$$

Rearranging this, we have

$$(\langle u, v \rangle_{\mathcal{V}})^2 \leq \|u\|_{\mathcal{V}}^2 \|v\|_{\mathcal{V}}^2. \quad (8.26)$$

Taking the square root of both sides leads to the inequality in the theorem. $\square$

8.1.5 ■ Weak derivatives

In many cases, our classical notion of a *strong* derivative is insufficient. For example, functions with “kinks” or nonsmooth features may be perfectly acceptable solutions to many of the PDEs

we consider, yet the derivative may not exist at some points in the domain. To this end, we consider the so-called *weak* derivative, defined using an integral formulation.

As a start, we consider *integrable* functions in $L^1(\Omega)$. In particular, we define the space of *locally* integrable functions

$$L^1_{\text{loc}}(\Omega) = \left\{ u(x) \mid \int_{\Omega} |u(x)\phi(x)| dx < \infty \text{ for every } \phi \in C_0^\infty(\Omega) \right\}, \quad (8.27)$$

where $C_0^\infty(\Omega)$ is the space of infinitely differentiable functions with compact support on open domain Ω . Note that $L^1(\Omega) \subset L^1_{\text{loc}}(\Omega)$, and $L^1_{\text{loc}}(\Omega)$ contains additional functions that do not satisfy growth restrictions at the boundary of the domain. For example, $L^1_{\text{loc}}((0, 1))$ contains the function $1/x$ (but $1/x \notin L^1((0, 1))$), and $L^1_{\text{loc}}(\mathbb{R})$ contains constant functions. With this, we define the weak derivative in Definition 8.14.

Definition 8.14: Weak derivative

Function $v \in L^1_{\text{loc}}(\Omega)$ is the weak partial derivative of $u \in L^1_{\text{loc}}(\Omega)$ in the k th component if

$$\int_{\Omega} v\phi dx = - \int_{\Omega} u \frac{\partial \phi}{\partial x_k} dx \quad (8.28)$$

for all $\phi \in C_0^\infty(\Omega)$. In the case of higher-order derivatives, let α be a multi-index. Then, v is the α weak partial derivative of u if

$$\int_{\Omega} v\phi dx = (-1)^{|\alpha|} \int_{\Omega} u \frac{\partial^{|\alpha|} \phi}{\partial_1^{\alpha_1} \partial_2^{\alpha_2} \dots \partial_n^{\alpha_n}} dx \quad (8.29)$$

for all $\phi \in C_0^\infty(\Omega)$. Using the notation $D^\alpha = \frac{\partial^{|\alpha|}}{\partial_1^{\alpha_1} \partial_2^{\alpha_2} \dots \partial_n^{\alpha_n}}$, we denote the weak derivative by $v := D_w^\alpha u$.

It is easy to see that the definition of the weak derivative above is motivated by the case where the function u has a classical derivative v . For example, when $v(x) = u'(x)$ with $x \in (0, 1)$, we can multiply both sides by the test function $\phi(x)$ and integrate to obtain $\int_0^1 v\phi dx = \int_0^1 u'\phi dx = -\int_0^1 u\phi' dx + u\phi|_0^1$, so $\int_0^1 v\phi dx = -\int_0^1 u\phi' dx$ for every $\phi(x)$ with compact support in $(0, 1)$.

Example 8.15: Weak derivatives

As an example of weak derivatives, consider the following three functions:

$$f_1(x) = 4(1 - x)x, \quad (8.30a)$$

$$f_2(x) = \begin{cases} 2x & \text{if } 0 \leq x \leq \frac{1}{2}, \\ 2 - 2x & \text{if } \frac{1}{2} < x \leq 1, \end{cases} \quad (8.30b)$$

$$f_3(x) = \begin{cases} -\sqrt{\frac{1}{2} - x} + \sqrt{\frac{1}{2}} & \text{if } 0 \leq x \leq \frac{1}{2}, \\ -\sqrt{x - \frac{1}{2}} + \sqrt{\frac{1}{2}} & \text{if } \frac{1}{2} < x \leq 1. \end{cases} \quad (8.30c)$$

The functions are displayed in Figure 8.1.3, where we see that there is a decrease in smoothness at $x = \frac{1}{2}$ and that the strong derivative does not exist for f_2 and f_3 . Yet, using Definition 8.14,

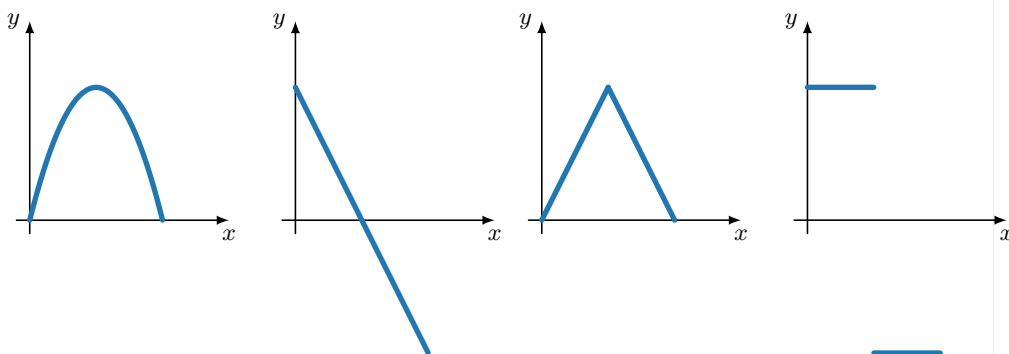
the weak derivative, which corresponds to the piecewise derivative for f_2 and f_3 , is defined as follows:

$$D_w f_1(x) = 4 - 8x, \quad (8.31a)$$

$$D_w f_2(x) = \begin{cases} 2 & \text{if } 0 \leq x < \frac{1}{2}, \\ \alpha & \text{if } x = \frac{1}{2}, \\ -2 & \text{if } \frac{1}{2} < x \leq 1, \end{cases} \quad (8.31b)$$

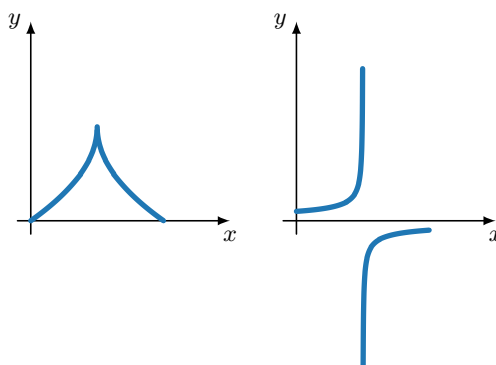
$$D_w f_3(x) = \begin{cases} \frac{1}{2\sqrt{\frac{1}{2}-x}} & \text{if } 0 \leq x < \frac{1}{2}, \\ \alpha & \text{if } x = \frac{1}{2}, \\ -\frac{1}{2\sqrt{x-\frac{1}{2}}} & \text{if } \frac{1}{2} < x \leq 1, \end{cases} \quad (8.31c)$$

where $\alpha \in \mathbb{R}$ is an arbitrary constant. Notice that the pointwise value of the weak derivative at $x = 1/2$ is immaterial for $f_2(x)$ and $f_3(x)$ due to the integral in the definition of the weak derivative. In this sense, the weak derivative defines an equivalence class of functions.



a. Left: $f_1(x)$. Right: $D_w f_1(x)$.

b. Left: $f_2(x)$. Right: $D_w f_2(x)$.



c. Left: $f_3(x)$. Right: $D_w f_3(x)$.

Figure 8.1.3. Functions $f_1(x)$, $f_2(x)$, and $f_3(x)$, and their associated weak derivatives.

8.1.6 ■ Sobolev spaces

With the weak derivative defined in the previous section, we introduce a special type of function space, the Sobolev space. For finite-element methods, the most relevant type of Sobolev space is usually a space that includes all functions that are in L^2 and that have (weak) derivatives up to some order in L^2 , and these spaces are Hilbert spaces. For example, we can define the Sobolev space for functions in one dimension as

$$\mathcal{V} = \{v \in L^2(\Omega) \mid D_w v \in L^2(\Omega)\}. \quad (8.32)$$

Recalling Example 8.15, functions $D_w f_1(x)$ and $D_w f_2(x)$ are both in $L^2(\Omega)$, so $f_1, f_2 \in \mathcal{V}$. However, $D_w f_3 \notin L^2(\Omega)$, so $f_3 \notin \mathcal{V}$.

To generalize to higher derivatives and dimensions, we first define the so-called (k, p) Sobolev space and corresponding Sobolev norm, which essentially define the space of functions with k weak derivatives that have finite L^p norm.

Definition 8.16: Sobolev space

Let $\Omega \subset \mathbb{R}^n$, k be a nonnegative integer, and $1 \leq p < \infty$. The (k, p) Sobolev space is given by

$$W^{k,p}(\Omega) = \{u : \Omega \rightarrow \mathbb{R} \mid \|u\|_{k,p} < \infty\}, \quad (8.33a)$$

with Sobolev norm defined as

$$\|u\|_{k,p}^p = \sum_{|\alpha| \leq k} \|D_w^\alpha u\|_p^p. \quad (8.33b)$$

For the special case of $p = 2$ and $k = 0$, note that $W^{0,2}(\Omega)$ is simply $L^2(\Omega)$. As mentioned above, the most relevant family of Sobolev spaces when working with finite-element methods comprises the ones with $p = 2$. Therefore, we introduce the standard notation for such spaces.

Definition 8.17: L^2 -based Sobolev spaces

Let $\Omega \subset \mathbb{R}^n$ and k be a nonnegative integer. Then, using Definition 8.16,

$$H^k(\Omega) = W^{k,2}(\Omega). \quad (8.34)$$

These spaces are inner-product spaces, with inner products

$$\langle u, v \rangle_{k,2} = \sum_{|\alpha| \leq k} \langle D_w^\alpha u, D_w^\alpha v \rangle, \quad (8.35)$$

where $\langle \cdot, \cdot \rangle$ denotes the $L^2(\Omega)$ inner product defined in Example 8.10 (equation (8.16)).

Remark 8.18: Inner product and norm notation for L^2 -based Sobolev spaces

For simplicity, we introduce the following notation for the $L^2(\Omega)$ and $H^k(\Omega)$ inner products and norms (for $k \geq 0$):

$$\langle u, v \rangle = \langle u, v \rangle_{0,2}, \quad \|u\| = \langle u, u \rangle^{1/2} = \|u\|_{0,2}, \quad (8.36a)$$

$$\langle u, v \rangle_k = \langle u, v \rangle_{k,2}, \quad \|u\|_k = \langle u, u \rangle_k^{1/2} = \|u\|_{k,2}. \quad (8.36b)$$

We also define the $H^k(\Omega)$ seminorm as

$$|u|_k^2 = \sum_{|\alpha|=k} \|D_w^\alpha u\|^2. \quad (8.37)$$

Note that the Sobolev spaces defined in Definition 8.16 are Banach spaces in general and Hilbert spaces in the special case $p = 2$, hence the $H^k(\Omega)$ notation in equation (8.34).

In the context of PDEs, we aim to find weak solutions that are in these Sobolev spaces. Since the solutions must satisfy the PDE plus some boundary conditions, we further specify the notation to account for this. Thus, we define

$$H_q^k(\Omega) := \{u \in H^k(\Omega) \mid u = q \text{ on } \Gamma_{\mathcal{D}}\}, \quad (8.38)$$

where $\Gamma_{\mathcal{D}} \subset \partial\Omega$ corresponds to the part of the boundary with Dirichlet boundary conditions. Then, for homogeneous boundary conditions, we look for solutions $u \in H_0^k$. Technically, $H_q^k(\Omega)$ is not a Hilbert space (or even a vector space) for $q \neq 0$, since for two functions in the (affine) space, u and v , $u + v$ would not have the correct boundary condition for the space. Fortunately, $H_0^k(\Omega)$ is still a Hilbert space, so we mostly consider H_0^k spaces and ignore further subtleties for now, revisiting this issue in Section 8.4.

Remark 8.19: Product spaces

For most of this section, we have been thinking of spaces of *scalar functions*, $u : \Omega \rightarrow \mathbb{R}$. However, all of the above applies to vector-valued functions $\mathbf{u} : \Omega \rightarrow \mathbb{R}^n$ as well. We denote a space of vector-valued functions with vector notation, e.g., $\mathbf{H}^1(\Omega)$, or we may use $(H^1(\Omega))^n$ to explicitly indicate the dimension. When defining inner products on these spaces, we simply sum the terms componentwise. For example,

$$\langle \mathbf{u}, \mathbf{v} \rangle_1 = \sum_{i=1}^n \left(\langle u_i, v_i \rangle + \sum_{|\alpha| \leq 1} \langle D_w^\alpha u_i, D_w^\alpha v_i \rangle \right) \quad (8.39a)$$

$$= \langle \mathbf{u}, \mathbf{v} \rangle + \langle \nabla \mathbf{u}, \nabla \mathbf{v} \rangle. \quad (8.39b)$$

In the latter expression, we generalize the notation for the $L^2(\Omega)$ inner product to vector and tensor spaces, where we first take the componentwise inner product of the vector or tensor objects and then integrate the resulting scalar function over the domain, Ω . We also follow the common convention of dropping the notation for weak derivatives, writing simply $\nabla \mathbf{u}$, for example, and we shall always interpret this as meaning weak derivatives when the corresponding strong derivatives are not well defined.

The H^k (and $\mathbf{H}^k$) spaces concern functions and their partial derivatives. However, we also consider cases where divergences or curls of vector-valued functions are of primary interest. Thus, further Sobolev spaces can be defined, for example,

$$\mathbf{H}(\text{div}, \Omega) := \{\mathbf{u} \in L^2(\Omega) \mid \nabla \cdot \mathbf{u} \in L^2(\Omega)\}, \quad (8.40a)$$

$$\mathbf{H}(\text{curl}, \Omega) := \{\mathbf{u} \in L^2(\Omega) \mid \nabla \times \mathbf{u} \in L^2(\Omega)\}, \quad (8.40b)$$

with corresponding inner products,

$$\langle \mathbf{u}, \mathbf{v} \rangle_{\text{div}} = \langle \mathbf{u}, \mathbf{v} \rangle + \langle \nabla \cdot \mathbf{u}, \nabla \cdot \mathbf{v} \rangle, \quad (8.41a)$$

$$\langle \mathbf{u}, \mathbf{v} \rangle_{\text{curl}} = \langle \mathbf{u}, \mathbf{v} \rangle + \langle \nabla \times \mathbf{u}, \nabla \times \mathbf{v} \rangle, \quad (8.41b)$$

and induced norms,

$$\|\mathbf{u}\|_{\text{div}}^2 = \langle \mathbf{u}, \mathbf{u} \rangle_{\text{div}} = \|\mathbf{u}\|^2 + \|\nabla \cdot \mathbf{u}\|^2, \quad (8.42a)$$

$$\|\mathbf{u}\|_{\text{curl}}^2 = \langle \mathbf{u}, \mathbf{u} \rangle_{\text{curl}} = \|\mathbf{u}\|^2 + \|\nabla \times \mathbf{u}\|^2. \quad (8.42b)$$

These will be used extensively in Chapter 10.

Now that we have the basics of functional analysis in place, we proceed to apply these ideas to determining the existence and uniqueness of weak solutions for linear elliptic PDEs of second order.

8.2 ■ Existence and uniqueness

Objectives

The focus of this section is on developing the framework for proving existence and uniqueness of solutions to the weak forms of elliptic PDEs. To start, we must outline the tools needed to establish key properties of these weak forms. For a given problem, you will consider questions such as the following:

- *Does a weak solution exist?*
- *Is the solution unique?*

In addition, you will try to determine under which conditions weak solutions exist and are unique:

- *What is a suitable space for the right-hand side of the PDE?*

Thus, in this section, you will discuss the Riesz representation theorem and apply it to a model problem. Then, you will use the Riesz representation theorem to establish the Lax–Milgram theorem, which allows us to obtain existence and uniqueness of weak solutions for PDEs. Finally, you will consider specialized tools, such as coercivity, continuity, bounded linear functionals, and the Poincaré inequality, to prove existence and uniqueness of weak solutions for some example problems.

To begin, consider a model problem,

$$-\nabla \cdot \nabla u + u = f(x) \quad \text{for } x \in \Omega, \quad (8.43)$$

with $u = 0$ on $\partial\Omega$ and with $f \in L^2(\Omega)$. Just as in Chapter 4, we multiply both sides of equation (8.43) by $v \in \mathcal{V}$ for some Hilbert space $\mathcal{V}$ and integrate over Ω . Here, the relevant integration by parts identity comes from the divergence theorem (see equations (2.89b) and (2.91)), giving us

$$\int_{\Omega} (-\nabla \cdot \nabla u) v \, d\mathbf{x} = \int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x} - \int_{\partial\Omega} (v \nabla u) \cdot \mathbf{n} \, ds. \quad (8.44)$$

As in Chapter 4, we find that the right choice of $\mathcal{V}$ is $H_0^1(\Omega)$, as we need the gradients of our functions to be in $L^2(\Omega)$ after integration by parts. Also, with this choice, $v \in H_0^1(\Omega)$ is zero on $\partial\Omega$, and the boundary integral is always zero. Thus, the weak form of the PDE in equation (8.43) is to find $u \in \mathcal{V} = H_0^1(\Omega)$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x} + \int_{\Omega} uv \, d\mathbf{x} = \int_{\Omega} f v \, d\mathbf{x} \quad \text{for all } v \in \mathcal{V}. \quad (8.45)$$

Recalling Section 4.1, we define a bilinear form $a(\cdot, \cdot)$ and linear functional $g(\cdot)$ as

$$a(u, v) = \langle \nabla u, \nabla v \rangle + \langle u, v \rangle, \quad (8.46a)$$

$$g(v) = \langle f, v \rangle, \quad (8.46b)$$

where, again, $\langle \cdot, \cdot \rangle$ denotes the $L^2(\Omega)$ inner product, as defined in Section 8.1.6. In the following, we establish properties of both $a(u, v)$ and $g(v)$ needed to show that the weak form in equation (8.45) has a unique solution. To do this, we rely on having a Hilbert space, $\mathcal{V}$.

One important observation is that $g(\cdot)$ is a *bounded* linear functional on the Hilbert space $H^1(\Omega)$ —see Definition 8.20. Bounded linear functionals have a specific representation in the Hilbert space, as we will see from the Riesz representation theorem (Theorem 8.22). For this, we need the dual space of $\mathcal{V}$, denoted by $\mathcal{V}^*$ —see Definition 8.21.

Definition 8.20: Bounded linear functional

Let $\mathcal{V}$ be a normed vector space with norm $\|\cdot\|_{\mathcal{V}}$. A linear functional, $g(\cdot)$, on $\mathcal{V}$ is a linear operator of the form $g : \mathcal{V} \rightarrow \mathbb{R}$. In addition, g is a bounded linear functional if there is a constant $c > 0$ such that

$$|g(v)| \leq c\|v\|_{\mathcal{V}} \quad \text{for all } v \in \mathcal{V}. \quad (8.47)$$

Definition 8.21: Dual space

Let $\mathcal{V}$ be a Hilbert space with the induced norm denoted by $\|\cdot\|_{\mathcal{V}}$. Then, the dual space, $\mathcal{V}^*$, is the space of all bounded linear functionals on $\mathcal{V}$:

$$\mathcal{V}^* = \{g : \mathcal{V} \rightarrow \mathbb{R} \mid g \text{ is a bounded linear functional}\}. \quad (8.48)$$

In addition, for operator $g \in \mathcal{V}^*$, the operator norm is defined as

$$\|g\|_{\mathcal{V}^*} = \sup_{u \in \mathcal{V}, u \neq 0} \frac{|g(u)|}{\|u\|_{\mathcal{V}}}. \quad (8.49)$$

For the model problem above, with $g(u) = \langle f, v \rangle$, we can show that g is a bounded linear functional since $f \in L^2(\Omega)$, just by using the Cauchy–Schwarz inequality, giving

$$|g(v)| = \left| \int_{\Omega} f(x)v \, dx \right| \leq \|f\| \|v\| \leq \|f\| \|v\|_1 \quad (8.50)$$

for any $v \in H_0^1$, so the constant in Definition 8.20 can be taken to be $\|f\|$, and $g(v)$ is a bounded linear functional for any $f \in L^2(\Omega)$. Observe, however, that $f(x)v$ may be integrable even if $f \notin L^2(\Omega)$. To formalize this, we introduce *negative-index* Sobolev spaces, where the norm on $f \in H^{-1}(\Omega)$ is given by

$$\|f\|_{-1} = \sup_{v \in H^1(\Omega)} \frac{\left| \int_{\Omega} f(x)v \, dx \right|}{\|v\|_1}. \quad (8.51)$$

Thus, the bound in equation (8.50) can be extended if $f \in H^{-1}(\Omega)$ to $|g(v)| \leq \|f\|_{-1} \|v\|_1$.

One often-used example to highlight the space $H^{-1}([-1, 1])$ is that of a Dirac distribution, or $f = D_w H(x)$, with $H(x)$ being the Heaviside function. In this case, we see that $\int_{[-1, 1]} f v \, dx = -\int_{[-1, 1]} H(x) D_w v(x) \, dx$ for all $v \in H_0^1([-1, 1])$. Since both $H(x)$ and $D_w v(x)$ are in L^2 , the integral is bounded. Therefore, $f = D_w H(x) \in H^{-1}([-1, 1])$, but f is not in $L^2([-1, 1])$. More details on H^{-1} and distributions can be found in [93, Ch. I.4].

8.2.1 ■ Riesz representation theorem

Next, we show how to associate the bounded linear functional $g(v)$ with a unique element of the space $\mathcal{V}$ through the Riesz representation theorem, presented in Theorem 8.22. This theorem also leads to establishing existence and uniqueness for our full weak problem.

Theorem 8.22: Riesz representation theorem (cf. [23, Thm. 2.5.8])

Let $\mathcal{V}$ be a Hilbert space with inner product $\langle \cdot, \cdot \rangle_{\mathcal{V}}$. Let g be a bounded linear functional on $\mathcal{V}$ —i.e., $g \in \mathcal{V}^*$. Then, there is a unique $u \in \mathcal{V}$ such that

$$g(v) = \langle u, v \rangle_{\mathcal{V}} \quad \text{for all } v \in \mathcal{V}. \quad (8.52)$$

Proof. Consider $g \in \mathcal{V}^*$ and write $\mathcal{K} = \text{Ker}(g)$, the kernel or nullspace of g . We note that $\mathcal{K}$ is a closed linear subspace of $\mathcal{V}$ since for any converging sequence $v_n \in \mathcal{K}$ with $v_n \rightarrow v$, we have that $|g(v)| = |g(v_n) - g(v)| \leq \|g\|_{\mathcal{V}^*} \|v_n - v\|_{\mathcal{V}} \rightarrow 0$, since g is linear and bounded. As a result, $v \in \mathcal{K}$.

First, to establish existence of u we consider two cases.

Case 1: $\mathcal{K} = \mathcal{V}$. In this case, for each $v \in \mathcal{V}$, $g(v) = 0$. Therefore, $u = 0$ is the unique element in $\mathcal{V}$ such that $g(v) = \langle u, v \rangle_{\mathcal{V}}$ for all $v \in \mathcal{V}$.

Case 2: $\mathcal{K} \neq \mathcal{V}$. From the Hilbert projection theorem (Theorem 8.12), there must exist an element $w \in \mathcal{K}^\perp$ with $w \neq 0$. Then, let $\alpha = g(w) \neq 0$ and observe that

$$g\left(\frac{g(v)}{\alpha}w\right) = \frac{g(v)}{\alpha}g(w) = g(v) \quad \text{for all } v \in \mathcal{V} \quad (8.53)$$

by linearity of g . As a result,

$$g(v - \frac{g(v)}{\alpha}w) = g(v) - g(v) = 0 \quad \text{for all } v \in \mathcal{V}. \quad (8.54)$$

Thus, for any $v \in \mathcal{V}$, $v - \frac{g(v)}{\alpha}w \in \mathcal{K}$, leading to

$$\langle v - \frac{g(v)}{\alpha}w, w \rangle_{\mathcal{V}} = 0, \quad (8.55)$$

since $w \in \mathcal{K}^\perp$. From this,

$$\left\langle \frac{g(v)}{\alpha}w, w \right\rangle_{\mathcal{V}} = \langle v, w \rangle_{\mathcal{V}} \quad \text{for all } v \in \mathcal{V}, \quad (8.56)$$

which yields

$$g(v) = \left\langle v, \frac{\alpha}{\langle w, w \rangle_{\mathcal{V}}}w \right\rangle_{\mathcal{V}} = \langle v, u \rangle_{\mathcal{V}} \quad \text{for all } v \in \mathcal{V}, \quad (8.57)$$

where we have established the existence of u with $u = \frac{\alpha}{\langle w, w \rangle_{\mathcal{V}}}w = \frac{g(w)}{\langle w, w \rangle_{\mathcal{V}}}w$.

Next, to establish the uniqueness of u , we suppose the contrary: $\exists \hat{u} \neq u$ such that $g(v) = \langle \hat{u}, v \rangle_{\mathcal{V}}$ for all $v \in \mathcal{V}$. Then,

$$g(v) = \langle u, v \rangle_{\mathcal{V}} = \langle \hat{u}, v \rangle_{\mathcal{V}} \quad \text{for all } v \in \mathcal{V}, \quad (8.58)$$

which shows that $\langle u - \hat{u}, v \rangle_{\mathcal{V}} = 0$ for all $v \in \mathcal{V}$. Thus, for $v = u - \hat{u}$, we see that $\langle u - \hat{u}, u - \hat{u} \rangle_{\mathcal{V}} = 0$, which results in $u - \hat{u} = 0$. $\square$

We gain some intuition into the Riesz representation theorem by considering a finite-dimensional example in Example 8.23, which shows the vector form of a linear functional on $\mathbb{R}^n$.

Example 8.23: Riesz representation theorem on $\mathbb{R}^n$

Let $\mathcal{V} = \mathbb{R}^n$ with the Euclidean inner product, and let $g : \mathcal{V} \rightarrow \mathbb{R}$ be a linear functional on $\mathcal{V}$. We would like to *associate* the functional $g(\cdot)$ with some element of the (Hilbert) vector space $\mathcal{V}$.

Any vector $\mathbf{v} \in \mathcal{V}$ can be written as

$$\mathbf{v} = v_1 \mathbf{e}^{(1)} + v_2 \mathbf{e}^{(2)} + \cdots + v_n \mathbf{e}^{(n)}, \quad (8.59)$$

where $\{\mathbf{e}^{(i)}\}$ forms the canonical basis for $\mathbb{R}^n$. Since $g(\cdot)$ is linear,

$$g(\mathbf{v}) = v_1 g(\mathbf{e}^{(1)}) + \cdots + v_n g(\mathbf{e}^{(n)}). \quad (8.60)$$

Defining $\mathbf{w}$ by $w_i = g(\mathbf{e}^{(i)})$, we then have that

$$g(\mathbf{v}) = \mathbf{v}^T \mathbf{w} = \langle \mathbf{v}, \mathbf{w} \rangle, \quad (8.61)$$

where the scalar product of vectors is the natural inner product on $\mathbb{R}^n$. As a result, we have identified a unique vector, $\mathbf{w}$, that represents $g(\cdot)$.

Returning now to our model problem, equation (8.45), we immediately obtain existence and uniqueness of the solution via the Riesz representation theorem, as seen in Example 8.24.

Example 8.24: Riesz representation theorem on $H_0^1(\Omega)$

For the model problem in equation (8.45), the bilinear form is equal to the $H^1(\Omega)$ inner product from Definition 8.17:

$$a(u, v) = \langle u, v \rangle + \langle \nabla u, \nabla v \rangle = \langle u, v \rangle_1. \quad (8.62)$$

Therefore, we rewrite the weak form in equation (8.45) as finding $u \in H_0^1(\Omega)$ such that

$$\langle u, v \rangle_1 = g(v) \text{ for all } v \in H_0^1(\Omega), \quad (8.63)$$

where we have chosen $H_0^1(\Omega)$ to account for the Dirichlet boundary conditions. Then, noting that we have already established that g is a bounded linear functional on $\mathcal{V} = H_0^1(\Omega)$ with norm $\|\cdot\|_1$ in equation (8.50) (with $f \in L^2(\Omega)$ or even $f \in H^{-1}(\Omega)$), the Riesz representation theorem establishes that the weak form in equation (8.45) has a unique solution.

8.2.2 ■ The Lax–Milgram theorem

While the Riesz representation theorem proved useful for equation (8.45), not all weak forms involve bilinear forms that are equal to the inner product on the relevant Hilbert space. For instance, consider a modified version of the model problem,

$$-\nabla \cdot \nabla u = f(x) \text{ for } x \in \Omega, \quad (8.64)$$

with $u = 0$ on $\partial\Omega$ and with $f \in L^2(\Omega)$. Then, the weak form is to find $u \in \mathcal{V} = H_0^1(\Omega)$ such that

$$\underbrace{\int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x}}_{a(u,v)} = \underbrace{\int_{\Omega} f(\mathbf{x})v \, d\mathbf{x}}_{g(v)} \text{ for all } v \in \mathcal{V}. \quad (8.65)$$

Again, $a(\cdot, \cdot) : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{R}$ is a *bilinear form* and $g(\cdot) : \mathcal{V} \rightarrow \mathbb{R}$ is a *linear functional*. Note that we have already established that $g(v)$ is in fact a *bounded* linear functional on $\mathcal{V} = H_0^1(\Omega)$ when $f \in L^2(\Omega)$; see equation (8.50).

The Riesz representation theorem of the previous section does not directly allow us to show existence and uniqueness of the solution to the weak problem in equation (8.65). To do so, we need to examine a few additional properties of the bilinear form $a(u, v)$. Definition 8.25 outlines the properties of *coercivity* and *continuity* of a bilinear form.

Definition 8.25: $\mathcal{V}$ -ellipticity (cf. [48, Def. II.2.4])

Given a Hilbert Space, $\mathcal{V}$, consider a bilinear form

$$a(\cdot, \cdot) : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{R}. \quad (8.66)$$

Then, $a(\cdot, \cdot)$ is coercive if there exists a constant $c_0 > 0$ such that

$$c_0 \|u\|_{\mathcal{V}}^2 \leq a(u, u) \quad \text{for all } u \in \mathcal{V}, \quad (8.67)$$

and $a(\cdot, \cdot)$ is continuous if there exists a constant $c_1 > 0$ such that

$$|a(u, v)| \leq c_1 \|u\|_{\mathcal{V}} \|v\|_{\mathcal{V}} \quad \text{for all } u, v \in \mathcal{V}. \quad (8.68)$$

If $a(\cdot, \cdot)$ is both coercive and continuous on $\mathcal{V}$, then $a(\cdot, \cdot)$ is said to be $\mathcal{V}$ -elliptic.

With this, we turn to a key theoretical result called the Lax–Milgram theorem. The Lax–Milgram theorem follows directly from the Riesz representation theorem if we place an additional assumption on the symmetry of $a(\cdot, \cdot)$, as we establish in Theorem 8.26. Here, we recall the discussion of Remark 4.1, that when we think of the function $u = 0 \in L^2(\Omega)$, we think of the equivalence class of functions where $u(\mathbf{x}) = 0$ for almost all $\mathbf{x} \in \Omega$.

Theorem 8.26: Lax–Milgram (symmetric bilinear forms)

Let $\mathcal{V}$ be a Hilbert space with inner product $\langle \cdot, \cdot \rangle_{\mathcal{V}}$. Assume that $a(\cdot, \cdot)$ is a symmetric bilinear form that is coercive and continuous on $\mathcal{V}$. In addition, assume that $g(\cdot)$ is a bounded linear functional on $\mathcal{V}$. Then, there exists a unique $u \in \mathcal{V}$ such that

$$a(u, v) = g(v) \quad \text{for all } v \in \mathcal{V}. \quad (8.69)$$

Proof. We observe several properties of $a(\cdot, \cdot)$ for u, v , and $w \in \mathcal{V}$ and $c \in \mathbb{R}$:

1. $a(u, v) = a(v, u)$ by the assumption of symmetry.
2. $a(cu + v, w) = ca(u, w) + a(v, w)$ due to bilinearity.
3. $a(u, u) \geq c_0 \|u\|_{\mathcal{V}}^2 \geq 0$ by coercivity.
4. If $u = 0$ almost everywhere, then $0 \leq a(u, u) \leq c_1 \|u\|_{\mathcal{V}}^2 \equiv 0$ by continuity. Thus, $a(u, u) = 0$.
5. If $a(u, u) = 0$, then $0 \leq c_0 \|u\|_{\mathcal{V}}^2 \leq a(u, u) = 0$ by coercivity. Thus, $u = 0$ almost everywhere.

Together, these show that $a(\cdot, \cdot)$ defines an inner product on $\mathcal{V}$. From the Riesz representation theorem with $(\mathcal{V}, a(\cdot, \cdot))$, there exists a unique $u \in \mathcal{V}$ such that $g(v) = a(u, v)$ for all $v \in \mathcal{V}$, completing the proof. $\square$

From the symmetric form of Lax–Milgram, we see that $\mathcal{V}$ -ellipticity of a symmetric bilinear form $a(\cdot, \cdot)$ means both that it defines an inner product on $\mathcal{V}$ and that the norm induced by that inner product is *equivalent* to the norm induced by $\langle \cdot, \cdot \rangle_{\mathcal{V}}$ —e.g., the inner product given by equation (8.35) in the case of $\mathcal{V} = W^{k,2}(\Omega)$. That is, by defining $\|u\|_a = \sqrt{a(u, u)}$ for $u \in \mathcal{V}$ we observe that

$$\sqrt{c_0}\|u\|_{\mathcal{V}} \leq \|u\|_a \leq \sqrt{c_1}\|u\|_{\mathcal{V}}. \quad (8.70)$$

The assumption of symmetry in Theorem 8.26 greatly simplifies the proof. We now turn to the more general form of the Lax–Milgram theorem in Theorem 8.27.

Theorem 8.27: Lax–Milgram (cf. [101, Thm. 6.2.7] and [149, Ch. 6, Thm. 6])

Let $\mathcal{V}$ be a Hilbert space with inner product $\langle \cdot, \cdot \rangle_{\mathcal{V}}$. Assume that $a(\cdot, \cdot)$ is a bilinear form that is coercive and continuous on $\mathcal{V}$. In addition, assume that $g(\cdot)$ is a bounded linear functional on $\mathcal{V}$. Then, there exists a unique $u \in \mathcal{V}$ such that

$$a(u, v) = g(v) \quad \text{for all } v \in \mathcal{V}. \quad (8.71)$$

Proof. We first consider any element $u \in \mathcal{V}$ and note that $a_u(v) := a(u, v)$ is a bounded linear functional on $\mathcal{V}$ due to continuity:

$$|a_u(v)| \leq c_1\|u\|_{\mathcal{V}}\|v\|_{\mathcal{V}}. \quad (8.72)$$

As a result, the Riesz representation theorem identifies a unique $t_u \in \mathcal{V}$ such that

$$a_u(v) = \langle v, t_u \rangle_{\mathcal{V}} \quad \text{for all } v \in \mathcal{V}. \quad (8.73)$$

Note that $a_u(v) = \langle v, t_u \rangle_{\mathcal{V}}$ defines a mapping $T : \mathcal{V} \rightarrow \mathcal{V}$: such that for any $u \in \mathcal{V}$, $t_u = Tu$ is defined through the Riesz representation theorem as above. We next show that T is linear, bounded, and onto $\mathcal{V}$.

T is linear. If we consider $\alpha u + w \in \mathcal{V}$ for $u, w \in \mathcal{V}$ and $\alpha \in \mathbb{R}$, then for any $v \in \mathcal{V}$,

$$\langle v, T(\alpha u + w) \rangle_{\mathcal{V}} = a_{\alpha u + w}(v) \quad (8.74a)$$

$$= a(\alpha u + w, v) \quad (8.74b)$$

$$= \alpha a(u, v) + a(w, v) \quad (8.74c)$$

$$= \alpha \langle v, Tu \rangle_{\mathcal{V}} + \langle v, Tw \rangle_{\mathcal{V}}. \quad (8.74d)$$

T is bounded. For $u \in \mathcal{V}$,

$$\|Tu\|_{\mathcal{V}}^2 = \langle Tu, Tu \rangle_{\mathcal{V}} \quad (8.75a)$$

$$= a_u(Tu) = a(u, Tu) \quad (8.75b)$$

$$\leq c_1\|Tu\|_{\mathcal{V}}\|u\|_{\mathcal{V}} \quad (\text{by continuity}). \quad (8.75c)$$

As a result, $\|Tu\|_{\mathcal{V}} \leq c_1\|u\|_{\mathcal{V}}$ for all u , so $\|T\| \leq c_1 < \infty$.

Range(T) is closed. Let $z_n = Tu_n$ be a sequence in Range(T). By definition of T , we have that $a(u_n, v) = \langle v, Tu_n \rangle_{\mathcal{V}} = \langle v, z_n \rangle_{\mathcal{V}}$ for all v , which means that

$$a(u_n - u_m, v) = \langle v, z_n - z_m \rangle_{\mathcal{V}}, \text{ and, consequently,} \quad (8.76a)$$

$$c_0\|u_n - u_m\|_{\mathcal{V}} \leq \|z_n - z_m\|_{\mathcal{V}}, \quad (8.76b)$$

by taking $v = u_n - u_m$ and using coercivity. If $z_n \rightarrow z \in \mathcal{V}$, then the sequence u_n must be Cauchy. However, since $\mathcal{V}$ is a Hilbert space, this means that u_n converges to some $u \in \mathcal{V}$. By continuity, then, $a(u_n, v) \rightarrow a(u, v)$ as a sequence in $\mathbb{R}$. We also have that $|\langle Tu_n - z, v \rangle_{\mathcal{V}}| \rightarrow 0$ by convergence of $z_n = Tu_n$; hence, $\langle v, Tu_n \rangle_{\mathcal{V}} \rightarrow \langle v, z \rangle_{\mathcal{V}}$. Since $a(u_n, v) = \langle v, Tu_n \rangle_{\mathcal{V}}$, this says that $a(u, v) = \langle v, z \rangle_{\mathcal{V}}$ and, consequently, $z = Tu$. Thus, since $z_n \rightarrow z \in \text{Range}(T)$, $\text{Range}(T)$ is closed.

T is onto $\mathcal{V}$. Suppose the converse—let $\mathcal{M} = \text{Range}(T)$ with $\mathcal{M} \subsetneq \mathcal{V}$. From the Hilbert projection theorem, Theorem 8.12, there exists $w \in \mathcal{M}^\perp$ with $w \neq 0$ and $\langle w, \bar{v} \rangle_{\mathcal{V}} = 0$ for all $\bar{v} \in \mathcal{M}$. Moreover, $\langle w, Tu \rangle_{\mathcal{V}} = 0$ for all $u \in \mathcal{V}$. However, for $u = w$,

$$0 = \langle w, Tw \rangle_{\mathcal{V}} = a(w, w) \neq 0, \quad (8.77)$$

which is a contradiction.

Existence of u . Now, the Riesz representation theorem also gives a unique $t_g \in \mathcal{V}$ such that

$$g(v) = \langle v, t_g \rangle_{\mathcal{V}} \quad \text{for all } v \in \mathcal{V}. \quad (8.78)$$

Since $T : \mathcal{V} \rightarrow \mathcal{V}$ is onto, there must exist a $u \in \mathcal{V}$ such that $t_g = Tu$, thus relating the bounded linear functional $g(\cdot)$ of the problem with the linear map, T , associated with the functional induced from the bilinear form $a(\cdot, \cdot)$. That is, $g(v) = \langle v, Tu \rangle_{\mathcal{V}} = a(u, v)$ for all $v \in \mathcal{V}$, showing existence of a $u \in \mathcal{V}$ that satisfies the weak form.

Uniqueness of u . Suppose that u is not unique, and that $t_g = T\hat{u}$ for some $\hat{u}$. Then, $a(u - \hat{u}, v) = 0$ for all $v \in \mathcal{V}$ and $a(u - \hat{u}, u - \hat{u}) = 0$, which shows that $u = \hat{u}$. This concludes the proof. $\square$

The Lax–Milgram theorem is one of the fundamental theorems in the analysis of finite-element methods. To see it in action, consider the weak problem at the start of this section, equation (8.65). To apply Lax–Milgram, we must first show continuity and coercivity of $a(\cdot, \cdot)$. However, coercivity requires one additional step: bounding $a(u, u) = \|\nabla u\|^2$ from below by $\|u\|_1^2$. For this, we introduce the Poincaré–Friedrichs inequality (or Poincaré inequality) in Theorem 8.28, which we use to prove coercivity in Example 8.30.

Theorem 8.28: Poincaré–Friedrichs inequality (cf. [176, Thm. 1.9])

Suppose $\Omega \subset \mathbb{R}^n$ is a bounded domain with Lipschitz boundary $\partial\Omega$ and that $\Gamma_{\mathcal{D}} \subset \partial\Omega$ has positive measure. Assume that $u \in H^1(\Omega)$ and has homogeneous Dirichlet boundary conditions on $\Gamma_{\mathcal{D}}$. Then, there exists a constant $C > 0$ such that

$$\|u\| \leq C \|\nabla u\|. \quad (8.79)$$

Remark 8.29: Proofs of the Poincaré–Friedrichs inequality

Standard proofs of the Poincaré–Friedrichs inequality use abstract completeness [176, Thm. 1.9] or compactness [52, Prop. 5.3.3] arguments to prove that we can define a norm on $H^1(\Omega)$ by $\|u\|_* = \|\nabla u\| + \int_{\Gamma_{\mathcal{D}}} |u| ds$ (or a similar expression), and that this norm is equivalent to the standard norm on $H^1(\Omega)$. Restricting this norm to $\mathcal{V} = \{u \in H^1(\Omega) \mid u = 0 \text{ on } \partial\Omega\}$

gives Theorem 8.28 after manipulating the inequality $\|u\|_1 \leq c\|u\|_*$ for some constant c . If we make the stronger assumption on the problem, that $\Gamma_{\mathcal{D}} = \partial\Omega$, then a simpler proof becomes possible using direct computation, as is presented in [48, Sec. II, Thm. 1.5], when Ω has a piecewise smooth boundary. That proof can be slightly generalized to allow for $\Gamma_{\mathcal{D}}$ to account for only part of $\partial\Omega$, but the technical restrictions on this generalization are difficult to state for general domains.

Example 8.30: Weak form of the Poisson problem

For model problem equation (8.65), we apply the Lax–Milgram theorem to show existence and uniqueness of a solution. To this end, we must show that the bilinear form is continuous and coercive. The linear functional g has already been established to be bounded in equation (8.50) when $f \in L^2(\Omega)$.

Continuity follows directly from the Cauchy–Schwarz inequality on $L^2(\Omega)$,

$$|a(u, v)| = \left| \int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x} \right| \quad (8.80a)$$

$$\leq \left(\int_{\Omega} |\nabla u|^2 \, d\mathbf{x} \right)^{\frac{1}{2}} \left(\int_{\Omega} |\nabla v|^2 \, d\mathbf{x} \right)^{\frac{1}{2}} \quad (8.80b)$$

$$\leq \left(\int_{\Omega} |u|^2 + |\nabla u|^2 \, d\mathbf{x} \right)^{\frac{1}{2}} \left(\int_{\Omega} |v|^2 + |\nabla v|^2 \, d\mathbf{x} \right)^{\frac{1}{2}} \quad (8.80c)$$

$$= c_1 \|u\|_1 \|v\|_1, \quad (8.80d)$$

showing continuity in the $H^1(\Omega)$ norm with $c_1 = 1$.

Coercivity follows using the Poincaré inequality,

$$a(u, u) = \|\nabla u\|^2 = \frac{1}{C^2 + 1} (C^2 \|\nabla u\|^2 + \|\nabla u\|^2) \quad (8.81a)$$

$$\geq \frac{1}{C^2 + 1} (\|u\|^2 + \|\nabla u\|^2), \quad (8.81b)$$

where C is the constant in the Poincaré inequality. With the assumptions of Theorem 8.27 in place, we conclude that the weak problem admits a unique solution in $H_0^1(\Omega)$.

So far, we have established tools that allow us to show whether a given weak form of a PDE is well posed. We revisit this in Section 8.4 for a broader class of operators and boundary conditions. Our next step, however, follows the same path as in Chapter 4, where we examine properties of the weak form restricted to finite-dimensional subspaces, $\mathcal{V}^h$, of $\mathcal{V}$. We recall Definition 4.3, which defines the Ritz–Galerkin approximation as the solution $u^h \in \mathcal{V}^h$ that satisfies

$$a(u^h, v^h) = g(v^h) \quad \forall v^h \in \mathcal{V}^h. \quad (8.82)$$

With a straightforward modification, the proof of Theorem 4.5 also applies to the more general multidimensional setting in this chapter, so long as $g(v)$ is a bounded linear functional on $\mathcal{V}$ and $a(u, v)$ is coercive on $\mathcal{V}$. Specifically, after equation (4.21a), we see that $\beta^T A \beta$ is strictly positive due to the coercivity of $a(\cdot, \cdot)$. As a result, the solution to the Ritz–Galerkin approximation in equation (8.82) exists and is unique. We next examine the quality of the approximation to the solution of the weak form that we can find in the finite-dimensional space $\mathcal{V}^h$.

8.3 ■ Types of elements and approximation properties

Objectives

This section explores the quality of a Ritz–Galerkin approximation of the solution to the weak form of a PDE. You will first consider a generalized form of Céa’s lemma from Section 4.2, reformulated to account for the coercivity and continuity constants introduced in the previous section, that *quantifies* how close the Ritz–Galerkin approximation can be to the “best approximation” over the subspace $\mathcal{V}^h$. Then, you will consider how to choose a “good” finite-element space. You will learn about two common types of finite-element spaces, known as P_k and Q_k , and the main theorems that show that they provide reliable approximations.

So far, we have mainly been interested in understanding the existence and uniqueness properties of the weak form of a PDE, solving for $u \in \mathcal{V}$ such that $a(u, v) = g(v)$ for all $v \in \mathcal{V}$. The next important question that we address is *how accurate to u is an approximation from a finite-dimensional subspace of $\mathcal{V}$?* That is, given a finite-dimensional subspace $\mathcal{V}^h \subset \mathcal{V}$, what are the bounds on the error, $u - u^h$, when $u^h \in \mathcal{V}^h$ satisfies

$$a(u^h, v^h) = g(v^h) \quad \text{for all } v^h \in \mathcal{V}^h. \quad (8.83)$$

As discussed in the previous section, we can easily extend the results from Section 4.2 to establish the existence and uniqueness of this *Ritz–Galerkin approximation*. The next step in Section 4.2 was to establish Céa’s lemma (Lemma 4.7), showing that (for the problem considered there) the Ritz–Galerkin solution has an optimality property: it is the best approximation to u in $\mathcal{V}^h$, when measured in the energy norm. The same idea applies more generally when $a(u, v)$ is coercive and continuous. Thus, we restate (and reprove) Céa’s lemma here in Lemma 8.31 in this more general setting, obtaining a *quasi-optimal* approximation over the space $\mathcal{V}^h \subset \mathcal{V}$. In the next subsections, we focus on developing finite-element spaces for which we quantify the best approximation to make sense of the bound in Céa’s lemma.

Lemma 8.31: Céa’s lemma [52, Thm. 2.8.1]

Let $\mathcal{V} \subset \mathcal{H}$ be a closed subspace of Hilbert space $\mathcal{H}$. Let $a(\cdot, \cdot)$ be a coercive and continuous bilinear form on $\mathcal{V}$. In addition, for a bounded linear functional $g(\cdot)$ on $\mathcal{V}$, let $u \in \mathcal{V}$ satisfy

$$a(u, v) = g(v) \quad \text{for all } v \in \mathcal{V}. \quad (8.84)$$

Consider a finite-dimensional subspace $\mathcal{V}^h \subset \mathcal{V}$ and $u^h \in \mathcal{V}^h$ that satisfies

$$a(u^h, v^h) = g(v^h) \quad \text{for all } v^h \in \mathcal{V}^h. \quad (8.85)$$

Then,

$$\|u - u^h\|_{\mathcal{V}} \leq \frac{c_1}{c_0} \min_{v^h \in \mathcal{V}^h} \|u - v^h\|_{\mathcal{V}}, \quad (8.86)$$

where c_0 and c_1 are the coercivity and continuity constants for $a(\cdot, \cdot)$, respectively.

Proof. Since $v^h \in \mathcal{V}^h \subset \mathcal{V}$, we have that $a(u, v^h) = g(v^h)$ for all $v^h \in \mathcal{V}^h$ from equation (8.84). Then,

$$a(u - u^h, v^h) = a(u, v^h) - a(u^h, v^h) = g(v^h) - g(v^h) = 0 \quad \text{for any } v^h \in \mathcal{V}^h. \quad (8.87)$$

This establishes the orthogonality property as in Lemma 4.6. Thus, for any $v^h \in \mathcal{V}^h$,

$$c_0 \|u - u^h\|_{\mathcal{V}}^2 \leq a(u - u^h, u - u^h) \quad \text{by coercivity} \quad (8.88a)$$

$$= a(u - u^h, u - v^h) + a(u - u^h, v^h - u^h) \quad (8.88b)$$

$$= a(u - u^h, u - v^h) \quad \text{since } v^h - u^h \in \mathcal{V}^h \quad (8.88c)$$

$$\leq c_1 \|u - u^h\|_{\mathcal{V}} \|u - v^h\|_{\mathcal{V}} \quad \text{by continuity.} \quad (8.88d)$$

Dividing by the common term $\|u - u^h\|_{\mathcal{V}}$ completes the proof. $\square$

Céa's lemma, Lemma 8.31, tells us that if $a(\cdot, \cdot)$ is a coercive and continuous bilinear form on a closed subspace, $\mathcal{V}$, of a Hilbert space, $\mathcal{H}$, and $\mathcal{V}^h \subset \mathcal{V}$ is finite-dimensional, then the solution of the weak form, u , and the Ritz–Galerkin approximation, u^h , are related by

$$\|u - u^h\|_{\mathcal{V}} \leq \frac{c_1}{c_0} \min_{v^h \in \mathcal{V}^h} \|u - v^h\|_{\mathcal{V}}, \quad (8.89)$$

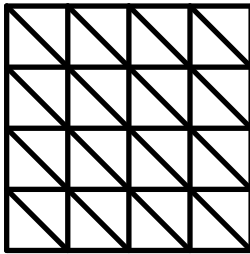
where c_0 and c_1 are the coercivity and continuity constants associated with the bilinear form. Assuming that the ratio c_1/c_0 is not unacceptably large (i.e., it does not increase dramatically for reasonable parameters of the PDE), this leaves us with two important questions to answer:

1. How do we choose $\mathcal{V}^h$?
2. How big is $\min_{v^h \in \mathcal{V}^h} \|u - v^h\|_{\mathcal{V}}$?

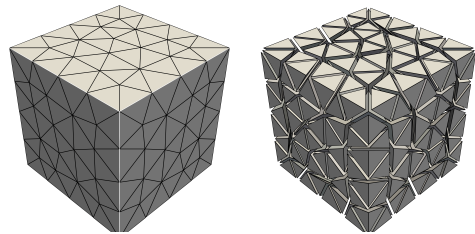
We start with the first question, which inherently addresses the second. We consider some standard types of elements, where the choice of $\mathcal{V}^h$ is split into two stages. First, we choose a *mesh* of the computational domain, Ω . Second, we choose piecewise polynomial approximation spaces over the mesh, imposing continuity or other conditions. We consider two general families of approaches to partitioning a domain into a mesh, based on the shapes of the resulting elements.

8.3.1 ■ Triangular and tetrahedral elements, P_k

While there are many ways to choose a mesh, $\mathcal{T}^h$, of the domain Ω , one common approach is to divide the domain into *simplices*, meaning triangles in two dimensions and tetrahedra in three dimensions—see Figure 8.3.1. This is straightforward if Ω is already polygonal (in 2D) or polyhedral (in 3D), since we can naturally subdivide it into triangles or tetrahedra, and those elements can be further subdivided to make a series of meshes, $\mathcal{T}^h$, with different mesh resolution, h . For nonpolygonal domains (such as the unit disk in 2D) or nonpolyhedral domains



a. 2D triangular mesh.



b. 3D tetrahedral mesh.

Figure 8.3.1. Meshing of domains using simplices. At left, a regular mesh of the unit square using triangles in 2D. At right, an irregular mesh on the unit cube using tetrahedra in 3D.

(such as the unit sphere in 3D), this adds an additional wrinkle, in that the computational domain (see Definition 2.12), Ω^h , only approximates the true domain, Ω . Such a mismatch must be suitably accounted for when imposing boundary conditions and when considering the following approximation results.

A natural way to think of the mesh, $\mathcal{T}^h = \{\tau\}$, is as a set of triangles (or tetrahedra). Once the mesh is chosen, we can set an approximation space by defining representations of functions over each element, τ , in the mesh, as well as rules for how functions on one element relate to those on their neighbors (imposing continuity, for example). For continuous approximations (that we say are *conforming* since continuous functions are in $H^1(\Omega)$), natural choices are the piecewise polynomial spaces

$$P_k(\Omega^h) = \left\{ v \in C^0(\Omega^h) \mid \forall \tau \in \mathcal{T}^h, \exists \{c_{\alpha,\beta}^\tau\}_{(\alpha,\beta) \in \mathcal{I}_k} \text{ such that} \right. \\ \left. v(\mathbf{x}) = \sum_{(\alpha,\beta) \in \mathcal{I}_k} c_{\alpha,\beta}^\tau x^\alpha y^\beta \text{ on } \tau \right\}, \quad (8.90a)$$

$$\text{where } \mathcal{I}_k = \{(\alpha, \beta) \mid \alpha, \beta \in \mathbb{Z}, 0 \leq \alpha, \beta \text{ and } \alpha + \beta \leq k\}. \quad (8.90b)$$

As a result, there are $|\mathcal{I}_k| = \frac{(k+1)(k+2)}{2}$ terms in the polynomial representation over each element.

For example, $P_1(\Omega^h)$ is the space of continuous piecewise linear functions on the mesh, and $P_2(\Omega^h)$ is the space of continuous piecewise quadratic functions, given by

$$P_1(\Omega^h) = \left\{ v \in C^0(\Omega^h) \mid \forall \tau \in \mathcal{T}^h, \exists \{c_{0,0}^\tau, c_{1,0}^\tau, c_{0,1}^\tau\} \text{ such that} \right. \\ \left. v(\mathbf{x}) = c_{0,0}^\tau + c_{1,0}^\tau x + c_{0,1}^\tau y \text{ on } \tau \right\} \quad (8.91)$$

and

$$P_2(\Omega^h) = \left\{ v \in C^0(\Omega^h) \mid \forall \tau \in \mathcal{T}^h, \exists \{c_{0,0}^\tau, c_{1,0}^\tau, c_{0,1}^\tau, c_{1,1}^\tau, c_{2,0}^\tau, c_{0,2}^\tau\} \text{ such that} \right. \\ \left. v(\mathbf{x}) = c_{0,0}^\tau + c_{1,0}^\tau x + c_{0,1}^\tau y + c_{1,1}^\tau xy + c_{2,0}^\tau x^2 + c_{0,2}^\tau y^2 \text{ on } \tau \right\}. \quad (8.92)$$

One of the reasons the spaces in equation (8.90) are natural is because it is easy to define a nodal basis for these finite-element spaces using the nodes of the mesh. For this reason, the spaces considered in this section are often called “Lagrange spaces,” as the nodal basis functions are naturally related to Lagrange polynomials. For $P_1(\Omega^h)$ in two dimensions, each triangle has three nodes, so a linear interpolation of nodal values readily defines the linear function in the triangle. The construction of the basis functions follows naturally, as illustrated in Example 8.32.

Example 8.32: Piecewise linears on one triangle ($P_1(\tau)$)

Take a triangle, τ , with three nodes, (x_1, y_1) , (x_2, y_2) , and (x_3, y_3) , as shown in Figure 8.3.2.

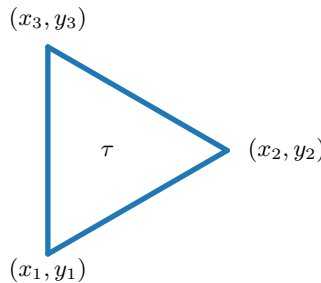


Figure 8.3.2. Labeling of vertices of a triangular element, τ .

We define three nodal basis functions over τ , written as

$$\phi_i(x, y) = a_i + b_i x + c_i y \quad (8.93)$$

for $i = 1, 2, 3$, with the property that $\phi_i(x_j, y_j) = 1$ if $i = j$ and $\phi_i(x_j, y_j) = 0$ if $i \neq j$ for $j = 1, 2, 3$ (note that we have simplified the coefficient notation $c_{\alpha, \beta}^\tau$ of equation (8.90a) to using simply a_i , b_i , and c_i for this example). Writing out equation (8.93), we have, for $i = 1$,

$$\phi_1(x_1, y_1) = a_1 + b_1 x_1 + c_1 y_1 = 1, \quad (8.94a)$$

$$\phi_1(x_2, y_2) = a_1 + b_1 x_2 + c_1 y_2 = 0, \quad (8.94b)$$

$$\phi_1(x_3, y_3) = a_1 + b_1 x_3 + c_1 y_3 = 0, \quad (8.94c)$$

which leads to the linear system to be solved for a_1, b_1, c_1 as

$$\begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{bmatrix} \begin{bmatrix} a_1 \\ b_1 \\ c_1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}. \quad (8.95)$$

We know that the system is uniquely solvable so long as the matrix is nonsingular, which occurs when its determinant is nonzero. From direct calculation,

$$\begin{aligned} \det \begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{bmatrix} &= \det \begin{bmatrix} 1 & x_1 & y_1 \\ 0 & x_2 - x_1 & y_2 - y_1 \\ 0 & x_3 - x_1 & y_3 - y_1 \end{bmatrix} \\ &= (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1). \end{aligned} \quad (8.96)$$

This equals zero whenever two points coincide, or when the three points are collinear, with $\frac{x_2 - x_1}{x_3 - x_1} = \frac{y_2 - y_1}{y_3 - y_1}$. If neither of these happens, we say that the element, τ , is *nondegenerate*, and we solve the system for the coefficients in $\phi_1(x, y)$ to get

$$a_1 = \frac{x_2 y_3 - y_2 x_3}{(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)}, \quad (8.97a)$$

$$b_1 = \frac{y_2 - y_3}{(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)}, \quad (8.97b)$$

$$c_1 = \frac{x_3 - x_2}{(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)}. \quad (8.97c)$$

This gives us

$$\phi_1(x, y) = \frac{(x_2 - x)(y_3 - y) - (x_3 - x)(y_2 - y)}{(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)}. \quad (8.98)$$

Similar calculations lead to the other two basis functions on τ :

$$\phi_2(x, y) = \frac{(y_3 - y_1)(x - x_1) + (x_1 - x_3)(y - y_1)}{(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)}, \quad (8.99a)$$

$$\phi_3(x, y) = \frac{(y_1 - y_2)(x - x_1) + (x_2 - x_1)(y - y_1)}{(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)}. \quad (8.99b)$$

By defining nodal basis functions for each $\tau \in \mathcal{T}^h$, we can use function values at the nodes of the triangular mesh as the degrees of freedom to represent a function in $P_1(\Omega^h)$. A similar calculation can also be done to show that nodal values make suitable degrees of freedom for the piecewise linear approximation on tetrahedra in three dimensions.

An important consequence of forcing each $\phi_i(x, y)$ to be a nodal basis function on each element is that it leads to basis functions that are continuous across element edges. For linear elements, it is easy to see that each $\phi_i(x, y)$ is a linear function going from value one at (x_i, y_i) to zero at (x_j, y_j) for $j \neq i$ along the two edges incident on node i , while it is uniformly zero along the edge between (x_j, y_j) and (x_k, y_k) for $j, k \neq i$. This is true on each triangle. We “assemble” the basis functions associated with each node over all of its adjacent elements into one basis function for $P_1(\Omega^h)$ associated with the node, simply by taking it to be piecewise defined over each of the adjacent triangles. This guarantees that when two triangles, τ_1 and τ_2 , share an edge, the basis functions associated with each of the nodes of the common edge between τ_1 and τ_2 take identical values along the edge, ensuring continuity of any function in $P_1(\Omega^h)$.

For the piecewise quadratic approximation space, $P_2(\Omega^h)$, we add function values at the midpoints of the edges of the elements in 2D to define six degrees of freedom per element, with basis functions of the form

$$\phi_i(x, y) = a_i + b_i x + c_i y + d_i x^2 + e_i xy + f_i y^2 \quad (8.100)$$

on each element, τ . Again defining these to be nodal basis functions, with $\phi_i(x_j, y_j) = 1$ if $i = j$ and $\phi_i(x_j, y_j) = 0$ if $i \neq j$ (now for six nodes: the three corners of τ and the three midpoints of its edges), leads to basis functions that are continuous across element edges when we assemble them over elements that share a node or edge. For higher-degree elements, we continue to add more approximation points to define the degrees of freedom, typically following the patterns associated with high-accuracy Gauss–Lobatto quadrature points on the element. We note that the definition of degrees of freedom for these elements is not unique; while the choice of a specific set of degrees of freedom for the space has no impact on the quality of approximation, it has a large impact on the conditioning of the linear systems that result from discretization. Thus, specific choices of degrees of freedom are often made to lead to favorable properties of the resulting discretization matrices.

Having defined the $P_k(\Omega^h)$ spaces, we now look to quantify the accuracy of finite-element Ritz–Galerkin approximations over such a space. In general, there are three factors that directly impact the quality of approximation: first, the smoothness of the solution, with better approximations being possible when the solution has more continuous derivatives; second, the quality of the mesh, measured in terms of both the size of the elements of the mesh and their shape quality; and third, the polynomial degree of the approximation space itself. In general, the theory describing the quality of Ritz–Galerkin approximations is somewhat complex, relying on properties of all of the above. Thus, in the following we state definitions and theorems to introduce the main ideas, but we omit proofs because they do not add much insight into the performance of the finite-element method. For a more rigorous treatment of these results, see, for example, [52].

First, we consider some definitions for the “quality” of a mesh in Definitions 8.33 and 8.34.

Definition 8.33: Diameter of a set

Given a set, $S \subset \mathbb{R}^n$, the *diameter* of S is given by $\text{diam}(S) = \sup_{x, y \in S} \|x - y\|$.

Definition 8.34: Nondegenerate and quasi-uniform mesh [52, Def. 4.4.13]

Let Ω be given and, for an infinite sequence of h values $\in (0, 1]$ with limit zero, let $\{\mathcal{T}^h\}$ be a family of meshes such that

$$\max_{\tau \in \mathcal{T}^h} \{\text{diam}(\tau)\} = h \text{diam}(\Omega). \quad (8.101)$$

For any $\tau \in \mathcal{T}^h$, let B_τ be the largest ball contained in τ such that for any $x \in \tau$, the closed convex hull of $\{x\} \cup B_\tau$ is contained in τ . We assume that such a ball B_τ exists with radius $r_\tau > 0$ for any $\tau \in \mathcal{T}^h$.

The family of meshes is nondegenerate if $\exists \rho > 0$ such that, $\forall h$ in the sequence and $\forall \tau \in \mathcal{T}^h$,

$$\text{diam}(B_\tau) \geq \rho \text{diam}(\tau). \quad (8.102)$$

The family of meshes is quasi-uniform if $\exists \rho > 0$ such that, $\forall h$ in the sequence,

$$\min_{\tau \in \mathcal{T}^h} \{\text{diam}(B_\tau)\} \geq \rho \max_{\tau \in \mathcal{T}^h} \{\text{diam}(\tau)\}. \quad (8.103)$$

If the family of meshes, $\mathcal{T}^h$, satisfies the criterion to be quasi-uniform, then it automatically satisfies the criterion to be nondegenerate, but the converse does not necessarily hold. Also note that starting from any base triangulation, $\mathcal{T}^{h_0}$, in two dimensions, the family of meshes generated by refining each triangle by connecting its edge midpoints yields a quasi-uniform family of meshes given by $\mathcal{T}^{h_0}, \mathcal{T}^{h_0/2}, \mathcal{T}^{h_0/4}$, etc.

Next, recall that in Section 4.3 we established the notion of an *interpolant* (defined in Definition 4.13). This was then used to give an approximation property for the space of piecewise linear functions (Theorem 4.14). We apply the same idea to the nodal bases that we have defined for $P_k(\Omega^h)$, which gives Theorem 8.35, and then give some examples for linears and quadratics in Example 8.36.

Theorem 8.35: Accuracy of $P_k(\Omega^h)$ [52, Thm. 4.4.20]

Let $\{\mathcal{T}^h\}$ for $0 < h \leq 1$ be a nondegenerate family of simplex meshes of a polyhedral domain, $\Omega \subset \mathbb{R}^n$. Given positive integer k with $k + 1 - n/2 > 0$, let $\mathcal{V}^h = P_k(\Omega^h)$ with a suitable choice of nodes for the degrees of freedom of $P_k(\Omega^h)$. Let I^h be such that $I^h w \in \mathcal{V}^h$ is the interpolant of $w \in C^0(\Omega)$. Then, there exists a constant, C , depending on the choice of nodes, n , k , and the constant ρ in equation (8.102) such that if $u \in H^{k+1}(\Omega)$, then

$$\left(\sum_{\tau \in \mathcal{T}^h} \|u - I^h u\|_s^2 \right)^{\frac{1}{2}} \leq C h^{k+1-s} |u|_{k+1} \quad (8.104)$$

for $0 \leq s \leq k + 1$.

Similar to our results in Chapter 4, Theorem 8.35 gives us a bound that we can use in C  a's lemma (Lemma 8.31) as an upper bound on the best approximation of the solution of the weak form in our finite-element space, $P_k(\Omega^h)$. As we saw in Chapter 4, however, C  a's lemma does not allow us to directly bound the L^2 norm of the error, $\|u - u^h\|$, only the H^1 norm (for H^1 -elliptic bilinear forms). To get L^2 error bounds for H^1 -elliptic problems, we need to extend the Aubin–Nitsche duality theorem, Theorem 4.9, to this more general setting. Similar results, however, do not automatically hold in two (or more) spatial dimensions. To prove the analogous bound, we make use of a property known as *elliptic regularity*, which states that solutions, $u \in \mathcal{V}$, to the weak form $a(u, v) = g(v) = \langle f, v \rangle$ for all $v \in \mathcal{V}$ must satisfy the additional bound that $|u|_2 \leq C\|f\|$. Such a bound is established for many second-order elliptic PDEs with smooth coefficients, but it depends on properties such as the shape of the domain and the boundary conditions imposed. A good reference to see how to prove elliptic regularity is [94].

With this, the idea of Theorem 4.9 is extended by considering the function $\phi \in \mathcal{V}$ such that $a(\phi, v) = \langle u - u^h, v \rangle$ for all $v \in \mathcal{V}$. From this definition of ϕ , we directly apply the elliptic regularity result to show that $|\phi|_2 \leq C\|u - u^h\|$. From equation (8.87), we also have that $a(u - u^h, v^h) = 0$ for all $v^h \in \mathcal{V}^h$, the same discrete space that u^h is taken from. Now, since

$u - u^h \in \mathcal{V}$, we have

$$\|u - u^h\|^2 = \langle u - u^h, u - u^h \rangle = a(\phi, u - u^h) \quad (8.105a)$$

$$= a(u - u^h, \phi) - a(u - u^h, I^h \phi) \quad (8.105b)$$

$$= a(u - u^h, \phi - I^h \phi) \quad (8.105c)$$

$$\leq c_1 \|u - u^h\|_1 \|\phi - I^h \phi\|_1. \quad (8.105d)$$

Note that $I^h \phi$ is in $\mathcal{V}^h$, which we take to be $P_k(\Omega^h)$. While Theorem 8.35 gives us higher-order approximation when applied to a smooth function, all we have assumed for ϕ is that it is in $H^2(\Omega)$, so we apply the theorem with $k = s = 1$, which gives us $\|\phi - I^h \phi\|_1 \leq Ch|\phi|_2$. Using the above elliptic regularity result, we then get that $\|\phi - I^h \phi\|_1 \leq \hat{C}h\|u - u^h\|$. This then gives us the bound that

$$\|u - u^h\|^2 \leq c_1 \hat{C}h \|u - u^h\|_1 \|u - u^h\| \quad (8.106a)$$

$$\Rightarrow \|u - u^h\| \leq \tilde{C}h \|u - u^h\|_1. \quad (8.106b)$$

Then, if $u \in H^{k+1}(\Omega)$, this allows us to gain one order of accuracy for the L^2 norm of the approximation, showing that $\|u - u^h\|$ is $\mathcal{O}(h^{k+1})$.

If we consider $u \in H^2(\Omega)$, then using piecewise linears ($k = 1$), gives

$$\|u - I^h u\| \leq Ch^2 |u|_2, \quad (8.107a)$$

$$\left(\sum_{\tau \in \mathcal{T}^h} \|u - I^h u\|_1^2 \right)^{\frac{1}{2}} = \|u - I^h u\|_1 \leq Ch |u|_2. \quad (8.107b)$$

Here, the so-called *broken* norm, expressed in equation (8.104), becomes the full $\|\cdot\|_1$ norm, since the interpolant $I^h u \in \mathcal{V}^h \subset H^1$.

If we have that $u \in H^2(\Omega)$, then we get the same approximation results for any choice of polynomial degree, as $u \in H^{k+1}(\Omega)$ is a requirement to achieve higher-order accuracy in Theorem 8.35. If we use piecewise quadratic elements ($k = 2$) and have $u \in H^3(\Omega)$, then

$$\|u - I^h u\| \leq Ch^3 |u|_3, \quad (8.108a)$$

$$\|u - I^h u\|_1 \leq Ch^2 |u|_3. \quad (8.108b)$$

Finally, we use the above approximation properties, along with Céa's lemma, to get an a priori bound on the error. For example, for $\mathcal{V} = H^1(\Omega)$ and $\mathcal{V}^h = P_1(\Omega^h)$,

$$\|u - u^h\|_1 \leq \underbrace{\frac{c_1}{c_0} \min_{v^h \in P_1(\Omega^h)} \|u - v^h\|_1}_{\text{Céa's lemma}} \leq \underbrace{Ch |u|_2}_{\text{Theorem 8.35}}, \quad (8.109)$$

where $C > 0$ is a constant independent of h (but dependent on the coercivity and continuity bounds). Similarly, for $P_2(\Omega^h)$,

$$\|u - u^h\|_1 \leq Ch^2 |u|_3. \quad (8.110)$$

An Aubin–Nitsche duality argument, as described above, allows us to get an $L^2(\Omega)$ error bound. Using the fact that the solution is in $H^2(\Omega)$ for $P_1(\Omega^h)$ or $H^3(\Omega)$ for $P_2(\Omega^h)$, and with appropriate boundary conditions, we obtain an extra order of accuracy in $L^2(\Omega)$ just like for the interpolant. In other words,

$$\|u - u^h\| \leq Ch^2 |u|_2 \quad \text{for } P_1(\Omega^h) \text{ elements,} \quad (8.111a)$$

$$\|u - u^h\| \leq Ch^3 |u|_3 \quad \text{for } P_2(\Omega^h) \text{ elements.} \quad (8.111b)$$

Example 8.36: $P_1(\Omega^h)$ and $P_2(\Omega^h)$

To illustrate these results, consider a two-dimensional version of the model problem from equation (8.64):

$$-\nabla \cdot \nabla u = f(x) \text{ for } x \in \Omega = [-1, 1]^2, \quad (8.112a)$$

$$u = 0 \text{ on } \partial\Omega. \quad (8.112b)$$

Choosing a test problem where we set the solution to

$$u(\mathbf{x}) = \sin\left(\pi \frac{x-1}{2}\right) \sin(\pi y) \quad (8.113)$$

yields the forcing function

$$f(\mathbf{x}) = \frac{5}{4}\pi^2 \sin\left(\pi \frac{x-1}{2}\right) \sin(\pi y). \quad (8.114)$$

Note that $u \in H^3(\Omega)$ and $f \in L^2(\Omega)$, thus satisfying all assumptions in the discussion above. In fact, this example has much better regularity than $u \in H^3(\Omega)$ and $f \in L^2(\Omega)$.

Figure 8.3.3 shows the L^2 norms of the error in the finite-element solution of this problem on uniform triangular meshes of the square $[-1, 1]^2$. We see that convergence for linear elements is nearly perfectly second order, as expected. Since the solution is sufficiently smooth, we see improved convergence for quadratic elements up to third order.

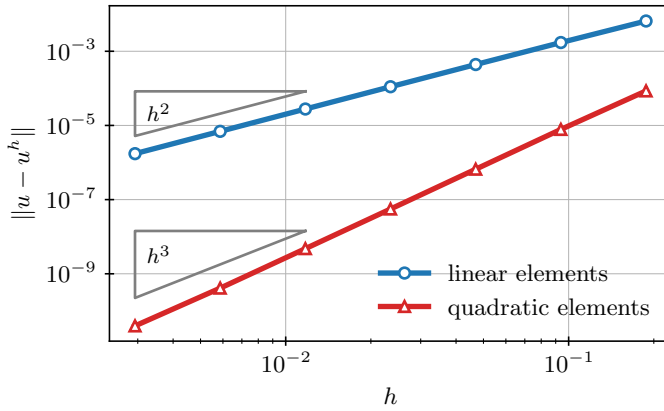
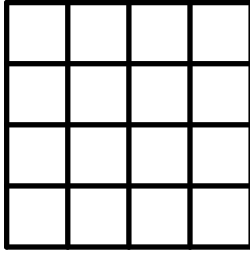


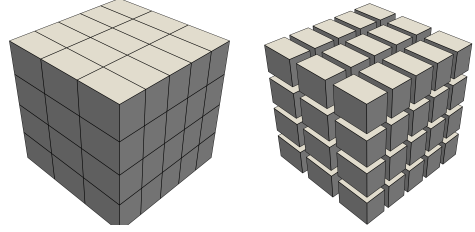
Figure 8.3.3. Finite-element convergence for Example 8.36, measured in the L^2 norm for a smooth solution and piecewise linear and quadratic elements.

8.3.2 ■ Quadrilateral and hexahedral elements, Q_k

Alternatively, we can work with quadrilateral elements in two dimensions and hexahedral elements in three dimensions, as shown in Figure 8.3.4. Here, the basic mechanics of the finite-element method are the same, but the use of nodal basis functions requires a larger function space on each element than in the case of simplices. For example, for $k = 1$, matching values at the four nodes of a quadrilateral (and eight nodes of a hexahedron) leads to the coefficients of a linear function being overdetermined. Again, for a given domain, it may be impossible to exactly mesh the domain using quadrilateral or hexahedral elements, and some approximation



a. 2D quadrilateral mesh



b. 3D hexahedral mesh

Figure 8.3.4. Meshing of domains using quadrilateral and hexahedral elements. At left, a regular mesh of the unit square using rectangles in 2D. At right, a regular mesh approximating the unit cube using hexahedra in 3D.

may be necessary when Ω is not a simple polygon or polyhedron. However, using these elements is somewhat more natural for problems posed on square or rectangular (or prismatic) domains.

The approximation spaces that we use on these elements are most naturally built up as tensor products of lower-dimensional elements. In two dimensions, we formally define the approximation spaces as

$$Q_k(\Omega^h) = \left\{ v \in C^0(\Omega^h) \mid \forall \tau \in \mathcal{T}^h, \exists \{c_{\alpha,\beta}^\tau\}_{(\alpha,\beta) \in \mathcal{I}_k} \text{ such that} \right. \\ \left. v(\mathbf{x}) = \sum_{(\alpha,\beta) \in \mathcal{I}_k} c_{\alpha,\beta}^\tau x^\alpha y^\beta \text{ on } \tau \right\}, \quad (8.115a)$$

$$\text{where } \mathcal{I}_k = \{(\alpha, \beta) \mid \alpha, \beta \in \mathbb{Z}, 0 \leq \alpha, \beta \text{ and } \alpha, \beta \leq k\}. \quad (8.115b)$$

The definition extends similarly to $Q_k(\Omega^h)$ for $\Omega \subset \mathbb{R}^3$, where the terms are of the form $x^\alpha y^\beta z^\gamma$ with $\alpha, \beta, \gamma \leq k$. Since the space is constructed by tensor products of one-dimensional spaces, “extra terms” arise in comparison with $P_k(\Omega^h)$; indeed, there are $|\mathcal{I}_k| = (k+1)^2$ terms in the polynomial over each element for the case of Q_k , whereas P_k has fewer with $|\mathcal{I}_k| = \frac{(k+1)(k+2)}{2}$. For example, when $k = 1$, the product term xy is added to the approximation on each element in addition to the terms present in $P_1(\Omega^h)$.

In practice, $Q_1(\Omega^h)$ is referred to as the space of piecewise *bilinears* and $Q_2(\Omega^h)$ as *bi-quadratics*. These are given by

$$Q_1(\Omega^h) = \left\{ v \in C^0(\Omega^h) \mid \forall \tau \in \mathcal{T}^h, \exists \{c_{0,0}^\tau, c_{1,0}^\tau, c_{0,1}^\tau, c_{1,1}^\tau\} \text{ such that} \right. \\ \left. v(\mathbf{x}) = c_{0,0}^\tau + c_{1,0}^\tau x + c_{0,1}^\tau y + c_{1,1}^\tau xy \text{ on } \tau \right\} \quad (8.116)$$

and

$$Q_2(\Omega^h) = \left\{ v \in C^0(\Omega^h) \mid \forall \tau \in \mathcal{T}^h, \exists \{c_{0,0}^\tau, c_{1,0}^\tau, c_{0,1}^\tau, c_{1,1}^\tau, c_{2,0}^\tau, c_{0,2}^\tau, c_{2,1}^\tau, c_{1,2}^\tau, c_{2,2}^\tau\} \text{ such that} \right. \\ \left. v(\mathbf{x}) = c_{0,0}^\tau + c_{1,0}^\tau x + c_{0,1}^\tau y + c_{1,1}^\tau xy + c_{2,0}^\tau x^2 + c_{0,2}^\tau y^2 \right. \\ \left. + c_{2,1}^\tau x^2 y + c_{1,2}^\tau x y^2 + c_{2,2}^\tau x^2 y^2 \text{ on } \tau \right\}. \quad (8.117)$$

As could be expected, since the approximation space has changed, the approximation properties also change, as expressed in Theorem 8.37. However, the same approach as above applies.

Using the approximation property, we have a bound on the error of a function approximated by its interpolant. Then, Céa's lemma tells us that the error between the exact solution and the finite-element solution is better than that interpolation error. Thus, we obtain an a priori error bound for our finite-element method using $Q_k(\Omega^h)$.

Theorem 8.37: Accuracy of $Q_k(\Omega^h)$ [52, Thm. 4.6.14]

Let $\{\mathcal{T}^h\}$ for $0 < h \leq 1$ be a nondegenerate family of quadrilateral/hexahedral meshes of a polyhedral domain, $\Omega \subset \mathbb{R}^n$. Given positive integer k with $k + 1 - n/2 > 0$, let $\mathcal{V}^h = Q_k(\Omega^h)$, with a suitable choice of nodes for the degrees of freedom of $Q_k(\Omega^h)$. Let I^h be such that $I^h w \in \mathcal{V}^h$ is the interpolant of $w \in C^0(\Omega)$. Then, there exists a constant, C , depending on the choice of nodes, n , k , and the constant ρ in equation (8.102) such that if $u \in H^\ell(\Omega)$, then

$$\|u - I^h u\| \leq Ch^{\min(k+1, \ell)} \left(\sum_{i=1}^n \left\| \frac{\partial^\ell u}{\partial x_i} \right\|^2 \right)^{\frac{1}{2}}, \quad (8.118a)$$

$$\|u - I^h u\|_1 \leq Ch^{\min(k+1, \ell)-1} \left(\sum_{i=1}^n \left\| \frac{\partial^\ell u}{\partial x_i} \right\|^2 \right)^{\frac{1}{2}}. \quad (8.118b)$$

If we consider $u \in H^2(\Omega)$, then, using $Q_1(\Omega^h)$,

$$\|u - I^h u\| \leq Ch^2 |u|_2, \quad (8.119a)$$

$$\|u - I^h u\|_1 \leq Ch |u|_2. \quad (8.119b)$$

With Céa's lemma and an Aubin–Nitsche duality argument, this yields the error bounds

$$\|u - u^h\| \leq Ch^2 |u|_2, \quad (8.120a)$$

$$\|u - u^h\|_1 \leq Ch |u|_2. \quad (8.120b)$$

Note that Theorem 8.37 does not use the seminorm on the right-hand side of the interpolation error bounds. However,

$$\left(\sum_{i=1}^n \left\| \frac{\partial^\ell u}{\partial x_i} \right\|^2 \right)^{\frac{1}{2}} \leq |u|_\ell,$$

leading to the looser bound.

If we use piecewise biquadratic elements ($Q_2(\Omega^h)$) and have $u \in H^3(\Omega)$, then

$$\|u - I^h u\| \leq Ch^3 |u|_3, \quad (8.121a)$$

$$\|u - I^h u\|_1 \leq Ch^2 |u|_3, \quad (8.121b)$$

and we have error bounds

$$\|u - u^h\| \leq Ch^3 |u|_3, \quad (8.122a)$$

$$\|u - u^h\|_1 \leq Ch^2 |u|_3. \quad (8.122b)$$

Similar results as in Figure 8.3.3 are seen for the test problem in Example 8.3.6 using quadrilateral elements, so we omit them here.

In the end, no matter the finite-dimensional space chosen, the story is the same: find an approximation property for the space, often using some kind of interpolant, and then take advantage of Céa's lemma to get a bound on the error. Choosing the “right” space is then about how good this approximation can be. Of course, ease of implementation can also be a deciding factor, which we address in Section 8.5.

8.4 ■ More general problems

Objectives

While the general theory proposed in Section 8.2 represents the main tools used in standard finite-element analysis (particularly for prototypical elliptic PDEs), we have yet to explore the limits of this theory. Here, our goal is to put this theory to use in more general settings and to see where limitations may arise. Specifically, in this section you will analyze a general model problem that leads to a nonsymmetric bilinear form and define the notion of weak coercivity to discuss how it yields well-posedness for some problems. Finally, you will consider the effects of boundary conditions on the analysis.

The preceding sections show how important the analytical tools of showing continuity and coercivity are to finite-element analysis, since our error bounds are determined by the constants arising in Céa's lemma and by the approximation properties of our discrete space, $\mathcal{V}^h$. So far, however, we have only shown coercivity and continuity for two problems:

$$\begin{cases} -\nabla \cdot \nabla u + u = f & \text{in } \Omega, \\ u = 0 & \text{on } \partial\Omega, \end{cases} \quad (8.123)$$

and

$$\begin{cases} -\nabla \cdot \nabla u = f & \text{in } \Omega, \\ u = 0 & \text{on } \partial\Omega. \end{cases} \quad (8.124)$$

In this section, we look to generalize both the PDE itself and the boundary conditions that we impose.

8.4.1 ■ Nonsymmetric bilinear forms

We now consider a much more general reaction-convection-diffusion equation on a domain $\Omega \subset \mathbb{R}^n$ as a model problem. We let $K(\mathbf{x})$ be a bounded, symmetric, and uniformly positive definite $n \times n$ matrix satisfying

$$0 < \lambda_0 \leq \frac{\mathbf{v}^T K(\mathbf{x}) \mathbf{v}}{\mathbf{v}^T \mathbf{v}} \leq \lambda_1 \text{ for all } \mathbf{v} \in \mathbb{R}^n \text{ and } \mathbf{x} \in \Omega, \quad (8.125)$$

where we emphasize that the constants, λ_0 and λ_1 , are independent of $\mathbf{x}$. Further, we let $\mathbf{b}(\mathbf{x})$ and $c(\mathbf{x})$ be bounded functions. Then, we consider the PDE

$$-\nabla \cdot K(\mathbf{x}) \nabla u + \mathbf{b}(\mathbf{x}) \cdot \nabla u + c(\mathbf{x})u = f(\mathbf{x}) \text{ for all } \mathbf{x} \in \Omega, \quad (8.126)$$

supplemented with homogeneous Dirichlet boundary conditions, $u(\mathbf{x}) = 0$ for $\mathbf{x} \in \partial\Omega$. We note that simpler families of PDEs arise by considering simplifications of the above, for example, by taking $K(\mathbf{x}) = k(\mathbf{x})I$, to have a (positive) scalar diffusion coefficient (in which case equation (8.125) translates to requiring that $0 < \lambda_0 \leq k(\mathbf{x}) \leq \lambda_1$ for all $\mathbf{x} \in \Omega$), and by allowing any of $K(\mathbf{x})$, $\mathbf{b}(\mathbf{x})$, or $c(\mathbf{x})$ to take on constant values. Multiplying by a test function, $v(\mathbf{x})$, and integrating by parts, we get the weak form of finding $u \in H_0^1(\Omega)$ such that

$$\langle K(\mathbf{x}) \nabla u, \nabla v \rangle + \langle \mathbf{b}(\mathbf{x}) \cdot \nabla u, v \rangle + \langle c(\mathbf{x})u, v \rangle = \langle f, v \rangle \text{ for all } v \in H_0^1(\Omega). \quad (8.127)$$

Thus, to apply the theory above, we want to show that the bilinear form

$$a(u, v) = \langle K(\mathbf{x}) \nabla u, \nabla v \rangle + \langle \mathbf{b}(\mathbf{x}) \cdot \nabla u, v \rangle + \langle c(\mathbf{x})u, v \rangle \quad (8.128)$$

is coercive and continuous on $H_0^1(\Omega)$.

We get continuity by bounding this term by term. For technical reasons, it is easiest to do this by rewriting $\langle K \nabla u, \nabla v \rangle = \langle K^{1/2} \nabla u, K^{1/2} \nabla v \rangle$, where we implicitly define the symmetric and positive-definite matrix square root pointwise, as the unique symmetric matrix that satisfies $(K^{1/2})^T K^{1/2} = K$. The other inequalities used below come from first applying the Cauchy–Schwarz inequality to the inner product in question and then using properties of the integral form of the norms to pull out terms that bound the effects of $K(\mathbf{x})$, $\mathbf{b}(\mathbf{x})$, and $c(\mathbf{x})$. For the first term in $a(u, v)$, we have

$$|\langle K(\mathbf{x}) \nabla u, \nabla v \rangle| \leq \|K^{1/2} \nabla u\| \cdot \|K^{1/2} \nabla v\| \leq \lambda_1 \|\nabla u\| \cdot \|\nabla v\| \quad (8.129a)$$

$$\leq \lambda_1 \|u\|_1 \|v\|_1, \quad (8.129b)$$

where the final inequality in equation (8.129a) comes from the bound that

$$\|K^{1/2} \nabla u\|^2 = \int_{\Omega} (\nabla u)^T K \nabla u \, d\mathbf{x} \leq \lambda_1 \int_{\Omega} (\nabla u)^T \nabla u \, d\mathbf{x} = \lambda_1 \|\nabla u\|^2. \quad (8.130)$$

Next, we have

$$|\langle \mathbf{b}(\mathbf{x}) \cdot \nabla u, v \rangle| \leq \|\mathbf{b}(\mathbf{x}) \cdot \nabla u\| \|v\|, \quad (8.131)$$

where we bound

$$\|\mathbf{b}(\mathbf{x}) \cdot \nabla u\|^2 = \int_{\Omega} (\nabla u)^T \mathbf{b} \mathbf{b}^T \nabla u \, d\mathbf{x} \quad (8.132a)$$

$$\leq \left(\max_{\mathbf{x} \in \Omega} \lambda_{\max}(\mathbf{b}(\mathbf{x}) \mathbf{b}^T(\mathbf{x})) \right) \int_{\Omega} (\nabla u)^T \nabla u \, d\mathbf{x} \quad (8.132b)$$

$$\leq \left(\max_{\mathbf{x} \in \Omega} \|\mathbf{b}(\mathbf{x})\|^2 \right) \int_{\Omega} (\nabla u)^T \nabla u \, d\mathbf{x}, \quad (8.132c)$$

where the largest eigenvalue of $\mathbf{b} \mathbf{b}^T$ satisfies $\lambda_{\max}(\mathbf{b} \mathbf{b}^T) = \|\mathbf{b}\|^2$ since $\mathbf{b} \mathbf{b}^T$ is a symmetric rank-one matrix whose only nonzero eigenvalue is $\|\mathbf{b}\|^2$. Taking $\max_{\mathbf{x} \in \Omega} \|\mathbf{b}(\mathbf{x})\| = \|\mathbf{b}\|_{\infty}$, we have

$$|\langle \mathbf{b}(\mathbf{x}) \cdot \nabla u, v \rangle| \leq \|\mathbf{b}(\mathbf{x}) \cdot \nabla u\| \|v\| \leq \|\mathbf{b}\|_{\infty} \|\nabla u\| \|v\| \quad (8.133a)$$

$$\leq \|\mathbf{b}\|_{\infty} \|u\|_1 \|v\|_1. \quad (8.133b)$$

Similarly, we have

$$|\langle c(\mathbf{x}) u, v \rangle| \leq \|c(\mathbf{x}) u\| \|v\| \leq \|c\|_{\infty} \|u\| \|v\| \quad (8.134a)$$

$$\leq \|c\|_{\infty} \|u\|_1 \|v\|_1. \quad (8.134b)$$

Taken together, these give us the continuity bound

$$|a(u, v)| \leq (\lambda_1 + \|\mathbf{b}\|_{\infty} + \|c\|_{\infty}) \|u\|_1 \|v\|_1. \quad (8.135)$$

For coercivity, we want to find a constant c_0 such that $|a(u, u)| \geq c_0 \|u\|_1^2$. From the lower bound on $K(\mathbf{x})$, we write

$$\langle K(\mathbf{x}) \nabla u, \nabla u \rangle \geq \lambda_0 \|\nabla u\|^2. \quad (8.136)$$

To treat the convection and reaction terms, we make use of a common trick to rewrite the convection term in “divergence form.” From the product rule, we have

$$\nabla \cdot (\mathbf{b}(\mathbf{x}) u) = (\nabla \cdot \mathbf{b}(\mathbf{x})) u + \mathbf{b}(\mathbf{x}) \cdot \nabla u. \quad (8.137)$$

Thus, we have the nonobvious identity that

$$\mathbf{b}(\mathbf{x}) \cdot \nabla u = \frac{1}{2} (\mathbf{b}(\mathbf{x}) \cdot \nabla u + \nabla \cdot (\mathbf{b}(\mathbf{x})u) - (\nabla \cdot \mathbf{b}(\mathbf{x})) u). \quad (8.138)$$

This, in turn, gives us

$$\langle \mathbf{b} \cdot \nabla u, u \rangle + \langle cu, u \rangle = \frac{1}{2} (\langle \mathbf{b} \cdot \nabla u, u \rangle + \langle \nabla \cdot (\mathbf{b}u), u \rangle) + \left\langle \left(c - \frac{1}{2} \nabla \cdot \mathbf{b} \right) u, u \right\rangle, \quad (8.139)$$

where we have dropped the dependence on $\mathbf{x}$ notation for simplicity. The trick is completed by computing

$$\langle \mathbf{b} \cdot \nabla u, u \rangle = \int_{\Omega} (\mathbf{b} \cdot \nabla u) u \, d\mathbf{x} = \int_{\Omega} (\nabla u) \cdot (\mathbf{b}u) \, d\mathbf{x} = - \int_{\Omega} u (\nabla \cdot (\mathbf{b}u)) \, d\mathbf{x} = - \langle \nabla \cdot (\mathbf{b}u), u \rangle, \quad (8.140)$$

where the boundary term from the divergence theorem vanishes due to the boundary conditions. Thus, two terms cancel in the above and we get

$$\langle \mathbf{b}(\mathbf{x}) \cdot \nabla u, u \rangle + \langle c(\mathbf{x})u, u \rangle = \left\langle \left(c(\mathbf{x}) - \frac{1}{2} \nabla \cdot \mathbf{b}(\mathbf{x}) \right) u, u \right\rangle. \quad (8.141)$$

We bound this from below by making the additional assumption that

$$C = \inf_{\mathbf{x}} \left(c(\mathbf{x}) - \frac{1}{2} \nabla \cdot \mathbf{b}(\mathbf{x}) \right) > 0. \quad (8.142)$$

It is important to recognize that this is the same assumption used in PDE analysis to prove existence and uniqueness of the solution to the equation (see [191, Ch. III.1] and references therein). With this, we then have

$$a(u, u) \geq \lambda_0 \|\nabla u\|^2 + C \|u\|^2 \geq c_0 \|u\|_1^2 \quad (8.143)$$

for $c_0 = \min(\lambda_0, C)$. In addition, if the infimum defining C is equal to zero, then we can also use the Poincaré–Friedrichs inequality to establish coercivity, as is done in earlier sections.

A natural question that arises is what to do when

$$\inf_{\mathbf{x}} \left(c(\mathbf{x}) - \frac{1}{2} \nabla \cdot \mathbf{b}(\mathbf{x}) \right) < 0. \quad (8.144)$$

Here, our ability to “fix” the problem with the coercivity proof is determined by the size of a suitably chosen Poincaré–Friedrichs inequality constant. However, since that may involve many factors, such as the diameter of the domain, we instead turn to different tools. A common approach is to use a generalized form of the existence and uniqueness theorem with a weaker form of coercivity. We say that a bilinear form, $a(\cdot, \cdot)$, is weakly coercive on $\mathcal{V}$ if there exists a constant $c_0 > 0$ such that

$$\inf_{u \in \mathcal{V}} \sup_{v \in \mathcal{V}} \frac{a(u, v)}{\|u\|_{\mathcal{V}} \cdot \|v\|_{\mathcal{V}}} \geq c_0. \quad (8.145)$$

A generalization of the Lax–Milgram theorem is then used, which states that if $a(u, v)$ is both continuous and weakly coercive, then we get unique solutions to the problem in the continuum (over $\mathcal{V}$). Note, however, that the condition holding on $\mathcal{V}$ is not sufficient to prove that it holds on subspaces (in contrast to classical coercivity), so the condition in equation (8.145) must be separately checked for each subspace, $\mathcal{V}^h \subset \mathcal{V}$, in a Ritz–Galerkin approximation. Related to

this are the uses of *Petrov–Galerkin* formulations, where we solve weak forms such as finding $u \in \mathcal{U}^h$ such that

$$a(u, v) = L(v) \text{ for all } v \in \mathcal{V}^h, \quad (8.146)$$

purposely choosing $\mathcal{U}^h \neq \mathcal{V}^h$ in order to get solvability at the discrete level.

Indeed, the challenge of finding spaces over which we can prove continuity and (weak) coercivity of the resulting bilinear form has driven a substantial amount of research into finite-element methods, particularly for problems with multiple solution components or higher-order derivative terms. We revisit this in Chapter 10, where we consider what to do for problems such as incompressible fluid mechanics and elasticity, as well as electromagnetic problems.

8.4.2 ■ Nonhomogeneous Dirichlet boundary conditions

To this point, we have exclusively considered the case of homogeneous Dirichlet boundary conditions, where $u(\mathbf{x}) = 0$ for $\mathbf{x} \in \partial\Omega$, allowing us to work in the space $H_0^1(\Omega)$. While this might seem at first to be a major limitation on finite-element methods, we now show how to transform nonhomogeneous Dirichlet conditions into homogeneous ones, using the same technique of superposition of solutions that is common when considering Fourier series solutions. In practice, this transformation is quite standard, so it is rarely even discussed in research papers on the finite-element method, but it is a critical step that can change the smoothness of the resulting solutions, which can greatly impact convergence rates.

Consider a PDE that, with homogeneous Dirichlet boundary conditions, yields the weak form of finding $u \in \mathcal{V} = H_0^1(\Omega)$ such that $a(u, v) = g(v)$ for all $v \in \mathcal{V}$. We now seek to replace that homogeneous condition with the nonhomogeneous boundary condition $u(\mathbf{x}) = q(\mathbf{x})$ for $\mathbf{x} \in \partial\Omega$, where we consider $q(\mathbf{x})$ to be known only on $\partial\Omega$. This implies that $u \in H_q^1(\Omega)$, as defined in equation (8.38). However, as noted there, we cannot apply our theory to functions in $H_q^1(\Omega)$ since it is not a Hilbert space. Instead, we write $u(\mathbf{x}) = \hat{u}(\mathbf{x}) + \hat{q}(\mathbf{x})$ for some functions $\hat{u} \in H_0^1(\Omega)$ and $\hat{q} \in H_q^1(\Omega)$, where $\hat{q}(\mathbf{x})$ is an extension of $q(\mathbf{x})$ to the entire domain. It can be shown that such a function always exists if $q(\mathbf{x})$ and the domain, Ω , satisfy certain assumptions, and we show how to construct one such function in Section 8.5.2. Assuming that we have obtained such a function $\hat{q}(\mathbf{x})$, we note that $u(\mathbf{x})$ satisfies the desired boundary condition if $\hat{u}(\mathbf{x})$ satisfies a homogeneous Dirichlet boundary condition on $\partial\Omega$. Thus, substituting $u = \hat{u} + \hat{q}$ into the weak form, we have

$$a(\hat{u} + \hat{q}, v) = g(v) \quad \text{for all } v \in \mathcal{V} = H_0^1(\Omega) \text{ or} \quad (8.147a)$$

$$a(\hat{u}, v) = g(v) - a(\hat{q}, v) \quad \text{for all } v \in \mathcal{V}. \quad (8.147b)$$

Note that this relies on the fact that the steps in Section 8.2 that brought us from the strong form in equation (8.43) to the weak form in equation (8.45) remain valid for $u \in H_q^1(\Omega)$. This follows since the steps rely only on the fact that $v \in H_0^1(\Omega)$, which is still the case here.

It is easy to see that if $a(\cdot, \cdot)$ is continuous, then $g(v) - a(\hat{q}, v)$ is a bounded linear functional on $\mathcal{V}$. Thus, if $a(\cdot, \cdot)$ is continuous and coercive, the problem of finding $\hat{u} \in \mathcal{V} = H_0^1(\Omega)$ such that equation (8.147b) is satisfied has a unique solution, and the solution of the nonhomogeneous problem is given by $u(\mathbf{x}) = \hat{u}(\mathbf{x}) + \hat{q}(\mathbf{x})$. A common construction for $\hat{q}(\mathbf{x})$ is to extend $q(\mathbf{x})$ to decay to zero at some short distance from $\partial\Omega$, so that $\hat{q}(\mathbf{x})$ is nonzero only in some neighborhood of the boundary. Again, this may not be possible depending on the shape of the domain, Ω , and the prescribed values of $q(\mathbf{x})$. Nevertheless, such a construction has a natural discrete analogue that we show in detail in Section 8.5, where we construct $\hat{q}^h \in \mathcal{V}^h$ to equal the given Dirichlet boundary data $q(\mathbf{x})$ at any node on the mesh that lies on the Dirichlet boundary and to take on value zero for all nodes not on the Dirichlet boundary. This construction gives a $\hat{q}^h(\mathbf{x})$

that satisfies the desired boundary values in all Dirichlet nodes and is also straightforward to implement.

The quality of a finite-element approximation depends on the smoothness of the solution. When u satisfies homogeneous Dirichlet boundary conditions, we typically infer its smoothness from that of the right-hand side and properties of the domain, using classical techniques from analysis. With nonhomogeneous Dirichlet boundary conditions, the smoothness of $\hat{u}$ depends now on properties of both the original right-hand side of the PDE and the extension of the nonhomogeneous boundary data, $\hat{q}$. Thus, the assumptions required to show smoothness of the solution are much more complicated, even on simple domains. As examples, we refer to the general theory in Grisvard's book [113] or the paper by Han and Kellogg [123] that summarizes suitable *compatibility conditions* on the boundary data and right-hand side function for second-order elliptic equations on the unit square.

8.4.3 ■ Non-Dirichlet boundary conditions

Next, we return to our generalized model problem, equation (8.126). We consider the case where $\partial\Omega$ can be partitioned into two subdomains, $\partial\Omega = \Gamma_{\mathcal{D}} \cup \Gamma_{\mathcal{R}}$, where $\Gamma_{\mathcal{D}} \cap \Gamma_{\mathcal{R}} = \emptyset$, and we impose Dirichlet boundary conditions on $\Gamma_{\mathcal{D}}$ and Robin boundary conditions on $\Gamma_{\mathcal{R}}$, requiring

$$\begin{cases} u(\mathbf{x}) = q_{\mathcal{D}}(\mathbf{x}) & \text{for } \mathbf{x} \in \Gamma_{\mathcal{D}}, \\ \mathbf{n} \cdot (K(\mathbf{x})\nabla u(\mathbf{x})) + \alpha u(\mathbf{x}) = q_{\mathcal{R}}(\mathbf{x}) & \text{for } \mathbf{x} \in \Gamma_{\mathcal{R}}, \end{cases} \quad (8.148)$$

where $\mathbf{n}$ is the outward unit normal vector on $\Gamma_{\mathcal{R}}$ and $\alpha \geq 0$ is a given parameter. In general, either $\Gamma_{\mathcal{D}}$ or $\Gamma_{\mathcal{R}}$ could be the empty set, but we focus on the case here where $\Gamma_{\mathcal{R}} \neq \emptyset$, as we have extensively considered the alternative up to this point. When $\alpha = 0$, we have the simpler situation of Neumann boundary conditions, so we do not treat this case separately.

We deal with the nonhomogeneous Dirichlet condition on $\Gamma_{\mathcal{D}}$ in the same way as in Section 8.4.2 by first extending $q_{\mathcal{D}}(\mathbf{x})$ to a function $\hat{q}_{\mathcal{D}} \in H^1(\Omega)$ and solving for $\hat{u} = u - \hat{q}_{\mathcal{D}}$, which satisfies homogeneous Dirichlet boundary conditions on $\Gamma_{\mathcal{D}}$. Using the problem-specific definition of $H_0^1(\Omega)$ from equation (8.38),

$$H_0^1(\Omega) = \{v \in H^1(\Omega) \mid v(\mathbf{x}) = 0 \text{ for } \mathbf{x} \in \Gamma_{\mathcal{D}}\}, \quad (8.149)$$

we can solve for $\hat{u} = u - \hat{q}_{\mathcal{D}} \in \mathcal{V} = H_0^1(\Omega)$ with test functions $v \in \mathcal{V}$ used in the weak form, as in equation (8.147b). Since the focus in this section is on the formulation with non-Dirichlet boundary conditions, we assume for simplicity that $q_{\mathcal{D}} = 0$ on $\Gamma_{\mathcal{D}}$, so $\hat{u} = u$, in the remainder of this section.

The key difference in deriving the weak form in the case of non-Dirichlet boundary conditions comes when we apply integration by parts to the second-order term in the strong form of the PDE, where the boundary integral no longer vanishes on the non-Dirichlet part of the boundary:

$$\int_{\Omega} -\nabla \cdot (K(\mathbf{x})\nabla u) v \, d\mathbf{x} = \int_{\Omega} (K(\mathbf{x})\nabla u) \cdot \nabla v \, d\mathbf{x} - \int_{\partial\Omega} v (K(\mathbf{x})\nabla u) \cdot \mathbf{n} \, ds \quad (8.150a)$$

$$\begin{aligned} &= \int_{\Omega} (K(\mathbf{x})\nabla u) \cdot \nabla v \, d\mathbf{x} - \int_{\Gamma_{\mathcal{D}}} v (K(\mathbf{x})\nabla u) \cdot \mathbf{n} \, ds \\ &\quad - \int_{\Gamma_{\mathcal{R}}} v (K(\mathbf{x})\nabla u) \cdot \mathbf{n} \, ds. \end{aligned} \quad (8.150b)$$

Since $v \in H_0^1(\Omega)$, the integral over $\Gamma_{\mathcal{D}}$ is zero. On $\Gamma_{\mathcal{R}}$, we rewrite our boundary condition as $(K(\mathbf{x})\nabla u) \cdot \mathbf{n} = q_{\mathcal{R}} - \alpha u$, yielding

$$- \int_{\Gamma_{\mathcal{R}}} v (K(\mathbf{x})\nabla u) \cdot \mathbf{n} \, ds = \alpha \int_{\Gamma_{\mathcal{R}}} uv \, ds - \int_{\Gamma_{\mathcal{R}}} q_{\mathcal{R}} v \, ds. \quad (8.151)$$

This gives

$$\int_{\Omega} -\nabla \cdot (K(\mathbf{x}) \nabla u) v \, d\mathbf{x} = \int_{\Omega} (K(\mathbf{x}) \nabla u) \cdot \nabla v \, d\mathbf{x} + \alpha \int_{\Gamma_{\mathcal{R}}} uv \, ds - \int_{\Gamma_{\mathcal{R}}} q_{\mathcal{R}} v \, ds. \quad (8.152)$$

Note that the boundary contribution here splits into two pieces. The first, $\alpha \int_{\Gamma_{\mathcal{R}}} uv \, ds$, is a bilinear form that must be included in $a(u, v)$. Here, the assumption that $\alpha \geq 0$ is important, since it results in a positive-semidefinite term that does not hurt any coercivity bound on the bilinear form. The second piece, $\int_{\Gamma_{\mathcal{R}}} q_{\mathcal{R}} v \, ds$, is a linear form in v and, as such, gets included in the linear form $g(v)$. The weak form is then to find $u \in \mathcal{V} = H_0^1(\Omega)$ such that

$$a(u, v) = g(v) \text{ for all } v \in \mathcal{V}, \quad (8.153)$$

with

$$a(u, v) = \langle K(\mathbf{x}) \nabla u, \nabla v \rangle + \langle \mathbf{b}(\mathbf{x}) \cdot \nabla u, v \rangle + \langle c(\mathbf{x}) u, v \rangle + \alpha \int_{\Gamma_{\mathcal{R}}} uv \, ds, \quad (8.154a)$$

$$g(v) = \langle f, v \rangle + \int_{\Gamma_{\mathcal{R}}} q_{\mathcal{R}} v \, ds. \quad (8.154b)$$

Continuity bounds on the new forms of $a(u, v)$ and $g(v)$ can still be shown, depending on properties of the domain, Ω , and the forcing function, $q_{\mathcal{R}}(\mathbf{x})$, as well as certain *trace inequalities* that relate the norm of a function on $\partial\Omega$ to its Sobolev norm(s) over Ω . In this setting, we may still want to use the Poincaré–Friedrichs inequality to prove coercivity. It is important to note that the detailed conditions on Theorem 8.28 are important here, since we no longer have that $\partial\Omega = \Gamma_{\mathcal{D}}$.

8.5 ■ Finite-element assembly

Objectives

In 1D, computing the finite-element approximation to the weak solution for a model problem by assembling the local matrices into a global matrix is relatively straightforward (see Section 4.4). The real power of the finite-element method, however, is that a variety of basis functions may be used on both structured and unstructured domains in two or more dimensions. In this section, you will consider an unstructured 2D mesh and investigate the assembly process. In particular, you will use an elementwise construction to compute the linear system corresponding to the discretized weak form using mappings from a *reference element*.

In previous sections, we have considered weak problems of the following form: find $u \in \mathcal{V}$ such that

$$a(u, v) = g(v) \text{ for all } v \in \mathcal{V} \quad (8.155)$$

for some space $\mathcal{V}$, establishing theoretical properties such as existence and uniqueness of a solution in Section 8.2 and approximation properties in Section 8.3. In this section, we outline the finite-element *assembly* process—the algorithm for constructing a linear system of the form

$$A\mathbf{u} = \mathbf{f} \quad (8.156)$$

associated with the weak form in equation (8.155).

In some settings, it may be desirable to compute only the *action* of A —i.e., the matrix-vector product $A\mathbf{v}$ for some vector $\mathbf{v}$ —without explicitly forming A . This occurs, for example, in

some timestepping schemes, or in combination with some preconditioners for solving the linear system. We call such schemes *matrix-free*. This section is focused, instead, on assembling the full system matrix, A , for a *matrix-full* approach, although many of the concepts extend to the matrix-free case.

As a model problem, we consider a simplification of the PDE in equation (8.126) and boundary conditions in equation (8.148),

$$-\nabla \cdot K(\mathbf{x}) \nabla u = f(\mathbf{x}) \quad \text{in } \Omega \subset \mathbb{R}^2, \quad (8.157a)$$

$$u = q_{\mathcal{D}}(\mathbf{x}) \quad \text{on } \Gamma_{\mathcal{D}}, \quad (8.157b)$$

$$\mathbf{n} \cdot (K(\mathbf{x}) \nabla u) = q_{\mathcal{N}}(\mathbf{x}) \quad \text{on } \Gamma_{\mathcal{N}}, \quad (8.157c)$$

for scalar coefficient function $K(\mathbf{x}) > 0$ with $\mathbf{x} \in \Omega$. A representative domain is depicted at the right of Figure 8.5.1, with the boundary, $\partial\Omega$, denoted by the smooth solid line. We use $\Gamma_{\mathcal{N}}$ to denote the part of the boundary with a Neumann boundary condition and $\Gamma_{\mathcal{D}}$ to denote the part of the boundary with a Dirichlet boundary condition, with $\partial\Omega = \Gamma_{\mathcal{N}} \cup \Gamma_{\mathcal{D}}$. We note that the PDE in equation (8.157a) lacks the convection and reaction terms from equation (8.126), but PDEs with these terms can be assembled by simple generalizations of the techniques described below applied to these terms. Similarly, we consider only a Neumann boundary condition on $\Gamma_{\mathcal{N}}$ instead of the Robin condition in equation (8.148). Following Definition 2.12, we define $\mathcal{T}^h$ as the triangulation of Ω consisting of n_{elem} elements, which defines the computational domain Ω^h .

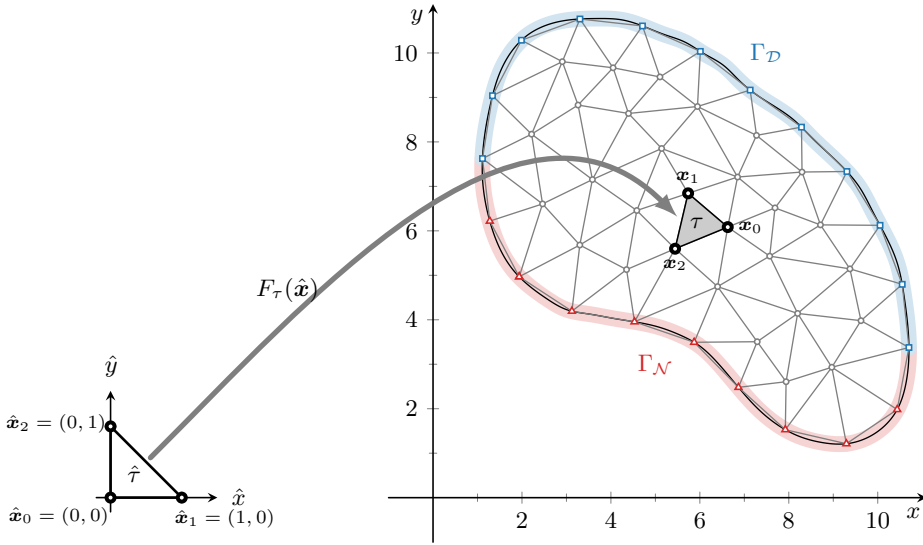


Figure 8.5.1. (left) Reference element, $\hat{\tau}$. (right) Example mesh, $\mathcal{T}^h$, and element $\tau \in \mathcal{T}^h$.

In a structured setting, determining a representative mesh spacing, h , is straightforward. However, for a fully unstructured mesh, the mesh elements may vary in both size and shape. The maximum diameter (i.e., longest edge) is often used to specify h ,

$$h = \max_{\tau \in \mathcal{T}^h} \text{diam}(\tau), \quad (8.158)$$

as illustrated in Figure 8.5.2. As in Section 8.3, approximation properties typically depend on h , in the form

$$\|u - u^h\| \leq Ch^p, \quad (8.159)$$

where $\|\cdot\|$ is most often an L^2 or an H^1 norm.

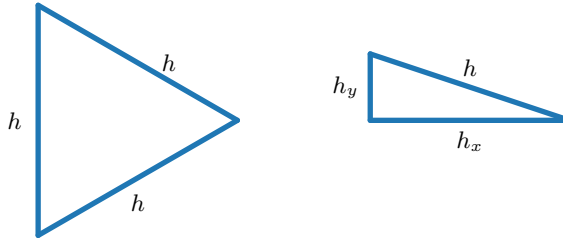


Figure 8.5.2. Example of two mesh elements with the same diameter, h .

Remark 8.38: Higher dimensions and other types of elements

The assembly methodology we discuss here is easily extended to higher dimensions and different $H^1(\Omega)$ -conforming element types, including quadrilateral/hexahedral elements. For $\mathbf{H}(\text{div}, \Omega)$ - and $\mathbf{H}(\text{curl}, \Omega)$ -conforming elements, as discussed in Section 10.2, certain generalizations of these techniques are needed [190]. These extensions rely only on the definition of an appropriate reference element and the basis functions defining the space. For simplicity, we stick with a two-dimensional problem on triangles as our example here.

The weak form of the problem associated with equation (8.157) is to find $u \in H_q^1(\Omega)$ (as defined in equation (8.38)) such that

$$\underbrace{\int_{\Omega} K(\mathbf{x}) \nabla u \cdot \nabla v \, d\mathbf{x}}_{a(u,v)} = \underbrace{\int_{\Omega} f v \, d\mathbf{x} + \int_{\Gamma_{\mathcal{N}}} q_{\mathcal{N}} v \, ds}_{g(v)} \quad (8.160)$$

for all $v \in H_0^1(\Omega)$ (as defined in equation (8.149)). A common approach to finite-element assembly is to now separate the assembly into two steps: (1) the assembly of the weak form in equation (8.160) over a discrete space, $\mathcal{V}^h$, that does not impose any Dirichlet boundary conditions, and (2) enforcement of the Dirichlet boundary conditions. This is the same approach as was used in Example 4.20, where we first assembled the system matrix associated with the problem with Neumann boundary conditions on both ends of the interval,

$$\begin{bmatrix} 1 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ & & -1 & 1 \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \end{bmatrix} = \frac{h}{2} \begin{bmatrix} 1 \\ 2 \\ 2 \\ 1 \end{bmatrix}, \quad (8.161)$$

and then used the homogeneous Dirichlet boundary condition for u_0 to eliminate it from the system, resulting in the reduced problem

$$\frac{1}{h} \begin{bmatrix} 2 & -1 \\ -1 & 2 & -1 \\ & -1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \frac{h}{2} \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}. \quad (8.162)$$

We discuss this second step in Section 8.5.2.

8.5.1 ■ Weak-form assembly

To begin, we ignore the Dirichlet boundary condition entirely and focus on assembling a weak form that satisfies

$$\mathbf{n} \cdot (K(\mathbf{x}) \nabla u) = \hat{q}_{\mathcal{N}}(\mathbf{x}) \text{ for } \mathbf{x} \in \partial\Omega, \quad (8.163)$$

where $\hat{q}_\mathcal{N}$ is chosen to match $q_\mathcal{N}$ on $\Gamma_\mathcal{N}$ and to be zero on $\Gamma_\mathcal{D}$. We note that this problem is not even well posed (given a solution, u , to the weak form in equation (8.160) without Dirichlet boundary conditions, all functions $u(\mathbf{x}) + c$ for any constant $c \in \mathbb{R}$ are also solutions of the weak form), but that we will impose the Dirichlet boundary conditions in Section 8.5.2 to fix this, just as the singular matrix in equation (8.161) becomes nonsingular when we impose the Dirichlet condition leading to equation (8.162). With this, we look to express the weak problem in equation (8.160) over a discrete space, $\mathcal{V}^h \subset H^1(\Omega)$; namely, find $u^h \in \mathcal{V}^h$ such that

$$a(u^h, v^h) = g(v^h) \text{ for all } v^h \in \mathcal{V}^h. \quad (8.164)$$

That is, we do not require the solution or test functions to vanish on the Dirichlet part of the boundary.

For this example, we (arbitrarily) take $\mathcal{V}^h = P_2(\Omega^h)$, selecting quadratic polynomials over each triangle, although the assembly is similar for elements of any order, or for quadrilateral elements, or for 3D meshes of tetrahedral or hexahedral elements. In an *element*-centric assembly, we cast the problem in a slightly different manner than written in equation (8.164) by splitting the integrals over elements. Since the boundary integral is only over edges of triangles that are adjacent to $\Gamma_\mathcal{N}$, we use $\mathcal{E}_\mathcal{N}$ to denote the set of edges of the mesh that are associated with $\Gamma_\mathcal{N}$. The problem is then to find $u^h \in \mathcal{V}^h$ such that

$$\sum_{\tau \in \mathcal{T}^h} \int_{\tau} K(\mathbf{x}) \nabla u^h \cdot \nabla v^h d\mathbf{x} = \sum_{\tau \in \mathcal{T}^h} \int_{\tau} f v^h d\mathbf{x} + \sum_{e \in \mathcal{E}_\mathcal{N}} \int_e q_\mathcal{N} v^h ds \quad (8.165)$$

for all $v^h \in \mathcal{V}^h$. We note that this weak form is consistent with the boundary condition in equation (8.163) when no Dirichlet boundary conditions are imposed on $\mathcal{V}^h$. Now, with a *basis*, $\phi_0, \dots, \phi_\ell$, of $\mathcal{V}^h$, we consider the assembly of $A\mathbf{u} = \mathbf{f}$, where

$$a_{i,j} = a(\phi_j, \phi_i) = \sum_{\tau \in \mathcal{T}^h} \int_{\tau} K(\mathbf{x}) \nabla \phi_j \cdot \nabla \phi_i d\mathbf{x}, \quad (8.166a)$$

$$f_i = g(\phi_i) = \sum_{\tau \in \mathcal{T}^h} \int_{\tau} f \phi_i d\mathbf{x} + \sum_{e \in \mathcal{E}_\mathcal{N}} \int_e q_\mathcal{N} \phi_i ds. \quad (8.166b)$$

The key observation in equation (8.166) is that contributions to the system $A\mathbf{u} = \mathbf{f}$ can be computed *elementwise*. The advantage of this scenario is that we can define integration routines (typically Gaussian quadrature) for a *reference element*, $\hat{\tau}$, and a *reference interval*, $\hat{e} := [0, 1]$. Then, each of the integrals in equation (8.166) is *mapped* to and evaluated on these reference domains. For this, we introduce a mapping, $F_\tau : \hat{\tau} \mapsto \tau$, for each element, τ , as shown in Figure 8.5.1, and a similar mapping, $F_e : \hat{e} \mapsto e$, for each edge, e . For simplices, the mappings F_τ and F_e are affine—i.e., F_τ and F_e are represented as linear maps plus translations,

$$F_\tau(\hat{\mathbf{x}}) = \hat{C}_\tau \hat{\mathbf{x}} + \hat{\mathbf{b}}_\tau, \text{ and } F_e(\hat{t}) = \hat{c}_e \hat{t} + \hat{\mathbf{b}}_e, \quad (8.167)$$

with $\hat{\mathbf{x}} \in \hat{\tau}$ and $\hat{t} \in \hat{e}$. Here, matrix $\hat{C}_\tau$ and translation vector $\hat{\mathbf{b}}_\tau$ define the mapping F_τ , and vectors $\hat{c}_e$ and $\hat{\mathbf{b}}_e$ define the parameterization of edge e in F_e .

If we consider a fixed triangle τ in the mesh $\mathcal{T}^h$, with vertices $\mathbf{x}_0, \mathbf{x}_1$, and $\mathbf{x}_2$, ordered counterclockwise as shown in Figure 8.5.1, and a reference element with vertices $\hat{\mathbf{x}}_0 = (0, 0)$, $\hat{\mathbf{x}}_1 = (1, 0)$, and $\hat{\mathbf{x}}_2 = (0, 1)$, the transformation for any $\hat{\mathbf{x}} = (\hat{x}, \hat{y}) \in \hat{\tau}$ is

$$F_\tau(\hat{\mathbf{x}}) = \begin{bmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{y} \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = J_{F_\tau} \hat{\mathbf{x}} + \mathbf{x}_0. \quad (8.168)$$

The Jacobian of F_τ , denoted by J_{F_τ} , is then

$$J_{F_\tau} = \begin{bmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{bmatrix}. \quad (8.169)$$

Similarly, if we consider a fixed edge, e , with endpoints $\mathbf{x}_0$ and $\mathbf{x}_1$, the transformation is given by

$$F_e(\hat{t}) = (\mathbf{x}_1 - \mathbf{x}_0)\hat{t} + \mathbf{x}_0, \quad (8.170)$$

giving Jacobian $J_{F_e} = F'_e(\hat{t}) = \mathbf{x}_1 - \mathbf{x}_0$. In what follows, we focus on the effects of the change of variables in the integrals over each element, $\tau \in \mathcal{T}^h$, but note that similar transformations follow in the same way for each edge, $e \in \mathcal{E}_\mathcal{N}$.

Since we wish to compute the integrals over the reference element, $\hat{\tau}$, we must consider the basis functions defined there. For example, for the linear basis of $P_1(\Omega^h)$, recall Example 8.32 to get

$$\hat{\phi}_0(\hat{\mathbf{x}}) = 1 - \hat{x} - \hat{y}, \quad (8.171a)$$

$$\hat{\phi}_1(\hat{\mathbf{x}}) = \hat{x}, \quad (8.171b)$$

$$\hat{\phi}_2(\hat{\mathbf{x}}) = \hat{y}. \quad (8.171c)$$

These are shown in Figure 8.5.3. Similarly, quadratic basis functions over the reference element are given by

$$\hat{\phi}_0(\hat{\mathbf{x}}) = (1 - \hat{x} - \hat{y})(1 - 2\hat{x} - 2\hat{y}), \quad (8.172a)$$

$$\hat{\phi}_1(\hat{\mathbf{x}}) = \hat{x}(2\hat{x} - 1), \quad (8.172b)$$

$$\hat{\phi}_2(\hat{\mathbf{x}}) = \hat{y}(2\hat{y} - 1), \quad (8.172c)$$

$$\hat{\phi}_3(\hat{\mathbf{x}}) = 4\hat{x}(1 - \hat{x} - \hat{y}), \quad (8.172d)$$

$$\hat{\phi}_4(\hat{\mathbf{x}}) = 4\hat{x}\hat{y}, \quad (8.172e)$$

$$\hat{\phi}_5(\hat{\mathbf{x}}) = 4\hat{y}(1 - \hat{x} - \hat{y}). \quad (8.172f)$$

These six basis functions are shown in Figure 8.5.4.

An important property of our nodal basis functions is that we can evaluate each $\phi_j(\mathbf{x})$ on the actual element, τ , by mapping them back to the corresponding basis function on the reference element, $\hat{\tau}$, using the inverse mapping F_τ^{-1} :

$$\phi_j(\mathbf{x}) = \hat{\phi}_j(F_\tau^{-1}(\mathbf{x})). \quad (8.173)$$

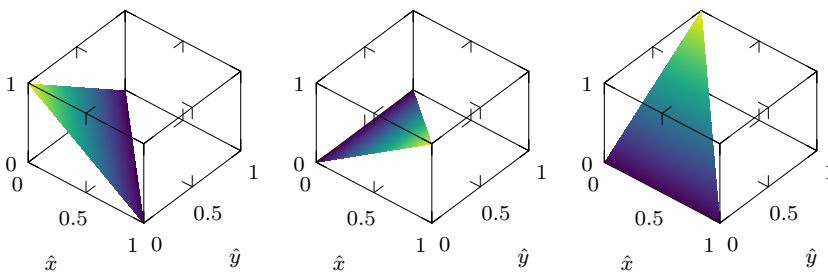


Figure 8.5.3. Linear basis functions on a reference triangle $\hat{\tau}$.

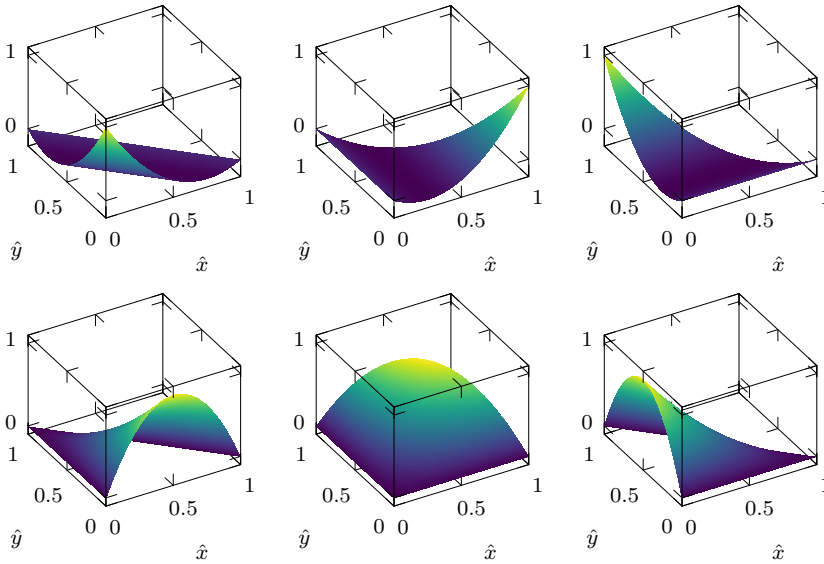


Figure 8.5.4. Quadratic basis functions on a reference triangle $\hat{\tau}$.

This allows us to compute $\nabla \phi_j(\mathbf{x})$ using the chain rule,

$$\nabla \phi_j(\mathbf{x}) = \nabla \left(\hat{\phi}_j(F_\tau^{-1}(\mathbf{x})) \right) = J_{F_\tau}^{-T} \nabla_{\hat{\mathbf{x}}} \left(\hat{\phi}_j(F_\tau^{-1}(\mathbf{x})) \right), \quad (8.174)$$

where $\nabla_{\hat{\mathbf{x}}}$ indicates the gradient in the reference triangle coordinates. Using this, the integrals over a given τ and $e \in \mathcal{E}_{\mathcal{N}}$ in the weak form become

$$\int_{\tau} K(\mathbf{x}) \nabla \phi_j(\mathbf{x}) \cdot \nabla \phi_i(\mathbf{x}) d\mathbf{x} = \int_{\hat{\tau}} K(F_\tau(\hat{\mathbf{x}})) \left(J_{F_\tau}^{-T} \nabla_{\hat{\mathbf{x}}} \hat{\phi}_j \right) \cdot \left(J_{F_\tau}^{-T} \nabla_{\hat{\mathbf{x}}} \hat{\phi}_i \right) |J_{F_\tau}| d\hat{\mathbf{x}}, \quad (8.175a)$$

$$\int_{\tau} f \phi_i d\mathbf{x} = \int_{\hat{\tau}} f(F_\tau(\hat{\mathbf{x}})) \hat{\phi}_i |J_{F_\tau}| d\hat{\mathbf{x}}, \quad (8.175b)$$

$$\int_e q_{\mathcal{N}} \phi_i ds = \int_{\hat{e}} q_{\mathcal{N}}(F_e(\hat{t})) \hat{\phi}_i |J_{F_e}| d\hat{t}, \quad (8.175c)$$

where we note that $|J_{F_\tau}|$ denotes the determinant of the Jacobian, while $|J_{F_e}|$ is the length of the vector J_{F_e} .

Remark 8.39: Quadrature order

In practice, these integrals are evaluated using numerical quadrature (see Section 2.5). Since they are done over the reference element or interval, only one set of quadrature points is needed for each. Also, the integration of $a(\phi_j, \phi_i)$ involves products of the basis functions of the finite-element space and/or their derivatives. Since these functions are polynomials of order k , the quadrature needs to have a sufficiently high order to integrate polynomials of order $2k$ exactly. Thus, a Gaussian quadrature of order $k+1$ is typically used and is all that is needed for constant $K(\mathbf{x})$. The integration of $g(\phi_i)$, however, may require higher-order accuracy if $f(\mathbf{x})$ and/or $q_{\mathcal{N}}(\mathbf{x})$ are rapidly varying over an element or edge, respectively.

Since the basis functions are “local,” entry (i, j) of the matrix requires contributions only from elements that support both basis functions ϕ_i and ϕ_j (and similarly for the right-hand side).

Thus, the assembly of the matrix and right-hand side proceeds by looping over the elements, mapping each element to the reference element, computing the necessary integrals with quadrature, and finally adding the contributions to the appropriate entries in the global matrices.

In the current example of $P_2(\Omega^h)$ in two dimensions, each element has six associated degrees of freedom, requiring 36 combinations of their products. This generates a 6×6 *local matrix* over each element. In turn, these entries are “mapped” to the global matrix by determining the global index of each entry. An example of the local and global ordering of degrees of freedom associated with element τ^k is given in Figure 8.5.5. As we did for the 1D case in Section 4.4, for each τ^k , we define a matrix P^k , with entries given by selecting the columns of the global identity matrix that correspond to the degrees of freedom on element τ^k , with order chosen so that the j th column of P^k is the column of the global identity that corresponds to the j th basis function on τ^k . With this, $A = \sum_{\tau^k \in \mathcal{T}^h} P^k A^k (P^k)^T$ and $\mathbf{f} = \sum_{\tau^k \in \mathcal{T}^h} P^k \mathbf{f}^k$. For the element illustrated in Figure 8.5.5, P^k would be a 22×6 matrix with the following entries indicating that global degrees of freedom 2, 7, 0, 20, 8, and 18 are on element τ^k with that local ordering:

$$P^k = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \quad (8.176)$$

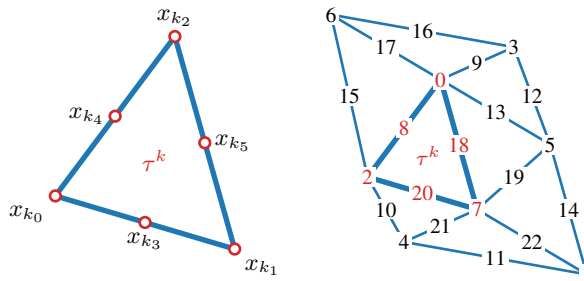


Figure 8.5.5. (left) Local ordering of degrees of freedom and (right) global ordering of degrees of freedom on an eight-element mesh using quadratic elements.

Algorithm 8.1: Finite-element assembly

Input : $\mathcal{T}^h$, list of elements
 $\mathcal{E}_{\mathcal{N}}$, list of edges associated with $\Gamma_{\mathcal{N}}$
 K , diffusion coefficient
 f , right-hand side function
 $q_{\mathcal{N}}$, Neumann boundary condition

Output : A , sparse matrix ($n_{\text{dof}} \times n_{\text{dof}}$)
 $\mathbf{f}$, right-hand side vector ($n_{\text{dof}} \times 1$)

```

1   $A \leftarrow 0, f \leftarrow 0$                                      {initialize the global matrix and right-hand side}
2   $\mathbf{w}^{\hat{\tau}}, \hat{\mathbf{x}}^{\hat{\tau}} \leftarrow \text{quadrature}(q^{\hat{\tau}})$          {quadrature weights and nodes of degree  $q^{\hat{\tau}}$  on reference triangle  $\hat{\tau}$ }
3   $\mathbf{w}^{\hat{e}}, \hat{\mathbf{t}}^{\hat{e}} \leftarrow \text{quadrature}(q^{\hat{e}})$              {quadrature weights and nodes of degree  $q^{\hat{e}}$  on reference interval  $\hat{e}$ }

4  for each  $\tau^k \in \mathcal{T}^h$                                        {loop over elements}
5       $\mathbf{x}_{k_0}, \dots, \mathbf{x}_{k_5} \leftarrow \tau^k$                  {location of degrees of freedom in element  $k$  (vertices ordered first)}
6       $J_{F_{\tau}} = [\mathbf{x}_{k_1} - \mathbf{x}_{k_0} \quad \mathbf{x}_{k_2} - \mathbf{x}_{k_0}]$          {element Jacobian}
7      Define  $P^k \in \mathbb{R}^{n_{\text{dof}} \times 6}$                          {columns of global identity over degrees of freedom of element  $k$ }
8       $A^k, \mathbf{f}^k \leftarrow \text{ASSEMBLE-VOLUME-INTEGRALS}(\mathbf{w}^{\hat{\tau}}, \hat{\mathbf{x}}^{\hat{\tau}}, \mathbf{x}_{k_0}, J_{F_{\tau}}, K, f)$ 
9       $A = A + P^k A^k (P^k)^T$                                    {accumulate to global matrix}
10      $\mathbf{f} = \mathbf{f} + P^k \mathbf{f}^k$                                    {accumulate to global right-hand side}

11     for each  $e \in \mathcal{E}_{\mathcal{N}}$  s.t.  $e \cap \partial\tau^k \neq \emptyset$          {each element edge on the Neumann boundary}
12          $\mathbf{x}_{e_0}, \mathbf{x}_{e_1} \leftarrow e$                          {identify endpoints of edge  $e$ }
13          $J_{F_e} = \mathbf{x}_{e_1} - \mathbf{x}_{e_0}$                              {tangent vector of edge from  $\mathbf{x}_{e_0}$  to  $\mathbf{x}_{e_1}$ }
14         Define  $P^e \in \mathbb{R}^{6 \times 3}$                              {columns of local identity over degrees of freedom of edge  $e$ }
15          $\mathbf{f}^e \leftarrow \text{ASSEMBLE-BOUNDARY-INTEGRALS}(\mathbf{w}^{\hat{e}}, \hat{\mathbf{t}}^{\hat{e}}, \mathbf{x}_{e_0}, J_{F_e}, q_{\mathcal{N}})$ 
16          $\mathbf{f} = \mathbf{f} + P^e (P^e)^T \mathbf{f}^e$                          {accumulate to global right-hand side}

```

The overall approach for assembly is outlined in Algorithm 8.1, with subroutines for assembling the volume integral terms and the boundary integral terms separately in Algorithm 8.2 and Algorithm 8.3, respectively. We note the high-level similarity between Algorithm 8.1 and the approach outlined in Section 4.4, where we loop over every element, τ^k , and use quadrature to assemble the element stiffness matrix, A^k , and right-hand side, $\mathbf{f}^k$, before using the local-to-global map, P^k , to integrate these into the global matrix and right-hand side. One new addition is accounting for boundary integrals as in Algorithm 8.3. Here, we now need a mapping, P^e , with entries given by selecting the columns of the local identity matrix that correspond to the local degrees of freedom on edge e of the given element, τ^k . Then, the particular edge contribution to the global right-hand side is added as $\mathbf{f} = \mathbf{f} + P^e (P^e)^T \mathbf{f}^e$. For the example in Figure 8.5.5, each P^e is a 6×3 matrix. Then, for instance, the bottom edge on τ^k containing local degrees of freedom x_{k_0} , x_{k_1} , and x_{k_3} would have mapping

$$P^e = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}. \quad (8.177)$$

Algorithm 8.2: ASSEMBLE-VOLUME-INTEGRALS

Input : $w^{\hat{\tau}}$, quadrature weights on the reference triangle
 $\hat{\mathbf{x}}^{\hat{\tau}}$, quadrature nodes on the reference triangle
 $\mathbf{x}_0$, reference vertex on the current element
 J_F , Jacobian of transformation from reference element
 K , diffusion coefficient
 f , right-hand side function
Output : A^k , local element matrix
 $\mathbf{f}^k$, local element right-hand side vector

```

1   $A^k = 0 \in \mathbb{R}^{6 \times 6}$                                      {initialize  $k$ th local element matrix}
2   $\mathbf{f}^k = 0 \in \mathbb{R}^6$                                      {initialize  $k$ th local element right-hand side vector}
3  for each  $w, (\hat{x}, \hat{y}) \in w^{\hat{\tau}}, \hat{\mathbf{x}}^{\hat{\tau}}$                      {loop over quadrature points/weights}
4       $\hat{\Phi} = \begin{bmatrix} (1 - \hat{x} - \hat{y})(1 - 2\hat{x} - 2\hat{y}) \\ \hat{x}(2\hat{x} - 1) \\ \hat{y}(2\hat{y} - 1) \\ 4\hat{x}(1 - \hat{x} - \hat{y}) \\ 4\hat{x}\hat{y} \\ 4\hat{y}(1 - \hat{x} - \hat{y}) \end{bmatrix}$                                      {basis evaluated at quadrature point}
5       $\nabla_{\hat{\mathbf{x}}} \hat{\Phi} = \begin{bmatrix} 4\hat{x} + 4\hat{y} - 3 & 4\hat{x} + 4\hat{y} - 3 \\ 4\hat{x} - 1 & 0 \\ 0 & 4\hat{y} - 1 \\ -8\hat{x} - 4\hat{y} + 4 & -4\hat{x} \\ 4\hat{y} & 4\hat{x} \\ -4\hat{y} & -4\hat{x} - 8\hat{y} + 4 \end{bmatrix}^T$                                      {gradients evaluated at quadrature point}
6       $\nabla \Phi = J_F^{-T} \nabla_{\hat{\mathbf{x}}} \hat{\Phi}$                                      {scale with the Jacobian}
7       $\tilde{\mathbf{x}} = J_F \hat{\mathbf{x}} + \mathbf{x}_0$                                      {transform coordinates to the physical element}
8       $A^k = A^k + w K(\tilde{\mathbf{x}}) (\nabla \Phi)^T \nabla \Phi |J_F|$                                      {add to the local element matrix}
9       $\mathbf{f}^k = \mathbf{f}^k + w f(\tilde{\mathbf{x}})^T \hat{\Phi} |J_F|$                                      {add to the local element right-hand side}

```

8.5.2 ■ Boundary conditions

The steps above assemble a system of the form $A\mathbf{u} = \mathbf{f}$ for basis functions over the entire mesh including all boundary nodes, subject to the Neumann boundary condition in equation (8.163) on all of $\partial\Omega$. Just as in Example 4.20, we now look to impose the Dirichlet boundary conditions on the system before we solve for $\mathbf{u}$. To do this, we partition the finite-element space into sets of Dirichlet (D) and non-Dirichlet (I) degrees of freedom, so that $D \cap I = \emptyset$ and $D \cup I$ covers all degrees of freedom in the space. For standard $P_k(\Omega)$ elements considered here, this partitioning is straightforward, with set D defined by all finite-element degrees of freedom associated with nodes $\mathbf{x}_k \in \Gamma_D$. The linear system assembled above is then permuted into a block ordering based on this partition that we write as

$$\begin{bmatrix} A_{II} & A_{ID} \\ A_{DI} & A_{DD} \end{bmatrix} \begin{bmatrix} \mathbf{u}_I \\ \mathbf{u}_D \end{bmatrix} = \begin{bmatrix} \mathbf{f}_I \\ \mathbf{f}_D \end{bmatrix}. \quad (8.178)$$

Algorithm 8.3: ASSEMBLE-BOUNDARY-INTEGRALS

Input : $w^{\hat{e}}$, quadrature weights on the reference interval
 $t^{\hat{e}}$, quadrature nodes on the reference interval
 $\mathbf{x}_0$, reference vertex on the current edge
 J_{F_e} , tangent vector of current edge
 q_N , Neumann boundary data
Output : $\mathbf{f}^e$, local edge right-hand side vector

```

1  $\mathbf{f}^e = 0 \in \mathbb{R}^3$                                      {initialize local edge right-hand side vector}
2 for each  $w, \hat{t} \in w^{\hat{e}}, t^{\hat{e}}$                          {loop over interval quadrature points/weights}
3    $\hat{\Phi} = \begin{bmatrix} (1-\hat{t})(1-2\hat{t}) \\ \hat{t}(2\hat{t}-1) \\ 4\hat{t}(1-\hat{t}) \end{bmatrix}$                                      {basis evaluated at quadrature point}
4    $\tilde{\mathbf{x}} = J_{F_e} \hat{t} + \mathbf{x}_0$                              {transform coordinates to the physical edge}
5    $\mathbf{f}^e = \mathbf{f}^e + w q_N(\tilde{\mathbf{x}})^T \hat{\Phi} |J_{F_e}|$            {add to the local edge right-hand side}
```

To impose the Dirichlet boundary conditions, we replace the equations for $\mathbf{u}_D$ with the discretization of the Dirichlet boundary values, writing

$$\begin{bmatrix} A_{II} & A_{ID} \\ 0 & I \end{bmatrix} \begin{bmatrix} \mathbf{u}_I \\ \mathbf{u}_D \end{bmatrix} = \begin{bmatrix} \mathbf{f}_I \\ \mathbf{q}_D \end{bmatrix}. \quad (8.179)$$

This is akin to the discussion in Section 8.4.2, where we define the function $\hat{u}$ by the finite-element function with coefficients given by the vector $\begin{bmatrix} \mathbf{u}_I \\ 0 \end{bmatrix}$ and the function $\hat{q}$ by the finite-element function with coefficients given by the vector $\begin{bmatrix} 0 \\ \mathbf{q}_D \end{bmatrix}$. This works for both the case of full Dirichlet boundary conditions in Section 8.4.2 and the case when $\Gamma_D \subset \partial\Omega$ as in Section 8.4.3.

The disadvantage of the system in equation (8.179) is that we have taken a symmetric weak form, $a(u, v)$, and a symmetric matrix, A , and rewritten it as a nonsymmetric matrix due to the *partial* elimination of the boundary conditions. However, given the triangular structure of the linear system, we can easily *fully* eliminate the Dirichlet boundary values to get

$$\begin{bmatrix} A_{II} & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \mathbf{u}_I \\ \mathbf{u}_D \end{bmatrix} = \begin{bmatrix} \mathbf{f}_I - A_{ID}\mathbf{q}_D \\ \mathbf{q}_D \end{bmatrix}. \quad (8.180)$$

Here, the term $\mathbf{f}_I - A_{ID}\mathbf{q}_D$ is a discrete analogue of the right-hand side in equation (8.147b). The resulting system (or the smaller system for just $\mathbf{u}_I$) is once again symmetric, so it can be solved using a Krylov method, such as the CG algorithm. The reduced system given in equation (8.162), following Example 4.20, is precisely the system $A_{II}\mathbf{u}_I = \mathbf{f}_I$, which always results when $\mathbf{q}_D = 0$.

Remark 8.40: Dirichlet boundary elimination during assembly

The Dirichlet boundary conditions can also be eliminated during the assembly phase. When looping over the degrees of freedom of each element, we check if each degree of freedom is on the boundary. If so, we modify the matrix entry to correspond to an identity when accumulating the global matrix, and we modify the right-hand side accordingly. This may sometimes be more efficient if the assembly is done once and is matrix-free. However, for time-dependent or nonlinear problems, it may be more useful to eliminate after the assembly, as described above.

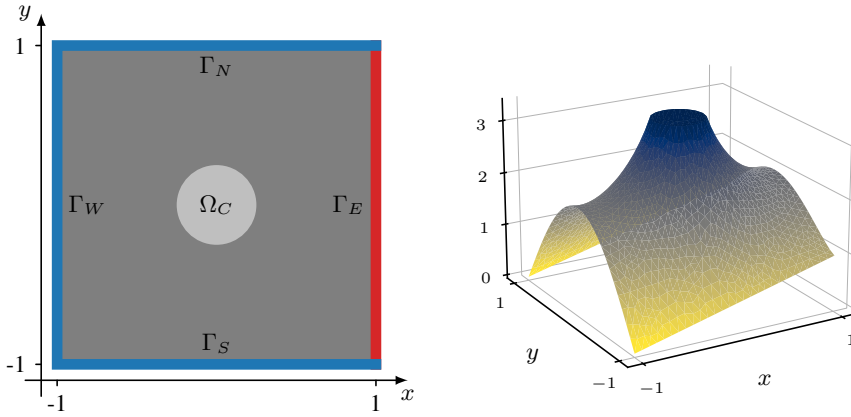


Figure 8.5.6. (left) Problem domain and (right) finite-element solution.

8.5.3 ■ Example

As an example, consider the problem in equation (8.157) on a domain $\Omega = [-1, 1]^2$, as given in Figure 8.5.6, highlighting a disk of radius 0.25 in the center of the domain as Ω_C and boundaries Γ_N , Γ_S , Γ_E , and Γ_W . For this problem, we consider Dirichlet data on $\Gamma_D = \Gamma_N \cup \Gamma_S \cup \Gamma_W$ and Neumann conditions on Γ_E . Specifically, consider

$$u = 2 \cos\left(\frac{\pi y}{2}\right) \quad \text{on } \Gamma_W, \quad u = \frac{1+x}{2} \quad \text{on } \Gamma_N \cup \Gamma_S, \quad \mathbf{n} \cdot \nabla u = 0 \quad \text{on } \Gamma_E. \quad (8.181)$$

In addition, we describe a diffusion coefficient $K(\mathbf{x})$ and forcing function $f(\mathbf{x})$ that have different values over the center disk, Ω_C :

$$f(\mathbf{x}) = \begin{cases} 100 & \text{if } \mathbf{x} \in \Omega_C, \\ 0 & \text{otherwise,} \end{cases} \quad K(\mathbf{x}) = \begin{cases} 100 & \text{if } \mathbf{x} \in \Omega_C, \\ 2 & \text{otherwise.} \end{cases} \quad (8.182)$$

From this description, we expect a flat portion within the center disk, due to the high diffusion and high forcing. At the boundaries, we should observe fixed data on the north, south, and west faces, while a flattening of the solution is expected at the east boundary, due to the Neumann condition. The (piecewise linear) finite-element solution reflects this, as shown in Figure 8.5.6.

8.6 ■ A posteriori error estimates

Objectives

In Section 8.3, we presented the approximation properties of various types of finite elements on given meshes, which lead to bounds on the possible error when approximating the solution to a PDE with that method. These bounds are *a priori* error estimates that are formally achieved in the asymptotic limit as the mesh parameter, h , goes to zero. In this section, you will consider *a posteriori* error estimates in the context of the finite-element method that, given a known approximation of the solution, we estimate its error locally or globally. Such error estimates should be “cheap” to compute and give the user an idea of how well the PDE is being solved. They can then be used for a variety of things, such as adaptive mesh refinement or to calculate other quantities of interest. In this section, you will investigate efficient and reliable error estimates and describe different types of estimators, such as residual-based and recovery-based error estimates.

Bounds on the error of an approximation, u^h , to the solution, u , of a weak form of a PDE follow naturally by applying the theory of Lax–Milgram, Céa’s lemma, and the approximation properties of the finite-element space. Examples of these bounds for approximations in $P_1(\Omega^h)$ and $P_2(\Omega^h)$, for instance, are given in Example 8.36. We refer to these error estimates as *a priori* bounds, as they are already known “before” we compute u^h . In contrast, once we have found the approximation, u^h , we would often like to know both how good it is and how to improve it. An *a posteriori* error estimate approximates the specific error “after” the solution procedure is completed and u^h is known. Denoting the finite-element error by $e^h = u - u^h$, a global *a posteriori* error estimate, θ , of $\|e^h\|_{\mathcal{V}}$ can act as a lower or upper bound on the error, as defined in Definition 8.41.

Definition 8.41: Efficient and reliable error estimators (cf. [187])

Let $u \in \mathcal{V}$ be the solution of a weak form of a PDE, and let $u^h \in \mathcal{V}^h$ be its Ritz–Galerkin approximation, with $e^h = u - u^h$. An error estimator, θ , for $\|e^h\|_{\mathcal{V}}$ is called efficient if it is a lower bound on the error,

$$\theta^2 \leq c_1 \|e^h\|_{\mathcal{V}}^2, \quad (8.183)$$

for some constant $c_1 > 0$. It is called reliable if it is an upper bound on the error,

$$c_0 \|e^h\|_{\mathcal{V}}^2 \leq \theta^2, \quad (8.184)$$

for some constant $c_0 > 0$.

In equations (8.183) and (8.184), we consider a *global* error estimate, θ , measured over the entire computational domain. The quantity

$$\eta = \frac{\theta}{\|e^h\|_{\mathcal{V}}} \quad (8.185)$$

is referred to as the effectivity index and measures the quality of the global estimation. When η approaches one, the error estimate exactly reflects the true error.

The main use of error estimators is to allow us to “fix” errors in an approximation in order to reduce this error. This leads to adaptive finite-element methods (AFEMs), which are discretization techniques for choosing optimal finite-element spaces with respect to computational complexity and accuracy. This is typically done by adaptively refining the computational grid based on where a localized version of the error estimate is large. Such adaptive mesh refinement (AMR) strategies follow an adaptive feedback loop:

SOLVE $\longrightarrow$ ESTIMATE $\longrightarrow$ MARK $\longrightarrow$ REFINE.

In this approach, we first solve a given discretized problem as we normally would on some relatively coarse initial grid and obtain some approximation to the solution. Then, using a local *a posteriori* error estimate, we estimate the error over the domain. After marking those elements on which the error estimate is large, we refine them with a goal of increasing the accuracy of the approximation. Solving on this new mesh, we expect an improved accuracy of the discrete approximation and can continue the loop by estimating the error of this new approximation, etc. Here, we note that we need the error estimator to be “local” and think of writing it as

$$\theta^2 = \sum_{\tau \in \mathcal{T}^h} \theta_{\tau}^2, \quad (8.186)$$

where the mesh triangulation is given by $\mathcal{T}^h = \{\tau\}$ and θ_τ is a local a posteriori error estimate defined on each element, τ . Thus, in the marking step of AMR, we also consider measures of the *local* effectivity index, $\eta_\tau = \frac{\theta_\tau}{\|e^h\|_\tau}$.

There are a variety of refinement techniques and marking strategies in the literature, and the field of AFEMs is quite mature. We refer to some classical texts on adaptivity and AMR ([28, 67, 6, 29, 179, 111, 187, 229] and the references therein) for more details. The remainder of this section is focused on the different types of a posteriori error estimates, mainly focused on residual-based ones, though we provide a brief overview of others, such as recovery-based and goal-oriented error estimates.

8.6.1 ■ Residual-based error estimates

Broadly speaking, residual-based error estimates can be broken down into two main categories: explicit and implicit. Explicit a posteriori error estimates use a direct computation of the residual of the finite-element approximation, typically defined *locally* on each element. Implicit methods require the solution of a local boundary-value problem. In general, explicit approaches are computationally cheap but are less robust than implicit methods. Hence, they typically only act as *error indicators* as opposed to efficient and reliable estimates.

Consider the model problem in equations (8.64) and (8.65),

$$-\nabla \cdot \nabla u = f(x) \text{ for } x \in \Omega, \quad (8.187)$$

with $u = 0$ on $\partial\Omega$ and with $f \in L^2(\Omega)$. The weak form is to find $u \in \mathcal{V}$ such that

$$\underbrace{\int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x}}_{a(u,v)} = \underbrace{\int_{\Omega} f(\mathbf{x})v \, d\mathbf{x}}_{g(v)} \text{ for all } v \in \mathcal{V}. \quad (8.188)$$

To start, we form the residual (or error) equation, namely,

$$a(e^h, v) = a(u - u^h, v) = a(u, v) - a(u^h, v) = g(v) - a(u^h, v) \quad \forall v \in \mathcal{V}. \quad (8.189)$$

This is further broken down into a summation over the elements in the domain:

$$a(e^h, v) = \sum_{\tau \in \mathcal{T}^h} \int_{\tau} (f v - \nabla u^h \cdot \nabla v) \, d\mathbf{x} \quad \forall v \in \mathcal{V}. \quad (8.190)$$

Integrating by parts over each element gives

$$a(e^h, v) = \sum_{\tau \in \mathcal{T}^h} \int_{\tau} r_{\tau} v \, d\mathbf{x} + \sum_{\gamma \in \mathcal{E}^h} \int_{\gamma} j_{\gamma} v \, ds \quad \forall v \in \mathcal{V}, \quad (8.191)$$

where $r_{\tau} = f(\mathbf{x}) + \Delta u^h$ is the interior residual restricted to element τ and $\mathcal{E}^h$ is the set of all interfaces of elements in $\mathcal{T}^h$. Here, $j_{\gamma} = [\nabla u^h \cdot \mathbf{n}]_{\gamma}$ is the jump in the normal component of the gradient across an element interface, with $\mathbf{n}$ being a unit normal to edge γ . In other words, $j_{\gamma} = (\nabla u^h)^1 \cdot \mathbf{n}_{\tau^1} + (\nabla u^h)^2 \cdot \mathbf{n}_{\tau^2}$ when γ is the boundary between elements τ^1 and τ^2 , which have outward unit normal vectors $\mathbf{n}_{\tau^1}$ and $\mathbf{n}_{\tau^2}$, and $(\nabla u^h)^1$ and $(\nabla u^h)^2$ signify evaluating ∇u^h along γ using the approximations on elements τ^1 and τ^2 , respectively. Note that for edges on the (Dirichlet) boundary, the jump is taken to be zero. See Section 10.5 for more details on evaluating jumps across element boundaries.

Next, using the orthogonality property of the finite-element weak form, $a(e^h, v^h) = 0$ for all $v^h \in \mathcal{V}^h$, we introduce an interpolant, I^h , from $\mathcal{V}$ to $\mathcal{V}^h$ such that

$$a(e^h, v) = \sum_{\tau \in \mathcal{T}^h} \int_{\tau} r_{\tau} (v - I^h v) \, d\mathbf{x} + \sum_{\gamma \in \mathcal{E}^h} \int_{\gamma} j_{\gamma} (v - I^h v) \, ds \quad \forall v \in \mathcal{V}. \quad (8.192)$$

Finally, using interpolation error estimates and other properties of the finite-element space, such as those described in Section 8.3, we obtain a bound on the error in the energy norm, $\|\cdot\|_{\mathcal{V}} = \sqrt{a(\cdot, \cdot)}$,

$$\|e^h\|_{\mathcal{V}}^2 \leq \sum_{\tau \in \mathcal{T}^h} \left(c_1 h_{\tau}^2 \|r_{\tau}\|_{0,\tau}^2 + c_2 h_{\tau} \sum_{\gamma \in \partial\tau} \|j_{\gamma}\|_{0,\gamma}^2 \right), \quad (8.193)$$

where the summation of the interfaces has been regrouped and scaled (by a factor of 1/2) to distribute the error equally across element interfaces and make the bound easier to compute. While the constants c_1 and c_2 , which depend on the approximation properties of the finite-element space, are not necessarily known, everything else on the right-hand side of equation (8.193) is computable from the current approximation. Thus, bounds as in equations (8.183) and (8.184) are obtained with constants related to the constants in equation (8.193). This then leads to a local, reliable a posteriori error estimate:

$$\theta_{\tau}^2 = h_{\tau}^2 \|r_{\tau}\|_{0,\tau}^2 + \frac{1}{2} h_{\tau} \sum_{\gamma \in \partial\tau} \|j_{\gamma}\|_{0,\gamma}^2. \quad (8.194)$$

Variations on these explicit residual-based bounds can be obtained for different PDEs and problems with different types of boundary conditions, but the constants in these bounds always depend only on the problem data given (e.g., right-hand sides and boundary conditions). Additionally, error estimates in other norms, such as $L^2(\Omega)$, are also obtained by using an Aubin–Nitsche duality argument, for example, in the style of Theorem 4.9.

While explicit methods are computationally inexpensive, some drawbacks include that they only give reliable (upper) bounds on the error, they involve possibly unknown constants (depending on approximation properties of the finite-element space), and they contain a residual of the approximation on both the interior and the boundary, making it difficult to decide what the best balance is. To obtain efficient estimates and ones without ambiguity on their weights, we consider implicit residual-based measures. In this setting, the error equation, given in equation (8.192), is solved using the residual as given data. Obviously, if this is solved exactly, we would have the exact error, but this would be impractical and computationally infeasible. Thus, instead, an auxiliary, local problem is solved,

$$a(w^h, v)_{\tau} = \int_{\tau} r_{\tau} v \, d\mathbf{x} + \sum_{\gamma \in \partial\tau} \int_{\gamma} j_{\gamma} v \, ds \quad \forall v \in \mathcal{V}_{\tau}, \quad (8.195)$$

where $\mathcal{V}_{\tau}$ is the function space restricted to an individual element. Then, the error estimate is taken to be $\theta_{\tau} = \|w^h\|_{\mathcal{V}}$. Unfortunately, the existence and uniqueness of solutions to the auxiliary problem may not be clear and, thus, variations on these local problems may include solving the problem over a patch of elements that permit ellipticity of the local bilinear form. Additionally, some postprocessing of the boundary data may be needed to account for the fact that the fluxes across interior element interfaces may not be known. In practice, such implicit methods involve solving smaller local problems with higher accuracy than the original solution process. This leads to more computationally intensive work, but it also leads to highly accurate, efficient, and reliable error estimates. Again, more details can be found in [6, 111].

Example 8.42: Adaptive mesh refinement with a residual-based error estimator

Consider the Poisson problem

$$-\nabla \cdot \nabla u = f \quad \text{on } [-1, 1]^2, \quad (8.196)$$

with homogeneous Dirichlet conditions on the boundary and with forcing function

$$f(x) = e^{-100((x-\frac{1}{4})^2 + (y-\frac{1}{4})^2)}. \quad (8.197)$$

We first find an approximate solution on an initial mesh using standard linear finite elements (see the left panel of Figure 8.6.1), leading to u^h . Next, we use u^h and equation (8.194) to estimate the error on each triangle, which generates θ_τ for each τ —this is shown in the middle panel of Figure 8.6.1. Finally, we select all triangles with $\theta_\tau > 0.001$ and mark them for refinement (those outlined in red in the middle panel of Figure 8.6.1); the refined mesh is shown in the right panel of Figure 8.6.1. The estimator captures an area of steep gradients due to the strong forcing function centered at $(\frac{1}{4}, \frac{1}{4})$.

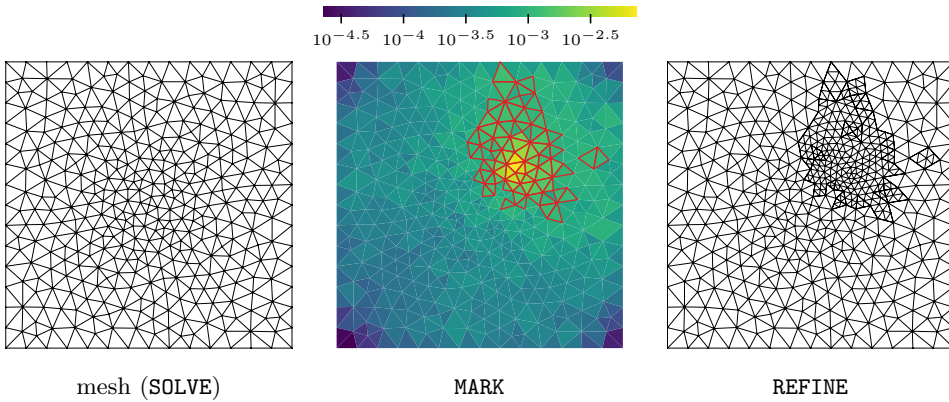


Figure 8.6.1. Steps in adaptive mesh refinement. (left) An initial coarse mesh, (middle) the elementwise error indicator θ_τ , and (right) a locally refined mesh.

8.6.2 ■ Other types of error estimates

There are a vast number of a posteriori error estimators for many applications and PDEs. We briefly describe two popular approaches below but omit many of the technical details. For a more complete description of the methods, see, for example, [187], and the references therein.

Recovery-based error estimates

A second type of a posteriori error estimator is a recovery-based method. Here, the approximate solution, u^h , is postprocessed to obtain a new solution, $\hat{u}^h$, which is hopefully more accurate than u^h (or at least allows for a more accurate estimate of its error). A common approach is to assume that large oscillations in u^h mean large error. Then, the goal is to actually get a better estimate of the gradient, $\nabla \hat{u}^h$. If for instance, $\|\nabla \hat{u}^h - \nabla u\| \leq c \|\nabla u^h - \nabla u\|$ for some constant $c < 1$, then

$$(1 - c) \|\nabla u^h - \nabla u\| \leq \|\nabla \hat{u}^h - \nabla u\| \leq (1 + c) \|\nabla u^h - \nabla u\|, \quad (8.198)$$

and the difference in the gradients is taken to be the new estimator,

$$\theta = \|\nabla \hat{u}^h - \nabla u^h\|. \quad (8.199)$$

Indeed, when $c \ll 1$, equation (8.199) provides an easily computable and accurate estimate of the error in the gradients, $\|\nabla u^h - \nabla u\|$ (cf. [187, Sec. 2.6.3]).

A popular approach, called the ZZ superconvergent patch recovery method [239], uses a least-squares fit with higher-order polynomials to obtain a nodal approximation of $\nabla \hat{u}^h$ over a patch of elements. The difference in this new approximation is then used as the error estimate, typically calculated elementwise:

$$\theta_\tau = \|\nabla \hat{u}^h - \nabla u^h\|_{0,\tau}. \quad (8.200)$$

Such approaches are popular for a variety of PDEs, including nonlinear ones, as they do not require using properties of the PDE itself. Under certain smoothness conditions, the error estimate in equation (8.200) is asymptotically exact. Modifications also exist for problems with discontinuities in the PDE parameters. Finally, we note that for problems that admit oscillatory behavior naturally, the error estimates will not be as precise.

Goal-oriented error estimates

Sometimes, we are not interested in estimating the error of the approximation in the energy norm or L^2 norm but are more concerned with estimating some property of the solution. In other words, we are more concerned with obtaining a *quantity of interest* than the actual solution. Such quantities might be stresses along a surface in an elasticity problem, vorticity of an eddy in a fluid problem, etc. Thus, approximating the error in this quantity of interest leads to *goal-oriented a posteriori error estimates*.

Consider the original primal problem, abstractly written as

$$a(u, v) = g(v) \quad \forall v \in \mathcal{V}, \quad (8.201)$$

and define the quantity of interest as $Q(u)$. This quantity may be a local or a global (i.e., averaged) measurement of the solution. Next, an auxiliary adjoint problem is formed (see, for example, [187, Sec. 2.6.4]) using the quantity of interest as data:

$$a(w, v) = Q(v), \quad \forall v \in \mathcal{V}. \quad (8.202)$$

The solution, w , is referred to as the influence function for a given choice of Q . Solving both the primal and dual problems discretely, we obtain approximations u^h and w^h . A goal-oriented error estimate is typically chosen as the difference between the quantities of interest applied to these two approximations:

$$\theta \approx Q(w^h) - Q(u^h). \quad (8.203)$$

The effectivity index is now computed as $\eta = \frac{\theta}{|Q(w^h) - Q(u^h)|}$. A variety of techniques can then be used to localize the estimate, similar to the residual-based and recovery-based methods described above for approximating the error in the solution or gradient.

In the end, whether one cares about estimating the error in the solution, the gradient, or some other quantity of interest, a posteriori error estimates allow for modifications to the solution process that can lead to better numerical approximations. Many marking strategies and refinement techniques for different finite-element methods on computational grids in multiple dimensions are specialized for a wide number of application areas. Adaptive techniques are a rich area of study and are crucial for efficiently solving PDEs.

8.7 ■ Finite-element methods for time-dependent PDEs

Objectives

In this section, you will see how the finite-element framework from the previous sections is extended to time-dependent PDEs using the method-of-lines approach.

To conclude this chapter, we briefly explain how the finite-element method can be applied to time-dependent PDEs. We illustrate the approach for the heat equation with the goal of finding $u(\mathbf{x}, t)$ that solves

$$\frac{\partial u}{\partial t} - \nabla \cdot K \nabla u = f(\mathbf{x}, t) \quad \text{for } \mathbf{x} \in \Omega, t > 0, \quad (8.204)$$

with boundary conditions $u = 0$ on $\partial\Omega$, initial condition $u(\mathbf{x}, 0) = u_0(\mathbf{x})$ at $t = 0$, and $f(\cdot, t) \in L^2(\Omega)$ for all t . We first multiply equation (8.204) by a test function, $v(\mathbf{x}) \in \mathcal{V} = H_0^1(\Omega)$, and integrate over spatial domain Ω , applying integration by parts to obtain the weak form: find $u(\mathbf{x}, t) \in \mathcal{V}$ such that

$$\int_{\Omega} \frac{\partial u}{\partial t} v \, d\mathbf{x} + \int_{\Omega} K \nabla u \cdot \nabla v \, d\mathbf{x} = \int_{\Omega} f v \, d\mathbf{x} \quad \text{for all } v \in \mathcal{V}. \quad (8.205)$$

The semidiscrete Galerkin formulation for a finite-dimensional subspace $\mathcal{V}^h$ of $\mathcal{V}$ is then given by finding, for all times t , $u^h(\mathbf{x}, t) \in \mathcal{V}^h \subset H_0^1(\Omega)$ such that

$$\int_{\Omega} \frac{\partial u^h}{\partial t} v^h \, d\mathbf{x} + \int_{\Omega} K \nabla u^h \cdot \nabla v^h \, d\mathbf{x} = \int_{\Omega} f v^h \, d\mathbf{x} \quad \text{for all } v^h \in \mathcal{V}^h. \quad (8.206)$$

Expressing $u^h(\mathbf{x}, t)$ in the spatial basis $\{\phi_1, \dots, \phi_n\}$ of $\mathcal{V}^h$ as

$$u^h(\mathbf{x}, t) = \sum_{j=1}^n u_j(t) \phi_j(\mathbf{x}), \quad (8.207)$$

we finally obtain the spatially discretized Galerkin form

$$\begin{aligned} \sum_{j=1}^n \frac{du_j(t)}{dt} \int_{\Omega} \phi_j(\mathbf{x}) \phi_i(\mathbf{x}) \, d\mathbf{x} + u_j(t) \int_{\Omega} K \nabla \phi_j(\mathbf{x}) \cdot \nabla \phi_i(\mathbf{x}) \, d\mathbf{x} \\ = \int_{\Omega} f(\mathbf{x}, t) \phi_i(\mathbf{x}) \, d\mathbf{x} \quad \text{for } i = 1, \dots, n. \end{aligned} \quad (8.208)$$

Defining the *mass matrix* M , the *stiffness matrix* A , and the right-hand side vector $\mathbf{f}(t)$ by

$$m_{i,j} = \langle \phi_j(\mathbf{x}), \phi_i(\mathbf{x}) \rangle, \quad (8.209a)$$

$$a_{i,j} = \langle K \nabla \phi_j(\mathbf{x}), \nabla \phi_i(\mathbf{x}) \rangle, \quad (8.209b)$$

$$f_i(t) = \langle f(\mathbf{x}, t), \phi_i(\mathbf{x}) \rangle, \quad (8.209c)$$

we write equation (8.208) in matrix form as

$$M \frac{d\mathbf{u}(t)}{dt} + A \mathbf{u}(t) = \mathbf{f}(t), \quad (8.210)$$

where $\mathbf{u}(t)$ represents the vector of coefficients for the finite-element solution at each timestep. This system of ODEs can be integrated in time by a suitable numerical ODE method in the method-of-lines framework, as discussed in Sections 6.5 and 9.6.

Example: implicit timestepping

As an example, we consider the finite-element discretization of equation (8.204) on $\Omega = [0, 1]^2$, with $K := 0.02$ and $f(\mathbf{x}, t) = 0$. We take $\mathcal{V}^h = Q_1(\Omega^h)$ (see Section 8.3.2), where $\mathcal{T}^h$ is given by a uniform $n_x \times n_y$ grid with $n_x = n_y$, so that $\Omega^h = \Omega$. The initial data is given by $u_0(x, y) = \sin(3\pi x) \sin(3\pi y)$, which results in an exact solution given by

$$u(x, y, t) = e^{-0.36\pi^2 t} \sin(3\pi x) \sin(3\pi y). \quad (8.211)$$

We define the discrete initial data, $\mathbf{u}_0$, as the finite-element interpolant of $u_0(x, y)$ in $\mathcal{V}^h$.

We consider two timestepping schemes for equation (8.210): implicit Euler and Crank–Nicolson. Using the finite-difference 1D schemes as a guide (see equation (5.10)), the implicit Euler discretization becomes

$$M \frac{\mathbf{u}_{\ell+1} - \mathbf{u}_\ell}{h_t} + A \mathbf{u}_{\ell+1} = 0 \quad (8.212a)$$

$$\Rightarrow (M + h_t A) \mathbf{u}_{\ell+1} = M \mathbf{u}_\ell, \quad (8.212b)$$

where M and A are defined by equation (8.209). From this, we see that each step requires solving a system of linear equations with the matrix $M + h_t A$. Similarly, the Crank–Nicolson discretization becomes

$$M \frac{\mathbf{u}_{\ell+1} - \mathbf{u}_\ell}{h_t} + A \frac{\mathbf{u}_{\ell+1} + \mathbf{u}_\ell}{2} = 0 \quad (8.213a)$$

$$\Rightarrow \left(M + \frac{h_t}{2} A \right) \mathbf{u}_{\ell+1} = \left(M - \frac{h_t}{2} A \right) \mathbf{u}_\ell. \quad (8.213b)$$

In this case, the system solve is with the matrix $M + \frac{h_t}{2} A$.

For this example, we look at the L^2 norm of the error at a given time as $h_t \rightarrow 0$ and $h_x, h_y \rightarrow 0$, holding the ratio constant by setting $h_t = h_x = h_y$. At each step, the linear system is solved with algebraic multigrid (see Section 11.3) to a relative residual tolerance of 10^{-12} . Figure 8.7.1 shows the L^2 error, $\|u(\mathbf{x}, T) - u^h(\mathbf{x}, T)\|$, at $T = 1.0$ (mesh size is calculated as $h = h_x = h_y$). Here, we see a clear trend toward first-order convergence for implicit Euler and second-order convergence for Crank–Nicolson as the mesh is refined, since the spatial discretization has

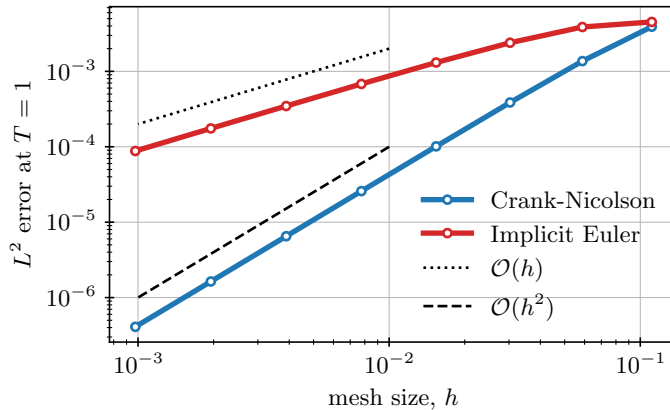


Figure 8.7.1. Convergence of implicit Euler and Crank–Nicolson schemes for a $Q_1(\Omega^h)$ finite-element discretization of the heat equation on uniform $n_x \times n_y$ meshes with $n_x = n_y$ and $h = h_t = h_x = h_y$. The error is calculated at $T = 1.0$.

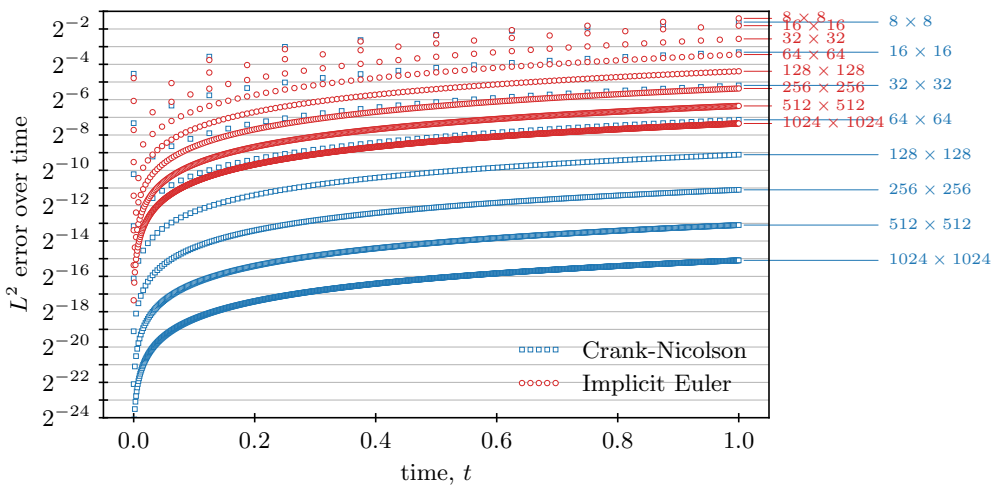


Figure 8.7.2. Error history (at each timestep) of implicit Euler and Crank–Nicolson schemes for a $Q_1(\Omega^h)$ finite-element discretization of the heat equation on uniform $n_x \times n_y$ meshes.

second-order accuracy in the L^2 norm. Figure 8.7.2 tracks the L^2 error at each timestep for each mesh size. As expected, the error grows over time, yet we observe a clear advantage of Crank–Nicolson over implicit Euler.

8.8 ■ Some further thoughts

In this chapter, we extended the results of Chapter 4 to the more general setting of PDEs in higher dimensions. While the basic principles were the same, a deeper review of functional analysis was required in order to properly present the methodology. For a more in-depth look at this analysis, there are many texts that present the details of finite-element methods from a variety of viewpoints — e.g., [43, 52, 48, 72] are all *essentials* on any finite-element bookshelf.

The literature on the analysis of finite-element methods is now extensive, but the success of finite-element approximations in practice is also attributed to significant advances in recent years in the algorithms and software to make these methods efficient and effective. There are many well-established software packages that enable straightforward use of finite elements. An incomplete list, based on programming language, includes

Python: Firedrake [122], FEniCS [11, 158], and DUNE [36];

C/C++: deal.II [15], MFEM [12], HAZmath [4], and FreeFEM [128];

MATLAB: iFEM [68].

Each of these allows a user to build discrete linear systems based on weak formulations, choosing from a variety of built-in finite-element spaces and corresponding bases. Many can easily be linked to existing solver packages for linear and nonlinear systems of algebraic equations or have their own solvers as well. Of course, there are a vast number of other finite-element packages, and some have a focus on specific settings, such as industrial applications.

Additionally, it should be noted that we have taken a specific view in this chapter, with a focus on Ritz–Galerkin-type finite elements, where the bases are local, compactly supported polynomial functions such as in Figures 8.5.3 and 8.5.4. However, another approach is to use higher-order functions instead, whether it be higher-order piecewise polynomials or other smooth, but

global, basis functions. Examples of the latter include spectral methods [110, 132, 217], which employ trigonometric functions as the basis, such as in Fourier series. We look at this idea more in Section 10.6, though there are many variants, such as pseudospectral and collocation methods. Additionally, isogeometric analysis [138] uses globally supported smooth basis functions, such as splines. In any case, each of these approaches tends to yield high-accuracy solutions with exponential convergence, assuming the solution to the PDE is smooth. The drawback is that due to the global support of the basis functions, the resulting discrete linear systems tend to be dense. This has significant implications in cases where the solution to a linear system is required (cf. Chapter 7), but also in the efficiency of the matrix assembly process. In the case of a standard Galerkin approach, the matrix equations associated with the bilinear forms can be assembled locally over each element, whereas globally supported basis functions often have degrees of freedom that interact with most other nodes, resulting in a more complex assembly. In this case, sum factorization techniques [169] may be employed in order to decompose the assembly of the stiffness matrix over each element more efficiently.

Finally, while all of the examples in this chapter considered polygonal domains, many applications require simulations on meshes with curved boundaries. Just as in Section 3.6 using curvilinear coordinates for finite-difference schemes, in the finite-element setting similar mappings are needed. This leads to *curvilinear elements*, where we represent the transformation from the reference element, $F_\tau(\hat{x})$, in Equation (8.167) by a higher-order finite-element basis. This is particularly advantageous when the basis functions used for the curvilinear mapping match those used for the finite-element approximation of the solution, known as using *isoparametric elements* [72, Section 4.3]. As an example, see Figure 8.8.1 where linear elements do not adequately represent the boundary of the unit disk, whereas quadratic curvilinear elements lead to a close fit.

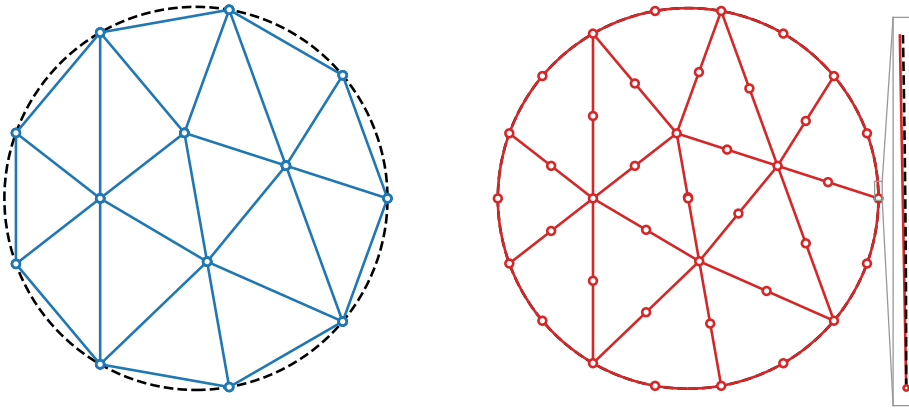


Figure 8.8.1. Meshing of the unit disk with 15 (left) linear and (right) quadratic curvilinear elements.

8.9 ■ Exercises

1. Consider the space of continuous functions on the interval $0 \leq x \leq 1$, $C^0([0, 1])$.
 - (a) Show that $\langle u, v \rangle = \int_0^1 u(x)v(x) dx$ is an inner product on this space.
 - (b) Show that the functional $f(u) = \int_0^{1/2} u(x) dx$ is a bounded linear functional on $C^0([0, 1])$.
 - (c) Does there exist a function $\phi \in C^0([0, 1])$ such that $f(u) = \langle \phi, u \rangle$ for all $u \in C^0([0, 1])$? If so, give the function. If not, explain why this does not contradict the Riesz representation theorem.

2. Let $a(\cdot, \cdot)$ be a coercive and continuous bilinear form on a Hilbert space, $\mathcal{V}$. Further assume that a is symmetric (meaning that $a(u, v) = a(v, u)$ for all $u, v \in \mathcal{V}$) and that $F(v)$ is a bounded linear functional on $\mathcal{V}$. Prove that, for any closed subspace, $\mathcal{U}$, of $\mathcal{V}$, $u \in \mathcal{U}$ satisfies $a(u, v) = F(v)$ for all $v \in \mathcal{U}$ if and only if u minimizes $g(w) = \frac{1}{2}a(w, w) - F(w)$ over all $w \in \mathcal{U}$. Does the same conclusion hold if a is not symmetric?

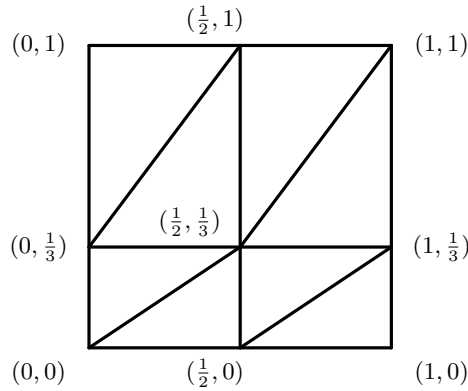
Hint: First explain the relationship between minimizers of $g(w)$ and those of the functions $g_v(x) = g(u + xv)$ for $x \in \mathbb{R}$ and $u, v \in \mathcal{U}$.

3. Let A be a symmetric and positive-definite $n \times n$ matrix.
- Show that the functional $a(\mathbf{u}, \mathbf{v}) = \mathbf{v}^T A \mathbf{u}$ is coercive and continuous with respect to the Euclidean norm in $\mathbb{R}^n$.
 - Explain how we can use part (a), along with the Lax–Milgram theorem, to say that $A\mathbf{u} = \mathbf{f}$ has a unique solution for any $\mathbf{f} \in \mathbb{R}^n$.
4. Consider

$$\begin{aligned} -u'' + u' + u &= f \quad \text{for } 0 < x < 1, \\ u(0) &= u(1) = 0, \end{aligned}$$

and $f \in L^2([0, 1])$.

- Find the weak form of this equation.
 - Show that the bilinear form on the left-hand side of the weak form is elliptic in $H_0^1([0, 1])$, with appropriate norm. *Hint:* Recall that for any real numbers, a and b , $ab \leq \frac{1}{2}(a^2 + b^2)$.
 - What can you conclude about weak solutions of this equation?
 - Can you conclude anything about the solution of the Ritz–Galerkin projection onto a finite-dimensional subset of $H_0^1([0, 1])$?
5. Consider the use of a piecewise linear approximation on a triangular grid of the unit square in two dimensions, where the grid has the additional property that each triangle has two sides parallel to the x - and y -axes. Such a mesh can be formed by forming a rectangular grid as the tensor product of two one-dimensional grids, $\{x_i\}_{i=0}^{n_x}$ and $\{y_j\}_{j=0}^{n_y}$, and then cutting each rectangle into two triangles. There are two possible “orientations” of triangles in such a grid; here, consider those with right angles in the lower-right and upper-left corners of the rectangles, which are cut from their top-right to bottom-left corners.
- Consider two generic elements, one, τ^1 , with corners (x_i, y_{j+1}) , (x_{i+1}, y_{j+1}) , and (x_i, y_j) , and the other, τ^2 , with corners (x_i, y_j) , (x_{i+1}, y_{j+1}) , and (x_{i+1}, y_j) . On each of these elements, write down the nodal basis functions associated with each of the three corners of the element.
 - Compute the *element stiffness matrices*, A^1 and A^2 , for each of these elements, τ^1 and τ^2 , corresponding to the weak form $a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x}$. If the three nodal basis functions are $\phi_i^{\tau^k}(x, y)$ for $i = 1, 2, 3$ and each element τ^k , then A^k is the 3×3 matrix with entries $(A^k)_{i,j} = \int_{\tau^k} \nabla \phi_j^{\tau^k} \cdot \nabla \phi_i^{\tau^k} \, d\mathbf{x}$.
 - Use the element stiffness matrices computed above to assemble the full stiffness matrix for the bilinear form $a(u, v)$ on the grid shown in the figure, with x -coordinates of the nodes given by $\{0, 1/2, 1\}$ and y -coordinates given by $\{0, 1/3, 1\}$. Assume homogeneous Neumann boundary conditions, so that there are no adjustments to the boundaries beyond the values given by the integrals.



- (d) Verify that your full stiffness matrix exactly matches the value of the bilinear form, $a(u, u)$, whenever u is a linear function, by evaluating u at the nodes to form vector $\mathbf{u}$ and verifying that $a(u, u) = \mathbf{u}^T \mathbf{A} \mathbf{u}$.
6. Consider the finite-element discretization of $-\nabla \cdot \nabla u = g(x, y)$ on a tensor-product grid of the unit square, with homogeneous Dirichlet boundary conditions on all four edges. Use a uniform grid with n_x intervals in the x -direction and n_y intervals in the y -direction.
- (a) Instead of using a triangular grid, consider the quadrilateral mesh defined by the tensor product of uniform grids Ω_x and Ω_y . For every node, (x_i, y_j) , define a basis function, $\phi_{i,j}(x, y) = \phi_i(x)\psi_j(y)$, where $\phi_i(x)$ is the one-dimensional linear basis function for node x_i on Ω_x and $\psi_j(y)$ is the one-dimensional linear basis function for node y_j on Ω_y . Write a piecewise definition of $\phi_{i,j}(x, y)$. We call this a *bilinear basis function*.
- (b) Show that the finite-element discretization matrix, A , for $-\nabla \cdot \nabla u = g(x, y)$ can be written in terms of Kronecker products of the one-dimensional *mass* and *stiffness* matrices

$$\begin{aligned}
 M^{(x)} \text{ with entries } m_{i,k}^{(x)} &= \int_0^1 \phi_k(x)\phi_i(x) dx, \\
 M^{(y)} \text{ with entries } m_{j,\ell}^{(y)} &= \int_0^1 \psi_\ell(y)\psi_j(y) dy, \\
 S^{(x)} \text{ with entries } s_{i,k}^{(x)} &= \int_0^1 \phi'_k(x)\phi'_i(x) dx, \text{ and} \\
 S^{(y)} \text{ with entries } s_{j,\ell}^{(y)} &= \int_0^1 \psi'_\ell(y)\psi'_j(y) dy.
 \end{aligned}$$

Hint: It is natural to think of entries in A in terms of double indices, with $a_{(i,j),(k,\ell)} = \int_{[0,1]^2} \nabla \phi_{k,\ell}(x, y) \cdot \nabla \phi_{i,j}(x, y) d\mathbf{x}$. You should be able to easily express $a_{(i,j),(k,\ell)}$ in terms of the entries above. The use of Kronecker products flattens this two-dimensional indexing into one-dimensional lexicographic ordering.

7. Consider the unforced heat equation on a polygonal domain, Ω :

$$\begin{aligned}
 \frac{\partial u}{\partial t} - \nabla \cdot K \nabla u &= 0 \text{ for } \mathbf{x} \in \Omega, t > 0, \\
 u(\mathbf{x}, 0) &= u_0(\mathbf{x}) \text{ for } \mathbf{x} \in \Omega, \\
 u(\mathbf{x}, t) &= 0 \text{ for } \mathbf{x} \in \partial\Omega, t > 0.
 \end{aligned}$$

- (a) Derive a *dissipation law* for a smooth solution, $u(\mathbf{x}, t)$, by evaluating the weak form in equation (8.205) with $v = u$. Use the Leibniz integration rule to express this in terms of $\frac{\partial}{\partial t} \|u(\cdot, t)\|^2$.
- (b) Explain why the semidiscrete approximation, $u^h(\mathbf{x}, t)$, defined by equation (8.206) satisfies the same dissipation law for any $\mathcal{V}^h \subset H_0^1(\Omega)$ (assuming that $\Omega^h = \Omega$). Express $\|u^h(\cdot, t)\|^2$ and its time derivative in terms of the mass and/or stiffness matrices.
- (c) Consider the fully discrete approximation using the Crank–Nicolson (implicit trapezoid) integrator that satisfies $\frac{1}{h_t} M(\mathbf{u}_{\ell+1} - \mathbf{u}_\ell) + \frac{1}{2} A(\mathbf{u}_{\ell+1} + \mathbf{u}_\ell) = \mathbf{0}$. Show that this satisfies a discretized dissipation law for $\|u^h(\cdot, t_\ell)\|^2$, the squared norm of the fully discrete solution at time t_ℓ , by proving that $\|u^h(\cdot, t_{\ell+1})\|^2 \leq \|u^h(\cdot, t_\ell)\|^2$ for all ℓ .

Chapter 9

Finite-volume methods for systems of conservation laws

Overview

In this chapter, we expand the discussion of finite-volume methods from Chapter 6 to the case of nonlinear hyperbolic systems in multiple spatial dimensions. We consider conservative systems of p coupled PDEs of the form

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot \mathbb{F}(\mathbf{u}) = \mathbf{0}, \quad (9.1)$$

where the conserved quantity $\mathbf{u} \in \mathbb{R}^p$ and, in 3D, the flux tensor $\mathbb{F}(\mathbf{u}) = [\mathbf{f}(\mathbf{u}), \mathbf{g}(\mathbf{u}), \mathbf{h}(\mathbf{u})] \in \mathbb{R}^{p \times 3}$ (the divergence operator acts rowwise on $\mathbb{F}$). Such equations have broad applications in many areas of science and engineering, e.g., as fluid dynamic models that describe aerodynamic or space physics flows.

Objectives

In this chapter, you will use the building blocks from Chapter 6 to derive robust methods for multidimensional hyperbolic systems that can handle discontinuities while controlling spurious oscillations. You will also consider two more-advanced high-order discretization techniques that go beyond the second-order accuracy attained in Chapter 6 and will be introduced to implicit time-integration methods for so-called *stiff* problems with disparate timescales. Using the techniques from this chapter, you will be able to

- observe the formation of shocks and identify the propagation of information in hyperbolic systems that describe physical systems such as compressible fluids and plasmas;
- extend finite-volume methods of first- and second-order accuracy to hyperbolic systems and multiple dimensions; and
- gain insight into and apply two advanced high-order methods for hyperbolic conservation laws: essentially nonoscillatory (ENO) methods and discontinuous Galerkin (DG) methods.

Chapter Outline

Section 9.1 Systems of hyperbolic conservation laws considers the theory of systems of hyperbolic conservation laws, discussing the properties of waves and shocks that inform the derivation of numerical methods. We introduce the Euler equations for compressible gas dynamics as an example of a nonlinear hyperbolic system.

Section 9.2 Finite-volume methods for systems of equations extends the scalar finite-volume methods from Chapter 6 to hyperbolic systems. The Lax–Friedrichs and Roe numerical flux functions will be presented and analyzed.

Section 9.3 Multidimensional problems extends the finite-volume approach to multiple spatial dimensions using the divergence theorem. We also consider boundary conditions and their numerical implementation.

Section 9.4 Essentially nonoscillatory finite-volume methods introduces the high-order accurate essentially nonoscillatory (ENO) and weighted essentially nonoscillatory (WENO) methods for hyperbolic conservation laws, providing a full derivation of the approach based on high-degree polynomial reconstruction in combination with high-order time integration that preserves stability in total variation.

Section 9.5 Discontinuous Galerkin schemes for hyperbolic conservation laws presents discontinuous Galerkin (DG) methods, another high-order discretization technique. These methods use finite element–type basis functions that are discontinuous at element interfaces and build on a weak formulation that employs numerical fluxes at those interfaces.

Section 9.6 Implicit time-integration for nonlinear PDEs discusses how problems with multiple disparate timescales may benefit from implicit time integration strategies, with stability properties that allow larger timesteps but require the solution of large systems of equations at each timestep.

9.1 ■ Systems of hyperbolic conservation laws

Objectives

The goal of this section is to describe several important theoretical properties of nonlinear hyperbolic systems that extend the properties of scalar conservation laws discussed in Section 6.1 and inform the derivation of numerical methods in the next sections. In particular, you will define hyperbolic systems of conservation laws and learn about their properties. Next, you will consider linear hyperbolic systems and construct solutions to linear Riemann problems. Finally, you will discuss nonlinear examples of the “shallow-water” and Euler equations, including properties of their wave and shock solutions.

Up to this point, we have only considered 1D scalar conservation laws, as in equation (6.2). However, most conservation law problems that arise in real-life applications feature coupled first-order systems of conservation laws in two or three spatial dimensions. First-order systems of conservation laws in 3D are described by

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot \mathbb{F}(\mathbf{u}) = \mathbf{0}, \quad (9.2)$$

where $\mathbb{F}(\mathbf{u}) = [\mathbf{f}(\mathbf{u}), \mathbf{g}(\mathbf{u}), \mathbf{h}(\mathbf{u})]$ and $\mathbf{u}$, $\mathbf{f}$, $\mathbf{g}$, and $\mathbf{h} \in \mathbb{R}^p$. In this notation, the divergence operator is taken to act rowwise on the $p \times 3$ tensor $\mathbb{F}$. Vector $\mathbf{u}$ is the vector of p conserved quantities, and $\mathbb{F}$ is the flux tensor, which is assumed to be a differentiable function of $\mathbf{u}$.

We initially focus on the properties of hyperbolic systems of conservation laws in one spatial dimension and derive numerical methods of finite-volume type for the 1D case, using the approaches that were derived for scalar 1D conservation laws in Chapter 6. Later, in Section 9.3, we extend these methods to spatial dimensions higher than one. Thus, we first consider the

system of conservation laws

$$\frac{\partial \mathbf{u}}{\partial t} + \frac{\partial \mathbf{f}(\mathbf{u})}{\partial x} = \mathbf{0}, \quad (9.3)$$

with quasi-linear form

$$\frac{\partial \mathbf{u}}{\partial t} + A(\mathbf{u}) \frac{\partial \mathbf{u}}{\partial x} = \mathbf{0}, \quad (9.4)$$

where

$$A(\mathbf{u}) = \frac{\partial \mathbf{f}(\mathbf{u})}{\partial \mathbf{u}} \in \mathbb{R}^{p \times p} \quad (9.5)$$

is the Jacobian matrix of the flux vector $\mathbf{f}(\mathbf{u})$.

Definition 9.1: Hyperbolic system of conservation laws [150, Sec. 6.4]

The PDE system in the conservation law form of equation (9.3) is called hyperbolic if the Jacobian of the flux, $A(\mathbf{u}) \in \mathbb{R}^{p \times p}$, has p real eigenvalues and a full set of p linearly independent eigenvectors.

In the following section, we relate the p real eigenvalues of $A(\mathbf{u})$ to p wave speeds and to p families of real characteristic curves, thus generalizing many of the properties of scalar conservation laws that were discussed in Chapter 6. For example, we extend the Rankine–Hugoniot relation of equation (6.17) for the propagation speed of jump discontinuities to hyperbolic systems. First, though, we discuss the case of linear hyperbolic systems, which are a systems generalization of linear advection, as in equation (6.52).

9.1.1 ■ Linear hyperbolic systems in 1D

In the case of a linear hyperbolic system, the flux function, $\mathbf{f}(\mathbf{u})$ in equation (9.3), is given by

$$\mathbf{f}(\mathbf{u}) = A\mathbf{u}, \quad (9.6)$$

with $A \in \mathbb{R}^{p \times p}$, giving conservation law

$$\frac{\partial \mathbf{u}}{\partial t} + A \frac{\partial \mathbf{u}}{\partial x} = \mathbf{0}. \quad (9.7)$$

Since the system is hyperbolic, A is diagonalizable with real eigenvalues according to Definition 9.1,

$$A R = R \Lambda, \quad (9.8)$$

where R and Λ are defined in terms of the p eigenvalues and p linearly independent eigenvectors of A as

$$R = \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \cdots & \mathbf{r}_{p-1} & \mathbf{r}_p \end{bmatrix}, \quad \Lambda = \begin{bmatrix} \lambda_1 & 0 & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & \cdots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & \cdots & 0 & \lambda_{p-1} & 0 \\ 0 & \cdots & 0 & 0 & \lambda_p \end{bmatrix}. \quad (9.9)$$

Since R contains the linearly independent eigenvectors of A , we define

$$L = R^{-1} \quad (9.10)$$

and rewrite (9.7) as

$$L \frac{\partial \mathbf{u}}{\partial t} + \Lambda L \frac{\partial \mathbf{u}}{\partial x} = \mathbf{0}. \quad (9.11)$$

This suggests a change of variables,

$$\mathbf{w} = L\mathbf{u}, \quad (9.12)$$

which fully diagonalizes the system in the new $\mathbf{w}$ variables:

$$\frac{\partial \mathbf{w}}{\partial t} + \Lambda \frac{\partial \mathbf{w}}{\partial x} = \mathbf{0}. \quad (9.13)$$

We then see immediately that a linear hyperbolic system corresponds to p fully decoupled linear advection equations:

$$\frac{\partial w_i(x, t)}{\partial t} + \lambda_i \frac{\partial w_i(x, t)}{\partial x} = 0, \quad i = 1, \dots, p. \quad (9.14)$$

This means that the solution of a linear hyperbolic system initial-value problem is known explicitly (following the scalar case discussed in Section 1.2.2), with

$$w_i(x, t) = w_i^{(0)}(x - \lambda_i t), \quad i = 1, \dots, p, \quad (9.15)$$

where the functions $w_i^{(0)}(x)$ are determined by expanding the initial conditions for $\mathbf{u}(x, 0)$ into the eigenbasis, with $\mathbf{w}^{(0)}(x) = L\mathbf{u}(x, 0)$. The w_i variables are called the *characteristic variables*, and the solution profile of each w_i is advected with wave speed λ_i . The p wave speeds, λ_i , define p families of straight characteristic curves with slope λ_i in the x - t plane, given by $x_i(t) = x_0 + \lambda_i t$. The p families each correspond to the single family of characteristic curves in Figure 1.2.1 in the scalar case.

Since $\mathbf{u} = R\mathbf{w}$, we express the solution of the PDE system in the original variables as

$$\mathbf{u}(x, t) = R\mathbf{w}(x, t) = \sum_{i=1}^p \mathbf{r}_i w_i^{(0)}(x - \lambda_i t), \quad (9.16)$$

where the $\mathbf{r}_i$ are the columns of R (the right eigenvectors of A). Equation (9.16) illustrates the important role the right eigenvector basis of A plays in the structure of solutions of linear hyperbolic systems.

Riemann problem for linear hyperbolic systems

It is instructive to consider the solution of Riemann problems for systems of conservation laws, similar to the case of Riemann problem solutions for scalar conservation law (6.2) as in Figure 6.3.1. In Example 9.2, we give a Riemann problem for a linear hyperbolic system and detail the solution using a change of variables. In subsequent sections, we discuss Riemann problems for nonlinear systems and explain how they are used as building blocks in finite-volume methods, as we saw in Chapter 6.

Example 9.2: Riemann problem for a linear hyperbolic system

Consider linear conservation law system (9.7) with

$$A = \begin{bmatrix} 1 & 3 \\ 3 & 1 \end{bmatrix} \quad (9.17)$$

and Riemann problem initial condition

$$\mathbf{u}(x, 0) = \begin{cases} \mathbf{u}^- & \text{if } x < 0, \\ \mathbf{u}^+ & \text{if } x > 0, \end{cases} \quad (9.18)$$

with

$$\mathbf{u}^- = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad \mathbf{u}^+ = \begin{bmatrix} 1 \\ -3 \end{bmatrix}. \quad (9.19)$$

We compute the solution of the Riemann problem as follows. First, we find the eigendecomposition $A = R\Lambda L$:

$$R = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \Lambda = \begin{bmatrix} 4 & 0 \\ 0 & -2 \end{bmatrix}, \quad L = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}. \quad (9.20)$$

With this, the system decouples into two independent scalar linear advection equations (cf. equation (9.14)) for the characteristic variables $\mathbf{w} = L\mathbf{u}$, with jumps in the two components of $\mathbf{w}$ at $(x, t) = (0, 0)$. This yields two shocks, in w_1 and w_2 , that propagate with speeds λ_1 and λ_2 , respectively. Thus, according to equation (9.16), we expect the initial jump in the original variables,

$$[\mathbf{u}] := \mathbf{u}^+ - \mathbf{u}^- = \begin{bmatrix} -1 \\ -3 \end{bmatrix}, \quad (9.21)$$

to split into two shocks with jump vectors proportional to the right eigenvectors of A . Using $\mathbf{w} = L\mathbf{u}$, the left and right states in the characteristic variables are expressed as

$$\mathbf{w}^- = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{w}^+ = \begin{bmatrix} -1 \\ 2 \end{bmatrix}. \quad (9.22)$$

Similarly, the initial jumps in the characteristic variables are computed from $\mathbf{w} = L\mathbf{u}$ as

$$[\mathbf{w}] = L[\mathbf{u}] = \begin{bmatrix} -2 \\ 1 \end{bmatrix}. \quad (9.23)$$

As a result, the initial jump in w_1 with $[w_1] = -2$ propagates to the right with speed $\lambda_1 = 4$, and the initial jump in w_2 with $[w_2] = 1$ propagates to the left with speed $\lambda_2 = -2$ (see Figure 9.1.1).

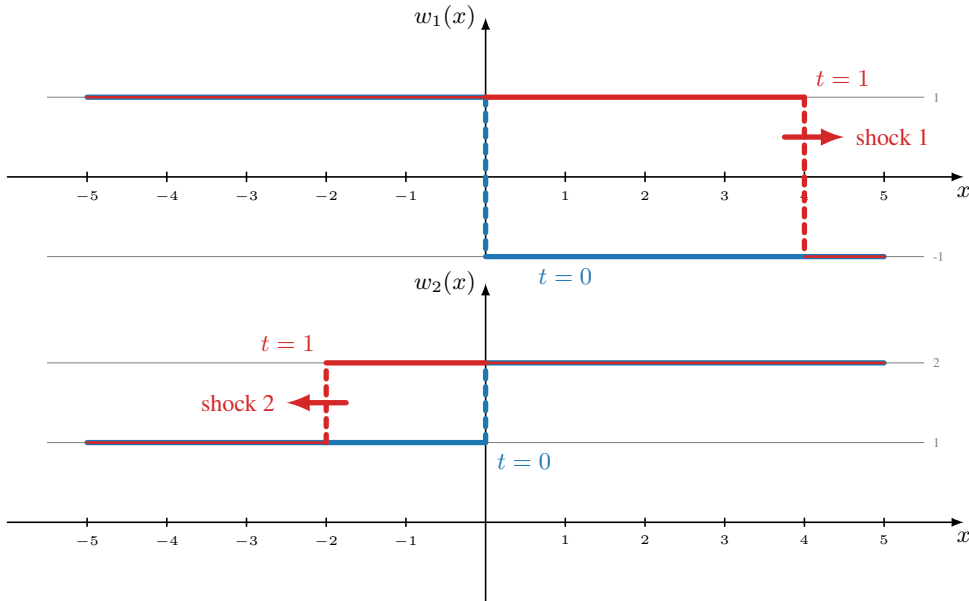


Figure 9.1.1. Solution $\mathbf{w}$ (top: w_1 ; bottom: w_2) and shock propagation for the Riemann problem of Example 9.2. Blue indicates the solution at $t = 0$ and red indicates $t = 1$.

This translates to two shocks in the original $\mathbf{u}$ variables propagating with speeds λ_1 and λ_2 as follows:

$$[\mathbf{u}] = R[\mathbf{w}] = \sum_{i=1}^p \mathbf{r}_i[w_i] = \begin{bmatrix} 1 \\ 1 \end{bmatrix} (-2) + \begin{bmatrix} 1 \\ -1 \end{bmatrix} 1 = \sum_{i=1}^p [\mathbf{u}_i]. \quad (9.24)$$

So, in the original variables, we have a first shock with jump

$$[\mathbf{u}_1] = \begin{bmatrix} -2 \\ -2 \end{bmatrix} \quad (9.25)$$

that propagates to the right with speed $\lambda_1 = 4$, and a second shock with jump

$$[\mathbf{u}_2] = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad (9.26)$$

that propagates to the left with speed $\lambda_2 = -2$ (see Figure 9.1.2).

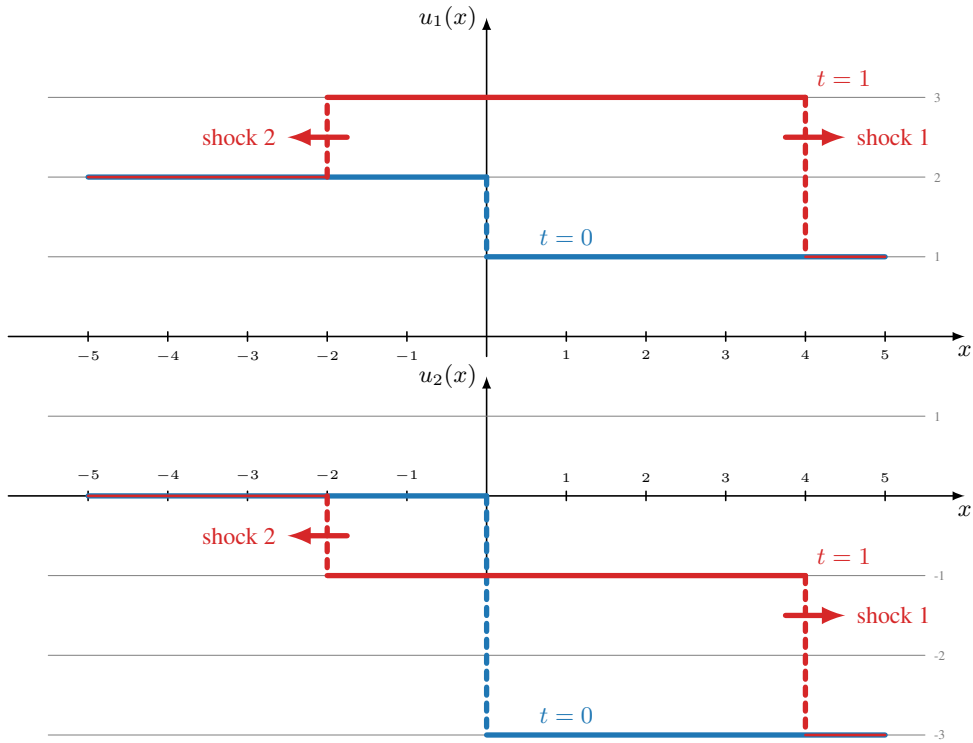
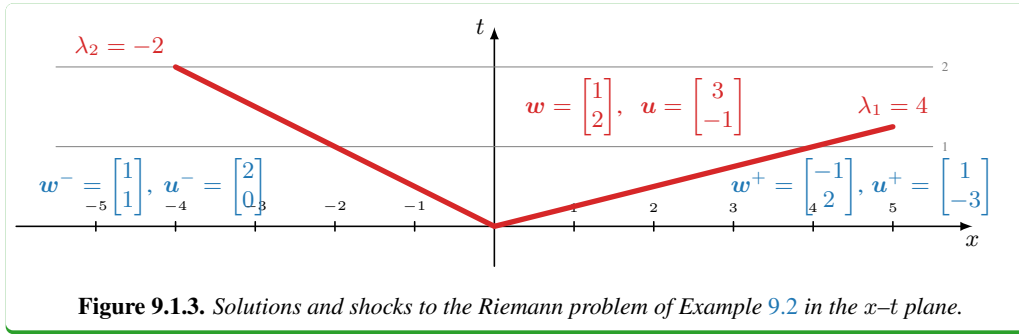


Figure 9.1.2. Solution $\mathbf{u}$ (top: u_1 ; bottom: u_2) and shock propagation for the Riemann problem of Example 9.2. Blue indicates the solution at $t = 0$ and red indicates $t = 1$.

In summary, the solution of a linear Riemann problem with $A \in \mathbb{R}^{p \times p}$ consists of p shocks that propagate with speeds λ_i ($i = 1, \dots, p$). The jumps of the p shocks are determined by decomposing $[\mathbf{u}]$ into the right eigenspace of A , $[\mathbf{u}] = R L [\mathbf{u}] = R[\mathbf{w}] = \sum_{i=1}^p \mathbf{r}_i[w_i] = \sum_{i=1}^p [\mathbf{u}_i]$. Figure 9.1.3 shows the solution of the Riemann problem in the $x-t$ plane for both the initial conditions and the solution at $x = 0$ for $t > 0$. In what follows, we may use Riemann problem solutions at $x = 0$ to determine numerical flux functions in finite-volume methods.



Classification of linear second-order PDEs

Next, we revisit the classification of scalar linear second-order PDEs in the context of our discussion on first-order systems. Consider the scalar second-order PDE

$$a u_{tt} + b u_{xt} + c u_{xx} + d u_t + e u_x = 0 \quad (a \neq 0), \quad (9.27)$$

and write it as a first-order system: define $v = u_t$ and $w = u_x$ such that $u_{tt} = v_t$, $u_{xt} = v_x$, and $u_{xx} = w_x$. Then,

$$v_t = u_{tt} = -(b u_{xt} + c u_{xx} + d u_t + e u_x)/a \quad (9.28a)$$

$$= -(b v_x + c w_x + d v + e w)/a, \quad (9.28b)$$

$$w_t = u_{xt} = v_x. \quad (9.28c)$$

This gives the first-order system

$$\frac{\partial \mathbf{u}}{\partial t} + A \frac{\partial \mathbf{u}}{\partial x} + B \mathbf{u} = 0, \quad (9.29)$$

with

$$\mathbf{u} = \begin{bmatrix} v \\ w \end{bmatrix}, \quad A = \begin{bmatrix} b/a & c/a \\ -1 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} d/a & e/a \\ 0 & 0 \end{bmatrix}. \quad (9.30)$$

The eigenvalues of A are given by

$$\lambda_{1,2} = \frac{b \pm \sqrt{b^2 - 4ac}}{2a}. \quad (9.31)$$

Then, extending Definition 9.1 to this setting, we say that the first-order PDE system in equation (9.29) and, thus, the corresponding scalar second-order PDE in equation (9.27), are

$$\begin{aligned} \text{hyperbolic} &\iff b^2 - 4ac > 0 \\ &\iff \lambda_{1,2} \text{ real, with a complete eigenvector set} \\ &\iff \text{two real characteristic curves exist;} \\ \text{parabolic} &\iff b^2 - 4ac = 0 \\ &\iff \lambda_{1,2} \text{ are real, but eigenvector set is incomplete;} \\ \text{elliptic} &\iff b^2 - 4ac < 0 \\ &\iff \lambda_{1,2} \text{ are complex} \\ &\iff \text{no real characteristic curves exist.} \end{aligned}$$

For example, for the wave equation, $u_{tt} - \alpha^2 u_{xx} = 0$, we have $a = 1$, $c = -\alpha^2$, and $b = d = e = 0$. We find $\lambda_{1,2} = \pm\alpha$, and the equation is hyperbolic. Similarly, for the heat equation, $u_t - u_{xx} = 0$, we have $c = -1$, $d = 1$, and $a = b = e = 0$, so $\lambda_{1,2} = 0$ and there is no complete set of eigenvectors, implying that the equation is parabolic. Finally, for the Poisson equation, $u_{xx} + u_{yy} = 0$, identifying y with t , we have $a = c = 1$ and $b = d = e = 0$, so $\lambda_{1,2} = \pm i$ and there are no real characteristic curves, meaning that the equation is elliptic.

9.1.2 ■ Systems of nonlinear hyperbolic conservation laws in 1D

We now give two examples of nonlinear hyperbolic systems that come from fluid dynamics, briefly explaining how concepts such as characteristic curves, shocks, rarefactions, and Riemann problems generalize to the nonlinear systems case.

Example 9.3: Shallow-water equations

The *shallow-water equations* are a system of conservation laws given by

$$\frac{\partial}{\partial t} \begin{bmatrix} h \\ hv_x \\ hv_y \end{bmatrix} + \frac{\partial}{\partial x} \begin{bmatrix} hv_x \\ hv_x^2 + \frac{1}{2}gh^2 \\ hv_x v_y \end{bmatrix} + \frac{\partial}{\partial y} \begin{bmatrix} hv_y \\ hv_x v_y \\ hv_y^2 + \frac{1}{2}gh^2 \end{bmatrix} = \mathbf{0}, \quad (9.32)$$

where $h \geq 0$ is the water height; v_x and v_y are the x and y components of the velocity vector, $\mathbf{v}$, of the shallow-water flow; and the constant g is the gravitational acceleration ($g \approx 9.81 \text{ m/s}^2$ at Earth's surface). Be careful! Make sure not to confuse the water height h with h_x and h_t , the mesh sizes in space and time.

Equation (9.32) is a conservation law of the form in (9.2), reduced to a two-dimensional spatial domain, where the first equation of the system expresses conservation of mass and the second and third equations express conservation of the momentum, $m_x = hv_x$ and $m_y = hv_y$ in the x and y directions, respectively. We define the vector $\mathbf{u}$ of *conserved variables* for conservation law (9.2) as

$$\mathbf{u} = \begin{bmatrix} h \\ m_x \\ m_y \end{bmatrix}. \quad (9.33)$$

The alternative set of variables (h, v_x, v_y) is referred to as the *primitive variables*. Using conservative variables, the 1D version of equation (9.32) becomes

$$\frac{\partial}{\partial t} \begin{bmatrix} h \\ m_x \\ m_y \end{bmatrix} + \frac{\partial}{\partial x} \begin{bmatrix} m_x \\ \frac{m_x^2}{h} + \frac{1}{2}gh^2 \\ \frac{m_x m_y}{h} \end{bmatrix} = \mathbf{0}, \quad (9.34)$$

with flux vector and flux Jacobian given by

$$\mathbf{f}(\mathbf{u}) = \begin{bmatrix} m_x \\ \frac{m_x^2}{h} + \frac{1}{2}gh^2 \\ \frac{m_x m_y}{h} \end{bmatrix} \quad (9.35)$$

and

$$A(\mathbf{u}) = \begin{bmatrix} 0 & 1 & 0 \\ -\left(\frac{m_x}{h}\right)^2 + gh & 2\frac{m_x}{h} & 0 \\ -\frac{m_x m_y}{h^2} & \frac{m_y}{h} & \frac{m_x}{h} \end{bmatrix} \quad (9.36)$$

Recalling that $\frac{m_x}{h} = v_x$, the flux Jacobian has real eigenvalues (wave speeds)

$$\lambda_{1,2} = v_x \pm \sqrt{gh}, \quad (9.37a)$$

$$\lambda_3 = v_x \quad (9.37b)$$

and a complete set of eigenvectors, so the system is hyperbolic. This system is used to study phenomena such as tsunamis and dam break problems. In addition, jump discontinuities may occur in relation to tidal bores and hydraulic jumps.

Example 9.3 describes the shallow-water equations for shallow fluid flow. This is a system with $p = 3$ conserved quantities, and the three eigenvalues in equation (9.37) (which we say correspond to three distinct *wave modes*) are the slopes of three families of characteristic curves in the $x-t$ plane. In contrast to characteristics for scalar conservation laws and linear hyperbolic systems, the characteristic curves of nonlinear hyperbolic systems are generally curved (because, for example, the value of $v_x + \sqrt{gh}$ is generally not constant on a characteristic with slope λ_1).

Further analysis reveals that two different types of wave modes can be distinguished for nonlinear hyperbolic systems and are outlined in Definition 9.4.

Definition 9.4: Wave types in hyperbolic systems [150, Ch. 7]

The wave mode corresponding to eigenvalue λ_i of hyperbolic system (9.4) is said to be

- genuinely nonlinear if

$$\mathbf{r}_i(\mathbf{u})^T \nabla_{\mathbf{u}} \lambda_i(\mathbf{u}) \neq 0 \quad \forall \mathbf{u} \text{ and} \quad (9.38)$$

- linearly degenerate if

$$\mathbf{r}_i(\mathbf{u})^T \nabla_{\mathbf{u}} \lambda_i(\mathbf{u}) = 0 \quad \forall \mathbf{u}, \quad (9.39)$$

where $\lambda_i(\mathbf{u})$ is the i th eigenvalue of the flux Jacobian of the system with corresponding right eigenvector, $\mathbf{r}_i(\mathbf{u})$. We note that since the Jacobian is a function of $\mathbf{u}$, both the eigenvector and the eigenvalue will also be functions of $\mathbf{u}$. Thus, it is sensible to consider the vector $\nabla_{\mathbf{u}} \lambda_i(\mathbf{u})$, which is the gradient of $\lambda_i(\mathbf{u})$ with respect to the conserved variables $\mathbf{u}$.

Genuinely nonlinear wave modes tend to steepen or rarefy, similar to the case of Burgers' equation, whereas linearly degenerate wave modes do not steepen or rarefy, similar to the case of the linear advection equation.

By computing the eigenvectors of the Jacobian in the shallow-water problem, equation (9.36), and verifying the conditions of Definition 9.4, it can be shown that the wave modes corresponding to $\lambda_{1,2}$ in Example 9.3 are genuinely nonlinear, and the wave mode corresponding to λ_3 is linearly degenerate. In particular, the following example shows that these types of linearly degenerate wave modes allow for generic solutions to the nonlinear equations that do not exhibit any steepening or rarefying but instead advect a solution profile with constant speed without deforming it, in exact analogy with the general solution in equation (1.16) for the 1D linear advection equation.

Example 9.5: Linearly degenerate wave solution for the shallow-water equations

Consider the shallow-water flow field (given in primitive variables)

$$\begin{bmatrix} h(x, t) \\ v_x(x, t) \\ v_y(x, t) \end{bmatrix} = \begin{bmatrix} \bar{h} \\ \bar{v}_x \\ g(x - \bar{v}_x t) \end{bmatrix}, \quad (9.40)$$

where $\bar{h}$ and $\bar{v}_x$ are constant in x and t . We see that this flow field, when converted to conserved variables, is a solution of equation (9.34). This type of solution is called a *shear wave* in fluid mechanics, with the varying y -velocity describing the shear velocity between fluid layers that are all advected in the x -direction with the same velocity, $\lambda_3 = \bar{v}_x$.

Both genuinely nonlinear and linearly degenerate wave modes can feature jump discontinuities. In terminology deriving from fluid mechanics, discontinuities associated with genuinely nonlinear wave modes are called *shocks*, and discontinuities associated with linearly degenerate wave modes are called *contact discontinuities*. At discontinuities, solutions of conservation law system (9.3) satisfy a systems version of the Rankine–Hugoniot relations of Theorem 6.4, as given in Theorem 9.6 (with the same proof as for Theorem 6.4).

Theorem 9.6: Rankine–Hugoniot relation for systems [150, Sec. 7.1]

Let $\hat{x}(t)$ be a curve describing a jump discontinuity in a weak solution of 1D conservation law (9.3). Then,

$$\mathbf{f}(\mathbf{u}^+) - \mathbf{f}(\mathbf{u}^-) = \hat{x}'(t)(\mathbf{u}^+ - \mathbf{u}^-), \quad (9.41)$$

where $\mathbf{u}^-$ and $\mathbf{u}^+$ are the values of the state vector $\mathbf{u}(x, t)$ at the left and right of the jump discontinuity.

Following the discussion of Riemann problems for linear hyperbolic systems in Section 9.1.1, we now discuss Riemann problem solutions for nonlinear hyperbolic systems with p conserved quantities. As in the case of linear systems, we expect that an initial discontinuity, $[\mathbf{u}] = \mathbf{u}^+ - \mathbf{u}^-$, will split up into p different waves. The parts of the initial discontinuity that correspond to genuinely nonlinear wave modes may either propagate as a shock or become a rarefaction, and the parts that correspond to linearly degenerate wave modes will propagate as contact discontinuities. The next example discusses the solution of a Riemann problem for the shallow-water equations.

Example 9.7: Dam-break problem for the shallow-water equations

Figure 9.1.4 shows the numerical solution of a Riemann problem for the 1D version of the shallow-water equations (equation (9.32), with units chosen such that $g = 1$, for simplicity), with left and right states

$$\mathbf{u}^- = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{u}^+ = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}. \quad (9.42)$$

This is called a *dam-break* problem since it models a stationary water basin with water height $h = 2.0$ that is initially separated from water with height $h = 1.0$ by a barrier at $x = 0$. At $t = 0$, the dam breaks and, since the y -velocity, v_y , vanishes at $t = 0$, $v_y(x, t) = 0$ for all x and t . As a result, the conservation law simplifies to a 2×2 system for h and v_x with eigenvalues of the flux Jacobian as $\lambda_{1,2} = v_x \pm \sqrt{gh}$, which are both genuinely nonlinear.

Figure 9.1.4 shows, as expected, that the initial discontinuity splits into two waves. The left-going wave is a rarefaction wave corresponding to $\lambda_2 = v_x - \sqrt{gh}$, describing the water behind the dam that gradually becomes aware that the dam has broken and smoothly accelerates to the right from its initial velocity of $v_x = 0.0$ to a velocity of around $v_x = 0.4$, as it encounters increasingly shallow water on its right. The right-going wave, corresponding to $\lambda_1 = v_x + \sqrt{gh}$, is a shock: it sees water to its right that is much shallower and rushes head-over-heels into the abyss, with a shock speed allowed by the Rankine–Hugoniot relations that encode the principles of mass and momentum conservation. The shock in the figure is relatively diffuse due to the use of a first-order method with just 100 finite-volume cells. Also, this simple simulation explains

why one can often find warning signs for flash floods along streams downstream from water dams: water height can indeed rise very quickly in a shock wave.

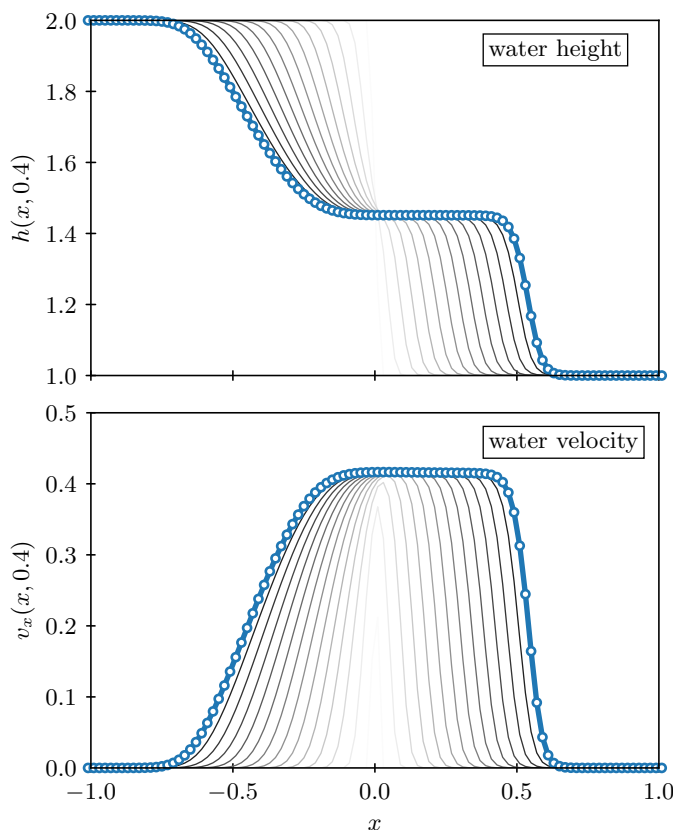


Figure 9.1.4. Dam-break problem for the shallow-water equations: (top) height, h ; (bottom) x -velocity, v_x . Blue curve indicates solution at $t = 0.4$ and fading black curves show history of solution back to $t = 0$.

In the next example, we describe the Euler equations of gas dynamics, which describe compressible fluid flows such as the flow of air around an airplane.

Example 9.8: Compressible Euler equations

The *Euler equations of compressible gas dynamics* are a conservation law system given by

$$\frac{\partial}{\partial t} \begin{bmatrix} \rho \\ \rho v_x \\ \rho v_y \\ \rho v_z \\ e \end{bmatrix} + \frac{\partial}{\partial x} \begin{bmatrix} \rho v_x \\ \rho v_x^2 + p \\ \rho v_x v_y \\ \rho v_x v_z \\ (e + p)v_x \end{bmatrix} + \frac{\partial}{\partial y} \begin{bmatrix} \rho v_y \\ \rho v_x v_y \\ \rho v_y^2 + p \\ \rho v_y v_z \\ (e + p)v_y \end{bmatrix} + \frac{\partial}{\partial z} \begin{bmatrix} \rho v_z \\ \rho v_x v_z \\ \rho v_y v_z \\ \rho v_z^2 + p \\ (e + p)v_z \end{bmatrix} = \mathbf{0}, \quad (9.43)$$

where $\rho \geq 0$ is the mass density of the fluid; v_x , v_y , and v_z are the x , y , and z components of the velocity vector of the fluid; and e is the total energy. The equations describe conservation of mass, momentum, and energy. The pressure, $p \geq 0$, can be expressed in terms of the five

variables (ρ, v_x, v_y, v_z, e) by using

$$e = \frac{p}{\gamma - 1} + \frac{1}{2} \|(v_x, v_y, v_z)\|_2^2, \quad (9.44)$$

which says that the total energy is the sum of the internal and kinetic energy of the fluid. Here, γ is a thermodynamic constant, for example, equal to $7/5$ for air. The conserved variables for the Euler equations are $(\rho, \rho v_x, \rho v_y, \rho v_z, e)$, and the variables (ρ, v_x, v_y, v_z, p) are usually referred to as the primitive variables.

The flux Jacobian for the 1D form of equation (9.43) has real eigenvalues (wave speeds)

$$\lambda_{1,2} = v_x \pm c, \quad (9.45a)$$

$$\lambda_{3,4,5} = v_x \quad (9.45b)$$

and a complete set of eigenvectors, so the system is hyperbolic. Here, c is the speed of sound, given by

$$c = \sqrt{\frac{\gamma p}{\rho}}. \quad (9.46)$$

The first two wave modes are genuinely nonlinear and correspond to sound waves, which may rarefy or steepen into shocks (e.g., the sonic boom generated by supersonic aircraft). The other three waves are linearly degenerate. Two of them correspond to shear waves in the y and z directions, and the third one is the so-called entropy wave, which allows for solutions in which a density profile advects with a constant velocity. Euler–Riemann problem solutions are composed of up to five waves—two of them are either a shock or a rarefaction, and the other three are contact discontinuities.

It is also notable that the *entropy* of the fluid can be defined by

$$s = \frac{p}{\rho^\gamma}. \quad (9.47)$$

As in the scalar case, a physically admissible shock associated with wave mode, i , has to satisfy the Lax entropy condition of Definition 6.7 for the associated eigenvalue, λ_i . This means that the characteristic curves of the i family need to enter the shock. It can be shown that the fluid entropy, s , increases when a fluid element crosses such a physically admissible shock, in accordance with the second law of thermodynamics. This is then an *a posteriori* justification for calling the condition in Definition 6.7 the Lax *entropy* condition and calling these admissible shocks *entropy satisfying*.

In the next sections, we discuss finite-volume methods for the nonlinear hyperbolic conservation laws we have described here, building on the properties of conservation law solutions that were introduced in this section.

9.2 ■ Finite-volume methods for systems of equations

Objectives

The goal of this section is to extend the scalar finite-volume methods from Section 6.3 to the case of nonlinear hyperbolic systems. Specifically, you will extend the scalar Lax–Friedrichs numerical flux function from equation (6.59) to the systems case and see how the Roe flux function improves on the Lax–Friedrichs flux by reducing the numerical diffusion through the use of a wave-by-wave eigenvector decomposition of the flux Jacobian.

We now extend the finite-volume methods developed for the 1D scalar conservation laws in Section 6.3 to the 1D nonlinear hyperbolic systems of equation (9.3). As in Section 6.2, we approximate cell averages of the vector of conserved quantities in finite-volume cell Ω_k at time t_ℓ , $\mathbf{u}_k(t_\ell)$, by the approximate cell averages $\mathbf{u}_{k,\ell}$, leading immediately to the systems version of the low-order explicit finite-volume method of equation (6.45):

$$\frac{\mathbf{u}_{k,\ell+1} - \mathbf{u}_{k,\ell}}{h_t} + \frac{\mathbf{f}^*(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}) - \mathbf{f}^*(\mathbf{u}_{k-1,\ell}, \mathbf{u}_{k,\ell})}{h_x} = \mathbf{0}. \quad (9.48)$$

Here, $\mathbf{f}^*(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})$ is a vector-valued numerical flux function. In what follows, we discuss two numerical flux functions for systems that are generalizations of the scalar numerical flux functions discussed in Section 6.3. For now, we limit our discussion to the low-order explicit finite-volume discretization of equation (9.48) and note that the schemes discussed here can be extended to second-order accuracy in time and space along the same lines as the approach for the scalar low-order finite-volume schemes in Section 6.5.

9.2.1 ■ The Lax–Friedrichs numerical flux function for systems

The scalar Lax–Friedrichs numerical flux function of equation (6.59) is directly extended to nonlinear systems as follows:

$$\mathbf{f}^*(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}) = \frac{\mathbf{f}(\mathbf{u}_{k,\ell}) + \mathbf{f}(\mathbf{u}_{k+1,\ell})}{2} - \frac{\alpha_{k+\frac{1}{2}}}{2}(\mathbf{u}_{k+1,\ell} - \mathbf{u}_{k,\ell}). \quad (9.49)$$

Here, the factor $\alpha_{k+\frac{1}{2}}$, which we have seen is proportional to the numerical diffusion in the discretization, is generalized from equation (6.61) to

$$\alpha_{k+\frac{1}{2}} = \max \left(\max_j |\lambda_j(\mathbf{u}_{k,\ell})|, \max_j |\lambda_j(\mathbf{u}_{k+1,\ell})| \right), \quad (9.50)$$

where $\lambda_j(\mathbf{u})$ is the j th eigenvalue of $\frac{\partial \mathbf{f}(\mathbf{u})}{\partial \mathbf{u}}$. The intuitive justification of this extension is from the scalar case: using the largest wave speed (in absolute value) on the left and right of the cell interface for $\alpha_{k+\frac{1}{2}}$ provides the right amount of numerical diffusion to make the scheme stable under the CFL limit and to preclude entropy-violating shocks. In the systems case, the choice in equation (9.50), which uses the *greatest* wave speed in magnitude on each side of the interface, adds sufficient numerical diffusion to *all* of the p wave modes to make each of the modes stable under the stability limit

$$h_t < \frac{h_x}{\max_k (\max_j |\lambda_j(\mathbf{u}_{k,\ell})|)}. \quad (9.51)$$

While this simple choice adds sufficient numerical diffusion for stability, it can add more diffusion than is required for some wave modes, which tends to decrease accuracy. The more sophisticated approach discussed next fine-tunes the numerical diffusion for each wave mode more carefully, with the potential for improved accuracy.

9.2.2 ■ The Roe numerical flux function

The Roe method for linear hyperbolic systems

We first consider the linear hyperbolic system in equation (9.7). Since, in the characteristic variables, the linear system decouples into p linear advection equations as in equation (9.13), it is

natural to apply the scalar finite-volume method (6.45) with the scalar Lax–Friedrichs flux function in equation (6.59) to each of the decoupled advection equations, resulting in the following finite-volume method for the linear system in characteristic variables $\mathbf{w}$:

$$\frac{\mathbf{w}_{k,\ell+1} - \mathbf{w}_{k,\ell}}{h_t} + \frac{\mathbf{g}_{k+\frac{1}{2},\ell}^* - \mathbf{g}_{k-\frac{1}{2},\ell}^*}{h_x} = \mathbf{0}, \quad (9.52)$$

where numerical flux function $\mathbf{g}_{k+\frac{1}{2},\ell}^*$ is given by

$$\mathbf{g}_{k+\frac{1}{2},\ell}^* = \frac{\Lambda \mathbf{w}_{k,\ell} + \Lambda \mathbf{w}_{k+1,\ell}}{2} - \frac{|\Lambda|}{2} (\mathbf{w}_{k+1,\ell} - \mathbf{w}_{k,\ell}). \quad (9.53)$$

Here,

$$|\Lambda| = \begin{bmatrix} |\lambda_1| & 0 & 0 & \cdots & 0 \\ 0 & |\lambda_2| & 0 & \cdots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & \cdots & 0 & |\lambda_{p-1}| & 0 \\ 0 & \cdots & 0 & 0 & |\lambda_p| \end{bmatrix}. \quad (9.54)$$

We then multiply equation (9.52) on the left by R and, using $\mathbf{u} = R\mathbf{w}$ and $\mathbf{w} = L\mathbf{u}$, we get back to the finite-volume method in equation (9.48) with the Roe numerical flux function

$$\mathbf{f}_{k+\frac{1}{2},\ell}^* = \frac{A\mathbf{u}_{k,\ell} + A\mathbf{u}_{k+1,\ell}}{2} - \frac{|A|}{2} (\mathbf{u}_{k+1,\ell} - \mathbf{u}_{k,\ell}), \quad (9.55)$$

where

$$|A| = R|\Lambda|L. \quad (9.56)$$

Extension to nonlinear systems

The Roe flux function for linear hyperbolic systems, equation (9.55), naturally extends to nonlinear systems as follows:

$$\mathbf{f}^*(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}) = \frac{\mathbf{f}(\mathbf{u}_{k,\ell}) + \mathbf{f}(\mathbf{u}_{k+1,\ell})}{2} - \frac{|\widehat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})|}{2} (\mathbf{u}_{k+1,\ell} - \mathbf{u}_{k,\ell}), \quad (9.57)$$

where, for example, $\widehat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})$ can be chosen to be the flux Jacobian $A(\mathbf{u})$ evaluated at the average of the conservative state variables on the left and the right of the cell interface:

$$\widehat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}) = A(\bar{\mathbf{u}}_{k+\frac{1}{2},\ell}), \quad (9.58a)$$

$$\bar{\mathbf{u}}_{k+\frac{1}{2},\ell} = \frac{\mathbf{u}_{k,\ell} + \mathbf{u}_{k+1,\ell}}{2}. \quad (9.58b)$$

In practice, it turns out that it is often preferable to do the averaging using the primitive state variables on the left and right of the interface, rather than the conservative variables, followed by converting the primitive average back to a conserved state variable, so that $A(\mathbf{u})$ can be evaluated. For example, in the compressible Euler equations of Example 9.8, averaging conservative state vectors may easily result in an effectively negative pressure for the averaged state if the kinetic energy in the left and/or right states is large relative to the internal energy. The algorithm

may then break down if a negative pressure arises—e.g., when computing the sound speed to determine eigenvalues.

The finite-volume method resulting from flux function (9.57) with the interface Jacobian in equation (9.58a) is often quite effective. However, it is also expensive: to determine $|\widehat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})|$, the right and left eigenvectors are computed at every cell interface and every timestep. Another drawback is that the scheme can produce entropy-violating shocks. This may happen for sonic rarefactions (i.e., a rarefaction in wave mode j where $\lambda_j = 0$ on some characteristic in the rarefaction wave) because the numerical diffusion (which is proportional to the j th eigenvalue of $|\widehat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})|$) may vanish or be very small in numerical flux function (9.57), and sufficient diffusion is required to ensure that the numerical method selects the entropy-satisfying rarefaction instead of an entropy-violating shock. This potential defect can be remedied by considering a so-called *entropy fix*, as explained in the next example.

Example 9.9: An entropy fix for the compressible Euler equations

Consider the genuinely nonlinear wave modes of the Euler equations of Example 9.8:

$$\lambda_{1,2} = v_x \pm c. \quad (9.59)$$

When using flux function (9.57) with the interface Jacobian in equation (9.58a), the numerical diffusion of wave mode 1 or 2 can be zero or close to zero in sonic rarefactions, where some numerical diffusion is required to avoid entropy-violating shocks. The following entropy fix prohibits numerical diffusion from getting too close to zero by smoothing the absolute value function near zero. In place of directly using $|\lambda_j|$, we use the corrected wave speed (in absolute value), $|\widehat{\lambda}_j|$, which is given by

$$|\widehat{\lambda}_j| = g(|\lambda_j|), \quad j = 1, 2, \quad (9.60)$$

where

$$g(|\lambda|) = \begin{cases} |\lambda| & \text{if } |\lambda| \geq \epsilon, \\ \frac{1}{2} \left(\frac{\lambda^2}{\epsilon} + \epsilon \right) & \text{if } |\lambda| < \epsilon, \end{cases} \quad (9.61)$$

with a suitably chosen $\epsilon > 0$; see Figure 9.2.1. As a result, since $g(|\lambda|)$ is continuously differentiable at $|\lambda| = \epsilon$, we see that $g(\epsilon) = \epsilon$ and $g'(\epsilon) = 1$, creating a smooth transition while padding the function away from zero. For a linearly degenerate wave mode, an entropy fix is unnecessary, since unphysical shocks cannot occur.

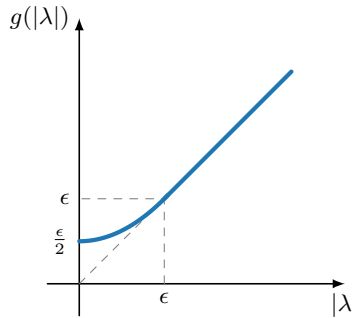


Figure 9.2.1. Entropy fix for the genuinely nonlinear wave modes of the compressible Euler or shallow-water equations.

The Roe flux function with Roe-averaging

As it turns out, when using the Roe flux defined in equation (9.57) with the flux Jacobian approximation of equation (9.58a), it is possible to further improve the accuracy for certain wave modes by selecting a different averaging function, $\bar{\mathbf{u}}_{k+\frac{1}{2},\ell}$, rather than the simple arithmetic mean used in equation (9.58b) (of either the conservative or the primitive state vector). In a so-called *Roe-averaged* flux, the average $\bar{\mathbf{u}}_{k+\frac{1}{2},\ell}$ in equation (9.58a) is chosen so that the following property is satisfied:

$$\hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}) (\mathbf{u}_{k+1,\ell} - \mathbf{u}_{k,\ell}) = \mathbf{f}(\mathbf{u}_{k+1,\ell}) - \mathbf{f}(\mathbf{u}_{k,\ell}) \quad \text{for any } \mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}. \quad (9.62)$$

While this may seem like a difficult condition to satisfy, Example 9.10 gives such an averaging function for the compressible Euler equations. Averages satisfying this condition have also been successfully derived for other relevant nonlinear conservation laws.

To see how the constraint in equation (9.62) may be advantageous for more accurately resolving certain jump discontinuities in the flow, consider a discontinuity between cell averages in cells Ω_k and Ω_{k+1} at time t_ℓ that corresponds to a single wave mode j . This discontinuity should propagate with speed s according to the Rankine–Hugoniot relation for systems, equation (9.41):

$$\mathbf{f}(\mathbf{u}_{k+1,\ell}) - \mathbf{f}(\mathbf{u}_{k,\ell}) = s(\mathbf{u}_{k+1,\ell} - \mathbf{u}_{k,\ell}). \quad (9.63)$$

If we define $\hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})$ with s as its j -mode eigenvalue, then the Riemann problem for the linearized conservation law,

$$\frac{\partial \mathbf{u}}{\partial t} + \hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}) \frac{\partial \mathbf{u}}{\partial x} = \mathbf{0}, \quad (9.64)$$

yields a discontinuity that propagates with the correct speed s , and numerical diffusion based on $\hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})$ will not be larger than is required for stability (where we recall that the eigenvalues $\hat{\lambda}_j$ of $\hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})$ provide the numerical diffusion in wave modes j that is necessary for the stability of each mode). This requires that

$$\hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}) (\mathbf{u}_{k+1,\ell} - \mathbf{u}_{k,\ell}) = s(\mathbf{u}_{k+1,\ell} - \mathbf{u}_{k,\ell}), \quad (9.65)$$

resulting in $\hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})$ with a right eigenvector of $\mathbf{u}_{k+1,\ell} - \mathbf{u}_{k,\ell}$ and s as the associated eigenvalue.

If equation (9.62) is satisfied for any $\mathbf{u}_{k,\ell}$ and $\mathbf{u}_{k+1,\ell}$, then both equations (9.65) and (9.64) are satisfied in the case of a discontinuity between cell averages in cells Ω_k and Ω_{k+1} that corresponds to a single wave mode j that propagates with speed s . This means that such waves receive the amount of numerical diffusion they require for stability, but not more. This is especially beneficial for contact discontinuities associated with linearly degenerate wave modes j and, in particular, contact discontinuities that are stationary: in this case, $\hat{\lambda}_j = 0$, and no numerical diffusion is necessary (either for numerical stability or for avoiding entropy-violating shocks). Roe-averaging approximates $A(\mathbf{u})$ at the cell interface in such a way that the computed $\hat{\lambda}_j$ in the Roe flux vanishes and the contact discontinuity is not smeared by (unnecessary) numerical diffusion.

Example 9.10: Roe-averaging for the compressible Euler equations

For the compressible Euler equations of Example 9.8, the average state, $\bar{\mathbf{u}}_{k+\frac{1}{2},\ell}$, which is used so that $A(\bar{\mathbf{u}}_{k+\frac{1}{2},\ell})$ in equation (9.58a) is computed to satisfy property (9.62), known as the *Roe-averaged state*, is given by

$$\bar{\rho}_{k+\frac{1}{2},\ell} = \sqrt{\rho_{k,\ell}\rho_{k+1,\ell}}, \quad (9.66a)$$

$$(\bar{v}_x)_{k+\frac{1}{2},\ell} = \frac{\sqrt{\rho_{k,\ell}}(v_x)_{k,\ell} + \sqrt{\rho_{k+1,\ell}}(v_x)_{k+1,\ell}}{\sqrt{\rho_{k,\ell}} + \sqrt{\rho_{k+1,\ell}}}, \quad (9.66b)$$

$$(\bar{v}_y)_{k+\frac{1}{2},\ell} = \frac{\sqrt{\rho_{k,\ell}}(v_y)_{k,\ell} + \sqrt{\rho_{k+1,\ell}}(v_y)_{k+1,\ell}}{\sqrt{\rho_{k,\ell}} + \sqrt{\rho_{k+1,\ell}}}, \quad (9.66c)$$

$$(\bar{v}_z)_{k+\frac{1}{2},\ell} = \frac{\sqrt{\rho_{k,\ell}}(v_z)_{k,\ell} + \sqrt{\rho_{k+1,\ell}}(v_z)_{k+1,\ell}}{\sqrt{\rho_{k,\ell}} + \sqrt{\rho_{k+1,\ell}}}, \quad (9.66d)$$

$$\bar{h}_{k+\frac{1}{2},\ell} = \frac{\sqrt{\rho_{k,\ell}}h_{k,\ell} + \sqrt{\rho_{k+1,\ell}}h_{k+1,\ell}}{\sqrt{\rho_{k,\ell}} + \sqrt{\rho_{k+1,\ell}}}, \quad (9.66e)$$

where $h_{k,\ell}$ is the specific total enthalpy, given by

$$h_{k,\ell} = \frac{e_{k,\ell} + p_{k,\ell}}{\rho_{k,\ell}}. \quad (9.67)$$

Using this Roe average for the Roe numerical flux function, (9.57), stationary contact discontinuities (which may occur, for example, in stationary aerodynamics simulations of air flow patterns around airplane wings with shocks) are computed with much less diffusion than using simple arithmetic averages like the one in equation (9.58b).

9.2.3 ■ Numerical example

We illustrate the local Lax–Friedrichs and Roe schemes for the one-dimensional form of the shallow-water equations from Examples 9.3 and 9.7. Here, we again consider the spatial domain $-1 < x < 1$, with initial condition

$$\mathbf{u}(x, 0) = \begin{cases} \mathbf{u}^- & \text{if } x < 0, \\ \mathbf{u}^+ & \text{if } x > 0, \end{cases} \quad (9.68)$$

with left and right states given by

$$\mathbf{u}^- = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{u}^+ = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}. \quad (9.69)$$

For both schemes, we integrate with the low-order explicit finite-volume method given in equation (9.48), using 100 cells for the interval $-1 < x < 1$ and timestep given by one-half times the CFL limit given in equation (9.51), integrating out to time $t = 0.4$. We use ghost cells (discussed in detail in Section 9.3) to impose Dirichlet boundary conditions to match the initial conditions on the two endpoints, but note that no changes to the solution are propagated near the endpoints before the final time in this example.

For the local Lax–Friedrichs discretization, we compute $\alpha_{k+\frac{1}{2}}$ as in equation (9.50), using the computed eigenvalues in equation (9.37) and noting that $v_x - \sqrt{gh} \leq v_x \leq v_x + \sqrt{gh}$, so

we only need to consider the maximal absolute value of the two genuinely nonlinear eigenvalues of the system. For the Roe scheme, we use the Roe-averaged state $\bar{\mathbf{u}}_{k+\frac{1}{2},\ell}$ given by

$$\bar{h}_{k+\frac{1}{2},\ell} = \frac{1}{2} (h_{k,\ell} + h_{k+1,\ell}), \quad (9.70a)$$

$$(\bar{v}_x)_{k+\frac{1}{2},\ell} = \frac{\sqrt{h_{k,\ell}} (v_x)_{k,\ell} + \sqrt{h_{k+1,\ell}} (v_x)_{k+1,\ell}}{\sqrt{h_{k,\ell}} + \sqrt{h_{k+1,\ell}}}, \quad (9.70b)$$

$$(\bar{v}_y)_{k+\frac{1}{2},\ell} = \frac{\sqrt{h_{k,\ell}} (v_y)_{k,\ell} + \sqrt{h_{k+1,\ell}} (v_y)_{k+1,\ell}}{\sqrt{h_{k,\ell}} + \sqrt{h_{k+1,\ell}}}, \quad (9.70c)$$

and exactly evaluate the flux Jacobian $\hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell}) = A(\bar{\mathbf{u}}_{k+\frac{1}{2},\ell})$ using the expression in equation (9.36). For each cell Ω_k , we form this matrix, compute its eigendecomposition, and then compute $|\hat{A}(\mathbf{u}_{k,\ell}, \mathbf{u}_{k+1,\ell})|$ using equation (9.56) with the entropy-fix approximation to the absolute value given in Example 9.9.

Numerical results are shown in Figure 9.2.2. While both schemes are rather diffusive (to be expected for the explicit low-order finite-volume framework that we are considering so far), we note that the rarefaction wave on the left is somewhat better resolved for the Roe scheme than for the local Lax–Friedrichs approximation, most noticeably in the approximation of v_x .

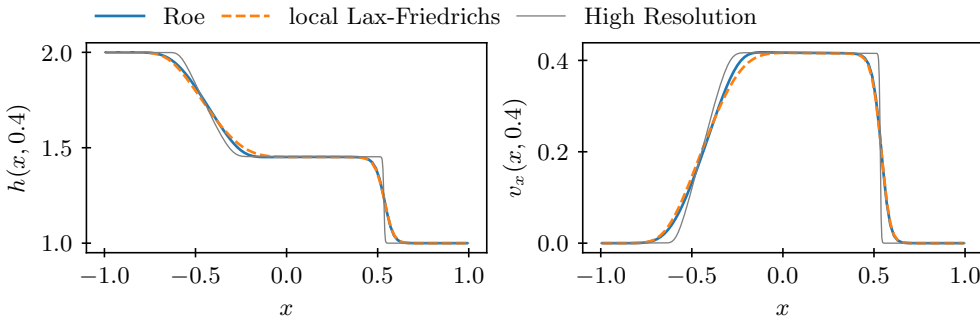


Figure 9.2.2. Comparison between local Lax–Friedrichs and Roe schemes for 1D shallow-water equations.

9.3 ■ Multidimensional problems

Objectives

The goal of this section is to extend the finite-volume approach from one to multiple spatial dimensions. In particular, you will first consider regular grids and do the extension using a “dimension-by-dimension” approach. Next, you will relate boundary conditions for hyperbolic systems to characteristic directions and learn how to implement them using ghost cells. Finally, you will derive finite-volume formulas on unstructured grids with polyhedral finite-volume cells using the divergence theorem.

9.3.1 ■ Finite-volume methods on Cartesian grids

We first look at finite-volume methods on regular Cartesian grids. For simplicity, we limit the discussion to the 2D case, but extensions to higher dimensions directly follow. Consider the

system of conservation laws in equation (9.2) in 2D,

$$\frac{\partial \mathbf{u}}{\partial t} + \frac{\partial \mathbf{f}(\mathbf{u})}{\partial x} + \frac{\partial \mathbf{g}(\mathbf{u})}{\partial y} = \mathbf{0}, \quad (9.71)$$

where $\mathbf{u}$, $\mathbf{f}$, and $\mathbf{g} \in \mathbb{R}^p$. Consider a Cartesian grid with constant h_x and h_y , as in Figure 9.3.1.

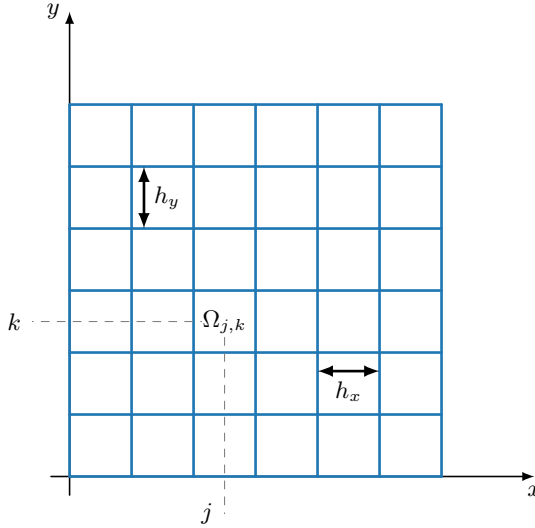


Figure 9.3.1. 2D Cartesian grid used for finite-volume method with cells $\Omega_{j,k}$.

The 1D explicit low-order finite-volume method in equation (9.48) naturally extends to the 2D Cartesian grid as follows:

$$\begin{aligned} \frac{\mathbf{u}_{j,k,\ell+1} - \mathbf{u}_{j,k,\ell}}{h_t} + \frac{\mathbf{f}^*(\mathbf{u}_{j,k,\ell}, \mathbf{u}_{j+1,k,\ell}) - \mathbf{f}^*(\mathbf{u}_{j-1,k,\ell}, \mathbf{u}_{j,k,\ell})}{h_x} \\ + \frac{\mathbf{g}^*(\mathbf{u}_{j,k,\ell}, \mathbf{u}_{j,k,\ell+1}) - \mathbf{g}^*(\mathbf{u}_{j,k-1,\ell}, \mathbf{u}_{j,k,\ell})}{h_y} = \mathbf{0}, \end{aligned} \quad (9.72)$$

where we note the double spatial index (j, k) so that $\mathbf{u}_{j,k,\ell}$ is the finite-volume approximation to the average value of $\mathbf{u}$ over cell $\Omega_{j,k}$ at time t_ℓ . For the numerical flux functions $\mathbf{f}^*$ and $\mathbf{g}^*$, we can use, for example, the systems Lax–Friedrichs flux function in equation (9.49) or the systems Roe flux function in equation (9.57). We note that the flux functions $\mathbf{f}$ and $\mathbf{g}$ in equation (9.71) are often essentially the same function, up to appropriate rotations of velocity and momentum vectors in terms of components perpendicular and tangential to the cell interfaces—see Examples 9.3 and 9.8.

9.3.2 ■ Boundary conditions and ghost cells

In order to discuss boundary conditions and their implementation for hyperbolic systems, we first consider linear hyperbolic systems in 1D as in equation (9.7). For definiteness, we consider a linear system of size $p = 2$, with eigenvalue matrix

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}, \quad (9.73)$$

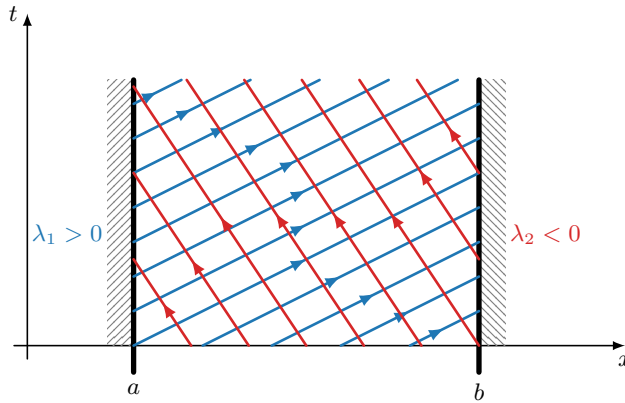


Figure 9.3.2. Characteristics and boundary conditions for a linear system with two variables in 1D.

where, for example, $\lambda_1 > 0$ and $\lambda_2 < 0$. In addition, the computational domain is $\Omega_{(x,t)} = [a, b] \times [0, \infty)$, with initial condition $\mathbf{u}(x, 0)$ at $t = 0$.

Figure 9.3.2 illustrates the two families of characteristic curves. Since characteristic variable $w_i(x, t)$ is conserved on characteristics with slope λ_i , it is clear that $w_1(a, t)$ is required as a boundary condition on the left boundary of the domain and $w_2(b, t)$ is required as a boundary condition on the right boundary of the domain, in accordance with the signs of λ_1 and λ_2 .

A convenient way to impose boundary conditions in finite-volume methods is to use so-called *ghost cells*. When applying a low-order finite-volume method (see equation (9.48)) to the problem in Figure 9.3.2, one ghost cell is added to each side of the computational domain. The left boundary condition is imposed at time t_ℓ by specifying $w_1(a, t_\ell)$ (or a more accurate approximation of the average value of $w_1(x, t_\ell)$ over the ghost cell from $a - h_x$ to a , such as $w_1(a - h_x/2, t_\ell)$, if known) in the left ghost cell and by extrapolating the numerical approximation of $w_2(x, t_\ell)$ in the first physical cell next to the boundary (with, for example, a constant extension) to the ghost cell. We use extrapolation in this way so that the value of $w_2(x, t)$ is carried outside the computational domain on the left by the λ_2 characteristics without introducing nonphysical oscillations. Similarly, the right boundary condition imposes $w_2(b, t_\ell)$ (or a more accurate approximation) in the ghost cell at the right boundary, and $w_1(x, t_\ell)$ is extrapolated from the first physical cell at the right boundary into the right ghost cell. In practice, we often don't need to convert the original $\mathbf{u}(x, t)$ variable into characteristic variables to impose the boundary conditions. Instead, it is often possible, in a case like the problem of Figure 9.3.2, to impose one component of $\mathbf{u}(x, t)$ on the left and the other one on the right and extrapolate the component of $\mathbf{u}(x, t)$ that is not imposed as a boundary condition on each side. How this is done depends, of course, on the boundary information that is provided in the problem formulation. For the second-order finite-volume methods of Section 6.5, two ghost cells are required on each side of the domain.

For nonlinear problems, the general idea is similar, but correctly imposing boundary conditions can be complicated in practice, since the signs of the eigenvalues may change depending on the solution at the boundaries. The same general principles also apply in multiple dimensions, with Figure 9.3.3 illustrating ghost cells for a first-order scheme on a 2D domain.

A specific type of boundary condition that often arises in multidimensional fluid flow problems (e.g., Euler or shallow-water flow; see Example 9.8 and Example 9.3) is the so-called *ideal wall* boundary condition, where fluid is allowed to flow freely along the wall but cannot penetrate the wall. In 2D, consider the velocity vector

$$\mathbf{v} = (v_x(\mathbf{x}), v_y(\mathbf{x}))^T \quad (9.74)$$

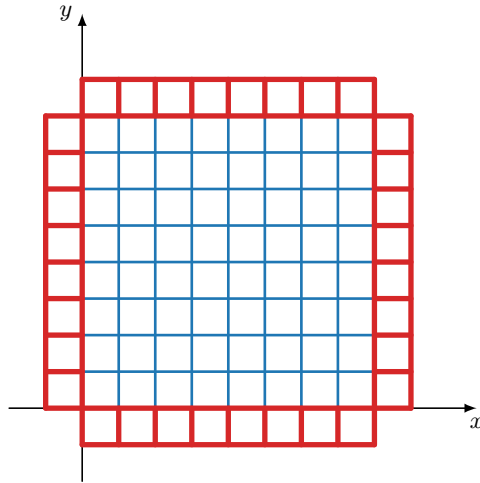


Figure 9.3.3. Ghost cells (in red) for a first-order finite-volume method in 2D.

at a point x on $\partial\Omega$ with normal $\mathbf{n}$. We expect the velocity perpendicular to the wall to be zero,

$$\mathbf{n} \cdot \mathbf{v} = 0, \quad (9.75)$$

while $\mathbf{n} \times \mathbf{v}$ (tangent to the wall) should be freely determined. To implement the ideal wall boundary conditions for velocity components that are *parallel* to the wall, constant extrapolation is used from the physical cells adjacent to the wall boundaries to the adjacent ghost cells. To maintain conservation of mass for velocity components that are perpendicular to the wall, the values are extrapolated with opposite sign, leading to an effective perpendicular velocity of zero at the wall interface. Figure 9.3.4 highlights this scenario on both the north- and east-facing boundaries of a domain.

For a concrete example, consider the two-dimensional shallow-water equations of Example 9.3 with a simple nondimensionalization to set the gravitational constant to $g = 1$. We

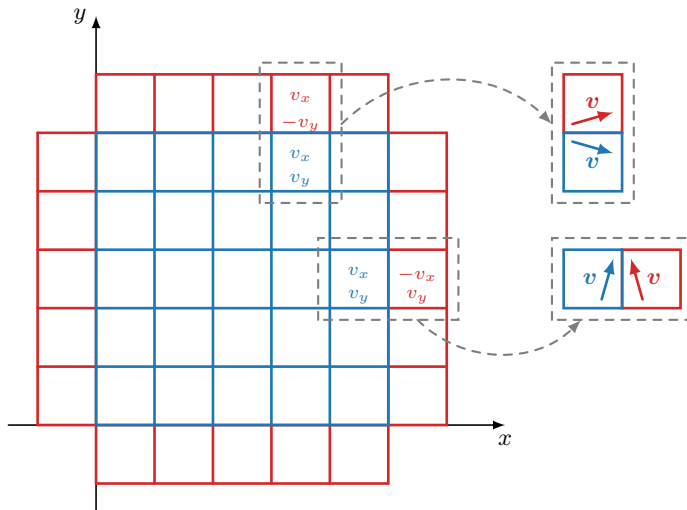


Figure 9.3.4. Ideal wall boundary conditions for shallow-water flow in 2D using ghost cells (in red).

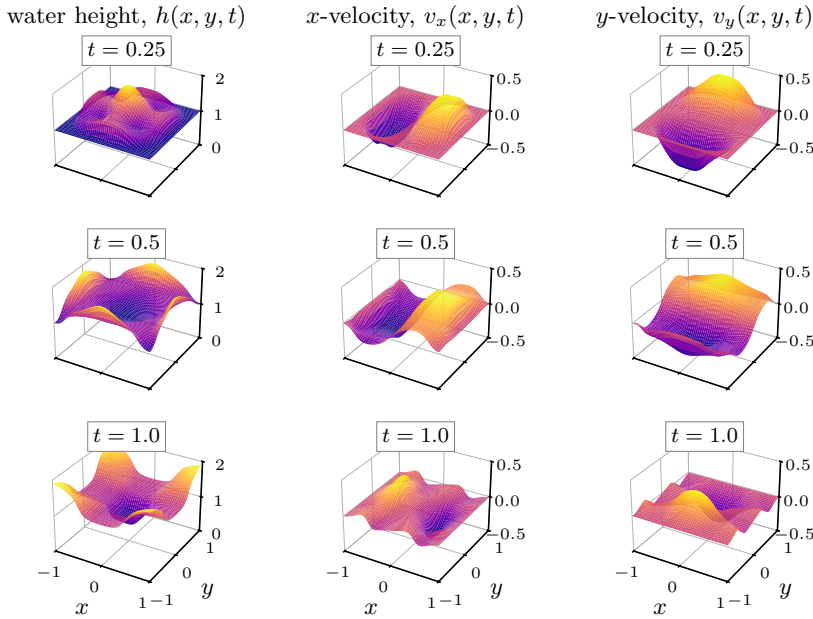


Figure 9.3.5. Snapshots of h , v_x , and v_y for the 2D shallow-water equations at times $t = 0.25$, 0.5 , and 1.0 using the local Lax–Friedrichs flux function.

discretize equation (9.32) on a 2D Cartesian grid using the explicit low-order scheme in equation (9.72) and the systems version of the local Lax–Friedrichs flux function from equation (9.49). Here, we consider the case of ideal wall boundary conditions for velocity and zero-flux boundary conditions for water height. This can be thought of as a model of water sloshing in an enclosed basin (like a sink or bathtub), so long as it does not overflow.

We take a square domain, $\Omega = [-1, 1]^2$, and start with a two-dimensional version of the dam-breaking problem from Example 9.7, where the initial velocity is zero and the initial water height is two if $(x, y) \in [-\frac{1}{2}, \frac{1}{2}]^2$ and one otherwise. In this example, we use 100 cells in each direction, and we take the timestep to satisfy the CFL condition

$$h_t = \frac{1}{4} \min_{j,k} \left(\min_i \frac{h_x}{|\lambda_i(\mathbf{u}_{j,k})|}, \min_i \frac{h_y}{|\mu_i(\mathbf{u}_{j,k})|} \right), \quad (9.76)$$

where λ_i are the eigenvalues of $\frac{\partial \mathbf{f}}{\partial \mathbf{u}}$ and μ_i are the eigenvalues of $\frac{\partial \mathbf{g}}{\partial \mathbf{u}}$.

Figure 9.3.5 shows snapshots of the three components of the solution to the shallow-water equations as time progresses for this example. Note that the flow quickly gets quite complex. At time $t = 0.25$, the initial added mass is spreading out from the center of the domain, in two sets of shock waves, one heading in the x -direction and the other in the y -direction. At time $t = 0.5$, the mass is now reflecting off the domain boundary, and the shock waves are starting to rebound. By $t = 1.0$, there are several reflections “bouncing” around the domain, including different behavior induced by the corners and edges of the domain. Importantly, although the solution given by the use of the local Lax–Friedrichs flux function is somewhat diffused, it remains perfectly mass conserving. Integrating the mass (equal to the volume at unit density) of water in the domain at the initial condition gives a value of five; Figure 9.3.6 shows the midpoint-rule mass calculation as a function of time, showing perfect conservation of the initial mass.

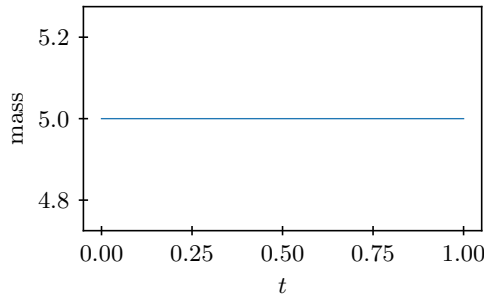


Figure 9.3.6. Integrated mass for the local Lax–Friedrichs solution of the 2D shallow-water equations.

9.3.3 ■ Finite-volume methods on unstructured grids

We next consider a more general partition of computational domain, Ω , into finite-volume cells, Ω_k , that are polygons in 2D or polyhedra in 3D. We call this an unstructured grid if it is not a logically Cartesian grid of quadrilaterals (in 2D) or hexahedra (in 3D). See Figure 9.3.7 for an example of an unstructured grid with triangles in 2D.

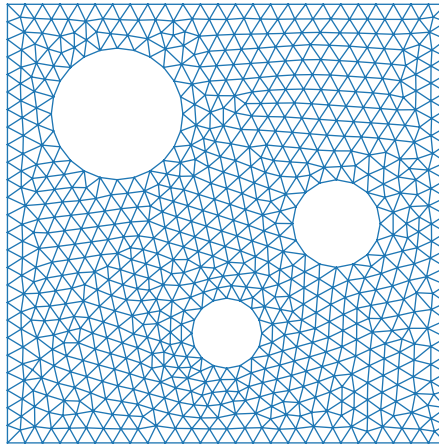


Figure 9.3.7. Example of an unstructured triangular grid in 2D.

Constructing finite-volume methods on such unstructured grids is relatively straightforward. Consider the system of conservation laws in equation (9.2), and integrate over a finite-volume cell Ω_k ,

$$\frac{d}{dt} \left(\int_{\Omega_k} \mathbf{u}(\mathbf{x}, t) d\mathbf{x} \right) + \int_{\partial\Omega_k} \mathbb{F}(\mathbf{u}(\mathbf{x}, t)) \mathbf{n} ds = 0, \quad (9.77)$$

where the divergence theorem is applied on the second term. In the case of a 2D problem, for example, this leads to the following extension of the 1D explicit low-order finite-volume method described in equation (9.48) on a 2D unstructured grid with triangular finite-volume cells,

$$\frac{V_k (\mathbf{u}_{k,\ell+1} - \mathbf{u}_{k,\ell})}{h_t} + \sum_{q=1}^3 \mathbb{F}_{q,\ell}^* \mathbf{n}_q l_q = \mathbf{0}, \quad (9.78)$$

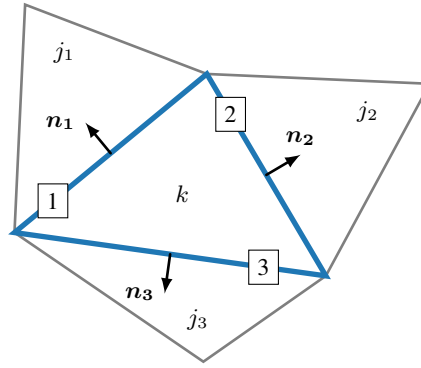


Figure 9.3.8. Cell k with outward-facing normals on edges 1, 2, and 3.

where $\mathbf{u}_{k,\ell}$ approximates the average of $\mathbf{u}$ over cell Ω_k at time t_ℓ . Here, V_k is the area of cell k , l_q is the length of edge q of the finite-volume cell, and $\mathbf{n}_q$ is the outward unit normal on edge q (see Figure 9.3.8). Further,

$$\mathbb{F}_{q,\ell}^* \mathbf{n}_q = \mathbb{F}^*(\mathbf{u}_{k,\ell}, \mathbf{u}_{j_q,\ell}) \quad (9.79)$$

is the normal numerical flux vector through edge q of the triangle, depending on cell average $\mathbf{u}_{k,\ell}$ in cell Ω_k at time t_ℓ and cell average $\mathbf{u}_{j_q,\ell}$ in cell Ω_{j_q} on the other side of edge q . It is common to divide equation (9.78) by V_k to get

$$\frac{\mathbf{u}_{k,\ell+1} - \mathbf{u}_{k,\ell}}{h_t} + \sum_{q=1}^3 \frac{l_q}{V_k} \mathbb{F}^*(\mathbf{u}_{k,\ell}, \mathbf{u}_{j_q,\ell}) = \mathbf{0}. \quad (9.80)$$

In this form, we can verify that the unstructured-grid finite-volume formula recovers the structured-grid formula in equation (9.72). For the numerical flux $\mathbb{F}^*$, we can again use, for example, the systems Lax–Friedrichs flux function in equation (9.49), or the systems Roe flux function in equation (9.57); these flux functions can be used for edges q that are not aligned with the coordinate axes, as long as rotations are used for the components of the velocity and momentum vectors in order to orient them perpendicular and parallel to the cell interfaces.

9.4 ■ Essentially nonoscillatory finite-volume methods

Objectives

The goal of this section is to introduce a family of advanced finite-volume methods with order of accuracy higher than two that handles discontinuous solutions without excessive spurious oscillations. In particular, you will build a family of high-degree polynomial reconstructions based on solution averages on shifted stencils that can be combined with a stencil-selection strategy to obtain a polynomial reconstruction that satisfies an essentially nonoscillatory (ENO) property. Then, you will develop intuition on how the weighted essentially nonoscillatory (WENO) method takes weighted combinations of the shifted polynomial reconstructions to increase the order of accuracy. Finally, you will derive a high-order finite-volume method that uses WENO polynomials combined with a special form of high-order Runge–Kutta time integration that satisfies a total-variation diminishing (TVD) property, applying it to hyperbolic systems in multiple dimensions.

Section 6.2.1 illustrated that centered-type higher-order finite-difference stencils lead to $\mathcal{O}(1)$ spurious oscillations at discontinuities with amplitude that does not decrease as the grid is refined. For scalar conservation laws in 1D, Section 6.5 presented second-order accurate methods that are provably oscillation-free by considering limiter-based nonlinear schemes that preserve the total-variation diminishing (TVD) property, but these approaches generally cannot be extended beyond second-order accuracy. In this section, we introduce the so-called *essentially nonoscillatory* (ENO) schemes, which relax the TVD requirement and can obtain an arbitrarily high order of accuracy while allowing small spurious oscillations. As a result, these methods are not *fully* nonoscillatory, but they are *essentially* nonoscillatory, in the sense that these small oscillations are expected to decay as the grid is refined. The discussion in this section is primarily based on [198, 131].

We focus our treatment of ENO schemes in 1D on scalar conservation laws, with extensions to multiple dimensions briefly discussed in Section 9.4.6. We consider a piecewise smooth function, $u(x)$, which may have a finite number of jump discontinuities or jumps in derivatives, and we consider the cell averages, u_k , of $u(x)$, over finite-volume cells Ω_k of equal size h_x . Our first goal is to construct, for each cell $\Omega_k = [x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}]$, a high-order accurate polynomial $q_k(x)$ that approximates $u(x)$ in Ω_k , using cell averages u_j for neighboring cells Ω_j —i.e., a polynomial $q_k(x)$ of degree at most $p-1$ based on p cell averages that approximates $u(x)$ with accuracy $\mathcal{O}(h_x^p)$, except when $u(x)$ is not sufficiently smooth over the stencil that defines $q_k(x)$. Our finite-volume method then uses $q_k(x)$ in cell Ω_k to reconstruct the values $u_{k+\frac{1}{2}}^-$ and $u_{k-\frac{1}{2}}^+$ at the interfaces of cell Ω_k with accuracy $\mathcal{O}(h_x^p)$ if $u(x)$ is smooth, writing

$$u_{k+\frac{1}{2}}^- = q_k(x_{k+\frac{1}{2}}), \quad (9.81a)$$

$$u_{k-\frac{1}{2}}^+ = q_k(x_{k-\frac{1}{2}}). \quad (9.81b)$$

We choose different polynomials $q_k(x)$ in each of the cells Ω_k , and we call the resulting piecewise polynomial approximation $q(x)$, given by

$$q(x) = q_k(x) \quad \text{if } x \in \Omega_k, \quad (9.82)$$

the piecewise polynomial reconstruction of $u(x)$ based on the cell averages u_k .

A finite-volume method based on this high-order reconstruction achieves accuracy of order $\mathcal{O}(h_x^p)$, except near discontinuities or other points where the solution is not sufficiently smooth. The ENO reconstruction procedure chooses the polynomials $q_k(x)$ in each cell Ω_k in such a way that they have favorable spurious oscillation properties near shocks. Recall that a high-order reconstruction with centered stencils leads to $\mathcal{O}(1)$ oscillations around discontinuities (see Figure 6.2.1). The main inspiration behind ENO finite-volume schemes is that, ideally, one would like to obtain high-order numerical approximations, $\mathbf{u}_\ell$, of conservation laws that satisfy the ENO condition,

$$TV(\mathbf{u}_{\ell+1}) \leq TV(\mathbf{u}_\ell) + \mathcal{O}(h_x^{p+1}); \quad (9.83)$$

i.e., spurious oscillations in $\mathbf{u}_{\ell+1}$ relative to $\mathbf{u}_\ell$ can be at most $\mathcal{O}(h_x^{p+1})$ in magnitude [126].

To pursue this, the ENO approach chooses the polynomial $q_k(x)$ in each cell, Ω_k , using the so-called *ENO reconstruction procedure*, which is based on applying the *ENO interpolation procedure* to the primitive function (antiderivative) of $u(x)$, denoted by $U(x)$ [199]. The piecewise polynomials resulting from the ENO interpolation procedure and the ENO reconstruction procedure both provably satisfy, under some conditions, ENO properties similar to the ENO property of equation (9.83). While, in the end, proving a property like equation (9.83) for the resulting

ENO finite-volume methods has turned out to be elusive and the theoretical understanding of the oscillation behavior remains incomplete, ENO finite-volume methods have been very successful for many applications, achieving high-order accuracy away from shocks while showing minimal spurious oscillations.

More specifically, the ENO reconstruction procedure proceeds as follows. For each cell, Ω_k , we consider p different stencils, each based on p cell averages, where these p stencils are shifted increasingly to the left and right relative to centered stencils; see Figure 9.4.1 for an example with $p = 3$. If the grid is sufficiently refined, and we choose the stencil where the function is the “smoothest” for each cell Ω_k , then the ENO procedure obtains a suitable polynomial, $q_k(x)$, on each cell to construct a piecewise polynomial approximation $q(x)$ that has accuracy $\mathcal{O}(h_x^p)$ except in cells where $u(x)$ is nonsmooth and that has small spurious oscillations that are expected to decrease as the grid is refined.

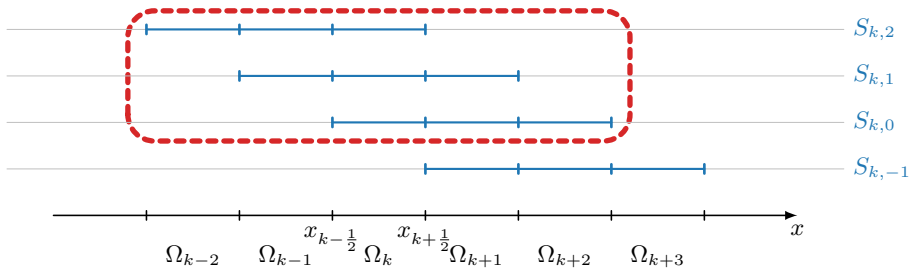


Figure 9.4.1. Stencils for the ENO and WENO procedures to determine polynomials $\phi_{k,r}(x)$ of degree $\leq p - 1$ to be used in finite-volume cell Ω_k , approximating a piecewise smooth function $u(x)$ with accuracy $\mathcal{O}(h_x^p)$ where $u(x)$ is smooth. Here, $p = 3$, and each stencil $S_{k,r}$ contains p finite-volume cells, and the p cell averages in these cells are used to determine the polynomial $\phi_{k,r}(x)$ corresponding to stencil $S_{k,r}$. The offset, $r = 0, \dots, p - 1$, is where the stencil starts to the left of cell Ω_k . The stencils $S_{k,0}$ to $S_{k,p-1}$ are used in the ENO and WENO procedures, and $S_{k,-1}$ is considered additionally for practical implementation of the methods.

The first task is to compute, for each cell Ω_k , p polynomials $\phi_{k,r}(x)$ of degree at most $p - 1$ that are determined by p cell averages on shifted stencils. The shifted stencils for cell Ω_k are given by

$$S_{k,r} = \{\Omega_{k-r}, \dots, \Omega_{k-r+p-1}\}, \quad r = 0, 1, \dots, p - 1, \quad (9.84)$$

and we seek to compute a polynomial $\phi_{k,r}(x)$ of degree $\leq p - 1$ for each stencil $S_{k,r}$ that is an $\mathcal{O}(h_x^p)$ approximation of a smooth function, $u(x)$, where the polynomial coefficients of $\phi_{k,r}(x)$ are determined using the p cell averages in stencil $S_{k,r}$. We limit our discussion to ENO schemes that use the p stencils with offsets $r = 0, 1, \dots, p - 1$, symmetric about Ω_k , but, as explained later, it is convenient for computations to also consider an additional stencil with offset $r = -1$. In the next step, the ENO reconstruction procedure selects the stencil and polynomial where the function is the “smoothest” for each cell, Ω_k .

Finally, we also consider so-called *weighted* ENO (WENO) schemes, which employ weighted linear combinations of the p polynomials $\phi_{k,r}(x)$ on stencils $S_{k,r}$, $r = 0, 1, \dots, p - 1$. These methods effectively use $2p - 1$ cell averages symmetrically about cell Ω_k to obtain finite-volume accuracy $\mathcal{O}(h_x^{2p-1})$. WENO schemes aim to preserve the ENO property by judiciously choosing the weights in the linear combination to remove the contribution of polynomials $\phi_{k,r}(x)$ on stencils where $u(x)$ is nonsmooth.

9.4.1 ■ Stencils for high-order reconstruction

Consider a piecewise smooth function, $u(x)$, and a grid with finite-volume cells, Ω_k , of equal length, h_x . For each cell Ω_k , we seek polynomials, $\phi_{k,r}(x)$, of degree $\leq p-1$, using the p cell averages, u_j , of $u(x)$ in the stencils, $S_{k,r}$. Since we are dealing with cell averages, which are obtained by integrating $u(x)$, we consider $U(x)$, the primitive function (antiderivative) of $u(x)$,

$$U(x) = \int_c^x u(\xi) d\xi, \quad (9.85)$$

for some constant reference point, c . Then, the cell average u_j over cell Ω_j can be computed using a finite differencing of $U(x)$:

$$u_j = \frac{\int_{\Omega_j} u(\xi) d\xi}{h_x} = \frac{U(x_{j+\frac{1}{2}}) - U(x_{j-\frac{1}{2}})}{h_x}. \quad (9.86)$$

Polynomial $\phi_{k,r}(x)$ is constructed on stencil $S_{k,r}$ by requiring that it have the same cell averages as $u(x)$:

$$\frac{\int_{\Omega_j} \phi_{k,r}(\xi) d\xi}{h_x} = u_j \quad \text{for } j = k-r, \dots, k-r+p-1. \quad (9.87)$$

It is then easy to see how $\phi_{k,r}(x)$ is computed by considering its primitive function, $\Phi_{k,r}(x)$, given by

$$\Phi_{k,r}(x) = \int_c^x \phi_{k,r}(\xi) d\xi, \quad (9.88)$$

where $\Phi_{k,r}(x)$ is now a polynomial of degree $\leq p$, since $\phi_{k,r}(x)$ is a polynomial of degree $\leq p-1$. If $\Phi_{k,r}(x)$ is chosen to interpolate $U(x)$ at the $p+1$ points $x_{k-r-\frac{1}{2}}, \dots, x_{k-r+p-1+\frac{1}{2}}$, then $\phi_{k,r}(x) = \Phi'_{k,r}(x)$ satisfies the cell averages condition in equation (9.87), since

$$\frac{\int_{\Omega_j} \phi_{k,r}(\xi) d\xi}{h_x} = \frac{\Phi_{k,r}(x_{j+\frac{1}{2}}) - \Phi_{k,r}(x_{j-\frac{1}{2}})}{h_x} = \frac{U(x_{j+\frac{1}{2}}) - U(x_{j-\frac{1}{2}})}{h_x} = u_j. \quad (9.89)$$

So instead of letting $\phi_{k,r}(x)$ interpolate the cell averages u_k of $u(x)$ in cell centers, we let its primitive function $\Phi_{k,r}(x)$ interpolate the values of the primitive function $U(x)$ at the cell interfaces, such that $\phi_{k,r}(x)$ has the same cell averages as $u(x)$, as expressed in equation (9.89).

From standard polynomial interpolation theory, we then have, if $u(x)$ is sufficiently smooth on $S_{k,r}$,

$$\Phi_{k,r}(x) = U(x) + \mathcal{O}(h_x^{p+1}) \quad \forall x \in S_{k,r}, \quad (9.90)$$

and, since $\phi_{k,r}(x) = \Phi'_{k,r}(x)$,

$$\phi_{k,r}(x) = u(x) + \mathcal{O}(h_x^p) \quad \forall x \in S_{k,r}. \quad (9.91)$$

We compute $\Phi_{k,r}(x)$ using the Lagrange form of the interpolating polynomial:

$$\Phi_{k,r}(x) = \sum_{m=0}^p U(x_{k-r+m-\frac{1}{2}}) \prod_{l=0, l \neq m}^p \frac{x - x_{k-r+l-\frac{1}{2}}}{x_{k-r+m-\frac{1}{2}} - x_{k-r+l-\frac{1}{2}}}. \quad (9.92)$$

To simplify further computations, we use the identity that

$$\sum_{m=0}^p \prod_{l=0, l \neq m}^p \frac{x - x_{k-r+l-\frac{1}{2}}}{x_{k-r+m-\frac{1}{2}} - x_{k-r+l-\frac{1}{2}}} = 1 \quad (9.93)$$

and the fact that we consider equispaced grids to obtain

$$\Phi_{k,r}(x) - U(x_{k-r-\frac{1}{2}}) = \sum_{m=0}^p \left(U(x_{k-r+m-\frac{1}{2}}) - U(x_{k-r-\frac{1}{2}}) \right) \prod_{l=0, l \neq m}^p \frac{x - x_{k-r+l-\frac{1}{2}}}{(m-l)h_x}. \quad (9.94)$$

Finally, rewriting the difference in the $U(x)$ values in equation (9.94) in terms of a sum over cell averages,

$$U(x_{k-r+m-\frac{1}{2}}) - U(x_{k-r-\frac{1}{2}}) = h_x \sum_{j=0}^{m-1} u_{k-r+j}, \quad (9.95)$$

we take the derivative of equation (9.94) to obtain

$$\phi_{k,r}(x) = \sum_{m=0}^p h_x \left(\sum_{j=0}^{m-1} u_{k-r+j} \right) \left(\frac{\sum_{l=0, l \neq m}^p \prod_{q=0, q \neq m, l}^p (x - x_{k-r+q-\frac{1}{2}})}{\prod_{l=0, l \neq m}^p (m-l)h_x} \right). \quad (9.96)$$

The reconstructed value at the right interface of cell Ω_k is then given by

$$\phi_{k,r}(x_{k+\frac{1}{2}}) = \sum_{j=0}^{p-1} c_{r,j} u_{k-r+j}, \quad (9.97)$$

with

$$c_{r,j} = \sum_{m=j+1}^p \left(\frac{\sum_{l=0, l \neq m}^p \prod_{q=0, q \neq m, l}^p (r-q+1)}{\prod_{l=0, l \neq m}^p (m-l)} \right). \quad (9.98)$$

To compute the reconstructed value $\phi_{k,r}(x_{k-\frac{1}{2}})$ at the left interface of cell Ω_k , we use

$$\phi_{k,r}(x_{k-\frac{1}{2}}) = \sum_{j=0}^{p-1} c_{r-1,j} u_{k-r+j}. \quad (9.99)$$

In the above derivation, we see that the reconstruction at $x_{k+\frac{1}{2}}$, which is the right boundary of cell Ω_k , is given by equation (9.97) for stencil $S_{k,r}$. Clearly, $x_{k-\frac{1}{2}}$ is the right boundary of cell Ω_{k-1} , so $\phi_{k,r}(x_{k-\frac{1}{2}})$ is given by $\phi_{k-1,r-1}(x_{k-\frac{1}{2}})$ for stencil $S_{k-1,r-1} = S_{k,r}$, which allows us to obtain equation (9.99) from equation (9.97), since $u_{(k-1)-(r-1)+j} = u_{k-r+j}$ and the $c_{r,j}$ do not depend on k for the case of constant grid spacing.

Table 9.4.1 gives the coefficients $c_{r,j}$ from equation (9.98) for $p = 1, 2, 3, 4$ and $-1 \leq r \leq p-1$. Notably, $c_{r,j}$ satisfies a symmetry condition of $c_{r,j} = c_{p-2-r,p-1-j}$. In the following, we limit our consideration to finite-volume methods that consider the p polynomials $\phi_{k,r}(x)$ on the stencils $S_{k,r}$ for $r = 0, 1, \dots, p-1$ that are symmetric about Ω_k . However, Table 9.4.1 also gives the coefficients for the stencils with $r = -1$ (see Figure 9.4.1), since they are useful for computing $\phi_{k,r}(x_{k-\frac{1}{2}})$ for the $S_{k,0}$ stencil using equation (9.99).

Table 9.4.1. Coefficients $c_{r,j}$ from equation (9.98) for high-order polynomial reconstruction in equation (9.97) using stencils with offset r as in Figure 9.4.1 for stencils with $p = 1, 2, 3, 4$ finite-volume cells and offsets $r = -1, 0, 1, \dots, p-1$.

p	r	$j=0$	$j=1$	$j=2$	$j=3$	ENO order (p)	WENO order ($2p-1$)
1	-1	1				1	1
	0	1					
2	-1	3/2	-1/2			2	3
	0	1/2	1/2				
	1	-1/2	3/2				
3	-1	11/6	-7/6	1/3		3	5
	0	1/3	5/6	-1/6			
	1	-1/6	5/6	1/3			
	2	1/3	-7/6	11/6			
4	-1	25/12	-23/12	13/12	-1/4	4	7
	0	1/4	13/12	-5/12	1/12		
	1	-1/12	7/12	7/12	-1/12		
	2	1/12	-5/12	13/12	1/4		
	3	-1/4	13/12	-23/12	25/12		

9.4.2 ■ ENO reconstruction

The ENO reconstruction procedure selects, for each cell Ω_k , the smoothest stencil among the p stencils $S_{k,r}$, $r = 0, \dots, p-1$, with p cells and uses the corresponding polynomial $\phi_{k,r}(x)$ of degree $\leq p-1$ for the reconstruction of $u(x)$ at points $x_{k-\frac{1}{2}}$ and $x_{k+\frac{1}{2}}$ with accuracy $\mathcal{O}(h_x^p)$ when $u(x)$ is sufficiently smooth. While we are primarily interested in polynomials that reconstruct $u(x)$, we continue to follow the procedure above and look for the smoothest polynomial, $\Phi_{k,r}(x)$ of degree $\leq p$, that interpolates the antiderivative, $U(x)$, at the cell interfaces. To understand the ENO interpolation procedure, it is useful to consider the Newton form of the interpolating polynomial instead of the Lagrange form given above. Here, we write

$$\Phi_{k,r}(x) = \sum_{j=0}^p U[x_{k-r-\frac{1}{2}}, \dots, x_{k-r+j-\frac{1}{2}}] \prod_{m=0}^{j-1} (x - x_{k-r+m-\frac{1}{2}}) \quad (9.100)$$

as the polynomial of degree $\leq p$ that interpolates $U(x)$, where $U[x_{k-r-\frac{1}{2}}, \dots, x_{k-r+j-\frac{1}{2}}]$ is the Newton divided difference of degree j , defined recursively as

$$\text{degree } 0: \quad U[x_{k-\frac{1}{2}}] = U(x_{k-\frac{1}{2}}), \quad (9.101a)$$

$$\text{degree } j: \quad U[x_{k-\frac{1}{2}}, \dots, x_{k+j-\frac{1}{2}}] = \frac{U[x_{k+\frac{1}{2}}, \dots, x_{k+j-\frac{1}{2}}] - U[x_{k-\frac{1}{2}}, \dots, x_{k+j-\frac{3}{2}}]}{x_{k+j-\frac{1}{2}} - x_{k-\frac{1}{2}}}. \quad (9.101b)$$

In principle, it is slightly cumbersome to use this form of divided differences, because we are focused on approximating $u(x)$ and not $U(x)$; however, we have the simple relationship that

$$U[x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}] = \frac{U[x_{k+\frac{1}{2}}] - U[x_{k-\frac{1}{2}}]}{x_{k+\frac{1}{2}} - x_{k-\frac{1}{2}}} = u_k, \quad (9.102)$$

which allows us to compute the divided differences of $U(x)$ of degree one and higher using the values u_k . Using equation (9.102) to evaluate the differences in equation (9.100), we write the polynomial reconstructions of $u(x)$ as

$$\phi_{k,r}(x) = \Phi'_{k,r}(x) = \sum_{j=1}^p U[x_{k-r-\frac{1}{2}}, \dots, x_{k-r+j-\frac{1}{2}}] \sum_{m=0}^{j-1} \prod_{l=0, l \neq m}^{j-1} (x - x_{k-r+l-\frac{1}{2}}), \quad (9.103)$$

noting that this does, indeed, depend only on divided differences of $U(x)$ of degree one and higher.

The main motivation to use the Newton form of the interpolating polynomials is the following property of divided differences:

$$\begin{cases} U[x_{k-\frac{1}{2}}, \dots, x_{k+j-\frac{1}{2}}] = \frac{U^{(j)}(\xi)}{j!} & \text{if } U(x) \text{ is smooth in } \Omega, \\ U[x_{k-\frac{1}{2}}, \dots, x_{k+j-\frac{1}{2}}] = \mathcal{O}\left(\frac{1}{h_x^j}\right) & \text{if } U(x) \text{ is discontinuous in } \Omega, \end{cases} \quad (9.104)$$

where $\xi \in \Omega = (x_{k-\frac{1}{2}}, x_{k+j-\frac{1}{2}})$. Thus, we define the stencil of the smoothest approximation of $U(x)$ as being the one that leads to the smallest divided-difference value. The ENO procedure to select the smoothest stencil, $S_{k,r}$, for cell Ω_k then proceeds as follows. We start from the base stencil $S_k = \{\Omega_k\}$ and extend S_k either on the left or the right by choosing

$$\begin{cases} S_k = \{\Omega_{k-1}, \Omega_k\} & \text{if } \left| U[x_{k-\frac{3}{2}}, x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}] \right| < \left| U[x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}, x_{k+\frac{3}{2}}] \right|, \\ S_k = \{\Omega_k, \Omega_{k+1}\} & \text{otherwise.} \end{cases} \quad (9.105)$$

We then continue adding cells on the left or right using the same procedure with divided differences of increasingly higher degree until the stencil contains p cells. Finally, the polynomial $\phi_{k,r}(x)$ is computed on the resulting smoothest stencil, $S_{k,r}$.

ENO-satisfying polynomial reconstruction

To provide some insight into how the ENO reconstruction procedure described above is related to ENO properties, we now explain how the ENO interpolation of a piecewise smooth function $U(x)$ satisfies an ENO condition. Here, we specifically consider the special case of a function $U(x)$ that is infinitely differentiable except at a finite number of jump discontinuities. Note that this does not directly relate to the situation where $u(x)$ is piecewise smooth, which is the relevant case for solutions of hyperbolic conservation laws with shocks, but it illustrates the intuition that lies at the basis of the ENO reconstruction approach.

Consider $U(x)$ on domain $\Omega = \bigcup \Omega_k$. We approximate $U(x)$ with the piecewise polynomial $Q(x)$ defined by

$$Q(x) = \Phi_{k,r}(x) \quad \text{if } x \in \Omega_k, \quad (9.106)$$

where $\Phi_{k,r}(x)$ is the polynomial of degree $\leq p$ on stencil $S_{k,r}$ that is determined, for each cell Ω_k , by the ENO stencil-selection procedure described above. We choose the (still-uniform) grid to be fine enough so that, for all cells Ω_k , $U(x)$ is fully smooth on the stencil $S_{k,r}$, as chosen by the ENO procedure, unless Ω_k is a “shocked cell” where $U(x)$ contains a jump discontinuity. This is just to ensure that every cell that does not contain a jump discontinuity has a stencil, $S_{k,r}$, that samples only smooth values of $U(x)$. Then, we have

$$\Phi_{k,r}(x) = U(x) + \mathcal{O}(h_x^{p+1}) \quad \forall x \in \Omega_k, \quad (9.107)$$

unless $U(x)$ has a discontinuity in Ω_k . If the grid and $Q(x)$ are chosen in this way, then $Q(x)$ satisfies the following ENO property: $\exists Z(x)$ such that

$$Q(x) = Z(x) + \mathcal{O}(h_x^{p+1}) \quad \forall x \in \Omega, \quad (9.108a)$$

$$TV(Z(x); \Omega) \leq TV(U(x); \Omega). \quad (9.108b)$$

This means that $Q(x)$ is $\mathcal{O}(h_x^{p+1})$ -close to $Z(x)$ and $Z(x)$ does not have spurious oscillations relative to $U(x)$. This is interpreted as the spurious oscillations in $Q(x)$ being at most $\mathcal{O}(h_x^{p+1})$ and disappearing with the order of the approximation as the grid is refined.

The function $Z(x)$ in equation (9.108) can simply be chosen as follows. In smooth cells, we choose $Z(x) = U(x)$; this satisfies the property expressed in equation (9.108a), using equation (9.107), and equation (9.108b) is automatically satisfied. In shocked cells, we choose $Z(x) = \Phi_{k,r}(x)$, so equation (9.108a) is automatically satisfied and the property in equation (9.108b) follows because the ENO polynomial, $\Phi_{k,r}(x)$, is monotone in Ω_k when Ω_k is a shocked cell. Hence,

$$TV(\Phi_{k,r}(x); \Omega_k) = \left| U(x_{k+\frac{1}{2}}) - U(x_{k-\frac{1}{2}}) \right| \leq TV(U(x); \Omega_k). \quad (9.109)$$

Theorem 9.11 shows that any of the polynomials, $\Phi_{k,r}(x)$, are monotone in a shocked cell Ω_k for the special, prototypical case of $U(x)$ being the Heaviside function; the proof can be extended to the case of a more general jump discontinuity (see [126]).

Theorem 9.11: ENO monotonicity in shocked cell for Heaviside function [126, Thm. 4.1]

Let

$$U(x) = \begin{cases} 0 & \text{if } x \leq 0, \\ 1 & \text{if } x > 0. \end{cases} \quad (9.110)$$

Assume $\Phi(x)$ interpolates $U(x)$ at $p+1$ equidistant points,

$$x_{\frac{1}{2}} < x_{\frac{3}{2}} < \cdots < x_{p+\frac{1}{2}}, \quad (9.111)$$

and the discontinuity lies in cell

$$\Omega_{j_0} = (x_{j_0-\frac{1}{2}}, x_{j_0+\frac{1}{2}}) \quad \text{for some } j_0 \text{ with } 1 \leq j_0 \leq p. \quad (9.112)$$

Then, $\Phi(x)$ is monotone in Ω_{j_0} .

Proof. For every cell Ω_j with $j \neq j_0$, we have

$$\Phi(x_{j-\frac{1}{2}}) = U(x_{j-\frac{1}{2}}) = U(x_{j+\frac{1}{2}}) = \Phi(x_{j+\frac{1}{2}}). \quad (9.113)$$

Since $\Phi(x)$ is of degree $\leq p$, it has at most p distinct roots, so $\Phi'(x)$ is of degree $\leq p-1$ and has at most $p-1$ distinct roots. Equation (9.113) indicates that $\Phi'(x)$ has $p-1$ distinct roots in the $p-1$ intervals Ω_j with $j \neq j_0$. Therefore, $\Phi'(x)$ does not have a root in Ω_{j_0} and, hence, $\Phi(x)$ is monotone in Ω_{j_0} . $\square$

As is shown under certain conditions in [126], similar total-variation boundedness properties hold for the ENO interpolation and the ENO reconstruction polynomials with more generality; see also the discussion in [199]. A significant general theoretical result that is known for the stability properties of the ENO reconstruction is the so-called *sign property* [131, 199], which shows that the sign of the jump in cell averages from u_k to u_{k+1} at interface $k + \frac{1}{2}$ is maintained in the reconstructed interface values. This precludes, for example, the reconstruction introducing sawtooth patterns when the cell averages are monotone. The sign property has been used to construct specialized ENO schemes that are provably entropy stable [131, 199]. However, it has not been possible to show ENO bounds as in equation (9.83) or other stability results for the high-order numerical approximations, $\mathbf{u}_\ell$, resulting from standard ENO finite-volume schemes, despite the widely observed success of these methods in practice.

9.4.3 ■ WENO reconstruction

We now take linear combinations of the p polynomials, $\phi_{k,r}(x)$, for each cell Ω_k to obtain a global approximation with accuracy $\mathcal{O}(h_x^{2p-1})$ for smooth $u(x)$ and discuss how the coefficients in the linear combination are chosen to preserve ENO properties. Such schemes are known as *weighted* ENO methods, or WENO schemes.

Combining p high-order stencils to obtain order h_x^{2p-1}

For cell Ω_k , we start from the p stencils $S_{k,r}$, $r = 0, \dots, p-1$, each with p cells, that are used in the ENO procedure to determine the p polynomials $\phi_{k,r}(x)$ of degree $\leq p-1$ with approximation accuracy $\mathcal{O}(h_x^p)$. If $u(x)$ was known to be smooth, then we could take a linear combination of the values of the p ENO polynomials at point $x_{k+\frac{1}{2}}$ to obtain an approximation, $v_{k+\frac{1}{2}}$, that is $\mathcal{O}(h_x^{2p-1})$ accurate:

$$v_{k+\frac{1}{2}} = \sum_{r=0}^{p-1} d_r \phi_{k,r}(x_{k+\frac{1}{2}}) = u(x_{k+\frac{1}{2}}) + \mathcal{O}(h_x^{2p-1}). \quad (9.114)$$

It is fairly easy to find coefficients d_r that achieve the desired $\mathcal{O}(h_x^{2p-1})$ accuracy. We rewrite equation (9.97) to indicate that the ENO weights $c_{r,j}^{(p)}$ are for stencil size p :

$$\phi_{k,r}(x_{k+\frac{1}{2}}) = \sum_{j=0}^{p-1} c_{r,j}^{(p)} u_{k-r+j}. \quad (9.115)$$

Substituting this into equation (9.114), we have that

$$v_{k+\frac{1}{2}} = \sum_{r=0}^{p-1} d_r \sum_{j=0}^{p-1} c_{r,j}^{(p)} u_{k-r+j}. \quad (9.116)$$

On the other hand, we want $v_{k+\frac{1}{2}}$ to be accurate with $\mathcal{O}(h_x^{2p-1})$, and we know a suitable polynomial reconstruction with that property using the stencil $\bigcup_{r=0}^{p-1} S_{k,r}$, namely, the high-order polynomial with stencil size $2p-1$ that is central about cell Ω_k , so we require

$$v_{k+\frac{1}{2}} = \sum_{j=0}^{2p-2} c_{p-1,j}^{(2p-1)} u_{k-(p-1)+j}. \quad (9.117)$$

Identifying the coefficients of $u_{k-(p-1)+j}$ for $j = 0, \dots, 2p-2$ in equation (9.117) with the corresponding coefficients in equation (9.116) then leads to $2p-1$ linear equations for the p

Table 9.4.2. Coefficients d_r in equation (9.114) for WENO reconstruction with order $2p - 1$, using p ENO polynomials with accuracy of order p for ENO stencils with $p = 1, 2, 3, 4$ finite-volume cells. The coefficients are computed by solving the linear system in equation (9.118).

p	d_0	d_1	d_2	d_3	WENO order ($2p - 1$)
1	1				1
2	2/3	1/3			3
3	3/10	6/10	1/10		5
4	4/35	18/35	12/35	1/35	7

unknowns d_r , $r = 0, \dots, p - 1$. As it turns out, due to symmetry in the $c_{r,j}$ coefficients, we only need to consider the first p equations, which are expressed, after some work, as

$$\sum_{r=0}^{p-1-l} d_r c_{r,r+l}^{(p)} = c_{p-1,p-1+l}^{(2p-1)} \quad \text{for } l = 0, \dots, p - 1. \quad (9.118)$$

Table 9.4.2 lists the solutions d_r of this linear system for $p = 1, 2, 3, 4$. Note that

$$\sum_{r=0}^{p-1} d_r = 1. \quad (9.119)$$

This reflects a basic consistency in the algorithm. If $u(x)$ is a constant function in the neighborhood around cell Ω_k that includes all of the ENO stencils considered, then (by construction) $\phi_{k,r}(x_{k+\frac{1}{2}})$ takes the same constant value as $u(x)$ does in that neighborhood. In this case, we expect $v_{k+\frac{1}{2}}$ to also match this value, which is implied by equation (9.119). Finally, at $x_{k-\frac{1}{2}}$, we use

$$v_{k-\frac{1}{2}} = \sum_{r=0}^{p-1} \bar{d}_r \phi_{k,r}(x_{k-\frac{1}{2}}) = u(x_{k-\frac{1}{2}}) + \mathcal{O}(h_x^{2p-1}), \quad (9.120)$$

where, by symmetry,

$$\bar{d}_r = d_{p-1-r}. \quad (9.121)$$

Choosing the WENO linear combination to avoid oscillations

We now consider changing the coefficients of the linear combination in equation (9.114), aiming to mimic the nonoscillatory properties of the ENO procedure while attempting to maintain $\mathcal{O}(h_x^{2p-1})$ accuracy where possible. To do so, we determine modified coefficients, ω_r , that reduce the contributions of stencils $S_{k,r}$ defined over regions on which the solution is nonsmooth, determining the modified reconstruction $\hat{u}_{k+\frac{1}{2}}$ as

$$\hat{u}_{k+\frac{1}{2}} = \sum_{r=0}^{p-1} \omega_r \phi_{k,r}(x_{k+\frac{1}{2}}), \quad (9.122)$$

with

$$\sum_{r=0}^{p-1} \omega_r = 1 \quad \text{and} \quad \omega_r \geq 0 \quad \forall r. \quad (9.123)$$

If we find a way to choose the modified coefficients ω_r such that

$$\omega_r = d_r + \mathcal{O}(h_x^{p-1}) \quad (9.124)$$

when the solution is smooth, then, using equations (9.119) and (9.123), we have

$$\hat{u}_{k+\frac{1}{2}} - v_{k+\frac{1}{2}} = \sum_{r=0}^{p-1} (\omega_r - d_r) \phi_{k,r}(x_{k+\frac{1}{2}}) - \sum_{r=0}^{p-1} (\omega_r - d_r) u(x_{k+\frac{1}{2}}) \quad (9.125a)$$

$$= \sum_{r=0}^{p-1} (\omega_r - d_r) (\phi_{k,r}(x_{k+\frac{1}{2}}) - u(x_{k+\frac{1}{2}})) \quad (9.125b)$$

$$= \mathcal{O}(h_x^{2p-1}), \quad (9.125c)$$

leading to

$$\hat{u}_{k+\frac{1}{2}} = u(x_{k+\frac{1}{2}}) + \mathcal{O}(h_x^{2p-1}), \quad (9.126)$$

by equation (9.114).

A suitable way to choose the ω_r is as follows:

$$\omega_r = \frac{\alpha_r}{\sum_{s=0}^{p-1} \alpha_s} \quad \text{with} \quad \alpha_r = \frac{d_r}{(\epsilon + \beta_r)^2}, \quad (9.127)$$

where typically $\epsilon = 10^{-6}$, but other choices exist. Here, the β_r are called *smoothness indicators* for the stencils $S_{k,r}$ and are chosen to satisfy

$$\beta_r = \begin{cases} \mathcal{O}(h_x^2) & \text{if } u(x) \text{ is smooth on } S_{k,r}, \\ \mathcal{O}(1) & \text{otherwise.} \end{cases} \quad (9.128)$$

The β_r are chosen to measure the variation in $\phi_{k,r}(x)$ by considering higher-order derivatives, similar to measuring total variation, and to how ENO uses approximations of the higher-order derivatives of $U(x)$ as in equation (9.104). In particular, a suitable choice for the β_r is

$$\beta_r = \sum_{l=1}^{p-1} \int_{x_{k-\frac{1}{2}}}^{x_{k+\frac{1}{2}}} h_x^{2l-1} \left(\frac{\partial^l \phi_{k,r}(x)}{\partial x^l} \right)^2 dx. \quad (9.129)$$

This gives, for $p = 2$ and $p = 3$,

$$p = 2: \quad \beta_0 = (u_{k+1} - u_k)^2, \quad \beta_1 = (u_k - u_{k-1})^2, \quad (9.130a)$$

$$p = 3: \quad \beta_0 = \frac{13}{12}(u_k - 2u_{k+1} + u_{k+2})^2 + \frac{1}{4}(3u_k - 4u_{k+1} + u_{k+2})^2, \quad (9.130b)$$

$$\beta_1 = \frac{13}{12}(u_{k-1} - 2u_k + u_{k+1})^2 + \frac{1}{4}(u_{k-1} - u_{k+1})^2, \quad (9.130c)$$

$$\beta_2 = \frac{13}{12}(u_{k-2} - 2u_{k-1} + u_k)^2 + \frac{1}{4}(u_{k-2} - 4u_{k-1} + 3u_k)^2. \quad (9.130d)$$

It can be shown that this choice implies the accuracy condition of equation (9.124). At $x_{k-\frac{1}{2}}$ we use, in a similar fashion,

$$\bar{\omega}_r = \frac{\bar{\alpha}_r}{\sum_{s=0}^{p-1} \bar{\alpha}_s} \quad \text{with} \quad \bar{\alpha}_r = \frac{\bar{d}_r}{(\epsilon + \beta_r)^2}. \quad (9.131)$$

In summary, for cell Ω_k , we get the WENO reconstructions of equation (9.122) and

$$\hat{u}_{k-\frac{1}{2}} = \sum_{r=0}^{p-1} \bar{\omega}_r \phi_{k,r}(x_{k-\frac{1}{2}}). \quad (9.132)$$

Finite-volume WENO scheme

We now use the WENO reconstructions of equations (9.122) and (9.132) in a finite-volume method. For the general finite-volume formula in equation (6.40) in semidiscrete form, we use the numerical flux formula at cell interface $k + \frac{1}{2}$ given by

$$\widehat{f}_{k+\frac{1}{2}}(t) = f^*(u_{k+\frac{1}{2}}^-(t), u_{k+\frac{1}{2}}^+(t)). \quad (9.133)$$

For example, we use the Lax–Friedrichs flux of equation (6.59). Here, the left interface value $u_{k+\frac{1}{2}}^-(t)$ is computed using the WENO reconstruction in cell Ω_k , evaluated at the right interface of cell Ω_k , using equation (9.122):

$$u_{k+\frac{1}{2}}^-(t) = \widehat{u}_{k+\frac{1}{2}}. \quad (9.134)$$

The right interface value $u_{k+\frac{1}{2}}^+(t)$ in equation (9.133) is computed using the WENO reconstruction in cell Ω_{k+1} , evaluated at the left interface of cell Ω_{k+1} , using equation (9.132):

$$u_{k+\frac{1}{2}}^+(t) = \widehat{u}_{(k+1)-\frac{1}{2}}. \quad (9.135)$$

9.4.4 ■ TVD Runge–Kutta methods for high-order time integration

While the ENO and WENO procedures carefully construct high-order polynomials in space that avoid large spurious oscillations in the approximations of $u_{k+\frac{1}{2}}^-$ and $u_{k+\frac{1}{2}}^+$ near discontinuities, further care must be taken so that high-order time integration does not reintroduce harmful oscillations. Consider the vector form of semidiscrete equation (6.40) that is the result of finite-volume discretization in space using ENO or WENO:

$$\frac{d\mathbf{u}(t)}{dt} + \mathbf{z}(\mathbf{u}) = \mathbf{0}. \quad (9.136)$$

We first consider time integration using the explicit Euler method, which is first-order accurate:

$$\mathbf{u}_{\ell+1} = \mathbf{u}_\ell - h_t \mathbf{z}(\mathbf{u}_\ell). \quad (9.137)$$

For the ENO and WENO methods that we consider here, it is often shown that this discrete evolution equation satisfies a stability property under a CFL condition, i.e.,

$$\|\mathbf{u}_{\ell+1}\| \leq \|\mathbf{u}_\ell\| \quad (9.138a)$$

$$\text{if } h_t \leq h_t^*. \quad (9.138b)$$

The stability condition may hold, e.g., in the discrete maximum norm, and for some schemes (e.g., certain low-order schemes) it may also hold in the TVD seminorm.

Since we aim to couple the high-order accuracy in space of an ENO or WENO scheme with high-order accuracy in time, we next consider the following family of s -stage explicit Runge–Kutta (RK) methods for integrating equation (9.136):

$$\mathbf{u}^{(0)} = \mathbf{u}_\ell, \quad (9.139a)$$

$$\mathbf{u}^{(i)} = \sum_{j=0}^{i-1} \left(\alpha_{i,j} \mathbf{u}^{(j)} - h_t \beta_{i,j} \mathbf{z}(\mathbf{u}^{(j)}) \right), \quad i = 1, \dots, s, \quad (9.139b)$$

$$\mathbf{u}_{\ell+1} = \mathbf{u}^{(s)}. \quad (9.139c)$$

Note that, for purposes that will become clear shortly, the RK method in equation (9.139) is written in a nonstandard form (known as the Shu–Osher form); however, this form is equivalent to the standard form. Also, for consistency, it is required that

$$\sum_{j=0}^{i-1} \alpha_{i,j} = 1 \quad \forall i, \quad (9.140)$$

which is seen by applying the method to the ODE system $\mathbf{u}' = \mathbf{0}$ with a constant solution. Theorem 9.12 identifies high-order RK time-integration methods of the form in equation (9.139) that maintain the stability condition in equation (9.138a) under a modified CFL condition.

Theorem 9.12: Stability of s -stage TVDRK method [131, Thm. 9.3]

Assume that

$$\alpha_{i,j} \geq 0 \text{ and } \beta_{i,j} \geq 0 \text{ for all } i, j \quad (9.141)$$

in equation (9.139), and that $\beta_{i,j} = 0$ if $\alpha_{i,j} = 0$. Assume further that the explicit Euler method (see equation (9.137)) satisfies the stability property of equation (9.138a) under the CFL condition in equation (9.138b). Then, the RK method satisfies the stability condition summarized in equation (9.138a) with a modified CFL condition of $h_t \leq ch_t^*$, where

$$c = \min_{\substack{i,j \\ \alpha_{i,j} \neq 0}} \frac{\alpha_{i,j}}{\beta_{i,j}}. \quad (9.142)$$

We call such an RK method a TVDRK method, since these methods were originally derived in the context of TVD-stable schemes.

Proof. Observe that

$$\mathbf{u}^{(i)} = \sum_{\substack{j=0 \\ \alpha_{i,j} \neq 0}}^{i-1} \alpha_{i,j} \left(\mathbf{u}^{(j)} - h_t \frac{\beta_{i,j}}{\alpha_{i,j}} \mathbf{z}(\mathbf{u}^{(j)}) \right), \quad i = 1, \dots, s, \quad (9.143)$$

since $\alpha_{i,j} = 0$ implies that $\beta_{i,j} = 0$. If, for a fixed i ,

$$h_t \frac{\beta_{i,j}}{\alpha_{i,j}} \leq h_t^* \quad \text{for all } j \text{ with } \alpha_{i,j} \neq 0 \quad (9.144)$$

or, equivalently,

$$h_t \leq \frac{\alpha_{i,j}}{\beta_{i,j}} h_t^* \quad \text{for all } j \text{ with } \alpha_{i,j} \neq 0, \quad (9.145)$$

then $\mathbf{u}^{(i)}$ is a convex linear combination of explicit Euler updates that satisfy the stability condition in equation (9.138a). If h_t is chosen to satisfy equation (9.145) for all i , then the following induction proof establishes the desired result. First, observe that

$$\|\mathbf{u}^{(0)}\| = \|\mathbf{u}_\ell\|, \quad (9.146a)$$

$$\|\mathbf{u}^{(1)}\| = \left\| \alpha_{10} \left(\mathbf{u}^{(0)} - h_t \frac{\beta_{10}}{\alpha_{10}} \mathbf{z}(\mathbf{u}^{(0)}) \right) \right\| \quad (9.146b)$$

$$\leq \|\mathbf{u}^{(0)}\| = \|\mathbf{u}_\ell\|. \quad (9.146c)$$

Assume that $\|\mathbf{u}^{(j)}\| \leq \|\mathbf{u}_\ell\|$ for $j = 0, \dots, i-1$. Then,

$$\|\mathbf{u}^{(i)}\| = \left\| \sum_{j=0}^{i-1} \alpha_{i,j} \left(\mathbf{u}^{(j)} - h_t \frac{\beta_{i,j}}{\alpha_{i,j}} \mathbf{z}(\mathbf{u}^{(j)}) \right) \right\| \quad (9.147a)$$

$$\leq \sum_{j=0}^{i-1} \alpha_{i,j} \left\| \mathbf{u}^{(j)} - h_t \frac{\beta_{i,j}}{\alpha_{i,j}} \mathbf{z}(\mathbf{u}^{(j)}) \right\| \quad (9.147b)$$

$$\leq \sum_{j=0}^{i-1} \alpha_{i,j} \|\mathbf{u}^{(j)}\| \quad (9.147c)$$

$$\leq \sum_{j=0}^{i-1} \alpha_{i,j} \|\mathbf{u}_\ell\| = \|\mathbf{u}_\ell\|, \quad (9.147d)$$

so

$$\|\mathbf{u}_{\ell+1}\| = \|\mathbf{u}^{(s)}\| \leq \|\mathbf{u}_\ell\|. \quad (9.148)$$

□

We give two specific TVDRK methods satisfying $\alpha_{i,j} \geq 0$ and $\beta_{i,j} \geq 0$ for all i, j , with $s = 2$ and $s = 3$ stages. The two-stage method is given by

$$\mathbf{u}^{(1)} = \mathbf{u}_\ell - h_t \mathbf{z}(\mathbf{u}_\ell), \quad (9.149a)$$

$$\mathbf{u}_{\ell+1} = \frac{1}{2} \mathbf{u}_\ell + \frac{1}{2} \mathbf{u}^{(1)} - \frac{1}{2} h_t \mathbf{z}(\mathbf{u}^{(1)}). \quad (9.149b)$$

This method has accuracy $\mathcal{O}(h_t^2)$ with $c = 1$ in Theorem 9.12. The three-stage method is given by

$$\mathbf{u}^{(1)} = \mathbf{u}_\ell - h_t \mathbf{z}(\mathbf{u}_\ell), \quad (9.150a)$$

$$\mathbf{u}^{(2)} = \frac{3}{4} \mathbf{u}_\ell + \frac{1}{4} \mathbf{u}^{(1)} - \frac{1}{4} h_t \mathbf{z}(\mathbf{u}^{(1)}), \quad (9.150b)$$

$$\mathbf{u}_{\ell+1} = \frac{1}{3} \mathbf{u}_\ell + \frac{2}{3} \mathbf{u}^{(2)} - \frac{2}{3} h_t \mathbf{z}(\mathbf{u}^{(2)}) \quad (9.150c)$$

and has accuracy $\mathcal{O}(h_t^3)$ with $c = 1$ in Theorem 9.12. This method is widely used with ENO and WENO discretizations. These TVDRK methods also belong to a broader class of time-integration methods known as *strong stability-preserving* methods.

9.4.5 ■ Example: WENO for Burgers' equation in 1D

Consider Burgers' equation (6.7) with initial data given by

$$u(x, 0) = v(x) + s(x), \quad (9.151a)$$

$$v(x) = \begin{cases} -3 & \text{if } x \in [0, 0.5], \\ 1 & \text{if } x \in (0.5, 1], \end{cases} \quad (9.151b)$$

$$s(x) = \begin{cases} 0.1 \sin(2\pi 15x) & \text{if } x \in [8/15, 11/15], \\ 0 & \text{otherwise,} \end{cases} \quad (9.151c)$$

and with periodic boundary conditions on the interval $0 \leq x \leq 1$. The initial condition consists of two Riemann problems, at $x = 0.5$ and $x = 1$, with a localized, small-amplitude sinusoidal wave-train perturbation, $s(x)$, superimposed onto the constant state between the two jumps, as shown in Figure 9.4.2a. The two initial discontinuities at $x = 0.5$ and $x = 1$ develop into a rarefaction and a left-moving shock, respectively, as shown in Figure 9.4.2b for a solution obtained on a fine grid.

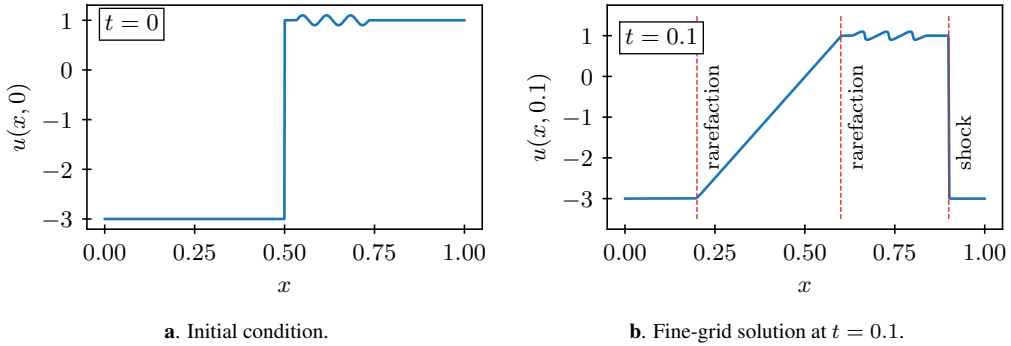


Figure 9.4.2. Initial condition and analytic front locations at $t = 0.1$ for Burgers' equation with WENO discretization.

The numerical results in Figure 9.4.3 are obtained with the WENO schemes of order 1, 3, and 5, from Section 9.4.3, using $p = 1, 2, 3$ and $h_x = 1/128$, and the third-order TVDRK method from equation (9.150). A reference solution is also provided using $p = 3$ at a resolution of $h_x = 1/1024$ for comparison of the accuracy of these methods. The numerical results confirm the absence of noticeable oscillations at the shock and at the weak discontinuities (i.e., the discontinuities in $u'(x)$) that border the rarefaction wave, as expected for the WENO scheme. Increasing the order of the scheme leads to substantial improvement in the accuracy of the rarefaction near the weak discontinuities and to a slight improvement in the steepness of the shock.

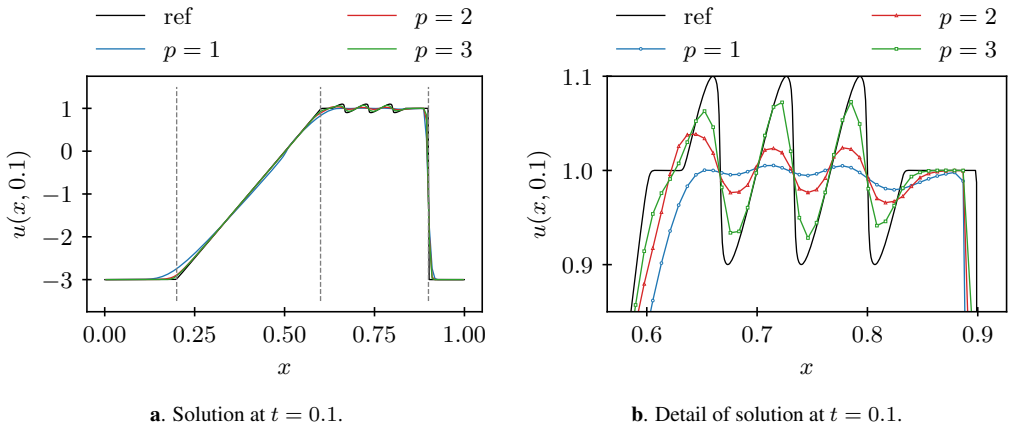


Figure 9.4.3. WENO test problem for Burgers' equation, with grid spacing $h_x = 1/128$ and the solution plotted at $t = 0.1$. The CFL constant is 0.5. The reference solution (ref in the legend) is solved with $p = 3$ and $h_x = 1/1024$.

Most important, Figure 9.4.3b shows that the accuracy of the sinusoidal wave train and the steepening of the decreasing slopes is improved as the order of the scheme is increased, for the same grid spacing. This illustrates the benefit of high-order methods for hyperbolic problems with solutions that contain shocks. The high-order methods locally drop to first-order accuracy near a shock, but they maintain $\mathcal{O}(h_x^{2p-1})$ accuracy away from shocks. Since information propagation in hyperbolic problems is directional with finite speed, the drop in order of accuracy at a shock does not necessarily pollute the accuracy of the numerical solution away from the shock, as seen in this example.

9.4.6 ■ WENO for systems and in multiple spatial dimensions

ENO and WENO schemes for systems of conservation laws follow the same principle as the schemes for scalar equations. The simplest approach is to apply the reconstruction component-wise to the conservative variables. However, for high orders or strongly nonlinear flows, this may lead to undesirable oscillations. In these cases, applying the reconstruction to local characteristic variables may work better. In this approach, for cell interface, $k + \frac{1}{2}$, of cell Ω_k , one computes the flux Jacobian matrix from equation (9.58a). The eigendecomposition of the Jacobian is used to transform the cell averages in stencils, $S_{k,r}$, from the conservative variables, $\mathbf{u}$, to characteristic variables, $\mathbf{w} = L\mathbf{u}$, as in equation (9.13). One then applies WENO reconstruction to the characteristic variables and transforms the reconstructed $\mathbf{w}_{k+\frac{1}{2}}$ back to conservative variables to compute the numerical flux at the cell interface.

In multiple spatial dimensions, we use a dimension-by-dimension approach. For example, for WENO with $p = 3$ and order $2p - 1 = 5$ on a 2D Cartesian grid, we need three Gauss points on every cell interface to approximate the flux integrals with sufficient accuracy over the boundary of the finite-volume cell as in equation (9.77). Here, we consider only uniform quadrilateral grids and index our cells as $\Omega_{j,k}$. We follow the procedure illustrated in Figure 9.4.4: for the Gauss points on the north and south interfaces of cell $\Omega_{j,k}$, we first do a 1D WENO reconstruction in the x -direction, using the cell averages in cells $\Omega_{j,k}$ for fixed values of k (along the red dotted lines in Figure 9.4.4). This step constructs polynomials in x that provide a high-order polynomial approximation of cell averages in the y -direction. This first set of WENO reconstructions is to

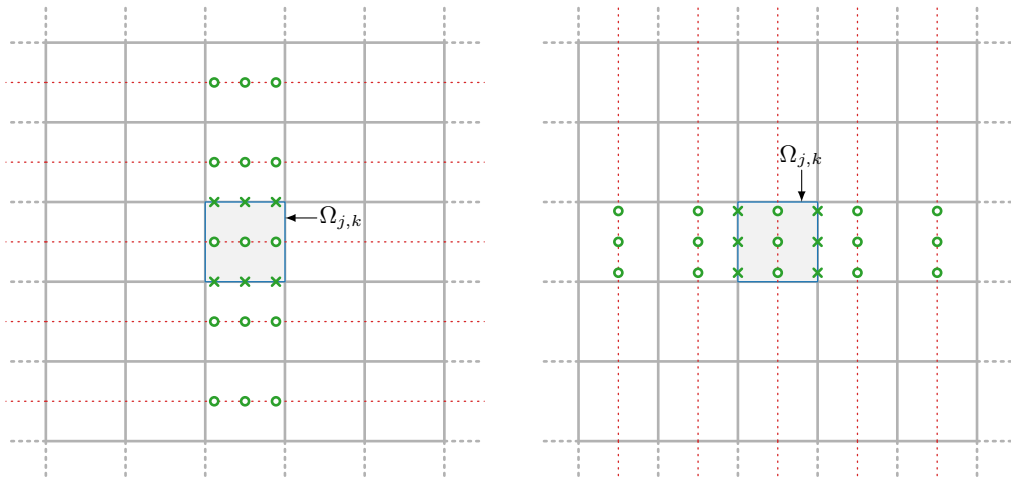


Figure 9.4.4. 2D WENO for $p = 3$: a two-phase method executing 1D WENO in the (left) x -direction, followed by the (right) y -direction.

be evaluated at the green points on each red line, using weights $c_{r,j}$ and d_r that depend on the location of the Gauss points. This is followed by a second set of WENO reconstructions, in the y -direction (along the vertical lines of green points, for cell $\Omega_{j,k}$), using the reconstructed y -averages in the green points, to determine the WENO reconstructions in the Gauss points (green crosses). Next, the two phases are repeated with switched roles for x and y to determine the WENO reconstructions in the Gauss points on the east and west interfaces of cell $\Omega_{j,k}$. It can be shown that this 2D approach has order of accuracy $2p - 1$.

9.5 ■ Discontinuous Galerkin schemes for hyperbolic conservation laws

Objectives

As an alternative to the WENO methods described in the previous section, the goal of this section is to introduce a second family of advanced methods for conservation laws with high-order accuracy, namely, discontinuous Galerkin (DG) methods. Specifically, you will use a weak formulation with finite-element basis functions that are discontinuous at element interfaces in order to implement DG methods using mass and stiffness matrices that are assembled elementwise. You will then explore various ways to stabilize DG solutions when shocks arise in nonlinear problems, including using filtering and limiting.

9.5.1 ■ The discontinuous Galerkin method

Finite-volume methods are designed to reflect the underlying conservation laws. For example, for scalar conservation law

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0, \quad (9.152)$$

recalling equation (6.40), the basic form of a semidiscretized finite-volume scheme for cell $\Omega_k = [x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}]$ is written as

$$\frac{du_k(t)}{dt} + \frac{\hat{f}_{k+\frac{1}{2}}(t) - \hat{f}_{k-\frac{1}{2}}(t)}{h_x} = 0. \quad (9.153)$$

The key component of equation (9.153) is the *numerical flux*, $\hat{f}_{k+\frac{1}{2}}$. The difference in numerical fluxes quantifies the net change to the cell average through the interfaces, an essential ingredient of finite-volume methods.

Finite-volume methods have proven to yield robust approximations for a variety of applications. However, finite-volume methods of order higher than two require increasingly large stencils, which may lead to complications at domain boundaries, and they suffer from difficulties with controlling spurious oscillations at discontinuities. High-order WENO methods effectively control spurious oscillations but also have large stencils, and it is difficult to extend the WENO approach to unstructured grids. Discontinuous Galerkin (DG) methods for hyperbolic conservation laws are another family of high-order methods that attempt to mitigate some of these issues. DG methods use discontinuous piecewise polynomial finite-element spaces in a weak formulation of the conservation law. As with other finite-element methods, they exploit locality and localize most computations to individual high-order elements. They incorporate aspects of the finite-volume approach to arrive at a high-order approximation that is locally and globally conservative. As such, DG methods have attractive properties, but spurious oscillations for problems with shocks remain a challenge.

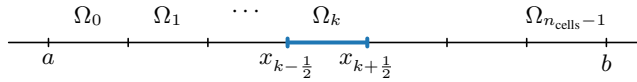


Figure 9.5.1. Grid of n_{cells} elements for the domain $\Omega = [a, b]$.

Consider a 1D domain, $\Omega = \bigcup_{k=0}^{n_{\text{cells}}-1} \Omega_k$, with n_{cells} cells (elements in the DG context), $\Omega_k = [x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}]$, and nodes $x_{k-\frac{1}{2}} < x_{k+\frac{1}{2}}$, as shown in Figure 9.5.1. Define $h_x = \max_k |\Omega_k|$. Similar to a (continuous) finite-element method, we introduce polynomials of degree at most p in a piecewise fashion; however, there is no requirement for continuity between cells (or mesh elements) Ω_k ,

$$\mathcal{V}^{h_x} = \left\{ v \mid v|_{\Omega_k} \in P_p(\Omega_k), \text{ for all } k \right\}, \quad (9.154)$$

where the superscript h_x in $\mathcal{V}^{h_x}$ means that the function space depends on the underlying mesh. This notation slightly diverges from that in Chapter 8 in order to be consistent with the notation used for the finite-volume method discussed in this chapter. As in the finite-volume case, we first consider a semidiscretization, where we approximate the solution by a function $u(t) \in \mathcal{V}^{h_x}$. We do not use $u^{h_x}(t)$ to denote the discrete solution here, simply to avoid additional complication to the notation that follows.

Note, in fact, that an *object* in $\mathcal{V}^{h_x}$ is actually not a *function* in the usual sense, since it is not required to be single valued at each point in x ; however, we think about these objects as functions in $L^2(\Omega)$, since the value of a function on a set of measure zero does not change its fundamental properties in $L^2(\Omega)$; henceforth, we ignore this subtlety. These functions, however, are generally not weakly differentiable in the sense of Definition 8.14. Indeed, consider three elements of a mesh as in Figure 9.5.2, with $p = 4$. The jumps at the interfaces $x_{k+\frac{1}{2}}$ preclude differentiability.

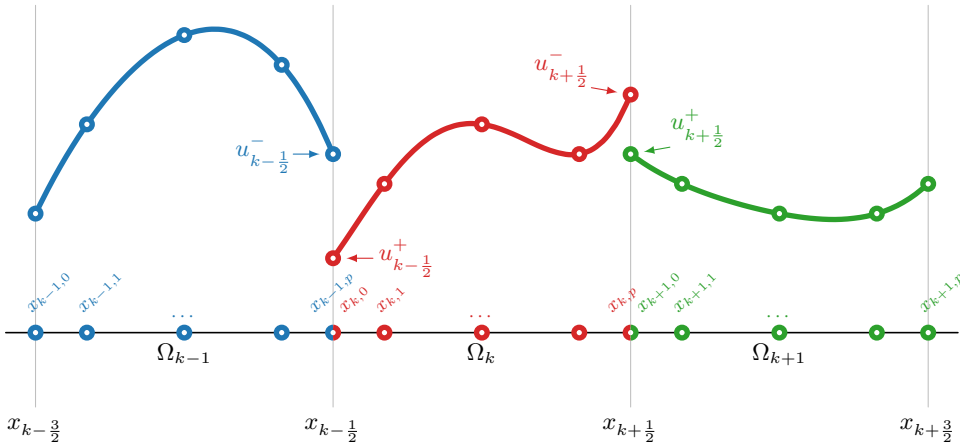


Figure 9.5.2. Example elements with $p = 4$. The left and right values at element interface $x_{k+\frac{1}{2}}$ are indicated by $u_{k+\frac{1}{2}}^-$ and $u_{k+\frac{1}{2}}^+$. The degrees of freedom are labeled according to spatial location $x_{k,0}, \dots, x_{k,p}$ for each element.

For $v(x) \in \mathcal{V}^{h_x}$, the restriction to each element, $v_k(x)$, is a polynomial of degree at most p . Here, we consider a *nodal* representation of the polynomial, writing $v_k(x)$ in element Ω_k in terms of a nodal basis, $\{\phi_{k,q}(x)\}_{q=0}^p$, where the basis functions, $\phi_{k,q}(x)$, satisfy $\phi_{k,q}(x_{k,j}) = \delta_{q,j}$ at the $p+1$ interpolation points, $x_{k,0}, \dots, x_{k,p}$, in element Ω_k , as shown in Figure 9.5.2. The nodal basis functions $\phi_{k,q}(x)$ are given by the Lagrange polynomials. Other approaches may use

Legendre or Chebyshev bases and are termed *modal*. To avoid ill-conditioning, the interpolation points are generally not equispaced, and a Gauss–Legendre–Lobatto (GLL) quadrature rule is typically used to select the node locations (later, we use the associated quadrature weights). In a reference element $\hat{\Omega} = [-1, 1]$, the $p + 1$ GLL nodes are defined as ± 1 and the $p - 1$ roots of $\frac{dL_p}{dx}$, where $L_p(x)$ is the degree- p Legendre polynomial (of the first kind). These nodes are mapped to the element Ω_k by the invertible mapping $F_k(\hat{x}) = x_{k-\frac{1}{2}} + (x_{k+\frac{1}{2}} - x_{k-\frac{1}{2}})(\frac{\hat{x}+1}{2})$, so that $x_{k,q} = F_k(\hat{x}_q)$ for each GLL node $\hat{x}_q \in \hat{\Omega}$. Furthermore, it can be shown that the Lagrange interpolating polynomials associated with the nodes $\hat{x}_q \in \hat{\Omega}$, $\hat{\phi}_q(\hat{x})$, and those associated with the nodes $x_{k,q} \in \Omega_k$, $\phi_{k,q}(x)$, are related as $\phi_{k,q}(F_k(\hat{x})) = \hat{\phi}_q(\hat{x})$. This property is important in the details of the implementation that follow. This is done on each element Ω_k , making the total number of degrees of freedom to define a scalar 1D function $v(x) \in \mathcal{V}^{h_x}$ equal to $n_{\text{cells}}(p + 1)$.

To construct a DG approximation, we consider the *weak* representation of

$$u_t + (f(u))_x = 0 \quad (9.155)$$

over each element, Ω_k . That is, let $u \in \mathcal{V}^{h_x}$, restricted to Ω_k , satisfy

$$\int_{\Omega_k} (u_t + (f(u))_x) v \, dx = 0 \quad (9.156)$$

for all $v \in \mathcal{V}^{h_x}$, restricted to Ω_k , for all times t . With integration by parts, this becomes

$$\int_{\Omega_k} (u_t v - f(u) v_x) \, dx + f(u_{k+\frac{1}{2}}^-) v_{k+\frac{1}{2}}^- - f(u_{k-\frac{1}{2}}^+) v_{k-\frac{1}{2}}^+ = 0 \quad (9.157)$$

for all $v \in \mathcal{V}^{h_x}$, restricted to Ω_k . Note that equation (9.156) and equation (9.157) are exactly equivalent, since we use the parts of u and v that are restricted to Ω_k , and are, hence, continuous polynomials over Ω_k , so integration by parts is valid. However, without modification, equation (9.157) cannot be the starting point for a useful finite-element method for hyperbolic conservation laws for two reasons. First, the unknowns in neighboring elements remain fully uncoupled without information flow between elements, so the global discrete problem cannot be well posed. Second, the flux flowing out of element Ω_k on the right, given by $f(u_{k+\frac{1}{2}}^-)$, will not be equal to the flux flowing into element Ω_{k+1} on the left, given by $f(u_{k+\frac{1}{2}}^+)$, so there is no local conservation and shocks may move with incorrect speeds.

To resolve these two issues, the DG method borrows the idea of a numerical flux from finite-volume methods and replaces $f(u_{k+\frac{1}{2}}^-)$ and $f(u_{k-\frac{1}{2}}^+)$ in equation (9.157) by the numerical fluxes $f^*(u_{k+\frac{1}{2}}^-, u_{k+\frac{1}{2}}^+)$ and $f^*(u_{k-\frac{1}{2}}^-, u_{k-\frac{1}{2}}^+)$, respectively. Note that this substitution does not impede high-order accuracy of the resulting method: for a DG method with order of accuracy s , the difference between $u_{k+\frac{1}{2}}^-$ and $u_{k+\frac{1}{2}}^+$ is expected to be of order h_x^s for smooth solutions and, by consistency of the numerical flux function, the difference between $f(u_{k+\frac{1}{2}}^-)$ and $f^*(u_{k+\frac{1}{2}}^-, u_{k+\frac{1}{2}}^+)$ is also of order h_x^s . This yields the DG weak form of finding $u \in \mathcal{V}^{h_x}$ such that

$$\int_{\Omega_k} (u_t v - f(u) v_x) \, dx + f^*(u_{k+\frac{1}{2}}^-, u_{k+\frac{1}{2}}^+) v_{k+\frac{1}{2}}^- - f^*(u_{k-\frac{1}{2}}^-, u_{k-\frac{1}{2}}^+) v_{k-\frac{1}{2}}^+ = 0 \quad (9.158)$$

for all k and all $v \in \mathcal{V}^{h_x}$. For the numerical flux in equation (9.158), we have the same choices as we considered for the finite-volume method. A popular choice is the local Lax–Friedrichs flux from equation (6.59) and equation (6.61). It is also easy to see why equation (9.158) now implies exact discrete conservation: taking $v = 1$ in equation (9.158) results in

$$\int_{\Omega_k} u_t \, dx + f^*(u_{k+\frac{1}{2}}^-, u_{k+\frac{1}{2}}^+) - f^*(u_{k-\frac{1}{2}}^-, u_{k-\frac{1}{2}}^+) = 0 \quad (9.159)$$

for each element Ω_k , and conservation of cell averages follows as in equation (6.46) using a telescoping sum argument.

Equation (9.158) describes the weak solution u in interval Ω_k . To construct the approximation, we expand $u(x, t)$ and $f(u(x, t))$ in the local Lagrange basis of element Ω_k , $\phi_{k,q}(x)$ for $q = 0, \dots, p$. To simplify notation when discussing the computations on a single element Ω_k , we drop the subscript k in the expansions of $u(x, t)$ and $f(u(x, t))$ on Ω_k and write

$$u(x, t)|_{\Omega_k} = \sum_{q=0}^p u_q(t) \phi_q(x), \quad (9.160a)$$

$$f(u(x, t))|_{\Omega_k} = \sum_{q=0}^p f_q(t) \phi_q(x). \quad (9.160b)$$

Here, the coefficients $u_q(t)$ and $f_q(t)$ are time dependent, and we write $u_q = u_q(t)$, for example, when the intent is clear. We use $\mathbf{u}_k$ and $\mathbf{f}_k$ to denote the vectors of u_q and f_q , respectively, on element k . In addition, note that $x_{k+\frac{1}{2}} = x_{k,p} = x_{k+1,0}$.

To obtain a finite-element discretization with the same number of equations as unknowns, we have to express the unknown coefficients $f_q(t)$ as a function of the coefficients $u_q(t)$. The simplest way to achieve this is to take $f_q(t) = f(u_q(t))$. This is a natural approach since the coefficients $u_q(t)$ are the nodal values of the approximation u in the points $x_{k,q}$, and $f_q(t)$ is the nodal value of the flux function f . In the case of linear conservation laws—e.g., linear advection with $f(u) = au$ —it is fully justified to take $f_q(t) = au_q(t)$ in equation (9.160b). For nonlinear flux functions, we normally assume that $f_q(t) = f(u_q(t))$, but other approaches exist that may perform better, for example, to deal with issues arising from aliased sampling; see Section 9.5.3.

The weak form of the problem on element Ω_k becomes finding $u_q(t)$ for $q = 0, \dots, p$ that satisfy

$$\begin{aligned} \int_{\Omega_k} \sum_{q=0}^p \frac{du_q(t)}{dt} \phi_q(x) \phi_r(x) dx - \int_{\Omega_k} \sum_{q=0}^p f_q(t) \phi_q(x) \frac{d\phi_r(x)}{dx} dx \\ + f^*(u_{k+\frac{1}{2}}^-(t), u_{k+\frac{1}{2}}^+(t)) \phi_r(x_{k,p}) - f^*(u_{k-\frac{1}{2}}^-(t), u_{k-\frac{1}{2}}^+(t)) \phi_r(x_{k,0}) = 0 \end{aligned} \quad (9.161)$$

for all $\phi_r(x)$, $r = 0, \dots, p$. Due to our use of a nodal basis with a node located at each element interface, we have that $u_{k+\frac{1}{2}}^-(t) = u_{k,p}(t)$, $u_{k+\frac{1}{2}}^+(t) = u_{k+1,0}(t)$, $u_{k-\frac{1}{2}}^-(t) = u_{k-1,p}(t)$, and $u_{k-\frac{1}{2}}^+(t) = u_{k,0}(t)$. Also, note that $\phi_r(x_{k,p})$ and $\phi_r(x_{k,0})$ each vanish in all but one of the $p+1$ equations, since $\phi_r(x_{k,j}) = \delta_{r,j}$ for all j .

To compute the integrals in equation (9.161), we use the same approach as in Section 8.5 and use a reference element to perform the computations. Using the mapping $x = F(\hat{x}) := x_{k-\frac{1}{2}} + (x_{k+\frac{1}{2}} - x_{k-\frac{1}{2}}) \left(\frac{\hat{x}+1}{2} \right)$ from $\hat{x} \in [-1, 1] \rightarrow \Omega_k$, with a Jacobian determinant of $|J_F| = \frac{h_x}{2}$, where $h_x = x_{k+\frac{1}{2}} - x_{k-\frac{1}{2}}$, the integral terms in equation (9.161) become

$$\sum_{q=0}^p \frac{du_q}{dt} \int_{\Omega_k} \phi_q(x) \phi_r(x) dx = \sum_{q=0}^p \frac{du_q}{dt} \int_{-1}^1 \phi_q(F(\hat{x})) \phi_r(F(\hat{x})) \frac{h_x}{2} d\hat{x} \quad (9.162a)$$

$$= \sum_{q=0}^p \frac{du_q}{dt} \int_{-1}^1 \hat{\phi}_q(\hat{x}) \hat{\phi}_r(\hat{x}) \frac{h_x}{2} d\hat{x} \quad (9.162b)$$

$$= \frac{h_x}{2} \left(M \frac{d\mathbf{u}}{dt} \right)_r, \quad (9.162c)$$

$$\sum_{q=0}^p f_q \int_{\Omega_k} \phi_q(x) \frac{d\phi_r(x)}{dx} dx = \sum_{q=0}^p f_q \int_{-1}^1 \phi_q(F(\hat{x})) \frac{d\phi_r(F(\hat{x}))}{d\hat{x}} d\hat{x} \quad (9.163a)$$

$$= \sum_{q=0}^p f_q \int_{-1}^1 \hat{\phi}_q(\hat{x}) \frac{d\hat{\phi}_r(\hat{x})}{d\hat{x}} d\hat{x} \quad (9.163b)$$

$$= (K\mathbf{f})_r \quad (9.163c)$$

for each $r = 0, \dots, p$. Here, M and K are the “mass” and “stiffness” matrices on the reference element $\hat{\Omega} = [-1, 1]$:

$$m_{r,q} = \int_{\hat{\Omega}} \hat{\phi}_q(\hat{x}) \hat{\phi}_r(\hat{x}) d\hat{x}, \quad k_{r,q} = \int_{\hat{\Omega}} \hat{\phi}_q(\hat{x}) \frac{d\hat{\phi}_r(\hat{x})}{d\hat{x}} d\hat{x}. \quad (9.164)$$

Note that M is symmetric but K is not.

Returning to equation (9.161), we see that the vectors with entries given by either $\phi_r(x_{k,0})$ or $\phi_r(x_{k,p})$ have all but one entry equal to zero, with the first entry in the vector with values $\phi_r(x_{k,0})$ being one, as is the last entry in the vector with values $\phi_r(x_{k,p})$. Denoting these vectors by δ_0 and δ_p , respectively, we then have the following semidiscretization: for each element Ω_k , $\mathbf{u}_k = \{u_{k,q}\}_{q=0}^p$ satisfies

$$M \frac{d\mathbf{u}_k}{dt} = \frac{2}{h_x} (K\mathbf{f}_k(\mathbf{u}_k) + f^*(u_{k-1,p}, u_{k,0})\delta_0 - f^*(u_{k,p}, u_{k+1,0})\delta_p), \quad (9.165a)$$

$$=: -\mathbf{z}(\mathbf{u}_{k-1}, \mathbf{u}_k, \mathbf{u}_{k+1}), \quad (9.165b)$$

where we have indicated explicitly that $\mathbf{f}_k$ is a function of $\mathbf{u}_k$.

9.5.2 ■ Implementation

Applying a timestepping scheme to update the solution vector $\mathbf{u}_k$ in each element Ω_k in equation (9.165a) is primarily a local calculation since only $u_{k-1,p}$ and $u_{k+1,0}$ couple to neighboring elements. The computation is largely a matrix-vector multiplication in the explicit case. In fact, we can implement the calculation in bulk—see Figure 9.5.3. In addition, for this problem, the calculation of both M and K can be performed “offline,” thereby rendering the construction of the right-hand side, $-\mathbf{z}(\mathbf{u}_k)$, in equation (9.165a) highly efficient (and well suited for graphics processing unit (GPU) computing [146]).

To construct M and K , given by equation (9.164), we note that computing the derivatives in K is complicated using a Lagrange basis for $\hat{\phi}_q$. Instead, we consider an expansion on the reference element in terms of a basis for which we can easily compute the derivatives, for example,

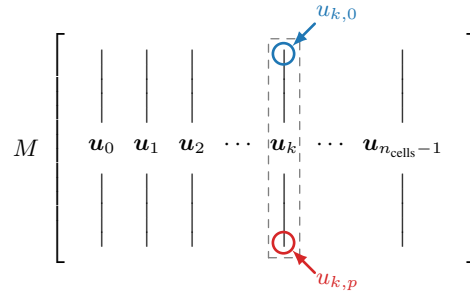


Figure 9.5.3. The mass matrix M in equation (9.165a) can be applied in bulk across all elements by writing $\mathbf{u}$ as a matrix with columns $\mathbf{u}_k$.

with $\widehat{\psi}(\widehat{x})$ representing a normalized Legendre basis,

$$v(\widehat{x}) = \sum_{i=0}^p v_i \widehat{\phi}_i(\widehat{x}) = \sum_{j=0}^p \widetilde{v}_j \widehat{\psi}_j(\widehat{x}), \quad (9.166)$$

where v and $\widetilde{v}$ are termed the nodal and modal coefficients, respectively. Then, by evaluating equation (9.166) at the nodal points, $\{\widehat{x}_i\}_{i=0}^p$, we obtain the relationship that

$$v_i = \sum_{j=0}^p \widetilde{v}_j \widehat{\psi}_j(\widehat{x}_i), \quad (9.167)$$

which is written in vector form as

$$\mathbf{v} = \begin{bmatrix} \widehat{\psi}_0(\widehat{x}_0) & \widehat{\psi}_1(\widehat{x}_0) & \cdots & \widehat{\psi}_p(\widehat{x}_0) \\ \widehat{\psi}_0(\widehat{x}_1) & \widehat{\psi}_1(\widehat{x}_1) & \cdots & \widehat{\psi}_p(\widehat{x}_1) \\ \vdots & \vdots & \ddots & \vdots \\ \widehat{\psi}_0(\widehat{x}_p) & \widehat{\psi}_1(\widehat{x}_p) & \cdots & \widehat{\psi}_p(\widehat{x}_p) \end{bmatrix} \widetilde{\mathbf{v}} = V \widetilde{\mathbf{v}}, \quad (9.168)$$

where V is the so-called *Vandermonde* matrix.

With this, the matrix M is computed as follows. From equation (9.164), we have $m_{r,q} = \langle \widehat{\phi}_q, \widehat{\phi}_r \rangle$. We now express $\widehat{\phi}_q$ in the modal basis as

$$\widehat{\phi}_q(\widehat{x}) = \sum_{i=0}^p \widetilde{v}_{q,i} \widehat{\psi}_i(\widehat{x}), \quad (9.169)$$

with, from equation (9.168),

$$\widetilde{\mathbf{v}}_q = V^{-1} \mathbf{e}^{(q)}, \quad (9.170)$$

where $\mathbf{e}^{(q)}$ is the q th canonical unit vector. Since the modal basis vectors $\widehat{\psi}$ are orthonormal on $\widehat{\Omega}$, we have

$$m_{r,q} = \langle \widehat{\phi}_q, \widehat{\phi}_r \rangle = \left\langle \sum_{i=0}^p \widetilde{v}_{q,i} \widehat{\psi}_i, \sum_{j=0}^p \widetilde{v}_{r,j} \widehat{\psi}_j \right\rangle \quad (9.171a)$$

$$= (\widetilde{\mathbf{v}}_q)^T \widetilde{\mathbf{v}}_r = \left(V^{-1} \mathbf{e}^{(q)} \right)^T V^{-1} \mathbf{e}^{(r)} \quad (9.171b)$$

$$= \left(\mathbf{e}^{(r)} \right)^T V^{-T} V^{-1} \mathbf{e}^{(q)}, \quad (9.171c)$$

so we obtain

$$M = V^{-T} V^{-1}. \quad (9.172)$$

Similarly, we write the entries of K as

$$k_{r,q} = \left\langle \widehat{\phi}_q, \frac{d\widehat{\phi}_r}{d\widehat{x}} \right\rangle = \left\langle \sum_{i=0}^p \widetilde{v}_{q,i} \widehat{\psi}_i, \sum_{j=0}^p \widetilde{v}_{r,j} \frac{d\widehat{\psi}_j}{d\widehat{x}} \right\rangle, \quad (9.173)$$

but we do not have the same orthogonality relationship between $\widehat{\psi}_i$ and $\frac{d\widehat{\psi}_j}{d\widehat{x}}$ to be directly able to evaluate these inner products. To express these entries, we first write $\frac{d\widehat{\psi}_j}{d\widehat{x}}$ in terms of the Lagrange basis (since it is a polynomial of degree $p-1$), as

$$\frac{d\widehat{\psi}_j}{d\widehat{x}}(\widehat{x}) = \sum_{k=0}^p \frac{d\widehat{\psi}_j}{d\widehat{x}}(\widehat{x}_k) \widehat{\phi}_k(\widehat{x}), \quad (9.174)$$

and then use equation (9.166) to express this in terms of the Legendre basis:

$$\frac{d\hat{\psi}_j}{d\hat{x}}(\hat{x}) = \sum_{k=0}^p \frac{d\hat{\psi}_j}{d\hat{x}}(\hat{x}_k) \sum_{\ell=0}^p \tilde{v}_{k,\ell} \hat{\psi}_\ell(\hat{x}). \quad (9.175)$$

With this, we revisit equation (9.173) to write

$$k_{r,q} = \left\langle \sum_{i=0}^p \tilde{v}_{q,i} \hat{\psi}_i, \sum_{j=0}^p \tilde{v}_{r,j} \sum_{k=0}^p \frac{d\hat{\psi}_j}{d\hat{x}}(\hat{x}_k) \sum_{\ell=0}^p \tilde{v}_{k,\ell} \hat{\psi}_\ell \right\rangle. \quad (9.176)$$

Defining the Vandermonde differentiation matrix,

$$V' = \begin{bmatrix} \hat{\psi}'_0(\hat{x}_0) & \hat{\psi}'_1(\hat{x}_0) & \cdots & \hat{\psi}'_p(\hat{x}_0) \\ \hat{\psi}'_0(\hat{x}_1) & \hat{\psi}'_1(\hat{x}_1) & \cdots & \hat{\psi}'_p(\hat{x}_1) \\ & & \ddots & \\ \hat{\psi}'_0(\hat{x}_p) & \hat{\psi}'_1(\hat{x}_p) & \cdots & \hat{\psi}'_p(\hat{x}_p) \end{bmatrix}, \quad (9.177)$$

and recognizing that equation (9.170) gives us $\tilde{v}_{q,i} = (V^{-1})_{i,q}$, we write the entries of K as

$$k_{r,q} = \sum_{i=0}^p \sum_{j=0}^p \sum_{k=0}^p (V^{-1})_{i,q} (V^{-1})_{j,r} (V')_{k,j} (V^{-1})_{i,k} \quad (9.178a)$$

$$= \sum_{i=0}^p \sum_{j=0}^p \sum_{k=0}^p (V^{-T})_{r,j} \left((V')^T \right)_{j,k} (V^{-T})_{k,i} (V^{-1})_{i,q}. \quad (9.178b)$$

This gives

$$K = V^{-T} (V')^T V^{-T} V^{-1} = V^{-T} (V')^T M. \quad (9.179)$$

A similar construction is also performed in higher dimensions; more information on the implementation of DG methods is found in [133, 143].

Example: linear advection

As an example, we consider the linear advection equation

$$\partial_t u + \partial_x u = 0, \quad (9.180)$$

with periodic boundary conditions on the interval $[0, 1]$ and with three different initial profiles, given by

$$u_0^A(x) = \frac{1 + \sin(2\pi x)}{2}, \quad (9.181a)$$

$$u_0^B(x) = \frac{1 + |\sin(2\pi x)|}{2}, \quad (9.181b)$$

$$u_0^C(x) = \begin{cases} \frac{1 + \sin(\pi x)}{2} & \text{if } x \leq \frac{1}{2}, \\ \frac{1 - \cos(\pi x)}{2} & \text{if } x > \frac{1}{2}. \end{cases} \quad (9.181c)$$

We note that u_0^A is continuous and differentiable, including across the endpoints of the interval, while u_0^B is continuous, but not differentiable across the periodic boundary (and at $x = \frac{1}{2}$), and

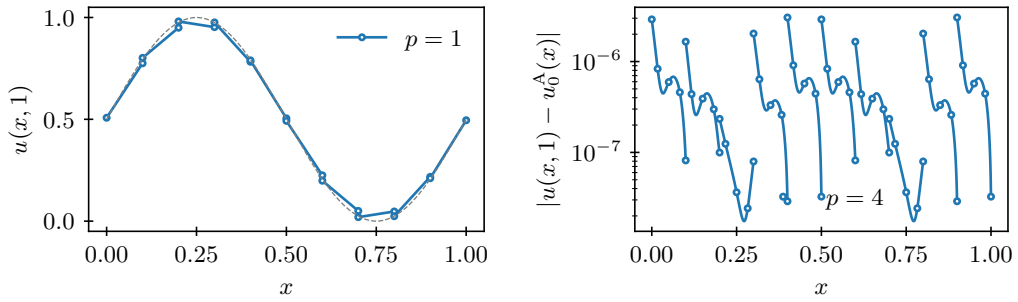


Figure 9.5.4. DG approximations for linear advection with $u(x, 0) = u_0^A(x)$ and $n_{\text{cells}} = 10$. (left) Solution for polynomial degree $p = 1$ after one period, compared with the exact solution (dashed). (right) Error after one period for degree $p = 4$.

u_0^C is discontinuous at $x = \frac{1}{2}$. We use the third-order TVDRK scheme of equation (9.150) as the timestepping scheme.

First consider Figure 9.5.4, where we advect the smooth profile, $u_0^A(x)$, over one period. We see that while the solution is smooth, the DG approximation is allowed to be discontinuous at element interfaces. The figure also provides a snapshot of the error in the case of $p = 4$.

In contrast, Figures 9.5.5 and 9.5.6 highlight what happens when we lose smoothness in the solution. Even for the continuous but nondifferentiable initial condition of u_0^B , we see higher

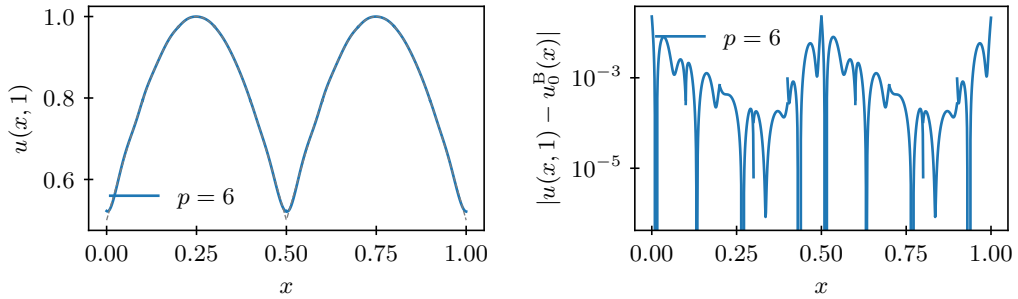


Figure 9.5.5. DG approximations for linear advection for degree $p = 6$ and $n_{\text{cells}} = 10$. (left) Approximation after one period for $u(x, 0) = u_0^B(x)$ compared with the exact solution (dashed). (right) Error after one period for $u(x, 0) = u_0^B(x)$.

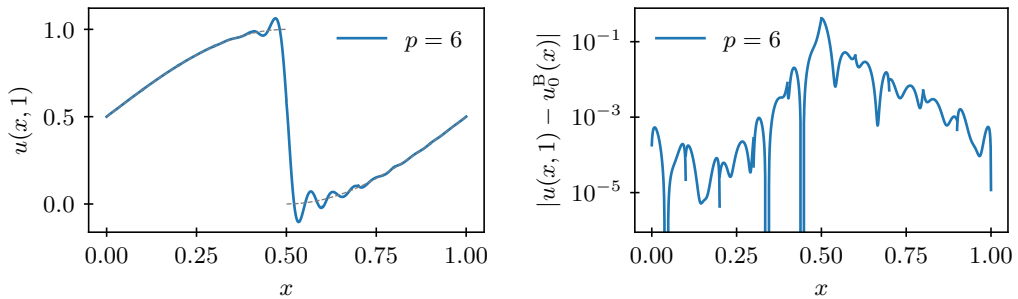


Figure 9.5.6. DG approximations for linear advection for degree $p = 6$ and $n_{\text{cells}} = 10$. (left) Approximation after one period for $u(x, 0) = u_0^C(x)$ compared with the exact solution (dashed). (right) Error after one period for $u(x, 0) = u_0^C(x)$.

error near the cusps in the solution, while the discontinuous initial condition of u_0^C exhibits significant oscillations at the jump discontinuity; we comment further on this in the next section.

9.5.3 ■ Nonlinear equations

As the linear advection example highlights, high-order DG methods work well for smooth problems, but it is a challenge to make them work properly for problems with discontinuities. Various approaches exist that attempt to improve the solution quality for nonsmooth flows, addressing issues with spurious oscillations (as observed in Figures 9.5.5 and 9.5.6). To explore this further, we consider Burgers' equation, where the nonlinear flux function presents additional challenges.

Slope limiters

A standard approach to combat spurious oscillations is the use of slope limiters, in the same fashion as discussed for finite-volume methods in Section 6.5. For high-order methods, the main trick with limiters is to limit only in regions with (nonphysical) oscillations by identifying cells in which to apply the limiter, either as a preprocessing step or by building this directly into the flux function itself. Indeed, slope limiters diminish the accuracy of the approximation if used in smooth regions. For this reason, so-called *discontinuity detectors* or *troubled cell indicators* are used to identify large deviations in the solution over a single or few consecutive cells, so that limiting is only triggered in regions close to a discontinuity. However, a shortcoming of this type of approach is that it tends to add additional parameters (that require tuning) to the overall approximation scheme. Alternatively, so-called *modal* limiting schemes [147] attempt to identify the variation in the approximation by assessing the modal coefficients directly and attempt to only limit where needed. This can be costly in practice, and it is a challenge to make limiters fully robust.

Slope limiters use the value of the numerical approximation and its derivatives in neighboring cells to estimate the curvature of the approximation. To construct slope limiters for DG methods, we consider the cell *averages* and reconstruct the solution in a cell with an appropriate slope, again similar to the approach taken for slope limiters for finite-volume methods (see, e.g., [131]). Since the Legendre polynomials integrate to zero for all but the constant mode (on the reference element), we directly compute the average, $\bar{u}$, of $u(F(\hat{x})) = \sum_{q=0}^p u_q \hat{\phi}_q(\hat{x}) = \sum_{q=0}^p \tilde{u}_q \hat{\psi}_q(\hat{x})$ from the first modal coefficient, recalling that $\hat{\psi}_0(\hat{x}) = \frac{1}{\sqrt{2}}$ since the modal (Legendre) basis functions are chosen to be orthonormal over the reference element $\hat{\Omega} = [-1, 1]$:

$$\bar{u} = \int_{\hat{\Omega}} u(F(\hat{x})) d\hat{x} = \sqrt{2} \tilde{u}_0, \quad \text{where } \tilde{\mathbf{u}} = V\mathbf{u}. \quad (9.182)$$

If a solution $u(x)$ requires limiting in cell Ω_k , we reconstruct $u(x)$ as a linear function with a modified slope over Ω_k (see Figure 9.5.7 for an illustration),

$$u(x) \leftarrow \bar{u}_k + s \psi_1(x), \quad (9.183)$$

where $\bar{u}_k$ is the average of $u(x)$ over Ω_k , equal to half that of $u(F(\hat{x}))$ when $u(x)|_{\Omega_k}$ is mapped back to $\hat{\Omega}$.

The approach of [74] is to compare the gradient of the approximation with the change in cell averages between neighboring cells. If the slopes are the same sign, then the minimum slope is taken; if the slopes change direction, then the slope of the reconstruction is set to zero. This is encoded through the minmod function in the following way. Consider a known approximate solution $u_k(x)$ in cell Ω_k , with corresponding cell averages $\bar{u}_{k-1}$, $\bar{u}_k$, and $\bar{u}_{k+1}$ in cells Ω_{k-1} ,

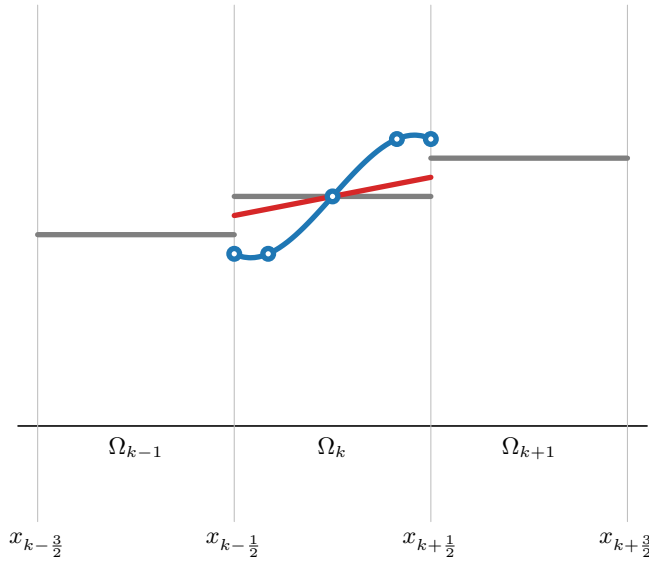


Figure 9.5.7. Reconstructing an approximation in cell Ω_k as a linear function. Solution averages in each element (gray), polynomial representation with $p = 4$ (blue), and limited linear reconstruction (red) are shown.

Ω_k , and Ω_{k+1} . Then, write the modified slope, s , in equation (9.183) as

$$s := \text{minmod} \left(\frac{du_k}{dx}(x), \frac{\bar{u}_{k+1} - \bar{u}_k}{h_x}, \frac{\bar{u}_k - \bar{u}_{k-1}}{h_x} \right) \quad (9.184)$$

for some approximation of $\frac{du_k}{dx}(x)$, where the minmod function is given by

$$\text{minmod}(\alpha, \beta, \gamma) = \begin{cases} \text{sign}(\alpha) \min(|\alpha|, |\beta|, |\gamma|) & \text{if } \text{sign}(\alpha) = \text{sign}(\beta) = \text{sign}(\gamma), \\ 0 & \text{otherwise.} \end{cases} \quad (9.185)$$

There are many variants to this slope limiter. In particular, $\frac{du_k}{dx}(x)$ may be computed from the linear term in the expansion above (proportional to $\tilde{u}_1$) or by evaluating the full derivative of $u_k(x)$ at the center of Ω_k . This type of slope limiter can be applied directly for second-order accurate DG methods with linear elements ($p = 1$), without the need for a troubled cell indicator, and a TVD property can be proven for the second-order DG method, similar to the case of second-order finite-volume methods [131]. For DG methods with $p > 1$ (i.e., order of accuracy higher than two), a troubled-cell indicator is needed. One simple strategy is to use the minmod function to detect troubled cells for DG methods by comparing the size and sign of changes in neighboring cell averages with the change in the value of the local polynomial between the midpoint and the endpoints of the element. See [74] and [131, Sec. 12.2.4]. However, it is hard to make this type of approach fully robust, and theoretical results on controlling oscillations for DG methods of order higher than two are limited.

As an example, we solve Burgers' equation with periodic boundary conditions and an initial profile with a jump discontinuity, as in Figure 9.5.8, using a DG method with degree $p = 6$. As seen in the figure, using the slope limiter and troubled-cell indicator from [74] (see also [131, Sec. 12.2.4]) reduces the oscillations near the discontinuity.

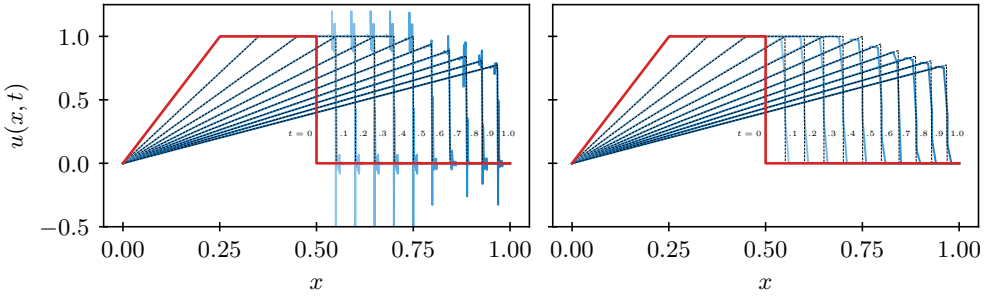


Figure 9.5.8. Solutions to Burgers' equation up to time $T = 1.0$ (left) without and (right) with slope limiting. Degree $p = 6$ with a timestep of 0.0001 is used with 100 elements. The initial condition is highlighted in red, shading from light blue to dark indicates an increase in time, and the exact solution is marked by a dashed black line at $t = 0.1, 0.2, \dots, 1.0$.

Aliasing errors

A second issue that arises is instabilities due to aliasing errors in the DG method for the case of nonlinear conservation laws. We return to the elementwise ODE system in equation (9.165a) with $u(x, t) = \sum_{q=0}^p u_q(t) \phi_q(x)$ (from equation (9.160a)). In the case of linear advection, we have $\mathbf{f}_k = a \mathbf{u}_k$ in equation (9.165a). For Burgers' equation, however, we need to choose an expression for $\mathbf{f}_k$ as a function of $\mathbf{u}_k$. The relation $f(u) = u^2/2$ suggests that if $u(x, t)$ is a degree- p polynomial, then $f(u)$ should be a degree- $2p$ polynomial. However, with our definition of $f(u) = \sum_{q=0}^p f_q(t) \phi_q(x)$, the choice $f_q(t) = f(u_q(t))$ introduces an aliasing error by representing $f(u)$ as a degree- p polynomial instead of a degree- $2p$ polynomial. In this way, we are unable to represent all relevant scales of $f(u)$. For smooth solutions, the effects of aliasing tend to be small; for other solutions, particularly where the solution is underresolved or exhibits sharp features, this aliasing error can render the scheme unstable.

There are several approaches to addressing this instability. For one, we could consider *projecting* $f(u)$ onto the degree- p space rather than interpolating the degree- p polynomial using $f_q(t) = f(u_q(t))$. This involves forming the degree- $2p$ polynomial $f(u) = u^2/2$ and then solving the local problem $\langle \tilde{f}, v \rangle = \langle f(u), v \rangle$ for all $v \in \mathcal{V}^{h_x}$. In this expression, $f(u(x))$ is a degree- $2p$ polynomial and $\tilde{f}(x)$ is the degree- p polynomial resulting from the projection. However, this type of projection is costly and may still require additional steps to mollify the source of the instability.

A more common approach is that of *filtering*. It is observed that aliasing often overemphasizes higher modes in the modal representation. Therefore, the goal of filtering is to attenuate these high-order modes. Considering the modal form of the solution from equation (9.166), we have

$$\tilde{\mathbf{u}} = V^{-1} \mathbf{u}. \quad (9.186)$$

From this, the modal coefficients $\tilde{\mathbf{u}}$ can be filtered with a diagonal filter matrix $\tilde{F}$ of the form

$$\tilde{f}_{q,q} = \begin{cases} 1 & \text{for } q < p_{\text{cutoff}}, \\ e^{\alpha \left(\frac{q+1-p_{\text{cutoff}}}{p+1-p_{\text{cutoff}}} \right)^{2s}} & \text{for } q \geq p_{\text{cutoff}}, \end{cases} \quad (9.187)$$

where we have three filtering parameters in this Gaussian-type filter: α is a scale parameter which is chosen to fully dampen mode $q = p$ (e.g., setting $\alpha = \ln(10^{-16})$), s is a strength parameter (increasing s reduces the strength of the filter), and p_{cutoff} is a cutoff parameter that sets the modes

to be filtered. With $\tilde{F}$, we define a nodal filter matrix F ,

$$F = V\tilde{F}V^{-1}. \quad (9.188)$$

Then, $\mathbf{u}_{\text{filtered}} = F\mathbf{u}$ takes $\mathbf{u}$ to modal form, applies the modal filter above, and converts the coefficients back to nodal form via V . Such filtering typically introduces additional dissipation into the numerical solution, which relates it to another approach to stabilization that we have discussed in Section 5.4: artificial dissipation or viscosity. Indeed, artificial dissipation can be implemented as a filter that modifies the modal polynomial coefficients, as above.

Another approach to handling the high-degree nature of $f(u)$ is to address the shortcomings within the integration term. Here, the concept of *overintegration* [145] is used to more accurately evaluate the second term of the integral in equation (9.158) by using higher-order quadrature.

Pros and cons of high-order methods for nonlinear hyperbolic conservation laws

While there has been substantial research and progress in the area of numerical methods for nonlinear hyperbolic conservation laws with order of accuracy greater than two, it is fair to say that controlling stability and spurious oscillations for problems with shocks remains a formidable challenge for high-order methods. Therefore, it is interesting to reflect for a moment on the relative merits of finite-volume methods, WENO schemes, and DG methods of order higher than two.

Among these three families of methods, the WENO schemes of Section 9.4 are the only class of high-order methods that provably and robustly handle discontinuous solutions without incurring detrimental spurious oscillations. However, these methods were developed for regular Cartesian grids and are not easily extended to more flexible grids that mesh complicated geometries. On the other hand, both high-order finite-volume methods (e.g., [210]) and high-order DG methods lack fully robust and parameter-free ways of controlling spurious oscillations, but they can be implemented naturally on unstructured grids.

The main advantage of DG methods is that they can be implemented with efficient arithmetic computations and parallelism, since computations are highly localized per element and can be implemented in highly structured block form using dense matrix operations that are efficient on, for example, GPUs. Also, since DG update stencils do not reach further than the directly neighboring elements, it is easy to implement high-order accurate boundary conditions. High-order finite-volume methods, in contrast, have stencils that grow in size as the order of accuracy of the scheme increases, which reduces locality of computations and makes parallelism and implementation of high-order accurate boundary conditions more challenging.

9.6 ■ Implicit time-integration for nonlinear PDEs

Objectives

The goal of this section is to discuss implicit time-integration methods for method-of-lines ODE systems that result from discretizing nonlinear time-dependent PDEs. Implicit time integration may be useful when a problem is *stiff*, meaning that it has dynamics that vary with multiple disparate timescales. Therefore, in this section, you will relate the idea of stiff PDEs to the need for small timesteps when using explicit ODE integration methods. Specifically, you will gain insight into the use of backward differentiation or implicit Runge–Kutta methods as implicit time integrators, combined with Newton-like nonlinear system solvers that use preconditioned iterative methods for solving the linear system in each Newton step.

In our discussion of finite-volume methods for nonlinear hyperbolic conservation laws in Chapter 6 and earlier in this chapter, we have so far only considered explicit time integration

methods for the semidiscrete ODE systems of the form

$$\frac{d\mathbf{u}(t)}{dt} + \mathbf{z}(\mathbf{u}(t), t) = \mathbf{0} \quad (9.189)$$

that arise from the spatial discretization of conservation laws—e.g., the general semidiscrete finite-volume form in equation (6.40). Explicit integration methods for equation (9.189) are suitable for many hyperbolic problems, but there are applications where the CFL timestep restriction of explicit schemes becomes a bottleneck in terms of computational efficiency. The CFL limit for explicit time integration of hyperbolic systems is generally of the form

$$h_t \leq c \frac{h_x}{|\lambda|}, \quad (9.190)$$

where λ is some maximal wave speed over all modes of the hyperbolic system over all cells in the discretization (cf. equation (9.51)).

ODE problems are called *stiff* when they are characterized by disparate timescales and the dynamics of interest happen on the long timescales. A bottleneck arises when the problem we want to solve is stiff, and numerical stability on the short timescales imposes a prohibitive timestep limitation relative to the dynamics that are of interest. In such cases, implicit methods that allow for larger timesteps may be more efficient, despite their higher cost per timestep.

One potential source of stiffness for hyperbolic PDEs is when the eigenvalues of the flux Jacobian, which correspond to the wave speeds in the system, have disparate magnitudes. In this case, the largest eigenvalues (which correspond to the fastest wave speeds and, hence, the shortest timescales) lead to a restriction of a very small timestep in equation (9.51), often much smaller than what would be required for accuracy of the numerical solution. Another source of stiffness is the addition of dissipative terms to the ideal, dissipation-free equations of the hyperbolic PDE system. Dissipative terms are characterized by second-order spatial derivatives; as we saw in Section 5.3.4, these second-order terms lead to stability constraints of the type

$$h_t \leq O(h_x^2), \quad (9.191)$$

and these easily become prohibitive when using explicit time integration. For these reasons, we briefly discuss some implicit time-integration methods for nonlinear hyperbolic systems, which allow us to take larger timesteps in the case of stiff problems.

Two important qualities of numerical schemes in this setting are known as A-stability and L-stability, which focus on what is known as the *Dahlquist test equation*, $\frac{du}{dt} = \lambda u$, for complex parameter λ . To state these results, we note that when we apply many numerical schemes to this problem, we can write a recursion of the form $u_{\ell+1} = \phi(\lambda h_t) u_\ell$, where the function $\phi(z)$ is determined by the scheme itself. This *stability function* tells us a lot about the scheme, as in the following definitions.

Definition 9.13: A-stability (see [121, Def. 3.3])

A time-integration scheme is said to be A-stable if its stability function, $\phi(z)$, satisfies $|\phi(z)| \leq 1$ for all z such that $\text{Re}(z) \leq 0$.

Definition 9.14: L-stability (see [121, Def. 3.7])

A time-integration scheme is said to be L-stable if it is A-stable and its stability function, $\phi(z)$, satisfies $\lim_{z \rightarrow \infty} \phi(z) = 0$.

9.6.1 ■ Backward differentiation methods

The first family of implicit methods that we consider is the *multistep* class of backward differentiation (BDF) methods. The lowest-order member of this class is the implicit (backward) Euler method:

$$\frac{\mathbf{u}_{\ell+1} - \mathbf{u}_\ell}{h_t} + \mathbf{z}(\mathbf{u}_{\ell+1}, t_{\ell+1}) = 0. \quad (9.192)$$

As we have seen in Chapter 5, implicit Euler is first-order accurate in time.

It also enjoys the property of being *A-stable*, which means that it is numerically stable for any h_t when the eigenvalues of the Jacobian of $\mathbf{z}(\mathbf{u}(t), t)$ all lie in the right half of the complex plane. We note that the discussion of stability above considers $\frac{du}{dt} = \lambda u$; moving $\mathbf{z}(\mathbf{u}(t), t)$ to the right-hand side of equation (9.189) switches signs, leading to the conclusion that we want the eigenvalues of the Jacobian of $\mathbf{z}(\mathbf{u}(t), t)$ to lie in the right half of the complex plane.

The second member of this class is the two-step BDF2 method:

$$\mathbf{u}_{\ell+1} - \frac{4}{3}\mathbf{u}_\ell + \frac{1}{3}\mathbf{u}_{\ell-1} + \frac{2}{3}h_t\mathbf{z}(\mathbf{u}_{\ell+1}, t_{\ell+1}) = \mathbf{0}. \quad (9.193)$$

BDF2 is also A-stable and is second-order accurate in h_t . Higher-order schemes are also possible, but it can be shown that these schemes are not A-stable. Consequently, they are not often used, as our main motivation for considering implicit timestepping is to gain stability.

Using either equation (9.192) or equation (9.193), we write the nonlinear algebraic equations in general form as

$$\mathbf{h}(\mathbf{u}_{\ell+1}) = 0. \quad (9.194)$$

The standard way to solve these equations for the new state, $\mathbf{u}_{\ell+1}$, given the state at the previous time level(s), is to use Newton's method,

$$\frac{\partial \mathbf{h}}{\partial \mathbf{u}}(\mathbf{u}_{\ell+1}^{(k)}) \mathbf{w}^{(k)} = -\mathbf{h}(\mathbf{u}_{\ell+1}^{(k)}), \quad (9.195a)$$

$$\mathbf{u}_{\ell+1}^{(k+1)} = \mathbf{u}_{\ell+1}^{(k)} + \mathbf{w}^{(k)}, \quad (9.195b)$$

where a natural choice for the initial guess is given by

$$\mathbf{u}_{\ell+1}^{(0)} = \mathbf{u}_\ell. \quad (9.196)$$

Here, $\mathbf{w}^{(k)}$ is the Newton update of the k th Newton step. For example, for the backward Euler method (9.192), equation (9.195) gives

$$\left(\frac{1}{h_t} I + \frac{\partial \mathbf{z}}{\partial \mathbf{u}}(\mathbf{u}_{\ell+1}^{(k)}, t_{\ell+1}) \right) \mathbf{w}^{(k)} = - \left(\frac{\mathbf{u}_{\ell+1}^{(k)} - \mathbf{u}_\ell}{h_t} + \mathbf{z}(\mathbf{u}_{\ell+1}^{(k)}, t_{\ell+1}) \right), \quad (9.197)$$

and a similar iteration is obtained for implicit time-integration methods of higher order like BDF2. The linear systems in equation (9.197) are typically large, sparse, and nonsymmetric, and iterative Krylov methods such as GMRES (see Section 7.8) are the methods of choice for solving them. The combined nonlinear-linear solvers are called Newton–Krylov methods.

For many problems, however, it is expensive to compute and store the matrix elements of the Jacobian matrices $\frac{\partial \mathbf{z}}{\partial \mathbf{u}}(\mathbf{u})$ as in equation (9.197) at every timestep. This is especially true for high-order accurate spatial discretizations. If this becomes a computational bottleneck, Newton–Krylov methods can be implemented in a so-called *Jacobian-free manner*. In these Jacobian-free methods, the Jacobian matrix is not explicitly computed or stored, but instead the matrix-vector products that need to be computed by the Krylov method are approximated using finite

differences that only require evaluation of the function $\mathbf{z}(\mathbf{u})$, like in explicit time-integration methods,

$$\frac{\partial \mathbf{z}}{\partial \mathbf{u}}(\mathbf{u}_{\ell+1}^{(k)}, t_{\ell+1}) \mathbf{w}^{(k)} \approx \frac{\mathbf{z}(\mathbf{u}_{\ell+1}^{(k)} + \mu \mathbf{w}^{(k)}, t_{\ell+1}) - \mathbf{z}(\mathbf{u}_{\ell+1}^{(k)}, t_{\ell+1})}{\mu}, \quad (9.198)$$

where the parameter μ is usually chosen as $\mu = O(\sqrt{\epsilon_{mach}})$.

As discussed in Section 7.10, Krylov methods like GMRES usually require suitable preconditioners for efficient convergence, and this is certainly also important for hyperbolic systems. Preconditioning can also be applied in the matrix-free case. For example, for a high-order accurate spatial discretization, it may be feasible to compute and store the Jacobian of a low-order spatial scheme and use an inexpensive approximate inverse of that low-order Jacobian to precondition the high-order linear system.

The approach sketched in this section for implicit time integration of nonlinear hyperbolic systems can also be applied to higher-order BDF methods, such as BDF3 and BDF4, or other implicit multistep methods. However, the famous *second Dahlquist barrier* establishes that no implicit multistep method of order greater than two can be A-stable [78]. Thus, if we aim to achieve higher-order accuracy, we typically turn to other high-order implicit methods with better stability properties, such as the implicit Runge–Kutta methods discussed in the next section.

9.6.2 ■ Implicit Runge–Kutta methods

We consider s -stage implicit Runge–Kutta methods in standard form:

$$\mathbf{u}_{\ell+1} = \mathbf{u}_{\ell} - h_t \sum_{i=1}^s b_i \boldsymbol{\kappa}_i, \quad (9.199a)$$

$$\boldsymbol{\kappa}_i = \mathbf{z}(\mathbf{u}_{\ell} - h_t \sum_{j=1}^s a_{i,j} \boldsymbol{\kappa}_j, t_{\ell} + h_t c_i), \quad i = 1, \dots, s. \quad (9.199b)$$

We note that the signs in equation (9.199) come from the semidiscrete form in equation (9.189), which is equivalent to solving $\frac{d\mathbf{u}}{dt} = -\mathbf{z}(\mathbf{u}(t), t)$. The coefficients of these methods are often represented in a *Butcher tableau*:

$$\begin{array}{c|cccc} c_1 & a_{1,1} & a_{1,2} & \cdots & a_{1,s} \\ c_2 & a_{2,1} & a_{2,2} & \cdots & a_{2,s} \\ \vdots & \vdots & & \ddots & \vdots \\ c_s & a_{s,1} & a_{s,2} & \cdots & a_{s,s} \\ \hline & b_1 & b_2 & \cdots & b_s \end{array} \quad (9.200)$$

When $a_{i,j} = 0$ for all $j \geq i$, the Runge–Kutta method is explicit, and each stage value, $\boldsymbol{\kappa}_i$, is computed directly from previous stage values at the cost of only evaluations of the function $\mathbf{z}(\mathbf{u}, t)$. Otherwise, the method is an implicit method, and we must solve a coupled system to determine the stage values. When $\mathbf{u} \in \mathbb{R}^m$ (as is the case when it represents a spatially discretized PDE), equation (9.199) represents a coupled nonlinear system of size $sm \times sm$. When m is large, the cost of solving this nonlinear system may be prohibitive, unless h_t is very small or good preconditioners are known for some linearization of the system. For this reason, the so-called *diagonally implicit Runge–Kutta* (DIRK) methods, for which

$$a_{i,j} = 0 \quad \text{if } j > i, \quad (9.201)$$

are often used. In this case, the large nonlinear system decouples into s smaller nonlinear systems of size $m \times m$ that are solved in sequence. Defining

$$\mathbf{h}_i(\boldsymbol{\kappa}_i) = \boldsymbol{\kappa}_i - \mathbf{z}(\mathbf{u}_\ell - h_t \sum_{j=1}^i a_{i,j} \boldsymbol{\kappa}_j, t_\ell + h_t c_i), \quad (9.202)$$

we solve the nonlinear algebraic equations $\mathbf{h}_i(\mathbf{z}_i) = \mathbf{0}$ successively using Newton's method (see equation (9.195)), with

$$\frac{\partial \mathbf{h}_i}{\partial \boldsymbol{\kappa}_i}(\boldsymbol{\kappa}_i) = I + h_t a_{i,i} \frac{\partial \mathbf{z}}{\partial \mathbf{u}} \left(\mathbf{u}_\ell - h_t \sum_{j=1}^i a_{i,j} \boldsymbol{\kappa}_j, t_\ell + h_t c_i \right). \quad (9.203)$$

A popular variant of this approach is the so-called *singly diagonally implicit Runge–Kutta* (SDIRK) method, in which the diagonal elements of the Butcher matrix A are all chosen to be equal:

$$a_{i,i} = \gamma \quad \forall i. \quad (9.204)$$

These approaches allow some further cost savings in the development of preconditioners for the resulting linearizations, since their Jacobians have a fixed dependence on $\frac{\partial \mathbf{z}}{\partial \mathbf{u}}$; however, they also sacrifice some accuracy compared to fully implicit Runge–Kutta methods, so their use must be carefully considered.

Two specific choices of SDIRK methods with two and three stages are the SDIRK2 method,

$$\begin{array}{c|cc} 1 - \frac{\sqrt{2}}{2} & 1 - \frac{\sqrt{2}}{2} & 0 \\ 1 & \frac{\sqrt{2}}{2} & 1 - \frac{\sqrt{2}}{2} \\ \hline & \frac{\sqrt{2}}{2} & 1 - \frac{\sqrt{2}}{2} \end{array}, \quad (9.205)$$

and the SDIRK3 method,

$$\begin{array}{c|ccc} \gamma & \gamma & 0 & 0 \\ \eta & \zeta & \gamma & 0 \\ 1 & \alpha & \beta & \gamma \\ \hline & \alpha & \beta & \gamma \end{array}, \quad (9.206)$$

where γ is the root of $x^3 - 3x^2 + 3x/2 - 1/6 = 0$ between $1/6$ and $1/2$, $\gamma \approx 0.435866521508459$; $\eta = (1 + \gamma)/2$; $\zeta = (1 - \gamma)/2$; $\alpha = -3\gamma^2/2 + 4\gamma - 1/4$; and $\beta = 3\gamma^2/2 - 5\gamma + 5/4$. These SDIRK2 and SDIRK3 methods are accurate with order two and three, respectively, and they are both A-stable [9]. They are also L-stable, which prevents the presence of nonphysical oscillations in numerical approximations to solutions of systems of ODEs.

SDIRK methods are particularly advantageous when combined with a simplified Newton approach for solving nonlinear systems of the form in equation (9.199), in which one replaces the Jacobians in equation (9.203) by approximate Jacobians that are chosen to be the same for all Newton steps applied to all nonlinear equations $\mathbf{h}_i(\boldsymbol{\kappa}_i) = \mathbf{0}$, e.g.,

$$\frac{\partial \mathbf{h}_i}{\partial \boldsymbol{\kappa}_i}(\boldsymbol{\kappa}_i^{(m)}) \approx I + h_t \gamma \frac{\partial \mathbf{z}}{\partial \mathbf{u}}(\mathbf{u}_\ell, t_\ell). \quad (9.207)$$

In this approach, all the successive nonlinear systems $\mathbf{h}_i(\boldsymbol{\kappa}_i) = \mathbf{0}$ are solved with simplified Newton iterations that use the same approximate Jacobian matrix, which only needs to be computed once per timestep. This is exploited further to efficiently solve the linear systems in the Newton iterations, e.g., by using a single (sparse) LU decomposition, or by reusing a precomputed preconditioner, e.g., ILU, when using iterative methods like GMRES. However, this simplified Newton approach does not retain the quadratic convergence speed of Newton's method.

9.6.3 ■ Implicit-explicit Runge–Kutta methods

The fundamental challenge of time integration for PDEs is balancing the need for stability (often driven by diffusive or fast wave modes) with the computational cost of solving coupled nonlinear systems in implicit schemes. One approach to try to balance these choices is the use of implicit-explicit (IMEX) schemes, where some terms in the PDE system are integrated using explicit timestepping methods (which are effective for slow wave modes, and inexpensive, even for nonlinear terms), while others are integrated using implicit timestepping methods (which effectively treat fast wave modes and diffusive terms, but at higher cost).

As an example, we consider the viscous form of Burgers' equation,

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad (9.208)$$

where ν is a viscosity parameter. We consider semidiscretizing the problem in space, using the local Lax–Friedrichs flux function for the flux term and a simple finite-difference approximation of $\frac{\partial u}{\partial x}$ at each endpoint of the interval $[x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}}]$. The resulting semidiscretization is then written as

$$\begin{aligned} \frac{du_k(t)}{dt} + \frac{f^*(u_k(t), u_{k+1}(t)) - f^*(u_{k-1}(t), u_k(t))}{h_x} \\ = \frac{\nu}{h_x} \left(\frac{u_{k+1}(t) - u_k(t)}{h_x} - \frac{u_k(t) - u_{k-1}(t)}{h_x} \right). \end{aligned} \quad (9.209)$$

Consider discretizing equation (9.209) in time using explicit Euler. Here, we know from Section 6.3.4 that if $\nu = 0$, the scheme is stable for a reasonable choice of h_t (such as $h_t = 0.5h_x / \max_j |f'(u_j)|$). However, we also know, from Section 5.3.4, that we would require $h_t \leq 0.5h_x^2/\nu$ to achieve stability for the heat equation without considering the flux term. Switching to an implicit Euler time discretization yields unconditional stability for the heat equation, but then we would need to use some form of linearization to solve the implicit system involving the local Lax–Friedrichs flux function. Note that, from equation (6.61), we are precluded from using Newton's method here, since the local Lax–Friedrichs flux function involves a discrete maximum that is not differentiable.

The resolution to this conundrum is to use an explicit Euler method to treat the flux term (which results in a reasonable CFL condition of $h_t = O(h_x)$) while using an implicit Euler method to treat the diffusive term (simultaneously avoiding the imposition of a stricter CFL condition and the need to solve a nonlinear system with the local Lax–Friedrichs fluxes). The resulting fully discrete system to solve at each timestep is then

$$\begin{aligned} \frac{u_{k,\ell+1} - u_{k,\ell}}{h_t} + \frac{f^*(u_{k,\ell}, u_{k+1,\ell}) - f^*(u_{k-1,\ell}, u_{k,\ell})}{h_x} \\ = \frac{\nu}{h_x} \left(\frac{u_{k+1,\ell+1} - u_{k,\ell+1}}{h_x} - \frac{u_{k,\ell+1} - u_{k-1,\ell+1}}{h_x} \right). \end{aligned} \quad (9.210)$$

We rearrange this to emphasize the implicit and explicit parts of the scheme, writing

$$\begin{aligned} u_{k,\ell+1} + \frac{\nu h_t}{h_x^2} (-u_{k+1,\ell+1} + 2u_{k,\ell+1} - u_{k-1,\ell+1}) \\ = u_{k,\ell} - \frac{h_t}{h_x} (f^*(u_{k,\ell}, u_{k+1,\ell}) - f^*(u_{k-1,\ell}, u_{k,\ell})). \end{aligned} \quad (9.211)$$

In this form, we recognize the left-hand side as a linear operator applied to $u_{\ell+1}$, for which standard iterative methods would be effective if $\frac{\nu h_t}{h_x^2}$ is relatively small (e.g., $O(1)$), while effective

multigrid and domain decomposition preconditioners (see Chapter 11) could be used if $\frac{\nu h_t}{h_x^2}$ is large. Since the resulting scheme is only first order in both time and space, we typically expect $h_t \approx h_x$, so standard multigrid preconditioners could be used to effectively advance the problem from time t_ℓ to $t_{\ell+1}$.

Higher-order IMEX schemes are also possible. Recognizing that the discretization of the diffusion term in equation (9.209) is second order, we could achieve a second-order spatial discretization using the second-order TVD scheme with minmod limiter from Section 6.5 to define $f^*(u^-, u^+)$. To achieve a second-order accurate IMEX scheme, we write the semidiscretized system as a sum of two terms, one to be treated implicitly and the other to be treated explicitly. Extending the notation from equation (9.189), we write the semidiscretized system as

$$\frac{d\mathbf{u}(t)}{dt} + \mathbf{z}_I(\mathbf{u}(t), t) + \mathbf{z}_E(\mathbf{u}(t), t) = \mathbf{0}, \quad (9.212)$$

where $\mathbf{z}_I$ denotes the part of the system to be integrated implicitly and $\mathbf{z}_E$ denotes the part of the system to be integrated explicitly. As is standard in Runge–Kutta methods, we prescribe the details of the integrator using a Butcher tableau, as in equation (9.200). Here, however, we prescribe *two* Butcher tableaus, one for the integration of the implicit part of the system and one for the explicit part. To simplify the schemes, we generally assume that the implicit scheme is a DIRK scheme. Using A^I and A^E to denote the two Butcher matrices (and similar notation for the rest of the Butcher tableaus), we rewrite the s -stage scheme from equation (9.199) as

$$\mathbf{u}_{\ell+1} = \mathbf{u}_\ell - h_t \sum_{i=1}^s (b_i^I \mathbf{z}_I(\tilde{\mathbf{u}}_i, t_\ell + h_t c_i^I) + b_i^E \mathbf{z}_E(\tilde{\mathbf{u}}_i, t_\ell + h_t c_i^E)), \quad (9.213a)$$

$$\tilde{\mathbf{u}}_i = \mathbf{u}_\ell - h_t \left(\sum_{j=1}^i a_{i,j}^I \mathbf{z}_I(\tilde{\mathbf{u}}_j, t_\ell + h_t c_j^I) + \sum_{j=1}^{i-1} a_{i,j}^E \mathbf{z}_E(\tilde{\mathbf{u}}_j, t_\ell + h_t c_j^E) \right) \quad (9.213b)$$

for $i = 1, \dots, s$. Here, we directly consider *approximations* to the solution, $\mathbf{u}(t)$, at the stage times, $t_\ell + h_t c_i^I$. It is common to choose the implicit and explicit schemes such that $c_i^I = c_i^E$, so that these times are the same for both parts of the scheme, making $\tilde{\mathbf{u}}_i$ a reasonable approximation to $\mathbf{u}(t_\ell + h_t c_i^I)$. In order to make this work, we expect that the first stage time has $c_1^I = c_1^E = 0$ to allow direct dependence on $\mathbf{u}_\ell$ but that this stage approximation does not enter into the implicit side of the calculation, so that A^I is given by the Butcher matrix for an implicit scheme padded with zero entries for its first row and column. With this, a second-order IMEX scheme comes from using the implicit and explicit midpoint methods, represented with (augmented) Butcher tableaus of

$$\begin{array}{c|cc} 0 & 0 & 0 \\ 1/2 & 0 & 1/2 \\ \hline & 0 & 1 \end{array} \quad \text{and} \quad \begin{array}{c|cc} 0 & 0 & 0 \\ 1/2 & 1/2 & 0 \\ \hline & 0 & 1 \end{array}, \quad (9.214)$$

respectively. Simplifying the expressions in equation (9.213), this yields

$$\tilde{\mathbf{u}}_1 = \mathbf{u}_\ell, \quad (9.215a)$$

$$\tilde{\mathbf{u}}_2 = \mathbf{u}_\ell - \frac{h_t}{2} \left(\mathbf{z}_I \left(\tilde{\mathbf{u}}_2, t_\ell + \frac{h_t}{2} \right) + \mathbf{z}_E(\tilde{\mathbf{u}}_1, t_\ell) \right), \quad (9.215b)$$

$$\mathbf{u}_{\ell+1} = \mathbf{u}_\ell - h_t \left(\mathbf{z}_I \left(\tilde{\mathbf{u}}_2, t_\ell + \frac{h_t}{2} \right) + \mathbf{z}_E \left(\tilde{\mathbf{u}}_2, t_\ell + \frac{h_t}{2} \right) \right). \quad (9.215c)$$

Combining the first two of these, we have

$$\tilde{\mathbf{u}}_2 = \mathbf{u}_\ell - \frac{h_t}{2} \left(\mathbf{z}_I \left(\tilde{\mathbf{u}}_2, t_\ell + \frac{h_t}{2} \right) + \mathbf{z}_E(\mathbf{u}_\ell, t_\ell) \right), \quad (9.216)$$

which we recognize as the Euler IMEX scheme in equation (9.210) with timestep $h_t/2$, where we treat z_I implicitly and z_E explicitly. Since the update in equation (9.215c) has only minimal additional computational cost (of evaluating these functions at $\tilde{\mathbf{u}}_2$), we conclude that the cost of the second-order midpoint rule IMEX scheme is almost the same as that of the first-order Euler IMEX scheme, making it feasible for this problem.

The IMEX Runge–Kutta formalism in equation (9.213) can be extended in many ways. For example, the second-order midpoint rule IMEX scheme is known to not be L-stable. To achieve better stability, the implicit midpoint rule scheme can be replaced with a second-order L-stable DIRK integrator, and a corresponding second-order explicit scheme can be found. Higher-order schemes are also possible (see [21]) and can be used in combination with the higher-order spatial discretizations of the flux terms in Sections 9.4 and 9.5 and standard higher-order discretizations of any diffusive terms in the PDE under consideration.

9.7 ■ Some further thoughts

The finite-volume methods derived in this chapter are illustrated for hyperbolic systems that have broad applications in compressible fluid dynamics, for example, the 2D shallow-water equations in equation (9.32) or the 3D Euler equations of gas dynamics in equation (9.43). The beauty of numerical methods for hyperbolic systems is that they apply, in principle, to any hyperbolic system: all that is needed is to derive the hyperbolic structure of the PDE system in terms of wave speeds and the Jacobian eigenvectors. Once expressions for these are known, we can automatically apply, for example, the approaches discussed in Section 9.2 to the system. As such, the methods discussed in this chapter can be used for much more complex physical settings than the shallow-water or Euler equations. For example, adding the magnetic field vector as an additional unknown function to the Euler equations in equation (9.43), along with two of Maxwell’s equations, results in the hyperbolic system of (compressible) ideal magnetohydrodynamics (MHD), which describes conducting plasmas, with applications in solar physics and nuclear fusion.

The methods we discussed in this chapter also are useful building blocks for simulating hyperbolic systems that are extended to include small dissipative effects in the physical model. For example, when viscosity and heat conduction are added to the compressible Euler equations, we obtain the compressible Navier–Stokes equations, which describe many technical systems of great everyday importance in, e.g., aerodynamics and combustion. To give a concrete example, we consider resistive MHD, which is a widely used extension of the compressible Euler equations in equation (9.43):

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0, \quad (9.217a)$$

$$\frac{\partial(\rho \mathbf{v})}{\partial t} + \nabla \cdot \left(\rho \mathbf{v} \mathbf{v} + I \left(p + \frac{\mathbf{B} \cdot \mathbf{B}}{2} \right) - \mathbf{B} \mathbf{B} \right) = \nabla \cdot \boldsymbol{\tau}, \quad (9.217b)$$

$$\frac{\partial \mathbf{B}}{\partial t} + \nabla \cdot (\mathbf{v} \mathbf{B} - \mathbf{B} \mathbf{v}) = -\nabla \times (\eta \mathbf{J}), \quad (9.217c)$$

$$\begin{aligned} \frac{\partial e}{\partial t} + \nabla \cdot \left((e + p + \frac{\mathbf{B} \cdot \mathbf{B}}{2}) \mathbf{v} - (\mathbf{v} \cdot \mathbf{B}) \mathbf{B} \right) \\ = \nabla \cdot (\mathbf{v} \cdot \boldsymbol{\tau}) - \nabla \cdot \mathbf{q} - \nabla \cdot \left(\eta \mathbf{J} \times \mathbf{B} \right), \end{aligned} \quad (9.217d)$$

$$\nabla \cdot \mathbf{B} = 0. \quad (9.217e)$$

These equations may, at first sight, contain many quantities that look unfamiliar, so we explain step by step how they relate to what we have learned in this chapter. To start, ρ is the mass

density of the plasma and $\mathbf{v} = (v_x, v_y, v_z)$ its fluid velocity. The magnetic field is denoted by $\mathbf{B} = (B_x, B_y, B_z)$, and $\mathbf{J}$ is the electric current density, typically related to $\mathbf{B}$ via Ampère's law (up to certain constants depending on the unit system used):

$$\mathbf{J} = \nabla \times \mathbf{B}. \quad (9.218)$$

The other unknowns, p and e , represent the pressure and energy of the system, respectively. Finally, η is the resistivity of the electromagnetic system, analogous to a fluid viscosity; $\mathbf{q}$ represents an energy source term such as heat flux; and τ is the fluid stress tensor, which we discuss more below.

If we disregard the right-hand side terms for the moment and set the magnetic field $\mathbf{B}$ equal to zero, then equations (9.217a)–(9.217d) revert to the compressible Euler equations of equation (9.43). Here, factors such as $\mathbf{v}\mathbf{v}$ denote a vector outer product, and I is the 3×3 identity matrix. As for the Euler equations, the gas pressure is expressed as a function of the energy, density, velocity, and magnetic field using

$$p = (\gamma - 1) \left(e - \frac{1}{2} \rho \|\mathbf{v}\|^2 - \frac{1}{2} \|\mathbf{B}\|^2 \right), \quad (9.219)$$

where γ is a thermodynamic constant. For the case of nonzero $\mathbf{B}$, equation (9.217c) and equation (9.217e) originate from Maxwell's equations. In particular, equation (9.217c) comes from Faraday's law of induction, and equation (9.217e) is Gauss's law for magnetism. Notably, if equation (9.217e) is satisfied at an initial time $t = 0$, then it remains satisfied for all $t > 0$ under the time evolution described by equations (9.217a)–(9.217d).

Without the right-hand side, equations (9.217a)–(9.217d) are known as the (compressible) ideal MHD equations, and they form a hyperbolic system with eight equations for eight unknown functions of space and time (namely, ρ , e , $\mathbf{v}$, and $\mathbf{B}$). The eigenvalues of the Jacobian, e.g., in the x -direction, are given by

$$\lambda_{1,2} = v_x \pm c_{fx}, \quad (9.220a)$$

$$\lambda_{3,4} = v_x \pm c_{Ax}, \quad (9.220b)$$

$$\lambda_{5,6} = v_x \pm c_{sx}, \quad (9.220c)$$

$$\lambda_7 = v_x, \quad (9.220d)$$

$$\lambda_8 = 0, \quad (9.220e)$$

where c_{fx} , c_{sx} , and c_{Ax} are the so-called *fast*, *slow*, and *Alfvén* wave speeds of ideal MHD, given by

$$c_{fx}^2 = \frac{1}{2} \left(\frac{\gamma p + \|\mathbf{B}\|^2}{\rho} + \sqrt{\left(\frac{\gamma p + \|\mathbf{B}\|^2}{\rho} \right)^2 - 4 \frac{\gamma p B_x^2}{\rho^2}} \right), \quad (9.221a)$$

$$c_{sx}^2 = \frac{1}{2} \left(\frac{\gamma p + \|\mathbf{B}\|^2}{\rho} - \sqrt{\left(\frac{\gamma p + \|\mathbf{B}\|^2}{\rho} \right)^2 - 4 \frac{\gamma p B_x^2}{\rho^2}} \right), \quad (9.221b)$$

$$c_{Ax}^2 = \frac{B_x^2}{\rho}. \quad (9.221c)$$

The finite-volume methods discussed in Section 9.2 can be directly applied to the ideal MHD system, for example, to simulate solar wind and space weather events [173]. There is, however, one numerical complication: while the PDE system satisfies the divergence constraint in equation (9.217e) for all times, numerical simulations do not automatically inherit this property, often

leading to numerical instability. Therefore, an additional numerical stabilization mechanism is required (see, e.g., [177] for a brief discussion and references).

Finally, we consider adding back in the right-hand side terms of equations (9.217a)–(9.217d) and note that these are dissipative terms that contain second-order derivatives of the unknown functions (like in the heat equation). Viscous effects in the compressible fluid are modeled using the fluid stress tensor τ in equation (9.217b) and equation (9.217d), given by

$$\tau = 2\mu \left(\varepsilon - \frac{1}{3}(\nabla \cdot \mathbf{v})\mathbf{I} \right), \quad (9.222)$$

where μ is the dynamic viscosity and

$$\varepsilon = \frac{1}{2} (\nabla \mathbf{v} + (\nabla \mathbf{v})^T) \quad (9.223)$$

is the fluid strain rate tensor. Heat conduction is modeled using the heat flux $\mathbf{q}$ in equation (9.217d), which according to Fourier's law of thermal conduction is written as

$$\mathbf{q} = -\kappa \nabla T, \quad (9.224)$$

where κ is the thermal conductivity and T is the temperature. The temperature is expressed as a function of pressure and density using an equation of state. For the Euler case where $\mathbf{B} = 0$, the system of equations is, again, known as the compressible Navier–Stokes equations. For the MHD case with nonzero $\mathbf{B}$, electric resistivity provides an additional dissipative effect, modeled by the terms containing the resistivity coefficient η in equation (9.217c) and equation (9.217d). Typically, the second-order derivatives in all of these dissipative terms are discretized using centered difference methods.

In summary, the numerical methods we have discussed in this chapter are quite versatile and can be applied to a variety of PDE systems that model physical systems governed by conservation laws.

9.8 ■ Exercises

1. Consider the one-dimensional shallow-water equations from Example 9.3, with conserved variables $\mathbf{u}$ and flux function $\mathbf{f}(\mathbf{u})$ given by

$$\mathbf{u} = \begin{bmatrix} h \\ m_x \\ m_y \end{bmatrix} \quad \text{and} \quad \mathbf{f}(\mathbf{u}) = \begin{bmatrix} m_x \\ \frac{m_x^2}{h} + \frac{gh^2}{2} \\ \frac{m_x m_y}{h} \end{bmatrix}.$$

Verify the expressions for the flux Jacobian and its eigenvalues given in the chapter, and find expressions for the eigenvectors. Use Definition 9.4 to verify the claim that the modes associated with two eigenvalues are genuinely nonlinear, while the third is a linearly degenerate wave mode.

2. A second form of the one-dimensional shallow-water equations arises when we consider flow over a nonconstant bottom topography, specified by differentiable function $b(x)$, where $h(x, t)$ now denotes the height of water above the bottom and $u(x, t) := v_x(x, t)$ denotes the horizontal velocity of the water. Then, we have the equations

$$\frac{\partial}{\partial t} \begin{bmatrix} h \\ hu \end{bmatrix} + \frac{\partial}{\partial x} \begin{bmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \end{bmatrix} = \begin{bmatrix} 0 \\ -ghb'(x) \end{bmatrix}.$$

- (a) Verify that, for any constant c , the solution $h(x, t) = c - b(x)$ and $u(x, t) = 0$ is a solution of these equations. This is known as the *lake-at-rest* solution of the shallow-water equations. Why?
- (b) A numerical approximation scheme for these equations is called a *well-balanced scheme* if it exactly preserves the lake-at-rest solution. Verify that the numerical scheme given by

$$\begin{aligned} & \frac{h_{k,\ell+1} - h_{k,\ell}}{h_t} + \frac{h_{k+1,\ell}u_{k+1,\ell} - h_{k-1,\ell}u_{k-1,\ell}}{2h_x} = 0, \\ & \frac{h_{k,\ell+1}u_{k,\ell+1} - h_{k,\ell}u_{k,\ell}}{h_t} + \frac{h_{k+1,\ell}(u_{k+1,\ell})^2 - h_{k-1,\ell}(u_{k-1,\ell})^2}{2h_x} \\ & + \frac{g(h_{k+1,\ell})^2 - (h_{k-1,\ell})^2}{2h_x} = -g \frac{h_{k+1,\ell} + h_{k-1,\ell}}{2} \frac{b_{k+1} - b_{k-1}}{2h_x}, \end{aligned}$$

is a well-balanced scheme.

3. Consider the linear hyperbolic system $\mathbf{u}_t + A \mathbf{u}_x = \mathbf{0}$ with

$$A = \begin{bmatrix} 9 & 4 \\ -12 & -5 \end{bmatrix}.$$

What is the solution of the Riemann problem with $\mathbf{u}^- = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ and $\mathbf{u}^+ = \begin{bmatrix} 5 \\ -8 \end{bmatrix}$? Describe the solution in detail. What is $\mathbf{u}^*$, the solution state between $\mathbf{u}^-$ and $\mathbf{u}^+$? To illustrate the solution, plot u_1 and u_2 as a function of x , for $t = 0$ and for $t = 1$, and plot the solution in the x - t plane (indicate in which parts of the plane the solution is $\mathbf{u}^-$, $\mathbf{u}^*$, and $\mathbf{u}^+$).

4. The time-dependent isentropic Euler equations in one spatial dimension are given by

$$\frac{\partial}{\partial t} \begin{bmatrix} \rho \\ \rho v \end{bmatrix} + \frac{\partial}{\partial x} \begin{bmatrix} \rho v \\ \rho v^2 + s \rho \gamma \end{bmatrix} = \mathbf{0},$$

with s and γ positive constants that represent the fluid entropy and adiabatic constant, respectively. Here, v is the fluid velocity in the x -direction, and you can assume that the density ρ is positive. These equations are derived from the compressible Euler equations by assuming that the entropy s is constant (this condition replaces the energy equation).

- (a) Write the system in quasi-linear form, and compute the eigenvalues and eigenvectors of the flux Jacobian. Is this system hyperbolic?
- (b) Write down the Rankine–Hugoniot relation for this system. Consider state $\mathbf{u}^- = \begin{bmatrix} \rho^- \\ \rho^- v^- \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. Give (nonlinear) equations that determine a state, $\mathbf{u}^+$, that is connected to $\mathbf{u}^-$ through a shock with speed $\hat{x}'(t) = 0$.
- (c) What is the physical significance of the form of the two eigenvalues (i.e., wave speeds) you find? *Hint:* Think about throwing a rock in a one-dimensional lake ($v = 0$) or in a river flowing with speed $v \neq 0$.
5. Let ψ_k represent the degree- k normalized Legendre polynomial on $[-1, 1]$. Consider a function $u(x)$ which can be represented by the infinite expansion

$$u(x) = \sum_{k=0}^{\infty} \tilde{u}_k \psi_k(x)$$

and an approximation $u_p(x)$ given by the truncated expansion

$$u_p(x) = \sum_{k=0}^p \tilde{u}_k \psi_k(x).$$

In either case, the coefficients are given by

$$\tilde{u}_k = \int_{-1}^1 u(x) \psi_k(x) dx.$$

The goal is to place a bound on the error in the approximation $u_p(x)$.

- (a) Derive an expression relating $\|u - u_p\|$ to the coefficients $\tilde{u}_k$.
- (b) The Legendre polynomials, ψ_k , satisfy the Sturm–Liouville problem

$$\partial_x(1 - x^2)\partial_x\psi_k(x) = -k(k+1)\psi_k(x).$$

Assuming that $u \in H^m([-1, 1])$ for some $m \geq 1$, use this identity to place a bound on $\tilde{u}_k$ of the form

$$|\tilde{u}_k| \leq ck^\alpha \|u\|_m$$

for some constant $c > 0$ and α depending on m . *Hint:* Use $m = 2q$ applications of integration by parts.

- (c) Combine the first two parts to establish a bound on the error of the form

$$\|u - u_p\|^2 \leq cp^\alpha \|u\|_m.$$

- (d) What does this say about the relationship between the smoothness of $u(x)$ and the coefficients in the Legendre expansion?

6. Consider the viscous wave equation

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

for constants $a, \nu > 0$ on spatial domain $\Omega = [0, 1]$ with periodic boundary conditions. Semidiscretize the equation on a uniform grid of n_x cells in space using the local Lax–Friedrichs flux function (which is equivalent to the FOU discretization of Section 6.3.1) for the first-order term and finite differences for the second-order term, leading to a semidiscretization of the same form as in equation (9.209).

- (a) Show that the semidiscretization can be written as $\frac{\partial}{\partial t} \mathbf{u} = A \mathbf{u}$ for some matrix A . Show that the vectors $u_k^{(j)} = e^{ikj2\pi/n_x}$ are eigenvectors of A for $0 \leq j < n_x$. Express the associated eigenvalues λ_j in terms of a and ν . Show that the real part of λ_j is negative for $1 \leq j < n_x$ and that $\lambda_0 = 0$.
- (b) Given an initial condition, $\mathbf{u}_0 = \sum_{j=0}^{n_x-1} c_{j,0} \mathbf{u}^{(j)}$, express the solution to the semidiscretization in terms of the eigenvectors and eigenvalues of A . What is $\lim_{t \rightarrow \infty} \mathbf{u}(t)$?
- (c) Suppose that n_x is even, and consider the mode with $j = n_x/2$. For values $a = 1$, $\nu = 0.1$, and $h_x = 0.01$, at what time, t , does the contribution of this mode to the solution of the semidiscrete scheme drop below 10^{-6} of its original magnitude? How much is this mode reduced in magnitude by time $t = 0.01$?

- (d) Consider taking a single timestep of the implicit trapezoid rule, given by

$$\mathbf{u}_{\ell+1} - \frac{h_t}{2} A \mathbf{u}_{\ell+1} = \mathbf{u}_{\ell} + \frac{h_t}{2} \mathbf{u}_{\ell}$$

for $\ell \geq 0$, with $h_t = 0.01$. How does the magnitude of mode $j = n_x/2$ in $\mathbf{u}_1$ compare with that at time $t = 0.01$ from part (c)?

- (e) Explain the benefit of an L-stable integration scheme compared to an A-stable scheme for this problem, noting that the implicit trapezoid rule is A-stable but not L-stable.

Part IV

Advanced topics

Chapter 10

Advanced discretizations

Overview

So far, we have mainly focused on “standard” discretization methods for classical PDEs, such as finite-difference and finite-element methods for scalar elliptic equations. However, as we have seen in the progression from Chapter 6 to Chapter 9, there are often complications when moving from a simple scalar PDE to a complex system of PDEs. Indeed, developing and analyzing such schemes remains a main focus in the scientific computing research literature. In this chapter, we present several established families of “advanced” discretizations for a PDE or system of PDEs. In particular, we introduce extensions of the finite-difference and finite-element methods for systems of PDEs with multiple unknowns. Many applications, such as Maxwell’s equations for electromagnetism and the Navier–Stokes equations for fluids, can be discretized with the techniques described here, in several different ways. We conclude this chapter with brief introductions to two families of techniques that are somewhat disjoint from the rest of the book, one where we purposely construct approximate solutions from a *nonconforming* space (one on which the original PDE fails to be guaranteed to be well defined) and one where we construct solutions from a very smooth approximation space.

Objectives

In this chapter, we consider a variety of discretization methods based on the finite-difference and finite-element approaches discussed in earlier chapters and mainly applied to systems of PDEs. While each section can be viewed independently, covering them in order presents the most logical structure of a story. Specifically, you will consider variations of finite-difference schemes to handle such systems, and you will introduce function spaces for solutions that are not in $H^1(\Omega)$ to modify the finite-element method appropriately. You will also recognize properties of systems of PDEs and identify several state-of-the-art discretization methods for various applications that are beyond the focus of the earlier chapters in this book.

Chapter Outline

Section 10.1 Staggered and mimetic finite-difference discretizations discusses the challenges in extending the finite-difference methodology to some systems of PDEs. Here, the marker-and-cell (MAC) and Yee staggered finite-difference schemes for the Stokes equations and Maxwell’s equations, respectively, are presented.

Section 10.2 Finite-element discretizations in spaces other than H^1 discusses some problems where we cannot directly apply the classical H^1 -conforming finite-element approach shown in Chapter 8. Here, $\mathbf{H}(\text{div})$ -, $\mathbf{H}(\text{curl})$ -, and H^2 -conforming elements are presented, along with a discussion of their approximation properties and some examples. The concept of the de Rham complex is also considered to discuss the *structure-preserving* properties of the discrete operators.

Section 10.3 Mixed finite-element methods introduces the mixed finite-element method for systems of PDEs. This section introduces the notion of an inf-sup condition, with application to Darcy flow and the Stokes equations. Several “pairs” of finite-element spaces are introduced and analyzed.

Section 10.4 Least-squares finite-element methods presents the first-order system least-squares finite-element approach for solving systems of PDEs. Here, equivalent systems of PDEs are constructed to be H^1 -elliptic, yielding a variety of useful properties, such as outputting symmetric positive-definite linear systems and efficient and reliable a posteriori error estimates.

Section 10.5 Discontinuous Galerkin finite-element methods discusses the construction of a nonconforming discontinuous Galerkin (DG) method for elliptic PDEs. We define numerical fluxes for the solution and its gradient to define variants such as the symmetric interior penalty method.

Section 10.6 Spectral methods presents spectral methods for solving PDEs without the need to construct discrete derivative operators. Here, the idea of a “fast Poisson solver” is introduced.

10.1 ■ Staggered and mimetic finite-difference discretizations

Objectives

The finite-difference discretizations of Chapter 3 can be adapted and extended to many purposes. One common difficulty arises when considering certain systems of PDEs, where using naïve discretization approaches leads to poor approximations. In this section, you will study two common systems of PDEs where this complication arises, those coming from the Stokes equations and Maxwell’s equations. Specifically, you will

- highlight the instability associated with standard finite-differencing techniques for the Stokes equations and construct a so-called *staggered* discretization, also known as the marker-and-cell (MAC) scheme, and
- summarize a staggered approach to Maxwell’s equations, called the Yee scheme.

10.1.1 ■ Marker-and-cell discretizations

The Stokes equations are given by

$$-\Delta \mathbf{u} + \nabla p = \mathbf{f}, \quad (10.1a)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (10.1b)$$

where $\mathbf{u}$ is the velocity of a highly viscous fluid and p is the fluid pressure. In the simplest case, boundary conditions are imposed only on $\mathbf{u}$, although this leaves a (trivial) nullspace in the system, since any constant shift in the pressure does not change its gradient. This is often resolved by adding a condition that either $p = 0$ at some point in the domain or p has zero mean (its integral over the domain equals zero). The Stokes equations and its boundary/compatibility conditions are discussed further in Section 10.3.3 in the context of the finite-element method. While important in their own right, the Stokes equations also serve as a useful prototype model for many problems in *incompressible* fluid and solid mechanics, such as models of Newtonian fluids or rubber-like materials.

In the following, we introduce the so-called *marker-and-cell* (MAC) discretization. But first, we look at what happens if we discretize the Stokes equations using a naïve approach. Consider the problem in two spatial dimensions, writing the two-component velocity as $\mathbf{u} = \begin{bmatrix} u \\ v \end{bmatrix}$. In terms of u and v , the equations are

$$-\Delta u + \partial_x p = f, \quad (10.2a)$$

$$-\Delta v + \partial_y p = g, \quad (10.2b)$$

$$\partial_x u + \partial_y v = 0. \quad (10.2c)$$

We then discretize these equations on a uniform mesh of the unit square, with $x_i = ih_x$ and $y_j = jh_y$ for $h_x = 1/n_x$ and $h_y = 1/n_y$, $0 \leq i \leq n_x$, $0 \leq j \leq n_y$. Approximating $u_{i,j} \approx u(x_i, y_j)$, $v_{i,j} \approx v(x_i, y_j)$, and $p_{i,j} \approx p(x_i, y_j)$, standard second-order differencing leads to

$$\frac{-u_{i-1,j} + 2u_{i,j} - u_{i+1,j}}{h_x^2} + \frac{-u_{i,j-1} + 2u_{i,j} - u_{i,j+1}}{h_y^2} + \frac{p_{i+1,j} - p_{i-1,j}}{2h_x} = f_{i,j}, \quad (10.3a)$$

$$\frac{-v_{i-1,j} + 2v_{i,j} - v_{i+1,j}}{h_x^2} + \frac{-v_{i,j-1} + 2v_{i,j} - v_{i,j+1}}{h_y^2} + \frac{p_{i,j+1} - p_{i,j-1}}{2h_y} = g_{i,j}, \quad (10.3b)$$

$$\frac{u_{i+1,j} - u_{i-1,j}}{2h_x} + \frac{v_{i,j+1} - v_{i,j-1}}{2h_y} = 0. \quad (10.3c)$$

Unfortunately, this discretization is unstable. As discussed above, with standard boundary conditions on the velocity, we expect a one-dimensional nullspace for the problem (associated with constant shifts in the pressure). Yet, with the centered finite-difference discretization of the first-derivative terms, we see that the system also has a so-called *red-black* or *checkerboard* mode in its nullspace. Consider $\hat{p}_{i,j} = (-1)^{i+j}$, that is, the vector over the mesh that takes value $+1$ if $i+j$ is even and -1 if $i+j$ is odd. Clearly, this is a highly oscillatory vector, since it takes both values of ± 1 at every pair of adjacent grid points on the mesh. However, it also satisfies $\hat{p}_{i-1,j} = \hat{p}_{i+1,j}$ and $\hat{p}_{i,j+1} = \hat{p}_{i,j-1}$ for every value of (i, j) . As a result, if u , v , and p are vectors defining a solution to equations (10.3a)–(10.3c), then not only does any shift of p by a constant also satisfy the equations, but so does $p + c\hat{p}$ for any constant c .

Direct options for removing this instability, but preserving the collocated structure of the discretization (where u , v , and p are all approximated at the same nodes), are generally not preferred. One option, for example, is to switch from using centered first-derivative approximations to one-sided approximations. However, this reduces the order of the scheme. Instead, in 1965, Harlow and Welch introduced the MAC scheme, a *staggered* discretization of the equations [124]. Their approach retains second-order accuracy without the instability of the collocated discretization, but at the expense of requiring a discretization with the discrete approximations to u , v , and p all located at different positions on the grid.

A natural way to do this is to consider what we need from the first derivatives in the system if we are to use second-order centered differencing to approximate them midway between two nodal values. Then, we require approximation points for u to be aligned in the y -direction with those for p , but staggered by a “half step,” so that if we use a fixed mesh width, h_x , we should have alternating approximation points for u and p along a line parallel to the x -axis, with spacing of $h_x/2$ between each approximation point for u and each of its neighboring approximation points for p . Similarly, we require the approximation points for v to be aligned in the x -direction with the approximation points for p , again staggered by distances of $h_y/2$. If we then take a regular mesh with spacing h_x and h_y for p , this defines the locations of approximation points for u and v shown in Figure 10.1.1. While we could directly identify the mesh for p with any (consistent) location in the cells, it is common to identify them as the cell centers, so that u is identified with approximations at the edge midpoints on the left and right of each cell and v is identified with approximations at the edge midpoints on the top and bottom of each cell. In three dimensions, a similar staggering is used, now with the pressure degrees of freedom at the center of each hexahedral cell and the velocity degrees of freedom represented by the normal component of velocity on each face of the cell.

On this mesh, we still use a regular five-point finite-difference stencil for the second-order derivatives of u and v , given that these degrees of freedom are also arrayed on a regular mesh. We index the mesh so that point (x_i, y_j) denotes the intersection of horizontal and vertical edges, and we identify discrete degrees of freedom with $u_{i,j} = u(x_i, y_{j+1/2})$ for $0 \leq i \leq n_x, 0 \leq j < n_y$; $v_{i,j} = v(x_{i+1/2}, y_j)$ for $0 \leq i < n_x, 0 \leq j \leq n_y$; and $p_{i,j} = p(x_{i+1/2}, y_{j+1/2})$ for $0 \leq i < n_x, 0 \leq j < n_y$, where $x_{i+1/2} = x_i + h_x/2$ and $y_{j+1/2} = y_j + h_y/2$ (see Figure 10.1.1). The staggering induced on the u and v approximation points results in standard centered differences of u in the x -direction and v in the y -direction that are collocated, giving a natural representation of the divergence of the vector velocity field, $\mathbf{u}$, at the cell centers (where p is approximated). The discretization becomes

$$\frac{-u_{i-1,j} + 2u_{i,j} - u_{i+1,j}}{h_x^2} + \frac{-u_{i,j-1} + 2u_{i,j} - u_{i,j+1}}{h_y^2} + \frac{p_{i,j} - p_{i-1,j}}{h_x} = f_{i,j}, \quad (10.4a)$$

$$\frac{-v_{i-1,j} + 2v_{i,j} - v_{i+1,j}}{h_x^2} + \frac{-v_{i,j-1} + 2v_{i,j} - v_{i,j+1}}{h_y^2} + \frac{p_{i,j} - p_{i,j-1}}{h_y} = g_{i,j}, \quad (10.4b)$$

$$\frac{u_{i+1,j} - u_{i,j}}{h_x} + \frac{v_{i,j+1} - v_{i,j}}{h_y} = 0. \quad (10.4c)$$

Remark 10.1: Notation

It is important to note that the definition of the degrees of freedom used for the naïve approach, equation (10.3), is different than for the MAC scheme, equation (10.4), even though the notation looks similar in the resulting equations. For simplicity of displaying the equations, we do not use the half-index notation in the MAC scheme even though the degrees of freedom are defined on midway points of the mesh. This is fully illustrated in Figure 10.1.1.

Importantly, the grid spacing is reflected in the right-hand side terms, with $f_{i,j} = f(x_i, y_{j+1/2})$ to match the staggering for u and $g_{i,j} = g(x_{i+1/2}, y_j)$ to match the staggering for v . This choice of degrees of freedom also allows natural imposition of Dirichlet boundary conditions on the normal component of velocity along any edge, which is a common form of Dirichlet boundary condition for this problem. However, we still need to take care when we have degrees of freedom adjacent to a boundary. Consider, for instance, the equation for $u_{i,0} = u(x_i, y_{1/2})$. This requires

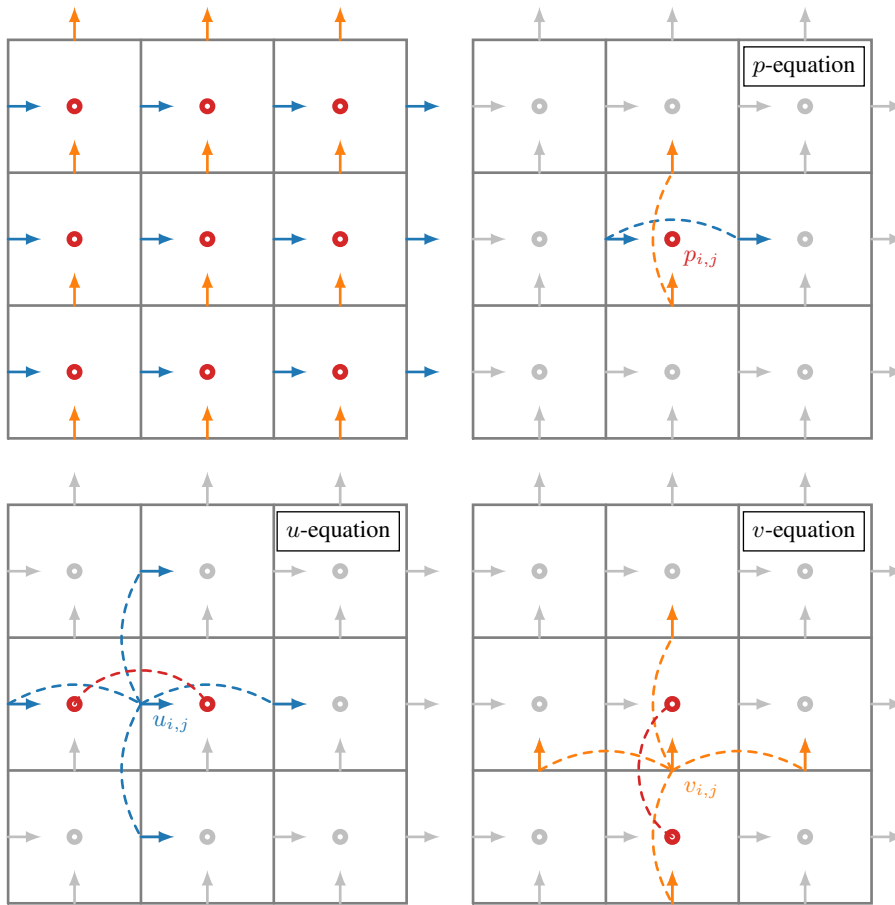


Figure 10.1.1. Degrees of freedom (upper left) on a regular two-dimensional mesh for the MAC scheme finite-difference discretization of the Stokes equations. The pressure is approximated at the nodes denoted by dots (in red), while u and v are approximated at the nodes denoted by horizontal and vertical arrows (in blue and in orange), respectively. Stencils for equations (10.4a)–(10.4c) are sketched in the lower left, lower right, and upper right, respectively.

knowledge of $u_{i,-1} = u(x_i, y_{-1/2})$, which is below the bottom boundary of the mesh. Assuming a Dirichlet boundary condition $u = q_D$ on the bottom boundary, though, we can approximate $u_{i,-1}$ with an average $\frac{u_{i,0} + u_{i,-1}}{2} = q_D$ and update the system accordingly.

Finally, we note that the MAC discretization does not suffer from the same checkerboard instability as the collocated discretization does; in fact, it is stable and second-order accurate for sufficiently smooth solutions. We illustrate this in Figures 10.1.2 and 10.1.3 on a simple 2D test problem with exact solution $u(x, y) = \sin(\pi x) \cos(\pi y)$, $v(x, y) = -\cos(\pi x) \sin(\pi y)$, and $p(x, y) = 1 - x$. Assuming a grid with $n_x = n_y$, such that $h := h_x = h_y = \frac{1}{n_x}$, we see the expected $\mathcal{O}(h^2)$ convergence of the scheme in the discrete L^2 norm. For comparison, we show results from the naïve approach in equation (10.3), where the pressure exhibits oscillatory behavior. This is highlighted in Figure 10.1.4, which plots the absolute error in the pressure. In both implementations, the one-dimensional nullspace is eliminated by fixing one degree of freedom of the pressure to equal the exact solution.

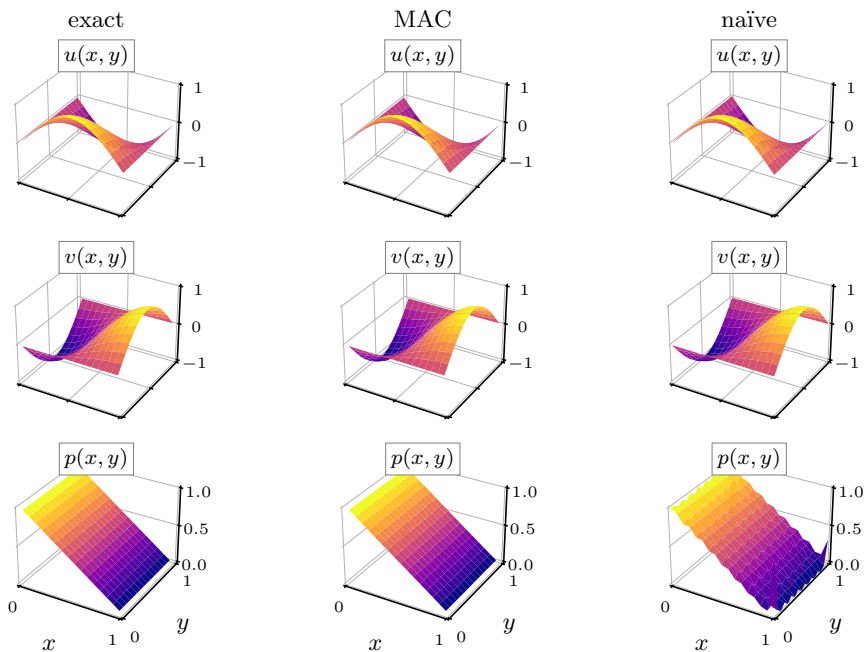


Figure 10.1.2. (left) Exact versus approximated solutions of the Stokes equations using the (middle) MAC discretization and (right) collocated (naïve) discretization. A staggered grid with $n_x = n_y = 16$ is used with unpreconditioned GMRES as the solver.

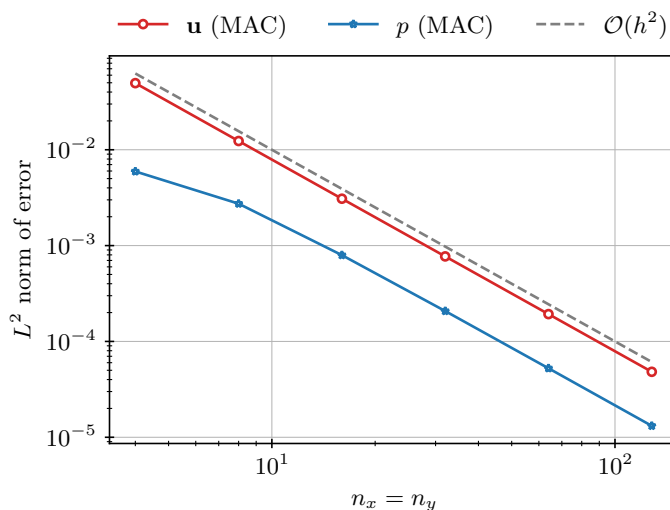


Figure 10.1.3. Discrete L^2 norm of the error for the Stokes equations using the MAC discretization, equation (10.4), versus mesh size. Here, a direct solver is used on the nonsingular system.

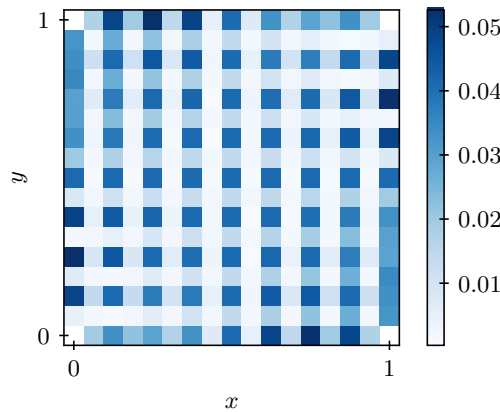


Figure 10.1.4. Absolute error in the pressure for the Stokes equations using the collocated (naïve) discretization, equation (10.3), on a 16×16 grid with unpreconditioned GMRES as the solver. Since the error at the four corner points is high, we mask those values in the plot in order to see the checkerboard pattern on the rest of the domain.

10.1.2 ■ The Yee scheme and mimetic finite-volume discretizations

We next consider a simplified form of Maxwell's equations. For a bounded, connected domain, $\Omega \in \mathbb{R}^3$, Maxwell's equations relate an electric field, $\mathbf{E}$, and a magnetic field, $\mathbf{B}$, with

$$\frac{\partial \mathbf{B}}{\partial t} + \nabla \times \mathbf{E} = \mathbf{0}, \quad (10.5a)$$

$$\frac{\partial \mathbf{E}}{\partial t} - \nabla \times \mathbf{B} = -\mathbf{J}, \quad (10.5b)$$

$$\nabla \cdot \mathbf{E} = 0, \quad (10.5c)$$

$$\nabla \cdot \mathbf{B} = 0 \quad (10.5d)$$

for $(\mathbf{x}, t) \in \Omega \times (0, T]$. This is a *simplified* version of Maxwell's equations and includes several assumptions. In particular, these equations are valid in a vacuum, and the system has been nondimensionalized, so that important physical constants, such as permittivity and permeability of the medium, are set equal to one. Additionally, we assume that the current density, $\mathbf{J}(\mathbf{x}, t)$, is known and satisfies $\nabla \cdot \mathbf{J} = 0$. Finally, we choose so-called *perfect conductor* boundary conditions—i.e., $\mathbf{B} \cdot \mathbf{n}|_{\partial\Omega} = 0$ and $\mathbf{n} \times \mathbf{E}|_{\partial\Omega} = \mathbf{0}$, where $\mathbf{n}$ is a normal vector to the boundary, $\partial\Omega$.

A standard trick is to consider the time-harmonic equations, where a Fourier transform in time is taken to reduce the problem to only a spatial one. Or, one can consider a simple finite-difference scheme for the time derivatives, again making the system depend only on the spatial domain. In many cases, these techniques allow us to reduce this system of first-order equations in two variables ($\mathbf{E}$ and $\mathbf{B}$) to a single second-order equation in one of the variables. One common form is the so-called *eddy-current* equation:

$$c_1 \mathbf{E} + \nabla \times (c_2 \nabla \times \mathbf{E}) = -\mathbf{J} \text{ in } \Omega \quad (10.6)$$

for some constants $c_1 \geq 0$ and $c_2 > 0$.

The standard finite-difference discretization for both forms of Maxwell's equations in this example is known as Yee's scheme [236] and was originally proposed for the first-order form of equations (10.5a) and (10.5b). In first-order form, the curl operators in Maxwell's equations

pose a particular challenge, since each component of the curl of $\mathbf{E}$ involves two (but not three) of the derivatives of $\mathbf{E}$ and must be meaningfully defined at the location of the corresponding component for $\mathbf{B}$, with a similar relationship between the components of the curl of $\mathbf{B}$ and those of $\mathbf{E}$. The solution to these challenges is, again, a careful staggering of the degrees of freedom for $\mathbf{B}$ and $\mathbf{E}$, as it was for the MAC scheme. Considering the first component of $\nabla \times \mathbf{E}$, $\partial_y E_3 - \partial_z E_2$, we see that we wish to represent B_1 on a mesh where discretization points for E_3 are on lines of the same x and z coordinates, but spaced by $h_y/2$ in the y -direction, and those for E_2 are on lines with the same x and y coordinates, but spaced by $h_z/2$ in the z -direction. Since $\partial_y B_1$ appears in the third component of $\nabla \times \mathbf{B}$ and $\partial_z B_1$ appears in the second component, we see that the spacing is fortuitous, enabling us to naturally evaluate the derivatives needed to “match” the time derivative of $\mathbf{B}$ with components of $\nabla \times \mathbf{E}$ and the time derivative of $\mathbf{E}$ with the components of $\nabla \times \mathbf{B}$. Similar staggering relationships are derived for the other components of $\mathbf{E}$ and $\mathbf{B}$, with the final depiction of the staggered mesh as shown in Figure 10.1.5. In two dimensions (known as a “slab” geometry, where $\mathbf{B}$ and $\mathbf{E}$ are taken to be independent of z), the scheme simplifies, since the curl of a two-dimensional vector field (represented by its tangential components on the edges) is naturally mapped to a scalar value at the cell center, and vice versa.

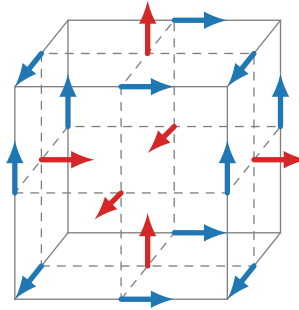


Figure 10.1.5. Degrees of freedom for $\mathbf{E}$ (in blue) and $\mathbf{B}$ (in red) on a regular three-dimensional mesh for the Yee scheme finite-difference discretization of Maxwell’s equations. Each arrow indicates the direction of the components of the discretization, with the y - z plane shown in the page and the positive x -direction coming out of the page.

Although very classical, Yee’s scheme is still the foundation of many commonly used computational methods for Maxwell’s equations to this day [118]. In particular, the broad family of mimetic finite-difference methods (cf. [153]) are seen as a natural extension of Yee’s scheme, using staggered finite-difference ideas to preserve continuum properties in consistent discretizations. The method is also at the center of the family of “finite-difference time-domain” solvers that are commonly used in many electromagnetic applications.

10.2 ■ Finite-element discretizations in spaces other than H^1

Objectives

The general theory for finite-element methods proposed in Chapter 8 considers elliptic PDEs and their variational forms for the case where solutions are in $H^1(\Omega)$. In fact, in Example 8.24, the energy norm associated with the weak form is exactly the $H^1(\Omega)$ norm. In these examples, we find natural bounds on the bilinear form in terms of the L^2 norms of the solution and its gradient. However, in many applications, such as electromagnetics and other coupled problems, bounds on the H^1 (semi)norm may not be necessary to achieve good performance, or may not

even be possible. Therefore, in this section, you will study finite-element spaces that are not derived from $H^1(\Omega)$. Specifically, you will

- consider $\mathbf{H}(\text{div}, \Omega)$ - and $\mathbf{H}(\text{curl}, \Omega)$ -conforming finite-element spaces, including their construction and application, and
- define several examples of $H^2(\Omega)$ -conforming spaces.

10.2.1 ■ Raviart–Thomas and Nédélec finite elements

We begin this section by revisiting Maxwell's equations, as presented in Section 10.1. Consider the variational form associated with equation (10.6): find $\mathbf{E} \in \mathcal{V}$ such that

$$c_1 \langle \mathbf{E}, \mathbf{F} \rangle + c_2 \langle \nabla \times \mathbf{E}, \nabla \times \mathbf{F} \rangle + c_2 \int_{\partial\Omega} (\nabla \times \mathbf{E}) \cdot (\mathbf{F} \times \mathbf{n}) \, ds = -\langle \mathbf{J}, \mathbf{F} \rangle \quad \forall \mathbf{F} \in \mathcal{V}, \quad (10.7)$$

where integration by parts is used to move the curl operator onto the test function, $\mathbf{F}$, and where $\mathbf{n}$ is the outward unit normal vector to $\partial\Omega$. Note that we again use $\langle \cdot, \cdot \rangle$ to denote the $\mathbf{L}^2(\Omega)$ inner product.

Observing this variational form, we notice two things. First, choosing $\mathcal{V} = \mathbf{H}^1(\Omega) := (H^1(\Omega))^3$ is not helpful, since we do not know anything about the gradient of $\mathbf{E}$, only its curl. However, we see that the volume integrals in the weak form define a bilinear form,

$$a(\mathbf{E}, \mathbf{F}) := c_1 \langle \mathbf{E}, \mathbf{F} \rangle + c_2 \langle \nabla \times \mathbf{E}, \nabla \times \mathbf{F} \rangle, \quad (10.8)$$

that is equivalent to the norm associated with the space $\mathbf{H}(\text{curl}, \Omega)$ defined in Remark 8.19 (at least for $c_1, c_2 > 0$). That is,

$$\mathbf{H}(\text{curl}, \Omega) := \{\mathbf{u} \in \mathbf{L}^2(\Omega) \mid \nabla \times \mathbf{u} \in \mathbf{L}^2(\Omega)\}, \quad (10.9a)$$

$$\|\mathbf{u}\|_{\text{curl}}^2 := \|\mathbf{u}\|^2 + \|\nabla \times \mathbf{u}\|^2. \quad (10.9b)$$

Second, the notion of essential versus natural boundary conditions is slightly different. If, for instance, $(\nabla \times \mathbf{E}) \times \mathbf{n} = 0$, the boundary integral vanishes *naturally* (using a triple-product identity). On the other hand, a condition on the Sobolev space could be explicitly *enforced* such that $\mathbf{F} \times \mathbf{n} = \mathbf{0}$, resulting in an essential boundary condition. As a result, we have homogeneous Dirichlet boundary conditions for a perfect conductor with $\mathbf{E} \times \mathbf{n} = \mathbf{0}$, leading to a compatible Hilbert space of

$$\mathcal{V} = \mathbf{H}_0(\text{curl}, \Omega) := \{\mathbf{u} \in \mathbf{H}(\text{curl}, \Omega) \mid \mathbf{u} \times \mathbf{n} = \mathbf{0} \text{ on } \partial\Omega\}. \quad (10.10)$$

A similar problem is constructed to create a variational problem in $\mathbf{H}(\text{div}, \Omega)$. Consider the PDE

$$c_1 \mathbf{B} - \nabla (c_2 \nabla \cdot \mathbf{B}) = \mathbf{f} \text{ in } \Omega \quad (10.11)$$

for some constants $c_1 \geq 0$ and $c_2 > 0$. The variational form associated with equation (10.11) is to find $\mathbf{B} \in \mathcal{V}$ such that

$$c_1 \langle \mathbf{B}, \mathbf{C} \rangle + c_2 \langle \nabla \cdot \mathbf{B}, \nabla \cdot \mathbf{C} \rangle - c_2 \int_{\partial\Omega} (\nabla \cdot \mathbf{B}) (\mathbf{C} \cdot \mathbf{n}) \, ds = \langle \mathbf{f}, \mathbf{C} \rangle \quad \forall \mathbf{C} \in \mathcal{V}, \quad (10.12)$$

where, again, integration by parts was used to get the boundary term. Here, with $a(\mathbf{B}, \mathbf{C}) := c_1 \langle \mathbf{B}, \mathbf{C} \rangle + c_2 \langle \nabla \cdot \mathbf{B}, \nabla \cdot \mathbf{C} \rangle$, it is natural to consider

$$\mathbf{H}(\operatorname{div}, \Omega) := \{\mathbf{u} \in \mathbf{L}^2(\Omega) \mid \nabla \cdot \mathbf{u} \in L^2(\Omega)\}, \quad (10.13a)$$

$$\|\mathbf{u}\|_{\operatorname{div}}^2 := \|\mathbf{u}\|^2 + \|\nabla \cdot \mathbf{u}\|^2, \quad (10.13b)$$

where we must, once again, carefully consider the boundary conditions. The essential boundary condition is to *enforce* that $\mathbf{C} \cdot \mathbf{n} = 0$. Thus, we have homogeneous Dirichlet boundary conditions associated with the normal component of the solution vector, $\mathbf{B} \cdot \mathbf{n}$. This suggests a Hilbert space of the form

$$\mathcal{V} = \mathbf{H}_0(\operatorname{div}, \Omega) := \{\mathbf{u} \in \mathbf{H}(\operatorname{div}, \Omega) \mid \mathbf{u} \cdot \mathbf{n} = 0 \text{ on } \partial\Omega\}. \quad (10.14)$$

In contrast, natural boundary conditions on this problem would require imposing the value of $\nabla \cdot \mathbf{B}$ on the boundary. As in Section 8.4, we note that the use of homogeneous boundary conditions here is only to simplify discussion, and these weak forms are easily extended to account for nonhomogeneous boundary data.

Remark 10.2: $L^2(\Omega)$ de Rham complex

The differential operators gradient, curl, and divergence have many special properties. For instance, $\nabla \times \nabla u = \mathbf{0}$ and $\nabla \cdot \nabla \times \mathbf{u} = 0$ for sufficiently smooth scalar and vector functions u and $\mathbf{u}$. Along with their associated Sobolev spaces, they form what is known as an *exact sequence* under the $L^2(\Omega)$ de Rham complex:

$$H^1(\Omega) \xrightarrow{\nabla} \mathbf{H}(\operatorname{curl}, \Omega) \xrightarrow{\nabla \times} \mathbf{H}(\operatorname{div}, \Omega) \xrightarrow{\nabla \cdot} L^2(\Omega) \longrightarrow 0. \quad (10.15)$$

Here, taking the gradient of a function in $H^1(\Omega)$ (which is allowed since it is bounded in the $L^2(\Omega)$ sense) gives a function in $\mathbf{H}(\operatorname{curl}, \Omega)$. For any function in $\mathbf{H}(\operatorname{curl}, \Omega)$, the curl yields a function in $\mathbf{H}(\operatorname{div}, \Omega)$. Finally, the divergence of a function in $\mathbf{H}(\operatorname{div}, \Omega)$ results in an $L^2(\Omega)$ function. There is an *exact sequence* of operations in this diagram, in the sense that the image of the map into a given space is equal to the kernel of the map out of that space, reflecting the identities that $\nabla \times \nabla u = \mathbf{0}$ and $\nabla \cdot \nabla \times \mathbf{u} = 0$. That these identities hold is denoted by the final $\rightarrow 0$ in the statement of the sequence.

When building finite-element subspaces for $\mathbf{H}(\operatorname{curl}, \Omega)$ and $\mathbf{H}(\operatorname{div}, \Omega)$, we seek to guarantee that the complex is satisfied for the subspaces. Discretizations that use such spaces are sometimes referred to as *structure preserving*, resulting in an exact sequence mimicking equation (10.15):

$$H_h^1(\Omega^h) \xrightarrow{\nabla^h} \mathbf{H}_h(\operatorname{curl}, \Omega^h) \xrightarrow{\nabla^h \times} \mathbf{H}_h(\operatorname{div}, \Omega^h) \xrightarrow{\nabla^h \cdot} L_h^2(\Omega^h) \longrightarrow 0, \quad (10.16)$$

where $H_h^1(\Omega^h) \subset H^1(\Omega)$, $\mathbf{H}_h(\operatorname{curl}, \Omega^h) \subset \mathbf{H}(\operatorname{curl}, \Omega)$, $\mathbf{H}_h(\operatorname{div}, \Omega^h) \subset \mathbf{H}(\operatorname{div}, \Omega)$, and $L_h^2(\Omega^h) \subset L^2(\Omega)$ are finite-dimensional subspaces and ∇^h , $\nabla^h \times$, and $\nabla^h \cdot$ are the discrete versions of gradient, curl, and divergence, respectively, defined on the computational mesh, Ω^h . If structure preserving, the latter operators satisfy similar properties as the continuous ones, such as $\nabla^h \times \nabla^h u_h = \mathbf{0}$, which is the case in the following examples. These ideas form the core of the finite-element exterior calculus (FEEC) methodology [18, 16].

For both cases outlined above, with variational problems in either $\mathbf{H}(\operatorname{curl}, \Omega)$ or $\mathbf{H}(\operatorname{div}, \Omega)$, we aim to pick finite-element subspaces of $\mathbf{H}(\operatorname{curl}, \Omega)$ and $\mathbf{H}(\operatorname{div}, \Omega)$ that allow us to leverage the general finite-element theory of Sections 8.2 and 8.3. From the boundary conditions, we see

that there are conditions on the tangential and normal components of the vector functions. To determine continuity requirements across interior elements, we consider two adjacent elements, τ_1 and τ_2 , with shared interface (face in 3D and edge in 2D) $f = \tau_1 \cap \tau_2$, and denote the outward unit normal vector with respect to τ_i on f as $\mathbf{n}_i$, so that $\mathbf{n}_1 = -\mathbf{n}_2$. Writing $\tau = \tau_1 \cup \tau_2$ with outward normal $\mathbf{n}_\tau$ on $\partial\tau$, we assume that $\mathbf{u} \in \mathbf{H}(\text{curl}, \tau)$ and ask what properties are required so that

$$\int_{\tau} \nabla \times \mathbf{u} \cdot \boldsymbol{\phi} \, d\mathbf{x} = \int_{\tau_1} \nabla \times \mathbf{u} \cdot \boldsymbol{\phi} \, d\mathbf{x} + \int_{\tau_2} \nabla \times \mathbf{u} \cdot \boldsymbol{\phi} \, d\mathbf{x} \quad (10.17)$$

for sufficiently smooth $\boldsymbol{\phi}$. Note that we assume a three-dimensional element and integrals here, but a similar argument can be made in 2D. To distinguish the value of $\mathbf{u}$ on τ_1 versus τ_2 , we define $\mathbf{u}_i = \mathbf{u}$ on τ_i . Then, we use the integration-by-parts formulas of Section 2.6.2 to arrive at

$$\int_{\tau} \nabla \times \mathbf{u} \cdot \boldsymbol{\phi} \, d\mathbf{x} = \int_{\tau} \mathbf{u} \cdot \nabla \times \boldsymbol{\phi} \, d\mathbf{x} + \int_{\partial\tau} \mathbf{u} \cdot (\boldsymbol{\phi} \times \mathbf{n}_\tau) \, ds \quad (10.18a)$$

$$= \int_{\tau_1} \nabla \times \mathbf{u}_1 \cdot \boldsymbol{\phi} \, d\mathbf{x} + \int_f \boldsymbol{\phi} \cdot (\mathbf{u}_1 \times \mathbf{n}_1) \, ds \quad (10.18b)$$

$$+ \int_{\tau_2} \nabla \times \mathbf{u}_2 \cdot \boldsymbol{\phi} \, d\mathbf{x} + \int_f \boldsymbol{\phi} \cdot (\mathbf{u}_2 \times \mathbf{n}_2) \, ds \quad (10.18c)$$

$$+ \underbrace{\int_{\partial\tau_1 \setminus f} \boldsymbol{\phi} \cdot (\mathbf{u}_1 \times \mathbf{n}_\tau) \, ds + \int_{\partial\tau_2 \setminus f} \boldsymbol{\phi} \cdot (\mathbf{u}_2 \times \mathbf{n}_\tau) \, ds + \int_{\partial\tau} \mathbf{u} \cdot (\boldsymbol{\phi} \times \mathbf{n}_\tau) \, ds}_0 \quad (10.18d)$$

$$= \int_{\tau_1} \nabla \times \mathbf{u}_1 \cdot \boldsymbol{\phi} \, d\mathbf{x} + \int_{\tau_2} \nabla \times \mathbf{u}_2 \cdot \boldsymbol{\phi} \, d\mathbf{x} \quad (10.18e)$$

$$+ \int_f \boldsymbol{\phi} \cdot (\mathbf{u}_1 \times \mathbf{n}_1 - \mathbf{u}_2 \times \mathbf{n}_1) \, ds. \quad (10.18f)$$

Here we have used the fact that $\partial\tau_1 \cup \partial\tau_2 = f \cup \partial\tau$, and the boundary integrals in equation (10.18d) cancel using the triple-product identity that $\mathbf{u} \cdot (\boldsymbol{\phi} \times \mathbf{n}_\tau) = -\boldsymbol{\phi} \cdot (\mathbf{u} \times \mathbf{n}_\tau)$. From this, we see that equation (10.17) holds if there is tangential continuity across the interface,

$$\mathbf{u}_1 \times \mathbf{n} = \mathbf{u}_2 \times \mathbf{n}, \quad (10.19)$$

where $\mathbf{n}$ is fixed as one of the unit normals to the interface, f . A similar derivation is made for $\nabla \cdot$ and $\mathbf{H}(\text{div}, \tau)$, leading to the following condition on normal continuity between elements:

$$\mathbf{n} \cdot \mathbf{u}_1 = \mathbf{n} \cdot \mathbf{u}_2. \quad (10.20)$$

To simplify the discussion, we assume that we start with a triangulation in 2D or a tetrahedral mesh in 3D, although similar finite-element spaces on quadrilateral or hexahedral meshes are also possible. These *simplicial* meshes are decomposed into their elements: faces, edges, and vertices. A global ordering of the faces, edges, and vertices is required as part of the mesh data, as well as a local ordering of those adjacent to each element. While this can be done in many different ways, as long as the ordering is consistent, we need to associate a “global” tangent vector, $\mathbf{t}_e$, with each edge and a “global” normal vector, $\mathbf{n}_f$, with each face. In the case of two dimensions, the edges and the faces are the same entities and we associate a normal with each edge as well—see Figure 10.2.1, where edge normals are selected to point from lower element index to higher element index.

As in the case for H^1 -conforming elements, we consider spaces of polynomials defined on an element, $\tau \in \Omega^h$, this time for vector-valued functions, $\mathbf{P}_k(\tau)$, with continuity requirements

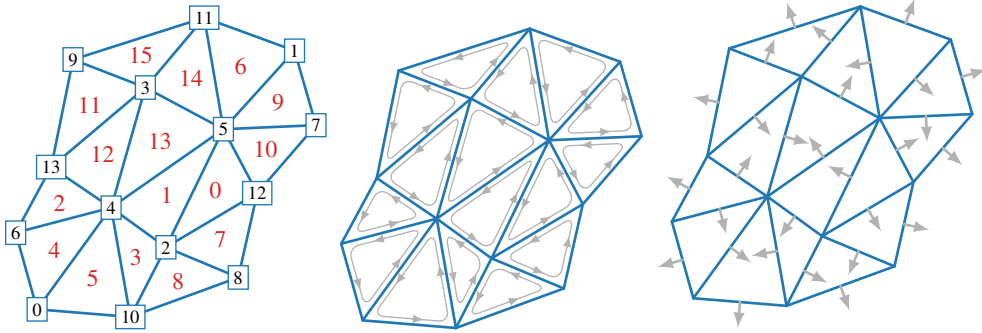


Figure 10.2.1. Example mesh orientation. (left) Mesh with 16 elements and 14 vertices; (middle) local counterclockwise orientation of the edges within each element; and (right) global normal directions for each face.

inherited from the discussion above. This leads to the Nédélec finite-element spaces for $\mathbf{H}(\text{curl}, \Omega)$ and the Raviart–Thomas and Brezzi–Douglas–Marini elements for $\mathbf{H}(\text{div}, \Omega)$ that are defined in Definitions 10.3 and 10.5 and Remarks 10.4 and 10.6.

Definition 10.3: Nédélec elements [174]

Let $\mathcal{S}_{k+1}(\tau) := \{\mathbf{p} \in \mathbf{P}_{k+1}(\tau) \mid \mathbf{p} \cdot \mathbf{x} = 0\}$, where $\mathbf{x}$ is a coordinate vector on element τ . Then, the Nédélec element of the first kind is defined by

$$\mathcal{N}_k(\tau) := \mathbf{P}_k(\tau) \oplus \mathcal{S}_{k+1}(\tau), \quad (10.21)$$

meaning that any function $\mathbf{u} \in \mathcal{N}_k(\tau)$ can be written as the sum of a function in $\mathbf{P}_k(\tau)$ and a function in $\mathcal{S}_{k+1}(\tau)$, noting that these spaces are not disjoint, so this decomposition is not unique. Equation (10.21) is supplemented by a requirement of the continuity of tangential components of the vector functions on edges of the elements, so that $\mathcal{N}_k(\Omega^h) \subset \mathbf{H}(\text{curl}, \Omega^h)$. On each element, the dimension of $\mathcal{N}_k(\tau)$ is $(k+1)(k+3)$ in two dimensions and $\frac{(k+1)(k+3)(k+4)}{2}$ in three dimensions. We also observe that, on each element, τ , the curl of a function in $\mathcal{N}_k(\tau)$ is in $\mathbf{P}_k(\tau)$ in two dimensions and $\mathbf{P}_k(\tau)$ in three dimensions.

Remark 10.4: Nédélec elements of the second kind [175]

A second family of Nédélec elements is available through a different construction, where the functions live in the full $\mathbf{P}_{k+1}(\tau)$ space. This results in an improved order of convergence for these elements in the $\mathbf{L}^2(\Omega)$ norm, being one order higher than $\mathcal{N}_k(\tau)$, and matching the expected order of $\mathbf{H}^1$ -conforming elements of the same degree. While the use of second-kind elements is becoming increasingly common, we only consider first-kind elements here.

Definition 10.5: Raviart–Thomas elements [185]

The Raviart–Thomas element is defined by

$$\mathcal{RT}_k(\tau) := \mathbf{P}_k(\tau) \oplus \mathbf{x}P_k(\tau), \quad (10.22)$$

meaning that each function $\mathbf{u} \in \mathcal{RT}_k(\tau)$ is written as the sum of a function in $\mathbf{P}_k(\tau)$ and one of

the form of a scalar function in $P_k(\tau)$ times the vector coordinate function, $\mathbf{x}$. Equation (10.22) is supplemented with the requirement of continuity of the normal components of the vector functions across faces of the elements, resulting in $\mathcal{RT}_k(\Omega^h) \subset \mathbf{H}(\text{div}, \Omega^h)$. On each element, the dimension of $\mathcal{RT}_k(\tau)$ is $(k+1)(k+3)$ in two dimensions and $\frac{(k+1)(k+2)(k+4)}{2}$ in three dimensions. Again, we observe that, on each element, τ , the divergence of a function in $\mathcal{RT}_k(\tau)$ is in $P_k(\tau)$.

Remark 10.6: BDM elements [56]

Similar to the Nédélec elements of the second kind, there is another construction of $\mathbf{H}(\text{div}, \Omega)$ -conforming elements where functions live in the full $\mathbf{P}_{k+1}(\tau)$ space. These are known as the Brezzi–Douglas–Marini (BDM) elements, which also increase the order of convergence in the $L^2(\Omega)$ norm by one order.

Remark 10.7: Order of Nédélec and Raviart–Thomas elements

There is inconsistency in the literature in the notation for the order of Nédélec and Raviart–Thomas spaces. This arises from two ways of thinking about the polynomial spaces being considered, and whether we consider what polynomials the Nédélec or Raviart–Thomas space *contains* versus which polynomial space the Nédélec or Raviart–Thomas space is *contained in*, leading to two types of indexing. One family of notation (as used here) states that the Nédélec space $\mathcal{N}_k(\tau)$ should *contain* the polynomial space $\mathbf{P}_k(\tau)$ but not $\mathbf{P}_{k+1}(\tau)$. This leads to the notation where the lowest-order space is $\mathcal{N}_0(\tau)$. The second family uses the opposite convention, wherein $\mathcal{N}_k(\tau)$ is *contained in* the polynomial space $\mathbf{P}_k(\tau)$ but not in the lower-order space $\mathbf{P}_{k-1}(\tau)$. In this notation, the lowest-order space is $\mathcal{N}_1(\tau)$. The same shift in indexing occurs for Raviart–Thomas spaces. Neither convention is inherently better than the other—they are both equally reasonable. However, it is important to be conscious of the existence of these two common notations and to be aware of which one is being used in any reference!

Next, we take a closer look at the construction of the elements above and their degrees of freedom. For simplicity, we only consider the lowest-order elements, which are a subset of linear polynomials on each element. However, first recall the continuous-conforming piecewise linear space, $P_1(\Omega^h)$, used as a finite-element subspace of $H^1(\Omega)$, referred to as linear Lagrange elements. In Example 8.32, a standard canonical linear nodal basis is constructed, and we define these here as ϕ_i^{grad} , where i represents each vertex on an element. The degrees of freedom, χ_i^{grad} , are then defined as function evaluations at the nodes. For scalar function $u \in H^1(\Omega^h)$, $\chi_i^{\text{grad}}(u) = u(x_i)$. Moreover, we define the interpolant of $u \in H^1(\Omega^h)$ simply as

$$(I^h u)(x) = \sum_i \chi_i^{\text{grad}}(u) \phi_i^{\text{grad}}(x). \quad (10.23)$$

An illustration of the degrees of freedom on a typical tetrahedron in three dimensions is shown in the top diagram of Figure 10.2.2. We now construct basis sets for both $\mathbf{H}(\text{curl}, \Omega)$ -conforming and $\mathbf{H}(\text{div}, \Omega)$ -conforming finite elements building from this formalism.

Lowest-order Nédélec elements ($\mathcal{N}_0(\Omega^h)$)

The lowest-order Nédélec elements (of the first kind) or edge elements, $\mathcal{N}_0(\Omega^h)$, comprise linear piecewise polynomials that have continuous tangential components across element interfaces, represented on the edges. Based on Definition 10.3, a vector function in $\mathcal{N}_0(\tau)$ takes the form

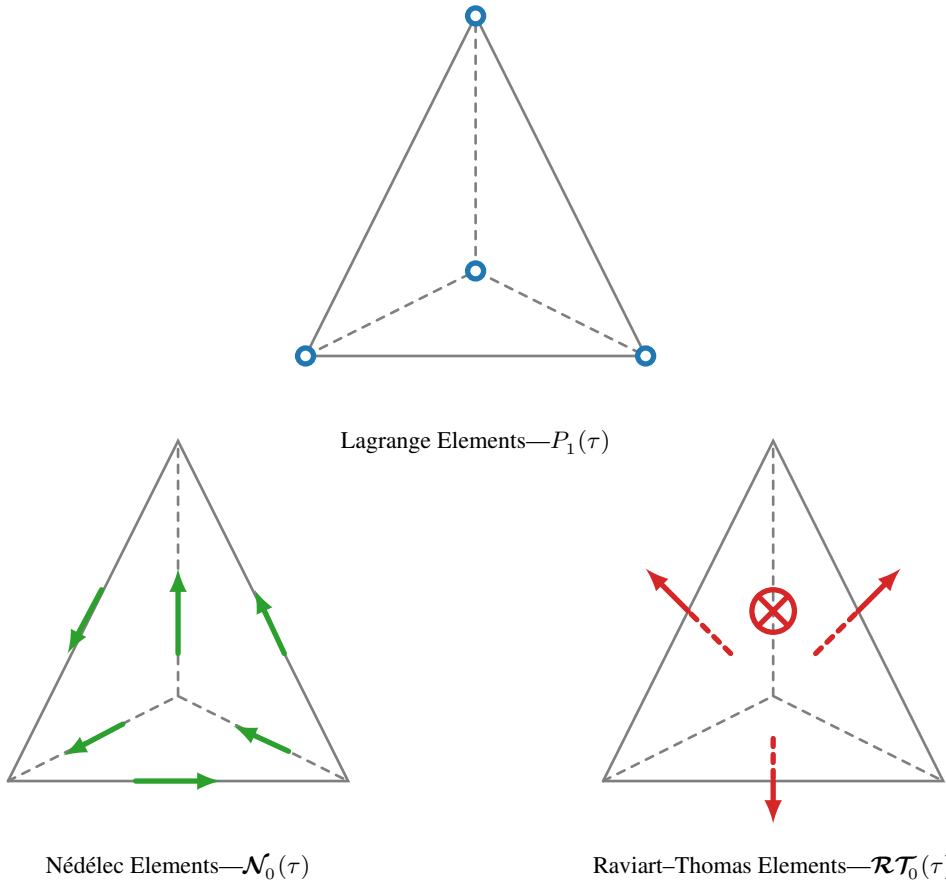


Figure 10.2.2. Degrees of freedom on a 3D tetrahedral element, τ , for the (top) piecewise linear Lagrange elements (in blue), $P_1(\tau)$; (lower left) lowest-order Nédélec elements (in green), $\mathcal{N}_0(\tau)$; and (lower right) lowest-order Raviart–Thomas elements (in red), $\mathcal{RT}_0(\tau)$. Arrows indicate components of a vector (either tangential on edges or normal on faces), and circles indicate function evaluations at nodes.

$\mathbf{v}(\mathbf{x}) = \mathbf{a}_\tau + \mathbf{b}_\tau \times \mathbf{x}$, where $\mathbf{a}_\tau$ and $\mathbf{b}_\tau$ are constant vectors in $\mathbb{R}^3$. In 3D, this gives six degrees of freedom to the element, which are represented by one degree of freedom for each edge on the element. There is a natural embedding to create similar elements in 2D, where we use a vector of length two for $\mathbf{a}_\tau$ and a scalar for $\mathbf{b}_\tau$, resulting in three degrees of freedom on the element, which are again represented by one degree of freedom for each edge on the element. These degrees of freedom can be given, for example, by defining the degrees of freedom for the space, χ_ℓ^{curl} , to be line integrals over edges. For vector function $\mathbf{u} \in \mathbf{H}(\text{curl}, \Omega^h)$,

$$\chi_\ell^{\text{curl}}(\mathbf{u}) = \frac{1}{|e_\ell|} \int_{e_\ell} \mathbf{u} \cdot \mathbf{t}_\ell ds, \quad (10.24)$$

where $|e_\ell|$ is the length of edge ℓ , and $\mathbf{t}_\ell$ is the tangent vector to that edge. The lower-left diagram of Figure 10.2.2 shows the corresponding degrees of freedom on a tetrahedron.

The basis functions for each degree of freedom are constructed directly from the linear Lagrange basis functions defined at the vertices of the element. To start, we let vertices i and j be the endpoints of edge e_ℓ , with the requirement that the global numbering of j be higher than that for i (this choice of ordering is made for consistency reasons only). Then, to satisfy the

constraints and also give the correct orientation, the appropriate basis functions for $\mathcal{N}_0(\Omega^h)$ are given by

$$\Psi_\ell^{\text{curl}} = |e_\ell| \left(\phi_i^{\text{grad}} \nabla \phi_j^{\text{grad}} - \phi_j^{\text{grad}} \nabla \phi_i^{\text{grad}} \right). \quad (10.25)$$

Here we have scaled the degrees of freedom and the basis functions by the lengths of the edges. This is arbitrary as long as $\chi_\ell^{\text{curl}}(\Psi_m^{\text{curl}}) = \delta_{\ell m}$. This specific choice of scaling does simplify the computation when using a midpoint rule to approximate the line integral, however.

Similar to Section 8.3 for $H^1(\Omega)$ -conforming elements, it can be shown that lowest-order Nédélec elements converge with $\mathcal{O}(h)$, where h is the size of the mesh, as expected for the case of linear polynomials. To verify this, we perform a series of experiments with manufactured solutions of equation (10.6) with $c_1 = c_2 = 1$ on $\Omega = [0, 1]^3$. The exact solution is defined as

$$\begin{aligned} \mathbf{E} &= \begin{bmatrix} \cos(\pi x) \sin(\pi y) \sin(\pi z) \\ \sin(\pi x) \cos(\pi y) \sin(\pi z) \\ -\sin(\pi x) \sin(\pi y) \cos(\pi z) \end{bmatrix} \\ \Rightarrow -\mathbf{J} &= \begin{bmatrix} (1 + 2\pi^2) \cos(\pi x) \sin(\pi y) \sin(\pi z) \\ (1 + 2\pi^2) \sin(\pi x) \cos(\pi y) \sin(\pi z) \\ -(1 + 4\pi^2) \sin(\pi x) \sin(\pi y) \cos(\pi z) \end{bmatrix}. \end{aligned} \quad (10.26)$$

We solve this problem on an $n_x \times n_y \times n_z$ tensor-product mesh with $n_x = n_y = n_z$, comprising $n_x + 1$ vertices in each coordinate direction. The cubes in the resulting hexahedral mesh are then regularly subdivided into six tetrahedral elements for each cube. We refer to this as a uniform tetrahedral mesh and note that $h = \mathcal{O}(n_x^{-1})$ with this mesh.

In general, using $\mathcal{N}_k(\Omega^h)$ elements yields convergence orders of $\mathcal{O}(h^{k+1})$ in both the $\mathbf{L}^2(\Omega)$ and $\mathbf{H}(\text{curl}, \Omega)$ norms. This is confirmed in Figure 10.2.3 for the lowest-order Nédélec case described above, as well as for higher-order spaces with $k = 1$ or 2. For these spaces, we introduce additional edge, face, and interior degrees of freedom as the order increases; see [174] for the construction.

Lowest-order Raviart–Thomas elements ($\mathcal{RT}_0(\Omega^h)$)

The lowest-order Raviart–Thomas elements or face elements, $\mathcal{RT}_0(\Omega^h)$, comprise linear piecewise polynomials that have continuous normal components across element faces. Based on Definition 10.5, a vector function in $\mathcal{RT}_0(\tau)$ takes the form $\mathbf{v}(\mathbf{x}) = \mathbf{a} + b\mathbf{x}$. With this, each face corresponds to a degree of freedom given by χ_ℓ^{div} , in either two or three dimensions. The degrees of freedom can be chosen to be normal flux integrals over faces, meaning that for vector function $\mathbf{u} \in \mathbf{H}(\text{div}, \Omega^h)$,

$$\chi_\ell^{\text{div}}(\mathbf{u}) = \frac{1}{|f_\ell|} \int_{f_\ell} \mathbf{u} \cdot \mathbf{n}_\ell \, ds, \quad (10.27)$$

where $|f_\ell|$ is the area of the face (length of edge in 2D) and $\mathbf{n}_\ell$ is the normal vector to each face. For consistency with the global ordering, a normal vector is chosen using the Lagrange basis functions as follows. Let i denote the index of the vertex opposite the corresponding face, f_ℓ . Then,

$$\mathbf{n}_\ell = \pm \frac{\nabla \phi_i^{\text{grad}}}{|\nabla \phi_i^{\text{grad}}|}, \quad (10.28)$$

where the sign is chosen based on the global numbering of elements (e.g., so that the normal points from the lower-numbered element to the higher-numbered element). The lower-right diagram of Figure 10.2.2 shows the corresponding degrees of freedom on a tetrahedron.

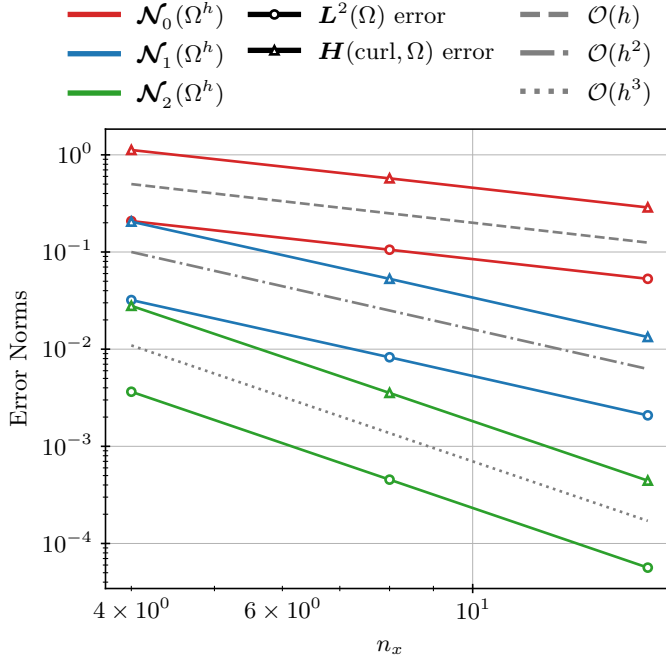


Figure 10.2.3. Error norms in $L^2(\Omega)$ and $H(\text{curl}, \Omega)$ versus mesh size for a simulation of the curl-curl problem defined in equation (10.7) using $\mathcal{N}_0(\Omega^h)$, $\mathcal{N}_1(\Omega^h)$, and $\mathcal{N}_2(\Omega^h)$. Here, $n_x + 1$ is the number of vertices of the tetrahedral mesh in each direction, and $h = \mathcal{O}(n_x^{-1})$.

The construction of basis functions differs for dimension but follows similar ideas as for the Nédélec elements. In two dimensions, a face is an edge, so again assume that i and j are the corresponding vertices of edge, e , such that the global numbering of j is higher than for i . The appropriate basis functions are then

$$\Psi_\ell^{\text{div}} = |e_\ell| \left(\phi_i^{\text{grad}} \nabla^\perp \phi_j^{\text{grad}} - \phi_j^{\text{grad}} \nabla^\perp \phi_i^{\text{grad}} \right), \quad (10.29)$$

where $\nabla^\perp = \begin{bmatrix} \partial_y \\ -\partial_x \end{bmatrix}$. This is simply a rotation of the Nédélec elements in 2D, which matches our intuition since the normal component is a rotation of the tangential component.

In three dimensions, assume that i , j , and k are the corresponding vertices of the current face, f_ℓ , with global numbering ordered from lowest to highest as i goes to k . Then, the basis functions are determined by

$$\begin{aligned} \Psi_\ell^{\text{div}} = 2|f_\ell| \bigg(& \phi_i^{\text{grad}} \left(\nabla \phi_j^{\text{grad}} \times \nabla \phi_k^{\text{grad}} \right) \\ & + \phi_j^{\text{grad}} \left(\nabla \phi_k^{\text{grad}} \times \nabla \phi_i^{\text{grad}} \right) + \phi_k^{\text{grad}} \left(\nabla \phi_i^{\text{grad}} \times \nabla \phi_j^{\text{grad}} \right) \bigg). \end{aligned} \quad (10.30)$$

As before, the scaling with the face area is done for consistency to ensure that $\chi_\ell^{\text{div}}(\Psi_m^{\text{div}}) = \delta_{\ell m}$.

Similar error analysis establishes that lowest-order Raviart–Thomas elements converge with $\mathcal{O}(h)$. To verify this, we again perform a series of experiments with manufactured solutions of equation (10.11) with $c_1 = c_2 = 1$ on a uniform tetrahedral mesh defined on $\Omega = [0, 1]^3$

(see above for description of the mesh). The exact solution is defined as

$$\begin{aligned} \mathbf{B} &= \begin{bmatrix} \sin(\pi x) \cos(\pi y) \cos(\pi z) \\ \cos(\pi x) \sin(\pi y) \cos(\pi z) \\ \cos(\pi x) \cos(\pi y) \sin(\pi z) \end{bmatrix} \\ \Rightarrow \mathbf{f} &= \begin{bmatrix} (1 + 3\pi^2) \sin(\pi x) \cos(\pi y) \cos(\pi z) \\ (1 + 3\pi^2) \cos(\pi x) \sin(\pi y) \cos(\pi z) \\ (1 + 3\pi^2) \cos(\pi x) \cos(\pi y) \sin(\pi z) \end{bmatrix}. \end{aligned} \quad (10.31)$$

As in the case of Nédélec elements, using $\mathcal{RT}_k(\Omega^h)$ elements yields convergence orders of $\mathcal{O}(h^{k+1})$ in both the $L^2(\Omega)$ and $\mathbf{H}(\text{div}, \Omega)$ norms. This is confirmed in Figure 10.2.4 for the lowest-order Raviart–Thomas case described above, as well as for higher-order spaces with $k = 1$ or 2.

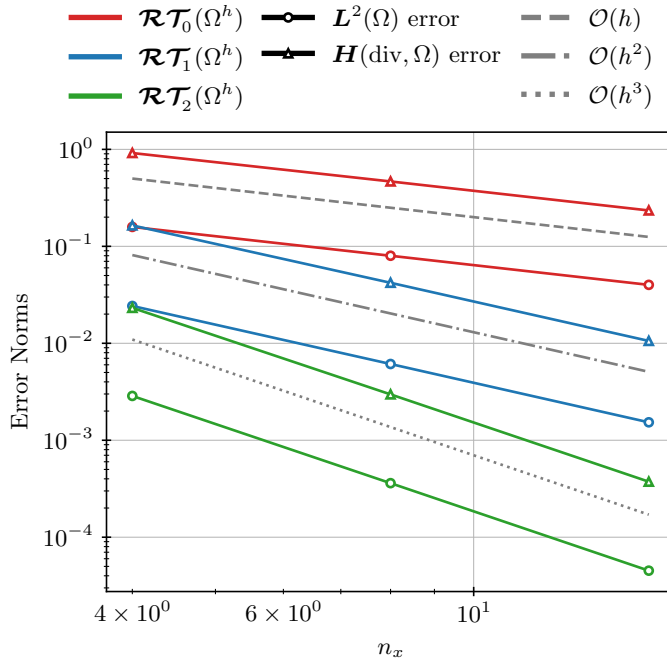


Figure 10.2.4. Error norms in $L^2(\Omega)$ and $\mathbf{H}(\text{div}, \Omega)$ versus mesh size for a simulation of the problem defined in equation (10.12) using $\mathcal{RT}_0(\Omega^h)$, $\mathcal{RT}_1(\Omega^h)$, and $\mathcal{RT}_2(\Omega^h)$. Here, $n_x + 1$ is the number of vertices of the tetrahedral mesh in each direction, and $h = \mathcal{O}(n_x^{-1})$.

Discrete properties

Finally, we end this subsection by highlighting the structure-preserving properties of the above spaces. Recall that we want certain mimetic properties to be satisfied, such as $\nabla^h \times \nabla^h u^h = \mathbf{0}$ and $\nabla^h \cdot \nabla^h \times \mathbf{u}^h = 0$. To start, using the notation from above, let $\{\phi_i^{\text{grad}}\}$, $\{\Psi_i^{\text{curl}}\}$, $\{\Psi_i^{\text{div}}\}$, and $\{\phi_i^{L^2}\}$ be the basis sets of $H_h^1(\Omega^h)$, $\mathbf{H}_h(\text{curl}, \Omega^h)$, $\mathbf{H}_h(\text{div}, \Omega^h)$, and $L_h^2(\Omega^h)$, respectively, with $\{\chi_i^{\text{grad}}\}$, $\{\chi_i^{\text{curl}}\}$, $\{\chi_i^{\text{div}}\}$, and $\{\chi_i^{L^2}\}$ defining the corresponding degrees of freedom, all of which are linear functionals on the appropriate continuum space. Then, we define the incidence matrices G , K , and D to represent the discrete operators ∇^h , $\nabla^h \times$, and $\nabla^h \cdot$ on the given simplicial mesh.

Using the de Rham complex—recall equation (10.15)—we see that the gradient operator should take a function in $H^1(\Omega)$ to a vector in $\mathbf{H}(\text{curl}, \Omega)$, which can then be projected to $\mathbf{H}_h(\text{curl}, \Omega^h)$ and represented via the degrees of freedom for $\mathbf{H}_h(\text{curl}, \Omega^h)$. The same can be accomplished for the curl and divergence operators with the various spaces in the exact sequence. Thus, we can define the incidence matrices as

$$G_{i,j} := \chi_i^{\text{curl}}(\nabla \phi_j^{\text{grad}}), \quad K_{i,j} := \chi_i^{\text{div}}(\nabla \times \Psi_j^{\text{curl}}), \quad D_{i,j} := \chi_i^{L^2}(\nabla \cdot \Psi_j^{\text{div}}). \quad (10.32)$$

With these operators, we inherit discrete properties for the Nédélec and Raviart–Thomas discretizations as summarized in Lemmas 10.8 and 10.9. Further details are found in [18, 16].

Lemma 10.8: Discrete curl grad [3]

Using Nédélec elements for $\mathbf{H}_h(\text{curl}, \Omega^h)$ and Lagrange elements for $H_h^1(\Omega^h)$ yields

$$KG = 0. \quad (10.33)$$

This corresponds to the discrete operation $\nabla^h \times \nabla^h u^h = \mathbf{0}$ for any function $u^h \in H_h^1(\Omega^h)$.

Proof. Due to the de Rham complex, equation (10.15), for $\phi_j^{\text{grad}} \in H_h^1(\Omega^h)$, we have $\nabla \phi_j^{\text{grad}} = \sum_{\ell} \chi_{\ell}^{\text{curl}}(\nabla \phi_j^{\text{grad}}) \Psi_{\ell}^{\text{curl}} \in \mathbf{H}_h(\text{curl}, \Omega^h)$. Thus,

$$(KG)_{i,j} = \sum_{\ell} K_{i,\ell} G_{\ell,j} \quad (10.34a)$$

$$= \sum_{\ell} \chi_i^{\text{div}}(\nabla \times \Psi_{\ell}^{\text{curl}}) G_{\ell,j} \quad (10.34b)$$

$$= \chi_i^{\text{div}} \left(\nabla \times \sum_{\ell} G_{\ell,j} \Psi_{\ell}^{\text{curl}} \right) \quad (10.34c)$$

$$= \chi_i^{\text{div}} \left(\nabla \times \sum_{\ell} \chi_{\ell}^{\text{curl}}(\nabla \phi_j^{\text{grad}}) \Psi_{\ell}^{\text{curl}} \right) \quad (10.34d)$$

$$= \chi_i^{\text{div}}(\nabla \times \nabla \phi_j^{\text{grad}}) \quad (10.34e)$$

$$= 0, \quad (10.34f)$$

since $\nabla \times \nabla u = \mathbf{0}$ for any smooth enough function u . Similarly, $G^T K^T = 0$, which is the discrete representation of the dual problem, namely, $\nabla \cdot \nabla \times \mathbf{u} = 0$ for any sufficiently smooth function $\mathbf{u}$. $\square$

Lemma 10.9: Discrete div curl [3]

Using Nédélec elements for $\mathbf{H}_h(\text{curl}, \Omega^h)$ and Raviart–Thomas elements for $\mathbf{H}_h(\text{div}, \Omega^h)$ yields

$$DK = 0. \quad (10.35)$$

This corresponds to the discrete operation $\nabla^h \cdot \nabla^h \times \mathbf{u}^h = 0$ for any function $\mathbf{u}^h \in \mathbf{H}_h(\text{curl}, \Omega^h)$.

Proof. Due to the de Rham complex, equation (10.15), for $\Psi_j^{\text{curl}} \in \mathbf{H}_h(\text{curl}, \Omega^h)$, we have $\nabla \times \Psi_j^{\text{curl}} = \sum_{\ell} \chi_{\ell}^{\text{div}} (\nabla \times \Psi_j^{\text{curl}}) \Psi_{\ell}^{\text{div}} \in \mathbf{H}_h(\text{div}, \Omega^h)$. Thus,

$$(DK)_{i,j} = \sum_{\ell} D_{i,\ell} K_{\ell,j} \quad (10.36a)$$

$$= \sum_{\ell} \chi_i^{L^2} (\nabla \cdot \Psi_{\ell}^{\text{div}}) K_{\ell,j} \quad (10.36b)$$

$$= \chi_i^{L^2} \left(\nabla \cdot \sum_{\ell} K_{\ell,j} \Psi_{\ell}^{\text{div}} \right) \quad (10.36c)$$

$$= \chi_i^{L^2} \left(\nabla \cdot \sum_{\ell} \chi_{\ell}^{\text{div}} (\nabla \times \Psi_j^{\text{curl}}) \Psi_{\ell}^{\text{div}} \right) \quad (10.36d)$$

$$= \chi_i^{L^2} (\nabla \cdot \nabla \times \Psi_j^{\text{curl}}) \quad (10.36e)$$

$$= 0, \quad (10.36f)$$

since $\nabla \cdot \nabla \times \mathbf{u}$ is 0 for any sufficiently smooth function $\mathbf{u}$. Similarly, $K^T D^T = 0$, which is the discrete version of the *dual* problem, namely, $\nabla \times \nabla u = 0$ for any smooth enough function u . $\square$

10.2.2 ■ H^2 -conforming discretizations

So far, we have only considered PDEs whose variational forms contain up to first derivatives. For these problems, the finite-element spaces that we consider need only be subsets of $H^1(\Omega)$ (or $\mathbf{H}(\text{div}, \Omega)$ or $\mathbf{H}(\text{curl}, \Omega)$), leading to approximations that are in $C^0(\Omega)$ (or have even less continuity). These spaces generally have basis functions that are continuous (at least partially), so that their (weak) derivatives exist, but they may have discontinuous derivatives across element boundaries. We next consider cases where higher derivatives appear in the variational forms, requiring greater regularity of the finite-element space than we have considered so far. We focus on the case where the PDE contains fourth-order derivatives, with the resulting bilinear forms requiring the approximation of second-order derivatives. Thus, we need to consider $H^2(\Omega)$ -conforming finite elements and discrete approximations that are in $C^1(\Omega)$. We motivate these elements with a PDE called the *biharmonic equation*.

The biharmonic equation

On a bounded, connected domain, $\Omega \in \mathbb{R}^d$, the biharmonic equation is a fourth-order PDE that appears in many applications of continuum mechanics, especially in modeling the elastic behavior of thin structures. It involves two actions of the Laplace operator:

$$\nabla^4 u := \nabla^2 \nabla^2 u = \nabla \cdot \nabla (\nabla \cdot \nabla u) = f. \quad (10.37)$$

We construct the variational form associated with equation (10.37) but would prefer to reduce the order of the derivatives acting on the solution. Thus, we perform two steps of integration by parts, and the weak problem statement becomes finding $u \in \mathcal{V}$ such that

$$\langle \nabla^2 u, \nabla^2 v \rangle + \int_{\partial\Omega} (\nabla(\nabla^2 u) \cdot \mathbf{n}) v \, ds - \int_{\partial\Omega} (\nabla^2 u) (\nabla v \cdot \mathbf{n}) \, ds = \langle f, v \rangle \quad \forall v \in \mathcal{V}, \quad (10.38)$$

where $\mathbf{n}$ is the outward unit normal vector to $\partial\Omega$.

Noting that the term $\langle \nabla^2 u, \nabla^2 v \rangle$ appears in equation (10.38), we need to choose the space $\mathcal{V}$ such that this term is guaranteed to be finite. A natural choice is to take $\mathcal{V} \subset H^2(\Omega)$, where the natural norm on functions in $H^2(\Omega)$ (defined in Definition 8.17) is

$$\|u\|_2^2 = \|u\|_0^2 + \|\nabla u\|_0^2 + \|\nabla \nabla u\|_0^2, \quad (10.39)$$

where the final term is the norm of the *Hessian* of u , the matrix of its second derivatives. Moreover, since two boundary integrals appear in the integration by parts, we require two boundary conditions on any segment of $\partial\Omega$. From the boundary integrals, we have two independent choices of essential (Dirichlet) versus natural (Neumann) boundary conditions:

$$\textbf{Essential:} \quad u = q_D^{(0)}, \quad \frac{\partial u}{\partial \mathbf{n}} = \nabla u \cdot \mathbf{n} = q_D^{(1)}; \quad (10.40a)$$

$$\textbf{Natural:} \quad \frac{\partial \nabla^2 u}{\partial \mathbf{n}} = \nabla(\nabla^2 u) \cdot \mathbf{n} = q_N^{(3)}, \quad \nabla^2 u = q_N^{(2)}. \quad (10.40b)$$

The subscripts denote whether the boundary condition is imposed strongly (as a Dirichlet condition) or weakly (as a Neumann condition), while the superscripts match the order of derivative on which the condition is imposed. We also note that determining boundary conditions for the bi-harmonic problem requires two partitions of the boundary into Dirichlet and Neumann segments, one for the first boundary integral (boundary conditions in the first column of equation (10.40)) and one for the second boundary integral (boundary conditions in the second column). When using only Dirichlet conditions for both integrals, we define

$$\mathcal{V}_0 = \left\{ v \in \mathcal{V} \mid v = 0 \text{ and } \frac{\partial v}{\partial \mathbf{n}} = 0 \right\}. \quad (10.41)$$

Proving $H^2(\Omega)$ -ellipticity of equation (10.38) is not necessarily trivial and depends on the type of boundary conditions used [52, Sec. 5.9]. In fact, for modeling thin plates, the weak form is often modified to ensure coercivity. Additionally, implementation issues arise when enforcing the derivative conditions, and Nitsche penalty methods are sometimes employed to enforce the boundary conditions instead [178, 142]. Thus, for simplicity, we consider Neumann boundary conditions, enforced weakly, and add a diffusion and reaction term to avoid any nullspaces in the linear system and to ensure $H^2(\Omega)$ -ellipticity. With this, the third-order natural boundary condition in equation (10.40) becomes $\frac{\partial(\Delta u - u)}{\partial \mathbf{n}} = q_N^{(3)}$, but the second-order natural boundary condition remains the same. In the end, the weak formulation considered is to find $u \in \mathcal{V}$ such that

$$\begin{aligned} \langle \nabla^2 u, \nabla^2 v \rangle + \langle \nabla u, \nabla v \rangle + \langle u, v \rangle &= \langle f, v \rangle \\ &- \int_{\partial\Omega} q_N^{(2)} v \, ds + \int_{\partial\Omega} q_N^{(3)} (\nabla v \cdot \mathbf{n}) \, ds \quad \forall v \in \mathcal{V}. \end{aligned} \quad (10.42)$$

We must now find a finite-element space $\mathcal{V}^h \subset \mathcal{V}$ that is a subset of $H^2(\Omega)$. Thus, it is not enough to consider a piecewise polynomial space, such as the Lagrange elements, which guarantee that functions are in $C^0(\Omega)$. Indeed, we need all functions in this space to be in $C^1(\Omega)$. A natural choice is to consider the idea of Hermite interpolation. Constructing piecewise polynomials based on Hermite polynomials allows for functions that are continuous across elements and have continuous derivatives. An example is given in Example 10.10.

Example 10.10: Cubic Hermite finite elements in 1D

In order to construct a cubic Hermite polynomial interpolant in 1D, four pieces of data are required: the function value at two points and the derivative value at two points, leading to a degree-three polynomial. For example, on the unit interval, $\hat{\tau} = [0, 1]$, the degrees of freedom associated with this interval are the function values at $x = 0$ and 1 and the derivative values at $x = 0$ and 1. Then, we define four basis functions that correspond to these four values. One example of such a basis is

$$\phi_1(x) = 1 - 3x^2 + 2x^3, \quad (10.43a)$$

$$\phi_2(x) = x - 2x^2 + x^3, \quad (10.43b)$$

$$\phi_3(x) = 3x^2 - 2x^3, \quad (10.43c)$$

$$\phi_4(x) = x^3 - x^2. \quad (10.43d)$$

Now, if we view $\hat{\tau}$ as the *reference* element and define appropriate maps from the reference interval to any element, τ , on a one-dimensional computational domain, Ω^h , we can define the cubic Hermite finite-element space as

$$\mathcal{V}^h = \{v \in C^1(\Omega^h) \mid \forall \tau \in \Omega^h, v(x) \text{ is a cubic polynomial on } \tau\}. \quad (10.44)$$

Cubic Hermite elements (Bogner–Fox–Schmit [45])

To extend Example 10.10 to higher dimensions, we take a tensor product of the 1D Hermite polynomials to define basis functions over a quadrilateral or hexahedral mesh. For example, in 2D, we define the Q_k elements on a quadrilateral, τ , as in Section 8.3.2,

$$Q_k(\tau) = \left\{ p : \tau \rightarrow \mathbb{R} \left| p(x, y) = \sum_{0 \leq i, j \leq k} \alpha_{i,j} x^i y^j \right. \right\}, \quad (10.45)$$

with a similar definition in 3D. Choosing tensor-product polynomials with $C^1(\Omega)$ continuity results in the so-called *Bogner–Fox–Schmit* elements on quadrilateral or hexahedral meshes, as presented in detail below.

Given a quadrilateral or hexahedral mesh $\mathcal{T}^h$ of a polygonal domain Ω , defining computational domain Ω^h , the cubic Hermite (or Bogner–Fox–Schmit) finite element is defined as

$$V^h(\Omega^h) = \{v \in C^1(\Omega) \mid v(\mathbf{x}) \in Q_3(\tau), \forall \tau \in \mathcal{T}^h\}. \quad (10.46)$$

On each element, there are 16 degrees of freedom in 2D and 64 degrees of freedom in 3D, corresponding to the tensor product of the 1D basis functions defined above. On the reference element, these basis functions are written as

$$\Psi_{i,j}(x, y) = \phi_i(x)\phi_j(y), \quad 1 \leq i, j \leq 4, \quad \text{for } \hat{\tau} = [0, 1]^2, \quad (10.47a)$$

$$\Psi_{i,j,k}(x, y, z) = \phi_i(x)\phi_j(y)\phi_k(z), \quad 1 \leq i, j, k \leq 4, \quad \text{for } \hat{\tau} = [0, 1]^3. \quad (10.47b)$$

In 2D, the degrees of freedom consist of four values at each vertex of an element, corresponding to the function value, u ; first derivatives in each coordinate direction (namely, u_x and u_y); and the mixed second derivative value, u_{xy} . In 3D, this becomes eight degrees of freedom at each

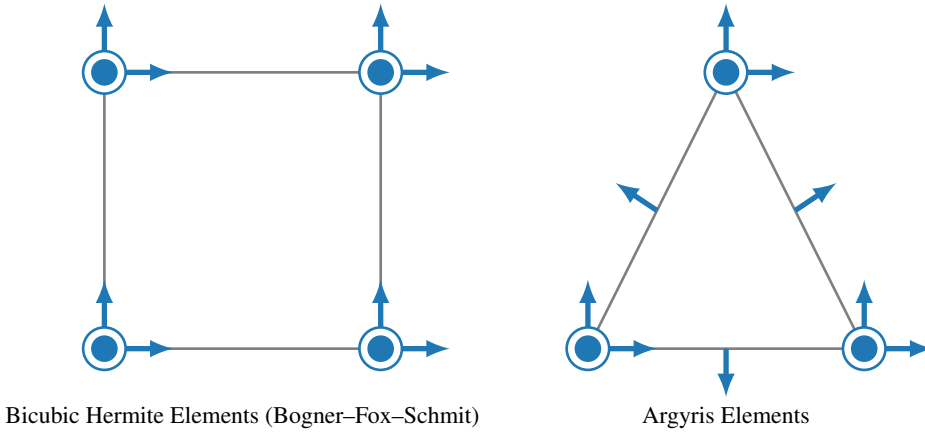


Figure 10.2.5. Degrees of freedom (in blue) for (left) a 2D cubic Hermite (Bogner–Fox–Schmit) element and (right) a 2D Argyris element. Filled-in circles represent function evaluations, arrows represent first-order derivative evaluations in a particular direction, and open circles represent second-order derivative evaluations.

vertex: the values of u , u_x , u_y , u_z , u_{xy} , u_{xz} , u_{yz} , and u_{xyz} . These are depicted for the 2D reference rectangle on the left side of Figure 10.2.5.

As with the other elements described above, but now assuming additional smoothness of the solution, that is, $u \in H^4(\Omega)$, error analysis provides bounds by defining an appropriate interpolation operator. Using cubic Hermite elements results in $\mathcal{O}(h^{4-m})$ convergence order for the error in the $H^m(\Omega)$ norm. This is confirmed in Figure 10.2.6 by a simple numerical experiment using the Bogner–Fox–Schmit element to solve a two-dimensional test problem of equation (10.42) with a manufactured solution, $u(x, y) = 100(x(1-x)y(1-y))^2$.

The quintic Argyris element in 2D [13]

The cubic Hermite element described above uses a cubic polynomial and 16 degrees of freedom (in 2D) to obtain a $C^1(\Omega)$ approximation and an $H^2(\Omega)$ -conforming finite element, and higher-order extensions to this element do exist. While the tensor-product construction provides a natural extension from the 1D case, similar Hermite elements can be defined on triangles (or on tetrahedra in 3D). In this case, we again have degrees of freedom corresponding to function values at each vertex of the simplex, as well as each of the first partial derivatives at each vertex. To account for the mixed derivative, we also need a degree of freedom at the *barycenter* of each triangle (in 3D, this requires four additional degrees of freedom, one at the barycenter of each triangular face of the tetrahedron). The difficulty in working with this element comes from the fact that it does not define an affine family of elements, as the partial derivatives on the reference element are mapped to tangential derivatives on the physical triangle. Thus, the construction of cubic Hermite elements on triangles is not straightforward. Instead, several other types of elements have been introduced that yield $C^1(\Omega)$ approximations on triangles or tetrahedra without the difficulties of the Hermite construction. This includes the Morley, Bell, and Argyris elements. We end this section with an example of the Argyris element.

Given a triangulation of a domain, $\Omega^h \subset \mathbb{R}^2$, with elements τ , the Argyris element uses fifth-order piecewise polynomial basis functions to approximate $C^1(\Omega^h)$ functions. This requires 21 degrees of freedom on each element, including the function value at all three vertices, the first-order partial derivatives in each coordinate direction at each vertex, all three second-order partial

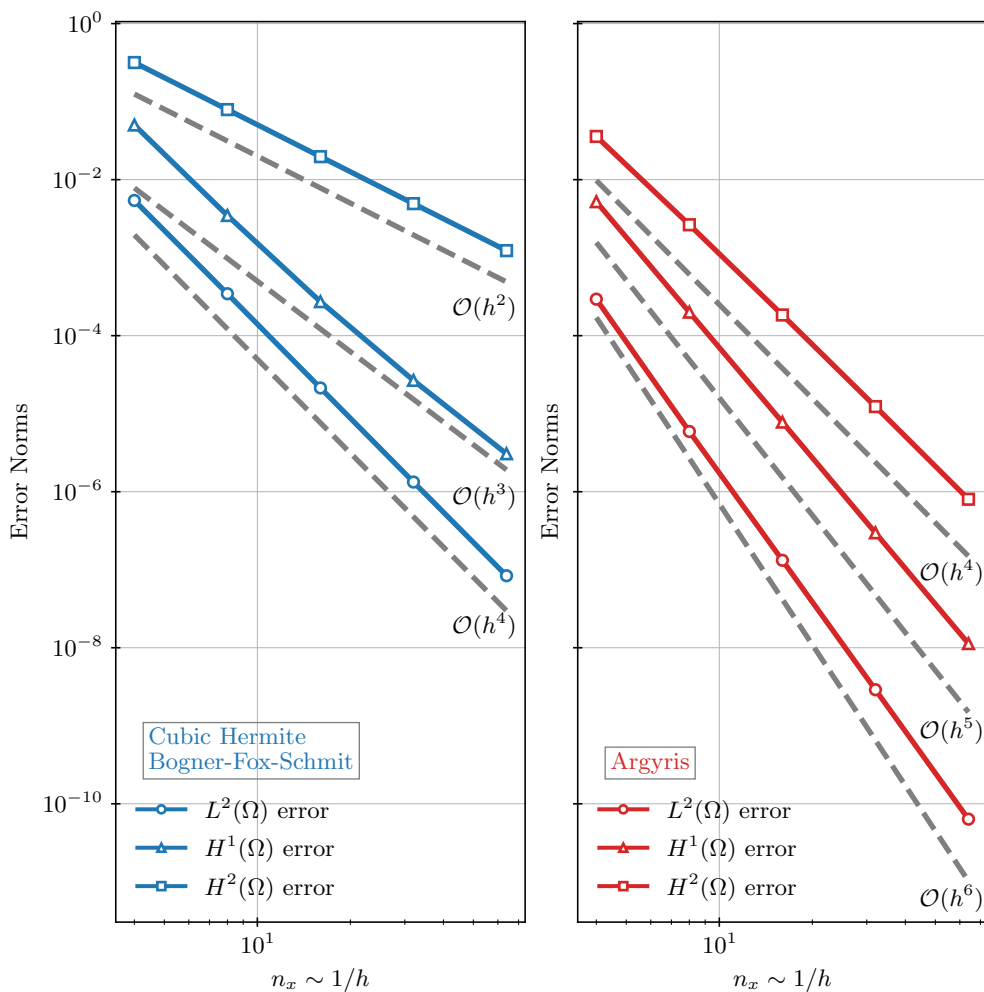


Figure 10.2.6. Error norms in $L^2(\Omega)$, $H^1(\Omega)$, and $H^2(\Omega)$ versus mesh size for a simulation of the biharmonic problem defined in equation (10.42) using the (left) cubic Hermite (or Bogner–Fox–Schmit) element and the (right) Argyris element in 2D. Here, we consider a uniform $n_x \times n_y$ mesh of the unit square with $n_x = n_y$. For the Argyris element, each square is divided into two right-triangular elements. In both cases, $h = \mathcal{O}(n_x^{-1})$.

derivatives at each vertex, and the normal component of the gradient on each face of the triangle. In total, this yields an approximation that is $C^1(\Omega^h)$ across elements yet has continuous second partial derivatives at the vertices. Basis functions are generally chosen in the usual way, such that each basis function maintains unit value for exactly one degree of freedom and gives zero for the others. A graphical representation of the degrees of freedom is depicted in the right side of Figure 10.2.5.

Again assuming enough smoothness in the solution, an error analysis yields $\mathcal{O}(h^{6-m})$ convergence order for the error in the $H^m(\Omega)$ norm, since we are now approximating our solutions with fifth-order polynomials. This is similarly confirmed in Figure 10.2.6 by a simple numerical experiment using a two-dimensional test problem of equation (10.42) with a manufactured solution of $u(x, y) = 100(x(1-x)y(1-y))^2$.

10.3 ■ Mixed finite-element methods

Objectives

As with the previous section, we continue to focus on extensions of the general finite-element theory presented in Chapter 8. Our goal now is to consider other *systems* of PDEs and to motivate the idea of *mixed finite-element methods*. In this case, you will specifically consider systems of PDEs with multiple unknowns, such as the electric and magnetic fields in Maxwell's equations discussed earlier, or the fluid velocity and pressure in Stokes or Darcy flow. The discretization of such systems involves unknowns that may not be functions in $H^1(\Omega)$ and may not all be in the same function space. This gives rise to the terminology of a *mixed finite-element method*, since we are “mixing” different types of function spaces in our finite-element discretization. In this section, you will study the properties of well-posedness for mixed variational problems. Specifically, you will

- consider a first-order formulation of the Poisson problem in order to motivate a general mixed variational form;
- revisit the well-posedness ideas of Lax–Milgram presented in Chapter 8 and be introduced to the concept of the inf-sup condition;
- contrast the differences between what is needed for the continuous variational form and the discrete setting; and
- give examples of well-posed finite-element formulations for the mixed formulation of the Poisson equation and for the Stokes equations.

To start, we again consider the second-order elliptic Poisson equation

$$-\nabla \cdot \nabla u = f(\mathbf{x}) \quad (10.48)$$

for $\mathbf{x} \in \Omega$, with $u = 0$ on $\partial\Omega$ and with $f \in L^2(\bar{\Omega})$. In Chapter 8, we transformed this second-order PDE with a solution $u \in C^2(\Omega)$ into a weak formulation with $u \in H_0^1(\Omega)$. However, to explore the idea of mixed methods, we instead rewrite this PDE as a first-order system by introducing an auxiliary variable, $\boldsymbol{\sigma} = \nabla u$. This choice of an auxiliary variable has a natural interpretation in the setting of flow through a porous medium, modeled by Darcy's law, where u represents the fluid pressure in the medium and $-\boldsymbol{\sigma}$ is known as the *Darcy velocity*. In other settings, we refer to $-\boldsymbol{\sigma}$ as the flux associated with density u (see Remark 1.5). With this definition of $\boldsymbol{\sigma}$, we rewrite the original PDE as the first-order system

$$\boldsymbol{\sigma} - \nabla u = \mathbf{0}, \quad (10.49a)$$

$$-\nabla \cdot \boldsymbol{\sigma} = f(\mathbf{x}). \quad (10.49b)$$

We now have two unknowns, u and $\boldsymbol{\sigma}$, and we need to find an appropriate finite-element space for each of them. Since the space for $\boldsymbol{\sigma}$ must be a space of vector-valued functions, it is natural to expect that the two unknowns live in different spaces. Ignoring, for the moment, what is needed from these spaces, we write $u \in \mathcal{V}$ and $\boldsymbol{\sigma} \in \mathcal{W}$ and multiply equation (10.49) by test functions $\mathbf{w} \in \mathcal{W}$ and $v \in \mathcal{V}$ and integrate over the domain to obtain a weak formulation to find $\boldsymbol{\sigma} \in \mathcal{W}$ and $u \in \mathcal{V}$ such that

$$\langle \boldsymbol{\sigma} - \nabla u, \mathbf{w} \rangle = 0 \quad \forall \mathbf{w} \in \mathcal{W}, \quad (10.50a)$$

$$\langle -\nabla \cdot \boldsymbol{\sigma}, v \rangle = \langle f, v \rangle \quad \forall v \in \mathcal{V}, \quad (10.50b)$$

where, as before, $\langle \cdot, \cdot \rangle$ denotes the standard L^2 inner product of either scalar or vector functions. It is typical at this stage to integrate by parts on one of the derivative terms (either ∇u or $\nabla \cdot \boldsymbol{\sigma}$)

to try to restore symmetry to the variational system. In order to do so, we must carefully consider the resulting boundary integral.

Remark 10.11: Nonhomogeneous boundary conditions

While integration by parts is simple when we prescribe $u = 0$ on all of $\partial\Omega$, we must be more careful for nonhomogeneous cases. Consider the more general boundary conditions for the Poisson problem,

$$-\nabla \cdot \nabla u = f(\mathbf{x}) \quad \text{for } \mathbf{x} \in \Omega, \quad (10.51a)$$

$$u = q_{\mathcal{D}}(\mathbf{x}) \quad \text{for } \mathbf{x} \in \Gamma_{\mathcal{D}}, \quad (10.51b)$$

$$\nabla u \cdot \mathbf{n} = q_{\mathcal{N}}(\mathbf{x}) \quad \text{for } \mathbf{x} \in \Gamma_{\mathcal{N}}, \quad (10.51c)$$

where $\mathbf{n}$ is the outward unit normal vector to the boundary $\partial\Omega = \Gamma_{\mathcal{D}} \cup \Gamma_{\mathcal{N}}$, and $\Gamma_{\mathcal{D}}$ and $\Gamma_{\mathcal{N}}$ are disjoint segments on which we impose Dirichlet and Neumann boundary conditions for u , respectively. Recall that, in the setting of Chapter 8, the Dirichlet boundary condition became an *essential* boundary condition that was imposed on the space for u , while the Neumann boundary condition became a *natural* boundary condition that was accounted for in the weak form. Here, a Neumann boundary condition for u becomes a Dirichlet boundary condition for σ , requiring that $\sigma \cdot \mathbf{n} = q_{\mathcal{N}}$.

Now consider the term $\langle -\nabla u, \mathbf{w} \rangle$ in equation (10.50a) and integrate by parts on it to obtain

$$\langle -\nabla u, \mathbf{w} \rangle = \int_{\Omega} -\nabla u \cdot \mathbf{w} \, d\mathbf{x} \quad (10.52a)$$

$$= - \int_{\partial\Omega} u \mathbf{w} \cdot \mathbf{n} \, ds + \int_{\Omega} u \nabla \cdot \mathbf{w} \, d\mathbf{x} \quad (10.52b)$$

$$= - \int_{\Gamma_{\mathcal{D}}} q_{\mathcal{D}} \mathbf{w} \cdot \mathbf{n} \, ds - \int_{\Gamma_{\mathcal{N}}} u \mathbf{w} \cdot \mathbf{n} \, ds + \int_{\Omega} u \nabla \cdot \mathbf{w} \, d\mathbf{x}. \quad (10.52c)$$

As a result, the condition that $\sigma \cdot \mathbf{n} = q_{\mathcal{N}}$ on $\Gamma_{\mathcal{N}}$ must now be enforced as an essential boundary condition on the restricted space $\mathcal{W}_0$, with $\mathbf{w} \cdot \mathbf{n} = 0$ on $\Gamma_{\mathcal{N}}$, to eliminate the integral over $\Gamma_{\mathcal{N}}$. On the other hand, we can enforce $u = q_{\mathcal{D}}$ as a natural boundary condition, leaving this integral in the weak form as a forcing term on the right-hand side. The “symmetry” that results in the variational form from this integration then allows us to write the system in a useful way.

Using the integration by parts from Remark 10.11 lets us rewrite equation (10.50) as the first-order system

$$\langle \sigma, \mathbf{w} \rangle + \langle u, \nabla \cdot \mathbf{w} \rangle = \int_{\Gamma_{\mathcal{D}}} q_{\mathcal{D}} \mathbf{w} \cdot \mathbf{n} \, ds \quad \forall \mathbf{w} \in \mathcal{W}_0, \quad (10.53a)$$

$$\langle -\nabla \cdot \sigma, v \rangle = \langle f, v \rangle \quad \forall v \in \mathcal{V}, \quad (10.53b)$$

noting that the right-hand side of equation (10.53a) is zero when we assume a homogeneous boundary condition of $q_{\mathcal{D}} = 0$, as we did to start this section. Additionally, to simplify the notation, we drop the subscript on $\mathcal{W}_0$ for the remainder of the section and assume that appropriate essential boundary conditions are built in as needed. From equation (10.53), we recognize that the left-hand side can be written in terms of two bilinear forms, which we define to be $a(\sigma, \mathbf{w}) = \langle \sigma, \mathbf{w} \rangle$ and $b(\mathbf{w}, u) = \langle u, \nabla \cdot \mathbf{w} \rangle$. For symmetry, we multiply the second equation

by -1 , yielding a symmetric saddle-point system:

$$a(\boldsymbol{\sigma}, \mathbf{w}) + b(\mathbf{w}, u) = 0 \quad \forall \mathbf{w} \in \mathcal{W}, \quad (10.54a)$$

$$b(\boldsymbol{\sigma}, v) = -\langle f, v \rangle \quad \forall v \in \mathcal{V}. \quad (10.54b)$$

We call such a problem a *saddle-point problem* because it corresponds to a constrained minimization problem, where we aim to minimize an energy associated with the bilinear form $a(\boldsymbol{\sigma}, \mathbf{w})$, subject to the constraint that $b(\boldsymbol{\sigma}, v)$ takes a prescribed value. The formulation in equation (10.54) corresponds to using a Lagrange multiplier to enforce that constraint, and the solution of the constrained minimization problem in this formulation is known to be a saddle point of the resulting Lagrangian functional.

10.3.1 ■ Continuous inf-sup conditions

Since we are now dealing with a *system* of PDEs, we need to carefully define continuity and (weak) coercivity in the product space, $\mathcal{W} \times \mathcal{V}$. Here, well-posedness is not (necessarily) an automatic consequence of Theorem 8.27 (the Lax–Milgram theorem) but, instead, it determines a sense of “compatibility” between the different spaces. This comes in the form of an *inf-sup condition*, as described below. To start, let us consider a slightly more general mixed formulation than in equation (10.54) of finding $\boldsymbol{\sigma} \in \mathcal{W}$ and $u \in \mathcal{V}$ that satisfy

$$a(\boldsymbol{\sigma}, \mathbf{w}) + b(\mathbf{w}, u) = g(\mathbf{w}) \quad \forall \mathbf{w} \in \mathcal{W}, \quad (10.55a)$$

$$b(\boldsymbol{\sigma}, v) = f(v) \quad \forall v \in \mathcal{V}, \quad (10.55b)$$

where $\mathcal{W}$ and $\mathcal{V}$ are Hilbert spaces and g and f are bounded linear functions on $\mathcal{W}$ and $\mathcal{V}$, respectively (meaning that they are elements of the dual spaces of $\mathcal{W}$ and $\mathcal{V}$). Looking at equation (10.54) above, we have $g(\mathbf{w}) = 0$ and $f(v) = -\langle f, v \rangle$.

The first part of a standard well-posedness result, whether in the scalar case of Chapter 8 or in the (mixed) systems case of this section, is to show continuity (or boundedness) of the bilinear forms. As a first step, we assume that both bilinear forms in equation (10.55) are continuous, as defined in Definition 8.25—i.e., there exist constants $c_a > 0$ and $c_b > 0$ such that

$$|a(\boldsymbol{\sigma}, \mathbf{w})| \leq c_a \|\boldsymbol{\sigma}\|_{\mathcal{W}} \|\mathbf{w}\|_{\mathcal{W}} \quad \forall \boldsymbol{\sigma}, \mathbf{w} \in \mathcal{W}, \quad (10.56a)$$

$$|b(\boldsymbol{\sigma}, v)| \leq c_b \|\boldsymbol{\sigma}\|_{\mathcal{W}} \|v\|_{\mathcal{V}} \quad \forall \boldsymbol{\sigma} \in \mathcal{W}, v \in \mathcal{V}. \quad (10.56b)$$

Next, we address coercivity. For simplicity, we start with the case $f = 0$. The central idea behind existence and uniqueness results for mixed systems is to separate the proof into two pieces, first looking for solutions $\boldsymbol{\sigma} \in \widehat{\mathcal{W}} := \{\mathbf{w} \in \mathcal{W} \mid b(\mathbf{w}, v) = 0 \ \forall v \in \mathcal{V}\}$, noting that $\widehat{\mathcal{W}}$ is a closed linear subspace of $\mathcal{W}$. Since this guarantees that equation (10.55b) is satisfied, we choose a test function for the first equation from the restricted space, $\mathbf{w} \in \widehat{\mathcal{W}}$, giving the equation

$$a(\boldsymbol{\sigma}, \mathbf{w}) = g(\mathbf{w}) \quad \forall \mathbf{w} \in \widehat{\mathcal{W}}. \quad (10.57)$$

This reduces the saddle-point system in equation (10.55) to the single (vector) equation in equation (10.57), for which we know how to apply the theory of Chapter 8. Thus, if $a(\cdot, \cdot)$ is coercive on $\widehat{\mathcal{W}}$, then it satisfies the conditions of the Lax–Milgram theorem (since $a(\cdot, \cdot)$ is continuous on $\mathcal{W}$, it is also continuous on $\widehat{\mathcal{W}}$ using the same norm), and there exists a unique solution, $\boldsymbol{\sigma}^*$, to the modified problem

$$a(\boldsymbol{\sigma}, \mathbf{w}) = g(\mathbf{w}) \quad \forall \mathbf{w} \in \widehat{\mathcal{W}}, \quad (10.58a)$$

$$b(\boldsymbol{\sigma}, v) = 0 \quad \forall v \in \mathcal{V} \quad (10.58b)$$

so long as $g(\mathbf{w})$ is a bounded linear functional on $\mathcal{W}$ (or $\widehat{\mathcal{W}}$).

All that is left is to determine if there is a unique u that satisfies the original system. To do this, we rewrite the first equation in equation (10.55), fixing the value of σ to be the solution, σ^* , of equation (10.58), giving

$$b(\mathbf{w}, u) = g(\mathbf{w}) - a(\sigma^*, \mathbf{w}) \quad \forall \mathbf{w} \in \mathcal{W}. \quad (10.59)$$

As above, we assume that g is a bounded linear functional and note that $a(\sigma^*, \cdot)$ is a bounded linear functional on $\mathcal{W}$ for fixed σ^* . Therefore, $g(\cdot) - a(\sigma^*, \cdot)$ is also a bounded linear functional. Combining this with the continuity of $b(\cdot, \cdot)$ on $\mathcal{W} \times \mathcal{V}$, we establish existence and uniqueness of u by showing some form of coercivity.

Recall the notion of weak coercivity in Section 8.4. One technical issue to address in this setting is that our bilinear functional acts on two different spaces, so we must modify equation (8.145), leading to Definition 10.12.

Definition 10.12: Generalized weak coercivity

A bilinear form $b(\mathbf{w}, v)$ is weakly coercive on $\mathcal{W} \times \mathcal{V}$ if there exists a constant $c_0 > 0$ such that

$$\inf_{v \in \mathcal{V}} \sup_{\mathbf{w} \in \mathcal{W}} \frac{b(\mathbf{w}, v)}{\|\mathbf{w}\|_{\mathcal{W}} \|v\|_{\mathcal{V}}} \geq c_0. \quad (10.60)$$

Remark 10.13: The “inf-sup” or “LBB” condition

The condition in equation (10.60) is often referred to as the *inf-sup condition*, and, in general, this framework is often referred to as the Babuška theory [27] for proving well-posedness of a mixed variational form (or the Brezzi theory [55] for saddle-point problems). For the special case of the mixed Poisson formulation presented in this section and the mixed formulation of the Stokes equations presented later, this specific version of the inf-sup condition is referred to as the Ladyzhenskaya–Babuška–Brezzi (LBB) condition or generalized LBB condition, respectively, and was first considered in [148].

To see why the inf-sup condition in equation (10.60) is sufficient to say that u is uniquely determined, we consider the representation of the bilinear form, b , as an operator. We define $\mathcal{V}^*$ and $\mathcal{W}^*$ to be the dual spaces of $\mathcal{V}$ and $\mathcal{W}$, respectively, as in Definition 8.21, and we define $B : \mathcal{W} \rightarrow \mathcal{V}^*$ by

$$\langle B\mathbf{w}, v \rangle = b(\mathbf{w}, v) \quad \forall \mathbf{w} \in \mathcal{W}, \quad \forall v \in \mathcal{V}, \quad (10.61)$$

where the inner product at left is the scalar L^2 inner product. Similarly, we define $B' : \mathcal{V} \rightarrow \mathcal{W}^*$ as

$$\langle B'v, \mathbf{w} \rangle = b(\mathbf{w}, v) \quad \forall \mathbf{w} \in \mathcal{W}, \quad \forall v \in \mathcal{V}, \quad (10.62)$$

where the inner product at left is now the vector L^2 inner product. In this form, $\widehat{\mathcal{W}} = \text{Ker}(B)$, and the inf-sup condition is sometimes referred to as a coercivity assumption on the kernel of the operator. The basic idea is that the inf-sup condition requires that for any $v \in \mathcal{V}$, there is a nonzero $\mathbf{w} \in \mathcal{W}$ such that $b(\mathbf{w}, v) \geq c_0 \|\mathbf{w}\|_{\mathcal{W}} \|v\|_{\mathcal{V}}$. Clearly, this nonzero $\mathbf{w}$ cannot be in $\widehat{\mathcal{W}}$; otherwise, $b(\mathbf{w}, v) = 0$ and the inequality could not hold for $c_0 > 0$. Further, it must be the case that for any $v \in \mathcal{V}$, the best choices of $\mathbf{w} \in \mathcal{W}$ must be those that are orthogonal to $\widehat{\mathcal{W}}$; otherwise, we have larger values of $\|\mathbf{w}\|_{\mathcal{W}}$ in the denominator, but no increase in value of $b(\mathbf{w}, v)$ in the numerator. This leads to an important property of B : if $b(\mathbf{w}, v)$ satisfies the inf-sup condition, then B is an isomorphism from $\widehat{\mathcal{W}}^\perp := \{\mathbf{z} \in \mathcal{W} \mid \langle \mathbf{z}, \mathbf{w} \rangle_{\mathcal{W}} = 0 \quad \forall \mathbf{w} \in \widehat{\mathcal{W}}\}$ onto $\mathcal{V}^*$. A consequence of this is that the inf-sup condition can be shown to be equivalent to the

following statements:

$$\|B\mathbf{w}\|_{\mathcal{V}^*} \geq c_0 \|\mathbf{w}\|_{\mathcal{W}} \quad \forall \mathbf{w} \in \widehat{\mathcal{W}}^\perp, \quad (10.63a)$$

$$\|B'v\|_{\mathcal{W}^*} \geq c_0 \|v\|_{\mathcal{V}} \quad \forall v \in \mathcal{V}. \quad (10.63b)$$

The second of these says that $b(\mathbf{w}, v)$ satisfies a coercivity relation that allows us to conclude that u is uniquely determined as the solution to

$$b(\mathbf{w}, u) = g(\mathbf{w}) - a(\boldsymbol{\sigma}^*, \mathbf{w}) \quad \forall \mathbf{w} \in \widehat{\mathcal{W}}^\perp. \quad (10.64)$$

From here, we generalize the above to the case when $f \neq 0$, again using the fact that B is an isomorphism from $\widehat{\mathcal{W}}^\perp$ onto $\mathcal{V}^*$. In particular, if the inf-sup condition holds, there must exist a unique $\boldsymbol{\sigma}_0 \in \widehat{\mathcal{W}}^\perp$ such that $B\boldsymbol{\sigma}_0 = f$ for every $f \in \mathcal{V}^*$, which means that $b(\boldsymbol{\sigma}_0, v) = f(v)$ for all $v \in \mathcal{V}$. Then, replacing $\boldsymbol{\sigma}$ with $\boldsymbol{\sigma} + \boldsymbol{\sigma}_0$ in equation (10.55) yields equation (10.58), and we are back at the case when $f = 0$, similar to the construction in Section 8.4.3. Thus, we arrive at the final result, given in Theorem 10.14.

Theorem 10.14: Well-posedness of mixed formulation [52, Sec. 12.2]

Let $\mathcal{W}$ and $\mathcal{V}$ be Hilbert spaces. Suppose that $a(\cdot, \cdot) : \mathcal{W} \times \mathcal{W} \rightarrow \mathbb{R}$ and $b(\cdot, \cdot) : \mathcal{W} \times \mathcal{V} \rightarrow \mathbb{R}$ are bounded bilinear functionals and that g and f are bounded linear functionals on $\mathcal{W}$ and $\mathcal{V}$, respectively. If a is coercive on $\widehat{\mathcal{W}} := \{\mathbf{w} \in \mathcal{W} \mid b(\mathbf{w}, v) = 0 \forall v \in \mathcal{V}\}$ and b satisfies the inf-sup condition in Definition 10.12, then there exists a unique solution, $(\boldsymbol{\sigma}, u) \in \mathcal{W} \times \mathcal{V}$, that solves equation (10.55).

Remark 10.15: Coercivity on the kernel of $b(\mathbf{w}, v)$

We highlight that Theorem 10.14 proves existence and uniqueness of solutions to the saddle-point problem in equation (10.55) for any bounded linear functionals $g(\mathbf{w})$ and $f(v)$ but that the coercivity condition that we check is always on the kernel of $b(\mathbf{w}, v)$. This follows from the discussion above the theorem that we need this condition (with $b(\mathbf{w}, v) = 0$ for all $v \in \mathcal{V}$) to uniquely solve for the component of $\boldsymbol{\sigma}$ in $\widehat{\mathcal{W}}$, and that the component of $\boldsymbol{\sigma}$ in $\widehat{\mathcal{W}}^\perp$ is used to satisfy equation (10.55b) with nonzero $f(v)$. More details on the general theory and for the specific examples below are found in many sources, including [52, 43] and the references therein.

Going back to the mixed Poisson problem in equations (10.53) and (10.54) with bilinear forms, $a(\boldsymbol{\sigma}, \mathbf{w}) = \langle \boldsymbol{\sigma}, \mathbf{w} \rangle$ and $b(\boldsymbol{\sigma}, v) = \langle \nabla \cdot \boldsymbol{\sigma}, v \rangle$, we see that no derivatives act on u in these forms, and only the divergence acts on $\boldsymbol{\sigma}$. Consequently, it is natural to choose the spaces to be $\mathcal{W} = \mathbf{H}(\text{div}, \Omega)$ and $\mathcal{V} = L^2(\Omega)$. The question is, then, whether equation (10.54) is well posed on this choice of spaces.

We note that assuming $f \in L^2$ is enough to guarantee that $-\langle f, \cdot \rangle$ is a bounded linear functional on $L^2(\Omega)$, so we first consider the continuity conditions on $a(\cdot, \cdot)$ and $b(\cdot, \cdot)$. Both are easily verified using the Cauchy–Schwarz inequality. Coercivity of $a(\cdot, \cdot)$ on $\widehat{\mathcal{W}}$ is also easy to verify by using the property that $\|\nabla \cdot \boldsymbol{\sigma}\|^2 = b(\boldsymbol{\sigma}, \nabla \cdot \boldsymbol{\sigma}) = 0$ for any $\boldsymbol{\sigma} \in \widehat{\mathcal{W}}$, since $\nabla \cdot \boldsymbol{\sigma}$ is in $L^2(\Omega)$. Thus, we have the following:

Continuity of $a(\cdot, \cdot)$ on $\mathcal{W}$:

$$\langle \boldsymbol{\sigma}, \mathbf{w} \rangle \leq \|\boldsymbol{\sigma}\| \|\mathbf{w}\| \leq (\|\boldsymbol{\sigma}\|^2 + \|\nabla \cdot \boldsymbol{\sigma}\|^2)^{1/2} (\|\mathbf{w}\|^2 + \|\nabla \cdot \mathbf{w}\|^2)^{1/2} = \|\boldsymbol{\sigma}\|_{\text{div}} \|\mathbf{w}\|_{\text{div}} \quad (10.65a)$$

$$\Rightarrow a(\boldsymbol{\sigma}, \mathbf{w}) \leq \|\boldsymbol{\sigma}\|_{\mathcal{W}} \|\mathbf{w}\|_{\mathcal{W}} \quad \forall \boldsymbol{\sigma}, \mathbf{w} \in \mathcal{W}. \quad (10.65b)$$

Continuity of $b(\cdot, \cdot)$ on $\mathcal{W} \times \mathcal{V}$:

$$\langle \nabla \cdot \boldsymbol{\sigma}, v \rangle \leq \|\nabla \cdot \boldsymbol{\sigma}\| \|v\| \leq (\|\boldsymbol{\sigma}\|^2 + \|\nabla \cdot \boldsymbol{\sigma}\|^2)^{1/2} \|v\| = \|\boldsymbol{\sigma}\|_{\text{div}} \|v\| \quad (10.66a)$$

$$\Rightarrow b(\boldsymbol{\sigma}, v) \leq \|\boldsymbol{\sigma}\|_{\mathcal{W}} \|v\|_{\mathcal{V}} \quad \forall \boldsymbol{\sigma} \in \mathcal{W}, v \in \mathcal{V}. \quad (10.66b)$$

Coercivity of $a(\cdot, \cdot)$ on $\widehat{\mathcal{W}}$:

$$\|\boldsymbol{\sigma}\|_{\text{div}}^2 = \|\boldsymbol{\sigma}\|^2 + \|\nabla \cdot \boldsymbol{\sigma}\|^2 = \|\boldsymbol{\sigma}\|^2 \quad (10.67a)$$

$$\Rightarrow a(\boldsymbol{\sigma}, \boldsymbol{\sigma}) \geq \|\boldsymbol{\sigma}\|_{\mathcal{W}}^2 \quad \forall \boldsymbol{\sigma} \in \widehat{\mathcal{W}}. \quad (10.67b)$$

The final piece is the inf-sup condition. With appropriate assumptions on the domain Ω , for any $v \in \mathcal{V}$, there is a $\phi \in H^2(\Omega)$ that solves $-\nabla \cdot \nabla \phi = v$ with homogeneous Dirichlet boundary conditions. For details, see the discussion on elliptic regularity in Section 8.3.1. Additionally, take $\boldsymbol{\sigma} = -\nabla \phi \in \mathcal{W}$. Then,

$$b(\boldsymbol{\sigma}, v) = \langle \nabla \cdot \boldsymbol{\sigma}, v \rangle = \langle -\nabla \cdot \nabla \phi, v \rangle = \langle v, v \rangle \quad (10.68a)$$

$$= \|v\|_{\mathcal{V}}^2, \quad (10.68b)$$

$$\|\boldsymbol{\sigma}\|_{\mathcal{W}}^2 = \|\nabla \cdot \boldsymbol{\sigma}\|^2 + \|\boldsymbol{\sigma}\|^2 \quad (10.68c)$$

$$= \|v\|^2 + \|\nabla \phi\|^2 = \|v\|^2 + \langle \nabla \phi, \nabla \phi \rangle \quad (10.68d)$$

$$= \|v\|^2 + \langle \phi, v \rangle \leq \|v\|^2 + \|\phi\| \|v\| \quad (10.68e)$$

$$\leq (1 + \beta) \|v\|_{\mathcal{V}}^2, \quad (10.68f)$$

where we use integration by parts and elliptic regularity of the Poisson problem with some constant $\beta > 0$, meaning that $\|\phi\| \leq \beta \|v\|$. Thus, for any $v \in \mathcal{V}$ and $c_0 = \frac{1}{\sqrt{1+\beta}}$,

$$c_0 \|v\|_{\mathcal{V}} \leq \frac{b(\boldsymbol{\sigma}, v)}{\|\boldsymbol{\sigma}\|_{\mathcal{W}}} \leq \sup_{\boldsymbol{w} \in \mathcal{W}} \frac{b(\boldsymbol{w}, v)}{\|\boldsymbol{w}\|_{\mathcal{W}}}. \quad (10.69)$$

Since this holds for all $v \in \mathcal{V}$, it also holds for the infimum; then, we get that $b(\boldsymbol{w}, v)$ is weakly coercive over $\boldsymbol{H}(\text{div}, \Omega) \times L^2(\Omega)$. Thus, the mixed formulation is well posed.

10.3.2 ■ Discrete inf-sup conditions and mixed Poisson

Next, we consider the finite-dimensional counterpart to the well-posedness results discussed above. Let $\mathcal{W}^h \subset \mathcal{W}$ and $\mathcal{V}^h \subset \mathcal{V}$ be finite-dimensional subspaces (finite-element spaces) defined on some triangulation of the domain with mesh parameter h . Then, we rewrite equation (10.55) discretely as finding $\boldsymbol{\sigma}^h \in \mathcal{W}^h$ and $u^h \in \mathcal{V}^h$ that satisfy

$$a(\boldsymbol{\sigma}^h, \boldsymbol{w}^h) + b(\boldsymbol{w}^h, u^h) = g(\boldsymbol{w}^h) \quad \forall \boldsymbol{w}^h \in \mathcal{W}^h, \quad (10.70a)$$

$$b(\boldsymbol{\sigma}^h, v^h) = f(v^h) \quad \forall v^h \in \mathcal{V}^h. \quad (10.70b)$$

However, we note that $\mathcal{V}^h$ is a proper subspace of $\mathcal{V}$. Thus, having $b(\boldsymbol{w}^h, v^h) = 0$ for all $v^h \in \mathcal{V}^h$ does not imply that $b(\boldsymbol{w}^h, v) = 0$ for all $v \in \mathcal{V}$. Due to this fact, knowing that b satisfies an inf-sup condition defined over $\mathcal{W} \times \mathcal{V}$ does not imply that it satisfies a similar condition over $\mathcal{W}^h \times \mathcal{V}^h$. Therefore, in order to prove well-posedness of a discrete mixed variational problem, we must prove the inf-sup condition for every pair of finite-element spaces, $(\mathcal{W}^h, \mathcal{V}^h)$, that we wish to consider. To do this, we first extend the notion of an inf-sup condition to the discrete case in Definition 10.16.

Definition 10.16: Discrete inf-sup condition

A family of finite-element spaces $(\mathcal{W}^h, \mathcal{V}^h)$ satisfies the *discrete inf-sup condition* if the bilinear form, $b(\mathbf{w}^h, v^h)$, is weakly coercive on $\mathcal{W}^h \times \mathcal{V}^h$, i.e., if there exists a constant $\widehat{c}_0 > 0$, independent of h , such that

$$\inf_{v^h \in \mathcal{V}^h} \sup_{\mathbf{w}^h \in \mathcal{W}^h} \frac{b(\mathbf{w}^h, v^h)}{\|\mathbf{w}^h\|_{\mathcal{W}^h} \|v^h\|_{\mathcal{V}^h}} \geq \widehat{c}_0. \quad (10.71)$$

Generally, proving coercivity of $a(\cdot, \cdot)$ at the continuous level implies that it is coercive at the discrete level as well. This means that proving the discrete inf-sup condition is the only additional requirement to prove well-posedness of the discrete mixed problem. There is, however, a special case for which the continuous inf-sup condition directly implies the discrete condition: namely, when there exists a Fortin operator, which is defined in Definition 10.17. The implications are detailed in Theorem 10.18.

Definition 10.17: Fortin operator [100]

Let $\mathcal{W}$ and $\mathcal{V}$ be Hilbert spaces, with $b : \mathcal{W} \times \mathcal{V} \rightarrow \mathbb{R}$ being a bounded bilinear functional, and let $\mathcal{W}^h \subset \mathcal{W}$ and $\mathcal{V}^h \subset \mathcal{V}$ be finite-element subspaces. We say that $\Pi_F : \mathcal{W} \rightarrow \mathcal{W}^h$ is a *Fortin operator* if

$$b(\mathbf{w}, v^h) = b(\Pi_F(\mathbf{w}), v^h) \quad \forall v^h \in \mathcal{V}^h \quad (10.72)$$

and if there exists a constant $C_F > 0$ such that

$$\|\Pi_F(\mathbf{w})\|_{\mathcal{W}} \leq C_F \|\mathbf{w}\|_{\mathcal{W}} \quad \forall \mathbf{w} \in \mathcal{W}. \quad (10.73)$$

Theorem 10.18: An inf-sup condition and a Fortin operator [43, Prop. 8.4.1]

Let $\mathcal{W}$ and $\mathcal{V}$ be Hilbert spaces, with $b : \mathcal{W} \times \mathcal{V} \rightarrow \mathbb{R}$ being a bounded bilinear functional, and let $\mathcal{W}^h \subset \mathcal{W}$ and $\mathcal{V}^h \subset \mathcal{V}$ be finite-element subspaces. If $b(\cdot, \cdot)$ satisfies the continuous inf-sup condition, Definition 10.12, and a Fortin operator exists, then the discrete inf-sup condition, Definition 10.16, also holds.

Proof. Let $v^h \in \mathcal{V}^h$. Then,

$$\sup_{\mathbf{w}^h \in \mathcal{W}^h} \frac{b(\mathbf{w}^h, v^h)}{\|\mathbf{w}^h\|_{\mathcal{W}}} \geq \sup_{\mathbf{w} \in \mathcal{W}} \frac{b(\Pi_F(\mathbf{w}), v^h)}{\|\Pi_F(\mathbf{w})\|_{\mathcal{W}}} \geq \frac{1}{C_F} \sup_{\mathbf{w} \in \mathcal{W}} \frac{b(\mathbf{w}, v^h)}{\|\mathbf{w}\|_{\mathcal{W}}} \geq \frac{c_0}{C_F} \|v^h\|_{\mathcal{V}}. \quad (10.74)$$

Thus, the discrete inf-sup condition is satisfied with $\widehat{c}_0 = \frac{c_0}{C_F}$. $\square$

From Theorem 10.18, for each pair of finite-element spaces, we must either prove the existence of a Fortin operator or check the discrete inf-sup condition directly. This then guarantees the existence of a unique solution to the finite-element problem.

Remark 10.19: Céa's lemma equivalent

As in the case of the scalar finite-element method, the approximation properties of the finite-element spaces are used to construct error estimates for the discrete solutions of a mixed formulation, assuming that the above well-posedness results hold. This leads to an analogous

version of Céa's lemma from Lemma 8.31. Namely, assume that σ and u are the exact solutions to equation (10.55) and σ^h and u^h are the discrete solutions to equation (10.70). If equation (10.70) satisfies the above-mentioned criteria for well-posedness, then the following error bound holds:

$$\|\sigma - \sigma^h\|_{\mathcal{W}} + \|u - u^h\|_{\mathcal{V}} \leq C \left(\inf_{w^h \in \mathcal{W}^h} \|\sigma - w^h\|_{\mathcal{W}} + \inf_{v^h \in \mathcal{V}^h} \|u - v^h\|_{\mathcal{V}} \right), \quad (10.75)$$

where $C > 0$ is independent of h but depends on the continuity, coercivity, and inf-sup conditions of $a(\cdot, \cdot)$ and $b(\cdot, \cdot)$ at the discrete level. The proof of this result is similar to the scalar case shown in the proof of Lemma 8.31. Full details are found, for example, in [52, Cor. 12.5.18]. From this, we see that the error bounds are defined entirely by both the approximation properties of the finite-element space and the stability results of the coercivity and inf-sup conditions. We therefore expect similar results to Chapter 8. For example, if linear elements are used for both spaces, we expect $\mathcal{O}(h)$ convergence if a discrete inf-sup condition is satisfied and $\mathcal{W}$ or $\mathcal{V}$ is an $H^1(\Omega)$ -type space.

In the following, we give two examples of finite-element spaces: one that satisfies a discrete inf-sup condition and one that does not. Again, more analysis and details are found in classic texts such as [43, 52].

$\mathcal{RT}_0$ - P_0^- for mixed Poisson

For the infinite-dimensional space, $\mathcal{W} = \mathbf{H}(\text{div}, \Omega)$, a natural choice of finite-element space on mesh Ω^h would be the low-order space, $\mathcal{W}^h = \mathcal{RT}_0(\Omega^h) \subset \mathbf{H}(\text{div}, \Omega)$, defined in Section 10.2. For $\mathcal{V} = L^2(\Omega)$, there are many choices for finite-dimensional $\mathcal{V}^h \subset \mathcal{V}$ on mesh Ω^h . Here, we use the discontinuous finite-element space $P_0^-(\Omega^h)$, defined as

$$P_0^-(\Omega^h) = \{u \in L^2(\Omega^h) \mid u \text{ is constant on each } \tau \in \mathcal{T}^h\}, \quad (10.76)$$

often referred to as the piecewise constant space on Ω^h . To prove well-posedness, we consider the two conditions of stability on the discrete spaces, namely, that $a(\cdot, \cdot)$ is coercive on the kernel of $\mathcal{W}^h$ and that the discrete inf-sup condition, Definition 10.16, holds.

Condition 1: Coercivity of $a(\cdot, \cdot)$ on kernel of $b(\cdot, \cdot)$ As in the continuous case, the coercivity of $a(\cdot, \cdot)$ is trivial. The discrete kernel is defined as

$$\widehat{\mathcal{W}}_h := \{w^h \in \mathcal{W}^h \mid \langle \nabla \cdot w^h, v^h \rangle = 0 \forall v^h \in \mathcal{V}^h\}. \quad (10.77)$$

Since $\mathcal{RT}_0$ consists of piecewise linear vector fields, their divergences are piecewise constants. Hence, $\text{div } \mathcal{W}^h \subset \mathcal{V}^h$ (meaning that, for any $\sigma^h \in \mathcal{W}^h$, $\nabla \cdot \sigma^h \in \mathcal{V}^h$), giving that for any $\sigma^h \in \widehat{\mathcal{W}}_h$,

$$\|\sigma^h\|_{\text{div}}^2 = \|\sigma^h\|^2 + \underbrace{\langle \nabla \cdot \sigma^h, \nabla \cdot \sigma^h \rangle}_{\in \mathcal{V}^h} \stackrel{0}{=} \|\sigma^h\|^2 = a(\sigma^h, \sigma^h). \quad (10.78)$$

Not quite a Fortin operator Next, to prove the discrete inf-sup condition, we take a similar approach as in the continuous case but first establish that the finite-element projection operators have similar properties to Fortin operators. To do this, we consider the Raviart–Thomas projection operator, $\Pi^h : \mathbf{H}^1(\Omega) \rightarrow \mathcal{RT}_0(\Omega^h)$, where, for a given $w \in \mathbf{H}^1(\Omega)$, we define $w^h = \Pi^h(w) \in \mathcal{RT}_0(\Omega^h)$ by its degrees of freedom, as in equation (10.27), requiring that

$$\frac{1}{|f_\ell|} \int_{f_\ell} (w^h \cdot n_\ell) ds = \frac{1}{|f_\ell|} \int_{f_\ell} (w \cdot n_\ell) ds \quad (10.79)$$

for all faces, ℓ , in the mesh. Note that Π^h cannot be a Fortin operator, since its domain is $\mathbf{H}^1(\Omega)$ and not $\mathbf{H}(\text{div}, \Omega)$. However, Π^h is a bounded operator in the sense that there exists a $c > 0$ independent of mesh size, h , for which

$$\|\Pi^h \mathbf{w}\|_{\text{div}} \leq c \|\mathbf{w}\|_1 \quad \forall \mathbf{w} \in \mathbf{H}^1(\Omega). \quad (10.80)$$

Similarly, we define the projection operator $Q^h : L^2(\Omega^h) \rightarrow \mathcal{V}^h$ by defining, for any $v \in L^2(\Omega)$, the piecewise constant function $v^h = Q^h(v) \in P_0^-(\Omega^h)$ by requiring that

$$\int_{\tau} v^h \, d\mathbf{x} = \int_{\tau} v \, d\mathbf{x} \quad (10.81)$$

for every $\tau \in \mathcal{T}^h$. Using Green's theorem, we see that for any $\tau \in \mathcal{T}^h$,

$$\int_{\tau} \nabla \cdot \Pi^h(\mathbf{w}) \, d\mathbf{x} = \int_{\partial\tau} \Pi^h(\mathbf{w}) \cdot \mathbf{n} \, ds = \int_{\partial\tau} \mathbf{w} \cdot \mathbf{n} \, ds = \int_{\tau} \nabla \cdot \mathbf{w} \, d\mathbf{x} = \int_{\tau} Q^h(\nabla \cdot \mathbf{w}) \, d\mathbf{x}, \quad (10.82)$$

by noting that $\partial\tau$ consists of all of the faces on the element, where the $\mathcal{RT}_0$ degrees of freedom are defined. Therefore,

$$\nabla \cdot \Pi^h(\mathbf{w}) = Q^h(\nabla \cdot \mathbf{w}) \quad \forall \mathbf{w} \in \mathbf{H}^1(\Omega). \quad (10.83)$$

Condition 2: The inf-sup condition Returning to the inf-sup condition, we take any $v^h \in \mathcal{V}^h$. If needed (i.e., for a domain with curved edges), we extend v^h by zero to Ω . Then, with appropriate assumptions on the domain Ω , we again use elliptic regularity to show that there is a function $\phi \in H^2(\Omega)$ that solves $-\nabla \cdot \nabla \phi = v^h$ on Ω , with homogeneous Dirichlet boundary conditions. Taking $\boldsymbol{\sigma} = -\nabla \phi \in \mathbf{H}^1(\Omega)$ and $\boldsymbol{\sigma}^h = \Pi^h(\boldsymbol{\sigma}) \in \mathcal{W}^h$ results in

$$b(\boldsymbol{\sigma}^h, v^h) = \langle \nabla \cdot \boldsymbol{\sigma}^h, v^h \rangle = \langle \nabla \cdot \Pi^h(\boldsymbol{\sigma}), v^h \rangle = \langle Q^h(\nabla \cdot \boldsymbol{\sigma}), v^h \rangle = \langle v^h, v^h \rangle \quad (10.84a)$$

$$= \|v^h\|_{\mathcal{V}^h}^2, \quad (10.84b)$$

$$\|\boldsymbol{\sigma}^h\|_{\mathcal{W}^h}^2 = \|\Pi^h \boldsymbol{\sigma}\|_{\text{div}}^2 \leq c \|\boldsymbol{\sigma}\|_1 \quad (10.84c)$$

$$\leq c \|v^h\|_{\mathcal{V}^h}, \quad (10.84d)$$

again using elliptic regularity of the Poisson problem to get the constant c . Thus, for any $v^h \in \mathcal{V}^h$, there exists $\hat{c}_0 > 0$ such that

$$\hat{c}_0 \|v^h\|_{\mathcal{V}^h} \leq \frac{b(\boldsymbol{\sigma}^h, v^h)}{\|\boldsymbol{\sigma}^h\|_{\mathcal{W}^h}} \leq \sup_{\mathbf{w}^h \in \mathcal{W}^h} \frac{b(\mathbf{w}^h, v^h)}{\|\mathbf{w}^h\|_{\mathcal{W}^h}}. \quad (10.85)$$

Since this holds for all $v^h \in \mathcal{V}^h = P_0^-(\Omega^h)$, it also holds for the infimum, and we have that $b(\mathbf{w}^h, v^h)$ is weakly coercive over $\mathcal{RT}_0(\Omega^h) \times P_0^-(\Omega^h)$. Thus, the mixed formulation is well posed on these spaces.

All of the above arguments also work for higher-order formulations of the Raviart–Thomas spaces, using the more general result that $\text{div } \mathcal{RT}_k(\Omega^h) \subset P_k^-(\Omega^h)$ and using a consistently defined projection operator. Here, $P_k^-(\Omega^h)$ indicates the space of piecewise polynomials of degree no more than k on Ω^h , but where continuity across elements is not guaranteed (commonly known as the *discontinuous* Lagrange finite-element space).

To confirm these results, we solve the Poisson problem, equation (10.54), on a two-dimensional unit square domain. We choose an exact solution given by $u(x, y) = \sin(\pi x) \sin(\pi y)$, leading to $f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$. We use the finite-element pairs $\mathcal{RT}_k(\Omega^h) \times P_k^-(\Omega^h)$ for various values of k on a “uniform” triangular $n_x \times n_y$ mesh with $n_x = n_y$, consisting of a

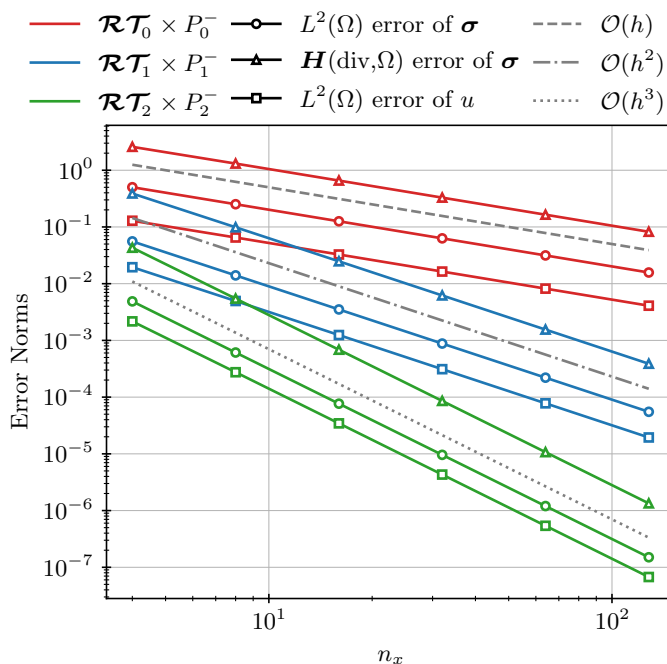


Figure 10.3.1. Error norms in $L^2(\Omega)$ and $\mathbf{H}(\text{div}, \Omega)$ versus mesh size for a simulation of the mixed Poisson problem defined in equation (10.54) using $\mathcal{RT}_k(\Omega^h) \times P_k^-(\Omega^h)$ for $0 \leq k \leq 2$. Here, $n_x + 1$ is the number of vertices of the triangular mesh in each direction, and $h = \mathcal{O}(n_x^{-1})$.

tensor product of $n_x + 1$ vertices in each direction. Each of the elements in the resulting $n_x \times n_x$ mesh of squares is then divided into two triangles, so that $h = \mathcal{O}(n_x^{-1})$. The convergence of σ and u with respect to the $\mathbf{H}(\text{div}, \Omega^h)$ and $L^2(\Omega^h)$ norms is shown in Figure 10.3.1, which confirms the expected $\mathcal{O}(h^{k+1})$ results for these stable pairs.

P_1 – P_1 for mixed Poisson

Because continuous piecewise linear elements work well for the scalar formulation of the Poisson equation, a natural extension is to choose piecewise linear elements for both unknowns in the mixed form—i.e., choose $\mathbf{P}_1(\Omega^h) \times P_1(\Omega^h)$ as the finite-element space for $\mathcal{W} \times \mathcal{V}$, where we use $\mathbf{P}_1(\Omega^h)$ to denote vector fields with each component in $P_1(\Omega^h)$. Unfortunately, such a choice is *unstable*.

Consider the case in one dimension with domain $[-1, 1]$, and observe that $\mathbf{H}(\text{div}, \Omega) = H^1(\Omega)$ in this case. The mixed formulation yields bilinear forms as follows:

$$a(\sigma, w) = \langle \sigma, w \rangle, \quad (10.86a)$$

$$b(w, v) = \langle w', v \rangle, \quad (10.86b)$$

with a discrete inf-sup condition of the form

$$\sup_{w^h \in P_1(\Omega^h)} \frac{\langle (w^h)', v^h \rangle}{\|w^h\|_1} \geq \widehat{c}_0 \|v^h\| \quad \forall v^h \in P_1(\Omega^h). \quad (10.87)$$

However, consider the case of a (nonconstant) piecewise linear function, v^h , that is zero at the midpoint of each element (for example, alternating between values of ± 1 at each node). Since w^h is also piecewise linear, $(w^h)'$ is piecewise constant. As a result, on every element,

$\int_{\tau} (w^h)' v^h dx = (w^h)' \int_{\tau} v^h dx = 0$. Thus, the only way to satisfy equation (10.87) is with $\widehat{c}_0 = 0$, which violates the requirement for an inf-sup condition in Definition 10.16.

A similar argument can be made in higher dimensions by constructing a piecewise linear function, v^h , with $\int_{\tau} v^h = 0$ on each element. Such a function is orthogonal to the space of piecewise constant functions. Thus, choosing $\mathcal{W}^h = \mathbf{P}_1(\Omega^h)$ gives $\operatorname{div} \mathcal{W}^h \subset P_0^-(\Omega^h)$, so $b(\mathbf{w}^h, v^h) = 0$ for any such function v^h , again forcing the constant in Definition 10.16 to be zero, which is not allowed.

Using the same two-dimensional test problem from the previous case, however, we see convergent approximations in Figure 10.3.2 (for the red lines). In fact, while we observe $\mathcal{O}(h)$ convergence in the $\mathbf{H}^1(\Omega^h)$ norm for σ , we get an $\mathcal{O}(h^2)$ convergence for both variables in the $L^2(\Omega^h)$ norm, as expected for a Ritz–Galerkin finite-element approximation on these spaces. However, the solution to this test problem is very smooth and does not exhibit the oscillatory behavior discussed above that violates the inf-sup condition. In Figure 10.3.2, we see that modifying the test problem to have a more oscillatory solution relative to the mesh, with $u(x, y) = \sin(100\pi x) \sin(100\pi y)$, reveals the expected breakdown (shown in the blue lines).

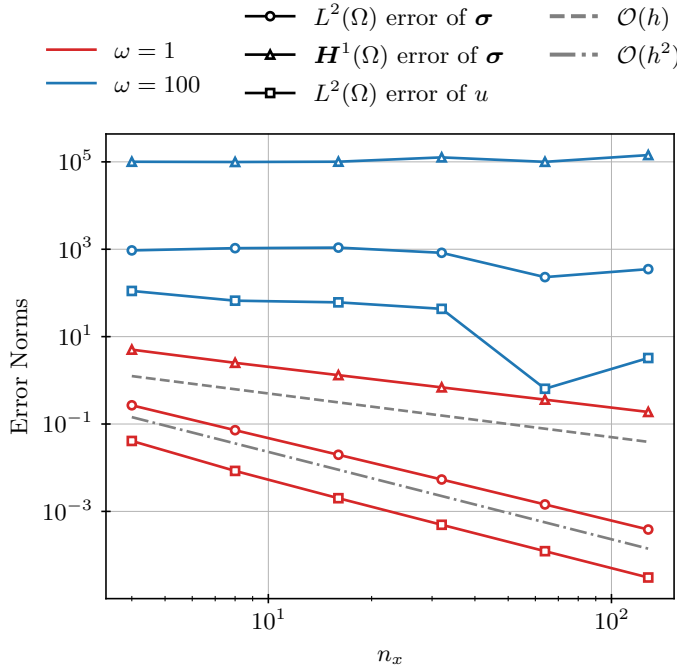


Figure 10.3.2. Error norms in $L^2(\Omega)$ and $\mathbf{H}^1(\Omega)$ versus mesh size for a simulation of the mixed Poisson problem defined in equation (10.54) using $\mathbf{P}_1(\Omega^h) \times P_1(\Omega^h)$, with solutions $u(x, y) = \sin(\omega\pi x) \sin(\omega\pi y)$ for $\omega = 1$ and 100. Here, $n_x + 1$ is the number of vertices of the triangular mesh in each direction and $h = \mathcal{O}(n_x^{-1})$.

10.3.3 ■ Weak formulation of the Stokes equations

To close out this section, we again consider the Stokes equations on a domain Ω ,

$$-\Delta \mathbf{u} + \nabla p = \mathbf{f}, \quad (10.88a)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (10.88b)$$

where $\mathbf{u}$ represents the velocity of a slow-moving, incompressible fluid and p is the pressure. Here, $\Delta \mathbf{u}$ denotes the *vector Laplacian* of the velocity and is a column vector with each entry being the scalar Laplacian of each component of $\mathbf{u}$: $(\Delta \mathbf{u})_i = \Delta u_i$. Again, more details are found in the texts [43, 52]; specific details appear in manuscripts cited below.

Remark 10.20: Symmetric gradient

It is common to write the Stokes equations using the *symmetric gradient*, i.e.,

$$-\nabla \cdot (2\epsilon(\mathbf{u})) + \nabla p = \mathbf{f}, \quad (10.89a)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (10.89b)$$

where $\epsilon(u) = \frac{1}{2} (\nabla \mathbf{u} + \nabla \mathbf{u}^\top)$, with $\nabla \mathbf{u}$ written as a tensor with $(\nabla \mathbf{u})_{ij} = \frac{\partial u_i}{\partial x_j}$ and the divergence operator acting on the rows of the tensor, yielding a column vector. When $\nabla \cdot \mathbf{u} = 0$, we can compute directly that $\Delta \mathbf{u} = \nabla \cdot (2\epsilon(\mathbf{u}))$. The symmetric gradient form arises naturally in many settings, particularly when considering generalized constitutive laws for non-Newtonian fluids. For this section, we generally use the simpler form with the vector Laplacian, though the theory below is derived similarly using the symmetric gradient.

We first consider the well-posedness of the continuous weak form and show that the continuous inf-sup condition holds. To obtain the mixed formulation, we multiply the equations by test functions in appropriate function spaces and integrate over the domain. Writing the (not yet specified) function spaces for $\mathbf{u}$ and p as $\mathcal{W}$ and $\mathcal{V}$, respectively, the weak variational formulation is as follows: find $\mathbf{u} \in \mathcal{W}$ and $p \in \mathcal{V}$ such that

$$\langle \nabla \mathbf{u}, \nabla \mathbf{v} \rangle - \int_{\partial\Omega} (\nabla \mathbf{u}) \mathbf{n} \cdot \mathbf{v} \, ds - \langle p, \nabla \cdot \mathbf{v} \rangle + \int_{\partial\Omega} p \mathbf{v} \cdot \mathbf{n} \, ds = \langle \mathbf{f}, \mathbf{v} \rangle \quad \forall \mathbf{v} \in \mathcal{W}, \quad (10.90a)$$

$$\langle -\nabla \cdot \mathbf{u}, q \rangle = 0 \quad \forall q \in \mathcal{V}, \quad (10.90b)$$

where, again, $\langle \cdot, \cdot \rangle$ denotes the standard L^2 inner product (on scalar-, vector-, or tensor-valued functions, as needed); $\mathbf{n}$ is the outward unit normal to the boundary, $\partial\Omega$; and integration by parts was performed on each of the two terms in the first equation. We note that $\langle \nabla \mathbf{u}, \nabla \mathbf{v} \rangle$ is now the L^2 inner product of a tensor (discussed above in Remark 10.20), defined by summing the componentwise inner products over the rows and columns of the tensors.

Remark 10.21: Boundary conditions

Considering the weak form in equation (10.90), we see that two types of boundary conditions can arise in the Stokes system, either by imposing values of $(\nabla \mathbf{u} - p\mathbf{I})\mathbf{n}$ on the boundary (leading to *natural* boundary conditions, where we include the boundary integrals in the weak form) or by imposing boundary conditions on $\mathbf{u}$ so that $\mathbf{v} \cdot \mathbf{n} = 0$ (leading to *essential* boundary conditions, where the boundary integrals are eliminated from the weak form). Note, however, that we have two choices in how we might impose the essential boundary conditions, by either requiring that values of $\mathbf{u} \cdot \mathbf{n}$ be given (allowing us to fix $\mathbf{v} \cdot \mathbf{n} = 0$ on the boundary) or requiring that values of $\mathbf{u}$ be given (allowing us to fix $\mathbf{v} = \mathbf{0}$ on the boundary and still eliminate the boundary integrals). Both of these can arise naturally. For example, we may be modeling a flow where part of the boundary represents a natural inflow or outflow from the domain, where the normal component of the velocity is specified. On the other hand, part of the boundary may be a rigid wall, where frictional forces lead to a so-called *no-slip* boundary condition that $\mathbf{u} = \mathbf{0}$ along the boundary.

For simplicity of the analysis, we now assume $\mathbf{u} = \mathbf{0}$ boundary conditions everywhere on $\partial\Omega$, although other boundary conditions are possible, following ideas similar to those in

Section 8.4.3. This yields the following mixed formulation of the Stokes equations: find $\mathbf{u} \in \mathcal{W}$ and $p \in \mathcal{V}$ such that

$$\langle \nabla \mathbf{u}, \nabla \mathbf{v} \rangle - \langle p, \nabla \cdot \mathbf{v} \rangle = \langle \mathbf{f}, \mathbf{v} \rangle \quad \forall \mathbf{v} \in \mathcal{W}, \quad (10.91a)$$

$$\langle -\nabla \cdot \mathbf{u}, q \rangle = 0 \quad \forall q \in \mathcal{V}. \quad (10.91b)$$

We write this in the generalized saddle-point form given in equation (10.55) by defining the bilinear and linear forms

$$a(\mathbf{u}, \mathbf{v}) = \langle \nabla \mathbf{u}, \nabla \mathbf{v} \rangle \quad \text{for } \mathbf{u}, \mathbf{v} \in \mathcal{W}, \quad (10.92a)$$

$$b(\mathbf{u}, q) = -\langle \nabla \cdot \mathbf{u}, q \rangle \quad \text{for } \mathbf{u} \in \mathcal{W}, q \in \mathcal{V}, \quad (10.92b)$$

$$g(\mathbf{v}) = \langle \mathbf{f}, \mathbf{v} \rangle \quad \text{for } \mathbf{v} \in \mathcal{W}, \quad (10.92c)$$

$$f(q) = 0 \quad \text{for } q \in \mathcal{V}. \quad (10.92d)$$

It is useful to recognize that, up to the minus sign, the bilinear form, $b(\cdot, \cdot)$, is the same as it is for the mixed formulation of the Poisson equation. From these forms, the natural function spaces to choose are $\mathcal{W} = \mathbf{H}_0^1(\Omega)$ and $\mathcal{V} = L^2(\Omega)$, where the subscript 0 on the $\mathbf{H}^1$ space indicates the Dirichlet boundary conditions.

Remark 10.22: Modified $L^2(\Omega)$

Recall that the well-posedness theory and inf-sup condition rely on the operator B induced by $b(\cdot, \cdot)$ mapping $\mathcal{W}$ onto $\mathcal{V}$ appropriately. For mixed Poisson, the divergence operator naturally maps $\mathbf{H}(\text{div}, \Omega)$ onto $L^2(\Omega)$. In the case of Stokes flow, however, this does not hold for $\mathbf{H}_0^1(\Omega)$ when we have Dirichlet boundary conditions on all of $\partial\Omega$. Indeed, given $\mathbf{u} \in \mathbf{H}_0^1(\Omega)$, $b(\mathbf{u}, 1) = -\int_{\Omega} \nabla \cdot \mathbf{u} \, dx = -\int_{\partial\Omega} \mathbf{u} \cdot \mathbf{n} \, ds = 0$, even though the constant function $1 \in L^2(\Omega)$. To avoid this causing problems, we modify the space by choosing

$$\mathcal{V} = L_0^2(\Omega) = \left\{ p \in L^2(\Omega) \mid \int_{\Omega} p \, dx = 0 \right\}. \quad (10.93)$$

Without this added condition, the same theoretical development as below would result in a pressure solution that is unique only up to an arbitrary constant shift. By constraining p to have zero mean, we remove the nullspace from the pressure space.

We also note that in the case of nonhomogeneous Dirichlet boundary conditions on all of $\partial\Omega$, where $\mathbf{u} = \mathbf{u}_b$ on $\partial\Omega$, we must satisfy the compatibility condition that $\int_{\partial\Omega} \mathbf{u}_b \cdot \mathbf{n} \, ds = 0$ in order for the theory below to be valid.

Well-posedness of the continuous Stokes problem

Considering equation (10.91) and the corresponding bilinear forms, our goal now is to show that the well-posedness conditions hold (see [108, Thm. 5.1]). Again, the assumption that $\mathbf{f} \in \mathbf{L}^2(\Omega)$ is enough to guarantee that $g(\mathbf{v})$ is a bounded linear functional on $\mathbf{H}_0^1(\Omega)$. We next consider the continuity conditions on $a(\cdot, \cdot)$ and $b(\cdot, \cdot)$. A useful tool in this is the inequality that $\|\nabla \cdot \mathbf{u}\| \leq d\|\nabla \mathbf{u}\|$ when $\Omega \subset \mathbb{R}^d$, which arises by simple use of the triangle inequality on $\|\nabla \cdot \mathbf{u}\|$.

Continuity of $a(\cdot, \cdot)$ on $\mathcal{W}$:

$$\langle \nabla \mathbf{u}, \nabla \mathbf{v} \rangle \leq \|\nabla \mathbf{u}\| \|\nabla \mathbf{v}\| \quad (10.94a)$$

$$\begin{aligned} &\leq (\|\mathbf{u}\|^2 + \|\nabla \mathbf{u}\|^2)^{1/2} (\|\mathbf{v}\|^2 + \|\nabla \mathbf{v}\|^2)^{1/2} = \|\mathbf{u}\|_1 \|\mathbf{v}\|_1 \\ &\Rightarrow a(\mathbf{u}, \mathbf{v}) \leq \|\mathbf{u}\|_{\mathcal{W}} \|\mathbf{v}\|_{\mathcal{W}} \quad \forall \mathbf{u}, \mathbf{v} \in \mathcal{W}. \end{aligned} \quad (10.94b)$$

Continuity of $b(\cdot, \cdot)$ on $\mathcal{W} \times \mathcal{V}$:

$$| - \langle \nabla \cdot \mathbf{u}, q \rangle | \leq \| \nabla \cdot \mathbf{u} \| \| q \| \quad (10.95a)$$

$$\begin{aligned} &\leq d \| \nabla \mathbf{u} \| \| q \| \leq d (\| \mathbf{u} \|^2 + \| \nabla \mathbf{u} \|^2)^{1/2} \| q \| = d \| \mathbf{u} \|_1 \| q \| \\ &\Rightarrow b(\mathbf{u}, q) \leq d \| \mathbf{u} \|_{\mathcal{W}} \| q \|_{\mathcal{V}} \quad \forall \mathbf{u} \in \mathcal{W}, q \in \mathcal{V}. \end{aligned} \quad (10.95b)$$

Next, we consider the coercivity of $a(\cdot, \cdot)$, which follows the same proof as for determining coercivity of the Poisson problem on $H_0^1(\Omega)$ in Example 8.30. Here, we use the Poincaré inequality (Korn's inequality for the symmetric gradient case), with constant $C > 0$, and obtain coercivity on *all* of $\mathcal{W}$ (including on $\widehat{\mathcal{W}}$). Since we have coercivity on all of $\mathcal{W}$, coercivity will hold on any subspace of $\mathcal{W}$, including *any* conforming finite-element space.

Coercivity of $a(\cdot, \cdot)$ on $\mathcal{W}_0$:

$$\| \nabla \mathbf{u} \|^2 \geq \frac{1}{C^2 + 1} (\| \mathbf{u} \|^2 + \| \nabla \mathbf{u} \|^2) \quad (10.96a)$$

$$\Rightarrow a(\mathbf{u}, \mathbf{u}) \geq \frac{1}{C^2 + 1} \| \mathbf{u} \|_{\mathcal{W}}^2 \quad \forall \mathbf{u} \in \mathcal{W}. \quad (10.96b)$$

The final piece is the inf-sup condition, established in [55], showing that the divergence operator mapping $\mathbf{H}_0^1(\Omega)$ to $L_0^2(\Omega)$ is surjective. With appropriate assumptions on the domain Ω , we again use elliptic regularity to say that for any $q \in L_0^2(\Omega)$, there is a $\phi \in H^2(\Omega) \cap L_0^2(\Omega)$ that solves $-\nabla \cdot \nabla \phi = q$ with homogeneous Neumann boundary conditions, $\nabla \phi \cdot \mathbf{n} = 0$ on $\partial\Omega$. Additionally, take $\mathbf{u} = \nabla \phi \in \mathbf{H}_0^1(\Omega)$. Note that this problem is well posed only because of the integral constraint on $L_0^2(\Omega)$, which is required when we consider the problem with Neumann boundary conditions on all of $\partial\Omega$. By construction, it must be the case that $\mathbf{u} \cdot \mathbf{n} = \nabla \phi \cdot \mathbf{n} = 0$. We omit a technical argument that also establishes that the tangential component of $\mathbf{u}$ vanishes on the boundary (see [108, Cor. 2.4]), giving $\mathbf{u} = \mathbf{0}$ on $\partial\Omega$. Then,

$$b(\mathbf{u}, q) = -\langle \nabla \cdot \mathbf{u}, q \rangle = \langle -\nabla \cdot \nabla \phi, q \rangle = \langle q, q \rangle \quad (10.97a)$$

$$= \| q \|_{\mathcal{V}}^2, \quad (10.97b)$$

$$\| \mathbf{u} \|_{\mathcal{W}}^2 = \| \nabla \mathbf{u} \|^2 + \| \mathbf{u} \|^2 \leq \| \phi \|_2^2 \quad (10.97c)$$

$$\leq \beta \| q \|_{\mathcal{V}}^2, \quad (10.97d)$$

where elliptic regularity of the Poisson problem in $H^2(\Omega)$ is used with some constant $\beta > 0$. Consequently, for any $q \in \mathcal{V}$ and $c_0 = \frac{1}{\sqrt{\beta}}$,

$$c_0 \| q \|_{\mathcal{V}} \leq \frac{b(\mathbf{u}, q)}{\| \mathbf{u} \|_{\mathcal{W}}} \leq \sup_{\mathbf{v} \in \mathcal{W}} \frac{b(\mathbf{v}, q)}{\| \mathbf{v} \|_{\mathcal{W}}}. \quad (10.98)$$

Since this holds for all $q \in \mathcal{V}$, it also holds for the infimum, and we get that $b(\mathbf{v}, q)$ is weakly coercive over $\mathbf{H}_0^1(\Omega) \times L_0^2(\Omega)$. Thus, the mixed formulation for Stokes is well posed.

10.3.4 ■ Mixed finite-element discretization of the Stokes equations

Now that we have determined that the Stokes mixed variational form is well posed in $\mathbf{H}_0^1(\Omega) \times L_0^2(\Omega)$, we take a closer look at which finite-element spaces might satisfy a corresponding discrete inf-sup condition. As discussed above, due to the form of $a(\cdot, \cdot)$, it is automatically coercive on the discrete space $\mathcal{W}^h$, provided that $\mathcal{W}^h \subset \mathbf{H}_0^1(\Omega)$, which is always the case in the discussion below. Similarly, both $a(\cdot, \cdot)$ and $b(\cdot, \cdot)$ are always continuous on the discrete spaces in our setting. Thus, we only need to check discrete inf-sup conditions for Stokes for each pair of spaces. First, we start with what might seem to be a natural choice.

$P_1(\Omega^h) - P_1(\Omega^h)$ for Stokes

We consider the choice of piecewise linears for both spaces, using the d -dimensional product space of piecewise linear vector fields, $\mathcal{W}^h = P_1(\Omega^h)$, and the scalar piecewise linear space, $\mathcal{V}^h = P_1(\Omega^h)$ with the mean-value zero condition for p . This pair of spaces can be shown to violate the inf-sup condition just as it does for the mixed Poisson problem in the previous section. In particular, there exists a nonzero piecewise linear function, q (with mean value zero), for which $\int_{\Omega} (\nabla \cdot \mathbf{v}) q \, dx = 0$ for all piecewise linear vector fields, $\mathbf{v}$, so that $b(\mathbf{v}, q) = 0$ for all $\mathbf{v} \in P_1(\Omega^h)$. This counterexample (the same one used for mixed Poisson) is often called a “spurious pressure mode” and is a nonconstant element of the kernel of the operator derived from $b(\cdot, \cdot)$. This instability can result in approximate solutions with highly oscillatory pressure errors, similar to those seen for the collocated finite-difference discretization in Section 10.1. A possible issue with this space is that the velocity space is not “big enough” compared to the pressure space to satisfy the incompressibility constraint. We next consider reducing the order of polynomials considered for the pressure space to see if this resolves the instability.

$P_1(\Omega^h) - P_0^-(\Omega^h)$ for Stokes

Here, we again consider piecewise linears for the velocity space but instead consider (discontinuous) piecewise constant functions for the pressure space $\mathcal{V}^h = P_0^-(\Omega^h)$. Unfortunately, while we are using a lower order space than before, the P_0^- space is too large for the discrete inf-sup condition to hold. To see this, we use a counting argument to show that the discrete divergence operator cannot possibly be full rank. Consider a two-dimensional example using a uniform triangulation of the domain $\Omega = [0, 1] \times [0, 1]$, split into an $n_x \times n_y$ grid with $n_x = n_y$. This gives a grid with $n_x + 1$ vertices in each direction, giving $(n_x + 1)^2$ vertices in total, splitting the resulting $n_x \times n_x$ mesh of squares into $2n_x^2$ triangular elements. Thus, the total number of degrees of freedom for the velocity space given by piecewise linear vector fields with Dirichlet boundary conditions is $2(n_x - 1)^2$, since there are two components of $\mathbf{u}$ and we have eliminated the boundary degrees of freedom. The pressure space, on the other hand, takes a single value on each of the $2n_x^2$ elements but is constrained to have zero mean, leading to $2n_x^2 - 1$ degrees of freedom. Subtracting the number of velocity degrees of freedom from the pressure degrees of freedom, we get

$$2n_x^2 - 1 - 2(n_x - 1)^2 = 2n_x^2 - 1 - 2n_x^2 + 4n_x - 2 = 4n_x - 3, \quad (10.99)$$

which is positive as long as $n_x \geq 1$. As a result, the discrete divergence operator associated with $b(\cdot, \cdot)$ has more rows (pressure degrees of freedom) than columns (velocity degrees of freedom) and cannot possibly have full row rank (or be a surjective operator), and the inf-sup condition cannot hold. Again, this yields “spurious pressure modes” in the kernel of $b(\cdot, \cdot)$.

One solution to “fix” the failings of the previous two examples is to enlarge the velocity space further, in order to not *underconstrain* the divergence condition. This leads to the well-known $P_2(\Omega^h) \times P_0^-(\Omega^h)$ mixed space in two dimensions.

$P_2(\Omega^h) - P_0^-(\Omega^h)$ in $\mathbb{R}^2$ for Stokes

Since the piecewise constant pressure space is too large to give an inf-sup stable pair of elements for the Stokes equations with piecewise linear velocities, we now consider the space of piecewise quadratic vector fields for the velocities, $\mathcal{W}^h = P_2(\Omega^h)$, keeping piecewise constants for pressure, $\mathcal{V}^h = P_0^-(\Omega^h)$, and restricting ourselves to two-dimensional domains. As we see below, these do produce a well-posed pairing, and the discrete inf-sup condition is shown via construction of a Fortin operator (see Theorem 10.18). However, to do this, we make use of the Cl  ment interpolant, which we now introduce.

Remark 10.23: The Clément interpolant

In Section 4.3, we introduced the notion of the finite-element interpolant, Definition 4.13. This is also used throughout Chapter 8, as well as in Section 10.2, in order to obtain approximation properties for the various finite-element spaces that are introduced. So far, each of the presented interpolation operators for the various cases considered has been applied to smooth functions—e.g., functions in $H^2(\Omega)$ or smoother spaces in Chapter 8. However, some applications consider discontinuous or nonsmooth functions, and the standard interpolation operators, as defined in Definition 4.13, are insufficient.

As we see below, when trying to construct a Fortin operator using $H^1(\Omega)$ -conforming finite elements, we end up trying to apply an interpolation bound on functions that are only guaranteed to be in $H^1(\Omega)$ and not a smoother space. For such functions, we cannot apply Theorem 8.35, since this would correspond to $k = 0$ (in the notation used there), violating the assumption that $k + 1 - n/2 > 0$ in $n > 1$ dimensions. While the standard finite-element interpolant is well defined for any function in $H^1(\Omega)$, the error bound in Theorem 8.35 relies on additional smoothness properties to prove that $\|u - I^h u\|_s$ is small for $s = 0$ or 1. Clément realized that an alternative construction of the interpolant allows these results to be extended to prove an $L^2(\Omega)$ error bound for the interpolant of functions in $H^1(\Omega)$ [73]. To do this, instead of using point evaluation to define the basis function coefficients, a local $L^2(\Omega)$ projection is used over the support of each basis function. For a triangular mesh Ω^h , we define the Clément interpolation operator, $I_C : H^1(\Omega) \rightarrow P_k(\Omega^h)$, by $I_C(v) = \sum_i v_i \phi_i$, with a local projection used to compute v_i . For every nodal basis function $\phi_i(\mathbf{x})$ in $P_k(\Omega^h)$, v_i is chosen by computing the L^2 projection of v into $P_k(S_i)$, where S_i is the set of elements over which $\phi_i(\mathbf{x})$ has nonzero support. Then, v_i is taken to equal the coefficient of $\phi_i(\mathbf{x})$ in this local projection. With this definition, for any $v \in H^\ell(\Omega)$ with $\ell \leq k + 1$, we can prove that there exist constants $c_1, c_2 > 0$, with

$$\|v - I_C(v)\|_m \leq c_1 h^{\ell-m} \|v\|_\ell \quad \forall 0 \leq m \leq \ell, \quad (10.100a)$$

$$\|I_C(v)\|_m \leq c_2 \|v\|_m, \quad 0 \leq m \leq 1. \quad (10.100b)$$

Of particular interest is the case where $\ell = 1$ and $m = 0$, where we now have the bound $\|v - I_C(v)\|_0 \leq c_1 h \|v\|_1$, as we would hope for (but do not get) from the classical interpolation result in Theorem 8.35.

For more rigorous details and definitions, we refer to Clément's original paper [73] or the finite-element text by Braess [48] (and references therein). In the following, we use one special type of Clément interpolant to prove the existence of a Fortin operator for the $P_2(\Omega^h) \times P_0^-(\Omega^h)$ approximation of the Stokes equations.

To construct the Fortin operator, $\Pi_F : \mathcal{W} \rightarrow \mathcal{W}^h$, we note that the first property required, equation (10.72), can be written as (changing notation to use $\mathbf{v}$ and q^h rather than $\mathbf{w}$ and v^h)

$$\int_{\Omega^h} \nabla \cdot (\Pi_F(\mathbf{v}) - \mathbf{v}) q^h d\mathbf{x} = 0 \quad \forall q^h \in \mathcal{V}^h, \quad (10.101a)$$

$$\sum_{\tau \in \mathcal{T}^h} \int_{\tau} \nabla \cdot (\Pi_F(\mathbf{v}) - \mathbf{v}) q_{\tau} d\mathbf{x} = 0 \quad \forall q_{\tau}, \quad (10.101b)$$

where q_{τ} is the value of the restriction of piecewise constant function $q^h(\mathbf{x})$ to element τ . Thus, the condition on Π_F is verified by checking its restriction to each element. Moreover, we apply the divergence theorem to rewrite the condition to be satisfied to depend only on the boundary of

each element, namely,

$$q_\tau \int_\tau \nabla \cdot (\Pi_F(\mathbf{v}) - \mathbf{v}) d\mathbf{x} = 0 \quad (10.102a)$$

$$\Rightarrow \int_{\partial\tau} (\Pi_F(\mathbf{v}) - \mathbf{v}) \cdot \mathbf{n}_\tau d\mathbf{s} = 0 \quad (10.102b)$$

for all elements, $\tau \in \mathcal{T}^h$, where $\mathbf{n}_\tau$ is the outward normal to the boundary of the element. In the two-dimensional case considered here, the boundary consists of the edges of the triangle, so it is enough to check that the condition holds on each edge. In a way, this makes moving from the $\mathbf{P}_1(\Omega^h)$ space to $\mathbf{P}_2(\Omega^h)$ natural, since we introduce edge degrees of freedom in $\mathbf{P}_2(\Omega^h)$. In three dimensions, adding bubble functions to the faces achieves similar results but does not correspond to the $\mathbf{P}_2(\Omega^h)$ space, since we need to consider $\mathbf{P}_3(\Omega^h)$ in order to introduce degrees of freedom on the faces. Nevertheless, for the two-dimensional case considered here, we focus on the boundary integral on each edge of the element, $e \in \partial\tau$, seeking an operator that satisfies

$$\int_e \Pi_F(\mathbf{v}) \cdot \mathbf{n}_e d\mathbf{s} = \int_e \mathbf{v} \cdot \mathbf{n}_e d\mathbf{s}. \quad (10.103)$$

We construct our Fortin operator in two steps. First, we construct an operator, $\Pi^h : H^1(\Omega) \rightarrow \mathbf{P}_2(\Omega^h)$, that satisfies equation (10.103) by defining Π^h so that

$$\Pi^h(\mathbf{v}(\mathbf{x})) = \mathbf{0} \quad \text{for all vertices } \mathbf{x}, \quad (10.104a)$$

$$\int_e \Pi^h(\mathbf{v}) d\mathbf{s} = \int_e \mathbf{v} d\mathbf{s} \quad \text{for all edges } e. \quad (10.104b)$$

It is easy to construct a function in $\mathbf{P}_2(\Omega^h)$ that satisfies equation (10.104a). Simultaneously satisfying equation (10.104b) is possible by noting that for the standard basis functions for $\mathbf{P}_2(\Omega^h)$, when equation (10.104a) is satisfied, each component of the integral of $\Pi^h(\mathbf{v})$ over edge e is determined only by the basis function associated with the component and the midpoint of e and is equal to the coefficient of the basis function (that we are free to choose) times the integral of the basis function. As a result, for any $\mathbf{v}$, it is possible to construct Π^h so that equation (10.104) is satisfied.

Unsurprisingly, this Π^h does not itself satisfy the conditions for a Fortin operator, because the other property in Definition 10.17 does not hold, namely, $\|\Pi_F(\mathbf{v})\|_{\mathcal{W}} \leq C_F \|\mathbf{v}\|_{\mathcal{W}}$. In fact, we can show (see [43]) that there is a constant, $c_3 > 0$, for which

$$\|\Pi^h(\mathbf{v})\|_1 \leq c_3(h^{-1}\|\mathbf{v}\| + \|\nabla \mathbf{v}\|). \quad (10.105)$$

We overcome this by introducing a Clément interpolation operator, $\Pi_C : \mathbf{H}_0^1(\Omega) \rightarrow \mathcal{W}^h$ (see Remark 10.23), that satisfies

$$\|\mathbf{v} - \Pi_C(\mathbf{v})\|_0 \leq c_1 h \|\mathbf{v}\|_1, \quad \|\mathbf{v} - \Pi_C(\mathbf{v})\|_1 \leq c_1 \|\mathbf{v}\|_1, \quad \|\Pi_C(\mathbf{v})\|_1 \leq c_2 \|\mathbf{v}\|_1 \quad (10.106)$$

for some constants $c_1, c_2 > 0$. Then, we define the Fortin operator as

$$\Pi_F(\mathbf{v}) = \Pi^h(I - \Pi_C)(\mathbf{v}) + \Pi_C(\mathbf{v}). \quad (10.107)$$

To verify equation (10.103), we have

$$\int_e \Pi_F(\mathbf{v}) d\mathbf{s} = \int_e \Pi^h(\mathbf{v} - \Pi_C(\mathbf{v})) d\mathbf{s} + \int_e \Pi_C(\mathbf{v}) d\mathbf{s} \quad (10.108a)$$

$$= \int_e (\mathbf{v} - \Pi_C(\mathbf{v})) d\mathbf{s} + \int_e \Pi_C(\mathbf{v}) d\mathbf{s} \quad (10.108b)$$

$$= \int_e \mathbf{v} d\mathbf{s} \quad \text{for all edges } e, \quad (10.108c)$$

and we can dot all terms with the edge-normal vector, $\mathbf{n}_e$, to prove equation (10.103), which implies that the first Fortin operator condition, equation (10.72), is satisfied. For the second condition, equation (10.73),

$$\|\Pi_F(\mathbf{v})\|_{\mathcal{W}} = \|\Pi^h(I - \Pi_C)(\mathbf{v}) + \Pi_C(\mathbf{v})\|_1 \quad (10.109a)$$

$$\leq c_3(h^{-1}\|(I - \Pi_C)(\mathbf{v})\|_0 + |(I - \Pi_C)(\mathbf{v})|_1) + c_2\|\mathbf{v}\|_1 \quad (10.109b)$$

$$\leq c_3c_1\|\mathbf{v}\| + c_3\|(I - \Pi_C)(\mathbf{v})\|_1 + c_3\|\mathbf{v}\|_1 \quad (10.109c)$$

$$\leq c_F\|\mathbf{v}\|_1 = c_F\|\mathbf{v}\|_{\mathcal{W}}. \quad (10.109d)$$

This gives Π_F as a Fortin operator on $\mathbf{P}_2(\Omega^h) \times P_0^-(\Omega^h)$ and implies that the discrete inf-sup condition holds.

In general, using this process of constructing a Fortin operator of the form

$$\Pi_F(\mathbf{v}) = \Pi_2(I - \Pi_1)(\mathbf{v}) + \Pi_1(\mathbf{v}) \quad (10.110)$$

is a standard approach. Most often, $\Pi_1(\mathbf{v})$ is the appropriate Cl  ment interpolant and Π_2 is constructed depending on the specific spaces being considered, as Π^h is constructed in this $\mathbf{P}_2(\Omega^h) \times P_0^-(\Omega^h)$ example.

Finally, we note that while this pair of spaces uses $\mathbf{P}_2(\Omega^h)$ for the velocity space, we do not see the convergence of the velocity approximation in the $\mathbf{H}^1(\Omega)$ energy norm consistent with Theorem 8.35. This is because the pressure is only approximated by piecewise constant functions and, therefore, lowers the order of the overall error estimate presented in Remark 10.19. Thus, we expect $\mathcal{O}(h)$ convergence of both $\mathbf{u}$ in the $\mathbf{H}^1(\Omega)$ norm and p in the $L^2(\Omega)$ norm. However, we do obtain an extra order of convergence for $\mathbf{u}$ in the $\mathbf{L}^2(\Omega)$ norm (to $\mathcal{O}(h^2)$), consistent with the elliptic regularity arguments discussed in Section 8.3.1. There are several techniques to regain the “optimal” quadratic convergence (or closer to it) that one would expect for $\mathbf{P}_2(\Omega^h)$ elements, though, including augmented Lagrangian approaches (cf. [44]) and using the Taylor–Hood element described next.

Remark 10.24: $\mathbf{P}_2(\Omega^h) \times P_0^-(\Omega^h)$ in three dimensions

While we are able to show that the $\mathbf{P}_2(\Omega^h) \times P_0^-(\Omega^h)$ finite-element pair is inf-sup stable in two dimensions, it is not known if this also holds in three dimensions. As alluded to earlier, the $\mathbf{P}_2(\Omega^h)$ element contains degrees of freedom on the vertices and edges, but not on the faces of the tetrahedron needed to build the Fortin operator constructed here. Thus, other examples which still keep the pressure in the piecewise constant space are used in practice, such as the face bubble augmented element, the MINI element (element-based bubble augmentation of $\mathbf{P}_1(\Omega^h)$), $\mathbf{P}_3(\Omega^h) \times P_0^-(\Omega^h)$, or even $\mathbf{Q}_2(\Omega^h) \times Q_0^-(\Omega^h)$ on hexahedral elements. For all these, there are known proofs of the discrete inf-sup condition.

$\mathbf{P}_{k+1}(\Omega^h) - \mathbf{P}_k(\Omega^h)$ (Taylor–Hood) for Stokes

As a final example of inf-sup stable mixed elements for Stokes, we consider a well-known, often-used finite-element pair. This space, known as the Taylor–Hood element, puts both the velocity and the pressure in continuous piecewise polynomial spaces. Thus, we choose $\mathcal{W}^h = \mathbf{P}_{k+1}(\Omega^h)$ and $\mathcal{V}^h = P_k(\Omega^h)$ for $k \geq 1$. The advantage of this approach is that we regain the expected “optimal” order of convergence for the velocity approximation, namely, $\mathcal{O}(h^{k+1})$ in the $\mathbf{H}^1(\Omega)$ norm. The proof of the discrete inf-sup condition is somewhat technical and tedious, so we omit it here. For complete details, see [57, 69].

The advantage of this element pair, in addition to the improved order of convergence, is mostly the ease of implementation, since simple elements are used. However, the disadvantage is that we do not obtain pointwise divergence-free velocity approximations, nor do we obtain elementwise mass conservation for the incompressibility constraint. We note that $\mathbf{P}_2(\Omega^h) \times P_0^-(\Omega^h)$ also does not achieve pointwise divergence-free approximations, but it does conserve mass continuity on average on the elements. Several modifications of the elements can be implemented, though, to regain conservation; see [140] for more details in this direction.

Numerical example

To test the above mixed methods, we consider a test problem with a manufactured solution. We use $\Omega = [0, 1] \times [0, 1]$ for simplicity and Dirichlet boundary conditions for $\mathbf{u}$ on all boundaries. Again, we use a uniform triangular mesh consisting of a tensor product of $n_x + 1$ by $n_y + 1$ vertices with $n_x = n_y$. The resulting $n_x \times n_x$ squares are then divided into two triangles. Thus, $h = \mathcal{O}(n_x^{-1})$. The exact (divergence-free) solution is given by

$$\mathbf{u} = \begin{bmatrix} \sin(\pi x) \cos(\pi y) \\ -\cos(\pi x) \sin(\pi y) \end{bmatrix}, \quad (10.111a)$$

$$p = \frac{1}{2} - x. \quad (10.111b)$$

In Figures 10.3.3 and 10.3.4, we see the convergent results for the $\mathbf{P}_2(\Omega^h) \times P_0^-(\Omega^h)$ and the Taylor–Hood elements of varying degree, obtaining the expected rates of convergence.

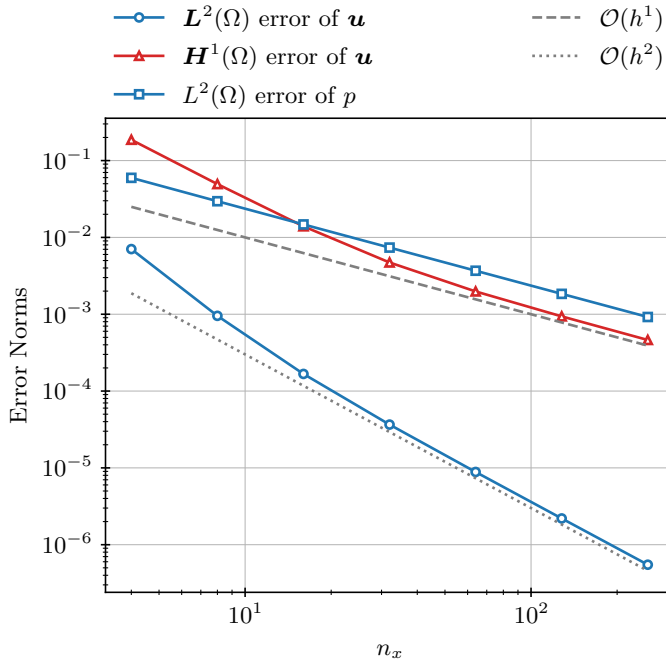


Figure 10.3.3. Error norms in $L^2(\Omega)$ and $H^1(\Omega)$ versus mesh size for a simulation of the Stokes problem defined in equation (10.91) using $\mathbf{P}_2(\Omega^h) \times P_0^-(\Omega^h)$ elements. Here, $n_x + 1$ is the number of vertices of the triangular mesh in each direction and $h = \mathcal{O}(n_x^{-1})$.

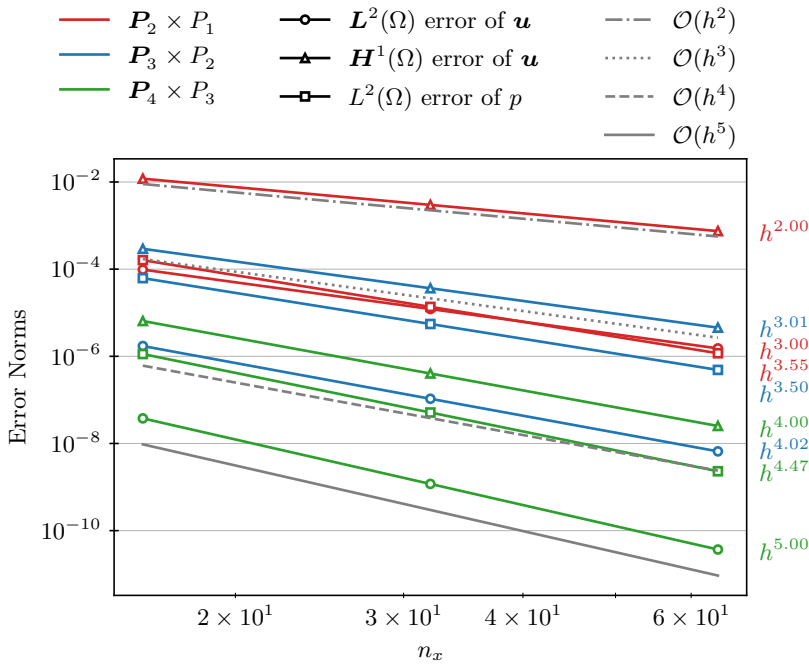


Figure 10.3.4. Error norms in $L^2(\Omega)$ and $H^1(\Omega)$ versus mesh size for a simulation of the Stokes problem defined in equation (10.91) using Taylor-Hood elements of different orders. Here, $n_x + 1$ is the number of vertices of the triangular mesh in each direction and $h = O(n_x^{-1})$.

10.4 ■ Least-squares finite-element methods

Objectives

In Section 8.4, we saw that proving coercivity can be complicated when considering more general scalar PDEs. Similarly, in Section 10.3, we saw that proving an inf-sup condition at both the continuous and discrete levels was needed to ensure well-posedness of standard finite-element discretizations for systems of PDEs. An alternative approach, particularly helpful in cases where proving (weak) coercivity is not straightforward, is to use a least-squares reformulation of the governing equations. In this section, you will study this approach and develop discretizations for elliptic (and other) PDEs that have some attractive properties. To do this, you will

- derive a least-squares functional associated with a PDE and discuss the useful properties, such as symmetry, that result;
- introduce the first-order systems least-squares (FOSLS) approach, transforming the PDE into a first-order system and, thus, reducing the regularity requirements on the finite-element spaces used for discretization;
- discuss a posteriori error estimates that arise; and
- demonstrate the FOSLS method on the Stokes equations, showing both advantages and disadvantages of the approach.

In the standard Galerkin approach to solving an elliptic PDE,

$$Lu = f, \quad (10.112)$$

we multiply by a test function, v , and integrate over the domain to obtain the weak form,

$$\langle Lu, v \rangle = \langle f, v \rangle \quad \forall v \in \mathcal{V}, \quad (10.113)$$

where $\mathcal{V}$ is some Hilbert space. Typically, we define the bilinear form for the PDE to be $a(u, v) = \langle Lu, v \rangle$ or some variation on this form, derived by integrating by parts and applying known boundary conditions. Given continuity and coercivity of this bilinear form, we say that $a(\cdot, \cdot)$ is $\mathcal{V}$ -elliptic. One way to view this condition is that $a(\cdot, \cdot)$ induces an inner product and norm on $\mathcal{V}$:

$$c_0 \|u\|_{\mathcal{V}}^2 \leq a(u, u) \leq c_1 \|u\|_{\mathcal{V}}^2 \quad (10.114a)$$

$$\Rightarrow a(u, u) \simeq \|u\|_{\mathcal{V}}^2. \quad (10.114b)$$

From here, the theorem of Lax–Milgram, Theorem 8.27, tells us that there is a unique solution to this weak form of the problem.

Alternatively, if the operator, L , is nonsingular, then minimization of the residual of the PDE can be done in a least-squares sense. To see this, define $G : \mathcal{V} \rightarrow \mathbb{R}$ to be the *least-squares functional*

$$G(u) = \|Lu - f\|^2 = \langle Lu - f, Lu - f \rangle, \quad (10.115)$$

and note that any solution to the PDE must be a minimizer of G , since $G(u) \geq 0$ by its construction, and any solution of $Lu = f$ yields $G(u) = 0$. On the other hand, minimizers of G can be characterized by the *Euler–Lagrange equations* of the functional. We understand these equations by first thinking of simpler cases. The extrema of a scalar function, $g(x)$, occur when $g'(x) = 0$. In multiple dimensions, there are two ways of characterizing extrema of a function $g(x)$, either as points where the gradient is zero, $\nabla g(x) = \mathbf{0}$, or as points where the directional derivative of g is zero in all directions, $\frac{\partial g}{\partial \mathbf{p}} = (\nabla g) \cdot \mathbf{p} = 0$ for all $\mathbf{p}$. The Euler–Lagrange equations are the generalization of this second characterization to functionals (which are functions, like G , that map from a vector space to the real numbers). We define the Gâteaux derivative of G to be the directional derivative of $G(u)$ in a given direction, $v \in \mathcal{V}$:

$$dG(u)[v] := \lim_{\gamma \rightarrow 0} \frac{G(u + \gamma v) - G(u)}{\gamma}. \quad (10.116)$$

Extreme values of $G(u)$ are characterized by requiring that $dG(u)[v] = 0$ for all $v \in \mathcal{V}$. Intuitively this makes sense, since if, at a point $u \in \mathcal{V}$, there is a direction $v \in \mathcal{V}$ for which $dG(u)[v] \neq 0$, then we can decrease the value of $G(u)$ by taking a small step in the direction of v and considering $G(u + \gamma v)$ for small (positive or negative) $\gamma \in \mathbb{R}$. As with finding extreme values of a function over $\mathbb{R}^n$, there are several technical details both with whether or not points where $dG(u)[v] = 0$ for all $v \in \mathcal{V}$ are extreme values of $G(u)$ or saddle points and with distinguishing minimizers from maximizers, but we omit discussion of these here (see, for example, [23, Sec. 5.3]).

For the functional defined in equation (10.115), we have

$$G(u + \gamma v) = \langle L(u + \gamma v) - f, L(u + \gamma v) - f \rangle \quad (10.117a)$$

$$= \langle Lu - f, Lu - f \rangle + 2\gamma \langle Lu - f, Lv \rangle + \gamma^2 \langle Lv, Lv \rangle, \quad (10.117b)$$

so

$$dG(u)[v] = 2\langle Lu - f, Lv \rangle = 0. \quad (10.118)$$

Thus, finding the extreme values of $G(u)$ over $u \in \mathcal{V}$ is equivalent to solving

$$\langle Lu, Lv \rangle = \langle f, Lv \rangle \quad \forall v \in \mathcal{V}. \quad (10.119)$$

This gives us a weak form with bilinear form $a(u, v) = \langle Lu, Lv \rangle$. Note that we can also directly find the extreme values of the finite-dimensional version of G , mapping from a discrete subspace, $\mathcal{V}^h \subset \mathcal{V}$, with

$$\min_{u^h \in \mathcal{V}^h} \|Lu^h - f\|^2 \rightarrow \langle Lu^h, Lv^h \rangle = \langle f, Lv^h \rangle \quad \forall v^h \in \mathcal{V}^h. \quad (10.120)$$

If the bilinear form, $\langle Lu, Lv \rangle$, associated with these problems is continuous and coercive in a norm on $\mathcal{V}$, then the Lax–Milgram theorem tells us that both the continuum and discrete problems have unique extreme values and, so, the extreme value of the continuum problem must be a minimizer corresponding to the unique solution to the weak form of the PDE. However, since we are now minimizing a “nice” quadratic functional, we obtain several advantageous properties.

One key advantage of this approach is that the discrete systems from conforming discretizations, where $\mathcal{V}^h \subset \mathcal{V}$, inherit well-posedness from the continuum, as continuity and coercivity of the functional (in some norm) at the continuous level automatically implies it for the discretization. Further advantages come in the linear system associated with the least-squares discretization. If we take $\{\phi_i\}_{i=1}^n$ to be a basis for $\mathcal{V}^h$, then the Ritz–Galerkin approach gives the discretized problem to solve for $\mathbf{u}$ such that

$$A\mathbf{u} = \mathbf{f}, \quad (10.121)$$

where $u^h = \sum_{i=1}^n u_i \phi_i$, $f_i = \langle f, L\phi_i \rangle$ and $a_{i,j} = \langle L\phi_j, L\phi_i \rangle$. We note that, by its construction, the matrix A is symmetric for any PDE operator L . Further, A is positive definite so long as L is not singular, since $\mathbf{u}^T A \mathbf{u} = \langle Lu, Lu \rangle > 0$ when L is nonsingular and $u \neq 0$. When A is symmetric and positive definite, we have many more options for iterative methods to solve the linear system, as discussed in Chapters 7 and 11.

However, there are also disadvantages to this approach. Consider, for example, the one-dimensional case of $Lu = -u'' = f$. The standard Ritz–Galerkin discretization, as presented in Chapter 4, yields the weak form of finding $u \in H^1(\Omega)$ such that $\langle u', v' \rangle = \langle f, v \rangle$ for all $v \in H^1(\Omega)$. As we learned in Section 4.3, we can discretize this problem with standard piecewise linear finite elements, getting $\mathcal{O}(h)$ error in the H^1 norm and $\mathcal{O}(h^2)$ error in the L^2 norm. Analysis of the resulting discretization matrix shows that the condition number of A grows like $\mathcal{O}(h^{-2})$. In contrast, the weak form for the least-squares formulation is to find $u \in H^2(\Omega)$ such that $\langle u'', v'' \rangle = -\langle f, v'' \rangle$ for all $v \in H^2(\Omega)$. Here, we note that the requirement that $u \in H^2(\Omega)$ comes from the fact that we need $\|u''\| < \infty$ for the bilinear form to be well defined, forcing us to consider using more complicated elements, such as the one-dimensional analogues of those discussed in Section 10.2. Moreover, we can show that the condition number of the resulting linear system now grows like $\mathcal{O}(h^{-4})$. In particular, for this example, both formulations lead to symmetric and positive-definite matrices, so even the advantages discussed above are of questionable value in this case, since the standard Ritz–Galerkin discretization possesses the same properties. For a more complete overview of least-squares finite-element methods, we refer to [117].

10.4.1 ■ FOSLS discretization

Given the inherent disadvantages of least-squares approaches, we have to ask if it is possible to reformulate our problems to keep the advantages but overcome the disadvantages. In this section, we show that the answer is “yes,” illustrating this with the same PDE considered in Section 8.4,

$$Lu = -\nabla \cdot K(\mathbf{x}) \nabla u + \mathbf{b}(\mathbf{x}) \cdot \nabla u + c(\mathbf{x})u = f(\mathbf{x}), \quad (10.122)$$

for $u \in H^2(\Omega) \cap H_0^1(\Omega)$ on a suitable domain, Ω . We know from Section 8.4 that standard coercivity and continuity bounds on this problem depend heavily on the coefficients in the equation, and that the system may fail to be coercive if $c(\mathbf{x}) - \frac{1}{2}\nabla \cdot \mathbf{b}(\mathbf{x}) \leq 0$ for any $\mathbf{x} \in \Omega$. In this case, weak coercivity of the continuum bilinear form does not imply weak coercivity of its discretization, and substantial work is required to show well-posedness for the weak form.

Remark 10.25: Using convection-reaction-diffusion for comparison

Clearly, not all problems of the form in equation (10.122) suffer from the disadvantages of a standard Galerkin formulation. Indeed, for simple problems, like $-\nabla \cdot \nabla u = f$, the standard Ritz–Galerkin weak form of finding $u \in H^1(\Omega)$ such that $\langle \nabla u, \nabla v \rangle = \langle f, v \rangle$ for all $v \in H^1(\Omega)$ can be derived from the minimization of the functional $G(u) = \frac{1}{2}\|\nabla u\|^2 - \langle f, u \rangle$, and, thus, there is no real need to attempt a different formulation. However, we consider the least-squares formulation of all problems of this form to illustrate the approach and to be able to compare and contrast its features more easily for the cases where both approaches are meaningful.

One possibility to avoid the need to formulate a discretization for $u \in H^2(\Omega)$ is to introduce a new dependent variable in order to transform the continuous PDE in equation (10.122) into a first-order system of PDEs. This introduction is what leads to the name of the first-order system least-squares (FOSLS) methodology.

Let $\boldsymbol{\sigma} = K\nabla u$, so that equation (10.122) is rewritten as

$$-\boldsymbol{\sigma} + K\nabla u = \mathbf{0}, \quad (10.123a)$$

$$-\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} \cdot (K^{-1}\boldsymbol{\sigma}) + cu = f. \quad (10.123b)$$

With this, we rewrite the second-order PDE, $Lu = f$, as a first-order system,

$$\hat{L} \begin{bmatrix} \boldsymbol{\sigma} \\ u \end{bmatrix} = \begin{bmatrix} -I & K\nabla \\ -\nabla \cdot + \mathbf{b} \cdot K^{-1} & cI \end{bmatrix} \begin{bmatrix} \boldsymbol{\sigma} \\ u \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ f \end{bmatrix}, \quad (10.124)$$

which we write as $\hat{L}\hat{u} = \hat{f}$, implicitly defining the augmented variables $\hat{u} = \begin{bmatrix} \boldsymbol{\sigma} \\ u \end{bmatrix}$ and $\hat{f} = \begin{bmatrix} \mathbf{0} \\ f \end{bmatrix}$. Up to this point, we have been nonspecific about boundary conditions, but adding variables to a system of equations demands that we be more careful. Suppose we are given the boundary conditions

$$u = q_{\mathcal{D}} \text{ on } \Gamma_{\mathcal{D}}, \quad (10.125a)$$

$$(K\nabla u) \cdot \mathbf{n} = q_{\mathcal{N}} \text{ on } \Gamma_{\mathcal{N}}, \quad (10.125b)$$

where $\partial\Omega = \Gamma_{\mathcal{D}} \cup \Gamma_{\mathcal{N}}$ with $\Gamma_{\mathcal{D}} \cap \Gamma_{\mathcal{N}} = \emptyset$ and $\mathbf{n}$ is the outward unit normal vector on the boundary. It is easy to rewrite the Neumann boundary condition on u as a Dirichlet boundary condition on $\boldsymbol{\sigma}$. To rewrite the Dirichlet boundary condition on u , we note that knowing $u(\mathbf{x})$ on $\Gamma_{\mathcal{D}}$ implies that we know the tangential component(s) of ∇u on $\Gamma_{\mathcal{D}}$, as $\nabla u \times \mathbf{n} = \nabla q_{\mathcal{D}} \times \mathbf{n}$. This notation denotes both the scalar tangential component of ∇u in two dimensions and the two vector components of ∇u in three dimensions. Thus, equation (10.125) leads directly to boundary conditions on $\boldsymbol{\sigma}$ given by

$$(K^{-1}\boldsymbol{\sigma}) \times \mathbf{n} = (\nabla q_{\mathcal{D}}) \times \mathbf{n} \text{ on } \Gamma_{\mathcal{D}}, \quad (10.126a)$$

$$\boldsymbol{\sigma} \cdot \mathbf{n} = q_{\mathcal{N}} \text{ on } \Gamma_{\mathcal{N}}. \quad (10.126b)$$

The boundary conditions in equation (10.126) can both be strongly imposed on $\boldsymbol{\sigma}$ along with the Dirichlet boundary condition on u in equation (10.125a), in contrast to the imposition of

boundary conditions for the mixed formulation in Section 10.3 (see Remark 10.11). That is, we strongly impose both the Dirichlet boundary condition on u and tangential boundary conditions on σ on parts of the boundary where we have Dirichlet boundary conditions on u , and we strongly impose a normal boundary condition on σ where we have Neumann boundary conditions on u . Including these “additional” boundary conditions on σ is generally important in the FOSLS methodology, yielding well-posed problems with better solution quality than when they are neglected—see [117, Ch. 12] for an expanded discussion on boundary conditions.

Introducing a least-squares minimization with the system in equation (10.124) leads to a least-squares functional as the sum of two components, namely,

$$G(u, \sigma; f) = \|K\nabla u - \sigma\|^2 + \|-\nabla \cdot \sigma + \mathbf{b} \cdot (K^{-1}\sigma) + cu - f\|^2. \quad (10.127)$$

From this we seek $(u, \sigma) \in \mathcal{V} \times \mathcal{W}$ such that

$$G(u, \sigma; f) = \min_{v \in \mathcal{V}, \mathbf{w} \in \mathcal{W}} G(v, \mathbf{w}; f), \quad (10.128)$$

where $\mathcal{V}$ and $\mathcal{W}$ are appropriate Hilbert spaces. For $K = I$, $\mathbf{b} = \mathbf{0}$, $c = 0$, the resulting Euler–Lagrange equations become finding $(u, \sigma) \in \mathcal{V} \times \mathcal{W}$ such that

$$\left\langle \begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \end{bmatrix} \begin{bmatrix} \sigma \\ u \end{bmatrix}, \begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ v \end{bmatrix} \right\rangle = \left\langle \begin{bmatrix} \mathbf{0} \\ f \end{bmatrix}, \begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ v \end{bmatrix} \right\rangle \quad (10.129)$$

for all $(v, \mathbf{w}) \in \mathcal{V} \times \mathcal{W}$. Formally, integrating by parts gives the problem of finding $(u, \sigma) \in \mathcal{V} \times \mathcal{W}$ such that

$$\left\langle \begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \end{bmatrix}^* \begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \end{bmatrix} \begin{bmatrix} \sigma \\ u \end{bmatrix}, \begin{bmatrix} \mathbf{w} \\ v \end{bmatrix} \right\rangle = \left\langle \begin{bmatrix} \mathbf{0} \\ f \end{bmatrix}, \begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ v \end{bmatrix} \right\rangle \quad (10.130)$$

for all $(v, \mathbf{w}) \in \mathcal{V} \times \mathcal{W}$, where L^* denotes the L^2 adjoint of a differential operator, defined so that $\langle Lu, v \rangle = \langle u, L^*v \rangle$. More generally, the Euler–Lagrange equations associated with minimizing $\|\hat{L}\hat{u} - \hat{f}\|^2$ always take the form of finding $\hat{u}$ such that

$$\langle \hat{L}\hat{u}, \hat{L}\hat{v} \rangle = \langle \hat{f}, \hat{L}\hat{v} \rangle \text{ for all } \hat{v}, \quad (10.131)$$

which we formally rewrite as

$$\langle \hat{L}^* \hat{L}\hat{u}, \hat{v} \rangle = \langle \hat{f}, \hat{L}\hat{v} \rangle \text{ for all } \hat{v}. \quad (10.132)$$

We note that this equivalence is only *formal*, since the action of replacing an operator on one side of an inner product with its adjoint on the other relies on certain boundary condition terms either being zero or being added to the form, as explained below. We note, however, that there is no methodological need to take the adjoints across the inner products in formulating the method, but doing so formally helps us develop intuition, as we do next.

Example 10.26: L^2 adjoint of a differential operator

Computing adjoints of differential operators is mostly an exercise in carefully integrating by parts. As an example, for one-dimensional domain Ω and starting with a single derivative, let $L = \frac{\partial}{\partial x}$. Integration by parts gives

$$\int_{\Omega} u_x v \, d\mathbf{x} = uv|_{\partial\Omega} - \int_{\Omega} uv_x \, d\mathbf{x}. \quad (10.133)$$

If we assume that u and/or v are zero on the boundary, then this gives

$$\left\langle \frac{\partial v}{\partial x}, v \right\rangle = - \left\langle u, \frac{\partial v}{\partial x} \right\rangle \Rightarrow \left[\frac{\partial}{\partial x} \right]^* = - \frac{\partial}{\partial x}. \quad (10.134)$$

Similar calculations in two and three dimensions (see Section 2.6) give

$$\begin{cases} \nabla^* = -\nabla \cdot & \text{assuming that } \mathbf{n} \cdot \boldsymbol{\sigma} = 0, \\ (\nabla \times)^* = \nabla \times & \text{assuming that } \mathbf{n} \times \boldsymbol{\sigma} = \mathbf{0}, \end{cases} \quad (10.135)$$

where $\mathbf{n}$ is the outward unit normal to the boundary.

We call the $\hat{L}^* \hat{L}$ operator the formal normal operator associated with the least-squares discretization of the first-order system $\hat{L}\hat{u} = \hat{f}$, as it represents the second-order operator whose Galerkin discretization is equivalent to the FOSLS discretization with $\hat{L}$, up to treatment of boundary conditions. Thus, it gives important insight into the appropriate Hilbert spaces to choose for $\hat{u}$ and $\hat{v}$ in equation (10.131). Continuing the example with $K = I$, $\mathbf{b} = \mathbf{0}$, and $c = 0$ from equation (10.130), we have the formal normal operator

$$\begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \end{bmatrix}^* \begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \end{bmatrix} = \begin{bmatrix} I - \nabla \nabla \cdot & -\nabla \\ \nabla \cdot & -\nabla \cdot \nabla \end{bmatrix}. \quad (10.136)$$

Dropping, for the moment, the off-diagonal terms in $\hat{L}^* \hat{L}$, we define a norm based on the diagonal of the formal normal as

$$\left\langle \begin{bmatrix} I - \nabla \nabla \cdot & 0 \\ 0 & -\nabla \cdot \nabla \end{bmatrix} \begin{bmatrix} \boldsymbol{\sigma} \\ u \end{bmatrix}, \begin{bmatrix} \boldsymbol{\sigma} \\ u \end{bmatrix} \right\rangle = \langle \boldsymbol{\sigma}, \boldsymbol{\sigma} \rangle + \langle \nabla \cdot \boldsymbol{\sigma}, \nabla \cdot \boldsymbol{\sigma} \rangle + \langle \nabla u, \nabla u \rangle \quad (10.137a)$$

$$= \|\boldsymbol{\sigma}\|_{\text{div}}^2 + |u|_1^2, \quad (10.137b)$$

where $\|\boldsymbol{\sigma}\|_{\text{div}}^2 = \|\boldsymbol{\sigma}\|^2 + \|\nabla \cdot \boldsymbol{\sigma}\|^2$ is the usual norm on $\mathbf{H}(\text{div}, \Omega)$. Thus, if we show continuity and coercivity for $(u, \boldsymbol{\sigma}) \in H^1(\Omega) \times \mathbf{H}(\text{div}, \Omega)$, we obtain a unique solution to the first-order system. Moreover, since $\hat{L}$ is first order, the least-squares formulation leads to improved numerical properties, such as a linear system with a condition number that scales like $\mathcal{O}(h^{-2})$. Showing continuity for the bilinear form in equation (10.129) is straightforward; proving coercivity is complex, but see [66] for the general case of $\hat{L}$ as in equation (10.124).

While this approach overcomes a key disadvantage of applying the least-squares methodology to the original second-order equation in equation (10.122), it still has numerical shortcomings. First, we have increased the number of degrees of freedom in the resulting linear systems, since we now approximate both u and $\boldsymbol{\sigma}$. Second, while $u \in H^1(\Omega)$, we need to address the approximation of $\boldsymbol{\sigma} \in \mathbf{H}(\text{div}, \Omega)$, which requires specialized elements, as introduced in Section 10.2. Moreover, approximating $\boldsymbol{\sigma} = K \nabla u$ in $\mathbf{H}(\text{div}, \Omega)$ leads to a structure where certain errors in $\boldsymbol{\sigma}$ can lead to small values of $G(u, \boldsymbol{\sigma}; f)$, even though they represent large errors in a relevant norm (see Example 10.27).

Example 10.27: Approximation in $H(\text{div}, \Omega)$ and $H^1(\Omega)$

Consider the case with $K = I$, $\mathbf{b} = \mathbf{0}$, and $c = 0$, leading to the functional

$$G(u, \boldsymbol{\sigma}; f) = \|\hat{L}\hat{u} - \hat{f}\|^2 = \|-\nabla \cdot \boldsymbol{\sigma} - f\|^2 + \|-\boldsymbol{\sigma} + \nabla u\|^2. \quad (10.138)$$

Assume that u and $\boldsymbol{\sigma}$ are the exact solutions to the first-order system (minimizers of $G(u, \boldsymbol{\sigma}; f)$)

and we evaluate $G(u, \boldsymbol{\sigma}; f)$, but add an error, $\mathbf{E}$, to $\boldsymbol{\sigma}$:

$$G(u, \boldsymbol{\sigma} + \mathbf{E}; f) = \|-\nabla \cdot (\boldsymbol{\sigma} + \mathbf{E}) - f\|^2 + \|-\boldsymbol{\sigma} - \mathbf{E} + \nabla u\|^2 = \|\nabla \cdot \mathbf{E}\|^2 + \|\mathbf{E}\|^2. \quad (10.139)$$

If $\mathbf{E}$ is relatively large, we expect that this would be reflected in the functional value. However, if $\mathbf{E}$ is divergence-free but highly oscillatory, then we can have

$$\nabla \cdot \mathbf{E} = 0, \quad \|\mathbf{E}\|^2 \approx 0, \quad \text{but } \|\nabla \mathbf{E}\|^2 \gg 1. \quad (10.140)$$

In two dimensions, one such function is given by $\mathbf{E} = \epsilon \begin{bmatrix} \sin(k\pi x) \cos(k\pi y) \\ -\cos(k\pi x) \sin(k\pi y) \end{bmatrix}$, which we can directly verify satisfies $\nabla \cdot \mathbf{E} = 0$. When ϵ is small, but k is large, we see that $\|\mathbf{E}\| = \mathcal{O}(\epsilon)$, but $\|\nabla \mathbf{E}\| = \mathcal{O}(k\epsilon)$, showing that the $\mathbf{H}^1(\Omega)$ norm of $\mathbf{E}$ can be large when the $\mathbf{H}(\text{div}, \Omega)$ norm is small. This is an inherent disadvantage of posing the problem with $\boldsymbol{\sigma} \in \mathbf{H}(\text{div}, \Omega)$.

An advantage of the FOSLS methodology, though, is that we have the freedom to add consistent terms to our functional in order to strengthen the natural norms for the problem, and these are immediately reflected in the discretization. Following Example 10.27, we seek to further augment the system of PDEs so that we can work with $\boldsymbol{\sigma} \in \mathbf{H}^1(\Omega)$, rather than $\mathbf{H}(\text{div}, \Omega)$, leading to stronger control on $\|\nabla \mathbf{E}\|$. As written so far, the system is not coercive in $\mathbf{H}^1(\Omega)$. However, we augment the system of equations by noting that $K^{-1}\boldsymbol{\sigma} = \nabla u$, so it must be curl-free, leading to the system

$$-\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} \cdot \nabla u + cu = f, \quad (10.141a)$$

$$-\boldsymbol{\sigma} + K\nabla u = \mathbf{0}, \quad (10.141b)$$

$$\nabla \times K^{-1}\boldsymbol{\sigma} = \mathbf{0}. \quad (10.141c)$$

We note that equation (10.141c) is a vector equation in three dimensions, but a scalar equation in two dimensions, where the curl of a vector field $\mathbf{F}$ becomes $\nabla \times \mathbf{F} = \nabla \times \begin{bmatrix} F_1 \\ F_2 \end{bmatrix} = \begin{bmatrix} \partial F_2 / \partial x \\ -\partial F_1 / \partial y \end{bmatrix}$.

As an example, in the special case when $K = I$, $\mathbf{b} = \mathbf{0}$, and $c = 0$, the formal normal for this system becomes

$$\begin{bmatrix} -I & \nabla & \nabla \times \\ -\nabla \cdot & 0 & 0 \end{bmatrix} \begin{bmatrix} -I & \nabla \\ -\nabla \cdot & 0 \\ \nabla \times & 0 \end{bmatrix} = \begin{bmatrix} I - \nabla \nabla \cdot + \nabla \times \nabla \times & -\nabla \\ \nabla \cdot & -\nabla \cdot \nabla \end{bmatrix} \quad (10.142a)$$

$$= \begin{bmatrix} I - \Delta & -\nabla \\ \nabla \cdot & -\Delta \end{bmatrix}, \quad (10.142b)$$

using the identity for the vector Laplacian that $-\Delta \mathbf{u} = -\nabla \nabla \cdot \mathbf{u} + \nabla \times \nabla \times \mathbf{u}$. It is easy to extend the proof of ellipticity discussed above for $(u, \boldsymbol{\sigma}) \in H^1(\Omega) \times \mathbf{H}(\text{div}, \Omega)$ to the case with $(u, \boldsymbol{\sigma}) \in H^1(\Omega) \times \mathbf{H}^1(\Omega)$ with the added curl constraint, at least on domains where $\mathbf{H}^1(\Omega) = \mathbf{H}(\text{curl}, \Omega) \cap \mathbf{H}(\text{div}, \Omega)$ —see [164] for more details when this is not the case. Furthermore, this formulation can now be discretized using simple Lagrange finite elements for u and $\boldsymbol{\sigma}$, resulting in linear systems with advantageous properties, even for problems for which the Galerkin weak form encounters difficulties.

10.4.2 ■ A posteriori error estimates

An additional useful property of least-squares formulations is that they naturally lead to efficient and reliable a posteriori error estimators, as discussed in Section 8.6. Let $(u^h, \boldsymbol{\sigma}^h) \in \mathcal{V}^h \times \mathcal{W}^h$ be a computed approximation to the exact solution, $(u, \boldsymbol{\sigma}) \in \mathcal{V} \times \mathcal{W}$, of the system $\hat{L}\hat{u} = \hat{f}$,

with $\hat{u} = [\sigma_u]$, and note that (u, σ) are also minimizers of $G(u, \sigma; f)$. Let $e^h = u - u^h$ and $E^h = \sigma - \sigma^h$ be the approximation errors in u and σ , respectively, defining $\hat{e}^h = \begin{bmatrix} e^h \\ E^h \end{bmatrix}$. We assume that the bilinear form resulting from the least-squares discretization, $\langle \hat{L}\hat{u}, \hat{L}\hat{v} \rangle$, is elliptic in $\mathcal{V} \times \mathcal{W}$. One way to express this is that $G(u, \sigma; 0) \simeq \|u\|_{\mathcal{V}}^2 + \|\sigma\|_{\mathcal{W}}^2$, where we use the $\simeq$ notation as in equation (10.114b) to denote equivalence to the (product) norm on $\mathcal{V} \times \mathcal{W}$. This equivalence occurs for the bilinear form from the discretization, which we directly express by evaluating the functional with a zero forcing term. Then, substituting the approximation into the least-squares functional, we have

$$G(u^h, \sigma^h; f) = \|\hat{L}\hat{u}^h - \hat{f}\|^2 = \|\hat{L}\hat{u}^h - \hat{L}\hat{u}\|^2 = \|\hat{L}(\hat{u}^h - \hat{u})\|^2 \quad (10.143a)$$

$$= \|\hat{L}\hat{e}^h\|^2 = G(e^h, E^h; 0) \simeq \|e^h\|_{\mathcal{V}}^2 + \|E^h\|_{\mathcal{W}}^2. \quad (10.143b)$$

Thus, the functional evaluated at the discrete approximation, $G(u^h, \sigma^h; f)$, approximates the error in the discrete approximation, (u^h, σ^h) , in the $\mathcal{V} \times \mathcal{W}$ norm. This is further motivation for adding the curl constraint, as discussed above, since then that norm is a product- $H^1(\Omega)$ norm. Due to continuity and coercivity of the bilinear form,

$$\langle \hat{L}\hat{e}^h, \hat{L}\hat{e}^h \rangle \leq c_1 (\|e^h\|_{\mathcal{V}}^2 + \|E^h\|_{\mathcal{W}}^2), \quad (10.144a)$$

$$\langle \hat{L}\hat{e}^h, \hat{L}\hat{e}^h \rangle \geq c_0 (\|e^h\|_{\mathcal{V}}^2 + \|E^h\|_{\mathcal{W}}^2), \quad (10.144b)$$

the functional acts as an efficient and reliable a posteriori error estimator, as defined in Definition 8.41, without any postprocessing steps required.

In addition, if (u^h, σ^h) is a minimizer of the functional over $\mathcal{V}^h \times \mathcal{W}^h$, then we can use standard arguments to establish a form of Céa's lemma, similar to Lemma 8.31. Since (u^h, σ^h) is a minimizer, we automatically get

$$G(u^h, \sigma^h; f) \leq G(I^h u, I^h \sigma; f) \leq c_1 (\|u - I^h u\|_{\mathcal{V}}^2 + \|\sigma - I^h \sigma\|_{\mathcal{W}}^2) \quad (10.145)$$

by continuity and the arguments above. By coercivity, we have

$$c_0 (\|e^h\|_{\mathcal{V}}^2 + \|E^h\|_{\mathcal{W}}^2) \leq G(u^h, \sigma^h; f). \quad (10.146)$$

Taken together, these give us

$$(\|e^h\|_{\mathcal{V}}^2 + \|E^h\|_{\mathcal{W}}^2) \leq \frac{c_1}{c_0} (\|u - I^h u\|_{\mathcal{V}}^2 + \|\sigma - I^h \sigma\|_{\mathcal{W}}^2). \quad (10.147)$$

Thus, the error of the approximation in the $\mathcal{V} \times \mathcal{W}$ norm, represented by the least-squares functional, is bounded by the interpolation error of the finite-element space in the same norm. When we use the curl-constraint formulation, and $\mathcal{V}$ and $\mathcal{W}$ are both H^1 spaces (scalar and vector), and their discrete counterparts are the Lagrange spaces discussed in Section 8.3, this gives us $\mathcal{O}(h^k)$ error bounds using $P_k(\Omega^h)$ elements, measured in the product H^1 norm. Similarly to the earlier discussion, we can also prove an Aubin–Nitsche duality theorem (under assumptions of elliptic regularity) to yield one order higher estimates in the L^2 product norm.

These properties of the error estimates allow for the FOSLS discretization approach to be complemented with adaptive mesh refinement techniques, using these simple a posteriori error estimates to properly mark elements for refinement (see, for example, [42, 162, 5]). This, in combination with the fact that the linear systems are symmetric and positive definite, gives robustness to the FOSLS approach in terms of its efficiency, despite the fact that there is an increase in the number of unknowns in the system.

10.4.3 ■ Stokes example

We end this section by applying the FOSLS approach to the Stokes equations,

$$-\Delta \mathbf{u} + \nabla p = \mathbf{f}, \quad (10.148a)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (10.148b)$$

where $\mathbf{u}$ represents the velocity of a slow-moving, incompressible fluid and p is the pressure. As in Section 10.3, we write the system in terms of the vector Laplacian, instead of using the symmetric gradient (see Remark 10.20). We illustrate two properties of FOSLS in this subsection. First, we show that the FOSLS functional does, indeed, converge as expected, indicating that it is a reliable error estimator. Second, we focus on the conservation of mass properties of the FOSLS discretization.

The constraint that $\nabla \cdot \mathbf{u} = 0$ is seen as a fundamental law in many fields involving incompressible fluid flow. Numerical schemes that do not respect this property, either exactly or approximately, are often viewed as unattractive schemes in these fields (see, for example, [140]), and the conservation properties of the FOSLS discretization have been a major concern with the methodology. In particular, for the Stokes system above, we would naturally form the FOSLS discretization by applying the least-squares principle to these equations, first reformulating equation (10.148a) into a first-order system by introducing $\boldsymbol{\sigma} = \nabla \mathbf{u}$ as a new variable and then summing the squared residuals for the remaining ones. This naturally leads to an error bound on the discrete approximations to $\mathbf{u}$, $\boldsymbol{\sigma}$, and p , similar to the discussion above for the simpler case of a scalar elliptic PDE, but no bound on how $\nabla \cdot \mathbf{u}$ varies from its expected zero value. It is natural to introduce weights in the FOSLS functional to highlight the importance of equation (10.148b) (see, for example, [82] and the references therein), but such weights impact the ellipticity constants and, thus, both the overall quality of approximation and the residual in the momentum equation. While there are many techniques to improve conservation (see [134]), we show below how this manifests in a standard implementation of FOSLS.

For simplicity, we consider a two-dimensional domain, though the ideas are easily extended to three spatial dimensions. We write the components of $\mathbf{u} = \begin{bmatrix} u \\ v \end{bmatrix}$ and model channel flow down a tube with domain $\Omega = [0, 8] \times [0, 1]$. We prescribe inflow and outflow velocities and zero pressure at the end of the channel, along with zero velocity along the top and bottom of the channel:

$$\mathbf{u} = \begin{bmatrix} \pi \sin(\pi y) \\ 0 \end{bmatrix}, \quad \text{inflow } (x = 0) \text{ and outflow } (x = 8), \quad (10.149a)$$

$$\mathbf{u} = \mathbf{0}, \quad \text{top } (y = 0) \text{ and bottom } (y = 1), \quad (10.149b)$$

$$p = 0, \quad \text{outflow } (x = 8). \quad (10.149c)$$

We prescribe $\mathbf{f} = \begin{bmatrix} \pi^3 \sin(\pi y) \\ 0 \end{bmatrix}$, so that $\mathbf{u} = \begin{bmatrix} \pi \sin(\pi y) \\ 0 \end{bmatrix}$ and $p = 0$ are the analytical solutions of equation (10.148). To quantify the quality of conservation of mass, we use the divergence theorem to write $\int_R \nabla \cdot \mathbf{u} \, dx = \int_{\partial R} \mathbf{u} \cdot \mathbf{n} \, ds$ for any region $R \subset \Omega$. In particular, since we consider channel flow parallel to the x -axis, we consider regions $R_x = [0, x] \times [0, 1]$ and note that ∂R_x consists of four edges of the rectangular region. Along the top and bottom of R_x , the Dirichlet boundary conditions enforce that $\mathbf{u} \cdot \mathbf{n} = 0$, so we get zero contributions to the conservation of mass from these edges. Along the left of R_x , $\mathbf{n} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$ and $\mathbf{u}$ is known. Thus, defining $m(x) = \int_0^1 u(x, y) \, dy$, $\int_{R_x} \nabla \cdot \mathbf{u} = 0$ implies that $m(x) = 2$ for all x . In Figures 10.4.1b and 10.4.2b, we plot $m(x)$ versus x to evaluate the conservation properties of the FOSLS discretizations.

We now present two ways to transform equations (10.148a) and (10.148b) into first-order systems by introducing either $\nabla \mathbf{u}$ or $\nabla \times \mathbf{u}$ as additional variables.

Velocity-gradient-pressure formulation

In the velocity-gradient-pressure formulation, we begin by letting

$$\boldsymbol{\sigma} = \nabla \mathbf{u} = \begin{bmatrix} u_x & u_y \\ v_x & v_y \end{bmatrix} = \begin{bmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{21} & \sigma_{22} \end{bmatrix}. \quad (10.150)$$

With this, we rewrite the Stokes equations as

$$-\nabla \cdot \boldsymbol{\sigma} + \nabla p = \mathbf{f}, \quad (10.151a)$$

$$\boldsymbol{\sigma} - \nabla \mathbf{u} = \mathbf{0}, \quad (10.151b)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (10.151c)$$

$$\nabla \times \boldsymbol{\sigma} = \mathbf{0}, \quad (10.151d)$$

$$\nabla \text{tr} \boldsymbol{\sigma} = \mathbf{0}, \quad (10.151e)$$

where we note that the divergence and curl operators act rowwise on $\boldsymbol{\sigma}$, so that they act on ∇u and ∇v in the usual way. In addition to the original Stokes equations, we have added the definition of the new variable, $\boldsymbol{\sigma} = \nabla \mathbf{u}$, and two new *consistent* equations, $\nabla \times \boldsymbol{\sigma} = \mathbf{0}$ (since $\boldsymbol{\sigma}$ is a gradient) and $\nabla \text{tr} \boldsymbol{\sigma} = \mathbf{0}$, noting that the trace of $\boldsymbol{\sigma}$, $\text{tr} \boldsymbol{\sigma}$, is equal to $\nabla \cdot \mathbf{u}$, so both it and its gradient are zero. As in the case of a scalar equation, we include $\nabla \times \boldsymbol{\sigma} = \mathbf{0}$ so that we can discretize $\boldsymbol{\sigma}$ in an H^1 space. We include the constraint on $\text{tr} \boldsymbol{\sigma}$ in order to more strongly enforce the divergence constraint by including it in equations for both $\mathbf{u}$ and $\boldsymbol{\sigma}$. In addition to the boundary conditions in equation (10.149), we must also include conditions for the auxiliary variable $\boldsymbol{\sigma}$. From the prescribed boundary conditions on $\mathbf{u}$, we can readily compute boundary conditions on some components of $\boldsymbol{\sigma}$ on each edge by taking tangential derivatives of the prescribed values for $\mathbf{u}$. Using the divergence condition that $u_x + v_y = 0$, we compute an additional boundary condition on each edge from the known value of one of the two terms on the left-hand side. With these, we prescribe

$$\sigma_{11} = 0, \quad \text{all boundaries}, \quad (10.152a)$$

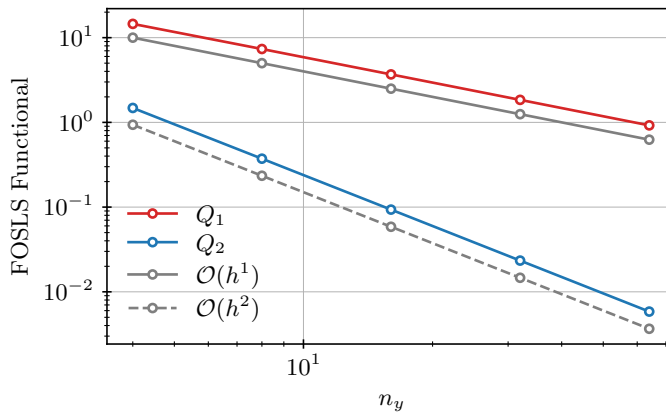
$$\sigma_{12} = \pi^2 \cos(\pi y), \quad \text{left and right}, \quad (10.152b)$$

$$\sigma_{21} = 0, \quad \text{top and bottom}, \quad (10.152c)$$

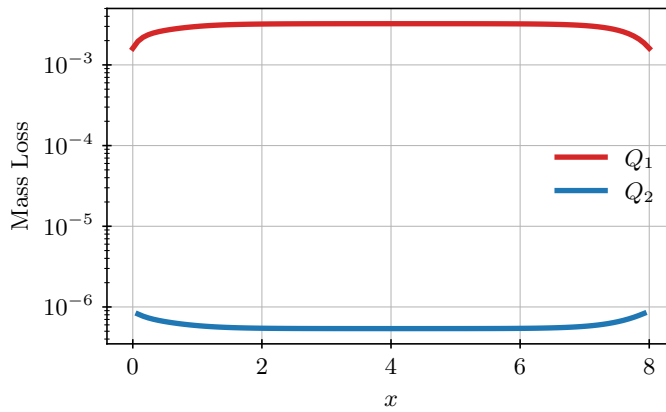
$$\sigma_{22} = 0, \quad \text{all boundaries}. \quad (10.152d)$$

With these boundary conditions, we find that the quality of the FOSLS discretization suffers somewhat, so we augment these boundary conditions with additional conditions on σ_{12} on the top and bottom based on the known analytical solution in the numerical results that follow. This, of course, relies on additional knowledge that we may not have access to in more general settings.

Figure 10.4.1a shows the convergence of the FOSLS functional versus mesh size, using both bilinear (Q_1) and biquadratic (Q_2) finite elements for all unknowns in the system. To make the elements uniform, we use a quadrilateral mesh of $n_x \times n_y$ elements with $n_x = 8n_y$, yielding $h = n_y^{-1}$. As expected, we see $\mathcal{O}(h)$ and $\mathcal{O}(h^2)$ convergence for the functional using Q_1 and Q_2 elements, respectively. Figure 10.4.1b shows the mass conservation along the domain for $n_y = 32$. Since the inflow and outflow are prescribed, we expect good mass conservation near the ends of the domain. However, we see a significant loss of mass toward the center of the channel (calculated as $|2 - m(x)|$) when using the bilinear discretization. Increasing the order of the finite elements to biquadratic, however, significantly improves the result, reducing the mass loss by several orders of magnitude. This is due to a more accurate representation of the solution down the channel. Mass conservation also improves by increasing the resolution, but not as significantly as by increasing the order of the space. For higher-dimensional problems, this can be more drastic, and better approximation properties are needed in order to capture the underlying physics.



a. FOSLS functional versus mesh size.



b. Mass loss along channel.

Figure 10.4.1. FOSLS functional values versus mesh size and mass loss along channel for a simulation of the Stokes problem defined in equation (10.149) using the velocity-gradient-pressure formulation. Here, $n_y + 1$ is the number of vertices of the triangular mesh in the y -direction and $h = \mathcal{O}(n_y^{-1})$.

Velocity-vorticity-pressure formulation

In the velocity-vorticity-pressure formulation, we first introduce the vorticity of the fluid, defined as $\mathbf{w} = \nabla \times \mathbf{u}$. Then, we make use of the vector calculus identity that

$$-\nabla \cdot \nabla \mathbf{u} = \nabla \times \nabla \times \mathbf{u} - \nabla(\nabla \cdot \mathbf{u}). \quad (10.153)$$

Since we consider an incompressible fluid, where $\nabla \cdot \mathbf{u} = 0$, this gives us that $-\nabla \cdot \nabla \mathbf{u} = \nabla \times \mathbf{w}$, and we rewrite the Stokes equations as

$$\nabla \times \mathbf{w} + \nabla p = \mathbf{f}, \quad (10.154a)$$

$$\mathbf{w} - \nabla \times \mathbf{u} = \mathbf{0}, \quad (10.154b)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (10.154c)$$

$$\nabla \cdot \mathbf{w} = 0, \quad (10.154d)$$

where the last equation is consistent with the fact that the vorticity is a curl of a vector field. Note that we write this equation as if $\mathbf{w}$ is a vector field (i.e., for the three-dimensional case) but that it is effectively scalar valued for two-dimensional domains. This system is quite different from

the second-order example considered in detail earlier, so it is worthwhile to consider the formal normal to ensure that we get ellipticity in spaces that we like for all of $\mathbf{u}$, w , and p . Writing the system as an operator, we have

$$\hat{L} \begin{bmatrix} w \\ p \\ \mathbf{u} \end{bmatrix} = \begin{bmatrix} \nabla \times & \nabla & 0 \\ I & 0 & -\nabla \times \\ 0 & 0 & \nabla \cdot \\ \nabla \cdot & 0 & 0 \end{bmatrix} \begin{bmatrix} w \\ p \\ \mathbf{u} \end{bmatrix}, \quad (10.155)$$

with formal normal

$$\hat{L}^* \hat{L} = \begin{bmatrix} \nabla \times & I & 0 & -\nabla \\ -\nabla \cdot & 0 & 0 & 0 \\ 0 & -\nabla \times & -\nabla & 0 \end{bmatrix} \begin{bmatrix} \nabla \times & \nabla & 0 \\ I & 0 & -\nabla \times \\ 0 & 0 & \nabla \cdot \\ \nabla \cdot & 0 & 0 \end{bmatrix} \quad (10.156a)$$

$$= \begin{bmatrix} I - \Delta & 0 & -\nabla \times \\ 0 & -\Delta & 0 \\ -\nabla \times & 0 & -\Delta \end{bmatrix}, \quad (10.156b)$$

where we use the same vector calculus identity as above. This formal normal indicates that we may choose all variables to be in H^1 spaces, making the choice of finite-element spaces straightforward.

Again, we must supplement the boundary conditions in equation (10.149) with those for the auxiliary variable, w . The original boundary conditions on $\mathbf{u}$ do not allow us to directly compute boundary conditions on w , so we impose these from the known analytical solution, giving

$$w = -\pi^2 \cos(\pi y), \quad \text{all boundaries}, \quad (10.157)$$

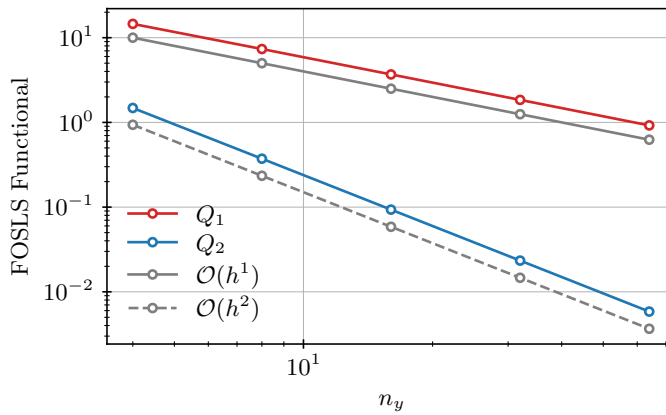
noting that w is actually a scalar for this two-dimensional example. Similarly to the velocity-gradient-pressure formulation above, Figure 10.4.2a shows the convergence of the FOSLS functional versus mesh size, using both bilinear (Q_1) and biquadratic (Q_2) finite elements for all unknowns in the system. As expected, we see an $\mathcal{O}(h)$ and $\mathcal{O}(h^2)$ convergence for the functional using Q_1 and Q_2 elements, respectively. Likewise, Figure 10.4.2b shows the mass conservation versus x along the domain for $n_y = 32$ and, again, we see a significant loss of mass toward the center of the channel for this discretization.

10.5 ■ Discontinuous Galerkin finite-element methods

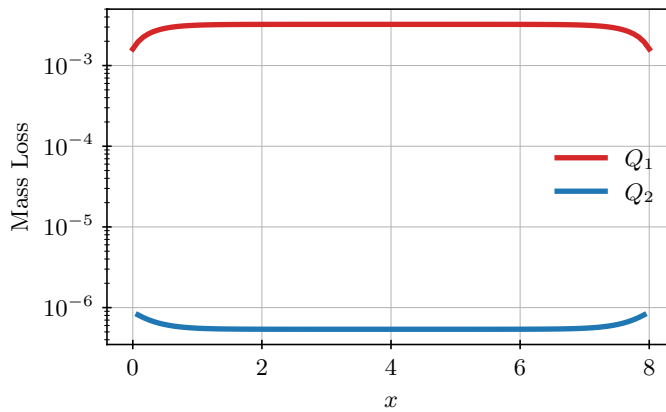
Objectives

As we saw in Section 9.5, discontinuous Galerkin (DG) methods offer a way to naturally extend finite-volume schemes to high-order approximations. The key step in the construction of DG schemes for conservation laws is the idea of the numerical flux. The goal of this section is to use similar techniques to devise DG methods for *elliptic* problems. Specifically, you will

- revisit the first-order form of the Poisson problem from Sections 10.3 and 10.4 to express the weak form locally on each element;
- define numerical fluxes for the scalar function u and its gradient to define the symmetric interior penalty method, one form of DG;
- highlight the numerical flux as a central component in stabilizing the approximation; and
- construct a DG weak form expressed over the elements and edges of elements in the mesh.



a. FOSLS functional versus mesh size.



b. Mass loss along channel.

Figure 10.4.2. FOSLS functional values versus mesh size and mass loss along channel for a simulation of the Stokes problem defined in equation (10.149) using the velocity-vorticity-pressure formulation. Here, $n_y + 1$ is the number of vertices of the triangular mesh in the y -direction and $h = \mathcal{O}(n_y^{-1})$.

To begin, consider the elliptic boundary-value problem (cf. equation (8.157))

$$-\nabla \cdot K(\mathbf{x}) \nabla u = f(\mathbf{x}) \quad \text{in } \Omega \subset \mathbb{R}^d, \quad (10.158a)$$

$$u = 0 \quad \text{on } \partial\Omega, \quad (10.158b)$$

where $K(\mathbf{x}) > 0$ is a smooth function and d is the dimension of the problem. Here, we have assumed homogeneous Dirichlet conditions along the boundary in order to simplify the discussion, but they can be generalized as in Section 8.4. In addition, we consider Ω^h to be a boundary-conforming computational domain for Ω —see Definition 2.12—and denote by $\mathcal{T}^h$ the set of elements in the mesh.

To construct a discontinuous Galerkin (DG) approximation, we consider the *first-order form* as in Sections 10.3 and 10.4 in order to introduce numerical fluxes for both u and ∇u . To this end, let $\boldsymbol{\sigma} = K(\mathbf{x}) \nabla u$, which leads to

$$-\nabla \cdot \boldsymbol{\sigma} = f, \quad (10.159a)$$

$$\boldsymbol{\sigma} - K \nabla u = \mathbf{0}, \quad (10.159b)$$

with the same Dirichlet boundary conditions on u . Note, however, that a Neumann-type boundary condition here, such as $\mathbf{n} \cdot (K(\mathbf{x})\nabla u) = 0$, leads to a Dirichlet-type boundary condition on $\boldsymbol{\sigma}$, as discussed in Remark 10.11. Unlike in a *continuous* Galerkin scheme, we now work with the so-called *broken* Sobolev spaces on the triangulation $\mathcal{T}^h$. For the current problem, we require the restriction of an approximation to element $\tau \in \Omega$ to be in $H^1(\tau)$ and define the scalar and vector-valued spaces as

$$\mathcal{V} = \{v \in L^2(\Omega) \mid v|_{\tau} \in H^1(\tau) \forall \tau \in \mathcal{T}^h\}, \quad (10.160a)$$

$$\mathcal{W} = \{\mathbf{w} \in \mathbf{L}^2(\Omega) \mid \mathbf{w}|_{\tau} \in \mathbf{H}^1(\tau) \forall \tau \in \mathcal{T}^h\}. \quad (10.160b)$$

The associated discrete subspaces are then defined as $\mathcal{V}^h = P_k^-(\Omega^h)$ and $\mathcal{W}^h = \mathbf{P}_k^-(\Omega^h)$, the spaces of discontinuous piecewise polynomial functions and vector fields of degree no more than k , respectively.

Next, we consider the weak form over each element τ : find $u \in \mathcal{V}^h$ and $\boldsymbol{\sigma} \in \mathcal{W}^h$ such that for all $\tau \in \mathcal{T}^h$,

$$\int_{\tau} -\nabla \cdot \boldsymbol{\sigma} v \, d\mathbf{x} = \int_{\tau} f v \, d\mathbf{x} \quad \text{for all } v \in P_k(\tau), \quad (10.161a)$$

$$\int_{\tau} (\boldsymbol{\sigma} - K\nabla u) \cdot \mathbf{w} \, d\mathbf{x} = 0 \quad \text{for all } \mathbf{w} \in \mathbf{P}_k(\tau). \quad (10.161b)$$

Then, integration by parts yields the following form:

$$\int_{\tau} \boldsymbol{\sigma} \cdot \nabla v \, d\mathbf{x} - \int_{\partial\tau} \mathbf{n}_{\tau} \cdot \hat{\boldsymbol{\sigma}} v \, ds = \int_{\tau} f v \, d\mathbf{x}, \quad (10.162a)$$

$$\int_{\tau} \boldsymbol{\sigma} \cdot \mathbf{w} \, d\mathbf{x} + \int_{\tau} u \nabla \cdot (K\mathbf{w}) \, d\mathbf{x} - \int_{\partial\tau} \hat{u} \mathbf{n}_{\tau} \cdot (K\mathbf{w}) \, ds = 0, \quad (10.162b)$$

where we have introduced $\hat{u}$ and $\hat{\boldsymbol{\sigma}}$ as *numerical fluxes* to approximate and represent the values of u and $\boldsymbol{\sigma}$ on the interfaces of each $\tau \in \mathcal{T}^h$ and use $\mathbf{n}_{\tau}$ to denote the outward unit normal to element τ . Defining the fluxes leads to a range of different methods with different stability and accuracy properties. Here, we focus on *interior penalty* DG methods; however, the extension to other forms through the definition of the numerical flux is straightforward [17].

Finally, we sum over all elements $\tau \in \mathcal{T}^h$ to arrive at a full weak form of the problem: find $u \in \mathcal{V}^h$ and $\boldsymbol{\sigma} \in \mathcal{W}^h$ such that

$$\int_{\Omega} \boldsymbol{\sigma} \cdot \nabla v \, d\mathbf{x} - \sum_{\tau} \int_{\partial\tau} \mathbf{n}_{\tau} \cdot \hat{\boldsymbol{\sigma}} v \, ds = \int_{\Omega} f v \, d\mathbf{x}, \quad (10.163a)$$

$$\int_{\Omega} \boldsymbol{\sigma} \cdot \mathbf{w} \, d\mathbf{x} + \int_{\Omega} u \nabla \cdot (K\mathbf{w}) \, d\mathbf{x} - \sum_{\tau} \int_{\partial\tau} \hat{u} \mathbf{n}_{\tau} \cdot (K\mathbf{w}) \, ds = 0 \quad (10.163b)$$

for all $v \in \mathcal{V}^h$ and $\mathbf{w} \in \mathcal{W}^h$. We note that the integrals over $\partial\tau$ are over the *faces* of the triangulation. To simplify the analysis (and the figures), for the rest of this section we consider only two-dimensional problems ($d = 2$), where faces are edges.

In the 2D setting, the interface terms involve summation over all edges of each element $\tau \in \mathcal{T}^h$. This is tricky to express as a global operation, since interior edges are traversed twice, while edges on the boundary are only included once (see Figure 10.5.1). To simplify the expression, we first introduce two quantities that are also used in the description of specific numerical fluxes. Given an interior edge, e , denote the two adjacent elements by τ^1 and τ^2 . Further, let $u^i = u|_{\tau^i}$ and $\boldsymbol{\sigma}^i = \boldsymbol{\sigma}|_{\tau^i}$ be the quantities local to the elements for $i = 1, 2$. Then, let $\bar{u}^i$ and $\bar{\boldsymbol{\sigma}}^i$ be the

traces of u^i and σ^i evaluated on edge e and $\mathbf{n}^i = \mathbf{n}_{\tau_i}$ be the outward normal for element i . With this, define the *jump* and *average* of $\bar{u}$ and $\bar{\sigma}$ on edge e as

$$\textbf{jump:} \quad [u]_e = \bar{u}^1 \mathbf{n}^1 + \bar{u}^2 \mathbf{n}^2, \quad [\sigma]_e = \bar{\sigma}^1 \cdot \mathbf{n}^1 + \bar{\sigma}^2 \cdot \mathbf{n}^2, \quad (10.164a)$$

$$\textbf{average:} \quad \{u\}_e = \frac{\bar{u}^1 + \bar{u}^2}{2}, \quad \{\sigma\}_e = \frac{\bar{\sigma}^1 + \bar{\sigma}^2}{2}. \quad (10.164b)$$

The averages are exactly as we might expect, except that the jump terms “exchange” scalar and vector quantities: the jump in u is a vector (taking into account the differences between outward normal vectors on elements one and two), while the jump in σ is a scalar. In the remainder of the section, we drop the subscript when it is clear which edge is referenced in the jump or average.

To help with the summations in equation (10.163) in 2D, we denote the set of edges, boundary edges, and interior edges as

$$E = \cup_{\tau} \partial\tau, \quad E^{\partial\Omega} = \partial\Omega = E \cap \partial\Omega, \quad E^{\circ} = E \setminus E^{\partial\Omega} \quad (10.165)$$

and derive the following identity for any $v \in \mathcal{V}^h$, $\mathbf{w} \in \mathcal{W}^h$:

$$\sum_{\tau \in \mathcal{T}^h} \int_{\partial\tau} v \mathbf{w} \cdot \mathbf{n}_{\tau} ds = \sum_{\tau \in \mathcal{T}^h} \int_{\partial\tau \cap E^{\circ}} v \mathbf{w} \cdot \mathbf{n}_{\tau} ds + \sum_{\tau \in \mathcal{T}^h} \int_{\partial\tau \cap E^{\partial\Omega}} v \mathbf{w} \cdot \mathbf{n}_{\tau} ds \quad (10.166a)$$

$$= \sum_{e \in E^{\circ}} \int_e \bar{v}^1 \bar{\mathbf{w}}^1 \cdot \mathbf{n}^1 + \bar{v}^2 \bar{\mathbf{w}}^2 \cdot \mathbf{n}^2 ds + \underbrace{\sum_{e \in E^{\partial\Omega}} \int_e v \mathbf{w} \cdot \mathbf{n} ds}_B \quad (10.166b)$$

$$= \sum_{e \in E^{\circ}} \frac{1}{2} \int_e \bar{v}^1 \mathbf{n}^1 \cdot \bar{\mathbf{w}}^1 + \bar{v}^1 \mathbf{n}^1 \cdot \bar{\mathbf{w}}^1 + \bar{v}^2 \mathbf{n}^2 \cdot \bar{\mathbf{w}}^2 + \bar{v}^2 \mathbf{n}^2 \cdot \bar{\mathbf{w}}^2$$

$$+ \underbrace{\bar{v}^1 \mathbf{n}^1 \cdot \bar{\mathbf{w}}^2 + \bar{v}^1 \mathbf{n}^2 \cdot \bar{\mathbf{w}}^2}_{=0 \text{ } (\mathbf{n}^1 = -\mathbf{n}^2)} + \underbrace{\bar{v}^2 \mathbf{n}^2 \cdot \bar{\mathbf{w}}^1 + \bar{v}^2 \mathbf{n}^1 \cdot \bar{\mathbf{w}}^1}_{=0 \text{ } (\mathbf{n}^1 = -\mathbf{n}^2)} ds$$

$$+ B \quad (10.166c)$$

$$= \sum_{e \in E^{\circ}} \frac{1}{2} \int_e \bar{v}^1 \mathbf{n}^1 \cdot \bar{\mathbf{w}}^1 + \bar{v}^1 \mathbf{n}^1 \cdot \bar{\mathbf{w}}^2 + \bar{v}^2 \mathbf{n}^2 \cdot \bar{\mathbf{w}}^2 + \bar{v}^2 \mathbf{n}^2 \cdot \bar{\mathbf{w}}^1$$

$$+ \bar{v}^1 \mathbf{n}^1 \cdot \bar{\mathbf{w}}^1 + \bar{v}^2 \mathbf{n}^1 \cdot \bar{\mathbf{w}}^1 + \bar{v}^1 \mathbf{n}^2 \cdot \bar{\mathbf{w}}^2 + \bar{v}^2 \mathbf{n}^2 \cdot \bar{\mathbf{w}}^2 ds$$

$$+ B \quad (10.166d)$$

$$= \sum_{e \in E^{\circ}} \int_e (\bar{v}^1 \mathbf{n}^1 + \bar{v}^2 \mathbf{n}^2) \cdot \frac{1}{2} (\bar{\mathbf{w}}^1 + \bar{\mathbf{w}}^2)$$

$$+ \frac{1}{2} (\bar{v}^1 + \bar{v}^2) (\bar{\mathbf{w}}^1 \cdot \mathbf{n}^1 + \bar{\mathbf{w}}^2 \cdot \mathbf{n}^2) ds + B \quad (10.166e)$$

$$= \sum_{e \in E^{\circ}} \int_e [v] \cdot \{\mathbf{w}\} + \{v\} \cdot [\mathbf{w}] ds + \sum_{e \in E^{\partial\Omega}} \int_e v \mathbf{w} \cdot \mathbf{n} ds, \quad (10.166f)$$

where we have used the fact that $\mathbf{n}^1 = -\mathbf{n}^2$. Thus, we write the integration by parts result as

$$\int_{\Omega} v \nabla \cdot \mathbf{w} dx = - \int_{\Omega} \nabla v \cdot \mathbf{w} dx + \sum_{e \in E^{\circ}} \int_e [v] \cdot \{\mathbf{w}\} + \{v\} \cdot [\mathbf{w}] ds + \sum_{e \in E^{\partial\Omega}} \int_e v \mathbf{w} \cdot \mathbf{n} ds. \quad (10.167)$$

Note that the same argument yields the same identity in three dimensions, but with summation over element faces, not edges.

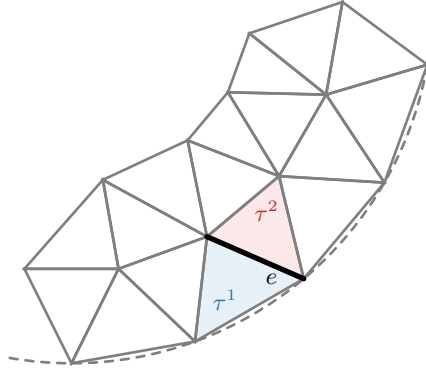


Figure 10.5.1. Two elements, τ^1 and τ^2 , with a shared edge e , where the true boundary of the domain is highlighted by a dashed line.

Our next step is to combine the system in equation (10.163) to arrive at single weak form in the scalar variable u (in contrast to the system of weak forms resulting from a FOSLS or mixed Galerkin approach in the previous sections). First, we rewrite equation (10.163) in terms of the edge integrals using the identity in equation (10.166):

$$\begin{aligned} \int_{\Omega} \boldsymbol{\sigma} \cdot \nabla v \, dx - \sum_{e \in E^{\circ}} \int_e [v] \cdot \{\widehat{\boldsymbol{\sigma}}\} + \{v\} \cdot [\widehat{\boldsymbol{\sigma}}] \, ds - \sum_{e \in E^{\partial\Omega}} \int_e v \widehat{\boldsymbol{\sigma}} \cdot \mathbf{n} \, ds \\ = \int_{\Omega} f v \, dx, \end{aligned} \quad (10.168a)$$

$$\begin{aligned} \int_{\Omega} \boldsymbol{\sigma} \cdot \mathbf{w} \, dx + \int_{\Omega} u \nabla \cdot (K\mathbf{w}) \, dx - \sum_{e \in E^{\circ}} \int_e [\widehat{u}] \cdot \{K\mathbf{w}\} + \{\widehat{u}\} \cdot [K\mathbf{w}] \, ds \\ - \sum_{e \in E^{\partial\Omega}} \int_e \widehat{u} K\mathbf{w} \cdot \mathbf{n} \, ds = 0. \end{aligned} \quad (10.168b)$$

We rewrite equation (10.168b) using equation (10.167) in order to get a new expression for $\int_{\Omega} \boldsymbol{\sigma} \cdot \mathbf{w} \, dx$:

$$\begin{aligned} \int_{\Omega} \boldsymbol{\sigma} \cdot \mathbf{w} \, dx - \int_{\Omega} \nabla u \cdot (K\mathbf{w}) \, dx - \sum_{e \in E^{\circ}} \int_e [\widehat{u} - u] \cdot \{K\mathbf{w}\} + \{\widehat{u} - u\} \cdot [K\mathbf{w}] \, ds \\ - \sum_{e \in E^{\partial\Omega}} \int_e (\widehat{u} - u) K\mathbf{w} \cdot \mathbf{n} \, ds = 0 \end{aligned} \quad (10.169a)$$

$$\begin{aligned} \Rightarrow \int_{\Omega} \boldsymbol{\sigma} \cdot \mathbf{w} \, dx = \int_{\Omega} \nabla u \cdot (K\mathbf{w}) \, dx + \sum_{e \in E^{\circ}} \int_e [\widehat{u} - u] \cdot \{K\mathbf{w}\} + \{\widehat{u} - u\} \cdot [K\mathbf{w}] \, ds \\ + \sum_{e \in E^{\partial\Omega}} \int_e (\widehat{u} - u) K\mathbf{w} \cdot \mathbf{n} \, ds. \end{aligned} \quad (10.169b)$$

Finally, using equation (10.169b) for the first term of equation (10.168a) with $\mathbf{w} = \nabla v$ yields a single weak form over the scalar variable:

$$\begin{aligned} \int_{\Omega} \nabla u \cdot K \nabla v \, d\mathbf{x} + \sum_{e \in E^{\circ}} \int_e [\hat{u} - u] \cdot \{K \nabla v\} + \{\hat{u} - u\} \cdot [K \nabla v] - [v] \cdot \{\hat{\sigma}\} - \{v\} \cdot [\hat{\sigma}] \, ds \\ + \sum_{e \in E^{\partial\Omega}} \int_e (\hat{u} - u) K \nabla v \cdot \mathbf{n} - v \hat{\sigma} \cdot \mathbf{n} \, ds = \int_{\Omega} f v \, d\mathbf{x}. \end{aligned} \quad (10.170)$$

At this point, we have the scaffolding for a DG method, but to fully define the approach requires defining fluxes $\hat{u}$ and $\hat{\sigma}$ for both interior and boundary edges. With two values $\bar{u}^1$ and $\bar{u}^2$ defined for u on edge e and two values $\bar{\sigma}^1$ and $\bar{\sigma}^2$ defined for σ on edge e , one might be tempted to approximate the numerical flux values with the average:

$$\hat{u} = \{u\}, \quad \hat{\sigma} = \{\sigma\}. \quad (10.171)$$

However, this results in an unstable weak form, requiring additional constraints on the interelement jump in the solution u . To address this, the symmetric interior penalty method form of DG incorporates this as a penalty in the form

$$\hat{u} = \{u\}, \quad \hat{\sigma} = \{\sigma\} - \mu_e [u] \quad \text{for } e \in E^{\circ}, \quad (10.172)$$

where μ_e is a positive weight parameter for edge e . On the boundary, we use

$$\hat{u} = 0 \quad \text{for } e \in E^{\partial\Omega}, \quad (10.173)$$

where we do not need a similar definition for $\hat{\sigma}$ due to the Dirichlet boundary terms on u eliminating the term that it contributes to in equation (10.170).

It is important to note that the flux is defined on an *edge*, requiring care in defining terms such as $\{\hat{u}\}$. Nevertheless, since the average and the jump are defined over *traces*, we can easily summarize these as

$$\{\hat{\sigma}\} = \{K \nabla u\} - \mu_e [u], \quad (10.174a)$$

$$[\hat{\sigma}] = 0. \quad (10.174b)$$

Extending this to the $\hat{u} - u$ terms, we have

$$\{\hat{u} - u\} = 0 \quad \text{for } e \in E^{\circ}, \quad (10.175a)$$

$$\hat{u} - u = 0 \quad \text{for } e \in E^{\partial\Omega}, \quad (10.175b)$$

$$[\hat{u} - u] = -[u] \quad \text{for } e \in E^{\circ}. \quad (10.175c)$$

Combining these terms with equation (10.170) and noting that we use Dirichlet boundary conditions so that $v = 0$ on $\partial\Omega$, we have

$$a(u, v) = \int_{\Omega} \nabla u \cdot K \nabla v \, d\mathbf{x} - \sum_{e \in E^{\circ}} \int_e [u] \cdot \{K \nabla v\} + [v] \cdot \{K \nabla u\} + \sum_{e \in E^{\circ}} \int_e \mu_e [v] \cdot [u] \, ds. \quad (10.176)$$

The weak form $a(u, v)$ is, indeed, symmetric, and a range of works have established stability and accuracy properties [17, 189] with a dependence on μ_e .

The penalty parameter, μ_e , is in place to stabilize the weak form of the problem. Small penalties lead to spurious solutions, while large penalties impact accuracy and conditioning (but remain stable)—see [133]. Consequently, there has been a focus on identifying precise values

for the stability parameter in order to balance this effect. Typically, μ_e will depend on properties of the neighboring elements of edge e , τ^1 and τ^2 . For example,

$$\mu_e = \alpha k^2 \max(h_{\tau^1}^{-1}, h_{\tau^2}^{-1}), \quad (10.177)$$

with h_τ denoting the diameter of element τ and k being the order of the discontinuous polynomials used. This form relies on selecting an adequate constant α , which can be difficult to identify in practice. A more direct form for μ_e , found in [197] for the case of constant $K(\mathbf{x})$, is written as the following (in two dimensions):

$$\mu_e = \frac{(k+1)(k+2)}{2} \max(\alpha_{\tau^1}, \alpha_{\tau^2}), \quad (10.178)$$

where α_τ is defined through edge lengths and the area of τ :

$$\alpha_\tau = \frac{|\partial\tau \setminus \partial\Omega| + 2|\partial\tau \cap \partial\Omega|}{2 \text{ area}(\tau)}. \quad (10.179)$$

Similar expressions are defined in one and three dimensions—see [197, Sec. 3].

10.5.1 ■ Example

As an example, consider equation (10.158) in two dimensions with $K(\mathbf{x}) = 1$ and solution

$$u(x, y) = \sin(\pi x) \sin(3\pi y). \quad (10.180)$$

While the exact solution is smooth, there is no such requirement on the numerical approximation. In fact, u^h is generally *discontinuous* since we have constructed u^h as a piecewise polynomial and have imposed interelement constraints or penalties for stability. Figure 10.5.2 highlights this for the case of $k = 1$ (linears) for two different choices of the penalty parameter; both are stable but offer increased penalty in the interelement jumps in the solution. Here, μ_e is set globally as αh^{-1} , where h is the minimum diameter of an element in the mesh. We see for $\alpha = 10$, for example, that the approximation retains significant jumps between elements. With stronger penalty terms (for example, $\alpha = 100$), the solution remains stable; however, loss of accuracy can occur.

Much of the analysis of approximation properties from previous sections applies here, and we expect an error dependence on h as well as k (the order of polynomials used). Figure 10.5.3

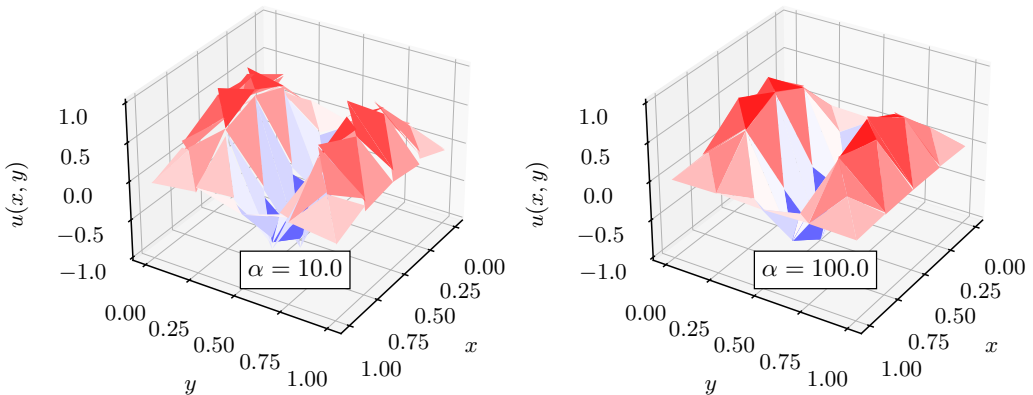


Figure 10.5.2. Two solutions with 64 elements and $K(\mathbf{x}) = 1$. (left) $\alpha = 10$ and (right) $\alpha = 100$.

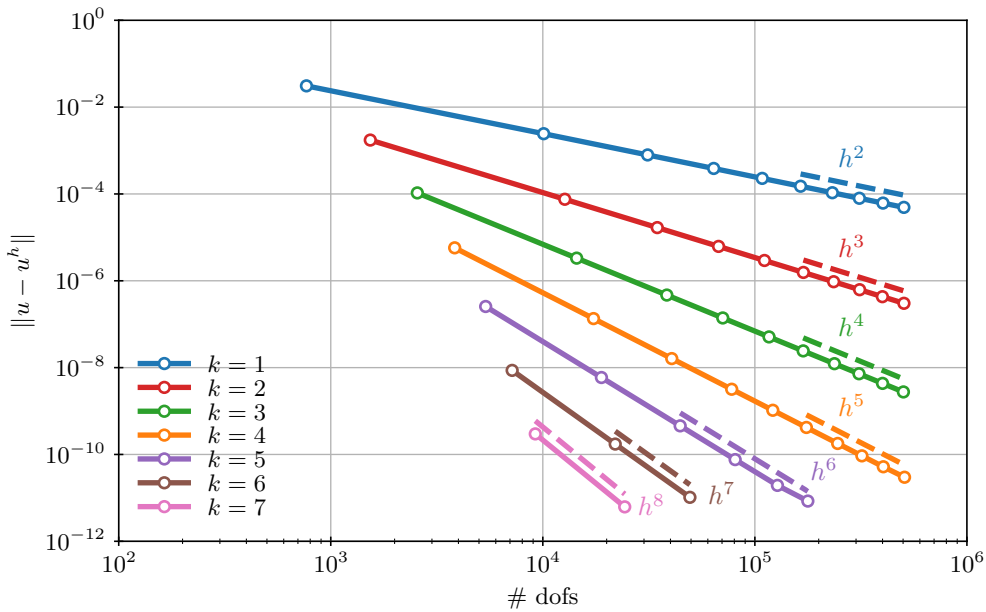


Figure 10.5.3. Convergence in the L^2 norm for the manufactured solution in equation (10.180) with $\mu_e = 100/h$. Solid lines represent computed $L^2(\Omega)$ errors in the approximation. Dashed lines represent the theoretical prediction of h^{k+1} for polynomial order k .

shows the decrease in error for each polynomial order k as the mesh size h is decreased. Here, we see that the L^2 error behaves as $\|u - u^h\| = \mathcal{O}(h^{k+1})$. Here, since different orders lead to different counts of degrees of freedom, we plot measured error vs. the number of degrees of freedom (# dofs), starting each order on an 8×8 mesh cut into triangles and refining from there.

10.6 ■ Spectral methods

Objectives

For the finite-difference discretizations of Chapter 3, the focus is on *approximating* derivative operators to find approximate solutions to PDEs. The *spectral methods* considered in this section take a second approach, directly approximating solutions without approximating the differential operator. Thus, in this section you will

- outline the basic form of a spectral method;
- connect the ideas in spectral approximations to those of Fourier series;
- analyze the high accuracy of spectral methods; and
- develop an example of efficient algorithms for arriving at spectral approximations.

Spectral methods play an important role in the numerical approximation of PDE solutions, efficiently offering unparalleled accuracy for certain problems, particularly those on one-dimensional periodic domains. They lose some of their advantages when considering problems with less smooth or nonperiodic solutions. In this section, we review the key ideas and technologies of spectral methods, but we do not provide a comprehensive overview. For further details on

spectral methods, we direct the reader to the excellent textbooks [110, 132, 217] and also note the closely related pseudospectral methods described, for example, in [99]. The presentation here focuses on the periodic case, as presented in [159, Ch. 5] and [132, Ch. 2], but the theory and methods extend to many other cases. Corresponding results for the infinite-domain case are presented in [217, Ch. 4], for example, and for finite domains with more general boundary conditions in [132, Ch. 6].

10.6.1 ■ Spectral discretization

We develop the spectral discretization methodology for a *periodic* model problem in 1D,

$$-u'' + bu = f \quad \text{for } 0 < x < 2\pi, \quad (10.181a)$$

$$u(0) = u(2\pi), \quad (10.181b)$$

$$u'(0) = u'(2\pi), \quad (10.181c)$$

where $b \geq 0$ is a constant, and we consider the interval of length 2π for convenience. The solution to equation (10.181) can be written in terms of a Fourier series as

$$u(x) = \sum_{k=-\infty}^{\infty} u_k \phi_k(x), \quad (10.182)$$

where $\phi_k(x) = \frac{1}{\sqrt{2\pi}} e^{ikx}$ with $i^2 = -1$. Recalling Definition 8.8, PDE theory tells us that any function $u \in L^2([0, 2\pi])$ can be written in the form of equation (10.182), with the coefficients u_k defined by

$$u_k = \int_0^{2\pi} u(x) \phi_{-k}(x) dx. \quad (10.183)$$

Another standard result establishes that the series representation in equation (10.182) converges in $L^2([0, 2\pi])$, although not pointwise if $u(x)$ has any discontinuities. A straightforward approximation is to truncate the sum and to write

$$u(x) \approx \sum_{k=-n}^n u_k \phi_k(x). \quad (10.184)$$

In the following, we ask the following question: when is this a good approximation to $u(x)$? We note that when we replace the boundary conditions in equation (10.181) with Dirichlet, Neumann, or Robin conditions, the same expansion and approximation can be considered, replacing the Fourier functions $\{\phi_k(x)\}$ with other orthogonal bases, such as solutions to the associated Sturm–Liouville problem [132, Ch. 4].

The quality of the approximation in equation (10.184) depends on the convergence of the Fourier series representation of $u(x)$, which in turn depends on the smoothness (and periodicity) of $u(x)$ itself. To make this precise, we identify an appropriate *periodic* normed function space that we use in the theory below. For integer $s > 0$, we recall the standard Sobolev space from Definition 8.16 on $[0, 2\pi]$ as

$$H^s([0, 2\pi]) = \{u : [0, 2\pi] \rightarrow \mathbb{R} \mid D_w^k u \in L^2([0, 2\pi]) \text{ for } 0 \leq k \leq s\}, \quad (10.185)$$

with D_w^k representing the k th weak derivative, as in Definition 8.14. The associated norm is

$$\|u\|_s^2 = \sum_{k \leq s} \|D_w^k u\|_0^2. \quad (10.186)$$

We note that requiring $u \in H^s([0, 2\pi])$ is enough to ensure that the weak derivatives of u up to order $s - 1$ are also strong derivatives, since they must be continuous on $[0, 2\pi]$; see [2]. Therefore, we extend the definition of $H^s([0, 2\pi])$ by enforcing degree- $(s - 1)$ continuity at the periodic boundary and by using the same norm. The resulting periodic Sobolev space is the appropriate generalization of $H_0^1([0, 2\pi])$ defined in equation (8.38) to the periodic problem in equation (10.181) and is given in Definition 10.28.

Definition 10.28: Periodic Sobolev spaces

For integer $s > 0$, the periodic Sobolev space is defined by

$$H_p^s([0, 2\pi]) = \left\{ u \in H^s([0, 2\pi]) \mid u^{(j)}(0) = u^{(j)}(2\pi) \text{ for } 0 \leq j \leq s - 1 \right\}, \quad (10.187)$$

with the norm $\|u\|_s$ defined by H^s (see equation (10.186)).

The essential ingredients in what follows are the amount of smoothness in u and that both u and its derivatives are periodic. This allows us to integrate by parts in the definition of the Fourier coefficients of u to get

$$u_k = \int_0^{2\pi} u(x) \frac{e^{-ikx}}{\sqrt{2\pi}} dx = \frac{-1}{-ik} \int_0^{2\pi} u'(x) \frac{e^{-ikx}}{\sqrt{2\pi}} dx \quad (10.188a)$$

$$= \cdots = \frac{1}{(ik)^s} \int_0^{2\pi} u^{(s)}(x) \frac{e^{-ikx}}{\sqrt{2\pi}} dx. \quad (10.188b)$$

This immediately leads to

$$|u_k| \leq \frac{\|u^{(s)}\|_0}{|k|^s}, \quad (10.189)$$

where the inequality follows from the Cauchy–Schwarz inequality applied to the final integral above.

A key feature of spectral discretizations is the *projection* of a function, u , onto a finite Fourier series. Given a function, u , we define the projection $\Pi_n u$ by

$$(\Pi_n u)(x) = \sum_{k=-n}^n u_k \phi_k(x) \quad (10.190)$$

for any integer $n > 0$. Then, Theorem 10.29 gives a useful bound on the error made by approximating u by $\Pi_n u$.

Theorem 10.29: Fourier projection error (cf. [159, Thm. 5.2])

Let $u \in H_p^s([0, 2\pi])$ for $s \geq 2$ and $n > 0$. Then,

$$\|u - \Pi_n u\|_r \leq \frac{\sqrt{2}}{\sqrt{(2s-3)}} n^{r-s+1/2} \|u^{(s)}\|_0 \text{ for } r = 0, 1. \quad (10.191)$$

Proof. Using the fact that $\int_0^{2\pi} \phi_k(x) \phi_\ell(x) dx = 0$ for $\ell \neq k$ and equation (10.189), we have

$$\|u - \Pi_n u\|_0^2 = \sum_{|k| > n} |u_k|^2 \leq \|u^{(s)}\|_0^2 \sum_{|k| > n} \frac{1}{|k|^{2s}}. \quad (10.192)$$

The sum in this case is over both positive and negative k , and each sum can be bounded by an

integral, giving

$$\|u - \Pi_n u\|_0^2 \leq 2\|u^{(s)}\|_0^2 \int_n^\infty \frac{dx}{x^{2s}} = 2\|u^{(s)}\|_0^2 \left(\frac{n^{1-2s}}{2s-1} \right). \quad (10.193)$$

By a similar argument, but noting that we need $s \geq 2$ for the sum to be convergent, we have

$$\|u' - (\Pi_n u)'\|_0^2 \leq \sum_{|k|>n} k^2 |u_k|^2 \leq 2\|u^{(s)}\|_0^2 \left(\frac{n^{3-2s}}{(2s-3)} \right). \quad (10.194)$$

□

The key idea behind a spectral method is that if $u \in H_p^s([0, 2\pi])$ for $s > 0$, then we achieve fast convergence of $\Pi_n u$ to u , so approximating u by its truncated Fourier expansion can be very efficient (and gets more efficient for smoother functions with larger s). We also note that if u satisfies stronger criteria—e.g., if there is an extension of u to the complex plane that is analytic on a slab $[0, 2\pi] \times [-\beta, \beta]$ —then we can obtain even faster convergence (see [159, Thm. 5.3]).

If all calculations in a spectral method are computed analytically (without finite-precision arithmetic), then we have a full specification of an effective method. To solve equation (10.181) where $b \geq 0$ is a constant, we first compute $f_k = \int_0^{2\pi} f(x)\phi_{-k}(x)dx$ for $-n \leq k \leq n$ and then take $u_k = f_k/(k^2 + b)$, and we know (from Theorem 10.29) that $(\Pi_n u)(x) = \sum_{k=-n}^n u_k \phi_k(x)$ is a good approximation to $u(x)$. We note that if $b = 0$, then we require $f_0 = \int_0^{2\pi} f(x)dx = 0$, and we assign $u_0 = 0$ to deal with the singularity of the periodic Laplace problem in a consistent way. If $u \in H_p^s([0, 2\pi])$, then we obtain a fixed-accuracy approximation to u with many fewer degrees of freedom using $\Pi_n u$ than with a finite-difference or piecewise linear finite-element approximation, with an $L^2([0, 2\pi])$ error bound like $n^{-s+1/2}$ instead of n^{-2} . We also note that the convergence bound depends only on u having sufficient smoothness (and periodicity) and not on our *knowledge* of how much smoothness u has. This is in contrast to higher-order finite-element methods, where our choice of function space depends on the smoothness we expect u to have in order to realize the possibility of achieving similar convergence bounds from Céa's lemma (Lemma 8.31) and Theorem 8.35.

10.6.2 ■ High-order spectral differencing

The complication in the above is that analytically evaluating the integrals $\int_0^{2\pi} f(x)\phi_{-k}(x)dx$ is computationally infeasible, necessitating the use of a quadrature approximation to the integral. We do this by implicitly defining a quadrature rule as follows. Given $n > 0$, let $h = 2\pi/(2n+1)$ and $x_j = jh$ for $0 \leq j \leq 2n$. For a given $u(x)$, we define $\{\tilde{u}_k\}_{k=-n}^n$ such that

$$u(x_j) = \sum_{k=-n}^n \tilde{u}_k \phi_k(x_j) \text{ for } j = 0, \dots, 2n. \quad (10.195)$$

In Section 10.6.3, we explore how we can evaluate $\{\tilde{u}_k\}$ by defining a $(2n+1) \times (2n+1)$ matrix, U , with entries $U_{j,k} = \phi_k(x_j)$, for which we efficiently compute either $\mathbf{u} = U\tilde{\mathbf{u}}$ or $\tilde{\mathbf{u}} = U^{-1}\mathbf{u}$, where $\mathbf{u}$ is the vector of function values at points x_j and $\tilde{\mathbf{u}}$ is the vector of coefficients. Defining the *Fourier interpolant* as $(I_n u)(x) = \sum_{k=-n}^n \tilde{u}_k \phi_k(x)$, we measure how well $I_n u$ approximates u —see Theorem 10.30.

Theorem 10.30: Fourier interpolation error (cf. [159, Thm. 5.4])

Let $u \in H_p^s([0, 2\pi])$ for $s \geq 2$ and $n > 0$. Then, there exists a constant $c > 0$ such that

$$\|u - I_n u\|_r \leq cn^{r-s+1/2} \|u^{(s)}\|_0 \text{ for } r = 0, 1. \quad (10.196)$$

Proof. This is an exercise in converging series, similar to that of Theorem 10.29. $\square$

With Theorem 10.30, we now have a feasible spectral method for equation (10.181) (when $b \geq 0$ is a constant). First, define $\{\tilde{f}_k\}_{k=-n}^n$ such that

$$f(x_j) = \sum_{k=-n}^n \tilde{f}_k \phi_k(x_j) \text{ for } j = 0, \dots, 2n. \quad (10.197)$$

Then, compute $\tilde{u}_k = \tilde{f}_k/(k^2 + b)$ for $-n \leq k \leq n$, noting that if $b = 0$, then we require that $\tilde{f}_0 = 0$ and assign $\tilde{u}_0 = 0$. Finally, set the interpolant as

$$(I_n u)(x) = \sum_{k=-n}^n \tilde{u}_k \phi_k(x). \quad (10.198)$$

In order to achieve a pointwise approximation of $I_n u$, we consider evaluating it at the points $\{x_j\}$ as

$$(I_n u)(x_j) = \sum_{k=-n}^n \tilde{u}_k \phi_k(x_j) \text{ for } j = 0, \dots, 2n. \quad (10.199)$$

The accuracy is summarized in Theorem 10.31.

Theorem 10.31: Spectral method accuracy (cf. [159, Thm. 5.7])

Let $n > 0$ be given, and suppose that $f \in H_p^s([0, 2\pi])$ for $s \geq 1$. Let $b \geq 0$ and $u \in H_p^{s+2}([0, 2\pi])$ be the solution of equation (10.181), and define $I_n u$ by equation (10.198). Then, there exists a constant $c > 0$ such that

$$\|u - I_n u\|_1 \leq cn^{-s} \|f^{(s)}\|_0. \quad (10.200)$$

Proof. By direct calculation, we write

$$\|u - I_n u\|_0^2 = \sum_{k=-n}^n |u_k - \tilde{u}_k|^2 + \sum_{|k| > n} |u_k|^2. \quad (10.201)$$

We bound the second term in the sum using Theorem 10.29, first writing $|u_k| = |f_k|/(k^2 + b)$ and then noting that $f \in H_p^s([0, 2\pi])$. For the first term in the sum, we rewrite u_k and $\tilde{u}_k$ in terms of f_k and $\tilde{f}_k$ as

$$\sum_{k=-n}^n |u_k - \tilde{u}_k|^2 = \sum_{k=-n}^n \frac{|f_k - \tilde{f}_k|^2}{(k^2 + b)^2}. \quad (10.202)$$

Then, using techniques similar to those from Theorem 10.30, we show that

$$\sum_{k=-n}^n \frac{|f_k - \tilde{f}_k|^2}{(k^2 + b)^2} \leq c \frac{\|f^{(s)}\|_0^2}{n^{2s}}. \quad (10.203)$$

A similar calculation holds for $\|u' - (I_n u)'\|_0^2$, giving the stated result. $\square$

We end by noting that this technique is only beneficial when we have a constant-coefficient differential operator and very smooth solutions to the resulting PDE. Even so, this provides accuracy like n^{-s} at the cost of just two matrix-vector products, with matrices U and U^{-1} plus $2n + 1$ scalar divisions. Thus, when s is large, a spectral method can be highly accurate and computationally efficient.

10.6.3 ■ The fast Fourier transform

We now turn our attention to deriving algorithms for the efficient application of matrix-vector products using the matrix U or its inverse, defined above as $U_{j,k} = \phi_k(x_j)$ for $-n \leq k \leq n$ and $x_j = 2\pi j/(2n + 1)$ for $0 \leq j \leq 2n$. Writing this out, we have

$$U_{j,k} = \frac{1}{\sqrt{2\pi}} e^{i \frac{2\pi jk}{2n+1}} \text{ for } 0 \leq j \leq 2n, \quad -n \leq k \leq n. \quad (10.204)$$

We first note that these entries are periodic in both j and k with period $2n + 1$. It simplifies notation to permute the indices so that we consider $0 \leq k \leq 2n$ rather than $-n \leq k \leq n$; however, it is important to realize that while this permutation serves to simplify notation in what follows, it plays a real role in the implementation of algorithms for working with U . As a result, we must remain aware of this permutation when we implement the algorithms that result.

With this column permutation, we write $w = e^{i2\pi/(2n+1)}$, giving $U_{j,k} = \frac{1}{\sqrt{2\pi}} w^{jk}$:

$$U = \frac{1}{\sqrt{2\pi}} \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & w & w^2 & \cdots & w^{2n} \\ 1 & w^2 & w^4 & \cdots & w^{4n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & w^{2n} & w^{4n} & \cdots & w^{(2n)^2} \end{bmatrix}. \quad (10.205)$$

One reason this notation is convenient is that $w^{2n+1} = 1$, directly from the definition of w . Moreover, if we consider the matrix-matrix product UU^H , where U^H is the Hermitian-transpose of U , we see that

$$(UU^H)_{j,\ell} = \frac{1}{2\pi} \sum_{k=0}^{2n} w^{jk} \overline{w^{k\ell}} = \frac{1}{2\pi} \sum_{k=0}^{2n} e^{i \frac{2\pi}{2n+1} (j-\ell)k}. \quad (10.206)$$

Writing $\rho = e^{i2\pi(j-\ell)/(2n+1)}$, we reveal two possibilities for the sum:

1. If $\rho = 1$, then $\sum_{k=0}^{2n} \rho^k = 2n + 1$.
2. If $\rho \neq 1$, then $\sum_{k=0}^{2n} \rho^k = \frac{1-\rho^{2n+1}}{1-\rho} = 0$, since $\rho^{2n+1} = 1$ just as w^{2n+1} does.

Thus, we have that $UU^H = \frac{2n+1}{2\pi} I$, so $U^{-1} = \frac{2\pi}{2n+1} U^H$.

While the above provides a useful relation between U and U^{-1} , we still need to answer our initial question regarding efficient computation of matrix-vector products with U and U^{-1} . The algorithm to do this is most easily expressed using an even number of points, so we first extend the summations above to $2n + 2$ points. We extend the definitions of $\{\tilde{u}_k\}$ and $\{\tilde{f}_k\}$ by an additional index, which not only increases the resulting accuracy but also allows us to define an algorithm that applies to vectors of even length. Specifically, equations (10.195) and (10.197) become

$$u(x_j) = \sum_{k=-n-1}^n \tilde{u}_k \phi_k(x_j) \quad \text{and} \quad f(x_j) = \sum_{k=-n-1}^n \tilde{f}_k \phi_k(x_j), \quad (10.207)$$

with $x_j = \frac{2\pi j}{2n+2}$ for $0 \leq j \leq 2n + 1$. As a result, taking $w = e^{i2\pi/(2n+2)}$, similar relations to those above still hold. In particular, we consider a generic matrix multiplication by U , writing

$$\begin{bmatrix} z_0 \\ z_1 \\ z_2 \\ \vdots \\ z_{2n+1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & w & w^2 & \cdots & w^{2n+1} \\ 1 & w^2 & w^4 & \cdots & w^{4n+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & w^{2n} & w^{4n} & \cdots & w^{(2n+1)^2} \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{2n+1} \end{bmatrix}, \quad (10.208)$$

which gives us $z_j = \sum_{k=0}^{2n+1} w^{jk} y_k$ for $0 \leq j \leq 2n + 1$.

Next, we split vector $\mathbf{y}$ into two subsequences, those entries with even index, $\{y_{2k_1}\}_{k_1=0}^n$, and those entries with odd index, $\{y_{2k_1+1}\}_{k_1=0}^n$. With this, we split the calculation of z_j for $0 \leq j \leq 2n + 1$ into two sums, writing

$$z_j = \sum_{k_1=0}^n w^{2jk_1} y_{2k_1} + \sum_{k_1=0}^n w^{j(2k_1+1)} y_{2k_1+1} = \sum_{k_1=0}^n w^{2jk_1} y_{2k_1} + w^j \sum_{k_1=0}^n w^{2jk_1} y_{2k_1+1}. \quad (10.209)$$

There are two cases. For $0 \leq j \leq n$, we directly consider the formula above. For $n + 1 \leq j \leq 2n + 1$, we write $j = n + 1 + j_1$ for $0 \leq j_1 \leq n$, giving

$$z_{n+1+j_1} = \sum_{k_1=0}^n w^{2(n+1+j_1)k_1} y_{2k_1} + w^{n+1+j_1} \sum_{k_1=0}^n w^{2(n+1+j_1)k_1} y_{2k_1+1} \quad (10.210a)$$

$$= \sum_{k_1=0}^n w^{2j_1 k_1} y_{2k_1} + w^{n+1+j_1} \sum_{k_1=0}^n w^{2j_1 k_1} y_{2k_1+1}. \quad (10.210b)$$

Recognizing that the two summations compute the same terms, we compute *both* z_{j_1} and z_{n+1+j_1} for $0 \leq j_1 \leq n$ by computing two sums over $n + 1$ entries plus two scalar multiplications for the weights, w^{j_1} and w^{n+1+j_1} , (known as *twiddle factors*) and two additions. We now apply this idea recursively to computing

$$\sum_{k_1=0}^n w^{2j_1 k_1} y_{2k_1} \quad \text{and} \quad \sum_{k_1=0}^n w^{2j_1 k_1} y_{2k_1+1} \quad \text{for } 0 \leq j_1 \leq n. \quad (10.211)$$

If $n + 1$ is also even, we do this again to compute each sum; if $(n + 1)/2$ is still even, we again recurse. Considering, then, the case where $2n + 2 = 2^\ell$ for integer ℓ , we build a model for the cost of the algorithm from the bottom up. For $\ell = 1$, the direct cost of computing the two entries is two multiply-add operations. For $\ell = 2$ ($2n + 2 = 4$), the cost is twice that for $\ell = 1$, plus

four multiply-add operations to combine the results. Similarly, for $\ell = 3$ ($2n + 2 = 8$), the cost is twice that for $\ell = 2$, plus eight multiply-add operations to combine the results. Thus, if $C(\ell)$ is the cost when $2n + 2 = 2^\ell$, we have the recurrence relation that $C(\ell) = 2C(\ell - 1) + 2^\ell$, with $C(1) = 2$. This is easily solved to give an explicit expression for $C(\ell)$, which we summarize in Theorem 10.32.

Theorem 10.32: Fast Fourier transform cost (cf. [132, Sec. 11.1.1])

If $C(1) = 2$ and $C(\ell) = 2C(\ell - 1) + 2^\ell$, then $C(\ell) = \ell 2^\ell$.

Proof. This follows from a simple proof by induction. For the base case, note that $C(1) = 1 \cdot 2^1 = 2$. For the inductive step, assume that $C(\ell - 1) = (\ell - 1)2^{\ell-1}$. Then,

$$C(\ell) = 2C(\ell - 1) + 2^\ell = (\ell - 1)2^\ell + 2^\ell = \ell 2^\ell. \quad (10.212)$$

□

There are several things to highlight here. First, the algorithm above is easily extended to other lengths of vectors, either based on prime factorizations of the vector length or using “zero padding” to extend vectors to appropriate lengths. Second, we can use a variant of this algorithm for multiplication by U^{-1} as well, so that the resulting cost for a spectral method using this *fast Fourier transform* is $\mathcal{O}(n \log_2 n)$ for data of length $\mathcal{O}(n)$. When considering accuracy of the approximate solution versus computational cost, this yields a highly efficient approach when solutions to the PDE are both periodic and smooth. Extensions to similar cases, such as a Fourier sine basis for Dirichlet boundary conditions, are also possible using the same approach to fast summation.

10.6.4 ■ Fast Poisson solvers

A natural question is what can be done when solutions are not smooth enough that the accuracy of spectral methods pays off. Here, we use a “trick” based on the matrix structure of the standard finite-difference (or finite-volume or finite-element) discretization matrices. With periodic boundary conditions, for a simple differential equation, the discretization matrix is circulant—see Definition 10.33 and a specific case in Example 10.34.

Definition 10.33: Circulant matrix

An $n \times n$ matrix, A , is circulant if each entry can be written as $a_{i,j} = \alpha_{(i-j) \bmod n}$. That is,

$$A = \begin{bmatrix} \alpha_0 & \alpha_{n-1} & \alpha_{n-2} & \cdots & \alpha_1 \\ \alpha_1 & \alpha_0 & \alpha_{n-1} & \cdots & \alpha_2 \\ \alpha_2 & \alpha_1 & \alpha_0 & \cdots & \alpha_3 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \alpha_{n-1} & \cdots & \alpha_2 & \alpha_1 & \alpha_0 \end{bmatrix}. \quad (10.213)$$

Example 10.34: Circulant finite-difference matrix

Consider the finite-difference discretization matrix for $-u'' + bu = f$ for $0 < x < 1$ with periodic boundary conditions $u(0) = u(1)$ and $u'(0) = u'(1)$ on a uniform mesh with $h = 1/n_x$.

The discretization matrix is written as

$$L^h = \begin{bmatrix} \frac{2}{h^2} + b & -\frac{1}{h^2} & & & -\frac{1}{h^2} \\ -\frac{1}{h^2} & \frac{2}{h^2} + b & -\frac{1}{h^2} & & \\ & -\frac{1}{h^2} & \frac{2}{h^2} + b & -\frac{1}{h^2} & \\ & & \ddots & \ddots & \ddots \\ -\frac{1}{h^2} & & & -\frac{1}{h^2} & \frac{2}{h^2} + b \end{bmatrix}. \quad (10.214)$$

In the notation of Definition 10.33, we then have $\alpha_0 = 2/h^2 + b$, with $\alpha_1 = \alpha_{n_x-1} = -1/h^2$ and $\alpha_j = 0$ for $2 \leq j \leq n_x - 2$.

It is straightforward to show that the columns of the $n \times n$ matrix, U , with $w = e^{i2\pi/n}$ are the eigenvectors of any circulant matrix, and that the eigenvalues are given by a simple formula involving w and the matrix entries. As a consequence, if we want to solve $L^h \mathbf{u}^h = \mathbf{f}^h$ for a circulant discretization matrix, we have a simple algorithm based on writing $L^h = U\Lambda U^{-1}$, where Λ is the diagonal matrix of eigenvalues of L^h . Since $U\Lambda U^{-1} \mathbf{u}^h = \mathbf{f}^h$, we have $\mathbf{u}^h = U\Lambda^{-1}U^{-1} \mathbf{f}^h$. Thus, we compute $\mathbf{u}^h$ with the following steps:

1. Compute $\mathbf{g}^h = U^{-1} \mathbf{f}^h$ using the fast Fourier transform.
2. Scale by the diagonal matrix, computing $\Lambda^{-1} \mathbf{g}^h$.
3. Compute $\mathbf{u}^h = U (\Lambda^{-1} \mathbf{g}^h)$ using the fast Fourier transform.

We call this a “fast Poisson solver,” since its $\mathcal{O}(n \log_2 n)$ cost is much smaller than the $\mathcal{O}(n^3)$ cost of dense Gaussian elimination. Such an algorithm has a natural extension to the Dirichlet problem using the fast sine transform analogue mentioned above. Extensions to 2D problems are limited to discretizations that give so-called *block circulant matrices with circulant blocks*, which is true, for example, of finite differences on uniform tensor-product meshes with full periodic boundary conditions. Unfortunately, however, it is difficult to extend these algorithms to problems on complex geometries or where there are variable coefficients.

10.7 ■ Some further thoughts

One goal of this chapter was to give a preview of a variety of different discretizations that work for various problems. Even within one general methodology like the finite-element method, there are several “flavors.” In particular, as seen in Sections 10.2 and 10.3, numerous types of elements can be used in a standard Galerkin setting. The Periodic Table of the Finite Elements [19] gives a helpful graphical overview of these different element types. In Figure 10.7.1, we include a snippet of the table,¹ highlighting some of the elements we discuss in this book.

Similarly, there many variations of the finite-element formulations. We saw some examples in Sections 10.4 and 10.5 using least squares and discontinuous Galerkin, but there are many more. Interested readers can explore such approaches, including virtual elements [37], which incorporate nonpolynomial functions into the finite-element space, and generalized [89] or extended [39] finite-element methods which enrich the finite-element spaces with discontinuous functions allowing for better approximations of local behavior and singularities. The latter methods are based on partition-of-unity methods [170], which are useful for modeling crack and fracture formation, for instance. A more recent development of these ideas, CutFEM [61], allows the elements to be “cut” at boundaries or interfaces in order to fit the geometry. This reduces the need for mesh refinement to accurately resolve the solution at such features.

¹<https://www-users.cse.umn.edu/~arnold/femtable/>



Figure 10.7.1. The Periodic Table of the Finite Elements [19]. Distributed under CC BY 4.0 (<https://creativecommons.org/licenses/by/4.0/deed.en>).

Finally, we note that there are *many* other types of discretizations that we do not discuss in this book. Particle-based methods, agent-based modeling, and physics-informed neural networks (PINNs) are a few, and the vast number of application areas that involve PDEs also introduce countless additional methods and variants. PINNs and related methods, in particular, have been the subject of substantial study and development in recent years (see, for example, [183, 77, 62]). Note that, in its basic form, the PINNs method from [183] minimizes the least-squares finite-element functional in equation (10.115), with the finite-element representation of u replaced by a neural network representation and the optimization problem solved numerically by methods such as LBFGS or stochastic gradient descent. In [183], initial and boundary conditions are imposed weakly, similar to the approach in [80, 81, 65], but with ad hoc weights for the boundary functional. It has to be noted that, to date, classical methods (such as finite elements) still consistently outperform PINNs in many settings when considering the accuracy yielded versus the total computational costs incurred [114]. Clearly, the study of discretization methods for PDEs remains an extremely rich area of research.

Chapter 11

Advanced linear solvers

Overview

A main takeaway from Chapter 7 is that the use of Krylov methods and simple preconditioners based on matrix splittings is suboptimal in terms of total cost to solution for the linear systems arising from discretization of elliptic PDEs. While these methods achieve optimal cost per iteration, the limiting factor is the convergence bound for these methods. For conjugate gradient (CG), in particular, we have

$$\|\mathbf{x} - \mathbf{x}_k\|_A \leq 2 \left(\frac{\sqrt{\text{cond}(A)} - 1}{\sqrt{\text{cond}(A)} + 1} \right)^k \|\mathbf{x} - \mathbf{x}_0\|_A \quad (11.1)$$

for symmetric and positive-definite A . For a standard discretization of a second-order elliptic PDE on a uniform mesh with mesh size h , a typical condition number is $\text{cond}(A) = \mathcal{O}(1/h^2)$. Thus, even with optimal cost per iteration, the number of iterations required to guarantee that $\|\mathbf{x} - \mathbf{x}_k\|_A$ achieves discretization-level accuracy grows (asymptotically) as $\mathcal{O}(1/h)$. While simple preconditioners improve on this, they remain suboptimal.

The best known matrix-splitting preconditioner for the 2D finite-difference Poisson problem on an $n_x \times n_x$ mesh comes from an incomplete Cholesky preconditioner, leading to a condition number of the preconditioned system that is $\mathcal{O}(n_x)$, so that the total solution cost is $\mathcal{O}(n_x^{5/2})$, coming from $\mathcal{O}(\sqrt{n_x})$ iterations of preconditioned CG with $\mathcal{O}(n_x^2)$ cost per iteration. There is a practical limit on the cost per iteration of a Krylov (or other polynomial) method, since a matrix-vector product with A is expected to cost $\mathcal{O}(n_x^2)$ for the example above, where A is an $n_x^2 \times n_x^2$ sparse matrix. Thus, the only way to achieve an optimal cost is to lower the bound that depends on $\text{cond}(A)$. This can be done in two ways. First, we can introduce a preconditioner that reduces the effective condition number of the iteration to an $\mathcal{O}(1)$ quantity. Then, the Chebyshev-based estimate above, equation (11.1), bounds the error in the approximate solution below discretization error in a constant number of steps. Or, we can precondition with the goal of *clustering* the spectrum of the preconditioned system, so that specialized bounds can be used to show convergence, again, in an $\mathcal{O}(1)$ number of iterations. In this chapter, we present two families of algorithms for achieving such goals, known as *domain decomposition* and *multigrid* methods.

Objectives

In this chapter, we present the basics for the design and analysis of domain decomposition and multigrid methods. In particular, you will learn how to recognize when each of these approaches should be considered as an effective tool. You will also study variants of the domain decomposition and multigrid methods, such as geometric versus algebraic multigrid, and understand in what contexts each of these effectively achieves the goal of being an “optimal” solver. Finally, you will extend these approaches to coupled systems of equations and to time-dependent settings.

Chapter Outline

Section 11.1 Domain decomposition methods presents the basic concepts behind a wide family of solution techniques that are commonly used to precondition Krylov methods (or directly used as iterative solvers). We discuss the basics of additive and multiplicative Schwarz methods, as well as nonoverlapping, overlapping, and optimized approaches. Two-level formulations are also introduced to achieve “optimal” methods.

Section 11.2 Multigrid methods presents the basic concepts of a *second* broad family of solution techniques that are commonly used to precondition Krylov methods (or directly used as iterative solvers). The focus is on geometric multigrid, where we discuss the complementary processes of relaxation (smoothing) and coarse-grid correction. Costs and convergence properties of basic two-grid and multigrid algorithms are analyzed.

Section 11.3 Algebraic multigrid extends the general framework described in the previous section from a geometric one to an algebraic one, introducing algebraic multigrid along with some of its variants. The notion of *algebraically smooth error* is considered, and two general formulations, smoothed aggregation and CF-AMG, are analyzed.

Section 11.4 Block-structured systems of equations extends the results of this chapter to linear and nonlinear systems that have block structure, usually derived from systems of PDEs, and describes both block factorization and monolithic multigrid methods. Split-step schemes are also introduced to address time-dependent problems.

11.1 ■ Domain decomposition methods

Objectives

One way to achieve the optimality that we seek for solving linear systems is to leverage direct methods when they are effective. We know that we cannot effectively use sparse Gaussian elimination directly on a large matrix, A , that results from discretization of an elliptic PDE in two or three dimensions; however, we can use such algorithms effectively on smaller problems. This naturally couples with the use of parallel computing, where for “large enough” systems, we would like to use multiple computing threads to solve a linear system, $A\mathbf{u} = \mathbf{f}$, in parallel. This leads to the basic idea of domain decomposition methods. Specifically, in this section, you will

- consider the basics of Schwarz methods in both the additive and multiplicative frameworks;
- discuss more-advanced techniques that include overlapping and optimized Schwarz; and
- show improved results with two-level formulations of Schwarz methods.

To set up the domain decomposition framework, we first partition a given discretized problem and the resulting $n \times n$ matrix, A , into s *subdomains* and define the preconditioner by constructing (exact) solutions to the subdomain problems. To formalize this idea, we define $\Omega = \{1, 2, \dots, n\}$ to be the set of degrees of freedom in the system and define a set of subdomains, $\{\Omega_i\}_{i=1}^s$, such that $\Omega = \cup_{i=1}^s \Omega_i$. For now, we consider the case where the subdomains are disjoint and $\Omega_i \cap \Omega_j = \emptyset$ if $i \neq j$. This is illustrated in Figure 11.1.1 in the case of four subdomains decomposing an $n_x \times n_x$ computational domain of the unit square (giving $n = n_x^2$ total degrees of freedom). We then consider a permutation of A wherein we order the degrees of freedom according to the index of their subdomain, with those in Ω_1 first, then those in Ω_2 , and so on. This induces a partitioning of the system matrix, A , as

$$A = \begin{bmatrix} A_{1,1} & A_{1,2} & \cdots & A_{1,s} \\ A_{2,1} & A_{2,2} & \cdots & A_{2,s} \\ \vdots & \vdots & & \vdots \\ A_{s,1} & A_{s,2} & \cdots & A_{s,s} \end{bmatrix}, \quad (11.2)$$

where $A_{i,j}$ describes how variables in Ω_j appear in the equations for Ω_i . On a parallel machine, process i is identified as “owning” the variables in Ω_i , thus requiring communication with other processes to obtain the values from Ω_j to compute Au .

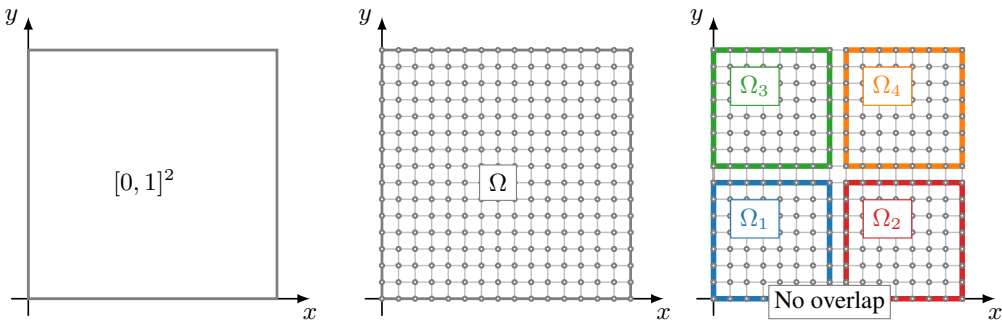


Figure 11.1.1. (left) The physical domain; (center) the computational domain with n total degrees of freedom; (right) four subdomains with no overlap: Ω_1 , Ω_2 , Ω_3 , and Ω_4 (cf. Figure 11.1.4).

11.1.1 ■ Basic Schwarz methods

Additive Schwarz

To define a basic domain decomposition preconditioner, we let R_i be the *restriction operator*, such that $R_i u = u_i$ gives the values of the n vector, u , only at the points in Ω_i . (Note that this notation is different than that used in Chapter 7, where u_i would have been the approximation to the global solution, u , after i iterations.) It is most natural to think of R_i as an $|\Omega_i| \times n$ matrix that, in reordered form, is given by s block columns of dimension $|\Omega_i| \times |\Omega_j|$,

$$R_i = \begin{bmatrix} 0 & \cdots & 0 & I & 0 & \cdots & 0 \end{bmatrix}, \quad (11.3)$$

where the first $i - 1$ and last $s - i$ block columns (i.e., those with $j \neq i$) are zero matrices and only the i th block column is the identity matrix. With this notation, $A_{i,j} = R_i A R_j^T$. In the spirit

of parallel computation, the simplest preconditioner for A is a block-diagonal one, with

$$A_D = \begin{bmatrix} A_{1,1} & & & \\ & A_{2,2} & & \\ & & \ddots & \\ & & & A_{s,s} \end{bmatrix}, \quad (11.4)$$

and the preconditioned system as $A_D^{-1}A$. Using the above notation, we write

$$A_D = \sum_{i=1}^s R_i^T (R_i A R_i^T) R_i \quad \text{and} \quad A_D^{-1} = \sum_{i=1}^s R_i^T (R_i A R_i^T)^{-1} R_i. \quad (11.5)$$

We call A_D the *nonoverlapping additive Schwarz* preconditioner for the decomposition $\Omega = \cup_{i=1}^s \Omega_i$.

To see how well this works, consider the one-dimensional model problem with homogeneous Dirichlet data

$$-u''(x) = f(x) \quad \text{for } 0 < x < 1, \quad (11.6a)$$

$$u(0) = u(1) = 0. \quad (11.6b)$$

The finite-difference discretization matrix for this problem on a uniform mesh with mesh size h is

$$A = \frac{1}{h^2} \begin{bmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ & & & \ddots \end{bmatrix}. \quad (11.7)$$

Since $A_D^{-1}A$ is similar to the symmetric matrix $A_D^{-1/2}AA_D^{-1/2}$, we consider an estimate to $\text{cond}(A_D^{-1}A) = \lambda_{\max}(A_D^{-1}A)/\lambda_{\min}(A_D^{-1}A)$. If λ is an eigenvalue of $A_D^{-1}A$, then there must exist a vector, $\mathbf{u}$, such that

$$A_D^{-1}A\mathbf{u} = \lambda\mathbf{u}, \quad (11.8a)$$

$$A\mathbf{u} = \lambda A_D\mathbf{u}, \quad (11.8b)$$

$$\mathbf{u}^T A\mathbf{u} = \lambda \mathbf{u}^T A_D\mathbf{u} \quad (11.8c)$$

$$\Rightarrow \lambda = \frac{\mathbf{u}^T A\mathbf{u}}{\mathbf{u}^T A_D\mathbf{u}}. \quad (11.8d)$$

Thus, we characterize $\text{cond}(A_D^{-1}A)$ by

$$\lambda_{\min}(A_D^{-1}A) = \min_{\mathbf{u} \neq \mathbf{0}} \frac{\mathbf{u}^T A\mathbf{u}}{\mathbf{u}^T A_D\mathbf{u}} \quad \text{and} \quad (11.9a)$$

$$\lambda_{\max}(A_D^{-1}A) = \max_{\mathbf{u} \neq \mathbf{0}} \frac{\mathbf{u}^T A\mathbf{u}}{\mathbf{u}^T A_D\mathbf{u}}. \quad (11.9b)$$

A lower bound on $\text{cond}(A_D^{-1}A)$ is then achieved by substituting in two fixed vectors that approximate the extreme values. This clearly gives a lower bound on the maximum eigenvalue

and an upper bound on the minimum. The ratio of these two values gives a lower bound on $\text{cond}(A_D^{-1}A)$.

For the lower bound on the maximum eigenvalue, we consider any canonical unit vector, $\mathbf{u} = \mathbf{e}^{(j)}$, where $e_i^{(j)} = 1$ if $i = j$ and is zero otherwise. For this vector, we have

$$\mathbf{u}^T A \mathbf{u} = \mathbf{u}^T A_D \mathbf{u} = 2/h^2, \quad (11.10)$$

giving the lower bound $\lambda_{\min}(A_D^{-1}A) \geq 1$. For an upper bound on the minimum eigenvalue, we consider the global constant vector, $\mathbf{u} = \mathbf{1}$. By direct calculation, since all but the first and last rows of A have row sum zero, while the row sum of the first and last rows is $1/h^2$, we have $\mathbf{1}^T A \mathbf{1} = 2/h^2$. Considering

$$\mathbf{1}^T A_D \mathbf{1} = \sum_{i=1}^s \mathbf{1}^T R_i^T (R_i A R_i^T) R_i \mathbf{1}, \quad (11.11)$$

we recognize that $R_i \mathbf{1} = \mathbf{1}_i$, the constant vector over subdomain Ω_i . It also holds that $A_{i,i} = R_i A R_i^T$ has similar properties to A (so long as $|\Omega_i| > 1$ and the degrees of freedom in Ω_i come from a connected section of the original mesh), with zero row sums over all but its first and last rows, where the row sums are $1/h^2$. Thus, $\mathbf{1}_i^T A_{i,i} \mathbf{1}_i = 2/h^2$ and $\mathbf{1}^T A_D \mathbf{1} = 2s/h^2$. This gives $\lambda_{\min}(A_D^{-1}A) \leq 1/s$ and, consequently, $\text{cond}(A_D^{-1}A) \geq s$.

The above bound shows that the achieved condition number estimate is not directly dependent on n but grows linearly with s . In a practical implementation, we typically fix the subdomain size, n/s , to be independent of n , so that the cost of solving the subdomain systems with $A_{i,i}$ does not grow with n . This, however, implies that we expect $s = \mathcal{O}(n)$, giving $\text{cond}(A_D^{-1}A) = \mathcal{O}(n)$. While this is an improvement on $\text{cond}(A) = \mathcal{O}(n^2)$ in this setting, it still does not offer optimal bounds based on the standard Chebyshev argument, nor does it offer any substantial clustering in the spectrum of $A_D^{-1}A$ that could be exploited. This argument extends similarly to realistic two- and three-dimensional problems, yielding bounds that degrade as problem size increases, again assuming that the subdomain sizes are fixed. While improved bounds are possible when A is diagonally dominant, the same is true for classical matrix splittings, so this is not considered a significant advantage for these approaches.

Example 11.1: Nonoverlapping additive Schwarz

Throughout this section, we apply the domain decomposition preconditioners that we develop to a simple model problem, coming from the finite-difference discretization of Poisson's equation, $-\Delta u = f$ for $f(x, y) = 2(x - x^2 + y - y^2)$, on two-dimensional uniform grids of the unit square, with fully eliminated homogeneous Dirichlet boundary conditions. We use the domain decomposition methods as preconditioners for FGMRES (see Section 7.10), solely so that we can take advantage of efficient implementation of the right-preconditioning framework for GMRES. For a stopping tolerance, we require that the norm of the residual be reduced by a relative factor of 10^6 . As a baseline, the left side of Figure 11.1.2 presents convergence results, showing the reduction in the residual norm versus iteration for the problem discretized on $n_x \times n_x$ meshes (hence, $n = n_x^2$ for the size of the resulting matrix) for $n_x = 128$ and 256, using FGMRES with no preconditioner and no restarting. As expected from the convergence theory in Section 7.8, we see that the number of Krylov iterations needed doubles when n_x does, since the condition number is proportional to n_x^2 .

Applying the additive Schwarz preconditioner with $s \times s$ subdomains leads to greatly improved convergence, as shown in the right side of Figure 11.1.2, where the faded lines show the convergence from unpreconditioned FGMRES. Here, we notice both the greatly improved convergence overall and the much diminished difference between the convergence results for

the two grid sizes. Note also the effect of the number of subdomains, with 4×4 subdomains leading to convergence in about 10 iterations fewer than 8×8 subdomains for both grid sizes.

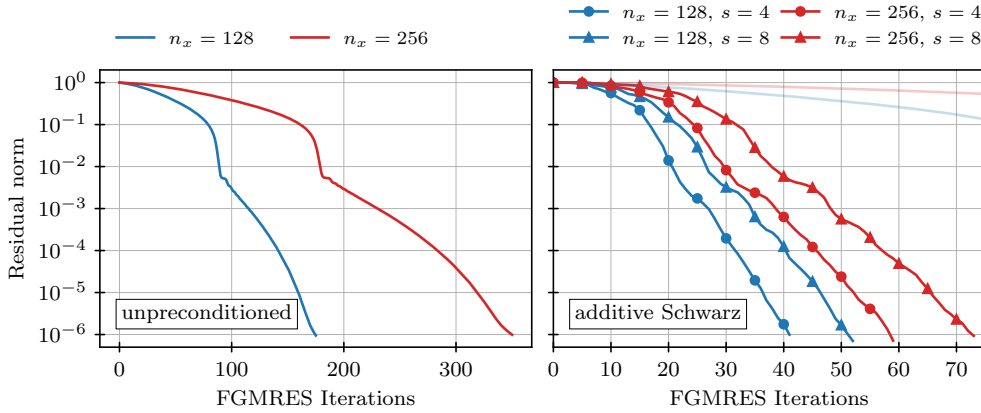


Figure 11.1.2. (left) Relative residual reduction by unpreconditioned FGMRES applied to the finite-difference Laplacian on $n_x \times n_x$ meshes. (right) Relative residual reduction by FGMRES applied to the finite-difference Laplacian on $n_x \times n_x$ meshes, using additive Schwarz preconditioning with $s \times s$ subdomains. Faded lines show the convergence of unpreconditioned FGMRES for comparison.

Multiplicative Schwarz

It is natural to think of the additive Schwarz preconditioner defined above as a block Jacobi method. From this perspective, it is also natural to consider the block Gauss–Seidel analogue, where updates that have been computed on previous subdomains are used immediately on the current block. The block-matrix form of equation (11.2) reveals this form of updating as a lower-triangular preconditioner,

$$A_L = \begin{bmatrix} A_{1,1} & 0 & \cdots & 0 \\ A_{2,1} & A_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A_{s,1} & A_{s,2} & \cdots & A_{s,s} \end{bmatrix}, \quad (11.12)$$

with the preconditioned system being $A_L^{-1}A$. In this case, A_L is no longer symmetric, so the preconditioned system can no longer be solved using MINRES or CG (if A is symmetric and if A_D is positive definite). However, this may be an acceptable compromise if the resulting iteration yields good convergence.

Algorithmically, the update above is straightforward to express as a stationary iteration. For given A , $\mathbf{f}$, and $\hat{\mathbf{u}}$, we loop over the subdomains, $1 \leq i \leq s$, and compute the successive updates as

$$\hat{\mathbf{u}} \leftarrow \hat{\mathbf{u}} + R_i^T (R_i A R_i^T)^{-1} R_i (\mathbf{f} - A \hat{\mathbf{u}}). \quad (11.13)$$

This is called the *nonoverlapping multiplicative Schwarz* iteration. The use of *additive* and *multiplicative* as identifiers for these iterations arises from the error-propagation forms of the stationary iterations. From above, we write

$$I - A_D^{-1}A = I - \sum_{i=1}^s R_i^T (R_i A R_i^T)^{-1} R_i A \quad (11.14)$$

and see the additive form of A_D^{-1} as expressed earlier. For $I - A_L^{-1}A$, we have to properly track the successive iterations, introducing more precise notation than $\hat{\mathbf{u}}$. To do this, given an initial guess, $\mathbf{u}^{(0)}$ (where we use superscripts to denote iterations to not conflict with subscripts denoting restriction to subdomains), we write $\mathbf{u}^{(i/s)}$ to denote the resulting approximation after the i th step, giving

$$\mathbf{u}^{(i/s)} = \mathbf{u}^{((i-1)/s)} + R_i^T (R_i A R_i^T)^{-1} R_i (\mathbf{f} - A \mathbf{u}^{((i-1)/s)}), \quad (11.15a)$$

$$\mathbf{e}^{(i/s)} = \mathbf{e}^{((i-1)/s)} - R_i^T (R_i A R_i^T)^{-1} R_i A \mathbf{e}^{((i-1)/s)} \quad (11.15b)$$

$$= (I - R_i^T (R_i A R_i^T)^{-1} R_i A) \mathbf{e}^{((i-1)/s)}. \quad (11.15c)$$

Writing $P_i = R_i^T (R_i A R_i^T)^{-1} R_i A$, this allows us to write

$$\mathbf{e}^{(1)} = (I - P_s)(I - P_{s-1}) \cdots (I - P_1) \mathbf{e}^{(0)}, \quad (11.16)$$

giving $I - A_L^{-1}A = (I - P_s)(I - P_{s-1}) \cdots (I - P_1)$, or

$$A_L^{-1} = (I - (I - P_s)(I - P_{s-1}) \cdots (I - P_1)) A^{-1}. \quad (11.17)$$

While this expression is somewhat unwieldy, it provides a route to define the matrix-vector product, $\mathbf{z} = A_L^{-1}\mathbf{v}$, needed to use the multiplicative Schwarz method as a preconditioner with GMRES. To do this, we define the partial product of the error propagators as

$$Q_i = (I - P_i)(I - P_{i-1}) \cdots (I - P_1), \quad (11.18)$$

with $Z_i = I - Q_i$ and $M_i = Z_i A^{-1}$, so that $A_L^{-1} = M_s$. We further define

$$T_i = P_i A^{-1} = R_i^T (R_i A R_i^T)^{-1} R_i, \quad (11.19)$$

so that we can conveniently write recurrences for Q_i , Z_i , and M_i , as

$$Q_i = (I - P_i) Q_{i-1}, \quad (11.20a)$$

$$\begin{aligned} Z_i &= I - (I - P_i) Q_{i-1} \\ &= I - (I - P_i)(I - Z_{i-1}) = Z_{i-1} + P_i - P_i Z_{i-1}, \end{aligned} \quad (11.20b)$$

$$\begin{aligned} M_i &= Z_i A^{-1} = Z_{i-1} A^{-1} + P_i A^{-1} - P_i Z_{i-1} A^{-1} \\ &= M_{i-1} + T_i - T_i A M_{i-1} = M_{i-1} + T_i (I - A M_{i-1}). \end{aligned} \quad (11.20c)$$

From this we see that the recurrences are readily initialized as $Q_1 = I - P_1$, $Z_1 = P_1$, and $M_1 = T_1$. Thus, we define $\mathbf{z}_i = M_i \mathbf{v}$, and, from the recurrence for M_i , we write

$$\mathbf{z}_i = M_i \mathbf{v} = M_{i-1} \mathbf{v} + T_i (I - A M_{i-1}) \mathbf{v} = \mathbf{z}_{i-1} + T_i (\mathbf{v} - A \mathbf{z}_{i-1}). \quad (11.21)$$

Since $\mathbf{z} = A_L^{-1} \mathbf{v} = M_s \mathbf{v}$, we compute it by following the above recurrence. One observation is that this is equivalent to applying a single (complete) sweep of the stationary iteration above to solving $A \mathbf{z} = \mathbf{v}$ with a zero initial guess for $\mathbf{z}$, as in equation (11.15a).

Remark 11.2: Schwarz methods as a preconditioner

In practice, this approach can be used to construct any well-defined stationary iteration as a preconditioner since, if

$$\mathbf{u}^{(k)} = \mathbf{u}^{(k-1)} + M \left(\mathbf{f} - A\mathbf{u}^{(k-1)} \right), \quad (11.22)$$

then, if $\mathbf{u}^{(0)} = \mathbf{0}$, we automatically have $\mathbf{u}^{(1)} = M\mathbf{f}$.

As in the case of (pointwise) Jacobi and Gauss–Seidel, typically the multiplicative Schwarz preconditioner leads to faster convergence than the additive Schwarz preconditioner, but only by a constant factor. In some cases, its convergence is analyzed using our earlier results, such as Theorem 7.36.

Example 11.3: Nonoverlapping multiplicative Schwarz

Here, we continue considering the problem from Example 11.1, but now we apply the multiplicative Schwarz preconditioner. Figure 11.1.3 shows the relative residual reduction, now with the data for unpreconditioned FGMRES and FGMRES preconditioned by additive Schwarz shown in faded lines for comparison. We note the further reduction in the number of iterations given by using the multiplicative approach over the additive one, but that the relative performance does not change: there is still degradation in convergence with increasing n_x and/or increasing s .

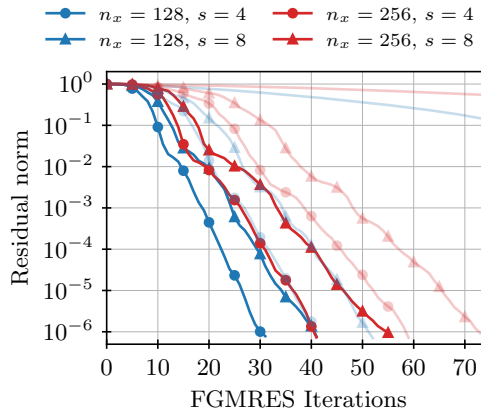


Figure 11.1.3. Relative residual reduction by FGMRES applied to the finite-difference Laplacian on $n_x \times n_x$ meshes, using multiplicative Schwarz preconditioning with $s \times s$ subdomains. Faded lines show the convergence of unpreconditioned FGMRES and FGMRES preconditioned by additive Schwarz for comparison.

Schur complement methods

In addition to the additive and multiplicative forms, a third approach is to partition A such that $A_{i,j} = 0$ for $i \neq j$ when $1 \leq i, j < s$, but with all blocks adjacent to subdomain s . This leads to a partitioning with block structure

$$A = \begin{bmatrix} A_{1,1} & 0 & \cdots & 0 & A_{1,s} \\ 0 & A_{2,2} & \cdots & 0 & A_{2,s} \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \cdots & A_{s-1,s-1} & A_{s-1,s} \\ A_{s,1} & A_{s,2} & \cdots & A_{s,s-1} & A_{s,s} \end{bmatrix}. \quad (11.23)$$

The block LU factorization of such an A is then

$$A = \begin{bmatrix} A_{1,1} & 0 & \cdots & 0 & 0 \\ 0 & A_{2,2} & \cdots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \cdots & A_{s-1,s-1} & 0 \\ A_{s,1} & A_{s,2} & \cdots & A_{s,s-1} & S \end{bmatrix} \begin{bmatrix} I & 0 & \cdots & 0 & A_{1,1}^{-1}A_{1,s} \\ 0 & I & \cdots & 0 & A_{2,2}^{-1}A_{2,s} \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \cdots & I & A_{s-1,s-1}^{-1}A_{s-1,s} \\ 0 & 0 & \cdots & 0 & I \end{bmatrix}, \quad (11.24)$$

where $S = A_{s,s} - \sum_{i=1}^{s-1} A_{s,i}A_{i,i}^{-1}A_{i,s}$. This leads to an *exact* algorithm to solve $Au = f$:

1. Solve $A_{i,i}v_i = f_i$ for $1 \leq i < s$.
2. Solve $Su_s = f_s - \sum_{i=1}^{s-1} A_{s,i}v_i$.
3. Solve $A_{i,i}u_i = f_i - A_{i,s}u_s$.

This is turned into an iterative method by approximating S and/or the solves with $A_{i,i}$ for $1 \leq i < s$ in each step. Such methods remain multiplicative methods, since the corrections to u_i for $1 \leq i < s$ are used to compute that for u_s but are fundamentally different than the general nonoverlapping multiplicative Schwarz method. In recognition of the important role the approximation of S plays in their convergence, they are typically called *Schur complement methods*.

11.1.2 ■ Advanced Schwarz methods

Overlapping Schwarz

Next, we relax the condition that the subdomains are disjoint. Again, we consider the discretized problem to be represented in terms of a domain given by $\Omega = \{1, 2, \dots, n\}$ and define subdomains $\{\Omega_i\}_{i=1}^s$ such that $\Omega = \cup_{i=1}^s \Omega_i$, but we drop the requirement that $\Omega_i \cap \Omega_j = \emptyset$ if $i \neq j$.

Our subdomains are often geometric in nature. For example, if we consider the (physical) domain of the unit interval, $[0, 1]$, we can divide this into two subintervals, $[0, 1/2]$ and $[1/2, 1]$, and a discrete degree of freedom at $x = 1/2$ would be shared between Ω_1 and Ω_2 . This is true in more general situations: if we partition the domain by its elements (or volumes), then a PDE discretized with nodal degrees of freedom leads to natural overlap on the interfaces between subdomains. We call this case *minimal overlap*. Otherwise, one way to categorize the overlap at the discrete level is by how many mesh widths of elements are in common between neighboring subdomains. In this style, we consider a nonoverlapping partition given by $\{\Omega_i^0\}_{i=1}^s$ such that $\Omega = \cup_{i=1}^s \Omega_i^0$ and $\Omega_i^0 \cap \Omega_j^0 = \emptyset$ for $i \neq j$ and then define

$$\Omega_i^\delta = \Omega_i^{\delta-1} \cup \{j \in \Omega \mid a_{j,k} \neq 0 \text{ for some } k \in \Omega_i^{\delta-1}\} \quad (11.25)$$

for integers $\delta \geq 1$. Here, the set being added to $\Omega_i^{\delta-1}$ is the set of all degrees of freedom in Ω that have a shared connection (in terms of the matrix system) with any of those in $\Omega_i^{\delta-1}$. This is illustrated in Figure 11.1.4 for a sparse system like that of the discretized Poisson equation.

If we define R_i^δ to be a restriction onto Ω_i^δ , we then define the solution step on subdomain i as

$$P_i^\delta = (R_i^\delta)^T \left(R_i^\delta A (R_i^\delta)^T \right)^{-1} R_i^\delta A = T_i^\delta A. \quad (11.26)$$

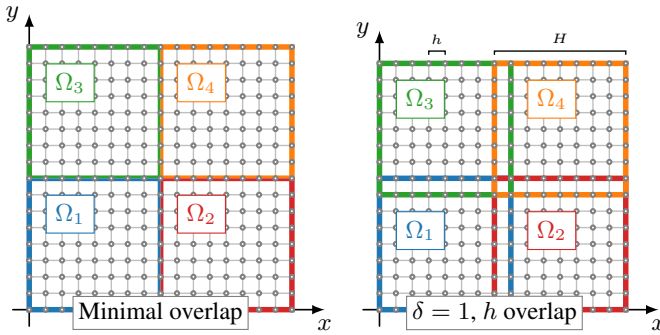


Figure 11.1.4. Four subdomains, Ω_1 , Ω_2 , Ω_3 , and Ω_4 , of a 2D computational domain with n degrees of freedom: (left) minimal overlap and (right) $\delta = 1$ overlap (cf. Figure 11.1.1).

From this, we easily define the overlapping additive and multiplicative Schwarz preconditioners as

$$(A_D^\delta)^{-1} = \sum_{i=1}^s T_i^\delta, \quad (11.27a)$$

$$(A_L^\delta)^{-1} = (I - (I - P_s^\delta) \cdots (I - P_1^\delta)) A^{-1}. \quad (11.27b)$$

We apply these preconditioners in much the same way as in the nonoverlapping case already considered, aside from being conscious of the overlap.

For multiplicative Schwarz, this is a natural thing to do and gives an improved method. This is simply because degrees of freedom in $\Omega_i^\delta \cap \Omega_j^\delta$ get updated in *both* step i and step j of the algorithm in a Gauss–Seidel-like manner. It is intuitive that this leads to better convergence since a new residual, $R_i^\delta(\mathbf{f} - A\hat{\mathbf{u}})$, is computed before each subdomain solve. For additive Schwarz, the advantage is less clear. Now, corrections to a degree of freedom in $\Omega_i^\delta \cap \Omega_j^\delta$ are computed independently in subdomains i and j and then simply added together to form the correction that leads to the next iterate. Convergence analysis for the overlapping case is also more difficult, since

$$(A_D^\delta)^{-1} = \sum_{i=1}^s (R_i^\delta)^T (R_i^\delta A (R_i^\delta))^{-1} R_i^\delta \quad (11.28)$$

cannot be directly inverted in terms of the subdomain blocks, as was possible in the nonoverlapping case.

Example 11.4: Minimal overlap additive and multiplicative Schwarz

Here, we continue Examples 11.1 and 11.3, now with additive and multiplicative Schwarz with minimal overlap. We note that the earlier results were computed on grids with $2^k \times 2^k$ interior grid points, so that they could easily be divided into 4×4 and 8×8 nonoverlapping subdomains. To get similarly nice partitions into minimally overlapping subdomains, we now increase the number of interior mesh points by one in each dimension, yielding a slightly different problem, but one that is easily comparable to the earlier results. Figure 11.1.5 shows convergence for both the additive method (at left) and the multiplicative method (at right), with faded lines showing the unpreconditioned FGMRES convergence and the FGMRES convergence with nonoverlapping Schwarz (additive at left, multiplicative at right) for comparison. We see that this leads to further improvements in convergence, but still degrading convergence with increasing n_x or s .

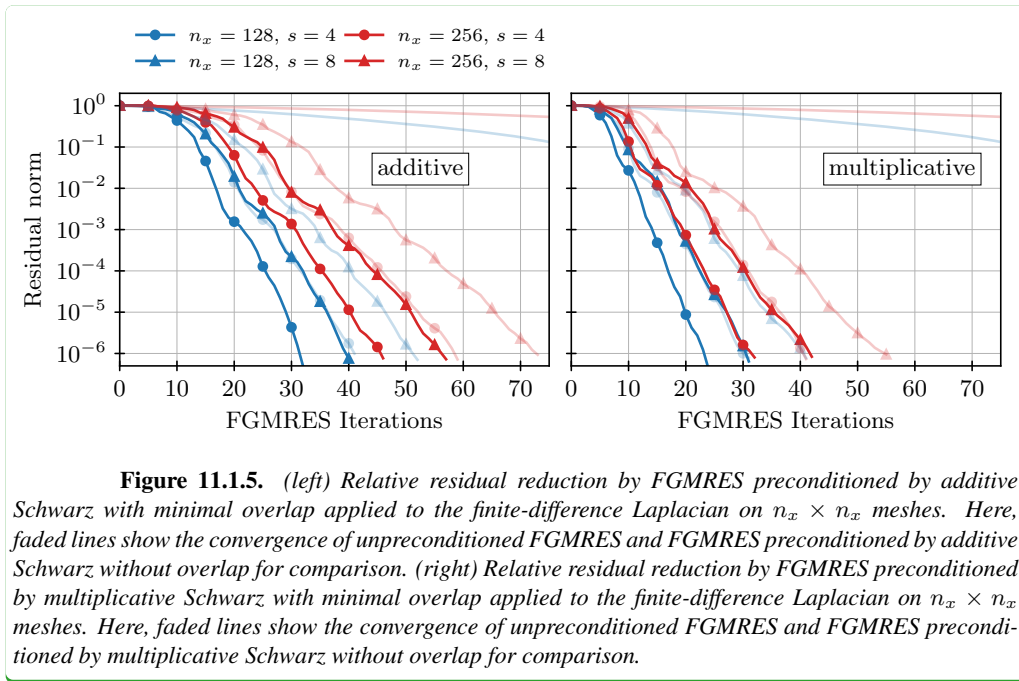


Figure 11.1.5. (left) Relative residual reduction by FGMRES preconditioned by additive Schwarz with minimal overlap applied to the finite-difference Laplacian on $n_x \times n_x$ meshes. Here, faded lines show the convergence of unpreconditioned FGMRES and FGMRES preconditioned by additive Schwarz without overlap for comparison. (right) Relative residual reduction by FGMRES preconditioned by multiplicative Schwarz with minimal overlap applied to the finite-difference Laplacian on $n_x \times n_x$ meshes. Here, faded lines show the convergence of unpreconditioned FGMRES and FGMRES preconditioned by multiplicative Schwarz without overlap for comparison.

Another disadvantage of overlapping additive Schwarz is a communication penalty that must be paid when using parallel computing. Suppose processor i stores all values of degrees of freedom in Ω_i^δ . Then, once $(R_i^\delta A (R_i^\delta)^T)^{-1} R_i^\delta \mathbf{v}$ has been computed on subdomain i , we need a communication step to update and exchange values for every processor (or subdomain) such that $\Omega_i^\delta \cap \Omega_j^\delta \neq \emptyset$. A natural way to avoid this is to have processor i only update degrees of freedom in the nonoverlapping partition, Ω_i^0 . This defines the *restricted additive Schwarz* (RAS) preconditioner [64],

$$(M_R^\delta)^{-1} = \sum_{i=1}^s (\hat{R}_i^\delta)^T (R_i^\delta A (R_i^\delta)^T)^{-1} R_i^\delta, \quad (11.29)$$

where $\hat{R}_i^\delta$ still maps from Ω to Ω_i^δ , but takes nonzero values only for its rows corresponding to degrees of freedom in Ω_i^0 . Note that for $\delta > 0$, M_R^δ is nonsymmetric. This has the same disadvantage as multiplicative Schwarz in comparison to additive, but the same potential benefit. As before, we should be prepared to accept a nonsymmetric RAS preconditioner if it leads to substantially better performance than classical additive Schwarz. Note that RAS still requires a communication step to update the residual on processor i from processors that own degrees of freedom in $\Omega_i^\delta \setminus \Omega_i^0$, as well as those degrees of freedom needed to compute the residuals at all points in Ω_i^δ .

A similar approach, called *additive Schwarz with harmonic extensions*, uses $\hat{R}_i^\delta$ in the restriction step, giving the preconditioner

$$(M_{\hat{R}}^\delta)^{-1} = \sum_{i=1}^s (R_i^\delta)^T (R_i^\delta A (R_i^\delta)^T)^{-1} \hat{R}_i^\delta, \quad (11.30)$$

which yields similar performance. One can attempt to symmetrize these approaches and use the

preconditioner

$$\sum_{i=1}^s \left(\hat{R}_i^\delta \right)^T \left(R_i^\delta A (R_i^\delta)^T \right)^{-1} \hat{R}_i^\delta, \quad (11.31)$$

but it does not work as well as either M_R^δ or M_R^δ [64, Rem. 2.5].

In practice, RAS is a substantially better preconditioner than additive Schwarz, even for small amounts of overlap, such as with $\delta = 1$ or 2. This observation leads to the natural question as to whether or not it is an optimal preconditioner. Deriving convergence bounds for such methods is outside of the scope of this book but can be expressed in terms of a few parameters of the domain decomposition. For regular meshes, we take h to be the discretization mesh width and H to be the physical size (in each dimension) of a typical subdomain (see right graphic in Figure 11.1.4). With δ again denoting the amount of overlap, a convergence bound for what is known as the symmetric RASHO variant (restricted additive Schwarz with harmonic overlap; see [63]) in two dimensions looks like

$$\begin{aligned} \text{cond} \left((M_R^\delta)^{-1} A \right) &\leq C \left((1 + \log(\delta + 1)) \left(1 + \log \left(\frac{H}{h} \right) \right) \right. \\ &\quad \left. + \frac{1}{H^2} \left(1 + \log(\delta + 1) + \frac{H}{(2\delta + 1)h} \right) \right). \end{aligned} \quad (11.32)$$

Key terms in this bound are

- H/h , the number of mesh points in each subdomain (in one dimension), which we expect to remain constant in order to not have the cost of solving the subdomain problems grow as h gets smaller;
- $\delta + 1$ and $2\delta + 1$, which grow with increasing overlap; and
- $1/H^2$, which grows as the problem size increases if we fix H/h , since the physical subdomain size decreases as h does in this case.

The last term here scales akin to the $\text{cond}(A_D^{-1}A) \geq s$ bound from above, since the number of $H \times H$ subdomains in $[0, 1]^2$ is precisely $1/H^2$. Thus, while RAS offers some improvement over classical additive Schwarz, it does not overcome the fundamental barrier to scalability therein.

Optimized Schwarz methods

RAS-type methods rely on their *volumetric* overlap to improve convergence over the nonoverlapping case. We now reconsider the case of *minimal overlap*, where the overlap is solely on the interface between subdomains. In the notation from above, this gives $\delta = 0$, and the RAS-type theory gives suboptimal bounds.

Consider the two-subdomain case, writing the global discretized problem as

$$\begin{bmatrix} A_{1,1} & 0 & A_{1,B} \\ 0 & A_{2,2} & A_{2,B} \\ A_{B,1} & A_{B,2} & A_{B,B} \end{bmatrix} \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \mathbf{u}_B \end{bmatrix} = \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_B \end{bmatrix}, \quad (11.33)$$

where we partition the degrees of freedom into subdomains Ω_1 and Ω_2 and the boundary between them, $\Omega_B = \Omega_1 \cap \Omega_2$. In this notation, we consider two subdomain problems:

$$\begin{bmatrix} A_{1,1} & \hat{A}_{1,B} \\ \hat{A}_{B,1} & \hat{A}_{B,B} \end{bmatrix} \begin{bmatrix} \mathbf{u}_1 \\ \hat{\mathbf{u}}_B \end{bmatrix} = \begin{bmatrix} \mathbf{f}_1 \\ \hat{\mathbf{f}}_B \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} A_{2,2} & \tilde{A}_{2,B} \\ \tilde{A}_{B,2} & \tilde{A}_{B,B} \end{bmatrix} \begin{bmatrix} \mathbf{u}_2 \\ \tilde{\mathbf{u}}_B \end{bmatrix} = \begin{bmatrix} \mathbf{f}_2 \\ \tilde{\mathbf{f}}_B \end{bmatrix}. \quad (11.34)$$

To define the iteration, we must now choose the additional matrices denoted above, $\hat{A}_{1,B}$, $\hat{A}_{B,1}$, $\hat{A}_{B,B}$, $\tilde{A}_{2,B}$, $\tilde{A}_{B,2}$, and $\tilde{A}_{B,B}$, as well as $\hat{\mathbf{f}}_B$ and $\tilde{\mathbf{f}}_B$. There are several natural possibilities.

The easiest approach is to use Dirichlet boundary conditions on Ω_B for both subdomains. This is equivalent to taking $\hat{A}_{B,B} = \tilde{A}_{B,B} = I$, with zero operators for $\hat{A}_{B,1}$ and $\tilde{A}_{B,2}$, and the natural subdomain operators above the diagonal. Thus, the two subdomain problems are written as

$$\begin{bmatrix} A_{1,1} & A_{1,B} \\ 0 & I \end{bmatrix} \begin{bmatrix} \mathbf{u}_1 \\ \hat{\mathbf{u}}_B \end{bmatrix} = \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{v}_B \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} A_{2,2} & A_{2,B} \\ 0 & I \end{bmatrix} \begin{bmatrix} \mathbf{u}_2 \\ \tilde{\mathbf{u}}_B \end{bmatrix} = \begin{bmatrix} \mathbf{f}_2 \\ \mathbf{w}_B \end{bmatrix}, \quad (11.35)$$

where $\mathbf{v}_B$ and $\mathbf{w}_B$ are the Dirichlet data to be imposed on each subdomain. Unfortunately, while this leads to subdomain problems that are relatively easy to solve, it does not lead to the best iteration. This is because neither subdomain directly updates an approximation of $\mathbf{u}_B$, since it simply reproduces the prescribed Dirichlet data on the interface. This, it turns out, is not a critical failure if the resulting iteration is used as a preconditioner, but it is also far from optimal.

A solution to the above is to use the Dirichlet data on only one subdomain, leading to the family of Dirichlet–Neumann methods. Taking Dirichlet data on Ω_1 as above, we now take $\tilde{A}_{2,B}$, $\tilde{A}_{B,2}$, $\tilde{A}_{B,B}$, and $\tilde{\mathbf{f}}_B$ so that

$$\begin{bmatrix} A_{2,2} & \tilde{A}_{2,B} \\ \tilde{A}_{B,2} & \tilde{A}_{B,B} \end{bmatrix} \begin{bmatrix} \mathbf{u}_2 \\ \tilde{\mathbf{u}}_B \end{bmatrix} = \begin{bmatrix} \mathbf{f}_2 \\ \tilde{\mathbf{f}}_B \end{bmatrix} \quad (11.36)$$

matches the discretization on $\Omega_2 \cup \Omega_B$ with a Neumann boundary condition imposed on the face corresponding to Ω_B . There are natural additive and multiplicative variants of this. For the additive form, we take the Dirichlet data, $\mathbf{v}_B$, for $\Omega_1 \cup \Omega_B$ from $\tilde{\mathbf{u}}_B$, the iterate on subdomain 2 at the previous iteration. Similarly, we take $\tilde{\mathbf{f}}_B$ to match the Neumann data that is calculated from the subdomain 1 solution on the previous iteration. Note that this exchange of information between the two subdomains is necessary, since otherwise the solution on each subdomain would not change between iterations. For the multiplicative form, we take the same Dirichlet data for subdomain 1, coming from the subdomain 2 solution at the previous iteration. Now, however, we take $\tilde{\mathbf{f}}_B$ to match the Neumann data generated by the solve on $\Omega_1 \cup \Omega_B$. In both cases, when we want to consider the global solution, we reassemble it from the subdomain solutions on Ω_1 and Ω_2 and from the vector $\tilde{\mathbf{u}}_B$, since $\hat{\mathbf{u}}_B$ is fixed by its Dirichlet data.

There is no real need for the asymmetry here, however, and we could also consider Neumann–Neumann methods, where Neumann boundary conditions are imposed on both subdomains. Again, there are additive and multiplicative forms that differ in whether or not the updated solution on subdomain 1 is used to compute the boundary data for subdomain 2 at each iteration. It is straightforward to generalize this to Robin-type boundary conditions, $au + b\frac{\partial u}{\partial n} = g$, on both $\Omega_1 \cup \Omega_B$ and $\Omega_2 \cup \Omega_B$, on the face of each subdomain corresponding to Ω_B . We recognize that this still fits into the framework of subdomain problems given above, just corresponding to specific choices of $\hat{A}_{1,B}$, $\hat{A}_{B,1}$, $\hat{A}_{B,B}$, $\hat{\mathbf{f}}_B$, $\tilde{A}_{2,B}$, $\tilde{A}_{B,2}$, $\tilde{A}_{B,B}$, and $\tilde{\mathbf{f}}_B$, where the matrices are fixed by the parameters in the boundary conditions and the right-hand side data is computed from the evolving solutions on the other subdomain.

In the context of Robin-type boundary conditions, what remains to be determined is how to choose the parameters. To see this, we first note that there is effectively only one parameter, since for $b \neq 0$, we can divide by b and write the equation as $\lambda u + \frac{\partial u}{\partial n} = \hat{g}$. Second, we can choose λ independently for each subdomain. In a Schwarz-type method, the boundary forcing term, g , is always naturally chosen to match the value of $\lambda u + \frac{\partial u}{\partial n}$ from the other subdomain, either at the previous iteration for the additive method or at the most recently computed value for the multiplicative method. There has been significant analysis in recent years of the choice of λ , primarily based on optimizing the continuum convergence rate, but the insight gained from continuum analysis translates well to the discrete case. Such *optimized* Schwarz methods [102, 103] lead to substantial improvements in convergence properties over classical Schwarz without the optimization, but they still do not lead to h -independent convergence.

11.1.3 ■ Two-level methods

It is quite telling that we have now introduced a number of variants of Schwarz methods (nonoverlapping additive, RAS, multiplicative, Dirichlet–Neumann, Neumann–Neumann, and optimized), but none achieve the desired preconditioner. Namely, we have not yet defined a preconditioner, M , such that $\text{cond}(MA) \leq C$ for a moderate constant C that does not depend adversely on mesh or problem size and the number of subdomains, when A comes from a standard discretization of a simple elliptic problem. A natural question to ask at this stage is, then, whether or not this goal can be achieved. It is, in fact, achievable, but it requires another ingredient in the preconditioner to be realized.

To develop these methods, we return to our original analysis of the nonoverlapping additive Schwarz method for the one-dimensional Poisson problem. In Section 11.1.1, we showed that $\lambda_{\max}(A_D^{-1}A) \geq 1$, while $\lambda_{\min}(A_D^{-1}A) \leq 1/s$, giving $\text{cond}(A_D^{-1}A) \geq s$. In practice, the upper bound on $\lambda_{\max}(A_D^{-1}A)$ is fairly reliable, as shown in the following result.

Theorem 11.5: Eigenvalue bound [194, Thm. 14.6]

Let A be symmetric and positive definite, and let $\Omega_i \subset \Omega = \{1, 2, \dots, n\}$ for $1 \leq i \leq s$, with $\Omega = \cup_{i=1}^s \Omega_i$. Let $\theta_k \subset \{1, 2, \dots, s\}$ for $1 \leq k \leq c$ have the properties that $\cup_{k=1}^c \theta_k = \{1, 2, \dots, s\}$ and if $\ell, m \in \theta_k$, then $R_\ell A R_m^T = 0$. Then, $\lambda_{\max}(A_D^{-1}A) \leq c$.

Proof. First, for any norm, $\lambda_{\max}(A_D^{-1}A) \leq \|A_D^{-1}A\|$, by the definitions of an eigenvalue and norm. Take the norm, then, to be the norm induced by A : $\lambda_{\max}(A_D^{-1}A) \leq \|A_D^{-1}A\|_A$.

By the definition of θ_k in the theorem, we write

$$A_D^{-1}A = \sum_{i=1}^s P_i = \sum_{k=1}^c \sum_{i \in \theta_k} P_i, \quad (11.37)$$

recalling that $P_i = R_i^T (R_i A R_i^T)^{-1} R_i A$ for $1 \leq i \leq s$. Note, in particular, that both P_i and $\sum_{i \in \theta_k} P_i$ are A -orthogonal projections, since $P_i^2 = P_i$, and $P_i P_j = 0$ if $i, j \in \theta_k$ by its definition.

For any A -orthogonal projection, P , we have that $\|P\|_A = 1$. Thus,

$$\|A_D^{-1}A\|_A \leq \sum_{k=1}^c \left\| \sum_{i \in \theta_k} P_i \right\|_A = c. \quad (11.38)$$

Consequently, $\lambda_{\max}(A_D^{-1}A) \leq c$. $\square$

The main message of this result is that as long as the overlap and adjacency between subdomains is kept under control, then $\lambda_{\max}(A_D^{-1}A)$ can be relied upon to be bounded by a moderate constant. Thus, to understand how to improve the preconditioner, we should consider in detail the bound on $\lambda_{\min}(A_D^{-1}A)$. For the one-dimensional Poisson problem considered above and in the last result, with contiguous nonoverlapping subdomains, this bound arises by considering the vectors of all ones either globally, $\mathbf{1}$, or on subdomain i , $\mathbf{1}_i$, and noting that $\mathbf{1}^T A \mathbf{1} = \mathbf{1}_i^T R_i A R_i^T \mathbf{1}_i = 2/h^2$, giving $\mathbf{1}^T A_D \mathbf{1} = 2s/h^2$ and $\lambda_{\min}(A_D^{-1}A) \leq 1/s$. Similar results hold for many other matrices that come from similar discretizations of elliptic PDEs.

A first attempt to fix this problem is to notice that the global mode, $\mathbf{1}$, can be identified as the “source” of the poor bound, so we could modify A_D to try to better match the action of A on this vector. That is, we define a rank-one perturbation,

$$A_{D'}^{-1} = A_D^{-1} + c \mathbf{1} \mathbf{1}^T, \quad (11.39)$$

where we choose constant c in order to make $\mathbf{1}^T A_{D'} \mathbf{1} \approx \mathbf{1}^T A \mathbf{1} = 2/h^2$. By the Sherman–Morrison formula,

$$A_{D'} = A_D - \frac{c}{1 + c \mathbf{1}^T A_D \mathbf{1}} A_D \mathbf{1} \mathbf{1}^T A_D, \quad (11.40)$$

so $\mathbf{1}^T A_{D'} \mathbf{1} = 2s/h^2 - c(2s/h^2)^2/(1 + c2s/h^2)$. Choosing $c = h^2/2$ gives $\mathbf{1}^T A_{D'} \mathbf{1} = \frac{s}{s+1} \frac{2}{h^2}$, so the upper bound on $\lambda_{\min}(A_{D'}^{-1} A)$ associated with this mode changes from $1/s$ to $1 + 1/s$. It is worthwhile to note that since $\mathbf{1}^T A \mathbf{1} = 2/h^2$,

$$A_{D'}^{-1} = A_D^{-1} + \mathbf{1} (\mathbf{1}^T A \mathbf{1})^{-1} \mathbf{1}^T, \quad (11.41)$$

showing that this scaling is also in the form of an A -orthogonal projection onto the span of the constant vector.

This modification fixes errors in the direction of the single mode, $\mathbf{1}$, but does not generally improve the eigenvalue bound, as we can find other vectors that achieve similar bounds. For example, if we consider the case where we have an even number of contiguous, nonoverlapping, equal-size subdomains indexed consecutively from left to right, we can define vector $\mathbf{u}$ by

$$u_j = \begin{cases} 1 & \text{if } j \in \Omega_i, 1 \leq i \leq s/2, \\ -1 & \text{otherwise,} \end{cases} \quad (11.42)$$

and show that $\mathbf{u}^T \mathbf{1} = 0$ and $\mathbf{u}^T A \mathbf{u} / \mathbf{u}^T A_{D'} \mathbf{u} = \mathcal{O}(1/s)$. In fact, there are many such modes.

As a second attempt, we recognize that the inherent difference between A and A_D comes from the *local* character of A_D , and that small eigenvalues (or Ritz values) of A come from global modes that are necessarily poorly represented in A_D . We should also recognize that the best correction to A_D over any space comes in the form of adding an A -orthogonal projection complementary to those already present. With this in mind, we consider defining a restriction operator, R_0 , such that $\text{Range}(R_0^T)$ contains many global modes, including $\mathbf{1}$ and the vector $\mathbf{u}$ from equation (11.42). Then, we extend our additive preconditioner to be

$$A_D = \sum_{i=0}^s R_i^T (R_i A R_i^T)^{-1} R_i \quad (11.43)$$

to include this global correction. A similar extension is made for the multiplicative preconditioner, writing $P_0 = R_0^T (R_0 A R_0^T)^{-1} R_0 A$, so that

$$A_L^{-1} A = I - (I - P_s)(I - P_{s-1}) \cdots (I - P_1)(I - P_0). \quad (11.44)$$

A common choice for R_0 in the nonoverlapping case is as an $s \times n$ matrix with

$$(R_0)_{i,j} = \begin{cases} 1 & \text{if } j \in \Omega_i, \\ 0 & \text{otherwise.} \end{cases} \quad (11.45)$$

In the overlapping case, we define

$$N_j = |\{i \mid j \in \Omega_i\}| \quad (11.46)$$

and then take

$$(R_0)_{i,j} = \begin{cases} 1/N_j & \text{if } j \in \Omega_i, \\ 0 & \text{otherwise.} \end{cases} \quad (11.47)$$

In either case, we have that $\mathbf{1} \in \text{Range}(R_0^T)$, making R_0^T a so-called *partition of unity*. In the nonoverlapping case discussed above, the problematic vector $\mathbf{u}$ in equation (11.42) is also in the range: $\mathbf{u} \in \text{Range}(R_0^T)$. In fact, many vectors $\mathbf{v}$ exist with small A norm, $\mathbf{v}^T A \mathbf{v} / \mathbf{v}^T \mathbf{v}$. For these

modes, we now get that $\mathbf{v}^T \mathbf{A} \mathbf{v} / \mathbf{v}^T \mathbf{A}_D \mathbf{v} \approx 1$. It is important to note that many such partitions of unity can be defined, and that many of these serve the same purpose. For stability reasons, preference is typically given to nonnegative operators, R_0 , with $(R_0)_{i,j} \geq 0$.

When we include a global restriction such as R_0 in the domain decomposition method, it is termed a *two-level* Schwarz method, referring to the fact that $R_0 \mathbf{A} R_0^T$ often represents a *coarse-level* discretization of the problem that led to $\mathbf{A}$ in the first place, since it necessarily couples global modes of the problem, and not just the local modes represented in the other subdomains.

Example 11.6: Two-level nonoverlapping additive Schwarz

Continuing with Examples 11.1, 11.3, and 11.4, Figure 11.1.6 presents results for two-level nonoverlapping additive Schwarz, with one-level results presented with faded lines for comparison. An important change in these results is that we now see very little dependence on the number of subdomains, since larger numbers of subdomains lead to a larger dimension of R_0 in the coarse-grid correction. However, we still see notable degradation of convergence with increasing n_x .

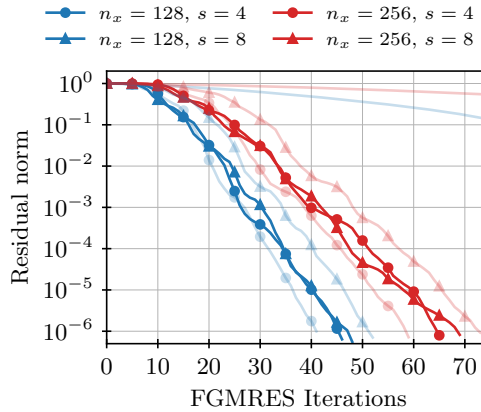


Figure 11.1.6. Relative residual reduction by FGMRES applied to the finite-difference Laplacian on $n_x \times n_x$ meshes, using two-level additive Schwarz preconditioning with $s \times s$ subdomains. Faded lines show the convergence of unpreconditioned FGMRES and FGMRES preconditioned by one-level additive Schwarz for comparison.

Without overlap, two-level Schwarz methods have improved performance from the single-level case but still do not achieve optimal performance. With volumetric overlap, we bound the condition number of the preconditioned system (see [52, Sec. 7.4]) based again on the fine-grid mesh size, h ; the subdomain size, H ; and the amount of overlap, δ , as

$$\text{cond}(\mathbf{A}_D^{-1} \mathbf{A}) \leq C \left(1 + \left(\frac{H}{\delta h} \right)^2 \right). \quad (11.48)$$

This bound is improved to $C \left(1 + \frac{H}{\delta h} \right)$ with additional assumptions on the domain and subdomains [88]. As a result, for a fixed subdomain size of H/h , we get a condition number bound that is independent of h and that improves as δ increases. Similar results hold for the multiplicative case. However, this bound disguises the fact that if H/h is fixed but $h \rightarrow 0$, then the size of the coarse-grid problem grows as h shrinks. If we are using standard direct solvers to solve the coarse-grid problem, then the cost per iteration of a preconditioned Krylov method grows as $H \rightarrow 0$, albeit in a more moderate way than that of a direct solver on the fine grid.

The case of minimal overlap has a range of outcomes, due to the different possible ways of applying boundary conditions to the subdomain problems. One common approach is based on the Neumann–Neumann preconditioner, with a slight modification to account for the over-

lap. For each subdomain, Ω_i , including its boundary, we let $\tilde{A}_{i,i}$ denote the subdomain matrix with Neumann boundary conditions imposed on the subdomain boundaries that are not domain boundaries. Taking the overlap counts, N_j in equation (11.46), for each degree of freedom, j , in the full-domain problem, we define the diagonal scaling matrix D with $d_{j,j} = 1/N_j$ and the modified additive Schwarz preconditioner as

$$A_D^{-1} = \sum_{i=1}^s (DR_i^T) \tilde{A}_{i,i}^{-1} R_i D. \quad (11.49)$$

The point of this scaling is that, for any point j in more than one subdomain, the residual is split and evenly distributed to each subdomain. Consequently, the corrections from each subdomain are evenly weighted in a linear combination to correct the current approximation. This provides a level of stability and improved performance over standard additive Schwarz without such weighting.

Remark 11.7: Singular matrices

For many problems, the discretized PDE system is only nonsingular due to boundary conditions imposed on the boundary of the domain—e.g., the Poisson equation with Dirichlet boundary conditions but no reaction term. In these settings, it is expected that $\tilde{A}_{i,i}$ may not be invertible, unless we have added properties of the PDE (e.g., a positive reaction coefficient in the Poisson case considered above) or partitioned into subdomains (so that each contains a piece of the Dirichlet boundary). When $\tilde{A}_{i,i}$ is not invertible, to construct the correction over Ω_i , it is common to simply find any solution of $\tilde{A}_{i,i} z_i = R_i D r$, for example, in the least-squares sense. If the subdomain systems are to be solved exactly, this simply replaces the inverse of $\tilde{A}_{i,i}$ in equation (11.49) by its pseudoinverse. If an inexact solve of the subdomain problems is to be used, standard iterative approaches for solving singular systems can be used.

To add a second level to the weighted additive Schwarz preconditioner from equation (11.49), one possibility is to proceed additively, as above. However, another common approach is to take a multiplicative combination of corrections between the two levels. Given a residual, r , we compute the action of the preconditioner on r in two stages:

$$z_1 = A_D^{-1} r, \quad (11.50a)$$

$$z = z_1 + R_0^T (R_0 A R_0^T)^{-1} R_0 (r - A z_1). \quad (11.50b)$$

Combining these, we write

$$z = A_D^{-1} r + R_0^T (R_0 A R_0^T)^{-1} R_0 (I - A A_D^{-1}) r. \quad (11.51)$$

This gives the preconditioner

$$M = A_D^{-1} + R_0^T (R_0 A R_0^T)^{-1} R_0 - R_0^T (R_0 A R_0^T)^{-1} R_0 A A_D^{-1}. \quad (11.52)$$

An important note about this preconditioner is that it is not, in general, symmetric, even with symmetric A and A_D^{-1} . This is addressed by using the second level in what is known as a *balancing* preconditioner, defined as a three-stage operation on the given residual, r :

$$z_1 = R_0^T (R_0 A R_0^T)^{-1} R_0 r, \quad (11.53a)$$

$$z_2 = z_1 + A_D^{-1} (r - A z_1), \quad (11.53b)$$

$$z = z_2 + R_0^T (R_0 A R_0^T)^{-1} R_0 (r - A z_2). \quad (11.53c)$$

In this form, if A and A_D^{-1} are symmetric and positive definite, then so is the resulting preconditioner. For finite-element discretizations of diffusion equations in two and three dimensions, it has been shown that [161]

$$\text{cond}(MA) \leq C (1 + \log^2(H/h)), \quad (11.54)$$

so long as the diagonal weighting matrix used in equation (11.49) is chosen in a compatible way.

Example 11.8: Two-level balancing preconditioner with minimal overlap

As a concluding example, we consider an algebraic variation on the balancing preconditioner where, instead of forming $\tilde{A}_{i,i}$ as the Neumann problem on each subdomain, we approximate that matrix by computing $R_i D A D R_i^T$, where D is the same diagonal matrix implementing the partition-of-unity scaling as above, with $d_{j,j} = 1/N_j$, for N_j defined as in equation (11.46). This construction corresponds to one possible implementation of Neumann boundary conditions in the finite-difference case, when ghost points are used and scaled appropriately (see Section 3.4 for more details).

In order to provide a fair comparison, at left of Figure 11.1.7 we first present results for a one-level additive Schwarz preconditioner with minimal overlap that also incorporates partition-of-unity scaling, with the faded lines showing convergence for the one-level additive case without overlap. In comparison with the left-hand plot in Figure 11.1.5, we see that the partition-of-unity scaling by itself does not have a large impact on convergence. The results in the right-hand plot of Figure 11.1.7, however, show further improvements in convergence. Now, we note that increasing s leads to *improved* performance due to the larger space included in the coarse-grid correction and that, overall, convergence is much improved but still not independent of n_x .

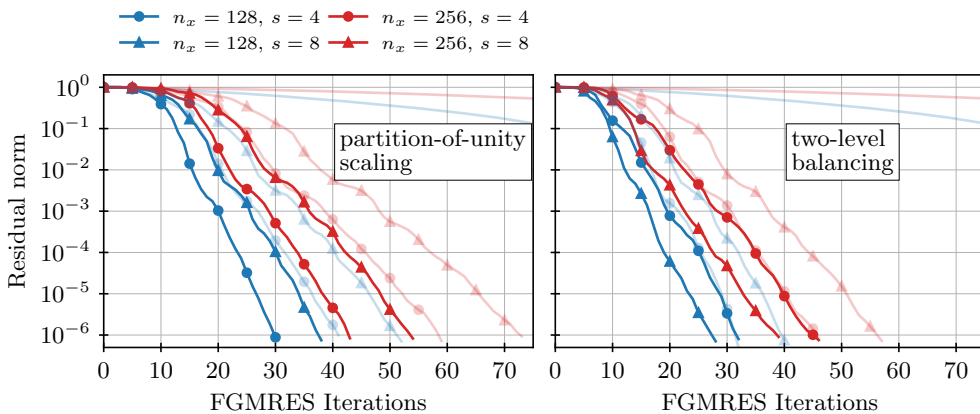


Figure 11.1.7. (left) Relative residual reduction by FGMRES preconditioned by additive Schwarz with minimal overlap and partition-of-unity scaling applied to the finite-difference Laplacian on $n_x \times n_x$ meshes. Here, faded lines show the convergence of unpreconditioned FGMRES and FGMRES preconditioned by additive Schwarz without overlap for comparison. (right) Relative residual reduction by FGMRES preconditioned by the two-level balancing preconditioner with minimal overlap applied to the finite-difference Laplacian on $n_x \times n_x$ meshes. Here, faded lines show the convergence of unpreconditioned FGMRES and FGMRES preconditioned by one-level additive Schwarz with overlap for comparison.

Similar techniques are possible for other methods with minimal overlap. In addition, a closely related approach is the family of methods known as finite-element tearing and interconnecting (FETI) preconditioners, which can be naturally applied to a much broader class of problems. In

all such cases, the key to success is in identifying global modes of the problem that the solution of the subdomain problems does a poor job of resolving. For many problems, these are known already from continuum analysis, such as globally smooth modes for Poisson, or the rigid-body modes of linear elasticity. If these modes are not known *a priori*, we can often compute good approximations to them, either directly or by solving an auxiliary problem, such as is done in adaptive multigrid [53] or the *generalized eigenvalues in the overlap* (GenEO) methods from domain decomposition [205, 87].

11.2 ■ Multigrid methods

Objectives

As we observed in Section 11.1.3, adding a second, global level is critical to attaining grid-independent convergence in a domain decomposition method. A natural question to ask is whether similar improvements in performance are gained by applying the same methodology in other ways. As you will demonstrate in this (and subsequent) sections, the answer to this question is “yes,” and the methodology has been used to generate effective linear solution strategies for a wide array of problems. Specifically, you will

- discuss the “smoothing” properties of the weighted Jacobi iteration;
- motivate the idea for a coarse-grid correction from another perspective;
- introduce the two-grid and multigrid algorithms and some variants; and
- analyze the cost and effectiveness of multigrid methods.

We begin our consideration of a “multiple grid” approach by revisiting the performance of the Jacobi iteration for the finite-difference discretization of the one-dimensional Poisson problem with Dirichlet boundary conditions on a uniform mesh, $x_0, x_1, \dots, x_{n_x}$, with $h = 1/n_x$. This discretization gives the linear system

$$(A\mathbf{u})_j = \frac{1}{h^2} (-u_{j-1} + 2u_j - u_{j+1}) = f_j \text{ for } 1 \leq j \leq n_x - 1, \quad (11.55)$$

where we implicitly take $u_0 = u_{n_x} = 0$. A direct calculation shows that the eigenvectors, $\mathbf{v}_i$, of A can be written as $(\mathbf{v}_i)_j = \sin\left(\frac{i\pi j}{n_x}\right)$ for $1 \leq i \leq n_x - 1$:

$$(A\mathbf{v}_i)_j = \frac{1}{h^2} \left(-\sin\left(\frac{i\pi(j-1)}{n_x}\right) + 2\sin\left(\frac{i\pi j}{n_x}\right) - \sin\left(\frac{i\pi(j+1)}{n_x}\right) \right) \quad (11.56a)$$

$$= \frac{1}{h^2} \left(2\sin\left(\frac{i\pi j}{n_x}\right) - 2\cos\left(\frac{i\pi}{n_x}\right) \sin\left(\frac{i\pi j}{n_x}\right) \right) \quad (11.56b)$$

$$= \frac{4}{h^2} \sin^2\left(\frac{i\pi}{2n_x}\right) \sin\left(\frac{i\pi j}{n_x}\right) = \frac{4}{h^2} \sin^2\left(\frac{i\pi}{2n_x}\right) (\mathbf{v}_i)_j, \quad (11.56c)$$

where we have used a half-angle identity in the final step. Thus, we conclude that $\mathbf{v}_i$ is an eigenvector of A with eigenvalue $\lambda_i(A) = \frac{4}{h^2} \sin^2\left(\frac{i\pi}{2n_x}\right)$ for $1 \leq i \leq n_x - 1$. For the smaller eigenvalues, using the approximation that $\sin(x) \approx x$ (for small x), we have

$$\lambda_i(A) \approx \frac{4}{h^2} \left(\frac{i\pi}{2n_x} \right)^2 = i^2 \pi^2 \text{ for } i \ll n_x. \quad (11.57)$$

Similarly, if we consider the continuum operator $-\partial_{xx}$ and a Fourier mode of the form $v(x) = \sin(i\pi x)$, then $-\partial_{xx}v(x) = i^2\pi^2v(x)$. This shows that the eigenvalues of the discrete operator converge to those of the continuum operator as $n_x \rightarrow \infty$.

With this, we write down the eigenvalues of the error-propagation operator for the weighted Jacobi iteration:

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \omega D^{-1}(\mathbf{f} - A\mathbf{u}_k) \quad (11.58a)$$

$$\Rightarrow \mathbf{e}_{k+1} = (I - \omega D^{-1}A) \mathbf{e}_k. \quad (11.58b)$$

Since $D = \frac{2}{h^2}I$, we then have that the vectors $\mathbf{v}_i$ are also eigenvectors of $I - \omega D^{-1}A$, with eigenvalues

$$\lambda_i(I - \omega D^{-1}A) = 1 - 2\omega \sin^2\left(\frac{i\pi}{2n_x}\right). \quad (11.59)$$

Since $\sin^2\left(\frac{i\pi}{2n_x}\right) < 1$ for $1 \leq i \leq n_x - 1$, we conclude that the weighted Jacobi iteration is convergent for $0 < \omega \leq 1$, since $|\lambda_i(I - \omega D^{-1}A)| < 1$ for $1 \leq i \leq n_x - 1$ in this case.

11.2.1 ■ The smoothing effect

A natural question is what is the “best” choice for the relaxation parameter, ω . We first observe that there is no choice of ω that results in a uniformly good reduction of error for small i while staying convergent. Indeed, the “first” eigenvalue (corresponding to $\mathbf{v}_1$, the eigenvector corresponding to the smallest eigenvalue of A) satisfies $|\lambda_1| \rightarrow 1$ as $n_x \rightarrow \infty$ for any ω that is independent of n_x . Figure 11.2.1 shows the eigenvalues of weighted Jacobi, indexed by eigenvector, $\mathbf{v}_i$, for three different values of ω , showing the significant impact that choosing ω has on convergence of error in the direction of some, but not all, eigenvectors.

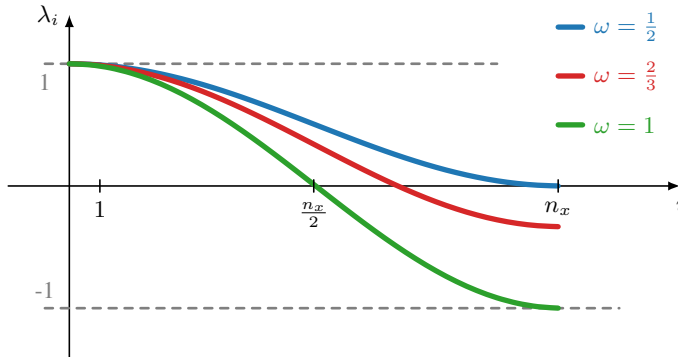


Figure 11.2.1. Convergence factor of weighted Jacobi relaxation versus frequency for different relaxation parameters ω .

Nevertheless, to optimize Jacobi convergence, we may be tempted to select ω in order to optimize $\rho(I - \omega D^{-1}A)$, leading to the condition that $|\lambda_1(I - \omega D^{-1}A)| = |\lambda_{n_x-1}(I - \omega D^{-1}A)|$, which gives $\omega = 1$ as shown in Figure 11.2.1. Another choice, however, is to choose ω to achieve a uniform performance bound over a range of values of i that are bounded away from small values. For example, if we consider the range $n_x/2 \leq i < n_x$, then the eigenvalues lie in the range

$$1 - 2\omega \sin^2(\pi/4) \geq \lambda_i(I - \omega D^{-1}A) > 1 - 2\omega \sin^2(\pi/2). \quad (11.60)$$

Evaluating $\sin(\pi/4)$ and $\sin(\pi/2)$ gives the bound $1 - \omega \geq \lambda_i (I - \omega D^{-1}A) > 1 - 2\omega$, and the optimal reduction over these modes occurs when we choose $\omega = 2/3$, so that the upper and lower bounds are equal but opposite in sign. For this choice, we then have the bound that

$$|\lambda_i (I - \omega D^{-1}A)| \leq 1/3 \text{ for } n_x/2 \leq i < n_x. \quad (11.61)$$

From here, we conclude that weighted Jacobi is very effective as a stationary iteration at damping errors in the upper half of the spectrum of A , regardless of the value of n_x . The same can be shown for Gauss–Seidel (although not for successive overrelaxation (SOR) with an optimal relaxation weight) and for a wide variety of similar problems, including two- and three-dimensional Poisson problems, with both finite-difference and finite-element discretizations.

A typical error, e_k , can be thought of in terms of its decomposition into the eigenmodes of the Jacobi iteration, just as in equation (7.31). Our observation that $\lambda_i \approx 1$ for small i but that $|\lambda_i| < 1/3$ for $n_x/2 \leq i < n_x$ implies that applying such an iteration to an initial approximation, $\mathbf{u}_0$, with error e_0 , leads to an error e_k that is dominated by modes with small i after even a few iterations. Given the form of the eigenvectors, the dominant eigenvectors are all seen to be “smooth” in that their values vary slowly relative to the mesh points. For example, consider the case with $n_x = 64$ shown in Figure 11.2.2. Here, we see that the initial error (in gray, based on a random initial guess) is highly oscillatory, reflecting an error composed of both high- and low-frequency functions. After a few Jacobi sweeps (iterations), the error is effectively smooth. As a result, we often refer to these iterations as *smoothers*, or as *relaxation methods*.

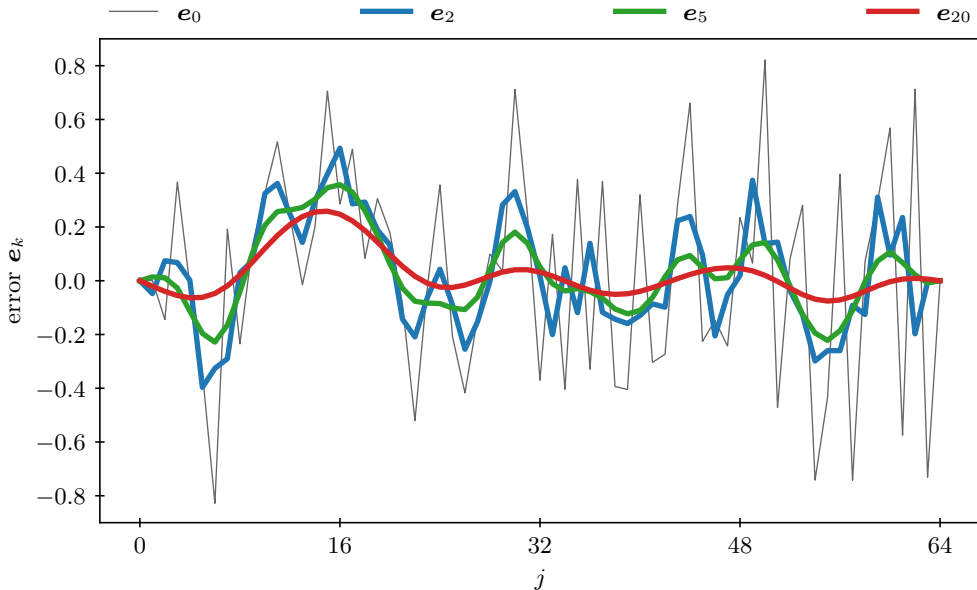


Figure 11.2.2. Jacobi relaxation with $\omega = 2/3$ and an initial error $e_0 = \mathbf{u} - \mathbf{u}_0$ after two, five, and 20 iterations.

As with the Schwarz methods considered above, we now ask if there is a way to correct these errors by a complementary iteration. To see how to do this, we note that the modes that are not effectively corrected by the weighted-Jacobi iteration with $\omega = 2/3$ are given by $(\mathbf{v}_i)_j = \sin(i\pi j/n_x)$ for $1 \leq i < n_x/2$. Taking n_x to be even (for convenience in what follows), we

rewrite this at even-indexed grid points as

$$(\mathbf{v}_i)_{2j} = \sin\left(\frac{i\pi(2j)}{n_x}\right) = \sin\left(\frac{i\pi j}{n_x/2}\right). \quad (11.62)$$

For $1 \leq i < n_x/2$, these correspond exactly to the eigenvectors of the same equation discretized on a grid with coarser mesh size, $H = 2/n_x = 2h$. This is true in general, that the dominant errors left over after a smoothing iteration can be accurately represented on *grid* $2h$, with half as many points in one dimension (and one-fourth as many in two dimensions, etc.). Our aim, now, is to complement the smoothing iteration on grid h with a *coarse-grid correction* from grid $2h$. To develop this methodology, we ask two questions:

1. What problem should we solve on grid $2h$?
2. How do we correct the approximation on grid h ?

Remark 11.9: Convergence factors and condition numbers

Notice that the discussion above focuses on the convergence factor, $\rho(I - \omega D^{-1}A)$, of weighted Jacobi relaxation rather than the condition number of the preconditioned matrix, $\omega D^{-1}A$. In part, this is because the choice of ω has an impact (as shown above) on the details of the convergence of the Jacobi iteration, but it does not have any impact on the condition number of $D^{-1}A$, since multiplying by $\omega > 0$ simply scales the largest and smallest eigenvalues or singular values of $D^{-1}A$ by ω .

For the two-grid and multigrid methods that we introduce next, we continue to focus on measuring convergence factors, now written as $\rho(I - M_{TG}A)$ for some operator M_{TG} . In many cases (such as when suitably weighted Jacobi relaxation is used both before and after a coarse-grid correction), the operator M_{TG} is symmetric and positive definite (see [212], for example, for sufficient conditions), and we relate $\rho(I - M_{TG}A)$ to $\text{cond}(M_{TG}A)$ as follows.

When both A and M_{TG} are symmetric and positive definite, then

$$\text{cond}(M_{TG}A) = \left\| (M_{TG}A)^{-1} \right\| \|M_{TG}A\| = \frac{\lambda_{\max}(M_{TG}A)}{\lambda_{\min}(M_{TG}A)}, \quad (11.63)$$

where we note that the eigenvalues of $M_{TG}A$ must be real and positive, since it is similar to the symmetric and positive-definite matrix $A^{1/2}M_{TG}A^{1/2}$. However, since $\rho(I - M_{TG}A)$ is the absolute value of an eigenvalue of $I - M_{TG}A$, it must be equal to either $1 - \lambda_{\min}(M_{TG}A)$ or $\lambda_{\max}(M_{TG}A) - 1$ and must satisfy

$$\rho(I - M_{TG}A) \geq 1 - \lambda_{\min}(M_{TG}A), \quad (11.64a)$$

$$-\rho(I - M_{TG}A) \leq 1 - \lambda_{\max}(M_{TG}A). \quad (11.64b)$$

From these, we conclude that $\lambda_{\min}(M_{TG}A) \geq 1 - \rho(I - M_{TG}A)$ and $\lambda_{\max}(M_{TG}A) \leq 1 + \rho(I - M_{TG}A)$. This gives us the bound that

$$\text{cond}(M_{TG}A) \leq \frac{1 + \rho(I - M_{TG}A)}{1 - \rho(I - M_{TG}A)}. \quad (11.65)$$

Thus, understanding how $\rho(I - M_{TG}A)$ depends on the mesh size or other problem parameters gives us a constructive bound on how $\text{cond}(M_{TG}A)$ depends on those same parameters. In particular, if we can show that $\rho(I - M_{TG}A) \leq c < 1$, independent of problem parameters, we have that $\text{cond}(M_{TG}A) \leq \frac{1+c}{1-c}$, which is also independent of problem parameters.

11.2.2 ■ Coarse-grid correction

We answer the second of these questions first by noting that, with regularly structured grids, geometric interpolation provides a natural map to take a vector on grid $2h$ onto a vector on grid h . Using this type of interpolation then leads to the *geometric* multigrid approach discussed in this section. For one-dimensional grids, a natural operator is that of *linear interpolation*: for grid points on the fine grid whose geometric position (point location) coincides with that of a coarse-grid node, we simply “inject” the value from the coarse grid to the fine grid; for grid points on the fine grid whose geometric location is between two coarse-grid nodes, we use a linear interpolant of the coarse-grid values to assign the fine-grid values. This defines a linear operator, P , that maps coarse-grid vectors to fine-grid vectors. Consider a uniform fine grid of mesh points $x_0^h, x_1^h, \dots, x_{n_x}^h$ with n_x even, and with mesh spacing given by $h = 1/n_x$. In addition, let $2h$ be the mesh spacing on the coarse grid, given by $x_0^{2h}, \dots, x_{n_x/2}^{2h}$. For a given coarse-level vector, $\mathbf{v}^{2h}$, we express P away from the boundaries by

$$(P\mathbf{v}^{2h})_{2j} = v_j^{2h}, \quad (11.66a)$$

$$(P\mathbf{v}^{2h})_{2j+1} = (v_j^{2h} + v_{j+1}^{2h})/2, \quad (11.66b)$$

where j is a coarse-level index. This is shown in Figure 11.2.3.

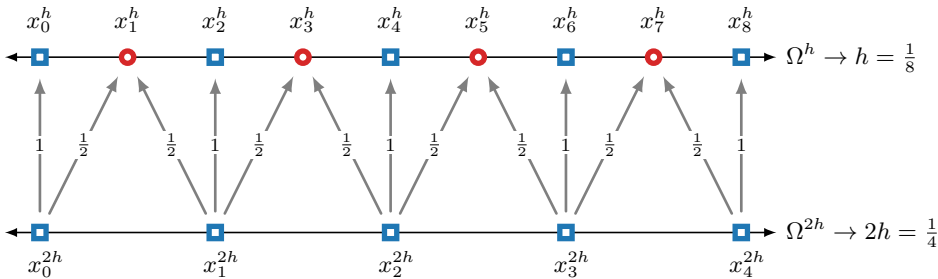


Figure 11.2.3. Interpolation pattern for one-dimensional grids with Neumann or Robin boundary conditions.

The boundary conditions require careful consideration. For example, with Neumann or Robin boundary conditions on both the left and right endpoints (and n_x even), the above prescription is well defined for all points on the fine grid, as illustrated in Figure 11.2.3. With Dirichlet boundary conditions, we typically eliminate degrees of freedom such as v_0 on both the fine and coarse grids. Here, then, the prescription for $(P\mathbf{v}^{2h})_0$ is not needed, and the one for $(P\mathbf{v}^{2h})_1$ is ill defined. However, since we consider the boundary degree of freedom to be *known* for a Dirichlet boundary condition, the error we expect to correct at the boundary should be zero. Thus, we usually correct the above prescription to assume that the correction from the boundary degree of freedom is zero, yielding $(P\mathbf{v}^{2h})_1 = v_1^{2h}/2$, although other scalings are possible depending on discretization and boundary conditions. On tensor-product grids in two dimensions, it is common to use tensor-product (e.g., Kronecker-product; see equation (3.20)) interpolation operators, again taking care of interpolation near boundaries, as pictured in Figure 11.2.4.

With an interpolation operator, P , specified to map vectors from grid $2h$ to grid h , we now propose a concrete correction to a given approximation, $\mathbf{u}_0^h$, to the solution of the grid h problem, $A\mathbf{u}^h = \mathbf{f}^h$, with a correction of the form $P\mathbf{u}^{2h}$. Here, $\mathbf{u}^{2h}$ is a vector computed (somehow)

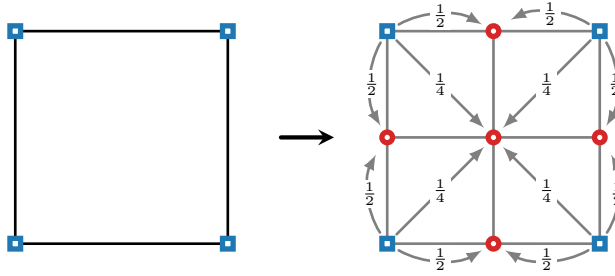


Figure 11.2.4. Interpolation pattern for two-dimensional tensor-product grids with a coarse cell at left and its uniform refinement at right.

on grid $2h$. Generally, we assume that $\mathbf{u}_0^h$ comes from applying a few steps of a smoothing iteration to another vector, but we do not rely on this assumption in what follows. Given $\mathbf{u}_0^h$ and $\mathbf{u}^{2h}$, we take our *corrected* approximation to be $\mathbf{u}_0^h + P\mathbf{u}^{2h}$ and now ask what the best possible approximation can be of this form.

As is typical, the use of the word “best” implies a metric that can be used to say that one corrected approximation is better than another. In our setting, we take this to be some matrix norm, $\|\mathbf{y}\|_M^2 = \mathbf{y}^T M \mathbf{y}$, for symmetric and positive-definite matrix, M , giving the optimization problem

$$\min_{\mathbf{u}^{2h}} \|\mathbf{u}^h - (\mathbf{u}_0^h + P\mathbf{u}^{2h})\|_M. \quad (11.67)$$

In the specific case of M as the identity matrix, this asks for the best approximation in the Euclidean ℓ_2 norm, while if $M = A^T A$, this asks for the correction that minimizes the residual after correction. We compute the minimum by taking the derivative and setting it equal to zero; for convenience, we first square the quantity to be minimized. This gives

$$\|\mathbf{u}^h - (\mathbf{u}_0^h + P\mathbf{u}^{2h})\|_M^2 = (\mathbf{e}_0^h - P\mathbf{u}^{2h})^T M (\mathbf{e}_0^h - P\mathbf{u}^{2h}) \quad (11.68a)$$

$$= (\mathbf{e}_0^h)^T M \mathbf{e}_0^h - 2(\mathbf{u}^{2h})^T P^T M \mathbf{e}_0^h + (\mathbf{u}^{2h})^T P^T M P \mathbf{u}^{2h}. \quad (11.68b)$$

Differentiating this with respect to $\mathbf{u}^{2h}$ and setting the resulting derivative to zero shows that the optimal correction is characterized as the solution, $\mathbf{u}^{2h}$, of

$$P^T M P \mathbf{u}^{2h} = P^T M \mathbf{e}_0^h = P^T M (\mathbf{u}^h - \mathbf{u}_0^h). \quad (11.69)$$

While this is elegant mathematically, it has the practical problem that the right-hand side may not be computable, since it depends on the unknown solution, $\mathbf{u}^h$. However, when A is symmetric and positive definite, we arrive at a particularly nice *coarse-grid problem* by taking $M = A$, which gives the minimizer satisfying

$$P^T A P \mathbf{u}^{2h} = P^T (\mathbf{f}^h - A \mathbf{u}_0^h). \quad (11.70)$$

We call the update $P\mathbf{u}^{2h}$ a *Galerkin* coarse-grid correction, and the coarse-grid operator $A^c = P^T A P$ is termed a *Galerkin* coarse-grid operator.

As with previous sections, we have used A rather than A^h to denote the grid h operator. However, we use the notation A^c to distinguish the operator from a finite-difference discretization on grid $2h$, which is denoted by A^{2h} . For finite-difference discretizations, it is common to

scale equation (11.70) by scaling P^T , writing $R := \frac{1}{2^d} P^T$, where d is the physical dimension of the problem discretized to yield A . Then, the coarse-grid equation becomes $RAP\mathbf{u}^{2h} = R(\mathbf{f} - A\mathbf{u}_0^h)$. One reason to do this is to ensure that the scaling from P^T matches the scaling introduced by the coarse-level *rediscretization* (i.e., directly constructing the discrete problem on the coarse grid), using A^{2h} . To see this, consider a relaxation method such as weighted Jacobi, with error-propagation operator G and a *weighted* (with γ) operator, $R = \gamma P^T$, resulting in a coarse-level correction process given by

$$\mathbf{u}^h \leftarrow \mathbf{u}^h + \gamma P(A^c)^{-1} P^T (\mathbf{f} - A\mathbf{u}^h), \quad (11.71)$$

where A^c represents the coarse-level operator, either the Galerkin operator with $A^c = P^T A P$ or the rediscretization operator with $A^c = A^{2h}$. A bound on convergence on this two-level method is then given by

$$\rho = \|G^T(I - \gamma P(A^c)^{-1} P^T A)G\|_A. \quad (11.72)$$

If $A^c = P^T A P$, then the two-level process is a projection when $\gamma = 1$ and, consequently, $\gamma = 1$ minimizes the bound convergence given by ρ . If we choose to rediscretize the problem on the coarse grid, giving $A^c = A^{2h}$, then A^c may not equal $P^T A P$. For example, for the one-dimensional Poisson problem discretized with finite differencing, a weighting of $\gamma = \frac{1}{2}$ is required to achieve the expected convergence (see Figure 11.2.5). Another related approach is that of *overcorrection*, which is sometimes applied in an algebraic context; see Section 11.3.

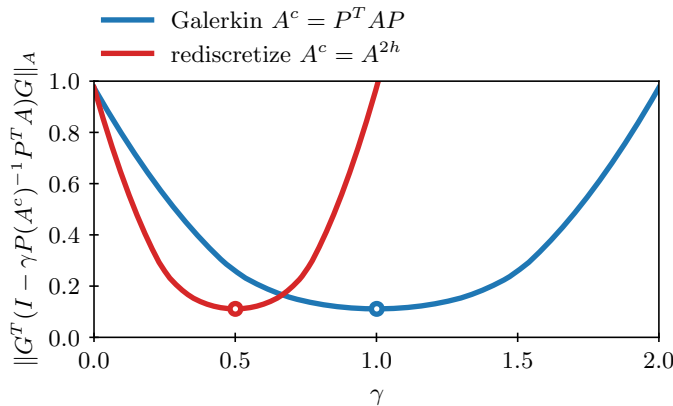


Figure 11.2.5. Convergence of weighted coarse-grid correction for the one-dimensional finite-difference Poisson operator.

We call R and P^T *restriction* operators, since it is easily shown that they naturally map from residuals on the fine grid h to grid $2h$. The interpretation of the right-hand side of equation (11.70) (or its scaled equivalent, $RAP\mathbf{u}^{2h} = R(\mathbf{f} - A\mathbf{u}_0)$) is straightforward: the restriction operator applied to the residual yields a forcing term that determines $\mathbf{u}^{2h}$. To understand the matrix on the left-hand side of equation (11.70), we compute RAP for the case of the one-dimensional Poisson equation discretized using finite differences on a uniform mesh with n_x even. This is easily done by considering canonical unit basis vectors, $\mathbf{e}^{(i, 2h)}$,

$$\left(\mathbf{e}^{(i, 2h)}\right)_j = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{otherwise,} \end{cases} \quad (11.73)$$

for i away from the boundaries. Direct calculation then yields

$$\left(Pe^{(i,2h)} \right)_j = \begin{cases} 1 & \text{if } j = 2i, \\ 1/2 & \text{if } j = 2i \pm 1, \\ 0 & \text{otherwise,} \end{cases} \quad (11.74a)$$

$$\left(APe^{(i,2h)} \right)_j = \begin{cases} 1/h^2 & \text{if } j = 2i, \\ -1/(2h^2) & \text{if } j = 2i \pm 2, \\ 0 & \text{otherwise,} \end{cases} \quad (11.74b)$$

$$\left(RAPe^{(i,2h)} \right)_j = \begin{cases} 2/(2h)^2 & \text{if } j = i, \\ -1/(2h)^2 & \text{if } j = i \pm 1, \\ 0 & \text{otherwise.} \end{cases} \quad (11.74c)$$

Here both $Pe^{(i,2h)}$ and $APe^{(i,2h)}$ are grid h vectors, while $RAPe^{(i,2h)}$ is again a vector on grid $2h$, giving column i of matrix RAP . Note also that the above calculation shows that RAP is exactly the discretization of the Poisson equation on grid $2h$. This is, of course, a consequence of the choice of scaling, $R = \frac{1}{2}P^T$. For finite-difference discretizations in d dimensions, we call the operator $R = \frac{1}{2^d}P^T$ the *full-weighting* restriction operator. While the relation between RAP and a direct discretization on grid $2h$ is only true for finite-difference discretizations in one dimension, it is common practice to replace RAP by direct discretization on grid $2h$ for finite-difference discretizations in two and three dimensions. For finite-element methods, the choice of $R = P^T$ is much more natural, and, in fact, for Galerkin finite-element discretizations, P^TAP is always the corresponding discretization matrix on grid $2h$ when P is the finite-element interpolation operator consistent with the basis functions used for discretization on grids h and $2h$.

The *two-grid* iteration (sometimes referred to as a two-grid cycle) arises from combining the two ingredients above: a smoothing iteration with the coarse-grid correction process. Given A and f from a discretization process on grid h and an initial guess, u_0 , on grid h , a typical two-grid iteration is written as in Algorithm 11.1. There, we consider ν_1 prerelaxation and ν_2 postrelaxation iterations specified by matrices M_1 and M_2 ; often, we take $M_1 = M_2$ or $M_1 = M_2^T$, but this depends on the problem under consideration. It is typical to take very small values of ν_1 and ν_2 ; often the most efficient performance is found for $\nu_1 + \nu_2 = 2$ or 3.

Writing the error-propagation operator for the corresponding stationary iteration as $I - M_{TGA}$, we view the *convergence factor* as the spectral radius $\rho(I - M_{TGA})$. If the algorithmic components and parameters are properly chosen, then convergence factors of 0.1 are often attained for simple model problems, as we show in the following examples. Unfortunately, the two-grid cycle is not an *efficient* iteration, since the grid $2h$ problem is not substantially smaller than the fine-grid problem on grid h . Thus, in the next subsection, we ask the question of whether or not we can make the algorithm more efficient, without sacrificing its effectiveness.

Example: the two-grid cycle in 1D

As an example, consider the 1D Poisson problem $-u_{xx} = f$ on $[0, 1]$ with homogeneous Dirichlet boundary conditions, discretized with finite differences and $n_x = 64$. We construct a right-hand side f so that $u(x) = \sin(\pi(3x)^2)$ exactly solves the problem and use a two-level multigrid scheme with standard linear interpolation and a Galerkin representation of the coarse-grid problem. Two pre- and postrelaxation sweeps of weighed Jacobi with $\omega = 2/3$ are used ($\nu_1 = \nu_2 = 2$ in Algorithm 11.1).

Figure 11.2.6 details the solution and error throughout a two-grid cycle. With the exact solution given by u and the (random) initial approximation given by $u^{(a)}$, the top panels show

Algorithm 11.1: Two-grid algorithm

Input : A , linear system matrix
 f , linear system right-hand side
 u_0 , initial guess
 R , restriction matrix
 P , interpolation matrix
 $M_{1,2}$, pre-, postrelaxation matrix
 $\nu_{1,2}$, number of pre-, postrelaxation sweeps
Output : u , approximation after one cycle

```

1  $u \leftarrow u_0$ 
2 for  $k = 1$  to  $\nu_1$ 
3    $u \leftarrow u + M_1(f - Au)$                                      {prerelaxation}
4    $f^{2h} \leftarrow R(f - Au)$                                      {restrict residual of current approximation  $u$ }
5   Solve  $RAPu^{2h} = f^{2h}$  (or  $A^{2h}u^{2h} = f^{2h}$ )                 {solve coarse-level problem for  $u^{2h}$ }
6    $u \leftarrow u + Pu^{2h}$                                          {coarse-grid correction}
7 for  $k = 1$  to  $\nu_2$ 
8    $u \leftarrow u + M_2(f - Au)$                                      {postrelaxation}

```

a poor approximation after only two sweeps of relaxation, given by $u^{(b)}$; however, the error already exhibits “smooth” features. The middle panels highlight the accuracy of the solution after coarse-grid correction, given by $u^{(c)}$, which is also observed in the error in the right plot. Finally, the approximate solution after completing the two-level iteration is given by $u^{(d)}$; here, we see both smooth error and high accuracy, reducing the error by more than a magnitude with just one two-grid cycle.

Example: the two-grid cycle in 1D and the role of relaxation

In the previous example, we observed the role of relaxation in rendering a smooth error for the upcoming coarse-grid correction step. One question is how much of a role relaxation plays in *convergence*. Here, we use the same example and setup but vary the number of prerelaxation (ν_1) and postrelaxation (ν_2) sweeps in the method.

Figure 11.2.7 shows the improved convergence in increasing the number of sweeps from one to six (in total) but also shows the diminishing returns on investing in more relaxation. For example, in moving from two relaxation sweeps to three, the method only requires one or two fewer iterations to reach the same residual reduction. This is attributed to coarse-grid correction, which limits the error reduction any further, and to the fact that relaxation itself is less effective with additional iterations. Notice also that, in this example, we see very good reduction in error when we use no prerelaxation at all. While the cycle is nonsymmetric, the error after one cycle will exhibit some level of smoothness, leading to an effective coarse-grid correction scheme in the subsequent cycle. Using one prerelaxation and one postrelaxation sweep is a common choice in practice, and we see the balance achieved with this cycle for this example. In other situations, a larger number of relaxation sweeps per cycle may lead to a more effective solver.

Example: the two-grid cycle in 2D

Following the previous 1D examples, we consider the same setup in 2D but on an $n_x \times n_y$ mesh of the unit square with $n_x = n_y = 64$ and with an exact solution given by $u(x, y) = \sin(\pi(2x)^2) \sin(\pi(2y)^2)$. We use Gauss–Seidel relaxation in this example (with $\nu_1 = \nu_2 = 2$).

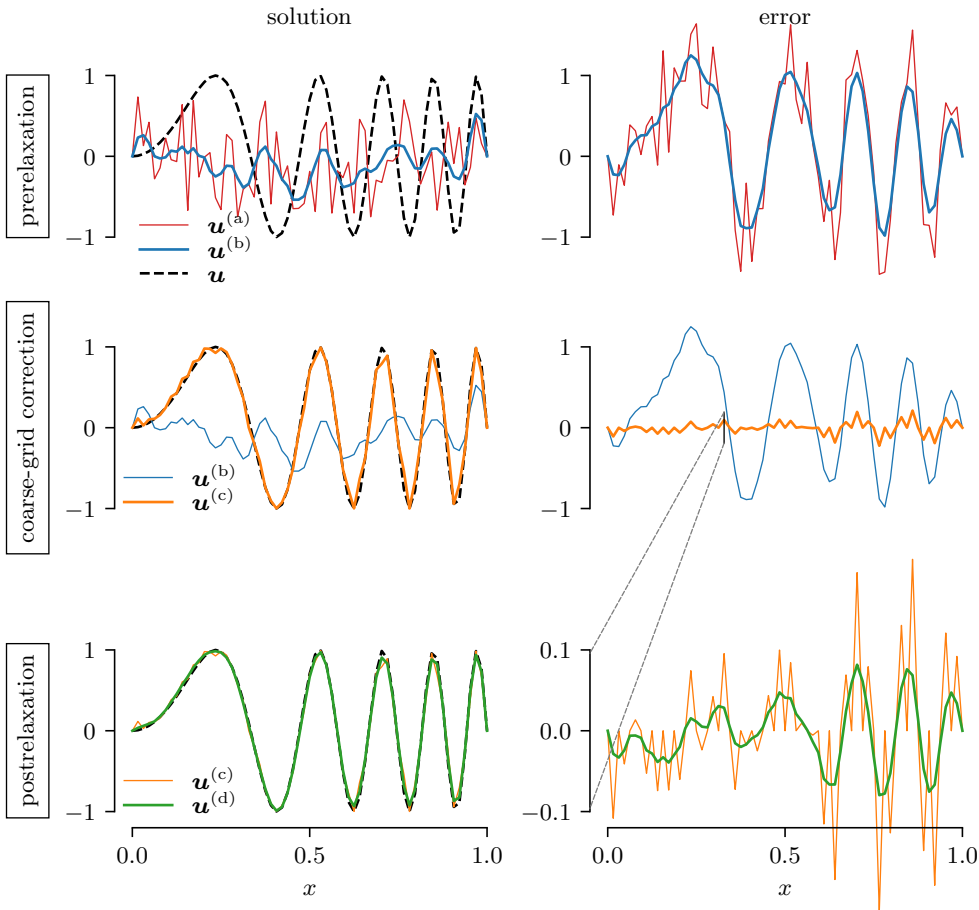


Figure 11.2.6. Solution and error throughout a two-level cycle applied to the finite-difference Laplacian in 1D with $n_x = 64$. The curves show the exact solution, $\mathbf{u}$; the initial approximation, $\mathbf{u}^{(a)}$; the approximation after relaxation, $\mathbf{u}^{(b)}$; the approximation after relaxation and coarse-grid correction, $\mathbf{u}^{(c)}$; and the approximation after one full (two-grid) iteration, $\mathbf{u}^{(d)}$. The solution is provided in the left panel; the right panel shows the corresponding error.

Figure 11.2.8 shows the random initial solution (and error) at the top. After two sweeps of relaxation, the middle figures reveal smooth features in the error while the solution remains largely unresolved. Finally, the bottom figures show a highly accurate approximation with slightly more than an order of magnitude drop in the size of the error after just one two-grid cycle.

11.2.3 ■ The multigrid μ -cycle

The main cost in Algorithm 11.1 is the solution to the linear system on grid $2h$. For a one-dimensional problem on a uniform mesh, with factor-two coarsening, A^{2h} is one-half the size of the original problem on grid h . Similarly, it is one-fourth the size in two dimensions, and one-eighth the size in three dimensions. In order to reduce this cost, we then have two apparent options: (1) to not revisit grid $2h$ at every iteration or (2) to not solve the grid $2h$ problem exactly.

While possible in some special cases, it is difficult to follow the first option in general. This is because there is often a small mismatch between the space of error reduced effectively by the

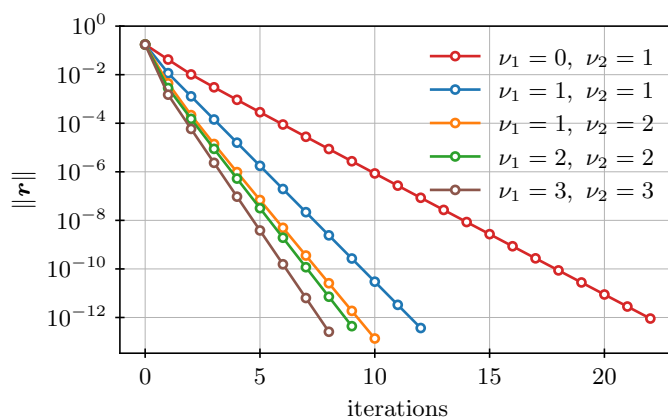


Figure 11.2.7. Residual history varying the number of prerelaxation (ν_1) and postrelaxation (ν_2) sweeps in a two-grid cycle applied to the finite-difference Laplacian in 1D with $n_x = 64$.

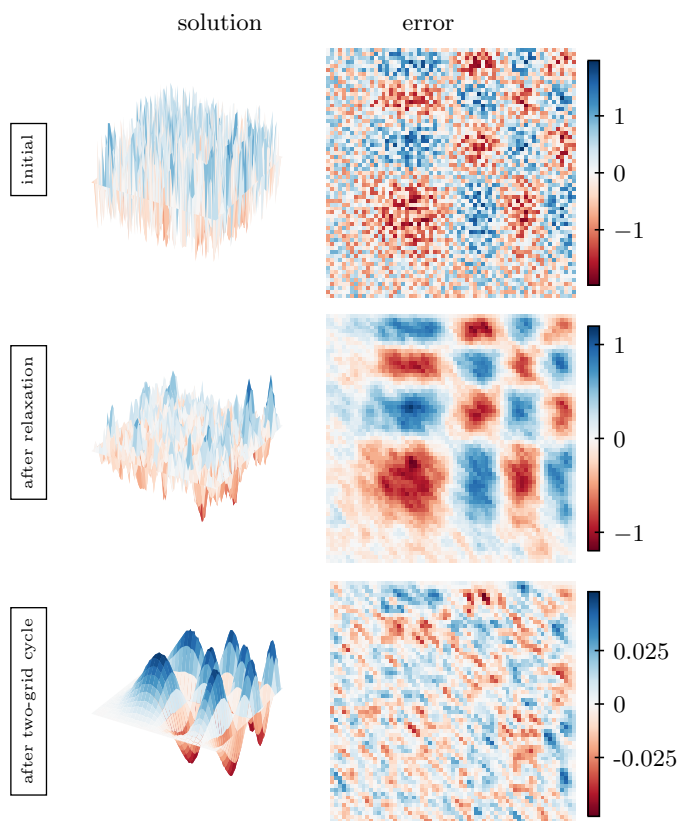


Figure 11.2.8. (left) Approximate solution and (right) error throughout a two-grid cycle applied to the finite-difference Laplacian in 2D on an $n_x \times n_y$ mesh with $n_x = n_y = 64$.

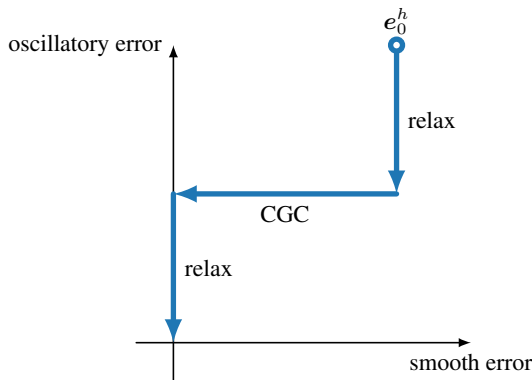


Figure 11.2.9. *Ideal two-grid error reduction when relaxation and coarse-grid correction (CGC) are perfectly orthogonal.*

fine-scale relaxation and the space of error reduced by the coarse-grid correction process. If these two spaces of error are orthogonal, then it would be possible to treat the error on the coarse grid once and only complement this with successive relaxation sweeps. In most settings, however, the space of errors reduced by coarse-grid correction is not perfectly orthogonal to those effectively damped by relaxation. In fact, the correction is in the range of interpolation, $\mathcal{R}(P)$, and then the only remaining error is in the nullspace of RA , $\mathcal{N}(RA)$. Thus, while coarse-grid correction in a two-grid process acts as an orthogonal projection in the A norm, subsequent relaxation steps reintroduce errors in these spaces that are only effectively treated by further coarse-grid correction. These two settings are illustrated in Figure 11.2.9, with the “ideal” (orthogonal) relation between spaces, and Figure 11.2.10, showing the more typical (nonorthogonal) iteration. Here, we assume that the relaxation process does not completely remove all of the oscillatory error in the first step, and we return to removing the rest after the coarse-grid correction(s).

The expected lack of orthogonality between the space of errors efficiently damped by relaxation and those eliminated by coarse-grid correction necessitates revisiting the coarse-grid problem at each step of an iteration process in order to remove errors in the range of interpolation that are reintroduced by relaxation. Therefore, we require an efficient way to (approximately) solve

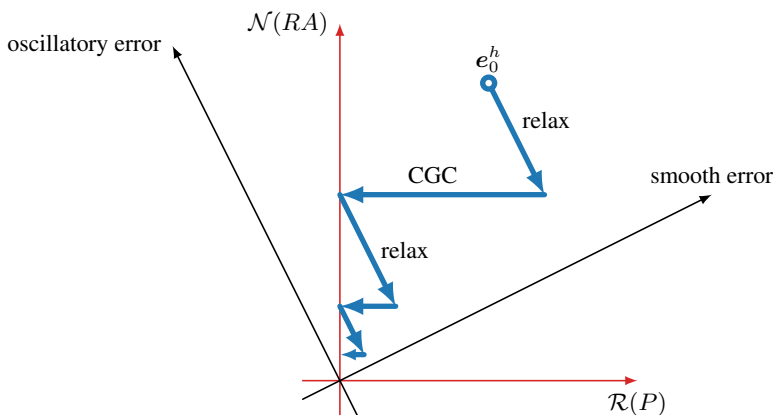


Figure 11.2.10. *Typical two-grid error-reduction when relaxation and coarse-grid correction (CGC) are not perfectly orthogonal.*

The key to achieving an efficient algorithm is appropriately balancing the amount of work that we do on each level in the hierarchy with the resulting error reduction on grid h . One variant of the multigrid idea, known as the algebraic multilevel iteration (or AMLI) [25, 26], uses a flexible Krylov solver on each level in order to solve each problem up to a prescribed tolerance before returning to the finer grid. While this offers a reasonable heuristic for a quickly converging algorithm, it is also more costly than is needed to effectively solve many problems. A more standard choice is to use a fixed cycling strategy, as shown in Algorithm 11.2, known as the multigrid μ -cycle. Note that Algorithm 11.2 suppresses the level in the notation, assuming that the algorithm is called on level ℓ of the hierarchy and that the hierarchical algorithm is started by being called on the finest grid, level 0. Typical choices in this algorithm are $\mu = 1$, known as the multigrid V-cycle, shown in Figure 11.2.11, and $\mu = 2$, known as the W-cycle, based on the shape of the iterations between levels when depicted as in Figure 11.2.12. To indicate

```

1 for  $k = 1$  to  $\nu_1$ 
2    $\mathbf{u} \leftarrow \mathbf{u} + M_1 (\mathbf{f} - A\mathbf{u})$  {prerelaxation}
3  $\mathbf{f}^{\ell+1} \leftarrow R(\mathbf{f} - A\mathbf{u})$  {restrict level- $\ell$  residual}
4 if  $\ell = \ell_{\max} - 1$  {on next-to-coarsest grid}
5    $\mathbf{u}^{\ell_{\max}} = (A^{\ell_{\max}})^{-1} \mathbf{f}^{\ell_{\max}}$  {exact solve on coarsest grid}
6 else
7    $\mathbf{u}^{\ell+1} \leftarrow \mathbf{0}$  {initialize coarse-level correction as zero}
8   for  $k = 1$  to  $\mu$ 
9      $\mathbf{u}^{\ell+1} \leftarrow \text{MU} (A^{\ell+1}, \mathbf{u}^{\ell+1}, \mathbf{f}^{\ell+1}, \dots, \ell + 1)$  {recursive call to a  $\mu$ -cycle}
10   $\mathbf{u} \leftarrow \mathbf{u} + P\mathbf{u}^{\ell+1}$  {coarse-grid correction}
11 for  $k = 1$  to  $\nu_2$ 
12    $\mathbf{u} \leftarrow \mathbf{u} + M_2 (\mathbf{f} - A\mathbf{u})$  {postrelaxation}

```

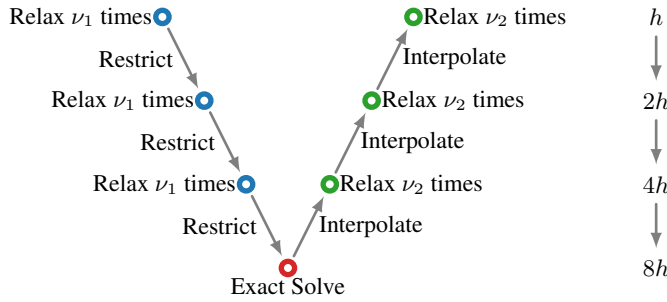


Figure 11.2.11. Multigrid V-cycle process, as in Algorithm 11.2 with $\mu = 1$.

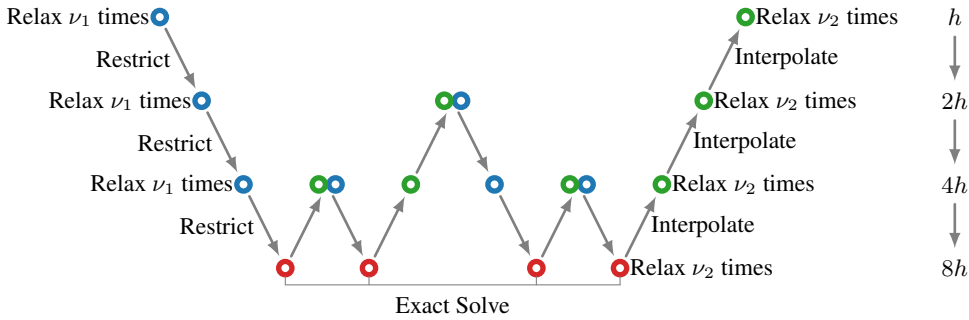


Figure 11.2.12. Multigrid W-cycle process, as in Algorithm 11.2 with $\mu = 2$.

the number of pre- and postrelaxation sweeps used in the cycle, a common notation is to write these as $V(\nu_1, \nu_2)$ - and $W(\nu_1, \nu_2)$ -cycles. While this cycling terminology is used for most variants of multigrid, if we are using geometric interpolation and restriction operators (typically with structured meshes), then this process is referred to as *geometric* multigrid (GMG).

If we revisit each level only a fixed number of times, we can easily express the cost of each cycle, assuming that the cost per level is proportional only to the number of degrees of freedom on that level. To do this, we opt to measure costs in terms of *work units* (WUs), where one WU is defined to be the cost of computing the residual, $\mathbf{f}^h - A^h \mathbf{u}^h$, on grid h . We assume this to be $\mathcal{O}(n)$, where n is the dimension of A^h . Considering iterations such as (weighted) Jacobi and Gauss–Seidel, we assume that each iteration of the relaxation scheme on grid h also costs one WU. For a problem in d dimensions with n_x grid points in each direction ($n = n_x^d$), problem size decreases by a factor of 2^d on each level. Thus, for the V-cycle (with $\mu = 1$), we estimate the cost of the cycle by accounting for the cost of relaxation as $(\nu_1 + \nu_2)(1 + 2^{-d} + 4^{-d} + \dots)$ WU $\approx (\nu_1 + \nu_2) \frac{2^d}{2^d - 1}$ WU. For $d = 1$, this gives $2(\nu_1 + \nu_2)$ WU, while it gives $\frac{4}{3}(\nu_1 + \nu_2)$ WU for $d = 2$ and $\frac{8}{7}(\nu_1 + \nu_2)$ WU for $d = 3$. Of course, there are more costs to the V-cycle than just relaxation, but they are typically smaller than those associated with relaxation, coming from a single residual evaluation on every level and the grid-transfer operations with matrices P and R on each level. For $\mu > 1$, each level in the hierarchy is visited multiple times. Using the indexed notation for levels, with the grid h fine level as $\ell = 0$, grid $2h$ as $\ell = 1$, etc., the relaxation cost of a μ -cycle becomes

$$(\nu_1 + \nu_2) (\mu^0 + \mu^1 2^{-d} + \mu^2 4^{-d} + \dots) \text{WU}. \quad (11.75)$$

For a W-cycle ($\mu = 2$) and $d = 1$, this leads to a total cost that is the same on each level, giving a cost per cycle of $(\nu_1 + \nu_2) \log_2 n_x$ WU (where again n_x is the size of the grid h problem). For

W-cycles in higher dimensions, though, we avoid the logarithmic term, with a relaxation cost of $2(\nu_1 + \nu_2)\text{WU}$ for $d = 2$ and $\frac{4}{3}(\nu_1 + \nu_2)\text{WU}$ for $d = 3$. Clearly, all of these costs are now either $\mathcal{O}(n)$ or $\mathcal{O}(n \log_2 n)$, which provides optimal cost per cycle. However, now that we have modified the two-grid cycle, we need to confirm that we have not lost the effectiveness with the transition from the two-grid to the multigrid algorithm.

Remark 11.10: Unwrapped cycles and F-cycles

When considering the V-cycle ($\mu = 1$), it is often more natural to consider an “unwrapped” form of the algorithm, without the recursive call. To do this, we note that each level of the “downward” and “upward” sweeps of the V-cycle consists of the same operations on each level of the hierarchy. Unwrapping the recursive call leads to Algorithm 11.3, which is conceptually simpler than Algorithm 11.2, although equivalent when $\mu = 1$. A similar approach yields the F-cycle algorithm, which arises from restricting the fine-grid residual all the way to the coarsest grid and then cycling to progressively finer grids, performing a V-cycle on each level. This cycle is akin to grid continuation (also known as full multigrid or nested iteration), but it always starts from the finest-grid problem. The F-cycle is presented in Algorithm 11.4.

Algorithm 11.3: Unwrapped multigrid V-cycle

Input : $A^0, \dots, A^{\ell_{\max}}$, level- ℓ operators
 $\mathbf{u}^0, \mathbf{f}^0$, initial approximation, fine-level right-hand side
 $R^0, \dots, R^{\ell_{\max}-1}, P^0, \dots, P^{\ell_{\max}-1}$, level- ℓ restriction and interpolation matrices
 $M_{1,2}^0, \dots, M_{1,2}^{\ell_{\max}-1}$, level- ℓ pre-, postrelaxation matrices
 $\nu_{1,2}$, number of pre-, postrelaxation sweeps
Output : $\mathbf{u}^0$, finest-grid approximation after one V-cycle

```

1 for  $\ell = 0, 1, \dots, \ell_{\max} - 1$ 
2   for  $k = 1$  to  $\nu_1$ 
3      $\mathbf{u}^\ell \leftarrow \mathbf{u}^\ell + M_1^\ell (\mathbf{f}^\ell - A^\ell \mathbf{u}^\ell)$                                      {prerelaxation}
4      $\mathbf{f}^{\ell+1} \leftarrow R^\ell (\mathbf{f}^\ell - A^\ell \mathbf{u}^\ell)$                                      {restrict level- $\ell$  residual}
5      $\mathbf{u}^{\ell+1} \leftarrow \mathbf{0}$                                                          {initialize coarse-level correction as zero}
6 Solve  $A^{\ell_{\max}} \mathbf{u}^{\ell_{\max}} = \mathbf{f}^{\ell_{\max}}$                                          {exact solve on coarsest grid}
7 for  $\ell = \ell_{\max} - 1, \ell_{\max} - 2, \dots, 0$ 
8    $\mathbf{u}^\ell \leftarrow \mathbf{u}^\ell + P^\ell \mathbf{u}^{\ell+1}$                                          {coarse-grid correction}
9   for  $k = 1$  to  $\nu_2$ 
10     $\mathbf{u}^\ell \leftarrow \mathbf{u}^\ell + M_2^\ell (\mathbf{f}^\ell - A^\ell \mathbf{u}^\ell)$                              {postrelaxation}

```

Example: the V(1,1)-cycle in 2D

We now return to the 2D domain decomposition examples given in Section 11.1, using the partition-of-unity scaling of Example 11.8 as a baseline. Using a multigrid preconditioner (for FGMRES), we construct regular $n_x \times n_y$ grids with $n_x = n_y$ taken to be a power of two. With Dirichlet conditions on the boundaries, the resulting interior matrix problem is of size $(n_x - 1) \times (n_y - 1)$, with matrices corresponding to the 2D Laplacian matrix discretized by finite differences given in equation (3.18). When the problem is coarsened by a factor of two in each direction, the coarse-level problem is of size $(n_x/2 - 1) \times (n_y/2 - 1)$. Bilinear interpolation is used between grids, and a Galerkin coarse-grid operator is constructed. In this example, weighted Jacobi is used as relaxation with $\omega = 4/5$ and one pre- and one postrelaxation sweep at each level. The V(1,1)-cycle coarsens to a single grid point.

Algorithm 11.4: Multigrid F-cycle

Input : $A^0, \dots, A^{\ell_{\max}}$, level- ℓ operators
 $\mathbf{u}^0, \mathbf{f}^0$, initial approximation, fine-level right-hand side
 $R^0, \dots, R^{\ell_{\max}-1}, P^0, \dots, P^{\ell_{\max}-1}$, level- ℓ restriction and interpolation matrices
 $M_{1,2}^0, \dots, M_{1,2}^{\ell_{\max}-1}$, level- ℓ pre-, postrelaxation matrices
 $\nu_{1,2}$, number of pre-, postrelaxation sweeps
Output : $\mathbf{u}^0$, finest-grid approximation after one F-cycle

```

1 for  $\ell = 0, 1, \dots, \ell_{\max} - 1$                                      {coarsen to coarsest grid}
2    $\mathbf{f}^{\ell+1} \leftarrow R^\ell (\mathbf{f}^\ell - A^\ell \mathbf{u}^\ell)$                      {restrict level- $\ell$  residual}
3    $\mathbf{u}^{\ell+1} \leftarrow \mathbf{0}$                                              {initialize coarse-level correction as zero}
4   Solve  $A^{\ell_{\max}} \mathbf{u}^{\ell_{\max}} = \mathbf{f}^{\ell_{\max}}$                          {exact solve on coarsest grid}
5   for  $m = \ell_{\max} - 1, \ell_{\max} - 2, \dots, 0$                        {start at the coarsest grid}
6      $\mathbf{u}^m \leftarrow \mathbf{u}^m + P^m \mathbf{u}^{m+1}$                              {coarse-grid correction}
7     for  $\ell = m, m + 1, \dots, \ell_{\max} - 1$                          {level- $m$  V-cycle}
8       for  $k = 1$  to  $\nu_1$ 
9          $\mathbf{u}^\ell \leftarrow \mathbf{u}^\ell + M_1^\ell (\mathbf{f}^\ell - A^\ell \mathbf{u}^\ell)$            {prerelaxation}
10         $\mathbf{f}^{\ell+1} \leftarrow R^\ell (\mathbf{f}^\ell - A^\ell \mathbf{u}^\ell)$                  {restrict level- $\ell$  residual}
11         $\mathbf{u}^{\ell+1} \leftarrow \mathbf{0}$                                        {initialize coarse-level correction as zero}
12      Solve  $A^{\ell_{\max}} \mathbf{u}^{\ell_{\max}} = \mathbf{f}^{\ell_{\max}}$                      {exact solve on coarsest grid}
13      for  $\ell = \ell_{\max} - 1, \ell_{\max} - 2, \dots, m$ 
14         $\mathbf{u}^\ell \leftarrow \mathbf{u}^\ell + P^\ell \mathbf{u}^{\ell+1}$                      {coarse-grid correction}
15        for  $k = 1$  to  $\nu_2$ 
16           $\mathbf{u}^\ell \leftarrow \mathbf{u}^\ell + M_2^\ell (\mathbf{f}^\ell - A^\ell \mathbf{u}^\ell)$            {postrelaxation}

```

Figure 11.2.13 compares multigrid convergence with that of domain decomposition from the previous section. We observe a significant reduction in the number of iterations to convergence using multigrid. In addition, as the grid size increases, the number of multigrid iterations remains constant; this highlights the scalability of multigrid as a preconditioner. The example also includes standalone multigrid (as compared with multigrid preconditioned FGMRES) at the right of Figure 11.2.13. While performance degrades without the use of FGMRES, the difference is small and multigrid as a solver remains effective as the grid size is increased.

Finally, the performance when using a W- or F-cycle applied to the same model problem yields nearly identical results. The power of these approaches is seen for more complex applications and PDEs. The differences between the various cycling approaches is explored a bit more in the next section on algebraic multigrid (AMG) methods, Section 11.3.

11.2.4 ■ Analysis of multigrid methods

We close this section by giving a heuristic picture of how a multigrid cycle can be effective. Just as we consider a hierarchy of models in a multigrid μ -cycle, we can also consider a hierarchical decomposition of the spectrum of a fine-grid matrix A^h coming from a one-dimensional model problem into its upper half, next quarter, then next eighth, and so on, as pictured in Figure 11.2.14. As we saw above, we choose the parameter in a weighted Jacobi iteration so that relaxation on the grid h problem effectively damps errors in the upper half of its spectrum. Similarly, relaxation on the grid $2h$ problem effectively damps errors in the upper half of the spectrum

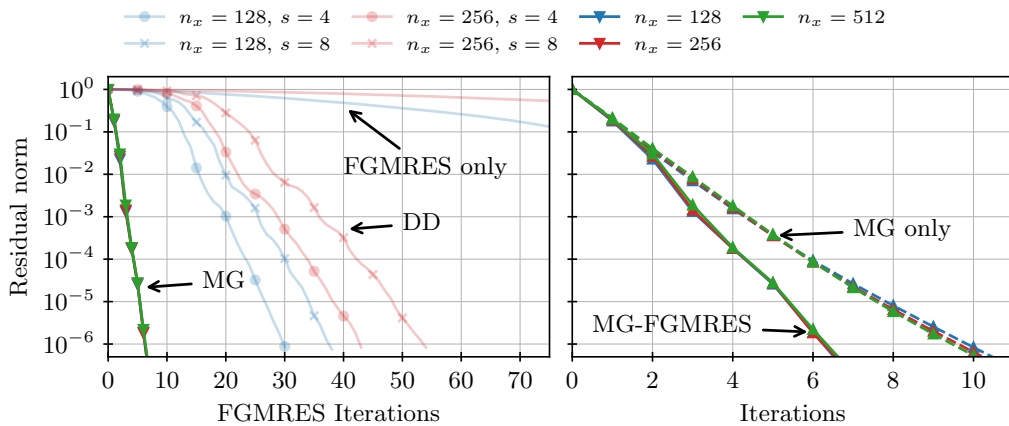


Figure 11.2.13. (left) Relative residual reduction by FGMRES preconditioned by a single $V(1, 1)$ cycle at each iteration applied to the finite-difference Laplacian on $n_x \times n_x$ meshes. Here, faded lines show the convergence of unpreconditioned FGMRES and additive Schwarz with minimal overlap, partition-of-unity scaling, and $s \times s$ subdomains as shown in Figure 11.1.7. (right) Relative residual reduction by FGMRES preconditioned by a single multigrid $V(1, 1)$ -cycle, as in the left panel, and by stationary multigrid $V(1, 1)$ -cycles without FGMRES acceleration.



Figure 11.2.14. Hierarchical decomposition of the spectrum of a 1D problem into segments to be handled on each grid of a multigrid process.

of A^{2h} , which is close to the next quarter of the grid h spectrum. This pattern continues: relaxation on the grid $4h$ problem effectively damps errors in the upper half of the spectrum of A^{4h} , which is close to the next eighth of the spectrum of A^h , and so on. Next, we aim to provide an outline of how this heuristic is formalized into a predictive theory for multigrid methods.

In general, multigrid convergence is controlled by two factors. First is the complementarity between relaxation and an idealized coarse-grid correction from grid $2h$. In essence, here we ask whether there is a good correction to the errors that relaxation damps inefficiently. Second is the effectiveness of the recursive strategy to solve the grid $2h$ problem. Here, rather than asking about the existence of a good correction, we ask if we can find a good correction efficiently. Recognize that, in order to be successful, we need both of these properties. In particular, good two-grid convergence is usually necessary in order to achieve good multigrid convergence. Once we know that we have good two-grid convergence, the details of the effectiveness of relaxation and other multigrid components for the grid $2h$ problem determine if V- or W-cycles are effective.

In general, there exist two families of approaches to multigrid convergence theory. One approach uses the variational setting of multigrid methods for finite-element discretizations, providing rigorous convergence bounds in this setting; however, the theoretical approach does not readily generalize to other discretizations, nor does it often provide so-called *predictive* bounds of multigrid convergence. That is, convergence theory based on variational properties often only provides bounds with unknown constants; while such bounds prove h -independent convergence properties, they offer no insight into the optimal choice of algorithmic components or parameters because they do not aim to offer accurate predictions of the spectral properties of the multigrid error-propagation operator. The second common family of approaches consists of those based

on generalizations of Toeplitz and circulant matrix classes. These approaches use similarity transformations that allow direct calculation of low-dimensional invariant subspaces of the error-propagation operator and, consequently, accurate prediction of convergence rates for stationary (or other) iterations. However, such *rigorous mode analysis* is only possible for a small class of operators for which diagonalization is possible via Fourier (and other) ansatzes. Rigorous mode analysis, though, is easily generalized into *local* mode analysis, which forms the basis for much of multigrid theory, applying techniques based on spectral transformations that are only valid for matrices from certain classes in more general settings. Detailed analysis of how and when local mode analysis is rigorously valid either for modifications of standard algorithms or in the limit as $h \rightarrow 0$ establishes the basis for how it is used as the major predictive analysis tool for multigrid methods.

To complement the spectral analysis of relaxation given above, we now consider rigorous mode analysis of the interpolation and restriction operators for one-dimensional uniform meshes. Recall that the eigenvectors of both the grid $h = 1/n_x$ finite-difference discretization of the Poisson operator and the weighted Jacobi iteration applied to that matrix are given by

$$(\mathbf{v}_i)_j = \sin\left(\frac{i\pi j}{n_x}\right) \text{ for } 1 \leq j \leq n_x - 1, \quad (11.76)$$

with eigenvalues

$$\lambda_i(A^h) = \frac{4}{h^2} \sin^2\left(\frac{i\pi}{2n_x}\right), \quad (11.77a)$$

$$\text{and } \lambda_i(I - \omega D^{-1}A^h) = 1 - 2\omega \sin^2\left(\frac{i\pi}{2n_x}\right) \quad (11.77b)$$

for $1 \leq i \leq n_x - 1$. This is easily extended to the coarse-grid operator on grid $2h = 2/n_x$, with eigenvectors

$$(\hat{\mathbf{v}}_i)_j = \sin\left(\frac{i\pi j}{n_x/2}\right) \text{ for } 1 \leq j \leq n_x/2 - 1 \quad (11.78)$$

and eigenvalues

$$\lambda_i(A^{2h}) = \frac{4}{(2h)^2} \sin^2\left(\frac{i\pi}{n_x}\right) \quad (11.79)$$

for $1 \leq i \leq n_x/2 - 1$.

Now consider restriction of a fine-grid eigenvector to coarse-grid node j :

$$(R\mathbf{v}_i)_j = \frac{1}{2}(\mathbf{v}_i)_{2j} + \frac{1}{4}\left((\mathbf{v}_i)_{2j+1} + (\mathbf{v}_i)_{2j-1}\right) \quad (11.80a)$$

$$= \frac{1}{2} \sin\left(\frac{2i\pi j}{n_x}\right) + \frac{1}{4} \left(\sin\left(\frac{i\pi(2j+1)}{n_x}\right) + \sin\left(\frac{i\pi(2j-1)}{n_x}\right) \right) \quad (11.80b)$$

$$= \frac{1}{2} \sin\left(\frac{2i\pi j}{n_x}\right) \left(1 + \cos\left(\frac{i\pi}{n_x}\right)\right). \quad (11.80c)$$

Thus, if $1 \leq i < n_x/2$, then $R\mathbf{v}_i = \mu_i(R)\hat{\mathbf{v}}_i$, with $\mu_i(R) = \frac{1}{2}\left(1 + \cos\left(\frac{i\pi}{n_x}\right)\right)$, and restriction is seen to map smooth fine-grid modes directly onto their coarse-grid analogues. For $n_x/2 < i \leq n_x - 1$, we write $i = n_x - \hat{i}$ for $1 \leq \hat{i} < n_x/2$ and write

$$\sin\left(\frac{2(n_x - \hat{i})\pi j}{n_x}\right) = \sin\left(2\pi j - \frac{2\hat{i}\pi j}{n_x}\right) = -\sin\left(\frac{2\hat{i}\pi j}{n_x}\right). \quad (11.81)$$

This gives $R\mathbf{v}_i = \mu_i(R)\hat{\mathbf{v}}_i$ for $\mu_i(R) = -\frac{1}{2}\left(1 + \cos\left(\frac{i\pi}{n_x}\right)\right)$, showing that the high-frequency modes on grid h are aliased with low-frequency modes on the coarse grid, with both modes i and $n_x - i$ on grid h being mapped onto $\hat{\mathbf{v}}_i$. Considering the restriction of a linear combination of these two modes, $R(c_i\mathbf{v}_i + c_{n_x-i}\mathbf{v}_{n_x-i})$, $\cos\left(\frac{i\pi}{n_x}\right)$ is positive for $1 \leq i < n_x/2$ and negative for $n_x/2 < i \leq n_x - 1$, so the smooth mode dominates the restriction operation. Finally, for $i = n_x/2$, we have that $\sin\left(\frac{2(n_x/2)\pi j}{n_x}\right) = \sin(\pi j) = 0$ and, consequently, this mode is in the nullspace of restriction and is not corrected by coarse-grid correction.

A similar calculation shows that the interpolation of an eigenvector from the coarse grid is mapped into a linear combination of the same mode and its aliased high frequency on the fine grid,

$$P\hat{\mathbf{v}}_i = c_i\mathbf{v}_i + s_i\mathbf{v}_{n_x-i}, \quad (11.82)$$

with $c_i > s_i$ for $1 \leq i < n_x/2$. This shows that the coarse-grid correction process returns a correction to the fine grid that is dominated by smooth modes but also includes some oscillatory components. With these two analyses, we conclude that the entire two-grid cycle can be analyzed in terms of two-dimensional invariant subspaces, $\text{span}\{\mathbf{v}_i, \mathbf{v}_{n_x-i}\}$ for $1 \leq i < n_x/2$, along with the one-dimensional space given by $\text{span}\{\mathbf{v}_{n_x/2}\}$. Thus, we consider an arbitrary linear combination of the two fine-grid *harmonic* modes, $c_i\mathbf{v}_i + c_{n_x-i}\mathbf{v}_{n_x-i}$, and analyze the actions of restriction and coarse-grid correction on an invariant subspace. This leads to a 2×2 block representation of the evolution of the coefficients c_i, c_{n_x-i} from before a two-grid cycle to after the cycle. The eigenvalues of these $n_x/2 - 1$ matrices (each of which is rank one) determine the convergence properties of the two-grid method. This can be extended to analyze a three-grid method, with invariant subspaces of size four, and so on. Note that rigorous analysis of successively deeper multigrid hierarchies requires analysis of larger and larger invariant subspaces. Thus, a compromise is often achieved where, for example, two-grid and three-grid analyses are considered and compared. If the prediction of a three-grid analysis is sufficiently close to that of the two-grid analysis, we infer that the multigrid cycle is likely to converge at the same rate.

It is in this setting, for example, that it is rigorously shown that a stationary iteration of a two-grid cycle for the 1D Poisson problem with one prerelaxation sweep and one postrelaxation sweep of weighted Jacobi with weight $\omega = 2/3$ yields an error-propagation operator whose spectral radius is around 0.1, meaning that the norm of the error in an approximation is reduced by almost an order of magnitude for each two-grid cycle. This sort of analysis has been extended to many more interesting and more complicated problems, and it is often used to identify near-optimal relaxation parameters for much more complicated relaxation schemes. While this approach is rigorous only for simple relaxation schemes and simple problems, the success of local Fourier analysis for a much wider class of problems is a major reason why it is often used as a multigrid analysis tool, even when it is not rigorously justified. More details on such local Fourier analysis are found in [219, 234], for example.

11.3 ■ Algebraic multigrid

Objectives

For many problems, *geometric* multigrid methods, as presented in Section 11.2, rapidly converge and are highly efficient. However, as we introduce complications into the problems we aim to solve, such as variable-diffusion coefficients and anisotropic or unstructured meshes, alternative formulations of multigrid are required in order to maintain scalable convergence. In this section, you will consider so-called *algebraic* multigrid (AMG) methods—methods that construct coarse problems and intergrid transfer operators automatically, albeit still following

the heuristics that were developed for the isotropic setting. Specifically, you will

- discuss the limitations of standard geometric multigrid and introduce the concepts of semicoarsening and line relaxation;
- be introduced to the notion of *algebraically smooth error* and understand the basic setup of an AMG method; and
- analyze and give examples of two general formulations of AMG: smoothed aggregation and CF-AMG.

We start this section by considering a case where the basic form of multigrid, as presented in Section 11.2, fails to yield satisfactory convergence. To see this, we consider a rotated anisotropic diffusion problem of the form

$$-\nabla \cdot \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & \varepsilon \end{bmatrix} \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix}^T \nabla u = f(x), \quad (11.83)$$

where $0 \leq \phi \leq \frac{\pi}{2}$ and ε is varied over several orders of magnitude. Such problems pose difficult challenges for linear solvers because of the character of the diffusion tensor given by

$$K = \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & \varepsilon \end{bmatrix} \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix}^T. \quad (11.84)$$

Recognize that the two matrices on the outside are orthogonal, so the transpose on the right is equivalent to an inverse of the same matrix; thus, we are given the diffusion tensor, K , in terms of its eigenvalue decomposition. When ε is far from one, we see very different diffusion character between the eigenvector associated with the unit eigenvalue, $\begin{bmatrix} \cos(\phi) \\ \sin(\phi) \end{bmatrix}$, and that associated with eigenvalue ε , $\begin{bmatrix} -\sin(\phi) \\ \cos(\phi) \end{bmatrix}$. This is what causes the PDE in equation (11.83) to be so challenging—the solution, $u(x, y)$, has dramatically different behavior when we vary (x, y) along the two directions given by these eigenvectors, and an effective linear solver for discretizations of equation (11.83) must be able to account for these differences. To study the effectiveness of multigrid as a solver for these problems, we consider the simplest case, with $f(x) = 0$, zero Dirichlet boundary conditions, and a *random* initial guess, u_0 (which is equal to the error, e_0 , in this setting).

Figure 11.3.1 shows the convergence of two-level multigrid using weighted Jacobi (with $\omega = \frac{4}{5}$) on a 32×32 grid discretized with linear finite elements. We see that convergence is satisfactory for small deviations of ε from 1.0. However, as the anisotropy increases ($\varepsilon \rightarrow 0$ or $\varepsilon \rightarrow \infty$), the convergence quickly deteriorates for any value of ϕ . Representative errors after 10 sweeps of relaxation are shown for $\varepsilon = 0.01$ for the case of $\phi = \frac{\pi}{4}$ in Figure 11.3.2a and for the case of $\phi = 0$ in Figure 11.3.2b. Here, we see that smoothness of the error after relaxation is retained in the direction of “strong connection” (aligned with angle ϕ , measured counterclockwise from the positive x -axis, corresponding to eigenvector $\begin{bmatrix} \cos(\phi) \\ \sin(\phi) \end{bmatrix}$), while there is a lack of smoothness in the orthogonal direction.

Semicoarsening and line relaxation

Noticing that relaxation does effectively smooth the error at least in one direction suggests that we might be able to regain good multigrid convergence by introducing coarsening in a complementary way. Indeed, if we consider the problem with a *grid-aligned* anisotropy, Figure 11.3.2

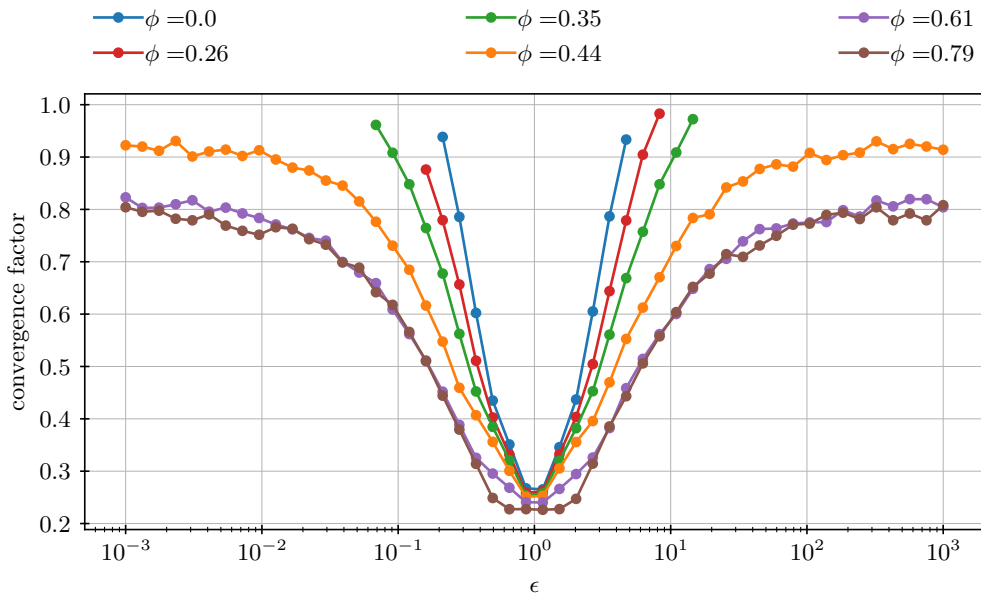


Figure 11.3.1. Geometric two-grid convergence applied to the anisotropic diffusion problem, equation (11.83), on a 32×32 mesh discretized with linear finite elements. The observed convergence factors at different angles ϕ and strength ε are shown. Only convergent tests are shown (i.e., iterations with convergence factors less than 1.0).

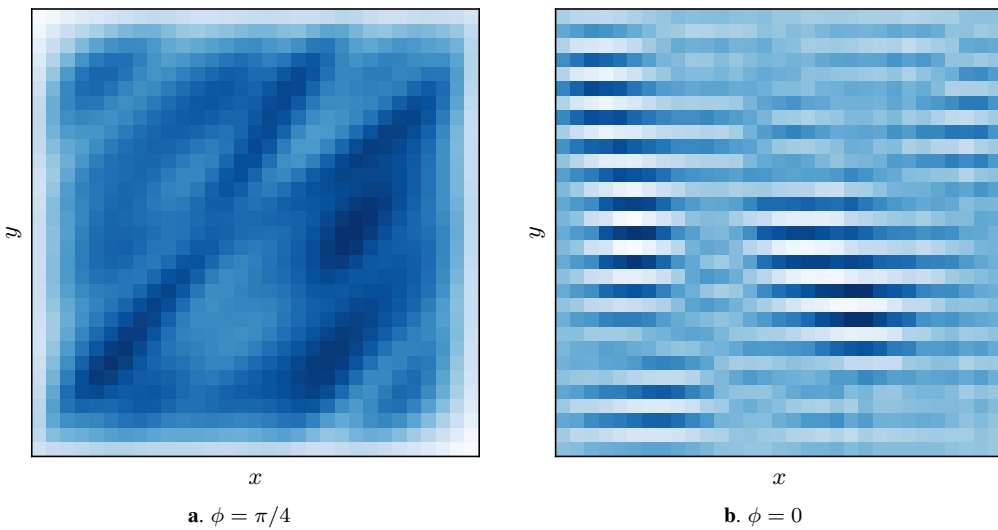


Figure 11.3.2. Error after 10 relaxation sweeps of weighted Jacobi ($\omega = \frac{4}{5}$) applied to the anisotropic diffusion problem, equation (11.83), on a 32×32 mesh discretized with linear finite elements with $\varepsilon = 0.01$.

shows that the error after relaxation is *only* smooth in one direction—i.e., the error after relaxation varies widely from one grid point to the next in the y -direction but varies slowly in the x -direction.

For this problem, *semicoarsening multigrid methods* are effective approaches. Semicoarsening refers to using regular coarsening on a geometrically structured mesh, but not in all dimen-

sions of the problem. Here, for example, we would coarsen only in the x -direction, which both addresses the strong connections in the matrix (those aligned with the eigenvector associated with the larger eigenvalue in equation (11.84)) and leads to less anisotropy on coarse-grid problems, since the anisotropy introduced on the coarse meshes by this coarsening counteracts that of the diffusion. When we coarsen in only one direction, natural interpolation and restriction operators arise as one-dimensional grid transfers along each line of points in the y -direction. This semi-coarsening strategy can be repeated until either the coarsest-grid problem is small enough to be solved directly or a (relatively) isotropic problem is reached, where regular multigrid principles can be applied directly.

Alternatively, we could instead use *line relaxation*, building the relaxation operator to act in the direction of the anisotropy. In the above example, this would be equivalent to using a *block* Jacobi method, where each block is a *line* of points in the x -direction. Then, a full coarsening scheme can be used, since the error after line relaxation is seen to be smooth in both directions.

11.3.1 ■ Basics of algebraic multigrid

The techniques of semicoarsening, line relaxation, and improved interpolation lead to significant improvements in geometric multigrid performance for a range of problems. Ultimately, however, we need a way to coarsen problems that are fully unstructured in nature, or problems for which the combinations above are ineffective. To this end, we now consider fully *algebraic* approaches: methods that use only the algebraic properties of the linear system $Au = f$, while requiring very little, if any, geometric information, such as grid coordinates or domain structure. One fundamental question in this setting is that of smoothness: can we generalize the concept of smoothness to a wider class of problems? In addition, for linear systems that are derived from PDEs on fully unstructured meshes, we require a new view of the coarse-grid problem that is meaningful.

Recalling the steps of a two-level V(1,0)-cycle from Section 11.2 for an initial error of e_0 , we have

$$e_1 \leftarrow \underbrace{(I - P(P^T A P)^{-1} P^T A)}_{\text{coarse-grid correction}} \underbrace{G}_{\text{relaxation}} e_0, \quad (11.85)$$

where G represents the error-propagation operator for the chosen relaxation method—e.g., $G = I - \omega D^{-1}A$ in the case of weighted Jacobi. Equation (11.85) directly highlights the important interplay between coarse-grid correction and the error remaining after relaxation.

If we consider the error after relaxation, Ge_0 , one tempting approach is to construct the interpolation operator, P , so that its range fully captures this error,

$$Ge_0 \in \mathcal{R}(P), \quad (11.86)$$

where we use $\mathcal{R}(P)$ to denote the range of the matrix P . Of course, doing this for any error e_0 is intractable in practice, but the result is motivating: if $Ge_0 = Pv$ for some v , then $e_1 \equiv 0$. So, if the dominant errors after relaxation are close to being in the range of interpolation, we have a nice complementarity between the relaxation and coarse-grid correction processes. To achieve this, the question then becomes: how do we characterize the dominant error remaining after relaxation?

Returning to weighted Jacobi and the 1D Poisson problem on a uniform mesh discretized with finite differences (see equation (11.55)), we characterize the error remaining after relaxation by the lowest (smoothest) Fourier modes (see Figure 11.2.1), which also satisfy $Ae \approx 0$. In general, for a given error e such that $\|e\| = \mathcal{O}(1)$, if the relaxation method is of the form $G = I - MA$, then we assume that we have run enough relaxation so that $Ge \approx e$ or that there is a sufficiently small $\delta > 0$ such that

$$\|Ge - e\| = \|(I - MA)e - e\| < \delta, \quad (11.87)$$

where we use $\|\cdot\|$ to denote the Euclidean norm of a vector, in contrast to the A norm, $\|\cdot\|_A$, also used in this section. This defines *algebraically smooth* error (see Definition 11.11). Then,

$$\|MAe\| < \delta \Rightarrow \|r\| < c\delta, \quad (11.88)$$

with the assumption that $c = 1/\sigma_{\min}(M) = \mathcal{O}(1)$. With this, we again see a connection between errors that yield small residuals and those that are slow to be reduced by relaxation. In addition, again assuming that $\|e\| = \mathcal{O}(1)$, we have

$$\|e\|_A^2 = \langle Ae, e \rangle \leq \|r\| \|e\| \ll 1. \quad (11.89)$$

Here, we see that an error remaining after relaxation should be smaller in the A norm (or the so-called *energy* norm) than its Euclidean norm.

Definition 11.11: Algebraically smooth error

Given a relaxation method with error-propagation operator G , the error, e , such that $\|e\| = \mathcal{O}(1)$ is considered algebraically smooth if

$$\|Ge - e\| < \delta \quad (11.90)$$

for some sufficiently small $\delta > 0$. That is, the relaxation method converges slowly for this type of error.

For model problems that are elliptic in nature, it can be shown that if e is algebraically smooth under weighted Jacobi relaxation, then entries of the error, e_i and e_j , must be close (with $e_i \approx e_j$) if $a_{i,j}/a_{i,i}$ is large; this is defined to be a *strong* connection in the graph of the matrix, a concept that we discuss in more detail below. Additionally, for structured problems such as those discussed in Section 11.2, the geometrically smooth error associated with low Fourier modes is also algebraically smooth, as defined in Definition 11.11, thus explaining why relaxation converges slowly in that setting.

There is an important connection here between the algebraically smooth error and strong connections. With this (or a similar) definition of the set of strong connections, we expect to be able to effectively approximate algebraically smooth errors at a point in the grid by their values at strongly connected degrees of freedom (other grid points, which we also refer to as nodes). Thus, sets of strong connections play an important role in the algebraic multigrid (AMG) coarsening process, as outlined in Algorithm 11.5. This algorithm describes a process of generating coarse

Algorithm 11.5: General AMG setup (* denotes additional parameters)

Input : $A \equiv A^0$

Output : $P^0, P^1, \dots, P^{\ell_{\max}-1}$, interpolation operators

$A^1, A^2, \dots A^{\ell_{\max}}$, coarse-grid operators

```

1 for  $\ell = 0, \dots$ 
2    $S \leftarrow \text{STRENGTH\_OF\_CONNECTION}(A^\ell, *)$            {determine strength of connection between nodes}
3    $C \leftarrow \text{COARSEN}(S, *)$                              {map points to set of coarse-grid points}
4    $P^\ell \leftarrow \text{INTERPOLATION}(A^\ell, S, C, *)$            {construct interpolation}
5    $A^{\ell+1} \leftarrow (P^\ell)^T A^\ell P^\ell$                    {form coarse-grid operator}
6   if  $\text{size}(A^{\ell+1}) < \text{minimum size}$ 
7     return

```

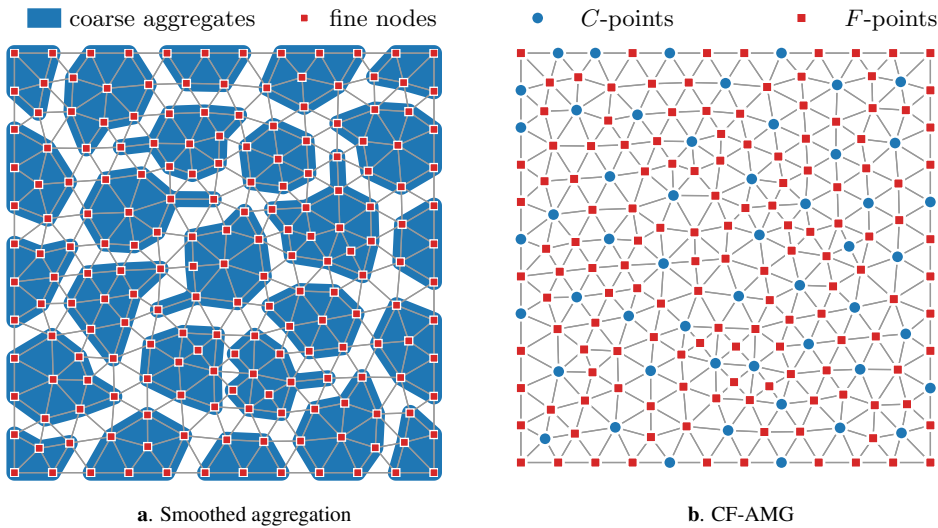



Figure 11.3.3. Coarse grids in AMG applied to a model 2D Poisson problem discretized with linear finite elements on an unstructured mesh and Neumann boundary conditions.

grids (not necessarily in the geometric sense) and the corresponding interpolation operators and is generally known as the *AMG setup*. On each grid, we measure strength of connection between nodes to produce an adjacency matrix, S , describing the strong connections, which we use to choose a *coarse-grid set*, C . From S , C , and A , we define an interpolation operator and a coarse-grid operator using the Galerkin condition. Then, we recurse to a coarser level and continue the coarsening process. Finally, with P^ℓ and A^ℓ defined for each level, ℓ , a V-cycle is executed in the same way as in geometric multigrid. In the next subsections, we expand on the components of Algorithm 11.5 and define specific AMG methods.

In this book, we consider two main approaches to AMG: smoothed aggregation AMG and CF-AMG. There are, of course, many other methods, as well as a range of variants on each of these main algorithms, but we focus on the two presented here as they represent the two most commonly used families of AMG algorithms. For more details on the different types of AMG approaches and their analysis, see [209, 235, 227] and the references therein. In smoothed aggregation, degrees of freedom for the coarse problem are defined by grouping (aggregating) the fine nodes. These aggregates define a mapping that serves as an initial interpolation operator that is later *smoothed*. In contrast, in CF-AMG, the fine nodes are split into either *C*-points (coarse points) or *F*-points (non-coarse, or fine-grid-only points). This splitting and the strength of connection between points are used to construct the interpolation. CF-AMG is commonly referred to as Ruge–Stüben–AMG (after their extensive work developing the methodology; see [192]) and also *classical* AMG. Figure 11.3.3 shows coarse grids for these two methods applied to a model 2D Poisson problem discretized with linear finite elements on an unstructured mesh and Neumann boundary conditions. This provides our first hint at the differences between the two AMG approaches.

Remark 11.12: Operator-based interpolation

There are, in essence, two important ideas in AMG. First, we have the idea that the Galerkin product gives us a reliable way to define a meaningful coarse-grid operator as long as we know how to interpolate to effectively correct algebraically smooth error. This frees us from needing to maintain structure in the coarse grids, as we will see in the following sections. Second, we

have the idea that, from any coarse grid, we can (and should) base our interpolation operators on the fine-grid matrix. This idea of *operator-based interpolation* predates the development of AMG and was first developed for structured grids in the setting of “black-box multigrid” (BoxMG) algorithms by Dendy and others [8, 85]. While BoxMG can be successfully applied to a smaller range of problems than AMG, it is able to leverage the structured-grid setting to achieve much better efficiency, through direct indexing into two-dimensional data structures, than is possible to achieve with AMG and typical sparse-matrix storage schemes.

In the remainder of this section, we highlight the key components in the basic form of these algorithms and identify several of the associated parameters that are used to fine-tune the convergence and computational complexity of the methods.

11.3.2 ■ Smoothed aggregation

To help guide the construction of a smoothed-aggregation AMG solver, we consider three model problems:

$$-u_{xx} = f(x) \quad \text{on } [0, 1] \quad (\text{Model 1D}),$$

$$-u_{xx} - u_{yy} = f(x, y) \quad \text{on } [0, 1]^2 \quad (\text{Model 2D}),$$

$$-\nabla \cdot Q\Lambda Q^T \nabla u = f(x, y) \quad \text{on } [0, 1]^2 \quad (\text{Anisotropic Model 2D}),$$

where Model 1D is discretized on a uniform mesh with spacing h and linear finite elements and Dirichlet boundary conditions, Model 2D is discretized on the unstructured mesh in Figure 11.3.3a with linear finite elements and Neumann boundary conditions, and Anisotropic Model 2D is discretized on a structured quadrilateral mesh with bilinear finite elements and Dirichlet boundary conditions. In the case of Anisotropic Model 2D, the diffusion tensor, $Q\Lambda Q^T$, is defined by setting a rotation angle, ϕ , and diffusion coefficient, ε , and taking

$$Q = \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix} \quad \text{and} \quad \Lambda = \begin{bmatrix} 1 & 0 \\ 0 & \varepsilon \end{bmatrix}. \quad (11.91)$$

For the numerical results below, we take $f(x) = 0$ and start with a random initial guess for u . Many of the ideas and definitions we present here for what follows on aggregation-based AMG are found in the paper by Vaněk, Mandel, and Brezina [223].

Strength of connection

To begin, we seek a measure of the so-called *strength* of each edge in the graph associated with matrix A to be used in line 2 of Algorithm 11.5. In smoothed aggregation, we typically consider a symmetric measure of strength (defined in Algorithm 11.6), where entry $a_{i,j}$ (edge in the graph of the matrix A) is defined to be a strong connection if

$$|a_{i,j}| \geq \theta \sqrt{|a_{i,i} a_{j,j}|} \quad (11.92)$$

for some threshold $\theta < 1$. That is, the i - j connection is considered strong if the off-diagonal entry is larger in magnitude than the diagonal entries of rows i and j . One motivation for this particular strength measure is that it should detect weak and strong directions in an anisotropic problem, where the size of the entries in rows i and j can change dramatically depending on the anisotropy in the problem.

Algorithm 11.6: SA STRENGTH_OF_CONNECTION($\cdot$)

Input : A , matrix
 θ , threshold
Output : S , strength matrix

```

1 for  $a_{i,j} \neq 0$                                      {for edge in the graph of  $A$ }
2   if  $|a_{i,j}| \geq \theta \sqrt{|a_{i,i} a_{j,j}|}$              {if the weight is large...}
3      $s_{i,j} = 1$                                      {mark as strong connection}
```

Consider Anisotropic Model 2D on a *structured* quadrilateral mesh with $\phi = \frac{\pi}{8}$ and $\varepsilon = 0.01$. The resulting stencil is given by

$$A_{\text{stencil}} = \begin{bmatrix} \text{NW} & \text{N} & \text{NE} \\ \text{W} & \text{O} & \text{E} \\ \text{SW} & \text{S} & \text{SE} \end{bmatrix} = \begin{bmatrix} 0.007 & 0.182 & -0.343 \\ -0.518 & 1.347 & -0.518 \\ -0.343 & 0.182 & 0.007 \end{bmatrix} \quad (11.93a)$$

$$= a_{ii} \begin{bmatrix} 0.005 & 0.135 & -0.255 \\ -0.385 & 1.000 & -0.385 \\ -0.255 & 0.135 & 0.005 \end{bmatrix}, \quad (11.93b)$$

where NW, N, NE, etc., refer to northwest, north, northeast, etc., in reference to the i th degree of freedom, which is labeled as the origin, O. In the last equation, since the diagonal of A is constant for this example, we extract $a_{i,i}$ for convenience. Figure 11.3.4 shows the value of $|a_{i,j}|/\sqrt{|a_{i,i} a_{j,j}|}$ for each edge. If a value of $\theta = 0.25$ is used as the threshold in equation (11.92), then the edges in red are considered strong connections.

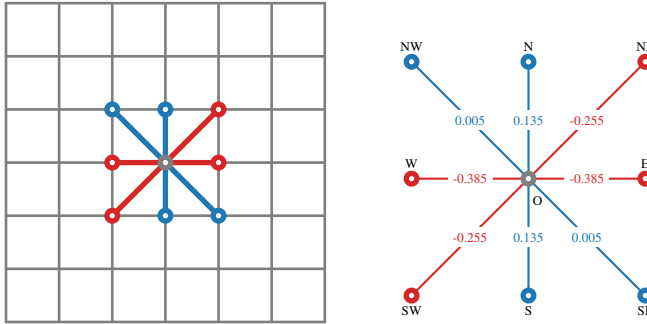


Figure 11.3.4. Strength of connection for Anisotropic Model 2D with $\phi = \frac{\pi}{8}$ and $\varepsilon = 0.01$. (left) Location of the reference node in the mesh. (right) Strong connections in red.

The isotropic case on an unstructured mesh is shown in Figure 11.3.5, where a threshold of $\theta = 0.1$ is highlighted. In practice, $\theta = 0.25$ is commonly used; however, this is one of many tuning parameters in AMG.

Coarsening

Next, let S denote the sparse matrix of strong connections from the previous section—i.e., a nonzero entry in element $s_{i,j}$ means that the connection between i and j is considered to be strong. Coarsening in smoothed aggregation is the process of forming *aggregates* or groups of

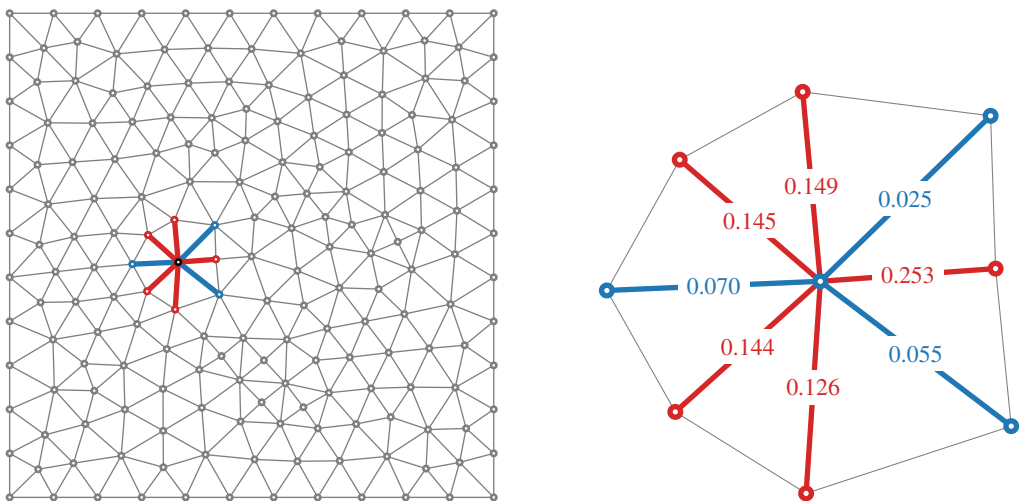


Figure 11.3.5. *Strength of connection for Model 2D. (left) Location of the reference node in the mesh. (right) Strong connections in red.*

nodes that represent the coarse degrees of freedom. Here, we present a greedy approach, noting that maximal independent sets and other graph-coloring algorithms are also used in practice.

To begin, we seek a grouping of the fine nodes (or degrees of freedom) $\{0, \dots, n-1\}$ into n_C distinct sets, or aggregates. Algorithm 11.7 (used in line 3 of Algorithm 11.5) constructs the aggregates in two passes:

First pass If a point and all strongly connected neighbors are unaggregated, then a new aggregate is formed.

Second pass Each remaining point is grouped with the aggregate of a strongly connected neighbor; otherwise a new aggregate is formed from this point.

Many variants exist to form these aggregates—e.g., when joining a strongly connected aggregate in the second pass, the point may join the smallest (or largest) existing aggregate.

Figure 11.3.6 shows an example for the three model problems outlined above. After two passes of the aggregation algorithm, aggregates follow the anisotropy for Anisotropic Model 2D, while the second pass is clearly visible for Model 2D, where additional nodes are added to the initial aggregates.

Interpolation

Interpolation is the key ingredient in a successful AMG method, and smoothed aggregation introduces a number of “knobs” that can be tuned in constructing interpolation operators that balance both convergence and the overall cost of the method. Recalling equation (11.86) in view of the two-level method (in equation (11.85)), we see that the *range* of interpolation should capture the algebraically smooth error.

Smoothed aggregation relies on the notion of a *near-nullspace* or a representation of a *prototype* for the algebraically smooth error (see Definition 11.11). We denote this vector or collection of m vectors by B , an $n \times m$ matrix, and refer to such vectors as *candidate vectors*. For a Poisson problem in 1D, such as Model 1D, the smallest eigenmodes of the operator A (and largest of the weighted Jacobi iteration matrix, G) are simply the smoothest Fourier modes on the grid. Interior

Algorithm 11.7: SA COARSEN($\cdot$)**Input :** S , strength matrix**Output :** C , aggregation adjacency matrix

```

1   $n_C \leftarrow 0$                                      {initialize the number of coarse nodes to zero}
2   $\mathcal{U} \leftarrow \{0, \dots, n-1\}$              {initialize every point as unaggregated}
3  for  $i \in \mathcal{U}$                                      {first pass}
4       $\mathcal{S}_i \leftarrow \{j \mid s_{i,j} \neq 0\}$          {all strongly connected neighbors of  $i$ }
5      if  $\mathcal{S}_i \subset \mathcal{U}$                                  {all neighbors are unaggregated}
6           $C_{i,n_C} \leftarrow 1$                          {set  $i$  in a new aggregate}
7           $C_{j,n_C} \leftarrow 1 \quad j \in \mathcal{S}_i$          {set all strong neighbors to the new aggregate}
8           $n_C \leftarrow n_C + 1$                          {increase the number of aggregates}
9           $\mathcal{U} \leftarrow \mathcal{U} \setminus (\{i\} \cup \mathcal{S}_i)$  {mark as aggregated}
10 for  $i \in \mathcal{U}$                                      {second pass}
11      $\mathcal{S}_i \leftarrow \{j \mid s_{i,j} \neq 0\}$          {all strongly connected neighbors of  $i$ }
12     if  $\exists j \in \mathcal{S}_i$  and  $m$  s.t.  $C_{j,m} = 1$          {find a strongly connected aggregate  $m$ }
13          $C_{i,m} \leftarrow 1$                          {add  $i$  to aggregate  $m$ }
14     else
15          $C_{i,n_C} \leftarrow 1$                          {set  $i$  as a new aggregate, itself}
16          $n_C \leftarrow n_C + 1$                          {increase the number of aggregates}
17      $\mathcal{U} \leftarrow \mathcal{U} \setminus \{i\}$              {mark as aggregated}

```

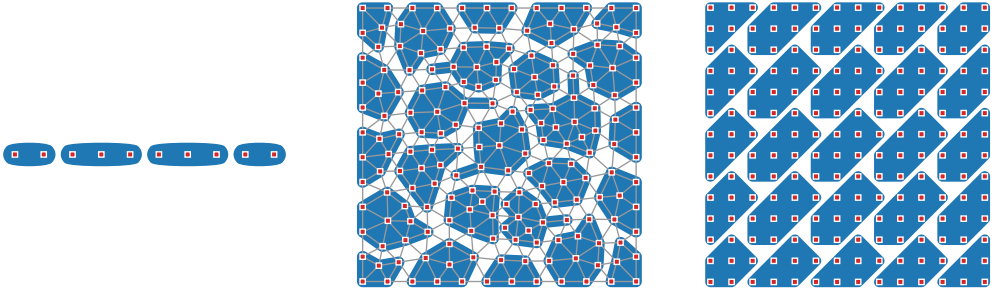


Figure 11.3.6. Aggregation. (left) Four aggregates are shown for Model 1D; (middle) the effects of the second pass of aggregation, where remaining nodes are added to neighboring aggregates, are clearly visible for Model 2D; and (right) the anisotropy is highlighted for Anisotropic Model 2D with $\theta = \frac{\pi}{8}$, $\varepsilon = 0.01$, on a regular $n_x \times n_x$ grid with $n_x = 14$.

to the problem, since the matrix has zero row sum, a *constant* vector is a reasonable representation of the near-nullspace. Therefore, one common approach is to select $B \equiv \mathbf{1}$. Alternatives include using the Fourier modes directly or presmoothing $\mathbf{1}$ to account for boundary conditions.

In other situations, the characterization of B may be more complex. For example, in problems with a strong geometric dependence, B may be constructed using the coordinates of the domain,

$$B = \begin{bmatrix} \mathbf{1} & \mathbf{x} & \mathbf{y} \end{bmatrix}, \quad (11.94)$$

where $\mathbf{x}$ and $\mathbf{y}$ are vectors of the coordinates of the nodes in the mesh. For elasticity problems, a common practice is to construct B using the *rigid-body modes*, defined by the nullspace of the

linear elasticity operator. In 2D, this can be expressed by

$$B = [\text{x-translation} \quad \text{y-translation} \quad \text{rotation}], \quad (11.95)$$

where each of these modes represents a displacement that leads to no stress in an elastic body away from boundaries, while six modes (three translations and rotations about three axes) arise in 3D. For other problems, an accurate characterization of the algebraically smooth error may be elusive, and more thorough construction is required. *Adaptive* smoothed aggregation [53] is one attempt at automatically constructing B (and the entire AMG hierarchy).

With B , we turn to constructing P so that, at least approximately, $\mathcal{R}(B) \subset \mathcal{R}(P)$. To do this, we first partition B across the aggregates using a simple *partition of unity*, creating a matrix with a number of rows equal to that of B and a number of columns equal to the number of aggregates times the number of columns of B . This partitioning leads to a potential problem, in that the columns of B may be distinct globally, but not locally, when restricted to a single aggregate. To detect this, we then do a QR factorization of the initial matrix, writing it as $T B^c$ for orthogonal matrix, T , with the coarse-grid representation of B given by the resulting B^c , which is triangular on each aggregate. For the case of Model 1D with $n = 10$ and four aggregates, using $m = 2$ candidate vectors leads to

$$\begin{bmatrix} b_{0,0} & b_{0,1} & & & & & & & & \\ & b_{1,0} & b_{1,1} & & & & & & & \\ & & & b_{2,0} & b_{2,1} & & & & & \\ & & & & b_{3,0} & b_{3,1} & & & & \\ & & & & & b_{4,0} & b_{4,1} & & & \\ & & & & & & b_{5,0} & b_{5,1} & & \\ & & & & & & & b_{6,0} & b_{6,1} & \\ & & & & & & & & b_{7,0} & b_{7,1} \\ & & & & & & & & & b_{8,0} & b_{8,1} \\ & & & & & & & & & & b_{9,0} & b_{9,1} \end{bmatrix} = \underbrace{\begin{bmatrix} t_{0,0} & t_{0,1} & & & & & & & & & \\ & t_{1,0} & t_{1,1} & & & & & & & & \\ & & & t_{2,0} & t_{2,1} & & & & & & \\ & & & & t_{3,0} & t_{3,1} & & & & & \\ & & & & & t_{4,0} & t_{4,1} & & & & \\ & & & & & & t_{5,0} & t_{5,1} & & & \\ & & & & & & & t_{6,0} & t_{6,1} & & \\ & & & & & & & & t_{7,0} & t_{7,1} & \\ & & & & & & & & & t_{8,0} & t_{8,1} \\ & & & & & & & & & & t_{9,0} & t_{9,1} \end{bmatrix}}_T \underbrace{\begin{bmatrix} b_{0,0}^c & b_{0,1}^c \\ & b_{1,1}^c \\ & & b_{2,0}^c & b_{2,1}^c \\ & & & b_{3,1}^c \\ & & & & b_{4,0}^c & b_{4,1}^c \\ & & & & & b_{5,1}^c \\ & & & & & & b_{6,0}^c & b_{6,1}^c \\ & & & & & & & b_{7,1}^c \end{bmatrix}}_{B^c}, \quad (11.96)$$

where B^c is now an 8×2 matrix, since we have two coarse degrees of freedom per aggregate and four aggregates, resulting in eight “points” on the coarse grid. Note that increasing the dimension of the smooth error candidates in B results in a larger coarse-grid problem, an aspect that we investigate in more detail below. While we do not focus on this here, an important practical detail is to examine the values of B^c on each aggregate to decide if each column in T is important to include in the resulting interpolation operator, and small entries in B^c may be used to eliminate some columns of T . It is also important to note that $\mathcal{R}(B)$ is, in fact, represented in the range of T with this construction (if no truncation is applied). However, we seek an improved interpolation operator and, thus, call T a *tentative* interpolation operator.

In the case of $m = 1$ and $B = \mathbf{1}$, we have a simplified construction for the aggregates shown in Figure 11.3.6, giving

$$\begin{bmatrix} 1 & & & & & & & & & \\ & 1 & & & & & & & & \\ & & 1 & & & & & & & \\ & & & 1 & & & & & & \\ & & & & 1 & & & & & \\ & & & & & 1 & & & & \\ & & & & & & 1 & & & \\ & & & & & & & 1 & & \\ & & & & & & & & 1 & \end{bmatrix} \rightarrow \begin{bmatrix} \sqrt{2}/2 & & & & & & & & & \\ & \sqrt{2}/2 & & & & & & & & \\ & & \sqrt{3}/3 & & & & & & & \\ & & & \sqrt{3}/3 & & & & & & \\ & & & & \sqrt{3}/3 & & & & & \\ & & & & & \sqrt{3}/3 & & & & \\ & & & & & & \sqrt{3}/3 & & & \\ & & & & & & & \sqrt{2}/2 & & \\ & & & & & & & & \sqrt{2}/2 & \end{bmatrix} \begin{bmatrix} \sqrt{2} \\ \sqrt{3} \\ \sqrt{3} \\ \sqrt{2} \end{bmatrix}, \quad (11.97)$$

leading to four degrees of freedom on the coarse grid and the interpolation basis represented in the left panel of Figure 11.3.7, where we observe *piecewise constant* interpolation. The “smoothed” aspect of smoothed aggregation arises in the improvement of the tentative interpolation operator T . Here, we use

$$P = (I - \omega_T D^{-1} A) T, \quad (11.98)$$

where ω_T is not required but often selected as the weighted Jacobi weight in pre-/postsMOOTHING. If A is symmetric and positive definite, then elements in the range of P have lower energy (lower A norm), thus improving the representation of the smooth error. By inspection, P also improves in accuracy. Figure 11.3.7 shows the effect of smoothing on piecewise constants, resulting in a column space of P that resembles piecewise *linears*. The construction of P is summarized in Algorithm 11.8, which is then used for line 4 of Algorithm 11.5.

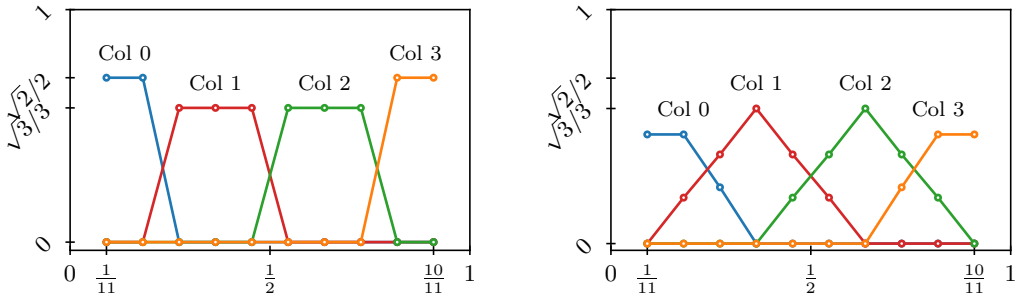


Figure 11.3.7. (left) Tentative interpolation operator and (right) interpolation after smoothing for Model 1D with $n_x = 10$, constant vector for B , and aggregates as in Figure 11.3.6.

Algorithm 11.8: SA INTERPOLATION($\cdot$)

Input : A , linear system matrix
 S , strength matrix
 C , $n \times n_c$ aggregation adjacency matrix
 B , $n \times m$ matrix of candidate vectors
Output : P , $n \times (mn_c)$ interpolation matrix
 B^c , $(mn_c) \times m$ matrix of coarse candidate vectors

```

1 for  $j = 1, \dots, n_c$ 
2    $B_{\text{loc}} \leftarrow \text{localize } B \text{ to aggregate } j$                                 {local basis over the aggregate}
3    $Q_{\text{loc}} R_{\text{loc}} = B_{\text{loc}}$                                                 {normalize the local basis}
4    $T \leftarrow \text{distribute } Q_{\text{loc}}$                                          {distribute the local basis to the global operator}
5    $B^c \leftarrow \text{distribute } R_{\text{loc}}$                                        {form the coarse-grid candidate vectors}
6    $P \leftarrow (I - \omega_T D^{-1} A) T$                                        {smooth the tentative interpolation operator}
```

The improved accuracy in P is costly, however. The increase in the number of nonzero entries in P propagates to the coarse-level matrix in the last step (line 5) of Algorithm 11.5, where the Galerkin product is used to form $A^c = P^T A P$. A common refinement of equation (11.98) is to smooth P *only* along strong connections.

Example

As an example, consider Model 2D with $f(x, y) = 0$ and a random initial guess, $\mathbf{u}_0$, for the solution $\mathbf{u} = \mathbf{0}$. We compare four different configurations of interpolation for smoothed aggregation based on the setup above:

Case 1 No interpolation smoothing: This results in a sparse representation of P ($= T$) and piecewise constant interpolation. A single vector $B = \mathbf{1}$ is used.

Case 2 One iteration of interpolation smoothing: With the same B as in Case 1, this improves interpolation.

Case 3 Two iterations of interpolation smoothing: With the same B as in Case 1, this again improves interpolation.

Case 4 One iteration of interpolation smoothing: In this case, we enrich the space spanned by B , using $B = [\mathbf{1} \quad \mathbf{x} \quad \mathbf{y}]$, where $\mathbf{x}$ and $\mathbf{y}$ denote the x - and y -coordinates (respectively) of the mesh.

In each case, we use symmetric Gauss–Seidel (requiring two iterations) for pre- and postrelaxation, leading to four sweeps in total on each level. For Case 1, we test both a V- and a W-cycle, while using only V-cycles for the latter three cases.

The AMG hierarchies are summarized in Table 11.3.1 for each of the solvers above. Here, we introduce the concept of *complexity*. The *operator* complexity is a measure of the cost of performing a single matrix-vector multiply on all levels of a multigrid method in comparison to the cost on just the finest grid,

$$c_{\text{op}} := \frac{\sum_{\ell} \text{nnz}(A^{\ell})}{\text{nnz}(A^0)}, \quad (11.99)$$

where $\text{nnz}(A)$ is the number of nonzero entries in the matrix A . Relatedly, the *cycle* complexity is a measure of the overall cost of a cycle compared to relaxation on the fine level. Assuming ν prerelaxation and postrelaxation sweeps in total, this is given by

$$c_{\text{cyc}} := \frac{\sum_{\ell} \mu^{\ell} \nu \text{nnz}(A^{\ell})}{\text{nnz}(A^0)}, \quad (11.100)$$

where $\mu = 1$ for a V-cycle and $\mu = 2$ for a W-cycle. Lastly, the *grid* complexity is a measure of the size of all grids in comparison to the finest,

$$c_{\text{grid}} := \frac{\sum_{\ell} n_{\ell}}{n_0}, \quad (11.101)$$

where n_{ℓ} is the number of rows or columns in A^{ℓ} . Each of these helps to assess the complexity of each iteration of the method, yet we also need to consider the rate of convergence to quantify the overall cost. For this, we measure the convergence factor ρ as the ratio of the norms of the last two residuals after converging in k iterations,

$$\rho := \frac{\|\mathbf{r}_k\|_2}{\|\mathbf{r}_{k-1}\|_2}, \quad (11.102)$$

approximating the asymptotic convergence factor of the method. Then, the *effective* convergence factor of an iteration, which measures the effective reduction in error per work unit, is defined as

$$\rho_{\text{eff}} := \rho^{1/c_{\text{cyc}}}. \quad (11.103)$$

Table 11.3.1. AMG statistics for four cases of smoothed aggregation applied to Model 2D with $f(x, y) = 0$. Here, n_ℓ represents the size of A^ℓ on each level ℓ ; nnz_ℓ is the number of nonzeros in A^ℓ .

	Case 1		Case 1, W-cycle		Case 2		Case 3		Case 4	
c_{op} :	1.13		1.13		1.2		1.28		2.8	
c_{cyc} :	4.51		5.15		4.8		5.11		11.2	
c_{grid} :	1.16		1.16		1.15		1.14		1.46	
ρ_{eff} :	0.938		0.936		0.791		0.755		0.789	
	n_ℓ	nnz_ℓ	n_ℓ	nnz_ℓ	n_ℓ	nnz_ℓ	n_ℓ	nnz_ℓ	n_ℓ	nnz_ℓ
level 0:	191	1243	191	1243	191	1243	191	1243	191	1243
level 1:	25	137	25	137	25	233	25	341	75	2097
level 2:	5	21	5	21	4	16	2	4	12	144

The statistics in Table 11.3.1 show the impact of interpolation smoothing. For each increase in smoothing on the interpolation operator, the number of nonzeros increases in the resulting level 1 coarse-grid operators (from 137 to 233 to 341), leading to higher operator complexities. Likewise, using additional vectors in B increases the size of the level 1 coarse space from 25 to 75 and gives a coarse-grid operator with more nonzeros than the fine-grid operator. The effective convergence factor is also tabulated in Table 11.3.1, which shows that the most efficient of these cycles is that in Case 3, although Cases 2 and 4 are not much less efficient.

Figure 11.3.8 shows the residual history for each of these cases. We observe a significant improvement in convergence from one smoothing iteration on interpolation: the convergence factor, ρ , improves from 0.75 to 0.32. Additional smoothing leads to a small improvement in convergence, to a factor of 0.24. Enriching B leads to another large improvement in the convergence factor, to 0.07, but more than doubles the cost per iteration, with an operator complexity jumping from around 1.2 to 2.8. Figure 11.3.8 also shows the use of a W-cycle for the case of no interpolation smoothing. A W-cycle can help address issues with insufficient interpolation by

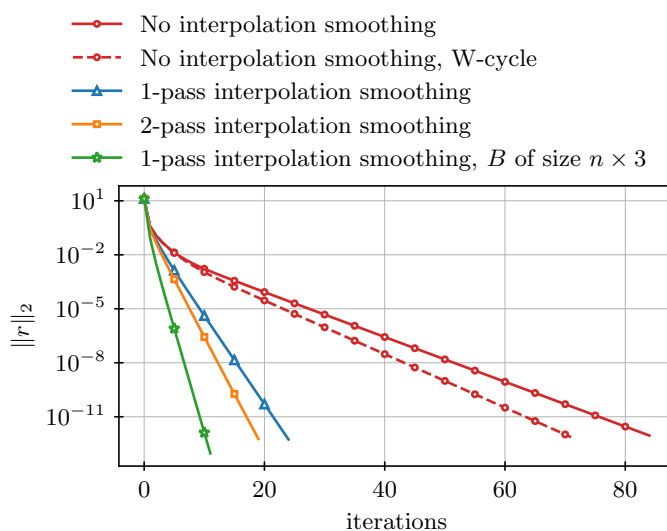


Figure 11.3.8. Residual history for difference cases of smoothed aggregation applied to Model 2D with $f(x, y) = 0$. Unless noted, a V-cycle is used.

revisiting coarser grids. In this case, the W-cycle leads to a modest improvement to $\rho_{\text{eff}} = 0.936$ (yet the cycle complexity grows from 4.51 in the case of a V-cycle to 5.15 for a W-cycle).

11.3.3 ■ Algebraic multigrid: C/F splittings

CF-AMG follows a similar approach to constructing an AMG hierarchy, again by considering the nature of algebraically smooth error. However, as Figure 11.3.3b shows, the coarse grids generated and the steps to construct interpolation are fundamentally different in CF-AMG than in smoothed aggregation AMG.

Strength of connection

The most commonly used strength measure in CF-AMG is nonsymmetric, measuring *dependence*: how much a value in an algebraically smooth error at point i *depends* on the value at point j . We say that point i strongly depends on j if

$$|a_{i,j}| \geq \theta \max_{k \neq i} |a_{i,k}| \quad (11.104)$$

for some threshold θ , meaning that $a_{i,j}$ is comparable to the largest coefficient(s) in row i . As in the smoothed aggregation case, the parameter θ can be tuned to vary the resulting coarse-grid selection. The full algorithm is given in Algorithm 11.9, which is then used in line 2 of Algorithm 11.5.

Algorithm 11.9: CF STRENGTH_OF_CONNECTION($\cdot$)

Input : A , matrix

θ , threshold

Output : S , strength matrix

1 for $a_{i,j} \neq 0$	2 if $ a_{i,j} \geq \theta \max_{k \neq i} a_{i,k} $	{for edge in the graph of A }
3 $s_{i,j} = 1$		{if the weight is large... } {mark as strong connection}

In S , we have also encoded the points that strongly depend on i . Paying careful attention to the index, j strongly depends on i if $s_{j,i} = 1$. Note though that, in general, we do not have $S = S^T$ even if A is symmetric, since strength is measured relative to entries in each row. One common variant on this algorithm is to use $-a_{i,j}$ and $-a_{i,k}$ in place of their absolute values, which is effective if A is an M-matrix or close to one (as is often the case for discretizations of isotropic diffusion problems on standard grids).

Coarsening

Unlike smoothed aggregation, the coarsening in CF-AMG selects a subset of the fine degrees of freedom for coarse representation. Consider the index set of all degrees of freedom $\Omega = \{0, \dots, n-1\}$. The goal of coarsening is to construct a *splitting* (or *partitioning*) of Ω into two disjoint subsets, $\Omega = C \cup F$, where points in C are labeled as coarse points and points in F are considered fine-only points.

In the next section, we see that interpolation maps grid functions on subset C onto Ω . As a result, we form C based on heuristics aimed to ensure the quality of this interpolation operator.

In particular, all F points should strongly depend on a sufficient number of C points in order to be effectively able to interpolate a correction to all points in the F set. To this end, for each point i , we define the set of coarse points upon which it strongly depends as C_i :

$$C_i = \{j \mid s_{j,i} = 1\} \cap C. \quad (11.105)$$

For each $i \in F$, the original approach to splitting in CF-AMG [192] *requires* that each point, j , that strongly depends on i either be in C itself (and, thus, be in C_i) or strongly depend on at least one point in C_i , so that we meaningfully account for the connection between i and j in interpolation. At the same time, the set, C , should be as small as possible in order to maintain the efficiency of the multigrid cycle. As a result, a common guide is to insist that all points in F satisfy these strong dependence criteria while seeking coarse grids, C , with minimal strong dependence between any two points $i, j \in C$. Algorithm 11.10 (used in line 3 of Algorithm 11.5) outlines the so-called *Ruge–Stüben* coarsening algorithm, although many variants and optimizations exist, particularly in a parallel setting [7].

Algorithm 11.10: CF COARSEN($\cdot$)

Input : S , strength matrix

Output : C , list of C -points

```

1 Initialize  $\mathcal{U} = \{0, \dots, n-1\}$ ,  $C = \emptyset$ ,  $F = \emptyset$                                 {set all points as unassigned}
2 for  $i \in \mathcal{U}$ 
3    $w_i := |\{j \mid s_{j,i} = 1\}|$                                                     {record the weight as the number of points that depend on it}
4 while  $\mathcal{U} \neq \emptyset$                                                             {first pass}
5    $i = \operatorname{argmax}_{j \in \mathcal{U}} w_j$                                                 {select the point with the largest weight}
6    $C \leftarrow C \cup \{i\}$                                                         {assign as a  $C$ -point}
7    $\mathcal{U} \leftarrow \mathcal{U} \setminus \{i\}$                                                 {mark as assigned}
8   for  $j$  such that  $s_{j,i} = 1$  and  $j \in \mathcal{U}$                                         {mark all neighbors...}
9      $F \leftarrow F \cup \{j\}$                                                     {assign as an  $F$ -point}
10     $\mathcal{U} \leftarrow \mathcal{U} \setminus \{j\}$                                               {mark as assigned}
11    for  $k$  such that  $s_{k,j} = 1$  and  $k \in \mathcal{U}$                                         {mark all neighbors' neighbors...}
12       $w_k \leftarrow w_k + 1$                                                     {increase weight}
13 for  $i \in F$                                                                     {second pass}
14   for  $j$  such that  $s_{i,j} = 1$ 
15     if  $j \in C$ 
16       continue                                                                {found a coarse connection}
17     if  $\nexists k \in C$  such that  $s_{i,k} = s_{j,k} = 1$ 
18        $C \leftarrow C \cup \{i\}$                                                   {assign as a  $C$ -point}
19        $F \leftarrow F \setminus \{i\}$                                               {remove as an  $F$ -point}
20       break                                                                    {go to the next fine  $i$ -point}

```

The first pass of Algorithm 11.10 is known as a maximal independent set algorithm, which selects a set of C -points guaranteed to have the property that no two C -points strongly depend on one another. While this ensures a small set of C -points, it does not guarantee that we can define an effective interpolation operator from C onto F . Thus, the second pass ensures that any two strongly connected F -points have a common strongly connected C -point by adding points to C if they fail this test.

As an example, consider the 2D problems defined in Model 2D and Anisotropic Model 2D. We show two levels of C/F -points in Figure 11.3.9, noting that these are necessarily nested—i.e., the C -points on level 1 are a subset of the C -points on level 0. The figure shows that the coarsening algorithm (and strength of connection, based on $\theta = 0.25$) picks up the direction of anisotropy for Anisotropic Model 2D, while the coarsening is roughly uniform for the isotropic problem.

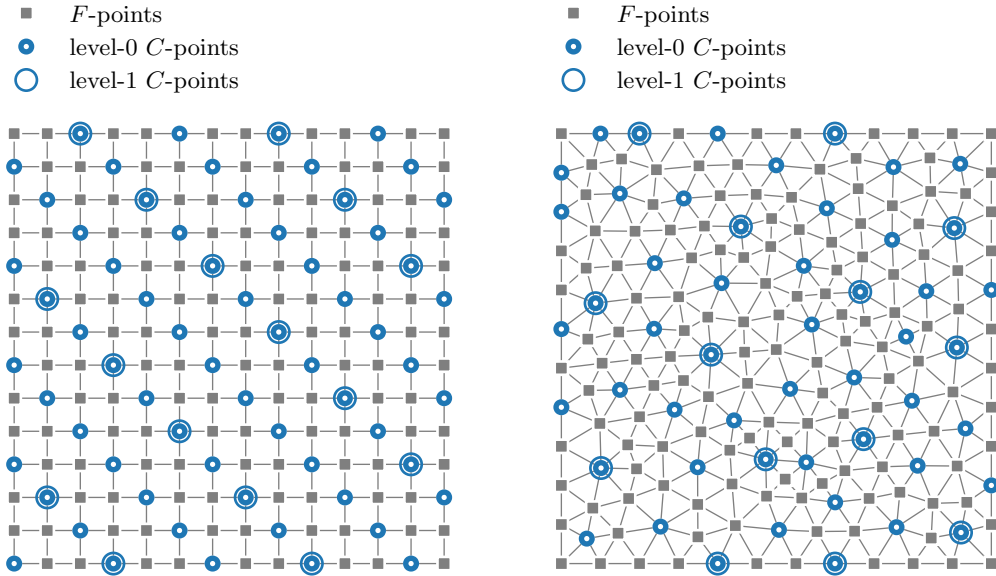


Figure 11.3.9. *Splitting in CF-AMG. (left) Coarsening two levels for Anisotropic Model 2D with $\phi = \pi/8$ and $\varepsilon = 0.01$ on a regular $n_x \times n_x$ grid with $n_x = 14$. (right) Coarsening two levels for Model 2D.*

Interpolation

If we assume a C/F splitting of points, as outlined above, then we can consider the transfer function $P : \mathbb{R}^{|C|} \rightarrow \mathbb{R}^{|F|}$. For simplicity, we reorder the matrix problem to put the F -points first, followed by the C -points, although we emphasize that this is only used to make the notation simpler and is not needed in practice. Considering the error equation, $Ae \approx 0$, for algebraically smooth error, $e = \begin{bmatrix} e_F \\ e_C \end{bmatrix}$, results in

$$\begin{bmatrix} A_{F,F} & A_{F,C} \\ A_{C,F} & A_{C,C} \end{bmatrix} \begin{bmatrix} e_F \\ e_C \end{bmatrix} \approx \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \quad (11.106)$$

Here, we see that the first equation associates the error at the F - and C -points as

$$A_{F,F}e_F + A_{F,C}e_C = 0, \quad (11.107)$$

so that

$$e_F = -A_{F,F}^{-1}A_{F,C}e_C. \quad (11.108)$$

In the coarse-grid correction process, we suppose that we have found a good representation of the error at the C -points, e_C , which leads to a so-called *ideal* form of interpolation:

$$\begin{bmatrix} e_F \\ e_C \end{bmatrix} = \underbrace{\begin{bmatrix} -A_{F,F}^{-1}A_{F,C} \\ I \end{bmatrix}}_{P_*} e_C. \quad (11.109)$$

Constructing P_* in equation (11.109) is far from ideal in practice: computing $A_{F,F}^{-1}$ is costly and results in a high number of nonzeros in interpolation, which leads to costly coarse-grid solves. On the other hand, the form of P_* motivates the construction of practical interpolation operators, P , of the form

$$P = \begin{bmatrix} W \\ I \end{bmatrix}, \quad (11.110)$$

with some weight matrix W .

Returning to the error equation for smooth error, we assume that $Ae = 0$ and focus on entry $i \in F$:

$$a_{i,i}e_i = - \sum_{j \neq i} a_{i,j}e_j. \quad (11.111)$$

If each j such that $a_{i,j} \neq 0$ is in C , then interpolation is readily constructed from equation (11.111)—i.e., each entry of W could be given by $-a_{i,j}/a_{i,i}$. This corresponds to the case that $A_{F,F}$ is diagonal, and computing the ideal form of interpolation in equation (11.109) is feasible. Generally, however, i is connected to a range of different types of points, j , as shown in Figure 11.3.10. We divide the points that i depends on into four groups,

$$\{j \neq i \mid a_{i,j} \neq 0\} = \Omega_i = \underbrace{C_i^s}_{\text{strong } C\text{-pts}} + \underbrace{F_i^s}_{\text{strong } F\text{-pts}} + \underbrace{C_i^w}_{\text{weak } C\text{-pts}} + \underbrace{F_i^w}_{\text{weak } F\text{-pts}}, \quad (11.112a)$$

where

$$C_i^s = \{j \neq i \mid s_{i,j} = 1\} \cap C, \quad (11.112b)$$

$$F_i^s = \{j \neq i \mid s_{i,j} = 1\} \cap F, \quad (11.112c)$$

$$C_i^w = (\Omega_i \cap C) \setminus C_i^s, \quad (11.112d)$$

$$F_i^w = (\Omega_i \cap F) \setminus F_i^s. \quad (11.112e)$$

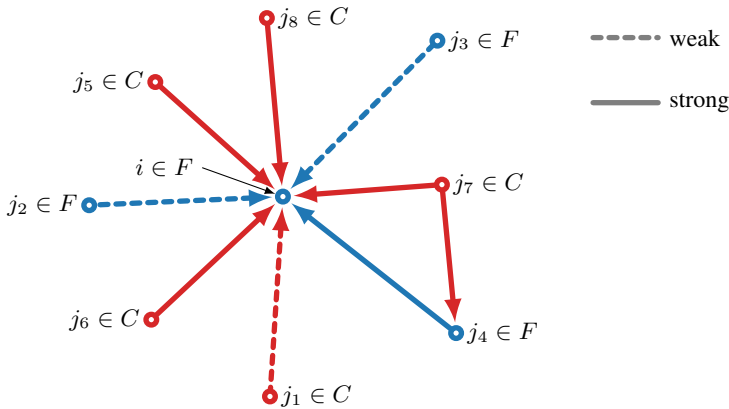


Figure 11.3.10. Example of strong and weak connections around $i \in F$. Point i depends strongly on points j_4, j_5, j_6, j_7 , and j_8 , while j_1, j_5, j_6, j_7 , and j_8 are in C .

Rewriting equation (11.111), we have

$$a_{i,i}e_i = - \sum_{j \in C_i^s} a_{i,j}e_j - \sum_{k \in F_i^s} a_{i,k}e_k - \sum_{\ell \in C_i^w} a_{i,\ell}e_\ell - \sum_{\ell \in F_i^w} a_{i,\ell}e_\ell. \quad (11.113)$$

Next, we make some assumptions to be able to rewrite this as a definition of the matrix W in equation (11.110). In the most common approach to interpolation for CF-AMG, we assume that $e_\ell \approx e_i$ for any weak connection. Substituting this into equation (11.113) leads to

$$\left(a_{i,i} + \sum_{\ell \in C_i^w \cup F_i^w} a_{i,\ell} \right) e_i = - \sum_{j \in C_i^s} a_{i,j} e_j - \sum_{k \in F_i^s} a_{i,k} e_k. \quad (11.114)$$

Then, to address the strong F connections in F_i^s , we approximate these values using the points in C_i^s on which they strongly depend. For each $k \in F_i^s$, this is typically done by approximating e_k as the weighted sum

$$e_k \approx \frac{\sum_{m \in C_i^s} a_{k,m} e_m}{\sum_{m \in C_i^s} a_{k,m}} = \frac{\sum_{j \in C_i^s} a_{k,j} e_j}{\sum_{m \in C_i^s} a_{k,m}}, \quad (11.115)$$

where we use the relabeling of the summation index in the numerator in what follows. Substituting this approximation into equation (11.114), we arrive at

$$\left(a_{i,i} + \sum_{\ell \in C_i^w \cup F_i^w} a_{i,\ell} \right) e_i = - \sum_{j \in C_i^s} a_{i,j} e_j - \sum_{k \in F_i^s} a_{i,k} \left(\frac{\sum_{j \in C_i^s} a_{k,j} e_j}{\sum_{m \in C_i^s} a_{k,m}} \right) \quad (11.116a)$$

$$= - \sum_{j \in C_i^s} a_{i,j} e_j - \sum_{j \in C_i^s} \sum_{k \in F_i^s} \left(\frac{a_{i,k} a_{k,j}}{\sum_{m \in C_i^s} a_{k,m}} \right) e_j \quad (11.116b)$$

$$= \sum_{j \in C_i^s} \left(-a_{i,j} - \sum_{k \in F_i^s} \left(\frac{a_{i,k} a_{k,j}}{\sum_{m \in C_i^s} a_{k,m}} \right) \right) e_j, \quad (11.116c)$$

which leads to the following weights on neighboring strong C -points:

$$e_i = \sum_{j \in C_i^s} \left(\frac{-a_{i,j} - \sum_{k \in F_i^s} \left(\frac{a_{i,k} a_{k,j}}{\sum_{m \in C_i^s} a_{k,m}} \right)}{a_{i,i} + \sum_{\ell \in C_i^w \cup F_i^w} a_{i,\ell}} \right) e_j. \quad (11.116d)$$

The construction of P via this formula is summarized in Algorithm 11.11 (again used in line 4 of Algorithm 11.5).

To see the difference of this interpolation from the ideal one, we construct interpolation operators P and $P_\star$ for Model 2D. The sparsity (after reordering) is shown in Figure 11.3.11. We clearly see the $\begin{bmatrix} W \\ I \end{bmatrix}$ structure of both operators and that the practical interpolation operator, P , mimics the entries in $P_\star$, but with much higher sparsity.

Example

As a final example, we construct a CF-AMG method for Model 2D and compare with the smoothed aggregation AMG (SA-AMG) constructed above (using a single pass of interpolation smoothing and a single candidate vector). The statistics for each method are summarized in Table 11.3.2. Typically, SA-AMG coarsens problems more aggressively than CF-AMG—here, we see a reduction in the number of points by a factor of 3.7 in the first coarsening for CF-AMG, while SA-AMG coarsens by a factor of 7.64 (on the first level), leading to a coarse grid with half as many degrees of freedom and roughly half as many nonzeros in the matrix, as reflected in

Algorithm 11.11: CF INTERPOLATION($\cdot$)**Input :** A , matrix S , strength matrix C , list of C -points**Output :** P , interpolation matrix

```

1  $W \leftarrow 0$                                      {initialize weight matrix}
2 for  $i \in F$ 
3    $\Omega_i \leftarrow \{j \neq i \mid a_{i,j} \neq 0\}$        {neighbors of point  $i$ }
4    $C_i^s \leftarrow \{j \neq i \mid s_{i,j} = 1\} \cap C$    {set strong  $C$ -points}
5    $F_i^s \leftarrow \{j \neq i \mid s_{i,j} = 1\} \cap F$    {set strong  $F$ -points}
6    $C_i^w \leftarrow (\Omega_i \cap C) \setminus C_i^s$          {set weak  $C$ -points}
7    $F_i^w \leftarrow (\Omega_i \cap F) \setminus F_i^s$        {set weak  $F$ -points}
8   for  $j \in C_i^s$ 
9      $w_{i,j} \leftarrow \frac{-a_{i,j} - \sum_{k \in F_i^s} \left( \frac{a_{i,k} a_{k,j}}{\sum_{m \in C_i^s} a_{k,m}} \right)}{a_{i,i} + \sum_{\ell \in C_i^w \cup F_i^w} a_{i,\ell}}$    {construct the weight based on equation (11.116d)}
10  $P = \begin{bmatrix} W \\ I \end{bmatrix}$                                      {form the sparse matrix for interpolation}

```

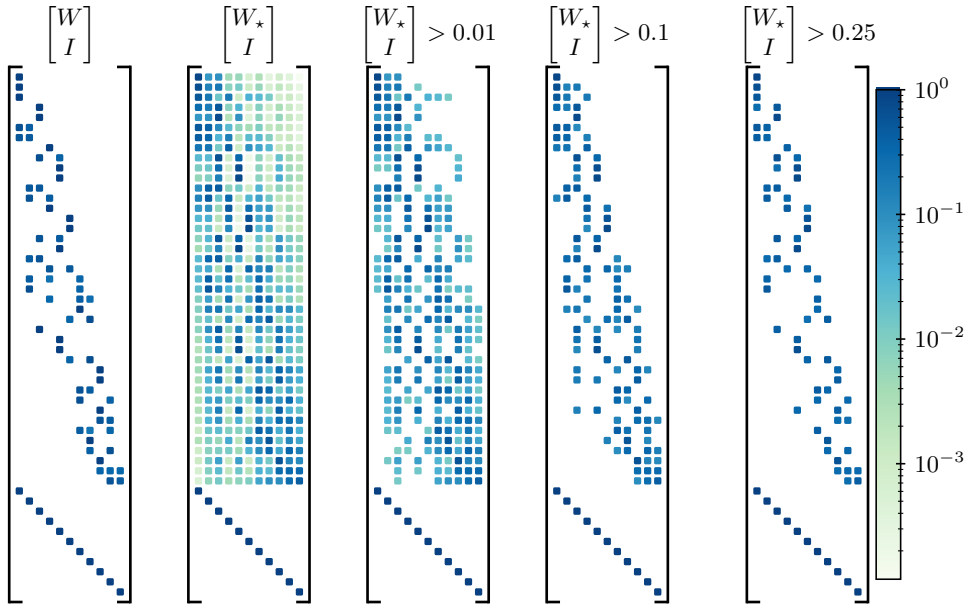


Figure 11.3.11. Nonzero patterns for CF-AMG interpolation constructed for Model 2D. On the left are shown the nonzero entries in P formed through Algorithm 11.11. The remaining figures show ideal interpolation, displaying $P_\star = \begin{bmatrix} W^* \\ I \end{bmatrix}$ with $W_\star = -A_{F,F}^{-1} A_{F,C}$, and an increase in filtering of small matrix values. For example, the rightmost figure shows ideal interpolation after setting entries less than or equal to 0.25 to zero.

Table 11.3.2. AMG statistics for CF-AMG and SA-AMG applied to Model 2D.

	CF-AMG		SA-AMG	
c_{op} :	1.41		1.2	
c_{cyc} :	5.65		4.8	
c_{grid} :	1.36		1.15	
ρ_{eff} :	0.858		0.790	
	n_ℓ	nnz_ℓ	n_ℓ	nnz_ℓ
level 0:	191	1243	191	1243
level 1:	51	407	25	233
level 2:	13	89	4	16
level 3:	4	16		

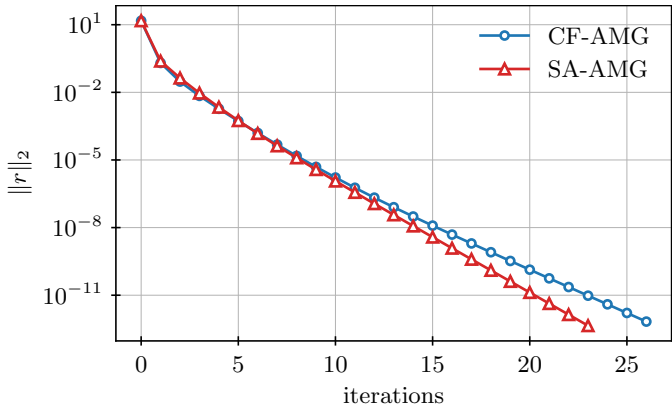


Figure 11.3.12. Residual history for CF-AMG and SA-AMG for Model 2D.

the operator complexity for the two methods. Despite the more aggressive coarsening, smoothed aggregation converges slightly faster for this example (with a convergence factor of 0.32 compared with 0.42 for CF-AMG), leading to a notable improvement in the effective convergence factor, as shown in Table 11.3.2. Residual convergence histories for the two methods are shown in Figure 11.3.12. Both CF-AMG and SA-AMG solvers are constructed using PyAMG [38] for this example.

11.4 ■ Block-structured systems of equations

Objectives

As discussed in Sections 10.3 and 10.4, numerical methods for approximating solutions to coupled systems of PDEs lead to block-structured matrices in the resulting linear systems. While some preconditioners and iterative methods can be directly applied to these systems without taking advantage of the structure present, often the best techniques are those that are derived with a full knowledge of the underlying system of PDEs and its discretization. In this section, you will explore three such approaches for common discretizations of the Stokes and Navier–Stokes equations, often used as the simplest examples of coupled systems and prototypes for many problems in fluid and solid mechanics. As described in Section 10.3, typical

discretizations of these equations lead to linear systems that are written as

$$\begin{bmatrix} A & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix}, \quad (11.117)$$

where, for the stationary Stokes equations with Dirichlet boundary conditions, for example, A represents the discretization of the *vector* Laplace operator (see Remark 10.20) acting on the discretized velocity vector, $\mathbf{u}$, and B represents the discretized gradient operator acting on the discretized pressure, $\mathbf{p}$, with B^T then representing minus the divergence operator. For both time-steady and time-dependent problems, you will consider two common families of methods:

- block-factorization preconditioners and
- monolithic multigrid methods.

For time-dependent problems, you will also be introduced to split-step schemes that are a special case of the IMEX schemes introduced in Section 9.6.

In this section, we consider discretizations of the time-steady Stokes equations,

$$-\Delta \mathbf{u} + \nabla p = \mathbf{f}, \quad (11.118a)$$

$$\nabla \cdot \mathbf{u} = 0; \quad (11.118b)$$

the time-dependent Stokes equations,

$$\frac{\partial \mathbf{u}}{\partial t} - \Delta \mathbf{u} + \nabla p = \mathbf{f}, \quad (11.119a)$$

$$\nabla \cdot \mathbf{u} = 0; \quad (11.119b)$$

and the time-dependent Navier–Stokes equations,

$$\frac{\partial \mathbf{u}}{\partial t} + Re(\mathbf{u} \cdot \nabla \mathbf{u}) - \Delta \mathbf{u} + \nabla p = \mathbf{f}, \quad (11.120a)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (11.120b)$$

where Re is the so-called *Reynolds number* of the flow. Discretization and linearization of these equations (using stable discretizations, as discussed in Sections 10.1 and 10.3) lead to systems of the form in equation (11.117), with the same B matrix but different A blocks. In the first two subsections, we fix ideas (when needed) by considering the time-steady Stokes equations, where A is a discretized vector Laplacian (see Remark 10.20), to develop key ideas in the simplest possible setting. Section 11.4.3 focuses on extending these ideas and developing new ones for the time-dependent case.

11.4.1 ■ Block-factorization preconditioners

A common class of preconditioners are based on the block factorization of the system matrix:

$$\begin{bmatrix} A & B \\ B^T & 0 \end{bmatrix} = \begin{bmatrix} A & 0 \\ B^T & -B^T A^{-1} B \end{bmatrix} \begin{bmatrix} I & A^{-1} B \\ 0 & I \end{bmatrix} \quad (11.121a)$$

$$= \begin{bmatrix} I & 0 \\ B^T A^{-1} & I \end{bmatrix} \begin{bmatrix} A & 0 \\ 0 & -B^T A^{-1} B \end{bmatrix} \begin{bmatrix} I & A^{-1} B \\ 0 & I \end{bmatrix}. \quad (11.121b)$$

The *Schur complement* of the original system is the matrix $-B^T A^{-1} B$, which plays a key role in the development of preconditioners for the block system. For the time-steady Stokes system, a surprising result, due to Verfürth [228], is that for any inf-sup stable finite-element method, $B^T A^{-1} B$ is spectrally equivalent to a mass matrix on the pressure space. Knowing that multigrid methods provide effective (spectrally equivalent) preconditioners for the vector Laplace operator, A , then opens the door to a family of preconditioners.

A simple block-factorization preconditioner for the coupled system is to approximately invert the block LU factorization in equation (11.121a) using a multigrid cycle for A and simple preconditioner for the mass-matrix approximation to the Schur complement. Given a block-structured residual, $\mathbf{r}$, which we decompose as above into $\mathbf{r}_u$ and $\mathbf{r}_p$, we define the action of the preconditioner as follows:

1. Approximately solve $A\mathbf{y} = \mathbf{r}_u$ using a few steps of a preconditioned iteration to generate $\mathbf{y}$.
2. Approximately solve $-M_p \mathbf{z}_p = \mathbf{r}_p - B^T \mathbf{y}$ using a few steps of a preconditioned iteration to generate $\mathbf{z}_p$.
3. Approximately solve $A\mathbf{z}_u = \mathbf{r}_u - B\mathbf{z}_p$ using a few steps of a preconditioned iteration to generate $\mathbf{z}_u$.

In principle, any polynomial method can be used for each of the subsolves in this algorithm. For common discretizations, such as Taylor–Hood, geometric or algebraic multigrid methods provide optimal algorithms for approximating the solutions of systems with A using one or two steps of a multigrid V-cycle, while the mass matrix, M_p , can be approximately inverted with a few steps of a stationary Jacobi iteration (or Chebyshev iteration [232]). The advantage of stationary (or Chebyshev) iterations is that the preconditioner becomes a stationary iteration, implicitly defining

$$\begin{bmatrix} \mathbf{z}_u \\ \mathbf{z}_p \end{bmatrix} = \mathcal{M} \begin{bmatrix} \mathbf{r}_u \\ \mathbf{r}_p \end{bmatrix}, \quad (11.122)$$

which is then used as a preconditioner for both GMRES and FGMRES.

In practice, it is common to simplify the above preconditioner into what are known as block-triangular and block-diagonal preconditioners [172] (see also [92] for more details on such approaches with applications in fluid dynamics). For the block-triangular preconditioner, we simply “discard” the second matrix in the block LU factorization in equation (11.121a), which is equivalent to omitting the third step in the above algorithm and taking $\mathbf{z}_u = \mathbf{y}$ from the first step. In the common case where the velocity space is of substantially larger dimension than the pressure space (and the preconditioner used to approximately invert A is more expensive to apply than that to approximately invert M_p), this saves almost half the cost per iteration. It is observed in practice that this does not greatly harm the convergence of an outer Krylov method for the solution of equation (11.117). Further cost savings are had by similarly discarding the block L and U factors in the block LDU factorization in equation (11.121b) and directly defining $\mathbf{z}_u$ by applying a stationary iteration to $A\mathbf{z}_u = \mathbf{r}_u$ and $\mathbf{z}_p$ by applying a stationary iteration to $M_p \mathbf{z}_p = \mathbf{r}_p$. For this block-diagonal preconditioner, the cost savings are minimal in comparison to the block-triangular preconditioner (only omitting a single matrix-vector product with B^T); however, the savings come from the fact that this defines (with suitable choices for the above iterations) a symmetric and positive-definite preconditioner for the symmetric-indefinite matrix in equation (11.117). Thus, the block-diagonal preconditioner is suitable for use as a preconditioner for MINRES. In more complicated problems, the cost savings for using MINRES in place of GMRES are seen to be worthwhile when many iterations are needed.

11.4.2 ■ Monolithic multigrid methods

An alternative approach to block-factorization preconditioners is to apply so-called *monolithic* multigrid methods to the coupled system in equation (11.117). In contrast to a block preconditioner, where multigrid might be applied just to the subsystem involving A (or even smaller subblocks of A), monolithic approaches maintain the coupling between $\mathbf{u}$ and $\mathbf{p}$ on all levels of the hierarchy. A standard approach to monolithic geometric multigrid is to coarsen the underlying mesh in much the same way as we do for an uncoupled problem, yielding a grid $2h$ problem that has the same structure of degrees of freedom as the original grid h problem. Interpolation from grid $2h$ to grid h is determined by independently interpolating each component of the system, leading to a composite interpolation operator of the form

$$P = \begin{bmatrix} P_u & 0 \\ 0 & P_p \end{bmatrix}, \quad (11.123)$$

where P_u is, itself, usually a composite interpolation operator of similar form to independently interpolate the two or three components of the velocity vector. Restriction takes a similar block form and may be the transpose of interpolation (as is natural for finite-element discretizations) or a scaled transpose (for finite-difference discretizations), or it may be chosen independently (e.g., for the case of Navier–Stokes, where A is strongly nonsymmetric). The coarse-grid operator can be formed as a Galerkin product with this P or via rediscrization (and the two coincide, for example, when equation (11.117) comes from a Galerkin finite-element discretization and P corresponds to the canonical finite-element interpolation operator for all components between grids $2h$ and h).

In order to define an effective monolithic multigrid approach, the relaxation scheme must be carefully chosen in order to properly resolve the coupling between the variables in the system. Note, in particular, that standard pointwise relaxation schemes, such as point (weighted) Jacobi and Gauss–Seidel, do not apply to systems of the form in equation (11.117) due to the zero diagonal entries. While some relaxation schemes, such as ILU-based approaches, can be applied to equation (11.117), they are typically less effective than those that are built on the structure of the saddle-point problem. There are four main classes of relaxation schemes that are in common use for coupled saddle-point problems:

1. Braess–Sarazin approaches [49] are based on the block factorization in equation (11.121a), with the realization that low-cost approximations to solves with A and $B^T A^{-1} B$ lead to effective relaxation schemes. In a typical Braess–Sarazin relaxation, we use the stationary iteration

$$\begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} + \begin{bmatrix} \alpha C & B \\ B^T & 0 \end{bmatrix}^{-1} \left(\begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix} - \begin{bmatrix} A & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} \right) \quad (11.124)$$

as a relaxation scheme. In place of A , we use the approximation αC , where C is often taken to be a diagonal matrix, such as the identity matrix or the diagonal of A , and α is an appropriately chosen relaxation parameter. In this setting, we must both assemble and solve a linear system with the approximate Schur complement, $-\frac{1}{\alpha} B^T C^{-1} B$. In contrast to the true Schur complement, this is efficiently assembled when C is diagonal; however, it still cannot be efficiently inverted. *Inexact* Braess–Sarazin approaches [240] overcome this by applying simple relaxation schemes, such as point (weighted) Jacobi or Gauss–Seidel, to the approximate Schur complement. While proper choice of the relaxation parameter(s) is critical in achieving good multigrid performance, Braess–Sarazin approaches have been successfully used in many contexts.

2. Uzawa relaxation [160] is most easily seen as the analogous change to Braess–Sarazin as moving from a Schur complement block-factorization preconditioner to a block lower-triangular preconditioner. That is, if Braess–Sarazin methods approximate the full block LU factorization in equation (11.121a), then Uzawa methods only approximate the inverse of the block L factor. This leads to a stationary iteration of the form

$$\begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} + \begin{bmatrix} \alpha C & 0 \\ B^T & -\hat{S} \end{bmatrix}^{-1} \left(\begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix} - \begin{bmatrix} A & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} \right), \quad (11.125)$$

where again C is an easily inverted approximation to A and now $\hat{S}$ is a similarly easy to invert approximation to either $B^T A^{-1} B$ or $B^T C^{-1} B$. While Uzawa iterations are less expensive than Braess–Sarazin iterations, they are also typically less effective. Whether Uzawa relaxation is competitive for a particular problem depends greatly on details of the problem and its discretization, as well as optimization of the underlying relaxation parameters; however, it is often seen that the added flexibility in inexact Braess–Sarazin methods pays off in improved performance relative to cost over Uzawa.

3. Distributed relaxation methods [50] rely on approximating the continuum relationship that $\Delta \nabla = \nabla \Delta$ at the discrete level (where we recognize the difference between the *vector* Laplacian at left and the scalar Laplacian at right). At the discrete level, we write this as $AB \approx BA_p$, where A_p is the discretization of the Laplace operator on the pressure space (for the case of the Stokes equations). If this holds, then we would have the basis for an alternative block LU factorization (with nonunit U factor) to that in equation (11.121a), writing

$$\begin{bmatrix} A & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} I & B \\ 0 & -A_p \end{bmatrix} = \begin{bmatrix} A & 0 \\ B^T & B^T B \end{bmatrix}. \quad (11.126)$$

Distributive relaxation approaches, thus, consist of a stationary iteration given by

$$\begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} + \begin{bmatrix} I & B \\ 0 & -A_p \end{bmatrix} \begin{bmatrix} \alpha C & 0 \\ B^T & \widehat{B^T B} \end{bmatrix}^{-1} \left(\begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix} - \begin{bmatrix} A & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} \right), \quad (11.127)$$

where C is an easy to invert approximation to A and $\widehat{B^T B}$ is an easy to invert approximation to $B^T B$. Originally proposed for finite-difference discretizations of the Stokes equations (where the discrete relationship $AB \approx BA_p$ holds away from boundaries), distributive relaxation has not been widely used for finite-element discretization outside of the context of Maxwell's equations, where the use of discrete exact sequences allows it to be effective [135, 136], although some investigation has been done [70, 230].

4. Vanka relaxation techniques [224] are, in contrast to the above, based on domain decomposition ideas rather than block factorizations. While the stationary iterations for a Vanka technique are exactly those of overlapping (single-level) additive and multiplicative Schwarz from Section 11.1, what is special in a Vanka relaxation scheme is the choice of subdomain decomposition. Here, the decomposition must be done in a way that is compatible with the structure of the discretization. An *algebraic* Vanka methodology is naturally defined by choosing the subdomains over the velocity degrees of freedom, Ω_i , such that

$$\Omega_i = \{j \mid b_{j,i} \neq 0\}. \quad (11.128)$$

That is, the i th subdomain is composed of the sets of velocity degrees of freedom associated with nonzero entries in the i th row of B^T along with the i th pressure degree

of freedom. Note that this naturally leads to overlap in standard discretizations of the Stokes equations, where a single velocity degree of freedom is connected to multiple pressure degrees of freedom. A *geometric* Vanka methodology is also possible, where the subdomains are constructed based on connectivity in the mesh. For the Taylor–Hood discretization of the Stokes equations, this leads to subdomains consisting of a single (nodal) pressure degree of freedom and all of the velocity degrees of freedom on elements adjacent to the node; depending on the treatment of coincidental zeros in the discrete gradient operator, this may coincide with the algebraic definition above. In recent years, Vanka relaxation techniques have been successfully applied in many contexts and, as a result, they are widely used in many applications.

All of these relaxation schemes have been successfully applied to (and analyzed for) finite-difference discretizations of the Stokes equations, where they were originally considered. Application of these methodologies to other problems is an active area of research in multigrid methods, with a growing list of successes.

11.4.3 ■ Time-dependent problems

In this section, we consider discretization in space and time, where the time discretization is achieved using the backward differentiation (BDF) or diagonally implicit Runge–Kutta (DIRK) methods introduced in Section 9.6. Extending these techniques to general implicit Runge–Kutta-type discretizations is possible but generally much more complex and a subject of much recent research [222, 165, 184, 203, 1].

Considering, first, the time-dependent Stokes equations in equation (11.119), we ask how the above preconditioners extend from steady-state to time-dependent linear problems. For DIRK- or BDF-type discretizations, we now have that A is a discretization of $I - ch_t\Delta$, a shifted Laplacian operator, where c is a constant dependent on the discretization scheme ($c = 2/3$ for BDF2, as in equation (9.193), or $c = a_{i,i}$, the diagonal entries of the Butcher tableau for DIRK schemes). Clearly, when h_t is sufficiently small, this is dominated by the identity term, which leads to a Schur complement that is closer to a discretized diffusion operator than a mass matrix. When h_t is large, on the other hand, the original approximation of the Schur complement by the mass matrix seems acceptable. This intuition was formalized in [166], where it was shown that a good block-diagonal preconditioner arises from applying multigrid to A for the velocity update and applying multigrid to the discretized Laplacian on the pressure space, shifted by ch_t times the mass matrix, for the pressure update. A similar shift works for distributed relaxation in multigrid, while the other multigrid relaxation schemes require no significant changes in their definition, since they already compute an approximate Schur complement depending on the values in A (for Braess–Sarazin and Uzawa) or do not need one (for Vanka).

For nonlinear problems, like the Navier–Stokes equations in equation (11.120), we have an additional complication in the need to linearize the equations at every timestep. For block-factorization preconditioners, substantial work has been devoted to deriving robust Schur complement approximations in the setting of both Newton and Picard linearizations (cf. [92]). Common approaches are known as “pressure convection-diffusion” (PCD) [200] and least-squares commutator (LSC) [91], relying on approximating an operator, A_p , such that $AB \approx BA_p$. The PCD approach takes A_p to be a discretized convection-diffusion operator similar to the vector convection-diffusion operator that arises in the velocity block, while the LSC approach computes A_p by approximately solving the least-squares problem of minimizing $\|AB - BA_p\|$. A third block-factorization approach is based on the idea of solving an equivalent system of equations

to equation (11.117), written as

$$\begin{bmatrix} A + \gamma B^T M_p^{-1} B & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix}, \quad (11.129)$$

and applying block factorization ideas to this system [40, 98]. This results in a much more difficult block to approximately invert for the velocity update, but a much simpler approximation of the Schur complement, again as a scaled mass matrix. Robust solvers for the velocity block, however, are well known [196], leading to a very effective preconditioning framework for difficult nonlinear problems. Once again, similar techniques are used to extend relaxation schemes for monolithic multigrid methods, and these are also well studied [141].

A third family of techniques for the Navier–Stokes equations are known as split-step or pressure-correction schemes and date back to some of the earliest work on numerical methods for fluid flow [71, 213]. The simplest forms of these schemes take advantage of explicit timestepping for the nonlinear operator in the velocity block coupled with implicit timestepping to advance the pressure, which is necessary because there is no time derivative of pressure in the equations. The simplest approach involves splitting the time derivative of equation (11.120a) into two pieces and solving

$$(\tilde{\mathbf{u}} - \mathbf{u}_\ell) + ch_t (Re(\mathbf{u}_\ell \cdot \nabla \mathbf{u}_\ell) - \Delta \mathbf{u}_\ell) = \mathbf{f}_\ell, \quad (11.130a)$$

$$(\mathbf{u}_{\ell+1} - \tilde{\mathbf{u}}) + ch_t \nabla \mathbf{p}_{\ell+1} = \mathbf{0}, \quad (11.130b)$$

$$\nabla \cdot \mathbf{u}^{\ell+1} = \mathbf{0}, \quad (11.130c)$$

where we ignore the details of spatial discretization by writing the continuum operators. Equation (11.130a) is readily solved for $\tilde{\mathbf{u}}$, accounting for the dominant terms in the momentum equation. To solve for $\mathbf{u}_{\ell+1}$ and $\mathbf{p}_{\ell+1}$, we solve equation (11.130b) for $\mathbf{u}_{\ell+1}$, noting that this depends on $\mathbf{p}_{\ell+1}$, and substitute this into equation (11.130c). This yields a so-called *pressure Poisson equation*,

$$-ch_t \nabla \cdot \nabla \mathbf{p}_{\ell+1} = -\nabla \cdot \tilde{\mathbf{u}}, \quad (11.131)$$

that we can solve for $\mathbf{p}_{\ell+1}$ using domain decomposition or multigrid, or even the fast Poisson solvers from Section 10.6. Once $\mathbf{p}_{\ell+1}$ is known, we again have an explicit update to compute $\mathbf{u}_{\ell+1}$ from it and $\tilde{\mathbf{u}}$. These techniques have many advantages over fully implicit approaches, but they come at a cost of limiting accuracy, and much work has gone into achieving higher-accuracy split-step schemes for Navier–Stokes [221, 116].

11.5 ■ Some further thoughts

Domain decomposition and multigrid methods are among the most commonly used linear solvers in modern scientific computing, finding applications in many areas of computational fluid and solid mechanics, electromagnetics, quantum field theory, finance, and more. As such, there is a wide array of modern textbooks and research monographs that describe these families of algorithms and their usage for many problems. For domain decomposition methods, we point to the book *An Introduction to Domain Decomposition Methods* by Dolean, Jolivet, and Nataf [87], along with classic texts such as *Domain Decomposition Methods for Partial Differential Equations* by Quarteroni and Valli [182] and *Domain Decomposition Methods—Algorithms and Theory* by Toselli and Widlund [216]. For multigrid methods, the text *A Multigrid Tutorial* [58] by Briggs, Henson, and McCormick offers a concise introduction to the methods and is nicely complemented by the more comprehensive monograph *Multigrid* by Trottenberg, Oosterlee, and Schüller [219]. A long-time classic in this field is the “1984 guide” to multigrid by Brandt, which was reprinted with some revisions [51]. While the treatment in this chapter is rather brief,

we also point out that much of the active research literature in both areas is approachable with this level of understanding, as are papers in the regular proceedings of the Domain Decomposition Conference series, or the many proceedings and special issues arising from the Copper Mountain Conference series on Multigrid and Iterative Methods or the International Multigrid Conferences.

Since both multigrid and domain decomposition can provide optimal preconditioners for wide classes of PDEs, a natural question that arises is which should be used for a particular problem. Unfortunately, this is not a question with an “easy” solution, since the answer depends on many factors, including the problem itself, the domain (both whether it is structured or unstructured and whether the problem is in two or three dimensions), the discretization scheme, the size of the discrete system, and the computer architecture on which we intend to run the simulation. A comprehensive answer is, thus, elusive, particularly given the ever-evolving state of the literature related to both methodologies.

It is important to recognize that the major factor in determining into which of these classes a particular discretization of a particular problem falls is one of effort, rather than one of possibility. There are no known problems for which either of multigrid or domain decomposition is known to be guaranteed to work poorly. This is largely due to the vast array of possible methods in both classes, making broad proofs of infeasibility impossible. Moreover, for essentially every problem seriously attempted with either family of approach, good results have (eventually) been found. Further, when both work well, they often give comparable performance when similarly optimized, with the choice of which method is preferable depending on the details of their implementation, the problem and its discretization, and the computational hardware available.

A more abstract mathematical comparison between the two families of approaches is found by considering the balancing preconditioner given in equation (11.53b), whose error-propagation form is written as

$$\mathbf{e}^{(k+1)} = \left(I - R_0^T (R_0 A R_0^T)^{-1} R_0 A \right) (I - A_D^{-1} A) \left(I - R_0^T (R_0 A R_0^T)^{-1} R_0 A \right) \mathbf{e}^{(k)}. \quad (11.132)$$

For comparison, we write the error-propagation form of the two-grid algorithm given in Algorithm 11.1 as

$$\mathbf{e}^{(k+1)} = (I - M_2 A) \left(I - P (A^c)^{-1} R A \right) (I - M_1 A). \quad (11.133)$$

Restricting the two-grid algorithm to a Galerkin formulation, with $R = P^T$ and $A^c = P^T A P$, gives

$$\mathbf{e}^{(k+1)} = (I - M_2 A) \left(I - P (P^T A P)^{-1} P^T A \right) (I - M_1 A) \mathbf{e}^{(k)} \quad (11.134a)$$

$$= (I - M_2 A) T (I - M_1 A) \mathbf{e}^{(k)}, \quad (11.134b)$$

where we denote the coarse grid correction by $T = I - P (P^T A P)^{-1} P^T A$. Now, we take advantage of a useful property of the coarse-grid correction operator when we use the Galerkin formulation that

$$I - P (P^T A P)^{-1} P^T A = \left(I - P (P^T A P)^{-1} P^T A \right) \left(I - P (P^T A P)^{-1} P^T A \right), \quad (11.135)$$

or simply $T = T^2$. This property (which is easily verified by direct computation) establishes that the two-grid operator is a *projection operator*, which is useful in more advanced multigrid theory. For now, we write the two-grid error-propagation operator as

$$I - M_{TG} A = (I - M_2 A) T^2 (I - M_1 A). \quad (11.136)$$

Next, we consider the spectrum of $I - M_{TG}A$ and recognize this operator as a product of two parts: $(I - M_2A)T$ and $T(I - M_1A)$. The significance of this is that the eigenvalues of a product of matrices are independent of the order of the product. Thus,

$$\sigma(I - M_{TG}A) = \sigma((I - M_2A)T^2(I - M_1A)) \quad (11.137a)$$

$$= \sigma(T(I - M_1A)(I - M_2A)T) \quad (11.137b)$$

$$= \sigma(T(I - (M_1 + M_2 - M_1AM_2)A)T). \quad (11.137c)$$

We observe a similarity, then, between the error-propagation operator for the balancing preconditioner and this last form. Matching notation, we see that the balancing preconditioner and the two-grid method have the same spectrum when $P = R_0^T$ and $A_D^{-1} = M_1 + M_2 - M_1AM_2$. The first condition (that $P = R_0^T$) simply requires that the two methods receive a correction from the same coarse space. The second, however, is unlikely to be true for the natural choices of multigrid relaxation schemes (defining M_1 and M_2) and additive (or other) Schwarz methods (defining typical choices of A_D), outside of the trivial case where $M_1 = A_D^{-1}$ and $M_2 = 0$. Indeed, even in practice it would be unusual to run multigrid and the balancing preconditioner in a context where $P = R_0^T$, since a typical choice for R_0 in the domain decomposition context is an $s \times n$ matrix with unit entries marking when point j is in subdomain i , while multigrid would naturally choose P to be $n \times (n/\alpha)$ for fixed coarsening factor, α , with entries corresponding to a geometric interpolation (or similar) operator.

In many ways, the key difference between multigrid and domain decomposition methods is, then, a difference in philosophy: domain decomposition typically relies on having a strong fine-grid preconditioning operator to make up for a weak coarse-grid correction process. Multigrid, in contrast, generally relies on having a strong coarse-grid correction term to make up for a weak fine-grid relaxation operator. Both viewpoints are, clearly, possible and valid, but they inherently yield different classes of algorithms. These differences also explain the regimes where each class of algorithms is most attractive. For domain decomposition approaches, the use of highly accurate subdomain solves makes these algorithms attractive for moderate-scale parallelism, up to a few tens or hundreds of processes, where small coarse-grid problems are efficiently gathered and solved. In contrast, parallel multigrid methods require more communication, but this pays off for very large problems, where an exact coarse-grid solve (as in domain decomposition) would be prohibitively expensive or inefficient. This notwithstanding, both families of approaches have been scaled to parallelism on the scale of thousands of processors (and, in some cases, tens or hundreds of thousands), and both have been adapted to modern accelerated architectures, such as GPUs. Quite simply, for any given discretization of a given problem, the best choice of algorithm for its solution depends most strongly on which community of researchers has put in the most effort.

Along these lines, it is a daunting task to go from the level of understanding of the methods proposed in this chapter to implementing and using the methods to their full power on discretizations of complicated systems of PDEs on modern parallel machines. Thankfully, due to a wide community of software developers and maintainers, there is little need to “hand-code” new implementations of many standard methods. The widely used Portable, Extensible Toolkit for Scientific Computation (PETSc) [31, 30, 32] provides high-quality parallel implementations of many linear and nonlinear solvers, including domain decomposition and multigrid methods. More specialized methods are also available in other packages, such as hypre [139, 97], which contains excellent parallel C/F AMG solvers [129], and Trilinos [130], which provides excellent smoothed aggregation solvers. If suitably configured, these packages also work together, with the ability to use solvers from hypre or Trilinos via their interfaces to PETSc. While the learning curve to use these packages can be steep, there are many excellent examples and tutorials available, including a recent book [60]. Further power is provided when these tools are tightly

integrated with a discretization framework, as is the case for Firedrake [122]. When used in this way, we have many powerful methods at our fingertips, at a tiny fraction of the development time to produce new code for established approaches.

Much of the focus of this book is on numerical techniques for *linear* PDEs; as such, this chapter and Chapter 7 are primarily devoted to solution techniques for *linear* systems of equations. However, many real-world applications are governed by nonlinear systems of PDEs, such as the Navier–Stokes equations and magnetohydrodynamics (MHD). This is discussed briefly in Section 9.6 with Newton–Krylov methods, where a Newton iteration is used to linearize a given PDE and then a standard (preconditioned) Krylov method, such as FGMRES, is used to solve the resulting linearized systems. However, this raises the question of whether multilevel ideas are directly applicable in a nonlinear setting. The full approximation scheme (FAS) [51] (originally known as “full approximation storage” multigrid) addresses coarse-grid correction in a nonlinear setting by reformulating the residual equation on the coarse grid. Consider a nonlinear algebraic problem of the form $\mathcal{A}(\mathbf{u}) = \mathbf{f}$, with solution $\mathbf{u}$. If $\mathbf{u}_k$ is an approximation to $\mathbf{u}$, then the residual satisfies

$$\mathbf{r} = \mathbf{f} - \mathcal{A}(\mathbf{u}_k) = \mathcal{A}(\mathbf{u}) - \mathcal{A}(\mathbf{u}_k) = \mathcal{A}(\mathbf{u}_k + \mathbf{e}_k) - \mathcal{A}(\mathbf{u}_k), \quad (11.138)$$

which we rewrite as

$$\mathcal{A}(\mathbf{u}_k + \mathbf{e}_k) = \mathcal{A}(\mathbf{u}_k) + (\mathbf{f} - \mathcal{A}(\mathbf{u}_k)). \quad (11.139)$$

With this view of the residual equation, FAS forms a coarse-level residual problem to solve for a coarse-level representation of $\mathbf{u}$, $\mathbf{u}_k^c$ by restricting each term and writing

$$\mathcal{A}^c(\mathbf{u}_k^c) = \mathcal{A}^c(P^T \mathbf{u}_k) + P^T (\mathbf{f} - \mathcal{A}(\mathbf{u}_k)). \quad (11.140)$$

From this, the coarse-level error is approximated as $\mathbf{e}_k^c = \mathbf{u}_k^c - P^T \mathbf{u}_k$. When this coarse-grid correction approach is complemented with nonlinear relaxation (such as the nonlinear generalizations of the Jacobi or Gauss–Seidel iterations), FAS multigrid can be a very effective nonlinear solver. Alternatively, akin to the Newton–Krylov methods discussed in Section 9.6, a Newton-multigrid approach, where Newton’s method is used to linearize the problem followed by standard multigrid iteration, has successfully been applied to many problems [58], or multigrid methods can be used as the preconditioners in standard Newton–Krylov approaches. One benefit of multilevel solvers comes in addressing the common issue of finding “good” initial guesses for Newton’s method. Here, ideas such as nested iteration, where the system is solved on a coarser grid at reduced cost to yield good initial guesses for the solution on finer grids, have been explored (see, for example [75, 5], in the context of least-squares finite-element methods applied to nonlinear problems). Similarly, continuation in a problem parameter [10] often leads to faster time to solution for complex problems because better starting guesses are used. For any particular problem, some or all of these approaches may be competitive. Thus, choosing suitable options is highly dependent on the application at hand.

Finally, we underscore that there is certainly no stagnation in the research area of developing new approaches to solving linear systems. Recent years have seen substantial work devoted to combinatorial preconditioning, following the pioneering work of Spielman and Teng [204]. While these approaches have not fully gained traction in practical scientific computing applications, their attractive theoretical properties suggest that they may become an important class of methods. More recently, a broader field of randomized numerical linear algebra [167] has emerged that may yet develop important and useful methods. While the focus of this book has been on discretization approaches that lead to sparse linear systems and, consequently, we have focused on preconditioning approaches suitable in this setting, other discretization approaches do result in dense matrices with different properties, where different solution techniques are appropriate, such as those based on hierarchical matrix representations [120, 46].

Bibliography

- [1] R. ABU-LABDEH, S. MACLACHLAN, AND P. E. FARRELL, *Monolithic multigrid for implicit Runge–Kutta discretizations of incompressible fluid flow*, Journal of Computational Physics, 478 (2023), p. 111961, <https://doi.org/10.1016/j.jcp.2023.111961>. (Cited in §11.4.3 (p. 558)).
- [2] R. A. ADAMS AND J. J. F. FOURNIER, *Sobolev Spaces*, vol. 140 of Pure and Applied Mathematics, Elsevier/Academic Press, second ed., 2003, <https://www.sciencedirect.com/bookseries/pure-and-applied-mathematics/vol/140>. (Cited in §10.6.1 (p. 489)).
- [3] J. H. ADLER, X. HU, AND L. T. ZIKATANOV, *Robust solvers for Maxwell’s equations with dissipative boundary conditions*, SIAM Journal on Scientific Computing, 39 (2017), pp. S3–S23, <https://doi.org/10.1137/16M1073339>. (Cited in §10.2.1 (p. 444), §10.2.1 (p. 444)).
- [4] J. H. ADLER, X. HU, AND L. T. ZIKATANOV, *HAZmath: A simple finite element, graph, and solver library*, 2023, <https://github.com/HAZmathTeam/hazmath>. (Cited in §8.8 (p. 355)).
- [5] J. H. ADLER, T. A. MANTEUFFEL, S. F. MCCORMICK, J. W. NOLTING, J. W. RUGE, AND L. TANG, *Efficiency based adaptive local refinement for first-order system least-squares formulations*, SIAM Journal on Scientific Computing, 33 (2011), pp. 1–24, <https://doi.org/10.1137/100786897>. (Cited in §10.4.2 (p. 476), §11.5 (p. 562)).
- [6] M. AINSWORTH AND J. T. ODEN, *A Posteriori Error Estimation in Finite Element Analysis*, Pure and Applied Mathematics, John Wiley & Sons, 2000, <https://doi.org/10.1002/9781118032824>. (Cited in §8.6 (p. 349), §8.6.1 (p. 350)).
- [7] D. M. ALBER AND L. N. OLSON, *Parallel coarse-grid selection*, Numerical Linear Algebra with Applications, 14 (2007), pp. 611–643, <https://doi.org/10.1002/nla.541>. (Cited in §11.3.3 (p. 548)).
- [8] R. E. ALCOUFFE, A. BRANDT, J. E. DENDY, JR., AND J. W. PAINTER, *The multi-grid method for the diffusion equation with strongly discontinuous coefficients*, SIAM Journal on Scientific and Statistical Computing, 2 (1981), pp. 430–454, <https://doi.org/10.1137/0902035>. (Cited in §11.3.1 (p. 539)).
- [9] R. ALEXANDER, *Diagonally implicit Runge–Kutta methods for stiff O.D.E.’s*, SIAM Journal on Numerical Analysis, 14 (1977), pp. 1006–1021, <https://doi.org/10.1137/0714068>. (Cited in §9.6.2 (p. 415)).

- [10] E. L. ALLGOWER AND K. GEORG, *Introduction to Numerical Continuation Methods*, vol. 45 of Classics in Applied Mathematics, SIAM, 2003, <https://doi.org/10.1137/1.9780898719154>. (Cited in §11.5 (p. 562)).
- [11] M. S. ALNÆS, J. BLECHTA, J. HAKE, A. JOHANSSON, B. KEHLET, A. LOGG, C. RICHARDSON, J. RING, M. E. ROGNES, AND G. N. WELLS, *The FEniCS project version 1.5*, Archive of Numerical Software, 3 (2015), pp. 9–23, <https://doi.org/10.11588/ans.2015.100.20553>. (Cited in §8.8 (p. 355)).
- [12] R. ANDERSON, J. ANDREJ, A. BARKER, J. BRAMWELL, J.-S. CAMIER, J. CERVENY, V. DOBREV, Y. DUDOUIT, A. FISHER, T. KOLEV, W. PAZNER, M. STOWELL, V. TOMOV, I. AKKERMAN, J. DAHM, D. MEDINA, AND S. ZAMPINI, *MFEM: A modular finite element methods library*, Computers & Mathematics with Applications, 81 (2021), pp. 42–74, <https://doi.org/10.1016/j.camwa.2020.06.009>. (Cited in §8.8 (p. 355)).
- [13] J. H. ARGYRIS, I. FRIED, AND D. W. SCHARPF, *The TUBA family of plate elements for the matrix displacement method*, The Aeronautical Journal, 72 (1968), pp. 701–709, <https://doi.org/10.1017/S000192400008489X>. (Cited in §10.2.2 (p. 448)).
- [14] R. J. ARMS, L. D. GATES, AND B. ZONDEK, *A method of block iteration*, Journal of the Society for Industrial and Applied Mathematics, 4 (1956), pp. 220–229, <https://doi.org/10.1137/0104012>. (Cited in §7.5 (p. 256)).
- [15] D. ARNDT, W. BANGERTH, B. BLAIS, M. FEHLING, R. GASSMÖLLER, T. HEISTER, L. HELTAI, U. KÖCHER, M. KRONBICHLER, M. MAIER, P. MUNCH, J.-P. PELTERET, S. PROELL, K. SIMON, B. TURCK SIN, D. WELLS, AND J. ZHANG, *The deal.II library, version 9.3*, Journal of Numerical Mathematics, 29 (2021), pp. 171–186, <https://doi.org/10.1515/jnma-2021-0081>. (Cited in §8.8 (p. 355)).
- [16] D. N. ARNOLD, *Finite Element Exterior Calculus*, vol. 93 of CBMS-NSF Regional Conference Series in Applied Mathematics, SIAM, 2018, <https://doi.org/10.1137/1.9781611975543>. (Cited in §10.2.1 (p. 436), §10.2.1 (p. 444)).
- [17] D. N. ARNOLD, F. BREZZI, B. COCKBURN, AND L. D. MARINI, *Unified analysis of discontinuous Galerkin methods for elliptic problems*, SIAM Journal on Numerical Analysis, 39 (2002), pp. 1749–1779, <https://doi.org/10.1137/S0036142901384162>. (Cited in §10.5 (p. 482), §10.5 (p. 485)).
- [18] D. N. ARNOLD, R. S. FALK, AND R. WINTHER, *Finite element exterior calculus, homological techniques, and applications*, Acta Numerica, 15 (2006), pp. 1–155, <https://doi.org/10.1017/S0962492906210018>. (Cited in §10.2.1 (p. 436), §10.2.1 (p. 444)).
- [19] D. N. ARNOLD AND A. LOGG, *Periodic table of the finite elements*, SIAM News, 47 (2014), pp. 1, 8–9, https://www.siam.org/media/ptfnj2mx/sn_november2014.pdf. (Cited in §10.7 (p. 495), §10.7 (p. 496)).
- [20] U. M. ASCHER AND C. GREIF, *A First Course in Numerical Methods*, vol. 7 of Computational Science & Engineering, SIAM, 2011, <https://doi.org/10.1137/9780898719987>. (Cited in §2.7 (p. 45), §5.5 (p. 173), §7.1 (p. 228), §7.7 (p. 265), §7.7 (p. 265)).

- [21] U. M. ASCHER, S. J. RUUTH, AND R. J. SPITERI, *Implicit-explicit Runge-Kutta methods for time-dependent partial differential equations*, Applied Numerical Mathematics, 25 (1997), pp. 151–167, [https://doi.org/10.1016/S0168-9274\(97\)00056-1](https://doi.org/10.1016/S0168-9274(97)00056-1). (Cited in §9.6.3 (p. 418)).
- [22] K. ATKINSON, *An Introduction to Numerical Analysis*, Wiley, second ed., 1991. (Cited in §2.7 (p. 45), §5.5 (p. 173), §7.7 (p. 265)).
- [23] K. ATKINSON AND W. HAN, *Theoretical Numerical Analysis: A Functional Analysis Framework*, vol. 39 of Texts in Applied Mathematics, Springer, third ed., 2009, <https://doi.org/10.1007/978-1-4419-0458-4>. (Cited in §8.1.1 (p. 303), §8.1.3 (p. 306), §8.2.1 (p. 316), §10.4 (p. 470)).
- [24] O. AXELSSON, *Iterative Solution Methods*, Cambridge University Press, 1994, <https://doi.org/10.1017/CB09780511624100>. (Cited in §7.7.1 (p. 269), §7.7.1 (p. 269)).
- [25] O. AXELSSON AND P. S. VASSILEVSKI, *Algebraic multilevel preconditioning methods*. I, Numerische Mathematik, 56 (1989), pp. 157–177, <https://doi.org/10.1007/BF01409783>. (Cited in §11.2.3 (p. 527)).
- [26] O. AXELSSON AND P. S. VASSILEVSKI, *Algebraic multilevel preconditioning methods*, II, SIAM Journal on Numerical Analysis, 27 (1990), pp. 1569–1590, <https://doi.org/10.1137/0727092>. (Cited in §11.2.3 (p. 527)).
- [27] I. BABUŠKA, *Error-bounds for finite element method*, Numerische Mathematik, 16 (1971), pp. 322–333, <https://doi.org/10.1007/BF02165003>. (Cited in §10.3.1 (p. 453)).
- [28] I. BABUŠKA AND W. C. RHEINBOLDT, *Error estimates for adaptive finite element computations*, SIAM Journal on Numerical Analysis, 15 (1978), pp. 736–754, <https://doi.org/10.1137/0715049>. (Cited in §8.6 (p. 349)).
- [29] I. BABUŠKA AND T. STROUBOULIS, *The Finite Element Method and its Reliability*, Oxford University Press, 2001, <https://doi.org/10.1093/oso/9780198502760.001.0001>. (Cited in §8.6 (p. 349)).
- [30] S. BALAY, S. ABHYANKAR, M. F. ADAMS, S. BENSON, J. BROWN, P. BRUNE, K. BUSCHELMAN, E. CONSTANTINESCU, L. DALCIN, A. DENER, V. EIJKHOUT, W. D. GROPP, V. HAPLA, T. ISAAC, P. JOLIVET, D. KARPEEV, D. KAUSHIK, M. G. KNEPLEY, F. KONG, S. KRUGER, D. A. MAY, L. C. MCINNES, R. T. MILLS, L. MITCHELL, T. MUNSON, J. E. ROMAN, K. RUPP, P. SANAN, J. SARICH, B. F. SMITH, S. ZAMPINI, H. ZHANG, H. ZHANG, AND J. ZHANG, *PETSc/TAO users manual*, Tech. Report ANL-21/39 - Revision 3.17, Argonne National Laboratory, 2022. (Cited in §11.5 (p. 561)).
- [31] S. BALAY, S. ABHYANKAR, M. F. ADAMS, S. BENSON, J. BROWN, P. BRUNE, K. BUSCHELMAN, E. M. CONSTANTINESCU, L. DALCIN, A. DENER, V. EIJKHOUT, W. D. GROPP, V. HAPLA, T. ISAAC, P. JOLIVET, D. KARPEEV, D. KAUSHIK, M. G. KNEPLEY, F. KONG, S. KRUGER, D. A. MAY, L. C. MCINNES, R. T. MILLS, L. MITCHELL, T. MUNSON, J. E. ROMAN, K. RUPP, P. SANAN, J. SARICH, B. F. SMITH, S. ZAMPINI, H. ZHANG, H. ZHANG, AND J. ZHANG, *PETSc Web page*, 2022, <https://petsc.org/>. (Cited in §11.5 (p. 561)).

- [32] S. BALAY, W. D. GROPP, L. C. MCINNES, AND B. F. SMITH, *Efficient management of parallelism in object oriented numerical software libraries*, in Modern Software Tools in Scientific Computing, E. Arge, A. M. Bruaset, and H. P. Langtangen, eds., Birkhäuser Boston Inc., 1997, pp. 163–202. (Cited in §11.5 (p. 561)).
- [33] W. R. BALL, *A Short Account of the History of Mathematics*, Dover Publications, 1960. (Cited in §1.4 (p. 16)).
- [34] R. BARRETT, M. BERRY, T. F. CHAN, J. DEMMEL, J. DONATO, J. DONGARRA, V. EIJKHOUT, R. POZO, C. ROMINE, AND H. VAN DER VORST, *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*, SIAM, 1994, <https://doi.org/10.1137/1.9781611971538>. (Cited in §7.12 (p. 297)).
- [35] G. BARTHOLOMEEUSEN, H. DE STERCK, AND G. SILLS, *Non-convex flux functions and compound shock waves in sediment beds*, in Hyperbolic Problems: Theory, Numerics, Applications, T. Y. Hou and E. Tadmor, eds., Springer, 2003, pp. 347–356, https://doi.org/10.1007/978-3-642-55711-8_31. (Cited in §6.6 (p. 219)).
- [36] P. BASTIAN, M. BLATT, A. DEDNER, N.-A. DREIER, C. ENGWER, R. FRITZE, C. GRÄSER, C. GRÜNIGER, D. KEMPF, R. KLÖFKORN, M. OHLBERGER, AND O. SANDER, *The DUNE framework: Basic concepts and recent developments*, Computers and Mathematics with Applications, 81 (2021), pp. 75–112, <https://doi.org/10.1016/j.camwa.2020.06.007>. (Cited in §8.8 (p. 355)).
- [37] L. BEIRÃO DA VEIGA, F. BREZZI, A. CANGIANI, G. MANZINI, L. D. MARINI, AND A. RUSSO, *Basic principles of virtual element methods*, Mathematical Models and Methods in Applied Sciences, 23 (2013), pp. 199–214, <https://doi.org/10.1142/S0218202512500492>. (Cited in §10.7 (p. 495)).
- [38] N. BELL, L. N. OLSON, J. SCHRODER, AND B. SOUTHWORTH, *PyAMG: Algebraic multigrid solvers in Python*, Journal of Open Source Software, 8 (2023), p. 5495, <https://doi.org/10.21105/joss.05495>. (Cited in §11.3.3 (p. 553)).
- [39] T. BELYTSCHKO AND T. BLACK, *Elastic crack growth in finite elements with minimal remeshing*, International Journal for Numerical Methods in Engineering, 45 (1999), pp. 601–620, [https://doi.org/10.1002/\(SICI\)1097-0207\(19990620\)45:5<601::AID-NME>3.0.CO;2-S](https://doi.org/10.1002/(SICI)1097-0207(19990620)45:5<601::AID-NME>3.0.CO;2-S). (Cited in §10.7 (p. 495)).
- [40] M. BENZI AND M. A. OLSHANSKII, *An augmented Lagrangian-based approach to the Oseen problem*, SIAM Journal on Scientific Computing, 28 (2006), pp. 2095–2113, <https://doi.org/10.1137/050646421>. (Cited in §11.4.3 (p. 559)).
- [41] A. BERMAN AND R. J. PLEMMONS, *Nonnegative Matrices in the Mathematical Sciences*, vol. 9 of Classics in Applied Mathematics, SIAM, 1994, <https://doi.org/10.1137/1.9781611971262>. Corrected republication of the work first published in 1979 by Academic Press. (Cited in §7.4 (p. 251), §7.6 (p. 262)).
- [42] M. BERNDT, T. A. MANTEUFFEL, AND S. F. MCCORMICK, *Local error estimates and adaptive refinement for first-order system least squares (FOSLS)*, ETNA: Electronic Transactions on Numerical Analysis, 6 (1997), pp. 35–43. (Cited in §10.4.2 (p. 476)).
- [43] D. BOFFI, F. BREZZI, AND M. FORTIN, *Mixed Finite Element Methods and Applications*, vol. 44 of Springer Series in Computational Mathematics, Springer, 2013, <https://doi.org/10.1007/978-3-642-36519-5>. (Cited in §8.8 (p. 355), §10.3.1 (p. 454), §10.3.2 (p. 456), §10.3.2 (p. 457), §10.3.3 (p. 461), §10.3.4 (p. 466)).

- [44] D. BOFFI AND C. LOVADINA, *Analysis of new augmented Lagrangian formulations for mixed finite element schemes*, *Numerische Mathematik*, 75 (1997), p. 405–419, <https://doi.org/10.1007/s002110050246>. (Cited in §10.3.4 (p. 467)).
- [45] F. K. BOGNER, R. L. FOX, AND L. A. SCHMIT, *The generation of interelement compatible stiffness and mass matrices by the use of interpolation formulas*, in *Proceedings of the Conference on Matrix Methods in Structural Mechanics*, Wright-Patterson Air Force Base, Ohio, 1966, pp. 395–443. (Cited in §10.2.2 (p. 447)).
- [46] S. BÖRM, *Efficient Numerical Methods for Non-Local Operators: H^2 -Matrix Compression, Algorithms and Analysis*, vol. 14 of EMS Tracts in Mathematics, European Mathematical Society, 2010, <https://doi.org/10.4171/091>. (Cited in §11.5 (p. 562)).
- [47] J. P. BOYD, *Chebyshev and Fourier Spectral Methods*, Dover Books on Mathematics, Dover Publications, second ed., 2001. (Cited in §4.5 (p. 126)).
- [48] D. BRAESS, *Finite Elements: Theory, Fast Solvers, and Applications in Solid Mechanics*, Cambridge University Press, third ed., 2007, <https://doi.org/10.1017/CB09780511618635>. (Cited in §1.4 (p. 17), §8.2.2 (p. 318), §8.2.2 (p. 321), §8.8 (p. 355), §10.3.4 (p. 465)).
- [49] D. BRAESS AND R. SARAZIN, *An efficient smoother for the Stokes problem*, *Applied Numerical Mathematics*, 23 (1997), pp. 3–19, [https://doi.org/10.1016/S0168-9274\(96\)00059-1](https://doi.org/10.1016/S0168-9274(96)00059-1). (Cited in §11.4.2 (p. 556)).
- [50] A. BRANDT AND N. DINAR, *Multigrid solutions to elliptic flow problems*, in *Numerical Methods for Partial Differential Equations*, S. V. Parter, ed., Academic Press, 1979, pp. 53–147, <https://doi.org/10.1016/B978-0-12-546050-7.50008-3>. (Cited in §11.4.2 (p. 557)).
- [51] A. BRANDT AND O. E. LIVNE, *Multigrid Techniques — 1984 Guide with Applications to Fluid Dynamics*, vol. 67 of *Classics in Applied Mathematics*, SIAM, 2011, <https://doi.org/10.1137/1.9781611970753>. (Cited in §11.5 (p. 559), §11.5 (p. 562)).
- [52] S. C. BRENNER AND L. R. SCOTT, *The Mathematical Theory of Finite Element Methods*, Springer, 2008, <https://doi.org/10.1007/978-0-387-75934-0>. (Cited in Preface (p. xiv), §4.1 (p. 106), §4.2 (p. 109), §4.2 (p. 110), §4.2 (p. 110), §4.2 (p. 111), §4.2 (p. 111), §4.2 (p. 112), §4.3 (p. 114), §4.3 (p. 114), §8.2.2 (p. 320), §8.3 (p. 322), §8.3.1 (p. 326), §8.3.1 (p. 326), §8.3.1 (p. 327), §8.3.2 (p. 331), §8.8 (p. 355), §10.2.2 (p. 446), §10.3.1 (p. 454), §10.3.1 (p. 454), §10.3.2 (p. 457), §10.3.3 (p. 461), §11.1.3 (p. 512)).
- [53] M. BREZINA, R. FALGOUT, S. MACLACHLAN, T. MANTEUFFEL, S. MCCORMICK, AND J. RUGE, *Adaptive smoothed aggregation (α SA)*, *SIAM Journal on Scientific Computing*, 25 (2004), pp. 1896–1920, <https://doi.org/10.1137/S1064827502418598>. (Cited in §11.1.3 (p. 515), §11.3.2 (p. 543)).
- [54] H. BREZIS AND F. BROWDER, *Partial differential equations in the 20th century*, *Advances in Mathematics*, 135 (1998), pp. 76–144, <https://doi.org/10.1006/aima.1997.1713>. (Cited in §1.4 (p. 16)).
- [55] F. BREZZI, *On the existence, uniqueness and approximation of saddle-point problems arising from Lagrangian multipliers*, *ESAIM: Mathematical Modelling and Numerical Analysis - Modélisation Mathématique et Analyse Numérique*, 8 (1974), pp. 129–151, <http://eudml.org/doc/193255>. (Cited in §10.3.1 (p. 453), §10.3.3 (p. 463)).

- [56] F. BREZZI, J. DOUGLAS, JR., AND L. D. MARINI, *Two families of mixed finite elements for second order elliptic problems*, Numerische Mathematik, 47 (1985), pp. 217–235, <https://doi.org/10.1007/BF01389710>. (Cited in §10.2.1 (p. 439)).
- [57] F. BREZZI AND R. S. FALK, *Stability of higher-order Hood–Taylor methods*, SIAM Journal on Numerical Analysis, 28 (1991), pp. 581–590, <https://doi.org/10.1137/0728032>. (Cited in §10.3.4 (p. 467)).
- [58] W. L. BRIGGS, V. E. HENSON, AND S. F. MCCORMICK, *A Multigrid Tutorial*, SIAM, second ed., 2000, <https://doi.org/10.1137/1.9780898719505>. (Cited in §11.5 (p. 559), §11.5 (p. 562)).
- [59] P. N. BROWN AND H. F. WALKER, *GMRES on (nearly) singular systems*, SIAM Journal on Matrix Analysis and Applications, 18 (1997), pp. 37–51, <https://doi.org/10.1137/S0895479894262339>. (Cited in §7.8.1 (p. 274)).
- [60] E. BUELER, *PETSc for Partial Differential Equations: Numerical Solutions in C and Python*, vol. 31 of Software, Environments, and Tools, SIAM, 2020, <https://doi.org/10.1137/1.9781611976311>. (Cited in §11.5 (p. 561)).
- [61] E. BURMAN, S. CLAUS, P. HANSBO, M. G. LARSON, AND A. MASSING, *CutFEM: Discretizing geometry and partial differential equations*, International Journal for Numerical Methods in Engineering, 104 (2015), pp. 472–501, <https://doi.org/10.1002/nme.4823>. (Cited in §10.7 (p. 495)).
- [62] S. CAI, Z. MAO, Z. WANG, M. YIN, AND G. E. KARNIADAKIS, *Physics-informed neural networks (PINNs) for fluid mechanics: A review*, Acta Mechanica Sinica, 37 (2021), pp. 1727–1738, <https://doi.org/10.1007/s10409-021-01148-1>. (Cited in §10.7 (p. 496)).
- [63] X.-C. CAI, M. DRYJA, AND M. SARKIS, *Restricted additive Schwarz preconditioners with harmonic overlap for symmetric positive definite linear systems*, SIAM Journal on Numerical Analysis, 41 (2003), pp. 1209–1231, <https://doi.org/10.1137/S0036142901389621>. (Cited in §11.1.2 (p. 508)).
- [64] X.-C. CAI AND M. SARKIS, *A restricted additive Schwarz preconditioner for general sparse linear systems*, SIAM Journal on Scientific Computing, 21 (1999), pp. 792–797, <https://doi.org/10.1137/S106482759732678X>. (Cited in §11.1.2 (p. 507), §11.1.2 (p. 508)).
- [65] Z. CAI, J. CHEN, AND M. LIU, *Least-squares ReLU neural network (LSNN) method for linear advection-reaction equation*, Journal of Computational Physics, 443 (2021), p. 110514, <https://doi.org/10.1016/j.jcp.2021.110514>. (Cited in §10.7 (p. 496)).
- [66] Z. CAI, R. LAZAROV, T. A. MANTEUFFEL, AND S. F. MCCORMICK, *First-order system least squares for second-order partial differential equations: Part I*, SIAM Journal on Numerical Analysis, 31 (1994), pp. 1785–1799, <https://doi.org/10.1137/0731091>. (Cited in §10.4.1 (p. 474)).
- [67] C. CARSTENSEN, *A posteriori error estimate for the mixed finite element method*, Mathematics of Computation, 66 (1997), p. 465–476, <https://doi.org/10.1090/s0025-5718-97-00837-5>. (Cited in §8.6 (p. 349)).

- [68] L. CHEN, *iFEM: An integrated finite element methods package in MATLAB*, tech. report, University of California at Irvine, 2009, <https://github.com/lyc102/ifem>. (Cited in §8.8 (p. 355)).
- [69] L. CHEN, *A simple construction of a Fortin operator for the two dimensional Taylor–Hood element*, Computers & Mathematics with Applications, 68 (2014), pp. 1368–1373, <https://doi.org/10.1016/j.camwa.2014.09.003>. (Cited in §10.3.4 (p. 467)).
- [70] L. CHEN, X. HU, M. WANG, AND J. XU, *A multigrid solver based on distributive smoother and residual overweighting for Oseen problems*, Numerical Mathematics: Theory, Methods and Applications, 8 (2015), pp. 237–252, <https://doi.org/10.4208/nmtma.2015.w09si>. (Cited in §11.4.2 (p. 557)).
- [71] A. J. CHORIN, *Numerical solution of the Navier-Stokes equations*, Mathematics of Computation, 22 (1968), pp. 745–762, <https://doi.org/10.1090/S0025-5718-1968-0242392-2>. (Cited in §11.4.3 (p. 559)).
- [72] P. G. CIARLET, *The Finite Element Method for Elliptic Problems*, vol. 40 of Classics in Applied Mathematics, SIAM, 2002, <https://doi.org/10.1137/1.9780898719208>. (Cited in §8.8 (p. 355), §8.8 (p. 356)).
- [73] P. CLÉMENT, *Approximation by finite element functions using local regularization*, Revue française d’automatique, informatique, recherche opérationnelle. Analyse numérique, 9 (1975), pp. 77–84. (Cited in §10.3.4 (p. 465), §10.3.4 (p. 465)).
- [74] B. COCKBURN AND C.-W. SHU, *TVB Runge-Kutta local projection discontinuous Galerkin finite element method for conservation laws II: General framework*, Mathematics of Computation, 52 (1989), pp. 411–435, <https://doi.org/10.2307/2008474>. (Cited in §9.5.3 (p. 408), §9.5.3 (p. 409)).
- [75] A. L. CODD, T. A. MANTEUFFEL, AND S. F. MCCORMICK, *Multilevel first-order system least squares for nonlinear elliptic partial differential equations*, SIAM Journal on Numerical Analysis, 41 (2003), pp. 2197–2209, <https://doi.org/10.1137/S0036142902404406>. (Cited in §11.5 (p. 562)).
- [76] L. COLLATZ, *The Numerical Treatment of Differential Equations*, Die Grundlehren der mathematischen Wissenschaften, Band 60, Springer-Verlag, third ed., 1960. (Cited in §3.7 (p. 99)).
- [77] S. CUOMO, V. SCHIANO DI COLA, F. GIAMPAOLO, G. ROZZA, M. RAISSI, AND F. PICCIALLI, *Scientific machine learning through physics-informed neural networks: Where we are and what’s next*, Journal of Scientific Computing, 92 (2022), p. 88, <https://doi.org/10.1007/s10915-022-01939-z>. (Cited in §10.7 (p. 496)).
- [78] G. DAHLQUIST, *A special stability problem for linear multistep methods*, BIT Numerical Analysis, 3 (1963), pp. 27–43, <https://doi.org/10.1007/BF01963532>. (Cited in §9.6.1 (p. 414)).
- [79] H. DE STERCK, B. LOW, AND S. POEDTS, *Complex magnetohydrodynamic bow shock topology in field-aligned low- β flow around a perfectly conducting cylinder*, Physics of Plasmas, 5 (1998), pp. 4015–4027, <https://doi.org/10.1063/1.873124>. (Cited in §6.6 (p. 219)).

- [80] H. DE STERCK, T. A. MANTEUFFEL, S. F. MCCORMICK, AND L. OLSON, *Least-squares finite element methods and algebraic multigrid solvers for linear hyperbolic PDEs*, SIAM Journal on Scientific Computing, 26 (2004), pp. 31–54, <https://doi.org/10.1137/S106482750240858X>. (Cited in §10.7 (p. 496)).
- [81] H. DE STERCK, T. A. MANTEUFFEL, S. F. MCCORMICK, AND L. OLSON, *Numerical conservation properties of $H(\text{div})$ -conforming least-squares finite element methods for the Burgers equation*, SIAM Journal on Scientific Computing, 26 (2005), pp. 1573–1597, <https://doi.org/10.1137/S1064827503430758>. (Cited in §10.7 (p. 496)).
- [82] J. M. DEANG AND M. D. GUNZBURGER, *Issues related to least-squares finite element methods for the Stokes equations*, SIAM Journal on Scientific Computing, 20 (1998), pp. 878–906, <https://doi.org/10.1137/S1064827595294526>. (Cited in §10.4.3 (p. 477)).
- [83] M. DELFOUR, M. FORTIN, AND G. PAYR, *Finite-difference solutions of a non-linear Schrödinger equation*, Journal of Computational Physics, 44 (1981), pp. 277–288, [https://doi.org/10.1016/0021-9991\(81\)90052-8](https://doi.org/10.1016/0021-9991(81)90052-8). (Cited in §5.5 (p. 174)).
- [84] J. W. DEMMEL, *Applied Numerical Linear Algebra*, SIAM, 1997, <https://doi.org/10.1137/1.9781611971446>. (Cited in §7.12 (p. 297)).
- [85] J. E. DENDY, *Black box multigrid*, Journal of Computational Physics, 48 (1982), pp. 366–386, [https://doi.org/10.1016/0021-9991\(82\)90057-2](https://doi.org/10.1016/0021-9991(82)90057-2). (Cited in §11.3.1 (p. 539)).
- [86] M. O. DEVILLE, P. F. FISCHER, AND E. H. MUND, *High-Order Methods for Incompressible Fluid Flow*, Cambridge Monographs on Applied and Computational Mathematics, Cambridge University Press, 2002, <https://doi.org/10.1017/CB09780511546792>. (Cited in §1.4 (p. 17)).
- [87] V. DOLEAN, P. JOLIVET, AND F. NATAF, *An Introduction to Domain Decomposition Methods: Algorithms, Theory, and Parallel Implementation*, SIAM, 2015, <https://doi.org/10.1137/1.9781611974065>. (Cited in §11.1.3 (p. 515), §11.5 (p. 559)).
- [88] M. DRYJA AND O. B. WIDLUND, *Domain decomposition algorithms with small overlap*, SIAM Journal on Scientific Computing, 15 (1994), pp. 604–620, <https://doi.org/10.1137/0915040>. (Cited in §11.1.3 (p. 512)).
- [89] C. DUARTE, I. BABUŠKA, AND J. ODEN, *Generalized finite element methods for three-dimensional structural mechanics problems*, Computers and Structures, 77 (2000), pp. 215–232, [https://doi.org/10.1016/S0045-7949\(99\)00211-4](https://doi.org/10.1016/S0045-7949(99)00211-4). (Cited in §10.7 (p. 495)).
- [90] I. S. DUFF, A. M. ERISMAN, AND J. K. REID, *Direct methods for sparse matrices*, Numerical Mathematics and Scientific Computation, Oxford University Press, second ed., 2017, <https://doi.org/10.1093/acprof:oso/9780198508380.001.0001>. (Cited in Preface (p. xiv), §7.1 (p. 229), §7.12 (p. 297)).
- [91] H. ELMAN, V. E. HOWLE, J. SHADID, R. SHUTTLEWORTH, AND R. TUMINARO, *Block preconditioners based on approximate commutators*, SIAM Journal on Scientific Computing, 27 (2006), pp. 1651–1668, <https://doi.org/10.1137/040608817>. (Cited in §11.4.3 (p. 558)).

- [92] H. ELMAN, D. SILVESTER, AND A. WATHEN, *Finite Elements and Fast Iterative Solvers: with Applications in Incompressible Fluid Dynamics*, Oxford University Press, 2014, <https://doi.org/10.1093/acprof:oso/9780199678792.001.0001>. (Cited in §11.4.1 (p. 555), §11.4.3 (p. 558)).
- [93] A. ERN AND J.-L. GUERMOND, *Finite elements I—Approximation and Interpolation*, vol. 72 of Texts in Applied Mathematics, Springer, 2021, <https://doi.org/10.1007/978-3-030-56341-7>. (Cited in §8.2 (p. 315)).
- [94] L. C. EVANS, *Partial Differential Equations*, vol. 19 of Graduate Studies in Mathematics, American Mathematical Society, second ed., 2010, <https://doi.org/10.1090/gsm/019>. (Cited in §1.4 (p. 17), §8.3.1 (p. 327)).
- [95] R. EYMARD, T. GALLOUËT, AND R. HERBIN, *Finite volume methods*, in Solution of Equation in $\mathbb{R}^n$ (Part 3), Techniques of Scientific Computing (Part 3), vol. 7 of Handbook of Numerical Analysis, Elsevier, 2000, pp. 713–1018, [https://doi.org/10.1016/S1570-8659\(00\)07005-8](https://doi.org/10.1016/S1570-8659(00)07005-8). (Cited in §3.5.1 (p. 77), §3.5.1 (p. 78), §3.5.1 (p. 79), §3.5.1 (p. 83), §3.5.2 (p. 86)).
- [96] V. FABER AND T. MANTEUFFEL, *Necessary and sufficient conditions for the existence of a conjugate gradient method*, SIAM Journal on Numerical Analysis, 21 (1984), pp. 352–362, <https://doi.org/10.1137/0721026>. (Cited in §7.11 (p. 291)).
- [97] R. D. FALGOUT, J. E. JONES, AND U. M. YANG, *The design and implementation of hypre, a library of parallel high performance preconditioners*, in Numerical Solution of Partial Differential Equations on Parallel Computers, vol. 51 of Lecture Notes in Computational Science and Engineering, Springer, 2006, pp. 267–294, https://doi.org/10.1007/3-540-31619-1_8. (Cited in §11.5 (p. 561)).
- [98] P. E. FARRELL, L. MITCHELL, AND F. WECHSUNG, *An augmented Lagrangian preconditioner for the 3D stationary incompressible Navier–Stokes equations at high Reynolds number*, SIAM Journal on Scientific Computing, 41 (2019), pp. A3073–A3096, <https://doi.org/10.1137/18M1219370>. (Cited in §11.4.3 (p. 559)).
- [99] B. FORNBERG, *A Practical Guide to Pseudospectral Methods*, vol. 1 of Cambridge Monographs on Applied and Computational Mathematics, Cambridge University Press, 1996, <https://doi.org/10.1017/CB09780511626357>. (Cited in §10.6 (p. 488)).
- [100] M. FORTIN, *An analysis of the convergence of mixed finite element methods*, RAIRO. Analyse numérique, 11 (1977), pp. 341–354, http://www.numdam.org/item/M2AN_1977__11_4_341_0/. (Cited in §10.3.2 (p. 456)).
- [101] A. FRIEDMAN, *Foundations of Modern Analysis*, Dover Publications, 1982. (Cited in §8.1.4 (p. 308), §8.2.2 (p. 319)).
- [102] M. GANDER, L. HALPERN, AND F. NATAF, *Optimized Schwarz methods*, in Proceedings of the 12th International Conference on Domain Decomposition, ddm.org, 2000, pp. 15–27, <http://www.ddm.org/DD12/Gander.pdf>. (Cited in §11.1.2 (p. 509)).
- [103] M. J. GANDER AND F. KWOK, *Optimal interface conditions for an arbitrary decomposition into subdomains*, in Domain Decomposition Methods in Science and Engineering XIX, vol. 78 of Lecture Notes in Computational Science and Engineering, Springer, 2011, pp. 101–108, https://doi.org/10.1007/978-3-642-11304-8_9. (Cited in §11.1.2 (p. 509)).

- [104] A. GEORGE, *Nested dissection of a regular finite element mesh*, SIAM Journal on Numerical Analysis, 10 (1973), pp. 345–363, <https://doi.org/10.1137/0710032>. (Cited in §7.2.3 (p. 241)).
- [105] A. GEORGE AND J. W. H. LIU, *Computer Solution of Large Sparse Positive-Definite Systems*, Prentice-Hall, 1981. (Cited in Preface (p. xiv), §7.1.1 (p. 231), §7.2 (p. 235), §7.2 (p. 235), §7.2.1 (p. 236), §7.2.2 (p. 238), §7.12 (p. 297)).
- [106] A. GEORGE AND J. W. H. LIU, *The evolution of the minimum degree ordering algorithm*, SIAM Review, 31 (1989), pp. 1–19, <https://doi.org/10.1137/1031001>. (Cited in §7.2.2 (p. 241), §7.2.2 (p. 241)).
- [107] J. A. GEORGE, *Computer Implementation of the Finite Element Method*, PhD thesis, Stanford University, 1971. AAI7205916. (Cited in §7.2.1 (p. 236)).
- [108] V. GIRAULT AND P.-A. RAVIART, *Finite Element Methods for Navier-Stokes Equations*, Springer, 1986, <https://doi.org/10.1007/978-3-642-61623-5>. (Cited in §10.3.3 (p. 462), §10.3.3 (p. 463)).
- [109] G. H. GOLUB AND C. F. VAN LOAN, *Matrix Computations*, Johns Hopkins Studies in the Mathematical Sciences, Johns Hopkins University Press, fourth ed., 2013. (Cited in Preface (p. xiv), §7.7.1 (p. 268), §7.12 (p. 297)).
- [110] D. GOTTLIEB AND S. A. ORSZAG, *Numerical Analysis of Spectral Methods*, vol. 26 of CBMS-NSF Regional Conference Series in Applied Mathematics, SIAM, 1977, <https://doi.org/10.1137/1.9781611970425>. (Cited in §8.8 (p. 356), §10.6 (p. 488)).
- [111] T. GRÄTSCH AND K.-J. BATHE, *A posteriori error estimation techniques in practical finite element analysis*, Computers & Structures, 83 (2005), pp. 235–265, <https://doi.org/10.1016/j.compstruc.2004.08.011>. (Cited in §8.6 (p. 349), §8.6.1 (p. 350)).
- [112] A. GREENBAUM, *Iterative Methods for Solving Linear Systems*, vol. 17 of Frontiers in Applied Mathematics, SIAM, 1997, <https://doi.org/10.1137/1.9781611970937>. (Cited in §7.11 (p. 291)).
- [113] P. GRISVARD, *Elliptic Problems in Nonsmooth Domains*, vol. 69 of Classics in Applied Mathematics, SIAM, 2011, <https://doi.org/10.1137/1.9781611972030>. (Cited in §8.4.2 (p. 336)).
- [114] T. G. GROSSMANN, U. J. KOMOROWSKA, J. LATZ, AND C.-B. SCHÖNLIEB, *Can physics-informed neural networks beat the finite element method?*, IMA Journal of Applied Mathematics, 89 (2024), pp. 143–174, <https://doi.org/10.1093/imamat/hxae011>. (Cited in §10.7 (p. 496)).
- [115] M. J. GROTE AND T. HUCKLE, *Parallel preconditioning with sparse approximate inverses*, SIAM Journal on Scientific Computing, 18 (1997), pp. 838–853, <https://doi.org/10.1137/S1064827594276552>. (Cited in §7.6 (p. 264)).
- [116] J. GUERMOND, P. MINEV, AND J. SHEN, *An overview of projection methods for incompressible flows*, Computer Methods in Applied Mechanics and Engineering, 195 (2006), pp. 6011–6045, <https://doi.org/10.1016/j.cma.2005.10.010>. (Cited in §11.4.3 (p. 559)).

- [117] M. D. GUNZBURGER AND P. B. BOCHEV, *Least-Squares Finite Element Methods*, Springer, 2009, <https://doi.org/10.1007/b13382>. (Cited in §10.4 (p. 471), §10.4.1 (p. 473)).
- [118] E. HABER, *Computational Methods in Geophysical Electromagnetics*, vol. 1 of Mathematics in Industry, SIAM, 2014, <https://doi.org/10.1137/1.9781611973808>. (Cited in §10.1.2 (p. 434)).
- [119] R. HABERMAN, *Applied Partial Differential Equations with Fourier Series and Boundary Value Problems*, Pearson Education, fifth ed., 2013. (Cited in §1.4 (p. 17), §5.1 (p. 131)).
- [120] W. HACKBUSCH, *Hierarchical Matrices: Algorithms and Analysis*, vol. 49 of Springer Series in Computational Mathematics, Springer, 2015, <https://doi.org/10.1007/978-3-662-47324-5>. (Cited in §11.5 (p. 562)).
- [121] E. HAIRER AND G. WANNER, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, vol. 14 of Springer Series in Computational Mathematics, Springer, 1996. (Cited in §9.6 (p. 412), §9.6 (p. 412)).
- [122] D. A. HAM, P. H. J. KELLY, L. MITCHELL, C. J. COTTER, R. C. KIRBY, K. SAGIYAMA, N. BOUZIANI, S. VORDERWUELBECKE, T. J. GREGORY, J. BETTERIDGE, D. R. SHAPERO, R. W. NIXON-HILL, C. J. WARD, P. E. FARRELL, P. D. BRUBECK, I. MARSDEN, T. H. GIBSON, M. HOMOLYA, T. SUN, A. T. T. MCRAE, F. LUPORINI, A. GREGORY, M. LANGE, S. W. FUNKE, F. RATHGEBER, G.-T. BERCEA, AND G. R. MARKALL, *Firedrake User Manual*, Imperial College London and University of Oxford and Baylor University and University of Washington, first ed., 2023, <https://doi.org/10.25561/104839>. (Cited in §8.8 (p. 355), §11.5 (p. 562)).
- [123] H. HAN AND R. B. KELLOGG, *Differentiability properties of solutions of the equation $-\epsilon^2 \Delta u + ru = f(x, y)$ in a square*, SIAM Journal on Mathematical Analysis, 21 (1990), pp. 394–408, <https://doi.org/10.1137/0521022>. (Cited in §8.4.2 (p. 336)).
- [124] F. H. HARLOW AND J. E. WELCH, *Numerical calculation of time-dependent viscous incompressible flow of fluid with free surface*, Physics of Fluids, 8 (1965), pp. 2182–2189, <https://doi.org/10.1063/1.1761178>. (Cited in §10.1.1 (p. 429)).
- [125] A. HARTEN, *High resolution schemes for hyperbolic conservation laws*, Journal of Computational Physics, 135 (1997), pp. 259–278, <https://doi.org/10.1006/jcph.1997.5713>. (Cited in §6.4.2 (p. 205)).
- [126] A. HARTEN, S. OSHER, B. ENGQUIST, AND S. R. CHAKRAVARTHY, *Some results on uniformly high-order accurate essentially nonoscillatory schemes*, Applied Numerical Mathematics, 2 (1986), pp. 347–377, [https://doi.org/10.1016/0168-9274\(86\)90039-5](https://doi.org/10.1016/0168-9274(86)90039-5). (Cited in §9.4 (p. 385), §9.4.2 (p. 391), §9.4.2 (p. 391), §9.4.2 (p. 392)).
- [127] M. T. HEATH, *Scientific Computing: An Introductory Survey, Revised Second Edition*, vol. 80 of Classics in Applied Mathematics, SIAM, 2018, <https://doi.org/10.1137/1.9781611975581>. (Cited in §2.7 (p. 45), §5.5 (p. 173)).
- [128] F. HECHT, *New development in freefem++*, Journal of Numerical Mathematics, 20 (2012), pp. 251–265, <https://doi.org/10.1515/jnum-2012-0013>. (Cited in §8.8 (p. 355)).

- [129] V. E. HENSON AND U. M. YANG, *BoomerAMG: A parallel algebraic multigrid solver and preconditioner*, Applied Numerical Mathematics. An IMACS Journal, 41 (2002), pp. 155–177, [https://doi.org/10.1016/S0168-9274\(01\)00115-5](https://doi.org/10.1016/S0168-9274(01)00115-5). (Cited in §11.5 (p. 561)).
- [130] M. A. HEROUX, R. A. BARTLETT, V. E. HOWLE, R. J. HOEKSTRA, J. J. HU, T. G. KOLDA, R. B. LEHOUCQ, K. R. LONG, R. P. PAWLOWSKI, E. T. PHIPPS, A. G. SALINGER, H. K. THORNQUIST, R. S. TUMINARO, J. M. WILLENBRING, A. WILLIAMS, AND K. S. STANLEY, *An overview of the Trilinos Project*, ACM Transactions on Mathematical Software, 31 (2005), pp. 397–423, <https://doi.org/10.1145/1089014.1089021>. (Cited in §11.5 (p. 561)).
- [131] J. S. HESTHAVEN, *Numerical Methods for Conservation Laws: From Analysis to Algorithms*, vol. 18 of Computational Science & Engineering, SIAM, 2018, <https://doi.org/10.1137/1.9781611975109>. (Cited in Preface (p. xiv), §6.1.4 (p. 186), §6.2.2 (p. 192), §6.2.3 (p. 194), §6.4.2 (p. 207), §6.4.2 (p. 207), §6.6 (p. 217), §6.6 (p. 219), §9.4 (p. 385), §9.4.2 (p. 392), §9.4.4 (p. 396), §9.5.3 (p. 408), §9.5.3 (p. 409)).
- [132] J. S. HESTHAVEN, S. GOTTLIEB, AND D. GOTTLIEB, *Spectral Methods for Time-Dependent Problems*, Cambridge Monographs on Applied and Computational Mathematics, Cambridge University Press, 2007, <https://doi.org/10.1017/CB09780511618352>. (Cited in §8.8 (p. 356), §10.6 (p. 488), §10.6.1 (p. 488), §10.6.3 (p. 494)).
- [133] J. S. HESTHAVEN AND T. WARBURTON, *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*, vol. 54 of Texts in Applied Mathematics, Springer, 2007, <https://doi.org/10.1007/978-0-387-72067-8>. (Cited in §9.5.2 (p. 406), §10.5 (p. 485)).
- [134] J. J. HEYS, E. LEE, T. A. MANTEUFFEL, AND S. F. MCCORMICK, *On mass-conserving least-squares methods*, SIAM Journal on Scientific Computing, 28 (2006), pp. 1675–1693, <https://doi.org/10.1137/050640928>. (Cited in §10.4.3 (p. 477)).
- [135] R. HIPTMAIR, *Multigrid method for $H(\text{div})$ in three dimensions*, Electronic Transactions on Numerical Analysis, 6 (1997), pp. 133–152, <https://etna.math.kent.edu/vol.6.1997/pp133-152.dir/>. (Cited in §11.4.2 (p. 557)).
- [136] R. HIPTMAIR, *Multigrid method for Maxwell's equations*, SIAM Journal on Numerical Analysis, 36 (1998), pp. 204–225, <https://doi.org/10.1137/S0036142997326203>. (Cited in §11.4.2 (p. 557)).
- [137] A. J. HOFFMAN, M. S. MARTIN, AND D. J. ROSE, *Complexity bounds for regular finite difference and finite element grids*, SIAM Journal on Numerical Analysis, 10 (1973), pp. 364–369, <https://doi.org/10.1137/0710033>. (Cited in §7.2.3 (p. 241), §7.2.3 (p. 243)).
- [138] T. J. R. HUGHES, J. A. COTTRELL, AND Y. BAZILEVS, *Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement*, Computer Methods in Applied Mechanics and Engineering, 194 (2005), pp. 4135–4195, <https://doi.org/10.1016/j.cma.2004.10.008>. (Cited in §8.8 (p. 356)).
- [139] *hypre: High performance preconditioners*. <https://llnl.gov/casc/hypre>, <https://github.com/hypre-space/hypre>. (Cited in §11.5 (p. 561)).

- [140] V. JOHN, A. LINKE, C. MERDON, M. NEILAN, AND L. G. REBHOLZ, *On the divergence constraint in mixed finite element methods for incompressible flows*, SIAM Review, 59 (2017), pp. 492–544, <https://doi.org/10.1137/15M1047696>. (Cited in §10.3.4 (p. 468), §10.4.3 (p. 477)).
- [141] V. JOHN AND L. TOBISKA, *Numerical performance of smoothers in coupled multigrid methods for the parallel solution of the incompressible Navier-Stokes equations*, International Journal for Numerical Methods in Fluids, 33 (2000), pp. 453–473, [https://doi.org/10.1002/1097-0363\(20000630\)33:4%3C453::AID-FLD15%3E3.0.CO;2-0](https://doi.org/10.1002/1097-0363(20000630)33:4%3C453::AID-FLD15%3E3.0.CO;2-0). (Cited in §11.4.3 (p. 559)).
- [142] M. JUNTUNEN AND R. STENBERG, *Nitsche's method for general boundary conditions*, Mathematics of Computation, 78 (2009), pp. 1353–1374, <https://doi.org/10.1090/S0025-5718-08-02183-2>. (Cited in §10.2.2 (p. 446)).
- [143] G. E. KARNIADAKIS AND S. J. SHERWIN, *Spectral/hp Element Methods for Computational Fluid Dynamics*, Numerical Mathematics and Scientific Computation, Oxford University Press, second ed., 2005, <https://doi.org/10.1093/acprof:oso/9780198528692.001.0001>. (Cited in §9.5.2 (p. 406)).
- [144] P. KEAST, *Moderate-degree tetrahedral quadrature formulas*, Computer Methods in Applied Mechanics and Engineering, 55 (1986), pp. 339–348, [https://doi.org/10.1016/0045-7825\(86\)90059-9](https://doi.org/10.1016/0045-7825(86)90059-9). (Cited in §2.5.2 (p. 42)).
- [145] R. M. KIRBY AND G. E. KARNIADAKIS, *De-aliasing on non-uniform grids: algorithms and applications*, Journal of Computational Physics, 191 (2003), pp. 249–264, [https://doi.org/10.1016/S0021-9991\(03\)00314-0](https://doi.org/10.1016/S0021-9991(03)00314-0). (Cited in §9.5.3 (p. 411)).
- [146] A. KLÖCKNER, T. WARBURTON, J. BRIDGE, AND J. S. HESTHAVEN, *Nodal discontinuous Galerkin methods on graphics processors*, Journal of Computational Physics, 228 (2009), pp. 7863–7882, <https://doi.org/10.1016/j.jcp.2009.06.041>. (Cited in §9.5.2 (p. 404)).
- [147] L. KRIVODONOVA, *Limiters for high-order discontinuous Galerkin methods*, Journal of Computational Physics, 226 (2007), pp. 879–896, <https://doi.org/10.1016/j.jcp.2007.05.011>. (Cited in §9.5.3 (p. 408)).
- [148] O. A. LADYZHENSKAYA, *The Mathematical Theory of Viscous Incompressible Flow*, vol. 2 of Mathematics and its Applications, Gordon and Breach, 1969. (Cited in §10.3.1 (p. 453)).
- [149] P. D. LAX, *Functional Analysis*, Pure and Applied Mathematics, Wiley-Interscience, 2002. (Cited in §8.2.2 (p. 319)).
- [150] R. J. LEVEQUE, *Numerical Methods for Conservation Laws*, Birkhäuser, 1992, <https://doi.org/10.1007/978-3-0348-8629-1>. (Cited in §6.2.3 (p. 193), §6.5.3 (p. 211), §9.1 (p. 363), §9.1.2 (p. 369), §9.1.2 (p. 370)).
- [151] R. J. LEVEQUE, *Finite Volume Methods for Hyperbolic Problems*, Cambridge Texts in Applied Mathematics, Cambridge University Press, 2002, <https://doi.org/10.1017/CBO9780511791253>. (Cited in Preface (p. xiv), §1.2 (p. 7), §1.4 (p. 17), §6.3.3 (p. 200), §6.4.1 (p. 202), §6.4.2 (p. 205), §6.4.2 (p. 205), §6.6 (p. 217), §6.6 (p. 217), §6.6 (p. 219), §6.6 (p. 219)).

- [152] R. J. LEVEQUE, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*, SIAM, 2007, <https://doi.org/10.1137/1.9780898717839>. (Cited in Preface (p. xiv), §3.3 (p. 61), §3.3 (p. 61), §3.3 (p. 62)).
- [153] K. LIPNIKOV, G. MANZINI, AND M. SHASHKOV, *Mimetic finite difference method*, Journal of Computational Physics, 257 (2014), pp. 1163–1227, <https://doi.org/10.1016/j.jcp.2013.07.031>. (Cited in §10.1.2 (p. 434)).
- [154] R. J. LIPTON AND R. E. TARJAN, *A separator theorem for planar graphs*, SIAM Journal on Applied Mathematics, 36 (1979), pp. 177–189, <https://doi.org/10.1137/0136016>. (Cited in §7.2.3 (p. 243)).
- [155] V. D. LISEIKIN, *Grid Generation Methods*, Springer, second ed., 2010, <https://doi.org/10.1007/978-90-481-2912-6>. (Cited in §3.6.1 (p. 95)).
- [156] J. W. H. LIU, *Modification of the minimum-degree algorithm by multiple elimination*, ACM Transactions on Mathematical Software, 11 (1985), pp. 141–153, <https://doi.org/10.1145/214392.214398>. (Cited in §7.2.2 (p. 241)).
- [157] J. D. LOGAN, *Applied Partial Differential Equations*, Undergraduate Texts in Mathematics, Springer, third ed., 2015, <https://doi.org/10.1007/978-3-319-12493-3>. (Cited in §1.4 (p. 17), §5.1 (p. 131)).
- [158] A. LOGG, K. MARDAL, AND G. N. WELLS, *Automated Solution of Differential Equations by the Finite Element Method*, Springer, 2012, <https://doi.org/10.1007/978-3-642-23099-8>. (Cited in §8.8 (p. 355)).
- [159] S. H. LUI, *Numerical Analysis of Partial Differential Equations*, Pure and Applied Mathematics, John Wiley & Sons, 2011, <https://doi.org/10.1002/9781118111130>. (Cited in §10.6 (p. 488), §10.6.1 (p. 489), §10.6.1 (p. 490), §10.6.2 (p. 491), §10.6.2 (p. 491)).
- [160] J. F. MAITRE, F. MUSY, AND P. NIGNON, *A fast solver for the Stokes equations using multigrid with a UZAWA smoother*, in Advances in Multi-Grid Methods, D. Braess, W. Hackbusch, and U. Trottenberg, eds., vol. 11 of Notes on Numerical Fluid Mechanics, Vieweg, 1984, pp. 77–83, https://doi.org/10.1007/978-3-663-14245-4_8. (Cited in §11.4.2 (p. 557)).
- [161] J. MANDEL, *Balancing domain decomposition*, Communications in Numerical Methods in Engineering with Biomedical Applications, 9 (1993), pp. 233–241, <https://doi.org/10.1002/cnm.1640090307>. (Cited in §11.1.3 (p. 514)).
- [162] T. MANTEUFFEL, S. MCCORMICK, J. NOLTING, J. RUGE, AND G. SANDERS, *Further results on error estimators for local refinement with first-order system least squares (FOSLS)*, Numerical Linear Algebra with Applications, 17 (2010), pp. 387–413, <https://doi.org/10.1002/nla.696>. (Cited in §10.4.2 (p. 476)).
- [163] T. A. MANTEUFFEL, *The Tchebychev iteration for nonsymmetric linear systems*, Numerische Mathematik, 28 (1977), pp. 307–327, <https://doi.org/10.1007/BF01389971>. (Cited in §7.7.2 (p. 271)).

- [164] T. A. MANTEUFFEL, S. F. MCCORMICK, J. RUGE, AND J. G. SCHMIDT, *First-order system LL^* (FOSLL*) for general scalar elliptic problems in the plane*, SIAM Journal on Numerical Analysis, 43 (2005), pp. 2098–2120, <https://doi.org/10.1137/S0036142903430402>. (Cited in §10.4.1 (p. 475)).
- [165] K.-A. MARDAL, T. K. NILSEN, AND G. A. STAFF, *Order-optimal preconditioners for implicit Runge–Kutta schemes applied to parabolic PDEs*, SIAM Journal on Scientific Computing, 29 (2007), pp. 361–375, <https://doi.org/10.1137/05064093X>. (Cited in §11.4.3 (p. 558)).
- [166] K.-A. MARDAL AND R. WINTHER, *Uniform preconditioners for the time dependent Stokes problem*, Numerische Mathematik, 98 (2004), pp. 305–327, <https://doi.org/10.1007/s00211-004-0529-6>. (Cited in §11.4.3 (p. 558)).
- [167] P.-G. MARTINSSON AND J. A. TROPP, *Randomized numerical linear algebra: foundations and algorithms*, Acta Numerica, 29 (2020), pp. 403–572, <https://doi.org/10.1017/s0962492920000021>. (Cited in §11.5 (p. 562)).
- [168] K. MATSSON AND J. NORDSTRÖM, *Summation by parts operators for finite difference approximations of second derivatives*, Journal of Computational Physics, 199 (2004), pp. 503–540, <https://doi.org/10.1016/j.jcp.2004.03.001>. (Cited in §3.7 (p. 100)).
- [169] J. MELENK, K. GERDES, AND C. SCHWAB, *Fully discrete hp-finite elements: fast quadrature*, Computer Methods in Applied Mechanics and Engineering, 190 (2001), pp. 4339–4364, [https://doi.org/10.1016/S0045-7825\(00\)00322-4](https://doi.org/10.1016/S0045-7825(00)00322-4). (Cited in §8.8 (p. 356)).
- [170] J. M. MELENK AND I. BABUŠKA, *The partition of unity finite element method: Basic theory and applications*, Computer Methods in Applied Mechanics and Engineering, 139 (1996), pp. 289–314, [https://doi.org/10.1016/S0045-7825\(96\)01087-0](https://doi.org/10.1016/S0045-7825(96)01087-0). (Cited in §10.7 (p. 495)).
- [171] R. B. MORGAN, *A restarted GMRES method augmented with eigenvectors*, SIAM Journal on Matrix Analysis and Applications, 16 (1995), pp. 1154–1171, <https://doi.org/10.1137/S0895479893253975>. (Cited in §7.8.2 (p. 278)).
- [172] M. F. MURPHY, G. H. GOLUB, AND A. J. WATHEN, *A note on preconditioning for indefinite linear systems*, SIAM Journal on Scientific Computing, 21 (2000), pp. 1969–1972, <https://doi.org/10.1137/S1064827599355153>. (Cited in §11.4.1 (p. 555)).
- [173] N. M. NARECHANIA, L. NIKOLIĆ, L. FRERET, H. DE STERCK, AND C. P. T. GROTH, *An integrated data-driven solar wind – CME numerical framework for space weather forecasting*, Journal of Space Weather and Space Climate, 11 (2021), p. 8, <https://doi.org/10.1051/swsc/2020068>. (Cited in §9.7 (p. 419)).
- [174] J. C. NÉDÉLEC, *Mixed finite elements in $\mathbb{R}^3$* , Numerische Mathematik, 35 (1980), pp. 315–341, <https://doi.org/10.1007/BF01396415>. (Cited in §10.2.1 (p. 438), §10.2.1 (p. 441)).
- [175] J. C. NÉDÉLEC, *A new family of mixed finite elements in $\mathbb{R}^3$* , Numerische Mathematik, 50 (1986), pp. 57–81, <https://doi.org/10.1007/BF01389668>. (Cited in §10.2.1 (p. 438)).

- [176] J. NEČAS, *Direct Methods in the Theory of Elliptic Equations*, Springer, 2012, <https://doi.org/10.1007/978-3-642-10455-8>. (Cited in §8.2.2 (p. 320), §8.2.2 (p. 320)).
- [177] T. B. NGUYEN, H. DE STERCK, L. FRERET, AND C. P. T. GROTH, *High-order implicit time-stepping with high-order central essentially-non-oscillatory methods for unsteady three-dimensional computational fluid dynamics simulations*, *International Journal for Numerical Methods in Fluids*, 94 (2022), pp. 121–151, <https://doi.org/10.1002/flid.5049>. (Cited in §9.7 (p. 420)).
- [178] J. NITSCHKE, *Über ein variationsprinzip zur lösung von Dirichlet-problemen bei verwendung von teilräumen, die keinen randbedingungen unterworfen sind*, *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 36 (1971), pp. 9–15, <https://doi.org/10.1007/bf02995904>. (Cited in §10.2.2 (p. 446)).
- [179] R. H. NOCHETTO, K. G. SIEBERT, AND A. VEESER, *Theory of adaptive finite element methods: An introduction*, in *Multiscale, Nonlinear and Adaptive Approximation*, R. DeVore and A. Kunoth, eds., Springer, 2009, pp. 409–542, https://doi.org/10.1007/978-3-642-03413-8_12. (Cited in §8.6 (p. 349)).
- [180] Y. PINCHOVER AND J. RUBINSTEIN, *An Introduction to Partial Differential Equations*, Cambridge University Press, 2005, <https://doi.org/10.1017/CB09780511801228>. (Cited in §1.4 (p. 17), §5.1 (p. 131)).
- [181] A. QUARTERONI, R. SACCO, AND F. SALERI, *Numerical Mathematics*, vol. 37 of *Texts in Applied Mathematics*, Springer, second ed., 2007, <https://doi.org/10.1007/b98885>. (Cited in §2.7 (p. 45)).
- [182] A. QUARTERONI AND A. VALLI, *Domain Decomposition Methods for Partial Differential Equations*, *Numerical Mathematics and Scientific Computation*, Oxford University Press, 1999, <https://doi.org/10.1093/oso/9780198501787.001.0001>. (Cited in §11.5 (p. 559)).
- [183] M. RAISSI, P. PERDIKARIS, AND G. E. KARNIADAKIS, *Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, *Journal of Computational Physics*, 378 (2019), pp. 686–707, <https://doi.org/10.1016/j.jcp.2018.10.045>. (Cited in §10.7 (p. 496)).
- [184] MD. M. RANA, V. E. HOWLE, K. LONG, A. MEEK, AND W. MILESTONE, *A new block preconditioner for implicit Runge–Kutta methods for parabolic PDE problems*, *SIAM Journal on Scientific Computing*, 43 (2021), pp. S475–S495, <https://doi.org/10.1137/20M1349680>. (Cited in §11.4.3 (p. 558)).
- [185] P. A. RAVIART AND J. M. THOMAS, *A mixed finite element method for 2-nd order elliptic problems*, in *Mathematical Aspects of Finite Element Methods*, I. Galligani and E. Magenes, eds., Springer, 1977, pp. 292–315, <https://doi.org/10.1007/BFb0064470>. (Cited in §10.2.1 (p. 438)).
- [186] E. REICH, *On the convergence of the classical iterative method of solving linear simultaneous equations*, *Annals of Mathematical Statistics*, 20 (1949), pp. 448–451, <https://doi.org/10.1214/aoms/1177729998>. (Cited in §7.4 (p. 255)).

- [187] S. I. REPIN, *A Posteriori Estimates for Partial Differential Equations*, vol. 4 of Radon Series on Computational and Applied Mathematics, Walter de Gruyter, 2008, <https://doi.org/doi:10.1515/9783110203042>. (Cited in §8.6 (p. 348), §8.6 (p. 349), §8.6.2 (p. 351), §8.6.2 (p. 352), §8.6.2 (p. 352)).
- [188] R. D. RICHTMYER AND K. W. MORTON, *Difference Methods for Initial-Value Problems*, vol. 4 of Interscience Tracts in Pure and Applied Mathematics, Interscience Publishers, John Wiley & Sons, second ed., 1967. (Cited in §5.1.3 (p. 137), §5.2 (p. 139), §5.2.1 (p. 143), §5.2.2 (p. 145), §5.2.2 (p. 146), §5.2.2 (p. 146)).
- [189] B. RIVIÈRE, *Discontinuous Galerkin Methods for Solving Elliptic and Parabolic Equations: Theory and Implementation*, vol. 35 of Frontiers in Applied Mathematics, SIAM, 2008, <https://doi.org/10.1137/1.9780898717440>. (Cited in §10.5 (p. 485)).
- [190] M. E. ROGNES, R. C. KIRBY, AND A. LOGG, *Efficient assembly of $H(\text{div})$ and $H(\text{curl})$ conforming finite elements*, SIAM Journal on Scientific Computing, 31 (2010), pp. 4130–4151, <https://doi.org/10.1137/08073901X>. (Cited in §8.5 (p. 339)).
- [191] H.-G. ROOS, M. STYNES, AND L. TOBISKA, *Robust Numerical Methods for Singularly Perturbed Differential Equations*, vol. 24 of Springer Series in Computational Mathematics, Springer-Verlag, second ed., 2008. (Cited in §8.4.1 (p. 334)).
- [192] J. W. RUGE AND K. STÜBEN, *Algebraic multigrid*, in Multigrid Methods, S. F. McCormick, ed., vol. 3 of Frontiers in Applied Mathematics, SIAM, 1987, pp. 73–130, <https://doi.org/10.1137/1.9781611971057.ch4>. (Cited in §11.3.1 (p. 538), §11.3.3 (p. 548)).
- [193] Y. SAAD, *ILUT: A dual threshold incomplete LU factorization*, Numerical Linear Algebra with Applications, 1 (1994), pp. 387–402, <https://doi.org/https://doi.org/10.1002/nla.1680010405>. (Cited in §7.6 (p. 263)).
- [194] Y. SAAD, *Iterative Methods for Sparse Linear Systems*, SIAM, second ed., 2003, <https://doi.org/10.1137/1.9780898718003>. (Cited in Preface (p. xiv), §3.3.2 (p. 65), §3.3.2 (p. 66), §7.3.2 (p. 249), §7.3.2 (p. 249), §7.4 (p. 250), §7.6 (p. 262), §7.6 (p. 262), §7.8 (p. 274), §7.8.2 (p. 279), §7.9.2 (p. 286), §7.11 (p. 291), §7.12 (p. 297), §11.1.3 (p. 510)).
- [195] Y. SAAD AND M. H. SCHULTZ, *GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems*, SIAM Journal on Scientific and Statistical Computing, 7 (1986), pp. 856–869, <https://doi.org/10.1137/0907058>. (Cited in §7.8.1 (p. 274)).
- [196] J. SCHÖBERL, *Multigrid methods for a parameter dependent problem in primal variables*, Numerische Mathematik, 84 (1999), pp. 97–119, <https://doi.org/10.1007/s002110050465>. (Cited in §11.4.3 (p. 559)).
- [197] K. SHAHBAZI, *An explicit expression for the penalty parameter of the interior penalty method*, Journal of Computational Physics, 205 (2005), pp. 401–407, <https://doi.org/10.1016/j.jcp.2004.11.017>. (Cited in §10.5 (p. 486), §10.5 (p. 486)).
- [198] C.-W. SHU, *Essentially non-oscillatory and weighted essentially non-oscillatory schemes for hyperbolic conservation laws*, tech. report, Nov 1997. ICASE Report No. 97-65. (Cited in §9.4 (p. 385)).

- [199] C.-W. SHU, *Essentially non-oscillatory and weighted essentially non-oscillatory schemes*, Acta Numerica, 29 (2020), pp. 701–762, <https://doi.org/10.1017/S0962492920000057>. (Cited in §9.4 (p. 385), §9.4.2 (p. 392)).
- [200] D. SILVESTER, H. ELMAN, D. KAY, AND A. WATHEN, *Efficient preconditioning of the linearized Navier–Stokes equations for incompressible flow*, Journal of Computational and Applied Mathematics, 128 (2001), pp. 261–279, [https://doi.org/10.1016/S0377-0427\(00\)00515-X](https://doi.org/10.1016/S0377-0427(00)00515-X). (Cited in §11.4.3 (p. 558)).
- [201] I. SINGER AND E. TURKEL, *High-order finite difference methods for the Helmholtz equation*, Computer Methods in Applied Mechanics and Engineering, 163 (1998), pp. 343–358, [https://doi.org/10.1016/S0045-7825\(98\)00023-1](https://doi.org/10.1016/S0045-7825(98)00023-1). (Cited in §3.7 (p. 99)).
- [202] G. L. SLEIJPEN AND D. R. FOKKEMA, *BiCGstab(l) for linear equations involving unsymmetric matrices with complex spectrum.*, ETNA. Electronic Transactions on Numerical Analysis, 1 (1993), pp. 11–32, <http://eudml.org/doc/118652>. (Cited in §7.11 (p. 295)).
- [203] B. S. SOUTHWORTH, O. KRZYSIK, W. PAZNER, AND H. DE STERCK, *Fast solution of fully implicit Runge–Kutta and discontinuous Galerkin in time for numerical PDEs, Part I: the linear setting*, SIAM Journal on Scientific Computing, 44 (2022), pp. A416–A443, <https://doi.org/10.1137/21M1389742>. (Cited in §11.4.3 (p. 558)).
- [204] D. A. SPIELMAN AND S.-H. TENG, *Nearly linear time algorithms for preconditioning and solving symmetric, diagonally dominant linear systems*, SIAM Journal on Matrix Analysis and Applications, 35 (2014), pp. 835–885, <https://doi.org/10.1137/090771430>. (Cited in §11.5 (p. 562)).
- [205] N. SPILLANE, V. DOLEAN, P. HAURET, F. NATAF, C. PECHSTEIN, AND R. SCHEICHL, *Abstract robust coarse spaces for systems of PDEs via generalized eigenproblems in the overlaps*, Numerische Mathematik, 126 (2014), pp. 741–770, <https://doi.org/10.1007/s00211-013-0576-y>. (Cited in §11.1.3 (p. 515)).
- [206] J. STOER AND R. BULIRSCH, *Introduction to Numerical Analysis*, vol. 12 of Texts in Applied Mathematics, Springer, third ed., 2002, <https://doi.org/10.1007/978-0-387-21738-3>. (Cited in §2.5.2 (p. 41)).
- [207] G. STRANG AND G. FIX, *An Analysis of the Finite Element Method*, Wellesley-Cambridge Press, second ed., 2008. (Cited in §2.5.2 (p. 42)).
- [208] J. C. STRIKWERDA, *Finite Difference Schemes and Partial Differential Equations*, vol. 88 of Classics in Applied Mathematics, SIAM, second ed., 2004, <https://doi.org/10.1137/1.9780898717938>. (Cited in Preface (p. xiv)).
- [209] K. STÜBEN, *An introduction to algebraic multigrid*, in Multigrid, U. Trottenberg, C. Oosterlee, and A. Schüller, eds., Academic Press, 2001, pp. 413–528. (Cited in §11.3.1 (p. 538)).
- [210] A. SUSANTO, L. IVAN, H. DE STERCK, AND C. P. T. GROTH, *High-order central ENO finite-volume scheme for ideal MHD*, Journal of Computational Physics, 250 (2013), pp. 141–164, <https://doi.org/https://doi.org/10.1016/j.jcp.2013.04.040>. (Cited in §9.5.3 (p. 411)).

- [211] P. K. SWEBY, *High resolution schemes using flux limiters for hyperbolic conservation laws*, SIAM Journal on Numerical Analysis, 21 (1984), pp. 995–1011, <https://doi.org/10.1137/0721062>. (Cited in §6.5.3 (p. 211)).
- [212] J. M. TANG, S. P. MACLACHLAN, R. NABBEN, AND C. VUIK, *A comparison of two-level preconditioners based on multigrid and deflation*, SIAM Journal on Matrix Analysis and Applications, 31 (2010), pp. 1715–1739, <https://doi.org/10.1137/08072084X>. (Cited in §11.2.1 (p. 518)).
- [213] R. TÉMAM, *Sur l'approximation de la solution des équations de Navier-Stokes par la méthode des pas fractionnaires (II)*, Archive for Rational Mechanics and Analysis, 33 (1969), pp. 377–385, <https://doi.org/10.1007/BF00247696>. (Cited in §11.4.3 (p. 559)).
- [214] J. W. THOMAS, *Numerical Partial Differential Equations: Finite Difference Methods*, vol. 22 of Texts in Applied Mathematics, Springer, 1995, <https://doi.org/10.1007/978-1-4899-7278-1>. (Cited in §5.4.1 (p. 165), §5.4.1 (p. 166), §5.4.1 (p. 166)).
- [215] J. W. THOMAS, *Numerical Partial Differential Equations: Conservation Laws and Elliptic Equations*, vol. 33 of Texts in Applied Mathematics, Springer, 1999, <https://doi.org/10.1007/978-1-4612-0569-2>. (Cited in §6.5.3 (p. 211)).
- [216] A. TOSELLI AND O. WIDLUND, *Domain Decomposition Methods—Algorithms and Theory*, vol. 34 of Springer Series in Computational Mathematics, Springer, 2005, <https://doi.org/10.1007/b137868>. (Cited in §11.5 (p. 559)).
- [217] L. N. TREFETHEN, *Spectral Methods in MATLAB*, vol. 10 of Software, Environments, and Tools, SIAM, 2000, <https://doi.org/10.1137/1.9780898719598>. (Cited in §8.8 (p. 356), §10.6 (p. 488)).
- [218] L. N. TREFETHEN AND D. BAU, III, *Numerical Linear Algebra*, SIAM, 1997, <https://doi.org/10.1137/1.9780898719574>. (Cited in §7.12 (p. 297)).
- [219] U. TROTTEBERG, C. W. OOSTERLEE, AND A. SCHÜLLER, *Multigrid*, Academic Press, 2001. With contributions by A. Brandt, P. Oswald, and K. Stüben. (Cited in §11.2.4 (p. 533), §11.5 (p. 559)).
- [220] H. A. VAN DER VORST, *Iterative Krylov Methods for Large Linear Systems*, vol. 13 of Cambridge Monographs on Applied and Computational Mathematics, Cambridge University Press, 2009, <https://doi.org/10.1017/CBO9780511615115>. (Cited in §7.12 (p. 297)).
- [221] J. VAN KAN, *A second-order accurate pressure-correction scheme for viscous incompressible flow*, SIAM Journal on Scientific and Statistical Computing, 7 (1986), pp. 870–891, <https://doi.org/10.1137/0907059>. (Cited in §11.4.3 (p. 559)).
- [222] J. VAN LENT AND S. VANDEWALLE, *Multigrid methods for implicit Runge–Kutta and boundary value method discretizations of parabolic PDEs*, SIAM Journal on Scientific Computing, 27 (2005), pp. 67–92, <https://doi.org/10.1137/030601144>. (Cited in §11.4.3 (p. 558)).
- [223] P. VANĚK, J. MANDEL, AND M. BREZINA, *Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems*, Computing, 56 (1996), pp. 179–196, <https://doi.org/10.1007/BF02238511>. (Cited in §11.3.2 (p. 539)).

- [224] S. P. VANKA, *Block-implicit multigrid solution of Navier-Stokes equations in primitive variables*, Journal of Computational Physics, 65 (1986), pp. 138–158, [https://doi.org/10.1016/0021-9991\(86\)90008-2](https://doi.org/10.1016/0021-9991(86)90008-2). (Cited in §11.4.2 (p. 557)).
- [225] J. VARAH, *A lower bound for the smallest singular value of a matrix*, Linear Algebra and Its Applications, 11 (1975), pp. 3–5, [https://doi.org/10.1016/0024-3795\(75\)90112-3](https://doi.org/10.1016/0024-3795(75)90112-3). (Cited in §3.3.2 (p. 66)).
- [226] R. S. VARGA, *Matrix Iterative Analysis*, vol. 27 of Springer Series in Computational Mathematics, Springer-Verlag, second ed., 2000, <https://doi.org/10.1007/978-3-642-05156-2>. (Cited in Preface (p. xiv), §3.3.2 (p. 64), §7.3.2 (p. 248), §7.3.2 (p. 249), §7.3.2 (p. 249), §7.4 (p. 250), §7.4 (p. 250), §7.4 (p. 251), §7.4 (p. 251), §7.4 (p. 252), §7.4 (p. 252), §7.4 (p. 253), §7.4 (p. 254), §7.4 (p. 255), §7.5 (p. 259), §7.5 (p. 259), §7.6 (p. 261), §7.6 (p. 262), §7.6 (p. 262), §7.6 (p. 262), §7.12 (p. 297)).
- [227] P. S. VASSILEVSKI, *Multilevel Block Factorization Preconditioners*, Springer, 2008. (Cited in §11.3.1 (p. 538)).
- [228] R. VERFÜRTH, *A combined conjugate gradient–multi-grid algorithm for the numerical solution of the Stokes problem*, IMA Journal of Numerical Analysis, 4 (1984), pp. 441–455, <https://doi.org/10.1093/imanum/4.4.441>. (Cited in §11.4.1 (p. 555)).
- [229] R. VERFÜRTH, *A Posteriori Error Estimation Techniques for Finite Element Methods*, Numerical Mathematics and Scientific Computation, Oxford University Press, 2013, <https://doi.org/10.1093/acprof:oso/9780199679423.001.0001>. (Cited in §8.6 (p. 349)).
- [230] M. WANG AND L. CHEN, *Multigrid methods for the Stokes equations using distributive Gauss-Seidel relaxations based on the least squares commutator*, Journal of Scientific Computing, 56 (2013), pp. 409–431, <https://doi.org/10.1007/s10915-013-9684-1>. (Cited in §11.4.2 (p. 557)).
- [231] R. WARMING AND B. HYETT, *The modified equation approach to the stability and accuracy analysis of finite-difference methods*, Journal of Computational Physics, 14 (1974), pp. 159–179, [https://doi.org/10.1016/0021-9991\(74\)90011-4](https://doi.org/10.1016/0021-9991(74)90011-4). (Cited in §5.5 (p. 173)).
- [232] A. WATHEN AND T. REES, *Chebyshev semi-iteration in preconditioning for problems including the mass matrix*, Electronic Transactions on Numerical Analysis, 34 (2008/09), pp. 125–135, <https://www.emis.de/journals/ETNA/vol.34.2008-2009/pp125-135.dir/pp125-135.pdf>. (Cited in §11.4.1 (p. 555)).
- [233] G. B. WHITHAM, *Linear and Nonlinear Waves*, Pure and Applied Mathematics, John Wiley & Sons, 1999, <https://doi.org/10.1002/9781118032954>. (Cited in §6.1.2 (p. 182), §6.1.2 (p. 182), §6.1.3 (p. 183)).
- [234] R. WIENANDS AND W. JOPPICH, *Practical Fourier Analysis for Multigrid Methods*, vol. 4 of Numerical Insights, Chapman & Hall/CRC, 2005, <https://doi.org/10.1201/9781420034998>. (Cited in §11.2.4 (p. 533)).
- [235] J. XU AND L. ZIKATANOV, *Algebraic multigrid methods*, Acta Numerica, 26 (2017), p. 591–721, <https://doi.org/10.1017/S0962492917000083>. (Cited in §11.3.1 (p. 538)).

- [236] K. YEE, *Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media*, IEEE Transactions on Antennas and Propagation, 14 (1966), pp. 302–307, <https://doi.org/10.1109/TAP.1966.1138693>. (Cited in §10.1.2 (p. 433)).
- [237] D. M. YOUNG, *Iterative Solution of Large Linear Systems*, Dover Publications, 2003. (Cited in §7.5 (p. 256), §7.5 (p. 257), §7.5 (p. 257), §7.12 (p. 297)).
- [238] C. ZENGER AND H. GIETL, *Improved difference schemes for the Dirichlet problem of Poisson's equation in the neighbourhood of corners*, Numerische Mathematik, 30 (1978), pp. 315–332, <https://doi.org/10.1007/BF01411846>. (Cited in §3.7 (p. 99)).
- [239] O. ZIENKIEWICZ AND J. ZHU, *Superconvergence and the superconvergent patch recovery*, Finite Elements in Analysis and Design, 19 (1995), pp. 11–23, [https://doi.org/10.1016/0168-874X\(94\)00054-J](https://doi.org/10.1016/0168-874X(94)00054-J). (Cited in §8.6.2 (p. 352)).
- [240] W. ZULEHNER, *A class of smoothers for saddle point problems*, Computing, 65 (2000), pp. 227–246, <https://doi.org/10.1007/s006070070008>. (Cited in §11.4.2 (p. 556)).

Index

- adjacency set, 235
- adjoint, 473
- algebraic multigrid (AMG), *see also*
 - multigrid methods
 - algebraically smooth error, 537
 - C/F AMG, 538
 - coarsening, 540
 - complexity, 545
 - interpolation, 541, 549
 - operator-based interpolation, 538
 - Ruge–Stüben (classical) AMG, 538
 - setup phase, 538
 - smoothed aggregation AMG, 538, 539
 - strength of connection, 539, 547
- algorithm
 - aggregation coarsening, 542
 - aggregation strength of connection, 540
 - algebraic interpolation, 544
 - AMG setup, 537
 - Arnoldi, 275
 - biconjugate gradient, 293
 - C/F coarsening, 548
 - C/F interpolation, 552
 - C/F strength of connection, 547
 - Chebyshev iteration, 268
 - conjugate gradient, 286, 289
 - conjugate gradient squared, 295
 - flexible GMRES, 288
 - generalized minimum residual (GMRES), 279
 - Lanczos, 282
 - Lanczos biorthogonalization, 292
 - minimum degree, 239
 - minimum residual (MINRES), 284
 - multigrid F-cycle, 530
 - multigrid μ -cycle, 527
 - multigrid two-grid cycle, 523
 - multigrid V-cycle, 529
 - nested dissection, 242
 - preconditioned conjugate gradient, 290
 - reverse Cuthill–McKee, 238
 - stabilized biconjugate gradient (BiCGSTAB), 296
- approximation assumption, 111
- approximation property
 - degree k , $\mathcal{V}_k^h$, 116
 - linear, $\mathcal{V}_1^h$, 114
 - quadratic, $\mathcal{V}_2^h$, 117
- backward differencing, *see* differencing
- basis
 - modal, 125
 - nodal, 113, 116, 121
- biharmonic equation, 445
- bilinear form, 105, 314, 317
 - nonsymmetric, 332
- Black–Scholes equations, 162–164
- boundary condition, 379
 - Dirichlet, 104, 335, 345, 446, 472
 - essential, 121, 435, 446, 451, 461
 - ghost cell, 379
 - natural, 121, 435, 446, 451, 461
 - Neumann, 70, 104, 336, 446, 472
 - periodic, 70
- bounded linear function, *see* functional, linear
- Burgers’ equation, 179–181, 184, 186, 397
 - inviscid, 179
 - viscous, 185, 416
- Cauchy–Schwarz inequality, 107, 309
- central differencing, *see* differencing
- characteristic curve, 7, 8, 16, 179–181, 184–186, 364, 380
- characteristic variable, 364
- Chebyshev nodes, 22
- checkerboard instability
 - see* red-black instability
- coercive, 318
 - weak, *see* inf-sup condition

- weakly, 334, 453
- compact support, 113
- compatibility condition, 336
- conservation, 189
 - of mass, 10, 11
- conservation law, 7, 9, 11, 12, 178
 - first-order, 7, 9, 178
 - second-order, 11
 - see also* hyperbolic conservation law
- conserved variables, 368, 369, 372
- consistency
 - see* finite-difference method, consistency of
- continuous, 318
- convergence
 - see* finite-difference method, convergence of
- convergence factor, asymptotic, 247
- convergence rate, 247
- corollary, *see* theorem, lemma, corollary
- Courant–Friedrichs–Lewy (CFL) condition, 149, 195, 373, 382, 412
- Crank–Nicolson, 134
- dam break problem, 370
- Darcy flow, 450
- de Rham complex, 436, 444
 - see also* exact sequence
- differencing
 - backward, 56, 131
 - central, 51, 91, 131
 - forward, 56, 131
- diffusion
 - numerical, 195, 196, 373
- discontinuity
 - contact, 370
 - jump, 182, 183, 219, 370
- discontinuous Galerkin (DG) method, 400
 - DG for elliptic equations, 480
 - DG for hyperbolic conservation laws, 400
 - aliasing errors, 410
 - filtering, 410
 - numerical conservation property, 402
 - slope limiter, 408
 - weak formulation, 402
 - filtering, 410
- discrete Fourier transforms, 150
- dispersion, 166
 - dispersion relation, 165
 - numerical, 167
 - phase velocity, 166
- dissipation, 166
 - numerical, 167
- domain decomposition, 499
 - nonoverlapping
 - additive Schwarz, 502
 - multiplicative Schwarz, 504
 - Schur complement methods, 504
 - overlapping Schwarz, 505
 - minimal overlap, 505
 - optimized Schwarz, 509
 - restricted additive Schwarz (RAS), 507
 - two-level Schwarz, 510
 - balancing Neumann–Neumann, 514
 - see also* multigrid methods
- dual space, 315
- elimination graph, 238
- elliptic PDE, 12, 14, 49, 103, 367
- ellipticity, *see* continuous, coercive
- energy inner product, 107
 - see also* inner product
- energy norm, 107
- entropy, 372
 - fix, 375
 - flux, 218
 - function, 218
 - Lax condition, *see* Lax entropy condition
 - solution, 184–186, 196, 218, 372
- equivalence class, 105
- error
 - a posteriori, 348, 351, 352, 475
 - a priori, 348
 - bound, 112, 116, 119
 - efficient, 348
 - estimate, 349, 351, 352
 - interpolation, 114, 116, 117, 327, 331
 - pointwise, 59
 - quadrature, *see* quadrature, error
 - reliable, 348
 - truncation error, *see* truncation error
- essentially nonoscillatory (ENO) method, 384
 - ENO monotonicity, 390, 391
 - ENO reconstruction, 387, 389
- Euler equations of compressible gas
 - dynamics, 371, 375, 377, 418
- Euler method
 - see* explicit or implicit time-integration method
- exact sequence, 436
 - see also* de Rham complex

- existence, 109, 315
- explicit method
 - Euler, 191
- explicit time-integration method
 - Euler, 134
 - Runge–Kutta, 208
 - TVD Runge–Kutta method, 395
- fast Fourier solver, 494
- fast Fourier transform, 492
- Fick’s law of diffusion, 12
- finite-difference method, 49, 129, 187, 188
 - approximation, 99, 130
 - consistency of, 61, 62, 143
 - convergence of, 61, 143
 - curvilinear coordinates, 87
 - discretization, 51, 130
 - examples, 51, 53, 58, 130, 132, 135, 136, 147
 - high-order, 99
 - meshless, 95
 - mimetic, 76, 90
 - multistage and multistep methods, 173
 - nonuniform grids, 56
 - stability of, 63, 143
 - see also* stability, von Neumann stability
 - staggered, 429
 - truncation error, *see* truncation error
 - see also* differencing
- finite-element method, 103, 107, 401
 - Argyris elements, 448
 - assembly, 122, 339
 - Bogner–Fox–Schmit elements, 447
 - Brezzi–Douglas–Marini (BDM) elements, 439
 - discontinuous Galerkin
 - see* discontinuous Galerkin (DG) method
 - Hermite elements, 447
 - least-squares, *see* FOSLS
 - linear basis, *see* basis
 - Nédélec elements, 438, 439
 - quadratic basis, *see* basis
 - Raviart–Thomas elements, 438, 441
 - Taylor–Hood elements, 467
- finite-volume method, 177, 186, 196, 361
 - conservation form, 77, 192, 193
 - for elliptic PDEs, 77, 83
 - finite-volume cell, 78, 189
 - first-order accurate, 190, 191
 - high-order accurate, 190, 384, 400
 - linear schemes, 207
 - multidimensional, 378
 - structured grid, 378
 - unstructured grid, 383
 - numerical conservation property, 192
 - polynomial reconstruction, 190, 208, 387
 - scalar, 177
 - second-order accurate, 208
 - system, 361, 372
- flux function, 7–9
 - concave, 186
 - convex, 186, 196
 - nonconvex, 219
 - numerical, *see* numerical flux function
- flux Jacobian, 363, 368, 369
- flux tensor, 362
- flux vector, 7, 10
- formal normal, 474
- Fortin operators, 456
- forward differencing, *see* differencing
- FOSLS (first-order system least-squares), 471
- Fourier’s law of heat conduction, 11
- function spaces
 - Banach, 305
 - C^∞ , 304
 - C^k , 106, 304
 - compact support, 304
 - complete, 302, 305, 307
 - conforming, 324
 - Hilbert, 307
 - inner, 307
 - L^1_{loc} , 310
 - L^2 , 143, 306
 - L^2 with zero mean, 462
 - L^p , 306
 - negative-index Sobolev, 315
 - normed, 305
 - piecewise linear, 324
 - P_k , 324
 - product, 313
 - Q_k , 330
 - Sobolev, 312, *see also* Sobolev spaces
 - vector, 303
- functional, linear, 314, 315, 317
- Gauss–Legendre–Lobatto (GLL) nodes, 402
- Gaussian elimination, 227
 - banded, 229
 - dense, 229
 - fill-in, 231

- sparse, 232
- Godunov's method, 197
- graph, 234, 236
- grid
 - logically rectangular, 87
 - quasi-uniform grid family, 31, 61
 - rectangular, 30, 53
 - structured, 378
 - uniform, 51
 - unstructured, 31, 383
 - see also* mesh
- heat equation, 12, 15, 134, 149, 160
- Hölder's inequality, 307
- hyperbolic conservation law, 7, 178, 362
 - integral form, 7, 189, 193
 - one-dimensional, 8, 178, 362
 - quasi-linear, 178, 363
 - scalar, 177
 - linear, 8
 - nonlinear, 7, 178
 - system, 361
 - linear, 363
 - nonlinear, 363, 368
 - three-dimensional, 7, 9, 362
 - weak solution, 181–186, 219
- hyperbolic PDE, 7, 9, 12, 14, 15, 129, 178, 367
- implicit time-integration method, 411
 - backward differentiation (BDF), 413
 - Euler, 134, 149
 - implicit Runge–Kutta, 414
 - DIRK, 414, 417
 - IMEX, 416
 - SDIRK, 415
- inf-sup condition
 - continuous inf-sup, 453
 - discrete inf-sup, 456
- inner product, 307
- integrable, locally, 310
- integration, *see* quadrature
- integration by parts, 104
- interior penalty method, 485
- interpolant, finite element, 114, 327, 331, 439
 - Clément, 465
- interpolant, Fourier, 490
- interpolation, 22
- iterative methods
 - biconjugate gradient, 292, 294
 - Chebyshev iteration, 268
 - conjugate gradient, 286, 289
 - conjugate gradient squared, 294
 - flexible GMRES, 288
 - Gauss–Seidel iteration, 246
 - GMRES, 275, 413
 - Jacobi iteration, 246, 516
 - MINRES, 283
 - polynomial, 245, 265, 273
 - preconditioned conjugate gradient, 290
 - stationary, 246
 - successive overrelaxation (SOR), 246, 255
- Jacobian, 87, 341
- Kronecker product, 55
- Krylov subspace, 274
- Ladyzhenskaya–Babuška–Brezzi (LBB)
 - condition, *see* inf-sup condition
- Laplace's equation, 12
- Lax entropy condition, 186, 219
- Lax–Wendroff method, 170, 188
- least-squares finite-element method, *see* FOSLS
- lemma, *see* theorem, lemma, corollary
- lexicographical ordering, 54
- linear advection equation, 8, 16, 194, 406
 - see also* one-way wave equation
- LU factorization, 227
 - see also* Gaussian elimination
- magnetohydrodynamics (MHD)
 - compressible, 418
- marker-and-cell, 429
- mass matrix, 353, 404
- matrix, 251
 - B -normal, 291
 - banded, 229
 - bandwidth of, 231
 - circulant, 494
 - diagonally dominant, 65, 249
 - graph, 235
 - Hessenberg, 281
 - mass, *see* mass matrix
 - nonnegative, 250
 - permutation, 233
 - positive definite, 66, 250
 - reducible, 250
 - stiffness, *see* stiffness matrix
 - Toeplitz, 67

- matrix splitting
 - see* iterative methods, stationary
- Maxwell's equation, 433, 435
- mesh
 - definition of, 30
 - diameter, 326, 338
 - hexahedral, 330
 - nondegenerate, 326
 - quadrilateral, 330
 - quasi-uniform mesh family, 326
 - tetrahedral, 324
 - triangular, 324
 - unstructured, 31
 - see also* grid
- method of lines, 135, 411
- metric tensor, 94
- Minkowski's inequality, 307
- minmod slope limiter, 210, 215, 408
- modal basis, *see* basis, modal
- modified equation analysis, 169
- monolithic multigrid
 - Braess–Sarazin relaxation, 556
 - distributed relaxation, 557
 - Uzawa relaxation, 557
 - Vanka relaxation, 557
 - see also* multigrid methods
- multigrid methods
 - algebraic multigrid, *see* algebraic multigrid
 - coarse-grid correction, 519
 - domain decomposition
 - see* domain decomposition
 - F-cycle, 529
 - Galerkin coarse-grid correction, 520
 - harmonic modes, 533
 - line relaxation, 536
 - local mode (Fourier) analysis, 532
 - monolithic multigrid
 - see* monolithic multigrid
 - μ -cycle, 528
 - semicoarsening, 535
 - smoothing property, 516
 - two-grid iteration, 522
 - V-cycle, 528
 - W-cycle, 528
- Navier–Stokes equations
 - compressible, 420
 - incompressible, 5
- Newton–Krylov method, 413, 415
- nodal basis, *see* basis, nodal
- norm
 - Euclidean, 60
 - k, p , 312
 - L^2 , 60, 143
 - matrix, 60, 143
 - maximum, 60
 - Sobolev, 312
 - vector, 60, 142
- numerical flux function, 190, 194, 195, 402
 - consistency, 192
 - first-order upwind, 194
 - Godunov, 198
 - Lax–Friedrichs, 195, 373, 395, 402
 - Lax–Wendroff, 195, 196
 - Roe, 373
- numerical stability, *see* stability
- one-way wave equation, 8, 132, 148, 156
 - see also* linear advection equation
- parabolic PDE, 12, 14, 15, 129, 367
- partial differential equation (PDE), 4, 6
 - classification, 12, 367
 - elliptic, *see* elliptic PDE
 - first-order, 6
 - hyperbolic, *see* hyperbolic PDE
 - linear, 6
 - nonlinear, 6
 - parabolic, *see* parabolic PDE
 - quasi-linear, 6
 - second-order, 6, 11, 12
 - semilinear, 6
- Poisson's equation, 12, 14
- polynomial
 - Legendre, 402, 405
- preconditioning, 245, 287
 - algebraic multigrid, *see* algebraic multigrid
 - block factorization, 555
 - domain decomposition
 - see* domain decomposition
 - incomplete LU factorization, 262, 263
 - multigrid, *see* multigrid
- primitive variables, 368, 372
- quadrature, 37
 - composite, 38
 - error, 41
 - Gaussian, 39
- Rankine–Hugoniot relation, 183, 184, 370

- rarefaction wave, 180, 182, 184–186, 200, 370
- reconstruct-evolve-average (REA) algorithm, 200
- red-black instability, 429
- regular splitting, 262
- regularity, elliptic, 327, 455, 458, 463
- Riemann problem, 184–186, 196, 219, 364, 370
 - Burgers' equation, 184, 197
 - linear hyperbolic system, 364
 - nonlinear hyperbolic system, 370
 - scalar conservation law, 184, 200, 219
- Ritz–Galerkin, 107, 321, 322
- Runge–Kutta method
 - see* implicit and explicit time-integration method
- shallow-water equations, 368, 381
- shock wave, 180, 182–186, 201, 219, 370
 - compound shock, 219
 - shock speed, 187
 - see also* Rankine–Hugoniot relation
 - sonic boom, 372
- simplified Newton method, 415
- slope limiter, 208, 211, 408
 - minmod, *see* minmod slope limiter
 - superbee, 215
 - van Leer, 215
- Sobolev spaces
 - $H(\text{curl}, \Omega)$, 313, 435
 - $H(\text{div}, \Omega)$, 313, 436, 474
 - H^k , 312
 - periodic, 489
- sound speed, 372
- sound wave, 372
- spectral methods, 488
- spurious oscillations, 188, 196, 385, 393
- stability, 61, 63, 143, 195, 373, 382
 - A-stable, 412
 - L-stable, 412
 - Lax–Richtmyer, 146, 148
 - von Neumann, *see* von Neumann stability
- stiffness matrix, 353, 404
- Stokes equations, 460, 477
- strong form, 104
- strong stability preserving (SSP) method, 397
- structure-preserving schemes, 76
- summation by parts, 99
- Taylor series, 23, 56
- theorem, lemma, corollary
 - Aubin–Nitsche duality, 111
 - Céa's lemma, 110, 322, 457, 476
 - divergence theorem, 7, 10, 43
 - Faber–Manteuffel, 291
 - Geršgorin theorem, 64, 248
 - Godunov's theorem, 207
 - Hilbert projection theorem, 308
 - Kahan, 253
 - Lax convergence theorem, 62, 145
 - Lax equivalence theorem, 146
 - Lax–Milgram, 318, 319
 - Lax–Wendroff theorem, 194
 - Ostrowski, 254
 - Perron–Frobenius, 251
 - Poincaré–Friedrichs inequality, 320
 - Riesz representation theorem, 316
 - Stein–Rosenberg, 252
 - Varah's theorem, 66
- Toeplitz matrix, 53
- Toeplitz operators, 151, 167
- total variation, 202, 385
 - of grid function, 205
 - scalar conservation laws, 202
- total-variation diminishing (TVD) method, 205
 - conditions, 205, 213
 - first-order upwind, 206
 - second-order methods, 211
 - TVD region, 215
- total-variation diminishing (TVD) property, 203, 385
 - discrete, 205
 - scalar conservation laws, 203
- traffic flow, 217
- tridiagonal matrix, 53
- truncation error
 - finite-difference method, 61, 139
- uniqueness, 109, 315
- vanishing-viscosity solution, 185, 186
- von Neumann stability, 150, 153–162
- Voronoi mesh, 85
- wave
 - genuinely nonlinear, 369
 - linearly degenerate, 369
 - rarefaction, *see* rarefaction wave

- shock, *see* shock wave
- sound, *see* sound wave
- wave equation, [8](#), [15](#)
- weak derivative, [310](#)
- weak form, [105](#), [322](#)
 - discontinuous Galerkin, [482](#)
- weak problem, *see* weak form
- weak solution
 - hyperbolic conservation law, *see* hyperbolic conservation law, weak solution
- weighted essentially nonoscillatory (WENO)
 - method, [384](#), [392](#)
 - hyperbolic systems, [399](#)
 - multiple spatial dimensions, [399](#)
- Yee scheme, [433](#)